



Arm®v8-M Architecture Reference Manual

Document number	DDI0553B.y
Document version	ID09082024
Document confidentiality	Non-confidential

Copyright © 2015 - 2024 Arm Limited or its affiliates. All rights reserved.

Release information

Date	Version	Changes
2024/Aug/09	B.y Non-Confidential EAC	• Twenty-fifth release of the the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2023/Dec/15	B.x Non-Confidential EAC	• Twenty-fourth release of the the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2023/Jul/07	B.w Non-Confidential EAC	• Twenty-third release of the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2022/Dec/16	B.v Non-Confidential EAC	• Twenty-second release of the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2022/Sep/28	B.u Non-Confidential EAC	• Twenty-first release of the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2022/Jun/30	B.t Non-Confidential EAC	• Twentieth release of the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2022/Apr/01	B.s Non-Confidential EAC	• Nineteenth release of the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2021/Dec/17	B.r Non-Confidential EAC	• Eighteenth release of the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2021/Sep/30	B.q Non-Confidential EAC	• Seventeenth release of the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2021/Jun/30	B.p Non-Confidential EAC	• Sixteenth release of the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2021/Mar/31	B.o Non-Confidential EAC	• Fifteenth release of the v8.0-M manual with integrated v8.1-M material, Custom Datapath Extension material and PACBTI Extension material
2020/Dec/18	B.n Non-Confidential EAC	• Fourteenth release of the v8.0-M manual with integrated v8.1-M material and Custom Datapath Extension material
2020/Oct/02	B.m Non-Confidential EAC	• Thirteenth release of the v8.0-M manual with integrated v8.1-M material and Custom Datapath Extension material
2020/Jun/30	B.l Non-Confidential EAC	• Twelfth release of the v8.0-M manual with integrated v8.1-M material and Custom Datapath Extension material
2020/Mar/31	B.k Non-Confidential EAC	• Eleventh release of the v8.0-M manual with integrated v8.1-M material
2019/Dec/17	B.j Non-Confidential EAC	• Tenth release of the v8.0-M manual with integrated v8.1-M material
2019/Oct/02	A.m Non-confidential EAC	• Eleventh EAC release

Date	Version	Changes
2019/Oct/02	B.i Non-Confidential EAC	• Ninth release of the v8.0-M manual with integrated v8.1-M material
2019/Jun/28	A.l Non-confidential EAC	• Tenth EAC release
2019/Jun/28	B.h Non-Confidential EAC	• Eighth release of the v8.0-M manual with integrated v8.1-M material
2019/Mar/29	A.k Non-confidential EAC	• Ninth EAC release
2019/Mar/29	B.g Non-Confidential EAC	• Seventh release of the v8.0-M manual with integrated v8.1-M material
2019/Feb/14	B.f Non-Confidential EAC	• Sixth release of the v8.0-M manual with integrated v8.1-M material
2018/Dec/14	A.j Non-confidential EAC	• Eighth EAC release
2018/Dec/14	B.e Confidential EAC	• Fifth release of the v8.0-M manual with integrated v8.1-M material
2018/Jun/29	A.i Non-confidential EAC	• Seventh EAC release
2018/Jun/29	B.d Confidential Beta	• Fourth release of the v8.0-M manual with integrated v8.1-M material
2018/Mar/31	A.h Non-confidential EAC	• Sixth EAC release
2018/Mar/31	B.c Confidential Beta	• Third release of the v8.0-M manual with integrated v8.1-M material
2017/Dec/15	A.g Non-confidential EAC	• Fifth EAC release
2017/Dec/15	B.b Confidential Beta	• Second release of the v8.0-M manual with integrated v8.1-M material
2017/Sep/29	A.f Non-confidential EAC	• Fourth EAC release
2017/Sep/29	B.a Confidential Beta	• First release of the v8.0-M manual with integrated v8.1-M material
2017/Jun/02	A.e Non-confidential EAC	• Third EAC release

Date	Version	Changes
2016/Nov/30	A.d Non-confidential EAC	• Second EAC release
2016/Sep/30	A.c Non-confidential EAC	• EAC release
2016/Jul/28	A.b Non-confidential Beta	• Beta release
2016/Mar/29	A.a Confidential Beta	• Beta release, limited circulation

Armv8-M Architecture Reference Manual

Copyright © 2015 - 2024 Arm Limited or its affiliates. All rights reserved. The copyright statement reflects the fact that some draft issues of this document have been released, to a limited circulation.

Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited (“Arm”). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm’s view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED “AS IS”. ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party’s products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word “partner” in reference to Arm’s customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm’s trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Copyright © 2015 - 2024 Arm Limited (or its affiliates). All rights reserved.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-20349 8 March 2024

Confidentiality Status

This document is Non-confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Product Status

The information in this document is final, that is for a developed product.

Web Address

<http://www.arm.com>

Contents

Arm®v8-M Architecture Reference Manual

Release information	ii
Armv8-M Architecture Reference Manual	v
Proprietary Notice	v
Confidentiality Status	vi
Product Status	vi
Web Address	vi

Preface

About this manual	xi
Using this manual	xli
Conventions	xlili
Typographical conventions	xlili
Signals	xlili
Numbers	xliv
Pseudocode descriptions	xliv
Assembler syntax descriptions	xliv
Additional reading	xlvi
Arm publications	xlvi
Other publications	xlvi
Feedback	xlvi
Feedback on this manual	xlvi
Inclusive Terminology Commitment	xlvi

Part A Armv8-M Architecture Introduction and Overview

Chapter A1

Introduction

A1.1	Architecture reference manual layout and terminology	49
A1.1.1	Structure of the architecture reference manual	49
A1.1.2	Scope of the architecture reference manual	50
A1.1.3	Intended audience	50
A1.1.4	Terminology, phrases	50
A1.1.5	Terminology, Armv8-M specific terms	51
A1.2	About the Armv8 architecture, and architecture profiles	52
A1.3	The Armv8-M architecture profile	53
A1.3.1	The Armv8-M instruction set	53
A1.3.2	Baseline implementation	53
A1.3.3	Nested Vectored Interrupt Controller	53
A1.4	Optional Extensions	55
A1.4.1	CDE - The Custom Datapath Extension	55
A1.4.2	Debug	55
A1.4.3	DSP - The Digital Signal Processing Extension.	56
A1.4.4	The DSP Debug Extension	56
A1.4.5	The Floating-point Extension	56
A1.4.6	The Main Extension	56
A1.4.7	MPU model	57
A1.4.8	M-Profile Vector Extension, MVE	57
A1.4.9	Pointer Authentication and Branch Target Identification Extension	57
A1.4.10	Performance Monitors Extension	57

A1.4.11	Reliability, Availability, and Serviceability	58
A1.4.12	Security Extension	58
A1.4.13	The System Timer Extension	58

Part B Armv8-M Architecture Rules

Chapter B1	Resets	
B1.1	Resets, Cold reset, and Warm reset	61
Chapter B2	Power Management	
B2.1	Power management	63
B2.1.1	The Wait for Event (WFE) instruction	63
B2.1.2	The Wait for Interrupt (WFI) instruction	64
B2.2	Sleep on exit	66
Chapter B3	Programmers' Model	
B3.1	PE modes, Thread mode and Handler mode	69
B3.2	Privileged and unprivileged execution	70
B3.3	Registers	71
B3.4	Special-purpose CONTROL register	73
B3.5	XPSR, APSR, IPSR, and EPSR	74
B3.5.1	Interrupt Program Status Register (IPSR)	74
B3.5.2	Execution Program Status Register (EPSR)	75
B3.6	Security states: Secure state, and Non-secure state	76
B3.7	Security states and register banking between Security states	77
B3.8	Stack pointer	78
B3.9	Exception numbers and exception priority numbers	80
B3.10	Exception enable, pending, and active bits	83
B3.11	Security states, exception banking	85
B3.12	Faults	88
B3.13	Priority model	93
B3.14	Secure address protection	98
B3.15	Security state transitions	99
B3.16	Function calls from Secure state to Non-secure state	101
B3.17	Function returns from Non-secure state	102
B3.18	Exception handling	104
B3.19	Exception entry, context stacking	106
B3.20	Exception entry, register clearing after context stacking	114
B3.21	Stack limit checks	115
B3.22	Exception return	118
B3.23	Integrity signature	123
B3.24	Exceptions during exception entry	124
B3.25	Exceptions during exception return	126
B3.26	Tail-chaining	127
B3.27	Exceptions, instruction resume, or instruction restart	129
B3.27.1	Basic definitions	129
B3.27.2	Instruction restart behavior	129
B3.27.3	Interrupt-continuable instructions	130
B3.27.4	Behavior of interrupt-continuable instructions	131
B3.27.5	Exceptions during interrupt-continuable instructions	132
B3.27.6	Exception-continuable instruction behavior	133
B3.27.7	Invalid ICI and ECI values	134
B3.28	Low overhead loops	135
B3.29	Branch future	139

B3.30	Vector tables	141
B3.31	Hardware-controlled priority escalation to HardFault	143
B3.32	Special-purpose mask registers, PRIMASK, BASEPRI, FAULTMASK, for configurable priority boosting	144
B3.33	Lockup	146
B3.33.1	Lockup behavior	147
B3.33.2	Exception-related lockup behavior	147
B3.34	Data independent timing	151
B3.35	Context Synchronization Event	153
B3.36	Coprocessor support	154
B3.37	The Custom Datapath Extension	155
B3.37.1	Overview of the Custom Datapath Extension	155
B3.37.2	Enabling CDE instructions	155
B3.37.3	Execution of CDE instructions	157

Chapter B4

Floating-point Support

B4.1	The optional Floating-point Extension, FPUv5	160
B4.2	About the Floating-point Status and Control Registers	162
B4.3	Registers for Floating-point data processing, S0-S31, or D0-D15	164
B4.4	Floating-point standards and terminology	166
B4.5	Floating-point data representable	167
B4.6	Floating-point encoding formats, half-precision, single-precision, and double-precision	168
B4.7	The IEEE 754 Floating-point exceptions	170
B4.8	The Flush-to-zero mode	171
B4.8.1	The Flush to zero mode half-precision calculations	172
B4.9	The Default NaN mode, and NaN handling	173
B4.10	The Default NaN	174
B4.11	Combinations of Floating-point exceptions	175
B4.12	Priority of Floating-point exceptions relative to other Floating-point exceptions	176

Chapter B5

Vector Extension

B5.1	Vector Extension operation	178
B5.2	Vector register file	180
B5.3	Lanes	181
B5.4	Beats	182
B5.5	Predication/conditional execution	185
B5.5.1	Loop tail predication	185
B5.5.2	VPT predication	186
B5.5.3	Effects of predication	189
B5.5.4	IT block	190
B5.6	MVE interleaving/de-interleaving loads and stores	191

Chapter B6

Pointer authentication and branch target identification Extension

B6.1	Implementing PAC and BTI	193
B6.1.1	Pointer authentication	194
B6.1.2	Branch target identification	198

Chapter B7

Memory Model

B7.1	Definition of the Armv8 memory model	203
B7.1.1	Locations	203
B7.2	Atomicity	206
B7.2.1	Single-copy atomicity	206
B7.2.2	Multi-copy atomicity	207
B7.2.3	Ordering and observability	207

B7.2.4	Ordering and observability at a Location	209
B7.2.5	Ordering constraints	213
B7.2.6	Completion and endpoint ordering	214
B7.2.7	External ordering constraints	216
B7.2.8	Globally-completes-before order	217
B7.2.9	Memory barriers	218
B7.2.10	Instruction Synchronization Barrier	219
B7.2.11	Data Memory Barrier	219
B7.2.12	Data Synchronization Barrier	220
B7.2.13	Consumption of Speculative Data Barrier	220
B7.2.14	Physical Speculative Store Bypass Barrier	221
B7.2.15	Speculative Store Bypass Barrier	222
B7.2.16	Synchronization requirements for System Control Space	222
B7.3	Memory accesses	223
B7.4	Address space	224
B7.5	Endianness	225
B7.6	Alignment behavior	228
B7.7	Concurrent modification and execution of instructions	229
B7.8	Access rights	231
B7.9	Normal memory	232
B7.10	Cacheability attributes	234
B7.11	Device memory	235
B7.12	Device memory attributes	237
B7.12.1	Gathering and non-Gathering Device memory attributes	238
B7.12.2	Reordering and non-Reordering Device memory attributes	238
B7.12.3	Early Write Acknowledgement and no Early Write Acknowledgement Device memory attributes	239
B7.13	Shareability attributes	240
B7.14	Shareability domains	241
B7.15	Memory access restrictions	243
B7.16	Mismatched memory attributes	244
B7.17	Load-Exclusive and Store-Exclusive accesses to Normal memory	246
B7.18	Load-Acquire and Store-Release accesses to memory	247
B7.19	Caches	249
B7.20	Cache identification	251
B7.21	Cache visibility	252
B7.22	Cache coherency	253
B7.23	Cache enabling and disabling	254
B7.24	Cache behavior at reset	255
B7.25	Behavior of Preload Data (PLD) and Preload Instruction (PLI) instructions with caches	256
B7.26	Branch predictors	257
B7.27	Cache maintenance operations	258
B7.28	Ordering of cache maintenance operations	262
B7.29	Branch predictor maintenance operations	263

Chapter B8

The System Address Map

B8.1	System address map	265
B8.2	The System region of the system address map	266
B8.3	The System Control Space (SCS)	268

Chapter B9

Synchronization and Semaphores

B9.1	Exclusive access instructions	270
B9.2	The local monitors	271
B9.3	The global monitor	273

	B9.3.1	Load-Exclusive and Store-Exclusive	274
	B9.3.2	Load-Exclusive and Store-Exclusive in Shareable memory	275
	B9.4	Exclusive access instructions and the monitors	277
	B9.5	Load-Exclusive and Store-Exclusive instruction constraints	278
Chapter B10		The Armv8-M Protected Memory System Architecture	
	B10.1	Memory Protection Unit	281
	B10.2	Security attribution	284
	B10.3	Security attribution unit (SAU)	287
	B10.4	IMPLEMENTATION DEFINED Attribution Unit (IDAU)	288
Chapter B11		The System Timer, SysTick	
	B11.1	The system timer, SysTick	290
Chapter B12		Nested Vectored Interrupt Controller	
	B12.1	NVIC definition	293
	B12.2	NVIC operation	294
Chapter B13		Debug	
	B13.1	Debug feature overview	297
		B13.1.1 Debug mechanisms	300
		B13.1.2 Debug resources	300
		B13.1.3 Trace	301
	B13.2	Accessing debug features	303
		B13.2.1 ROM table	303
		B13.2.2 Debug System registers	306
		B13.2.3 CoreSight and identification registers	306
	B13.3	Debug authentication interface	308
		B13.3.1 Halting debug authentication	309
		B13.3.2 Non-invasive debug authentication	312
		B13.3.3 DebugMonitor exception authentication	313
		B13.3.4 DAP access permissions	315
	B13.4	Debug event behavior	321
		B13.4.1 About debug events	321
		B13.4.2 Debug stepping	327
		B13.4.3 Vector catch	332
		B13.4.4 Breakpoint instructions	334
		B13.4.5 External debug request	335
	B13.5	Debug state	337
	B13.6	Exiting Debug state	342
	B13.7	Multiprocessor support	343
		B13.7.1 Cross-halt event	343
		B13.7.2 External restart request	343
Chapter B14		Debug and Trace Components	
	B14.1	Instrumentation Trace Macrocell	345
		B14.1.1 About the ITM	345
		B14.1.2 ITM operation	346
		B14.1.3 Timestamp support	349
		B14.1.4 Synchronization support	352
		B14.1.5 Continuation bits	353
	B14.2	Data Watchpoint and Trace unit	354
		B14.2.1 About the DWT	354
		B14.2.2 DWT unit operation	355
		B14.2.3 Constraints on programming DWT comparators	360
		B14.2.4 CMPMATCH trigger events	365

B14.2.5	Matching in detail	366
B14.2.6	DWT match restrictions and relaxations	369
B14.2.7	DWT trace restrictions and relaxations	370
B14.2.8	CYCCNT cycle counter and related timers	372
B14.2.9	Profiling counter support	374
B14.2.10	Program Counter sampling support	375
B14.3	Embedded Trace Macrocell	378
B14.4	Trace Port Interface Unit	380
B14.5	Flash Patch and Breakpoint unit	382
B14.5.1	About the FPB unit	382
B14.5.2	FPB unit operation	382
B14.5.3	Cache maintenance	384

Chapter B15

The Performance Monitors Extension

B15.1	Counters	386
B15.2	Accuracy of the performance counters	387
B15.3	Security, access, and modes	388
B15.4	Attributability	389
B15.5	Coexistence with the DWT Performance Monitors	390
B15.6	Interrupts and Debug events	391
B15.7	Performance Monitors and Debug state	392
B15.8	List of supported architectural and microarchitectural events	393
B15.9	Generic architectural and microarchitectural events	404
B15.9.1	CTI_TRIGOUT (Cross-trigger Interface output trigger, for = 4 to 7)	404
B15.9.2	TRCEXTOUT (PE Trace Unit external output, for = 0 to 3)	404
B15.9.3	DWT_CMPMATCH (DWT comparator match, for = 0 to 7)	404
B15.9.4	LI_CACHE_REFILL (Level instruction cache refill)	404
B15.9.5	LD_CACHE_REFILL (Level data cache refill)	404
B15.9.6	LD_CACHE_MISS_RD (Level data cache miss on read)	405
B15.9.7	LD_CACHE_WB (Level data cache write-back)	405
B15.9.8	LI_CACHE (Level instruction cache access)	406
B15.9.9	LD_CACHE (Level data cache access)	406
B15.9.10	LD_CACHE_RD (Level data cache access, read)	406
B15.10	Common event descriptions	407
B15.11	Required PMU events	430
B15.12	IMPLEMENTATION DEFINED event numbers	431

Chapter B16

Reliability, Availability, and Serviceability (RAS) Extension

B16.1	Overview	433
B16.2	Taxonomy of errors	434
B16.2.1	Architectural error propagation	434
B16.2.2	Architecturally infected, contained, and uncontained	434
B16.2.3	Architecturally consumed errors	435
B16.2.4	Other errors	435
B16.3	Generating error exceptions	436
B16.3.1	Error correction and deferment	438
B16.4	Error Synchronization Barrier (ESB)	439
B16.4.1	ESB and Unrecoverable errors	439
B16.4.2	ESB and other containable errors	439
B16.4.3	ESB and other errors	440
B16.5	Implicit Error Synchronization (IESB)	441
B16.6	Fault handling	443
B16.7	RAS error records	445
B16.8	Multiple BusFault exceptions	448
B16.9	Minimal RAS implementation	449

Part C Armv8-M Instruction Set

Chapter C1

Instruction Set Overview

C1.1	Instruction set	452
C1.2	Format of instruction descriptions	453
C1.2.1	The title	453
C1.2.2	A short description	453
C1.2.3	The instruction encoding or encodings	453
C1.2.4	Any alias conditions, if applicable	455
C1.2.5	Standard assembler syntax fields	456
C1.2.6	Pseudocode describing how the instruction operates	458
C1.2.7	Use of labels in UAL instruction syntax	459
C1.2.8	Using syntax information	460
C1.3	Conditional execution	461
C1.3.1	Conditional instructions	462
C1.3.2	Pseudocode details of conditional execution	462
C1.3.3	Conditional execution of undefined instructions	463
C1.3.4	Interaction of undefined instruction behavior with UNPREDICTABLE or CONSTRAINED_UNPREDICTABLE instruction behavior	463
C1.3.5	ITSTATE	463
C1.3.6	Pseudocode details of ITSTATE operation	464
C1.3.7	SVC and ITSTATE	464
C1.3.8	CONSTRAINED_UNPREDICTABLE behavior and IT blocks	465
C1.4	Instruction set encoding information	467
C1.4.1	UNDEFINED and UNPREDICTABLE instruction set space	467
C1.4.2	Pseudocode descriptions of operations on general-purpose registers and the PC	467
C1.4.3	Use of 0b1111 as a register specifier	468
C1.4.4	Use of 0b1101 as a register specifier	469
C1.4.5	16-bit T32 instruction support for SP	470
C1.4.6	Branching	470
C1.4.7	Instruction set, interworking and interstating support	471
C1.5	Modified immediate constants	473
C1.5.1	Operation of modified immediate constants	473
C1.5.2	Modified immediate values for MVE instructions	474
C1.5.3	Modified immediate constants for floating-point instructions	475
C1.6	NOP-compatible hint instructions	476

Chapter C2

Instruction Specification

C2.1	Top level T32 instruction set encoding	478
C2.2	16-bit T32 instruction encoding	479
C2.2.1	Shift (immediate), add, subtract, move, and compare	479
C2.2.2	Data-processing (two low registers)	481
C2.2.3	Special data instructions and branch and exchange	482
C2.2.4	Load/store (register offset)	483
C2.2.5	Load/store word/byte (immediate offset)	483
C2.2.6	Load/store halfword (immediate offset)	483
C2.2.7	Load/store (SP-relative)	484
C2.2.8	Add PC/SP (immediate)	484
C2.2.9	Miscellaneous 16-bit instructions	485
C2.2.10	Load/store multiple	487
C2.2.11	Conditional branch, and Supervisor Call	487
C2.3	32-bit T32 instruction encoding	489
C2.3.1	Coprocessor, floating-point, and vector instructions	489
C2.3.2	Load/store (multiple, dual, exclusive, acquire-release)	504

C2.3.3	Data-processing (shifted register)	508
C2.3.4	Branches and miscellaneous control	510
C2.3.5	Data-processing (modified immediate)	513
C2.3.6	Data-processing (plain binary immediate)	514
C2.3.7	Load/store single	516
C2.3.8	Data-processing (register)	522
C2.3.9	Multiply, multiply accumulate, and absolute difference	524
C2.3.10	Long multiply and divide	525
C2.4	Alphabetical list of instructions	527
C2.4.1	ADC (immediate)	528
C2.4.2	ADC (register)	530
C2.4.3	ADD (SP plus immediate)	532
C2.4.4	ADD (SP plus register)	534
C2.4.5	ADD (immediate)	537
C2.4.6	ADD (immediate, to PC)	540
C2.4.7	ADD (register)	542
C2.4.8	ADR	545
C2.4.9	AND (immediate)	547
C2.4.10	AND (register)	549
C2.4.11	ASR (immediate)	551
C2.4.12	ASR (register)	553
C2.4.13	ASRL (immediate)	555
C2.4.14	ASRL (register)	556
C2.4.15	ASRS (immediate)	557
C2.4.16	ASRS (register)	559
C2.4.17	AUT	561
C2.4.18	AUTG	563
C2.4.19	B	565
C2.4.20	BF, BFX, BFL, BFLX, BFCSEL	567
C2.4.21	BFC	571
C2.4.22	BFI	573
C2.4.23	BIC (immediate)	575
C2.4.24	BIC (register)	577
C2.4.25	BKPT	579
C2.4.26	BL	580
C2.4.27	BLX, BLXNS	581
C2.4.28	BTI	583
C2.4.29	BX, BXNS	584
C2.4.30	BXAUT	586
C2.4.31	CBNZ, CBZ	588
C2.4.32	CDP, CDP2	589
C2.4.33	CINC	591
C2.4.34	CINV	592
C2.4.35	CLREX	593
C2.4.36	CLRM	594
C2.4.37	CLZ	595
C2.4.38	CMN (immediate)	597
C2.4.39	CMN (register)	598
C2.4.40	CMP (immediate)	600
C2.4.41	CMP (register)	602
C2.4.42	CNEG	605
C2.4.43	CPS	606
C2.4.44	CSDB	608
C2.4.45	CSEL	609
C2.4.46	CSET	611

C2.4.47	CSETM	612
C2.4.48	CSINC	613
C2.4.49	CSINV	615
C2.4.50	CSNEG	617
C2.4.51	CX1	619
C2.4.52	CX1D	621
C2.4.53	CX2	623
C2.4.54	CX2D	625
C2.4.55	CX3	627
C2.4.56	CX3D	629
C2.4.57	DBG	631
C2.4.58	DMB	632
C2.4.59	DSB	633
C2.4.60	EOR (immediate)	634
C2.4.61	EOR (register)	636
C2.4.62	ESB	638
C2.4.63	FLDMDBX, FLDMIAX	639
C2.4.64	FSTMDBX, FSTMIAX	641
C2.4.65	ISB	643
C2.4.66	IT	644
C2.4.67	LCTP	646
C2.4.68	LDA	647
C2.4.69	LDAB	648
C2.4.70	LDAEX	649
C2.4.71	LDAEXB	650
C2.4.72	LDAEXH	651
C2.4.73	LDAH	652
C2.4.74	LDC, LDC2 (immediate)	653
C2.4.75	LDC, LDC2 (literal)	656
C2.4.76	LDM, LDMIA, LDMFD	659
C2.4.77	LDMDB, LDMEA	664
C2.4.78	LDR (immediate)	667
C2.4.79	LDR (literal)	671
C2.4.80	LDR (register)	673
C2.4.81	LDRB (immediate)	675
C2.4.82	LDRB (literal)	678
C2.4.83	LDRB (register)	680
C2.4.84	LDRBT	682
C2.4.85	LDRD (immediate)	683
C2.4.86	LDRD (literal)	685
C2.4.87	LDREX	687
C2.4.88	LDREXB	688
C2.4.89	LDREXH	689
C2.4.90	LDRH (immediate)	690
C2.4.91	LDRH (literal)	693
C2.4.92	LDRH (register)	695
C2.4.93	LDRHT	697
C2.4.94	LDRSB (immediate)	698
C2.4.95	LDRSB (literal)	700
C2.4.96	LDRSB (register)	702
C2.4.97	LDRSBT	704
C2.4.98	LDRSH (immediate)	705
C2.4.99	LDRSH (literal)	707
C2.4.100	LDRSH (register)	709
C2.4.101	LDRSHT	711

C2.4.102 LDRT	712
C2.4.103 LE, LETP	713
C2.4.104 LSL (immediate)	715
C2.4.105 LSL (register)	717
C2.4.106 LSLL (immediate)	719
C2.4.107 LSLL (register)	720
C2.4.108 LSLS (immediate)	721
C2.4.109 LSLS (register)	723
C2.4.110 LSR (immediate)	725
C2.4.111 LSR (register)	727
C2.4.112 LSRL (immediate)	729
C2.4.113 LSRS (immediate)	730
C2.4.114 LSRS (register)	732
C2.4.115 MCR, MCR2	734
C2.4.116 MCRR, MCRR2	736
C2.4.117 MLA	738
C2.4.118 MLS	740
C2.4.119 MOV (immediate)	741
C2.4.120 MOV (register)	743
C2.4.121 MOV, MOVS (register-shifted register)	747
C2.4.122 MOVT	750
C2.4.123 MRC, MRC2	751
C2.4.124 MRRC, MRRC2	753
C2.4.125 MRS	755
C2.4.126 MSR (register)	760
C2.4.127 MUL	765
C2.4.128 MVN (immediate)	767
C2.4.129 MVN (register)	768
C2.4.130 NOP	770
C2.4.131 ORN (immediate)	771
C2.4.132 ORN (register)	773
C2.4.133 ORR (immediate)	775
C2.4.134 ORR (register)	777
C2.4.135 PAC	779
C2.4.136 PACBTI	781
C2.4.137 PACG	783
C2.4.138 PKHBT, PKHTB	785
C2.4.139 PLD (literal)	787
C2.4.140 PLD, PLDW (immediate)	788
C2.4.141 PLD, PLDW (register)	790
C2.4.142 PLI (immediate, literal)	791
C2.4.143 PLI (register)	793
C2.4.144 POP (multiple registers)	794
C2.4.145 POP (single register)	796
C2.4.146 PSSBB	797
C2.4.147 PUSH (multiple registers)	798
C2.4.148 PUSH (single register)	800
C2.4.149 QADD	801
C2.4.150 QADD16	802
C2.4.151 QADD8	803
C2.4.152 QASX	804
C2.4.153 QDADD	805
C2.4.154 QDSUB	806
C2.4.155 QSAX	807
C2.4.156 QSUB	808

C2.4.157	QSUB16	809
C2.4.158	QSUB8	810
C2.4.159	RBIT	811
C2.4.160	REV	812
C2.4.161	REV16	814
C2.4.162	REVSH	816
C2.4.163	ROR (immediate)	818
C2.4.164	ROR (register)	820
C2.4.165	RORS (immediate)	822
C2.4.166	RORS (register)	824
C2.4.167	RRX	826
C2.4.168	RRXS	827
C2.4.169	RSB (immediate)	828
C2.4.170	RSB (register)	830
C2.4.171	SADD16	832
C2.4.172	SADD8	833
C2.4.173	SASX	834
C2.4.174	SBC (immediate)	835
C2.4.175	SBC (register)	837
C2.4.176	SBFX	839
C2.4.177	SDIV	841
C2.4.178	SEL	842
C2.4.179	SEV	843
C2.4.180	SG	844
C2.4.181	SHADD16	846
C2.4.182	SHADD8	847
C2.4.183	SHASX	848
C2.4.184	SHSAX	849
C2.4.185	SHSUB16	850
C2.4.186	SHSUB8	851
C2.4.187	SMLABB, SMLABT, SMLATB, SMLATT	852
C2.4.188	SMLAD, SMLADX	854
C2.4.189	SMLAL	856
C2.4.190	SMLALBB, SMLALBT, SMLALTB, SMLALTT	858
C2.4.191	SMLALD, SMLALDX	860
C2.4.192	SMLAWB, SMLAWT	862
C2.4.193	SMLSDD, SMLSDDX	864
C2.4.194	SMLSDD, SMLSDDX	866
C2.4.195	SMMLA, SMMLAR	868
C2.4.196	SMMLS, SMMLSR	870
C2.4.197	SMMUL, SMMULR	872
C2.4.198	SMUAD, SMUADX	873
C2.4.199	SMULBB, SMULBT, SMULTB, SMULTT	875
C2.4.200	SMULL	877
C2.4.201	SMULWB, SMULWT	878
C2.4.202	SMUSD, SMUSDX	879
C2.4.203	SQRSHR (register)	881
C2.4.204	SQRSHRL (register)	882
C2.4.205	SQSHL (immediate)	883
C2.4.206	SQSHLL (immediate)	884
C2.4.207	SRSRHR (immediate)	885
C2.4.208	SRSRHL (immediate)	886
C2.4.209	SSAT	887
C2.4.210	SSAT16	888
C2.4.211	SSAX	889

C2.4.212	SSBB	890
C2.4.213	SSUB16	891
C2.4.214	SSUB8	892
C2.4.215	STC, STC2	893
C2.4.216	STL	896
C2.4.217	STLB	897
C2.4.218	STLEX	898
C2.4.219	STLEXB	900
C2.4.220	STLEXH	902
C2.4.221	STLH	904
C2.4.222	STM, STMIA, STMEA	905
C2.4.223	STMDB, STMFD	908
C2.4.224	STR (immediate)	911
C2.4.225	STR (register)	914
C2.4.226	STRB (immediate)	916
C2.4.227	STRB (register)	919
C2.4.228	STRBT	921
C2.4.229	STRD (immediate)	922
C2.4.230	STREX	924
C2.4.231	STREXB	926
C2.4.232	STREXH	928
C2.4.233	STRH (immediate)	930
C2.4.234	STRH (register)	933
C2.4.235	STRHT	935
C2.4.236	STRT	936
C2.4.237	SUB (SP minus immediate)	937
C2.4.238	SUB (SP minus register)	939
C2.4.239	SUB (immediate)	941
C2.4.240	SUB (immediate, from PC)	944
C2.4.241	SUB (register)	945
C2.4.242	SVC	947
C2.4.243	SXTAB	948
C2.4.244	SXTAB16	949
C2.4.245	SXTAH	951
C2.4.246	SXTB	952
C2.4.247	SXTB16	954
C2.4.248	SXTH	955
C2.4.249	TBB, TBH	957
C2.4.250	TEQ (immediate)	958
C2.4.251	TEQ (register)	959
C2.4.252	TST (immediate)	961
C2.4.253	TST (register)	962
C2.4.254	TT, TTT, TTA, TTAT	964
C2.4.255	UADD16	966
C2.4.256	UADD8	967
C2.4.257	UASX	968
C2.4.258	UBFX	969
C2.4.259	UDF	971
C2.4.260	UDIV	972
C2.4.261	UHADD16	973
C2.4.262	UHADD8	974
C2.4.263	UHASX	975
C2.4.264	UHSAX	976
C2.4.265	UHSUB16	977
C2.4.266	UHSUB8	978

C2.4.267	UMAAL	979
C2.4.268	UMLAL	981
C2.4.269	UMULL	983
C2.4.270	UQADD16	985
C2.4.271	UQADD8	986
C2.4.272	UQASX	987
C2.4.273	UQRSHL (register)	988
C2.4.274	UQRSHLL (register)	989
C2.4.275	UQSAX	990
C2.4.276	UQSHL (immediate)	991
C2.4.277	UQSHLL (immediate)	992
C2.4.278	UQSUB16	993
C2.4.279	UQSUB8	994
C2.4.280	URSHR (immediate)	995
C2.4.281	URSHRL (immediate)	996
C2.4.282	USAD8	997
C2.4.283	USADA8	998
C2.4.284	USAT	999
C2.4.285	USAT16	1000
C2.4.286	USAX	1001
C2.4.287	USUB16	1002
C2.4.288	USUB8	1003
C2.4.289	UXTAB	1004
C2.4.290	UXTAB16	1005
C2.4.291	UXTAH	1007
C2.4.292	UXTB	1008
C2.4.293	UXTB16	1010
C2.4.294	UXTH	1011
C2.4.295	VABAV	1013
C2.4.296	VABD	1015
C2.4.297	VABD (floating-point)	1017
C2.4.298	VABS	1019
C2.4.299	VABS (floating-point)	1021
C2.4.300	VABS (vector)	1023
C2.4.301	VADC	1025
C2.4.302	VADD	1027
C2.4.303	VADD (floating-point)	1029
C2.4.304	VADD (vector)	1031
C2.4.305	VADDLV	1033
C2.4.306	VADDV	1035
C2.4.307	VAND	1037
C2.4.308	VAND (immediate)	1039
C2.4.309	VBIC (immediate)	1040
C2.4.310	VBIC (register)	1042
C2.4.311	VBRSR	1044
C2.4.312	VCADD	1046
C2.4.313	VCADD (floating-point)	1048
C2.4.314	VCLS	1050
C2.4.315	VCLZ	1052
C2.4.316	VCMLA (floating-point)	1054
C2.4.317	VCMP	1057
C2.4.318	VCMP (floating-point)	1060
C2.4.319	VCMP (vector)	1062
C2.4.320	VCMPE	1067
C2.4.321	VCMUL (floating-point)	1070

C2.4.322	VCTP	1073
C2.4.323	VCVT (between double-precision and single-precision)	1074
C2.4.324	VCVT (between floating-point and fixed-point)	1075
C2.4.325	VCVT (between floating-point and fixed-point) (vector)	1078
C2.4.326	VCVT (between floating-point and integer)	1080
C2.4.327	VCVT (between single and half-precision floating-point)	1082
C2.4.328	VCVT (floating-point to integer)	1084
C2.4.329	VCVT (from floating-point to integer)	1086
C2.4.330	VCVT (integer to floating-point)	1088
C2.4.331	VCVTA	1090
C2.4.332	VCVTB	1092
C2.4.333	VCVTM	1094
C2.4.334	VCVTN	1096
C2.4.335	VCVTP	1098
C2.4.336	VCVTR	1100
C2.4.337	VCVTT	1102
C2.4.338	VCX1	1104
C2.4.339	VCX1 (vector)	1106
C2.4.340	VCX2	1108
C2.4.341	VCX2 (vector)	1110
C2.4.342	VCX3	1112
C2.4.343	VCX3 (vector)	1114
C2.4.344	VDDUP, VDWDUP	1116
C2.4.345	VDIV	1119
C2.4.346	VDUP	1121
C2.4.347	VEOR	1123
C2.4.348	VFMA	1125
C2.4.349	VFMA (vector by scalar plus vector, floating-point)	1127
C2.4.350	VFMA, VFMS (floating-point)	1129
C2.4.351	VFMA (vector by vector plus scalar, floating-point)	1131
C2.4.352	VFMS	1133
C2.4.353	VFNMA	1135
C2.4.354	VFNMS	1137
C2.4.355	VHADD	1139
C2.4.356	VHCADD	1141
C2.4.357	VHSUB	1143
C2.4.358	VIDUP, VIWDUP	1145
C2.4.359	VINS	1148
C2.4.360	VLD2	1149
C2.4.361	VLD4	1151
C2.4.362	VLDM	1153
C2.4.363	VLDR	1157
C2.4.364	VLDR (System Register)	1160
C2.4.365	VLDRB, VLDRH, VLDRW	1163
C2.4.366	VLDRB, VLDRH, VLDRW, VLDRD (vector)	1168
C2.4.367	VLLDM	1174
C2.4.368	VLSTM	1177
C2.4.369	VMAX, VMAXA	1180
C2.4.370	VMAXNM	1183
C2.4.371	VMAXNM, VMAXNMA (floating-point)	1185
C2.4.372	VMAXNMV, VMAXNMAV (floating-point)	1187
C2.4.373	VMAXV, VMAXAV	1189
C2.4.374	VMIN, VMINA	1191
C2.4.375	VMINNM	1194
C2.4.376	VMINNM, VMINNMA (floating-point)	1196

C2.4.377 VMINNMV, VMINNMAV (floating-point)	1198
C2.4.378 VMINV, VMINAV	1200
C2.4.379 VMLA	1202
C2.4.380 VMLA (vector by scalar plus vector)	1204
C2.4.381 VMLADAV	1206
C2.4.382 VMLALDAV	1209
C2.4.383 VMLALV	1211
C2.4.384 VMLAS (vector by vector plus scalar)	1212
C2.4.385 VMLAV	1214
C2.4.386 VMLS	1215
C2.4.387 VMLSDAV	1217
C2.4.388 VMLSDAV	1220
C2.4.389 VMOV (between general-purpose register and half-precision register)	1222
C2.4.390 VMOV (between general-purpose register and single-precision register)	1223
C2.4.391 VMOV (between two general-purpose registers and a doubleword register)	1224
C2.4.392 VMOV (between two general-purpose registers and two single-precision registers)	1226
C2.4.393 VMOV (general-purpose register to vector lane)	1228
C2.4.394 VMOV (half of doubleword register to single general-purpose register)	1230
C2.4.395 VMOV (immediate)	1231
C2.4.396 VMOV (immediate) (vector)	1233
C2.4.397 VMOV (register)	1235
C2.4.398 VMOV (register) (vector)	1237
C2.4.399 VMOV (single general-purpose register to half of doubleword register)	1238
C2.4.400 VMOV (two 32-bit vector lanes to two general-purpose registers)	1239
C2.4.401 VMOV (two general-purpose registers to two 32-bit vector lanes)	1241
C2.4.402 VMOV (vector lane to general-purpose register)	1243
C2.4.403 VMOVL	1245
C2.4.404 VMOVN	1247
C2.4.405 VMOVX	1249
C2.4.406 VMRS	1250
C2.4.407 VMSR	1253
C2.4.408 VMUL	1256
C2.4.409 VMUL (floating-point)	1258
C2.4.410 VMUL (vector)	1260
C2.4.411 VMULH, VRMULH	1262
C2.4.412 VMULL (integer)	1264
C2.4.413 VMULL (polynomial)	1266
C2.4.414 VMVN (immediate)	1268
C2.4.415 VMVN (register)	1270
C2.4.416 VNEG	1271
C2.4.417 VNEG (floating-point)	1273
C2.4.418 VNEG (vector)	1275
C2.4.419 VNMLA	1277
C2.4.420 VNMLS	1279
C2.4.421 VNMUL	1281
C2.4.422 VORN	1283
C2.4.423 VORN (immediate)	1285
C2.4.424 VORR	1286
C2.4.425 VORR (immediate)	1288
C2.4.426 VPNOT	1290
C2.4.427 VPOP	1291
C2.4.428 VPSEL	1293
C2.4.429 VPST	1295
C2.4.430 VPT	1296

C2.4.431	VPT (floating-point)	1302
C2.4.432	VPUSH	1304
C2.4.433	VQABS	1306
C2.4.434	VQADD	1307
C2.4.435	VQDMLADH, VQRDMLADH	1309
C2.4.436	VQDMLAH, VQRDMLAH (vector by scalar plus vector)	1312
C2.4.437	VQDMLASH, VQRDMLASH (vector by vector plus scalar)	1314
C2.4.438	VQDMLSDH, VQRDMLSDH	1316
C2.4.439	VQDMULH, VQRDMULH	1319
C2.4.440	VQDMULL	1323
C2.4.441	VQMOVN	1325
C2.4.442	VQMOVUN	1327
C2.4.443	VQNEG	1329
C2.4.444	VQRSHL	1330
C2.4.445	VQRSHRN	1332
C2.4.446	VQRSHRUN	1334
C2.4.447	VQSHL, VQSHLU	1336
C2.4.448	VQSHRN	1340
C2.4.449	VQSHRUN	1342
C2.4.450	VQSUB	1344
C2.4.451	VREV16	1346
C2.4.452	VREV32	1348
C2.4.453	VREV64	1350
C2.4.454	VRHADD	1352
C2.4.455	VRINT (floating-point)	1354
C2.4.456	VRINTA	1356
C2.4.457	VRINTM	1358
C2.4.458	VRINTN	1360
C2.4.459	VRINTP	1362
C2.4.460	VRINTR	1364
C2.4.461	VRINTX	1366
C2.4.462	VRINTZ	1368
C2.4.463	VRMLALDAVH	1370
C2.4.464	VRMLALVH	1372
C2.4.465	VRMLSLDAVH	1373
C2.4.466	VRSHL	1375
C2.4.467	VRSHR	1377
C2.4.468	VRSHRN	1379
C2.4.469	VSBC	1381
C2.4.470	VSCCLRM	1383
C2.4.471	VSEL	1385
C2.4.472	VSHL	1388
C2.4.473	VSHLC	1391
C2.4.474	VSHLL	1393
C2.4.475	VSHR	1396
C2.4.476	VSHRN	1398
C2.4.477	VSLI	1400
C2.4.478	VSQRT	1402
C2.4.479	VSRI	1403
C2.4.480	VST2	1405
C2.4.481	VST4	1407
C2.4.482	VSTM	1409
C2.4.483	VSTR	1413
C2.4.484	VSTR (System Register)	1415
C2.4.485	VSTRB, VSTRH, VSTRW	1418

C2.4.486	VSTRB, VSTRH, VSTRW, VSTRD (vector)	1423
C2.4.487	VSUB	1429
C2.4.488	VSUB (floating-point)	1431
C2.4.489	VSUB (vector)	1433
C2.4.490	WFE	1435
C2.4.491	WFI	1436
C2.4.492	WLS, DLS, WLSTP, DLSTP	1437
C2.4.493	YIELD	1440

Part D Armv8-M Registers and Payload Specification

Chapter D1

Register and Payload Specification

D1.1	Register index	1443
D1.1.1	Special and general-purpose registers	1443
D1.1.2	Payloads	1444
D1.1.3	Instrumentation Macrocell	1444
D1.1.4	Data Watchpoint and Trace	1445
D1.1.5	Flash Patch and Breakpoint	1445
D1.1.6	Performance Monitoring Unit	1446
D1.1.7	Reliability, Availability and Serviceability Extension Fault Status Register	1446
D1.1.8	Implementation Control Block	1447
D1.1.9	SysTick Timer	1447
D1.1.10	Nested Vectored Interrupt Controller	1447
D1.1.11	System Control Block	1447
D1.1.12	Memory Protection Unit	1448
D1.1.13	Security Attribution Unit	1448
D1.1.14	Debug Control Block	1449
D1.1.15	Software Interrupt Generation	1449
D1.1.16	Reliability, Availability and Serviceability Extension Fault Status Register	1449
D1.1.17	Floating-Point Extension	1449
D1.1.18	Cache Maintenance Operations	1449
D1.1.19	Debug Identification Block	1450
D1.1.20	Implementation Control Block (NS alias)	1450
D1.1.21	SysTick Timer (NS alias)	1450
D1.1.22	Nested Vectored Interrupt Controller (NS alias)	1451
D1.1.23	System Control Block (NS alias)	1451
D1.1.24	Memory Protection Unit (NS alias)	1452
D1.1.25	Debug Control Block (NS alias)	1452
D1.1.26	Software Interrupt Generation (NS alias)	1452
D1.1.27	Reliability, Availability and Serviceability Extension Fault Status Register (NS Alias)	1452
D1.1.28	Floating-Point Extension (NS alias)	1452
D1.1.29	Cache Maintenance Operations (NS alias)	1453
D1.1.30	Debug Identification Block (NS alias)	1453
D1.1.31	Trace Port Interface Unit	1453
D1.2	Alphabetical list of registers	1455
D1.2.1	ACTLR, Auxiliary Control Register	1456
D1.2.2	AFSR, Auxiliary Fault Status Register	1457
D1.2.3	AIRCR, Application Interrupt and Reset Control Register	1458
D1.2.4	APSR, Application Program Status Register	1463
D1.2.5	BASEPRI, Base Priority Mask Register	1465
D1.2.6	BFAR, BusFault Address Register	1466
D1.2.7	BFSR, BusFault Status Register	1467
D1.2.8	BPIALL, Branch Predictor Invalidate All	1470

D1.2.9	CCR, Configuration and Control Register	1471
D1.2.10	CCSIDR, Current Cache Size ID register	1475
D1.2.11	CFSR, Configurable Fault Status Register	1477
D1.2.12	CLIDR, Cache Level ID Register	1478
D1.2.13	CONTROL, Control Register	1480
D1.2.14	CPACR, Coprocessor Access Control Register	1483
D1.2.15	CPPWR, Coprocessor Power Control Register	1485
D1.2.16	CPUID, CPUID Base Register	1488
D1.2.17	CSSELR, Cache Size Selection Register	1490
D1.2.18	CTR, Cache Type Register	1492
D1.2.19	Dn, Floating-point Double-precision register, n = 0 - 15	1494
D1.2.20	DAUTHCTRL, Debug Authentication Control Register	1495
D1.2.21	DAUTHSTATUS, Debug Authentication Status Register	1498
D1.2.22	DCCIMVAC, Data Cache line Clean and Invalidate by Address to PoC	1501
D1.2.23	DCCISW, Data Cache line Clean and Invalidate by Set/Way	1502
D1.2.24	DCCMVAC, Data Cache line Clean by Address to PoC	1503
D1.2.25	DCCMVAU, Data Cache line Clean by address to PoU	1504
D1.2.26	DCCSW, Data Cache Clean line by Set/Way	1505
D1.2.27	DCIDR0, SCS Component Identification Register 0	1506
D1.2.28	DCIDR1, SCS Component Identification Register 1	1507
D1.2.29	DCIDR2, SCS Component Identification Register 2	1508
D1.2.30	DCIDR3, SCS Component Identification Register 3	1509
D1.2.31	DCIMVAC, Data Cache line Invalidate by Address to PoC	1510
D1.2.32	DCISW, Data Cache line Invalidate by Set/Way	1511
D1.2.33	DCRDR, Debug Core Register Data Register	1512
D1.2.34	DCRSR, Debug Core Register Select Register	1513
D1.2.35	DDEVARCH, SCS Device Architecture Register	1518
D1.2.36	DDEVTYPE, SCS Device Type Register	1520
D1.2.37	DEMCR, Debug Exception and Monitor Control Register	1522
D1.2.38	DFSR, Debug Fault Status Register	1529
D1.2.39	DHCSR, Debug Halting Control and Status Register	1531
D1.2.40	DLAR, SCS Software Lock Access Register	1538
D1.2.41	DLSR, SCS Software Lock Status Register	1539
D1.2.42	DPIDR0, SCS Peripheral Identification Register 0	1541
D1.2.43	DPIDR1, SCS Peripheral Identification Register 1	1542
D1.2.44	DPIDR2, SCS Peripheral Identification Register 2	1543
D1.2.45	DPIDR3, SCS Peripheral Identification Register 3	1544
D1.2.46	DPIDR4, SCS Peripheral Identification Register 4	1545
D1.2.47	DPIDR5, SCS Peripheral Identification Register 5	1546
D1.2.48	DPIDR6, SCS Peripheral Identification Register 6	1547
D1.2.49	DPIDR7, SCS Peripheral Identification Register 7	1548
D1.2.50	DSCEMCR, Debug Set Clear Exception and Monitor Control Register	1549
D1.2.51	DSCSR, Debug Security Control and Status Register	1551
D1.2.52	DWT_CIDR0, DWT Component Identification Register 0	1553
D1.2.53	DWT_CIDR1, DWT Component Identification Register 1	1554
D1.2.54	DWT_CIDR2, DWT Component Identification Register 2	1555
D1.2.55	DWT_CIDR3, DWT Component Identification Register 3	1556
D1.2.56	DWT_COMPn, DWT Comparator Register, n = 0 - 14	1557
D1.2.57	DWT_CPICNT, DWT CPI Count Register	1559
D1.2.58	DWT_CTRL, DWT Control Register	1561
D1.2.59	DWT_CYCCNT, DWT Cycle Count Register	1566
D1.2.60	DWT_DEVARCH, DWT Device Architecture Register	1567
D1.2.61	DWT_DEVTYPE, DWT Device Type Register	1569
D1.2.62	DWT_EXCCNT, DWT Exception Overhead Count Register	1570
D1.2.63	DWT_FOLD CNT, DWT Folded Instruction Count Register	1571

D1.2.64	DWT_FUNCTIONn, DWT Comparator Function Register, n = 0 - 14	1572
D1.2.65	DWT_LAR, DWT Software Lock Access Register	1577
D1.2.66	DWT_LSR, DWT Software Lock Status Register	1578
D1.2.67	DWT_LSUCNT, DWT LSU Count Register	1580
D1.2.68	DWT_PCSR, DWT Program Counter Sample Register	1581
D1.2.69	DWT_PIDR0, DWT Peripheral Identification Register 0	1583
D1.2.70	DWT_PIDR1, DWT Peripheral Identification Register 1	1584
D1.2.71	DWT_PIDR2, DWT Peripheral Identification Register 2	1585
D1.2.72	DWT_PIDR3, DWT Peripheral Identification Register 3	1586
D1.2.73	DWT_PIDR4, DWT Peripheral Identification Register 4	1587
D1.2.74	DWT_PIDR5, DWT Peripheral Identification Register 5	1588
D1.2.75	DWT_PIDR6, DWT Peripheral Identification Register 6	1589
D1.2.76	DWT_PIDR7, DWT Peripheral Identification Register 7	1590
D1.2.77	DWT_SLEPCNT, DWT Sleep Count Register	1591
D1.2.78	DWT_VMASKn, DWT Comparator Value Mask Register, n = 0 - 14	1593
D1.2.79	EPSR, Execution Program Status Register	1595
D1.2.80	ERRADDRn, Error Record Address Register, n = 0 - 55	1598
D1.2.81	ERRADDR2n, Error Record Address 2 Register, n = 0 - 55	1599
D1.2.82	ERRCTRLn, Error Record Control Register, n = 0 - 55	1601
D1.2.83	ERRDEVID, Error Record Device ID Register	1604
D1.2.84	ERRFRn, Error Record Feature Register, n = 0 - 55	1605
D1.2.85	ERRGSRn, RAS Fault Group Status Register	1609
D1.2.86	ERRIIDR, Error Implementer ID Register	1610
D1.2.87	ERRMISC0n, Error Record Miscellaneous 0 Register, n = 0 - 55	1611
D1.2.88	ERRMISC1n, Error Record Miscellaneous 1 Register, n = 0 - 55	1614
D1.2.89	ERRMISC2n, Error Record Miscellaneous 2 Register, n = 0 - 55	1615
D1.2.90	ERRMISC3n, Error Record Miscellaneous 3 Register, n = 0 - 55	1616
D1.2.91	ERRMISC4n, Error Record Miscellaneous 4 Register, n = 0 - 55	1617
D1.2.92	ERRMISC5n, Error Record Miscellaneous 5 Register, n = 0 - 55	1618
D1.2.93	ERRMISC6n, Error Record Miscellaneous 6 Register, n = 0 - 55	1619
D1.2.94	ERRMISC7n, Error Record Miscellaneous 7 Register, n = 0 - 55	1620
D1.2.95	ERRSTATUSn, Error Record Primary Status Register, n = 0 - 55	1621
D1.2.96	EXC_RETURN, Exception Return Payload	1628
D1.2.97	FAULTMASK, Fault Mask Register	1630
D1.2.98	FNC_RETURN, Function Return Payload	1631
D1.2.99	FPCAR, Floating-Point Context Address Register	1632
D1.2.100	FPCCR, Floating-Point Context Control Register	1633
D1.2.101	FPCTX, Floating-point context payload	1639
D1.2.102	FPDSCR, Floating-Point Default Status Control Register	1641
D1.2.103	FPSCR, Floating-point Status and Control Register	1643
D1.2.104	FP_CIDR0, FP Component Identification Register 0	1649
D1.2.105	FP_CIDR1, FP Component Identification Register 1	1650
D1.2.106	FP_CIDR2, FP Component Identification Register 2	1651
D1.2.107	FP_CIDR3, FP Component Identification Register 3	1652
D1.2.108	FP_COMPn, Flash Patch Comparator Register, n = 0 - 125	1653
D1.2.109	FP_CTRL, Flash Patch Control Register	1654
D1.2.110	FP_DEVARCH, FPB Device Architecture Register	1656
D1.2.111	FP_DEVTYPE, FPB Device Type Register	1658
D1.2.112	FP_LAR, FPB Software Lock Access Register	1659
D1.2.113	FP_LSR, FPB Software Lock Status Register	1660
D1.2.114	FP_PIDR0, FP Peripheral Identification Register 0	1662
D1.2.115	FP_PIDR1, FP Peripheral Identification Register 1	1663
D1.2.116	FP_PIDR2, FP Peripheral Identification Register 2	1664
D1.2.117	FP_PIDR3, FP Peripheral Identification Register 3	1665
D1.2.118	FP_PIDR4, FP Peripheral Identification Register 4	1666

D1.2.119	FP_PIDR5, FP Peripheral Identification Register 5	1667
D1.2.120	FP_PIDR6, FP Peripheral Identification Register 6	1668
D1.2.121	FP_PIDR7, FP Peripheral Identification Register 7	1669
D1.2.122	FP_REMAP, Flash Patch Remap Register	1670
D1.2.123	HFSR, HardFault Status Register	1671
D1.2.124	ICIALLU, Instruction Cache Invalidate All to PoU	1673
D1.2.125	ICIMVAU, Instruction Cache line Invalidate by Address to PoU	1674
D1.2.126	ICSR, Interrupt Control and State Register	1675
D1.2.127	ICTR, Interrupt Controller Type Register	1681
D1.2.128	ID_AFR0, Auxiliary Feature Register 0	1682
D1.2.129	ID_DFR0, Debug Feature Register 0	1683
D1.2.130	ID_ISAR0, Instruction Set Attribute Register 0	1685
D1.2.131	ID_ISAR1, Instruction Set Attribute Register 1	1687
D1.2.132	ID_ISAR2, Instruction Set Attribute Register 2	1689
D1.2.133	ID_ISAR3, Instruction Set Attribute Register 3	1691
D1.2.134	ID_ISAR4, Instruction Set Attribute Register 4	1693
D1.2.135	ID_ISAR5, Instruction Set Attribute Register 5	1695
D1.2.136	ID_MMFR0, Memory Model Feature Register 0	1697
D1.2.137	ID_MMFR1, Memory Model Feature Register 1	1699
D1.2.138	ID_MMFR2, Memory Model Feature Register 2	1700
D1.2.139	ID_MMFR3, Memory Model Feature Register 3	1701
D1.2.140	ID_PFR0, Processor Feature Register 0	1703
D1.2.141	ID_PFR1, Processor Feature Register 1	1705
D1.2.142	IPSR, Interrupt Program Status Register	1707
D1.2.143	ITM_CIDR0, ITM Component Identification Register 0	1708
D1.2.144	ITM_CIDR1, ITM Component Identification Register 1	1709
D1.2.145	ITM_CIDR2, ITM Component Identification Register 2	1710
D1.2.146	ITM_CIDR3, ITM Component Identification Register 3	1711
D1.2.147	ITM_DEVARCH, ITM Device Architecture Register	1712
D1.2.148	ITM_DEVTYPE, ITM Device Type Register	1714
D1.2.149	ITM_LAR, ITM Software Lock Access Register	1716
D1.2.150	ITM_LSR, ITM Software Lock Status Register	1717
D1.2.151	ITM_PIDR0, ITM Peripheral Identification Register 0	1719
D1.2.152	ITM_PIDR1, ITM Peripheral Identification Register 1	1720
D1.2.153	ITM_PIDR2, ITM Peripheral Identification Register 2	1721
D1.2.154	ITM_PIDR3, ITM Peripheral Identification Register 3	1722
D1.2.155	ITM_PIDR4, ITM Peripheral Identification Register 4	1723
D1.2.156	ITM_PIDR5, ITM Peripheral Identification Register 5	1724
D1.2.157	ITM_PIDR6, ITM Peripheral Identification Register 6	1725
D1.2.158	ITM_PIDR7, ITM Peripheral Identification Register 7	1726
D1.2.159	ITM_STIMn, ITM Stimulus Port Register, n = 0 - 255	1727
D1.2.160	ITM_TCR, ITM Trace Control Register	1729
D1.2.161	ITM_TERN, ITM Trace Enable Register, n = 0 - 7	1733
D1.2.162	ITM_TPR, ITM Trace Privilege Register	1734
D1.2.163	LO_BRANCH_INFO, Loop and branch tracking information	1735
D1.2.164	LR, Link Register	1736
D1.2.165	MAIR_ATTR, Memory Attribute Indirection Register Attributes	1737
D1.2.166	MMFAR, MemManage Fault Address Register	1739
D1.2.167	MMFSR, MemManage Fault Status Register	1740
D1.2.168	MPU_CTRL, MPU Control Register	1743
D1.2.169	MPU_MAIR0, MPU Memory Attribute Indirection Register 0	1745
D1.2.170	MPU_MAIR1, MPU Memory Attribute Indirection Register 1	1746
D1.2.171	MPU_RBAR, MPU Region Base Address Register	1747
D1.2.172	MPU_RBAR_An, MPU Region Base Address Register Alias, n = 1 - 3	1749
D1.2.173	MPU_RLAR, MPU Region Limit Address Register	1751

D1.2.174 MPU_RLAR_An, MPU Region Limit Address Register Alias, n = 1 - 3	1753
D1.2.175 MPU_RNR, MPU Region Number Register	1755
D1.2.176 MPU_TYPE, MPU Type Register	1756
D1.2.177 MSPLIM, Main Stack Pointer Limit Register	1757
D1.2.178 MVFR0, Media and VFP Feature Register 0	1758
D1.2.179 MVFR1, Media and VFP Feature Register 1	1760
D1.2.180 MVFR2, Media and VFP Feature Register 2	1763
D1.2.181 NSACR, Non-secure Access Control Register	1764
D1.2.182 NVIC_IABRn, Interrupt Active Bit Register, n = 0 - 15	1766
D1.2.183 NVIC_ICERn, Interrupt Clear Enable Register, n = 0 - 15	1767
D1.2.184 NVIC_ICPRn, Interrupt Clear Pending Register, n = 0 - 15	1768
D1.2.185 NVIC_IPRn, Interrupt Priority Register, n = 0 - 123	1769
D1.2.186 NVIC_ISERn, Interrupt Set Enable Register, n = 0 - 15	1770
D1.2.187 NVIC_ISPRn, Interrupt Set Pending Register, n = 0 - 15	1771
D1.2.188 NVIC_ITNSn, Interrupt Target Non-secure Register, n = 0 - 15	1773
D1.2.189 PAC_KEY_P_n, Pointer Authentication Key Privileged, n = 0 - 3	1774
D1.2.190 PAC_KEY_U_n, Pointer Authentication Key Unprivileged, n = 0 - 3	1775
D1.2.191 PC, Program Counter	1776
D1.2.192 PMU_AUTHSTATUS, Performance Monitoring Unit Authentication Status Register	1777
D1.2.193 PMU_CCFILTR, Performance Monitoring Unit Cycle Counter Filter Register	1780
D1.2.194 PMU_CCNTR, Performance Monitoring Unit Cycle Counter Register	1781
D1.2.195 PMU_CIDR0, Performance Monitoring Unit Component Identification Register 0	1782
D1.2.196 PMU_CIDR1, Performance Monitoring Unit Component Identification Register 1	1783
D1.2.197 PMU_CIDR2, Performance Monitoring Unit Component Identification Register 2	1784
D1.2.198 PMU_CIDR3, Performance Monitoring Unit Component Identification Register 3	1785
D1.2.199 PMU_CNTENCLR, Performance Monitoring Unit Count Enable Clear Register	1786
D1.2.200 PMU_CNTENSET, Performance Monitoring Unit Count Enable Set Register	1788
D1.2.201 PMU_CTRL, Performance Monitoring Unit Control Register	1790
D1.2.202 PMU_DEVARCH, Performance Monitoring Unit Device Architecture Register	1792
D1.2.203 PMU_DEVTYPE, Performance Monitoring Unit Device Type Register	1794
D1.2.204 PMU_EVCNTRn, Performance Monitoring Unit Event Counter Register	1795
D1.2.205 PMU_EVTYPERn, Performance Monitoring Unit Event Type and Filter Register	1796
D1.2.206 PMU_INTENCLR, Performance Monitoring Unit Interrupt Enable Clear Register	1797
D1.2.207 PMU_INTENSET, Performance Monitoring Unit Interrupt Enable Set Register	1799
D1.2.208 PMU_OVSCLR, Performance Monitoring Unit Overflow Flag Status Clear Register	1801
D1.2.209 PMU_OVSSET, Performance Monitoring Unit Overflow Flag Status Set Register	1803
D1.2.210 PMU_PIDR0, Performance Monitoring Unit Peripheral Identification Register 0	1805
D1.2.211 PMU_PIDR1, Performance Monitoring Unit Peripheral Identification Register 1	1806
D1.2.212 PMU_PIDR2, Performance Monitoring Unit Peripheral Identification Register 2	1807

D1.2.213 PMU_PIDR3, Performance Monitoring Unit Peripheral Identification Register 3	1808
D1.2.214 PMU_PIDR4, Performance Monitoring Unit Peripheral Identification Register 4	1809
D1.2.215 PMU_SWINC, Performance Monitoring Unit Software Increment Register	1810
D1.2.216 PMU_TYPE, Performance Monitoring Unit Type Register	1811
D1.2.217 PRIMASK, Exception Mask Register	1813
D1.2.218 PSPLIM, Process Stack Pointer Limit Register	1814
D1.2.219 Qn, Vector register, n = 0 - 7	1815
D1.2.220 Rn, General-Purpose Register, n = 0 - 12	1816
D1.2.221 RETPSR, Combined Exception Return Program Status Registers	1817
D1.2.222 REVIDR, Revision ID Register	1819
D1.2.223 RFSR, RAS Fault Status Register	1820
D1.2.224 Sn, Floating-point Single-precision register, n = 0 - 31	1822
D1.2.225 SAU_CTRL, SAU Control Register	1823
D1.2.226 SAU_RBAR, SAU Region Base Address Register	1825
D1.2.227 SAU_RLAR, SAU Region Limit Address Register	1826
D1.2.228 SAU_RNR, SAU Region Number Register	1828
D1.2.229 SAU_TYPE, SAU Type Register	1829
D1.2.230 SCR, System Control Register	1830
D1.2.231 SFAR, Secure Fault Address Register	1832
D1.2.232 SFSR, Secure Fault Status Register	1833
D1.2.233 SHCSR, System Handler Control and State Register	1836
D1.2.234 SHPR1, System Handler Priority Register 1	1844
D1.2.235 SHPR2, System Handler Priority Register 2	1846
D1.2.236 SHPR3, System Handler Priority Register 3	1847
D1.2.237 SP, Current Stack Pointer Register	1849
D1.2.238 SP_NS, Current Stack Pointer register (Non-secure)	1850
D1.2.239 STIR, Software Triggered Interrupt Register	1851
D1.2.240 SYST_CALIB, SysTick Calibration Value Register	1852
D1.2.241 SYST_CSR, SysTick Control and Status Register	1854
D1.2.242 SYST_CVR, SysTick Current Value Register	1857
D1.2.243 SYST_RVR, SysTick Reload Value Register	1859
D1.2.244 TPIU_ACPR, TPIU Asynchronous Clock Prescaler Register	1860
D1.2.245 TPIU_CIDR0, TPIU Component Identification Register 0	1861
D1.2.246 TPIU_CIDR1, TPIU Component Identification Register 1	1862
D1.2.247 TPIU_CIDR2, TPIU Component Identification Register 2	1863
D1.2.248 TPIU_CIDR3, TPIU Component Identification Register 3	1864
D1.2.249 TPIU_CLAIMCLR, TPIU Claim Tag Clear Register	1865
D1.2.250 TPIU_CLAIMSET, TPIU Claim Tag Set Register	1867
D1.2.251 TPIU_CSPSR, TPIU Current Parallel Port Sizes Register	1869
D1.2.252 TPIU_DEVID, TPIU Device Identifier Register	1870
D1.2.253 TPIU_DEVTYPE, TPIU Device Type Register	1872
D1.2.254 TPIU_FFCR, TPIU Formatter and Flush Control Register	1874
D1.2.255 TPIU_FFSR, TPIU Formatter and Flush Status Register	1876
D1.2.256 TPIU_LAR, TPIU Software Lock Access Register	1878
D1.2.257 TPIU_LSR, TPIU Software Lock Status Register	1879
D1.2.258 TPIU_PIDR0, TPIU Peripheral Identification Register 0	1881
D1.2.259 TPIU_PIDR1, TPIU Peripheral Identification Register 1	1882
D1.2.260 TPIU_PIDR2, TPIU Peripheral Identification Register 2	1883
D1.2.261 TPIU_PIDR3, TPIU Peripheral Identification Register 3	1884
D1.2.262 TPIU_PIDR4, TPIU Peripheral Identification Register 4	1885
D1.2.263 TPIU_PIDR5, TPIU Peripheral Identification Register 5	1886
D1.2.264 TPIU_PIDR6, TPIU Peripheral Identification Register 6	1887
D1.2.265 TPIU_PIDR7, TPIU Peripheral Identification Register 7	1888

D1.2.266	TPIU_PSCR, TPIU Periodic Synchronization Control Register	1889
D1.2.267	TPIU_SPPR, TPIU Selected Pin Protocol Register	1891
D1.2.268	TPIU_SSPPR, TPIU Supported Parallel Port Sizes Register	1893
D1.2.269	TT_RESP, Test Target Response Payload	1894
D1.2.270	UFSR, UsageFault Status Register	1897
D1.2.271	VPR, Vector Predication Status and Control Register	1900
D1.2.272	VTOR, Vector Table Offset Register	1902
D1.2.273	XPSR, Combined Program Status Registers	1903

Part E Armv8-M Pseudocode

Chapter E1

Arm Pseudocode Definition

E1.1	About the Arm pseudocode	1907
E1.1.1	General limitations of Arm pseudocode	1907
E1.2	Data types	1908
E1.2.1	General data type rules	1908
E1.2.2	Bitstrings	1908
E1.2.3	Integers	1909
E1.2.4	Reals	1909
E1.2.5	Booleans	1910
E1.2.6	Enumerations	1910
E1.2.7	Structures	1911
E1.2.8	Tuples	1912
E1.2.9	Arrays	1912
E1.3	Operators	1914
E1.3.1	Relational operators	1914
E1.3.2	Boolean operators	1914
E1.3.3	Bitstring operators	1915
E1.3.4	Arithmetic operators	1916
E1.3.5	The assignment operator	1917
E1.3.6	Precedence rules	1918
E1.3.7	Conditional expressions	1918
E1.3.8	Operator polymorphism	1918
E1.4	Statements and control structures	1920
E1.4.1	Statements and Indentation	1920
E1.4.2	Function and procedure calls	1920
E1.4.3	Conditional control structures	1921
E1.4.4	Loop control structures	1922
E1.4.5	Special statements	1923
E1.4.6	Comments	1924
E1.5	Built in functions	1925
E1.6	Arm pseudocode definition index	1926
E1.7	Additional functions	1928
E1.7.1	IsSee()	1928
E1.7.2	IsUndefined()	1928

Chapter E2

Pseudocode Specification

E2.1	Alphabetical Pseudocode List	1930
E2.1.1	_AdvanceVPTState	1930
E2.1.2	_CommitState	1930
E2.1.3	_ITStateChanged	1930
E2.1.4	_Mem	1930
E2.1.5	_NextInstrAddr	1930
E2.1.6	_NextInstrITState	1930

Contents

E2.1.7	_PCChanged	1930
E2.1.8	_PendingFetchFault	1930
E2.1.9	_PendingReturnOperation	1931
E2.1.10	_RName	1931
E2.1.11	_S	1931
E2.1.12	_SP	1931
E2.1.13	Abs	1932
E2.1.14	AccessAttributes	1932
E2.1.15	AccType	1932
E2.1.16	ActivateException	1932
E2.1.17	ActiveFPState	1933
E2.1.18	AddressDescriptor	1933
E2.1.19	AddrType	1933
E2.1.20	AddWithCarry	1933
E2.1.21	AdvSIMDEExpandImm	1934
E2.1.22	AlgorithmPAC	1934
E2.1.23	Align	1934
E2.1.24	ArchVersion	1934
E2.1.25	ASR	1935
E2.1.26	ASR_C	1935
E2.1.27	BeatComplete	1935
E2.1.28	BeatSchedule	1936
E2.1.29	BigEndian	1936
E2.1.30	BigEndianReverse	1936
E2.1.31	BitCount	1937
E2.1.32	BitReverseShiftRight	1937
E2.1.33	BranchCall	1937
E2.1.34	BranchReturn	1937
E2.1.35	BranchTo	1938
E2.1.36	BTIEnabled	1938
E2.1.37	BusFaultBarrier	1939
E2.1.38	CallSupervisor	1939
E2.1.39	CanDebugAccessFP	1939
E2.1.40	CanHaltOnEvent	1939
E2.1.41	CanPendMonitorOnEvent	1940
E2.1.42	CdeImpDefValue	1940
E2.1.43	CheckCDEDecodeFaults	1940
E2.1.44	CheckCPDecodeFaults	1940
E2.1.45	CheckCPEnabled	1941
E2.1.46	CheckDecodeFaults	1941
E2.1.47	CheckFPDecodeFaults	1941
E2.1.48	CheckPermission	1942
E2.1.49	ClearEventRegister	1943
E2.1.50	ClearExclusiveByAddress	1943
E2.1.51	ClearExclusiveLocal	1943
E2.1.52	ClearInFlightInstructions	1943
E2.1.53	ComparePriorities	1943
E2.1.54	ComputePAC	1943
E2.1.55	Cond	1945
E2.1.56	ConditionHolds	1945
E2.1.57	ConditionPassed	1946
E2.1.58	ConstrainUnpredictable	1946
E2.1.59	ConstrainUnpredictableBits	1946
E2.1.60	ConstrainUnpredictableBool	1946
E2.1.61	ConstrainUnpredictableInteger	1946

E2.1.62	ConsumeExcStackFrame	1946
E2.1.63	ConsumptionOfSpeculativeDataBarrier	1947
E2.1.64	Coproc_Accepted	1947
E2.1.65	Coproc_DoneLoading	1947
E2.1.66	Coproc_DoneStoring	1947
E2.1.67	Coproc_GetOneWord	1947
E2.1.68	Coproc_GetTwoWords	1948
E2.1.69	Coproc_GetWordToStore	1948
E2.1.70	Coproc_InternalOperation	1948
E2.1.71	Coproc_SendLoadedWord	1948
E2.1.72	Coproc_SendOneWord	1948
E2.1.73	Coproc_SendTwoWords	1948
E2.1.74	CoprocType	1948
E2.1.75	CountLeadingSignBits	1949
E2.1.76	CountLeadingZeroBits	1949
E2.1.77	CPDef	1949
E2.1.78	CreateException	1949
E2.1.79	CreatePAC	1950
E2.1.80	CurrentCond	1950
E2.1.81	CurrentMode	1950
E2.1.82	CurrentModelsPrivileged	1951
E2.1.83	CX_op0	1951
E2.1.84	CX_op1	1951
E2.1.85	CX_op2	1951
E2.1.86	CX_op3	1951
E2.1.87	D	1952
E2.1.88	DAPCheck	1952
E2.1.89	DataMemoryBarrier	1953
E2.1.90	DataSynchronizationBarrier	1953
E2.1.91	DeActivate	1953
E2.1.92	Debug_authentication	1954
E2.1.93	DebugCanMaskInts	1954
E2.1.94	DebugEventCause	1954
E2.1.95	DebugRegisterTransfer	1954
E2.1.96	DecodeExecute	1959
E2.1.97	DecodeImmShift	1959
E2.1.98	DecodeRegShift	1959
E2.1.99	DefaultCond	1959
E2.1.100	DefaultExclInfo	1960
E2.1.101	DefaultMemoryAttributes	1960
E2.1.102	DefaultPermissions	1961
E2.1.103	DerivedLateArrival	1962
E2.1.104	DeviceType	1963
E2.1.105	DWT_AddressCompare	1963
E2.1.106	DWT_CycCountMatch	1963
E2.1.107	DWT_DataAddressMatch	1963
E2.1.108	DWT_DataMatch	1964
E2.1.109	DWT_DataValueMatch	1965
E2.1.110	DWT_InstructionAddressMatch	1966
E2.1.111	DWT_InstructionMatch	1967
E2.1.112	DWT_ValidMatch	1968
E2.1.113	Elem	1968
E2.1.114	EndOfInstruction	1968
E2.1.115	EventRegistered	1969
E2.1.116	ExceptionActiveBitCount	1969

E2.1.117	ExceptionDetails	1969
E2.1.118	ExceptionEnabled	1970
E2.1.119	ExceptionEntry	1971
E2.1.120	ExceptionPriority	1971
E2.1.121	ExceptionReturn	1972
E2.1.122	ExceptionTaken	1973
E2.1.123	ExceptionTargetsSecure	1974
E2.1.124	ExclInfo	1975
E2.1.125	ExclusiveMonitorsPass	1976
E2.1.126	ExecBeats	1976
E2.1.127	ExecuteFPCheck	1977
E2.1.128	ExecutionPriority	1978
E2.1.129	Extend	1979
E2.1.130	ExternalInvasiveDebugEnabled	1979
E2.1.131	ExternalNoninvasiveDebugEnabled	1979
E2.1.132	ExternalSecureInvasiveDebugEnabled	1979
E2.1.133	ExternalSecureNoninvasiveDebugEnabled	1980
E2.1.134	ExternalSecureSelfHostedDebugEnabled	1980
E2.1.135	ExtType	1980
E2.1.136	FaultNumbers	1980
E2.1.137	FetchInstr	1980
E2.1.138	FindMemPriv	1982
E2.1.139	FindPriv	1982
E2.1.140	FixedToFP	1982
E2.1.141	FPAbs	1983
E2.1.142	FPAdd	1983
E2.1.143	FPB_CheckBreakPoint	1983
E2.1.144	FPB_CheckMatchAddress	1983
E2.1.145	FPCompare	1984
E2.1.146	FPConvertNaN	1984
E2.1.147	FPConvertNaNBase	1984
E2.1.148	FPDefaultNaN	1985
E2.1.149	FPCDiv	1985
E2.1.150	FPCDoubleToHalf	1986
E2.1.151	FPCDoubleToSingle	1986
E2.1.152	FPExc	1986
E2.1.153	FPHalfToDouble	1987
E2.1.154	FPHalfToSingle	1987
E2.1.155	FPInfinity	1987
E2.1.156	FPMMax	1988
E2.1.157	FPMMaxNormal	1988
E2.1.158	FPMMaxNum	1988
E2.1.159	FPMMin	1989
E2.1.160	FPMMinNum	1989
E2.1.161	FPMul	1989
E2.1.162	FPMulAdd	1990
E2.1.163	FPNeg	1991
E2.1.164	FPPProcessException	1991
E2.1.165	FPPProcessNaN	1991
E2.1.166	FPPProcessNaNs	1992
E2.1.167	FPPProcessNaNs3	1992
E2.1.168	FPRound	1993
E2.1.169	FPRoundBase	1993
E2.1.170	FPRoundCV	1994
E2.1.171	FPRoundInt	1994

Contents

E2.1.172	FPSingleToDouble	1995
E2.1.173	FPSingleToHalf	1996
E2.1.174	FPSqrt	1996
E2.1.175	FPSub	1996
E2.1.176	FPToFixed	1997
E2.1.177	FPToFixedDirected	1998
E2.1.178	FPType	1999
E2.1.179	FPUnpack	1999
E2.1.180	FPUnpackBase	1999
E2.1.181	FPUnpackCV	2000
E2.1.182	FPZero	2000
E2.1.183	FunctionReturn	2001
E2.1.184	GenerateCoproprocessorException	2002
E2.1.185	GenerateDebugEventResponse	2002
E2.1.186	GenerateIntegerZeroDivide	2002
E2.1.187	GetActiveChains	2002
E2.1.188	GetCurlInstrBeat	2003
E2.1.189	GetInstrExecState	2003
E2.1.190	GetMveScalarReadRegs	2004
E2.1.191	GetMveScalarWriteRegs	2008
E2.1.192	Halt	2010
E2.1.193	Halted	2011
E2.1.194	HaltingDebugAllowed	2011
E2.1.195	HandleException	2011
E2.1.196	HandleExceptionTransitions	2011
E2.1.197	HandleLO	2013
E2.1.198	HasArchVersion	2014
E2.1.199	HaveAlgorithmPAC	2014
E2.1.200	HaveDebugMonitor	2014
E2.1.201	HaveDSPExt	2014
E2.1.202	HaveDWT	2014
E2.1.203	HaveFPB	2014
E2.1.204	HaveFPEExt	2015
E2.1.205	HaveHaltingDebug	2015
E2.1.206	HaveITM	2015
E2.1.207	HaveLOBExt	2015
E2.1.208	HaveMainExt	2015
E2.1.209	HaveMve	2015
E2.1.210	HaveMveOrFPEExt	2015
E2.1.211	HavePACBTIExt	2015
E2.1.212	HaveSecurityExt	2016
E2.1.213	HaveSysTick	2016
E2.1.214	HaveUDE	2016
E2.1.215	HighestPri	2016
E2.1.216	HighestSetBit	2016
E2.1.217	Hint_Debug	2016
E2.1.218	Hint_PreloadData	2017
E2.1.219	Hint_PreloadDataForWrite	2017
E2.1.220	Hint_PreloadInstr	2017
E2.1.221	Hint_Yield	2017
E2.1.222	IDAUCheck	2017
E2.1.223	IgnoreFaultsType	2017
E2.1.224	InITBlock	2017
E2.1.225	InstrCanChain	2018
E2.1.226	InstrExecState	2019

E2.1.227	InstrType	2019
E2.1.228	InstructionAdvance	2020
E2.1.229	InstructionExecute	2020
E2.1.230	InstructionsInFlight	2022
E2.1.231	InstructionSynchronizationBarrier	2022
E2.1.232	InstStateCheck	2022
E2.1.233	Int	2023
E2.1.234	IntegerZeroDivideTrappingEnabled	2023
E2.1.235	InvalidateFPRegs	2023
E2.1.236	InVPTBlock	2023
E2.1.237	IsAccessible	2024
E2.1.238	IsActiveForState	2024
E2.1.239	IsAligned	2024
E2.1.240	IsBKPTInstruction	2024
E2.1.241	IsCdeBeatWiseInstruction	2025
E2.1.242	IsCPEnabled	2025
E2.1.243	IsCPInstruction	2025
E2.1.244	IsCplInstructionSecureOnly	2026
E2.1.245	IsDebugState	2026
E2.1.246	IsDWTConfigUnpredictable	2026
E2.1.247	IsDWTEnabled	2028
E2.1.248	IsExceptionTargetConfigurable	2028
E2.1.249	IsExclusiveGlobal	2029
E2.1.250	IsExclusiveLocal	2029
E2.1.251	IsFirstBeat	2029
E2.1.252	IsIrqValid	2029
E2.1.253	IsLastBeat	2029
E2.1.254	IsLastLowOverheadLoop	2029
E2.1.255	IsLEInstruction	2030
E2.1.256	IsLoadStoreClearMultInstruction	2030
E2.1.257	IsMveAccessFPSCR_C	2030
E2.1.258	IsMveBeatWiseInstruction	2030
E2.1.259	IsMveLoadStoreInstruction	2032
E2.1.260	IsOnes	2032
E2.1.261	IsPPB	2032
E2.1.262	IsReqExcPriNeg	2032
E2.1.263	IsReturn	2033
E2.1.264	IsSCS	2033
E2.1.265	IsSecure	2033
E2.1.266	IsZero	2033
E2.1.267	IsZeroBit	2034
E2.1.268	ITAdvance	2034
E2.1.269	ITSTATE	2034
E2.1.270	ITSTATEType	2034
E2.1.271	LastInITBlock	2034
E2.1.272	LoadWritePC	2034
E2.1.273	LockedUp	2035
E2.1.274	Lockup	2035
E2.1.275	LookupRName	2035
E2.1.276	LookupSP	2036
E2.1.277	LookupSP_with_security_mode	2036
E2.1.278	LookupSPLim	2036
E2.1.279	LowestSetBit	2037
E2.1.280	LR	2037
E2.1.281	LSL	2037

Contents

E2.1.282	LSL_C	2037
E2.1.283	LSR	2037
E2.1.284	LSR_C	2038
E2.1.285	LTPSIZE	2038
E2.1.286	MAIRDecode	2038
E2.1.287	MarkExclusiveGlobal	2039
E2.1.288	MarkExclusiveLocal	2039
E2.1.289	Max	2039
E2.1.290	MaxExceptionNum	2040
E2.1.291	MemA	2040
E2.1.292	MemA_MVE	2040
E2.1.293	MemA_with_priv	2040
E2.1.294	MemA_with_priv_security	2041
E2.1.295	MemD_with_priv_security	2043
E2.1.296	MemI	2044
E2.1.297	MemO	2044
E2.1.298	MemoryAttributes	2044
E2.1.299	MemType	2045
E2.1.300	MemU	2045
E2.1.301	MemU_unpriv	2045
E2.1.302	MemU_with_priv	2045
E2.1.303	MergeExclInfo	2046
E2.1.304	Min	2047
E2.1.305	MonitorCanPreempt	2047
E2.1.306	MPUCheck	2047
E2.1.307	NextInstrAddr	2049
E2.1.308	NextInstrITState	2049
E2.1.309	NoninvasiveDebugAllowed	2049
E2.1.310	Ones	2049
E2.1.311	PACCellInvShuffle	2049
E2.1.312	PACCellShuffle	2050
E2.1.313	PACEnabled	2050
E2.1.314	PACInvSub	2050
E2.1.315	PACKey	2051
E2.1.316	PACKeys	2051
E2.1.317	PACMult	2052
E2.1.318	PACSubQ3	2052
E2.1.319	PACSubQ5	2052
E2.1.320	PC	2053
E2.1.321	PEMode	2053
E2.1.322	PendingDebugHalt	2053
E2.1.323	PendingDebugMonitor	2053
E2.1.324	PendingExceptionDetails	2053
E2.1.325	PendReturnOperation	2054
E2.1.326	Permissions	2054
E2.1.327	PMU_CounterIncrement	2054
E2.1.328	PMU_HandleOverflow	2055
E2.1.329	PolynomialMult	2055
E2.1.330	PopStack	2056
E2.1.331	PreserveFPState	2058
E2.1.332	Privilege	2060
E2.1.333	ProcessorID	2060
E2.1.334	PushCalleeStack	2060
E2.1.335	PushStack	2061
E2.1.336	Q	2062

E2.1.337	R	2063
E2.1.338	RaiseAsyncBusFault	2063
E2.1.339	RasImpDefValue	2063
E2.1.340	RawExecutionPriority	2063
E2.1.341	Replicate	2064
E2.1.342	ResetRegs	2064
E2.1.343	RestrictedNSPri	2064
E2.1.344	RF	2064
E2.1.345	RFD	2065
E2.1.346	RName	2065
E2.1.347	RNames	2066
E2.1.348	ROR	2066
E2.1.349	ROR_C	2066
E2.1.350	RotCell	2066
E2.1.351	RoundDown	2066
E2.1.352	RoundTowardsZero	2067
E2.1.353	RoundUp	2067
E2.1.354	RRX	2067
E2.1.355	RRX_C	2067
E2.1.356	RSPCheck	2067
E2.1.357	RZ	2067
E2.1.358	S	2068
E2.1.359	Sat	2068
E2.1.360	SatQ	2068
E2.1.361	SAttributes	2068
E2.1.362	SCS_UpdateStatusRegs	2068
E2.1.363	SecureDebugMonitorAllowed	2068
E2.1.364	SecureHaltingDebugAllowed	2069
E2.1.365	SecureNoninvasiveDebugAllowed	2069
E2.1.366	SecurityCheck	2069
E2.1.367	SecurityState	2071
E2.1.368	SendEvent	2071
E2.1.369	SerializeVFP	2071
E2.1.370	SetActive	2071
E2.1.371	SetDWTDebugEvent	2071
E2.1.372	SetEventRegister	2072
E2.1.373	SetExclusiveMonitors	2072
E2.1.374	SetITSTATEAndCommit	2072
E2.1.375	SetPending	2072
E2.1.376	SetThisInstrDetails	2073
E2.1.377	SetVPTMask	2073
E2.1.378	Shift	2073
E2.1.379	Shift_C	2073
E2.1.380	SignedSat	2074
E2.1.381	SignedSatQ	2074
E2.1.382	SignExtend	2074
E2.1.383	Sleeping	2074
E2.1.384	SleepOnExit	2074
E2.1.385	SP	2074
E2.1.386	SP_Main	2075
E2.1.387	SP_Main_NonSecure	2075
E2.1.388	SP_Main_Secure	2075
E2.1.389	SP_Process	2075
E2.1.390	SP_Process_NonSecure	2076
E2.1.391	SP_Process_Secure	2076

E2.1.392	SpeculativeSynchronizationBarrier	2076
E2.1.393	SRTYPE	2076
E2.1.394	Stack	2076
E2.1.395	StandardFPSCRValue	2077
E2.1.396	SteppingDebug	2077
E2.1.397	SynchronizeBusFault	2078
E2.1.398	T32ExpandImm	2078
E2.1.399	T32ExpandImm_C	2078
E2.1.400	TailChain	2079
E2.1.401	TakePreserveFPEXCEPTION	2079
E2.1.402	TakeReset	2080
E2.1.403	ThisInstr	2081
E2.1.404	ThisInstrAddr	2081
E2.1.405	ThisInstrITState	2081
E2.1.406	ThisInstrLength	2082
E2.1.407	TopLevel	2082
E2.1.408	TTRSP	2083
E2.1.409	TweakCellInvRot	2084
E2.1.410	TweakCellRot	2084
E2.1.411	TweakInvShuffle	2084
E2.1.412	TweakShuffle	2085
E2.1.413	UnprivHaltingDebugAllowed	2085
E2.1.414	UnprivHaltingDebugEnabled	2085
E2.1.415	UnsignedSat	2085
E2.1.416	UnsignedSatQ	2086
E2.1.417	UpdateDebugEnable	2086
E2.1.418	UpdateFPCCR	2086
E2.1.419	ValidateAddress	2087
E2.1.420	ValidateExceptionReturn	2089
E2.1.421	ValidatePAC	2090
E2.1.422	VCX_op0	2090
E2.1.423	VCX_op1	2090
E2.1.424	VCX_op2	2090
E2.1.425	VCX_op3	2091
E2.1.426	Vector	2091
E2.1.427	VectorCatchDebug	2091
E2.1.428	VFPEXCBarrier	2092
E2.1.429	VFPEXExpandImm	2092
E2.1.430	VFPNEGmul	2092
E2.1.431	VFPsmallRegisterBank	2093
E2.1.432	ViolatesSPLim	2093
E2.1.433	VPTActive	2093
E2.1.434	VPTAdvance	2093
E2.1.435	WaitForEvent	2094
E2.1.436	WaitForInterrupt	2094
E2.1.437	ZeroExtend	2094
E2.1.438	Zeros	2094

Part F Debug Packet Protocols

Chapter F1

ITM and DWT Packet Protocol Specification

F1.1	About the ITM and DWT packets	2097
F1.1.1	Uses of ITM and DWT packets	2097
F1.1.2	ITM and DWT protocol packet headers	2097

F1.1.3	Packet transmission by the trace sink	.2098
F1.2	Alphabetical list of DWT and ITM packets	.2099
F1.2.1	Data Trace Data Address packet	.2099
F1.2.2	Data Trace Data Value packet	.2100
F1.2.3	Data Trace Match packet	.2102
F1.2.4	Data Trace PC Value packet	.2103
F1.2.5	Event Counter packet	.2105
F1.2.6	Exception Trace packet	.2106
F1.2.7	Extension packet	.2107
F1.2.8	Global Timestamp 1 packet	.2109
F1.2.9	Global Timestamp 2 packet	.2111
F1.2.10	Instrumentation packet	.2113
F1.2.11	Local Timestamp 1 packet	.2114
F1.2.12	Local Timestamp 2 packet	.2116
F1.2.13	Overflow packet	.2117
F1.2.14	Periodic PC Sample packet	.2118
F1.2.15	PMU overflow packet	.2119
F1.2.16	Synchronization packet	.2120

Part G Glossary

Preface

This preface introduces the Armv8-M Architecture Reference Manual. It contains the following sections:

[About this manual.](#)

[Using this manual.](#)

[Conventions.](#)

[Additional reading.](#)

[Feedback.](#)

About this manual

This manual documents the microcontroller profile of version 8 of the Arm Architecture, the Armv8-M architecture profile. For short definitions of all the Armv8 profiles, see [A1.2 About the Armv8 architecture, and architecture profiles](#).

This manual has the following parts:

Part A Provides an introduction to the Armv8-M architecture.

Part B Describes the architectural rules.

Part C Describes the T32 instruction set.

Part D Describes the registers.

Part E Describes the Armv8-M pseudocode.

Part F Describes the packet protocols.

Using this manual

The information in this manual is organized into parts, as described in this section.

Part A, Armv8-M Architecture Introduction and Overview

Part A gives an overview of the Armv8-M architecture profile, including its relationship to the other Arm PE architectures. It introduces the terminology that describes the architecture, and gives an overview of the optional architectural extensions. It contains the following chapter:

[Chapter A1 *Introduction*](#)

Read this for an introduction to the Armv8-M architecture.

Part B, Armv8-M Architecture Rules

Part B describes the architecture rules. It contains the following chapters:

[Chapter B1 *Resets*](#)

Read this for a description of the reset rules.

[Chapter B2 *Power Management*](#)

Read this for a description of the power management rules.

[Chapter B3 *Programmers' Model*](#)

Read this for a description of the programmers model rules.

[Chapter B4 *Floating-point Support*](#)

Read this for a description of the floating-point support rules.

[Chapter B5 *Vector Extension*](#)

Read this for a description of the Vector Extension support rules.

[Chapter B7 *Memory Model*](#)

Read this for a description of the memory model rules.

[Chapter B8 *The System Address Map*](#)

Read this for a description of the system address map rules.

[Chapter B9 *Synchronization and Semaphores*](#)

Read this for a description of the rules on non-blocking synchronization of shared memory.

[Chapter B10 *The Armv8-M Protected Memory System Architecture*](#)

Read this for a description of the protected memory system architecture rules.

[Chapter B11 *The System Timer, SysTick*](#)

Read this for a description of the system timer rules.

[Chapter B12 *Nested Vectored Interrupt Controller*](#)

Read this for a description of the *Nested Vectored Interrupt Controller* (NVIC) rules.

[Chapter B13 *Debug*](#)

Read this for a description of the debug rules.

[Chapter B14 *Debug and Trace Components*](#)

Read this for a description of the debug and trace component rules.

[Chapter B15 *The Performance Monitors Extension*](#)

Read this for a description of the Performance Monitors Extension.

[Chapter B16 *Reliability, Availability, and Serviceability \(RAS\) Extension*](#)

Read this for a description of the Reliability, Availability, and Serviceability (RAS) Extension.

Part C, Armv8-M Instructions

Part C describes the instructions. It contains the following chapters:

[Chapter C1 *Instruction Set Overview*](#)

Read this for an overview of the instruction set and the instruction set encoding.

[Chapter C2 *Instruction Specification*](#)

Read this for a description of each instruction, arranged by instruction mnemonic.

Part D, Armv8-M Registers

Part D describes the registers. It contains the following chapter:

[Chapter D1 *Register and Payload Specification*](#)

Read this for a description of the registers.

Part E, Armv8-M Pseudocode

Part E describes the pseudocode. It contains the following chapters:

[Chapter E1 *Arm Pseudocode Definition*](#)

Read this for a definition of the pseudocode that Arm documentation uses.

[Chapter E2 *Pseudocode Specification*](#)

Read this for a description of the pseudocode.

Part F, Packet Protocols

Part F describes the packet protocols. It contains the following chapter:

[Chapter F1 *ITM and DWT Packet Protocol Specification*](#)

Read this for a description of the protocol for packets that are used to send the data generated by the ITM and DWT to an external debugger.

Conventions

The following sections describe conventions that this manual can use:

[Typographical conventions.](#)

[Signals.](#)

[Numbers.](#)

[Pseudocode descriptions.](#)

[Assembler syntax descriptions.](#)

Typographical conventions

The typographical conventions are:

italic

Introduces special terminology, and denotes citations.

bold

Denotes signal names, and is used for terms in descriptive lists, where appropriate.

`monospace`

Used for assembler syntax descriptions, pseudocode, and source code examples.

Also used in the main text for instruction mnemonics and for references to other items appearing in assembler syntax descriptions, pseudocode, and source code examples.

SMALLCAPS

Used for a few terms that have specific technical meanings, and that are included in the Glossary.

Colored text Indicates a link. This can be:

- A URL, for example <https://developer.arm.com/>.
- A cross-reference, that includes the page number of the referenced information if it is not on the current page, for example, [Chapter B2 *Power Management*](#).
- A link, to a chapter or appendix, or to a glossary entry, or to the section of the manual that defines the colored term, for example [tail-chaining](#).

Signals

In general this manual does not define processor signals, but it does include some signal examples and recommendations.

The signal conventions are:

Signal level The level of an asserted signal depends on whether the signal is active-HIGH or active-LOW. Asserted means:

- HIGH for active-HIGH signals.
- LOW for active-LOW signals.

Lowercase n At the start or end of a signal name denotes an active-LOW signal.

Numbers

Numbers are normally written in decimal. Binary numbers are preceded by `0b`, and hexadecimal numbers by `0x`. In both cases, the prefix and the associated value are written in a `monospace` font, for example `0xFFFF0000`.

For both binary and hexadecimal numbers, where a bit is represented by the letter `x`, the value is irrelevant. For example a value expressed as `0b1x` can be either `0b11` or `0b10`.

To improve readability, long numbers can be written with an underscore separator between every four characters, for example `0xFFFF_0000_0000_0000`. Ignore any underscores when interpreting the value of a number.

Pseudocode descriptions

This manual uses a form of pseudocode to provide precise descriptions of the specified functionality. This pseudocode is written in a `monospace` font, and is described in [Chapter E1 Arm Pseudocode Definition](#).

Assembler syntax descriptions

This manual contains numerous syntax descriptions for assembler instructions and for components of assembler instructions. These are shown in a `monospace` font, and use the conventions described in [C1.2.5 Standard assembler syntax fields](#).

Additional reading

This section lists relevant publications from Arm and third parties.

See <https://developer.arm.com>, for access to Arm documentation.

Arm publications

- *Arm® Debug Interface Architecture Specification ADIv5.0 to ADIv5.2* (ARM IHI 0031).
- *Arm® Debug Interface Architecture Specification ADIv6.0* (ARM IHI 0074).
- *Arm® CoreSight™ Architecture Specification* (ARM IHI 0029).
- *Arm® Embedded Trace Macrocell Architecture Specification ETMv4* (ARM IHI 0064).
- *Embedded Trace Macrocell® ETMv3 Architecture Specification* (ARM IHI 0014).
- *Arm® v6-M Architecture Reference Manual* (ARM DDI 0419).
- *Arm® v7-M Architecture Reference Manual* (ARM DDI 0403).
- *Arm® Architecture Reference Manual, for A-profile architecture* (ARM DDI 0487).
- *Arm® Armv8-M Faults on Instruction Fetch* (ARM DDI 0624).
- *Arm® Armv8-M Faults on Exception Handling* (ARM DDI 0625).

Other publications

The following publications are referred to in this manual, or provide more information:

- ANSI/IEEE Std 754-1985 and ANSI/IEEE Std 754-2008, IEEE Standard for Binary Floating-Point Arithmetic. Unless otherwise indicated, references to IEEE 754 refer to either issue of the standard.

Note

This manual does not adopt the terminology defined in the 2008 issue of the standard.

- JEP106, Standard Manufacturers Identification Code, JEDEC Solid State Technology Association.

Feedback

Arm welcomes feedback on its documentation.

Feedback on this manual

If you have any comments or suggestions for additions and improvements, create a ticket at <https://support.developer.arm.com>

- The title, Armv8-M Architecture Reference Manual.
- The number, DDI0553B.y
- The section name to which your comments refer.
- The rule identifier(s) to which your comments apply, if applicable.
- A concise explanation of your comments.

Arm also welcomes general suggestions for additions and improvements.

Note

Arm tests PDFs only in Adobe Acrobat and Acrobat Reader, and cannot guarantee the appearance or behavior of any manual when viewed with any other PDF reader.

Inclusive Terminology Commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used terms that can be offensive. Arm strives to lead the industry and to create change.

Previous issues of this manual included terms that can be offensive. We have replaced these terms.

If you find offensive terms in this manual, please contact terms@arm.com.

Part A

Armv8-M Architecture Introduction and Overview

Chapter A1

Introduction

This chapter introduces the Armv8 architecture, the architecture profiles it defines, and the Armv8-M architecture profile defined by this manual. It contains the following sections:

A1.1 Architecture reference manual layout and terminology.

A1.2 About the Armv8 architecture, and architecture profiles.

A1.3 The Armv8-M architecture profile.

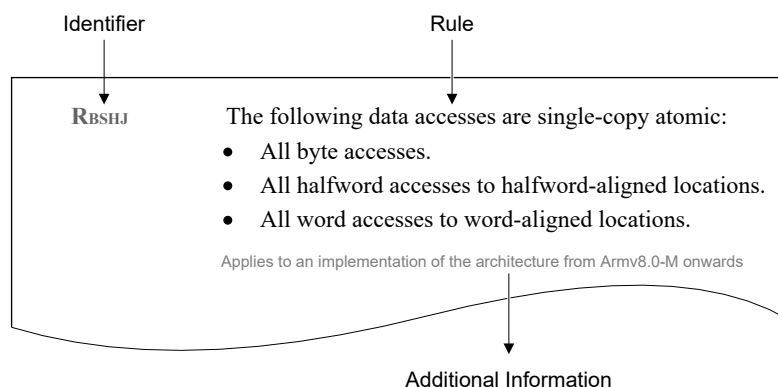
A1.1 Architecture reference manual layout and terminology

This section describes the structure and scope of this architecture reference manual. This section also describes the terminology that this architecture reference manual uses. It does not constitute part of the architecture reference manual, and must not be interpreted as implementation guidance.

A1.1.1 Structure of the architecture reference manual

This architecture reference manual describes the behavior of the processing element as a set of individual rules.

Each rule is clearly identified by the letter R, followed by a random group of subscript letters that do not reflect any intended order or priority, for example R_{BSHJ}. In the following example, R_{BSHJ} is simply a random rule identifier that has no significance apart from uniquely identifying a rule in this manual.



Rules must not be read in isolation, and where more than one rule relating to a particular feature exists, individual rules are grouped into sections and subsections to provide the proper context. Where appropriate, these sections contain a short introduction to aid the reader.

An implementation that conforms to all the rules described in this manual constitutes an Armv8-M compliant implementation. An implementation whose behavior deviates from these rules is not compliant with the Armv8-M architecture.

Some sections contain additional information and guidance that do not constitute rules. This information and guidance is provided purely as an aid to understanding the architecture. Information statements are clearly identified by the letter I, followed by a random group of subscript letters, for example I_{PRTD}.

A line below each rule or information statement gives additional information indicating the architecture version, the extensions that are required for the rule or information statement to apply, and any other notes. Some extensions depend on the implementation of other extensions, for example [FP](#).

Note

Arm strongly recommends that implementers read *all* chapters and sections of this architecture reference manual to ensure that an implementation is compliant.

An implementation that conforms to all the rules described in this architecture reference manual but chooses to ignore any additional information and guidance is compliant with the Armv8-M architecture.

In the following parts of this architecture reference manual, architectural rules are not identified by a specific prefix and a random group of subscript letters:

- Parts of [Chapter B15 The Performance Monitors Extension](#).

Applies to an implementation of the architecture Armv8.1-M onward.

- Parts of [Part C Armv8-M Instruction Set](#).
- [Part D Armv8-M Register and Payload Specification](#).
- [Part E Armv8-M Pseudocode](#).
- [Part F Armv8-M Debug Packet Protocols](#).

A1.1.2 Scope of the architecture reference manual

This architecture reference manual contains only rules and information that relate specifically to the Armv8-M architecture. It does not include any information about other Arm architectures, nor does it describe similarities between Armv8-M and other architectures.

Readers must not assume that the rules provided in this manual are applicable to an Armv7-M or Armv6-M implementation, nor must they assume that the rules that are applicable to an Armv7-M or Armv6-M implementation are equally applicable to an Armv8-M implementation.

A1.1.3 Intended audience

This manual is written for users who want to design, implement, or program an Armv8-M PE in a range of Arm-compliant implementations from simple uniprocessor implementations to complex multiprocessor systems. It does not assume familiarity with previous versions of the M-Profile architecture.

The architecture reference manual provides a precise, accurate, and correct set of rules that must be followed in order for an Armv8-M implementation to be architecturally compliant. It is an explicit reference manual, and not a general introduction to, or user guide for, the Armv8-M architecture.

A1.1.4 Terminology, phrases

This subsection identifies some standard words and phrases that are used in the Arm architecture documentation. These words and phrases have an Arm-specific definition, which is described in this section.

Architecturally visible

Something that is visible to the controlling agent. The controlling agent might be software.

Arm recommends

A particular usage that ensures consistency and usability. Following all the rules listed in this manual leads to a predictable outcome that is compliant with the architecture, but might produce an unexpected output. Adhering to a recommendation ensures that the output is as expected.

Arm strongly recommends

Something that is essentially mandatory, but that is outside the scope of the architecture described in this manual. Failing to adhere to a strong recommendation can break the system, although the PE itself remains compliant with the architecture that is described in this manual.

Finite time

An action will occur at some point in the future. Finite time does not make any statement about the time involved. However, delaying an action longer than is absolutely necessary might have an adverse impact on performance.

Permitted

Allowed behavior.

Required

Mandatory behavior.

Support

The implementation has implemented a particular feature.

A1.1.5 Terminology, Armv8-M specific terms

For definitions of Armv8-M specific terms, see the [Glossary](#).

A1.2 About the Armv8 architecture, and architecture profiles

Armv8-M is documented as one of a set of architecture profiles.

Arm defines three architecture profiles:

A Application profile:

- Supports a *Virtual Memory System Architecture* (VMSA) based on a *Memory Management Unit* (MMU).
- Supports the A64, A32, and T32 instruction sets.

R Real-time profile:

- Supports the AArch64 or AArch32 Execution states.
- Supports A64, or A32 and T32 instruction sets.
- Supports a *Protected Memory System Architecture* (PMSA) based on a *Memory Protection Unit* (MPU).
- Supports a VMSA.

M Microcontroller profile, described in this manual:

- Implements a programmers' model designed for low-latency interrupt processing, with hardware stacking of registers and support for writing interrupt handlers in high-level languages.
- Optionally implements a variant of the PMSA based on an MPU.
- Supports a variant of the T32 instruction set.

This Architecture Reference Manual describes only the Armv8-M profile.

A1.3 The Armv8-M architecture profile

The M-Profile architecture includes:

- The opportunity to include simple pipeline designs offering leading edge system performance levels in a broad range of markets and applications.
- Highly deterministic operation:
 - Single or low cycle count execution.
 - Minimal interrupt latency, with short pipelines.
 - Capable of cacheless operation.
- Excellent targeting of C/C++ code. This aligns with the Arm programming standards in this area:
 - Exception handlers are standard C/C++ functions, entered using standard calling conventions.
- Design support for deeply embedded systems:
 - Low pincount devices.
- Support for debug and software profiling for event-driven systems.

A line below each rule or information statement indicates the architecture version, the extensions that are required for the rule or information statement to apply, and any other notes. Some extensions depend on the implementation of other extensions, for example [FP](#).

A1.3.1 The Armv8-M instruction set

Armv8-M only supports execution of T32 instructions.

For more information about the instructions, see [Chapter C1 Instruction Set Overview](#) and Chapter C2, Instruction Specification.

A1.3.2 Baseline implementation

The simplest Armv8.0-M implementation, without any of the optional extensions, is a Baseline implementation. The Armv8.0-M Baseline offers improvements over previous M-Profile architectures in the following areas:

- An improved *Memory Protection Unit* (MPU) model.
- Alignment with Armv8-A and Armv8-R memory types.
- Stack pointer limit checking.
- Improved support for multi-processing.
- Better alignment with C11 and C11++ standards.
- Enhanced debug capabilities.

In this architecture reference manual a Baseline implementation is referred to as an implementation without the Main Extension.

If an implementation does not include the Main Extension [CPUID.Architecture](#) reads as 0b1100.

Applies to an implementation of the architecture Armv8.0-M onward.

A1.3.3 Nested Vectored Interrupt Controller

The Nested Vectored Interrupt Controller (NVIC) can support up to 496 external interrupts. The NVIC is always implemented and is used for:

- Integrated interrupts.
- Exception handling.
- Priority management.
- Automatic vectoring.
- Automatic state preservation.

For more information see [Chapter B12 Nested Vectored Interrupt Controller](#).

A1.4 Optional Extensions

A1.4.1 CDE - The Custom Datapath Extension

The Custom Datapath Extension is an OPTIONAL feature available from the Armv8-M architecture. An implementation that includes the Custom Datapath Extension must implement all of the features that are required by the Main Extension ([M](#)), and might implement the following OPTIONAL features:

- The features that are provided by the Floating-point Extension ([FP](#)).
- The features that are provided by the Armv8.1 M Vector Extension ([MVE](#)).

CDE Instructions that operate on the [S](#) or [D](#) register file require either [FP](#) or [MVE](#). CDE Instructions that operate on the [Q](#) register file require [MVE](#).

For more information see [B3.37 The Custom Datapath Extension](#).

Note

The Custom Datapath Extension can also be referred to as Arm Custom Instructions for Armv8-M.

Applies to an implementation of the architecture Armv8.0-M onward.

A1.4.2 Debug

The Armv8-M architecture introduces:

- Enhanced breakpoint and watchpoint functionality.
- Improvements to the Instrumentation Trace Macrocell (ITM).
- Comprehensive trace and self-hosted debug extensions to make embedded software easier to debug and trace.

Halting debug (HDBG) is an OPTIONAL feature that allows the debugger to halt the PE.

The following OPTIONAL peripherals might be implemented:

- The Data Trace and Watchpoint Unit - a memory-mapped peripheral providing two options:
 - Comparators and watchpoints for data, instruction and address matching (DWTD).
 - Trace packet generation (DWTT).
- The Embedded Trace Macrocell Unit - a memory-mapped peripheral providing trace information (ETM).
- The Instrumentation Trace Macrocell Unit - a memory-mapped interface for the generation of instrumentation trace packets (ITM).
- The Flash Patch and Breakpoint Unit - a memory-mapped peripheral providing breakpoints (FPB).
- The Trace Port Interface Unit - a memory-mapped peripheral providing an output path and trace sink for the DWT, ETM and ITM (TPIU).

The Main Extension is required to generate trace packets.

Support for the Armv8-M debug architecture is indicated in [ID_DFR0.MProfDbg](#).

For more information see [B13.1 Debug feature overview](#).

Applies to an implementation of the architecture Armv8.0-M onward.

In an Armv8.1-M implementation, the optional *Unprivileged Debug Extension* adds support for unprivileged debug. UDE requires the Main Extension, ([M](#)), to be implemented.

The restrictions on access to a number of registers is relaxed to allow a debugger to write to the register when the PE is not in Debug state.

Support for the Armv8-M debug architecture is indicated in [ID_DFR0.UDE](#).

Applies to an implementation of the architecture Armv8.1-M onward.

For more information about debug, see [Chapter B13 Debug](#) and [Chapter B14 Debug and Trace Components](#).

A1.4.3 DSP - The Digital Signal Processing Extension.

The Digital Signal Processing Extension, DSP, is an OPTIONAL feature. The DSP adds support for [SIMD](#) instructions.

A PE that implements the DSP Extension must implement the Main Extension ([M](#)).

Support for DSP is indicated in the following registers:

- [ID_ISAR1](#).
- [ID_ISAR2](#).
- [ID_ISAR3](#).

Applies to an implementation of the architecture Armv8.0-M onward.

A1.4.4 The DSP Debug Extension

Armv8.1-M adds OPTIONAL debug support for the [DSP](#).

If the DSP Debug Extension, DSPDE, is implemented, the following features must also be implemented:

- Halting debug, [HDBG](#).
- The Main Extension, [M](#).
- The Digital Signal Processing Extension, [DSP](#).

If [MVE](#) and [DWTD](#) are implemented then DSPDE must be implemented.

Applies to an implementation of the architecture Armv8.1-M onward.

A1.4.5 The Floating-point Extension

A PE that implements the Floating-point Extension must implement the Main Extension ([M](#)).

The Floating-point Extension supports either single-precision floating-point instructions or both single-precision and double-precision floating-point instructions.

Applies to an implementation of the architecture Armv8.0-M onward.

HP - Half-precision floating-point instructions

The Armv8.1-M architecture extends the Floating-point extension to include half-precision data types. If single-precision data types are implemented, half-precision data types must also be implemented.

Applies to an implementation of the architecture Armv8.1-M onward.

A1.4.6 The Main Extension

A PE that implements the Main Extension implements the System Timer Extension.

Note

- A PE with the Main Extension is also referred to as a Mainline implementation.
- A PE without the Main Extension is also referred to as a Baseline implementation. A Baseline implementation has a subset of the instructions, registers, and features, of a Mainline implementation.
- Armv7-M compatibility requires the Main Extension.

- Armv6-M compatibility is provided by all Armv8-M implementations.

Support for the Main Extension is indicated in [CPUID.Architecture](#).

Applies to an implementation of the architecture Armv8.0-M onward.

A1.4.7 MPU model

The Armv8-M architecture provides a system address map and permits implementations to include an OPTIONAL MPU. The optional MPU uses the Protected Memory System Architecture (PMSAv8) and contains improved flexibility in the MPU region definition.

The Armv8.1-M architecture introduces a further attribute for MPU regions, [MPU_RLAR.PXN](#).

Applies to an implementation of the architecture Armv8.1-M onward.

For further information see [Chapter B10 The Armv8-M Protected Memory System Architecture](#).

A1.4.8 M-Profile Vector Extension, MVE

This extension provides operations on various SIMD data types.

It consists of MVE-I (integer) and MVE-F (floating-point).

A PE that implements MVE-I requires the features that are provided by the Main Extension, [\(M\)](#), and the DSP Extension [\(DSP\)](#).

A PE that implements MVE-F requires the implementation of the Floating-point Extension [\(FP\)](#), and MVE-I.

For more information see [Chapter B5 Vector Extension](#).

Support for MVE is indicated in ID_MVFR1.MVE.

Note

The Armv8-M MVE can also be referred to as Arm Helium™ for Armv8-M.

Applies to an implementation of the architecture Armv8.1-M onward.

A1.4.9 Pointer Authentication and Branch Target Identification Extension

The OPTIONAL Pointer Authentication and Branch Target Identification Extension (PACBTI) for Armv8.1-M adds support for the authentication of the contents of a register before that register is used as a load or as the target of an indirect branch, and for the authentication of the stack pointers.

For more information see [Chapter B6 Pointer authentication and branch target identification Extension](#).

Support for the PACBTI extension is indicated by [ID_ISAR5.PACBTI](#).

Applies to an implementation of the architecture Armv8.1-M onward.

A1.4.10 Performance Monitors Extension

In an Armv8.1-M implementation, the optional *Performance Monitors Extension* adds support for a Performance Monitor Unit (PMU).

For more information see [Chapter B15 The Performance Monitors Extension](#).

Support for the Performance Monitors Extension is indicated by [PMU_TYPE.N](#).

Applies to an implementation of the architecture Armv8.1-M onward.

A1.4.11 Reliability, Availability, and Serviceability

In an Armv8.1-M implementation, the *Reliability, Availability, and Serviceability (RAS) Extension* adds additional debug support. The minimum RAS Extension is mandatory in an Armv8.1-M implementation.

For more information see [Chapter B16 Reliability, Availability, and Serviceability \(RAS\) Extension](#).

Support for RAS is indicated by [ID_PFR0.RAS](#).

Applies to an implementation of the architecture Armv8.1-M onward.

A1.4.12 Security Extension

The Armv8-M architecture introduces the OPTIONAL Security Extension which provides a mechanism to protect sensitive data, areas of memory and other resources. This is achieved through the use of instructions and hardware to create a Secure state that is protected from a Non-secure state.

From Armv8.1-M Thread mode re-entrancy can be disabled on [SG](#) instructions.

Applies to an implementation of the architecture Armv8.1-M onward.

In a PE with the Main Extension, support for the Security Extension is indicated in [ID_PFR1.Security](#).

Note

The Armv8-M Security Extension can also be referred to as Arm TrustZone for Armv8-M.

A1.4.13 The System Timer Extension

The System Timer Extension, also known as SysTick, provides a decrementing, wrap-on-zero, clear-on-write counter.

It is an OPTIONAL feature of a PE without the Main Extension, and it is IMPLEMENTATION DEFINED if none, one or two counters are implemented. If one timer is implemented it is shared between Security states, if the Security Extension is implemented. If two timers are implemented each Security state has a dedicated timer.

In an implementation with the Main Extension one timer is implemented per Security state.

For more information see [Chapter B11 The System Timer, SysTick](#).

Part B

Armv8-M Architecture Rules

Chapter B1

Resets

This chapter specifies the Armv8-M reset rules. It contains the following section:

[B1.1 *Resets, Cold reset, and Warm reset.*](#)

B1.1 Resets, Cold reset, and Warm reset

R _{BDPL}	There are two resets: • Cold reset. • Warm reset. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{CTPC}	It is not possible to have a Cold reset without also having a Warm reset. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{FNNX}	On a Cold reset, registers that have a defined reset value contain that value. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LQYD}	Registers and fields which have a defined value for a Cold reset retain their value over a Warm reset, unless a value is specified for a Warm reset. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{YMHN}	On a Warm reset, the PE performs the actions that are described by the TakeReset() pseudocode. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{WSZN}	AIRCR.SYSRESETREQ is used to request a Warm reset. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HFRS}	For AIRCR.SYSRESETREQ, the architecture does not guarantee that the reset takes place immediately. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[Chapter B13 Debug.](#)

[Chapter B16 Reliability, Availability, and Serviceability \(RAS\) Extension.](#)

Chapter B2

Power Management

This chapter specifies the Armv8-M power management rules. It contains the following section:

[B2.1 *Power management*.](#)

[B2.2 *Sleep on exit*.](#)

B2.1 Power management

I_{HCTL} The following instructions and pseudocode functions hint to the PE hardware that it can suspend execution and enter a low-power state:

- [WFE](#).
- [WFI](#).
- [WaitForEvent\(\)](#).
- [WaitForInterrupt\(\)](#).
- [SleepOnExit\(\)](#).

Applies to an implementation of the architecture Armv8.0-M onward.

B2.1.1 The Wait for Event (WFE) instruction

R_{DCMH} When a [WFE](#) instruction is executed, if the state of the Event register is clear, the PE can suspend execution and enter a low-power state.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{RPZM} The Event register is a single-bit register for each PE in the system.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{LNFV} Software cannot read, and cannot write to, the Event register directly.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BPBR} The Event register for a PE is set by any of the following:

- Any WFE wakeup event.
- Exception entry.
- Exception return.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{MMZW} When the Event register is set, it is an indication that an event has occurred since the register was last cleared, and that the event might require some action by the PE.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{HDV} When a [WFE](#) instruction is executed, if the state of the Event register is set, the instruction clears the register and completes immediately. In this case execution is not suspended.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NNDK} If the PE enters a low-power state on a [WFE](#) instruction, it remains in that low-power state until it receives a *WFE wakeup event*. When the PE recognizes a WFE wakeup event, the [WFE](#) instruction completes. The following are WFE wakeup events:

- The execution of a [SEV](#) instruction by any other PE in the same multiprocessor system.
- When [SCR.SEVONPEND](#) bit associated with a security state is one, interrupts transitioning from the inactive to the pending state that target that security state are wakeup events.
- Any exception at a priority that would preempt the current execution priority, taking into account any active exceptions and including the effects of any software-controlled priority boosting by [AIRCR.PRIS](#) == 1 and [PRIMASK](#), [FAULTMASK](#), or [BASEPRI](#).
- If debug is enabled, a debug event.
- Any IMPLEMENTATION DEFINED event.
- A Warm reset.

Applies to an implementation of the architecture Armv8.0-M onward.

R _{CXMT}	A reset clears the Event register. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{TRPP}	When SCR.SLEEPDEEP is 1 and SCR.SEVONPEND is 1, it is IMPLEMENTATION DEFINED whether any exception entering the pending state will act as a <i>WFE wakeup event</i> and wake up the PE. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{YRDC}	The Armv8-M architecture does not define the exact nature of the low-power state that is entered on an instruction, except that it does not cause a loss of memory coherency . <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{TZJZ}	Arm recommends that software always uses the WFE instruction in a loop. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[B3.13 Priority model](#).

[WaitForEvent\(\)](#).

[SendEvent\(\)](#).

[SetEventRegister\(\)](#)

[ClearEventRegister\(\)](#)

[EventRegistered\(\)](#)

B2.1.2 The Wait for Interrupt (WFI) instruction

R _{HRMJ}	When a WFI instruction is executed, the PE can suspend execution and enter a low-power state. If it does, it remains in that state until it receives a <i>WFI wakeup event</i> . When the PE recognizes a WFI wakeup event, the WFI instruction completes. The following are WFI wakeup events: • A Warm reset. • Any asynchronous exception at a priority that, ignoring the effect of PRIMASK (so that behavior is as if PRIMASK is 0), would preempt any currently active exceptions. • An IMPLEMENTATION DEFINED WFI wakeup event. • If debug is enabled, a debug event. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{WCB D}	A debug event will be a wakeup event if either: • Halting Debug is permitted for the current Security state of the PE and the PE is required to enter Debug state. • A DebugMonitor exception is asserted and the exception has sufficient priority to preempt the current priority of the PE. Setting DEMCR.MON_PEND to one is not required to cause a wakeup event. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB. Note, DebugMonitor exceptions require M.</i>
I _{PDKN}	An unprivileged debug event will be a wakeup event if unprivileged debug is permitted for the current Security state and privilege mode of the PE. If unprivileged debug is not enabled the event will be pended and taken when unprivileged debug is permitted. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DB && UDE.</i>

I_{CGNL}

Arm recommends that software always uses the [WFI](#) instruction in a loop.

*Applies to an implementation of the architecture **Armv8.0-M** onward.*

See also:

[B3.13 Priority model.](#)

[Chapter B13 Debug](#)

[WaitForInterrupt\(\)](#)

B2.2 Sleep on exit

R_{JXGW} It is IMPLEMENTATION DEFINED whether the [SleepOnExit \(\)](#) function causes the PE to enter a low-power state during a return from the only active exception and to Thread mode.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BHSL} The PE is permitted to enter a low-power state on return from the only active exception when all the following are true:

- [EXC_RETURN.Mode](#) == 1.
- [SCR.SLEEPONEXIT](#) == 1.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WWDW} If the sleep-on-exit function is enabled, it is IMPLEMENTATION DEFINED at which point in the exception return process the PE enters a low-power state.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{LLQF} The wakeup events for the sleep-on-exit function are identical to the [WFI](#) instruction wakeup events.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.13 Priority model.](#)

[SleepOnExit \(\)](#)

[B3.22 Exception return.](#)

Chapter B3

Programmers' Model

This chapter specifies the Armv8-M programmers' model architecture rules. It contains the following sections:

- B3.1 PE modes, Thread mode and Handler mode.*
- B3.2 Privileged and unprivileged execution.*
- B3.3 Registers.*
- B3.4 Special-purpose CONTROL register.*
- B3.5 XPSR, APSR, IPSR, and EPSR.*
- B3.6 Security states: Secure state, and Non-secure state.*
- B3.7 Security states and register banking between Security states.*
- B3.8 Stack pointer.*
- B3.9 Exception numbers and exception priority numbers.*
- B3.10 Exception enable, pending, and active bits.*
- B3.11 Security states, exception banking.*
- B3.12 Faults.*
- B3.13 Priority model.*
- B3.14 Secure address protection.*
- B3.15 Security state transitions.*
- B3.16 Function calls from Secure state to Non-secure state.*
- B3.17 Function returns from Non-secure state.*

B3.18 *Exception handling.*

B3.19 *Exception entry, context stacking.*

B3.20 *Exception entry, register clearing after context stacking.*

B3.21 *Stack limit checks.*

B3.22 *Exception return.*

B3.23 *Integrity signature.*

B3.24 *Exceptions during exception entry.*

B3.25 *Exceptions during exception return.*

B3.26 *Tail-chaining.*

B3.27 *Exceptions, instruction resume, or instruction restart.*

B3.28 *Low overhead loops.*

B3.29 *Branch future.*

Applies to an implementation of the architecture Armv8.1-M onward.

B3.30 *Vector tables.*

B3.31 *Hardware-controlled priority escalation to HardFault.*

B3.32 *Special-purpose mask registers, PRIMASK, BASEPRI, FAULTMASK, for configurable priority boosting.*

B3.33 *Lockup.*

B3.34 *Data independent timing.*

Applies to an implementation of the architecture Armv8.1-M onward.

B3.35 *Context Synchronization Event.*

B3.36 *Coprocessor support.*

B3.37 *The Custom Datapath Extension.*

B3.1 PE modes, Thread mode and Handler mode

R_{CNMS}

There are two PE modes:

- Thread mode.
- Handler mode.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{FDVT}

A common usage model for the PE modes is:

- **Thread mode:** Applications.
- **Handler mode:** OS kernel and associated functions, that manage system resources.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RPKP}

The PE handles all exceptions in Handler mode.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CMQP}

Thread mode is selected on reset.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.2 Privileged and unprivileged execution.](#)

[B3.5.1 Interrupt Program Status Register \(IPSR\).](#)

[B3.6 Security states: Secure state, and Non-secure state.](#)

B3.2 Privileged and unprivileged execution

R_{WVRK}

Thread mode

Execution can be privileged or unprivileged.

Handler mode

Execution is always privileged.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{WCFH}

CONTROL.nPRIV determines whether execution in Thread mode is unprivileged.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{SBQF}

In a PE without the Main Extension, it is IMPLEMENTATION DEFINED whether **CONTROL.nPRIV** can be set to 1.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - !M.

R_{JSSW}

Execution privilege can determine whether a resource is accessible.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{GNSC}

Privileged execution typically has access to more resources than unprivileged execution.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.1 PE modes, Thread mode and Handler mode.](#)

B3.3 Registers

R_{KGST}

There are the following types of registers:

General-purpose registers, all 32-bit:

- R0-R12 (**Rn**).
- R13. This is the stack pointer (**SP**).
- R14. This is the Link Register (**LR**).

Program Counter, 32-bit:

- R15 is the Program Counter (**PC**).

Special-purpose registers:

- Mask Registers:
 - 1-bit exception mask register, **PRIMASK**.
 - 8-bit base priority mask register, **BASEPRI**.
 - 1-bit fault mask register, **FAULTMASK**.
- A **CONTROL** register.
- Two 32-bit stack pointer limit registers, **MSPLIM** and **PSPLIM**, if the Main Extension is not implemented the Non-secure versions of these registers are RAZ/WI.
- A combined 32-bit Program Status Register (**XPSR**), comprising:
 - Application Program Status Register (**APSR**).
 - Interrupt Program Status Register (**IPSR**).
 - Execution Program Status Register (**EPSR**).
- Vector Predication and Control Register (**VPR**).
- Floating-point Status and Control Register (**FPSCR**).

Memory-mapped registers:

All other registers.

Applies to an implementation of the architecture Armv8.0-M onward. Note, Armv8.1-M is required for MVE and PACBTI.

I_{CJWV}

A 32-bit combined exception return Program Status Register, **RETPSR**, contains a payload of the saved state derived from the **XPSR**.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DHVL}

Extensions might add more registers to the base register set.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{BLXF}

SP refers to the active stack pointer, the Main stack pointer or the Process stack pointer.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{PLRT}

If the Main Extension is implemented, the **LR** is set to 0xFFFFFFFF on Warm reset.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M**.*

R_{QHMH}

If the Main Extension is not implemented, the **LR** becomes UNKNOWN on a Warm reset.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **!M**.*

R_{PLNS}

The **PC** is loaded with the reset handler start address on Warm reset.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{JPCB} The **PC** contains the instruction address of the instruction currently being executed. If an instruction reads the value of the **PC**, the value used is that of **PC** plus 4.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XHHC} Except for writes to the **CONTROL** register, any change to a special-purpose register by a **CPS** or **MSR** instruction is guaranteed:

- Not to affect that **CPS** or **MSR** instruction, or any instruction preceding it in program order.
- To be visible to all instructions that appear in program order after the **CPS** or **MSR**.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MHJK} All unallocated or reserved values of fields with allocated values within the registers that are described in this reference manual behave, unless otherwise stated in the register description, in one of the following ways:

- The encoding maps onto any of the allocated values, but otherwise does not cause CONSTRAINED UNPREDICTABLE behavior.
- The encoding causes effects that could be achieved by a combination of more than one of the allocated encodings.
- The encoding causes the field to have no functional effect.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{PDJC} Reads of registers described as write-only (WO) behave as RES0.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XKXP} A write to a register or field described as read-only (RO) does not cause modification of the read-only register or field.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[Chapter B8 The System Address Map.](#)

[B3.32 Special-purpose mask registers, PRIMASK, BASEPRI, FAULTMASK, for configurable priority boosting.](#)

[B3.4 Special-purpose CONTROL register.](#)

[B3.21 Stack limit checks.](#)

[B3.5 XPSR, APSR, IPSR, and EPSR.](#)

[B1.1 Resets, Cold reset, and Warm reset.](#)

[Part D Register and Payload Specification.](#)

B3.4 Special-purpose CONTROL register

R _{CSPP}	MRS and MSR instructions can be used to access the CONTROL register. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{GKVQ}	Privileged execution can write to the CONTROL register. The PE ignores unprivileged writes to the CONTROL register. All reads of the CONTROL register, regardless of privilege, are allowed. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SNGJ}	The architecture requires a Context synchronization event to guarantee that a change to the CONTROL register will affect the execution of instructions appearing later in the program order. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HVGB}	The PE automatically updates CONTROL.SPSEL on exception entry and exception return. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{NMBL}	CONTROL.SPSEL selects the stack pointer when the PE is in Thread mode. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

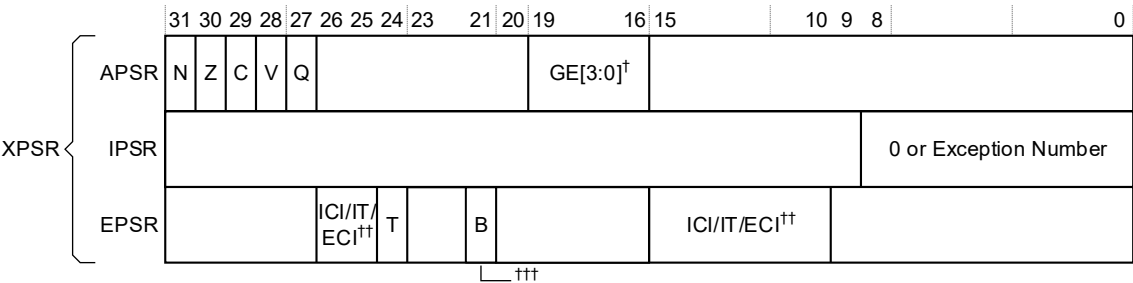
See also:

[B3.35 Context Synchronization Event](#).

[CONTROL](#), *Control Register*.

B3.5 XPSR, APSR, IPSR, and EPSR

R_{ZFHH} The [APSR](#), [IPSR](#), and [EPSR](#) combine to form one register, the XPSR:



[†] Reserved if the DSP Extension is not implemented
^{††} Reserved if the Main Extension is not implemented. ECI requires implementing the MVE Extension.
^{†††} B requires implementing the PACBTI Extension.

All unused bits in any of the [APSR](#), [IPSR](#), or [EPSR](#), or any unused bits in the combined [XPSR](#), are reserved.

Applies to an implementation of the architecture Armv8.0-M onward. Note, ECI functionality only available in an Armv8.1-M implementation.

R_{XGTP} The [MRS](#) and [MSR](#) instructions recognize the following mnemonics for accessing the [APSR](#), [IPSR](#) or [EPSR](#), or a combination of them:

Mnemonic	Registers accessed
APSR	APSR
IPSR	IPSR
EPSR	EPSR
IAPSR	IPSR and APSR
EAPSR	EPSR and APSR
IEPSR	IPSR and EPSR
XPSR	APSR , IPSR , and EPSR

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WLFR} Arm deprecates using `MSR APSR` without a `<bits>` qualifier as an alias for `MSR APSR_nzcvq`.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.3 Registers](#).

[APSR, Application Program Status Register](#).

[B3.5.1 Interrupt Program Status Register \(IPSR\)](#).

[B3.5.2 Execution Program Status Register \(EPSR\)](#).

B3.5.1 Interrupt Program Status Register (IPSR)

R _{DTBJ}	When the PE is in Thread mode, the IPSR value is zero. When the PE is in Handler mode: • In the case of a taken exception, the IPSR holds the exception number of the exception being handled. • When there has been a function call from Secure state to Non-secure state, the IPSR has the value of 1. The PE updates the IPSR on exception entry and return. <i>Applies to an implementation of the architecture Armv8.0-M onward. Note, Secure state requires S.</i>
R _{XTCC}	The PE ignores writes to the IPSR by MSR instructions. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{RDRX}	If DCRDR is used to change the value of IPSR, then the value of IPSR becomes UNKNOWN. If DCRDR attempts to set IPSR to an illegal value, then the UNKNOWN value is set to one of the known legal values. <i>Applies to an implementation of the architecture Armv8.0-M only. The extension requirements are - HDBG.</i>
R _{VRPL}	Writes to IPSR using the DCRDR and DCRSR mechanism are ignored. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - HDBG.</i> See also: B3.5 XPSR, APSR, IPSR, and EPSR. B3.16 Function calls from Secure state to Non-secure state. IPSR, Interrupt Program Status Register BX, BXNS

B3.5.2 Execution Program Status Register (EPSR)

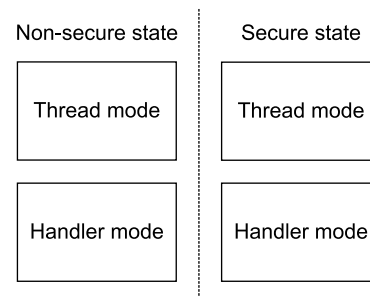
R _{KSCH}	A reset sets EPSR.T to the value of bit[0] of the reset vector. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SQLX}	When EPSR.T is: 0: Any attempt to execute any instruction generates: • An INVSTATE UsageFault, in a PE with the Main Extension. • A HardFault, in a PE without the Main Extension. 1: The Instruction set state is T32 state and all instructions are decoded as T32 instructions. <i>Applies to an implementation of the architecture Armv8.0-M onward. Note, UsageFault requires M.</i>
I _{XBWX}	The intent is that the Instruction set state is always T32 state. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LBJQ}	All EPSR fields read as zero using an MRS instruction. The PE ignores writes to the EPSR by an MSR instruction. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i> See also: B3.5 XPSR, APSR, IPSR, and EPSR. B3.5.2 Execution Program Status Register (EPSR).

B3.6 Security states: Secure state, and Non-secure state

R_{HKKL}

A PE with the Security Extension has two Security states:

- Secure state.
 - Secure Thread mode.
 - Secure Handler mode.
- Non-secure state.
 - Non-secure Thread mode.
 - Non-secure Handler mode.



Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{PBGT}

If the Security Extension is implemented, memory areas and other critical resources that are marked as secure can only be accessed when the PE is executing in Secure state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{HWFV}

A PE with the Security Extension resets into Secure state on a Warm reset.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{PLGH}

A PE without the Security Extension resets into Non-secure state on a Warm reset.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - !S.

See also:

[B3.1 PE modes, Thread mode and Handler mode.](#)

[B3.2 Privileged and unprivileged execution.](#)

[B3.7 Security states and register banking between Security states.](#)

[B3.11 Security states, exception banking.](#)

[B3.15 Security state transitions.](#)

[Chapter B5 Vector Extension.](#)

Applies to an implementation of the architecture Armv8.1-M onward.

B3.7 Security states and register banking between Security states

I_{MGRQ}

In a PE with the Security Extension, some registers are banked between the Security states. When a register is banked in this way, there is a distinct instance of the register in Secure state and another distinct instance of the register in Non-secure state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{BHDK}

In a PE with the Security Extension:

- The general-purpose registers that are banked are:
 - R13. This is the stack pointer (SP).
- The special-purpose registers that are banked are:
 - The Mask registers, PRIMASK, BASEPRI, and FAULTMASK.
 - Some bits in the CONTROL register.
 - The Main and Process stack pointer Limit registers, MSPLIM and PSPLIM.
- All registers in the System Control Space (SCS) have a Non-secure alias which is only accessible from Secure state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

I_{GBWT}

For MRS and MSR (register) instructions, SYSm[7] in the instruction encoding specifies whether the Secure or the Non-secure instance of a Banked register is accessed:

Access from	SYSm[7]	
	0	1
Secure state	Secure instance	Non-secure instance
Non-secure state	Non-secure instance	RAZ/WI

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{ZJKL}

This specification uses the following naming convention to identify a Banked register:

- <register name>_S: The Secure instance of the register.
- <register name>_NS: The Non-secure instance of the register.
- <register name>: The instance that is associated with the current Security state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

See also:

[B3.3 Registers.](#)

[B3.6 Security states: Secure state, and Non-secure state.](#)

[B3.8 Stack pointer.](#)

[B8.3 The System Control Space \(SCS\).](#)

B3.8 Stack pointer

R_{RDLR} In a PE with the Security Extension, four stacks and four stack pointer registers are implemented:

Stack	Stack pointer register	
Secure	Main	MSP_S
	Process	PSP_S
Non-secure	Main	MSP_NS
	Process	PSP_NS

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [S](#).

R_{TGHV} In a PE without the Security Extension, two stacks and two stack pointer registers are implemented:

Stack	Stack pointer register
Main	MSP
Process	PSP

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [!S](#).

R_{LDGJ} On exception return the Armv8-M architecture only supports doubleword aligned stack pointers.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{ZJBC} If on exception return, the stack pointer used for the unstacking operations is not doubleword aligned the behavior is CONSTRAINED UNPREDICTABLE. When calculating the address of the unstacking operations either of the following behaviors are permitted:

- Treating the stack pointer as the actual value.
- Treating the stack pointer as if it were aligned.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{NPJB} If the stack pointer being adjusted on exception return is not doubleword aligned the behavior is CONSTRAINED UNPREDICTABLE. Either of the following behaviors are permitted:

- $SP = SP + framesize + (sprealign \ll 2)$
- $SP = (SP + framesize) \text{ OR } (sprealign \ll 2)$

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{TXRW} In Handler mode, the PE uses the main stack.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{DMLS} In Thread mode, [CONTROL.SPSEL](#) determines whether the PE uses the main or process stack.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{BTVD} In a PE without the Security Extension, MSP is selected and initialized on reset.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [!S](#).

R_{MDXK} In a PE with the Security Extension, the Secure main stack, MSP_S, is selected and initialized on reset.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [S](#).

I_{LVWN} On Warm reset, the selected Stack Pointer either the MSP or MSP_S, is set to the value contained in the Vector table, as described in [TakeReset\(\)](#).

Applies to an implementation of the architecture *Armv8.0-M* onward. Note, S is required for MSP_S.

R_{XPWM} Bits [1:0] of the MSP or PSP, in either Security state, are [RES0H](#), so that all stack pointers are always guaranteed to be word-aligned.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{MQVJ} Where an instruction states that the [SP](#) is UNPREDICTABLE and [SP](#) is used:

- The value that is read or written from or to the [SP](#) is UNKNOWN.
- The instruction is permitted to be treated as UNDEFINED.
- If the [SP](#) is being written, it is UNKNOWN whether a stack limit check is applied.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{JXJM} After the successful completion of an exception entry stacking operation, the stack pointer of the stack pushed because of the exception entry is doubleword-aligned.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{PWRQ} Arm recommends that the Secure stacks be located in Secure memory.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

See also:

[B3.6 Security states: Secure state, and Non-secure state.](#)

[B3.1 PE modes, Thread mode and Handler mode.](#)

[B3.19 Exception entry, context stacking.](#)

[B3.30 Vector tables.](#)

[B3.3 Registers.](#)

[B3.21 Stack limit checks.](#)

B3.9 Exception numbers and exception priority numbers

I_{DCJS} Each exception has an associated *exception number* and an associated *priority number*.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CMTc} In a PE with the Main Extension, the exceptions, their associated numbers, and their associated priority numbers are as follows:

Exception	Exception Number	Priority Number
Reset	1	-4 (Highest Priority)
Secure HardFault when AIRCR.BFHFNMINS is 1 ^a	3	-3
NMI	2	-2
Secure HardFault when AIRCR.BFHFNMINS is 0	3	-1
Non-secure HardFault	3	-1
MemManage fault	4	Configurable
BusFault	5	Configurable
UsageFault	6	Configurable
SecureFault	7	Configurable
Reserved	8-10	-
SVCall	11	Configurable
DebugMonitor	12	Configurable
Reserved	13	-
PendSV	14	Configurable
SysTick	15	Configurable
External Interrupt 0	16	Configurable
-	-	-
-	-	-
-	-	-
External interrupt N	16+N	Configurable

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#). Note, S is required for Secure faults.

R_{MGNV} In a PE without the Main Extension, the exceptions, their associated numbers, and their associated priority numbers are as follows:

Exception	Exception Number	Priority Number
Reset	1	-4 (Highest Priority)
Secure HardFault when AIRCR.BFHFNMINS is 1	3	-3
NMI	2	-2
Secure HardFault when AIRCR.BFHFNMINS is 0	3	-1
Non-secure HardFault	3	-1
Reserved	4-10	-
SVCall	11	Configurable
Reserved	12-13	-
PendSV	14	Configurable
SysTick	15	Configurable
External Interrupt 0	16	Configurable
-	-	-
-	-	-
-	-	-
External interrupt N	16+N	Configurable

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [!M](#). Note, *ST* is required for SysTick exception.

I_{FPJD}

The number of supported external interrupts is configurable, to a maximum of 496.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{QQT}

The architecture permits an implementation to omit external configurable interrupts where no external device is connected to the corresponding interrupt pin. Where an implementation omits such an interrupt, the corresponding pending, active, enable, and priority registers are RES0.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{QWTM}

In a PE with the Main Extension, the following exceptions with configurable priority numbers can be configured with [SHPR1](#)- [SHPR3](#) in the System Control Block (SCB):

- MemManage Fault.
- BusFault.
- UsageFault.
- SecureFault (if the Security Extension is implemented).
- SVCall.
- DebugMonitor exception.
- PendSV.
- SysTick.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [M](#).

I_{SGBC}

In a PE without the Main Extension the following exceptions with configurable priority numbers can be configured with [SHPR2](#) and [SHPR3](#) in the System Control Block (SCB):

- SVCall.
- PendSV.
- SysTick.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [!M](#).

I_{JQPH} All other configurable exceptions can be configured using the **NVIC_IPRn.PRI_<n>** register fields.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NFSM} Configurable priority numbers start at 0, the highest configurable exception priority number.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GGCP} In a PE with the Main Extension, the number of configurable priority numbers is an IMPLEMENTATION DEFINED power of two in the range 8-256:

Number of priority bits of SHPRIn.PRI_n implemented	Number of configurable Priority numbers	Minimum Priority Number (highest priority)	Maximum Priority Number (lowest priority)
3	8	0	0b11100000 = 224
4	16	0	0b11110000 = 240
5	32	0	0b11111000 = 248
6	64	0	0b11111100 = 252
7	128	0	0b11111110 = 254
8	256	0	0b11111111 = 255

All low-order bits of SHPRIn.PRI_n that are not implemented as priority bits are **RAZ/WI**, as shown in the maximum priority number column.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M**.*

R_{CMGH} In a PE without the Main Extension, the number of configurable priority numbers is 4:

Number of priority bits of SHPRIn.PRI_n implemented	Number of configurable Priority numbers	Minimum Priority Number (highest priority)	Maximum Priority Number (lowest priority)
2	4	0	0b11000000 = 192

SHPRn.PRI_n[5:0] are **RAZ/WI**, as shown in the maximum priority number column.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **!M**.*

See also:

[B3.11 Security states, exception banking.](#)

[B3.12 Faults.](#)

[B3.13 Priority model.](#)

[SHPR1](#), [SHPR2](#), [SHPR3](#).

[NVIC_IPRn](#).

[ExecutionPriority\(\)](#)

B3.10 Exception enable, pending, and active bits

I_{YPBJ} The [SHCSR](#), [ICSR](#), [DEMCR](#), [NVIC_IABRn](#), [NVIC_ISPRn](#), [NVIC_ISERn](#) contain exception enable, pending, and active fields. [STIR](#) can be used to pend exceptions.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{GHGW} The following exceptions are always enabled and therefore do not have an exception enable bit:

- Secure HardFault or Non-secure HardFault if the Security Extension is not implemented.
- NMI.
- SVCall.
- PendSV.

Applies to an implementation of the architecture Armv8.0-M onward. Note, S is required for Secure HardFault.

I_{SVFP} Interrupts can be pended from software by:

- Privileged execution can pend interrupts by writing to the [NVIC_ISPRn](#).
- When [CCR.USERSETMPEND](#) is 1, unprivileged execution can pend interrupts by writing to the [STIR](#).
- Non-secure accesses are only permitted to pend interrupts configured to target the Non-secure state and controlled by [NVIC_ITNSn](#).

Applies to an implementation of the architecture Armv8.0-M onward.

I_{TRJJ} The following table identifies the fault enable, status and active bits:

Fault, Enable (SHCSR) and Trap Bits	Status bit	Pending bit SHCSR , ICSR	Active bit SHCSR
Secure HardFault	HFSR.VECTTBL HFSR.FORCED HFSR.DEBUGEVT	HARDFaultPENDED	HARDFaultACT
NMI	-	PENDNMISET	NMIACT
HardFault AIRCR.BFHFNMINS	HFSR.VECTTBL HFSR.FORCED HFSR.DEBUGEVT	HARDFaultPENDED	HARDFaultACT
MemmanageFault MEMFAULTENA	MMFSR.IACCVIOL MMFSR.DACCVIOL MMFSR.MUNSTKERR MMFSR.MSTKERR MMFSR.MLSPERR	MEMFaultPENDED	MEMFaultACT
BusFault BUSFAULTENA	BFSR.IBUSERR BFSR.PRECISERR BFSR.IMPRESISERR BFSR.UNSTKERR BFSR.STKERR BFSR.LSPERR	BUSFaultPENDED	BUSFaultACT

UsageFault	UFSR.UNDEFINSTR	USGFAULTPENDEd	USGFAULTACT
USGFAULTENA	UFSR.INVSTATE		
	UFSR.INVPC		
	UFSR.NOCP		
	UFSR.STKOF		
CCR.UNALIGN_TRP	UFSR.UNALIGNED	-	-
CCR.DIV_0_TRP	UFSR.DIVBYZERO	-	-
SecureFault	SFSR.INVEP	SECUREFAULTPENDEd	SECUREFAULTACT
SECUREFAULTENA	SFSR.INVIS		
	SFSR.INVER		
	SFSR.AUVIOL		
	SFSR.INVTRAN		
	SFSR.LSPERR		
	SFSR.LSERR		
SVCall	-	SVCALLPENDEd	SVCALLACT
DebugMonitor	-	DEMCR.MON_PEND	MONITORACT
DEMCR.MON_EN			
PendSV	-	PENDSVSET	PENDSVACT
SysTick	-	PENDSTSET	SYSTICKACT
SYST_CSR.ENABLE and SYST_CSR.TICKINT			
External Interrupt	-	NVIC_ISPRn NVIC_ICPRn	NVIC_IABRn
NVIC_ICERn			
NVIC_ISERn			

Applies to an implementation of the architecture *Armv8.0-M* onward.

B3.11 Security states, exception banking

R_{JQM}

Some exceptions are banked. A banked exception has all the following:

- Banked enabled, pending, and active bits.
- A banked [SHPRn.PRI](#) field.

The following table shows which exceptions are banked:

Exception	Banked
Reset	No
HardFault	Yes (conditionally)
NMI	No
MemManage fault	Yes
BusFault	No
UsageFault	Yes
SecureFault	No
SVCall	Yes
DebugMonitor	No
PendSV	Yes
SysTick	Yes
External interrupt 0	No
-	-
-	-
-	-
External interrupt N	No

Applies to an implementation of the architecture Armv8.0-M onward. Note, some exceptions require M, S, DebugMonitor exception or ST.

R_{DXYK}

A banked synchronous exception targets the Security state that it is taken from, except for the following cases:

- If a NOCP UsageFault is generated due to the relevant field of [NSACR](#) being zero, the UsageFault will target the Secure state.
- If a NOCP UsageFault is generated due to the relevant [CPPWR.SU*](#) field being one, the UsageFault will target:
 - The Secure state if the corresponding [CPPWR.SUSm](#) bit is set to one.
 - The current Security state if the corresponding [CPPWR.SUSm](#) bit is set to zero.
- If an instruction triggers lazy floating-point state preservation, then the banked exception will be raised as if the current Security state was the same as the state that activated the lazy floating-point state preservation, as indicated by [FPCCR.S](#).
- Banked faults and exceptions which arise from instruction fetch will target the Security state associated with the instruction address instead of the current Security state.
- MemManage Faults will target the Security state associated with the bank of the MPU used to check the access.

- Where Non-secure HardFault is enabled, because [AIRCR.BFHFNMINS](#) is set to 1, the following applies:
 - HardFault exceptions generated through escalation will target the Security state of the original exception before its escalation to HardFault.
 - A HardFault generated as a result of a failed vector fetch will target the Security state of the exception raised during the failed vector fetch and not the current Security state.
- MemManage Faults triggered by stacking the [State context registers](#) or the [Additional state context registers](#) on exception entry including [tail chaining](#) targets the Security state associated with the background state, as indicated by [EXC_RETURN.S](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#). Note, a UsageFault requires M, Floating-point state requires FP.

R_GVPG	If AIRCR.BFHFNMINS == 0, then all Non-secure HardFaults are disabled and escalated to Secure HardFaults. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_WLGH	Where an implementation has two SysTick timers, one in each Security state, each timer targets its owning Security state and not the current Execution state of the PE. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S && ST.</i>
I_DDKC	NMI can be configured to target either Security state, by using AIRCR.BFHFNMINS. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I_HGFM	BusFault can be configured to target either Security state, by using AIRCR.BFHFNMINS. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_MQWN	SecureFault always targets Secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
I_WSSL	The DebugMonitor exception targets Secure state if the status bit DEMCR.SDME is 1. Otherwise, it targets Non-secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I_DQLX	Each external interrupt, 0-N, targets the Security state that its NVIC_ITNSn[bit number] dictates. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_HXRW	When <exception> targets Secure state, the Non-secure view of its priority field, any enabled, pending, and active bits, are RAZ/WI. <exception> is one of: • NMI. • BusFault. • DebugMonitor. • External interrupt N. • In a PE without the Main Extension, and a single instance of the SysTick Timer, SysTick. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S. Note, a BusFault exception requires M, a DebugMonitor exception requires DebugMonitor exception.</i>
R_DWXH	When an exception is taken to the other Security state, the PE automatically transitions to the target Security state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
I_QRDM	Secure software must ensure that when changing the target Security state of an exception, the exception is not pending or active. A change to the Security state of an exception when the exception is pending or active might lead to UNPREDICTABLE behavior. <i>Applies to an implementation of the architecture Armv8.0-M onward. Note, S.</i>

See also:

[B3.9 Exception numbers and exception priority numbers.](#)

[B3.30 Vector tables.](#)

[SHCSR, System Handler Control and State Register.](#)

B3.12 Faults

I_{NHTB}

There are the following Fault Status Registers:

- HardFault Status Register [HFSR](#).
- MemManage Fault Status Register [MMFSR](#).
- BusFault Status Register [BFSR](#).
- UsageFault Status Register [UFSR](#).
- SecureFault Status Register [SFSR](#).
- Debug Fault Status Register [DFS](#).
- Auxiliary Fault Status Register [AFSR](#). The contents of this register are IMPLEMENTATION DEFINED.
- RAS Fault Status Register [RFSR](#).

In a PE with the Main Extension, the [BFSR](#), [MMFSR](#), and [UFSR](#) combine to form one register, called the Configurable Fault Status Register ([CFSR](#)).

There are the following Fault Address Registers:

- MemManage Fault Address Register ([MMFAR](#)).
- BusFault Address Register ([BFAR](#)).
- SecureFault Address Register ([SFAR](#)).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#). Note, RFSR requires RAS.

R_{XMRH}

Unless otherwise stated, [MMFAR](#) is updated only for a MemManage fault on a data access.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{DDJJ}

Unless otherwise stated, [BFAR](#) is updated only for a synchronous BusFault on a data access.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{BFFR}

Unless otherwise stated, [SFAR](#) is set due to SAU/IDAU violations in the following cases:

- Data accesses.
- exception stacking and unstacking accesses, including stacking during tail chaining.
- Lazy state preservation accesses.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#) & [S](#).

R_{FLDT}

Each fault address register has an associated valid bit. When the PE updates the fault address register, the PE sets the valid bit to one.

Fault address register	Valid bit
MMFAR	MMFSR.MM FARVALID
BFAR	BFSR.BFARVALID
SFAR	SFSR.SFARVALID

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{TSCP}

If a fault occurs that would otherwise set a FAR valid bit to one, which has already been set by an earlier fault it is IMPLEMENTATION DEFINED whether:

- The FAR register is updated with the new syndrome.
- The FAR register retains the syndrome of the original fault.

Applies to an implementation of the architecture Armv8.0-M onward.

R _{TSCG}	If the Security Extension is not implemented, it is IMPLEMENTATION DEFINED whether separate BFAR and MMFAR are implemented. If one shared fault address register is implemented, then on a fault that would otherwise update the shared fault address register, if one of the other valid bits is set to one, it is IMPLEMENTATION DEFINED whether: • The shared fault address register is updated, the valid bit for the fault is set, and the other valid bit is cleared. • The shared fault address register is not updated, and the valid bits are not changed. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && !S.</i>
R _{XPWN}	If the Security Extension is implemented, it is IMPLEMENTATION DEFINED whether separate fault address registers for each fault are implemented. If one shared fault address register per Security state is implemented, then on a fault that would otherwise update the shared fault address register, if one of the other FAR valid bits for the same Security state is set to one, it is IMPLEMENTATION DEFINED whether: • The shared fault address register is updated, the valid bit for the fault is set, and the other valid bit is cleared. • The shared fault address register is not updated, and the valid bits are not changed. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && S.</i>
R _{GBKY}	AIRCR.BFHFNMINs dictates which fault registers can be shared: • SFAR and MMFAR_S, and when AIRCR.BFHFNMINs is zero, BFAR. • MMFAR_NS and when AIRCR.BFHFNMINs is one, BFAR. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && S.</i>
I _{DGDG}	For example in an implementation that contains a shared Fault Address Register, the shared fault register might be written to with the address of the memory location being accessed that caused a BusFault fault, and BFSR.BFARVALID is set to one. If the BFSR.BFARVALID is still set when a MemManage fault subsequently occurs that might update the shared fault address register it is an IMPLEMENTATION DEFINED choice between: • The shared fault address register being updated with the syndrome that caused the MemManage fault, the BFSR.BFARVALID being cleared and MMFSR.MMFArVALID for the MemManage fault being set. • The address of the memory location that caused the BusFault is retained and the fault address information generated by the MemManage fault is discarded without clearing BFSR.BFARVALID or updating MMFSR.MMFArVALID for the MemManage fault. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && !S.</i>
I _{NSSB}	If a Secure MemManage fault occurs that targets Secure state, as described by <code>ExceptionTargetsSecure()</code>, the Secure shared fault register is written to with the address of the memory location that caused the MemManage fault, and MMFSR_S.MMFArVALID is set to one. If the MMFSR_S.MMFArVALID is still set when a SecureFault subsequently occurs it is an IMPLEMENTATION DEFINED choice between: • The shared fault address register being updated with the address of the memory location that that caused the SecureFault, MMFSR_S.MMFArVALID being cleared and SFSR.SFARVALID being set. • The address of the memory location that caused the MemManageFault is retained and the fault address information generated by the SecureFault is discarded without out clearing MMFSR_S.MMFArVALID or updating SFSR.SFARVALID. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && S.</i>
I _{SMVQ}	Arm strongly recommends that if a separate BFAR is implemented, the associated BFAR and BFSR.BFARVALID bit is cleared when changing AIRCR.BFHFNMINs so as not to expose the last accessed address to Non-secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && S.</i>
I _{LRNV}	When the Security extension is implemented: • If AIRCR.BFHFNMINs is zero, a read of BFAR_NS will return 0.

- If [AIRCR.BFHFNMINS](#) is one, a read of [BFAR_NS](#) and [BFAR_S](#) might return the same value. This behavior cannot be relied on if there is a change to [AIRCR.BFHFNMINS](#) between the reads of [BFAR](#) in different Security states.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#) & [S](#).

$\mathbb{I}_{\text{KJPM}}$

In a PE with the Main Extension, the faults are:

Exception Number	Exception		Fault Status Bit
3	HardFault	HardFault on Vector table entry read error	HFSR.VECTTBL
		HardFault on fault escalation	HFSR.FORCED
		HardFault on BKPT escalation	HFSR.DEBUGEVT
4	MemManage Fault	MemManage fault on an instruction fetch	MMFSR.IACCVIOL
		MemManage Fault on direct data access	MMFSR.DACCVIOL
		MemManage Fault on context unstacking by hardware.	MMFSR.MUNSTKERR
		MemManage Fault on context stacking by hardware.	MMFSR.MSTKERR
		A MemManage fault on Floating-point lazy state preservation	MMFSR.MLSPERR
5	BusFault	BusFault on an instruction fetch, precise	BFSR.IBUSERR
		BusFault on a data access, precise	BFSR.PRECISERR
		BusFault on a data access, imprecise	BFSR.IMPRESISERR
		BusFault on a context unstacking by hardware	BFSR.UNSTKERR
		BusFault on context stacking by hardware	BFSR.STKERR
		A BusFault on Floating-point lazy state preservation	BFSR.LSPERR
6	UsageFault	UsageFault, undefined instruction	UFSR.UNDEFINSTR
		UsageFault, invalid Instruction set state because EPSR.T is 0 or because an attempt was made to execute an instruction that is not ICI or ECI compatible when there is a valid EPSR.ICI , ECI value.	UFSR.INVSTATE
		UsageFault, failed integrity check on exception return or function	UFSR.INVPC

Exception Number	Exception	Fault Status Bit
	return.	
	UsageFault, no coprocessor	UFSR.NOCP
	UsageFault, stack overflow	UFSR.STKOF
	UsageFault, unaligned access	UFSR.UNALIGNED
	UsageFault, divide by zero when CCR.DIV_0_TRP is 1	UFSR.DIVBYZERO
7	SecureFault	
	SecureFault, invalid Secure state entry point	SFSR.INVEP
	SecureFault, invalid integrity signature when unstacking	SFSR.INVIS
	SecureFault, invalid exception return	SFSR.INVER
	SecureFault, attribution unit violation	SFSR.AUVIOL
	SecureFault, invalid transition from Secure state	SFSR.INVTRAN
	SecureFault, lazy Floating-point state preservation error	SFSR.LSPERR
	SecureFault, lazy Floating-point state error	SFSR.LSERR

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M**. Note, Secure Faults require S, EPSR.ECI requires MVE.

R_{XVNN} Exception vector reads use the system address map.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{LLRP} In an Armv8.1-M implementation when a HardFault is generated as a result of a read of the Vector table HFSR.FORCED is not set. In an Armv8.0-M implementation it is IMPLEMENTATION DEFINED whether a HardFault generated as a result of Vector table read sets HFSR.FORCED.

Applies to an implementation of the architecture Armv8.1-M onward.

I_{XJQC} RAS faults can generate BusFaults and these are recorded in RFSR.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.

I_{NKHG} In a PE without the Main Extension, the enable, pending, and active bits in SHCSR are RES0 for those faults that are only included in a PE with the Main Extension.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M**.

R_{WHBK} In a PE without the Main Extension, the faults are:

Exception number	Exception
3	HardFault

All other faults described in the architecture behave as if they are disabled, and escalate to HardFault.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

I_{CCXG}

For the exact circumstances under which each of the Armv8-M faults is generated, see the appropriate Fault Status Register description.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

See also:

[B3.9 Exception numbers and exception priority numbers.](#)

[B3.31 Hardware-controlled priority escalation to HardFault.](#)

[Chapter B13 Debug.](#)

[Part D Register and Payload Specification.](#)

B3.13 Priority model

I_{CTFJ}

An exception, other than reset, has the following possible states:

Active:

An exception that either:

- Is being handled.
- Was being handled. The handler was preempted by a handler for a higher priority exception.

Pending:

An exception that has been generated, but that is not active.

Inactive:

The exception has not been generated.

Active and pending:

One instance of the exception is active, and a second instance of the exception is pending. Only asynchronous exceptions can be active and pending. Synchronous exceptions are either inactive, pending, or active.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CJDM}

Lower priority numbers take precedence over higher priority numbers.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RGCH}

The instruction stream that is executing can be interrupted by an exception with a high enough priority. That is, the numerical priority of the exception is lower value than the current group execution priority.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{VJPG}

The current group execution priority is determined by performing the following steps:

- At each step the highest priority, that is the lowest numerical value, is selected.
- The priority of highest priority active exception is determined as follows:
 - If the Main Extension is implemented [AIRCR.PRIGROUP](#) is applied to the priority value of each active exception.
 - If the Security Extension is implemented [AIRCR.PRIS](#) is applied to Non-secure exceptions.
 - If there are no active exceptions the value 256 is used.
- If the Main Extension is implemented the effects of [BASEPRI](#) are applied after the [BASEPRI](#) value has been adjusted by [AIRCR.PRIGROUP](#) and the Non-secure value is adjusted by [AIRCR.PRIS](#) if the Security Extension is implemented.
- If the Main Extension is implemented the effects of [FAULTMASK](#) are applied. If the Security Extension is implemented [AIRCR.PRIS](#) is applied to any priority boosting resulting from [FAULTMASK_NS](#)
- [PRIMASK](#) is applied. If the Security Extension is implemented [AIRCR.PRIS](#) is applied to any priority boosting resulting from [PPRIMASK_NS](#).

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RKCQ}

Execution at a particular priority can only be preempted by an exception with a lower group priority value.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DPSP}

In a PE with the Main Extension, [BASEPRI](#) and each [SHPRn.PRI_m](#) and [NVIC_IPRn.PRI_Nm](#) are 8-bit fields that [AIRCR.PRIGROUP](#) splits into two fields, a group priority field and a subpriority field:

BASEPRI, SHPRn.PRI_m[7:0], and NVIC_IPRn.PRI_Nm[7:0]		
AIRCR.PRIGROUP value	Group priority field	Subpriority field
0	[7:1]	[0]
1	[7:2]	[1:0]
2	[7:3]	[2:0]
3	[7:4]	[3:0]
4	[7:5]	[4:0]
5	[7:6]	[5:0]
6	[7]	[6:0]
7	-	[7:0]

In a PE without the Main Extension, the priority fields are two bits wide, and AIRCR.PRIGROUP is RES0, therefore each SHPRn.PRI_m and NVIC_IPRn.PRI_Nm field is treated as a group priority.

SHPRn.PRI_m[5:0] and NVIC_IPRn.PRI_Nm[5:0] are RES0 in a PE without the Main Extension.

All low order bits of BASEPRI, SHPRn.PRI, and NVIC_IPRn.PRI_Nm are not implemented as priority bits are RAZ/WI.

Applies to an implementation of the architecture Armv8.0-M onward.

R_WQWK

When AIRCR.PRIS is 1, each Non-secure exception NVIC_IPRn.PRI_Nm priority field value [7:0] has the following sequence applied to it, it:

1. Is divided by two.
2. The constant 0x80 is then added to it.

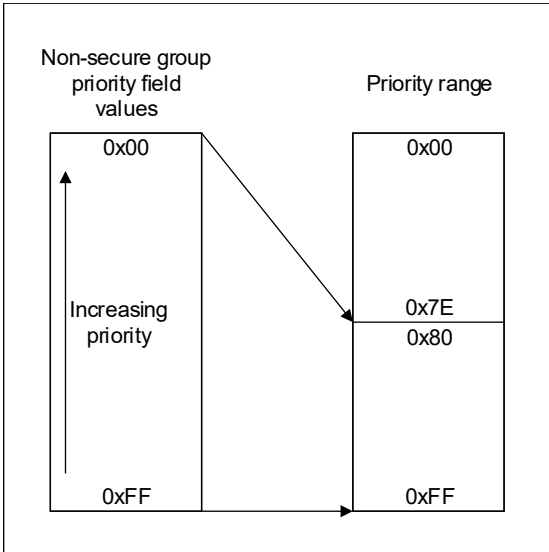
This is equivalent to the priority field value output_pri = '1':input_pri[7:1] and the priority comparisons are done on the effective field value after the division by 2 + 0x80 has been performed.

This maps the Non-secure priority field values to the bottom half of the priority range. When this sequence is applied, any effects of AIRCR.PRIGROUP have already been taken into account.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S. Note, Subpriority requires M.

I_XFVH

The following diagram shows an example. In this example, all 8 bits of the Non-secure exception priority fields are implemented as priority bits and AIRCR.PRIGROUP_NS is set to 0.



In this example, the mapping is:

Non-secure exception priority	Mapped to
0x00	0x80
0x02	0x81
0x04	0x82
0x06	0x83
.	.
.	.
.	.
0xFE	0xFF

In this example, Secure exceptions in the range 0x00-0x7F have priority over all Non-secure exceptions.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - *M* & *S*.

R_{CQRV}

After applying [AIRCR.PRIS](#):

- If there are multiple pending exceptions, the pending exception with the lowest group priority field value takes precedence.
- If multiple pending exceptions have the same group priority field value, the pending exception with the lowest subpriority field value takes precedence.
- If multiple pending exceptions have the same group priority field value and the same subpriority field value, the pending exception with the lowest exception number takes precedence.
- If a pending Secure exception and a pending Non-secure exception both have the same group priority field value, the same subpriority field value, and the same exception number, the Secure exception takes precedence.

Applies to an implementation of the architecture *Armv8.0-M* onward. Note, a Secure exception requires *S*.

I_{WPCP}

In a PE without the Main Extension but with the Security Extension, when [AIRCR.PRIS](#) is set to 1 the Non-secure exception is mapped to the lower half of the priority range, as shown in the table:

Non-secure group priority value	Mapped to
0x00	0x80
0x40	0xA0
0x80	0xC0
0xC0	0xE0

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S** && **M**.

R_{FNND}

When determining the highest priority pending exception it is IMPLEMENTATION DEFINED whether:

- A single priority comparison tree is used and **AIRCR.PRIGROUP** is not applied.
- Separate priority comparison trees are used for the Secure and Non-secure exceptions. **AIRCR.PRIGROUP** is applied to the outputs of each tree before they are compared to determine the overall highest priority pending exception.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{VPKQ}

Arm strongly recommends a single priority comparison tree is used and **AIRCR.PRIGROUP** is not applied.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{NCDS}

The following is an example of exceptions with different priorities:

This example considers the following exceptions, that all have configurable priority numbers:

- A has the highest priority.
- B has medium priority.
- C has lowest priority.

Example sequence of events:

1. No exception is active and no priority boosting is active.
2. B is generated. The PE takes exception B and starts executing the handler for it. Exception B is now active and the current execution priority is that of B.
3. A is generated. A is higher priority, therefore A preempts B, and the PE starts executing the handler for A. Exception A is now active and the current execution priority is that of A. Exception B remains active.
4. C is generated. C has the lowest priority, therefore it is pending.
5. After completing A, the PE restarts the handler for B. After completing B, the PE takes exception C and starts executing the handler for it. C is now active and the current execution priority is that of C.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{DSZC}

An implementation is permitted to extend the exception priorities to 9-bits so that the LSB of Non-secure exceptions is not discarded when applying **AIRCR.PRIS**.

When **AIRCR.PRIS** is not applied, the exception priority is extended to 9-bits by appending '0' to the least significant end of the priority value.

When applying **AIRCR.PRIGROUP** to determine the group priority, only the most significant 8-bits of the 9-bit priority are used.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S** && **M**.

I_{RYTJ}

The LSB of an extended 9-bit exception priority can be thought of as a fractional sub-priority bit.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S** && **M**.

See also:

[B3.9 Exception numbers and exception priority numbers.](#)

B3.32 Special-purpose mask registers, PRIMASK, BASEPRI, FAULTMASK, for configurable priority boosting.

B3.31 Hardware-controlled priority escalation to HardFault.

`ExceptionPriority()`.

`ExecutionPriority()`.

`ComparePriorities()`.

`RawExecutionPriority()`.

B3.14 Secure address protection

R_{CHJX} *NS-Req* defines the Security state that the PE or DAP requests that a memory access is performed in.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{MSNJ} *NS-Attr* marks a memory access as Secure or Non-secure.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{VHRL} For PE data accesses, *NS-Req* is equal to the current Security state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{XSPQ} For PE and DAP accesses, *NS-Attr* is determined as follows:

NS-Req	Security attribute of the location being accessed	NS-Attr
Non-secure	Non-secure	Non-secure
	Secure	N/a
Secure	Non-secure	Non-secure
	Secure	Secure

A memory transaction is terminated with an error if the *NS-Req* is Non-secure and the Security attribute of the location being accessed is Secure. For PE generated accesses this error causes a SecureFault with the appropriate syndrome to be generated.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{BBBJ} If the memory transaction is an exception vector fetch access, *NS-Req* is set to the Security state of the exception.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{FLRG} If the memory transaction is part of stacking or unstacking an exception stack frame, the *NS-Req* is determined by the Security state of the background context, as indicated by [EXC_RETURN.S](#)

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{LDDJ} If the memory transaction is part of an interstating function return, the *NS-Req* is always Secure for the unstacking of the function return stack frame.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{LSTF} If the memory transaction is part of lazy Floating-point state preservation, then *NS-req* is determined by [FPCCR.S](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#) && [FP](#).

See also:

[B3.15 Security state transitions.](#)

[B3.17 Function returns from Non-secure state.](#)

[B3.19 Exception entry, context stacking.](#)

[B13.3.4 DAP access permissions.](#)

[B10.2 Security attribution.](#)

B3.15 Security state transitions

R_{PQHT}

Transitions between Security states are performed when branching to an address in a different Security state, as shown in the table below:

Current Security state	Security attribute of the branch target address	Conditions for a change in Security state
Secure	Non-secure	Change to Non-secure state if the branch was an <i>interstating branch</i> instruction, BXNS or BLXNS , with the least significant bit of its target address set to 0.
Non-secure	Secure and is Non-secure callable	Change to the Secure state if the branch target address contains an SG instruction which is fetched and successfully executed.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

I_{KWMP}

[SG](#) instructions in Secure memory are valid entry points to Secure code. They prevent Non-secure code from being able to jump to arbitrary addresses in Secure code.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{XWPC}

Any attempt to execute an instruction in Non-secure memory while the PE is in secure state causes an INVTRAN SecureFault to be generated.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

I_{MSCF}

An interstating branch with the least significant bit of the target address set to zero causes the PE to transition to the Non-secure state. A subsequent attempt to execute an instruction from Non-secure memory will therefore not generate an INVTRAN SecureFault.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{JKJD}

On transition from Non-secure to Secure state, if there is no [SG](#) instruction or the whole instruction at the branch target address is not flagged as Non-secure callable, the execution of the next instruction will generate an INVEP SecureFault.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{YGTR}

If sequential instruction execution crosses from Non-secure memory to Secure memory, then if the Secure memory entry point contains an [SG](#) instruction and the whole of the instruction at the Secure memory entry point is flagged with the Secure attribute and is Non-secure callable, it is CONSTRAINED UNPREDICTABLE whether:

- The PE changes to Secure state.
- An INVEP SecureFault is generated.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#). Note, an INVEP SecureFault requires M.

I_{NGXH}

Except for an [SG](#) instruction, it is not possible to execute Secure code in Non-secure state. It is not possible to execute Non-secure code in Secure state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{QVPC}

When all the following conditions are met, the value at the location indicated by the current Secure stack pointer is loaded from memory:

- The [SG](#) instruction is executed in Non-secure state.
- Either the SAU or IDAU, or both, indicate that the [SG](#) instruction was fetched from Secure memory.
- The PE is executing in Thread mode.

The load of the value indicated by the current Secure stack pointer is performed with the privilege level indicated by [CONTROL_S.nPriv](#) and NS-req set to Secure.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - [S](#).*

R_{XCXC}

An INVEP SecureFault is raised if the all of the following are true:

- [CCR_S.TRD](#) is set to 1.
- Either, or both, of the following conditions are met:
 - [CONTROL_S.SPEL](#) is 0.
 - The top 31 bits of the value at the location indicated by the current Secure stack pointer loaded from memory matches the top 31 bits of 0xFEFA125A.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - [S](#).*

See also:

[C1.4.7 Instruction set, interworking and interstating support.](#)

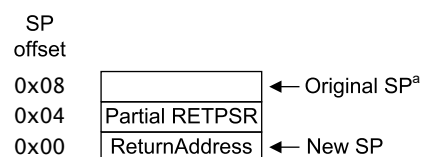
[Chapter B10 The Armv8-M Protected Memory System Architecture.](#)

B3.16 Function calls from Secure state to Non-secure state

R_{CVBB}

If a [BLXNS](#) interstating branch generates a change from Secure state to Non-secure state, then before the Security state change:

- The return address, which is the address of the instruction after the instruction that caused the function call, the [IPSR](#) value and [CONTROL.SFPA](#) are stored onto the current stack, as shown in the following figure. ReturnAddress[0] is set to 1 to indicate a return to the T32 instruction set state. The [IPSR](#) is stacked in the partial [RETPSR](#), and [CONTROL.SFPA](#) is stacked in bit [20] of the partial [RETPSR](#).



- If the PE is in Handler mode, [IPSR](#) has the value of 1.
- The [FNC_RETURN](#) value is saved in the [LR](#).

Applies to an implementation of the architecture [Armv8.0-M](#) onward. The extension requirements are - [S](#). Note, [CONTROL.SFPA](#) requires FP.

R_{MNOV}

Behavior is CONSTRAINED UNPREDICTABLE when a function call stack frame is not doubleword-aligned, and one of the following behaviors must occur:

- The instruction uses the current value of the stack pointer.
- The instruction behaves as though bits [2:0] of the stack pointer are 0b000.

Applies to an implementation of the architecture [Armv8.0-M](#) onward. The extension requirements are - [S](#).

I_{KWZD}

Arm recommends that when Secure code calls a Non-secure function, any registers not passing function arguments are set to 0.

Applies to an implementation of the architecture [Armv8.0-M](#) onward. The extension requirements are - [S](#).

See also:

[C1.4.7 Instruction set, interworking and interstating support.](#)

B3.17 Function returns from Non-secure state

R_{HPPG}

An interstating function return begins when one of the following instructions loads a **FNC_RETURN** value into the PC:

- A **POP (multiple registers)** or **LDM** that includes loading the PC.
- An **LDR** instruction with the PC as a destination.
- A **BX** instruction with any register.
- A **BXNS** instruction with any register.

On detecting a **FNC_RETURN** value in the PC:

- The **FNC_RETURN** stack frame is unstacked.
- **EPSR.IT** is set to 0x00.
- The following *integrity checks on function return* are performed:
 - A check that **IPSR** is zero or 1 before the value of it is restored.
 - A check that if the function return is being performed in Thread mode the stacked **RETPSR.Exception** value is zero.
 - A check that if the function return is being performed in Handler mode the stacked **RETPSR.Exception** value is nonzero.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

I_{QBJN}

Behavior is CONSTRAINED UNPREDICTABLE when a function call stack frame is not doubleword-aligned.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{DWTF}

The **FNC_RETURN** value is:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	S

Bits[31:1]

This is what identifies the value as an **FNC_RETURN** value.

Bit[0], S: The function return was from:

0: Secure state.

1: Non-secure state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{FRSP}

Any fault raised on function return is synchronous to the instruction that loaded the **FNC_RETURN** value into the **PC**.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{PBNM}

Any failed integrity check on function return generates a Secure INVPC UsageFault.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{FGNB}

If **FNC_RETURN** does not fail the integrity checks then the PE behaves as follows:

- ReturnAddress bits [31:1] is written to the **PC**.
- ReturnAddress bit [0] is written to **EPSR.T**.
- The partial **RETPSR** is written to **IPSR** Exception and **CONTROL.SFPA**.
- **EPSR.IT** is cleared to zero.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#). Note, CONTROL.SFPA requires FP.

R_{LNFB} If the exception indicated by [RETPSR.Exception](#) is not supported by the PE [IPSR.Exception](#) is set to an UNKNOWN value.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{BGTC} If a precise BusFault is raised during [FNC_RETURN](#) stack frame unstacking, then it is permitted to raise the fault regardless of the state of [CCR.BFHFNMIGN](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

See also:

[B3.16 Function calls from Secure state to Non-secure state.](#)

[B3.31 Hardware-controlled priority escalation to HardFault.](#)

[B3.33 Lockup.](#)

B3.18 Exception handling

I _{GCFQ}	This section describes the hardware behavior on entry into an exception handler when an exception has been recognized. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{KLWH}	If an Asynchronous exception occurs it is required to be architecturally pended within a finite time. Once an asynchronous exception has been pended, it is required to be taken within finite time if the asynchronous exception is enabled and has sufficient priority to preempt the current execution priority. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{KFRF}	If an exception was pending but is changed to not pending before it is taken, then the architecture permits the exception to be taken but does not require that the exception is taken. If the exception is taken it must be taken before the first Context synchronization event after the exception was changed to not pending. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{YFHR}	An exception that does not cause lockup sets both: • The pending bit of its handler, or the pending bit of the HardFault handler, to 1. • The associated fault status information. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{VLDB}	When a pending exception has a lower group priority value than current execution, including accounting for any priority adjustment by AIRCR.PRIS, the pending exception preempts current execution. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{WYWW}	Preemption of current execution causes the following basic sequence: 1. The return address is determined and stacked along with the State context registers. 2. The Additional state context may be stacked depending on the value of EXC_RETURN.DCRS and the Security transition being made. 3. The Floating-point context may be stacked, which might be any one of the following: • No stacking or preservation of the Floating-point context. • Stacking the basic Floating-point context. • Stacking the basic Floating-point context and the Additional floating-point context. • Activation of Lazy Floating-point state preservation. 4. LR is set to EXC_RETURN. 5. The Floating-point registers may be cleared, depending on the Security state transition. 6. The following flags are also cleared: • IT State is cleared, if the Main Extension is implemented. • CONTROL.FPCA is cleared, if the Floating-point Extension is implemented. • CONTROL.SFPA is cleared, if the Floating-point Extension and the Security Extension are implemented. 7. The exception to be taken is chosen, and IPSR.Exception is set accordingly. The setting of IPSR.Exception to a nonzero value causes the PE to change to Handler mode. 8. CONTROL.SPSEL is set to 0, to indicate the selection of the main stack, dependent on the Security state being targeted. 9. The pending bit of the exception to be taken is set to 0. The active bit of the exception to be taken is set to 1. 10. The Security state is changed to the Security state of the exception that is being activated.

11. The stacked [State context registers](#) is either set to UNKNOWN or cleared to zero based on the Security state transition being made and the architectural features implemented.
12. The stacked [Additional state context registers](#) is either preserved, set to UNKNOWN, or cleared to zero based on the Security state transition being made and the architectural features implemented.
13. [EPSR.T](#) is set to bit[0] of the exception vector for the exception to be taken.
14. The [PC](#) is set to the exception vector for the exception to be taken.

Applies to an implementation of the architecture Armv8.0-M onward. Note, some steps might require additional extensions.

R_{NJVF}

When, during exception entry, the target Security state of an exception differs from the Security state of the memory the exception vector targets:

- An INVEP SecureFault is generated if the exception is Non-secure and the exception vector targets Secure memory.
 - The INVEP SecureFault can be avoided if the exception is associated with Non-secure state and is targeting an [SG](#) instruction that is located in memory that is Secure and Non-secure callable.
- An INVTRAN SecureFault is generated if the exception is Secure and the exception vector targets Non-secure memory.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#). Note, an INVEP or INVTRAN SecureFault requires M.

R_{QLHB}

The return address is one of the following:

- For a synchronous exception, other than an SVCcall exception and a [SVC](#) instruction that escalates to HardFault, the address of the instruction that caused the exception.
- For an asynchronous exception, the address of the next instruction in the program order.
- For an SVCcall exception and a [SVC](#) instruction that escalates to HardFault, the address of the next instruction in the program order.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XKDD}

The least significant bit of the return address from an exception is RES0.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{HZDC}

The possible faults, and their ordering, that might be encountered during exception handling are described in *Arm® Armv8-M Faults on Exception Handling* (ARM DDI 0625).

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.10 Exception enable, pending, and active bits.](#)

[B3.13 Priority model.](#)

[B3.19 Exception entry, context stacking.](#)

[B3.20 Exception entry, register clearing after context stacking.](#)

[B3.30 Vector tables.](#)

[B3.21 Stack limit checks.](#)

[B3.24 Exceptions during exception entry.](#)

[Chapter B5 Vector Extension](#)

Applies to an implementation of the architecture Armv8.1-M onward.

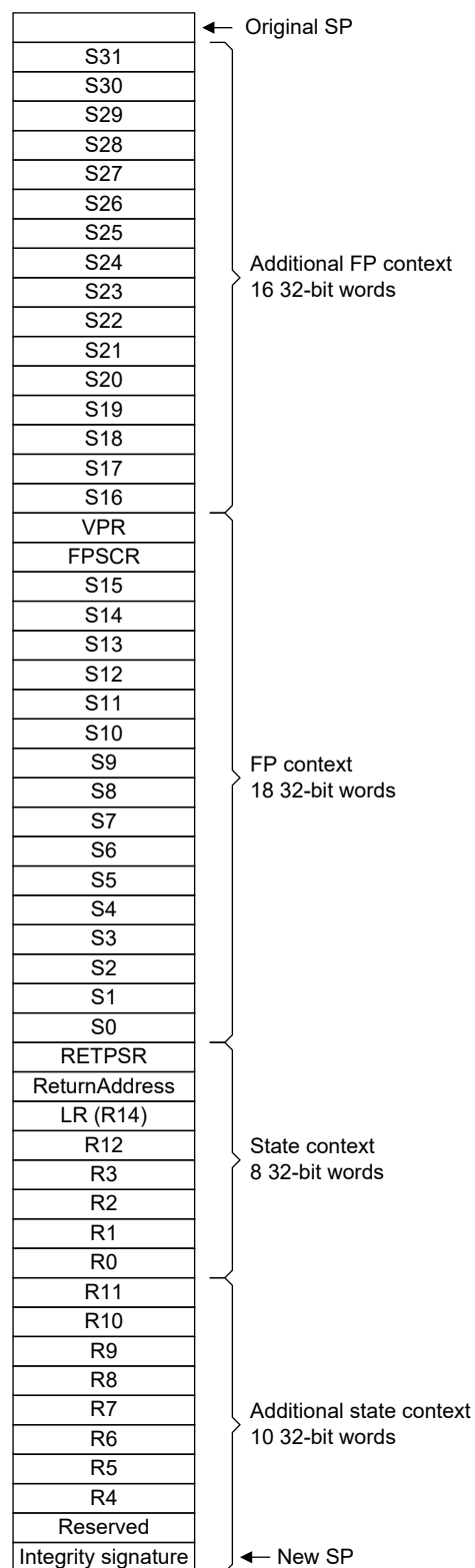
Arm® Armv8-M Faults on Exception Handling (ARM DDI 0625).

B3.19 Exception entry, context stacking

R_{QDKQ} When stacking the context on exception entry, including [tail-chaining](#), full descending stacks are used.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MFPR} The following diagram shows the format stack frame generated on exception entry.



Applies to an implementation of the architecture *Armv8.0-M* onward.

R _{PWWG}	The exception stack frame is divided into the following four sections: • State context registers. • Additional state context registers. • floating-point context registers. • Additional floating-point context registers. The State context registers are always stacked on exception entry, except in tail-chaining. The presence of the other sections depends on the type of exception transition being performed and whether or not there is an active floating-point context. Unless otherwise stated if a section is not stacked, the other sections are shifted upwards to reduce the amount of stack space required. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SHNX}	On taking an exception, excluding tail-chaining that requires a transition from Secure to Non-secure state, the PE hardware saves Additional state context registers. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{BLQS}	If a Secure exception is taken from a Secure context of execution, it is IMPLEMENTATION DEFINED whether: • The Additional state context register stacking is not attempted and EXC_RETURN.DCRS is set to 1. • The Additional state context register stacking is attempted and EXC_RETURN.DCRS is set to 0. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
I _{TQYR}	If a higher priority Secure exception, that does not require the stacking of the Additional state context registers occurs during exception entry after the PE has begun stacking the additional state context, but before any exception handler has started execution, in preparation to take a Secure exception the PE might: • Discard the stacking of the Additional state context registers and EXC_RETURN.DCRS is set to 1. • Complete the stacking of the Additional state context registers and EXC_RETURN.DCRS is set to 0. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{SGKY}	If CONTROL.FPCA is 1 when the exception is taken, excluding tail-chaining, then in addition to the state context being saved, the floating-point context is handled in one of the following ways: • Stack the Floating-point context. • Reserve space on the stack for saving the Floating-point context in the future. This is called <i>lazy Floating-point state preservation</i>. If CONTROL.FPCA is 0, then there is no floating-point context active. No stacking or lazy state preservation of the floating-point context is performed. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP MVE.</i>
R _{NXDF}	On taking an exception when CONTROL.FPCA is 1 and FPCCR.S.TS is 1 and the background state is Secure state, the PE hardware either stacks both the Floating-point context and Additional floating-point context, or when lazy Floating-point state preservation is enabled, reserves space on the stack for both the Floating-point context and Additional floating-point context. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - (FP MVE) && S.</i>
R _{VNSK}	If one or more of the following exceptions is generated during stacking operations on exception entry, including tail-chaining, the PE is permitted to abandon any remaining stacking operations: • MemManage fault. • BusFault. • AUVIOL SecureFault. • STKOF UsageFault. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

R_{WCHZ} An exception taken during exception entry that is not related to stacking the [state context](#) and [Additional state context](#), for example an NOCP UsageFault or LSERR SecureFault, the PE must complete stacking of the [state context](#) and [Additional state context](#).

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BKVD} On an exception, the [RETPSR](#) value that is stacked is all the following:

- The [APSR](#), [IPSR](#), and [EPSR](#).
- [CONTROL.SFPA](#), in [RETPSR.SFPA](#) if the background state is Secure state.

Applies to an implementation of the architecture Armv8.0-M onward. Note, [CONTROL.SFPA](#) requires S && (FP || MVE).

R_{PBVF} On exception entry, excluding [tail-chaining](#), if the stack pointer belonging to the context that is being preempted is not doubleword aligned, the stack pointer is decremented by four to enforce doubleword alignment. [RETPSR.SPREALIGN](#) is set to one to indicate that the stack pointer has been decremented by four. If the stack pointer has not been realigned [RETPSR.SPREALIGN](#) is set to zero.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{ZWQX} In a PE with the Floating-point Extension or MVE, because setting [FPCCR.ASPEN](#) to one causes the PE to automatically set [CONTROL.FPCA](#) to 1 on the execution of a floating-point or MVE instruction, setting [FPCCR.ASPEN](#) to one means that for any software that uses floating-point or MVE the PE hardware automatically either:

- Stack [Floating-point context](#) on taking an exception.
- Activate lazy *Floating-point state preservation* on taking an exception.

If [CONTROL.FPCA](#) == 1, it is [FPCCR.LSPEN](#) that determines whether the PE hardware performs stacking or lazy Floating-point state preservation:

0 : The PE hardware automatically stacks the floating-point registers.

1: On taking an exception, the PE hardware activates lazy Floating-point state preservation and sets all of:

- The [FPCAR](#), to point to the reserved S0 stack address.
- [FPCCR.LSPACT](#) to one.
- [FPCCR.{USER, THREAD, HFREADY, MMRDY, BFRDY, SFRDY, MONRDY, UFRDY, SPLIMVIOL}](#), to record the permissions and fault possibilities to be applied to any subsequent [Floating-point context](#) and [Additional floating-point context](#) save.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [FP](#) || [MVE](#).

I_{ZBMH} Executing a [VLSTM](#) instruction can activate lazy Floating-point state preservation.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [FP](#) || [MVE](#).

R_{VBHY} On activating lazy Floating-point state preservation:

- If the activation of lazy floating-point preservation was triggered by exception entry, and the stacking operation would violate the stack pointer limit [FPCCR.SPLIMVIOL](#) is set to one, otherwise [FPCCR.SPLIMVIOL](#) is set to zero.
- If the current Security state is Secure then [FPCCR.S](#) is set to one, otherwise zero.
- If the current privilege level is privileged [FPPCR.USER](#) is set to zero. If the current privilege level is unprivileged [FPPCR.USER](#) is set to one.
- If the current mode is Thread [FPPCR.THREAD](#) is set to one. If the current mode is Handler [FPPCR.THREAD](#) is set to zero.
- If the lazy Floating-point state preservation was triggered by exception entry the [FPCAR](#), is set to point to the reserved S0 stack address.
- If the activation of lazy floating-point preservation was triggered by a [VLSTM](#) instruction [FPCAR](#) is set to the address specified by that instruction.

- **FPCCR.LSPACT** to one.
- Set **FPCCR.{USER, THREAD, HFRDY, MMRDY, BFRDY, SFRDY, MONRDY, UFRDY}** to one or zero, to record the permissions and fault possibilities to be applied to any subsequent **Floating-point context** and **Additional floating-point context** stacking. The fields will be set to one according to the following conditions:
 - If the execution priority value, as defined by **ExecutionPriority()**, is greater than minus one then **FPCCR.HFRDY** is set to one, otherwise this field is set to zero.
 - If **SHCSR.BUSFAULTENA** is set and the execution priority value, as defined by **ExecutionPriority()**, is greater than the priority value for a BusFault **FPCCR.BFRDY** is set to one, otherwise this field is set to zero.
 - If **SHCSR.MEMFAULTENA** is set and the execution priority value, as defined by **ExecutionPriority()**, is greater than the priority value for a MemManage Fault, the **FPCCR.MMRDY** bit for the current Security state is set to one, otherwise this field is set to zero.
 - If **SHCSR_S.USGFAULTENA** is set and the execution priority value, as defined by **ExecutionPriority()**, is greater than the priority value for a Secure UsageFault **FPCCR_S.UFRDY** is set to one, otherwise this field is set to zero.
 - If **SHCSR_NS.USGFAULTENA** is set and the execution priority value, as defined by **ExecutionPriority()**, is greater than the priority value for a Non-secure UsageFault **FPCCR_NS.UFRDY** is set to one, otherwise this field is set to zero.
 - If **SHCSR.SECUREFAULTENA** is set and the execution priority value, as defined by **ExecutionPriority()**, is greater than the priority value for a SecureFault **FPCCR.SFRDY** is set to one, otherwise this field is set to zero.
 - If **CanPendMonitorOnEvent()** returns TRUE, **FPCCR.MONRDY** is set to one, otherwise this field is set to zero.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

R_{GHDJ}

Execution of a floating-point or MVE instruction while lazy Floating-point state preservation is active, that is when the banked **FPCCR.LSPACT** is one for the Security state indicated by **FPCCR.S**, triggers the preservation of the floating-point state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

R_{YTOC}

If an attempt is made to execute a floating-point or MVE instruction while lazy Floating-point state preservation is active, the access permissions that **CPACR** and **NSACR** define are checked against the context that activated lazy Floating-point state preservation, in addition to the checks relating to memory access permissions.

- If no permission violation is detected, the PE:
 1. If **AIRCR.IESB** is set to one an implicit Error synchronization barrier is performed.
 2. Saves **Floating-point context** to the reserved area on the stack, as identified by the **FPCAR**.
 3. Saves the **Additional floating-point context** if **FPCCR.TS** == 1 for the Security state indicated by **FPCCR.S**.
 4. Clears the floating-point registers that were saved to the stack if **FPCCR_S.TS** == 1 and **FPCCR.S** == 1. Otherwise, the PE sets the floating-point registers to an UNKNOWN value.
 5. Sets **FPCCR.LSPACT** to 0 to indicate that lazy Floating-point state preservation is no longer active.
 6. If **AIRCR.IESB** is set to one an implicit Error synchronization barrier is performed.
 7. Processes the floating-point instruction.
- If a **CPACR** or **NSACR** permission violation is detected, the PE generates a NOCP UsageFault and does not save or clear the Floating-point registers to the reserved area on the stack.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

R_{KWNM}

Any faults that are caused by saving the floating-point registers during exception entry escalate in the same way as a fault on stacking the state context.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

R _{QFWH}	If a fault is generated during lazy Floating-point state preservation then whether to escalate the fault determined by using the FPCCR.*RDY fields rather than the current execution priority. If the exception is enabled, the exception priority value of the fault is compared with the current execution priority value. If the exception priority value is not sufficient to preempt the execution priority, then the fault is pended. The fault will be taken after the execution priority reduces to a level to allow the fault to preempt execution. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP MVE.</i>
R _{BQFC}	An LSERR SecureFault is raised on exception entry, excluding tail-chaining if all of the the following conditions are met: • CONTROL.FPCA was set for the background context from which the exception is being taken. • The banked version of FPCCR.LSPACT indicated by FPCCR_S.S is set. If the LSERR SecureFault is raised, all of the following occur: • The existing lazy state activation is not modified. • The registers associated with the Floating-point context are not modified. • The registers associated with the Additional Floating-point context are not modified. • No stacking operations are performed. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - (FP MVE) && S.</i>
R _{LGNS}	When all the following conditions are met on exception entry, the PE generates a Secure NOCP UsageFault, skips all Floating-point register saving, clearing or lazy Floating-point state preservation activation and does not allocate space on the stack for floating-point registers: • CONTROL.FPCA == 1. • NSACR.CP10 is 0. • The Background state is Non-secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - (FP MVE) && S.</i>
R _{LRTP}	It is IMPLEMENTATION DEFINED whether beat-wise instructions that check for lazy Floating-point state preservation (by directly or indirectly calling PreserveFPState()) do so before the execution of every beat or once before the first beat to be executed of an instruction. The first beat might not be beat 0 if the instruction is executed with a nonzero ECI value. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R _{QYYT}	When performing lazy Floating-point state preservation that does not cause the execution of the current instruction to terminate, it is IMPLEMENTATION DEFINED whether only the FPCCR.LSPACT flag associated with the Security state indicated by FPCCR.S is cleared or the FPCCR.LSPACT flags are cleared for both Security states. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - (FP MVE) && S.</i>
I _{TWPT}	Arm recommends that when performing lazy Floating-point state preservation both the Secure and Non-secure FPCCR.LSPACT flags should be cleared. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - (FP MVE) && S.</i>
I _{GGMG}	For Armv8.0-M implementations Arm recommends that a NOCP UsageFault takes precedence over UNDEFINSTR faults for all instructions that fall into the range described by the IsCPInstruction() function, except for the following instructions: • VLLDM. • VLSTM. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{QLGM}	From Armv8.1-M, a NOCP UsageFault takes precedence over UNDEFINSTR faults for all instructions that fall into the range described by the IsCPInstruction() function, except for the following instructions: • VLLDM.

- [VLSTM](#).
- [VSCCLRM](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. Note, For further information see the instruction descriptions.

R_{DKCR}

For the following instructions, UNDEFINSTR faults take precedence over NOCP UsageFaults when they are attempting to access FPCXT payloads from Non-secure state:

- [VLDLDR](#) (System Register).
- [VMRS](#).
- [VMSR](#).
- [VSTR](#) (System Register).

Applies to an implementation of the architecture *Armv8.1-M* onward. Note, For further information see the instruction descriptions.

I_{HGGX}

If CP10 is not implemented or disabled, executing an [MVE](#) vector instruction generates a NOCP UsageFault.

Applies to an implementation of the architecture *Armv8.1-M* onward.

R_{NPLD}

The instruction encoding space `0b111x_1111_xxxx_xxxx_xxxx_xxxx_xxxx_xxxx` is considered to be part of CP10 and therefore NOCP UsageFaults are prioritized over UNDEFINSTR UsageFaults in the same way as for other coprocessor instructions.

Applies to an implementation of the architecture *Armv8.1-M* onward.

R_{FVTL}

The value in [CONTROL.SFPA](#) is set automatically by hardware on any of the following events:

- An [SG](#) instruction fetched from Secure memory and executed in Non-secure state clears [CONTROL.SFPA](#) to 0.
- A [BXNS](#) instruction that causes a transition from Secure state to Non-secure state clears [CONTROL.SFPA](#) to 0.
- A [BLXNS](#) instruction that causes a transition from Secure state to Non-secure state preserves the value in [CONTROL.SFPA](#) in the [FNC_RETURN](#) stack frame and then clears [CONTROL.SFPA](#) to 0.
- A valid instruction that loads [FNC_RETURN](#) into the PC sets [CONTROL.SFPA](#) to the value retrieved from the [FNC_RETURN](#) payload.
- [CONTROL.SFPA](#) is saved and restored on exception entry or return in the [RETPSR](#) value in the stack frame.
- Exception entry, including tail chaining, clears [CONTROL.SFPA](#) to 0.
- If the value of [FPCCR.ASPEN](#) is one, then any floating-point or MVE instructions excluding [VLLDM](#) and [VLSTM](#) or [FPCXT_NS](#) access instructions executed in Secure state sets the value of [CONTROL.SFPA](#) to one. If the value of [FPCCR.ASPEN](#) is one and the value of [CONTROL.SFPA](#) is zero when a floating-point instruction is executed in the Secure state, the [FPSCR](#) value is taken from the values set in [FPDSCR](#).

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - ([FP](#) || [MVE](#)) && [S](#).

I_{GJGL}

To ensure future compatibility Arm recommends that the value used to seal the top of the stack is `0xFEFE5EDA5`. This value has the following properties:

- It is not a valid [FNC_RETURN](#) or [EXC_RETURN](#) value.
- It is not the integrity signature used to secure the bottom of the stack frame and cannot be used to inadvertently mark the stack as containing a valid exception stack frame.
- The value starts with `0xF` and is therefore not a valid instruction address and will result in a fault if interpreted as a [FNC_RETURN](#) stack frame.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{FKBH}

If a MemManage fault, BusFault, or AUVIOL SecureFault occurs on a stacking memory access during exception entry, then stacking of [Additional state context](#) is optional.

Whether or not the option of stacking the [Additional state context](#) is exercised, the clearing of [Additional state context](#) is still required on exception entry as described in [ExceptionTaken\(\)](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{HGZQ}

The order in which registers are saved to the stack during exception entry or during lazy Floating-point state preservation is not architected.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{KPYX}

The possible faults, and their ordering, that might be encountered during Floating-point lazy state preservation are described in *Arm® Armv8-M Faults on Exception Handling* (ARM DDI 0625).

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.8 Stack pointer.](#)

[B3.20 Exception entry, register clearing after context stacking.](#)

[B3.23 Integrity signature.](#)

[B3.24 Exceptions during exception entry.](#)

[PushStack\(\)](#).

[UpdateFPCCR\(\)](#).

[MergeExcInfo\(\)](#).

Arm® Armv8-M Faults on Exception Handling (ARM DDI 0625).

B3.20 Exception entry, register clearing after context stacking

R_{MRDS}

On exception entry, including [tail-chaining](#), the PE sets the [State Context](#), [APSR](#) and [EPSR](#) to:

- Zero if the exception being take does not target the Secure state or Armv8.1-M is implemented.
- UNKNOWN in all other cases.

If stacked the PE sets the [Additional state context](#) to:

- Zero if the background state is Secure and the exception targets the Non-secure state.
- UNKNOWN if all the following conditions are met:
 - The background state is Secure.
 - The exception targets the Secure state.
 - [EXC_RETURN.DCRS](#) is zero.
- Otherwise the [Additional state context](#) values are not changed.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{JWBK}

The following operations are performed on exception entry if the [Floating-point context](#) has been stacked:

- If [FPCCR.S.TS](#) is 0 when the [Floating-point context](#) is pushed to the stack, and the registers are set to an UNKNOWN value after stacking.
- If [FPCCR.S.TS](#) is 1 when the [Floating-point context](#) and [Additional floating-point context](#) are both pushed to the stack, and the registers are set to zero after stacking.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [FP](#) && [S](#).

See also:

[B3.19 Exception entry, context stacking.](#)

[B3.26 Tail-chaining.](#)

B3.21 Stack limit checks

R_{PCRT} A PE that does not implement the Main Extension, and does not implement the Security Extension does not implement stack limit checking.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - !M && !S.

R_{NHBX} In a PE without the Main Extension but with the Security Extension, there are two stack limit registers in Secure state for the purposes of stack limit checking.

Security state	Stack	Stack limit registers
Secure	Main	MSPLIM_S
	Process	PSPLIM_S

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S && !M.

R_{JPEX} In a PE with the Main Extension but without the Security Extension, there are two stack limit registers:

Stack	Stack limit registers
Main	MSPLIM
Process	PSPLIM

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && !S.

R_{XQDS} In a PE with the Main Extension and the Security Extension, there are four stack limit registers:

Security state	Stack	Stack limit registers
Secure	Main	MSPLIM_S
	Process	PSPLIM_S
Non-secure	Main	MSPLIM_NS
	Process	PSPLIM_NS

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && S.

I_{KDPG} A stack can descend to its stack limit value. Any attempt to descend the stack further than its stack limit value is a violation of the stack limit.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{TCXN} xSPLIM_x[2:0] are treated as RES0, so that all stack pointer limits are always guaranteed to be doubleword-aligned. Bits [31:3] of the xSPLIM_x registers are writable.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DKSR} Stack limit checks are performed during the creation of a stack frame for all of the following:

- Exception entry.
- [Tail-chaining](#) from a Secure to a Non-secure exception.
- A function call from Secure code to Non-secure code.

Applies to an implementation of the architecture Armv8.0-M onward. Note, Secure exceptions and secure code require S.

R_{ZLZG} On a violation of a stack limit during either exception entry or [tail-chaining](#):

- A synchronous STKOF UsageFault is generated.
- The stack pointer is set to the stack limit value.
- Push operations to addresses below the stack limit value are not performed.

- It is IMPLEMENTATION DEFINED whether push operations to addresses equal to or above the stack limit value are performed.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{YXCQ} If on exception entry or [tail-chaining](#) the stack frame being written wraps around the top or bottom of the 32-bit memory address space, it is IMPLEMENTATION DEFINED whether stack limit checking is applied.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CCSC} On a violation of a Secure stack limit during the stacking of a FNC_RETURN stack frame:

- A synchronous STKOF UsageFault is generated.
- Push operations to addresses below the stack limit value are not performed.
- It is IMPLEMENTATION DEFINED whether push operations to addresses equal to or above the stack limit value are performed.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{GGRH} Unstacking operations are not subject to stack limit checking.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BRLT} Updates to the stack pointer by the [MSR](#) instruction targeting [SP_NS](#) are subject to stack limit checking. Updates to the stack pointer and stack pointer limit by any other [MSR](#) instruction are not subject to stack limit checking.

[LDR](#) instructions write to two registers, the address register and the destination register. Updates to the stack pointer by the [LDR](#) instructions are only subject to stack limit checking if the stack pointer is the address register. It is IMPLEMENTATION DEFINED whether stack limit checking is performed if the [SP](#) is the destination register for a load instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{HRDG} If a STKOF UsageFault is generated when the Non-secure stack pointer is accessed through a [MSR{SP_NS}](#) instruction in Secure state, the STKOF UsageFault can target either the Secure or Non-secure state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

I_{JSPJ} Arm recommends that a STKOF UsageFault generated by a [MSR](#) instruction in Secure state accessing the Non-secure stack pointer should target the Non-secure state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

I_{BJHX} When an instruction updates the stack pointer, if it results in a violation of the stack limit, it is the modification of the stack pointer that generates the exception, rather than an access that uses the out-of-range stack pointer.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{RRDX} [CCR.STKOFHFNMIGN](#) controls whether stack limit violations are IGNORED while executing at a requested execution priority that is negative.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XCQL} It is UNKNOWN whether a stack limit check is performed on any use of the SP marked as UNPREDICTABLE.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{QFPF} A write to the current stack pointer by an instruction subject to stack limit checking with a value less than the stack limit will generate a STKOF UsageFault.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{ZKSV} Setting the stack limit to a value above the current [SP](#) does not directly cause a STKOF UsageFault. However, the next instruction to modify the [SP](#) will cause a STKOF UsageFault if the [SP](#) remains below the stack limit after the update to the [SP](#) has been performed.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DSJN} There is no architectural requirement for stack limit checking to be carried out on exception return as the current stack pointer will only increment and will not decrement.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{XRXD} There is no architectural requirement for stack limit checking to be carried out on instruction resume for an [interrupt-continuable instruction](#). Stack limit checking is required on instruction resume for an [exception-continuable instruction](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{QMRP} If an instruction attempts to make an access below the stack limit, it is UNKNOWN whether a store performing a write-back to the current Stack Pointer will generate a STKOF UsageFault where the value written to the current stack pointer is greater than the stack limit.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#) || [S](#).

R_{BRCX} When a STKOF UsageFault is generated by an instruction:

- No accesses below the stack limit will be performed.
- It is IMPLEMENTATION DEFINED whether an access by the instruction to locations above the stack limit will be performed.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.8 Stack pointer](#).

[B3.26 Tail-chaining](#).

B3.22 Exception return

R_{KPSS}

The PE begins an exception return when both of the following are true:

- The PE is in Handler mode.
- One of the following instructions loads an **EXC_RETURN** value, 0xFFFFXXXX, into the **PC**:
 - A **POP (multiple registers)** or **LDM** that includes loading the **PC**.
 - An **LDR** with the **PC** as a destination.
 - A **BX** with any register.
 - A **BXNS** with any register.

When both of these are true, then on detecting an **EXC_RETURN** value in the **PC**, the PE unstacks the exception stack frame and resumes execution of the unstacked context.

An **EXC_RETURN** value loaded into the **PC** will be treated as an address when any of the following conditions apply:

- The value is loaded by an instruction other than those listed.
- The value is loaded from the vector table by a vector fetch.
- The value is loaded when the PE is in Thread mode.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NSRH}

If a field in the **EXC_RETURN** with a reserved value is set to a value that conflicts with the reserved value the behavior is UNPREDICTABLE.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NCQN}

On exception return the following procedures are carried out:

1. In a PE with the Security Extension, the integrity check that is called the **EXC_RETURN.ES validation check**, as follows:
 - If the PE was in Non-secure state when **EXC_RETURN** was loaded into the **PC** and **EXC_RETURN.ES** is one, an INVER SecureFault is generated and **EXC_RETURN.ES** has an **Effective value** of zero for the rest of the exception return process.
 - If **EXC_RETURN.DCRS** and **EXC_RETURN.ES** are both zero, an INVER SecureFault is generated.
2. A check that the exception number being returned from, as held in the **IPSR**, is shown as active. When the Security Extension is implemented the exception must be active for the Security state indicated by **EXC_RETURN.ES**. If the exception being returned from can be configured to target either Security state, then it must target the Security state indicated by **EXC_RETURN.ES**. If this check fails:
 - An INVPC UsageFault is generated.
 - The UsageFault targets the Security state that the exception return instruction was executed in.
3. A check that if the return is to Thread mode, the value that is restored to the **IPSR** from the **RETPSR** is zero, or that if the return is to Handler mode, the value that is restored to the **IPSR** from the **RETPSR** is nonzero. If this check fails:
 - An INVPC UsageFault is generated.
 - The UsageFault targets the background Security state as indicated by **EXC_RETURN.S**.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DQLL}

On returning from Non-secure state, if **EXC_RETURN.ES** causes an INVER integrity check failure, the subsequent **EXC_RETURN.DCRS** bit that is presented in the **LR** on entry to the resulting derived exception is permitted to be UNKNOWN.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

I _{TLXJ}	Arm recommends that the subsequent EXC_RETURN.DCRS bit that is presented in the LR on entry to the derived exception is not UNKNOWN, and is set to one. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{CDCR}	When returning from an exception, the PE will clear FAULTMASK.FM for the Security state indicated by EXC_RETURN.ES if the raw execution priority is equal to or greater than zero. If the raw execution priority is less than zero the following exceptions are cleared: • If the raw execution priority is -1 and AIRCR.BFHFNMINS is zero indicating Secure HardFault. • If the raw execution priority is -1 and AIRCR.BFHFNMINS is one indicating Non-secure HardFault. • If the raw execution priority is -2, indicating NMI. • If the raw execution priority is -3, indicating Secure HardFault. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{VJKL}	If a fault is generated during the exception return process, the PE will tail-chain into that exception. If tail-chaining is not possible, because after escalation to HardFault the priority is still insufficient to pre-empt the background code, the PE will enter lockup. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{RPGL}	On an exception return that successfully returns to the Background state, with no tail-chaining or failed integrity checks, the Security state is set to EXC_RETURN.S. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
I _{CTWL}	In a PE with the Security Extension, after a successful exception return to the Background state, the PE is in the correct Security state before the next instruction from the background code is executed. This means that in the case where the Background state is Secure state, there is no need for an SG instruction at the exception return address. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{QRWD}	On exception return, and there are no generated exceptions to tail-chain into, the inverse of EXC_RETURN.FType is written to CONTROL.FPCA. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{LNPQ}	Arm recommends that FPCCR.CLRONRET is set to 1, to ensure hardware automatically clears the Floating-point context registers to zero on exception return. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP MVE.</i>
R _{HTQX}	Any faults that are caused by restoring floating-point registers during exception return escalate in the same way as faults on unstacking state context. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{JMGQ}	An LSERR SecureFault is generated on exception return if all of the following conditions are met: • CONTROL.FPCA is 1. • FPCCR.CLRONRET is 1. • FPCCR.S.LSPACT is 1. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S && (FP MVE).</i>
R _{LMSY}	The PE generates an LSERR SecureFault on exception return before unstacking the Floating-point context or Additional floating-point context, when the following conditions are met: • EXC_RETURN.FType is 0. • Secure lazy floating-point state preservation is active, that is, FPCCR.S.LSPACT is 1. • The return is to Non-secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S && (FP MVE).</i>

R_{XLTP}

A NOCP UsageFault is generated on exception return if all of the following conditions are met:

- **CONTROL.FPCA** == 1.
- **FPCCR.LSPACT** == 0.
- **FPCCR.CLRONRET** == 1.
- Access to CP10 from the Security state of the returning exception, as indicated by **EXC_RETURN.ES**, is disabled by **NSACR**, **CPACR**, or **CPPWR**.

The target Security state of the NOCP UsageFault is as follows:

- Secure state, if blocked by **NSACR**.
- The same Security state as the returning exception as indicated by **EXC_RETURN.ES**, if blocked by **CPACR**.
- If the access is blocked by **CPPWR**, the NOCP UsageFault targets the Secure state if **CPPWR.SUS10** == 1. Otherwise, the NOCP UsageFault targets the same Security state as the returning exception as indicated by **EXC_RETURN.ES**.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP || MVE.

R_{QFCN}

The PE hardware will clear **S0-S15**, the **FPSCR** and **VPR** to zero, when all of the following conditions apply:

- **CONTROL.FPCA** is 1.
- **FPCCR.CLRONRET** is 1.
- **CPACR**, **CPPWR**, and **NSACR** if the Security extension is implemented, grant access to CP10 for the Security state of the returning exception, as indicated by **EXC_RETURN.ES**.
- If the Security extension is implemented, **FPCCR.S.LSPACT** is 0.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

R_{NLHJ}

On an exception return that successfully returns to background state, the PE performs the following steps:

- If **EXC_RETURN.FType** is one no floating-point registers are unstacked.
- If **EXC_RETURN.FType** is zero and **FPCCR.LSPACT** is one for the background Security state, as indicated by **EXC_RETURN.S**, **FPCCR.LSPACT** is cleared to zero and no floating-point registers are unstacked.
- Otherwise, the **Floating-point context** is unstacked, and if **FPCCR.TS** is one for the background Security state, as indicated by **EXC_RETURN.S**, the **Additional Floating point context** is unstacked.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

R_{RXJC}

A check of **CPACR.CP10**, for the Security state indicated by **EXC_RETURN.S**, and the relevant fields in **NSACR** and **CPPWR** is undertaken, before unstacking the floating-point registers.

If the check fails, a NOCP UsageFault is generated. The target Security state of the NOCP UsageFault is:

- Secure state, if blocked by **NSACR**.
- If the access is blocked by **CPPWR**, the NOCP UsageFault targets the Secure state if **CPPWR.SUS10** == 1.
- Otherwise, the NOCP UsageFault targets the same Security state as the returning exception as indicated by **EXC_RETURN.S**.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

R_{WCSC}

S0 to **S31**, **FPSCR**, and **VPR** are not modified if the checks prior to unstacking fail.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE. Note, VPR requires MVE.

R_{HNNW}

If the PE abandons unstacking of the floating-point registers to **tail-chain** into another exception, then if the Security Extension is implemented, the PE clears to zero any floating-point registers, **FPSCR**, and **VPR** that would have been unstacked.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - (FP || MVE) && S. Note, VPR requires MVE.

R_{LMNG} If the PE abandons unstacking of the floating-point registers to [tail-chain](#) into another exception, then if the Security Extension is not implemented, then the floating-point registers, [FPSCR](#), and [VPR](#) that would have been unstacked become UNKNOWN.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ([FP](#) || [MVE](#)) && [!S](#). Note, VPR requires MVE.

R_{QVKT} The PE will enter an IMPLEMENTATION DEFINED sleep mode if all the following are true:

- The return is to Thread Mode.
- [SCR.SLEEPONEXIT](#) is one.
- There are no further active exceptions after deactivating the returning exception.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WCGV} Following completion of the validation requirements of the [EXC_RETURN](#) value the PE returns to execution and the following occurs:

- The registers are restored from the stack frame, in accordance with the [EXC_RETURN](#) flags:
 - If [EXC_RETURN.S](#) is one, either the Secure Main or Process stack is used. If [EXC_RETURN.S](#) is zero, either the Non-secure Main or Process stack is used.
 - If [EXC_RETURN.ES](#) is 1, [EXC_RETURN.SPSEL](#) is written to [CONTROL_S.SPSEL](#).
 - If [EXC_RETURN.ES](#) is 0, [EXC_RETURN.SPSEL](#) is written to [CONTROL_NS.SPSEL](#).
 - The Secure or Non-secure bank of [CONTROL.SPSEL](#) is read as indicated by [EXC_RETURN.S](#). If the read [CONTROL.SPSEL](#) is one, the Process stack pointer is used, otherwise the Main stack pointer is used.
- The [State context](#) is popped from the stack.
- If [EXC_RETURN.S](#) is one, and any of the following conditions are met, the [Additional state context](#) is popped from the stack:
 - [EXC_RETURN.ES](#) is zero.
 - [EXC_RETURN.DCRS](#) is zero.
- If [EXC_RETURN.Mode](#) is one the PE transitions to Thread mode.
- [APSR](#), [EPSR](#), and [IPSR](#) are restored from [RETPSR](#).
- The [PC](#) is set to `ReturnAddress [31:1]: '0'`.
- Bit[0] of the `ReturnAddress` is discarded.
- If [EXC_RETURN.S](#) is one then [CONTROL.SFPA](#) is set to [RETPSR.SFPA](#).

Applies to an implementation of the architecture Armv8.0-M onward.

I_{GTDP} The order in which registers, including floating-point registers, are restored from the stack is not architected.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{LTMK} If the return unstacks the [Additional state context registers](#), the PE will check the Integrity Signature popped from the stack. If the Integrity Signature popped from the stack does not match the expected signature, an INVIS SecureFault is generated.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DQWV} The unstacking memory operations triggered by an exception return are treated as if the operations are a load instruction executed in the Security state being returned to, as indicated by [EXC_RETURN.S](#). Any faults generated by the unstacking memory operations will behave in the same way as a fault on a load instruction, and target the appropriate Security state.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[Chapter B2 Power Management.](#)

[B3.18 Exception handling.](#)

[B3.23 Integrity signature.](#)

[B3.33 Lockup.](#)

[Chapter B5 Vector Extension.](#)

*Applies to an implementation of the architecture **Armv8.1-M** onward.*

[ExceptionReturn\(\)](#)

[ValidateExceptionReturn\(\)](#)

B3.23 Integrity signature

I_{LHHR}

In a PE without the Floating-point Extension register file, the integrity signature value is:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	1	1	1	1	0	1	0	0	0	0	1	0	0	1	0	0	1	0	1	1	0	1	1

In a PE with the Floating-point Extension register file, the integrity signature value is:

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	1	1	1	1	0	1	0	0	0	0	1	0	0	1	0	0	1	0	1	1	0	1	SFTC

Stack Frame Type Check $\neg$

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S**.

R_{PJPQ}

When **EXC_RETURN.S** is one and the **additional state context** is unstacked on exception return and the unstacked integrity signature does not contain a valid value, including if SFTC does not match **EXC_RETURN.FType**, an INVIS SecureFault is generated.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S**.

I_{FFTS}

The integrity signature is an XN address. When performing a function return from Non-secure code, if the integrity signature value is restored to the PC as the function return address, a MemManage fault is generated when the PE attempts execution.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S**.

See also:

[B3.19 Exception entry, context stacking.](#)

[B3.22 Exception return.](#)

B3.24 Exceptions during exception entry

I_{LBGQ}

During exception entry exceptions can occur, for example asynchronous exceptions, or the exception entry sequence itself might cause an exception, for example a MemManage fault on the push to the stack.

Any exception that occurs during exception entry is a late-arriving exception, and:

- The exception that caused the original entry sequence is the *original exception*.
- The priority of the code stream running at the time of the original exception is the *preempted priority*.

When the exception entry sequence itself causes an exception, the latter exception is a *derived exception*.

The following mechanism is called *late-arrival preemption*. The PE takes a late-arriving exception during an exception entry if the late-arriving exception is higher priority, including accounting for any priority adjustment by [AIRCR.PRIS](#) or any required escalation to HardFault.

Late-arriving exceptions are subject to the rules on escalation to HardFault and [Lockup](#).

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NMTT}

In late-arrival preemption:

- The late-arriving exception uses the exception entry sequence started by the original exception. The original exception remains pending.
- The PE takes the original exception after returning from the late-arriving exception.
- The PE ignores non-terminal faults on taking a derived exception on late-arrival preemption.

The pseudocode [DerivedLateArrival\(\)](#) describes this.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MRTT}

For Derived exceptions, late-arrival preemption is mandatory.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BXTB}

For late-arriving asynchronous exceptions, it is IMPLEMENTATION DEFINED whether late-arrival preemption is used. If the PE does not implement late-arrival preemption for late-arriving asynchronous exceptions, late-arriving asynchronous exceptions become pending.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{MPNB}

If a higher priority late-arriving Secure exception occurs during entry to a Non-secure exception when the Background state is Secure, it is IMPLEMENTATION DEFINED whether:

- The stacking of the additional state context is rolled back.
- The stacking of the additional state context is completed and [EXC_RETURN.DCRS](#) is set to 0.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{GDNT}

If the group priority value of a derived exception is higher than or equal to the preempted priority:

- If the derived exception is a DebugMonitor exception, it is IGNORED.
- Otherwise, the PE escalates the derived exception to HardFault or [Lockup](#) if the HardFault cannot be taken due to the current priority.

Applies to an implementation of the architecture Armv8.0-M onward. Note, a DebugMonitor Exception requires the Debug-Monitor exception.

I_{NJCW}

The architecture does not specify the point during exception entry at which the PE recognizes the arrival of an asynchronous exception.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MOBK}

The possible faults, and their ordering, that might be encountered during exception entry are described in *Arm® Armv8-M Faults on Exception Handling* (ARM DDI 0625).

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.9 Exception numbers and exception priority numbers.](#)

[B3.13 Priority model.](#)

[B3.18 Exception handling.](#)

[B3.26 Tail-chaining.](#)

[B3.31 Hardware-controlled priority escalation to HardFault.](#)

[B3.33 Lockup.](#)

Arm® Armv8-M Faults on Exception Handling (ARM DDI 0625).

B3.25 Exceptions during exception return

I _{KXPV}	During exception return exceptions can occur, for example asynchronous exceptions, or the exception return might itself cause an exception. Any exception that occurs during exception return is a <i>late-arriving exception</i>. When the exception return sequence itself causes an exception, the latter exception is a derived exception. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{TRFM}	When a late-arriving exception during exception return has a lower priority value than the priority being returned to, the PE takes the late-arriving exception by using tail-chaining. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{MBNG}	The architecture does not specify the point during exception return at which the PE recognizes the arrival of an asynchronous exception. If a PE recognizes an asynchronous exception after an exception return has completed, there is no opportunity to tail-chain the asynchronous exception. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HHSX}	If the group priority value of a derived exception during exception return is equal to or higher than the execution priority being returned to: • If the derived exception is a DebugMonitor exception, the PE ignores the derived exception. • Otherwise, the PE escalates the derived exception to HardFault and the escalated exception is tail-chained or the PE enters lockup if the HardFault cannot be taken due to the execution priority being returned to. <i>Applies to an implementation of the architecture Armv8.0-M onward. Note, a DebugMonitor Exception requires the Debug-Monitor exception.</i>
R _{DHFK}	If the priority value of a derived exception during exception return, after priority escalation if appropriate, is a lower priority value than the execution priority being returned to, the PE uses tail-chaining to take the derived exception. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{ZWMN}	The possible faults, and their ordering, that might be encountered during exception return are described in <i>Arm® Armv8-M Faults on Exception Handling</i> (ARM DDI 0625). <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[B3.9 Exception numbers and exception priority numbers.](#)

[B3.13 Priority model.](#)

[B3.22 Exception return.](#)

[B3.26 Tail-chaining.](#)

[B3.33 Lockup](#)

[DebugMonitor exception.](#)

Arm® Armv8-M Faults on Exception Handling (ARM DDI 0625).

B3.26 Tail-chaining

R_{FKXX}

Tail-chaining behavior is as follows:

On detecting an **EXC_RETURN** value in the **PC**, if there is a pending exception or a derived exception is raised that has a lower group priority value than the execution priority being returned to, the PE hardware:

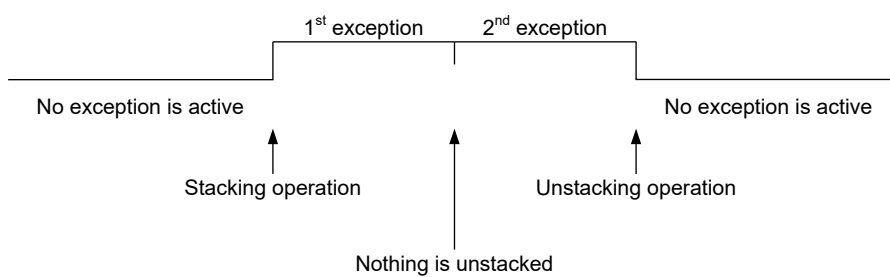
1. Does not unstack the exception stack frame.
2. Takes the pending exception or derived exception.
3. When **tail-chaining** the PE will not execute any instructions from the background state that has been preempted by the original exception.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{FCGM}

Tail-chaining removes unstacking and restacking operations. In the following example the second exception is a **tail-chained exception**:

All in Non-secure state:



Applies to an implementation of the architecture Armv8.0-M onward.

I_{SJSX}

The **tail-chaining** of asynchronous exceptions is an IMPLEMENTATION DEFINED optimization.

Applies to an implementation of the architecture Armv8.0-M onward.

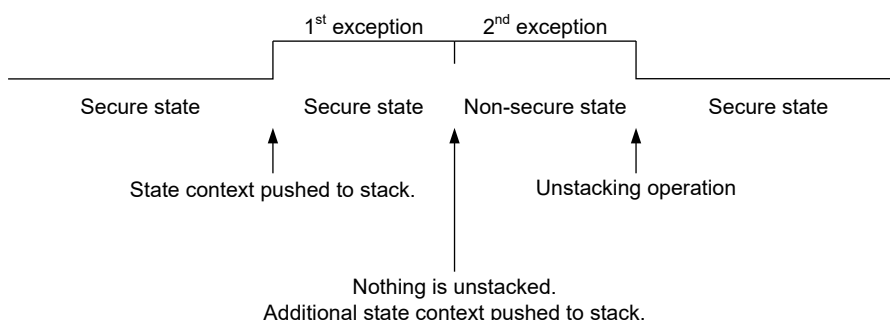
I_{RWDT}

If **tail-chaining** avoids a derived exception on exception return, for example a MemManage fault on an unstacking operation, the derived exception might instead be generated on the return from the last tail-chained exception.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{PXVB}

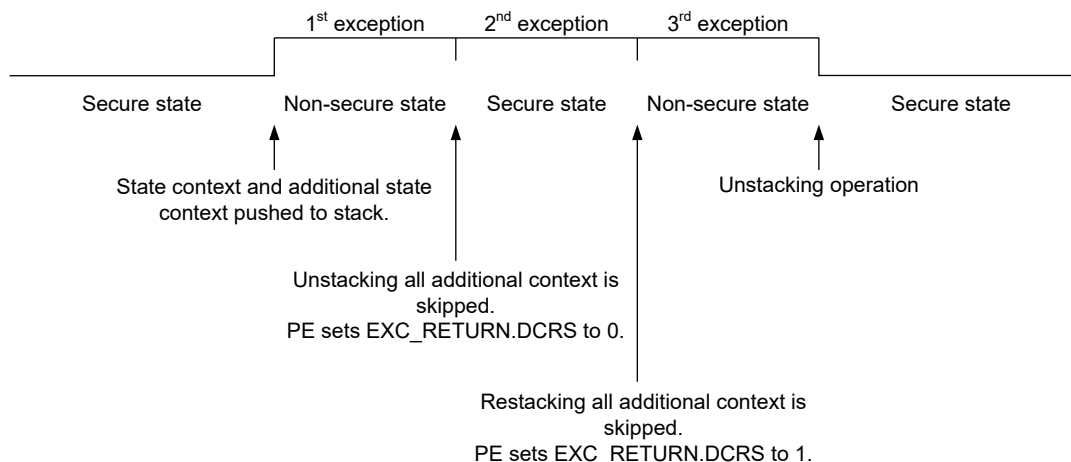
When the Background state is Secure state, as indicated by **EXC_RETURN.S**, if **tail-chaining** causes a change of Security state from Secure to Non-secure, additional context is saved on taking the Non-secure exception if it has not already been saved as indicated by **EXC_RETURN.DCRS**:



Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S**.

I_{TKLM}

When an exception is **tail-chained**, and the returning exception is Secure, **EXC_RETURN.DCRS** keeps track of whether the additional context is stacked. The following figure is an example:



Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S**.

I_{XHFS}

When an exception **tail-chains** for both Secure and Non-secure state, the tail-chained exception cannot rely on the state context registers containing the values they had when no exception was active.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{CGZH}

When an exception **tail-chains**, and if the background state was Secure, the tail-chained exception cannot rely on the additional state context registers containing the values they had when no exception was active.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{JMHS}

If the PE recognizes a new asynchronous exception while it is **tail-chaining**, and the new asynchronous exception has a higher priority than the next tailed-chained exception, the PE can, instead, take the new asynchronous exception, using late-arrival preemption.

This rule is true even if the next tail-chained exception is a derived exception. The PE can, instead, take the new asynchronous exception. If it does, the derived exception becomes pending.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{PLBN}

Before entering a **tail-chained** exception handler during an exception return, the banked **FPCCR.LSPACT** field associated with the background Security state, as indicated by the **EXC_RETURN.S** bit, must be set to one if it was one before the exception return.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP** || **MVE**.

See also:

[B3.19 Exception entry, context stacking.](#)

[B3.25 Exceptions during exception return.](#)

B3.27 Exceptions, instruction resume, or instruction restart

B3.27.1 Basic definitions

I_{WNNP}	Load multiple, Store multiple or Clear multiple instructions that support instruction resume are called interrupt-continuable instructions . <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R_{PGRC}	The PE can take an exception during execution of a interrupt-continuable instruction , effectively halting the instruction, and resuming execution of the instruction after returning from the exception. This is called <i>instruction resume</i> . <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
I_{FFGR}	Instruction resume is an OPTIONAL optimisation. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R_{KRLL}	The PE can abandon execution of an interrupt-continuable instruction to take an exception, and after returning from the exception, restart the interrupt-continuable instruction again from the start of the instruction. This is called <i>instruction restart</i> . <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I_{VVXG}	If MVE is implemented the architecture supports the interruption of beat-wise instructions in the middle of multiple partially executed instructions, these instructions are exception-continuable instructions . <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>

B3.27.2 Instruction restart behavior

R_{KCMD}	To support <i>instruction restart</i> , singleword load and store instructions do not update the destination registers when the PE takes an exception during execution. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{VBVG}	If during the execution of a load or store dual instruction to Normal memory, an exception is taken between the two memory accesses then the behavior is as follows: - Any register used by the load instruction, that is not used in the generation of the address used by the load instruction, can contain an UNKNOWN value. - Any register used in the generation of the address used by a load instruction is restored to its initial value. - Any location being stored to by a store instruction can have an UNKNOWN value. Examples of load and store dual instructions are LDRD (literal) or STRD (immediate). <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R_{WGRG}	Where a fault is taken during the execution of a VLLDM instruction the PE abandons the stacking of the Secure floating-point register contents and does not modify the floating-point registers, so that on return from the fault the instruction can be restarted. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S && (FP MVE).</i>
I_{JJQX}	If the PE is using instruction restart, Arm recommends that Load Multiple, or Store Multiple instructions are not used with data in Device memory. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

B3.27.3 Interrupt-continuable instructionsR_{GPQG}

The interrupt-continuable instructions are as follows:

- LDM.
- LDMDB.
- STM.
- STMDB.
- POP (multiple registers).
- Push (multiple registers).

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **M**.

R_{RDHK}

In a PE with the Floating-point Extension, the floating-point interrupt-continuable instructions are as follows:

- VLDM.
- VSTM.
- VPOP.
- VPUSH.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP** || **MVE**.

R_{FQDX}

It is IMPLEMENTATION DEFINED whether the **CLRM** instruction is interrupt-continuable.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **M**.

R_{QRXG}

It is IMPLEMENTATION DEFINED whether the **VSCCLRM** instruction is interrupt-continuable.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **M**.

R_{SXPP}

It is IMPLEMENTATION DEFINED whether, if there is an asynchronous exception, instruction execution can be interrupted after triggering lazy Floating-point state preservation, but before updating the ownership of the floating-point registers.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP** || **MVE**.

R_{TLVX}

If an instruction can trigger the update of **CONTROL.FPCA** and initialise **FPSCR**, **VPR** without raising an exception, and were that instruction to be abandoned due to an exception, it is IMPLEMENTATION DEFINED if those registers are preserved.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP** || **MVE**.

R_{YLDS}

The **VLLDM** and **VLSTM** instructions are not interrupt-continuable. It is IMPLEMENTATION DEFINED whether **VLLDM** and **VLSTM** instructions abort or complete when an interrupt occurs.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **M**.

R_{XBGJ}

When executing the **VLLDM** instruction, and when **FPCCR.LSPACT** is zero, if the instruction does not raise a NOCP UsageFault, nor an UNALIGNED UsageFault, and is either interrupted or aborted due to a memory fault, then any registers that would be loaded by this instruction must be cleared.

If no registers have been updated as a result of the interrupted **VLLDM** instruction, the registers are permitted to retain their former values.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **M**.

R_{BFBP}

When the PE is using instruction resume, **EPSR.ICI** is set to a nonzero value that is the continuation state of the interrupt-continuable instruction:

- For **LDM**, **LDMDB**, **STM**, **STMDB**, **POP (multiple registers)**, **PUSH (multiple registers)** and **CLRM** instructions, **EPSR.ICI** contains the number of the first register in the register list that is to be loaded, stored or cleared after instruction resume.

- For the floating-point instructions `VLDM`, `VSTM`, `VPOP`, `VPUSH`, and `VSCCLRM` `EPSR.ICI` contains the number of the lowest numbered doubleword Floating-point Extension register that was not loaded, stored or cleared before the PE took the exception.

The `EPSR.ICI` values shown in the following table are valid `EPSR.ICI` values:

<code>EPSR[26:25]</code>	<code>EPSR[15:12]</code>	<code>EPSR[11:10]</code>
<code>ICI[7:6] = 0b00</code>	<code>ICI[5:2] = reg_num</code>	<code>ICI[1:0] = 0b00</code>
<code>ICI[7:6] = 0b00</code>	<code>ICI[5:2] = 0b0000</code>	<code>ICI[1:0] = 0b00</code>

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **M**. Note, some instructions listed require *FP*.

R_{PCZR}

If `EPSR.ICI` contains a valid `ICI` nonzero value and the register number that it contains is either:

- Not in the register list of the [interrupt-continuable instructions](#).
- The first register in the register list of the [interrupt-continuable instructions](#).

Then, when an interrupt-continuable instruction is executed, behavior is a CONSTRAINED UNPREDICTABLE choice between the following:

- The instruction generates an `INVSTATE UsageFault`.
- The instruction completes execution.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **M**.

R_{SZFC}

When an instruction that is permitted to be interrupt-continuable completes execution and is outside of an IT block, the `EPSR.ICI` field is cleared.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **M**.

I_{GZQD}

The requirement to clear `EPSR.ICI` applies regardless of whether the PE supports *instruction resume*, or whether the PE has implemented an instruction that is optionally interrupt-continuable as interrupt-continuable.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **M**.

B3.27.4 Behavior of interrupt-continuable instructions

R_{ZLVP}

If the `VSCCLRM` instruction supports resuming from a nonzero `EPSR.ICI` value, then the `ICI` value will correspond to the Floating-point registers in the register list and not the `VPR` register. When the last Floating-point register in the register list is cleared, the `VPR` register must also be cleared and the instruction must be completed.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **S** && (**MVE** || **FP**).

R_{KLPL}

The PE does not generate an `INVSTATE UsageFault` if the `VSCCLRM` instruction does not support instruction resume from a nonzero `EPSR.ICI` value. If an attempt is made to resume this instruction from a nonzero `EPSR.ICI` value, then the nonzero `EPSR.ICI` value is ignored and the instruction is restarted.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **S** && (**MVE** || **FP**).

R_{JXKQ}

If the PE uses instruction resume during a load multiple interrupt-continuable instruction, then after the instruction is interrupted, the values of all registers in the register list are UNKNOWN, except for the following:

- Registers that are marked by `EPSR.ICI` as already loaded.
- The base register.
- The `PC`.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **M**.

R_{GZCT} If the PE uses instruction resume during a clear multiple interrupt-continuable instruction, then after the instruction is interrupted, the values of all registers in the register list are UNKNOWN, except for the registers that are marked by [EPSR.ICI](#) as already cleared.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

B3.27.5 Exceptions during interrupt-continuable instructions

R_{PQRT} When the PE uses instruction resume to resume a Load Multiple instruction with the [PC](#) in its register list, the PE must, if the PC was loaded before generation of the exception, restore the [PC](#) before taking the exception.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{TGGR} In a PE that does not implement [EPSR.ICI](#), or when a fault is taken during any Load Multiple or Store Multiple instruction, including [PUSH \(multiple registers\)](#) and [POP \(multiple registers\)](#), the PE uses instruction restart and the [Base register](#) is restored to the original value. If the instruction is outside of an IT block [EPSR.ICI](#) is cleared to zero.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{LDZW} If the PE implements interrupt-continuable instructions and takes an interrupt during any Load Multiple or Store Multiple instruction including aliases, for example [PUSH \(multiple registers\)](#) and [POP \(multiple registers\)](#), and all the following apply:

- Is not in an IT block.
- The instruction is using [SP](#) as the [Base register](#).
- The instruction instruction performs write-back of [SP](#).

Then if the [SP](#) presented to the exception entry sequence is lower than any element pushed by an [STM](#), or not yet popped by an [LDM](#), the [SP](#) is set to the lowest value in the list and the instruction resumes following the exception return using the continuation state held in [EPSR.ICI](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{FDST} If the PE takes an interrupt during any Load Multiple or Store Multiple instruction which is not in an IT block, then if the instruction is not using [SP](#) as the [Base register](#) and write-back to the [Base register](#) is specified:

- For Decrement Before variants of the instruction, the [Base register](#) is set to the initial value this corresponds to the address in memory of the lowest register in the list.
- For Increment After variants of the instruction, the [Base register](#) is set to the final value this corresponds to the address in memory of the highest register in the list.
- In both cases, the PE uses instruction resume and [EPSR.ICI](#) holds the continuation state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{FYMK} [EPSR.ICI](#) is cleared to zero and the [Base register](#) is restored to its original value if write-back is being used if any of the following occur during the execution of an interrupt-continuable instruction:

- A synchronous exception.
- The PE takes an asynchronous fault.
- The PE takes an interrupt and the [interrupt-continuable instruction](#) is in an IT block.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{BRGS} If the PE takes an interrupt during a Load Multiple instruction which is not in an IT block, then if the instruction's base register in its register list, and the exception was taken after loading the base register:

- The base register is restored to the original value.
- The [EPSR.IT/ICI](#) fields can be set to a value that corresponds to an IMPLEMENTATION DEFINED register that after returning from the exception, load at least the base register and any subsequent locations.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

B3.27.6 Exception-continuable instruction behavior

I_QSSP	MVE divides vector operations between Beats. The architecture permits Beats from multiple instructions to be performed at the same time. An exception can be taken after any Beat of a vector operation. On an exception the value written to EPSR.ECI indicates which Beats of the vector operations in progress have been completed. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R_GFXK	The architecture supports taking exceptions in the middle of multiple partially executed exception-continuable instructions. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R_NTYR	For exceptions that occur in the middle of a beat-wise vector exception-continuable instruction that is executing: • The exception return address points to the oldest incomplete instruction. • RETPSR.ECI in the exception stack frame stores information about how many beats of the instruction at the return address, and how many beats of the subsequent instruction, have already been completely executed. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I_ZPTT	In EPSR, XPSR, and RETPSR, the ECI and ICI fields, and ITSTATE overlap. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I_GRFW	The architecture tracks the completion of beats within vector instructions. Because the Element size can be smaller than the beat size, it is possible that an exception might be generated for a beat that has only partially completed. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R_QDSP	If execution of a beat is abandoned, then: • RETPSR.ECI only indicates that a beat is completed if all the Elements that are associated with the beat have been completed. • If a destination vector register is not the same as a source vector register for an instruction that is abandoned because of an exception, the parts of a vector destination register that are associated with an abandoned beat, and all subsequent beats of the abandoned instruction, are set to an UNKNOWN value if the parts of the destination register would have been written to had the abandoned instruction been completed. It must not be possible for parts of a destination register to become UNKNOWN if they are not affected by the instruction, and would retain their previous values as if the instruction had not been abandoned. • Any scalar destination registers, the VPR state, and the FPSCR.QC flag record all the architecture state updates that are associated with the fully completed beats. Updates that are associated with the abandoned beat and all subsequent beats of the instruction are not recorded. Partial stores to locations that might be accessed by the abandoned beat and all subsequent beats might be observed. Loads to locations of the abandoned beats and all subsequent beats might be observed. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R_YWJX	The return address for the instruction fetch fault, for example a MemManage fault, is always the address of the instruction that triggered the fault. The fault is taken after all the preceding instructions have completed. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R_GRG7	If an exception return is performed to an UNDEFINED instruction encoding within an otherwise valid MVE beat-wise instruction, for example a VAND with the D bit set to one, and the ECI value being returned to is invalid, it is IMPLEMENTATION DEFINED whether an UNDEFINSTR or INVSTATE UsageFault is raised. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R_KDNH	Architecture state updates that are associated with an Architecture tick are observed as one of the following: • All updates to the architecture state are observable.

- Partial updates to the architecture states (both to the registers and to memory) are permitted for instructions that were terminated by an exception and can be restarted without data corruption. In other words, an instruction that is capable of instruction resume or restart and there is a valid [EPSR.ECI](#) value.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

I_{RQDN}

If an exception is taken during the execution of overlapping beat-wise executable instructions, this might become architecturally visible.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

I_{GRDV}

Vector load or store instructions, that might be abandoned and subsequently restart the execution of a beat, might cause multiple accesses to the same memory location to be performed.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

See also:

[Chapter B5 Vector Extension](#).

B3.27.7 Invalid ICI and ECI values

R_{PCQF}

If none of the following conditions are met, an attempt to execute an instruction causes the PE to generate an INVSTATE UsageFault if either [EPSR.ICI](#) or [EPSR.ECI](#) contain valid nonzero values:

- The instruction is a [BKPT](#) instruction or an FPB generated breakpoint.
- [EPSR.ICI](#) is a valid nonzero value and the instruction is an [interrupt-continuable instruction](#).
- MVE is implemented, [EPSR.ECI](#) is a valid nonzero value, and the instruction is beat-wise executable.
- MVE is implemented, [EPSR.ECI](#) is a valid nopnzero value, and the instruction is an [LETP](#) loop end instruction.

The INVSTATE UsageFault targets the current Security state.

The execution of the [BKPT](#) instruction, FPB generated breakpoint, or [LETP](#) loop end instruction does not advance any of the register fields that are used for interrupt-continuation or beat-wise execution tracking.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

See also:

[Chapter B5 Vector Extension](#).

B3.28 Low overhead loops

I _{JRNR}	The LOB Extension is a mandatory feature of the Armv8.1-M architecture. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.</i>
R _{BJZM}	If neither MVE nor the Floating-point extension are implemented LTPSIZE is a fixed integer value of four. The pseudocode function LTPSIZE () describes this. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB && !MVE && !FP.</i>
R _{BCPH}	If Floating-point extension is implemented LTPSIZE is a fixed value of four and the value of LTPSIZE is held in FPSCR.LTPSIZE. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB && FP && !MVE.</i>
R _{RXNJ}	If MVE is implemented LTPSIZE is not fixed and the value of LTPSIZE is held in FPSCR.LTPSIZE, which might not read as four when LOB tail predication is in progress. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB && MVE.</i>
R _{FTZN}	The Armv8.1-M architecture supports low overhead loops using: • WLS - While Loop Start. • DLS - Do Loop Start. • LE - Loop End. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.</i>
R _{WGGG}	An implementation that includes MVE has the following additional instructions that can be used in or in the creation of low overhead loops: • WLSTP - While Loop Start with Tail Predication. • DLSTP - Do Loop Start with Tail Predication. • LCTP - Loop Clear with Tail Predication. • LETP - Loop End with Tail Predication. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB && MVE.</i>
R _{FDVJ}	Instructions within the loop can read and write the loop iteration count, which is in the LR. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.</i>
R _{BZWN}	LR is updated when LOB handling causes the PC to return to the start of the loop. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.</i>
I _{GXDN}	The following is a trivial <code>memcpy</code> example which uses the T1 variant of the LE instruction. The T1 variant uses LR: <pre> 1 memcpy: 2 PUSH {R0, LR} 3 WLS LR, R2, loopEnd //R2=size 4 loopStart: 5 LDRSB R3, [R1], #1 // R1 = srcPtr, R3 = temp reg 6 STRB R3, [R0], #1 // R0 = destPtr 7 LE LR, loopStart 8 9 loopEnd: 10 POP {R0, PC} </pre>

The **WLS** and **LE** instructions cause the loop body to be executed *n* times, where *n* is specified by the value of R2. In this example, the low overhead loop instructions operate as follows:

- If the iteration count that is passed to the [WLS](#) instruction is nonzero, the loop iteration count is copied to [LR](#). If the iteration count is zero, the [WLS](#) instruction jumps to the end of the loop.
- If additional iterations of the loop are required when the [LE](#) instruction is executed (as indicated by the value in [LR](#)), the iteration count decrements [LR](#) and branches back to the start of the loop. The [LE](#) instruction also caches the loop branch information in [LO_BRANCH_INFO](#). Subsequent iterations might not be required to re-execute the [LE](#) instruction.
- If [LR](#) indicates that no further iterations are required, the PE branches over the [LE](#) instruction when execution reaches the last instruction in the body of the loop.

The [LE](#) T2 variant of the [LE](#) instruction does not include [LR](#) as an argument and can be used where the number of iterations is not known in advance.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#).

I_{CRKM}

The T3 variant of the [LE](#) instruction is [LETP](#), which is a tail predicated loop. Tail predicated loops can be used if the iteration count is not known in advance. A trivial `memcpy` example of the [LETP](#) instruction is shown here:

```

1      memcpy:
2          PUSH    {R0, LR}
3          WLSSTP.8    LR, R2, vectLoopEnd    //R2 = element / byte count
4
5      vectLoopStart:
6          VLDRB.8    Q0, [R1], #16    // R1 = srcPtr
7          VSTRB.8    Q0, [R0], #16    // R0 = destPtr
8          LETP      LR, vectLoopStart
9
10     vectLoopEnd:
11         POP      {R0, PC}

```

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#) && [MVE](#).

R_{FZXX}

The [LE](#), [LETP](#) instruction caches the loop branch information in [LO_BRANCH_INFO](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#).

R_{NMGH}

When executing a [LE](#) instruction, [LR](#) decrements by one on each iteration of the loop.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#).

R_{HSNB}

An `INVSTATE UsageFault` is raised if a [LE](#) instruction is executed and [FPSCR.LTPSIZE](#) does not read as four.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#).

R_{CCRF}

Executing an [LETP](#) instruction when [FPSCR.LTPSIZE](#) is between zero and four inclusive will not generate an `INVSTATE UsageFault`.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#) && [MVE](#).

R_{VRQV}

For low overhead loop instructions, [LR](#) stores the loop iteration count. For a tail predicated low overhead loop instruction, [LR](#) stores the number of vector elements to be processed.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#). Note, [MVE](#) required for tail predication.

R_{HNLF}

When executing a [LETP](#) instruction, [LR](#) decrements by the element width indicated in [FPSCR.LTPSIZE](#). When [FPSCR.LTPSIZE](#) is not set to four tail predication is applied according to the value in [LR](#). The number of elements is calculated by dividing the vector width, 128, by the element width in [FPSCR.LTPSIZE](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#) && [MVE](#).

R_{GMVB}

The following events update the low overhead loop flags, as indicated by [LO_BRANCH_INFO.VALID](#) bit.

Event	LO_BRANCH_INFO
Reset	Cleared
LE , LETP instruction	Conditionally set
BF , BFX , BFL , BFLX , BFCSEL instruction	Set
Context synchronization event	Cleared
BXNS or BLXNS instruction that cause a Security State transition	Cleared
SG instruction that causes a transition from Non-secure to Secure state	Cleared
Unstacking a FNC_RETURN stack frame	Cleared
SVC instruction	Cleared
Any instruction that modifies the PC when LO_BRANCH_INFO.BF is set	Cleared
IMPLEMENTATION DEFINED events	Cleared

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#). Note, some rows require *S*.

I_{GHRX}

Clearing the [LO_BRANCH_INFO.VALID](#) bit on an [SVC](#) instruction guarantees that the `ReturnAddress` is the instruction directly after the [SVC](#) instruction.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#).

I_{SWHC}

If the debugger expects predictable control flow, then Arm recommends that the implicit branches are disabled and that the associated cache is cleared.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#).

R_{GHQJ}

For implementations that include [MVE](#), the architecture permits the architecturally overlapping execution of a vector instruction at the end of the loop with an instruction at the start of the next iteration of the loop, except when:

- The vector instructions at the end of the loop write to [LR](#).
- The instruction at the start of the loop reads or writes to [LR](#).
- Data dependencies between vector instructions are violated.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#) && [MVE](#).

R_{XZJF}

When a new [Floating-point context](#) is created and [FPCCR.ASPEN](#) is set to one the PE automatically initializes [FPSCR.LTPSIZE](#) to four.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#) && ([MVE](#) || [FP](#)).

I_{JLJL}

When a new [Floating-point context](#) is created and [FPCCR.ASPEN](#) is set to zero it is the responsibility of software to correctly initialize [FPSCR.LTPSIZE](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#) && ([MVE](#) || [FP](#)).

I_{RRPL}

When a [WLSTP](#) or a [DLSTP](#) instruction is used to initialize a loop, the loop end instruction must be an [LETP](#) instruction. If an [LE](#) instruction is used in this scenario, the predication applied might be incorrect.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#) && [MVE](#).

I_{TPKG}

When a [WLS](#) or a [DLS](#) instruction is used to initialize a loop, the loop end instruction must be an [LE](#) instruction.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [LOB](#).

R_{RKZN}

The execution of an implicit or explicit [LE](#), [LETP](#) instruction is CONSTRAINED UNPREDICTABLE anywhere within an IT block. When the instruction is committed for execution, one of the following occurs:

- An UNDEFINED exception is taken.
- ITSTATE is cleared to 0.
- The instructions are executed as if they had passed the condition code check and the ITSTATE is advanced.
- The instructions execute as NOPs, as if they had failed the condition code check and the ITSTATE is not advanced.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - [LOB](#).*

See also:

[B5.5.1 Loop tail predication](#).

[WLS](#), [DLS](#), [WLSTP](#), [DLSTP](#).

[LE](#), [LETP](#).

[LTPSIZE](#).

B3.29 Branch future

I_{TBQH} The Armv8.1-M architecture supports branch future instructions (BF instructions). The BF instruction and its variants are requests to the PE to perform a branch in the future. The variants of the branch future instruction are BF, BFX, BFL, BFLX, and BFCSEL.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.

I_{VCGC} An example of a BF branch point is as follows:

```
1 start:
2     BFX b_label, LR          // Set up BF at b_label
3     ADD r0, r0, r1
4     ADD r0, r0, r2
5     ADD r0, r0, r3
6     // This is the BF branch point
7 b_label:
8     BX LR                   // Executed if LO_BRANCH_INFO invalid
```

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.

R_{SVXL} If the last instruction immediately before the BF branch point writes to LR, and a BFL or BFLX set up the BF branch point, then LR is set to the return address, and not to the value that is generated by the instruction at the BF branch point.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.

R_{FZVC} BF initializes the LO_BRANCH_INFO register to cause a low overhead branch just before execution reaches the specified label, that is the branch point.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.

R_{JTGY} When BF causes a branch, this branch occurs at the branch point. The instruction after the branch point is not executed if the branch is taken.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.

R_{JVJF} Inserting the BF branch point in the middle of a T32 instruction results in one of the following CONSTRAINED UNPREDICTABLE behaviors:

- It executes as a NOP.
- It raises an UNDEFINED instruction fault.
- It executes normally and the branch that is associated with the BF instruction is taken.
- It executes normally and the branch that is associated with the BF instruction is not taken.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.

R_{VGJR} If the BF branch point is in an IT block, and it does not immediately precede the last instruction in the IT block, then the following CONSTRAINED UNPREDICTABLE behaviors can result:

- The instruction executes normally and the branch that is associated with the BF instruction is not taken. The BF instruction can be treated as a NOP.
- The instruction before the BF branch point raises an UNDEFINED instruction fault.
- ITSTATE is cleared to 0.
- Taking the branch that is associated with the BF instruction causes ITSTATE to become UNKNOWN.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.

R_{HLVZ} If a BF branch point is within an IT block, the branch that was created by the BF instruction is not affected by the IT condition.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.

R_{JFHP} When executing in Handler mode, [BF](#) instructions that attempt to cause a branch to [EXC_RETURN](#) behave as NOPs.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [LOB](#).

R_{BGSF} In an implementation that includes the Security Extension, [BF](#) instructions that attempt to cause a branch to [FNC_RETURN](#) behave as NOPs.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [LOB](#) && [S](#).

R_{MCKJ} Taking a branch that is created by the BF instruction clears ITSTATE to 0.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [LOB](#).

See also:

[Chapter C1 Instruction Set Overview](#).

[B3.28 Low overhead loops](#).

[C1.3.5 ITSTATE](#).

[BF](#), [BFX](#), [BFL](#), [BFLX](#), and [BFCSEL](#).

Applies to an implementation of the architecture [Armv8.1-M](#) onward.

B3.30 Vector tables

R_{NWFF}

In a PE with the Security Extension, two vector tables are implemented, the Secure Vector table and the Non-secure Vector table, and it is IMPLEMENTATION DEFINED which of the following is true:

- The PE supports configurability of each vector table base, and two Vector Table Offset Registers, [VTOR_S](#) and [VTOR_NS](#), are provided for this purpose.
- The PE does not support configurability of either vector table base, and [VTOR_S](#) and [VTOR_NS](#) are WI.

If the PE supports configurability of each vector table base:

- Exceptions that target Secure state use [VTOR_S](#) to determine the base address of the Secure vector table.
- Exceptions that target Non-secure state use [VTOR_NS](#) to determine the base address of the Non-secure vector table.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{GTJQ}

In a PE without the Security Extension, a single vector table is implemented, and it is IMPLEMENTATION DEFINED which of the following is true:

- The PE supports configurability of the vector table base, and a single Vector Table Offset Register, [VTOR](#), is provided for this purpose.
- The PE does not support configurability of the vector table base, and [VTOR](#) is WI.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [!S](#).

I_{WFGX}

Arm recommends that [VTOR_S](#) points to memory that is Secure and not Non-secure callable.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{WPRT}

A vector table contains both:

- The initialization value for the main stack pointer on reset.
- The start address of each exception handler.

The *exception number* defines the order of entries.

Word offset in vector table	Value that is held at offset
0	Initial value for the main stack pointer on reset.
1	Start address for the reset handler.
Exception number	Start address for the handler for the exception with that number
.	.
.	.
.	.
Exception number	Start address for the handler for the exception with that number

Applies to an implementation of the architecture Armv8.0-M onward.

R_{LFDL}

In a PE with a configurable vector table base, the vector table is naturally-aligned to a power of two, with an alignment value that is:

- A minimum of 128 bytes.
- Greater than or equal to (Number of Exceptions supported x4).

Applies to an implementation of the architecture Armv8.0-M onward.

R _{VDPD}	Vector fetches for entries beyond the natural alignment of the associated VTOR occur from an UNKNOWN entry within the vector table. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{P LSB}	Arm recommends that it is ensured that the vector table and VTOR are aligned so that the entry for the highest taken exception falls within the natural alignment of the table, and at a minimum that the vector table is 128 byte aligned. A PE might impose further restrictions on the VTOR. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{ZVWS}	If a vector fetch causes a Security attribution unit violation or an implementation defined attribution unit violation or a BusFault, a secure VECTTBL HardFault is raised. If the exception priority prevents any secure VECTTBL HardFault preempting, one of the following occurs: • The PE enters Lockup at the priority of the original exception. • The original exception transitions from the pending to the active state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S. Note, SAU and IDAU require M.</i>
R _{XPPT}	For all vector table entries other than the entry at offset 0, if bit[0] is not set to 1, the first instruction in the exception results in an INVSTATE UsageFault or a HardFault. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{BVSC}	For all vector table entries other than the entry at offset 0, bit[0] defines EPSR.T on exception entry. Setting bit[0] to 1 indicates that the exception handler is in the T32 instruction set state. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{WRJW}	A vector fetch may be performed using the instruction interface, and avoid DWT matches and watchpoints being generated. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[B10.4 IMPLEMENTATION DEFINED Attribution Unit \(IDAU\).](#)

[B10.3 Security attribution unit \(SAU\).](#)

[B3.9 Exception numbers and exception priority numbers.](#)

[B3.5.2 Execution Program Status Register \(EPSR\).](#)

B3.31 Hardware-controlled priority escalation to HardFault

R_{GNVS} If a synchronous exception with an equal or lower priority to execution is pending, the PE hardware escalates it to become a HardFault. This rule applies to all synchronous exceptions and DebugMonitor exceptions that are caused by the [BKPT](#) instruction. This rule does not apply to asynchronous exceptions and all other DebugMonitor exceptions. If the HardFault cannot be taken the PE enters [Lockup](#).

Applies to an implementation of the architecture Armv8.0-M onward. Note, DebugMonitor exception requires M.

R_{HPLM} [FPCCR](#).*RDY (not the current execution priority) determines the escalation of synchronous exceptions generated because of lazy floating-point state preservation. This means that an asynchronous exception might be pended.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{TKCW} When the execution priority is less than HardFault and a disabled configurable-priority exception occurs, the PE escalates the configurable-priority fault to HardFault.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{DQRR} A fault that has been escalated to a HardFault, and not pended, retains the return address behavior of the original fault and sets [HFSR.FORCED](#) to 1.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{NHFZ} When [AIRCR.IESB](#) is one, a detected asynchronous BusFault will be handled synchronously.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

See also:

[B3.9 Exception numbers and exception priority numbers.](#)

[DebugMonitor exception.](#)

[B3.33 Lockup.](#)

[B3.11 Security states, exception banking.](#)

[Chapter B16 Reliability, Availability, and Serviceability \(RAS\) Extension](#)

[RaiseAsyncBusFault\(\)](#)

B3.32 Special-purpose mask registers, PRIMASK, BASEPRI, FAULTMASK, for configurable priority boosting

I_BNJJG

In a PE with the Main Extension, the [PRIMASK](#), [FAULTMASK](#), and [BASEPRI](#) registers can be used as follows. A PE without the Main Extension implements [PRIMASK](#), but does not implement [FAULTMASK](#) and [BASEPRI](#).

PRIMASK

In a PE without the Security Extension:

- Setting this bit to one boosts the current execution priority to 0, masking all exceptions with an equal or lower priority.

In a PE with the Security Extension:

- Setting [PRIMASK_S](#) to one boosts the current execution priority to 0.
- If [AIRCR.PRIS](#) is:

0:

Setting [PRIMASK_NS](#) to one boosts the current execution priority to 0.

1:

Setting [PRIMASK_NS](#) to one boosts the current execution priority to 0x80.

In a PE with the Security Extension, when the current execution priority is boosted to a particular value, all exceptions with an equal or lower priority are masked.

FAULTMASK

In a PE without the Security Extension:

- Setting this bit to one boosts the current execution priority to -1, masking all exceptions with an equal or lower priority.

In a PE with the Security Extension, if [AIRCR.BFHFNMINS](#) is:

0:

Setting [FAULTMASK_S](#) to one boosts the current execution priority to -1.

If [AIRCR.PRIS](#) is:

0: Setting [FAULTMASK_NS](#) to one boosts the current execution priority to 0.

1: Setting [FAULTMASK_NS](#) to one boosts the current execution priority to 0x80.

1:

Setting [FAULTMASK_S](#) to one boosts the current execution priority to -3.

Setting [FAULTMASK_NS](#) to one boosts the current execution priority to -1.

In a PE with the Security Extension, when the current execution priority is boosted to a particular value, all exceptions with an equal or lower priority are masked.

BASEPRI

In a PE without the Security Extension:

- This field can be set to a priority number between 1 and the maximum supported priority number. This boosts the current execution priority to that number, masking all exceptions with an equal or lower priority.

In a PE with the Security Extension:

- [BASEPRI_S](#) can be set to a priority number between 1 and the maximum supported priority number.

- If [AIRCR.PRIS](#) is:

- 0: [BASEPRI_NS](#) can be set to a priority number between 1 and the maximum supported priority number.
- 1: [BASEPRI_NS](#) can be set to a priority number between 1 and the maximum supported priority number. The value in [BASEPRI_NS](#) is then mapped to the bottom half of the priority range, so that the current execution priority is boosted to the mapped-to value in the bottom half of the priority range, that is from 0×80 to the supported maximum.

In a PE with the Security Extension, when the current execution priority is boosted to a particular value, all exceptions with an equal or lower priority are masked.

Applies to an implementation of the architecture Armv8.0-M onward. Note, [FAULTMASK](#) and [BASEPRI](#) require M.

R_FHMC

The [PRIMASK](#), [FAULTMASK](#), and [BASEPRI](#) priority boosting mechanisms only boost the group priority, not the subpriority.

Applies to an implementation of the architecture Armv8.0-M onward. Note, [FAULTMASK](#) and [BASEPRI](#) require M.

R_SKB_J

Without the Security Extension:

- An exception return sets [FAULTMASK](#) to 0.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - !S && M.

R_HRTM

With the Security Extension:

- An exception return sets [FAULTMASK](#) to 0 if the *raw execution priority* is greater than or equal to 0. [EXC_RETURN.ES](#) indicates which banked instance of [FAULTMASK](#) is set to 0.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S && M.

I_LS_X_J

The *raw execution priority* is:

- The execution priority minus the effects of any configurable [PRIMASK](#), [FAULTMASK](#), or [BASEPRI](#) priority boosting.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.

I_GBV_L

The *requested execution priority* for a Security state is negative when any of the following are true:

- The banked [FAULTMASK](#) bit is 1, including when [AIRCR.PRIS](#) is also 1.
- A HardFault is active.
- An NMI is active and targets the Security state for which the requested execution priority is being calculated .

Applies to an implementation of the architecture Armv8.0-M onward. Note, [FAULTMASK](#) requires M.

R_QZW_N

The [PRIMASK](#) and [FAULTMASK](#) values can be set by the execution of a [CPS](#) instruction. The effects of the instruction are guaranteed to be visible when either of the following conditions apply:

- If the [CPSID](#) variant is executed, the PE serializes the change so that it is visible to instructions appearing in program order after completion of [CPSID](#) instruction.
- If the [CPSIE](#) variant is executed, the effects of the instruction are only guaranteed to be visible after a [Context synchronization event](#).

Applies to an implementation of the architecture Armv8.0-M onward. Note, [FAULTMASK](#) requires M.

See also:

[B3.13 Priority model](#).

[B3.9 Exception numbers and exception priority numbers](#).

B3.33 Lockup

I _{RKJB}	Lockup is a PE state where the PE stops executing instructions in response to an error for which escalation to an appropriate HardFault handler is not possible because of the current execution priority. An example is a synchronous exception that escalates to a Secure HardFault, but cannot escalate to a Secure HardFault because a Secure HardFault is already active. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{FSFR}	Arm recommends that an implementation provides a LOCKUP signal that, when the PE is in lockup, signals to the external system that the PE is in lockup. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{MBTM}	When the PE is in lockup: • DHCSR.S_LOCKUP reads as 1. • The PC reads as 0xEFFFFFFE. This is an XN address. • The PE stops fetching and executing instructions. • If the implementation provides an external LOCKUP signal, LOCKUP is asserted HIGH. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{DWKP}	Asynchronous BusFaults do not cause lockup. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{KTQM}	When a BusFault does not cause lockup, the value that is read or written to the location that generated the BusFault is UNKNOWN. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HTVD}	ITSTATE does not advance when the PE is in lockup. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XQSR}	Exit from lockup is only by one of the following: • A Cold reset. • A Warm reset. • Entry to Debug state. • Preemption by an NMI exception. <i>Applies to an implementation of the architecture Armv8.0-M onward. Note, entry to Debug state requires HDBG.</i>
I _{XKJB}	Arm recommends that entry into Lockup is treated as fatal and requires the PE to be reset. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HJNP}	Exit from lockup causes both DHCSR.S_LOCKUP and, if implemented, the external LOCKUP signal, to be deasserted. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SPPN}	On an exit from lockup by entry to Debug state, or by preemption by another exception, the return address is 0xEFFFFFFE. <i>Applies to an implementation of the architecture Armv8.0-M onward. Note, entry to Debug state requires HDBG.</i>
I _{CRHJ}	After exit from lockup by entry to Debug state, or by preemption by another exception, a subsequent return from Debug state or that exception without modifying the return address attempts to execute from 0xEFFFFFFE. Execution from this address is guaranteed to generate an IACCVIOL MemManage fault, causing the PE to reenter lockup if the execution priority has not been modified. Modification of the return address would enable execution to be resumed, however Arm recommends treating entry to lockup as fatal and requiring the PE to be reset.

Applies to an implementation of the architecture *Armv8.0-M* onward. Note, entry to Debug state requires HDBG.

See also:

[B3.13 Priority model](#).

[Chapter B13 Debug](#).

B3.33.1 Lockup behavior

Instruction execution

R_{VGMR}

A synchronous exception taken during instruction execution, excluding faults caused by lazy floating-point state preservation, results in **lockup** when:

- The synchronous exception would otherwise escalate to a Secure HardFault and any of the following is true:
 - Secure HardFault is already active.
 - NMI is active and [AIRC.R.BFHFNMIN](#) is 0.
 - [FAULTMASK.S.FM](#) is 1.
 - Non-secure HardFault is active and [AIRC.R.BFHFNMIN](#) is 0.
- The synchronous exception would otherwise escalate to a Non-secure HardFault and any of the following is true:
 - Non-secure HardFault or Secure HardFault is already active.
 - NMI is active.
 - [FAULTMASK.NS.FM](#) or [FAULTMASK.S.FM](#) is 1.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [S](#). Note, [FAULTMASK](#) requires *M*.

R_{RNKB}

When [FPCCR.LSPACT](#) is 1, a NOCP UsageFault, [AU](#) violation, [MPU](#) violation, or synchronous BusFault during lazy Floating-point state preservation causes **lockup** if all the following are true:

- [FPCCR.HFRDY](#) is 0.
- The [FPCCR.*RDY](#) bit associated with the original exception is 0.
- The current execution priority is high enough to prevent preemption.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [FP](#). Note, an MPU violation requires MPU, an SAU violation requires *S*.

R_{XHMT}

Entry to **lockup** during instruction execution, including lazy floating-point state preservation, causes:

- Any Fault Status Registers associated with the exception to be updated.
- No update to the pending exception state or to the active exception state.
- The [PC](#) to be set to 0xFFFFFFFF.
- [EPSR.IT](#) is permitted to become UNKNOWN. Arm recommends that entry to **lockup** does not modify the [EPSR.IT](#) bits.

In addition, [HFSR.FORCED](#) is not changed.

Applies to an implementation of the architecture *Armv8.0-M* onward.

B3.33.2 Exception-related lockup behavior

Vector or stack pointer error on reset

R_{BHVG}

On reset, if reading the vector table to obtain either the vector for the reset handler or the initialization value for the main stack pointer causes a BusFault, the PE enters **lockup** in HardFault with the following behavior:

- **HFSR.VECTTBL** is set to 1.
- In a PE with the Security Extension, Secure HardFault is made active. That is, **SHCSR.S.HARDFULTACT** is set to 1.
- In a PE without the Security Extension, HardFault is made active. That is, **SHCSR.HARDFULTACT** is set to 1.
- An UNKNOWN value is loaded into the main stack pointer.
- The **IPSR** is set to 0.
- **EPSR.T** is UNKNOWN.
- **EPSR.IT** is set to zero.
- The **PC** is set to 0xEFFFFFFE.

Applies to an implementation of the architecture Armv8.0-M onward. Note, a Secure HardFault requires S.

Errors on preemption and stacking for exception entry

R_{VKTX}

An **AU** violation, **MPU** violation, NOCP UsageFault, STKOF UsageFault, LSERR SecureFault, or synchronous BusFault during context stacking causes **lockup** when:

- The exception would escalate to a Secure HardFault if any of the following is true:
 - Secure HardFault is already active.
 - NMI is active and **AIRCR.BFHFNMINs** is 0.
 - **FAULTMASK.S.FM** is 1.
 - Non-secure HardFault is active and **AIRCR.BFHFNMINs** is 0.
- The exception would escalate to a Non-secure HardFault if any of the following is true:
 - Non-secure HardFault or Secure HardFault is already active.
 - NMI is active.
 - **FAULTMASK.NS.FM** or **FAULTMASK.S.FM** is 1.

In these cases, the point of PE **lockup** is when, after the exception to be taken has been chosen, the handler for that exception is entered. These cases do not in themselves cause any additional exception to become pending.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S. Note, an AU violation requires S, an MPU violation requires MPU, a UsageFault requires M, a SecureFault requires S.

R_{QSSB}

When an **AU** violation, **MPU** violation, NOCP UsageFault, STKOF UsageFault, LSERR SecureFault, or synchronous BusFault occurs during context stacking, it is IMPLEMENTATION DEFINED whether the PE continues to stack any of the remaining context.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S. Note, an AU violation requires S, an MPU violation requires MPU, a UsageFault requires M, a SecureFault requires S, LSERR requires FP.

R_{GJJG}

At the point of encountering an **AU** violation, **MPU** violation, NOCP UsageFault, STKOF UsageFault, LSERR SecureFault, or synchronous bus error during context stacking, the PE:

- Updates any Fault Status Registers associated with the error.
- Does not change **HFSR.FORCED**.

At the point of **lockup**:

- All state, including the **LR**, **IPSR**, and active and pending bits, is modified as though the fault on context stacking had never occurred, other than the following:
 - **EPSR.T** becomes UNKNOWN.
 - **EPSR.IT** is set to zero.
 - The **PC** is set to 0xEFFFFFFE.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S**. Note, an AU violation requires S, an MPU violation requires MPU, a UsageFault requires M, a SecureFault requires S, LSERR requires FP.

Vector read error on NMI or HardFault entry

R_{CTKP}

On entry to an NMI or HardFault, if reading the vector table to obtain the vector for the NMI or HardFault handler causes a bus error, the PE enters **lockup** with the following behavior:

- **HFSR.VECTTBL** is set to 1.
- The **IPSR** is updated to hold the exception number of the exception taken.
- The active bit of the exception that is taken is set to 1.
- The pending bit of the exception that is taken is cleared to 0.
- **EPSR.T** is UNKNOWN.
- **EPSR.IT** is set to zero.
- The **LR** is set to the **EXC_RETURN** value that would have been used had the fault not occurred.
- The **PC** is set to 0xEFFFFFFE.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{NMRW}

Because **AU** violations on vector reads are required to be treated as late-arriving, they cannot cause **lockup**, and instead result in a higher priority exception being taken. Vector reads always use the system address map and cannot generate **MPU** violations.

Applies to an implementation of the architecture *Armv8.0-M* onward.

Lockup on exception return

R_{NZCD}

A fault that is generated by any failed *integrity check on exception return* is generated after either the active bit for the returning exception, or the active bit for NMI or HardFault, has been cleared to 0, and if applicable, after **FAULTMASK** has also been cleared to 0.

Context unstacking is performed after any clearing of exception active bits or **FAULTMASK**, that is required by the exception return, has been made visible.

A fault that is generated by a failed integrity check on exception return or a synchronous exception during context unstacking causes **lockup** when:

- The exception would escalate to a Secure HardFault and any of the following is true:
 - Secure HardFault is already active.
 - NMI is active and **AIRCR.BFHFNMINs** is 0.
 - **FAULTMASK_S.FM** is 1.
 - Non-secure HardFault is active and **AIRCR.BFHFNMINs** is 0.
- The exception would escalate to a Non-secure HardFault and any of the following is true:
 - Non-secure HardFault or Secure HardFault is already active.
 - NMI is active.
 - **FAULTMASK_NS.FM** or **FAULTMASK_S.FM** is 1.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S**.

R_{TCJR}

When the PE enters **lockup** because of a fault on exception return that is generated by a failed integrity check or by context unstacking, the PE:

- Updates any Fault Status Registers associated with the error.
- Sets **IPSR** to 0, if **EXC_RETURN** for the returning exception indicated a return to Thread mode.
- Sets **IPSR** to 3, if **EXC_RETURN** for the returning exception indicated a return to Handler mode.

- Sets the stack pointer that is used for unstacking to the value it would have had if the fault had not occurred.
 - If the [RETPSR](#) load faults, the SP is 64-bit aligned.
- Updates [CONTROL.FPCA](#), based on [EXC_RETURN.FType](#).
- [CONTROL.SFPA](#) becomes UNKNOWN.
- Sets the [PC](#) to 0xEFFFFFFE.
- The following registers become UNKNOWN:
 - [APSR](#).
 - [EPSR](#).
 - [FPSCR](#).
 - [R0-R12](#).
 - [LR](#)
 - [S0-S31](#).
 - [VPR](#)

*Applies to an implementation of the architecture **Armv8.0-M** onward. Note, [CONTROL.FPCA](#) and [SFPA](#), [FPSCR](#) and [S0-S31](#) require FP || MVE.*

See also:

[B3.22 Exception return](#).

B3.34 Data independent timing

I _{JFVR}	The Armv8.1-M architecture supports Data independent timing operations. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DIT.</i>
R _{DNPM}	DIT behavior only applies if the instruction passes its Condition code check. The instruction remains subject to the rules of the architecture but is permitted to have a different execution time when compared to the same instruction that had passed the Condition code check. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DIT.</i>
R _{NXXV}	When AIRC.R.DIT is set to 1, unless otherwise specified, the time required for Data independent timing operations is independent of all values that are accessed by operations from the following registers: • FPCSR.{N, Z, C, V}. • APSR. • General-purpose registers. • Floating-point Extension registers (S0-S31, D0-D15, and Q0-Q7). • In a limited number of cases, VPR.P0. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DIT. Note, Floating-point registers require FP or MVE, and VPR requires MVE.</i>
R _{CWVH}	When AIRC.R.DIT is set to 1, this affects the following features: • Exception handling. In addition to the standard set of registers, the following operations also exhibit Data independent timing for accesses to VPR.P0: – Exception entry. – Tail-chaining. – Lazy floating-point state preservation. – Exception return. • EPCSR.ICI. Whether a PE uses ICI for load/store multiple instructions is not dependent on the data values that are loaded or saved. This excludes the address that is being targeted. • Beat wise execution. Whether a Data independent timing vector instruction overlaps with another vector instruction is not dependent on the data values being processed by the data independent timing vector instruction. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DIT. Note, VPR and vector instructions requires MVE.</i>
I _{FNSB}	For more information on instructions that are required to comply with Data independent timing see the individual instruction descriptions. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DIT.</i>
R _{KBBC}	The execution of an instruction must be timing invariant with respect to values in registers which are not accessed by the instruction directly or indirectly. For example, the timing of <code>MOV R1, R0</code> must not be affected by the values held in R8 or R9. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DIT.</i>
R _{RWJW}	For non-architected accesses, all instructions, including instructions that are not listed as Data independent timing instructions, exhibit Data independent timing with respect to data that is held in specified DIT registers that are not architecturally accessed by the instruction. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DIT.</i>

See also:

[B5.4 Beats.](#)

[Chapter C2, Instruction Specification](#)

B3.35 Context Synchronization Event

R_{QXWD}

The architecture requires a Context synchronization event to guarantee visibility of any change to any memory-mapped register described in the architecture. Following a [Context synchronization event](#) a completed write to a memory-mapped register is visible to an indirect read by an instruction appearing in program order after the context synchronization event.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DQHW}

A context synchronization event is one of the following:

- Performing an ISB operation. An ISB operation is performed when an [ISB](#) instruction is executed and does not fail its condition code check.
- Taking an exception, including [tail-chaining](#).
- Returning from an exception.
- Entry into Debug state.
- Exit from Debug state.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{QNDW}

Security state transitions through interstating branches and returns are not Context synchronization events.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{FJBW}

Between any change to special purpose registers which require synchronization or memory-mapped registers and a subsequent [Context synchronization event](#), it is IMPLEMENTATION DEFINED whether an indirect read of the register by the PE uses the old or new values, and may vary with each use of the unsynchronized value.

Where multiple changes are made to to special purpose registers which require synchronization or memory-mapped registers before a [Context synchronization event](#), each value might independently be the old or new value.

Applies to an implementation of the architecture Armv8.0-M onward.

B3.36 Coprocessor support

R _{BSLX}	Coprocessor support is OPTIONAL. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
I _{JBMG}	When coprocessors are not supported, the fields in CPACR , NSACR , and CPPWR that are associated with the unsupported coprocessor are RAZ/WI. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{XSQH}	The architecture supports 0-16 coprocessors, CP0 to CP15. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{HJDH}	If not used by CDE, CP0 to CP7 are IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{XPRQ}	It is IMPLEMENTATION DEFINED whether CP0 to CP7 can be used from both Secure and Non-secure states or whether the coprocessor is enabled for only Secure or Non-secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M. Note, Secure state requires S.</i>
I _{QSRC}	Arm reserves CP8 to CP15. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{LKZM}	CP10 to CP11 are reserved to support the Floating-point Extension, and CP10 controls the CP11 Floating-point instructions. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{CSQD}	From version 8.1-M of the architecture, access control for CP10 also controls CP8, CP9, CP14, and CP15. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>
R _{LPMK}	The state that is associated with Floating-point unit described in CPPWR.SU10 applies to S registers, D registers, and FPSCR. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP MVE.</i>
R _{VLNJ}	From version 8.1-M of the architecture, the state that is associated with the Floating-point unit described in CPPWR.SU10 also applies to the Q registers and VPR . <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R _{XXDG}	Instructions that are issued to unimplemented or disabled coprocessors result in a NOCP UsageFault. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{RMLV}	If a coprocessor cannot complete an instruction, an UNDEFINSTR UsageFault is generated. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>

See also:

[Chapter B4 Floating-point Support.](#)

[CPACR, Coprocessor Access Control Register](#)

[CPPWR, Coprocessor Power Control Register](#)

B3.37 The Custom Datapath Extension

B3.37.1 Overview of the Custom Datapath Extension

I_{QNBG}

CDE introduces three classes of two instructions in the co-processor instruction space:

- Three classes operate on the general-purpose register file, including the condition code flags `APSR_nzcv`.
- Three classes operate on the Floating-point or SIMD register file only.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **CDE**. Note, (FP || MVE) required for Floating-point register file. MVE is only available in an Armv8.1-M implementation.*

I_{JWBF}

A Custom Datapath instruction operating on the Floating-point or SIMD register files uses one of:

- 32-bit **S** registers.
- 64-bit **D** registers.
- 128-bit **Q** registers.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **CDE** && (FP || MVE). Note, Q registers require MVE, MVE is only available in an Armv8.1-M implementation.*

I_{RBLJ}

The three classes of CDE instructions are defined by the following patterns:

- <operation code> <destination register>.
- <operation code> <destination register>, <source register>.
- <operation code> <destination register>, <source register 1>, <source register 2>.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **CDE**.*

I_{FZRZ}

The destination register of a Custom Datapath instruction might be optionally read, as well as written.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **CDE**.*

I_{BXBG}

The operation code can be split between a true operation code in the custom datapath and an immediate value used in the custom datapath. The architecture does not prescribe any split.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **CDE**.*

I_{NFMB}

Immediate consequences of the above are:

- No operations on the Floating-point or SIMD registers can set condition codes.
- There are no instructions that support the use of all of, or any combination of the following:
 - **S** registers.
 - **D** registers.
 - **Q** registers.
 - The general-purpose register file.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **CDE**. Note, Q registers require MVE, MVE is only available in an Armv8.1-M implementation.*

I_{TVRP}

Operations on the general-purpose register file operate on 32-bit registers, or a dual-register consisting of a 64-bit value constructed from an even numbered general-purpose register and its immediately following odd numbered pair.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **CDE**.*

B3.37.2 Enabling CDE instructions

I _{CXBC}	Custom Datapath instructions can be found within, and are associated with, the existing coprocessor encoding and numbering spaces. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>
R _{XLTS}	Custom Datapath instructions fall into encoding spaces associated with a <i>coprocessor</i> number in the range 0 to 7 inclusive. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>
I _{WWNQ}	Enabling the coprocessor space in which the Custom Datapath Extension is implemented is the same as other IMPLEMENTATION DEFINED coprocessors. The function <code>IsCPEEnabled()</code> describes this. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE. Note, S required for Secure state.</i>
R _{VSVW}	If a coprocessor is associated with the Custom Datapath Extension, that coprocessor cannot execute the following instructions: • CDP, CDP2. • LDC, LDC2 (immediate). • LDC, LDC2 (literal). • MCR, MCR2. • MCRR, MCRR2. • MRC, MRC2. • MRRC, MRRC2. • STC, STC2. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>
R _{SKRX}	Execution of a Custom Datapath instruction that accesses the Floating-point or SIMD register file causes <i>Lazy Floating-point stacking</i> as specified by the architecture. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE && (FP MVE).</i>
R _{LQTN}	When executing a CDE instruction the PE checks that the coprocessor associated with CDE is enabled. If access to another coprocessor is required, for example the Floating-point Extension or MVE, a second coprocessor check is carried out. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE && (FP MVE).</i>
R _{GNRV}	The order in which the PE carries out the <code>ExecuteFPCheck()</code> function and <code>Coproc_Accepted()</code> function for execution, that is after decoding the instruction, of CDE instructions accessing the S, D or Q register files is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE && (FP MVE).</i>
I _{ZCCK}	If the execution of a CDE instruction requires access to the Floating-point or MVE register file the Floating-point Extension or MVE must be enabled using CPACR or NSACR dependent on Security state. Before the execution of a CDE instruction that requires access to the Floating-point Extension or MVE register file, the following registers are checked to ensure that CP10 is enabled: • CPACR. • NSACR. • CPPWR. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE && (FP MVE).</i>
I _{MXW}	Armv8-M double-precision Floating-point Extension implements 16 “D” registers, D0 to D15. The instructions defined by the Custom Datapath Extension are capable of indexing registers D0 to D31. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE && (FP MVE).</i>

I _{LPY}	Armv8.1-M MVE implements eight “Q” registers, Q0 to Q7. The instructions defined by the Custom Datapath Extension are capable of indexing registers Q0 to Q15. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - CDE && MVE.</i>
R _{LBNN}	Execution of a Custom Datapath instruction that attempts to access an unimplemented Floating-point or SIMD register, is CONSTRAINED UNPREDICTABLE and either of the following behaviors can occur: • The instruction is UNDEFINED. • The instruction is treated as a NOP. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE && (FP MVE).</i> See also: B3.36 Coprocessor support CPACR, Coprocessor Access Control Register CPPWR, Coprocessor Power Control Register NSACR, Non-secure Access Control Register

B3.37.3 Execution of CDE instructions

R _{QGNK}	The source and destination registers for any Custom Datapath instruction are restricted to those that are specified by the instruction pseudocode. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>
R _{GPLC}	The operation of a Custom Datapath instruction cannot be stateful, and cannot operate directly on memory. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>
R _{XFCV}	It is IMPLEMENTATION DEFINED which Custom Datapath instructions are implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>
R _{SVBN}	An unimplemented Custom Datapath instruction whose associated coprocessor is not disabled is UNDEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>
R _{RTQN}	The execution of an unimplemented immediate value, accumulator, non-accumulator, single or double precision variant in the encoding of a Custom Datapath instruction is CONSTRAINED UNPREDICTABLE and either of the following behaviors can occur: • The instruction is UNDEFINED. • The instruction is treated as a NOP. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>
R _{SGPM}	Which coprocessors adhere to the Custom Datapath Extension or the Arm architecture coprocessor instruction set is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>
I _{LCVM}	Arm strongly recommends that CDE instructions must conform with data independent timing. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - CDE && DIT.</i>
I _{HZCW}	Arm recommends that the execution of a CDE instruction behaves the same no matter the registers or variants used. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - CDE.</i>

R_{RKQV}

If the Performance Monitors Extension is implemented CDE instructions exhibit the following behavior:

- Events that arise as a result of CDE instructions, including [INST_RETIRE](#), [INST_SPEC](#), [OP_RETIRE](#) and [OP_SPEC](#), are counted as they would be in relation to any other instruction.
- IMPLEMENTATION DEFINED events can be configured to count CDE related operations.
- CDE instructions do not directly access memory. PMU events which are counted as a result of memory accesses are not counted on the execution of a CDE instruction.
- CDE instructions do not form a class of instructions which are counted by the defined PMU events. For example CDE instructions will not cause [BR_RETIRE](#), [LD_RETIRE](#), or [MVE_INST_RETIRE](#) to be counted.
- Events which are unrelated to direct operations, for example the [L1I_CACHE_REFILL](#), will count if the cache is refilled no matter the cause of the cache refill.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [CDE](#) && [PMU](#).

R_{VPLL}

When executing a CDE scalar dual instruction the CDE enabled coprocessor must process general-purpose register pairs according to the PE's current endianness.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [CDE](#).

I_{RRTS}

All of the rules required for the M-Profile Vector Extension and the Low Overhead Loop and Branch Future Extension apply to all CDE beat-wise compatible instructions.

This includes the following, but is not limited to:

- Exception-continuable behavior.
- Overlapping of beat-wise instructions.
- VPT predication.
- Tail predicated low overhead loops.

The CDE instructions are as follows:

- [VCX1 \(vector\)](#).
- [VCX2 \(vector\)](#).
- [VCX3 \(vector\)](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [CDE](#) && [MVE](#) && [LOB](#).

See also:

[Chapter C1 Instruction Specification](#).

[Chapter B15 The Performance Monitors Extension](#).

Chapter B4

Floating-point Support

This chapter specifies the Armv8-M Floating-point support rules. It contains the following sections:

- B4.1 The optional Floating-point Extension, Fpv5.*
- B4.2 About the Floating-point Status and Control Registers.*
- B4.3 Registers for Floating-point data processing, S0-S31, or D0-D15.*
- B4.4 Floating-point standards and terminology.*
- B4.5 Floating-point data representable.*
- B4.6 Floating-point encoding formats, half-precision, single-precision, and double-precision.*
- B4.7 The IEEE 754 Floating-point exceptions.*
- B4.8 The Flush-to-zero mode.*
- B4.9 The Default NaN mode, and NaN handling.*
- B4.10 The Default NaN.*
- B4.11 Combinations of Floating-point exceptions.*
- B4.12 Priority of Floating-point exceptions relative to other Floating-point exceptions.*

B4.1 The optional Floating-point Extension, FPUv5

I_{VBNH} The optional Floating-point Extension defines a *Floating Point Unit* (FPU). Coprocessors 10 and 11 support the Extension.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

I_{WPHK} The scalar Floating-point Extension can be implemented with or without MVE-F.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP.

I_{RXQX} Floating-point is sometimes abbreviated to FP.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

R_{GQBM} The version of Floating-point Extension that is supported is FPUv5.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

I_{FGSG} FPUv5 provides all of the following:

- Single-precision arithmetic operations.
- Optional double-precision arithmetic operations.
- Conversions between integer, double-precision, single-precision, and half-precision formats.
- Registers for Floating-point processing S0-S31, or D0-D15.
- Data transfers, between Arm general-purpose registers and FPUv5 Extension registers S0-S31, or D0-D15, of single-precision and double-precision values.
- A **Flush-to-zero mode** that software can enable or disable.
- An optional *alternative half-precision* interpretation of the IEEE 754 half-precision encoding format.

FPUv5 adds the following System registers:

- The **FPSCR**, to the CP10 and CP11 System register space.
- The **FPCAR**, **FPCCR**, **FPDSCR**, **MVFR0**, **MVFR1**, and **MVFR2**, to the *System Control Block* (SCB).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

I_{TVZF} From Armv8.1-M onwards, FPUv5 provides Half-precision arithmetic operations.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP.

I_{PVBQ} When the Floating-point Extension is implemented, some software tools might require the following information:

Extension	Single-precision arithmetic operations only	Single and double-precision arithmetic operations
FPUv5	FPUv5-SP-D16-M	FPUv5-D16-M

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

I_{FTDS} When the Floating-point Extension is implemented, software can interrogate **MVFR0**, **MVFR1**, and **MVFR2** to discover the Floating-point features that are implemented.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

I_{JDJQ} To use the Floating-point Extension, software must enable access to CP10, by writing to **CPACR.CP10**.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

R_{PDMV} The value of **CPACR.CP11** is UNKNOWN if it is not programmed to the same value as **CPACR.CP10**.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

See also:

[B8.1 System address map.](#)

[B4.2 About the Floating-point Status and Control Registers.](#)

[B4.3 Registers for Floating-point data processing, S0-S31, or D0-D15.](#)

[B4.8 The Flush-to-zero mode.](#)

[B4.9 The Default NaN mode, and NaN handling.](#)

[B4.6 Floating-point encoding formats, half-precision, single-precision, and double-precision.](#)

B4.2 About the Floating-point Status and Control Registers

I_{FQTM} For implementations of the Armv8.1-M architecture, [FPCXT](#) and [VPR](#) provide additional controls.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [FP](#).

R_{HCJS} The register map of the coprocessor System register space is as follows.

Location	Register	Information
0b0001	FPSCR.{N, Z, C, V}	Access to flags

All locations that are not explicitly listed in this table are reserved, and accesses to these locations result in UNPREDICTABLE behavior.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [FP](#).

R_{KHDZ} The register map of the coprocessor System register space is as follows.

Location	Register	Information
0b0001	FPSCR.{N, Z, C, V}	Access to flags
0b0010	FPSCR.{N, Z, C, V, QC}	Access to flags, including MVE saturation flag
0b1100	VPR	Privileged access to this register only
0b1101	VPR.P0	Access to P0 field
0b1110	FPCXT_NS	Saves and restores the Non-secure FP context
0b1111	FPCXT_S	Saves and restores the Secure FP context

All locations that are not explicitly listed in this table are reserved, and accesses to these locations result in UNPREDICTABLE behavior.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [FP](#) || [MVE](#).

I_{GJWP} Software can use [VMRS](#) and [VMSR](#) instructions to access the Floating-point Status and Control registers.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [FP](#).

I_{DSSS} Software can use [VMRS](#), [VMSR](#), [VLDR \(System Register\)](#), and [VSTR \(System Register\)](#) instructions to access [FPCXT](#), [VPR](#), and the Floating-point Status and Control registers.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [FP](#) || [MVE](#).

R_{RRTZ} Accesses to the [FPCXT](#) will behave as NOPs unless at least one of MVE or the Floating-point Extension is implemented.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [FP](#) || [MVE](#).

R_{WXZV} Accesses to the [FPCXT](#) are UNDEFINED from the Non-secure state.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - ([FP](#) || [MVE](#)) && [S](#).

R_{FXBJ} Execution of Floating-point instructions that generate Floating-point exceptions update the appropriate status fields of [FPSCR](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [FP](#).

See also:

[B3.36 Coprocessor support](#).

[B4.1 The optional Floating-point Extension, Fpv5](#).

FPSCR, Floating Point Status and Control Register.

B4.3 Registers for Floating-point data processing, S0-S31, or D0-D15

R_{TWCB}

The registers that FPv5 adds for Floating-point processing are visible as either:

- 32 single-precision registers, [S0-S31](#).
- 16 double-precision registers, [D0-D15](#).

These map as follows:

S0-S31		D0-D15	
S0		D0	
S1			
S2		D1	
S3			
S4		D2	
S5			
S6		D3	
S7			
S28		D14	
S29			
S30		D15	
S31			

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP** || **MVE**.

R_{XWJQ}

After a Warm reset, the values of [S0-S31](#) or [D0-D15](#) are UNKNOWN.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP** || **MVE**.

R_{WGFL}

If a **VLDM** indexes **S** and **D** registers outside of the range defined by **VFPSmallRegisterBank()** the instruction will behave as a NOP in respect of the unimplemented **S** or **D** registers.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **FP** || **MVE**.

R_{RDCQ}

If a **VSTM** indexes **S** and **D** registers outside of the range defined by **VFPSmallRegisterBank()** the behavior is as follows:

- For unimplemented **S** registers the instructions behaves as NOP and no writes to memory occur.
- For unimplemented **D** registers the instruction behaves either as a NOP or it is permissible for some writes to the addresses specified by the instruction to occur but the values written will be UNKNOWN.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **FP** || **MVE**.

R_{DLNJ}

If a **VSCCLRM** indexes **S** and **D** registers outside of the range defined by **VFPSmallRegisterBank()** the instruction will behave as a NOP in respect of the unimplemented **S** or **D** registers.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **FP** || **MVE**.

R_{NHLX}

If a **VLSTM** instruction is executed an LSERR SecureFault is raised if all the following conditions are met:

- **CONTROL_S.FPCA** is set.
- The banked version of **FPCCR.LSPACT** indicated by **FPCCR_S.S** is set.

If the LSERR SecureFault is raised, all of the following occurs:

- The existing lazy state activation is not modified.
- The registers associated with the [Floating-point context](#) are not modified.

- The registers associated with the [Additional floating-point state context](#) are not modified.
- No stacking operations are performed.

*Applies to an implementation of the architecture **Armv8.0-M** onward. The extension requirements are - **S** && (**FP** || **MVE**).*

See also:

[B4.1 The optional Floating-point Extension, Fpv5.](#)

[B3.18 Exception handling.](#)

B4.4 Floating-point standards and terminology

I_{XNMN} There are two editions of the IEEE 754 standard:

- IEEE 754-1985.
- IEEE 754-2008.

In this manual, references to IEEE 754 that do not include the year apply to either edition.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [FP](#).

I_{MQFS} The Floating-point terminology that this manual uses differs from that used in IEEE 754-2008 as follows:

This manual	IEEE 754-2008
Normalized	Normal
Denormal, or denormalized	Subnormal
Round towards Minus Infinity (RM)	roundTowardNegative
Round towards Plus Infinity (RP)	roundTowardPositive
Round towards Zero (RZ)	roundTowardZero
Round to Nearest (RN)	roundTiesToEven
Round to Nearest with Ties to Away	roundTiesToAway
Rounding mode	Rounding-direction attribute

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [FP](#).

I_{BGPn} The following is called *Arm standard Floating-point operation*:

- IEEE 754-2008 plus the following configuration:
 - [Flush-to-zero mode](#) enabled.
 - Default [NaN](#) mode enabled.
 - Round to Nearest mode selected.
 - Alternative half-precision interpretation not selected.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [FP](#).

See also:

IEEE 754-2008, IEEE Standard for Floating-point Arithmetic, August 2008.

[B4.8 The Flush-to-zero mode.](#)

[B4.9 The Default NaN mode, and NaN handling.](#)

[B4.6 Floating-point encoding formats, half-precision, single-precision, and double-precision.](#)

B4.5 Floating-point data representable

R_{FWXC}

FPv5 supports the following, as defined by IEEE 754:

- Normalized numbers.
- Denormalized numbers.
- Zeros, +0 and -0.
- Infinities, $+\infty$ and $-\infty$.
- NaNs, signaling NaNs and quiet NaN.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FP**.*

See also:

[B4.4 Floating-point standards and terminology.](#)

IEEE 754-2008, IEEE Standard for Floating-point Arithmetic, August 2008.

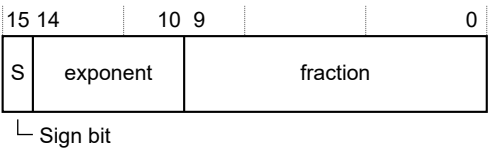
[B4.6 Floating-point encoding formats, half-precision, single-precision, and double-precision.](#)

B4.6 Floating-point encoding formats, half-precision, single-precision, and double-precision

R_{RHKS} The half-precision, single-precision, and double-precision encoding formats are those defined by IEEE 754-2008, in addition to an alternative half-precision format.

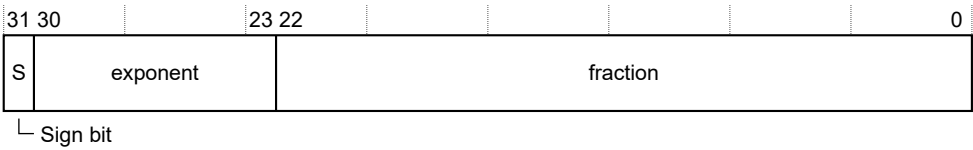
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

I_{LGTJ} The half-precision encoding format is:



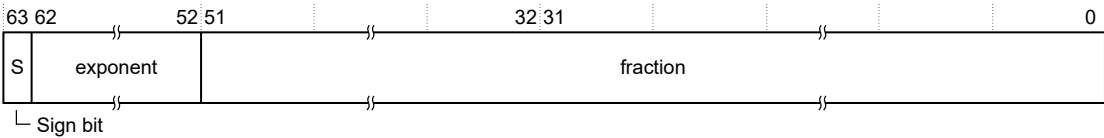
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

I_{CWBP} The single-precision encoding format is:



Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

I_{FVWV} The double-precision encoding format is:



Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.

R_{RWRW} The interpretations of the half-precision, single-precision, and double-precision encoding formats are as follows.

Half-precision

There are two interpretations of the half-precision encoding formats:

- The interpretation that is defined by IEEE 754-2008.
- An alternative half-precision interpretation, indicated by **FPSCR.AHP**.

Single-precision

The interpretation that is defined by IEEE 754-2008.

Double-precision

The interpretation that is defined by IEEE 754-2008. See the following table:

E (biased exponent)	T (trailing significand)	S (sign bit)	T [51]	Value
Zero for all formats.	Nonzero	-	-	A denormalized number.
-	Zero	0	-	Zero, +0
-	-	1	-	Zero, -0
Zero < E < 0x1F, if one of the half precision formats.	-	-	-	A normalized number.
Zero < E < 0xFF, if single-precision format.	-	-	-	-
Zero < E < 0x7FF, if double-precision format.	-	-	-	-
0x1F, if half-precision format, IEEE interpretation.	Nonzero	-	0	A signaling NaN
0xFF, if single-precision format.	-	-	1	A quiet NaN
0x7FF, if double-precision format.	Zero	0	-	Infinity, $+\infty$
-	Zero	1	-	Infinity, $-\infty$
0x1F, if half-precision, alternative half-precision interpretation.	-	-	-	A normalized number.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP**.

R_{DPHH}

The value of a normalized number is equal to:

Half-precision: $(-1)^S \times 2^{(E-15)} \times (1.T)$

Single-precision: $(-1)^S \times 2^{(E-127)} \times (1.T)$

Double-precision: $(-1)^S \times 2^{(E-1023)} \times (1.T)$

The value of a denormalized number is equal to:

Half-precision: $(-1)^S \times 2^{-14} \times (0.T)$

Single-precision: $(-1)^S \times 2^{-126} \times (0.T)$

Double-precision: $(-1)^S \times 2^{-1022} \times (0.T)$

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP**.

R_{PKXD}

Denormalized numbers can be flushed to zero. FPUv5 provides a **Flush-to-zero mode**.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP**.

See also:

IEEE 754-2008, IEEE Standard for Floating-point Arithmetic, August 2008.

[B4.5 Floating-point data representable.](#)

B4.7 The IEEE 754 Floating-point exceptions

R_{BCC}L

The IEEE 754 Floating-point exceptions are:

Invalid Operation: This exception is as IEEE 754-2008 (7.2) describes.

Division by zero: This exception is as IEEE 754-2008 (7.3) describes, with the following assumption:

- For the reciprocal and reciprocal square root estimate functions the dividend is assumed to be +1.0.

Overflow: This exception is as IEEE 754-2008 (7.4) describes.

Underflow: This exception is as IEEE 754-2008 (7.5) describes, with the additional clarification that:

- Assessing whether a result is tiny and nonzero is done before rounding.

Inexact: This exception is as IEEE 754-2008 (7.6) describes.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FP**.*

I_{JCWS}

The criteria for the Underflow exception to be generated are different in [Flush-to-zero mode](#).

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FP**.*

I_{NFHK}

The corresponding status flags for the IEEE 754 Floating-point exceptions are FPSCR.{IOC, DZC, OFC, UFC, IXC}.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FP**.*

See also:

IEEE 754-2008, IEEE Standard for Floating-point Arithmetic, August 2008.

[B4.8 The Flush-to-zero mode](#).

B4.8 The Flush-to-zero mode

I _{XGFP}	Software can enable Flush-to-zero mode by setting FPSCR.FZ to 1. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
I _{WMKJ}	Using Flush-to-zero mode is a deviation from IEEE 754. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{JQHX}	Half-precision Floating-point numbers are exempt from Flush-to-zero mode . <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{LVCG}	In an Armv8.1-M implementation Half-precision Floating-point numbers are subject to Flush-to-zero mode . <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP.</i>
R _{VJSF}	When Flush-to-zero mode is enabled, all single-precision denormalized inputs and double-precision denormalized inputs to Floating-point operations are treated as though they are zero, that is they are flushed to zero. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{GGQW}	In an Armv8.1-M implementation when Flush-to-zero mode is enabled, all half-precision denormalized inputs to Floating-point operations are treated as though they are zero, that is they are flushed to zero. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP.</i>
R _{KBJJ}	When an input to a Floating-point operation is flushed to zero, the PE generates an Input Denormal exception. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{SBCK}	Input Denormal exceptions are only generated in Flush-to-zero mode . <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{WJDM}	When Flush-to-zero mode is enabled, the sequence of events for an input to a Floating-point operation is: 1. Flush to Zero processing takes place. If appropriate, the input is flushed to zero and the PE generates an Input Denormal exception. 2. Tests for the generation of any other Floating-point exceptions are done after Flush to Zero processing. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{PHPT}	When Flush-to-zero mode is enabled, the result of a Floating-point operation is treated as if it is zero if, before rounding, it satisfies the condition: $0 < \text{Abs}(\text{result}) < \text{MinNorm}$, where: • MinNorm is 2^{-126} for single-precision. • MinNorm is 2^{-1022} for double-precision. The result is said to be flushed to zero. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{QPQF}	When the result of a Floating-point operation is flushed to zero, the PE generates an Underflow exception. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{TPVD}	In Flush-to-zero mode , the PE generates Underflow exceptions only when a result is flushed to zero. This uses different criteria than when Flush-to-zero mode is disabled. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>

R _{RTPH}	When a Floating-point number is flushed to zero, the sign is preserved. That is, the sign bit of the zero matches the sign bit of the number being flushed to zero. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && FP.</i>
R _{RWRT}	The PE does not generate an Inexact exception when a Floating-point number is flushed to zero. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
I _{SQCJ}	The corresponding status flag for the Input Denormal exception is FPSCR.IDC . <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>

B4.8.1 The Flush to zero mode half-precision calculations

Applies to an implementation of the architecture Armv8.1-M onward.

I _{MMKS}	In an Armv8.1-M implementation Flush-to-zero mode mode is extended to include half-precision calculations. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP. Note, !8.0.</i>
I _{NPCG}	Software can enable Flush-to-zero mode for half-precision calculations by setting FPSCR.FZ16 to 1. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP. Note, !8.0.</i>
R _{KZFH}	When Flush-to-zero mode is enabled, the result of a Floating-point operation is treated as if it is zero if, before rounding, it satisfies the condition: $0 < \text{Abs}(\text{result}) < \text{MinNorm}$, where: • MinNorm is 2^{-14} for half-precision. • MinNorm is 2^{-126} for single-precision. • MinNorm is 2^{-1022} for double-precision. The result is said to be flushed to zero. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP. Note, !8.0.</i>
R _{HJZZ}	The Effective value of FPSCR.FZ16 is zero when converting real values and integers from one Floating-point format to another. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP. Note, !8.0.</i>
R _{LCDV}	When Flush-to-zero mode is enabled for half-precision Floating-point and a half-precision Floating-point number is flushed to zero an Input Denormal Floating-point exception will not be generated. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - FP. Note, !8.0.</i>

See also:

[B4.7 The IEEE 754 Floating-point exceptions.](#)

B4.9 The Default NaN mode, and NaN handling

I _{FGPN}	Software can enable Default NaN mode by setting FPSCR.DN to 1. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
I _{DJVH}	Using Default NaN mode is a deviation from IEEE 754. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{QMQC}	When Default NaN mode is enabled, the <i>Default NaN</i> is the result of both: • All Floating-point operations that produce an untrapped Invalid Operation exception. • All Floating-point operations whose inputs include at least one quiet NaN but no signaling NaNs. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{NPRL}	IEEE 754 specifies that: • An operation that produces an untrapped Invalid Operation exception returns a quiet NaN as its result. When Default NaN mode is disabled, behavior complies with this and adds: • If the Invalid Operation exception was generated because one of the inputs to the operation was a signaling NaN, the quiet NaN result is equal to the first signaling NaN input with its most significant bit set to 1. • The quiet NaN result is the Default NaN otherwise. The <i>first signaling NaN input</i> means the first argument, in the left-to-right ordering of arguments, that is passed to the pseudocode function describing the operation. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
R _{VCSB}	IEEE 754 specifies that: • An operation using a Quiet NaN as an input, but no signaling NaNs as inputs, returns one of its quiet NaN inputs as its result. When Default NaN mode is disabled, behavior complies with this and adds: • The Quiet NaN result is the first Quiet NaN input. The <i>first quiet NaN input</i> means the first argument, in the left-to-right ordering of arguments, that is passed to the pseudocode function describing the operation. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i>
I _{LXLF}	Depending on the Floating-point operation, the exact value of a Quiet NaN result might differ in both sign and the number of T bits from its source. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP.</i> See also: B4.10 The Default NaN. B4.6 Floating-point encoding formats, half-precision, single-precision, and double-precision.

B4.10 The Default NaN

R_FQFG

The Default NaN is:

Field	Half-precision, IEEE 754-2008 interpretation	Single-precision	Double-precision
S	0	0	0
E	0x1F	0xFF	0x7FF
T	bit[9] == 1, bits[8:0] == 0	bit[22] == 1, bits[21:0] == 0	bit[51] == 1, bits[50:0] == 0

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP**.

See also:

- [B4.6 Floating-point encoding formats, half-precision, single-precision, and double-precision.](#)
- [B4.9 The Default NaN mode, and NaN handling.](#)

B4.11 Combinations of Floating-point exceptions

I_{BTTH}

In compliance with IEEE 754:

- An Inexact Floating-point exception can occur with an Overflow Floating-point exception.
- An Inexact Floating-point exception can occur with an Underflow Floating-point exception.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FP**.*

R_{LEFH}

An Input Denormal exception can occur with other Floating-point exceptions.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FP**.*

See also:

[B4.7 The IEEE 754 Floating-point exceptions.](#)

[B4.8 The Flush-to-zero mode.](#)

B4.12 Priority of Floating-point exceptions relative to other Floating-point exceptions

R_{PLHJ}

Some Floating-point instructions specify more than one Floating-point operation. In these cases, an exception on one operation is higher priority than an exception on another operation when generation of the second exception depends on the result of the first operation. Otherwise, it is UNPREDICTABLE which exception is higher priority.

*Applies to an implementation of the architecture **Armv8.0-M** onward. The extension requirements are - **FP**.*

See also:

[B4.7 The IEEE 754 Floating-point exceptions.](#)

Chapter B5

Vector Extension

This chapter specifies the optional Armv8.1-M Vector Extension rules. It contains the following sections:

[B5.1 *Vector Extension operation.*](#)

[B5.2 *Vector register file.*](#)

[B5.3 *Lanes.*](#)

[B5.4 *Beats.*](#)

[B5.5 *Predication/conditional execution.*](#)

[B5.6 *MVE interleaving/de-interleaving loads and stores.*](#)

*Applies to an implementation of the architecture **Armv8.1-M** onward.*

B5.1 Vector Extension operation

I _{LRKM}	MVE-I operates on 32-bit, 16-bit, and 8-bit data types, including Q7 , Q15 , Q31 integer values. MVE-F operates on half-precision and single-precision floating-point values. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>
R _{JYRS}	Vector instructions operate on a fixed vector width of 128 bits. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I _{MSHF}	Integer MVE , MVE-I, instructions can be implemented with or without the scalar Floating-point Extension. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I _{SLHH}	Floating-point MVE , MVE-F, instructions require the Floating-point extension to be implemented. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I _{LXMG}	An implementation that includes MVE also includes the DSP Extension. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>
R _{QRWY}	Vector operations are divided in two orthogonal ways: • Lanes. • Beats. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I _{FNXY}	The word Element is used in this specification to refer to the data that is put into a lane. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>
R _{VXBF}	Multiple lanes can be executed per beat. There are four beats per vector instruction. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R _{BDHN}	The pseudocode for each vector instruction is executed four times, one time for each beat. The GetCurInstrBeat() function returns the current beat number and predication details. These determine which of the lanes are operated on during the current execution of the code. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R _{LFDY}	Multiple Element writes that are generated by the same vector store instruction by the same Observer can be observed in any order, with the exception that writes to the same location by different Elements are observed in order of increasing vector element number. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R _{FBRZ}	If CCR.BFHFNMIGN is set to one and a multi access load or store instruction generates a BusFault while executing at a requested negative priority, the BusFault may affect all of the accesses generated by that instruction and not just the access that generated the BusFault. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I _{KPFV}	In this case, the values in the destination registers for the load operation are UNKNOWN. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>

See also:

[B3.28 Low overhead loops](#).

[B3.18 Exception handling](#).

B5.2 *Vector register file.*

B5.3 *Lanes.*

B5.4 *Beats.*

B5.2 Vector register file

R_{BBYH}

MVE defines eight vector registers, **Q<n>**, that alias onto the Floating-point Extension register file.

```
1 Q[0][127:96] = S3, Q[0][95:64] = S2, Q[0][63:32] = S1, Q[0][31:0] = S0
2 Q[1][127:96] = S7, Q[1][95:64] = S6, Q[1][63:32] = S5, Q[1][31:0] = S4
3 ...
4 Q[7][127:96] = S31, Q[7][95:64] = S30, Q[7][63:32] = S29, Q[7][31:0] = S28
```

These registers map as follows:

S0-S31		D0-D15		Q0-Q7	
S0		D0		Q0	
S1					
S2		D1			
S3					
S4		D2		Q1	
S5					
S6		D3			
S7					
⋮		⋮		⋮	
S28		D14		Q7	
S29					
S30		D15			
S31					

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **MVE**.

R_{PPPV}

If CP10 is enabled, access to vector register 0-7 is permitted, unless otherwise stated in the individual instruction descriptions.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **MVE**.

I_{MFLD}

To reduce pressure on the vector register file, many vector instructions can use scalar arguments from the general-purpose register file.

Applies to an implementation of the architecture *Armv8.1-M* onward.

I_{WWPZ}

After a Warm reset, the values of **Q0-Q7** are UNKNOWN.

Applies to an implementation of the architecture *Armv8.1-M* onward.

See also:

[C1.4 Instruction set encoding information.](#)

B5.3 Lanes

R_{DWVD} The lane width of the operation to be performed is specified by the instruction that is being executed.
Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{QSBC} The permitted lane widths, and lane operations per beat, are:

- For a 64-bit lane size, a beat performs half of the lane operation.
- For a 32-bit lane size, a beat performs a one lane operation.
- For a 16-bit lane size, a beat performs a two lane operations.
- For an 8-bit lane size, a beat performs a four lane operations.

Bit positions	127				96				64				32				31				0			
A)	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0								
B)	7		6		5		4		3		2		1		0									
C)	3				2				1				0											
D)	1								0															

- A) 8-bit lane numbers
- B) 16-bit lane numbers
- C) 32-bit lane numbers
- D) 64-bit lane numbers

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

See also:
[Chapter C2, Instruction specification](#)

B5.4 Beats

I_{YYZD} A vector instruction executes beats sequentially, from beat 0-3.

Bit position	127	96	95	64	63	32	31	0
Beat number	3			2		1		0

Applies to an implementation of the architecture Armv8.1-M onward.

I_{PCBB} The number of beats for each tick describes how much of the architectural state is updated for each [Architecture tick](#) in the common case. In a trivial implementation, an [Architecture tick](#) might be one clock cycle:

- In a single-beat system, one beat might occur for each tick.
- In a dual-beat system, two beats might occur for each tick.
- In a quad-beat system, four beats might complete for each tick.

Applies to an implementation of the architecture Armv8.1-M onward.

R_{XTZH} It is IMPLEMENTATION DEFINED how many beats are executed for each [Architecture tick](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

I_{MWSJ} The number of beats per tick might change at runtime and is not required to be constant.

Applies to an implementation of the architecture Armv8.1-M onward.

R_{QJJP} Multiple faults might occur within a single [Architecture tick](#). In this case, only one fault is raised. The fault that is generated is determined using the following priorities:

- The fault from the oldest instruction takes priority.
- If multiple faults are associated with the oldest faulting instruction, the fault that was generated by the lowest numbered [Beat](#) takes priority.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

I_{LHXJ} An exception can be taken on any beat of a vector instruction. [RETPSR.ECI](#) in the exception stack frame stores information about how many beats of the instruction at the return address and how many beats of the subsequent instruction have been executed.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{VVVG} When the execution of a beat-wise instruction raises a MemManage, BusFault, or SecureFault as a result of SAU or IDAU permission violations, the Fault Address Register associated with the exception is permitted to report any address associated with any lane of the instruction, including lanes predicated false, if all of the following are true:

- The lane associated with the address being reported belongs to the same beat as the lane that caused the fault.
- The address being reported is in the same 32-byte naturally-aligned address range as the lane that caused the fault.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{FXDM} If multiple lanes of a beat-wise instruction raise a MemManage, BusFault, or SecureFault as a result of SAU or IDAU permission violations within a single [Architecture tick](#), it is IMPLEMENTATION DEFINED which lane's fault is raised.

Regardless of which lane's fault is raised, a valid ECI value must be generated, and the ECI value must be consistent with the updates made to other architecture state, for example the [Q](#) or [R](#) register values.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

Chapter B5. Vector Extension
B5.4. Beats

I_{ZBKX} A dual-beat overlap system implies that the last two beats of a vector instruction can overlap with the first two beats of the next vector instruction.

Applies to an implementation of the architecture Armv8.1-M onward.

I_{GNLS} The following is an example of a dual-beat system where two beats are executed per [Architecture tick](#). The figure labels are:

Tick [Architecture tick](#).

A0-A3 Beats of the `VLDWR` instruction.

B0-B3 Beats of the `VMUL` instruction.

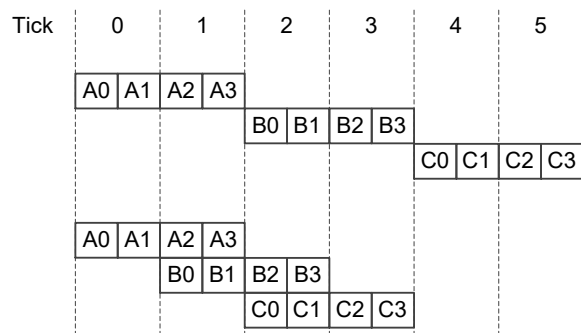
C0-C3 Beats of the `VSHR` instruction.

Vector instructions not overlapping

```
VLDWR.U32 Q1, [R0], #16
VMUL.I32  Q0, Q1,  Q2
VSHR.U32  Q0, Q0,  #1
```

Vector instructions overlapping

```
VLDWR.U32 Q1, [R0], #16
VMUL.I32  Q0, Q1,  Q2
VSHR.U32  Q0, Q0,  #1
```



[EPSR.ECI](#) explains how beats are captured in the ECI field.

Applies to an implementation of the architecture Armv8.1-M onward.

R_{VWBD} The PE can resume execution of an [exception continuable instruction](#) from any valid ECI value, even if the PE cannot generate all the ECI values.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

R_{KRNF} Instructions that are subject to beat-wise execution can only overlap if they are consecutive in the execution order.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

R_{FKCG} The architecturally visible overlap of instructions is only permitted for instructions subject to beat-wise execution if:

- The overlap does not violate data dependencies between instruction beats.
- The overlap is not between two instructions subject to beat-wise execution that both access memory.
- In a low overhead loop, the overlap does not violate [LR hazard](#).
- The overlap is not between an instruction before a [BF branch point](#) and the instruction at the target of the BF.
- An implicit [LE](#), [LETP](#) instruction is executed at the end of a loop body when [LO_BRANCH_INFO](#) is valid and the instruction after the implicit [LE](#), [LETP](#) instruction in execution order is subject to beat-wise execution.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

I_{NQKF} Vector instructions are permitted to overlap if the data dependency is at beat granularity and not at instruction granularity.

Applies to an implementation of the architecture Armv8.1-M onward.

R_{PRSG} After each [Architecture tick](#), the architectural instruction overlap is representable by a valid [EPSR.ECI](#) value.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - [MVE](#).*

R_{TBDV} The [PC](#) is only updated when all beats of an instruction have completed.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - [MVE](#).*

See also:

[B5.5.1 Loop tail predication](#).

[B3.29 Branch future](#).

[B3.27.6 Exception-continuable instruction behavior](#).

[Chapter C2, Instruction specification](#).

B5.5 Predication/conditional execution

R_{RSLP}

MVE includes predication that enables the independent masking of each lane within a vector operation. It supports the following predication mechanisms:

- Loop tail predication. This eliminates the requirement for special vector tail handling code after loops where the number of **Elements** to be processed is not a multiple of the number of **Elements** in the vector.
- VPT predication. This enables data-dependent conditions that are based on data value comparisons to mask each vector lane separately.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **MVE**.*

R_{QGLY}

Loop tail predication and VPT predication operate separately. The resulting predication flags from each mechanism are ANDed together so that a lane of a vector operation is only active if both the loop tail predication and the VPT predication conditions are true.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **MVE**.*

See also:

[B5.5.1 Loop tail predication.](#)

[B5.5.2 VPT predication.](#)

[B5.5.3 Effects of predication.](#)

B5.5.1 Loop tail predication

R_{DCZN}

Low overhead loops can be used with vector instructions, for example with a word-based memory copy instruction. The number of words to copy might not be a multiple of the vector length, therefore loop tail predication can eliminate any additional tail handling steps.

MVE includes special loop tail predication instructions, **WLSTP**, **DLSTP**, **LETP**, and **LCTP**, that operate as follows:

- The source register of the loop start instruction contains the number of vector **Elements** that are to be processed, instead of the iteration count.
- The loop start instruction sets **FPSCR.LTPSIZE** to the requested **Element** size. This alters the amount by which the **Element** count in LR is decremented at the end of each loop iteration.
- On the last iteration of the loop, the values in LR and **FPSCR.LTPSIZE** determines the number of vector lanes that are to be masked.
- After the last instruction of the last loop iteration has been executed, tail predication is disabled by setting **FPSCR.LTPSIZE** to 0b100.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **MVE**.*

I_{RBP}

The active floating-point state is defined by `ActiveFPState()`.

Applies to an implementation of the architecture Armv8.1-M onward.

R_{GQSH}

To prevent the inadvertent creation of floating-point contexts and the predication of vector operation outside of a loop, **FPSCR.LTPSIZE** behaves as follows:

- **FPSCR.LTPSIZE** reads as 0b100 if there is no active floating-point state.
- **FPSCR.LTPSIZE** is set to 0b100 if any of the following events occur:
 - On the last iteration of a loop by either the execution of an **LETP** instruction, or by execution reaching the end of the loop body when **LO_BRANCH_INFO** is valid and the floating-point context is active, that is **CONTROL.FPCA** is one.
 - An **LCTP** instruction is executed.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **MVE**.*

I _{NRTL}	Arm recommends that tail predicated loop start instructions are only used with a tail predicated loop end instruction. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>
R _{BDJB}	FPDSCR.LTPSIZE always reads as 0b100, and therefore the floating-point contexts that are automatically initialized are created with predication disabled. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R _{JHLP}	The behavior of a beat-wise capable instruction that modifies LR and is within a tail predicated low overhead loop is CONSTRAINED UNPREDICTABLE, the permitted behaviors are either of: • An UNDEFINSTR UsageFault is generated. • The instruction, and any adjacent instructions that are permitted to overlap, are subject to UNKNOWN predication. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>

B5.5.2 VPT predication

R _{ZFLV}	Comparison-based predication is supported by vector predication blocks. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R _{CVTC}	A vector predication block is called a VPT block. A VPT block is defined as the <i>n</i> instructions following a VPT or VPST instruction, where <i>n</i> is the number of instructions that the VPT or VPST instruction defines as being subject to predication conditions. The predication conditions are stored in the VPR register. <i>n</i> is less than or equal to 4. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>
R _{BVPG}	The instructions in a VPT block can be subject to either the condition or to the inverse of the condition. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I _{RVJJ}	A VCMP (vector) or a VCMP (floating-point) instruction can be placed inside a VPT block. VCMP instructions update the predication flag on completion, therefore affecting the subsequent instructions in the VPT block. The subsequent instructions in the VPT block are subject to the predicates of the VPT block and the updates caused by the VCMP instructions. The execution of successive VCMP instructions permits the creation of complex predication conditions. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>
I _{PHJP}	Allowing instructions to be subject to either the condition or the inverse of the condition enables the instructions in both the THEN (T decorator) and the ELSE (E decorator) parts of an IF statement to be predicated with a single VPT instruction. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>

R_{VWGT}

The following table shows the [VPT](#) instruction variants, mask field encodings, and the associated decorators that are placed on the subsequent instructions.

Instruction name	Mask value	Number of subsequent instructions to be predicated	<v> instruction decorator			
			First	Second	Third	Fourth
VPT	0b1000	1	T	-	-	-
VPTT	0b0100	2	T	T	-	-
VPTE	0b1100	2	T	E	-	-
VPTTT	0b0010	3	T	T	T	-
VPTTE	0b0110	3	T	T	E	-
VPTEE	0b1010	3	T	E	E	-
VPTET	0b1110	3	T	E	T	-
VPTTTT	0b0001	4	T	T	T	T
VPTTTE	0b0011	4	T	T	T	E
VPTTEE	0b0101	4	T	T	E	E
VPTTET	0b0111	4	T	T	E	T
VPTEEE	0b1001	4	T	E	E	E
VPTEET	0b1011	4	T	E	E	T
VPTETT	0b1101	4	T	E	T	T
VPTETE	0b1111	4	T	E	T	E

The same encoding format is used for [VPST](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

R_{LKXQ}

[VPR](#) contains a MASK field for each pair of beats of a vector instruction. This permits beat-wise overlapping of the [VPT](#) or [VPST](#) instructions with the surrounding vector instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

R_{KHCV}

The state of [VPR](#) is UNKNOWN when use of a VPT block results in CONSTRAINED UNPREDICTABLE behavior.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

R_{XHQ}

The following conditions result in CONSTRAINED UNPREDICTABLE behavior when they apply to a VPT block:

- The presence of a non-VPT compatible instruction in a VPT block. This includes:
 - All instructions that are not part of [MVE](#), with the exception of [BKPT](#).
 - [MVE](#) instructions that are marked as not being VPT compatible.
- A [BF branch point](#) within a VPT block.
- Branching into a VPT block.
- Exception return or returns from Debug state if [VPR.{MASK23, MASK01}](#) is not consistent with the position returned to in the VPT block.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#). Note, Debug state requires HDBG.

R_{DZWZ}

The CONSTRAINED UNPREDICTABLE behavior for a VPT block is one of the following:

- The VPT or VPST instruction generates an UNDEFINED Instruction fault.
- The instruction that causes the CONSTRAINED UNPREDICTABLE behavior does one of the following:
 - It raises an UNDEFINED Instruction fault.
 - It executes normally.
 - It has UNKNOWN predication applied.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{HXSZ}

In a VPT block, the VPR state is only advanced after the completion of a pair of beats within a VPT compatible instruction that is subject to beat-wise execution.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{XXKG}

The CONSTRAINED UNPREDICTABLE behavior for a VPT compatible instruction executed outside a VPT block when the VPR mask is nonzero is one of the following:

- It raises an UNDEFINED Instruction fault.
- It executes normally.
- It has UNKNOWN predication applied.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{LQDR}

Outside of a VPT block, when VPR.{MASK32, MASK01} is nonzero, the execution of a non-VPT compatible instruction is not UNPREDICTABLE and does not advance VPT state.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{XMHZ}

In the case of an exception return or a return from Debug state, the instruction that exhibits the CONSTRAINED UNPREDICTABLE behavior is defined as the instruction that is being returned to.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE. Note, Debug state requires HDBG.

R_{MBQX}

For a BF branch point within a VPT block, the instruction that exhibits the CONSTRAINED UNPREDICTABLE behavior can be one of the following:

- The instruction before the BF branch point.
- The instruction after the BF branch point.
- The instruction at the BF branch target address.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{SZQK}

For the purposes of the CONSTRAINED UNPREDICTABLE behavior described in this section, a memory location is considered to be in VPT block until:

- The VPT or VPST instruction has been removed.
- All the addresses that are covered by the VPT block have been invalidated in the instruction cache (if implemented).
- A subsequent Context synchronization event has occurred.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

I_{KKNH}

There are similarities between VPT/VPR and IT/ITSTATE, but there are also some important differences as follows:

- Unlike IT, the VPT instruction performs the actual comparison in addition to applying the result to the subsequent instructions. As such, VPT can be considered as the vectorized combination of CMP and IT.
- There are multiple MASK fields in VPR that handle partial instruction execution caused by exceptions during the overlapping of instructions.
- The MASK fields are similar to ITSTATE[3:0] and encode both the number of instructions outstanding in the current VPT block, and whether these instructions are subject to the THEN or the ELSE condition.

Applies to an implementation of the architecture Armv8.1-M onward.

R_{HJGT} **VPR.P0** contains one predication bit per 8-bit lane. The **VPR** mask bits cause the VPR predication bits to be inverted if the corresponding mask bit is set to 1. The mask bits that are shifted out toggle the current predication condition and are not part of the predication condition. The value of **VPR.MASK01** affects bits[7:0] of **VPR.P0** and the value of **VPR.MASK23** affects bits [15:8] of **VPR.P0**.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{PXY} The **VPR** predication bits are not inverted after executing the last instruction in a VPT block.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

I_{LRVJ} The state in the **VPR** register can be accessed directly using **VMRS**, **VMSR**, **VLDR** (System Register), and **VSTR** (System Register) instructions. Setting **VPR** using a **VMSR** or **VLDR** (System Register) instruction does not make the instructions that follow **VMSR** or **VLDR** (System Register) part of a VPT block.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{MPTV} Execution of a VPT compatible instruction outside of a VPT block with a nonzero value in **VPR.{MASK23, MASK01}** results in CONSTRAINED UNPREDICTABLE behavior and does one of the following:

- It raises an UNDEFINED Instruction fault.
- It executes normally.
- It has UNKNOWN predication applied.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{FJLH} It is IMPLEMENTATION DEFINED which of the following behaviors occur if a prohibited unprivileged read from **VPR** is attempted:

- Returns zeroes.
- Behaves as a NOP.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

See also:

[Chapter C1 Instruction Set Overview](#).

B5.5.3 Effects of predication

I_{TZPD} The exact effects of a false predication value are defined in the instruction pseudocode.

Applies to an implementation of the architecture Armv8.1-M onward.

R_{SYDC} Vector predication has no effect on scalar instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{ZLSJ} For non-load instructions for vector register file writes, predication is always performed at byte level granularity, regardless of the **Element** size that is specified by the vector instruction.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{VHSN} For non-load instructions, the predicate flags determine if the destination register byte is updated with the new value or if the previous value is preserved. For load instructions, where lanes are predicated false, the corresponding parts of the destination register are set to 0.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{LYPK} For base pointer write-back, vector predication does not affect address write-back in load and store instructions. This applies both when the address is in a scalar register, and when it is in a vector register.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{LNDP} The predication flag determines whether a lane operation is performed. For [Element](#) sizes of more than 8 bits for the types of instruction listed here, the LSB of the corresponding group of predicate flags determines:

- For vector operations that perform reduction across the vector and produce a scalar result, whether the value is accumulated or not.
- For vector store instructions, whether the store occurs or not.
- For vector load instructions, whether the value that is loaded or whether zeros are written to that element of the destination register.
- The setting of the [FPSCR.QC](#) saturation flags.

For predication, 64-bit vector memory load/store operations are treated as if they were a pair of 32-bit operations.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

I_{CJRK} The relation between lane width and bits in [VPR.P0](#) is as follows:

Lane width	Bits in VPR.P0
32 bits	[12, 8, 4, 0]
16 bits	[14, 12, 10, 8, 6, 4, 2, 0]
8 bits	[15:0]

Applies to an implementation of the architecture Armv8.1-M onward.

See also:

[B5.5.1 Loop tail predication.](#)

[B5.5.2 VPT predication.](#)

B5.5.4 IT block

R_{PZDX} Instructions that are subject to beat-wise execution are not permitted in IT blocks. For the exceptions to this rule, see the decode pseudocode in the individual instruction descriptions. In these exceptional cases, beat-wise execution is not performed and the instruction does not overlap with other instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

B5.6 MVE interleaving/de-interleaving loads and stores

I_{VWWN} For implementations that include [MVE](#), data streams can be interleaved and de-interleaved with strides of 2 and 4, using `VLD2/VLD4` and `VST2/VST4`.

*Applies to an implementation of the architecture **Armv8.1-M** onward.*

R_{CMZP} The interleaving and de-interleaving instructions always operate on 128 bits of data at a time.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - [MVE](#).*

I_{XGTN} When using `VLD4`, each of the four instructions loads 128 bits of data, and partially updates the four destination vector registers. The memory offsets and destination register sections that are accessed are arranged so that when all four instructions have been executed, the de-interleaving operation has been performed.

<code>VLD40.32 {Q0-Q3}, [Rn]</code>	Q3	<code>S15=Mem[60]</code>	<code>S14=Mem[44]</code>	<code>S13=Mem[28]</code>	<code>S12=Mem[12]</code>
<code>VLD41.32 {Q0-Q3}, [Rn]</code>	Q2	<code>S11=Mem[56]</code>	<code>S10=Mem[40]</code>	<code>S9=Mem[24]</code>	<code>S8=Mem[8]</code>
<code>VLD42.32 {Q0-Q3}, [Rn]</code>	Q1	<code>S7=Mem[52]</code>	<code>S6=Mem[36]</code>	<code>S5=Mem[20]</code>	<code>S4=Mem[4]</code>
<code>VLD43.32 {Q0-Q3}, [Rn]!</code>	Q0	<code>S3=Mem[48]</code>	<code>S2=Mem[32]</code>	<code>S1=Mem[16]</code>	<code>S0=Mem[0]</code>

*Applies to an implementation of the architecture **Armv8.1-M** onward.*

R_{NSFK} The assembly syntax for `VLD2/VLD4` and `VST2/VST4` lists the range of vector registers to be accessed. Only the lowest numbered register is encoded in the opcode. If this register number plus the number of registers to be accessed is greater than 7 (the highest numbered vector register) behavior is a CONSTRAINED UNPREDICTABLE choice of the following:

- The instruction is UNDEFINED.
- The instruction is treated as a NOP.
- One or more of the vector registers become UNKNOWN. If the instruction specifies write-back, the base register becomes UNKNOWN. No other general-purpose registers are affected.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - [MVE](#).*

Chapter B6

Pointer authentication and branch target identification Extension

This chapter specifies the optional Armv8.1-M Pointer Authentication Extension and Branch Target Identification Extension rules. It contains the following sections:

[B6.1 *Implementing PAC and BTI.*](#)

[B6.1.1 *Pointer authentication.*](#)

[B6.1.2 *Branch target identification.*](#)

*Applies to an implementation of the architecture **Armv8.1-M** onward.*

B6.1 Implementing PAC and BTI

R _{QYTS}	The PACBTI Extension is OPTIONAL. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>
R _{MKNJ}	If the PACBTI Extension is implemented ID_ISAR5.PACBTI will read as a nonzero value. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>
I _{VRBT}	The PAC and BTI features can be independently enabled or disabled for every Security state and privilege level. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>

B6.1.1 Pointer authentication

R_{WRRP}

PAC can be enabled or disabled as follows:

- [CONTROL.PAC_EN](#) for privileged mode.
- [CONTROL.UPAC_EN](#) for unprivileged mode.

The pseudocode function [PACEnabled\(\)](#) describes this.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PACBTI](#).

R_{KRRP}

In privileged mode [CONTROL.PAC_EN](#) and [CONTROL.UPAC_EN](#) can be accessed using [MSR](#) and [MRS](#) instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PACBTI](#).

R_{VCCY}

In unprivileged mode [CONTROL.UPAC_EN](#) can be read using the [MRS](#) instruction.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PACBTI](#).

R_{VLRL}

For pointer authentication, there are four 128-bit keys:

	Non-secure state	Secure state
Privileged	PAC_KEY_P_NS	PAC_KEY_P_S
Unprivileged	PAC_KEY_U_NS	PAC_KEY_U_S

The keys can be accessed by privileged software through the [MSR](#) and [MRS](#) instructions where:

- The register values [PAC_KEY_P_3](#) : [PAC_KEY_P_2](#) : [PAC_KEY_P_1](#) : [PAC_KEY_P_0](#) are concatenated to form [PAC_KEY_P](#).
- The register values [PAC_KEY_U_3](#) : [PAC_KEY_U_2](#) : [PAC_KEY_U_1](#) : [PAC_KEY_U_0](#) are concatenated to form [PAC_KEY_U](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PACBTI](#).

R_{LPQW}

For the Pointer authentication instructions, it is IMPLEMENTATION DEFINED whether the encrypted value is generated using:

- The 5-round variant of the QARMA algorithm, known as QARMA5, as described in *The QARMA Block Cipher Family*, Roberto Avanzi, *Qualcomm Product Security Initiative*, when the value of [ID_ISAR5.PACBTI](#) is 0b0001.
- An IMPLEMENTATION DEFINED algorithm when the value of [ID_ISAR5.PACBTI](#) is 0b0010.
- The 3-round variant of the QARMA algorithm, known as QARMA3, as described in *The QARMA Block Cipher Family*, Roberto Avanzi, *Qualcomm Product Security Initiative*, when the value of [ID_ISAR5.PACBTI](#) is 0b0100.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PACBTI](#).

R_{LDRC}

When an IMPLEMENTATION DEFINED algorithm is used, the following applies:

- The algorithm must use the same input arguments as the [ComputePAC\(\)](#) function.
- The algorithm must provide the same result when:
 - The same arguments are passed to the algorithm (values in the registers and keys).
 - The PE is executing at the same Security and privilege level.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PACBTI](#).

I_{KRZP} The IMPLEMENTATION DEFINED algorithm is permitted to apply further cryptographic rounds based on the Security and privilege level of the PE. Using the same input arguments and key values at different Security and privilege levels might produce different results.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

I_{KRXB} The following functions describe the QARMA algorithms supported by the **PACBTI** Extension:

- **ComputePAC()**
- **PACCellInvShuffle()**
- **PACCellShuffle()**
- **PACInvSub()**
- **PACMult()**
- **PACSubQ3()**
- **PACSubQ5()**
- **RotCell()**
- **TweakCellInvRot()**
- **TweakCellRot()**
- **TweakInvShuffle()**
- **TweakShuffle()**

Arm recommends that the QARMA family of algorithms should be used for pointer authentication.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

I_{NPTP} The **PAC** is generated by **PACBTI**, **PAC**, and **PACG** instructions.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

I_{PRJQ} The input arguments of the pseudocode function **ComputePAC()** are:

- Most significant bits of the 32-bit pointer/address/data are zero extended to 64 bits.
- Most significant bits of the 32-bit modifier are zero extended to 64 bits.
- The 128-bit cryptographic key.

The pseudocode function **CreatePAC()** describes this.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R_{TRZN} If a **PAC** authentication instruction fails to validate the **PAC** an INVSTATE UsageFault is generated. The fault will be generated if any of the input arguments used for generating the **PAC** are modified before validating the **PAC**, any of the three general-purpose registers or the cryptographic key. Instructions **AUT**, **BXAUT**, and **AUTG** authenticate **PAC**.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R_{XZVW} When **PACEnabled()** is FALSE:

- **PAC** generating instructions do not create the **PAC** and do not update any registers. Any other functionality provided by the instructions is unaffected.
- **PAC** authentication instructions do not validate the **PAC** and do not generate the INVSTATE UsageFault. Any other functionality provided by the instructions is unaffected.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

I_{PLRN} The output of the cryptographic algorithm is a 32-bit **PAC**, so it is possible for different combinations of input arguments and cryptographic keys to produce an identical **PAC**. This type of cryptographic collision cannot be detected by the **PAC** authentication instructions and an INVSTATE UsageFault will not be generated.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

I_{KFDR} The **PACBTI** Extension does not distinguish between privileged Thread mode execution and Handler mode execution. For example, a pointer authenticated in Handler mode can be validated in privileged Thread mode without generating an INVSTATE UsageFault.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

I_{ZGXX} There is no architectural restriction on the number of instructions that can be executed between:

- An instruction that generates **PAC** and an instruction that validates **PAC**. For example, **PAC** and **AUT**.
- An instruction that validates **PAC** and the return instruction. For example, **AUT** and **BX**.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

I_{FRLR} Arm recommends that between the authentication of a pointer and any subsequent reference to that pointer, the unprotected pointer should not be saved and restored from memory.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

I_{BRTY} Examples of using **PAC** are shown below.

```

1   LDR      R4, =func1      // Load function address.
2   BLX      R4              // Call function which is protected by PAC-AUT.
3 ret1:
4   MOV      R1, #AA         // #AA value stored in R1 register.
5   MOV      R2, #BB         // #BB value stored in R2 register.
6   LDR      R4, =func2
7   BLX      R4              // Call function which is protected by PAC-AUT.
8 ret2:
9   AUTG     R0, R1, R2      // Validate R0, R1, and R2 registers. If any of the
10                                // values or the secret key have changed, then an
11                                // INVSTATE UsageFault is generated.
12                                // Not supported on legacy implementations.
13   ...
14 func1:
15   PAC      R12, LR, SP     // Computes the cryptographic code using LR, SP, and
16                                // a secret key, then stores it in R12.
17                                // This instruction works on legacy implementations.
18   PUSH     {R4-R6, R12, LR} // Stack LR if it needs to be used before the AUT.
19   ...
20   POP      {R4-R6, R12, LR} // Restore the value of LR before authenticating.
21   AUT      R12, LR, SP     // Computes PAC using LR, SP, and a secret key, and
22                                // then compares it against the value in R12.
23                                // If the two values match then the LR is validated,
24                                // otherwise an INVSTATE UsageFault is generated.
25                                // This instruction works on legacy implementations.
26   BX       LR              // Return to 'ret1'.
27 func2:
28   PAC      R12, LR, SP     // Calculate PAC for LR.
29   ...
30   PACG     R0, R1, R2      // Calculate PAC where desired value is in R1 (#AA).
31                                // The modifier value is in R2 (#BB).
32                                // Computed PAC is stored in R0 register.
33                                // Not supported on legacy implementations.
34   ...
35   BXAUT    R12, LR, SP     // Validate LR and branch to LR ('ret2').
36                                // Not supported on legacy implementations.
```

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

See also:

B3.4 Special-purpose CONTROL register

B6.1.2 Branch target identification

R_{JKHH} BTI can be enabled or disabled as follows:

- **CONTROL.BTI_EN** for privileged mode.
- **CONTROL.UBTI_EN** for unprivileged mode.

The pseudocode function **BTIEnabled()** describes this.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R_{LVTR} In privileged mode **CONTROL.BTI_EN** and **CONTROL.UBTI_EN** can be accessed using **MSR** and **MRS** instructions.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R_{PCZX} In unprivileged mode **CONTROL.UBTI_EN** can be read using the **MRS** instruction.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R_{FDGT} When **BTI** is enabled for the target Security and privilege state then any instruction that sets **EPSR.B** or **LO_BRANCH_INFO.BTI** to one is referred to as a **BTI setting** instruction. **BTIEnabled()** describes this.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R_{KRRX} Any instruction that clears **EPSR.B** to zero is referred to as a **BTI clearing** instruction.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R_{HRRL} **EPSR.B** is automatically stacked and cleared to zero on exception entry and restored on exception return.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

I_{LVMN} **EPSR.B** is not stacked and cleared to zero as a part of the **FNC_RETURN** stack frame.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R_{DFGH} If **BTI** is enabled, for the target Security state and privilege level, then **EPSR.B** is set to one when any of the following **BTI setting** instructions are executed:

- **BLX**.
- **BLXNS**.
- When the register holding the branch address is not the **LR**:
 - **BX**.
 - **BXNS**.
- When the address is loaded into the **PC**:
 - **LDR (register)**.
 - **LDR (literal)**.
- When the address is loaded into the **PC** and the base address register is either not the **SP** or the **SP** and write-back of the **SP** does not occur:
 - **LDR (immediate)**.
 - **LDM, LDMIA, LDMFD**.
 - **LDMDB, LDMEA**.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R_{DDFR} The **LO_BRANCH_INFO.BTI** bit is set to one when any of the following **BTI setting** instructions are executed:

- **BFLX**.
- When the register holding the branch address is not the **LR**:
 - **BFX**.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PACBTI**.*

R _{HJGG}	When the BTI setting Branch Future branch is taken, EPSR.B is set to one when all of the following are true: • BTIEnabled() is TRUE. • LO_BRANCH_INFO.BTI is set to one. • LO_BRANCH_INFO.VALID is set to one. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>
I _{PRKZ}	The pseudocode functions HandleLO(), GetInstrExecState(), and InstructionAdvance() show how a BTI setting Branch Future instruction sets EPSR.B to one. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>
R _{TVHG}	Any instruction that is not BTI setting and sets the LO_BRANCH_INFO.VALID bit to one must also clear the LO_BRANCH_INFO.BTI bit to zero. For example, the LE and LETP instructions. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>
I _{HLLR}	The BXAUT instruction is not a BTI setting instruction regardless of which register is used as the branch target. Equivalent behavior with BTI setting can be achieved through an AUT or AUTG instruction followed by a BX instruction. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>
I _{LPCT}	The POP (single register), which is an alias of LDR (immediate), and POP (multiple registers), which is an alias of LDM, LDMIA, LDMFD, are not BTI setting instructions. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>
I _{KDLB}	For transitions from Non-secure to Secure state, if the SG instruction is targeted by a BTI setting branch then the CONTROL_NS.BTI_EN and CONTROL_NS.UBTI_EN bits associated with the privilege level of that branch determine whether the EPSR.B is set. The SG instruction is BTI clearing, so if EPSR.B is set, it will be cleared even if BTI is not enabled. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI && S.</i>
I _{QNTB}	For transitions from Secure to Non-secure state, if the transition is caused by a BTI setting branch, for example BXNS, then the CONTROL_NS.BTI_EN and CONTROL_NS.UBTI_EN bits associated with the privilege level of the Non-secure state will be checked to determine whether the EPSR.B is set. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI && S.</i>
R _{XZCV}	The following instructions are BTI clearing: • BTI. • SG. • PACBTI. If the EPSR.B is set, the BTI clearing instructions will clear it to zero even if BTI is not enabled. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>
R _{YBDN}	When EPSR.B is set the next executed instruction must be a BTI clearing instruction or a BKPT instruction otherwise an INVSTATE UsageFault is generated. If the next instruction is a BTI clearing instruction EPSR.B is cleared. In the case of the BKPT instruction EPSR.B is not cleared and remains set and an INVSTATE UsageFault is not generated. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>
I _{ZLLP}	When PAC functionality is required but creating a landing pad using PACBTI is undesirable then either a PAC or PACG instruction can be used. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI.</i>

I_{YYZG}

Examples of using BTI are shown below.

```

1 // Current Security state is Non-secure
2   LDR    R4, =testBti // Load function address.
3   LDR    R3, =ret1
4   BLX    R4           // BTI is enabled for Non-secure state, set EPSR.B.
5 ret1:
6   BTI                    // BTI clearing instruction.
7   LDR    R4, =funcTestBti
8   BLX    R4           // Set EPSR.B.
9 ret2:
10  LDR    R4, =nscBti
11  BLX    R4           // Target address is in NSC region, set EPSR.B.
12 end:
13  ...
14 // Function in Non-secure state.
15 testBti:
16  BTI                    // BTI clearing instruction, valid entry point.
17  ...                  // LR not used for return, so PAC-AUT is not required.
18  ...                  // But using LR would be recommended in this scenario.
19  BX      R3           // Branch to 'ret1'. LR is not used, so the branch
20                      // target must be a BTI clearing instruction.
21 // Function in Non-secure state.
22 funcTestBti:
23  PACBTI R12, LR, SP    // BTI clearing instruction, valid entry point.
24  ...                  // Stacking and unstacking R12, LR, and SP is safe
25  ...                  // as any corruption will be detected by BXAUT.
26  BXAUT  R12, LR, SP    // Validate LR and return to 'ret2', address in LR.
27                      // The combined authenticate and branch instruction
28                      // is not backwards compatible with legacy software.
29 // Function in Non-secure state.
30 leafFunc:
31  BTI                    // Since BTI is enabled, a BTI clearing instruction
32  ...                  // is required.
33  BX      leafRet       // Does not set BTI, uses FNC_RETURN.
34  ...
35 // Secure and Non-secure callable region.
36 nscBti:
37  SG                    // Clear BTI, regardless of the Secure BTI settings.
38  B      secure        // Call Secure function.
39  ...
40 // Function in Secure state.
41 secure:
42  PAC      R12, LR, SP  // BTI clearing PAC is not required.
43  ...
44  LDR      R6, =leafFunc
45  BLXNS   R6           // Call Non-secure function.
46                      // Instruction implicitly checks BTI settings for the
47                      // current privilege state in CONTROL_NS.
48 leafRet:
49  ...                  // BTI clearing instruction not required.
50  AUT      R12, LR, SP  // Validate LR.
51  ...                  // Stacking and unstacking LR and SP is not safe.
52  ...                  // Corruption of the registers will not be detected.
53  BXNS    LR           // Branch to 'end'. Does not set BTI.

```

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **PACBTI**.

Chapter B7

Memory Model

This chapter specifies the Armv8-M memory model architecture rules. It contains the following sections:

- B7.1 Definition of the Armv8 memory model.*
- B7.2.1 Single-copy atomicity.*
- B7.3 Memory accesses.*
- B7.4 Address space.*
- B7.5 Endianness.*
- B7.6 Alignment behavior.*
- B7.7 Concurrent modification and execution of instructions.*
- B7.8 Access rights.*
- B7.9 Normal memory.*
- B7.10 Cacheability attributes.*
- B7.11 Device memory.*
- B7.12 Device memory attributes.*
- B7.13 Shareability attributes.*
- B7.14 Shareability domains.*
- B7.15 Memory access restrictions.*
- B7.16 Mismatched memory attributes.*
- B7.17 Load-Exclusive and Store-Exclusive accesses to Normal memory.*

B7.18 Load-Acquire and Store-Release accesses to memory.

B7.19 Caches.

B7.20 Cache identification.

B7.21 Cache visibility.

B7.22 Cache coherency.

B7.23 Cache enabling and disabling.

B7.24 Cache behavior at reset.

B7.25 Behavior of Preload Data (PLD) and Preload Instruction (PLI) instructions with caches.

B7.26 Branch predictors.

B7.27 Cache maintenance operations.

B7.28 Ordering of cache maintenance operations.

B7.29 Branch predictor maintenance operations.

B7.1 Definition of the Armv8 memory model

I_{JRPK}

This section describes observation and ordering in the Armv8 memory model. It contains the following subsections:

- [B7.1.1 Locations.](#)
- [B7.2.3 Ordering and observability.](#)
- [B7.2.5 Ordering constraints.](#)
- [B7.2.7 External ordering constraints.](#)
- [B7.2.6 Completion and endpoint ordering.](#)
- [B7.2.9 Memory barriers.](#)

Applies to an implementation of the architecture Armv8.0-M onward.

B7.1.1 Locations

I_{CKCT}

The Armv8 memory model provides a set of definitions that are used to constrain the permitted sequences of accesses to memory. The Armv8 memory model defines:

- The ordering of observation of memory accesses between different Observers.
- The ordering of arrival of memory accesses arriving at an endpoint.
- The mechanisms to control the ordering of observation of memory accesses and the arrival of memory accesses at an endpoint.

Applies to an implementation of the architecture Armv8.0-M onward.

Locations, memory effects, and observers

Location

R_{GZWZ}

A *Location* is a single byte that is associated with an address in the physical address space.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{LDTD}

As part of its execution, an instruction might generate [memory effects](#). Observers in the system might observe the [memory effects](#) of that instruction on a Location.

Applies to an implementation of the architecture Armv8.0-M onward.

Effects

R_{KFVX}

The Effects of an instruction can be:

- Register effects.
- Memory effects.
- Barrier effects.
- Branching effects.

The effects of an instruction I_1 are said to appear in program order before the effects of an instruction I_2 if and only if I_1 occurs before I_2 in the order specified by the program. Each effect generated by an instruction has a unique identifier, which characterizes it among the events generated by the same instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

Register effects

R_{CLDD}

The *Register effects* of an instruction are register reads or register writes of that instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{PTMD}

For an instruction that accesses registers, a register read effect is generated for each register read by the instruction and a register write effect is generated for each register written by the instruction. An instruction may generate both read and write Register effects.

Applies to an implementation of the architecture Armv8.0-M onward.

Memory effects

R_{JXFX}

The *memory effects* of an instruction are the read, write, or barrier effects of that instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{ZRCJ}

For an instruction that accesses memory:

- A read effect is generated for each [Location](#) that is read by the instruction.
- A write effect is generated for each [Location](#) that is written by the instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{VJRP}

An instruction can generate both read and write effects.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MTJR}

The [memory effects](#) of an instruction I₁ are said to appear in program order before the [memory effects](#) of instruction I₂ if and only if I₁ occurs before I₂ in program order.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{WDFL}

For the purposes of describing the Armv8 memory model, all read and write effects access only Normal memory [Locations](#) in a common Shareability domain. Where this section refers to a read, write, or memory barrier without any qualification, then it is referring to the corresponding [memory effect](#).

Applies to an implementation of the architecture Armv8.0-M onward.

Branching effects

R_{DHWH}

The *Branching effects* of an instruction are effects which correspond to a branching decision being taken.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{JXQC}

Conditional and compare-and-swap instructions do not create Branching effects.

Applies to an implementation of the architecture Armv8.0-M onward.

Intrinsic order

R_{DBQB}

There is a per-instruction *Intrinsic order* relation that provides a partial order over the effects of that instruction, according to the operation of that instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

Observer

R_{KWHH}

An *Observer* refers to either a PE, or some other memory accessing agent that can generate reads from or writes to memory.

Applies to an implementation of the architecture Armv8.0-M onward.

Overlapping accesses

R_{QJCH}

Two [memory effects](#) overlap if and only if they access the same [location](#).

*Applies to an implementation of the architecture **Armv8.0-M** onward.*

Common Shareability domain

R_{RZFF}

A *common Shareability domain* for a program is the smallest Shareability domain that contains all of the active Observers of the [memory effects](#) generated by a program.

*Applies to an implementation of the architecture **Armv8.0-M** onward.*

B7.2 Atomicity

B7.2.1 Single-copy atomicity

R_{ZJRK}

Single-copy atomic memory access operations generated by different observers have the following properties:

1. For a pair of single-copy atomic write operations W_1 and W_2 with at least one **overlapping** location, if any of the **overlapping** writes generated by W_1 are **coherence-after** the corresponding writes generated by W_2 , then all of the **overlapping** writes generated by W_1 are **coherence-after** the corresponding writes generated by W_2 .
2. For a single-copy atomic read operation R_1 that has at least one **overlapping** location with a single-copy atomic write operation W_2 , if one of the **overlapping** reads generated by R_1 **reads-from** one of the **overlapping** writes generated by W_2 , then none of the **overlapping** writes generated by W_2 are **coherence-after** the corresponding **overlapping** reads generated by R_1 .

Applies to an implementation of the architecture Armv8.0-M onward.

R_{SGSJ}

For explicit memory accesses the following rules apply:

- A read that is generated by a load instruction that loads a single general-purpose register and is aligned to the size of the read in the instruction is single-copy atomic.
- A write that is generated by a store instruction that stores a single general-purpose register and is aligned to the size of the write in the instruction is single-copy atomic.
- Reads that are generated by a Load Pair instruction that loads two general-purpose registers and is aligned to the size of the load to each register are treated as two single-copy atomic reads, one for each register being loaded.
- Writes that are generated by a Store Pair instruction that stores two general-purpose registers and are aligned to the size of the store of each register are treated as two single-atomic writes, one for each register being stored.
- Load-Exclusive Pair instructions of two 32-bit quantities and Store-Exclusive Pair instructions of two 32-bit quantities are single-copy atomic.
- Reads to SIMD and floating-point registers of a single 32-bit or smaller quantity that is aligned to the size of the quantity being read are treated as single-copy atomic.
- Writes from SIMD and floating-point registers of a single 32-bit or smaller quantity that is aligned to the size of the quantity being stored are treated as single-copy atomic.

Applies to an implementation of the architecture Armv8.0-M onward. Note, FP required for Floating-point register file, S0-S31 and D0-D16. SIMD instructions require DSP.

R_{KDVX}

If MVE is implemented, the following rules apply for explicit memory accesses:

- Element reads to SIMD and floating-point registers of 32-bit or smaller elements, where each element is aligned to the size of the quantity being loaded, have each element treated as a single-copy atomic read.
- Element writes from SIMD and floating-point registers of 32-bit or smaller elements, where each element is aligned to the size of the quantity being stored, have each element treated as a single-copy atomic store.
- Reads to SIMD and floating-point registers of a 64-bit value that is 64-bit aligned in memory are treated as a pair of single-copy atomic 32-bit reads.
- Writes from SIMD and floating-point registers of a 64-bit value that is 64-bit aligned in memory are treated as a pair of single-copy atomic 32-bit writes.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R _{XDKD}	All accesses to a single byte are single-copy atomic. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{DFJD}	All other memory accesses are treated as streams of accesses to bytes, and no atomicity between accesses to bytes is ensured by the architecture. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{GQXM}	If, according to these rules, an instruction is executed as a series of accesses exceptions, including interrupts, can be taken during that sequence, regardless of the memory type being accessed. If any of these exceptions are returned from using their preferred return address, the instruction that generated the sequence of accesses is re-executed, and so any access performed before the exception was taken is repeated. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

B7.2.2 Multi-copy atomicity

I _{BCHK}	In a multiprocessing environment, writes to memory are <i>multi-copy atomic</i> if and only if all of the following are true: • All writes to the same location are observed in the same order by all Observers, although some of the Observers might not observe all of the writes. • A read of a location does not return the value of a write to that location until all Observers have observed that write. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{GJGP}	Writes to Normal memory are not required to multi-copy atomic. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LBGB}	Writes to Device memory with the Gathering attribute are not required to be multi-copy atomic. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{WHJR}	Writes to Device memory with the non-Gathering attribute that is single-copy atomic are also multi-copy atomic. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[B7.11 Device memory.](#)

[B7.9 Normal memory.](#)

[B7.18 Load-Acquire and Store-Release accesses to memory.](#)

[B7.6 Alignment behavior.](#)

[B7.2.5 Ordering constraints.](#)

B7.2.3 Ordering and observability

I _{JLGR}	The Armv8 memory model permits reordering of memory accesses. This description defines the constraints on the reordering of memory accesses using: • Register value dependencies to establish order between instructions on a PE. • Ordering constraints to establish order between accesses to a Location. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
-------------------	--

Register value dependencies

I_{DNJX}

The Armv8 memory model defines the following dependencies between instructions:

- Dependency through registers.
- Address dependency.
- Data dependency.
- Control dependency.

Applies to an implementation of the architecture Armv8.0-M onward.

Dependency through registers

R_{LNLD}

A *Dependency through registers* from first effect E_1 to a second effect E_2 exists with a PE if and only if at least one of the following applies:

- E_1 is a register write, W_1 , which has been generated by a Store Exclusive, E_2 is a register read R_2 and R_2 [reads-from-register](#) W_1 .
- E_1 and E_2 have been generated by the same instruction and E_1 is before E_2 in the Intrinsic Order of that instruction.
- There is a Dependency through registers from E_1 to a third effect E_3 , and there is a Dependency through registers from E_3 to E_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Address dependency

R_{NPWZ}

An *address dependency* from a memory read R_1 to a [Memory effect](#), RW_2 , exists if and only if there is a Dependency through registers from R_1 to a Register effect, E_3 generated by RW_2 , and E_3 affects the address part of RW_2 , and either:

- RW_2 is a memory write effect W_2 .
- RW_2 is a memory read effect, R_2 , and there is no [branching effect](#) on D_4 such that there is a Dependency through registers from R_1 to D_4 and from D_4 to R_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Data dependency

R_{FNSF}

A *data dependency* from a read R_1 to a subsequent write W_2 exists if and only if there is a register dependency from the data value returned by R_1 to the data value written by W_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Control dependency

R_{KFXC}

A *Control dependency* from a memory read R_1 to a [Memory effect](#) RW_2 exists if and only if either:

- There is a Dependency through registers from R_1 to a [branching effect](#) B_2 and B_3 is in program order before RW_2 .
- There is a Dependency through registers from R_1 to the determination of a synchronous exception on an instruction generating an effect RW_3 , and RW_2 appears in program order after RW_3 .

Applies to an implementation of the architecture Armv8.0-M onward.

B7.2.4 Ordering and observability at a Location

I_{XQBW}

Memory effects on a [Location](#) have the following relations:

- Reads-from-register.
- Reads-from.
- Local-read-successor.
- Local write successor.
- Data dependency.
- Coherence order.
- Coherence-after.
- Completion.
- Global completion.
- Overlapping accesses.
- Observed-by.
- DMB

Applies to an implementation of the architecture Armv8.0-M onward.

Reads-from-register

R_{ZQXF}

The *reads-from-register* relation couples reads and writes to the same register such that each register read is paired with exactly one register write in the execution of a program. A read R_2 Reads-from-register a write W_1 to the same register if and only if R_2 takes its data from W_1 . By construction W_1 must be in program order before R_2 and there must be no intervening write to the same register in program order between W_1 and R_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Reads-from

R_{CMSK}

A *reads-from* relation that couples reads and writes to the same [Location](#) so that each read is paired with a single write in the program. A read R_2 of a [Location](#) reads-from a write W_1 to the same [Location](#) if and only if R_2 takes its data from W_1 .

Applies to an implementation of the architecture Armv8.0-M onward.

I_{MHCW}

The reads-from relation represents a read being satisfied by a write and then returning the written data.

Applies to an implementation of the architecture Armv8.0-M onward.

Local-read-successor

R_{SZNB}

A read from R_2 of a [Location](#) is the *Local-read-successor* of a write from W_1 from the same [Observer](#) to the same [Location](#) if and only if W_1 appears in program order before R_2 and there is not a write from W_3 from the same [Observer](#) to the same [Location](#) appearing in program order between W_1 and R_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Local write successor

R_{FRJQ}

A write W_2 of a [Location](#) is the *Local write successor* of a write W_1 from the same [Observer](#) to the same [Location](#) if and only if W_1 appears in program order before W_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Data dependency

R_{HHFZ} A Data dependency from a memory read, R_1 , to a memory write, W_2 , exists if and only if there is a dependency through registers from R_1 to a [register effect](#), E_3 , generated by W_2 and E_3 affects the data part of W_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Coherence order

R_{XLMs} A coherence order relation for each [Location](#) in the program that provides a total order on all writes from all coherent Observers to that [Location](#), starting with a notional write of the initial value.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{HGfQ} The coherence order of a [Location](#) represents the order in which writes to the [Location](#) arrive at memory.

Applies to an implementation of the architecture Armv8.0-M onward.

Coherence-after

R_{VSMJ} A write W_2 to a [Location](#) is Coherence-after another write W_1 to the same [Location](#) if and only if W_2 is sequenced after W_1 in the coherence order of the [Location](#).

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WJvX} A write W_2 to a [Location](#) is Coherence-after a read R_1 of the same [Location](#) if and only if R_1 [reads-from](#) a write W_3 to the same [Location](#) and W_2 is Coherence-after W_3 .

Applies to an implementation of the architecture Armv8.0-M onward.

Completion

R_{LHTR} A read R_1 of a [Location](#) by an [Observer](#) is Complete when a write to the same [Location](#) by a different [Observer](#) is Coherence-after R_1 .

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RQRH} A write W_1 to a [Location](#) by an [Observer](#) is Complete when both:

1. A write to the same [Location](#) by a different [Observer](#) will be Coherence-after W_1 .
2. A read to the same [Location](#) by a different [Observer](#) will be Read-from a write that is Coherence-after W_1 .

Applies to an implementation of the architecture Armv8.0-M onward.

Global completion

R_{HNRN} A [reads-from](#) R_1 of a [Location](#) by an [Observer](#) is Globally complete if it is Complete and the write that it [reads-from](#) is also Complete.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{SWKN} A write W_1 to a [Location](#) by an [Observer](#) is Globally complete if it is Complete.

Applies to an implementation of the architecture Armv8.0-M onward.

Overlapping accesses

R_{QBJR} Two memory effects overlap if and only if they access the same [Location](#). Two instructions overlap if and only if one or more of their generated memory effects overlap.

Applies to an implementation of the architecture Armv8.0-M onward.

Observed-by

- R_{MSFX} A read or a write RW_1 from an **Observer** is *Observed-by* a write W_2 from a different **Observer** if and only if W_2 is **coherence-after** RW_1 .
Applies to an implementation of the architecture Armv8.0-M onward.
- R_{DLDS} A write W_1 from an **Observer** is observed-by a read R_2 from a different **Observer** if and only if R_2 **reads-from** W_1 .
Applies to an implementation of the architecture Armv8.0-M onward.
- I_{WHVK} The observed-by relation only relates accesses generated by different **Observers**.
Applies to an implementation of the architecture Armv8.0-M onward.

DMB SY

- R_{JTHS} Unless a specific Shareability domain is defined, a **DMB SY** applies to the Common shareability domain.
Applies to an implementation of the architecture Armv8.0-M onward.
- R_{FNCH} All properties that apply to **DMB SY** also apply to the corresponding **DSB**.
Applies to an implementation of the architecture Armv8.0-M onward.

B7.2.4.1 Ordering relations

- I_{WPKC} In addition to the ordering relations for a single **Location**, the Armv8 memory model also provides ordering relations to describe the ordering of memory effects to multiple **Locations**.
Applies to an implementation of the architecture Armv8.0-M onward.

Dependency-ordered-before

- R_{ZZLM} A dependency creates externally-visible order between a read and another **memory effect** generated by the same **Observer**. A read R_1 is *dependency-ordered-before* a read or write RW_2 from the same **Observer** if and only if R_1 appears in program order before RW_2 and any of the following cases apply:
- There is an address dependency or a **data dependency** from R_1 to RW_2 .
 - RW_2 is a write W_2 and there is a **control dependency** from R_1 to W_2 .
 - RW_2 is a read R_2 generated by an instruction appearing in program order after an instruction I_3 that generates a **Context synchronization event**, and there is a **dependency through registers** from R_1 to I_3 .
 - RW_2 is a write W_2 appearing in program order after a read or a write RW_3 and there is an address dependency from R_1 to RW_3 .
 - RW_2 is a **local-read-successor** R_2 of a write, W_3 , and there is an address dependency or **data dependency** from R_1 to w_3 .

Applies to an implementation of the architecture Armv8.0-M onward.

Atomic-ordered-before

- R_{NXVW} Load-Exclusive and Store-Exclusive instructions provide some ordering guarantees, even in the absence of dependencies. A read or a write RW_1 is *atomic-ordered-before* a read or a write RW_2 from the same **Observer** if and only if RW_1 appears in program order before RW_2 and either of the following cases apply:
- RW_1 is a read R_1 and RW_2 is a write W_2 so that R_1 and W_2 are generated by an atomic instruction or a successful Load-Exclusive/Store-Exclusive instruction pair to the same **Location**.

- RW_1 is a write W_1 generated by an atomic instruction or a successful Store-Exclusive instruction and RW_2 is a read R_2 generated by an instruction with Acquire or AcquirePC semantics so that R_2 [reads-from](#) W_1 .

Applies to an implementation of the architecture Armv8.0-M onward.

Single-copy-atomic-ordered-before

R_{DFDV}

A read R_1 is *single-copy-atomic-ordered-before* another read R_2 if and only if all of the following statements are true:

- R_1 and R_2 are reads from the same instruction.
- R_1 is not a [local-read-successor](#) of a write.
- R_2 is a [local-read-successor](#) of a write.

Applies to an implementation of the architecture Armv8.0-M onward.

Barrier-ordered-before

R_{XPQ}

Barrier instructions order prior [memory effects](#) before subsequent [memory effects](#) generated by the same [Observer](#). A read or a write RW_1 is *barrier-ordered-before* a read or a write RW_2 from the same [Observer](#) if and only if RW_1 appears in program order before RW_2 and any of the following cases apply:

- RW_1 appears in program order before a [DMB](#) that appears in program order before RW_2 .
- RW_1 appears in program order before an atomic instruction with both Acquire and Release semantics that appears in program order before RW_2 .
- RW_1 is a write W_1 generated by an instruction with Release semantics and RW_2 is a read R_2 generated by an instruction with Acquire semantics.
- RW_1 is a read R_1 and R_1 is generated by an instruction with Acquire semantics.
- RW_2 is a write W_2 and W_2 is generated by an instruction with Release semantics.

Applies to an implementation of the architecture Armv8.0-M onward.

Locally-ordered-before

R_{ZQLS}

Dependencies, Local write successor, Load/Store-Exclusive, atomic, and barrier instructions can be composed with an [Observer](#) to create externally-visible order. A read or write RW_1 is *locally-ordered-before* a read or write RW_2 from the same [Observer](#) if and only if any of the following apply:

- RW_1 is a write, W_1 , and RW_2 is a write, W_2 , that is equal to or generated by the same instruction as a [local-write-successor](#) of RW_1 .
- RW_1 is [dependency-ordered-before](#) RW_2 .
- RW_1 is [atomic-ordered-before](#) RW_2 .
- RW_1 is [barrier-ordered-before](#) RW_2 .
- RW_1 is locally-ordered-before RW_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Ordered-before

R_{GZXJ} An arbitrary pair of [memory effects](#) is ordered if it can be linked by a chain of ordered accesses consistent with external observation. A read or a write RW_1 is *ordered-before* a read or a write RW_2 if and only if any of the following cases apply:

- RW_1 is observed-by RW_3 which is generated by the same instruction as RW_2 .
- RW_1 is locally-ordered-before RW_2 .
- RW_1 is ordered-before a read or write that is ordered-before RW_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

I_{VPCD} For a read or write, RW_1 , from an [Observer](#) that is ordered before a read or a write, RW_2 from a different [Observer](#), the External visibility requirement requires that RW_2 is not Observed-by RW_1 . This means that architecturally-well-formed execution must not exhibit a cycle in the Ordered-before relation.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{MFCK} Informally, if a [memory effect](#), M_1 , from an [Observer](#) appears in program order before a [memory effect](#), M_2 , from the same [Observer](#), then M_1 will be seen to occur before M_2 by all Observers in the system.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.35 Context Synchronization Event](#)

[Chapter B9 Synchronization and Semaphores](#)

B7.2.5 Ordering constraints

Other-multi-copy atomic

R_{MXHD} In an *other-multi-copy atomic* system, it is required that a write from an [Observer](#), if observed by a different [Observer](#), is then observed by all other Observers that access the [Location](#) coherently. It is, however, permitted for an [Observer](#) to observe its own writes prior to making them visible to other Observers in the system.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XCGK} The Armv8 memory model is described as being other-multi-copy atomic.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{QPKH} The other-multi-copy atomic property of the Armv8 memory model is enforced by placing constraints on the possible executions of a program. Those executions that meet the constraints given by the ordering model are said to be *architecturally well-formed*. An implementation that is executing a program is only permitted to exhibit behavior consistent with an architecturally well-formed execution.

Applies to an implementation of the architecture Armv8.0-M onward.

Architecturally well-formed

R_{KNXM} An architecturally well-formed execution must satisfy both of the following requirements:

- Internal visibility.
- External visibility.

Applies to an implementation of the architecture Armv8.0-M onward.

Internal visibility requirement

R_{RZDJ}

For a read or a write RW_1 that appears in program order before a read or a write RW_2 to the same [Location](#), the internal visibility requirement requires that exactly one of the following statements is true:

- RW_2 is a write W_2 that is [Coherence-after](#) RW_1 .
- RW_1 is a write W_1 and RW_2 is a read R_2 such that either:
 - R_2 [reads-from](#) W_1 .
 - R_2 [reads-from](#) another write that is [Coherence-after](#) W_1 .
- RW_1 and RW_2 are both reads R_1 and R_2 such that R_1 [reads-from](#) a write W_3 and either:
 - R_2 [reads-from](#) W_3 .
 - R_2 [reads-from](#) another write that is [Coherence-after](#) W_3 .

Applies to an implementation of the architecture Armv8.0-M onward.

I_{JJBB}

If a [memory effect](#) M_1 from an [Observer](#) appears in program order before a [memory effect](#) M_2 from the same [Observer](#), then M_1 will be seen to occur before M_2 by that [Observer](#).

Applies to an implementation of the architecture Armv8.0-M onward.

External visibility requirement

R_{DVLW}

For a read or a write RW_1 from an [Observer](#) that is [ordered-before](#) a read or a write RW_2 from a different [Observer](#), the external visibility constraint requires that RW_2 is not observed-by RW_1 . This means that an architecturally well-formed execution must not exhibit a cycle in the [ordered-before](#) relation.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{XRFG}

If a [memory effect](#) M_1 from an [Observer](#) is ordered-before another [memory effect](#) M_2 , from a different [Observer](#), then M_1 will be seen to occur before M_2 by all [Observers](#) in the system.

Applies to an implementation of the architecture Armv8.0-M onward.

B7.2.6 Completion and endpoint ordering

I_{XSQR}

Interaction between [Observers](#) in a system is not restricted to communication through shared variables in coherent memory. For example, an [Observer](#) could configure an interrupt controller to raise an interrupt on another [Observer](#) as a form of message passing. These interactions typically involve an additional agent, which defines the instruction sequence that is required to establish communication links between different [Observers](#). When these forms of interaction are used together with shared variables, a [DSB](#) instruction can be used to enforce ordering between them.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MNQT}

A read, R_1 , to a [Location](#) is complete for a Shareability domain when any write to the same [Location](#) by an [Observer](#) within the Shareability domain will be [Coherence-after](#) R_1 .

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DXPG}

A write W_1 to a [Location](#) is complete for a Shareability domain when all of the following are true:

- Any write to the same [Location](#) by an [Observer](#) within the Shareability domain will be [Coherence-after](#) W_1 .
- Any read to the same [Location](#) by an [Observer](#) within the Shareability domain will either [reads-from](#) W_1 or [reads-from](#) a write that is [Coherence-after](#) W_1 .

Applies to an implementation of the architecture Armv8.0-M onward.

R _{GTSJ}	A cache maintenance operation is complete for a Shareability domain when the memory effects of the instruction are complete for that Shareability domain. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{RHHF}	The completion of any cache maintenance operation includes its completion on all PEs that are affected by both the operation and the DSB instruction that is required to guarantee visibility of the maintenance operation. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{GWDJ}	These completion rules mean that, for example, a cache maintenance operation that operates to the PoC completes only after memory at the PoC has been updated. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{JHGR}	Additionally, for Device-nGnRnE memory, a read or write of a Location in a memory-mapped peripheral that exhibits side-effects is complete only when the read or write both: • Can begin to affect the state of the memory-mapped peripheral. • Can trigger all associated side-effects, whether they affect other peripheral devices, PEs, or memory. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{CTMQ}	This requirement for Device-nGnRnE memory is consistent with the memory access having reached the peripheral endpoint. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

Peripherals

I _{LRMH}	This section defines a memory-mapped peripheral and the total order of reads and writes to a peripheral which is defined as the peripheral coherence order. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
-------------------	--

Memory-mapped peripheral

R _{FVCC}	A <i>memory-mapped peripheral</i> occupies a memory region of IMPLEMENTATION DEFINED size and can be accessed using load and store instructions. Memory effects to a memory-mapped peripheral can have side-effects, such as causing the peripheral to perform an action. Values that are read from addresses within a memory-mapped peripheral might not correspond to the last data value written to those addresses. As such, memory effects to a memory-mapped peripheral might not appear in the reads-from or Coherence-after relations. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
-------------------	---

Peripheral coherence order

R _{TCDN}	The <i>peripheral coherence order</i> of a memory-mapped peripheral is a total order on all reads and writes to that peripheral. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{QHFV}	The peripheral coherence order for a memory-mapped peripheral signifies the order in which accesses arrive at the endpoint. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{JWMW}	For a read or a write RW ₁ and a read or a write RW ₂ to the same peripheral, then RW ₁ will appear in the peripheral coherence order for the peripheral before RW ₂ if either of the following cases apply: • RW₁ and RW₂ are accesses using <i>Non-cacheable</i> or <i>Device</i> attributes and RW₁ is ordered-before RW₂. • RW₁ and RW₂ are accesses using Device-nGnRE or Device-nGnRnE attributes and RW₁ appears in program order before RW₂.

Applies to an implementation of the architecture Armv8.0-M onward.

Out-of-band-ordered-before

R_{CHWP} A read or a write RW_1 is *out-of-band-ordered-before* a read or a write RW_2 if and only if either of the following cases apply:

- RW_1 appears in program order before a [DSB](#) instruction that begins an IMPLEMENTATION DEFINED instruction sequence indirectly leading to the generation of RW_2 .
- RW_1 is [ordered-before](#) a read or a write RW_1 and RW_1 is out-of-band-ordered-before RW_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

R_{VSNV} If a [memory effect](#) M_1 is out-of-band-ordered-before a read or a write M_2 , then M_1 is seen to occur before M_2 by all [Observers](#).

Applies to an implementation of the architecture Armv8.0-M onward.

B7.2.7 External ordering constraints

I_{ZCPH} The Armv8 memory model provides the following three alternative representations of the external ordering constraint:

- External visibility requirement.
- External completion requirement.
- External global completion requirement.

Applies to an implementation of the architecture Armv8.0-M onward.

Completes-before order

R_{QNHS} The *Completes-before order* is a total order that corresponds to the order in which memory effects complete within the system. The following effects constitute a single entry in the completes-before order:

- Writes from the same instruction.
- Reads from the same instruction which read from external writes.
- Writes from the same instruction which read from the same internal write.

All other reads constitute distinct entries in the completes-before order.

Applies to an implementation of the architecture Armv8.0-M onward.

Completes-before

R_{JCKC} A read or write RW_1 *completes-before* a read or write RW_2 if and only if RW_1 appears in the completes-before order before RW_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Deriving Reads-from and Coherence order from the Completes-before order

I_{WPBF} The completes-before order can be used to resolve the [Reads-from](#) relation for every memory access in the system.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{LQJQ}

For a read, R_1 , of a **Location** by an **Observer**, then:

- If there is a write W_2 to the same **Location** from the same **Observer**, R_1 will read-from W_2 if all of the following are true:
 - W_2 appears in program order before R_1 .
 - R_1 completes-before W_2 .
 - There are no writes to the same **Location** appearing in program order between W_2 and R_1 .
- If there is a write W_2 to same **Location** from a different **Observer**, R_1 will read-from W_2 if all of the following are true:
 - W_2 completes-before R_1 .
 - There are no writes to the **Location** appearing in program order or in the completes-before order between W_2 and R_1 .
- If no such write exists R_1 **reads-from** the initial value of the **Location**.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{TCLS}

The Coherence order of writes to a **Location** is the order in which those writes appear in the completes-before order. The final value of each **Location** is determined by the final write to each **Location** in the completes-before order. If no such write exists for a given **Location**, the final value of the **Location** is the initial value of the **Location**.

Applies to an implementation of the architecture Armv8.0-M onward.

External completion requirement

R_{NLVB}

A read or write, RW_1 , *Globally-completes-before* a read or a write, RW_2 , if and only if any of the following are true:

- RW_1 is **locally-ordered-before** RW_2 .
- RW_2 is a read, R_2 , and R_1 is single-copy-ordered-before R_2

Applies to an implementation of the architecture Armv8.0-M onward.

B7.2.8 Globally-completes-before order

R_{ZPSM}

The *globally-completes-before order* is a total order that corresponds to the order in which memory effects globally-complete with the system. The following effects constitute a single entry in the globally-completes-before order:

- Writes from the same instruction.
- Reads from the same instruction which read from external writes.
- Reads from the same instruction which read from the same internal write.

All other reads constitute distinct entries in the globally-completes-before order.

Applies to an implementation of the architecture Armv8.0-M onward.

Globally-completes-before

R_{FDKZ}

A read or write, RW_1 , *globally-completes-before* a read or write, RW_2 , if and only if RW_1 appears in the globally-completes-before order before RW_2 .

Applies to an implementation of the architecture Armv8.0-M onward.

Deriving reads-from and coherence order from the globally-completes-before order

I_{WKHN}	The globally-completes-before order can be used to resolve the reads-from and coherence order relation for every memory access in the system. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{ZJDF}	A read, R_1, of a Location by an Observer reads-from its closest preceding write in the globally-completes-before order to the same Location. If no such write exists, then R_1 reads-from the initial value of the Location. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{GWVQ}	The coherence order of writes to a Location is the order in which those writes appear in the globally-completes-before order. The final value of each Location is determined by the final write to each Location in the globally-completes-before order. If no such write exists for a Location, the final value is the initial value of that Location. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

External global completion requirement

R_{KWDN}	For a read or write, RW_1, that is locally-ordered-before a read or a write, RW_2, the <i>external global completion requirement</i> requires that RW_1 globally-completes-before RW_2 if and only if any of the following statements are true: • RW_1 is locally-ordered-before RW_2 and either: – RW_1 is a write. – RW_1 is a read, R_1, and either: * R_1 does not locally-read-from a write. * R_1 locally-read-from a write that is locally-ordered-before RW_2. • RW_1 is a read, R_1, and RW_2 is a read, R_2, and R_1 is single-copy-ordered-before R_2. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
------------	--

B7.2.9 Memory barriers

R_{WRCT}	The Arm architecture supports out-of-order completion of instructions. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{NBQC}	The Armv8-M architecture supports the following memory barriers: • <i>Instruction Synchronization Barrier</i> (ISB). • <i>Data Memory Barrier</i> (DMB). • <i>Data Synchronization Barrier</i> (DSB). • <i>Consumption of Speculative Data Barrier</i> (CSDB). • <i>Physical Speculative Store Bypass Barrier</i> (PSSBB). • <i>Speculative Store Bypass Barrier</i> (SSBB). <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{LQXF}	The DMB and DSB memory barriers affect reads and writes to the memory system that are generated by Load/Store instructions and data or unified cache maintenance operations that are executed by the PE. Instruction fetches are not explicit accesses. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

B7.2.10 Instruction Synchronization Barrier

R_{STMG} An **ISB** ensures that all instructions that come after the **ISB** instruction in program order are fetched from the cache or memory after the **ISB** instruction has completed.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{RZFB} Using an **ISB** ensures that the effects of context-changing operations executed before the **ISB** are visible to the instructions fetched after the **ISB**.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{VJKH} Any context-changing operations appearing in program order after the **ISB** instruction only take effect after the **ISB** has been executed.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[InstructionSynchronizationBarrier\(\)](#).

[Context synchronization event](#)

B7.2.11 Data Memory Barrier

R_{MPSG} The required Shareability for a **DMB** is *Full system*, and applies to all observers in the Shareability domain.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GVDL} A **DMB** only affects memory accesses and the operation of data cache and unified cache maintenance operations, and has no effect on the ordering of any other instructions.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{HFTX} A **DMB** that ensures the completion of cache maintenance instructions has an access type of both loads and stores.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{CNSK} The principle of the **DMB** is to introduce order between memory accesses that are specified to be affected by the **DMB** supplied as arguments to the **DMB** instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RGHH} The **DMB** instruction ensures that all affected memory accesses by the PE executing the **DMB** instruction that appear in program order before the **DMB** instruction and those instructions which originate from a different PE, are observed-by each PE before any affected memory accesses that are in program order after the **DMB** are observed-by that PE.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{CLHH} The use of the **DMB** instruction between the Memory effects of instructions are **barrier-ordered-before**.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[DataMemoryBarrier\(\)](#).

[B7.14 Shareability domains](#).

B7.2.12 Data Synchronization Barrier

I_{BGJS} A **DSB** is a memory barrier that ensures that memory accesses that occur before the **DSB** instruction have completed before the completion of the **DSB** instruction. The **DSB** acts as a stronger barrier than a **DMB** instruction and all ordering that is created by a **DMB** is also generated by a **DSB**.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NKJW} The required Shareability for a **DSB** is Full system and applies to all observers in the Shareability domain.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{KMGH} A **DSB** completes when all of the following conditions apply:

- All explicit memory accesses that are observed by PEE before the **DSB** is executed and are of the required access types, and are from observers in the same required Shareability domain as PEE, are complete for the set of observers in the required Shareability domain.
- If the required access types of the **DSB** is reads and writes, then all cache and branch predictor maintenance instructions that are issued by PEE before the **DSB** are complete for the required Shareability domain.
- All explicit accesses to the **System Control Space** that result in a context altering operation issued by PEE before the **DSB** are complete.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{KMBX} No instruction that appears in program order after the **DSB** instruction can execute until the **DSB** completes.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{KNDF} The order in which a series of **implicit** and **explicit** accesses occurs is only guaranteed in one of the following scenarios or if a **DSB** instruction is executed between the accesses:

- An **implicit** read of **MPU_RNR** triggered by an **explicit access** to the **MPU_RBAR**, **MPU_RBAR_An**, **MPU_RLAR** or **MPU_RLAR_An** must occur after, and observe the updated value of, an **explicit** write to the **MPU_RNR** that occurred before the **explicit access** to the base or limit registers.
- An **implicit** read of **SAU_RNR** triggered by an **explicit access** to the **SAU_RBAR**, or **SAU_RLAR** must occur after, and observe the updated value of, an **explicit** write to the **SAU_RNR** that occurred before the **explicit access** to the base or limit registers.
- **Explicit accesses** to the PPB space in an exception handler must occur after, and observe the values of, **implicit** writes during raising and entry to that exception. For example, an **LDR** instruction executed by a BusFault handler that accesses the **BFSR** must observe the updates to the **BFSR** that occur during raising and entry to the BusFault exception.
- For a set of PPB registers that directly access common architecture state, for example the Write-one-to-clear and Write-one-to-set registers **NVIC_ICER** and **NVIC_ISER**, an **implicit** read of the common architecture state triggered by an **explicit** read must occur after and observe the value of an **explicit** write that occurred before the **explicit** read to the common architecture state.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[DataSynchronizationBarrier\(\)](#).

[B7.14 Shareability domains](#).

B7.2.13 Consumption of Speculative Data Barrier

R_{CTSR} The **CSDB** is a memory barrier that prevents instructions that appear in program order after the barrier completes from determining any part of the value of data derived from speculatively executed load instructions that appeared in program order before completion of the **CSDB** memory barrier.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.

I_{LZDK} When a **CSDB** instruction is executed, but before the **CSDB** completes and there are three instructions:

1. A load instruction speculatively executed in program order before the barrier that might or might not be architecturally executed.
2. A Conditional Move instruction that has passed its condition code check and does not have an address dependency for an input register on the speculatively executed load.
3. A load, store, data, or instruction preload appearing in program order after the barrier, which has an address dependency on the Conditional Move instruction.

The speculative execution of the load, store, data, or instruction preload does not influence the allocation of cache entries to determine any part of the value of the speculatively executed load instruction by an evaluation of the cache entries which have been allocated or evicted.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.

I_{DDTH} When a **CSDB** instruction is executed, but before the **CSDB** completes and there are three instructions:

1. A load instruction speculatively executed in program order before the barrier that might or might not be architecturally executed.
2. A Conditional Move instruction that has no dependency to pass the condition tests or for an input register on the speculatively executed load.
3. An indirect branch instruction, appearing in program order after the barrier, that is dependent on the Conditional Move instruction for the target address of the indirect branch.

The speculative execution of the indirect branch does not influence the allocation of cache entries to determine any part of the value of the speculatively executed load instruction by an evaluation of the cache entries which have been allocated or evicted.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.

R_{JWCV} A **CSDB** instruction cannot be executed speculatively.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.

I_{QZKB} A **CSDB** can be inserted speculatively and completed when it is known not to be speculative.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.

R_{WGCX} The **CSDB** instruction is not available in an implementation without the Main Extension.

Applies to an implementation of the architecture Armv8.0-M onward. Note, !M.

I_{PCSF} Arm recommends that a combination of **DSB** and an **ISB** is inserted to prevent consumption of speculative data.

Applies to an implementation of the architecture Armv8.0-M onward. Note, !M.

B7.2.14 Physical Speculative Store Bypass Barrier

I_{CCNK} The **PSSBB** prevents speculative loads from:

- Returning data older than the most recent store to the same physical address appearing in program order before the load.
- Returning data from stores using the same physical address appearing in program order after the load.

Applies to an implementation of the architecture Armv8.0-M onward.

B7.2.15 Speculative Store Bypass Barrier

I_{HWND}

The [SSBB](#) prevents speculative loads from:

- Returning data older than the most recent store to the same address appearing in program order before the load.
- Returning data from stores using the same address appearing in program order after the load.

Applies to an implementation of the architecture Armv8.0-M onward.

B7.2.16 Synchronization requirements for System Control Space

R_{SJQJ}

A [DSB](#) guarantees that all writes to the System Control Space have been completed.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NPDJ}

The execution of a [DSB](#) alone does not guarantee that the side effects of writes to the [System Control Space](#) are visible. A Context synchronization event is required for side effects of a write to the [System Control Space](#) to be visible.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{HMNM}

A Context synchronization event guarantees that the side effects of any completed writes to the System Control Space are visible after the Context synchronization event.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B8.3 The System Control Space \(SCS\).](#)

B7.3 Memory accesses

I _{XRDS}	The memory accesses that are referred to in describing the memory model are instruction fetches from memory and load or store data accesses. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LKQN}	The instruction operation uses the MemA () or MemU () helper functions. If the Main Extension is not implemented unaligned accesses using the MemU () helper functions generate an alignment fault. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{BENF}	A memory access is governed by: • Whether the access is a read or a write. • The address alignment. • Data endianness. • Memory attributes. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{FLFQ}	Memory reads that are generated by MVE instructions using MemA_MVE () are allowed to access bytes that are not explicitly accessed by the instruction if both of the following are true: • The bytes that are accessed are in a 32-byte window that is aligned to 32 bytes. • That window contains at least one byte that is explicitly accessed by the instruction. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I _{SKLB}	Arm recommends that software does not use vector load/store instructions with data in Device memory. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>
R _{LMBL}	If an MVE load or store operation results in an access to the Private Peripheral Bus (PPB) address space, within the System region of the system address map, the behavior of the accesses is CONSTRAINED UNPREDICTABLE and one of the following occurs: • It generates a Bus Fault. • The specified access to the PPB address space is performed. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>

See also:

[B7.1 Definition of the Armv8 memory model.](#)

[B7.9 Normal memory.](#)

[B7.11 Device memory.](#)

[B7.15 Memory access restrictions.](#)

[B8.2 The System region of the system address map.](#)

B7.4 Address space

R _{FFMK}	The address space is a single, flat address space of 2^{32} bytes. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SNPV}	In the address space, byte addresses are unsigned numbers in the range $0-(2^{32}-1)$. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{RGBT}	If an address calculation overflows or underflows the address space, it wraps around. Address calculations are modulo 2^{32}. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{JTKM}	Normal sequential execution cannot overflow the top of the address space, because the top of memory always has the execute-never (XN) memory attribute. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{BPMP}	One or more accesses that target or wrap around the top or bottom bytes of memory, access a sequence of words at increasing memory addresses, effectively incrementing the address by four for each load or store. If this calculation overflows the top of the address space, the result is UNPREDICTABLE. <i>Applies to an implementation of the architecture Armv8.0-M onward. Note, The encodings of some instructions require M, the encodings of some instructions require FP.</i>

See also:

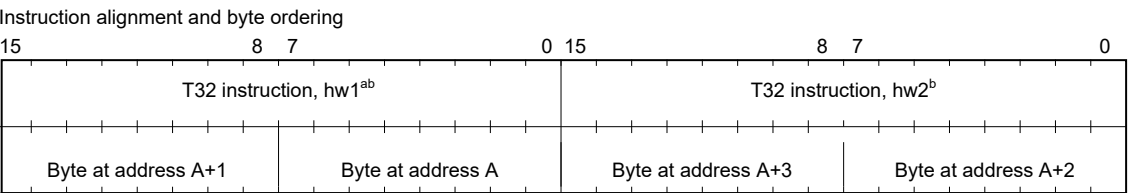
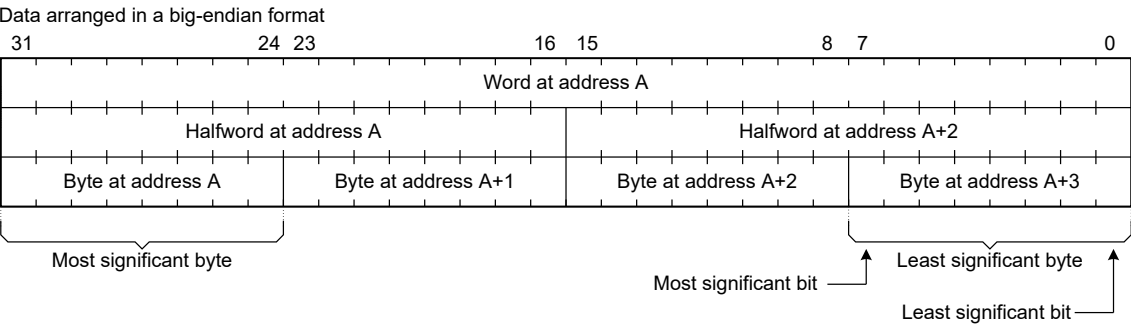
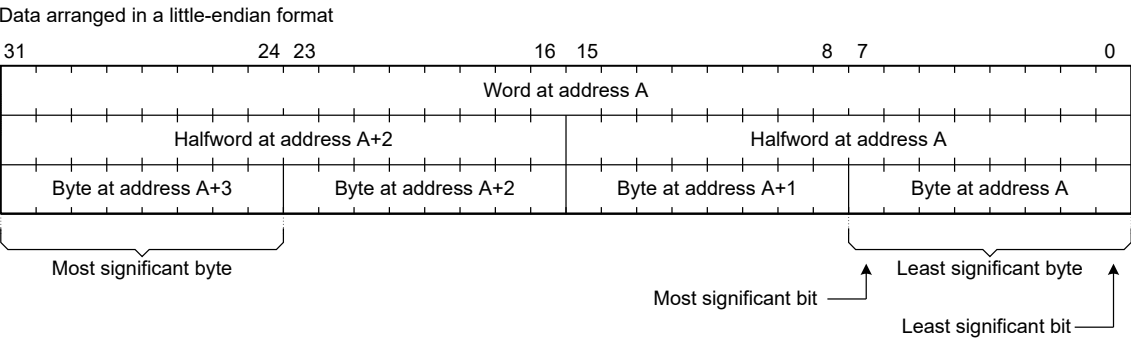
[Chapter B8 The System Address Map.](#)

B7.5 Endianness

I_CTVV

In memory:
The following figures show the relationship between:

- The word at address A.
- The halfwords at addresses A and A+2.
- The bytes at addresses A, A+1, A+2, and A+3.



- a) Bits[15:0]: this is hw 1 for a T32 instruction with a 16-bit encoding
- b) Bits[31:0]: this is hw1 and hw2 for a T32 instruction with a 32-bit encoding

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_JJQL

Instruction fetches are always little-endian, which means that the PE assumes a little-endian arrangement of instructions in memory.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_MNSB

All accesses to the Private Peripheral Bus (PPB) are always little-endian, which means that the PE assumes a little-endian arrangement of the PPB registers.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{TFKG} The endianness of data accesses is IMPLEMENTATION DEFINED, as indicated by [AIRC.R.ENDIANNESS](#).

Applies to an implementation of the architecture Armv8.0-M onward.

R_{KPCF} [AIRC.R.ENDIANNESS](#) is either:

- Implemented with a static value.
- Configured by a hardware input on reset.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XDJV} Instructions that cause a memory access that crosses the PPB boundary are CONSTRAINED UNPREDICTABLE if [AIRC.R.ENDIANNESS](#) is set to 1. The permitted behavior is one of the following:

- The instruction behaves as a NOP.
- The instruction raises an UNALIGNED UsageFault.
- If the instruction that crossed the PPB boundary was a load, the value of the destination register becomes UNKNOWN.
- If the instruction that crossed the PPB boundary was a store, the value of the memory locations accessed becomes UNKNOWN.

Applies to an implementation of the architecture Armv8.0-M onward. Note, a UsageFault requires M.

R_{QHWC} For data accesses, the following table shows the data element size that endianness applies to, for endianness conversion purposes.

Instruction class	Instructions	Element size
Load or store byte	LDR{S}B{T}, LDAB , LDAEXB , STLB , STLEXB , STRB {T}, TBB , LDREXB , STREXB	Byte
Load or store halfword	LDR{S}H{T}, LDAH , LDAEXH , STLH , STLEXH , and STRH {T}, TBH , LDREXH , STREXH	Halfword
Load or store word	LDR{T}, LDA , LDAEX , STL , STLEX , and STR {T}, LDREX , STREX , VLDR.F32 , VSTR.F32	Word
Load or store two words	LDRD (literal), LDRD (immediate), STRD (immediate), VLDR.64 , VSTR.64	Word
Load or store multiple words	LDM {IA, DB}, STM {IA, DB}, PUSH (multiple registers), POP (multiple registers), LDC , STC , VLDM VSTM , VPUSH , VPOP , BLX , BLXNS , BX , BXNS VLLDM , VLSTM	Word

Applies to an implementation of the architecture Armv8.0-M onward.

I_{WSHX} Instructions only support 32bit single copy atomicity.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XNVS} The following instructions change the endianness of data that is loaded or stored:

- [REV](#)

Reverse word (four bytes) in a register, for transforming 32-bit representations.

- [REVSH](#)

Reverse halfword and sign extend, for transforming signed 16-bit representations.

- [REV16](#)

Reverse packed halfwords in a register for transforming unsigned 16-bit representations.

*Applies to an implementation of the architecture **Armv8.0-M** onward.*

B7.6 Alignment behavior

R_{LKGV}	All instruction fetches are halfword-aligned. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{CWNS}	If the following instructions are not aligned, as indicated, any data access will generate an alignment fault: • Non halfword-aligned <code>LDAH</code>, <code>LDREXH</code>, <code>LDAEXH</code>, <code>STLH</code>, <code>STLEXH</code>, <code>STREXH</code>, <code>VLDLDR.16</code>, <code>VLDLDRH</code>, <code>VLDLDRH (vector)</code>, <code>VLD2.16</code>, <code>VLD4.16</code>, <code>VSTR.16</code>, <code>VSTRH</code>, <code>VSTRH (vector)</code>, <code>VST2.16</code>, and <code>VST4.16</code>. • Non word-aligned <code>LDREX</code>, <code>LDAEX</code>, <code>STLEX</code>, <code>STREX</code>, <code>LDRD (immediate)</code>, <code>LDRD (literal)</code>, <code>LDMIA</code>, <code>LDMDB</code>, <code>POP (multiple registers)</code>, <code>LDC (immediate)</code>, <code>LDC (literal)</code>, <code>VLDM</code>, <code>VLDR (System Register)</code>, <code>VPOP</code>, <code>LDA</code>, <code>STL</code>, <code>STRD (immediate)</code>, <code>STMIA</code>, <code>STMDB</code>, <code>PUSH (multiple registers)</code>, <code>STC</code>, <code>VLDRW</code>, <code>VLDRW (vector)</code>, <code>VLDRD (vector)</code>, <code>VLDR.32</code>, <code>VLDR.64</code>, <code>VLD2.32</code>, <code>VLD4.32</code>, <code>VSTR</code>, <code>VSTR (System Register)</code>, <code>VST2.32</code>, <code>VST4.32</code>, <code>VSTM</code>, <code>VSTRW</code>, <code>VSTRW (vector)</code>, <code>VSTRD (vector)</code>, <code>VSTR.32</code>, <code>VSTR.64</code>, and <code>VPUSH</code>. • Non doubleword-aligned <code>VLLDM</code>, and <code>VLSTM</code>. <i>Applies to an implementation of the architecture Armv8.0-M onward. Note, See the instruction descriptions for the required extensions.</i>
R_{MHCM}	If <code>CCR.UNALIGN_TRP</code> is set to 1, the following are unaligned data accesses that generate an alignment fault: • Non halfword-aligned <code>LDR{S}H{T}</code>, and <code>STRH{T}</code>. • Non halfword-aligned <code>TBH</code>. • Non word-aligned <code>LDR{T}</code>, and <code>STR{T}</code>. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{JLGS}	Unaligned accesses are only supported if the Main Extension is implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R_{WCVX}	Accesses to Device memory are always aligned. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{PZTT}	If the Main Extension is not implemented, unaligned accesses generate an alignment HardFault. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - !M.</i>
$R_{RND S}$	Alignment faults are synchronous and generate an UNALIGNED UsageFault. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R_{BNBX}	The CONSTRAINED UNPREDICTABLE behavior of unaligned loads and stores is one of the following: • Generate an UNALIGNED UsageFault. • Perform the specified load or store to the unaligned Location. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R_{LPVP}	Unaligned loads and stores perform the specified load and store to the unaligned Location. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i> See also: B7.9 Normal memory. B7.11 Device memory.

B7.7 Concurrent modification and execution of instructions

I _{TFGC}	The Armv8 architecture limits the set of instructions that can be executed by one thread of execution as they are being modified by another thread of execution without requiring explicit synchronization. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XWVK}	Unless otherwise stated, concurrent modification and execution of instructions results in a CONSTRAINED UNPREDICTABLE choice of any behavior that can be achieved by executing any sequence of instructions from the same Security state or the same Privilege level. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{BFPB}	For instructions that can be concurrently modified, the PE executes either: • The original instruction. • The modified instruction. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{NNQK}	A 16-bit instruction can be concurrently modified, where the 16-bit instruction before modification and the 16-bit modification is any of the following: • B. • BX. • BLX. • BKPT. • NOP. • SVC. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{KMZG}	The hw1 of a 32-bit BL immediate instruction can be concurrently modified to the most significant halfword of another BL immediate instruction. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HKGP}	The hw1 of a 32-bit BL immediate instruction can be concurrently modified to a 16-bit B, BLX, BKPT, or SVC instruction. This modification also works in reverse. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{FGBT}	The hw2 of a 32-bit BL immediate instruction can be concurrently modified to the hw2 of another BL instruction with a different immediate. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{NTVD}	The hw2, of a 32-bit B immediate instruction with a condition field can be concurrently modified to the hw2 of another 32-bit B immediate instruction with a condition field with a different immediate. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{CMZX}	The hw2 of a 32-bit B immediate instruction without a condition field can be concurrently modified to the hw2 of another 32-bit B immediate instruction without a condition field. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[B7.5 Endianness.](#)

B.

BL.

BLX, BLXNS.

B7.8 Access rights

R_{JTGX} The possible results of an instruction fetch are described in *Arm® Armv8-M Faults on Instruction Fetch* (ARM DDI 0624).

Applies to an implementation of the architecture Armv8.0-M onward.

R_{JHYT} If the [EPSR.T](#) is zero it is IMPLEMENTATION DEFINED whether the instruction opcode is treated as a halfword or word on instruction fetch, with respect to fetch faults.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.9 Exception numbers and exception priority numbers.](#)

[Chapter B10 The Armv8-M Protected Memory System Architecture.](#)

Arm® Armv8-M Faults on Instruction Fetch (ARM DDI 0624).

B7.9 Normal memory

I _{NVRF}	Memory locations that are <i>idempotent</i> have the following properties: • Read accesses can be repeated with no side effects. • Repeated read accesses return the last value that is written to the resource being read. • Read accesses can fetch additional memory locations with no side-effects. • Write accesses can be repeated with no side-effects, if the contents of the Location that is accessed are unchanged between the repeated writes or as the result of an exception. • Unaligned accesses can be supported. • Accesses can be merged before accessing the target memory system. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{HKLB}	As a consequence of the properties speculative reads can occur to a Normal memory location, combined with the possibility of intermediate buffering and forwarding of data means that mapping of non-idempotent memory locations as Normal memory can cause memory accesses to such locations to return UNKNOWN values, and there may be UNPREDICTABLE effects on memory-mapped peripherals mapped in such a way. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{QGCF}	The PE is permitted to treat regions of memory assigned the memory type Normal memory as idempotent. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{CGJX}	Normal memory can be marked as Cacheable or Non-cacheable. Normal memory is assigned Cacheability attributes. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LCPJ}	Normal Non-cacheable memory is always treated as shareable. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{PKXL}	Speculative data accesses to Normal memory are permitted. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{WLVR}	A write to Normal memory completes in finite time. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{WLCV}	A write to a Non-cacheable Normal memory Location reaches the endpoint for that Location in the memory system in finite time. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SNTN}	Writes to a memory location with the Normal memory type that are either Non-cacheable or Write-through cacheable for both the Inner and Outer cacheability must reach the endpoint for that location in the memory system in finite time. Two writes to the same location where at least one is using the Normal memory type might be merged before they reach the endpoint unless there is an ordered-before relationship between the two writes. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{NHFQ}	For multi-register Load/Store instructions that access Normal memory, the architecture does not define the order in which the registers are accessed. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{CFHV}	There is no requirement for the memory system beyond the PE to be able to identify the size of the elements accessed. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

R_{JDSF}

If non-idempotent memory locations are mapped with the Normal memory type, then:

- Speculative read accesses can cause accesses to the non-idempotent memory locations that would not occur as part of a simple sequential execution, so corrupting the state of the non-idempotent memory location.
- Writes to a non-idempotent memory locations might be merged or split meaning that the number and size of writes seen by the memory location might not be the number and size that occur as part of a simple sequential execution, so corrupting the state of the non-idempotent memory location.

*Applies to an implementation of the architecture **Armv8.0-M** onward.*

See also:

[B7.3 Memory accesses.](#)

[B7.14 Shareability domains.](#)

[B7.10 Cacheability attributes.](#)

[B7.17 Load-Exclusive and Store-Exclusive accesses to Normal memory.](#)

[MAIR_ATTR, Memory Attributes Indirection Register Attributes.](#)

B7.10 Cacheability attributes

R_{KXJV} The architecture provides Cacheability attributes that are defined independently for each of two conceptual levels of cache:

- The Inner cache.
- The Outer cache.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XRWS} The Cacheability attributes are:

- Non-cacheable.
- Write-Through Cacheable.
- Write-Back Cacheable.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{QXW} It is IMPLEMENTATION DEFINED whether Write-Through Cacheable and Write-Back Cacheable can have the additional attribute Transient or Non-transient.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{LDXP} The Transient attribute is a [memory hint](#) that indicates that the benefit of caching is for a short period. The architecture does not define what is meant by a *short period*.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CFKN} Cacheability attributes other than Non-cacheable can be complemented by the following cache allocation hints, which are independent for read and write accesses:

- Read-Allocate, Transient Read-Allocate, or No Read-Allocate.
- Write-Allocate, Transient Write-Allocate, or No Write-Allocate.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DRTR} The architecture does not require an implementation to make any use of cache allocation hints.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{FQSS} Any cacheable Normal memory region is treated as Read-Allocate, No Write-Allocate unless it is explicitly assigned other cache allocation hints.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{FRVF} A Cacheable [Location](#) with no Read-Allocate and no Write-Allocate hints is not the same as a Non-cacheable [Location](#). A Non-cacheable [Location](#) has coherency guarantees for all [Observers](#) within the system that do not apply to a [Location](#) that is Cacheable, no Read-Allocate, no Write-Allocate.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{FTKW} All data accesses to Non-cacheable Normal memory locations are data coherent to all [Observers](#).

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.9 Normal memory](#).

B7.11 Device memory

I _{BXHS}	Device memory is a <i>memory type</i> that is assigned to regions of memory where accesses can have side effects. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{WTZL}	Device memory is not cacheable. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LDDN}	Device memory is always treated as shareable. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{PQXS}	Speculative data accesses cannot be made to Device memory. However, for instructions that access a sequence of word-aligned words the accesses might occur multiple times. This includes instructions that cause the stacking or unstacking of, for example, a FNC_RETURN stack frame that require access to a sequence of word-aligned words. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{NXFX}	When MVE is implemented accesses to Device memory by vector load/store instructions are permitted to make multiple accesses to the same location. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R _{CSKG}	Any unaligned access to Device memory generates an UNALIGNED UsageFault exception. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{YMTK}	Device memory is assigned a combination of <i>Device memory attributes</i>. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LFTG}	A write to Device memory completes in finite time. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{FSCD}	A write to a Device memory location reaches the endpoint for that location in the memory system in finite time. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{VDMB}	Two writes of Device memory type, to the same location might be merged before they reach the endpoint unless either of the following conditions apply: • Both writes have the non-Reordering attribute. • There is an ordered-before relationship between the two writes. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{GTTO}	A completed write to a Device memory location is globally observed for the Shareability domain in finite time without the requirement for cache maintenance instructions or barriers. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XMCH}	If the content of a Device memory location changes without a direct write to the location, the change is observed for the Shareability domain in finite time. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{KJHG}	For an instruction fetch from Device memory, if a branch causes the Program Counter (PC) to point to an area of memory that is not marked as execute-never, the implementation can either: • Treat the fetch as if it is to a location in Normal Non-cacheable memory. • Take an IACCVIOL MemManage fault.

Applies to an implementation of the architecture Armv8.0-M onward. Note, a MemManage fault requires M.

R_{DFJX}

There is no requirement for the memory system beyond the PE to be able to identify the size of the elements that are accessed, for instructions that load the following from Device memory:

- More than one general-purpose register.
- One or more registers from the floating-point register file.

Applies to an implementation of the architecture Armv8.0-M onward. Note, FP or, from Armv8.1-M MVE required for floating-point register file.

R_{KVHT}

For an LDM, STM, LDRD, or STRD instruction with a register list that includes the PC, the architecture does not define the order in which the registers are accessed.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{SFPK}

For an LDM, STM, VLDM, VSTM, VPOP or VPUSH instruction with a register list that does not include the PC, all registers are accessed in the order that they appear in the register list, for Device memory with the non-Reordering attribute.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.3 Memory accesses.](#)

[B7.13 Shareability attributes.](#)

[B7.12 Device memory attributes.](#)

[B7.14 Shareability domains.](#)

B7.12 Device memory attributes

R_{VNSJ} Each Device memory region is assigned a combination of Device memory attributes. The attributes are:

Gathering, G and nG: The *Gathering* and *non-Gathering* attributes.

Reordering, R and nR: The *Reordering* and *non-Reordering* attributes.

Early Write Acknowledgement, E and nE: The *Early Write Acknowledgement* and *no Early Write Acknowledgement* attributes.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CFFC} Each Device memory region is assigned one of the combinations in the following table:

Memory Ordering	Name	nG	nR	nE	G	R	E
Strong	Device-nGnRnE	Y	Y	Y	-	-	-
↓	Device-nGnRE	Y	Y	-	-	-	Y
↓	Device-nGRE	Y	-	-	-	Y	Y
Weak	Device-GRE	-	-	-	Y	Y	Y

Applies to an implementation of the architecture Armv8.0-M onward.

R_{LJKD} Weaker memory can be accessed according to the rules specified for stronger memory:

- Memory with the:
 - G attribute can be accessed according to the rules specified for the nG attribute.
 - nG attribute cannot be accessed according to the rules specified for the G attribute.
- Memory with the:
 - R attribute can be accessed according to the rules specified for the nR attribute.
 - nR attribute cannot be accessed according to the rules specified for the R attribute.

Because the nE attribute is a hint:

- An implementation is permitted to perform an access with the E attribute in a manner consistent with the requirements specified by the nE attribute.
- An implementation is permitted to perform an access with the nE attribute in a manner consistent with the relaxations allowed by the E attribute.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{FJXX} For Device-GRE and Device-nGRE memory, the use of barriers is required to order accesses.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{PVCY} Memory accesses that are generated by vector instructions that target any type of Device memory operate as if the access had targeted a Device-GRE region.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [MVE](#).

See also:

- [B7.12.1 Gathering and non-Gathering Device memory attributes.](#)
- [B7.12.2 Reordering and non-Reordering Device memory attributes.](#)
- [B7.12.3 Early Write Acknowledgement and no Early Write Acknowledgement Device memory attributes.](#)
- [B7.11 Device memory.](#)

B7.12.1 Gathering and non-Gathering Device memory attributes

G attribute

R_{DBSX}

If multiple accesses of the same type, read or write, are to:

- The same location, with the G attribute, they can be merged into a single transaction.
- Different locations, all with the G attribute, they can be merged into a single transaction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{KCMX}

Gathering of accesses that are separated by a memory barrier is not permitted.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{JSRD}

Gathering of accesses that are generated by a Load-Acquire/Store-Release is not permitted.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MGKJ}

A read can come from intermediate buffering of a previous write if:

- The accesses are not separated by a [DMB](#) or [DSB](#) barrier.
- The accesses are not separated by any other ordering construction that requires that the accesses are in order, for example a combination of Load-Acquire and Store-Release.
- The accesses are not generated by a Store-Release instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{SRDS}

The architecture only defines programmer visible behavior. Therefore, if a programmer cannot tell whether Gathering has occurred, Gathering can be performed.

Applies to an implementation of the architecture Armv8.0-M onward.

nG attribute

R_{GVTf}

Multiple accesses to a [Location](#) with the nG attribute cannot be merged into a single transaction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BTWD}

A read of a [Location](#) with the nG attribute cannot come from a cache or a buffer, but comes from the endpoint for that address in the memory system.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.18 Load-Acquire and Store-Release accesses to memory.](#)

B7.12.2 Reordering and non-Reordering Device memory attributes

R attribute

R_{RPTB}

This attribute imposes no restrictions or relaxations.

Applies to an implementation of the architecture Armv8.0-M onward.

nR attribute

R_{DFXL}

If the access is to a:

- Peripheral, it arrives at the peripheral in program order. If there is a mixture of accesses to Device nGnRE and Device-nGnRnE in the same peripheral, these accesses occur in program order.
- Non-peripheral, this attribute imposes no restrictions or relaxations.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{BDWB}

The IMPLEMENTATION DEFINED size of the [single peripheral](#) is the same as applies for the ordering guarantee that is provided by the [DMB](#) instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NDHC}

The non-Reordering attribute does not require any additional ordering, other than the ordering that applies to Normal memory, between:

- Accesses with the non-Reordering attribute and accesses with the Reordering attribute.
- Accesses with the non-Reordering attribute and accesses to Normal memory.
- Accesses with the non-Reordering attribute and accesses to different peripherals of IMPLEMENTATION DEFINED size.

Applies to an implementation of the architecture Armv8.0-M onward.

B7.12.3 Early Write Acknowledgement and no Early Write Acknowledgement Device memory attributes

E attribute

R_{PVSH}

The E attribute imposes no restrictions or relaxations.

Applies to an implementation of the architecture Armv8.0-M onward.

nE attribute

R_{FWFR}

Assigning the nE attribute recommends that only the endpoint of the write access returns a write acknowledgment of the access, and that no earlier point in the memory system returns a write acknowledgment.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{FQWQ}

The E attribute is treated as a hint. Arm strongly recommends that this hint is not ignored by a PE, but is made available for use by the system.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.2.9 Memory barriers.](#)

B7.13 Shareability attributes

R_{CJRF}

Each Normal cacheable memory region is assigned one of the following Shareability attributes:

- *Non-shareable.*
- *Inner-shareable.*
- *Outer-shareable.*

Applies to an implementation of the architecture Armv8.0-M onward.

R_{PDVV}

For Non-shareable memory, hardware is not required to make data accesses by different observers, other than the Debug Access Port, coherent. If a number of observers share the memory, cache maintenance instructions, in addition to the barrier operations that are required to ensure memory ordering, can ensure that the presence of caches does not lead to coherency issues.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.3 Memory accesses.](#)

[B7.9 Normal memory.](#)

[B7.11 Device memory.](#)

[B7.14 Shareability domains.](#)

[B7.27 Cache maintenance operations.](#)

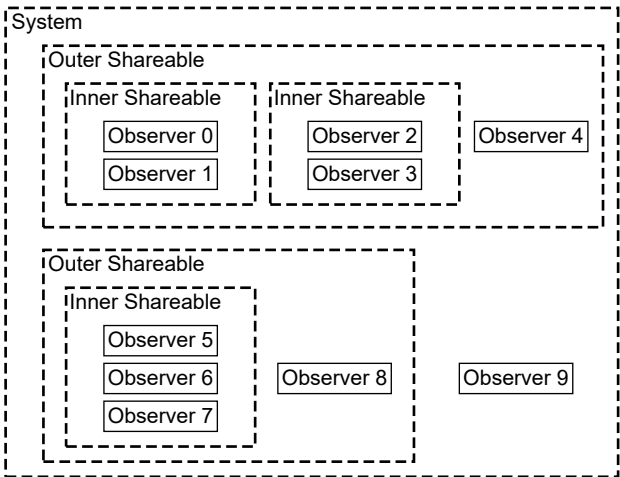
B7.14 Shareability domains

R_{JMHL} There are two conceptual Shareability domains:

- The Inner Shareability domain.
- The Outer Shareability domain.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{XQWM} The following diagram shows the Shareability domains:



Applies to an implementation of the architecture Armv8.0-M onward.

R_{MCPs} All observers in an Inner Shareability domain are data coherent for data accesses to memory that has the *Inner-shareable Shareability attribute*.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{SVCR} All observers in an Outer Shareability domain are data coherent for data accesses to memory that has the *Outer-shareable Shareability attribute*.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{JMFS} Each **Observer** is a member of only a single Inner Shareability domain.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BNWH} Each **Observer** is a member of only a single Outer Shareability domain.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{FVBG} All members of the same Inner Shareability domain are always members of the same Outer Shareability domain.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WFMV} Accesses to a shareable **Location** are coherent within the Shareability domain of that **Location**.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DHJF} An Inner Shareability domain is a subset of an Outer Shareability domain, although it is not required to be a proper subset.

Applies to an implementation of the architecture Armv8.0-M onward.

R_XHJL

Hardware is required to ensure coherency and ordering within the Shareability domain if all of the following apply:

- Before writing to a [Location](#) not using the Write-Back attribute, a [Location](#) in the caches that might have been written with the Write-Back attribute by an agent has been invalidated or cleaned.
- After writing the [Location](#) with the Write-Back attribute, the [Location](#) has been cleaned from the caches to make the write visible to external memory.
- Before reading the [Location](#) with a cacheable attribute, the cache [Location](#) has been invalidated, or cleaned and invalidated.
- A `DMB` barrier instruction has been executed, with a scope that applies to the common Shareability of the accesses, between any accesses to the same [Location](#) that use different attributes.

*Applies to an implementation of the architecture **Armv8.0-M** onward.*

See also:

[B7.2.3 Ordering and observability.](#)

[B7.13 Shareability attributes.](#)

B7.15 Memory access restrictions

R_{LTWZ}

For two explicit memory read effects to any two adjacent [locations](#) in memory, p and $p + I$, generated by the same instruction, and for two explicit memory write effects to any two adjacent [locations](#) in memory, p and $p + I$, that are generated by the same instruction:

- The locations p and $p + I$ must have the same memory type and Shareability attributes, otherwise the results are a CONSTRAINED UNPREDICTABLE choice of the following:
 - Each memory access that is generated by the instruction uses the memory type and Shareability attribute that is associated with its own [location](#).
 - The instruction executes as a NOP.
 - The instruction generates an UNALIGNED UsageFault.
- Except for possible differences in cache allocation hints, Arm deprecates having different Cacheability attributes for accesses to any two bytes that are generated by the same instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BFKS}

If the accesses of an instruction that cause multiple accesses to any type of Device memory cross the boundary of a memory region then the behavior is a CONSTRAINED UNPREDICTABLE choice of the following:

- All memory accesses that are generated by the instruction are performed as if the presence of the boundary had no effect on memory accesses.
- All memory accesses that are generated by the instruction are performed as if the presence of the boundary had no effect on memory accesses, except that there is no guarantee of ordering between memory accesses.
- The instruction executes as a NOP.
- The instruction generates an UNALIGNED UsageFault.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{YZWN}

Speculative instruction fetches can be made to memory, including Device memory, unless the [Location](#) is marked as execute-never.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.3 Memory accesses](#).

B7.16 Mismatched memory attributes

R_{XHTK} Memory locations are accessed with *mismatched attributes* if all accesses to the [Location](#) do not use a common definition of all the following memory attributes of that [Location](#):

- Memory type - Device or Normal.
- Shareability.
- Cacheability, for the same level of the Inner or Outer cache, but excluding any cache allocation hints.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{VKHJ} When a memory [Location](#) is accessed with mismatched attributes, the only permitted effects are one or more of the following:

- Uniprocessor semantics for reads and writes to that memory [Location](#) might be lost. This means:
 - A read of the memory [Location](#) by one agent might not return the value that was most recently written to that memory [Location](#) by the same agent.
 - Multiple writes to the memory [Location](#) by one agent with different memory attributes might not be ordered in program order.
- There might be a loss of coherency when multiple agents attempt to access a memory [Location](#).
- There might be a loss of the properties that are derived from the memory type.
- If all Load-Exclusive/Store-Exclusive instructions that are executed across all threads to access a given memory [Location](#) do not use consistent memory attributes, the exclusive monitor state becomes UNKNOWN.
- Bytes that are written without the Write-Back cacheable attribute and that are within the same Write-Back granule as bytes that are written with the Write-Back cacheable attribute might have their values reverted to the old values as a result of cache Write-Back.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NJLB} The loss of the properties that are associated with mismatched memory type attributes refers only to the following properties of Device memory that are additional to the properties of Normal memory:

- Prohibition of speculative read accesses.
- Prohibition on Gathering.
- Prohibition on Reordering.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{HCCD} Any agent that reads a memory [Location](#) with mismatched attributes using the same common definition of the Shareability and Cacheability attributes is guaranteed to access it coherently, to the extent required by that common definition of the memory attributes, only if all the following conditions are met:

- All aliases to the memory [Location](#) with write permission both use a common definition of the Shareability and Cacheability attributes for the memory [Location](#), and have the Inner Cacheability attribute the same as the Outer Cacheability attribute.
- All aliases to a memory [Location](#) use a definition of the Shareability attributes that encompasses all the agents with permission to access the [Location](#).

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GBKH} The possible permitted effects that are caused by mismatched attributes for a memory [Location](#) are defined more precisely if all the mismatched attributes define the memory [Location](#) as one of:

- Any Device memory type.
- Normal Inner Non-cacheable, Outer Non-cacheable memory.

In these cases, the only permitted software-visible effects of the mismatched attributes are one or more of the following:

- Possible loss of properties that are derived from the memory type when multiple agents attempt to access the memory [Location](#).
- Possible reordering of memory transactions to the same memory [Location](#) with different memory attributes, potentially leading to a loss of coherency or uniprocessor semantics. Any possible loss of coherency or uniprocessor semantics can be avoided by inserting [DMB](#) barrier instructions between accesses to the same memory [Location](#) that might use different attributes.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{VVBS}

If the mismatched attributes for a [Location](#) mean that multiple cacheable accesses to the [Location](#) might be made with different Shareability attributes, then ordering and coherency are guaranteed only if:

- Each PE that accesses the [Location](#) with a cacheable attribute performs a clean and invalidate of the [Location](#) before and after accessing that [Location](#).
- A [DMB](#) barrier with scope that covers the full Shareability of the accesses is placed between any accesses to the same memory [Location](#) that use different attributes.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{VCXW}

If multiple agents attempt to use Load-Exclusive or Store-Exclusive instructions to access a [Location](#), and the accesses from the different agents have different memory attributes associated with the [Location](#), the exclusive monitor state becomes UNKNOWN.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{ZKKV}

All cache maintenance instructions and memory transactions must be completed, or ordered by the use of barrier operations, if they are not naturally ordered by the use of a common address.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{TPWG}

Arm strongly recommends that software does not use mismatched attributes for aliases of the same [Location](#). An implementation might not optimize the performance of a system that uses mismatched aliases.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[Chapter B10 The Armv8-M Protected Memory System Architecture.](#)

[B7.14 Shareability domains.](#)

[B7.10 Cacheability attributes.](#)

[B7.11 Device memory.](#)

[B7.9 Normal memory.](#)

[B7.17 Load-Exclusive and Store-Exclusive accesses to Normal memory.](#)

B7.17 Load-Exclusive and Store-Exclusive accesses to Normal memory

R_{KDWC}

For Normal memory that is:

- Non-shareable, it is IMPLEMENTATION DEFINED whether Load-Exclusive and Store-Exclusive instructions take account of the possibility of accesses by more than one [Observer](#).
- Shareable, Load-Exclusive, and Store-Exclusive instructions take account of the possibility of accesses by more than one [Observer](#).

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.9 Normal memory](#).

[B7.3 Memory accesses](#).

B7.18 Load-Acquire and Store-Release accesses to memory

I_{VXGZ}

The principle of a Load-Acquire instruction is to introduce order between the memory access generated by the Load-Acquire instruction and the memory accesses appearing in program order after the Load-Acquire instruction, ensuring that the memory access generated by the Load-Acquire instruction is all of the following:

- Observed-by each PE.
- Observed before any of the memory accesses appearing in program order after the Load-Acquire instruction are observed by that PE.
- Observed to the extent that each PE is required to observe the access coherently.

The Load-Acquire instructions create order between the Memory effects of instructions as described in the definition of [barrier-ordered-before](#).

Applies to an implementation of the architecture Armv8.0-M onward.

I_{ZDWZ}

The principle of a Store-Release instruction is to introduce order between the memory accesses generated by the PE, PEe, executing the Store-Release instruction, together with those that originate from a different PE, to the extent that PEe is required to observe them coherently, Observed-by the PEe before executing the Store-release.

The use of Store-Release instructions create order between the Memory effects of instructions as described in the definition of [barrier-ordered-before](#).

Applies to an implementation of the architecture Armv8.0-M onward.

I_{VVTX}

The following table summarizes the Load-Acquire/Store-Release instructions:

Data type	Load-Acquire	Store-Release	Load-Acquire Exclusive	Store-Release Exclusive
32-bit word	LDA	STL	LDAEX	STLEX
16-bit halfword	LDAH	STLH	LDAEXH	STLEXH
8-bit byte	LDAB	STLB	LDAEXB	STELXB

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XBRM}

A Store-Release followed by a Load-Acquire is observed in program order by each [Observer](#) within the Shareability domain of the memory address being accessed by the Store-Release and the memory address being accessed by the Load-Acquire.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RRFK}

For a Load-Acquire, observers in the Shareability domain of the address that is accessed by the Load-Acquire observe accesses in the following order:

1. The read caused by the Load-Acquire.
2. Reads and writes caused by loads and stores that appear in program order after the Load-Acquire for which the Shareability of the address that is accessed by the load or store requires that the [Observer](#) observes the access.

There are no other ordering requirements on loads or stores that appear before the Load-Acquire.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WLWT}

For a Store-Release, observers in the Shareability domain of the address that is accessed by the Store-Release observe accesses in the following order:

1. All of the following for which the Shareability of the address that is accessed requires that the [Observer](#) observes the access:
 - Reads and writes caused by loads and stores that appear in program order before the Store-Release.
 - Writes that were observed by the PE executing the Store-Release before it executed the Store-Release.

2. The write caused by the Store-Release.

There are no other ordering requirements on loads or stores that appear in program order after the Store-Release.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{HCKR} All Store-Release instructions are multi-copy atomic when they are observed with Load-Acquire instructions.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DGXR} A Load-Acquire to an address in a memory-mapped peripheral of an arbitrary system-defined size that is defined as any type of Device memory access ensures that all memory accesses using Device memory types to the same memory-mapped peripheral that are architecturally required to be observed after the Load-Acquire will arrive at the memory-mapped peripheral after the memory access of the Load-Acquire.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CKRC} A Store-Release to an address in a memory-mapped peripheral of an arbitrary system-defined size that is defined as any type of Device memory access ensures that all memory accesses using Device memory types to the same memory-mapped peripheral that are architecturally required to be observed before the Store-Release will arrive at the memory-mapped peripheral before the memory access of the Store-Release.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GJHK} If a Load-Acquire to a memory address in a memory-mapped peripheral of an arbitrary system-defined size that is defined as any type of Device memory access has observed the value that is stored to that address by a Store-Release, then any memory access to the memory-mapped peripheral that is architecturally required to be ordered before the memory access of the Store-Release will arrive at the memory-mapped peripheral before any memory access to the same peripheral that is architecturally required to be ordered after the memory access of the Load-Acquire.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WRLC} Load-Acquire and Store-Release access only a single data element.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{KCTN} Load-Acquire and Store-Release accesses are single-copy atomic.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BXRP} If a Load-Acquire or Store-Release instruction accesses an address that is not aligned to the size of the data element being accessed, the access generates an alignment fault.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NVRJ} A Store-Release Exclusive instruction only has the release semantics if the store is successful.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.14 Shareability domains.](#)

[B7.11 Device memory.](#)

B7.19 Caches

I_JSPB	When a memory Location is marked with a Normal Cacheable memory attribute, determining whether a copy of the memory Location is held in a cache can depend on many aspects of the implementation, such as the following factors: • The size, line length, and associativity of the cache. • The cache allocation algorithm. • Activity by other elements of the system that can access the memory. • Speculative instruction fetching algorithms. • Speculative data fetching algorithms. • Interrupt behaviors. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_QGSG	An implementation can include multiple levels of cache, up to a maximum of seven levels, in a hierarchical memory system. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I_STRV	The lower the cache level, the closer the cache is to the PE. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_PDSR	Entries for addresses with a Normal cacheable attribute can be allocated to an enabled cache at any time. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_JGBL	The allocation of a memory address to a cache Location is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_SBGJ	A cache entry covers at least 16 bytes and no more than 2KB of contiguous address space, aligned to its size. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_XXBW	Where a breakdown in coherency can occur, data coherency of the caches is controlled in an IMPLEMENTATION DEFINED manner. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_JVJN	The architecture cannot guarantee whether: • A memory Location that is present in the cache remains in the cache. • A memory Location that is not present in the cache is brought into the cache. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_PHWM	If the cache is disabled, no new allocation of memory locations into the cache occurs. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_LJQB	The allocation of a memory Location into a cache cannot cause the most recent value of that memory Location to become invisible to an Observer, if it had previously been visible to that Observer. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_QRLS	If the cache is enabled, it is guaranteed that no memory Location that does not have a cacheable attribute is allocated into the cache. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_XXVH	If the cache is enabled, it is guaranteed that no memory Location is allocated to the cache if the access permissions for that Location are so that the Location cannot be accessed by reads and cannot be accessed by writes. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

R_{SCKQ}	Any cached memory Location is not guaranteed to remain incoherent with the rest of memory. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{ROXN}	If an implementation permits cache hits when the Cacheability control fields force all memory locations to be treated as Non-cacheable, then the cache initialization routine: • Provides a mechanism to ensure the correct initialization of the caches. • Is documented clearly as part of the documentation of the device. In particular, if an implementation permits cache hits when the Cacheability controls force all memory locations to be treated as Non-cacheable, and the cache contents are not invalidated at reset, the initialization routine avoids any possibility of running from an uninitialized cache. It is acceptable for an initialization routine to require a fixed instruction sequence to be placed in a restricted range of memory. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{WDBP}	It is UNPREDICTABLE whether the Location is returned from cache or from memory when: • The Location is not marked as cacheable but is contained in the cache. This situation can occur if a Location is marked as Non-cacheable after it has been allocated into the cache. • The Location is marked as cacheable and might be contained in the cache, but the cache is disabled. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{NDNN}	The architecture allows copies of control values or data values to be cached. The existence of such copies can lead to CONSTRAINED UNPREDICTABLE behavior, if the cache has not been correctly invalidated following a change of the control or data values. Unless otherwise stated, the behavior of the PE is consistent with: • The old value. • The new value. • An amalgamation of the old and new values. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I_{BMPQ}	The choice between the behaviors might, in some implementations, vary for each use of a control or data value. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[B7.20 Cache identification.](#)

[B7.23 Cache enabling and disabling.](#)

[B7.10 Cacheability attributes.](#)

[B7.24 Cache behavior at reset.](#)

[B7.28 Ordering of cache maintenance operations.](#)

[B7.16 Mismatched memory attributes.](#)

B7.20 Cache identification

R_{WBGH}	A PE controls the implemented caches using: • A single Cache Type Register, CTR. • A single Cache Level ID Register, CLIDR. • A single Cache Size Selection Register, CSSELR. • For each implemented cache, across all levels of caching, a Cache Size Identification Register, CCSIDR. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{XJTL}	The number of levels of cache is IMPLEMENTATION DEFINED and can be determined from the Cache Level ID Register. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I_{PPSB}	Cache sets and Cache ways are numbered from 0. Usually the set number is an IMPLEMENTATION DEFINED function of an address. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

B7.21 Cache visibility

R_{QLVB} A completed write to a memory [Location](#) that is Non-cacheable or [Location](#) Write-Through Cacheable for a level of cache made by an [Observer](#) accessing the memory system inside the level of cache is visible to all observers accessing the memory system outside the level of cache without the need of explicit cache maintenance.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RCHC} A completed write to a memory [Location](#) that is Non-cacheable for a level of cache made by an [Observer](#) accessing the memory system outside the level of cache is visible to all observers accessing the memory system inside the level of cache without the need of explicit cache maintenance.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.10 Cacheability attributes](#).

B7.22 Cache coherency

R_{NNDJ}

Data coherency of caches is ensured:

- When caches are not used.
- As a result of cache maintenance operations.
- By the use of hardware coherency mechanisms to ensure coherency of data accesses to memory for cacheable locations by observers in different Shareability domains.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CPGW}

Hardware is not required to ensure coherency between instruction caches and memory, even for regions of memory with the Shareability attribute.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.27 Cache maintenance operations.](#)

[B7.2.9 Memory barriers.](#)

[B7.13 Shareability attributes.](#)

B7.23 Cache enabling and disabling

I _{PPLL}	The Configuration and Control Register, CCR , enables and disables caches across all levels of cache that are visible to the PE. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HTLD}	It is IMPLEMENTATION DEFINED whether the CCR.DC and CCR.IC bits affect the memory attributes that are generated by an enabled MPU. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && MPU.</i>
I _{TNHX}	An implementation can use control bits in the Auxiliary Control Register, ACTLR , for finer-grained control of cache enabling. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SMDL}	For instruction fetches and data accesses, NS-Attr determines which banked instance, either Secure or Non-secure, of CCR.IC or CCR.DC is used. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{DSTQ}	If the MPU is disabled, MPU_CTRL.ENABLE == 0, the CCR.DC and CCR.IC bits determine the cache state for cacheable regions of the system address map. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && MPU.</i>

See also:

[B7.20 Cache identification.](#)

[B7.19 Caches.](#)

[B7.24 Cache behavior at reset.](#)

[B3.14 Secure address protection.](#)

B7.24 Cache behavior at reset

R _{KCFK}	All caches are disabled at reset. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{JMBT}	An implementation can require the use of a specific cache initialization routine to invalidate its storage array before it is enabled: • The exact form of any required cache initialization routine is IMPLEMENTATION DEFINED. • If a required initialization routine is not performed, the state of an enabled cache is UNPREDICTABLE. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{TVKQ}	If an implementation permits cache hits when the cache is disabled, the cache initialization routine provides a mechanism to ensure the correct initialization of the caches. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{CJGV}	If an implementation permits cache hits when the cache is disabled and the cache contents are not invalidated at reset, the initialization routine avoids any possibility of running from an uninitialized cache. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{JSQQ}	An initialization routine can require a fixed instruction sequence to be placed in a restricted range of memory. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{JCTD}	Arm recommends that whenever an invalidation routine is required, it is based on the Armv8-M cache maintenance operations. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[B7.19 Caches.](#)

[B7.23 Cache enabling and disabling.](#)

[B7.27 Cache maintenance operations.](#)

B7.25 Behavior of Preload Data (PLD) and Preload Instruction (PLI) instructions with caches

I_{CQLR}	PLD and PLI are memory system hints and their effect is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I_{TPPK}	The instructions PLD and PLI do not generate exceptions but the memory system operations might generate an imprecise fault (asynchronous exception) because of the memory access. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{QNGJ}	A PLD instruction does not cause any effect to the caches or memory other than the effects that, for permission or other reasons, can be caused by the equivalent load from the same location with the same context and at the same privilege level and Security state. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{SFNK}	A PLD instruction does not access Device-nGnRnE or Device-nGnRE memory. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{HNLN}	A PLI instruction does not cause any effect to the caches or memory other than the effects that, for permission or other reasons, can be caused by the fetch resulting from changing the PC to the location specified by the PLI instruction with the same context and at the same privilege level and Security state. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{MRFG}	A PLI instruction cannot access memory that has the Device-nGnRnE or Device-nGnRE attribute. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

PLD, PLDW (immediate).

PLD (literal).

PLD, PLDW (register).

PLI (immediate, literal).

PLI (register).

B7.26 Branch predictors

I_{GTPB} Branch predictor hardware typically uses a form of cache to hold branch information.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MTBD} Branch predictors are not architecturally visible.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{CVCV} The [BPIALL](#) operation is provided for timing and determinism.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

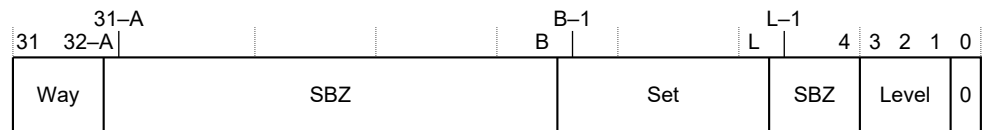
[B7.29 Branch predictor maintenance operations.](#)

B7.27 Cache maintenance operations

I _{MRMG}	Cache maintenance operations act on particular memory Locations . <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{JJLL}	Following a Clean operation, updates made by an Observer that controls the cache are made visible to other Observers that can access memory at the point to which the operation is performed. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{VRBP}	The cleaning of a cache entry from a cache can overwrite memory that has been written by another Observer only if the entry contains a Location that has been written to by an Observer in the Shareability domain of that memory Location . <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SJFS}	Following an invalidate operation, updates made visible by observers that access memory at the point to which the invalidate is defined are made visible to an Observer that controls the cache. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{PGXK}	An invalidate operation might result in the loss of updates to the Locations affected by the operation that have been written by observers that access the cache. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{TKBD}	If the address of an entry on which the invalidate operates does not have a Normal cacheable attribute, or if the cache is disabled, then an invalidate operation ensures that this address is not present in the cache. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{JTXK}	If the address of an entry on which the invalidate operates has the Normal cacheable attribute, the cache invalidate operation cannot ensure that the address is not present in an enabled cache. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SDVP}	A clean and invalidate operation behaves as the execution of a clean operation followed immediately by an invalidate operation. Both operations are performed to the same Location . <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{VKSN}	The clean operation cleans from the level of cache that is specified through at least the next level of cache away from the PE. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{GFXB}	The invalidate operation invalidates only at the level specified. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{KVSM}	For set/way operations and for All (entire cache) operations, the cache maintenance operation is to the next level of caching. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{JTWT}	For address operations, the cache maintenance operation is to the Point of coherency (PoC) or to the Point of unification (PoU) depending on the settings in CLIDR.{LoC, LOUU} . <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XLHX}	Data cache maintenance operations affect data caches and unified caches. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

- R_{QKMF}** Instruction cache maintenance operations only affect instruction caches.
Applies to an implementation of the architecture Armv8.0-M onward.
- R_{RSVL}** Cache maintenance operations are memory mapped, 32-bit write-only operations.
Applies to an implementation of the architecture Armv8.0-M onward.
- R_{NSHH}** Cache maintenance operations can have one of the following side effects:
- Any **Location** in the cache might be cleaned.
 - Any unlocked **Location** in the cache might be cleaned and invalidated.
- Applies to an implementation of the architecture Armv8.0-M onward.*
- R_{DWMMR}** The **ICIMVAU**, **DCIMVAC**, **DCCMVAU**, **DCCMVAC**, and **DCCIMVAC** operations require the physical address in the memory map but it does not have to be cache-line aligned.
Applies to an implementation of the architecture Armv8.0-M onward.
- R_{HCTC}** For **DCISW**, **DCCSW**, and **DCCISW**, the **STR** operation identifies the **cache line** to which it applies by specifying the following:
- The cache set the line belongs to.
 - The way number of the line in the set.
 - The **cache level**.

The format of the register data for a set/way operation is:



Where:

A = $\text{Log}_2(\text{ASSOCIATIVITY})$, rounded up to the next integer if necessary.

B = (L + S).

L = $\text{Log}_2(\text{LINELEN})$.

S = $\text{Log}_2(\text{NSETS})$, rounded up to the next integer if necessary. **ASSOCIATIVITY**, **LINELEN** (line length, in bytes), and **NSETS** (number of sets) have their usual meanings and are the values for the cache level being operated on.

The values of **A** and **S** are rounded up to the next integer.

Level = ((Cache level to operate on) - 1). For example, this field is 0 for operations on an L1 cache, or 1 for operations on an L2 cache.

Set = The number of the set to operate on.

Way = The number of the way to operate on.

- If **L** == 4 then there is no **SBZ** field between the set and level fields in the register.
- If **A** == 0 there is no way field in the register, and register bits[31:B] are **SBZ**.
- If the level, set, or way field in the register is larger than the size implemented in the cache, then the effect of the operation is UNPREDICTABLE.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RSBX} After the completion of an instruction cache maintenance operation, a [Context synchronization event](#) guarantees that the effects of the cache maintenance operation are visible to all instruction fetches that follow the [Context synchronization event](#).

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DHJQ} Arm recommends that, wherever possible, all caches that require maintenance to ensure coherency are included in the caches affected by the architecturally-defined cache maintenance operations.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{LRGS} It is IMPLEMENTATION DEFINED whether the [DCIMVAC](#) and [DCISW](#) operations, when performed from Non-secure state either:

- Clean any data that might be Secure data before invalidating it.
- Do not invalidate Secure data.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{VKDF} [ICIALLU](#), [ICIMVAU](#), [DCCMVAU](#), [DCCMVAC](#), [DCCSW](#), [DCCIMVAC](#), [DCCISW](#), and [BPIALL](#) operations on Secure data might be ignored if the operation was performed from Non-secure state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

I_{MLLC} The following is the sequence of cache cleaning operations for a line of self-modifying code:

```

1 // Enter this code with <Rx> containing the new 32-bit instruction and <Ry>;
2 // containing the address of the instruction.
3 // Use STRH in the first line instead of STR for a 16-bit instruction.
4 STR <Rx>, [<Ry>]           // Write instruction to memory
5 DSB                       // Ensure write is visible
6 MOV <Rt>, 0xE000E000       // Create pointer to base of System Control Space
7 STR <Ry>, [<Rt>, #0xF64]   // Clean data cache by address to point of unification
8 DSB                       // Ensure visibility of the data cleaned from the cache
9 STR <Ry>, [<Rt>, #0xF58]   // Invalidate instruction cache by address to PoU
10 STR <Ry>, [<Rt>, #0xF78]  // Invalidate branch predictor
11 DSB                       // Ensure completion of the invalidations
12 ISB                       // Synchronize fetched instruction stream

```

Applies to an implementation of the architecture Armv8.0-M onward.

R_{HXMM} If the Security attribution of memory is changed, it is IMPLEMENTATION DEFINED whether cache maintenance operations are required to keep the system state valid.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{JFGF} In the cache maintenance instructions that operate by Set/Way, if any index argument is larger than the value supported by the implementation, then the behavior is CONSTRAINED UNPREDICTABLE and one of the following occurs:

- The instruction generates a BusFault.
- The instruction performs cache maintenance on one of the following:
 - No [cache lines](#).
 - A single arbitrary [cache line](#).
 - Multiple arbitrary [cache lines](#).

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[Cache Maintenance Operations](#).

[Cache Maintenance Operations \(NS alias\)](#).

[B7.2.3 Ordering and observability](#).

B7.10 Cacheability attributes.

B7.28 Ordering of cache maintenance operations

R_{GCNB} All cache and branch predictor maintenance operations that do not specify an address execute, relative to each other, in program order.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GXNL} All cache maintenance operations that specify an address:

- Execute in program order relative to all cache operations that do not specify an address.
- Execute in program order relative to all cache maintenance operations that specify the same address.
- Can execute in any order relative to cache maintenance operations that specify a different address.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RTJG} There is no restriction on the ordering of data or unified cache maintenance operation by address relative to any explicit load or store.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MJPP} There is no restriction on the ordering of a data or unified cache maintenance operation by set/way relative to any explicit load or store.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{VXXZ} A [DSB](#) instruction can be inserted to enforce ordering as required.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{SWBG} For the [IC_{IALLU}](#) operation, the value in the register specified by the [STR](#) instruction that performs the operation is ignored.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{ZQQZ} If the Security attribution of cached memory is changed, either by the SAU or IDAU, then the following cache maintenance sequence should be used:

1. Execute a [DSB](#) instruction.
2. Clean the affected lines in data or unified caches using the [DCC*](#) operation.
3. Execute a [DSB](#) instruction.
4. Change the security attribution of the address range.
5. Execute a [DSB](#) instruction.
6. Invalidate the affected lines in all caches using the [DCI*](#) and [ICI*](#) operations.
7. Execute a [Context synchronization event](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

See also:

[B7.2.12 Data Synchronization Barrier](#).

[B10.2 Security attribution](#).

[B7.27 Cache maintenance operations](#).

B7.29 Branch predictor maintenance operations

R _{HVXX}	Branch predictor maintenance operations are independent of cache maintenance operations. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{NSRK}	A Context synchronization event that follows a branch predictor maintenance operation guarantees that the effects of the branch predictor maintenance operation are visible to all instructions after the Context synchronization event . <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{DRQT}	When performing a BPIALL operation, the value written to the BPIALL register is ignored. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LXHX}	As a side effect of a branch predictor maintenance operation, any entry in the branch predictor might be invalidated. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[Cache Maintenance Operations](#).

[Cache Maintenance Operations \(NS alias\)](#).

[BPIALL](#), [Branch Predictor Invalidate All](#).

[B7.2.9 Memory barriers](#).

[DSB](#).

Chapter B8

The System Address Map

This chapter specifies the Armv8-M system address map rules. It contains the following sections:

B8.1 System address map.

B8.2 The System region of the system address map.

B8.3 The System Control Space (SCS).

B8.1 System address map

R_{FQSD}

The address space is divided into the following regions:

Address	Region	Memory type	XN?	Cache	Shareability	Example usage
0x00000000 - 0x1FFFFFFF	Code	Normal	-	WT RA	Non-shareable	Typically ROM or flash memory.
0x20000000 - 0x3FFFFFFF	SRAM	Normal	-	WBWA RA	Non-shareable	SRAM region typically used for on-chip RAM.
0x40000000 - 0x5FFFFFFF	Peripheral	Device, nGnRE	XN	-	-	On-chip peripheral address space.
0x60000000 - 0x7FFFFFFF	RAM	Normal	-	WBWA RA	Non-shareable	Memory with write-back, write allocate cache attribute for L2 and L3 cache support.
0x80000000 - 0x9FFFFFFF	RAM	Normal	-	WT RA	Non-shareable	Memory with Write-Through cache attribute.
0xA0000000 - 0xBFFFFFFF	Device	Device, nGnRE	XN	-	-	Peripherals accessible to all Requesters.
0xC0000000 - 0xDFFFFFFF	Device	Device, nGnRE	XN	-	-	Peripherals accessible only to the PE.
0xE0000000 - 0xE00FFFFF	System PPB	Device, nGnRE	XN	-	-	1 MB region reserved as the PPB. This supports key resources, including the System Control Space, and debug features.
0xE0100000 - 0xFFFFFFFF	System Vendor_SYS	Device, nGnRE	XN	-	-	Vendor System Region.

WT - Write-Through.

RA - Read-allocate.

WBWA - Write-back, write-allocate.

XN - Memory with the execute-never memory attribute.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{XXGQ}

The term boundary is used to indicate the divide between memory regions stated in the system address map.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MBRB}

An access that crosses a boundary is UNPREDICTABLE. This rule also applies to the 0xFFFFFFFF - 0x00000000 boundary.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DNBD}

An unaligned or multi-word access that crosses a 0.5GB memory region boundary is UNPREDICTABLE.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B8.2 The System region of the system address map.](#)

[B7.4 Address space.](#)

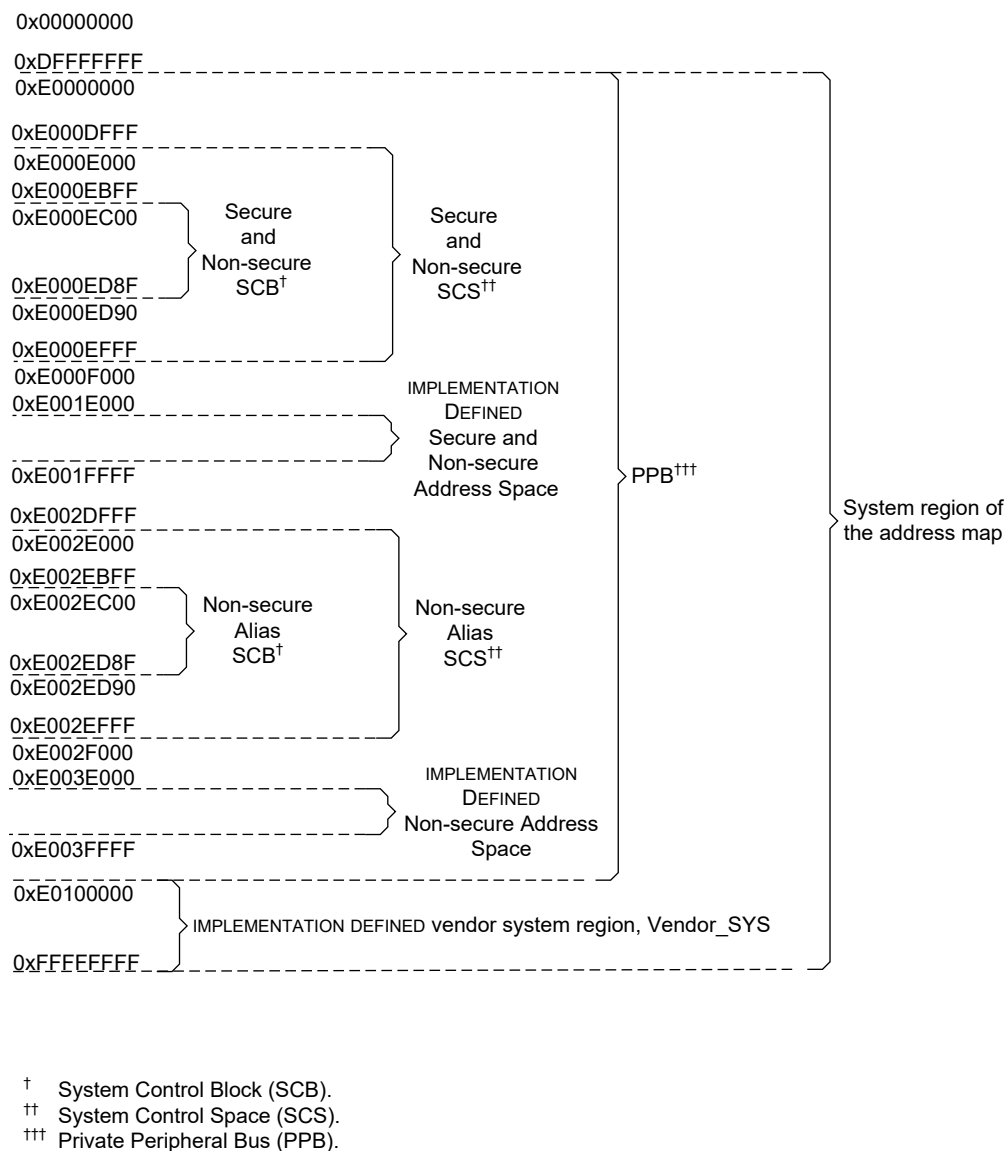
[B7.3 Memory accesses.](#)

[B7.19 Caches.](#)

B8.2 The System region of the system address map

R_{MHGM}

The system region of the system address map is as follows:



Applies to an implementation of the architecture Armv8.0-M onward. Note, The IMPDEF S & NS Address Space and IMPDEF NS Address space are only available in an Armv8.1-M implementation.

R_{MXRW}

In a PE without the Security Extension, the Non-secure SCS is RAZ/WI and any unprivileged access to the Non-secure SCS results in a BusFault.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are -M && !S. Note, if !M a HardFault is generated.

I_{FWLM}

Arm recommends that Vendor_SYS is divided as follows:

- 0xE0100000-0xFFFFFFFF is reserved.

- Vendor resources start at 0xF0000000.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{MSFV} Arm strongly recommends that the IMPLEMENTATION DEFINED Secure and Non-secure Address space, 0x00000000 - 0x0000FFFF, and the IMPLEMENTATION DEFINED Non-secure alias Address space, 0xE003E000 - 0xE003FFFF, are implemented symmetrically.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - S.

R_{RTKK} Any access to the PPB that results in a BusFault will be reported synchronously.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GLNG} Privileged accesses to unimplemented registers are RES0.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NDML} Unprivileged accesses to unimplemented registers will generate a synchronous BusFault unless otherwise stated.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DQQS} Unprivileged accesses can be enabled to the Software Trigger Interrupt Register, STIR in the System Control Space by programming a control bit in the Configuration and Control Register, CCR.USERSETMPEND.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NSWG} If a write to a PPB register triggers a BusFault the PPB register targeted by the write must not be updated. This applies regardless of whether the BusFault is subsequently ignored due to CCR.BFHFNMIGN.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RJHJ} If the exception entry context stacking, exception return context unstacking, lazy floating-point state preservation, or the stacking or unstacking of a FNC_RETURN stack frame, results in an access to an address within the PPB space the behavior of the access is CONSTRAINED UNPREDICTABLE and is one of the following:

- Generates a BusFault.
- Perform the specified access to the PPB space.

This does not apply to the VLSTM instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B8.1 System address map.](#)

[B8.3 The System Control Space \(SCS\).](#)

[STIR, Software Triggered Interrupt Register.](#)

[CCR, Configuration and Control Register.](#)

[B13.1.2 Debug resources.](#)

B8.3 The System Control Space (SCS)

R _{CQVK}	The System Control Space (SCS) provides registers for control, configuration, and status reporting. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{CFPK}	The Secure view of the NS alias is identical to the Non-secure view of normal addresses unless otherwise stated. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{BMLS}	The side effects of any access to the SCS that performs a context-altering operation take effect when the access completes. A DSB instruction can be used to guarantee completion of a previous SCS access. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{WQOB}	A context synchronization event guarantees that the side effects of a previous SCS access are visible to all instructions in program order following the context synchronization event . <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[B8.2 The System region of the system address map.](#)

[System Control Block.](#)

[System Control Block \(NS alias\).](#)

[Debug Control Block.](#)

[Debug Control Block \(NS alias\).](#)

[STIR, Software Triggered Interrupt Register.](#)

[SYST_CSR, SysTick Control and Status Register.](#)

[Chapter B12 Nested Vectored Interrupt Controller.](#)

[Chapter B10 The Armv8-M Protected Memory System Architecture.](#)

Chapter B9

Synchronization and Semaphores

This chapter specifies the Armv8-M architecture rules for exclusive access instructions and non-blocking synchronization of shared memory. It contains the following sections:

B9.1 Exclusive access instructions.

B9.2 The local monitors.

B9.3 The global monitor.

B9.4 Exclusive access instructions and the monitors.

B9.5 Load-Exclusive and Store-Exclusive instruction constraints.

B9.1 Exclusive access instructions

R_{LQDX} Armv8 provides non-blocking synchronization of shared memory, using synchronization primitives for accesses to both Normal and Device memory.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RGCP} The synchronization primitives and associated instructions are as follows:

Function	T32 instruction
Load-Exclusive	
Byte	LDREXB, LDAEXB
Halfword	LDREXH, LDAEXH
Word	LDREX, LDAEX
Store-Exclusive	
Byte	STREXB, STLEXB
Halfword	STREXH, STLEXH
Word	STREX, STLEX
Clear-Exclusive	
	CLREX

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MWFP} A Load-Exclusive instruction performs a load from memory, and:

- The executing PE marks the memory address for exclusive access.
- The local monitor of the executing PE transitions to the Exclusive Access state.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{JHMH} The size of the marked memory block is called the *Exclusives reservation granule* (ERG), and is an IMPLEMENTATION DEFINED value that is of a power of 2 size, in the range 4 - 512 words.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MTTN} A marked block of the ERG is created by ignoring the least significant bits of the memory address. A marked address is any address within this marked block.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{FMXK} In some implementations the [CTR](#) identifies the Exclusives reservation granule. Where this is not the case, the Exclusives reservation granule is treated as having the maximum of 512 words.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B9.2 The local monitors.](#)

[B9.3 The global monitor.](#)

[B9.4 Exclusive access instructions and the monitors.](#)

[B9.5 Load-Exclusive and Store-Exclusive instruction constraints.](#)

B9.2 The local monitors

R_{QTFP} Any non-aborted attempt by the same PE to use a Store-Exclusive instruction to modify any address is guaranteed to clear the marking.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NJWC} When a PE writes using any instruction other than a Store-Exclusive instruction:

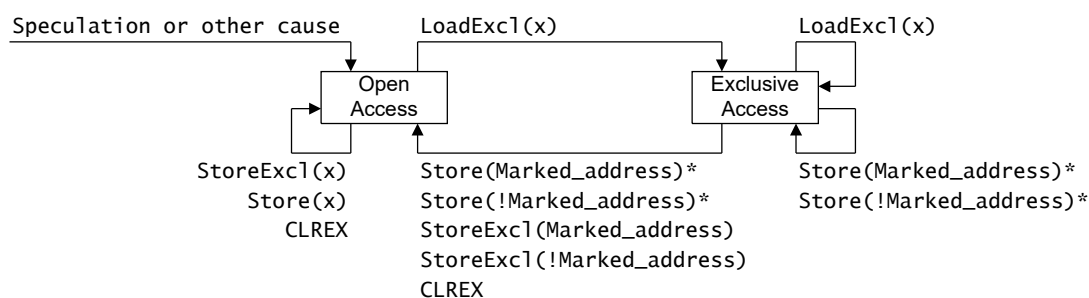
- If the write is to a physical address that is not marked as Exclusive Access by its local monitor and that local monitor is in the Exclusive Access state, it is IMPLEMENTATION DEFINED whether the write affects the state of the local monitor.
- If the write is to a physical address that is marked as Exclusive Access by its local monitor, it is IMPLEMENTATION DEFINED whether the write affects the state of the local monitor.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{PFFT} It is IMPLEMENTATION DEFINED whether a store to a marked physical address causes a mark in the local monitor to be cleared if that store is by an **Observer** other than the one that caused the physical address to be marked.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{KXNM} The state machine for the local monitor is shown here.



Operations marked * are possible alternative IMPLEMENTATION DEFINED options.

In the diagram: LoadExc1 represents any Load-Exclusive instruction

StoreExc1 represents any Store-Exclusive instruction

Store represents any other store instruction.

Any LoadExc1 operation updates the marked address to the most significant bits of the address x used for the operation.

The local monitor only transitions to the Exclusive Access state as the result of the architectural execution of one of the operations shown in the diagram.

Any transition of the local monitor to the Open Access state that is not caused by the architectural execution of an operation shown here does not indefinitely delay forward progress of execution.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WTHJ} The local monitor does not hold any physical address, but instead treats any access as matching the address of the previous Load-Exclusive instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{JWQS} A local monitor implementation can be unaware of Load-Exclusive and Store-Exclusive instructions from other PEs.

Applies to an implementation of the architecture Armv8.0-M onward.

R _{KJQW}	The architecture does not require a load instruction by another PE that is not a Load-Exclusive instruction to have any effect on the local monitor. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XMML}	It is IMPLEMENTATION DEFINED whether the transition from Exclusive Access to Open Access state occurs when the Store or StoreExcl is from another Observer. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{MRS}	The architecture permits a local monitor to transition to the Open Access state as a result of speculation, or from some other cause. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HRHC}	An exception return clears the local monitor. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

See also:

[B9.4 Exclusive access instructions and the monitors.](#)

B9.3 The global monitor

R _{FKFB}	For each PE in the system, the global monitor: • Can hold at least one marked block. • Maintains a state machine for each marked block it can hold. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{VDLP}	For each PE, the architecture only requires global monitor support for a single marked address. Any situation that might benefit from the use of multiple marked addresses on a single PE is CONSTRAINED UNPREDICTABLE. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{NNDC}	The global monitor can either reside in a block that is part of the hardware on which the PE executes or exist as a secondary monitor at the memory interfaces. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{XTLH}	The IMPLEMENTATION DEFINED aspects of the monitors mean that the global monitor and the local monitor can be combined into a single unit, provided that the unit performs the global monitor and the local monitor functions defined in this manual. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{KDWM}	For shareable memory locations, in some implementations and for some memory types, the properties of the global monitor require functionality outside the PE. Some system implementations might not implement this functionality for all locations of memory. In particular, this can apply to: • Any type of memory in the system implementation that does not support hardware cache coherency. • Non-cacheable memory, or memory treated as Non-cacheable, in an implementation that does support hardware cache coherency. In such a system, it is defined by the system: • Whether the global monitor is implemented. • If the global monitor is implemented, which address ranges or memory types it monitors. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{QJNL}	The only memory types for which it is architecturally guaranteed that a global exclusive monitor is implemented are: • Inner Shareable, Inner Write-Back, Outer Write-Back Normal memory with Read allocation hint and Write allocation hint and not transient. • Outer Shareable, Inner Write-Back, Outer Write-Back Normal memory with Read allocation hint and Write allocation hints and not transient. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HBJJ}	The set of memory types that support atomic instructions includes all of the memory types for which a global monitor is implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{HLHS}	If the global monitor is not implemented for an address range or memory type, then performing a Load-Exclusive/Store-Exclusive instruction to such a location, in the absence of any other fault, has one or more of the following effects: • The instruction generates BusFault. • The instruction generates a DACCVIOL MemManage fault. • The instruction is treated as a NOP. • The Load-Exclusive instruction is treated as if it were accessing a Non-shareable location, but the state of the local monitor becomes UNKNOWN.

- The Store-Exclusive instruction is treated as if it were accessing a Non-shareable location, but the state of the local monitor becomes UNKNOWN.
- The value held in the result register of the Store-Exclusive instruction becomes UNKNOWN.

Applies to an implementation of the architecture Armv8.0-M onward. Note, a MemManage Fault requires M && MPU, a BusFault requires M.

R_{FQRT}

For write transactions generated by non-PE observers that do not implement exclusive accesses or other atomic access mechanisms, the effect that writes have on the global monitor and the local monitor that are used by an Arm PE is IMPLEMENTATION DEFINED. The writes might not clear the global monitors of other PEs for:

- Some address ranges.
- Some memory types.

Applies to an implementation of the architecture Armv8.0-M onward.

B9.3.1 Load-Exclusive and Store-Exclusive

R_{RXVB}

The global monitor only supports a single outstanding exclusive access to shareable memory for each PE.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GXLf}

The architecture does not require a load instruction by another PE, that is not a Load-Exclusive instruction, to have any effect on the global monitor.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MPKM}

A Load-Exclusive instruction by one PE has no effect on the global monitor state for any other PE.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MFGC}

A Store-Exclusive instruction performs a conditional store to memory:

- The store is guaranteed to succeed only if the physical address accessed is marked as exclusive access for the requesting PE and both the local monitor and the global monitor state machines for the requesting PE are in the Exclusive Access state. In this case:
 - A status value of 0 is returned to a register to acknowledge the successful store.
 - The final state of the global monitor state machine for the requesting PE is IMPLEMENTATION DEFINED.
 - If the address accessed is marked for exclusive access in the global monitor state machine for any other PE then that state machine transitions to Open Access state.
- If no address is marked as exclusive access for the requesting PE, the store does not succeed:
 - A status value of 1 is returned to a register to indicate that the store failed.
 - The global monitor is not affected and remains in Open Access state for the requesting PE.
- If a different physical address is marked as exclusive access for the requesting PE, it is IMPLEMENTATION DEFINED whether the store succeeds or not:
 - If the store succeeds a status value of 0 is returned to a register, otherwise a value of 1 is returned.
 - If the global monitor state machine for the PE was in the Exclusive Access state before the Store-Exclusive instruction it is IMPLEMENTATION DEFINED whether that state machine transitions to the Open Access state.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NNMG}

In a shared memory system, the global monitor implements a separate state machine for each PE in the system. The state machine for accesses to shareable memory by PE(n) can respond to all the shareable memory accesses visible to it.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WKPJ} In a shared memory system, the global monitor implements a separate state machine for each **Observer** that can generate a Load-Exclusive or a Store-Exclusive instruction in the system.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NWWH} Whenever the global monitor state for a PE changes from Exclusive access to Open access, an event is generated and held in the Event register for that PE. This register is used by the Wait for Event mechanism.

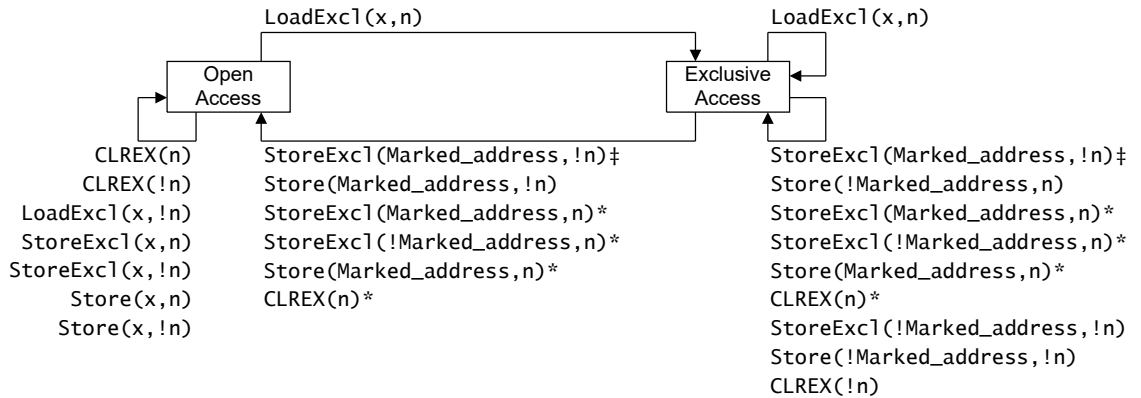
Applies to an implementation of the architecture Armv8.0-M onward.

B9.3.2 Load-Exclusive and Store-Exclusive in Shareable memory

R_{HKQT} A Load-Exclusive instruction from shareable memory performs a load from memory, and causes the physical address of the access to be marked as exclusive access for the requesting PE. This access can also cause the exclusive access mark to be removed from any other physical address that has been marked by the requesting PE.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GDMD} The state machine for PE(n) in a global monitor is as follows.



Any LoadExc1 operation updates the marked address to the most significant bits of the address x used for the operation.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RGFK} Whether a Store-Exclusive instruction successfully updates memory or not depends on whether the address accessed matches the marked shareable memory address for the PE issuing the Store-Exclusive instruction, and whether the local monitor and the global monitor are in the exclusive state.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{QVWF} When the global monitor is in the Exclusive Access state, it is IMPLEMENTATION DEFINED whether a CLREX instruction causes the global monitor to transition from Exclusive Access to Open Access state.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DLMP} A Load-Exclusive instruction can only update the marked shareable memory address for the PE issuing the Load-Exclusive instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BSGB}

It is IMPLEMENTATION DEFINED:

- Whether a modification to a Non-shareable memory location can cause a global monitor to transition from Exclusive Access to Open Access state.
- Whether a Load-Exclusive instruction to a Non-shareable memory location can cause a global monitor to transition from Open Access to Exclusive Access state.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B9.4 Exclusive access instructions and the monitors.](#)

B9.4 Exclusive access instructions and the monitors

R_{VXWN} The Store-Exclusive instruction defines the register to which the status value of the monitors is returned.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DTRN} A Store-Exclusive instruction performs a conditional store to memory that depends on the state of the local monitor:

- **If the local monitor is in the Exclusive Access state:**

- If the address of the Store-Exclusive instruction is the same as the address that has been marked in the monitor by an earlier Load-Exclusive instruction, then the store occurs. Otherwise, it is IMPLEMENTATION DEFINED whether the store occurs.
- A status value is returned to a register:
 - * If the store took place the status value is 0.
 - * Otherwise, the status value is 1.
- The local monitor of the executing PE transitions to the Open Access state.

- **If the local monitor is in the Open Access state:**

- No store takes place.
- A status value of 1 is returned to a register.
- The local monitor remains in the Open Access state.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DFNB} A Store-Exclusive instruction performs a store to Shareable memory that depends on the state of both the local monitor and the global monitor:

- **If both the local monitor and the global monitor are in the Exclusive Access state:**

- If the address of the Store-Exclusive instruction is the same as the address that has been marked in the monitor by an earlier Load-Exclusive instruction, then the store occurs. Otherwise, it is IMPLEMENTATION DEFINED whether the store occurs.
- A status value is returned to a register:
 - * If the store took place the status value is 0.
 - * Otherwise, the status value is 1.
- The local monitor of the executing PE transitions to the Open Access state.

- **If either the local monitor or the global monitor is in the Open Access state:**

- No store takes place.
- A status value of 1 is returned to a register.
- The local monitor of the executing PE transitions to the Open Access state.
- The global monitor that is associated with the executing PE transitions to the Open Access state.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B9.2 The local monitors.](#)

[B9.3 The global monitor.](#)

B9.5 Load-Exclusive and Store-Exclusive instruction constraints

I _{RTW}	The Load-Exclusive and Store-Exclusive instructions are intended to work together as a pair, for example a LDREX/STREX pair or a LDREXB/STREXB pair. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{BHPN}	The architecture does not require an address or size check as part of the <code>IsExclusiveLocal()</code> function. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LHLG}	If two StoreExcl instructions are executed without an intervening LoadExcl instruction the second StoreExcl instruction returns a status value of 1. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{DVRQ}	The architecture does not require every LoadExcl instruction to have a subsequent StoreExcl instruction. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{JXXS}	If the transaction size of a StoreExcl instruction is different from the preceding LoadExcl instruction in the same thread of execution, behavior is a CONSTRAINED UNPREDICTABLE choice of: • The StoreExcl either passes or fails, and the status value returned by the StoreExcl is UNKNOWN. • The block of data of the size of the larger of the transaction sizes used by the LoadExcl/StoreExcl pair at the address accessed by the LoadExcl/StoreExcl pair, is UNKNOWN. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{GVWN}	The hardware only ensures that a LoadExcl/StoreExcl pair succeeds if the LoadExcl and the StoreExcl have the same transaction size. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XLSE}	Forward progress can only be made using LoadExcl/StoreExcl loops if, for any LoadExcl/StoreExcl loop within a single thread of execution if both of the following are true: • There are no explicit memory accesses, pre-loads, direct or indirect register writes, cache maintenance instructions, SVC instructions, ISB barriers or exception returns between the Load-Exclusive and the Store-Exclusive. • The following conditions apply between the Store-Exclusive having returned a fail result and the retry of the Load-Exclusive: – There are no stores to any location within the same Exclusives reservation granule that the Store-Exclusive is accessing. – There are no direct or indirect register writes, other than changes to the flag fields in APSR or FPSCR, caused by data processing or comparison instructions. – There are no direct or indirect cache maintenance instructions, SVC instructions, or exception returns. The Exclusives monitor can be cleared at any time without an application-related cause, provided that such clearing is not systematically repeated so as to prevent the forward progress in finite time of at least one of the threads that is accessing the Exclusives monitor. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{RFXR}	Keeping the LoadExcl and the StoreExcl operations close together in a single thread of execution minimizes the chance of the exclusive monitor state being cleared between the LoadExcl instruction and the StoreExcl instruction. Therefore, for best performance, Arm strongly recommends a limit of 128 bytes between LoadExcl and StoreExcl instructions in a single thread of execution. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

R _{PKQF}	The architecture sets an upper limit of 2048 bytes on the Exclusives reservation granule that can be marked as exclusive. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{PGGN}	For performance reasons, Arm recommends that objects that are accessed by exclusive accesses are separated by the size of the exclusive reservations granule. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XPDN}	After taking a BusFault or a MemManage fault, the state of the exclusive monitors is UNKNOWN. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{FCRN}	For the memory location accessed by a LoadExcl/StoreExcl pair, if the memory attributes for a StoreExcl instruction are different from the memory attributes for the preceding LoadExcl instruction in the same thread of execution, behavior is CONSTRAINED UNPREDICTABLE. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{DMJW}	The effect of a data or unified cache invalidate, clean, or clean and invalidate instruction on a local exclusive monitor or a global exclusive monitor that is in the Exclusive Access state is CONSTRAINED UNPREDICTABLE, and the instruction might clear the monitor, or it might leave it in the Exclusive Access state. For address-based maintenance instructions, this also applies to the monitors of other PEs in the same Shareability domain as the PE executing the cache maintenance instruction, as determined by the Shareability domain of the address being maintained. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{MDHL}	Arm strongly recommends that implementations ensure that the use of such maintenance instructions by a PE in the Non-secure state cannot cause a denial of service on a PE in the Secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{RRTJ}	In the event of repeatedly-contending LoadExcl/StoreExcl instruction sequences from multiple PEs, an implementation ensures that forward progress is made by at least one PE. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

Chapter B10

The Armv8-M Protected Memory System Architecture

This chapter specifies the Armv8-M *Protected Memory System Architecture* (PMSAv-8) rules, and in particular the rules for the optional *Memory Protection Unit* (MPU) and the optional *Security Attribution Unit* (SAU). It contains the following sections:

[B10.1 *Memory Protection Unit*.](#)

[B10.2 *Security attribution*.](#)

[B10.3 *Security attribution unit \(SAU\)*.](#)

[B10.4 *IMPLEMENTATION DEFINED Attribution Unit \(IDAU\)*.](#)

B10.1 Memory Protection Unit

R _{HPNK}	In an implementation that includes the Protected Memory System Architecture (PMSA), system address space is protected by a Memory Protection Unit (MPU). <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{TBPJ}	PMSAv8-M only supports a unified memory model. All enabled regions support instruction and data accesses. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{HBNG}	Memory attributes are determined from the system address map or by using an MPU. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{BXCN}	MPU support in Armv8-M is optional. If the MPU is implemented at least one MPU region must be provided. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{MCCL}	The system address map can be configured to provide a background region for privileged accesses. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{JVJC}	When the MPU is disabled or not present, accesses use memory attributes from the system address map. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{KLHL}	If the MPU is enabled, attributes for memory accesses that hit in a single region are provided by the hit region. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{DBBM}	The MPU divides the memory into regions. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{JVCN}	An individual MPU region is defined by: Address >= MPU_RBAR.BASE:'00000' && Address <= MPU_RLAR.LIMIT:'11111' <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{MNDS}	The number of supported MPU regions is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
I _{WTCL}	Because the MPU_TYPE register is banked, an implementation can have a different number of MPU regions, including no MPU regions, for each Security state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S && MPU.</i>
I _{GLXD}	All MPU regions are aligned to a multiple of 32 bytes. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{BPGB}	The PE can fetch and execute instructions from each MPU region according to the value of MPU_RBAR.XN. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{ZLHD}	Unless otherwise stated, all load, store, and instruction fetch transactions are subject to an MPU check. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{DNXT}	If MPU_CTRL.ENABLE is zero, MPU checks are carried out against the system address map and not against any defined MPU regions. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>

Chapter B10. The Armv8-M Protected Memory System Architecture
B10.1. Memory Protection Unit

I _{HSCD}	The MPU check is one of a number of checks carried out on any load, store or instruction fetch transaction including alignment, security attribution checks, and a check for any BusFaults. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{HBBJ}	Exception vector reads from the Vector Address Table always use the system address map. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{VPJQ}	For the purposes of MPU checks, any change in the current execution priority resulting from a MSR or CPS requires a Context synchronization event to guarantee visibility. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{VCTC}	Any load, store or instruction fetch transactions to the PPB, within the range 0xE0000000–0xE0FFFFFF, are checked against the system address map. Instruction fetches to this region generate an IACCVIOL MemManage fault. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LLLP}	Any MPU check performed for a load, store or instruction fetch transaction will generate a precise MemManage Fault if any of the following is true: • The address accessed by the load, store or instruction fetch transaction matches more than one MPU region. • The load, store or instruction fetch transaction does not match all of the access conditions for the MPU region being accessed. • The load, store or instruction fetch transaction only matches the system address map and the system address map is not enabled. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{KDJG}	The MPU is restricted in how it can change the memory attributes for addresses in VENDOR_SYS space, that is, for addresses in the region 0xE0100000–0xFFFFFFFF. Unless otherwise stated, system space is always XN (Execute Never) and it is always Device-nGnRE. If the MPU maps this to a type other than Device-nGnRE, it is UNKNOWN whether the region is treated as Device-nGnRE or as Device-nGnRnE. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{KMTF}	Unless otherwise stated for data accesses, the MPU memory attribution and privilege checking uses the configuration registers that correspond to the current Security state of the PE. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU && S.</i>
R _{RLBR}	For instruction fetches, the MPU memory attribution and privilege checking uses the configuration registers associated with the security of the address of the instruction. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{RHSF}	If the requested execution priority is negative, the MPU_CTRL.HFNIMENA bit controls whether accesses are checked against the MPU regions or the system address map. The requested execution priority is negative if any of the following are true: • The HardFault exception is active for the Security state associated with the MPU. • The NMI exception is active for the Security state associated with the MPU. • FAULTMASK.FM is set to one for the Security state associated with the MPU. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - MPU.</i>
R _{RJJL}	If the PE is executing in privileged mode, a MemManage fault is generated when it attempts to execute an instruction from an MPU region with MPU_RLAR.PXN set to one. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MPU.</i>
R _{MKJC}	Setting the MPU_RNR.REGION field to a value that does not correspond to an implemented memory region is CONSTRAINED UNPREDICTABLE as follows: • Any subsequent read of MPU_RNR.REGION returns an UNKNOWN value.

- Any read of a register that is in an unimplemented region returns an UNKNOWN value.
- Any write to a register indirected by [MPU_RNR.REGION](#) causes all state that is indirected by that register to become UNKNOWN.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [MPU](#).

See also:

[B8.1 System address map.](#)

[B7.8 Access rights.](#)

[B7.12 Device memory attributes.](#)

[B7.13 Shareability attributes.](#)

[B7.15 Memory access restrictions.](#)

[B7.16 Mismatched memory attributes.](#)

[B7.17 Load-Exclusive and Store-Exclusive accesses to Normal memory.](#)

[B7.18 Load-Acquire and Store-Release accesses to memory.](#)

[MPU_CTRL](#), [MPU Control Register](#).

[TT_RESP](#), [Test Target Response Payload](#).

B10.2 Security attribution

I _{SBSJ}	The Secure Attribution Unit and the Implementation Defined Attribution Unit are collectively referred to as the Attribution Unit (AU). <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{JGHS}	The Security Extension defines three levels of memory security attribution. In ascending order of security, these are: 1. The Non-secure attribute. 2. The Secure attribute and is Non-secure callable. 3. The Secure attribute and is not Non-secure callable. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{RPKG}	The following units can provide security attribution information: • A <i>Security attribution unit</i> (SAU) inside the PE. • An <i>IMPLEMENTATION DEFINED attribution unit</i> (IDAU) external to the PE. The presence of such a unit is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{MGXN}	The attribution information from the SAU is used unless the IDAU specifies attributes with a higher security, in which case the IDAU attributes override the SAU attributes. This rule does not apply to architecturally defined ranges exempt from memory attribution. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{NJGR}	An <i>attribution unit</i> (AU) violation is defined as being a violation raised by either the SAU or the IDAU. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{QGV5}	All boundaries between address ranges with different security attributes are aligned to 32-byte boundaries. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.</i>
R _{LDTN}	The behavior of the following address ranges is fixed, so they are exempt from memory attribution by both the SAU and IDAU:

Ranges exempt from checking security violation

The following address ranges are marked with the Security state indicated by NS-Req, that is, the current state of the PE for non-debug accesses. This marking sets the NS-Attr to NS-Req:

0xE0000000 - 0xE0003FFF: ITM, DWT, FPB, PMU.

0xE0005000 - 0xE0005FFF: RAS error record registers.

0xE000E000 - 0xE000EFFF: SCS Secure and Non-secure range.

0xE002E000 - 0xE002EFFF: SCS Non-secure alias range.

0xE0040000 - 0xE0041FFF: TPIU, ETM.

0xE00FF000 - 0xE00FFFFFF: ROM table.

0xE0000000 - 0xFFFFFFFF: for instruction fetch only.

Additional address ranges specified by the IDAU.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S. Note, some address ranges require DB, RAS or PMU.

R_{FGDW} The behavior of the following address range is fixed:

0xF0000000 – 0xFFFFFFFF

If the PE implements the Security Extension, this memory range is always marked as Secure and not Non-secure callable for instruction fetches.

If the Security Extension is not present, this range is marked as Non-secure.

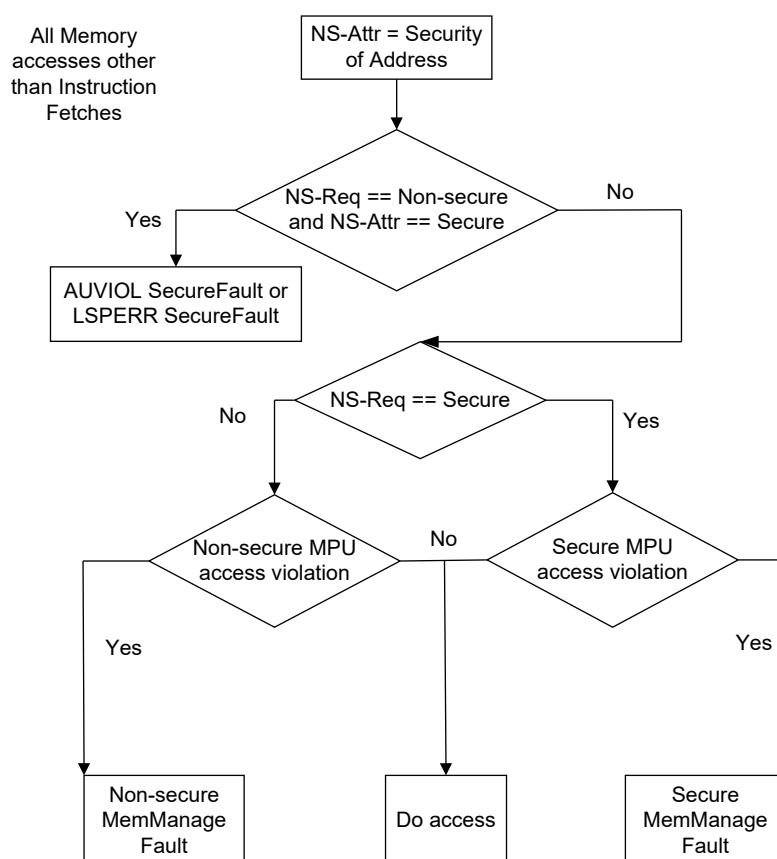
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

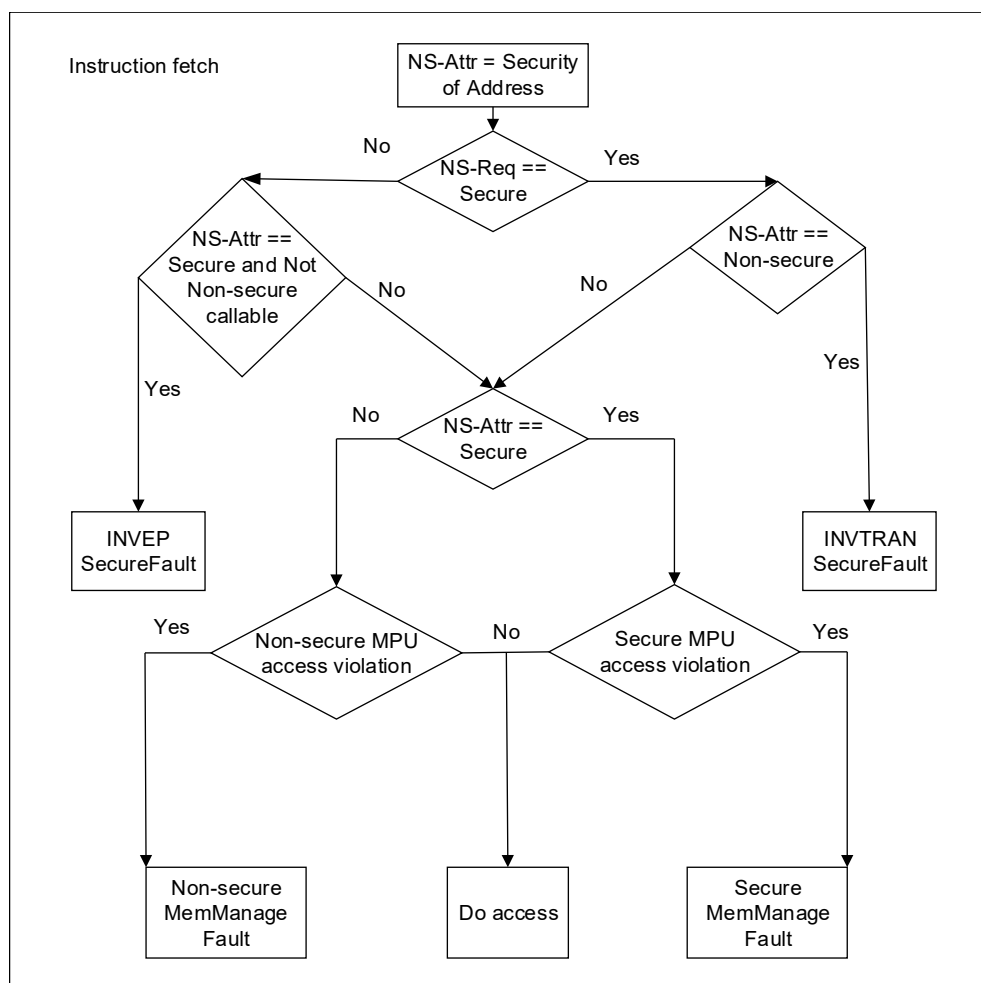
R_{PHJY} Speculative instruction fetches are permitted to a Security attribution exempt address if either of the following hold true:

- The Secure MPU region check reports the address as not being execute-never.
- The Non-secure MPU region check reports the address as not being execute-never.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S && MPU.

I_{VPWL} The Security attribution and MPU check sequence, for all data accesses which are not instruction fetches and accesses for instruction fetches are shown in the following diagrams.





Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **S**.

R_{FTFR}

If the memory access is caused by the execution of an **SG** instruction the load from the stack is undertaken with the NS-Req set to Secure even though the **SG** instruction was executed from the Non-secure state.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **S**.

See also:

[B10.3 Security attribution unit \(SAU\).](#)

[B10.4 IMPLEMENTATION DEFINED Attribution Unit \(IDAU\).](#)

B10.3 Security attribution unit (SAU)

R_{VFLR} The SAU configuration defines an IMPLEMENTATION DEFINED number of memory regions. The number of regions is indicated by [SAU_TYPE.SREGION](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

I_{PPLK} The memory regions defined by the SAU configuration are referred to as SAU_REGIONn, where n is a number from 0 - ([SAU_TYPE.SREGION](#)-1).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{RVFP} The SAU region configuration fields can only be accessed indirectly using the window registers, [SAU_RNR](#) shown in the following table.

SAU region configuration field	Associated window register field
SAU_REGIONn.ENABLE	SAU_RLAR.ENABLE
SAU_REGIONn.NSC	SAU_RLAR.NSC
SAU_REGIONn.BADDR	SAU_RBAR.BADDR
SAU_REGIONn.LADDR	SAU_RLAR.LADDR

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{KRSC} When the SAU is enabled, an address is defined as matching a region in the SAU if the following is true:
Address >= SAU_REGIONn.BADDR: '00000' && Address <= SAU_REGIONn.LADDR: '11111'

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{MPJC} Memory is marked as Secure by default. However, if the address matches a region with SAU_REGIONn.ENABLE set to 1 and SAU_REGIONn.NSC set to 0, then memory is marked as Non-secure.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{WGDK} An address that matches multiple SAU regions is marked as Secure and not Not-secure callable regardless of the attributes specified by the regions that matched the address.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{GVFQ} When the SAU is not enabled:

- Addresses are not checked against the SAU regions.
- The attribution of the address space is determined by the [SAU_CTRL.ALLNS](#) field.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{MBJN} To permit lockdown of the SAU configuration, it is IMPLEMENTATION DEFINED whether [SAU_RLAR](#), [SAU_RBAR](#), [SAU_CTRL](#), and [SAU_RNR](#) are writable.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

R_{BBCT} Setting the [SAU_RNR.REGION](#) field to a value that does not correspond to an implemented memory region is CONSTRAINED UNPREDICTABLE as follows:

- Any subsequent read of [SAU_RNR.REGION](#) returns an UNKNOWN value.
- Any read of a register indirected by [SAU_RNR.REGION](#) returns an UNKNOWN value.
- Any write to a register indirected by [SAU_RNR.REGION](#) causes all state that is indirected by that register to become UNKNOWN.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - S.

B10.4 IMPLEMENTATION DEFINED Attribution Unit (IDAU)

R_{MVCM}

The IDAU can provide the following Security attribution information for an address:

- Security attribution exempt. This specifies that the address is exempt from security attribution. This information is combined with the address ranges that are architecturally required to be exempt from attribution.
- Non-secure. This specifies if the address is Secure or Non-secure.
- Non-secure callable. This specifies if code at the address can be called from Non-secure state. This attribute is only valid if the address is marked as Secure.
- Region number. This is the region number that matches the address, and is only used by the [TT](#) instruction.
- Region number valid. This specifies that the region number is valid. This field has no effect on the attribution of the address, and is only used by the [TT](#) instruction.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{GPDN}

The IDAU must ensure that for each region all the following are true:

- Each region must have a unique identification number.
- All the addresses in the region must have the same attributes, including the Non-secure Callable attribute.
- The region must be a contiguous range of addresses.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

See also:

[TT](#), [TTT](#), [TTA](#), [TTAT](#).

[B10.2 Security attribution](#).

Chapter B11

The System Timer, SysTick

This chapter specifies the Armv8-M system timer rules. It contains the following section:

[B11.1 *The system timer, SysTick.*](#)

B11.1 The system timer, SysTick

R_{BQRG}

In a PE without the Main Extension and without the Security Extensions, either:

- No system timers are implemented.
- One system timer, SysTick, is implemented.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - !M && !S.

R_{PDDL}

In a PE without the Main Extension but with the Security Extension, one of the following is true:

- No system timers are implemented.
- One system timer, SysTick, is implemented. ICSR.STTNS determines which Security state owns the SysTick.
- Two system timers are implemented:
 - SysTick, Secure instance.
 - SysTick, Non-secure instance.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - !M && S.

R_{CNTG}

In a PE with the Main Extension but without the Security Extension, one system timer, SysTick, is implemented.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && !S.

R_{XPCW}

In a PE with the Main and Security Extensions, two system timers are implemented:

- SysTick, Secure instance.
- SysTick, Non-secure instance.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M && S.

I_{DXSQ}

There are the following SysTick registers:

- SysTick Control and Status Register (SYST_CSR).
- SysTick Reload Value Register (SYST_RVR).
- SysTick Current Value Register (SYST_CVR).
- SysTick Calibration Value Register (SYST_CALIB).

In a PE with the Security Extension and a SysTick instance dedicated to each Security state, these registers are banked.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ST.

I_{VHDT}

Each implemented SysTick is a 24-bit decrementing, wrap-on-zero, clear-on-write counter:

- When enabled, the counter counts down from the value in SYST_CVR. When it reaches zero, SYST_CVR is reloaded with the value held in SYST_RVR on the next clock edge.
- Reading SYST_CVR returns the value of the counter at the time of the read access.
- When the counter reaches zero, it sets SYST_CSR.COUNTFLAG to 1. Reading SYST_CSR.COUNTFLAG clears it to 0.
- A write to SYST_CVR clears both SYST_CVR and SYST_CSR.COUNTFLAG to 0. SYST_CVR is then reloaded with the value held in SYST_RVR on the next clock edge.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ST.

R_{TLGK}

Writing the value zero to SYST_RVR disables the SysTick on the next wrap-on-zero. The value zero is held by the counter after the wrap. This is true even when SYST_CSR.ENABLE is 1.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ST.

R_{TTFT}

A write to SYST_CVR does not cause a SysTick exception.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ST.

I _{VDJQ}	Setting SYST_CSR.TICKINT to 1 causes the SysTick exception to become pending on the SysTick reaching zero. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ST.</i>
I _{PPGV}	Arm recommends that before enabling a SysTick by SYST_CSR.ENABLE, software writes the required counter value to the SYST_RVR, and then writes to the SYST_CVR to clear the SYST_CVR to zero. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ST.</i>
I _{MMRQ}	Software can optionally use SYST_CALIB.TENMS to scale the counter to other clock rates within the dynamic range of the counter. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ST.</i>
R _{QSKV}	When the PE is halted in Debug state, any implemented SysTicks do not decrement. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ST && HDBG.</i>
I _{RWFQ}	Each implemented SysTick is clocked by a reference clock, either the PE clock or an external system clock. It is IMPLEMENTATION DEFINED which clock is used as the external reference clock. Arm recommends that if an external system clock is used, the relationship between the PE clock and the external clock is documented, so that system timings can be calculated taking into account metastability, clock skew, and jitter. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ST.</i>

Chapter B12

Nested Vectored Interrupt Controller

This chapter specifies the Armv8-M *Nested Vectored Interrupt Controller* (NVIC) rules. It contains the following sections:

[B12.1 NVIC definition.](#)

[B12.2 NVIC operation.](#)

B12.1 NVIC definition

R_{XJJQ} An Armv8-M PE includes an integral interrupt controller.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{QHBN} The Interrupt Controller Type Register (ICTR) defines the number of external interrupt lines that are supported, as well as the number of NVIC registers that are implemented. This is specified in the following table:

ICTR.INTLINESNUM	Maximum number	The index, n, of the highest implemented:	
	of interrupts	NVIC_IPRn	all other NVIC registers
0b0000	32	7	0
0b0001	64	15	1
0b0010	96	23	2
0b0011	128	31	3
0b0100	160	39	4
0b0101	192	47	5
0b0110	224	55	6
0b0111	256	63	7
0b1000	288	71	8
0b1001	320	79	9
0b1010	352	87	10
0b1011	384	95	11
0b1100	416	103	12
0b1101	448	111	13
0b1110	480	119	14
0b1111	496	123	15

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DLTR} As an example of the effect of ICTR.INTLINESNUM, if ICTR.INTLINESNUM = 0b0100, then:

- 160 external interrupt lines are supported.
- The registers NVIC_IPR0 to NVIC_IPR39 are implemented, and NVIC_IPR40 to NVIC_IPR123 are not implemented.
- The registers NVIC_ISER0 to NVIC_ISER4 are implemented, and NVIC_ISER5 to NVIC_IPR15 are not implemented. The same range of registers is implemented for all other NVIC registers.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[ICTR, Interrupt Controller Type Register.](#)

[Nested Vectored Interrupt Controller Block.](#)

[Nested Vectored Interrupt Controller Block\(NS alias\).](#)

B12.2 NVIC operation

R _{SNVK}	It is IMPLEMENTATION DEFINED which NVIC interrupts are implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{SGCR}	When a particular NVIC interrupt line is not implemented, the register fields that are associated with it are reserved. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{CCVJ}	Only an interrupt that is both pending and enabled with sufficient priority can preempt PE execution. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{GYCN}	The following conditions will cause the pending state associated with that interrupt to be set: • The input associated with the interrupt is HIGH while the active state associated with the interrupt is clear, and a simultaneous write to NVIC_ICPRn to clear the pending state is not being performed. • The input associated with the interrupt transitions from LOW to HIGH while the active state associated with the interrupt is set. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{BLWR}	When an interrupt is disabled interrupt assertion causes the interrupt to become pending, but the interrupt cannot become active. If an interrupt is active when it is disabled it remains in the active state until cleared by reset or by an exception return. Clearing the relevant enable bit prevents any new activation of the associated interrupt. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{WTFS}	The Armv8-M interrupt behavior provides compatibility with both active-high level-sensitive and pulse-sensitive interrupt signaling: • For level-sensitive interrupts, the associated exception handler runs one time for each occurrence as long as the level is cleared before the exception handler returns. If the level of the input is HIGH after the exception handler returns, the exception will be pended again. • For pulse-sensitive interrupts, the associated exception handler runs one time only, regardless of the number of pulses that the NVIC sees before the exception handler is entered. If a pulse occurs after the exception handler has been entered, the exception will be pended again. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{HVQQ}	For some implementations, pulse-sensitive interrupt signals are held long enough to ensure that the PE can sample them reliably. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{QKFW}	All NVIC interrupts have a programmable priority value and an associated exception number. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XNQW}	NVIC interrupts can be enabled and disabled by writing to their corresponding Interrupt Set-Enable or Interrupt Clear-Enable register bit field. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{WGDJ}	An implementation can hard-wire interrupt enable bits to zero if the associated interrupt line does not exist. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{RSDJ}	An implementation can hard-wire interrupt enable bits to one if the associated interrupt line cannot be disabled. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

R_{NRJV}

It is IMPLEMENTATION DEFINED for each NVIC interrupt line supported whether an NVIC interrupt supports either or both setting and clearing of the associated pending state under software control.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.9 Exception numbers and exception priority numbers.](#)

[B3.13 Priority model.](#)

[Nested Vectored Interrupt Controller Block.](#)

[Nested Vectored Interrupt Controller Block\(NS alias\).](#)

Chapter B13

Debug

This chapter specifies the Armv8-M debug rules. It contains the following sections:

[B13.1 *Debug feature overview.*](#)

[B13.2 *Accessing debug features.*](#)

[B13.3 *Debug authentication interface.*](#)

[B13.4 *Debug event behavior.*](#)

[B13.5 *Debug state.*](#)

[B13.6 *Exiting Debug state.*](#)

[B13.7 *Multiprocessor support.*](#)

B13.1 Debug feature overview

R_{WXRJ} The debug configuration of an implementation is IMPLEMENTATION DEFINED.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{VTGP} The following table describes the features of the Armv8-M debug architecture.

Feature	Main Extension	Baseline Implementation
DebugMonitor exception	Always implemented	Never implemented
Halting debug	Optional	Optional
EDBGRQ <i>External Halt signal</i>	Optional, requires Halting debug	Optional, requires Halting debug
Flash Patch and Breakpoint unit - FPB	Optional, requires Halting debug	Optional, requires Halting debug
Data Watchpoint and Trace Unit - DWT <i>Debug functionality - DWTD</i>	Optional, requires Halting debug	Optional, requires Halting debug
<i>Trace functionality - DWTT</i>	Requires ITM and Debug functionality	Never implemented
Instrumentation Trace Macrocell - ITM	Optional, requires Halting debug	Never implemented
Cross Trigger Interface - CTI	Requires ETM or Halting Debug	Requires ETM or Halting Debug
Trace Port Interface Unit - TPIU	Requires ITM or ETM	Requires ETM
Embedded Trace Macrocell - ETM	Optional, requires Halting debug	Optional, requires Halting debug
Performance Monitors Unit - PMU	Optional, requires Halting debug	Never implemented
Unprivileged Debug Extension - UDE	Optional, requires Halting debug	Never implemented
DSP Debug Extension - DSPDE	Optional, requires Halting debug	Never implemented

Applies to an implementation of the architecture Armv8.0-M onward. Note, CTI requires HDBG or ETM.

R_{FHRN} The following optional debug components are not part of the Armv8-M architecture:

- The *Cross-Trigger Interface (CTI)*.
- The CoreSight basic trace router (MTB).
- The Embedded Trace Macrocell (ETM).

Applies to an implementation of the architecture Armv8.0-M onward. Note, CTI requires HDBG or ETM.

I_{ZNHD} The recommended Debug implementation levels are:

- Minimum.
- Basic.
- Comprehensive.
- Program trace.

Minimum

In an implementation that includes the Main Extension, the minimum level contains support for the DebugMonitor exception, including:

- The [BKPT](#) instruction.
- [DEMCR](#) Monitor debug features.
- Monitor entry from External debug requests.
- [DFSR](#).

[DHCSR](#), [DCRSR](#), [DCRDR](#), and the Halting debug features in [DFSR](#) and [DEMCR](#) are RES0.

[ID_DFR0](#) is RAZ.

In an implementation that does not include the Main Extension there is no debug support.

[DFSR](#), [DHCSR](#), [DCRSR](#), [DCRDR](#), and [DEMCR](#) are RES0.

[ID_DFR0](#) is RAZ.

Basic

In an implementation that includes the Main Extension, the basic level adds support for Halting debug with:

- A Debug Access Port and ROM table.
- [DHCSR](#), [DCRSR](#), [DCRDR](#), and the Halting debug features in [DEMCR](#) are implemented.
- FPB with at least two breakpoints.
- DWT with at least:
 - One watchpoint that supports instruction, data address, and data value matching.
 - [DWT_PCSR](#).
- Optional support for a CTI in a multiprocessor system.

Support for the basic implementation is identified by [ID_DFR0](#).

In an implementation that includes Armv8.1-M, the DSP Debug Extension adds support for DSPDE. Support for the DSP Debug Extension is part of the basic level of debug.

In an implementation that does not include Armv8.1-M, there is no support for the DSP Debug Extension.

In an implementation that does not include the Main Extension, the basic level adds support for Halting debug with:

- A Debug Access Port and ROM table.
- [SHCSR](#), [DFSR](#), [DHCSR](#), [DCRSR](#), [DCRDR](#), and [DEMCR](#) are implemented. Access for the PE is IMPLEMENTATION DEFINED.
- FPB with at least two breakpoints.
- DWT with at least:
 - One watchpoint that supports instruction, data address, and data value matching.
 - [DWT_PCSR](#).
- Optional support for a CTI in a multiprocessor system.

Support for the basic implementation is identified by [ID_DFR0](#). In an Armv8.0 implementation [ID_DFR0](#) is RAZ, unless Halting debug is implemented.

Comprehensive

In an implementation that includes the Main Extension, the comprehensive level adds basic trace support with:

- An ITM.
- DWT with:
 - Trace support.

- Profiling support.
- Cycle counter.
- TPIU.

In an implementation that does not include the Main Extension, there is no support for the comprehensive level.

Program trace

In an implementation that includes the Main Extension, Program trace adds support for ETMs.

In an implementation that does not include the Main Extension, Program trace adds supports for ETMs and TPIUs.

An Armv8.1-M implementation introduces further optional debug extensions:

Performance Monitoring

In an implementation that includes Armv8.1-M, Performance Monitoring adds support for PMU. Support for the Performance Monitors extension is part of the comprehensive level of debug.

In an implementation that does not include Armv8.1-M, there is no support for Performance Monitoring.

Unprivileged Debug Extension

In an implementation that does not include Armv8.1-M, there is no support for the Unprivileged Debug Extension. Support for the Unprivileged Debug Extension is part of the basic level of debug.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

See also:

[B13.1.1 Debug mechanisms.](#)

[Halting debug.](#)

[DebugMonitor exception.](#)

[B13.4.4 Breakpoint instructions.](#)

[B14.1 Instrumentation Trace Macrocell.](#)

[B14.2 Data Watchpoint and Trace unit.](#)

[B14.3 Embedded Trace Macrocell.](#)

[B14.4 Trace Port Interface Unit.](#)

[B14.5 Flash Patch and Breakpoint unit.](#)

[DEMCR, Debug Exception and Monitor Control Register.](#)

[DFSR, Debug Fault Status Register.](#)

[DHCSR, Debug Halting Control and Status Register.](#)

[DCRDR, Debug Core Register Data Register.](#)

[DCRSR, Debug Core Register Select Register.](#)

[ID_DFR0, Debug Feature Register.](#)

[DWT_PCSR, DWT Program Control Sample Register .](#)

B13.1.1 Debug mechanisms

R_{HWCH} Armv8-M supports a range of invasive and non-invasive debug mechanisms. The *invasive debug mechanisms* are:

- The ability to halt the PE. This provides a run-stop debug model.
- Debugging code using the DebugMonitor exception. This provides less intrusive debug than halting the PE.

The *non-invasive debug techniques* are:

- Generating application trace by writing to the *Instrumentation Trace Macrocell* (ITM), causing a low level of intrusion.
- Non-intrusive program trace and profiling.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**. Note, M is required for the DebugMonitor exception and ITM.*

I_{LBLF} When the PE is halted, it is in *Debug state*.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

I_{SXVR} When the PE is not halted, it is in *Non-debug state*.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

I_{HWPQ} Non-invasive debug components do not guarantee that they do not make any changes to the behavior or performance of the PE. Any changes that do occur must not be severe however, as this will reduce the usefulness of event counters for performance measurement and profiling. This does not include any change to program behavior that results from the same program being instrumented to use a Non-invasive debug feature, for example the Performance Monitors, or from some other performance monitoring process being run concurrently with the process being profiled. As such, a reasonable variation in performance is permissible.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DMQP} The *Unprivileged Debug Extension*, UDE, allows for a finer-grained control of debug access to the PE. UDE allows conditional debug capabilities when the PE is in an unprivileged mode, including support for the DebugMonitor exception and the optional Halting debug and non-invasive debug.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **UDE**.*

See also:

[B13.2 Accessing debug features.](#)

B13.1.2 Debug resources

R_{TZVG} In the system address map, debug resources are in the *Private Peripheral Bus* (PPB) region.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{FBHD} Except for the resources in the SCS, each debug component occupies a fixed 4KB address region.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

R_{WXTK} The debug resources in the SCS are:

- The *Debug Control Block* (DCB).
- Debug controls in the *System Control Block* (SCB).

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

I_{KKBT} If the Main Extension is implemented, then support for DebugMonitor is implemented. If the Main Extension is not implemented, then DebugMonitor is not supported.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

Chapter B13. Debug
B13.1. Debug feature overview

R_{VMGD} ROM table entries identify which optional debug components are implemented.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB**.

R_{RRLF} The addresses of the debug resources are:

Address range	Debug Resource
0xE0000000–0xE0000FFF	Instrumentation Trace Macrocell (ITM)
0xE0001000–0xE0001FFF	Data Watchpoint and Trace Unit (DWT)
0xE0002000–0xE0002FFF	Flashpatch and Breakpoint Unit (FPB)
0xE0003000–0xE0003FFF	Performance Monitor Unit (PMU)
0xE000E000–0xE000EFFF	Secure SCS
	0xE000EC00–0xE000ED8F Secure and Non-secure System Control Block (SCB)
	0xE000EDF0–0xE000EEFF Secure and Non-secure Debug Control Block (DCB)
0xE002E000–0xE002EFFF	Non-secure SCS
	0xE002EC00–0xE002ED8F Non-secure System Control Block (SCB)
	0xE002EDF0–0xE002EEFF Non-secure Debug Control Block (DCB)
0xE0040000–0xE0040FFF	Trace Port Interface Unit (TPIU), when not implemented as a shared resource otherwise reserved.
0xE0041000–0xE0041FFF	Embedded Trace Macrocell (ETM)
0xE0042000–0xE00FEFFF-	IMPLEMENTATION DEFINED
0xE00FF000–0xE00FFFFF	ROM table

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB**.

See also:

[B14.1 Instrumentation Trace Macrocell.](#)

[B14.2 Data Watchpoint and Trace unit.](#)

[B14.5 Flash Patch and Breakpoint unit.](#)

[Chapter B8 The System Address Map.](#)

[B13.2.2 Debug System registers.](#)

[B14.4 Trace Port Interface Unit.](#)

[B14.3 Embedded Trace Macrocell.](#)

[B13.2.1 ROM table.](#)

[B13.2 Accessing debug features.](#)

B13.1.3 Trace

R_{CZKQ} Trace can be generated by using the:

- Embedded Trace Macrocell (ETM).
- Instrumentation Trace Macrocell (ITM).
- Data Watchpoint and Trace (DWT) unit.
- Performance Monitoring Unit (PMU).

Chapter B13. Debug
B13.1. Debug feature overview

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - *ITM* && (*ETM* || *DWTT* || *PMU*).

R_{TBHB}

A debug implementation that generates trace includes a trace sink, such as a TPIU.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - (*ETM* || *DWTT* || *PMU*) && *ITM* && *TPIU*.

I_{RJKJ}

A TPIU can be either the Armv8-M TPIU implementation, or an external system resource.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - *ETM* || *ITM* || *DWTT* || *PMU*.

See also:

[ITM and DWT Packet Protocol Specification](#).

The applicable ETM Architecture Specification.

B13.2 Accessing debug features

R _{WVSZ}	The mechanism by which an external debugger accesses the PE and system is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
I _{QPHR}	A debugger can use a <i>Debug Access Port</i> (DAP) interface, such as the DAP described in the <i>Arm®Debug Interface v5 Architecture Specification</i> (ADIV5) and <i>Arm®Debug Interface v6 Architecture Specification</i> (ADIV6), to interrogate a system for Memory Access Ports (MEM-APs). The base register in a Memory Access Port provides the address of the ROM table, or the first in a series of ROM tables in a ROM table hierarchy. The Memory Access Port can then fetch the ROM table entries. Arm recommends implementation of at least an ADIV5 DAP for compatibility with tools. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
I _{TTLK}	The DAP interface supports word, halfword and byte accesses. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{KSJT}	The Debug Access Port is an Observer, and observes all accesses that the PE makes. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{JRHS}	Software configures and controls the debug features through memory-mapped registers. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{WPGQ}	A write from a DAP memory access are complete when the DAP reports the write as complete. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{WCQK}	For SCS registers, a write from a DAP is complete when the write has completed and the SCS register has been updated. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
I _{SXFW}	The <i>Arm®Debug Interface v5 Architecture Specification</i> (ADIV5) and <i>Arm®Debug Interface v6 Architecture Specification</i> (ADIV6) specify a Memory Access Port (MEM-AP), and a memory access completes when it is capable of being observed by the MEM-AP and other observers in the system. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>

See also:

[B13.2.1 ROM table.](#)

[B13.3.4 DAP access permissions.](#)

[B7.2.3 Ordering and observability.](#)

The *Arm®Debug Interface v5 Architecture Specification*.

The *Arm®Debug Interface v6 Architecture Specification*.

B13.2.1 ROM table

I _{XFEV}	The ROM table is a table of entries providing a mechanism to identify the debug infrastructure that is supported by the implementation. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
-------------------	--

Chapter B13. Debug

B13.2. Accessing debug features

I _{FWPG}	The ROM table indicates the implemented debug components, and the position of those components in the memory map. See the <i>Arm® Debug Interface v5 Architecture Specification</i> for the format of a ROM table entry. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
I _{PHJJ}	For an Armv8-M ROM table, all entry offsets are negative. The ROM table entry points to the top of a 4KB page, the offset points to the bottom of that page that contains the Peripheral and Component ID registers. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{GPPX}	The ROM table is implemented if any other debug component is implemented or a Debug Access Port is implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{BQSP}	Bit[0] of the ROM table entries indicates whether the corresponding debug component is implemented and is accessible through the PPB at the indicated address. If the corresponding debug component is not implemented, this bit has a value of 0. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{NDQW}	If a debug component is implemented, debug registers can provide additional information about the implemented features of that debug component. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{ZZGJ}	The format of the ROM table is:

Chapter B13. Debug
B13.2. Accessing debug features

Offset	Value	Name	Description
0x000	0xFFFF0F003	ROMSCS	Points to the SCS at 0xE000E000
0x004	0xFFFF02002 or 0xFFFF02003	ROMDWT	Points to the Data Watchpoint and Trace unit at 0xE0001000
0x008	0xFFFF03002 or 0xFFFF03003	ROMFPB	Points to the Flash Patch and Breakpoint unit at 0xE0002000
0x00C	0xFFFF01002 or 0xFFFF01003	ROMITM	Points to Instrumentation Trace unit at 0xE0000000.
0x010	0xFFFF41002 or 0xFFFF41003	ROMTPIU	Points to the Trace Port Interface Unit.
0x014	0xFFFF42002 or 0xFFFF42003	ROMETM	Points to the Embedded Trace Macrocell.
0x018	0xFFFF04002 or 0xFFFF04003	ROMPMU	Points to the Performance Monitoring Unit at 0xE0003000.
-	0x00000000	End	End of table marker. It is IMPDEF whether the table is extended with pointers to other system debug resources. The table entries always terminate with a null entry.
0x020 - 0xEFC	-	Not used	Reserved for additional ROM table entries.
0xF00 - 0xFC8	-	Reserved	Reserved, not used for ROM table entries.
0xFCC	0x00000001	MEMTYPE	Bit [0] is set to 1 to indicate that resources other than those listed in the ROM table are accessible in the same 32-bit address space, using the DAP.Bits [31:1] of the MEMTYPE entry are RES0.
0xFD0	IMP DEF	PIDR4	CIDRx values are fully defined for the ROM table, and are CorseSight compliant.
0xFD4	0	PIDR5	PIDRx values are CoreSight compliant or RAZ.
0xFD8	0	PIDR6	
0xFDC	0	PIDR7	
0xFE0	IMP DEF	PIDR0	
0xFE4	IMP DEF	PIDR1	
0xFE8	IMP DEF	PIDR2	
0xFEC	IMP DEF	PIDR3	
0xFF0	0x0000000D	CIDR0	
0xFF4	0x00000010	CIDR1	
0xFF8	0x00000005	CIDR2	
0xFFC	0x000000B1	CIDR3	

Accesses to the ROMITM cannot cause a non-existent memory exception.

It is IMPLEMENTATION DEFINED whether the ETM and TPIU are a shared resource and whether the resource is managed by the local PE or a different resource.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB** and those indicated in the table. Note, PMU is only available in an Armv8.1-M implementation.*

RJDGV

If a PMU is implemented, the end-of-table marker is 0x1C and has a value of 0x00000000. It is IMPLEMENTATION DEFINED whether the table is extended with pointers to other System debug resources.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **DB** & **PMU**.

See also:

[B13.2.3 CoreSight and identification registers.](#)

B13.2.2 Debug System registers

R_{RHDW}

The debug provision in the *System Control Block* (SCB) comprises:

- Two handler-related flag bits, **ICSR.ISRPREEMPT** and **ICSR.ISRPENDING**.
- The **DFSR**.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.

See also:

[Part D Register and Payload Specification.](#)

[Debug Control Block.](#)

B13.2.3 CoreSight and identification registers

I_{CMLH}

Arm recommends that CoreSight-compliant ID registers are implemented to allow identification and discovery of the components to a debugger.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.

R_{QCWD}

The address spaces that are reserved in each of the debug components for IMPLEMENTATION DEFINED ID registers and CoreSight compliance are:

Debug Component	Space reserved for ID registers	Space reserved for CoreSight compliance
ITM	0xE0000FD0-0xE0000FFC	0xE0000FA0-0xE0000FCC
DWT	0xE0001FD0-0xE0001FFC	0xE0001FA0-0xE0001FCC
FPB	0xE0002FD0-0xE0002FFC	0xE0002FA0-0xE0002FCC
PMU	0xE0003FD0-0xE0003FFC	0xE0003FA0-0xE0003FCC
SCS	0xE000EFD0-0xE000EFFC	0xE000EFA0-0xE000EFCC
TPIU	0xE0040FD0-0xE0040FFC	0xE0040FA0-0xE0040FCC
ETM	0xE0041FD0-0xE0041FFC	0xE0041FA0-0xE0041FCC
ROM table	0xE00FFFD0-0xE00FFFFC	0xE00FFFA0-0xE00FFFCC

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**. Note, PMU available only in an Armv8.1-M implementation.

R_{VWSX}

For the ROM table, the ID register space is used for a set of CoreSight-compliant ID registers.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.

R _{HXDK}	For all components other than the ROM table, if the registers in the ID register space are not used for ID registers they are RAZ. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{VQPM}	If CoreSight-compliant ID registers are implemented, the Class field in Component ID Register 1 is: • 0x1 for the ROM table. • 0x9 for other components. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
I _{HQSR}	The Part number in the PIDR registers must be assigned a unique value for each implementation, or Unique Component Identifier, as with all other CoreSight components. CoreSight permits that two or more functionally different components are permitted to share the same Part number, so long as they have different values of the DEVTYPE or DEVARCH registers. Each component has its own DEVTYPE and DEVARCH registers, for example: • DDEVTYPE. • DDEVARCH. • DWT_DEVTYPE. • DWT_DEVARCH. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
I _{CTBF}	The Part number in the PIDR registers do not need to be unique for different implementation options of the same part. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>

B13.3 Debug authentication interface

R_{GWTN} The following pseudocode functions provide an abstracted description of the authentication interface:

- `ExternalInvasiveDebugEnabled()`.
- `ExternalSecureInvasiveDebugEnabled()`.
- `ExternalNoninvasiveDebugEnabled()`.
- `ExternalSecureNoninvasiveDebugEnabled()`.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

R_{SWWT} For an implementation using the CoreSight signals **DBGEN**, **NIDEN**, **SPIDEN**, and **SPNIDEN**:

- `ExternalInvasiveDebugEnabled()` returns TRUE if **DBGEN** is asserted.
- `ExternalSecureInvasiveDebugEnabled()` returns TRUE if both **DBGEN** and **SPIDEN** are asserted.
- `ExternalNoninvasiveDebugEnabled()` returns TRUE if either **NIDEN** or **DBGEN** is asserted.
- `ExternalSecureNoninvasiveDebugEnabled()` returns TRUE if both of the following conditions apply:
 - Either **NIDEN** or **DBGEN** is asserted.
 - Either **SPNIDEN** or **SPIDEN** is asserted.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

I_{FCPK} The unprivileged debug capabilities enabled by `DAUTHCTRL.UIDEN` being set to 1 are available regardless of the state of the following:

- `DAUTHCTRL.INTSPNIDEN`.
- `DAUTHCTRL.SPNIDENSEL`.
- `DAUTHCTRL.INTSPIDEN`.
- `DAUTHCTRL.SPIDENSEL`.
- **DBGEN**.
- **SPIDEN**.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **HDBG** && **UDE**. Note, *S* is required for Secure Behavior.*

R_{HVGN} For any implementation of the authentication interface, if `ExternalInvasiveDebugEnabled()` is FALSE, then `ExternalSecureInvasiveDebugEnabled()` is FALSE.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

R_{JWCS} For any implementation of the authentication interface, if `ExternalNoninvasiveDebugEnabled()` is FALSE, then `ExternalSecureNoninvasiveDebugEnabled()` is FALSE.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

R_{XCMD} For any implementation of the authentication interface, if `ExternalInvasiveDebugEnabled()` is TRUE, then `ExternalNoninvasiveDebugEnabled()` is TRUE.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

R_{LCHH} For any implementation of the authentication interface, if `ExternalSecureInvasiveDebugEnabled()` is TRUE, then `ExternalSecureNoninvasiveDebugEnabled()` is TRUE.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

I_{MSRG} Secure self-hosted debug is controlled by the authentication interface. The pseudocode function [ExternalSecureSelfHostedDebugEnabled\(\)](#) provides an abstracted description of this authentication interface.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

R_{GLWM} Between a change to the debug authentication interface and a following [Context synchronization event](#), it is UNPREDICTABLE whether the PE uses the old or the new values.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB**.*

See also:

[B13.3.1 Halting debug authentication.](#)

[B13.3.3 DebugMonitor exception authentication.](#)

[B13.3.2 Non-invasive debug authentication.](#)

[B13.3.4 DAP access permissions.](#)

B13.3.1 Halting debug authentication

I_{DMFG} Halting debug authentication is controlled by the IMPLEMENTATION DEFINED authentication interface function [ExternalInvasiveDebugEnabled\(\)](#), and if the Security Extension is implemented, the IMPLEMENTATION DEFINED authentication interface function [ExternalSecureInvasiveDebugEnabled\(\)](#).

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**. Note, External Secure invasive debug requires S.*

R_{PHWV} Halting is prohibited in privileged mode in all states if the function [ExternalInvasiveDebugEnabled\(\)](#) returns FALSE. If UDE is not implemented this rule applies to all privilege modes.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **HDBG**.*

R_{JXTX} When the PE is halted, the PE behaves as if [ExternalInvasiveDebugEnabled\(\)](#) is TRUE. The pseudocode function [HaltingDebugAllowed\(\)](#) describes this.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

I_{BCZM} If the Security Extension is not implemented, there are two Halting debug authentication modes:

ExternalInvasiveDebugEnabled()	DHCSR.S_HALT	Halting debug authentication mode
FALSE	0	Halting is prohibited.
FALSE	1	Halting is allowed.
TRUE	X	Halting is allowed.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG** && **!S** && **!UDE**.*

R_{BMRJ} Halting is prohibited in privileged modes in Secure state if any of the following conditions are TRUE:

- The IMPLEMENTATION DEFINED [ExternalInvasiveDebugEnabled\(\)](#) returns FALSE.
- [DAUTHCTRL.SPIDENSEL](#) is set to 1 and [DAUTHCTRL.S.INTSPIDEN](#) is set to 0.
- [DAUTHCTRL.SPIDENSEL](#) is set to 0 and [ExternalSecureInvasiveDebugEnabled\(\)](#) returns FALSE.

The pseudocode function `SecureHaltingDebugEnabled` describes this.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG** && **S**.

I_{XKFR}

If the Security Extension is not implemented and the Unprivileged Debug Extension is implemented, Halting debug has the following authentication modes:

<code>HaltingDebugEnabled()</code>	<code>DAUTHCTRL.UIDEN</code>	Halting debug authentication mode
FALSE	0	Halting is prohibited in all modes.
FALSE	1	Halting is allowed in unprivileged mode. Halting is prohibited in privileged mode.
TRUE	X	Halting is allowed in all modes.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **HDBG** && **!S** && **UDE**.

R_{HXP}

Halting in unprivileged modes is prohibited in Secure state if all of the following are true:

- `SecureHaltingDebugEnabled()` returns FALSE.
- `DAUTHCTRL.S.UIDEN` is set to 0.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **UDE** && **HDBG** && **S**.

R_{XFKR}

Halting in unprivileged modes is prohibited in Non-secure state if all of the following are true:

- `HaltingDebugEnabled()` returns FALSE.
- `DAUTHCTRL.S.UIDEN` is set to 0.
- `DAUTHCTRL.NS.UIDEN` is set to 0.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **UDE** && **HDBG** && **S**.

I_{VKHZ}

`DAUTHCTRL.NS.UIDEN` controls whether debug request can be raised from unprivileged mode in Non-secure state. `DAUTHCTRL.S.UIDEN` controls whether debug requests can be raised from Secure or Non-secure state. When `DAUTHCTRL.S.UIDEN` is set to 1 `DAUTHCTRL.NS.UIDEN` has an Effective value of 1.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **HDBG** && **UDE**. Note, **S** is required for Secure behavior.

R_{VQJZ}

The value of `DHCSR.S_SDE`, `DHCSR.S_SUIDE` and `DHCSR.S_NSUIDE` are updated when the PE is not halted as described by the function `UpdateDebugEnabled()`.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **HDBG**. Note, **S** is required for Secure behavior. **UDE** is required for Unprivileged Debug.

R_{KBKM}

If the PE is in Non-debug state the following condition is true:

- `DHCSR.S_SDE` reads as one if the following is true, and reads as zero otherwise:
 - `SecureHaltingDebugEnabled()` returns TRUE.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG** && **!UDE**. Note, **S** is required for Secure behavior.

R_{CVKR}

If the PE is in Non-debug state the following conditions are true:

- `DHCSR.S_SDE` reads as one if any one of the following are true, and reads as zero otherwise:
 - `SecureHaltingDebugEnabled()` returns TRUE.
 - `UnprivHaltingDebugEnabled(TRUE)` returns TRUE.
- `DHCSR.S_SUIDE` reads as one if all of the following are true and reads as zero otherwise:

- `SecureHaltingDebugEnabled()` returns FALSE.
- `UnprivHaltingDebugEnabled(TRUE)` returns TRUE.
- **DHCSR.S_NSUIDE** reads as one if both of the following are true and reads as zero otherwise:
 - `HaltingDebugEnabled()` returns FALSE.
 - `UnprivHaltingDebugEnabled(FALSE)` returns TRUE.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **HDBG** && **UDE**. Note, *S* is required for Secure behavior.

R_KMXG

If the PE is in Debug state:

- **DHCSR.S_SDE** reads as one if either of the following is true, and reads as zero otherwise:
 - The PE entered Debug state from Secure state.
 - The PE entered Debug state from Non-secure state when `SecureHaltingDebugEnabled()` returned TRUE.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG** && **!UDE**. Note, *S* is required for Secure behavior.

R_TNMM

If the PE is in Debug state:

- **DHCSR.S_SDE** reads as one if any one of the following is true, and reads as zero otherwise:
 - The PE entered Debug state from Secure state.
 - The PE entered Debug state from Non-secure state when `SecureHaltingDebugEnabled()` returned TRUE.
 - The PE entered Debug state from Non-secure state when `UnprivHaltingDebugEnabled(TRUE)` returned TRUE.
- **DHCSR.S_SUIDE** reads as one if when the PE entered Debug state both of the following were true, and reads as zero otherwise:
 - `SecureHaltingDebugEnabled()` returned FALSE.
 - `UnprivHaltingDebugEnabled(TRUE)` returned TRUE.
- **DHCSR.S_NSUIDE** reads as one if when the PE entered Debug state both the following were true, and reads as zero otherwise:
 - `HaltingDebugEnabled()` returned FALSE.
 - `UnprivHaltingDebugEnabled(FALSE)` returned TRUE.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **HDBG**. Note, *S* is required for Secure behavior.

I_LDTR

If the Security Extension is implemented, there are three Halting debug authentication modes:

HaltingDebugEnabled()	DHCSR.S_SDE	Halting debug authentication mode
FALSE	X	Halting is prohibited.
TRUE	0	Halting is allowed in Non-secure state. Halting is prohibited in Secure state.
	1	Halting is allowed.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG** && **S** && **!UDE**.

I_{LQVH}

In an Armv8.1-M PE, unprivileged debug is authentication is as follows:

Halting DebugAllowed()	SecureHalting DebugAllowed()	DAUTHCTRL_S. UIDEN	DAUTHCTRL_NS. UIDEN	Halting debug authentication mode
FALSE	FALSE	0	0	Halting is prohibited.
FALSE	FALSE	0	1	Halting is allowed in Non-secure unprivileged mode. Halting is prohibited in Secure unprivileged mode. Halting is prohibited in Non-secure privileged mode. Halting is prohibited in Secure privileged mode.
FALSE	FALSE	1	X	Halting is allowed in Secure and Non-secure unprivileged mode. Halting is prohibited in Secure and Non-secure privileged mode.
TRUE	FALSE	0	X	Halting is allowed in Non-secure state and in all privilege modes.
TRUE	FALSE	1	X	Halting is allowed in Non-secure state and in Secure unprivileged mode.
TRUE	TRUE	X	X	Halting is allowed in all Security states and privilege modes.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **HDBG** && **UDE**. Note, *S* required for Secure state.

R_{FXCB}

When **DHCSR.C_DEBUGEN** == 0 or the PE is in a state in which Halting is prohibited, the PE does not enter Debug state.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG**. Note, *S* is required for Secure behavior.

See also:

[CanHaltOnEvent\(\)](#).

B13.3.2 Non-invasive debug authentication

R_{GFTG}

Non-invasive authentication is controlled by the IMPLEMENTATION DEFINED function:

[ExternalNoninvasiveDebugEnabled\(\)](#).

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB** && **!S**.

R_{HXQD}

Secure Non-invasive authentication is controlled by the IMPLEMENTATION DEFINED functions:

- [ExternalSecureNoninvasiveDebugEnabled\(\)](#).
- [ExternalNoninvasiveDebugEnabled\(\)](#).

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB** && **S**.

R _{CFNB}	When <code>HaltingDebugEnabled()</code> is TRUE, the PE behaves as if <code>ExternalNoninvasiveDebugEnabled()</code> returns TRUE. The pseudocode function <code>NoninvasiveDebugEnabled()</code> describes this. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB && UDE.</i>
R _{LNCF}	Non-invasive debug of an operation is prohibited unless any of the following return TRUE: • <code>ExternalNoninvasiveDebugEnabled()</code>. • <code>HaltingDebugEnabled()</code>. • <code>UnprivHaltingDebugEnabled()</code> and the PE is in unprivileged mode. The pseudocode function <code>NoninvasiveDebugEnabled()</code> describes this. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DB && UDE.</i>
R _{QMRF}	Non-invasive debug is prohibited if the functions <code>SecureNoninvasiveDebugEnabled()</code> and <code>NoninvasiveDebugEnabled()</code> both return FALSE. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB && S.</i>
R _{GZZX}	Non-invasive debug of an operation in Secure state is prohibited unless any of the following are true: • <code>DHCSR.S_SDE</code> is set to one and UDE is not implemented or is not permitted in Secure state. • The PE is in unprivileged mode and <code>UnprivHaltingDebugEnabled(TRUE)</code> returns TRUE. • <code>DAUTHCTRL.SPNIDENSEL</code> is set to 1 and <code>DAUTHCTRL_S.INTSPIDEN</code> is set to 1. • <code>ExternalSecureNoninvasiveDebugEnabled()</code> returns TRUE. The pseudocode function <code>SecureNoninvasiveDebugEnabled()</code> describes this. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB && S && UDE.</i>
R _{LXRX}	The PE does not generate any trace or profiling data when non-invasive debug of that operation is prohibited. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{VYGT}	If non-invasive debug of Secure operations is prohibited, the PE does not generate any trace or profiling data that contains secure information or is attributable to secure operations. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB && S.</i>
R _{PSSV}	If non-invasive debug of privileged operations is prohibited, the PE does not generate any trace or profiling data that contains privileged information or is attributable to privileged operations. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DB && UDE.</i>
R _{TWDH}	If non-invasive debug is prohibited in the current Security state, an ETM behaves as described in the relevant ETM architecture. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB && S && ETM.</i> See also: <code>NoninvasiveDebugEnabled()</code>. <code>SecureNoninvasiveDebugEnabled()</code>. B14.2.2 DWT unit operation.

B13.3.3 DebugMonitor exception authentication

R _{MXTM}	DebugMonitor exception authentication is only available if the Main Extension is implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
-------------------	--

R_{LQCN} DebugMonitor exception authentication is controlled by the IMPLEMENTATION DEFINED authentication interface function `ExternalSecureSelfHostedDebugEnabled()`.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M** && **S**.*

R_{GTSQ} Unless otherwise stated DebugMonitor exceptions are never generated for Secure privileged operations if all of the following conditions are true:

- **DAUTHCTRL.FSDMA** is set to 0.
- Any of the following conditions are true:
 - **DAUTHCTRL.SPIDENSEL** is set to 1 and **DAUTHCTRL.INTSPIDEN** is set to 0.
 - **DAUTHCTRL.SPIDENSEL** is set to 0 and `ExternalSecureSelfHostedDebugEnabled()` returns FALSE.

The pseudocode function `SecureDebugMonitorAllowed()` describes this.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M** && **S**. Note, **DAUTHCTRL.FSDMA** is only available in an Armv8.1-M implementation.*

I_{PWJZ} If UDE is implemented **DAUTHCTRL.FSDMA** allows the Secure DebugMonitor exception to be enabled independently of Halting debug and `ExternalSecureSelfHostedDebugEnabled()`. This field does not control the DebugMonitor exception permissions directly, instead **DAUTHCTRL.FSDMA** is used as an input into **DEMCR.SDME**, as described by `UpdateDebugEnable()` and `SecureDebugMonitorAllowed()`.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **M** && **S** && **UDE**.*

R_{WXM} When a DebugMonitor exception is not pending and is not active:

- **DEMCR.SDME** is set to 1 if `SecureDebugMonitorAllowed()` is TRUE.
- **DEMCR.SDME** is zero otherwise.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M**.*

R_{CPPN} If `SecureDebugMonitorAllowed()` returned TRUE when a DebugMonitor exception is pending or active:

- **DEMCR.SDME** is set to 1 when the DebugMonitor exception became pending or active.
- **DEMCR.SDME** is zero otherwise.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M**.*

R_{YFPK} If **DEMCR.SDME** == 1, **SHPR3.PRI_12** behaves as RAZ/WI when accessed from Non-secure state.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M** && **S**.*

I_{HNGD} When set to 1, **DEMCR.MON_PEND** remains set to 1 until one of the following occurs:

- The DebugMonitor exception is taken
- A write to **DEMCR** sets the **DEMCR.MON_PEND** to 0.
- The PE is reset.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M**.*

R_{SCZH} DebugMonitor exceptions are never generated for unprivileged operations if all of the following conditions are true:

- **DEMCR.MON_EN** is 0.
- **DEMCR.UMON_EN** is 0.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **UDE** && **M**.*

R_{CKPB} DebugMonitor exceptions are never generated for Secure unprivileged operations when all of the following are true:

- `SecureDebugMonitorAllowed()` returns FALSE.
- **DEMCR.UMON_EN** is 0 and **DEMCR.MON_EN** is 0.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **M** && **S** && **UDE**.

See also

DEMCR, Debug Exception Monitor Control Register.

`ExternalSecureSelfHostedDebugEnabled()`.

`SecureDebugMonitorAllowed()`.

`CanPendMonitorOnEvent()`.

`UpdateDebugEnabled()`.

B13.3.4 DAP access permissions

R_{BFSB} When `HaltingDebugEnabled()` is TRUE the external debugger can issue requests to the entire physical address space through the DAP. When the DAP accesses a System register the response to the DAP is defined by the access permissions of the register being accessed.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB**.

R_{FFPN} The external debugger is capable of requesting Secure and Non-secure accesses through the DAP.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB** && **S**.

R_{VDPK} The external debugger is capable of requesting Privileged and Unprivileged accesses through the DAP.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DB** && **UDE**.

I_{XNTP} **DAUTHCTRL.UIDAPEN** indicates software intent to allow the external debugger unprivileged access through the DAP to selected PPB registers.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DB**. Note, **DAUTHCTRL.UIDAPEN** requires **UDE**.

R_{TRLZ} When either **DAUTHCTRL.S.UIDAPEN** or **DAUTHCTRL.NS.UIDAPEN** is set, and no other debug authorization is active, the PE permits the external debugger to request unprivileged access through the DAP to the System PPB space. If the external debugger makes a privileged request, when **DAUTHCTRL.UIDAPEN** is set in either Security state the PE demotes the request to an unprivileged request. An unprivileged access will return an error to the DAP unless otherwise specified by the register description.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DB**. Note, **DAUTHCTRL.UIDAPEN** requires **UDE**.

R_{TXZB} If `HaltingDebugEnabled()` is FALSE, then:

- Unless otherwise stated, external debugger accesses through the DAP to addresses in the range 0x00000000 to 0xFFFFFFFF return an error to the DAP.
- If either bank of **DAUTHCTRL.UIDAPEN** is one, external debugger accesses through the DAP to addresses outside the range 0xE0000000 to 0xE00FFFFF return an error to the DAP.
- If neither bank of **DAUTHCTRL.UIDAPEN** is one, external debugger accesses are permitted as detailed in the table below:

Address Range	Component or description	NoninvasiveDebugAllowed()	
		FALSE	TRUE
0x00000000 – 0xDFFFFFFF	Rest of Memory	DAP error	DAP error
0xE0000000 – 0xE00FFFFFFF	PPB		
0xE00xxFB0 – 0xE00xxFB7	CoreSight Software Lock registers	DAP error	RW
0xE00xxFD0 – 0xE00xxFFF	CoreSight peripheral and component ID registers	RO	RO
0xE0000000 – 0xE0000FCF	ITM	DAP error	RW
0xE0001000 – 0xE0001FCF	DWT	DAP error	RW
0xE0003000 – 0xE0003FCF	PMU	DAP error	RW
0xE0040000 – 0xE0040FFF	TPIU	RW	RW
0xE0041000 – 0xE0041FFF	ETM	RW	RW
0xE0042000 – 0xE00FEFFF	IMPDEF	IMPDEF	IMPDEF
0xE00FF000 – 0xE00FFFFFFF	ROM table	RO	RO
	All other PPB regions	DAP error	DAP error
0xE0100000 – 0xFFFFFFFF	Vendor_SYS	DAP error	RW

Applies to an implementation of the architecture **Armv8.0-M** onward. The extension requirements are - **DB**. Note, **DAUTHC-TRL.UIDAPEN** requires **UDE**. **UDE** and **PMU** are only available in an **Armv8.1-M** implementation.

I_{LGXS} Where this rule does not prevent an external debugger access to a register through the DAP, the access permissions are defined by the register. The access permissions are set out in the Usage Constraints section of each register description.

Applies to an implementation of the architecture **Armv8.0-M** onward. The extension requirements are - **DB**.

R_{KBQZ} A DAP memory access calls **MemD_with_priv_security()** and **DAPCheck()**.

Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - **DB**. Note, Secure accesses require **S** and unprivileged accesses require **UDE**.

I_{DKCB} The architecture does not describe how a DAP requests Secure or Non-secure memory accesses. In the recommended **ADIV5** Memory Access Port (MEM-AP), Arm recommends that, when **CSW.DEVICEEN** is set to one:

- **CSW[30]**, **CSW.Prot[6]**, selects a Secure or Non-secure access:
 - **0**: Request a Secure access.
 - **1**: Request a Non-secure access.
- **CSW[23]**, **CSW.SPIDEN**, must be one. This is because the DAP can always request a Secure access.

When **CSW.DEVICEEN** is zero the MEM-AP is incapable of requesting any accesses. In **ADIV6** **CSW[23]** is **SDeviceEN**, but the behavior remains the same.

Applies to an implementation of the architecture **Armv8.0-M** onward. The extension requirements are - **DB** && **S**.

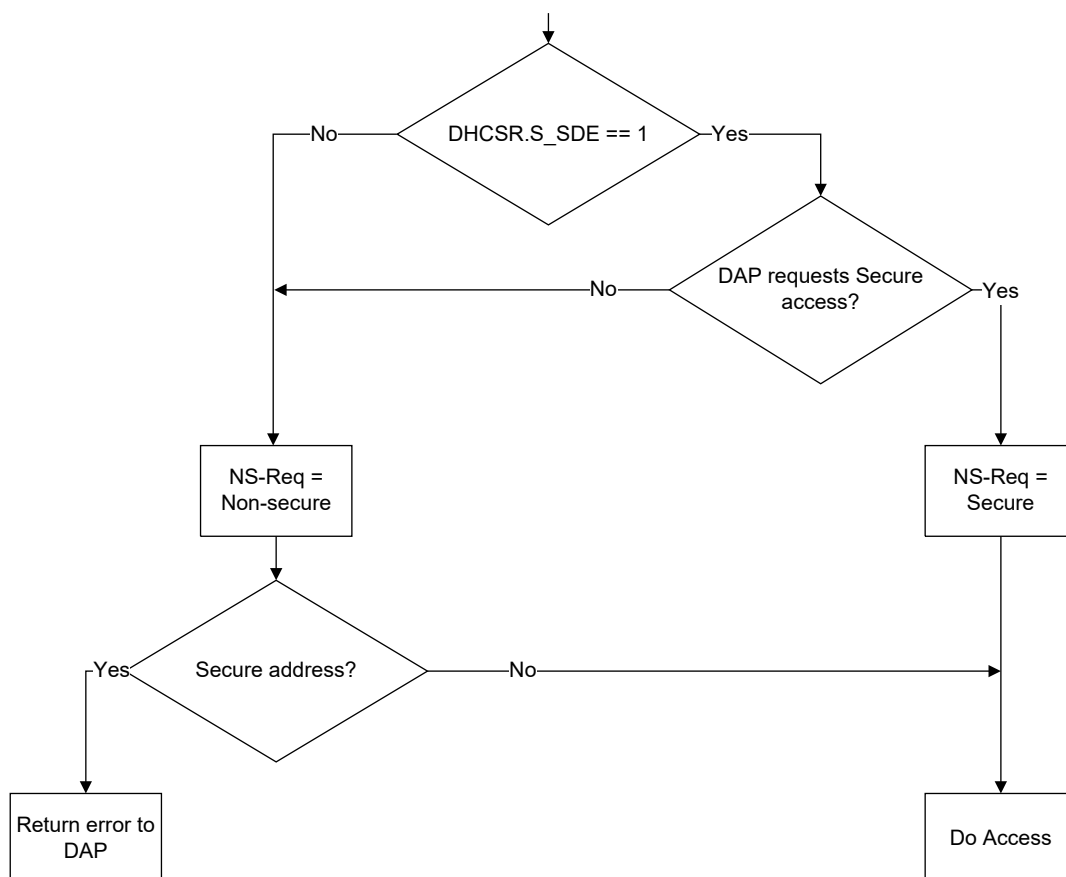
I_{RVWM} The architecture does not describe how a DAP requests Privileged or Unprivileged memory accesses. In the recommended **ADIV5** or higher Memory Access Port (MEM-AP), Arm recommends that the **IMPLEMENTATION DEFINED** field **CSW[27:24]**, **PROT**, flags are used to define Privileged or Unprivileged accesses.

Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - **DB** && **UDE**.

I_{MRMX} In a CoreSight DAP, the **SPIDEN** input to the **Armv8-M** MEM-AP is independent of the **SPIDEN** input of the PE, and must be tied HIGH when **CSW.DEVICEEN** is one. When **CSW.DEVICEEN** is zero the MEM-AP is disabled and is incapable of making any requests for memory accesses.

Applies to an implementation of the architecture **Armv8.0-M** onward. The extension requirements are - **DB** && **S**.

- R_{JHBC}** If **DHCSR.S_SDE** == 1, and the DAP requests a Secure access, NS-Req is set to Secure.
*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB** && **S**.*
- R_{LVBG}** If either **DHCSR.S_SDE** == 0 or the DAP requests a Non-secure access, NS-Req set to Non-secure.
*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB** && **S**.*
- R_{WMRR}** DAP accesses are checked by the IDAU and the SAU, if applicable. That is, if NS-Req on a DAP access specifies Non-secure access, and the IDAU or SAU prohibits Non-secure access to the address, an error response is returned to the DAP.



*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB** && **S**.*

- R_{VTTN}** Unless otherwise stated DAP accesses are not checked by the MPU.
*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB** && **MPU**.*
- R_{TYBC}** The DAP NS-Req signal is ignored for accesses to the SCS registers. Instead, NS-Req is derived from the **DSCSR.SBRSELEN** and **DSCSR.SBRSEL**. The derived NS-Req is used for all permissions checks, including the privilege checks and demotions.
*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DB** && **S**.*
- R_{WBNK}** DAP accesses to SCS registers in the range 0xE000E000-0xE000EFFF are affected by the values of **DSCSR.SBRSELEN**, and **DSCSR.SBRSEL**, as well as by the current Security state of the PE. The following table shows the effect of these factors on the register being viewed.

	DSCSR.SBRSELEN	DSCSR.SBRSEL	Current Security state of the PE	View of register accessed
	1	0	X	Non-secure.
	1	1	X	Secure.
	0	X	Non-secure.	Non-secure.
	0	X	Secure.	Secure.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB** && **S**.

I_{KLPQ} When **DHCSR.S_SDE** == 0, then **DSCSR.SBRSELEN** reads as 1 and **DSCSR.SBRSEL** reads as 0, which selects the Non-secure bank of registers.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB** && **S**.

R_{PMNV} Permitted DAP accesses to the region 0xE002E000-0xE002EFFF are RAZ/WI if the access is privileged or unprivileged. Privileged accesses never return a DAP error, and it is IMPLEMENTATION DEFINED whether an unprivileged access returns a DAP error.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB**. Note, This rule also applies to unprivileged accesses via *Armv8.1-M* UDE.

I_{HBGQ} An *Armv8.1-M* PE with UDE extends the existing DAP access regime for the Main Extension.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DB** && **UDE**.

R_{RXKV} When **DAUTHCTRL.UIDAPEN** is set, unprivileged debugger accesses to reserved locations within the PPB are treated as RES0 and do not return a DAP error.

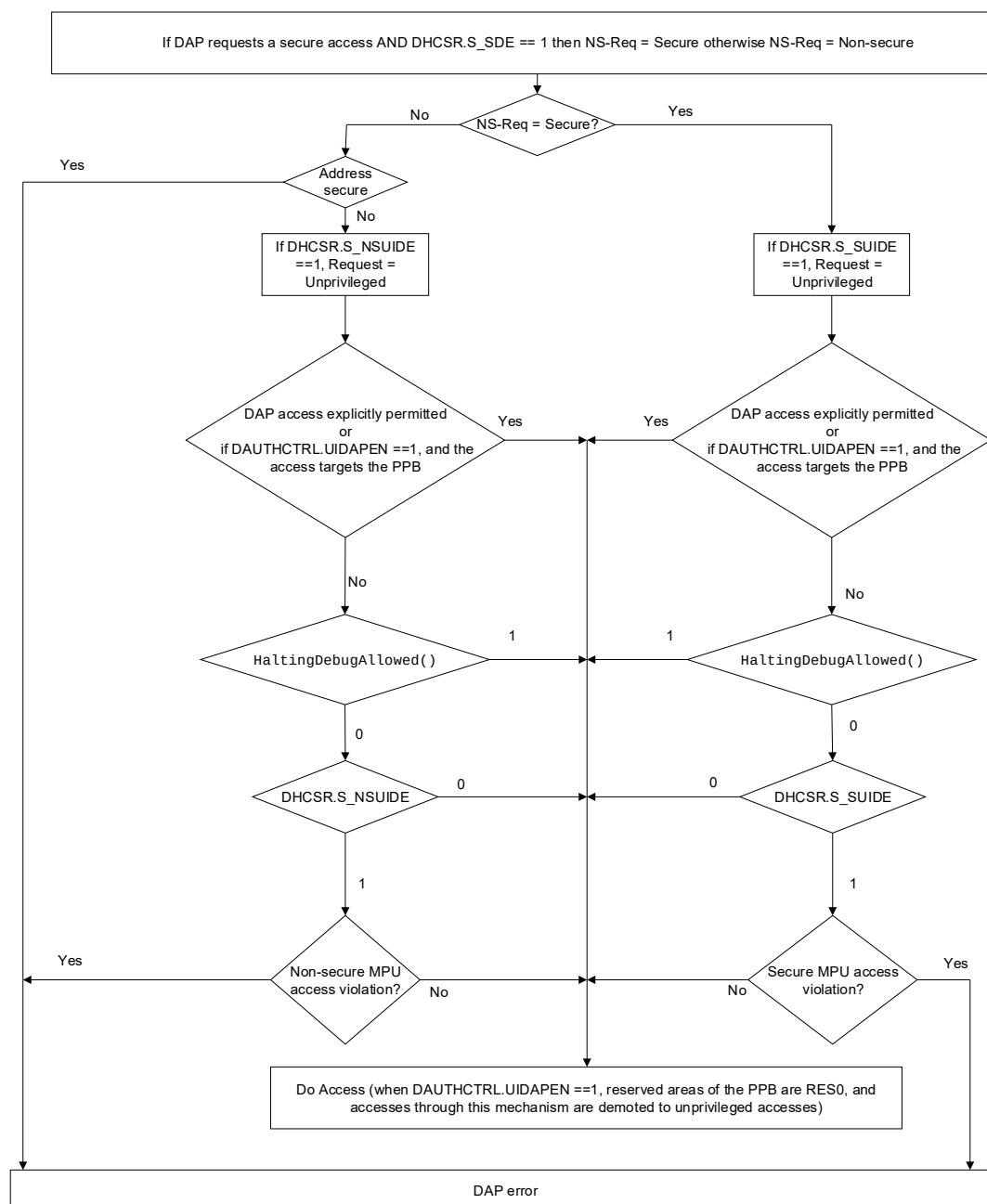
Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **UDE**.

R_{MRPC} A privileged DAP request through the Unprivileged Debug Extension mechanism is demoted to an unprivileged access and is subject to MPU checks. Both privileged and unprivileged requests are subject to MPU checks.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **UDE** && **MPU**.

R_{WSQR}

The DAP access process is extended when the Unprivileged Debug Extension is implemented, and `HaltingDebugAllowed()` returns FALSE, as shown in the following diagram.



Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **UDE** && **MPU**.

I_{LBGV}

When `HaltingDebugAllowed()` returns TRUE, DAP accesses follow the behavior specified by R_{WMRR}.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **UDE** && **MPU**.

See also:

[B3.14 Secure address protection.](#)

[Chapter B10 *The Armv8-M Protected Memory System Architecture.*](#)

B13.4 Debug event behavior

B13.4.1 About debug events

I _{CBWT}	An event that is triggered for debug reasons is known as <i>a debug event</i>. Applies to an implementation of the architecture <i>Armv8.0-M</i> onward. The extension requirements are - DB.
R _{PQKW}	A debug event that is not ignored causes one of the following to occur: • If Halting debug is implemented and enabled, entry to Debug state. • A HardFault exception. • Lockup. Applies to an implementation of the architecture <i>Armv8.0-M</i> onward. The extension requirements are - DB. Note, entry to Debug state requires <i>HDBG</i>.
R _{QLTQ}	A debug event that is not ignored, can cause a DebugMonitor exception to occur. Applies to an implementation of the architecture <i>Armv8.0-M</i> onward. The extension requirements are - M.
R _{MNKP}	The HardFault exceptions or Lockup that are caused by debug events are generated by: • A BKPT instruction that is executed when the PE can neither halt nor generate a DebugMonitor exception. • In some circumstances, the FPB. Applies to an implementation of the architecture <i>Armv8.0-M</i> onward. The extension requirements are - M. Note, an FPB requires FPB.
R _{WCPW}	The debug events are as follows.

Debug event	Actions
Step	Halt or DebugMonitor exception.
Halt Request	Halt
Breakpoint	Halt, DebugMonitor exception, or Hardfault.
Watchpoint	Halt or DebugMonitor exception.
Vector catch	Halt only
PMU Overflow	Halt or DebugMonitor exception
External	Halt or DebugMonitor exception.

R _{LDRZ}	Applies to an implementation of the architecture <i>Armv8.0-M</i> onward. Note, a DebugMonitor exception requires <i>M</i>. Halt requires <i>HDBG</i>. PMU is only available in an <i>Armv8.1-M</i> implementation. The DFSR contains status bits for each debug event. These bits are set to 1 when a debug event causes the PE to halt or generate a DebugMonitor exception, and are then write-one-to-clear. The following table shows which bit is set for each debug event.
-------------------	--

Event cause	DFSR bit
Step	HALTED
Halt request	HALTED
Breakpoint	BKPT
Watchpoint	DWTTRAP
Vector catch	VCATCH
PMU Overflow	PMU
External	EXTERNAL

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **M** || **HDBG**. Note, PMU is only available in an *Armv8.1-M* implementation.

R_{HNRV} It is IMPLEMENTATION DEFINED whether the **DFSR** debug event bits are updated when an event is ignored or the debug event is escalated to a HardFault.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB**.

I_{KLLX} In an implementation of the *Armv8.1-M* architecture Arm recommends that the **DFSR** debug event bits are updated when an event is ignored. For example, when a DebugMonitor event is not taken or is escalated to a HardFault.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DB**.

I_{MCZX} Arm recommends that any **BKPT** executed should set **DFSR.BKPT** regardless of any authentication for halting, DebugMonitor or HardFault or Lockup behavior.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB**.

R_{DVCR} If a **BKPT** instruction cannot halt and cannot generate a DebugMonitor exception then escalation to HardFault occurs.

The effect of **DEMCR.SDME** on the target Security state of an escalated Hardfault is an IMPLEMENTATION DEFINED choice, so that the target Security state of any escalated HardFault will be either:

- Non-secure state when the current Security state is Non-secure and **AIRCR.BFHFNMINs** is 1 and **DEMCR.SDME** is ignored.
- Secure state when any of the following are true:
 - The current Security state is Secure.
 - **AIRCR.BFHFNMINs** is 0.
 - **DEMCR.SDME** is 1 and is not ignored.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB** && **M**. Note, *S* required for Secure state.

R_{TBWL} If a **BKPT** instruction that cannot halt and cannot generate a DebugMonitor exception then escalation to HardFault occurs. The target Security state of the generated Hardfault is the Secure state if any of the following are true:

- **DEMCR.SDME** is set to one.
- **AIRCR.BFHFNMINs** is zero.
- The current Security state is Secure.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DB** && **S**. Note, *M* is always implemented in an *Armv8.1* implementation.

I_{ZFKK} When escalating a DebugMonitor exception to HardFault **DEMCR.SDME** should be checked regardless of the value of **DEMCR.MON_EN**.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DB**. Note, *S* required for Secure state.

Chapter B13. Debug
B13.4. Debug event behavior

I _{NSMV}	Debug events are either synchronous or asynchronous. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{VSVN}	The synchronous debug events are: • Breakpoint debug events, caused by execution of a BKPT instruction or by a match in the FPB. • Vector catch debug events, caused when one or more DEMCR.VC_* bits are set to 1, and the PE takes the corresponding exception. • Step debug events, caused by DHCSR.C_STEP or DEMCR.MON_STEP. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{PVGM}	A single instruction can generate several synchronous debug events. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{WJFB}	Synchronous debug events are associated with the instruction that generated them and are taken instead of executing the instruction. The PE does not generate any other synchronous exception or debug event that might have occurred as a result of executing the instruction. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{RNRD}	The Step debug event is taken on the instruction following the instruction being stepped. This means that prioritization of the event applies relative to any other exception or debug event for the following instruction, not for the instruction being stepped. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{JSPS}	If multiple debug events and exceptions are generated on the same instruction, they are prioritized as follows: 1. Halt request (halting only), including where DHCSR.C_HALT is set by DHCSR.C_STEP of the previous instruction. 2. Highest-priority pending exception that is eligible to be taken. If the Main Extension is implemented, this might be a DebugMonitor exception, if DEMCR.MON_PEND == 1. This includes where DEMCR.MON_PEND is set by DEMCR.MON_STEP of the previous instruction. 3. Vector catch. 4. Fault from an instruction fetch, including synchronous BusFault error. 5. Breakpoint that is signaled by an FPB unit. 6. BKPT instruction or other exception that results from decoding the instructions. This includes the cases where exceptions from the instruction are UNDEFINED, an unimplemented or disabled coprocessor is targeted, or the EPSR.T bit has a value of 0. 7. Other synchronous exception that is generated by executing the instruction, including an exception that is generated by a memory access that is generated by the instruction. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB. Note, not all of the debug features listed might be implemented in a particular implementation.</i>
R _{BQVF}	The highest-priority synchronous debug event is reported in the DFSR. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{FWQQ}	It is UNPREDICTABLE whether synchronous debug events that occur on the same instruction as a debug event with a higher priority are reported in the DFSR. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DB.</i>
R _{TKRS}	The asynchronous debug events are: • Watchpoint debug events caused by a match in the DWT, including instruction address match watchpoints. • Halt request debug events, where either: – A debugger write that has set DHCSR.C_HALT to 1 and DHCSR.C_DEBUGEN set to 1. – A software write that sets DHCSR.C_HALT to 1 when DHCSR.C_DEBUGEN was set to 1.

- External debug request debug events caused by assertion of an IMPLEMENTATION DEFINED external debug request.
- PMU Overflow caused by an overflow of a PMU counter that is configured to generate an interrupt.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DB**. Note, PMU is only available in an *Armv8.1-M* implementation.

R_{MPMC}

When **DHCSR.C_DEBUGEN** == 0 or the PE is in a state in which halting is prohibited, **DHCSR.C_HALT** and **DHCSR.C_STEP** are ignored, and these bits have an **Effective value** of 0.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG**.

See also:

[B3.13 Priority model](#).

[Halting debug](#).

[DebugMonitor exception](#).

[B13.4.3 Vector catch](#).

[B15.6 Interrupts and Debug events](#).

Applies to an implementation of the architecture *Armv8.1-M* onward.

[GenerateDebugEventResponse\(\)](#).

Halting debug

R_{WLCF}

Setting the **DHCSR.C_DEBUGEN** bit to 1 enables Halting debug.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG**.

R_{JXMW}

Setting the **DAUTHCTRL.UIDEN** bit to 1, enables halting debug in unprivileged state. Setting the Secure instance of this bit enables unprivileged debug in both Secure and Non-secure state.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **HDBG** && **UDE**.

R_{RZTG}

A debug event sets **DHCSR.C_HALT** to 1 if all of the following conditions apply:

- The debug event supports generating entry to Debug state.
- **DHCSR.C_DEBUGEN** == 1.
- Unless otherwise stated, halting is allowed.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG**.

R_{THLS}

If **DHCSR.C_HALT** has a value of 1 and halting is allowed, the PE halts and enters Debug state.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG**.

R_{FKWB}

A debug event that sets **DHCSR.C_HALT** to 1 pends entry to Debug state.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG**.

I_{VBXN}

A debug event might set **DHCSR.C_HALT** and remain pending through execution in a mode or state where Halting debug is prohibited, which might not be a finite time.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG**.

R_{HQTX}

When [SecureHaltingDebugEnabled\(\)](#) returns FALSE and [HaltingDebugEnabled\(\)](#) returns TRUE, a Debug event that sets **DHCSR.C_HALT** to 1 will cause all of the following:

- The PE to halt on transition from Secure state to Non-secure state for any of the following reasons:
 - Exception entry.
 - Exception return.

- Non-secure function call.
- Function return.

- The PE will halt and enter Debug state before completing the first instruction in Non-secure state.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_VDMZ When `UnprivHaltingDebugEnabled(FALSE)` returns TRUE a debug event that sets **DHCSR.C_HALT** to 1 will cause the PE to halt and enter Debug state on transition from a privileged mode to an unprivileged mode before completing the first instruction in unprivileged mode.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **UDE** && **HDBG**.*

R_XSRJ If **DHCSR.C_HALT** has a value of 1 or **EDBGRQ** is asserted before a **Context synchronization event**, and halting is allowed after the **Context synchronization event**, then the PE halts and enters Debug state before the first instruction following the Context synchronization event completes its execution.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG** || **EDBGRQ**.*

R_JXQF **DFSR** is updated at the same time as the PE sets **DHCSR.C_HALT** to 1.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_TXWB If an instruction that is being stepped or an instruction that generates a debug event reads **DFSR** or **DHCSR**, the value that is read for the relevant **DFSR** bit or for **DHCSR.C_HALT** is UNKNOWN.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_FRJC For asynchronous debug events, if halting is allowed, the PE enters Debug state in finite time.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_VJKX Entering Debug state has no architecturally defined effect on the Event Register and exclusive monitors.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

I_JNGH **DHCSR.C_SNAPSTALL** might allow imprecise entry into the Debug state, for example by forcing any stalled load or store instructions to be abandoned.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_BTBJ If **DHCSR.C_DEBUGEN** == 0 or `HaltingDebugEnabled()` == FALSE, **DHCSR.C_SNAPSTALL** is ignored and has an **Effective value** of 0.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_HLNF If **DHCSR.S_SDE** == 0, **DHCSR.C_SNAPSTALL** ignores writes from the debugger.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG** && **S**.*

R_CBLC If **UDE** is implemented and **DHCSR.S_SUIDE** == 1, debugger writes to **DHCSR.C_SNAPSTALL** are ignored.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **HDBG** && **UDE** && **S**.*

R_RKBK When the PE is in a state in which halting is prohibited, if **DHCSR.C_HALT** == 1 and **DHCSR.C_DEBUGEN** == 1, then **DHCSR.C_HALT** remains set unless it is cleared by a direct write to **DHCSR**. If the PE enters a state in which halting is allowed while **DHCSR.C_HALT** is set to 1, then the PE enters Debug state.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

See also:

[DHCSR, Debug Halting Control and Status Register.](#)

[B13.4.2 Debug stepping.](#)

[B13.5 Debug state.](#)

DebugMonitor exception

I _{DPPC}	The DebugMonitor exception is only available if the Main Extension is implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{VMDP}	Unless otherwise stated, a debug event sets DEMCR.MON_PEND to 1 if all of the following conditions apply: • The debug event supports generating DebugMonitor exceptions and does not generate an entry into debug state. • One of the following conditions apply: – DEMCR.MON_EN is set to 1. – DEMCR.UMON_EN is set to 1 and the debug event was generated by an unprivileged operation. • One of the following conditions apply: – The Security Extension is not implemented. – DEMCR.SDME is set to 1. – The debug event was generated by a Non-secure operation. • The DebugMonitor exception group priority is not sufficient to preempt current execution priority. The pseudocode function CanPendMonitorOnEvent () describes this. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - M && UDE.</i>
R _{VJH}	If a Debug event that does not generate an entry to Debug state, and the architecture does not require that the debug event set DEMCR.MON_PEND to 1, then: • The PE escalates a DebugMonitor synchronous exception that is generated by executing a BKPT instruction to a HardFault. • The PE might set DEMCR.MON_PEND to 1 for a watchpoint debug event. • The PE ignores the other debug event. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - M && UDE.</i>
R _{CHXQ}	A debug event that sets DEMCR.MON_PEND to 1 pends a DebugMonitor exception. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{VSPX}	DEMCR.MON_PEND is cleared to 0 when the PE takes a DebugMonitor exception. This means that a value of 1 for DEMCR.MON_PEND might never be observed for a synchronous DebugMonitor exception. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{BRXT}	DFSR is updated at the same time as the PE sets DEMCR.MON_PEND to 1. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{BKHP}	If an instruction that is being stepped or that generates a debug event reads DFSR or DEMCR, the value that is read for the relevant DFSR bit or for DEMCR.MON_PEND is UNKNOWN. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{VFLQ}	For asynchronous debug events, if taken as a DebugMonitor exception, and if the current priority is lower than the DebugMonitor exception group priority, a DebugMonitor exception is taken in finite time. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{JVSC}	A direct write to DEMCR can set DEMCR.MON_PEND to 1 at any time to make the DebugMonitor exception pending or can set DEMCR.MON_PEND to 0 to remove a pending DebugMonitor exception. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>

R _{XPBN}	When DEMCR.MON_PEND == 1, the PE takes the DebugMonitor exception according to the exception prioritization rules, regardless of the value of DEMCR.SDME and DEMCR.MON_EN . <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{NVQT}	From Armv8.1 when DEMCR.MON_PEND == 1, the PE takes the DebugMonitor exception according to the exception prioritization rules, regardless of the value of DEMCR.SDME , DEMCR.MON_EN and DEMCR.UMON_EN . <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - M && UDE.</i>
R _{QZHW}	Unless otherwise stated, synchronous and asynchronous DebugMonitor exceptions can only cause preemption at instruction boundaries. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>
R _{TMNT}	For beat-wise executable instructions, synchronous and asynchronous DebugMonitor exceptions can cause preemption at either beat or instruction boundaries. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
I _{PJJD}	DebugMonitor exceptions on their own cannot cause instruction resume or instruction restart. However, if another exception preempts an execution-continuable instruction, then the PE might take that exception along with DebugMonitor exception as a tail-chained or late-arriving exception. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>

See also:

[B14.2.2 DWT unit operation.](#)

[B14.5.2 FPB unit operation.](#)

[B3.27 Exceptions, instruction resume, or instruction restart.](#)

B13.4.2 Debug stepping

R _{HMCN}	The Armv8-M architecture supports debug stepping in both Halting debug and for the DebugMonitor exception. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG M. Note, might require the DebugMonitor exception.</i>
R _{THTG}	It is IMPLEMENTATION DEFINED whether stepping a WFE or WFI instruction causes the WFE or WFI instruction to: • Retire and take the debug event. • Go into a sleep state and take the debug event only when another wake up event occurs. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG M.</i>
R _{LLVC}	If a debug event wakes a WFE or WFI instruction, then on taking the debug event, the instruction has retired. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG M.</i>
I _{ZLPK}	If the PE exits from a sleep state due to an asynchronous debug event the PE will perform all of the following before halting the PE: • Complete the exception return, including the consumption of the Stack frame. • Clear the exclusive monitor state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG M.</i>
I _{CMYW}	The debug architecture includes support for stepping over vector instructions. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>

R_{XQYC} The debug architecture does not support Halting debug stepping or DebugMonitor stepping over Warm reset.
*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG** || **M**.*

See also:

[Halting debug stepping.](#)

[Debug monitor stepping.](#)

Halting debug stepping

I_{QMXC} A debugger can use Halting debug stepping to exit from Debug state, execute a single instruction, and then reenter Debug state.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_{SWKC} Halting debug stepping is active when all of the following apply:

- **DHCSR.C_DEBUGEN** is set to 1, Halting debug is enabled, and halting is allowed.
- **DHCSR.C_STEP** is set to 1, halting stepping is enabled.
- The PE is in Non-debug state.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_{ZVKC} When the PE exits Debug state and Halting debug stepping becomes active, the PE performs a Halting debug step as follows:

1. Performs one of the following:
 - Completes the next instruction without generating any exception.
 - Takes any pending exception entry of sufficient priority, without completing the next instruction. The PE performs an exception entry sequence that stacks the next instruction context. This context might include instruction continuation bits if the next instruction was partly executed and supports instruction resume. The exception might be a pending exception, or an exception generated by the execution of the next instruction. Taking a pending exception might include entry to [Lockup](#).
 - Completes the execution of the next instruction, and then takes any pending exception of sufficient priority. The PE performs an exception entry sequence that stacks the following instruction context. Taking an exception might include entry to [Lockup](#).
 - If the next instruction is an exception return instruction, completes the next instruction, tail-chaining to enter a new exception handler. In each case where the PE performs an exception entry sequence it does so according to the exception priority and late-arrival rules, meaning derived and late-arriving exceptions might preempt the exception entry sequence. If the PE enters [Lockup](#) as a consequence of step one the PE will resolve the Lockup signal before halting the PE. The exception behavior is not recursive. Only a single [PushStack\(\)](#) update can occur in a step sequence.
2. Sets **DFSR.HALTED** and **DHCSR.C_HALT** to 1. A read of the **DFSR.HALTED** or the **DHCSR.C_HALT** bit performed by the stepped instruction returns an UNKNOWN value.
3. After the Halting debug step, before executing the following instruction, because **DHCSR.C_HALT** is set the PE will halt and enter Debug state if halting is still allowed. However, if halting is prohibited after the Halting debug step then the PE does not enter Debug state and **DHCSR.C_HALT** remains set.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_{LJJB} If the implementation includes the Armv8.1-M profile Vector Extension, a Halting debug step has the following additional possible actions when stepping over vector instructions. The PE:

- Attempts to complete the execution of all in-flight vector instructions, does not execute any new instructions, and generates a synchronous exception that is taken.

- Completes the execution of all in-flight vector instructions, does not start the execution of new instructions, and does not generate an exception.
- Completely executes the next vector instruction, without overlapping execution, and does not generate an exception.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **HDBG** && **MVE**.*

I_{LTX}

The debugger can optionally set **DHCSR.C_MASKINTS** to 1 to prevent PENDSV, SysTick, and external interrupts from being taken. If a permitted exception becomes active, the PE steps into the exception handler and halts before executing the first instruction of the associated exception handler.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_{NSMR}

DHCSR.C_MASKINTS is ignored and has an **Effective value** of 0 if any of the following are true:

- **HaltingDebugAllowed()** returns FALSE.
- **DHCSR.C_DEBUGEN** is 0.
- The exception is Secure and **DHCSR.S_SDE** is 0.
- UDE is implemented, the exception is Secure and **DHCSR.S_SUIDE** is 1.
- UDE is implemented and **DHCSR.S_NSUIDE** is 1.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**. Note, S is required for Secure state, and UDE is only available in an Armv8.1-M implementation.*

R_{MBCB}

DHCSR.{C_HALT, C_STEP, C_MASKINTS} can be written in a single write to **DHCSR**, as follows:

DHCSR write			
assumes that DHCSR.C_DEBUGEN and DHCSR.S_HALT are both set to 1 when the write occurs and the PE is halted.			
C_HALT	C_STEP	C_MASKINTS	Effect
0	0	0	Exit Debug state and start instruction execution. Exceptions can become active and prioritized according to the priority rules and the configuration of exceptions.
0	0	1	Exit Debug state and start instruction execution. PendSV, SysTick and, external interrupts are disabled, otherwise exceptions can become active and prioritized according to the priority rules.
0	1	0	Exit Debug state, step an instruction and halt. Exceptions can become active and prioritized according to the priority rules.
0	1	1	Exit Debug state, step an instruction and halt. PendSV, SysTick and, external interrupts are disabled, otherwise exceptions can become active and prioritized according to the priority rules.
1	X	X	Remain in Debug state.

The write to **DHCSR** assumes that the PE is halted when the write occurs and the write is not UNPREDICTABLE.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **HDBG**.

Debug monitor stepping

I_{DXCT}

A debugger can use debug monitor stepping to return from the DebugMonitor exception handler, execute a single instruction, and then reenter the DebugMonitor exception handler.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DebugMonitor exception**.

R_{MLRM}

Debug monitor stepping is active when all of the following apply:

- **DHCSR.C_DEBUGEN** is set to 0 or the PE is in a state in which halting is prohibited.
- **DEMCR.MON_EN** is set to 1, that is Monitor debug is enabled.
- **DEMCR.MON_STEP** is set to 1, that is monitor stepping is enabled.
- **DEMCR.SDME** == 1 or the PE is in Non-secure state.
- Execution priority is below the priority of the DebugMonitor exception.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **M**.

R_{MJCF}

From Armv8.1-M Debug monitor stepping is active when all of the following apply:

- **DHCSR.C_DEBUGEN** is set to 0 or the PE is in a state in which halting is prohibited.
- **DEMCR.MON_EN** is set to 1, or **DEMCR.UMON_EN** is set to 1 and the instruction is executed in unprivileged mode, that is Monitor debug is enabled.
- **DEMCR.MON_STEP** is set to 1, that is monitor stepping is enabled.
- **DEMCR.SDME** == 1 or the PE is in Non-secure state.
- Execution priority is below the priority of the DebugMonitor exception.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - M && UDE.

R_{MWFT}

When DebugMonitor stepping becomes active, the PE performs a DebugMonitor step as follows:

1. It performs one of the following:

- It completes the next instruction without generating any exception.
- It takes any pending exception of sufficient priority. The PE performs an exception entry sequence that stacks the next instruction context. The exception might be a pending exception, or it might be an exception generated by the execution of the next instruction. Taking a pending exception might include entry to [Lockup](#).
- If the next instruction is an exception return instruction, the PE completes the next instruction, tail-chaining to enter a new exception handler according to the normal exception priority and late-arrival rules.

If the PE performs an exception entry sequence as part of step 1, the PE stacks the next instruction context. This context might include instruction continuation bits if the next instruction was partly executed and supports instruction resume.

If the PE enters [Lockup](#) as a consequence of step one the PE will resolve the Lockup signal before entering the DebugMonitor exception if the DebugMonitor exception has sufficient priority to pre-empt execution after Lockup has been resolved.

2. If the execution priority is below the priority of the DebugMonitor exception after step 1, the PE sets [DEMCR.MON_PEND](#) and [DFSR.HALTED](#) to 1. A read of [DEMCR.MON_PEND](#) or [DFSR.HALTED](#) by the stepped instruction returns an UNKNOWN value.

3. Before executing the following instruction, the PE takes any pending exception with sufficient priority.

If step 2 set [DEMCR.MON_PEND](#) to 1, then the DebugMonitor exception is pending. However, it is UNPREDICTABLE whether the PE uses the new value or the old value of [DEMCR.MON_PEND](#) in determining the highest priority exception. This means that:

- Another exception might preempt execution before the DebugMonitor exception is taken, and the exception might be lower priority than the DebugMonitor exception. However, this is a [Context synchronization event](#) and the PE uses the new value of [DEMCR.MON_PEND](#) to determine the highest priority exception before executing the next instruction. This includes entry into [Lockup](#).
- If no other exceptions are pending, the PE takes the DebugMonitor exception.

Derived and late-arriving exceptions might preempt the exception entry sequence.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.

R_{JKJM}

If the implementation includes the Armv8.1-M profile Vector Extension, a Debug monitor step has the following additional possible actions when stepping over vector instructions. The PE:

- Attempts to complete the execution of all in-flight vector instructions, does not execute any new instructions, and generates a synchronous exception that is taken.
- Completes the execution of all in-flight vector instructions, does not start the execution of new instructions, and does not generate an exception.
- Completely executes the next vector instruction, without overlapping execution, and does not generate an exception.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - M && MVE.

I_{GPSX}

In all other cases, the DebugMonitor exception preempting execution returns control to the DebugMonitor exception handler. Unless that handler clears [DEMCR.MON_STEP](#) to 0, returning from the handler performs the next debug monitor step.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.

I_{KPKX} If, after the debug monitor stepping process, the taking of an exception means that the execution priority is no longer below that of the DebugMonitor exception, the values of **DEMCR.MON_STEP** and **DEMCR.MON_PEND** mean that debug monitor stepping process continues when execution priority falls back below the priority of the DebugMonitor exception.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M**.*

See also:

DHCSR.

B13.4.3 Vector catch

I_{TVRX} Vector catch is the mechanism for generating a debug event and entering Debug state on entry to a particular exception handler or reset.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M**.*

R_{JCXR} Vector catching is only supported by Halting debug.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_{PBVX} The conditions for a vector catch, other than reset vector catch, are:

- **DHCSR.C_DEBUGEN** == 1 and halting is allowed for the Security state the exception is targeting.
- The associated exception enable bit is set.
- The associated active bit is set.
- The associated vector catch enable bit.
- An exception is taken to the relevant exception handler. The associated fault status register status bit is set to 1.

When these conditions are met, the PE sets **DHCSR.C_HALT** to 1 and enters Debug state before executing the first instruction of the exception handler.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

I_{XDGP} Late arrival and derived exceptions might occur, preempting the exception targeted by the vector catch and postponing when the PE halts.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_{XKMH} The following table defines the exception, Fault status bit, and Vector catch bit.

Exception	Fault status bit	Vector catch bit DEMCR
HardFault	HFSR.VECTTBL	VC_INTERR
	HFSR.FORCED	VC_HARDERR
	HFSR.DEBUGEVT	VC_HARDERR
BusFault	BFSR.IBUSERR	VC_BUSERR
	BFSR.PRECISERR	VC_BUSERR
	BFSR.IMPRECISERR	VC_BUSERR
	BFSR.UNSTKERR	VC_INTERR
	BFSR.STKERR	VC_INTERR
	BFSR.LSPERR	VC_INTERR
DebugMonitor	HFSR.DEBUGEVT	-
MemManage fault	MMFSR.IACCVIOL	VC_MMERR
	MMFSR.DACCVIOL	VC_MMERR
	MMFSR.MUNSTKERR	VC_INTERR
	MMFSR.MSTKERR	VC_INTERR
	MMFSR.MLSPERR	VC_INTERR
NMI	-	-
PENDSV	-	-
UsageFault	UFSR.UNDEFINSTR	VC_STATERR
	UFSR.INVSTATE	VC_STATERR
	UFSR.INVPC	VC_STATERR
	UFSR.NOCP	VC_NOCPERR
	UFSR.STKOF	VC_INTERR
	UFSR.UNALIGNED	VC_CHKERR
	UFSR.DIVBYZERO	VC_CHKERR
SecureFault	SFSR.INVEP	VC_SFERR
	SFSR.INVIS	VC_SFERR
	SFSR.INVER	VC_SFERR
	SFSR.AUVIOL	VC_SFERR
	SFSR.INVTRAN	VC_SFERR
	SFSR.LSPERR	VC_SFERR
	SFSR.LSERR	VC_SFERR
SVCall	-	-
SysTick	-	-

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - *M*.

R_{LKNL} When **DHCSR.C_DEBUGEN** == 0 or the PE is in a state in which halting is prohibited, all **DEMCR.VC_** bits, other than **DEMCR.VC_CORERESET**, are ignored.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG** && **S**.*

R_{VZDB} If the PE resets into Secure state when both of:

- **DHCSR.C_DEBUGEN** == 1
- **DEMCR.VC_CORERESET** == 1

The PE will pend a Halt request and will halt and enter into Debug state when halting is permitted, setting **DFSR.VCATCH** to 1.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **HDBG** && **S**.*

R_{WRMQ} The PE pends a Vector catch event when all of the following is true:

- The PE has reset into Secure state.
- **DHCSR.C_DEBUGEN** == 1.
- **DEMCR.VC_CORERESET** == 1.
- Halting debug is not allowed in Secure state.

The PE does not halt until either it enters Non-secure state or debug is allowed in Secure state.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG** && **S**.*

R_{MSWM} If UDE is implemented and all of the following are true:

- **DHCSR.C_DEBUGEN** == 1,
- **DEMCR.VC_CORERESET** == 1,
- **UnprivHaltingDebugEnabled(FALSE)** returns TRUE,
- The PE resets into Secure state,

The PE will pend a Vector Catch debug event, if debug is prohibited in Secure state, and does not halt until either the PE enters Non-secure state or debug is permitted in Secure state.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **HDBG** && **UDE**.*

See also:

[B1.1 Resets, Cold reset, and Warm reset.](#)

[B3.10 Exception enable, pending, and active bits.](#)

[B3.13 Priority model.](#)

[B3.12 Faults.](#)

[B3.9 Exception numbers and exception priority numbers.](#)

[B3.24 Exceptions during exception entry.](#)

[B3.25 Exceptions during exception return.](#)

[Chapter B1 Resets.](#)

B13.4.4 Breakpoint instructions

R_{CRJG} When **DHCSR.C_DEBUGEN** == 0 or when the PE is in a state in which halting is prohibited, the **BKPT** instruction does not generate an entry to Debug state. If no DebugMonitor exception is generated, the **BKPT** instruction generates a HardFault exception or enters Lockup state.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{NXTW} A **BKPT** instruction halts the PE if either of the following cases are true:

1. **HaltingDebugAllowed()** returns TRUE and the following are true:
 - **DHCSR.C_DEBUGEN** is set to 1.
 - The PE is not halted.
 - If the Security extension is implemented and the PE is executing in Secure state all of the following are true:
 - **DHCSR.S_SDE** is set to 1.
 - UDE is not implemented or **DHCSR.S_SUIDE** is set to 0.
2. **HaltingDebugAllowed()** returns FALSE and all of the following are true:
 - UDE is implemented and the PE is executing Non-secure state and unprivileged mode and **DHCSR.S_NSUIDE** is set to 1.
 - UDE is implemented and the PE is executing in Secure state and unprivileged mode and **DHCSR.S_SUIDE** is set to 1.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **HDBG** && **UDE**.*

R_{ZJLD} A **BKPT** instruction generates a DebugMonitor exception if it does not halt the PE and all of the following conditions apply:

- The DebugMonitor exception priority is greater than the current execution priority.
- The Security Extension is not implemented, the instruction is executed in Non-secure state, or **DEMCR.SDME** == 1.
- One of the following conditions apply:
 - **DEMCR.MON_EN** == 1
 - The Unprivileged Debug Extension is implemented, **DEMCR.UMON_EN** == 1, and the instruction is executed in unprivileged mode.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **M** && **UDE**.*

B13.4.5 External debug request

R_{XZCP} When the PE is in Non-debug state, an external agent can signal an external debug request.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GTGX} An external debug request can cause a debug event, that causes either:

- Entry to Debug state.
- If the Main Extension is implemented, a DebugMonitor exception.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **M** || **HDBG**.*

R_{FGCV} The PE ignores external debug requests when it is in Debug state.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **HDBG**.*

R_{BXRD} When **DHCSR.C_DEBUGEN** == 0 or the PE is in a state in which halting is prohibited, an External debug request does not generate an entry to Debug state and is ignored if no DebugMonitor exception is generated.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BNBR} An External debug request that does not halt the PE will set **DEMCR.MON_PEND** to 1 if all of the following are true:

- The DebugMonitor exception is enabled. That is if either of the following are true:
 - **DEMCR.MON_EN** is set to 1.

- [DEMCR.UMON_EN](#) is set to 1 and the PE is executing in unprivileged mode.

- The priority of the DebugMonitor exception is sufficient to preempt the current execution priority.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [M](#) && [UDE](#).

I_{FSTF} [R_BNBR](#) describes the conditions in which [DEMCR.MON_PEND](#) will be set on the assertion of an External debug request. [R_KQTP](#) and [R_RVXG](#) set the bounds of the timing for when [DEMCR.MON_PEND](#) will be set, so that privileged software can update [DEMCR](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [M](#).

R_{KQTP} In the absence of a [Context synchronization event](#) an External debug request that does not cause the PE to halt will set [DEMCR.MON_PEND](#) to 1 in finite time.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

R_{RVXG} A [Context synchronization event](#) is sufficient to ensure that an External debug request has set [DEMCR.MON_PEND](#) for the context being switched away from.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [M](#).

See also:

[B13.4 Debug event behavior](#).

[DFSR.EXTERNAL](#).

B13.5 Debug state

R _{RMKS}	In Halting debug, debug events allow an external debugger to halt the PE. The PE then enters Debug state. When the PE is in Debug state: • The PE stops executing instructions from the location indicated by the PC, and is instead controlled by the external debug interface. • The PE cannot service any interrupts. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
R _{QDCP}	In Debug state, the PE clears the DHCSR.S_REGRDY bit to 0 when the debugger writes to DCRSR and the PE then sets the bit to 1 when the transfer between the DCRDR and R0-R12 (Rn), Special-purpose register, Floating-point Extension register, or DebugReturnAddress completes. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG. Note, Floating-point registers are RES0 if FP is not implemented.</i>
I _{FKSM}	To transfer a word to a general-purpose register, to a Special-purpose register, to a Floating-point Extension register, or to DebugReturnAddress, a debugger: 1. Writes the required word to DCRDR. 2. Writes to the DCRSR, with the REGSEL value indicating the required register, and the REGWnR bit set to 1 to indicate a write access. This clears the DHCSR.S_REGRDY bit to 0. 3. If required, polls DHCSR until DHCSR.S_REGRDY reads-as-one. This shows that the PE has transferred the DCRDR value to the selected register. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
I _{CMBB}	To transfer a word from a general-purpose register, from a Special-purpose register, from a Floating-point Extension register, or from DebugReturnAddress, a debugger: 1. Writes to DCRSR, with the REGSEL value indicating the required register, and the REGWnR bit as 0 to indicate a read access. This clears the DHCSR.S_REGRDY bit to 0. 2. Polls DHCSR until DHCSR.S_REGRDY reads-as-one. This shows that the PE has transferred the value of the selected register to DCRDR. 3. Reads the required value from DCRDR. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
R _{VLVD}	In Debug state, following a write to DCRSR that clears the DHCSR.S_REGRDY bit to 0, the behavior is UNPREDICTABLE if any of the following occur before the PE sets DHCSR.S_REGRDY to 1: • The PE exits Debug state, other than because of a Warm reset. • The debugger writes to DCRDR or DCRSR. If the DCRSR.REGWnR bit was set to 0 and the debugger reads from DCRDR before the PE sets DHCSR.S_REGRDY to 1, then the read returns an UNKNOWN value. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
R _{JKBB}	When using the DCRDR, DCRSR and DHCSR.S_REGRDY mechanism to write to XPSR, all bits of the XPSR are fully accessible. The effect of writing an illegal value is UNPREDICTABLE. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
I _{RXQB}	The DCRDR, DCRSR and DHCSR.S_REGRDY mechanism differs from the behavior of MSR or MRS instruction accesses to the XPSR, where some bits are ignored on writes. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>

R _{QLRN}	When the PE is halted the Debugger can write to: • The DebugReturnAddress. • EPSR.IT/ICI bits. On exiting Debug state the PE starts from DebugReturnAddress. The Debugger must ensure that the EPSR.IT and EPSR.ICI bits are consistent with DebugReturnAddress, otherwise instruction execution will be UNPREDICTABLE. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
R _{XZJN}	The debugger can write to the EPSR.IT/ICI/ECI bits. If the debugger does this, it writes a value consistent with the instruction to be executed on exiting Debug state, otherwise instruction execution will be UNPREDICTABLE. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - HDBG && MVE.</i>
I _{RRFN}	The debugger can always set FAULTMASK to 1, but doing so might cause unexpected behavior on exit from Debug state. An MSR instruction cannot set FAULTMASK to 1 when the execution priority is -1 or higher. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
R _{XRRQ}	The debugger can write to the EPSR.IT/ICI bits, and on exiting Debug state any interrupted LDM or STM instruction will use these new values. Clearing the ICI bits to 0 will cause the interrupted LDM or STM instruction to restart or continue. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
R _{QLTB}	The debugger can write to the EPSR.ECI bits, and on exiting Debug state any interrupted vector instructions will use these new values. Clearing the ECI bits to 0 will cause the interrupted vector instruction to restart. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - HDBG && MVE.</i>
R _{BMHD}	When the PE is in Debug state, an indirect write to a Special-purpose register caused by an access by the debugger to a register within the System Control Block (SCB) is guaranteed to be visible after the access to the register within the SCB completed to a subsequent: • Access to the Special-purpose register through DCRDR. • Indirect read of the Special-purpose register made for an access of any register through DCRDR or any register within the System Control Block. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
R _{MDJX}	When the PE is in Debug state, a write to a Special-purpose register made by the debugger through the DCRDR is guaranteed to be visible after the write is observed to be completed in DHCSR.S_REGRDY to a subsequent: • Access of any register through DCRDR or any register within the System Control Block. • Indirect read of the Special-purpose register made for an access to any register through DCRDR or any register within the System Control Block. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
I _{DMTG}	A read or write of a register through DCRDR starts with a write to DCRSR. Where the architecture guarantees that a previous access is visible to a subsequent access through DCRDR, this means the write to DCRSR is made after the point where the previous access is visible. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - HDBG.</i>
I _{WFHL}	Armv8.1-M introduces restrictions on the DCRDR and DCRSR mechanism for unprivileged debug access to the floating-point registers when lazy Floating-point state preservation is active or CPACR.CP10 associated with the Security state of the floating-point context and additional floating-point context restricts access. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - HDBG && UDE.</i>

R _{HTXM}	An access using the DCRDR and DCRSR mechanism to the FPSCR, VPR or the floating-point register file will return RAZ/WI if CanDebugAccessFP() returns FALSE. When CanDebugAccessFP() returns TRUE, DHCSR.S_FPD is set to 0. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - HDBG && UDE. Note, Floating-point registers are RES0 if FP is not implemented.</i>
R _{FMRB}	An access to the following registers using the DCRDR and DCRSR mechanism when DHCSR.S_SUIDE or DHCSR.S_NSUIDE are set to 1 will return zeros to the DCRDR: • Secure Main Stack Pointer • MSPLIM • CONTROL • FAULTMASK • PRIMASK • BASEPRI <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - HDBG && UDE.</i>
R _{JFLV}	An access to XPSR using the DCRDR and DCRSR mechanism when DHCSR.S_SUIDE or DHCSR.S_NSUIDE are set to 1 will return the EAPSR values to the DCRDR. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - HDBG && UDE.</i>
I _{YRGH}	The debugger can access the PAC keys using the DCRDR and DCRSR mechanism. The DCRDR and DCRSR mechanism is described by DebugRegisterTransfer(). <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI && HDBG.</i>
R _{LRXR}	The PAC_KEY_Pn and PAC_KEY_Un registers can be accessed using the DCRDR and DCRSR mechanism. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PACBTI && HDBG. Note, The Security extension is required for access to Secure register variants.</i>
R _{PCSN}	Permission for an unprivileged debugger to access the above registers using the DCRDR and DCRSR mechanism is dependant on the Current Security state, as well as the values of DHCSR.S_SDE, DHCSR.S_SUIDE and DHCSR.S_NSUIDE. These access permissions are detailed in the following table:

Current PE Security	DHCSR			Access PAC_KEY		
	S_SDE	S_SUIDE	S_NSUIDE	Current	Secure	Non-secure
NS	0	0	0	Y	N	Y
NS	0	0	1	N	N	N
NS	0	1	0	Not applicable		
NS	0	1	1	Not applicable		
NS	1	0	0	Y	Y	Y
NS	1	0	1	Not applicable		
NS	1	1	0	Y	N	Y
NS	1	1	1	N	N	N
S	0	0	0	Not applicable		
S	0	0	1	Not applicable		
S	0	1	0	Not applicable		
S	0	1	1	Not applicable		
S	1	0	0	Y	Y	Y
S	1	0	1	Not applicable		
S	1	1	0	N	N	Y
S	1	1	1	N	N	N

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **PACBTI** && **HDBG** && **UDE** && **S**.

R_{KLQR} A debugger can access the **CONTROL.UPAC_EN**, **PAC_EN**, **UBTI_EN**, **BTI_EN** using the **DCRDR** and **DCRSR** mechanism.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **PACBTI** && **HDBG** && **UDE**. Note, The Security extension is required for access to Secure register variants.

R_{BHTP} Permission for an unprivileged debugger to access the above registers using the **DCRDR** and **DCRSR** mechanism is dependant on the Current Security state, as well as the values of **DHCSR.S_SDE**, **DHCSR.S_SUIDE** and **DHCSR.S_NSUIDE**. These access permissions are detailed in the following table:

Current PE Security	DHCSR			Access CONTROL		Access CONTROL_S		Access CONTROL_NS	
	S_SDE	S_SUIDE	S_NSUIDE	PAC_EN/ BTI_EN	UPAC_EN/ UBTI_EN	PAC_EN/ BTI_EN	UPAC_EN/ UBTI_EN	PAC_EN/ BTI_EN	UPAC_EN/ UBTI_EN
NS	0	0	0	Y	Y	N	N	Y	Y
NS	0	0	1	N	Y	N	N	N	Y
NS	0	1	0	Not applicable					
NS	0	1	1	Not applicable					
NS	1	0	0	Y	Y	Y	Y	Y	Y
NS	1	0	1	Not applicable					
NS	1	1	0	Y	Y	N	Y	Y	Y
NS	1	1	1	N	Y	N	Y	N	Y
S	0	0	0	Not applicable					
S	0	0	1	Not applicable					
S	0	1	0	Not applicable					
S	0	1	1	Not applicable					
S	1	0	0	Y	Y	Y	Y	Y	Y
S	1	0	1	Not applicable					
S	1	1	0	N	Y	N	Y	Y	Y
S	1	1	1	N	Y	N	Y	N	Y

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **PACBTI** && **HDBG** && **UDE** && **S**.

See also:

DCRDR, Debug Core Register Data Register.

DCRSR, Debug Core Data Select Register.

`DebugRegisterTransfer()`

Applies to an implementation of the architecture *Armv8.1-M* onward.

B13.6 Exiting Debug state

- R_{BFGT} The PE exits Debug state:
- When the debugger writes 0 to [DHCSR.C_HALT](#).
 - On receipt of an external restart request.
 - On Warm reset.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [HDBG](#).

- R_{GGMJ} For synchronous debug events DebugReturnAddress is:

Synchronous debug event	DebugReturnAddress
Breakpoint debug events (BKPT or FPB Match)	Address of the breakpointed instruction.
Vector Catch debug events	Address of the first instruction of the exception handler.
Step debug events	Address of the next instruction to be executed in simple sequential execution order following the instruction that was stepped. If an exception was taken during stepping, this is the first instruction of the exception handler.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [HDBG](#).

- R_{XCCB} Bit[0] of a DebugReturnAddress value is RAZ/SBZ. When writing a DebugReturnAddress, writing bit [0] of the address does not affect the [EPSR.T](#) bit.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [HDBG](#).

- R_{HNKB} Exiting Debug state has no architecturally defined effect on the Event Register and exclusive monitors.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [HDBG](#).

- R_{WKSD} If software clears [DHCSR.C_HALT](#) to 0 when the PE is in Debug state, a subsequent read of the [DHCSR](#) that returns 1 for both [DHCSR.C_HALT](#) and [DHCSR.S_HALT](#) indicates that the PE has reentered Debug state because it has detected a new debug event.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [HDBG](#).

- R_{FKXH} Before leaving Debug state caused by an imprecise entry into Debug state the system is reset.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [HDBG](#).

See also:

[B13.5 Debug state](#)

B13.7 Multiprocessor support

R_{QXLS} Systems that support debug of more than one PE, either within a single device or as heterogeneous PEs in a more complex system, require each PE to support all of the following to enable cross-triggering of debug events between PEs:

- An external debug request.
- A cross-halt event.
- An external restart request.

Support for these features is OPTIONAL in other systems.

Applies to an implementation of the architecture Armv8.0-M onward.

B13.7.1 Cross-halt event

R_{DLCV} When the PE enters Debug state, it signals to an external agent that it is entering Debug state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [HDBG](#).

B13.7.2 External restart request

R_{ZKVV} When the PE is in Debug state, an external agent can signal an external restart request that causes the PE to exit Debug state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [HDBG](#).

R_{WJST} An external restart request is not ordered with respect to accesses to memory-mapped registers. It is UNPREDICTABLE whether an access to a memory-mapped register from a DAP completes before an external restart request.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [HDBG](#).

I_{VNDK} A debugger ensures that any read or write of a memory-mapped register by the DAP completes before issuing an external restart request.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [DB](#).

R_{NJQN} The PE ignores external restart requests when it is in Non-debug state.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B13.6 Exiting Debug state](#).

Chapter B14

Debug and Trace Components

This chapter specifies the Armv8-M debug and trace component rules. It contains the following sections:

B14.1 Instrumentation Trace Macrocell.

B14.2 Data Watchpoint and Trace unit.

B14.3 Embedded Trace Macrocell.

B14.4 Trace Port Interface Unit.

B14.5 Flash Patch and Breakpoint unit.

B14.1 Instrumentation Trace Macrocell

B14.1.1 About the ITM

R_{GDNG} The *Instrumentation Trace Macrocell* (ITM) provides a memory-mapped register interface that applications can use to generate Instrumentation packets.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

I_{BXWJ} The ITM is only available if the Main Extension is implemented.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{LMXS} The ITM generates Instrumentation packets, Synchronization packets, and the following protocol packets:

- Overflow packets.
- Local timestamp packets.
- Global timestamp packets.
- Extension packets.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{XQRX} The ITM combines the following packets into a single trace stream:

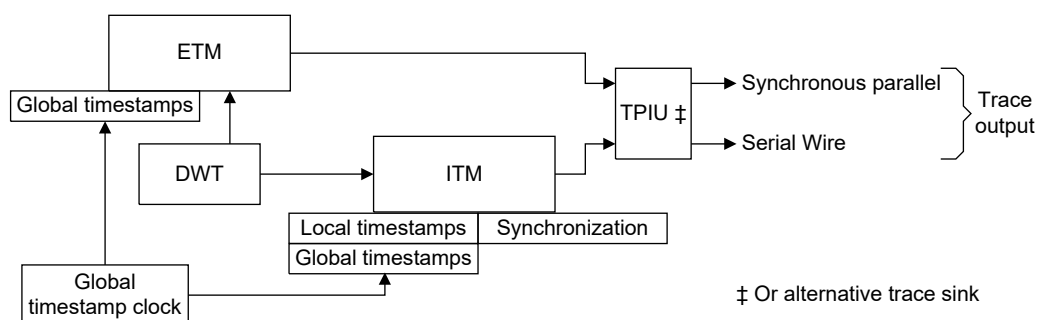
- Instrumentation packets.
- Synchronization packets.
- Protocol packets.
- Hardware source packets that are generated by the DWT.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{MXMN} If the implementation includes the PMU, the PMU Hardware source packets are included in the single trace stream.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **ITM** && **PMU**.*

I_{FQLR} The following figure shows how the ITM relates to other debug components.



*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{BWJJ} When multiple sources are generating data at the same time, the ITM arbitrates using the following priorities:

Synchronization, when required: Priority level -1, highest.

Instrumentation: Priority level 0.

Hardware source: Priority level 1.

Local timestamps: Priority level 2.

Global timestamp 1: Priority level 3.

Global timestamp 2: Priority level 4.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

See also:

[Global timestamping](#).

[B14.2 Data Watchpoint and Trace unit](#).

[ITM and DWT Packet Protocol Specification](#).

B14.1.2 ITM operation

R_{NKSC}

The ITM consists of:

- Up to 256 stimulus port registers, [ITM_STIMn](#).
- Up to eight enable registers, [ITM_TERN](#).
- An access control register, [ITM_TPR](#).
- A general control register, [ITM_TCR](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{MFDV}

The number of [ITM_STIMn](#) registers is an IMPLEMENTATION DEFINED multiple of eight. Software can discover the number of supported stimulus ports by writing all ones to the [ITM_TPR](#), and then reading how many bits are set to 1.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{CGVD}

If the ITM is disabled or not implemented, any Secure or Non-secure write to [ITM_STIMn](#) is ignored.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#) && [S](#).

R_{NJTR}

Unprivileged and privileged software can always read all ITM registers.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{FFXF}

If the ITM is not implemented, the ITM registers are RAZ/WI.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{CSFV}

The [ITM_TPR](#) defines whether each group of eight [ITM_STIMn](#) registers, and their corresponding [ITM_TERN](#) bits, can be written by an unprivileged access.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{PTXV}

[ITM_STIMn](#) registers are 32-bit registers that support the following word-aligned accesses:

- Byte accesses, to access register bits[7:0].
- Halfword accesses, to access register bits[15:0].
- Word accesses, to access register bits[31:0].

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{LNMW}

Non-word-aligned accesses are UNPREDICTABLE.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{NQVK}

[ITM_TCR.ITMENA](#) is a global enable bit for the ITM. A Cold reset clears this bit to 0, disabling the ITM.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R _{VRGP}	The ITM_TERN registers provide an enable bit for each stimulus port. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{NTCR}	When software writes to an enabled ITM_STIMn register, the ITM combines the identity of the port, the size of the write access, and the data that is written, into an Instrumentation packet that it writes to a stimulus port output buffer. The ITM transmits packets from the output buffer to a trace sink. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{XPRK}	If an ITM implementation supports more than 32 stimulus ports, paging is used to indicate the stimulus port number. The stimulus port number is defined by the A field of the Instrumentation packet, byte 0 bits [7:3]. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{QBCH}	An Extension packet is issued to define the page number, 0 to 7, with each subsequent Instrumentation packet. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{HWBQ}	Whenever a debugger receives an Instrumentation packet, it uses the the page number from the last Extension packet received, or a page number of 0 if the debugger has not received an Extension packet since the debugger last received a Synchronization packet. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - ITM.</i>
R _{MKFV}	The ITM does not generate trace if DEMCR.TRCENA == 0 or either of the following are true: • NoninvasiveDebugEnabled() == FALSE and the PE is in Non-secure state. • SecureNoninvasiveDebugEnabled() == FALSE and the PE is in Secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM. Note, S required for Secure state.</i>
R _{PXSX}	The ITM does not generate trace if the PE is in unprivileged mode when UnprivHaltingDebugEnabled() == FALSE for the current Security state. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - ITM && UDE.</i>
R _{ZKDG}	It is IMPLEMENTATION DEFINED whether no other ITM packets are generated when ITM trace is not generated. Arm strongly recommends that no ITM packet is generated when ITM trace is not generated. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{GRNM}	The size of the stimulus port output buffer is IMPLEMENTATION DEFINED, but has at least one entry. The stimulus port output buffer is shared by all ITM_STIMn registers. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{SXNK}	When the stimulus port output buffer is full, if software writes to any ITM_STIMn register, the ITM discards the write data, and generates an Overflow packet. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{SRPP}	Reading the ITM_STIMn register of any enabled stimulus port returns a value indicating the output buffer status and that the port is enabled. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{XVVB}	Reading an ITM_STIMn register when the ITM is disabled, or when the individual stimulus port is disabled in the corresponding ITM_TERN register, returns the value indicating that the output buffer cannot accept data because the port is disabled. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{FXSL}	Hardware source packets that are generated by any source use a separate output buffer. The output buffer status that is obtained by reading an ITM_STIMn register is not affected by trace that is generated by any other source. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM && DWTT.</i>

R _{RGCV}	Stalling is supported through an optional control, ITM_TCR.STALLENA. When implemented and set to 1, the ITM can stall the PE to guarantee delivery of the following Hardware source packets: • Data Trace Data Address. • Data Trace Data Value. • Data Trace Match. • Data Trace PC Value. • Exception Trace. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{NFJN}	Stalling does not affect the DWT counters. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM && DWTT.</i>
R _{NWVT}	Stalling does not affect the PMU counters. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - ITM && PMU.</i>
R _{TNDP}	The ITM might generate an Overflow packet while the PE is stalled, if the DWT generates: • A Hardware source packet other than a Data trace packet or Exception packet. • A Data Trace PC value packet or Data Trace Match packet from a Cycle Counter comparator. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{CRKK}	The ITM does not stall the PE in Secure state if SecureHaltingDebugEnabled() == FALSE. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM && S.</i>
R _{GRHW}	The ITM does not stall the PE if HaltingDebugEnabled() == FALSE. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{BGCP}	The ITM does not stall the PE in such a way as to deadlock the system. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{FRJG}	The ITM does not stall the PE if the trace output is disabled. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{XRVL}	The ITM does not stall for writes to the ITM_STIMn registers. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{HDLH}	Instrumentation trace packets appear in the trace output in the order in which writes arrive at the ITM_STIMn registers. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{XNHX}	It is IMPLEMENTATION DEFINED whether an ITM requires flushing of trace data to guarantee that data is output. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{TSXR}	If periodic flushing is required, the ITM flushes trace data: • When a Synchronization packet is generated. • When trace is disabled, meaning that either DEMCR.TRCENA is cleared to 0 or one or more of ITM_TCR.{TXENA, SYNCENA, TSENA, SYNCENA} is cleared to 0, and the buffered trace includes at least one corresponding packet type. • In response to other IMPLEMENTATION DEFINED flush requests from the system. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>

R_{MKFS} If a system supports multiple trace streams, the debugger writes a unique nonzero trace ID value to the **ITM_TCR.TraceBusID** field. The system uses this value to identify the individual trace streams. To avoid trace stream corruption, before modifying the **ITM_TCR.TraceBusID** a debugger does the following:

- It clears the **ITM_TCR.ITMENA** bit to 0, to disable the ITM.
- It polls the **ITM_TCR.BUSY** bit until it returns to 0, indicating that the ITM is inactive.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

B14.1.3 Timestamp support

R_{RVL} Timestamps provide information on the timing of event generation regarding their visibility at a trace output port.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{TFDG} An Armv8-M PE can implement either or both of the following types of timestamp:

- Local timestamps.
- Global timestamps.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

Local timestamping

R_{RMX} Local timestamps provide delta timestamp values, meaning each local timestamp indicates the elapsed time since generating the previous local timestamp.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{WBG} The ITM generates the local timestamps from the timestamp counter in the ITM unit.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{XLBH} The timestamp counter size is an IMPLEMENTATION DEFINED value that is less than or equal to 28 bits.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{GPXT} It is IMPLEMENTATION DEFINED whether the ITM supports synchronous clocking of the timestamp counter mode.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{SRJH} It is IMPLEMENTATION DEFINED whether the ITM and TPIU support asynchronous clocking of the timestamp counter mode.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{GHP} **ITM_TCR.TSENA** enables Local timestamp packet generation.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{FSWG} When local timestamping is enabled and the DWT or ITM transfers a Hardware source or instrumentation trace packet to the appropriate output FIFO, and the timestamp counter is nonzero, the ITM:

- Generates a Local timestamp packet.
- Resets the timestamp counter to zero.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R_{BRL} If the timestamp counter overflows, it continues counting from zero and the ITM generates an Overflow packet and transmits an associated Local timestamp packet at the earliest opportunity. If higher priority trace packets delay transmission of this Local timestamp packet, the timestamp packet has the appropriate nonzero local timestamp value.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **ITM**.*

R _{XFRH}	The ITM can generate a Local timestamp packet relating to a single event packet, or to a stream of back-to-back packets if multiple events generate a packet stream without any idle time. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{QJJB}	Local timestamp packets include status information that indicates any delay in one or both of: • Transmission of the timestamp packet relative to the corresponding event packet. • Transmission of the corresponding event packet relative to the event itself. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{NDCK}	If the ITM cannot generate a Local timestamp packet synchronously with the corresponding event packet, the timestamp count continues to increment until the ITM can generate a Local timestamp packet. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{TBMX}	The ITM compresses the count value in the timestamp packet by removing leading zeros, and transmits the smallest packet that can hold the required count value. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
I _{SQLG}	To prevent overflow, Arm recommends that the ITM emits a Local timestamp packet before the timestamp counter overflows. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>

Local timestamp clocking options

R _{DSTG}	If the implementation supports both synchronous and asynchronous clocking of the local timestamp counter, ITM_TCR.SWOENA selects the clocking mode. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{BDWS}	When software selects synchronous clocking, when local timestamping is enabled, the PE clock drives the timestamp counter, and the counter increments on each PE clock cycle. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
I _{JQJD}	When software selects synchronous clocking, whether local timestamps are generated in Debug state is IMPLEMENTATION DEFINED. Arm recommends that entering Debug state disables local timestamping, regardless of the value of the ITM_TCR.TSENA bit. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{JDRD}	When software selects asynchronous clocking, and enables local timestamping, the TPIU output interface clock drives the timestamp counter, through a configurable prescaler. The rate of asynchronous clocking depends on the output encoding scheme. This clock might be asynchronous to the PE clock. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{NGDW}	When asynchronous clocking is implemented, whether the incoming clock signal can be divided before driving the local timestamping counter is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{RMTN}	If the implementation supports division of the incoming asynchronous clock signal, ITM_TCR.TSPrescale sets the prescaler divide value. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.</i>
R _{SKCP}	Software only selects asynchronous clocking when the TPIU is configured to use an output mode that supports asynchronous clocking. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM && TPIU.</i>

R_{JGCF} When software selects asynchronous clocking and the TPIU asynchronous interface is idle, the ITM holds the timestamp counter at zero. This means that the ITM does not generate a local timestamp on the first packet after an idle on the asynchronous interface.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#) && [TPIU](#).

See also:

[B14.4 Trace Port Interface Unit](#).

Global timestamping

I_{DKSD} Global timestamps provide absolute timestamp values, which are based on a system global timestamp clock. They provide synchronization between different trace sources in the system.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{HBWD} If an implementation includes Global timestamping, the ITM generates *Global timestamp* (GTS) packets, which are based on a global timestamp clock.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{KWQJ} The size of the global timestamp is either 48 bits or 64 bits. The choice between these two options is IMPLEMENTATION DEFINED.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{SRDF} To transfer the global timestamp, two formats of Global timestamp packets are defined:

- The first packet format, Global timestamp 1 packet, holds the value of the least significant timestamp bits[25:0], and wrap and clock change indicators.
- The second packet format, Global timestamp 2 packet, holds the value of the high-order timestamp bits:
 - Bits[47:26], if a 48-bit global timestamp is supported.
 - Bits[63:26], if a 64-bit global timestamp is supported.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{VGBT} The ITM generates a full Global timestamp packet, consisting of Global timestamp 1 packet Global timestamp 2 packet, in the following circumstances:

- When software first enables global timestamps, by changing the value of the [ITM_TCR.GTSFREQ](#) field from zero to a nonzero value.
- When the system asserts the clock ratio change signal in the external ITM timestamp interface.
- In response to a Synchronization packet request, even if [ITM_TCR.SYNCENA](#) == 0.
- When the ITM has to generate a global timestamp, and the ITM detects that the value of the high-order bits of the global timestamp have changed since the Global timestamp 2 packet was last generated.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{XQWL} If the global timestamp generated by the ITM does not have to be a full global timestamp, the ITM generates only a single Global timestamp 1 packet.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{DJLN} When the ITM generates a global timestamp, it does so after a non-delayed Instrumentation or Hardware Source packet. The Global Timestamp 1 packet is always associated with the most recently output non-delayed Instrumentation or Hardware Source packet.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{WDCX}

When the ITM generates a full global timestamp:

1. The ITM first generates the Global timestamp 1 packet with timestamp bits[25:0], with the applicable bit of the Wrap and ClockCh bits in that packet set to 1 to indicate that the high-order bits of the timestamp will also be output. This is the packet that the ITM outputs immediately after a non-delayed trace packet.
2. Because of packet prioritization, the ITM might have to transmit other trace packets before it can output the Global timestamp 2 packet that contains the high-order bits of the timestamp. It might also have to transmit another Global timestamp packet. If so, it outputs the Global timestamp 1 packet with timestamp bits[25:0] and the Wrap bit set to 1.
3. The ITM later generates the Global timestamp 2 packet with the high-order timestamp bits for the most recently transmitted Global timestamp 1 packet.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

See also:

[B14.1.4 Synchronization support](#).

[B14.1.5 Continuation bits](#).

[ITM and DWT Packet Protocol Specification](#).

B14.1.4 Synchronization support

I_{LRJT}

An external debugger uses Synchronization Packets to recover bit-to-byte alignment information in a serial data stream.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

I_{LVGD}

Synchronization packets are independent of timestamp packets.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{QRFL}

If the ITM connects to a parallel trace port interface, it must generate periodic Synchronization packets. An ITM connected to an asynchronous serial trace port interface can generate Synchronization packets

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

I_{JNJV}

Arm recommends that software disables Synchronization packets when using an asynchronous serial trace port, to reduce the data stream bandwidth.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#).

R_{RMND}

If [ITM_TCR.SYNCENA](#) == 1, the ITM outputs a Synchronization packet:

- When it is first enabled.
- If [DWT_CYCCNT](#) is implemented and [DWT_CTRL.SYNCTAP](#) is nonzero, in response to a Synchronization packet request from the DWT unit.
- If [TPIU_PSCR](#) is implemented, in response to a Synchronization packet request from the TPIU:
 - If [DWT_CYCCNT](#) is not implemented, [TPIU_PSCR](#) is implemented.
 - If [DWT_CYCCNT](#) is implemented, it is IMPLEMENTATION DEFINED whether [TPIU_PSCR](#) is implemented.
- In response to other IMPLEMENTATION DEFINED Synchronization packet requests from the system.
- On exit from Debug state.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ITM](#). Note, might require additional extensions as described in the rule.

R_{NXLD} When the ITM issues a Synchronization packet the ITM clears the value of the Page register to 0.
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM. Note, When the ITM supports more than 32 Stimulus ports.

See also:

[DWT_CTRL.SYNCTAP](#).

B14.1.5 Continuation bits

I_{BFMX} A Synchronization packet consists of a bit stream of at least 47 zero bits followed by a one bit. The final bit is the byte alignment marker, and therefore bit[7] of the last byte of a Synchronization packet is always one.
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.

R_{JNVH} The longest Extension packet is always 5 bytes. In an Extension packet, bit[7] of each byte, including the header byte, but not including the last byte of a 5-byte packet, is a continuation bit, C. Bit[7] of the last byte of a 5-byte Extension packet is part of the extension field. Bit[7] of the last byte of a fewer-than-5-byte Extension packet is always zero.
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.

R_{XFTL} For all other protocol packets, bit[7] of each byte, including the header byte, but not including the last byte of a 7-byte packet, is a continuation bit, C. Bit[7] of the last byte of a packet is always zero.
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.

R_{BBSF} Each packet type defines its maximum packet length. Except for Global timestamp 2 and Synchronization packets, the longest defined packet is 5 bytes.
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.

R_{DPJG} The continuation bit, C, is defined as:

0: This is the last byte of the packet.

1: This is not the last byte of the packet.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - ITM.

B14.2 Data Watchpoint and Trace unit

B14.2.1 About the DWT

R_{QQLQ}

The *Data Watchpoint and Trace* (DWT) unit provides the following features:

- Comparators that support:
 - Use as a single comparator for instruction address matching or data address matching.
 - Use in linked pairs for instruction address range matching or data address range matching.
- Generation, on a comparator match, of:
 - A debug event that causes the PE either to enter Debug state or, if the Main Extension is implemented, to take a DebugMonitor exception.
 - Signaling a match to an ETM, if implemented.
 - Signaling a match to another external resource.
- External instruction address sampling using an instruction address sample register.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && (Debug-Monitor exception || HDBG). Note, some comparator matches require ETM.

R_{KBMX}

If the Main Extension is implemented, the DWT provides the following features:

- An optional cycle counter.
- Comparators that support:
 - Use as a single comparator for cycle counter matching, if the cycle counter is implemented.
 - Use as a single comparator for data value matching.
 - Use in linked pairs for data value matching at a specific data address.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && M.

R_{DVJV}

If the Main Extension and the ITM are implemented, the DWT provides the following trace generation features:

- Generating one or more trace packets on a comparator match.
- Generating periodic trace packets for software profiling.
- Exception trace.
- Performance event counters that generate trace.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && M && ITM.

R_{CPXJ}

If **DWT_CTRL.NOTRCPKT** is 1, there is no DWT trace support.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{FKFP}

If **DWT_CTRL.NOCYCCNT** is 1, there is no cycle counter support.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{BKGF}

If **DWT_CTRL.NOPRFCNT** is 1, there is no event counter support.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{HFTT}

The **DWT_CTRL.NUMCOMP** field indicates the number of implemented DWT comparators, which is in the range 0-15.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R _{VM} SD	It is optional whether the DWT supports Data value masking. If Data value masking is supported, <code>DWT_DEVARCH.REVISION == 0b0001</code> . <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DSPDE.</i>
R _{WQ} LX	If the Main Extension is not implemented, Cycle counter, Data value, Linked data value, and Data address with value comparators and all trace features are not supported. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - !M && DWTT.</i>
R _{SS} WT	Data trace packets are only generated for comparators 0-3. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{CR} HX	When a DWT implementation includes one or more comparators, which comparator features are supported, and by which comparators, is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

B14.2.2 DWT unit operation

I _{WT} SS	For each implemented comparator, a set of registers defines the comparator operation. For comparator <i>n</i> : • <code>DWT_COMPn</code> holds a value for the comparison. • <code>DWT_FUNCTIONn</code> defines the operation of the comparator. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{KG} NZ	<code>DWT_FUNCTION.ID</code> can be interpreted in the following way: • Bit 0, if set, indicates matches on the Cycle counter. • Bit 1, if set, indicates matches on Instruction Addresses. • Bit 2, if set, indicates matches on a Data Value. • Bit 3 indicates matches on a Data Address, and when one of the first four comparators indicates a match on Data Address with Value. This bit is always set. • Bit 4, if set, indicates linked matches. That is a Data Address Limit, an Instruction Address Limit if bit 1 is set, and Linked Data Value if bit 2 is set. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _F SSY	Arm recommends that odd numbered comparators support linking. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{LZ} MQ	<code>DWT_VMASKn</code> is an additional register that defines the comparator operation. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DSPDE.</i>
R _{DW} TC	The DWT checks for a cycle counter match on each increment of the Cycle Counter. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _X BRD	A <i>Secure match</i> is a match that is generated by one of the following: • Vector fetches where NS-Req has a value of Secure for the operation. • The hardware stacking or unstacking of registers, where NS-Req has a value of Secure for the operation, on any of: – Exception entry. – Exception exit. – Function call entry. – Function return. – Lazy state preservation.

- A privileged or unprivileged operation that is generated by an instruction that is executed in Secure state, including:
 - An Instruction address match for an instruction that is executed in Secure state.
 - A Data address or Data value match for a load or store that is generated by an instruction that is executed in Secure state.
- A Secure match can be generated by a cycle counter match in Secure state if `DWT_CTRL.CYCDISS == 1`.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT** && **S**.*

`I_DWTB`

If the Cycle Counter increments on entry or exit from Secure state no Secure match is generated.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT** && **S**.*

`R_BSKH`

An *unprivileged match* is a match that is generated by one of the following:

- The hardware stacking or unstacking of registers, where the operation is unprivileged on any of:
 - Exception entry.
 - Stacking of additional context on tail-chaining.
 - Exception exit.
 - Function call entry.
 - Function return.
 - Lazy state preservation.
- An operation that is generated by an instruction that is executed in an unprivileged mode, including:
 - An instruction address match for an instruction that is executed in unprivileged mode.
 - A Data address or Data value match for a load or store that is generalized by an instruction that is executed in an unprivileged mode.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **DWTT** && **S** && **UDE**.*

`R_NDFP`

Privileged matches are prohibited if one or more of the following conditions apply:

- `DEMCR.TRCENA == 0` or `NoninvasiveDebugAllowed(TRUE) == FALSE`. If UDE is not implemented all matches will be prohibited.
- `DWT_FUNCTION.ACTION` specifies a debug event and all the following conditions apply:
 - `HaltingDebugAllowed() == FALSE` or `DHCSR.C_DEBUGEN == 0`.
 - The Main Extension is not implemented or `DEMCR.MON_EN == 0`.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT**.*

`R_FRGS`

Unprivileged matches are prohibited if one or more of the following conditions apply:

- `DEMCR.TRCENA == 0`. or `NoninvasiveDebugAllowed(FALSE) == FALSE`.
- `DWT_FUNCTION.ACTION` specifies a debug event and all of the following are true:
 - `HaltingDebugAllowed() == FALSE` or `DHCSR.C_DEBUGEN == 0`.
 - UDE is not implemented or `UnprivHaltingDebugAllowed() == FALSE` or `DHCSR.C_DEBUGEN == 0`.
 - The Main extension is not implemented or `DEMCR.MON_EN == 0`.
 - UDE is not implemented or `DEMCR.UMON_EN == 0`.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT**.*

`I_JWHK`

If UDE is not implemented, the `NoninvasiveDebugAllowed()` function ignores the privilege mode of the match and a match will be generated if the function returns TRUE.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT**.*

R_{MJBG}

Secure matches are prohibited if one or more of the following conditions applies:

- **DWT_FUNCTION.ACTION** specifies a trace or trigger event and the following condition applies:
 - `SecureNoninvasiveDebugAllowed(TRUE) == FALSE`.
- **DWT_FUNCTION.ACTION** specifies a debug event and all of the following conditions apply:
 - `DHCSR.S_SDE == 0`.
 - The Main Extension is not implemented or `DEMCR.SDME == 0`.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **DWTT** && **S**. Note, M required if `DEMCR.SDME == 1`.*

R_{HCFP}

For address and value comparisons, the control register values and the current execution priority and Security state relate to the state of the PE when it generated the transaction that is being matched against.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT** && **S**.*

R_{FFKV}

Between a change to the debug authentication interface, **DHCSR** or **DEMCR**, that disables debug and a following **context synchronization event**, it is UNPREDICTABLE whether the DWT uses the old values or the new values.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT**.*

R_{VTNJ}

Where the DWT operation rules prohibit a match being generated, a match is not generated, even if the programmers' model defines it as being UNPREDICTABLE whether a comparator generates a match as the result of the way in which the DWT is programmed.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT**.*

R_{LFXC}

DWT_CTRL.FOLDEVTENA, **LSUEVTENA**, **SLEEPEVTENA**, **EXCEVTENA**, and **CPIEVTENA** are ignored and these fields have an **Effective value** of 0 if any of the following are true:

- `DEMCR.TRCENA == 0`.
- `NoninvasiveDebugAllowed(FALSE) == FALSE`.
- `NoninvasiveDebugAllowed(TRUE)` returns FALSE, and the PE is executing in a privileged mode.
- The Security Extension is implemented and `SecureNoninvasiveDebugAllowed(FALSE)` returns FALSE, and the PE is executing in Secure state.
- The Security Extension and UDE are implemented, `SecureNoninvasiveDebugAllowed(TRUE)` returns FALSE, and the PE is executing in Secure state and privileged mode.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT** && **UDE**. Note, UDE only available in an Armv8.1-M implementation.*

R_{BDDP}

The DWT does not generate any trace packets for an operation if any of the following are true:

- `DEMCR.TRCENA == 0`.
- `NoninvasiveDebugAllowed(FALSE)` returns FALSE.
- `NoninvasiveDebugAllowed(TRUE)` returns FALSE, and the operation is executed in a privileged mode.
- The Security Extension is implemented, `SecureNoninvasiveDebugAllowed(FALSE)` returns FALSE, and the operation is executed in Secure state.
- The Security Extension and UDE are implemented, `SecureNoninvasiveDebugAllowed(TRUE)` returns FALSE, and the operation is executed in Secure state and privileged mode.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **DWTT** && **UDE**. Note, UDE only available in an Armv8.1-M implementation.*

R _{FHWV}	If <code>SecureNoninvasiveDebugEnabled(TRUE) == FALSE</code>, <code>DWT_CTRL.FOLDEVTENA</code>, <code>LSUEVTENA</code>, <code>SLEEPEVTENA</code>, <code>EXCEVTENA</code>, and <code>CPIEVTENA</code> are ignored and these fields have an Effective value of 0 in Secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && S.</i>
R _{WSRR}	If <code>SecureNoninvasiveDebugEnabled()</code> == FALSE, Exception trace packets are not generated if the exception number in the packet represents a Secure exception: • Exception entry packets are not generated for exceptions that are taken to Secure state. • Exception exit packets are not generated for exits from Secure state. • Exception return packets are not generated for returns to Secure state. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && S.</i>
R _{DFWR}	Exception trace packets appear in the same order as for a simple sequential execution of the exception handling. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{BSRR}	When the DWT generates a match, <code>DWT_FUNCTION.MATCHED</code> is set to 1, unless the comparator is a Data address limit or Instruction address limit comparator, in which case <code>DWT_FUNCTION.MATCHED</code> is UNKNOWN. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{GSSG}	When the DWT generates a match, then if <code>DWT_FUNCTION.ACTION</code> specifies a debug event, then <code>DHCSR.C_HALT</code> is set to 1 if all of the following conditions are true: • Either of the following conditions apply: – <code>HaltingDebugEnabled()</code> returns TRUE. – <code>UnprivHaltingDebugEnabled(TRUE)</code> returns TRUE and the operation is unprivileged. • <code>DHCSR.C_DEBUGEN</code> == 1. • <code>DHCSR.S_HALT</code> == 0. • One of the following conditions apply: – <code>SecureHaltingDebugEnabled()</code> returns TRUE. – <code>SecureUnprivHaltingDebugEnabled()</code> returns TRUE and the operation is unprivileged. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DWTT && UDE.</i>
R _{CJLL}	When the DWT generates a match, then if <code>DWT_FUNCTION.ACTION</code> specifies a debug event, <code>DEMCR.MON_PEND</code> is set to 1 if all of the following conditions apply: • The PE does not set <code>DHCSR.C_HALT</code> to 1 as a result of the match. • <code>DEMCR.MON_EN</code> is set to 1, or all of the following apply: – The Unprivileged Debug Extension is implemented. – The match is an unprivileged match. – <code>DEMCR.UMON_EN</code> is set to 1. • Either: – The watchpoint was not generated by a lazy Floating-point state preservation access and the DebugMonitor exception priority was sufficient to preempt the current execution priority. – The watchpoint was generated by lazy Floating-point state preservation and <code>FPCCR.MONRDY</code> is set to 1. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DWTT && M && UDE.</i>

R _{FTBG}	When the DWT generates a match, then a Data trace match packet is generated, if all of the following conditions apply: • <code>SecureNoninvasiveDebugAllowed()</code> == FALSE. • <code>DWT_FUNCTION.ACTION</code> specifies generating a Data trace PC value packet. • The instruction address that would be included in the packet refers to an instruction that was executed in Secure state. Unless otherwise stated, the type of trace packet that is specified by <code>DWT_FUNCTION.ACTION</code> is generated. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && M && S.</i>
R _{RGWF}	When the DWT generates a match and <code>DWT_FUNCTION.ACTION</code> specifies generating a Data trace PC value packet, then the type of trace packet that is specified by <code>DWT_FUNCTION.ACTION</code> is ignored and a Data trace match packet is generated in its place, if one or more of the following conditions apply for the instruction, that is the subject of the match, referred to by the instruction address that would otherwise be included in the Data trace PC value packet: • The Unprivileged Debug Extension is implemented, <code>NoninvasiveDebugAllowed(TRUE)</code> returns FALSE, and the instruction was executed in a privileged mode. • The Security Extension and the Unprivileged Debug Extension are implemented, the instruction was executed in a Secure privileged mode, and <code>SecureNoninvasiveDebugAllowed(TRUE)</code> returns FALSE. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DWTT && M && S && UDE.</i>
R _{FNDW}	An access that results in a MemManage fault or SecureFault exception because of the alignment, SAU, IDAU, or MPU checks, is not observed by the DWT, and cannot generate a match. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && (S M && MPU).</i>
R _{PGJB}	The DWT treats hardware accesses to the stack as data accesses: • For registers pushed to the stack by hardware as part of an exception entry or lazy Floating-point state preservation. • For registers popped from the stack by hardware as part of an exception return. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{NQNR}	The DWT treats hardware accesses to the stack as data accesses: • For registers pushed to the stack by hardware as part of a Non-secure function call. • For registers popped from the stack by hardware as part of a Non-secure function return. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && S.</i>
R _{SFSC}	Where a hardware access to the stack generates a Data trace PC value packet, the PC value in the packet will be as follows: • On exception entry or a function call, the PC value will be the return address for the exception or function call. • On lazy Floating-point state preservation the PC value is the address of the instruction that triggered the lazy Floating-point state preservation. • On exception return or Non-secure function return the PC value is either: – The address of the instruction that caused the exception return or the Non-secure function return. – The <code>EXC_RETURN</code> or <code>FNC_RETURN</code> payload value used in the exception return or the Non-secure function return. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

R _{YZLM}	Watchpoints that occur as a result of DWT Unit events will be taken when all in-flight instructions have completed. However, under rare circumstances, the architecturally-visible overlap of instructions might be observable. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DWTT && MVE.</i>
I _{BPHG}	If a higher priority exception preempts the generated watchpoint, then the in-flight instruction might be visible when the PE enters Debug state or a DebugMonitor exception is subsequently taken. The debugger and software can observe the stacked value of EPSR.ECI to determine the overlap status. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DWTT && MVE && HDBG DebugMonitor exception.</i>
R _{HKDY}	Watchpoints can be triggered from a number of comparators based on configurable events. If triggered from an instruction address comparator, the watchpoint is triggered whenever the instruction attempts to execute. Predication has no effect on the watchpoint address comparator. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DWTT && MVE.</i>
R _{SJCQ}	Predication affects data address and value comparators, and a watchpoint is triggered if the memory access is not predicated. It is not triggered if the access is not performed. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DWTT && MVE.</i>
R _{ZHVG}	If Data value matching is supported, then the DWT implements the DWT_VMASK<n> registers for each implemented comparator <i>n</i> that supports Data value matching. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DSPDE.</i>
R _{KCYJ}	Armv8.1-M introduces implicit branching for BF instructions, and this behavior might induce implicit changes in the program flow. In the case where a breakpoint or watchpoint is set in program order after the branch point and the implicit branch is taken, the breakpoint or watchpoint will have no effect. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB.</i>
R _{FBRP}	It is UNKNOWN whether a Watchpoint or a Breakpoint that is targeted at an LE or LETP instruction will have any effect. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - LOB && (DWTD FPB).</i>

B14.2.3 Constraints on programming DWT comparators

R _{MSPS}	If a DWT comparator, <n>, or pair of comparators, <n> and <n+1>, is programmed with a reserved combination of DWT_FUNCTION.MATCH and DWT_FUNCTION.ACTION, then it is UNPREDICTABLE whether any comparator: • Behaves as if disabled. • Generates a match, setting the DWT_FUNCTION.MATCHED bit to an UNKNOWN value, and one or more of the following: – Asserts CMPMATCH. – Generates a debug event. – Generates one or more trace packets in any order. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{GPLQ}	Combinations of DWT_FUNCTION.MATCH and DWT_FUNCTION.ACTION that are not specified as valid combinations are reserved. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

R_{JHZK} It is IMPLEMENTATION DEFINED which values of [DWT_FUNCTION.MATCH](#) are valid for counter <n>. [DWT_FUNCTION.ID](#) defines which values are valid. Values that are not valid are reserved.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [DWTT](#).

R_{CNNH} The valid combinations of [DWT_FUNCTION.MATCH](#) and [DWT_FUNCTION.ACTION](#) for a single comparator, and the events and Data trace packets that the comparator can generate from matching a single access, are identified in the following table.

In the table:

-: means that the packet or event is not generated.

Yes: means that the packet or event is generated on a comparator match.

Comparator Type	MATCH	ACTION	Debug Event	Data Trace			
				Match Packet	PC Value Match Packet	Data Address Packet	Data Value Packet
Disabled	0b0000	0bxx	-	-	-	-	-
Cycle Counter	0b0001	0b00	-	-	-	-	-
		0b01	Yes	-	-	-	-
		0b10	-	Yes	-	-	-
		0b11	-	-	Yes	-	-
Instruction Address	0b0010	0b00	-	-	-	-	-
		0b01	Yes	-	-	-	-
		0b10	-	Yes	-	-	-
Data address	0b01xx (not 0b0111)	0b00	-	-	-	-	-
		0b01	Yes	-	-	-	-
		0b10	-	Yes	-	-	-
		0b11	-	-	Yes	-	-
Data value	0b10xx (not 0b1011)	0b00	-	-	-	-	-
		0b01	Yes	-	-	-	-
		0b10	-	Yes	-	-	-
Data address with value	0b11xx (not 0b1111)	0b10	-	-	-	-	Yes
		0b11	-	-	Yes	-	Yes

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [DWTT](#). Note, Cycle counter, Data value and Data address with value are only available if *M* is implemented.

Instruction address range

R_{DKHG} To match an instruction that is in an instruction address range, the following conditions are met:

- The first comparator, <n-1>, is programmed for *Instruction address*.
- The second comparator, <n>, is programmed for *Instruction address limit*.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [DWTT](#).

R_{LNQD}

The valid combinations of [DWT_FUNCTION.MATCH](#) and [DWT_FUNCTION.ACTION](#) for an instruction address range, and the events and data trace packets that matching a single access can generate, are specified in the following table.

In the table:

-: means that the packet or event is not generated.

First: means that the packet or event is generated by the first comparator match.

Second: means that the packet or event is generated by the second comparator match.

MATCH		ACTION		Data Trace				
<n-1>	<n>	<n-1>	<n>	Debug Event	Match packet	PC Value packet	Data Address packet	Data Value packet
0b0000	0b0011	0bxx	0bxx	-	-	-	-	-
0b0010	0b0011	0b00	0b00	-	-	-	-	-
		0b00	0b11	-	-	Second	-	-
		0b01	0b00	First	-	-	-	-
		0b10	0b00	-	First	-	-	-

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [DWTT](#) && [M](#).

R_{VDRJ}

If the Main Extension is not implemented the valid combinations of [DWT_FUNCTION.MATCH](#) and [DWT_FUNCTION.ACTION](#) for an instruction address range, and the events and data trace packets that matching a single access can generate, are specified in the following table.

In the table:

-: means that the packet or event is not generated.

First: means that the packet or event is generated by the first comparator match.

Second: means that the packet or event is generated by the second comparator match.

MATCH		ACTION		Data Trace				
<n-1>	<n>	<n-1>	<n>	Debug Event	Match packet	PC Value packet	Data Address packet	Data Value packet
0b0000	0b0011	0bxx	0bxx	-	-	-	-	-
0b0010	0b0011	0b00	0b00	-	-	-	-	-
		0b01	0b00	First	-	-	-	-

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [DWTT](#) && [!M](#).

Data address range

R_{LDGR}

To match a data access in a data address range, the following conditions are met:

- The first comparator, <n-1>, is programmed for either *Data address* or *Data address with value*.
- The second comparator, <n>, is programmed for *Data address limit*.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [DWTT](#) && [M](#).

R_{PSBJ}

The valid combinations of [DWT_FUNCTION.MATCH](#) and [DWT_FUNCTION.ACTION](#) for a data address range, and the events and data trace packets that matching a single access can generate, are specified in the following table.

In the table:

-: means that the packet or event is not generated.

First: means that the packet or event is generated by the first comparator match.

Second: means that the packet or event is generated by the second comparator match.

MATCH		ACTION		Data Trace				
<n-1>	<n>	<n-1>	<n>	Debug Event	Match packet	PC Value packet	Data Address packet	Data Value packet
0b0000	0b0111	0bxx	0bxx	-	-	-	-	-
0b01xx	0b0111	0b00	0b00	-	-	-	-	-
(not 0b0111)		0b00	0b11	-	-	-	Second	-
		0b01	0b00	First	-	-	-	-
		0b10	0b00	-	First	-	-	-
		0b11	0b00	-	-	First	-	-
(not 0b1111)		0b11	0b11	-	-	First	Second	-
		0b10	0b00	-	-	-	-	First
		0b10	0b11	-	-	-	Second	First
		0b11	0b00	-	-	First	-	First
		0b11	0b11	-	-	First	Second	First

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [DWTT](#).

R_{HDMX}

If the Main Extension is not implemented the valid combinations of and for a data address range, and the events and data trace packets that matching a single access can generate, are specified in the following table.

In the table:

-: means that the packet or event is not generated.

First: means that the packet or event is generated by the first comparator match.

Second: means that the packet or event is generated by the second comparator match.

MATCH		ACTION		Data Trace				
<n-1>	<n>	<n-1>	<n>	Debug Event	Match packet	PC Value packet	Data Address packet	Data Value packet
0b0000	0b0111	0bxx	0bxx	-	-	-	-	-
0b01xx	0b0111	0b00	0b00	-	-	-	-	-
(not 0b0111)		0b00	0b11	-	-	-	Second	-
		0b01	0b00	First	-	-	-	-

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - [DWTT](#) && [!M](#).

Data value at specific address

R _{KFHV}	Matching data values at specific data addresses is possible only if the Main Extension is implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{NNXD}	To match a data value at a specific data address, the following conditions are met: • The first comparator, <n-1>, is programmed for either <i>Data address</i> or <i>Data address with value</i>. • The second comparator, <n>, is programmed for <i>Linked data value</i>. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{JKGJ}	The first comparator matches any access that matches the address. The second matches only accesses that match the address and the data value. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{NTSD}	The valid combinations of DWT_FUNCTION.MATCH and DWT_FUNCTION.ACTION for a linked data value, and the events and data trace packets that matching a single access can generate, are specified in the following table. In the table: -: means that the packet or event is not generated. First: means that the packet or event is generated by the first comparator match. Second: means that the packet or event is generated by the second comparator match. Both: means that a first packet is generated by a first comparator match, even if the Linked data value comparator does not match, and a second packet is generated by the second comparator match, if both comparators match.

MATCH		ACTION		Data Trace				
<n-1>	<n>	<n-1>	<n>	Debug Event	Match packet	PC Value packet	Data Address packet	Data Value packet
0b0000	0b1011	0bxx	0bxx	-	-	-	-	-
0b01xx	0b1011	0b00	0b00	-	-	-	-	-
(not 0b0111)		0b00	0b01	Second	-	-	-	-
		0b00	0b10	-	Second	-	-	-
		0b01	0b00	First	-	-	-	-
		0b01	0b10	First	Second	-	-	-
		0b10	0b00	-	First	-	-	-
		0b10	0b01	Second	First	-	-	-
		0b11	0b00	-	-	First	-	-
		0b11	0b01	Second	-	First	-	-
		0b11	0b10	-	Second	First	-	-
0b11xx	0b1011	0b10	0b00	-	-	-	-	First
(not 0b1111)		0b10	0b01	Second	-	-	-	First
		0b10	0b10	-	Second	-	-	First
		0b11	0b00	-	-	First	-	First
		0b11	0b01	Second	-	First	-	First
		0b11	0b10	-	Second	First	-	First

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

I_{XTTW}

Software should be aware of the order in which it enables or disables pairs of comparators that are linked so as to avoid UNPREDICTABLE, or unexpected, behaviors.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

B14.2.4 CMPMATCH trigger events

I_{VNCC}

The **CMPMATCH** events signal watchpoint matches.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R_{PRJG}

The implementation of **CMPMATCH** is IMPLEMENTATION DEFINED.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R_{FTWC}

If an ETM is implemented, **CMPMATCH** events are output to the ETM.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT** & **ETM**.

R_{TMZX}

If an ETM is not implemented, the effect of **CMPMATCH** is IMPLEMENTATION DEFINED, including whether the trigger event has any observable effect or whether observable effects are visible to other components in the system.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R_{XXKM}

For all enabled watchpoints, if **DWT_FUNCTION_n** is not programmed as an Instruction address limit comparator and is not programmed as a Data address limit comparator, **CMPMATCH[n]** is triggered on a comparator match.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{GVHS} For all enabled watchpoints, if **DWT_FUNCTION_n** is programmed as an Instruction address limit or Data address limit comparator, it is UNPREDICTABLE whether **CMPMATCH_[n]** is triggered on a comparator match.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

B14.2.5 Matching in detail

Instruction address matching in detail

R_{GNVB} The DWT checks all instructions that are executed by a simple sequential execution of the program and do not generate any exception for an instruction address match, including conditional instructions that fail their condition code check.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{NQGR} An instruction might be checked by the DWT for an instruction address match if it either:

- Is executed by a simple sequential execution of the program and generates a synchronous exception.
- Would be executed by the sequential execution of the program but is abandoned because of an asynchronous exception.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{KJJC} Speculative instruction prefetches, other than those that would be executed by the sequential execution of the program but that are abandoned because of asynchronous exceptions, do not generate matches.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{DDST} For all instruction address matches, if bit[0] of the comparator address has a value of 1, it is UNPREDICTABLE whether a match is generated when the other address bits match.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{KLXM} For single instruction address matches, an instruction matches if the address of the first byte of the instruction matches the comparator address.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{FXFM} For single address matches, if the instruction at address A is a 4-byte T32 instruction, and the address A+2 matches but the address A does not match, it is UNPREDICTABLE whether a match is generated.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{DNKD} For instruction address range matches, an instruction at address A matches if the address A lies between the lower comparator address, which is specified by comparator <n-1>, and the limit comparator address, which is specified by comparator <n>. Both addresses are inclusive to the range.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{JNXZ} For instruction address range matches, if the instruction at address A is a 4-byte T32 instruction, and the address A+2 lies in the range but the address A does not lie in the range, it is UNPREDICTABLE whether a match is generated.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{MLMQ} For instruction address range matches, if so configured, a Data trace PC value packet or Data trace match packet is generated for the first instruction that is executed in the range.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

I_VHHW	For instruction address range matches, if so configured, a branch or sequential execution that stays within the range does not necessarily generate a new packet. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_HMNx	For instruction address range matches, if so configured, CMPMATCH [n-1] is triggered for each instruction that is executed inside the range, where n-1 is the lower of the two comparators that configure the range. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

Data address matching in detail

R_BPWC	For all Data Address matches, all bits of the comparator address are considered. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_GSLX	Speculative reads might generate data address matches. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_WWBH	Speculative writes do not generate data address matches. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_VJFB	Prefetches into a cache do not generate data address matches. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_CMRP	For single data address matches, an access matches if any accessed byte lies between the comparator address and a limit that is defined by DWT_FUNCTION.DATAVSIZE . <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_KHRF	For single data address matches, the comparator address is naturally-aligned to DWT_FUNCTION.DATAVSIZE otherwise generation of watchpoint events is UNPREDICTABLE. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_KKRJ	For data address range matches, an access matches if any accessed byte lies between the lower comparator address, which is specified by comparator <n-1>, and the limit comparator address, which is specified by comparator <n>. Both addresses are inclusive to the range. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_CFMx	For data address range matches, DWT_FUNCTION.DATAVSIZE is set to 0b00 for both the lower comparator address and the limit comparator address otherwise it is UNPREDICTABLE whether or not a match is generated. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

Data value matching in detail

R_BMSM	Data value matching is only possible if the Main Extension is implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_FVFQ	Speculative reads might generate data value matches. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_VGJF	Speculative writes do not generate data value matches. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R_MLFK	Prefetches into a cache do not generate data value matches. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

R _{RMDB}	For data value matches, if the access size is smaller than DWT_FUNCTION.DATAVSIZE , there is no match. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{ZDPM}	For unlinked data value matches, an access matches if all bytes of any naturally-aligned subset, the size of which is specified by DWT_FUNCTION.DATAVSIZE , of the access match the data value in DWT_COMPn . <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{ZHXP}	The data value in DWT_COMPn is in little-endian order with respect to memory. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{HMS}	If the access is unaligned then this might generate a higher priority alignment fault, depending on the instruction type, profile, and configuration. In these cases no match is generated. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{SQKS}	For unlinked data value matches, if an access is unaligned, it is IMPLEMENTATION DEFINED whether it either treated as: • A sequence of byte accesses. • A sequence of naturally-aligned accesses covering the accessed bytes. For a read, this access might access more bytes than the original access. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{QRPW}	For linked data value matching, if an access is larger than DWT_FUNCTION.DATAVSIZE , then only the naturally-aligned subset of the access of size DWT_FUNCTION.DATAVSIZE at the matching address is compared for a match. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{QVRK}	For linked data value matching, the data address comparator address is naturally-aligned to DWT_FUNCTION.DATAVSIZE , and the DWT_FUNCTION.DATAVSIZE values for both comparators are the same. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{KRCV}	A Data value comparator that is linked to a Data address comparator does not change the behavior of the address comparator. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{KQJB}	For each comparator <i>n</i> that is configured to Data Value or Linked Data Value matching it is UNPREDICTABLE whether comparator <i>n</i> generates a match when for bit <i>m</i> =31-0, if any of the following are true: • DWT_FUNCTION<n>.DATAVSIZE specifies halfword or byte comparison and DWT_COMPn[31:16] is not equal to DWT_COMPn[15:0]. • DWT_FUNCTION<n>.DATAVSIZE specifies byte comparison and DWT_COMPn[15:8] is not equal to DWT_COMPn[7:0]. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{ZKRZ}	For each comparator <i>n</i> that is configured to Data Value or Linked Data Value matching the value matches if, for each bit[<i>m</i>] any of the following are true: • Bit[<i>m</i>] of the value is equal to DWT_COMPn[<i>m</i>]. • DWT_VMASKn is implemented, DWT_COMPn[<i>m</i>] is set to 0, and DWT_VMASKn[<i>m</i>] is set to 1. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - DSPDE.</i>
R _{BVLK}	For each comparator <i>n</i> that is configured for Data Value or Linked Value matching, if DWT_VMASKn is implemented, then it is UNPREDICTABLE whether comparator <i>n</i> generates a match when, for bit <i>m</i> = 0-31, all of the following are true:

- `DWT_VMASKn[m]` is set to 1.
- `DWT_COMPN[m]` is not set to 0.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DSPDE**.

R_{HRTJ} For each comparator *n* that is configured for Data Value or Linked Value matching, if `DWT_VMASKn` is implemented, then it is UNPREDICTABLE whether comparator *n* generates a match if any of the following are true:

- `DWT_FUNCTION<n>.DATAVSZIE` specifies halfword or byte comparison and `DWT_VMASKn[31:16]` is not equal to `DWT_VMASKn[15:0]`.
- `DWT_FUNCTION<n>.DATAVSZIE` specifies byte comparison and `DWT_VMASKn[15:8]` is not equal to `DWT_VMASKn[7:0]`.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DSPDE**.

R_{SMHR} For each comparator *n* that is configured for neither Data Value nor Linked Data Value matching, `DWT_VMASKn` is ignored if it is implemented.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **DSPDE**.

See also:

`DWT_AddressCompare()`.

`DWT_ValidMatch()`.

`DWT_InstructionAddressMatch()`.

`DWT_DataAddressMatch()`.

`DWT_DataValueMatch()`.

B14.2.6 DWT match restrictions and relaxations

R_{FRWG} It is IMPLEMENTATION DEFINED whether the DWT treats a fetch from the exception vector table as part of an exception entry or reset as a data access or ignores these accesses, for the purposes of DWT comparator matches.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R_{DTWH} A fetch by the DWT from the exception vector table as part of an exception entry is never treated as an instruction fetch.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R_{JQHW} If a return is tail-chained, it is IMPLEMENTATION DEFINED whether hardware accesses the stack and therefore IMPLEMENTATION DEFINED whether the DWT can generate events or trace.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R_{VJTK} The DWT does not match accesses from the DAP.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R_{MNBX} Any executed `NOP` or `IT` that matches an appropriately configured instruction address watchpoint causes a match.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R_{SLPX} It is IMPLEMENTATION DEFINED whether a failed **STREX** instruction can generate a data access match.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R_{NHLN} If an instruction or operation makes multiple or unaligned data accesses, then it is UNPREDICTABLE whether any nonmatching access generated by an instruction that generated a matching access is treated as a matching access.

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **DWTT**.

R _{CSSQ}	If an instruction or operation makes multiple or unaligned data accesses, then CMPMATCH is triggered for each matching access. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{VFXT}	If an instruction or operation makes multiple or unaligned data accesses, then, if so configured, only a data value match of at least a part of the value that is guaranteed to be single-copy atomic can generate a match. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{WJNR}	If an instruction or operation makes multiple or unaligned data accesses, then, if so configured, for a matching data access that generates a debug event, if permitted, DHCSR.C_HALT or DEMCR.MON_PEND, as applicable, is set to 1. A pending DebugMonitor exception does not interrupt the multiple accesses, but another interrupt might, which means that the debug event might be taken before the multiple operations complete. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{QCJL}	It is UNKNOWN if the DWT will match on the addresses of accesses that generate a BusFault. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{QVHL}	It is IMPLEMENTATION DEFINED whether a stored value for an access that generates a BusFault: • Can generate a data value match. • Can be traced. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{KLFC}	For a load access that returns a BusFault, any data that is returned by the memory system is invalid, and the DWT does not: • Generate a data value match. • Generate a Data trace data value packet. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{TQCF}	A data access that generates any fault other than a BusFault does not generate a data address or data value match at the DWT and is not traced. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{FRHP}	DWT matches are generated asynchronously. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{THHR}	A DSB barrier guarantees that the effect of a DWT match is visible to a subsequent read of DWT_FUNCTION.-MATCHED, DHCSR, or DEMCR. In the absence of a DSB barrier, the effect is only guaranteed to be visible in finite time. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{HPGH}	The effects of a DWT match never affect instructions appearing in program order before the operation that generates the match. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

See also:

[B3.26 Tail-chaining.](#)

B14.2.7 DWT trace restrictions and relaxations

R _{PGCS}	If a single instruction makes multiple single-copy atomic accesses, such as the multiple-byte accesses from an unaligned access or a predicated vector load/store operation, the DWT might gather a sequence of consecutive bytes from the multiple accesses, and trace it as a single access. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{HDKK}	Where a single instruction or operation, or multiple instructions, generate multiple accesses that each generate one or more trace packets, then if the architecture guarantees the order in which a pair of these accesses is observed by the PE, the first trace packets that are generated for each of those accesses appear in the trace output in the same order. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{WSKK}	Where a single instruction or operation, or multiple instructions, generate multiple accesses that each generate one or more trace packets, then if the architecture does not guarantee the order of the accesses, the order of the trace packets in the trace output is not defined. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{XCNB}	If a single instruction or operation makes multiple or unaligned data accesses, then, if so configured, only the first access is guaranteed to generate a Data trace PC value packet, Data trace data address packet, or Data trace match packet. If the architecture does not guarantee the order of the accesses, the first access might be any of the accesses. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{XVBT}	If a single instruction or operation makes multiple or unaligned data accesses, then, if so configured, a Data trace data value packet is generated for each matching access. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{QSCF}	If a single instruction or operation makes unaligned data accesses, it is UNPREDICTABLE how many Data trace data value packets are generated for each unaligned matching access. An implementation might overread, meaning that more data outside the access might be traced. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{KXBL}	If a single instruction or operation makes multiple or unaligned data accesses, then, if so configured, for a matching data access that generates a Data trace data value packet, at least that part of the value that is guaranteed to be single-copy atomic is traced. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{QWQS}	Duplicate Data trace PC value packets, Data trace data address packets, and Data trace data value packets from a single access are not generated for a single access. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{BSRR}	Where a comparator or linked pair of comparators generates multiple packet types for a single access, the packets appear in the trace output in the following order: 1. Data trace PC value packet. 2. Data trace match packet, generated by a Data address, Data address with value, or Data address limit comparator match. 3. Data trace data address packet. 4. Data trace match packet, generated by a Data value comparator match or a Linked Data Value match. 5. Data trace data value packet. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{QXBC}	Where a comparator or linked pair of comparators generates multiple packet types for a single access, packets are not interleaved with packets that are generated by other accesses by the same comparator or linked pair of comparators. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

R _{RHNF}	Where a comparator or linked pair of comparators generates a trace packet for a single access, if a comparator other than this comparator or this linked pair of comparators generates a trace packet of the same type for the same access, then only one of these packets is output. It is IMPLEMENTATION DEFINED which comparator is chosen. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{MJXG}	Arm recommends that the packet from the lowest-numbered comparator is output. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{DKMV}	Where a comparator or linked pair of comparators generates multiple packet types for a single access, if any of the packets cannot be output and an Overflow packet is generated, then the remaining packets for that access are not generated. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{LNBW}	Where a comparator or linked pair of comparators generates multiple packet types for a single access, packets might be interleaved with packets that are generated for the same access by comparators other than this comparator or this linked pair of comparators. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

B14.2.8 CYCCNT cycle counter and related timers

R _{SVPW}	CYCCNT is an optional free-running 32-bit cycle counter. If the DWT unit implements CYCCNT then DWT_CTRL.NOCYCCNT is RAZ. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{XDVS}	The cycle counter, DWT_CYCCNT, and the POSTCNT counter are disabled when DEMCR.TRCENA == 0, but are not otherwise affected by debug authentication. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{RTJR}	The cycle counter does not count in Secure state when DWT_CTRL.CYCDISS is set to 1. This is independent of Secure debug authentication. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && S.</i>
R _{KRFP}	When implemented and enabled, CYCCNT increments on each cycle of the PE clock, using the same definition of cycle as the Event counters when counting cycles in power-saving modes. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{NFJW}	When the counter overflows it transparently wraps to zero. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{GXJK}	DWT_CTRL.CYCCNTENA enables the CYCCNT counter. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{BKCG}	POSTCNT is a 4-bit countdown counter derived from CYCCNT, that acts as a timer for the periodic generation of Periodic PC sample packets or Event counter packets, when these packets are enabled. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{MGGL}	Periodic PC sample packets are not the same as the Data trace PC value packets that are generated by the DWT comparators. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{DKGR}	The DWT does not support the generation of Periodic PC sample packets or Event packets if it does not implement the CYCCNT timer and DWT_CTRL.NOTRCPKT is RAO. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

R_{RNTV} The [DWT_CTRL.CYCTAP](#) bit selects the CYCCNT tap bit for POSTCNT.

CYCTAP bit	CYCCNT tap at	POSTCNT clock rate
0	Bit[6]	(PE clock)/64
1	Bit[10]	(PE clock)/1024

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{SXKK} A write to [DWT_CTRL](#) will initialize POSTCNT to the previous value of [DWT_CTRL.POSTINIT](#) if all of the following are true:

- [DWT_CTRL.PCSAMPLENA](#) was set to 0 prior to the write.
- [DWT_CTRL.CYCEVTENA](#) was set to 0 prior to the write.
- The write sets either [DWT_CTRL.PCSAMPLENA](#) or [DWT_CTRL.CYCEVTENA](#) to 1.

It is UNPREDICTABLE whether any other write to [DWT_CTRL](#) that alters the value of [DWT_CTRL.PCSAMPLENA](#) and [DWT_CTRL.CYCEVTENA](#) sets POSTCNT to [DWT_CTRL.POSTINIT](#) or leaves POSTCNT unchanged.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{XFRM} When either [DWT_CTRL.PCSAMPLENA](#) or [DWT_CTRL.CYCEVTENA](#) is set to 1, and the CYCCNT tap bit transitions, either from 0 to 1 or from 1 to 0:

- If POSTCNT is nonzero, POSTCNT decrements by 1.
- If POSTCNT is 0, the DWT:
 - Reloads POSTCNT from [DWT_CTRL.POSTPRESET](#).
 - Generates a Periodic PC Sample packets if [DWT_CTRL.PCSAMPLENA](#) is set to 1.
 - Generates an Event Counter packet with the Cyc bit set to 1 if [DWT_CTRL.CYCEVTENA](#) is set to 1.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

I_{PNNs} The enable bit for the POSTCNT counter underflow event is [DWT_CTRL.CYCEVTENA](#). There is no overflow event for the CYCCNT counter. When CYCCNT overflows it wraps to zero transparently. Software cannot access the POSTCNT value directly, or change this value.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

I_{JRVV} This means that, to initialize POSTCNT, software:

1. Ensures that [DWT_CTRL.CYCEVTENA](#) and [DWT_CTRL.PCSAMPLENA](#) are set to 0. This can be achieved with a single write to [DWT_CTRL](#). This is also the reset value of these bits.
2. Writes the required initial value of POSTCNT to the [DWT_CTRL.POSTINIT](#) field, leaving [DWT_CTRL.CYCEVTENA](#) and [DWT_CTRL.PCSAMPLENA](#) set to 0.
3. Sets either [DWT_CTRL.CYCEVTENA](#) or [DWT_CTRL.PCSAMPLENA](#) to 1 to enable the POSTCNT counter.

Each of these are separate writes to [DWT_CTRL](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{KNHF} Disabling CYCCNT stops POSTCNT.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{TMHN} Writes to [DWT_CTRL.POSTINIT](#) are ignored if either [DWT_CTRL.CYCEVTENA](#) was set to 1 or [DWT_CTRL.PCSAMPLENA](#) was set to 1 prior to the write.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

B14.2.9 Profiling counter support

I _{HXPV}	If the Main Extension is implemented event counter support is an optional Non-invasive debug feature. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && M.</i>
R _{WHWR}	If event counter support is implemented the DWT provides five 8-bit Event counters for software profiling: • DWT_FOLDCNT. • DWT_LSUNCT. • DWT_EXCCNT. • DWT_SLEEP CNT. • DWT_CPICNT. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{GLMJ}	DWT event counters do not increment when the PE is halted. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{BRGW}	The Event counters provide broadly accurate and statistically useful count information. However, the architecture allows for a reasonable degree of inaccuracy in the counts. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{WMNV}	The Event counters and CYCCNT use the same definition of cycle in particular when counting cycles in power-saving modes. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && M.</i>
I _{GNWQ}	To keep the implementation and validation cost low, a reasonable degree of inaccuracy in the counts is acceptable. Arm does not define a reasonable degree of inaccuracy but recommends the following guidelines: • Under normal operating conditions, the Event counters present an accurate value count. • Entry to or exit from Debug state can be a source of inaccuracy. • Under very unusual, non-repeating pathological cases, the counts can be inaccurate. An implementation does not introduce inaccuracies that can be triggered systematically by the execution of normal pieces of software. As the Event counters include counters for measuring exception overhead, this includes the operation of exceptions. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{CHKR}	Arm strongly recommends that an implementation document any particular scenarios where significant inaccuracies in the Event counters are expected. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{MWQ}	At entry and exit from an exception or sleep state, the exact attribution of cycles to the exception and cycles to the sleep overhead counters is IMPLEMENTATION DEFINED. Arm recommends that the overhead cycles are attributed to the overhead counters. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{MPQN}	The architecture does not define the point in a pipeline where the particular instruction increments an Event counter, relative to the point where the incremented counter can be read. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{LMPK}	An Event counter overflows on every 256th event that is counted and then wraps to 0. If the appropriate counter overflow event is enabled in DWT_CTRL the DWT outputs an Event counter packet with the appropriate counter flag set to 1. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>

R_{LHMB} Setting one of the enable bits to 1 clears the corresponding counter to 0.
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

I_{QRP} The following equation holds:
$$ICNT = CNT_{CYCLES} + CNT_{FOLD} - (CNT_{LSU} + CNT_{EXC} + CNT_{SLEEP} + CNT_{CPI})$$

Where:

$ICNT$: is the total number of instructions Architecturally executed.

CNT_{CYCLES} : is the number of cycles counted by DWT_CYCCNT.

CNT_{FOLD} : is the number of instructions counted by DWT_FOLDCNT.

CNT_{LSU} : is the number of cycles counted by DWT_LSUNCT.

CNT_{EXC} : is the number of cycles counted by DWT_EXCCNT.

CNT_{SLEEP} : is the number of cycles counted by DWT_SLEEPcnt.

CNT_{CPI} : is the number of cycles counted by DWT_CPICNT.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

See also:

[B14.4 Trace Port Interface Unit.](#)

Generating Overflow packets from Event counters

R_{KWDH} If an Event counter wraps to zero and the previous Event counter packet has been delayed and has not yet been output, and the counter flag in the previous Event counter packet is set to 0, then it is IMPLEMENTATION DEFINED whether:

- The DWT attempts to generate a second Event counter packet.
- The DWT updates the delayed Event counter packet to include the new wrap event.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{HKTL} If an Event counter wraps to zero and the previous Event counter packet has been delayed and has not yet been output, and the counter flag in the previous Event counter packet is set to 1, the DWT attempts to generate a second Event counter packet.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{VPXK} If the DWT unit attempts to generate a packet when its output buffer is full, an Overflow packet is output.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{SFFL} The size of the DWT output buffer is IMPLEMENTATION DEFINED.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

B14.2.10 Program Counter sampling support

R_{FXWL} Program Counter sampling is an optional component provided through DWT_PCSR.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

I_{LNJL} Program Counter sampling is independent of PC sampling provided by:

- Periodic PC sample packets.
- Data trace PC value packets generated as a result of a DWT comparator match.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

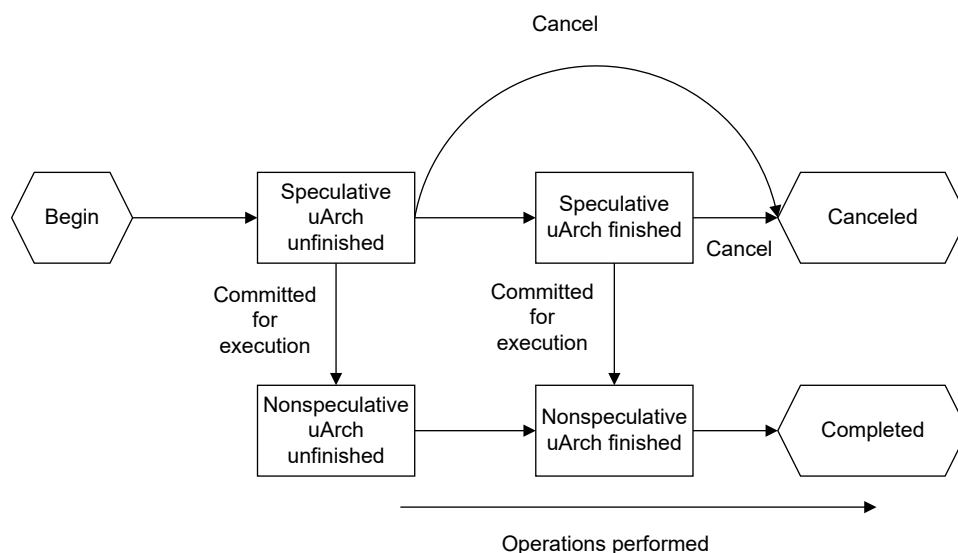
I_{KVFB} The architecture does not define the delay between an instruction being executed by the PE and its address being written to **DWT_PCSR**.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{NGNT} When **DWT_PCSR** returns a value other than 0xFFFFFFFF, the returned value is an instruction that has been committed for execution. It is IMPLEMENTATION DEFINED whether an instruction that failed its condition code check is considered as committed for execution. A read of **DWT_PCSR** does not return the address of an instruction that has been fetched but not committed for execution.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

I_{TNVH} The following diagram describes when instructions are committed for execution:



Applies to an implementation of the architecture Armv8.0-M onward.

I_{KCBH} Arm recommends that instructions that fail the condition code check are considered as committed instructions.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{WPMF} **DWT_PCSR** is able to sample references to branch targets. It is IMPLEMENTATION DEFINED whether it can sample references to other instructions.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

I_{SJVK} Arm recommends that **DWT_PCSR** can sample a reference to any instruction.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R_{LMDG} The branch target for a conditional branch that fails its Condition code check is the instruction that immediately follows the conditional branch instruction. The branch target for an exception is the exception vector address.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.

R _{NWKP}	Periodic sampling of DWT_PCSR provides broadly accurate and statistically useful profile information. However, the architecture allows for a reasonable degree of inaccuracy in the sampled data. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{TJTS}	To keep the implementation and validation cost low, a reasonable degree of inaccuracy in the counts is acceptable. Arm does not define <i>a reasonable degree of inaccuracy</i> but recommends the following guidelines: • In exceptional circumstances, such as a change in Security state or other boundary condition, it is acceptable for the sample to represent an instruction that was not committed for execution. • Under unusual non-repeating pathological cases, the sample can represent an instruction that was not committed for execution. These cases are likely to occur as a result of asynchronous exceptions, such as interrupts, where the chance of a systematic error in sampling is very unlikely. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
I _{KVJM}	Arm strongly recommends that an implementation document any particular scenarios where significant inaccuracies in the sampled data are expected. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT.</i>
R _{YPHZ}	A read of DWT_PCSR will return 0xFFFFFFFF if any of the following are true: • The PE is in Debug state. • The address of a recently executed instruction is not available. • NoninvasiveDebugAllowed(FALSE) returns FALSE. • The Unprivileged Debug Extension is implemented, NoninvasiveDebugAllowed(TRUE) returns FALSE, and the instruction was executed in a privileged mode. • The Security Extension is implemented and SecureNoninvasiveDebugAllowed(FALSE) returns FALSE, and the instruction was executed in Secure state. • The Security Extension and the Unprivileged Debug Extension are implemented, SecureNoninvasiveDebugAllowed(TRUE) returns FALSE, and the instruction was executed in Secure state and privileged mode. • DEMCR.TRCENA is set to 0. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - DWTT && UDE && S. Note, UDE Is only available in an Armv8.1-M implementation.</i>

B14.3 Embedded Trace Macrocell

I_{LCCX} An *Embedded Trace Macrocell* (ETM) is an optional non-invasive debug feature of an Armv8-M implementation.
Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ETM](#).

R_{NGTT} An ETM implementation complies with one of the following versions of the ETM architecture:

	Data trace	Security Extension	
		Implemented	Not implemented
Implemented		ETMv3 not permitted	ETMv3 not permitted
		ETMv4, version 4.2 or later	ETMv4, version 4.0 or later
Not Implemented		ETMv3, version 3.5 or later	ETMv3, version 3.5
		ETMv4, version 4.2 or later	ETMv4, version 4.0 or later

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ETM](#).

R_{QWRD} If Armv8.1-M is implemented and an ETM is implemented, at least ETMv4 version 4.5 is required. Data trace is not supported in ETMv4 version 4.5 and is not supported in Armv8.1-M.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [ETM](#).

R_{LPJM} If an ETM is implemented a trace sink is also implemented. If the trace sink that is implemented is the TPIU it is CoreSight compliant, and complies with the TPIU architecture for compatibility with Arm and other CoreSight-compatible debug solutions.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ETM](#).

R_{NLNS} When an Armv8-M implementation includes an ETM, the **CMPMATCH[N]** signals from the DWT unit are available as control inputs to the ETM unit.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ETM](#).

R_{NJDK} If the Main Extension is not implemented, it is IMPLEMENTATION DEFINED whether the ETM is accessible only to the debugger and is RES0 to software.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ETM](#) && [!M](#).

R_{WPBN} If the ETMv3 is implemented the debugger programs the ETMTRACEIDR with a unique nonzero Trace ID for the ETM trace stream.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ETM](#).

R_{TJSF} If the ETMv4 is implemented the debugger programs the TRCTRACEIDR with a unique nonzero Trace ID for the ETM trace stream.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ETM](#).

I_{MNBN} The ETM is not directly affected by [DEMCR.TRCENA](#) being set to 0.

A system might employ significant power saving techniques when tracing is not active, such as gating of clocks directly or indirectly based on trace enable signals, however these techniques must not affect the ability to program the ETM. Depending on the PE implementation, responsibility for ensuring this might fall on the system architecture into which the PE is inserted.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [ETM](#).

See also:

Arm® *CoreSight™ Architecture Specification*.

[B14.2.4 CMPMATCH trigger events.](#)

B14.4 Trace Port Interface Unit

I _{PWXP}	The <i>Trace Port Interface Unit</i> (TPIU) support for Armv8-M provides an output path for trace data from the DWT, ITM, and ETM. The TPIU is a trace sink. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{CRTQ}	It is IMPLEMENTATION DEFINED whether the TPIU supports a parallel trace port output. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{GTRP}	It is IMPLEMENTATION DEFINED whether the TPIU supports low-speed asynchronous serial port output using NRZ encoding. This operates as a traditional UART. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{LKQT}	It is IMPLEMENTATION DEFINED whether the TPIU supports medium-speed asynchronous serial port output using Manchester encoding. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
I _{SDDK}	Arm recommends that the TPIU provides both parallel and asynchronous serial ports, for maximum flexibility with external capture devices. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{HJXK}	Whether the trace port clock is synchronous to the PE clock is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{PKKS}	It is IMPLEMENTATION DEFINED whether the TPIU is reset by a Cold reset or has an independent Cold reset. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{JBKJ}	Software ensures that all trace is output and flushed to the trace sink before setting the DEMCR.TRCENA bit to 0. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
I _{MNBK}	The TPIU is not directly affected by DEMCR.TRCENA being set to 0 or <code>NoninvasiveDebugAllowed()</code> being FALSE. A system might employ significant power saving techniques when tracing is not active, such as gating of clocks directly or indirectly based on trace enable signals, however these techniques must not affect the ability to program the TPIU. Depending on the PE implementation, responsibility for ensuring this might fall on the system architecture into which the PE is inserted. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{JLCQ}	The output formatting modes that are supported by the TPIU are IMPLEMENTATION DEFINED. They are: • Bypass. • Continuous. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{DMFP}	Bypass mode is only supported if a serial port output is supported. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{RRJP}	Continuous mode is supported if the parallel trace port is implemented. Continuous mode is selected when the parallel trace port is used. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>
R _{FCFT}	Continuous mode is supported if the ETM is implemented. Continuous mode is selected when the ETM is used. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - TPIU.</i>

R_{BRSK}

When [ITM_TCR.SWOENA](#) is set to 1 and [TPIU_SPPR.TXMODE](#) is either 0b01 or 0b10 a trace port clock signal from the TPIU drives the timestamp counter, through the configurable prescaler [TPIU_ACPR](#).

*Applies to an implementation of the architecture **Armv8.0-M** onward. The extension requirements are - [TPIU](#) && [ITM](#).*

See also:

[TPIU_FFCR](#), *Formatter and Flush Control Register*.

[B14.1 Instrumentation Trace Macrocell](#).

[B14.3 Embedded Trace Macrocell](#).

[Chapter B1 Resets](#).

B14.5 Flash Patch and Breakpoint unit

B14.5.1 About the FPB unit

R _{FTWL}	The <i>Flash Patch and Breakpoint</i> (FPB) unit supports setting breakpoints on instruction fetches. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FPB.</i>
I _{BPFS}	The name Flash Patch and Breakpoint unit is historical and the architecture does not support remapping functionality. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FPB.</i>
R _{GDWW}	The number of implemented instruction address comparators is IMPLEMENTATION DEFINED. Software can discover the number of implemented instruction address comparators from FP_CTRL.NUM_CODE . <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FPB.</i>

See also:

[Chapter B8 The System Address Map.](#)

[B14.2.7 DWT trace restrictions and relaxations.](#)

[Part D Register and Payload Specification.](#)

B14.5.2 FPB unit operation

R _{RKFD}	The FPB contains the following register types: • A general control register, FP_CTRL. • Comparator registers. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FPB.</i>
R _{BKKW}	Each implemented instruction address comparator supports breakpoint generation. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FPB.</i>
R _{FNQF}	The FP_CTRL register provides a global enable bit for the FPB, and ID fields that indicate the numbers of instruction address comparison and literal comparison registers implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FPB.</i>
R _{CKBL}	When configured for breakpoint generation, instruction address comparators can be configured to match any halfword-aligned addresses in the whole system address map. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FPB.</i>
R _{XPXS}	Instruction address comparators match only on instruction fetches. The FPB treats hardware accesses to the stack as data accesses for registers that are: • Pushed to the stack by hardware as part of an exception entry or lazy Floating-point state preservation. • Popped from the stack by hardware as part of an exception return. • Pushed to the stack by hardware as part of a Non-secure function return. • Popped from the stack by hardware as part of a Non-secure function call. It is IMPLEMENTATION DEFINED whether the FPB treats a fetch from the exception vector table as part of an exception entry as a data access, or ignores these accesses, for the purposes of FPB address comparator matches. The fetch is never be treated as an instruction fetch. The FPB does not match accesses from the DAP.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FPB**.*

I_{CNBW} Bit[0] of each instruction fetch address is always 0.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FPB**.*

R_{CJJK} When an Instruction address matching comparator is configured for breakpoint generation, a match on the address of a 32-bit instruction is configured to match the first halfword or both halfwords of the instruction.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FPB**.*

R_{WSXN} If a Breakpoint debug event is generated by the FPB on the second halfword of a 32-bit T32 instruction, it is UNPREDICTABLE whether the breakpoint generates a debug event.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FPB**.*

R_{KPPY} An FPB match specifying a Breakpoint debug event generates a Breakpoint debug event that halts the PE if all of the following conditions are true:

- `HaltingDebugAllowed()` returns TRUE.
- `DHCSR.C_DEBUGEN == 1`.
- `DHCSR.S_HALT == 0`.
- The matching instruction is executed in Non-secure state, or `DHCSR.S_SDE == 1`.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FPB**.*

R_{JNVD} If the UDE is implemented an FPB match specifying a Breakpoint debug event generates a Breakpoint debug event that halts the PE in unprivileged mode if either of the following conditions are true:

- `HaltingDebugAllowed()` returns TRUE
- `UnprivHaltingDebugAllowed()` returns TRUE.

and the all of the following conditions are true:

- `DHCSR.S_HALT == 0`.
- The Security Extension is not implemented, the matching instruction is executed in Non-secure state, or `UnprivHaltingDebugAllowed(TRUE)` returns TRUE.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **FPB** && **UDE**.*

R_{HXMP} An FPB match specifying a Breakpoint debug event generates a DebugMonitor exception if it does not halt the PE and all of the following conditions are true:

- `DEMCR.MON_EN == 1`.
- `DHCSR.S_HALT == 0`.
- The DebugMonitor exception group priority is greater than the current execution priority.
- The Security Extension is not implemented, the matching instruction is executed in Non-secure state, or `DEMCR.SDME == 1`.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FPB**.*

R_{LTML} An FPB match specifying a Breakpoint debug event generates a DebugMonitor exception if it does not halt the PE and either of the following conditions are true:

- `DEMCR.MON_EN == 1`.
- The Unprivileged Debug Extension is implemented, `DEMCR.UMON_EN == 1` and the PE is executing in unprivileged mode.

and all of the following conditions apply:

- `DHCSR.S_HALT == 0`.
- The DebugMonitor exception group priority is greater than the current execution priority.
- The Security Extension is not implemented, the matching instruction is executed in Non-secure state, or `DEMCR.SDME == 1`.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **FPB** && **UDE**.*

R_{BFPK} An FPB match that specifies a Breakpoint debug event might be ignored if it does not meet the conditions for generating either:

- A Breakpoint debug event that halts the PE.
- A DebugMonitor exception.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FPB**.*

R_{LQDT} Between a change to the debug authentication interface, **DHCSR**, **DEMCR** or **DAUTHCTRL**, and a following **Context synchronization event**, it is CONSTRAINED UNPREDICTABLE whether breakpoints generated by the FPB:

- Generate a Breakpoint debug event based on the old values and either:
 - If the Main Extension is implemented, generate a DebugMonitor exception.
 - Halts the PE.
- Are escalated to HardFault or **Lockup**.
- Are ignored.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FPB**.*

R_{DZWB} Breakpoints that occur as a result of FPB Unit events behave in the same way as a scalar **BKPT** instruction. All in-flight instructions are completed before halting.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **FPB** && **MVE**.*

I_{YVFX} Entry to debug state and debug monitor is delayed until all in-flight instructions have completed.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **FPB** && **MVE**.*

See also:

[Halting debug.](#)

[B13.4.1 About debug events.](#)

[GenerateDebugEventResponse\(\)](#)

[InstructionExecute\(\)](#)

Applies to an implementation of the architecture Armv8.1-M onward.

B14.5.3 Cache maintenance

R_{BWSW} Instruction caches are not permitted to cache breakpoints that are generated by a Flash Patch and Breakpoint unit.

*Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - **FPB**.*

Chapter B15

The Performance Monitors Extension

This chapter specifies the optional Armv8.1-M Performance Monitors Extension. It contains the following sections:

[B15.1 Counters.](#)

[B15.2 Accuracy of the performance counters.](#)

[B15.3 Security, access, and modes.](#)

[B15.4 Attributability.](#)

[B15.5 Coexistence with the DWT Performance Monitors.](#)

[B15.6 Interrupts and Debug events.](#)

[B15.7 Performance Monitors and Debug state](#)

[B15.8 List of supported architectural and microarchitectural events.](#)

[B15.9 Generic architectural and microarchitectural events.](#)

[B15.10 Common event descriptions.](#)

[B15.11 Required PMU events.](#)

[B15.12 IMPLEMENTATION DEFINED event numbers.](#)

Applies to an implementation of the architecture Armv8.1-M onward.

B15.1 Counters

I_WBLR	The Performance Monitors Extension describes an optional non-invasive debug component that allows performance events to be identified and counted. An implementation of the Performance Monitors Extension is known as the Performance Monitoring Unit (PMU). <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_XVBC	There is space for a maximum of 31 IMPLEMENTATION DEFINED event counters in the PMU. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_KXXN	Each performance event counter is a 16-bit general-purpose counter. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
I_NNBX	By chaining counters in pairs, the counter range can be increased by halving the number of counters. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_VVVF	If the PMU is implemented, a minimum of two 16-bit performance event counters are required, and one 32-bit cycle counter. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
I_JFVV	Each performance event counter can be configured to count any of the performance events that might be supported by an implementation. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_SLDG	It is IMPLEMENTATION DEFINED whether PMU_EVCNTRn supports generating an interrupt. If the interrupt is not supported PMU_INTENSET.Pn is RAZ/WI. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_MGPL	Each performance event counter can be configured to increment on each occurrence of a specified performance event. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_VXPJ	The dedicated 32-bit cycle counter is hard-wired to count cycles. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.</i>
R_YVZG	In case of a counter chain event, the architecture guarantees that, if PMU_CTRL.FZO is zero, the unsigned overflow of the lower half of the counter and subsequent increment of the upper half of the counter are counted within the same cycle. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_CLBS	In case of a counter chain event, it is UNPREDICTABLE whether, if PMU_CTRL.FZO is one, the upper half of the counter increments after an unsigned overflow of the lower half of the counter. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_RVJT	If a counter is configured to a performance event that is not supported on a specific implementation, the counter value is UNKNOWN. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>

B15.2 Accuracy of the performance counters

I_{RYDQ}

The Performance Monitors provide broadly accurate and statistically useful count information. To keep the implementation and validation costs low, a reasonable degree of inaccuracy in the counts is acceptable.

Arm does not define a reasonable degree of inaccuracy, but recommends the following guidelines:

- Under normal operating conditions, the counters present an accurate value of the count.
- In exceptional circumstances, such as a change in Security state or other boundary condition, it is acceptable for the count to be inaccurate.
- Under very unusual, non-repeating, pathological cases, the counts can be inaccurate. These cases are likely to occur as a result of asynchronous exceptions, such as interrupts, where the chance of a systematic error in the count is very unlikely.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

R_{XCDG}

A reasonable degree of inaccuracy in the PMU is permitted, if this does not create systematic inaccuracies in normal operating conditions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

I_{JZGS}

In normal operating conditions:

- Top level counters for a category of PMU events will reflect the orthogonal items in their sub-categories, when the sub-categories provide complete coverage. For example, a MEM_ACCESS event will be the sum of events which count cache accesses.
- Where the sub-categories do not provide comprehensive coverage, the top level counters will count greater than or equal to the sum of the sub-categories.
- Architectural events will be consistently applied but allowance should be made for edge conditions when enabling or disabling counters.
- Arm recommends that barrier instructions are used when enabling or disabling the PMU or individual counters.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

See also:

[B7.2.9 Memory barriers](#).

B15.3 Security, access, and modes

I _{NPZL}	The access to the PMU using the DAP mirrors that of the DWT. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R _{DVWK}	The counters do not increment when: • The PE is in Secure state and privileged mode and <code>SecureNoninvasiveDebugAllowed(TRUE) == FALSE</code>. • The PE is in Secure state and unprivileged mode and <code>SecureNoninvasiveDebugAllowed(FALSE) == FALSE</code>. • The PE is in Non-secure state and privileged mode and <code>NoninvasiveDebugAllowed(TRUE) == FALSE</code>. • The PE is in Non-secure state and unprivileged mode and <code>NoninvasiveDebugAllowed(FALSE) == FALSE</code>. • The PE is in Debug state. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU. Note, Secure state requires S.</i>
R _{KVPL}	It is IMPLEMENTATION DEFINED whether PMU event counters increment, or retain their previous values in low-power state. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R _{NNSQ}	In lookup, it is UNKNOWN whether the counters increment. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R _{ZHBY}	If PMU_CTRL.DP is zero, the PMU cycle counter increments regardless of the Security state of the PE. Otherwise, the PMU cycle counter incrementation is disabled when the PE is in Secure state. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU && S.</i>

See also:

[B14.2 Data Watchpoint and Trace unit](#).

[B13.5 Debug state](#).

B15.4 Attributability

R_{GBQN} An event that is caused by the PE that is counting the event is [Attributable](#). If an agent other than the PE that is counting the events causes an event, these events are [Unattributable](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

R_{XXLL} All architecturally defined events are [Attributable](#), unless otherwise stated in the PMU event list.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

R_{XGRG} Events caused by the execution of an instruction are [Attributable](#) to the Security state and privilege mode the instruction was executed in.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [S](#).

R_{LDGQ} Events are not counted if any of the following are true:

- The event is [Attributable](#) to Secure state and privileged mode and `SecureNoninvasiveDebugEnabled(TRUE) == FALSE`.
- The event is [Attributable](#) to Secure state and unprivileged mode and `SecureNoninvasiveDebugEnabled(FALSE) == FALSE`.
- The event is [Attributable](#) to Non-secure state and privileged mode and `NoninvasiveDebugEnabled(TRUE) == FALSE`.
- The event is [Attributable](#) to Non-secure state and unprivileged mode and `NoninvasiveDebugEnabled(FALSE) == FALSE`.

The event is [Attributable](#) to Secure state are not counted if `SecureNoninvasiveDebugEnabled() == FALSE`.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

R_{WJDJ} [Attributable](#) Events caused by the following are [Attributable](#) to the NS-Req and privilege for the access:

- Vector fetches.
- The hardware stacking or unstacking of registers on any of:
 - Exception entry.
 - Exception exit.
 - Function call entry.
 - Function return.
 - Lazy state preservation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#). Note, Secure state requires [S](#).

R_{VHLM} For each [Unattributable](#) event it is IMPLEMENTATION DEFINED whether the [Unattributable](#) event is counted when counting [Attributable](#) events is prohibited.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

See also:

[B15.10 Common event descriptions](#).

B15.5 Coexistence with the DWT Performance Monitors

R_{NKCB} The PMU cycle counter **PMU_CCNTR** is an alias of the **DWT_CYCCNT** register. All derived PMU functions are available whenever the PMU enables the cycle counter. If both the PMU and the DWT are implemented, **DWT_CTRL.NOCYCCNT** is RAZ.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PMU** && **DWTD**.*

R_{ZBWD} When **PMU_CTRL.E** == 1:

- A read of **DWT_CPICNT**, **DWT_EXCCNT**, **DWT_FOLDCNT**, **DWT_LSUCNT**, and **DWT_SLEEPCNT** return an UNKNOWN value.
- **DWT_CPICNT**, **DWT_EXCCNT**, **DWT_FOLDCNT**, **DWT_LSUCNT**, and **DWT_SLEEPCNT** might increment at random.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PMU** && **DWTT**.*

R_{KDKT} When any of **DWT_CTRL.FOLDEVTENA**, **LSUEVTENA**, **SLEEPEVTENA**, **EXCEVTENA**, **CPIEVTENA** == 1:

- A read of any **PMU_EVCNTRn** counters returns an UNKNOWN value.
- A read of the PMU overflow flags in **PMU_OVSSET** and **PMU_OVSCLR** return UNKNOWN values.
- The **PMU_EVCNTRn** counters might increment at random.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PMU** && **DWTT**.*

R_{NDFK} When **PMU_CTRL.E** == 1 and any of **DWT_CTRL.CYCEVTENA**, **FOLDEVTENA**, **LSUEVTENA**, **SLEEPEVTENA**, **EXCEVTENA**, **CPIEVTENA** == 1:

- The generation of Event Counter packets by the DWT is UNPREDICTABLE.
- If any of the **PMU_INTENSET.Pn** flags are set to 1, the generation of interrupts is UNPREDICTABLE.
- If **PMU_CTRL.TRO** == 1 the generation of Event Counter packets by the PMU is UNPREDICTABLE.
- The PMU and DWT counters might increment at random.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PMU** && **DWTT**.*

R_{KBZG} It is permissible for an implementation to allow the PMU or DWT to overwrite any packets held in the trace buffer on enabling the PMU and disabling the DWT or enabling the DWT and disabling the PMU without requiring software to flush output to the trace sink or TPIU.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PMU** && **DWTT**.*

R_{RGQN} At any time:

- A write to any of **DWT_CPICNT**, **DWT_EXCCNT**, **DWT_FOLDCNT**, **DWT_LSUCNT**, or **DWT_SLEEPCNT**, including the indirect write of 0 when writing 1 to any of **DWT_CTRL.CYCEVTENA**, **FOLDEVTENA**, **LSUEVTENA**, **SLEEPEVTENA**, **EXCEVTENA**, **CPIEVTENA**, sets the **PMU_EVCNTRn** to UNKNOWN values.
- A write of **PMU_EVCNTRn**, including the indirect write of 0 when writing 1 to **PMU_CTRL.P** to 1, sets **DWT_CPICNT**, **DWT_EXCCNT**, **DWT_FOLDCNT**, **DWT_LSUCNT**, and **DWT_SLEEPCNT** to UNKNOWN values.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PMU** && **DWTT**.*

I_{KLJG} It is permissible for PMU and DWT event counters to be aliased. Cycle counters **PMU_CCNTR** and **DWT_CYCCNT** are always aliased.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PMU** && **DWTT**.*

See also:

[B14.4 Trace Port Interface Unit](#)

B15.6 Interrupts and Debug events

I _{RJNK}	Counters can be configured to generate interrupts or debug events on overflow. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R _{ZDMH}	If a counter is configured to generate an interrupt when it overflows, DEMCR.MON_PEND is set to 1 to pend a DebugMonitor exception with DFSR.PMU set to 1 if one of the following applies: • The counter is configured to count events attributable to unprivileged mode and DEMCR.UMON_EN is set to 1. • The counter is configured to count events attributable to privileged mode and DEMCR.MON_EN is set to 1. The associated field in PMU_OVSSET or PMU_OVSCLR indicates which counter triggered the exception. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R _{WGGD}	For each implemented event counter <i>m</i>, and the cycle counter, unsigned overflow of the counter halts the PE if all of the following conditions apply: • PMU_INTENSET_m is set to 1 for the event counter, or PMU_INTENSET.C is set to 1 for the cycle counter. • DHCSR.C_PMOV is set to 1. • CanHaltOnEvent() returns TRUE for the Security state and privilege mode of the event being counted. As a result of the above: • DHCSR.C_HALT is set to 1. • DFSR.PMU is set to 1. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU && DSPDE && UDE. Note, UDE is required for unprivileged events.</i>
R _{YKXR}	For each implemented event counter <i>m</i>, and the cycle counter, unsigned overflow of the counter pends a DebugMonitor exception if it does not halt the PE and all of the following conditions apply: • PMU_INTENSET_m is set to 1 for the event counter, or PMU_INTENSET.C is set to 1 for the cycle counter. • DEMCR.MON_EN is set to 1 for privileged events. or DEMCR.UMON_EN is set to 1 for unprivileged events. • The PE is in Non-secure state or DEMCR.SDME is set to 1. • The group priority of the DebugMonitor exception is not sufficient to preempt the current execution priority. If the priority of the DebugMonitor exception is not sufficient to preempt execution the PE will set DEMCR.MON_PEND to 1. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
I _{HSYK}	The CTI uses the PMU as an event source. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i> See also: PMU_HandleOverflow(). Chapter B13 Debug

B15.7 Performance Monitors and Debug state

R _{JSTB}	Events that count cycles are not counted in Debug state. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R _{WTSQ}	Events Attributable to the operations issued by the debugger through the external debug interface are not counted in Debug state. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R _{HFRP}	For each Unattributable event, it is IMPLEMENTATION DEFINED whether it is counted when the PE is in Debug state. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>

B15.8 List of supported architectural and microarchitectural events

I_CFXV	Arm recommends the use of implementation-specific events based on performance behaviors of the underlying microarchitecture. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_GNJH	Events 0x0000-0xBFFF are either defined or reserved for future common events. Implementations can implement any of these events. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
R_TPLX	Implementations can define additional events that are specific to the implementation outside the specified reserved space. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
I_FHHC	The list of common events for the PMU is as follows. The Event types are: Arch Architectural event. These events are the same across all implementations. uArch Microarchitectural event. These events might vary across different implementations. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.</i>
I_PTHP	In this section: Architecturally executed Instruction architecturally executed is a class of event that counts for each instruction of the specified type. Architecturally executed means that the program flow is of the type that the counted instruction would be executed in a Simple sequential execution of the program. Therefore an instruction that has been executed and retired is defined to be architecturally executed even if the PE discards the results of the speculative execution. If an instruction that would be executed in a Simple sequential execution of the program that generates a synchronous exception, it is IMPLEMENTATION DEFINED whether the instruction is counted. Each architecturally executed instruction is counted once, even if the implementation splits the instruction into multiple operations. Instructions that have no visible effect on the architectural state of the PE are architecturally executed if they form part of the architecturally executed program flow. The point where such instructions are retired is IMPLEMENTATION DEFINED. Examples of instructions that have no visible effects are: • A NOP. • A conditional instruction that fails its Condition code check. • A Compare and Branch on Zero, CBZ, instruction that does not branch. • A Compare and Branch on Nonzero, CBNZ, instruction that does not branch. The point at which an event causes an event counter to be updated is not defined. Instruction architecturally executed, Condition code check pass Instruction architecturally executed, Condition code check pass is a class of events that explicitly do not occur for: • A conditional instruction that fails its Condition code check. • A Compare and Branch on Zero, CBZ, instruction that does not branch. • A Compare and Branch on Nonzero, CBNZ, instruction that does not branch. • A Store-Exclusive instruction that does not write to memory. Otherwise the definition of architecturally executed is the same as Instruction architecturally executed. A branch that is architecturally executed, with condition code check pass is also described as a branch taken.

Instruction memory access A PE acquires instructions for execution through instruction fetches. Instruction fetches might be due to:

- Fetching instructions that are architecturally executed.
- The result of the execution of a `PLI` instruction.
- Speculation that a particular instruction might be executed in the future.

The relationship between the fetch of an individual instruction and an instruction memory access is IMPLEMENTATION DEFINED. For example, an implementation might fetch many instructions including a non-integer number of instructions in a single instruction memory access.

Memory-read operations A PE accesses memory through memory-read operations and memory-write operations. A memory-read operation might be due to:

- The result of an architecturally executed memory-reading instruction.
- The result of a Speculatively executed memory-reading instruction.

For levels of cache hierarchy beyond the Level 1 caches, memory-read operations also include accesses made as part of a refill of another cache closer to the PE. Such refills might be due to any of the items in the following, non-exhaustive, list:

- Memory-read operations or Memory-write operations that miss in the cache.
- The execution of a preload data instruction.
- The execution of an instruction preload instruction on a unified cache.
- The execution of a cache maintenance operation. A preload instruction or cache maintenance operation is not, in itself, an access to that cache. However, as preload instruction or cache maintenance operation might generate cache refills which are then treated as memory-read operations beyond that cache.
- Speculation that a future instruction might access that memory location.
- Instruction memory accesses.

The relationship between memory-read instructions and memory-read operations is IMPLEMENTATION DEFINED.

Memory-write operations Memory-write operations might be due to:

- The result of an architecturally executed memory-writing instruction.
- The result of a Speculatively executed memory-writing instruction.

Speculatively executed memory-writing instructions that do not become architecturally executed must not alter the architecturally defined view of memory. They can, however, generate a memory-write operation that is later undone in some implementation-specific way.

For levels of cache hierarchy beyond the Level 1 caches, memory-write operations also include accesses made as part of a write-back from another cache closer to the PE. A write-back of this type might be due to any of the following non-exhaustive list:

- Evicting a dirty line from the cache, to allocate a cache line for a cache refill.
- The execution of a cache maintenance operation. A cache maintenance operation is not in itself an access to that cache. However, a cache maintenance operation might generate write-backs which are then treated as memory-write operations beyond that cache.
- The result of a coherency request from another PE.

The relationship between memory-writing instructions and memory-write operations is IMPLEMENTATION DEFINED.

The data written back from a cache that is shared with other PEs might not be the data that was written by the PE that performs the operation that leads to the write-back. Nevertheless, the event is counted as a write-back event for that PE.

Microarchitectural operation It is permissible for an implementation of a PE to break down instructions into separate, smaller, operations. The use of Microarchitectural operations (micro-ops) is IMPLEMENTATION DEFINED. An instruction might create one or more micro-ops at any point in the execution pipeline. For the

purpose of event counting, the micro-ops are counted. The definition of a micro-op is implementation-specific. An architecture instruction might create more than one micro-op for each instruction. Micro-ops might also be removed or merged in the execution stream, so an architecture instruction might create no micro-ops for an instruction. Any arbitrary translation of instructions to an equivalent sequence of micro-ops is permitted. The PE must strictly follow the architecturally visible behavior no matter the underlying division of an instruction into micro-ops.

The counting of operations can indicate the workload on the PE. However, there is no requirement for operations to represent similar amounts of work, and direct comparisons between different microarchitectures are not meaningful.

For example, an implementation might split an [LDM](#) instruction of six registers into six micro-ops, one for each load, and a seventh address-generation operation to determine the base address or write-back address. Therefore an instruction might be split into multiple countable events.

Operations speculatively executed There is no architecturally guaranteed relationship between a Speculatively executed micro-op and an architecturally executed instruction. The results of such an operation can also be discarded, if it transpires that the operation was not required, following a mispredicted branch. Therefore, Armv8-M defines these events as operations speculatively executed, where appropriate.

Slot An implementation of a PE might be able to execute multiple micro-ops in a single processor cycle. The maximum number of micro-ops that can be executed might vary at different points in the execution pipeline.

To allow profiling of the utilization of the resource of the PE, an implementation-specific point in the execution pipeline is chosen where the maximum number of micro-ops that can be executed is an IMPLEMENTATION DEFINED fixed value.

Software change of the PC Some events relate to instructions that cause a software change of the [PC](#). This includes all:

- Branch instructions.
- Memory-reading instructions that explicitly write to the [PC](#).
- Data-processing instructions that explicitly write to the [PC](#).
- Exception return instructions.

It is IMPLEMENTATION DEFINED whether any or all of the following are treated as software changes to the [PC](#):

- A [BKPT](#).
- An UNDEFINSTR Fault.
- Context synchronization barrier [ISB](#) instructions.

Speculatively executed Many events relate to speculatively executed operations. Here, speculatively executed means the PE did some work associated with one or more instructions but the instructions were not necessarily architecturally executed.

The definition of speculatively executed does not mean only those operations that are executed speculatively and later abandoned, for example due to a branch misprediction or fault. That is, speculatively executed operations must count operations on both false and correct execution paths.

Different groups of events can have different IMPLEMENTATION DEFINED definitions of speculatively executed. Such groups share a common base type, which the event name denotes. Each of the events in the previous example is of the base type, operation speculatively executed. For groups of events with a common base type, speculatively executed operations are all counted on the same basis, which normally means at the same point in the pipeline. It is possible to compare the counts and make meaningful observations about the program being profiled.

Within these groups, events are commonly defined with reference to a particular architecture instruction or group of instructions. In the case of speculatively executed operations this means operations with semantics that map to that type of instruction.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

Chapter B15. The Performance Monitors Extension
B15.8. List of supported architectural and microarchitectural events

Event number	Event type	Event mnemonic	Description
0x0000	Arch	SW_INCR	Instruction Architecturally executed , condition code check pass, software increment
0x0001	uArch	L1I_CACHE_REFILL	Attributable Level 1 instruction cache refill
0x0003	uArch	L1D_CACHE_REFILL	Attributable Level 1 data cache refill
0x0004	uArch	L1D_CACHE	Attributable Level 1 data cache access
0x0006	Arch	LD_RETIRE	Instruction Architecturally executed , condition code check pass, load
0x0007	Arch	ST_RETIRE	Instruction Architecturally executed , condition code check pass, store
0x0008	Arch	INST_RETIRE	Instruction Architecturally executed
0x0009	Arch	EXC_TAKEN	Exception taken
0x000A	Arch	EXC_RETURN	Instruction Architecturally executed , condition code check pass, exception return
0x000C	Arch	PC_WRITE_RETIRE	Instruction Architecturally executed , condition code check pass, software change of the PC
0x000D	Arch	BR_IMMED_RETIRE	Instruction Architecturally executed , immediate branch
0x000E	Arch	BR_RETURN_RETIRE	Instruction Architecturally executed , condition code check pass, procedure return
0x000F	Arch	UNALIGNED_LDST_RETIRE	Instruction Architecturally executed , condition code check pass, unaligned load or store
0x0010	uArch	BR_MIS_PRED	Mispredicted or not predicted branch speculatively executed
0x0011	uArch	CPU_CYCLES	Cycle
0x0012	uArch	BR_PRED	Predictable branch speculatively executed
0x0013	uArch	MEM_ACCESS	Data memory access
0x0014	uArch	L1I_CACHE	Attributable Level 1 instruction cache access
0x0015	uArch	L1D_CACHE_WB	Attributable Level 1 data cache write-back

Event number	Event type	Event mnemonic	Description
0x0016	uArch	L2D_CACHE	Attributable Level 2 data cache access
0x0017	uArch	L2D_CACHE_REFILL	Attributable Level 2 data cache refill
0x0018	uArch	L2D_CACHE_WB	Attributable Level 2 data cache write-back
0x0019	uArch	BUS_ACCESS	Attributable Bus access
0x001A	uArch	MEMORY_ERROR	Local memory error
0x001B	uArch	INST_SPEC	Operation speculatively executed
0x001D	uArch	BUS_CYCLES	Bus cycle
0x001E	Arch	CHAIN	For an odd-numbered counter, increments when an overflow occurs on the preceding even-numbered counter on the same PE
0x001F	uArch	L1D_CACHE_ALLOCATE	Attributable Level 1 data cache allocation without refill
0x0020	uArch	L2D_CACHE_ALLOCATE	Attributable Level 2 data cache allocation without refill
0x0021	Arch	BR_RETIRED	Instruction Architecturally executed , branch
0x0022	uArch	BR_MIS_PRED_RETIRED	Instruction Architecturally executed , mispredicted branch
0x0023	uArch	STALL_FRONTEND	No operation issued due to the frontend
0x0024	uArch	STALL_BACKEND	No operation issued due to the backend
0x0027	uArch	L2I_CACHE	Attributable Level 2 instruction cache access
0x0028	uArch	L2I_CACHE_REFILL	Attributable Level 2 instruction cache refill
0x0029	uArch	L3D_CACHE_ALLOCATE	Attributable Level 3 data cache allocation without refill
0x002A	uArch	L3D_CACHE_REFILL	Attributable Level 3 data cache refill
0x002B	uArch	L3D_CACHE	Attributable Level 3 data cache access
0x002C	uArch	L3D_CACHE_WB	Attributable Level 3 data cache write-back
0x0036	uArch	LL_CACHE_RD	Last level data cache read
0x0037	uArch	LL_CACHE_MISS_RD	Last level data cache read miss
0x0039	uArch	L1D_CACHE_LMISS_RD	Level 1 data cache long-latency read miss
0x003A	uArch	OP_RETIRED	Micro-operation architecturally

Event number	Event type	Event mnemonic	Description
			executed
0x003B	uArch	OP_SPEC	Micro-operation speculatively executed
0x003C	uArch	STALL	No operation sent for execution
0x003D	uArch	STALL_SLOT_BACKEND	No operation sent for execution on a slot due to the backend
0x003E	uArch	STALL_SLOT_FRONTEND	No operation sent for execution on a slot due to the frontend
0x003F	uArch	STALL_SLOT	No operation sent for execution on a slot
0x0040	uArch	L1D_CACHE_RD	Level 1 data cache read
0x0100	uArch	LE_RETIRED	Loop end instruction architecturally executed, entry registered in the LO_BRANCH_INFO cache
0x0101	uArch	LE_SPEC	Loop end instruction speculatively executed, entry registered in the LO_BRANCH_INFO cache
0x0104	uArch	BF_RETIRED	Branch future instruction Architecturally executed , condition code check pass, and registers an entry in the LO_BRANCH_INFO cache
0x0105	uArch	BF_SPEC	Branch future instruction speculatively executed, condition code check pass, and registers an entry in the LO_BRANCH_INFO cache
0x0108	uArch	LE_CANCEL	LO_BRANCH_INFO cache containing a valid loop entry cleared while not in the last iteration of the loop
0x0109	uArch	BF_CANCEL	LO_BRANCH_INFO cache containing a valid BF entry cleared and associated branch not taken
0x0114	Arch	SE_CALL_S	Call to secure function, resulting in security state change
0x0115	Arch	SE_CALL_NS	Call to non-secure function, resulting in security state change
0x0118	Arch	DWT_CMPMATCH0	DWT comparator 0 match
0x0119	Arch	DWT_CMPMATCH1	DWT comparator 1 match
0x011A	Arch	DWT_CMPMATCH2	DWT comparator 2 match

Chapter B15. The Performance Monitors Extension
B15.8. List of supported architectural and microarchitectural events

Event number	Event type	Event mnemonic	Description
0x011B	Arch	DWT_CMPMATCH3	DWT comparator 3 match
0x011C	Arch	DWT_CMPMATCH4	DWT comparator 4 match
0x011D	Arch	DWT_CMPMATCH5	DWT comparator 5 match
0x011E	Arch	DWT_CMPMATCH6	DWT comparator 6 match
0x011F	Arch	DWT_CMPMATCH7	DWT comparator 7 match
0x0130	uArch	PAC_RETIRED	PAC instruction architecturally executed
0x0131	uArch	PAC_SPEC	PAC instruction speculatively executed
0x0134	uArch	AUT_RETIRED	AUT instruction architecturally executed
0x0135	uArch	AUT_SPEC	AUT instruction speculatively executed
0x013C	uArch	STALL_BACKEND_PAC	Stall cycles caused by PAC operations
0x0200	Arch	MVE_INST_RETIRED	MVE instruction architecturally executed
0x0201	uArch	MVE_INST_SPEC	MVE instruction speculatively executed
0x0204	Arch	MVE_FP_RETIRED	MVE floating-point instruction Architecturally executed
0x0205	uArch	MVE_FP_SPEC	MVE floating-point instruction speculatively executed
0x0208	Arch	MVE_FP_HP_RETIRED	MVE half-precision floating-point instruction Architecturally executed
0x0209	uArch	MVE_FP_HP_SPEC	MVE half-precision floating-point instruction speculatively executed
0x020C	Arch	MVE_FP_SP_RETIRED	MVE single-precision floating-point instruction Architecturally executed
0x020D	uArch	MVE_FP_SP_SPEC	MVE single-precision floating-point instruction speculatively executed
0x0214	Arch	MVE_FP_MAC_RETIRED	MVE floating-point multiply or multiply-accumulate instruction Architecturally executed
0x0215	uArch	MVE_FP_MAC_SPEC	MVE floating-point multiply or multiply-accumulate instruction speculatively executed
0x0224	Arch	MVE_INT_RETIRED	MVE integer instruction Architecturally executed
0x0225	uArch	MVE_INT_SPEC	MVE integer instruction speculatively executed
0x0228	Arch	MVE_INT_MAC_RETIRED	MVE integer multiply or multiply-

Event number	Event type	Event mnemonic	Description
			accumulate instruction architecturally executed
0x0229	uArch	MVE_INT_MAC_SPEC	MVE integer multiply or multiply-accumulate instruction speculatively executed
0x0238	Arch	MVE_LDST_RETIRED	MVE load or store instruction Architecturally executed
0x0239	uArch	MVE_LDST_SPEC	MVE load or store instruction speculatively executed
0x023C	Arch	MVE_LD_RETIRED	MVE load instruction architecturally executed
0x023D	uArch	MVE_LD_SPEC	MVE load instruction speculatively executed
0x0240	Arch	MVE_ST_RETIRED	MVE store instruction architecturally executed
0x0241	uArch	MVE_ST_SPEC	MVE store instruction speculatively executed
0x0244	uArch	MVE_LDST_CONTIG_RETIRED	MVE contiguous load or store instruction Architecturally executed
0x0245	uArch	MVE_LDST_CONTIG_SPEC	MVE contiguous load or store instruction speculatively executed
0x0248	uArch	MVE_LD_CONTIG_RETIRED	MVE contiguous load instruction Architecturally executed
0x0249	uArch	MVE_LD_CONTIG_SPEC	MVE contiguous load instruction speculatively executed
0x024C	uArch	MVE_ST_CONTIG_RETIRED	MVE contiguous store instruction Architecturally executed
0x024D	uArch	MVE_ST_CONTIG_SPEC	MVE contiguous store instruction speculatively executed
0x0250	uArch	MVE_LDST_NONCONTIG_RETIRED	MVE non-contiguous load or store instruction Architecturally executed
0x0251	uArch	MVE_LDST_NONCONTIG_SPEC	MVE non-contiguous load or store instruction speculatively executed
0x0254	uArch	MVE_LD_NONCONTIG_RETIRED	MVE non-contiguous load instruction Architecturally executed
0x0255	uArch	MVE_LD_NONCONTIG_SPEC	MVE non-contiguous load instruction speculatively executed
0x0258	uArch	MVE_ST_NONCONTIG_RETIRED	MVE non-contiguous store instruction

Event number	Event type	Event mnemonic	Description
			Architecturally executed
0x0259	uArch	MVE_ST_NONCONTIG_SPEC	MVE non-contiguous store instruction speculatively executed
0x025C	Arch	MVE_LDST_MULTI_RETIRED	MVE memory instruction targeting multiple registers architecturally executed
0x025D	uArch	MVE_LDST_MULTI_SPEC	MVE memory instruction targeting multiple registers speculatively executed
0x0260	Arch	MVE_LD_MULTI_RETIRED	MVE memory load instruction targeting multiple registers architecturally executed
0x0261	uArch	MVE_LD_MULTI_SPEC	MVE memory load instruction targeting multiple registers speculatively executed
0x0264	Arch	MVE_ST_MULTI_RETIRED	MVE memory store instruction targeting multiple registers architecturally executed
0x0265	uArch	MVE_ST_MULTI_SPEC	MVE memory store instruction targeting multiple registers speculatively executed
0x028C	uArch	MVE_LDST_UNALIGNED_RETIRED	MVE unaligned memory load or store instruction Architecturally executed
0x028D	uArch	MVE_LDST_UNALIGNED_SPEC	MVE unaligned memory load or store instruction speculatively executed
0x0290	uArch	MVE_LD_UNALIGNED_RETIRED	MVE unaligned load instruction Architecturally executed
0x0291	uArch	MVE_LD_UNALIGNED_SPEC	MVE unaligned load instruction speculatively executed
0x0294	uArch	MVE_ST_UNALIGNED_RETIRED	MVE unaligned store instruction Architecturally executed
0x0295	uArch	MVE_ST_UNALIGNED_SPEC	MVE unaligned store instruction speculatively executed
0x0298	uArch	MVE_LDST_UNALIGNED_NONCONTIG_RETIRED	MVE unaligned non-contiguous load or store instruction architecturally executed
0x0299	uArch	MVE_LDST_UNALIGNED_NONCONTIG_SPEC	

Event number	Event type	Event mnemonic	Description
			MVE unaligned non-contiguous load or store instruction speculatively executed
0x02A0	Arch	MVE_VREDUCE_RETIRE	MVE vector reduction instruction Architecturally executed
0x02A1	uArch	MVE_VREDUCE_SPEC	MVE vector reduction instruction speculatively executed
0x02A4	Arch	MVE_VREDUCE_FP_RETIRE	MVE floating-point vector reduction instruction Architecturally executed
0x02A5	uArch	MVE_VREDUCE_FP_SPEC	MVE floating-point vector reduction instruction speculatively executed
0x02A8	Arch	MVE_VREDUCE_INT_RETIRE	MVE integer vector reduction instruction Architecturally executed
0x02A9	uArch	MVE_VREDUCE_INT_SPEC	MVE integer vector reduction instruction speculatively executed
0x02B8	uArch	MVE_PRED	Cycles where one or more predicated beats Architecturally executed
0x02CC	uArch	MVE_STALL	Stall cycles caused by an MVE instruction
0x02CD	uArch	MVE_STALL_RESOURCE	Stall cycles caused by an MVE instruction due to resource conflicts
0x02CE	uArch	MVE_STALL_RESOURCE_MEM	Stall cycles caused by an MVE instruction due to memory resource conflicts
0x02CF	uArch	MVE_STALL_RESOURCE_FP	Stall cycles caused by an MVE instruction due to floating-point resource conflicts
0x02D0	uArch	MVE_STALL_RESOURCE_INT	Stall cycles caused by an MVE instruction due to integer resource conflicts
0x02D3	uArch	MVE_STALL_BREAK	Stall cycles caused by an MVE chain break
0x02D4	uArch	MVE_STALL_DEPENDENCY	Stall cycles caused by MVE register dependency
0x4007	uArch	ITCM_ACCESS	Instruction TCM access
0x4008	uArch	DTCM_ACCESS	Data TCM access
0x4010	uArch	TRCEXTOUT0	ETM external output 0
0x4011	uArch	TRCEXTOUT1	ETM external output 1

Event number	Event type	Event mnemonic	Description
0x4012	uArch	TRCEXTOUT2	ETM external output 2
0x4013	uArch	TRCEXTOUT3	ETM external output 3
0x4018	uArch	CTI_TRIGOUT4	Cross-trigger Interface output trigger 4
0x4019	uArch	CTI_TRIGOUT5	Cross-trigger Interface output trigger 5
0x401A	uArch	CTI_TRIGOUT6	Cross-trigger Interface output trigger 6
0x401B	uArch	CTI_TRIGOUT7	Cross-trigger Interface output trigger 7

Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - **PMU**.

B15.9 Generic architectural and microarchitectural events

This section provides descriptions that apply to multiple events.

B15.9.1 CTI_TRIGOUT (Cross-trigger Interface output trigger , for = 4 to 7)

The counter counts for each event from CTI output trigger . Note: CTI output triggers are input events to the PMU and PE Trace Unit.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

B15.9.2 TRCEXTOUT (PE Trace Unit external output , for = 0 to 3)

The counter counts for each event signaled by the ETM external event .

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

B15.9.3 DWT_CMPMATCH (DWT comparator match, for = 0 to 7)

The counter increments for each successful comparator match indicated by DWT comparator . The comparator does not increment if the comparator is not implemented or is disabled

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

B15.9.4 LI_CACHE_REFILL (Level instruction cache refill)

The counter counts each access that is counted by [L<n>I_CACHE](#) that returns instructions from beyond the Level instruction cache. Beyond in this context means a Level cache, where m>n, or memory.

The event indicates to software that the access missed in the Level instruction cache and might have a significant performance impact because of the additional latency, compared to an access that hits in the Level instruction cache.

The definition of Level cache is IMPLEMENTATION DEFINED, and does not necessarily correspond to the [cache levels](#) that are defined by the CLIDR mechanism. Instead, increasing values of correspond to increasing average additional latency.

The counter does not count:

- Accesses where the miss does not have a significant impact on performance.
- A miss that does not cause a new cache refill but is satisfied from a previous miss.

If the cache is shared, only accesses that are [Attributable](#) to this PE are counted.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

B15.9.5 LD_CACHE_REFILL (Level data cache refill)

The counter counts each access that is counted by [L<n>D_CACHE](#) that returns data from beyond the Level data cache. Beyond in this context means a Level cache, where m>n, or memory.

Each access to a [cache line](#) that causes a new linefill is counted, including those from instructions that generate multiple accesses, such as load or store multiples, and [PUSH](#) and [POP](#) instructions. In particular, the counter counts accesses to the Level cache that cause a refill. A refill includes any access that causes data to be fetched from outside the Level 1 to the Level cache, even if the data is ultimately not allocated into the Level cache.

The definition of Level cache is IMPLEMENTATION DEFINED, and does not necessarily correspond to the [cache levels](#) that are defined by the CLIDR mechanism. Instead, increasing values of correspond to increasing average additional latency.

The counter does not count:

- Accesses that do not cause a new Level cache refill but are satisfied by refilling data from a previous miss.
- Accesses to a [cache line](#) that generate a memory access but not a new linefill, such as Write-Through writes that hit in the cache.
- Cache maintenance instructions.
- A write that writes an entire line to the cache and does not fetch any data from outside the Level cache.
- A write that misses in the cache, and writes through the cache without allocating a line.

If the cache is shared, only accesses that are [Attributable](#) to this PE are counted.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

B15.9.6 LD_CACHE_MISS_RD (Level data cache miss on read)

The counter counts each access that is counted by `L<n>D_CACHE_RD` that returns data from beyond the Level data or unified cache. Beyond in this context means a Level cache, where $m > n$, or memory.

The event indicates to software that the access missed in the Level data or unified cache and might have a significant performance impact because of the additional latency, compared to an access that hits in the Level data or unified cache.

The definition of Level cache is IMPLEMENTATION DEFINED, and does not necessarily correspond to the [cache levels](#) that are defined by the CLIDR mechanism. Instead, increasing values of correspond to increasing average additional latency.

The counter does not count:

- Accesses where the miss does not have a significant impact on performance.
- A miss that does not cause a new cache refill but is satisfied from a previous miss.

If the cache is shared, only accesses that are [Attributable](#) to this PE are counted.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

B15.9.7 LD_CACHE_WB (Level data cache write-back)

The counter counts every write-back of data from the Level data or unified cache.

The counter counts each write-back that causes data to be written from the Level cache to outside of the Level cache. For example, the counter counts the following cases:

- A write-back that causes data to be written to a Level<n+1> cache or memory.
- A write-back of a recently fetched [cache line](#) that has not been allocated to the Level cache.
- Transfers of data from the Level cache to outside of this cache that are made as a result of a coherency request. The conditions that determine which of these are counted for transfers to other Level caches within the same multiprocessor cluster are IMPLEMENTATION DEFINED.

Each write-back is counted one time, even if multiple accesses are required to complete the write-back.

Whether write-backs that are made as a result of cache maintenance instructions are counted is IMPLEMENTATION DEFINED. The counter does not count:

- The invalidation of a [cache line](#) without any write-back to a Level<n+1> cache or memory.
- Writes from the PE that write through the Level cache to outside of the Level cache.

An Unattributable write-back event occurs when a requestor outside the PE makes a coherency request that results in write-back. If the cache is shared, then an Unattributable write-back event is not counted. If the cache is not shared, then the event is counted. It is IMPLEMENTATION DEFINED whether a write of a whole [cache line](#) that is not the result of the eviction of a line from a cache, is counted.

If the cache is shared, only accesses that are [Attributable](#) to this PE are counted.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - **PMU**.*

B15.9.8 LI_CACHE (Level instruction cache access)

The counter counts each [Attributable](#) access to at least the Level instruction cache. Each access to other Level instruction memory structures, such as refill buffers, is also counted.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - **PMU**.*

B15.9.9 LD_CACHE (Level data cache access)

The counter counts each [Attributable](#) memory-read or [Attributable](#) memory-write access to at least the Level data or unified cache. Each access to a [cache line](#) is counted, including the multiple accesses of instructions, such as LDM or STM. Each access to other Level data or unified cache memory structures is also counted.

The counter does not count cache maintenance instructions.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - **PMU**.*

B15.9.10 LD_CACHE_RD (Level data cache access, read)

The counter operates the same way as [L<n>D_CACHE](#), with the one exception that the counter counts only memory-read accesses.

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - **PMU**.*

B15.10 Common event descriptions

The common events that can be supported by the PMU counters are defined in this section. For the common features, the counters normally increment only one time for each event. The individual event descriptions include any exceptions to this. In the definitions, the term *Architecturally executed* means that the instruction flow is one where the counted instruction would have been executed in a simple sequential execution model.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

SW_INCR (0x0000, Architectural)

The counter increments on writes to the PMU_SWINC register. If the PE performs two *Architecturally executed* writes to the PMU_SWINC register without an intervening *Context synchronization event*, then the counter is incremented twice.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

L1I_CACHE_REFILL (0x0001, Microarchitectural)

See L<n>I_CACHE_REFILL.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

L1D_CACHE_REFILL (0x0003, Microarchitectural)

See L<n>D_CACHE_REFILL.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

L1D_CACHE (0x0004, Microarchitectural)

See L<n>D_CACHE.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

LD_RETIRED (0x0006, Architectural)

The counter increments for every executed memory-reading instruction.

LD_RETIRED does not count the return status value of a Store-Exclusive instruction. It is IMPLEMENTATION DEFINED whether the following preload instructions count as memory-reading instructions:

- PLD (literal).
- PLD, PLDW (immediate).
- PLI (immediate, literal).
- PLI (register).

Arm recommends that if these instruction are not implemented as NOPs, then they are counted as memory-reading instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

ST_RETIRED (0x0007, Architectural)

The counter increments for every executed memory-writing instruction. The counter does not increment for a Store-Exclusive instruction that fails.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

INST_RETIRED (0x0008, Architectural)

The counter increments for every [Architecturally executed](#) instruction.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

EXC_TAKEN (0x0009, Architectural)

The counter increments on each exception entry. The counter does not further increment in the case of a late or derived exception, but it is incremented when [tail-chaining](#). This corresponds to calls to the [ActivateException\(\)](#) pseudocode function.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

EXC_RETURN (0x000A, Architectural)

The counter increments on each exception return. This occurs when the PE is in Handler mode, and one of the following is executed and loads an [EXC_RETURN](#) value into the PC:

- A [POP](#) or LDM that includes loading the [PC](#).
- An LDR with the [PC](#) as a destination.
- A [BX](#) with any register.

This counter also increments on [tail-chaining](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

PC_WRITE_RETIRED (0x000C, Architectural)

The counter increments for every software change of the [PC](#). This includes all of the following:

- Branch instructions.
- Loop start instructions.
- Loop end instructions.
- Memory-reading instructions that explicitly write to the [PC](#).
- Data-processing instructions that explicitly write to the [PC](#).

If the PACBTI Extension is implemented, the counter counts [Architecturally executed BXAUT](#) instructions that pass their PAC checks. It is IMPLEMENTATION DEFINED whether the counter increments for any or all of:

- [BKPT](#) instructions.
- An exception generated because an instruction is UNDEFINED.
- The exception-generating instructions, [SVC](#), and [UDF](#).

It is IMPLEMENTATION DEFINED whether an [ISB](#) is counted as a software change of the [PC](#). The counter does not increment for exceptions other than those explicitly identified in these lists.

Conditional branches are only counted if the branch is taken.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - PMU.

BR_IMMED_RETIRED (0x000D, Architectural)

The counter counts all immediate branch instructions that are [Architecturally executed](#), which includes the immediate variants of any B branch instruction or CBNZ instruction. Conditional branches are always counted, regardless of whether the branch is taken. If an [ISB](#) is counted as a software change of the PC instruction, then it is IMPLEMENTATION DEFINED whether an [ISB](#) is counted as an immediate branch instruction.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

BR_RETURN_RETIRED (0x000E, Architectural)

The counter counts function return instructions.

For example the following, non-exhaustive, list shows instructions that are counted:

- [BX](#) LR.
- [MOV](#) PC, LR.
- [POP](#) ..., PC.
- [LDR](#) PC, [SP], #offset.
- A [BXNS](#) with any register.
- Any indirect branch with [FNC_RETURN](#) as the target address.

If the PACBTI Extension is implemented, the counter counts [Architecturally executed](#) [BXAUT](#) instructions that pass their PAC checks. The following is a non-exhaustive list of instructions that are not counted:

- [BX](#) with any register that is not [LR](#).
- Any [MOV](#) PC where the register being written to is not the [LR](#).
- [LDM](#) SP, ..., PC because write-back is not specified.
- [LDR](#) PC, [SP, #offset] because this specifies the wrong addressing mode.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

UNALIGNED_LDST_RETIRED (0x000F, Architectural)

The counter counts each memory-reading instruction or memory-writing instruction access that would generate an UNALIGNED UsageFault when [CCR.UNALIGN_TRP](#) is 1. It is IMPLEMENTATION DEFINED whether this event counts accesses that generate a fault, including accesses that do generate an UNALIGNED UsageFault.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

BR_MIS_PRED (0x0010, Microarchitectural)

The counter counts each correction to the predicted program flow that occurs because of a misprediction from, or no prediction from, the branch prediction resources, and that relates to instructions that the branch prediction resources are capable of predicting. If no program-flow prediction resources are implemented, Arm recommends that the counter counts all branches that are not taken.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

CPU_CYCLES (0x0011, Microarchitectural)

The counter increments on every cycle.

All counters are subject to changes in clock frequency, including when a [WFI](#) or [WFE](#) instruction stops the clock. This means that it is **CONSTRAINED UNPREDICTABLE** whether or not CPU_CYCLES continues to increment when the clocks are stopped by [WFI](#) and [WFE](#) instructions.

Unlike [PMU_CCNTR](#), this count is not affected by [PMU_CTRL.DP](#), or by [PMU_CTRL.C](#):

- The counter is not incremented in prohibited regions, and is not affected by [PMU_CTRL.DP](#).
- The counter is reset when event counters are reset by [PMU_CTRL.P](#), never by [PMU_CTRL.C](#).

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

BR_PRED (0x0012, Microarchitectural)

The counter counts every branch or other change in the program flow that the branch prediction resources are capable of predicting. If all branches are subject to prediction, then all branches are predictable branches. If branches are decoded before the branch predictor, so that the branch prediction logic dynamically predicts only some branches, for example conditional and indirect branches, then it is **IMPLEMENTATION DEFINED** whether other branches are counted as predictable branches. Arm recommends that all branches are counted.

An implementation might include other structures that predict branches, such as a loop buffer that predicts short backwards direct branches as taken. Each execution of such a branch is a predictable branch. Terminating the loop might generate a misprediction event that is counted by [BR_MIS_PRED](#). If no program-flow prediction resources are implemented, this event is optional, but Arm recommends that BR_PRED counts all branches.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MEM_ACCESS (0x0013, Microarchitectural)

The counter counts memory-read or memory-write operations that the PE made. The counter increments whether the access results in an access to a Level 1 data or unified cache, a Level 2 data or unified cache, or neither of these. The counter does not increment as a result of:

- Instruction memory accesses.
- Cache maintenance instructions.
- Write-back from any cache.
- Refilling of any cache.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

L1I_CACHE (0x0014, Microarchitectural)

See [L<n>I_CACHE](#).

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

L1D_CACHE_WB (0x0015, Microarchitectural)

See [L<n>D_CACHE_WB](#).

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

L2D_CACHE (0x0016, Microarchitectural)

See [L<n>D_CACHE](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L2D_CACHE_REFILL (0x0017, Microarchitectural)

See [L<n>D_CACHE_REFILL](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L2D_CACHE_WB (0x0018, Microarchitectural)

See [L<n>D_CACHE_WB](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

BUS_ACCESS (0x0019, Microarchitectural)

The counter counts memory-read or memory-write operations that access outside of the boundary of the PE and its closely-coupled caches. This counter does not include accesses to or from a TCM. Where this boundary lies with respect to any implemented caches is IMPLEMENTATION DEFINED.

The definition of a bus access is IMPLEMENTATION DEFINED but physically it is a single request rather than a burst, (that is, for each bus cycle for which the bus is active). Bus accesses include refills of, and write-backs from, data, instruction, and unified caches. Whether bus accesses include operations that do use the bus but that do not explicitly transfer data is IMPLEMENTATION DEFINED. An Unattributable bus access occurs when a requestor outside the PE makes a request that results in a bus access, for example, a coherency request.

If the bus is shared, then only [Attributable](#) bus accesses are counted. If the bus is not shared, then all bus accesses are counted.

Where an implementation has multiple buses at this boundary, this event counts the sum of accesses across all buses. If a bus supports multiple accesses per cycle, for example through multiple channels, the counter increments one time for each channel that is active on a cycle, and so it might increment by more than one in any given cycle. The maximum increment in any given cycle is IMPLEMENTATION DEFINED.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MEMORY_ERROR (0x001A, Microarchitectural)

The counter counts every occurrence of a memory error that is signaled by memory closely coupled to this PE. The definition of local memories is IMPLEMENTATION DEFINED but includes caches and tightly-coupled memories. Memory error refers to a physical error that is detected by the hardware, such as a parity or ECC error. It includes errors that are correctable and those that are not. It does not include errors that are defined in the architecture, such as MPU or SAU faults.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

INST_SPEC (0x001B, Microarchitectural)

The counter counts instructions that are speculatively executed by the PE. This includes instructions that are subsequently not architecturally executed. As a result, this event counts a larger number of instructions than the number of instructions [Architecturally executed](#). The definition of speculatively executed is IMPLEMENTATION DEFINED.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

BUS_CYCLES (0x001D, Microarchitectural)

The counter increments on every cycle of the external memory interface of the PE.

If the implementation clocks the external memory interface at the same rate as the processor hardware, the counter counts every cycle.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

CHAIN (0x001E, Architectural)

Even-numbered counters never increment as a result of this event. This means the CHAIN event links the odd-numbered counter with the preceding even-numbered counter to provide a 32-bit counter.

The CHAIN event means a system can provide N 16-bit counters, N/2 32-bit counters, or a mixture of 16-bit counters and 32-bit counters. The increment of both counters is atomic with respect to software or external counter access.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L1D_CACHE_ALLOCATE (0x001F, Microarchitectural)

The counter increments on every [Attributable](#) write that writes an entire line into the Level 1 cache without fetching from outside the Level 1 cache, for example a write from a coalescing buffer of a full cache line.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L2D_CACHE_ALLOCATE (0x0020, Microarchitectural)

The counter increments on every [Attributable](#) write that writes an entire line into the Level 2 cache without fetching from outside the Level 1 or Level 2 caches, for example:

- A write-back from a Level 1 to a Level 2 cache.
- A write from a coalescing buffer of a full [cache line](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

BR_RETIRE (0x0021, Architectural)

The counter counts all branches on the [Architecturally executed](#) path that would incur a cost if mispredicted.

It is IMPLEMENTATION DEFINED whether this includes each of:

- Unconditional direct branch instructions.
- Exception-generating instructions.
- Exception return instructions.
- Context synchronization instructions.

Conditional branches are always counted, regardless of whether the branch is taken.

Arm recommends that BR_RETIRE counts Unconditional direct branch instructions.

If [PC_WRITE_RETIRE](#) and BR_RETIRE are both implemented, the PE must treat the following types of instruction in the same way for both events:

- [BKPT](#) instructions.
- UNDEFINED instructions.
- [SVC](#) instructions.
- Context synchronization barrier instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

BR_MIS_PRED_RETIRE (0x0022, Microarchitectural)

The counter counts all instructions counted by [BR_RETIRE](#) that were not correctly predicted. If no program-flow prediction resources are implemented, this event counts all retired not-taken branches.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

STALL_FRONTEND (0x0023, Microarchitectural)

The counter counts every cycle counted by the [CPU_CYCLES](#) event on which no operation was issued because there are no operations available to issue for this PE from the frontend. The division between frontend and backend is IMPLEMENTATION DEFINED. Frontend and backend events count at the same point in the pipeline.

For a simplified pipeline model of Fetch > Decode > Issue > Execute > Retire, Arm recommends that the events are counted when instructions are dispatched from Decode to Issue.

On a given cycle, both events might be counted if the backend is unable to accept any operations and there are no operations available to issue from the frontend.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

STALL_BACKEND (0x0024, Microarchitectural)

The counter counts every cycle counted by the [CPU_CYCLES](#) event on which no operation was issued because either:

- The backend is unable to accept any of the operations available for issue for this PE.
- The backend is unable to accept any operations.

For example, the back end might be unable to accept operations because of a resource conflict or non-availability. The division between frontend and backend is IMPLEMENTATION DEFINED. Frontend and backend events count at the same point in the pipeline.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L2I_CACHE (0x0027, Microarchitectural)

See [L<n>I_CACHE](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L2I_CACHE_REFILL (0x0028, Microarchitectural)

See [L<n>I_CACHE_REFILL](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L3D_CACHE_ALLOCATE (0x0029, Microarchitectural)

The counter increments on every [Attributable](#) write that writes an entire line into the Level 3 cache without fetching from outside the Level 1, Level 2, or Level 3 cache, for example:

- A write-back from a Level 2 to Level 3 cache.
- A write from a coalescing buffer of a full [cache line](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L3D_CACHE_REFILL (0x002A, Microarchitectural)

See [L<n>D_CACHE_REFILL](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L3D_CACHE (0x002B, Microarchitectural)

See [L<n>D_CACHE](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L3D_CACHE_WB (0x002C, Microarchitectural)

See [L<n>D_CACHE_WB](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

LL_CACHE_RD (0x0036, Microarchitectural)

See [L<n>D_CACHE_RD](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

LL_CACHE_MISS_RD (0x0037, Microarchitectural)

See [L<n>D_CACHE_MISS_RD](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#).

L1D_CACHE_LMISS_RD (0x0039, Microarchitectural)

See [L<n>D_CACHE_MISS_RD](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#).

OP_RETIRED (0x003A, Microarchitectural)

The counter counts each operation counted by [OP_SPEC](#) that is later committed to the architectural state of this PE.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#).

OP_SPEC (0x003B, Microarchitectural)

The counter counts the number of [Attributable](#) instructions or operations that are sent for execution by this PE, including those that are not committed to the architectural state of this PE.

This event might be an alias for [INST_SPEC](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#).

STALL (0x003C, Microarchitectural)

The counter counts every [Attributable](#) cycle on which no [Attributable](#) instruction operation was sent for execution for this PE.

The division between frontend and backend is IMPLEMENTATION DEFINED. [STALL](#), [STALL_FRONTEND](#) and [STALL_BACKEND](#) events count at the same point in the pipeline.

For a simplified pipeline model of Fetch, Decode, Issue, Execute, Retire, Arm recommends that the events are counted when instructions are dispatched from Decode to Issue.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#).

STALL_SLOT_BACKEND (0x003D, Microarchitectural)

Counts each slot counted by [STALL_SLOT](#) where no [Attributable](#) instruction or operation was sent for execution because either:

- The backend was unable to accept the instruction operation available for this PE on the slot.
- The backend is unable to accept any operations on the slot.

The division between frontend and backend is IMPLEMENTATION DEFINED. [STALL_SLOT](#), [STALL_SLOT_FRONTEND](#) and [STALL_SLOT_BACKEND](#) events count at the same point in the pipeline.

On a given cycle, both the [STALL_SLOT_FRONTEND](#) and the [STALL_SLOT_BACKEND](#) event might be counted if the backend is unable to accept any operations and there are no operations available to issue from the frontend.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#).

STALL_SLOT_FRONTEND (0x003E, Microarchitectural)

Counts each slot counted by [STALL_SLOT](#) where no [Attributable](#) instruction or operation was sent for execution because there was no [Attributable](#) instruction or operation available to issue for this PE from the frontend for the slot.

The division between frontend and backend is IMPLEMENTATION DEFINED. [STALL_SLOT](#), [STALL_SLOT_FRONTEND](#) and [STALL_SLOT_BACKEND](#) events count at the same point in the pipeline.

On a given cycle, both the [STALL_SLOT_FRONTEND](#) and the [STALL_SLOT_BACKEND](#) event might be counted if the backend is unable to accept any operations and there are no operations available to issue from the frontend.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

STALL_SLOT (0x003F, Microarchitectural)

The counter counts on each [Attributable](#) cycle the number of instruction or operation slots that were not occupied by an instruction or operation [Attributable](#) to this PE.

The definition of a slot is IMPLEMENTATION DEFINED, but there is a fixed number of slots, WIDTH, that are available on each cycle, so that the formula $\text{STALL_SLOT} / (\text{CPU_CYCLES} \times \text{WIDTH})$ gives the utilization of the slots of the processor by [Attributable](#) instruction or operations of this PE. Each slot can hold at most one instruction or operation each cycle.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

L1D_CACHE_RD (0x0040, Microarchitectural)

See [L<n>D_CACHE_RD](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

LE_RETIRE (0x0100, Microarchitectural)

The counter increments for every [Architecturally executed](#) loop end ([LE](#)) instruction, when that instruction registers an entry inside the [LO_BRANCH_INFO](#) cache.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

LE_SPEC (0x0101, Microarchitectural)

The counter increments for every speculatively executed loop end ([LE](#)) instruction, when that instruction registers an entry inside the [LO_BRANCH_INFO](#) cache.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

BF_RETIRE (0x0104, Microarchitectural)

The counter increments for every [Architecturally executed](#) branch future instruction, when that instruction registers an entry inside the [LO_BRANCH_INFO](#) cache.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

BF_SPEC (0x0105, Microarchitectural)

The counter increments for every speculatively executed branch future instruction, when that instruction registers an entry inside the [LO_BRANCH_INFO](#) cache.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

LE_CANCEL (0x0108, Microarchitectural)

The [LO_BRANCH_INFO](#) cache was cleared while it contained a valid loop entry as set up by a loop end instruction, and that did not coincide with the last iteration of the loop.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

BF_CANCEL (0x0109, Microarchitectural)

The [LO_BRANCH_INFO](#) cache was cleared while it contained a valid branch entry as set up by a branch future instruction, and that did not coincide with a taken implicit branch.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

SE_CALL_S (0x0114, Architectural)

The counter increments for every [Architecturally executed SG](#) instruction that results in a Security state transition from Non-secure state to Secure state.

Arm recommends that this counter increments regardless of the Security state filters that are applied to the PMU.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

SE_CALL_NS (0x0115, Architectural)

The counter increments for every [Architecturally executed BLXNS](#) instruction that results in a Security state transition from Secure state to Non-secure state.

Arm recommends that this counter increments regardless of the Security state filters that are applied to the PMU.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

DWT_CMPMATCH0 (0x0118, Architectural)

See [DWT_CMPMATCH<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

DWT_CMPMATCH1 (0x0119, Architectural)

See [DWT_CMPMATCH<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

DWT_CMPMATCH2 (0x011A, Architectural)

See [DWT_CMPMATCH<n>](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#) && [DSPDE](#).

DWT_CMPMATCH3 (0x011B, Architectural)

See [DWT_CMPMATCH<n>](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#) && [DSPDE](#).

DWT_CMPMATCH4 (0x011C, Architectural)

See [DWT_CMPMATCH<n>](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#) && [DSPDE](#).

DWT_CMPMATCH5 (0x011D, Architectural)

See [DWT_CMPMATCH<n>](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#) && [DSPDE](#).

DWT_CMPMATCH6 (0x011E, Architectural)

See [DWT_CMPMATCH<n>](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#) && [DSPDE](#).

DWT_CMPMATCH7 (0x011F, Architectural)

See [DWT_CMPMATCH<n>](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#) && [DSPDE](#).

PAC_RETIRED (0x0130, Microarchitectural)

The counter counts each operation counted by [INST_RETIRED](#) where the [Architecturally executed](#) instruction is [PAC](#), [PACBTI](#), or [PACG](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#).

PAC_SPEC (0x0131, Microarchitectural)

The counter counts each operation counted by [INST_SPEC](#) where the speculatively executed instruction is [PAC](#), [PACBTI](#), or [PACG](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [PMU](#).

AUT_RETIRED (0x0134, Microarchitectural)

The counter counts each operation counted by `INST_RETIRED` where the [Architecturally executed](#) instruction is `AUT` or `AUTG`.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

AUT_SPEC (0x0135, Microarchitectural)

The counter counts each operation counted by `INST_SPEC` where the speculatively executed instruction is `AUT` or `AUTG`.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

STALL_BACKEND_PAC (0x013C, Microarchitectural)

The counter counts each operation counted by `STALL_BACKEND` where no operation can be issued due to a PAC operation being in progress. This includes PAC stalls as a result of `PAC`, `PACBTI`, `PACG`, `AUT`, and `AUTG` instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_INST_RETIRED (0x0200, Architectural)

The counter increments for each [Architecturally executed MVE](#) instruction that is subject to beat-wise execution. This includes instructions that are partially or fully predicated, and instructions that resume execution after returning from an exception.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_INST_SPEC (0x0201, Microarchitectural)

The counter increments for each speculatively executed `MVE` instruction that is subject to beat-wise execution. This includes instructions that are subsequently not [Architecturally executed](#). As a result, this event might count a larger number of instructions than the number of instructions [Architecturally executed](#). The definition of speculatively executed is IMPLEMENTATION DEFINED.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_FP_RETIRED (0x0204, Architectural)

The counter increments for each [Architecturally executed MVE](#) instruction, as counted by `MVE_INST_RETIRED`, that operates on floating-point data. This includes `MVE` conversion instructions that convert to or from floating-point representation, as well as floating-point vector compare and vector predicate instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_FP_SPEC (0x0205, Microarchitectural)

The counter increments for each speculatively executed [MVE](#) instruction, as counted by [MVE_INST_SPEC](#), that operates on floating-point data. This includes [MVE](#) conversion instructions that convert to or from floating-point representation, as well as floating-point vector compare and vector predicate instructions.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_FP_HP_RETIRE (0x0208, Architectural)

The counter increments for each [Architecturally executed MVE](#) floating-point instruction that operates on floating-point data, as counted by [MVE_FP_RETIRE](#), and which additionally operates on half-precision data. This includes [MVE](#) conversion instructions that convert to or from half-precision floating-point representation.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_FP_HP_SPEC (0x0209, Microarchitectural)

The counter increments for each speculatively executed [MVE](#) floating-point instruction that operates on floating-point data, as counted by [MVE_FP_SPEC](#), and which additionally operates on half-precision data. This includes [MVE](#) conversion instructions that convert to or from half-precision floating-point representation.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_FP_SP_RETIRE (0x020C, Architectural)

The counter increments for each [Architecturally executed MVE](#) floating-point instruction that operates on floating-point data, as counted by [MVE_FP_RETIRE](#), and which additionally operates on single-precision data. This includes [MVE](#) conversion instructions that convert to or from single-precision floating-point representation.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_FP_SP_SPEC (0x020D, Microarchitectural)

The counter increments for each speculatively executed [MVE](#) floating-point instruction that operates on floating-point data, as counted by [MVE_FP_SPEC](#), and which additionally operates on single-precision data. This does not include [MVE](#) conversion instructions that convert to or from single-precision floating-point representation.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_FP_MAC_RETIRE (0x0214, Architectural)

The counter increments for each [Architecturally executed MVE](#) floating-point instruction that performs a multiply or multiply-accumulate operation. This includes instructions that perform fused multiply-accumulation.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_FP_MAC_SPEC (0x0215, Microarchitectural)

The counter increments for each speculatively executed [MVE](#) floating-point instruction that performs a multiply or multiply-accumulate operation. This includes instructions that perform fused multiply-accumulation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_INT_RETIRED (0x0224, Architectural)

The counter increments for each [Architecturally executed MVE](#) beat-wise integer or fixed-point instruction. This does not include any floating-point conversion instructions. This does include integer vector compare and vector predicate instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_INT_SPEC (0x0225, Microarchitectural)

The counter increments for each speculatively executed [MVE](#) beat-wise integer or fixed-point instruction. This does not include any floating-point conversion instructions. This does include integer vector compare and vector predicate instructions.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_INT_MAC_RETIRED (0x0228, Architectural)

The counter increments for each [Architecturally executed MVE](#) integer or fixed-point instruction, as counted by [MVE_INT_RETIRED](#), which additionally performs a multiply or multiply-accumulate operation. This includes reducing variants.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_INT_MAC_SPEC (0x0229, Microarchitectural)

The counter increments for each speculatively executed [MVE](#) integer or fixed-point instruction, as counted by [MVE_INT_SPEC](#) which additionally performs a multiply or multiply-accumulate operation. This includes reducing variants.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_RETIRED (0x0238, Architectural)

The counter increments for each [Architecturally executed MVE](#) load instruction that writes data to a Q register or store instruction that sources data from a Q register.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_SPEC (0x0239, Microarchitectural)

The counter increments for each speculatively executed [MVE](#) load instruction that writes data to a Q register or store instruction that sources data from a Q register.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_RETIRED (0x023C, Architectural)

The counter increments each time [MVE_LDST_RETIRED](#) increments as a result of a load operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_SPEC (0x023D, Microarchitectural)

The counter increments each time [MVE_LDST_SPEC](#) increments as a result of a load operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_RETIRED (0x0240, Architectural)

The counter increments each time [MVE_LDST_RETIRED](#) increments as a result of a store operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_SPEC (0x0241, Microarchitectural)

The counter increments each time [MVE_LDST_SPEC](#) increments as a result of a store operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_CONTIG_RETIRED (0x0244, Microarchitectural)

The counter increments each time a contiguous memory load or store instruction is [Architecturally executed](#), and results in an optimal number of memory accesses. The counter increments for scatter-gather instructions that are promoted to contiguous accesses. The counter increments regardless of alignment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_CONTIG_SPEC (0x0245, Microarchitectural)

The counter increments each time a contiguous memory load or store instruction is speculatively executed, and results in an optimal number of memory accesses. The counter increments for scatter-gather instructions that are promoted to contiguous accesses. The counter increments regardless of alignment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_CONTIG_RETIRED (0x0248, Microarchitectural)

The counter increments each time [MVE_LDST_CONTIG_RETIRED](#) increments as a result of a load operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_CONTIG_SPEC (0x0249, Microarchitectural)

The counter increments each time [MVE_LDST_CONTIG_SPEC](#) increments as a result of a load operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_CONTIG_RETIRED (0x024C, Microarchitectural)

The counter increments each time [MVE_LDST_CONTIG_RETIRED](#) increments as a result of a store operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_CONTIG_SPEC (0x024D, Microarchitectural)

The counter increments each time [MVE_LDST_CONTIG_SPEC](#) increments as a result of a store operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_NONCONTIG_RETIRED (0x0250, Microarchitectural)

The counter increments each time a non-contiguous memory load or store instruction is [Architecturally executed](#), and results in an increased number of accesses compared to a contiguous operation. The counter is incremented one time, regardless of the additional number of memory requests that are required to complete the execution of that instruction. The counter does not increment if a scatter-gather instruction is promoted to contiguous memory accesses.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_NONCONTIG_SPEC (0x0251, Microarchitectural)

The counter increments each time a non-contiguous memory load or store instruction is speculatively executed, and results in an increased number of accesses compared to a contiguous operation. The counter is incremented one time, regardless of the additional number of memory requests that are required to complete the execution of that instruction. The counter does not increment if a scatter-gather instruction is promoted to contiguous memory accesses.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_NONCONTIG_RETIRED (0x0254, Microarchitectural)

The counter increments each time [MVE_LDST_NONCONTIG_RETIRED](#) increments as a result of a load operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_NONCONTIG_SPEC (0x0255, Microarchitectural)

The counter increments each time [MVE_LDST_NONCONTIG_SPEC](#) increments as a result of a load operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_NONCONTIG_RETIRED (0x0258, Microarchitectural)

The counter increments each time [MVE_LDST_NONCONTIG_RETIRED](#) increments as a result of a store operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_NONCONTIG_SPEC (0x0259, Microarchitectural)

The counter increments each time [MVE_LDST_NONCONTIG_SPEC](#) increments as a result of a store operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_MULTI_RETIRED (0x025C, Architectural)

The counter increments whenever a [VLD2x](#), [VST2x](#), [VLD4x](#), or [VST4x](#) instruction is [Architecturally executed](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_MULTI_SPEC (0x025D, Microarchitectural)

The counter increments whenever a [VLD2x](#), [VST2x](#), [VLD4x](#), or [VST4x](#) instruction is speculatively executed.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_MULTI_RETIRED (0x0260, Architectural)

The counter increments whenever a [VLD2x](#), or [VLD4x](#) instruction is [Architecturally executed](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_MULTI_SPEC (0x0261, Microarchitectural)

The counter increments whenever a [VLD2x](#), or [VLD4x](#) instruction is speculatively executed.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_MULTI_RETIRED (0x0264, Architectural)

The counter increments whenever a [VST2x](#), or [VST4x](#) instruction is [Architecturally executed](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_MULTI_SPEC (0x0265, Microarchitectural)

The counter increments whenever a [VST2x](#), or [VST4x](#) instruction is speculatively executed.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_UNALIGNED_RETIRED (0x028C, Microarchitectural)

The counter increments when [MVE_LDST_RETIRED](#) increments as a result of an instruction that requires one or more additional memory accesses because of misalignment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_UNALIGNED_SPEC (0x028D, Microarchitectural)

The counter increments when [MVE_LDST_SPEC](#) increments as a result of an instruction that requires one or more additional memory accesses because of misalignment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_UNALIGNED_RETIRED (0x0290, Microarchitectural)

The counter increments when [MVE_LDST_UNALIGNED_RETIRED](#) increments as a result of a load operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LD_UNALIGNED_SPEC (0x0291, Microarchitectural)

The counter increments when [MVE_LDST_UNALIGNED_SPEC](#) increments as a result of a load operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_UNALIGNED_RETIRED (0x0294, Microarchitectural)

The counter increments when [MVE_LDST_UNALIGNED_RETIRED](#) increments as a result of a store operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_ST_UNALIGNED_SPEC (0x0295, Microarchitectural)

The counter increments when [MVE_LDST_UNALIGNED_SPEC](#) increments as a result of a store operation.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_UNALIGNED_NONCONTIG_RETIRED (0x0298, Microarchitectural)

This counter increments whenever both [MVE_LDST_UNALIGNED_RETIRED](#) and [MVE_LDST_NONCONTIG_RETIRED](#) increment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_LDST_UNALIGNED_NONCONTIG_SPEC (0x0299, Microarchitectural)

This counter increments whenever both [MVE_LDST_UNALIGNED_SPEC](#) and [MVE_LDST_NONCONTIG_SPEC](#) increment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_VREDUCE_RETIRED (0x02A0, Architectural)

The counter increments whenever an [MVE](#) instruction that operates on a vector to produce a scalar result that is stored in a general-purpose register is [Architecturally executed](#). This includes only instructions that have the ‘V’ suffix.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_VREDUCE_SPEC (0x02A1, Microarchitectural)

The counter increments whenever an [MVE](#) instruction that operates on a vector to produce a scalar result that is stored in a general-purpose register is speculatively executed. This includes only instructions that have the ‘V’ suffix.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_VREDUCE_FP_RETIRE (0x02A4, Architectural)

The counter increments whenever both [MVE_VREDUCE_RETIRE](#) and [MVE_FP_RETIRE](#) increment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_VREDUCE_FP_SPEC (0x02A5, Microarchitectural)

The counter increments whenever both [MVE_VREDUCE_SPEC](#) and [MVE_FP_SPEC](#) increment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_VREDUCE_INT_RETIRE (0x02A8, Architectural)

The counter increments whenever both [MVE_VREDUCE_RETIRE](#) and [MVE_INT_RETIRE](#) increment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_VREDUCE_INT_SPEC (0x02A9, Microarchitectural)

The counter increments whenever both [MVE_VREDUCE_SPEC](#) and [MVE_INT_SPEC](#) increment.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_PRED (0x02B8, Microarchitectural)

The counter increments on each [Architecture tick](#) where one or more beats of an [MVE](#) beat-wise instruction architecturally completes, and where one or more of these beats is partially or fully predicated false by VPR or loop-tail predication.

If a beat is interrupted by an exception and does not architecturally complete, or if the beat is masked by [EPSR.ECI](#) when resuming execution following an exception, Arm recommends that the counter does not increment.

The ratio $(MVE_PRED * BEATS_PER_TICK) / (4 * MVE_INST_RETIRE)$ then offers an approximate insight into the proportion of [MVE](#) instructions affected by predication, where BEATS_PER_TICK is an IMPLEMENTATION DEFINED average number of beats, including from distinct overlapping instructions, executed per [Architecture tick](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

MVE_STALL (0x02CC, Microarchitectural)

The counter counts every cycle counted by the [CPU_CYCLES](#) event on which no operation was issued as a direct result of an [MVE](#) instruction that is either currently executing or attempting to execute.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_STALL_RESOURCE (0x02CD, Microarchitectural)

The counter increments whenever [MVE_STALL](#) increments, and where the cause is attributable to an [MVE](#) instruction failing to execute because there are no available resources in the PE capable of executing that instruction.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_STALL_RESOURCE_MEM (0x02CE, Microarchitectural)

The counter increments whenever [MVE_STALL](#) increments, and where the cause is attributable to an [MVE](#) instruction failing to execute because there are no available memory resources in the PE capable of executing that instruction.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_STALL_RESOURCE_FP (0x02CF, Microarchitectural)

The counter increments whenever [MVE_STALL](#) increments, and where the cause is attributable to an [MVE](#) instruction failing to execute because there are no available floating-point resources in the PE capable of executing that instruction.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_STALL_RESOURCE_INT (0x02D0, Microarchitectural)

The counter increments whenever [MVE_STALL](#) increments, and where the cause is attributable to an [MVE](#) instruction failing to execute because there are no available integer resources in the PE capable of executing that instruction.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_STALL_BREAK (0x02D3, Microarchitectural)

The counter increments whenever [MVE_STALL](#) increments, and where the cause is attributable to waiting for the completion of an in-flight [MVE](#) instruction. A possible example is when an [MVE](#) chainable instruction completes before executing a scalar instruction. Arm recommends that the counter increments only if no other specific attributable cause can be identified.

Applies to an implementation of the architecture [Armv8.1-M](#) onward. The extension requirements are - [PMU](#).

MVE_STALL_DEPENDENCY (0x02D4, Microarchitectural)

The counter increments whenever [MVE_STALL](#) increments, and where the cause is attributable to the subsequent instruction being delayed to resolve a register RAW conflict.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

ITCM_ACCESS (0x4007, Microarchitectural)

The counter counts memory read or memory write operations that the PE made to an instruction TCM.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

DTCM_ACCESS (0x4008, Microarchitectural)

The counter counts memory read or memory write operations that the PE made to a data or unified TCM.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

TRCEXTOUT0 (0x4010, Microarchitectural)

See [TRCEXTOUT<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

TRCEXTOUT1 (0x4011, Microarchitectural)

See [TRCEXTOUT<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

TRCEXTOUT2 (0x4012, Microarchitectural)

See [TRCEXTOUT<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

TRCEXTOUT3 (0x4013, Microarchitectural)

See [TRCEXTOUT<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

CTI_TRIGOUT4 (0x4018, Microarchitectural)

See [CTI_TRIGOUT<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

CTI_TRIGOUT5 (0x4019, Microarchitectural)

See [CTI_TRIGOUT<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

CTI_TRIGOUT6 (0x401A, Microarchitectural)

See [CTI_TRIGOUT<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

CTI_TRIGOUT7 (0x401B, Microarchitectural)

See [CTI_TRIGOUT<n>](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#) && [DSPDE](#).

B15.11 Required PMU events

R_FXXR

The architecture requires that the PMU supports at least the following common events:

- 0x0000 **SW_INCR**: Instruction **Architecturally executed**, condition code check pass, software increment.
- 0x0003 **L1D_CACHE_REFILL**: **Attributable** Level 1 data cache refill.
 - This event is only required if the implementation includes a Level 1 data or unified cache.
- 0x0004 **L1D_CACHE**: **Attributable** Level 1 data cache access.
 - This event is only required if the implementation includes a Level 1 data or unified cache.
- 0x0008 **INST_RETIRED**: Instruction **Architecturally executed**.
- 0x0011 **CPU_CYCLES**: Cycle.
- 0x0021 **BR_RETIRED**: Instruction **Architecturally executed**, branch.
 - This event is only required if the implementation includes program flow prediction.
- 0x0022 **BR_MIS_PRED_RETIRED**: Instruction **Architecturally executed**, mispredicted branch.
 - This event is only required if the implementation includes program flow prediction.
- 0x0023 **STALL_FRONTEND**: No operation issued due to the frontend.
- 0x0024 **STALL_BACKEND**: No operation issued due to the backend.
- 0x0200 **MVE_INST_RETIRED**: **MVE** instruction **Architecturally executed**.
 - This event is only required if the implementation includes **MVE**.
- 0x0238 **MVE_LDST_RETIRED**: **MVE** load or store instruction **Architecturally executed**.
 - This event is only required if the implementation includes **MVE**.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **PMU**.*

B15.12 IMPLEMENTATION DEFINED event numbers

R_{ZJSZ} For IMPLEMENTATION DEFINED event numbers, each counter is independently defined to either:

- Increment only one time for each event.
- Count the duration for which an event occurs.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

I_{LCNR} Arm recommends that implementers establish a standardized numbering scheme for their IMPLEMENTATION DEFINED events, with common definitions and common count numbers applied to all of their implementations. In general, Arm recommends standardization across implementations with common features. However, Arm recognizes that attempting to standardize the encoding of microarchitectural features across too wide a range of implementations is not productive.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

I_{RKHN} Arm strongly recommends that the IMPLEMENTATION DEFINED events allow the user to measure the utilization of any microarchitectural features that the implementation considers important for system performance, and any significant deviations from optimal performance.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [PMU](#).

Chapter B16

Reliability, Availability, and Serviceability (RAS) Extension

This chapter specifies the Armv8.1-M Reliability, Availability, and Serviceability (RAS) Extension. A minimal implementation of this is mandatory for any implementation of the Armv8.1-M architecture, but any additional RAS features are optional.

This chapter contains the following sections:

[B16.1 Overview.](#)

[B16.2 Taxonomy of errors.](#)

[B16.3 Generating error exceptions.](#)

[B16.4 Error Synchronization Barrier \(ESB\).](#)

[B16.5 Implicit Error Synchronization \(IESB\).](#)

[B16.6 Fault handling.](#)

[B16.7 RAS error records.](#)

[B16.8 Multiple BusFault exceptions.](#)

[B16.9 Minimal RAS implementation.](#)

*Applies to an implementation of the architecture **Armv8.1-M** onward. The extension requirements are - **RAS**.*

B16.1 Overview

I _{ZTMS}	For a detailed description of possible RAS errors, see the <i>Arm® Reliability, Availability, and Serviceability (RAS) Specification, Armv8, for the Armv8-A architecture profile</i>. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{TXGN}	A minimum implementation of the RAS Extension is required for an implementation of the v8.1-M architecture. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{RSMP}	ID_PFR0.RAS is nonzero when the RAS Extension is implemented. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{CTQJ}	An error is a deviation from correct service. For the purpose of describing the RAS Extension, deviation from correct service is defined using the following terms: • A <i>failure</i> is the event of deviation from correct service. This includes data corruption, data loss, and service loss. • An <i>error</i> is the deviation. • A <i>fault</i> is the cause of the error. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{FCTN}	When a PE accesses memory or other state, an error might be detected in that memory or state, and corrected, deferred, or signaled to the PE as a Detected error. It is IMPLEMENTATION DEFINED whether an error that is detected by a consumer of a write from a PE is signaled to the PE and becomes a Detected error that is consumed by the PE. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{LTTV}	The nodes that are included as part of a PE, including an Armv8-M PE, are IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{TLFS}	A single error might generate multiple exceptions. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{NJVH}	Software must be aware that errors might be double-reported. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>

See also:

[B16.9 Minimal RAS implementation](#).

B16.2 Taxonomy of errors

I_{GNNQ}

The architecture does not specify techniques for:

- Fault prevention.
- Fault removal.
- Fault injection.
- Testing.

These are IMPLEMENTATION DEFINED and are outside the scope of this architecture.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

B16.2.1 Architectural error propagation

R_{MVFK}

For a PE, Error propagation applies to the propagation of detected errors between the general-purpose registers or the Floating-point Extension register file, and any program-visible architectural state of the PE, including:

- Other general-purpose registers and the Floating-point Extension register file.
- Memory-mapped registers.
- Special-purpose registers.
- Memory.

That is, the error is propagated by:

- A store of a corrupt value.
- A write of a corrupt value to a System register or a special-purpose register. Infecting a System register state might mean that the PE generates transactions that would not otherwise be permitted.
- Any operation that would not have been permitted to occur had the error not been activated, including:
 - A load or instruction fetch that would not have been permitted, including those from hardware speculation or prefetching.
 - A store to an incorrect address or a store that would not have been made or not permitted.
 - A direct or indirect write to a special-purpose or System register that would not have been made or not permitted.
 - Assertion of any signal, such as an interrupt, that would not have been asserted.
- Any operation not occurring that would have occurred had the error not been activated.
- Taking an asynchronous exception.
- The PE discarding data that it holds in a modified state.
- Any other loss of uniprocessor semantics, ordering or coherency.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

R_{RGPS}

The propagated error is **silently propagated** if it is not signaled to a consumer as a **Detected** error.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

R_{JKVQ}

The features that a PE includes to contain an error are IMPLEMENTATION DEFINED, and it is IMPLEMENTATION DEFINED whether an error can be signaled to the consumer as a **Detected** error.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

B16.2.2 Architecturally infected, contained, and uncontained

R_{VKZT} **Infected**, **Poisoned**, **Containable** and **Uncontainable** apply to all program-visible architectural state of the PE, including general-purpose registers, the Floating-point Extension register file, special purpose registers, System registers, and memory.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

R_{ZPCB} An error is **Uncontainable** by the PE if the error is **silently propagated**, unless it is contained because all of the following are true:

- The corrupt value is in the general-purpose register or in the Floating-point Extension register file.
- The error has only been **silently propagated** by an instruction that occurs in program order after one of the following:
 - Taking a BusFault that is generated by the error.
 - An Error Synchronization Barrier operation that synchronizes the error.
- The error is not **silently propagated** in any other way.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

B16.2.3 Architecturally consumed errors

R_{RNNF} For a PE, an error is architecturally consumed if any of the following are true:

- An instruction commits the corruption into the visible state of the PE.
- The error is on an instruction fetch and the instruction is committed for execution.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

R_{MKBF} The PE takes action for a detected, architecturally consumed error either by:

- Generating an error exception.
- Entering a failure mode.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

B16.2.4 Other errors

I_{TCZS} Errors from software faults are outside the scope of the RAS Extension error recovery architecture.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

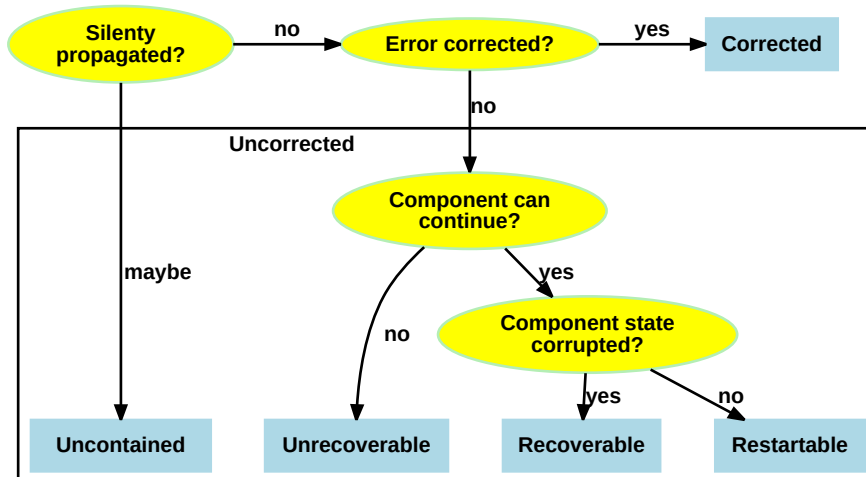
I_{KDJP} From within the PE itself, other errors might be detected. These are not errors that are detected by the architectural model of the PE and so are treated like errors that are detected by another component. Other components might report errors to a PE using error recovery interrupts. An example of this is when the cache, not the PE, detects a RAM error. Other components might report errors to a PE using error recovery interrupts.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

B16.3 Generating error exceptions

I_{WZHQ}

The following diagram shows the taxonomy of consumed errors.



Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **RAS**.

R_{PVJQ}

An error exception is generated for all detected RAS errors that are neither **Corrected** nor **Deferred** errors. These error exceptions are signaled to, and consumed by, a PE and are not silently propagated.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **RAS**.

R_{RPFQ}

A **Corrected** error is detected and corrected by the PE, and is not silently propagated.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **RAS**.

I_{NDMC}

In normal circumstances a **Corrected** error no longer infects the node. In IMPLEMENTATION DEFINED circumstances, the **Corrected** error might remain latent in the node.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **RAS**.

I_{PSKL}

For an **Uncontainable** error, if the error cannot be isolated to an application, the system must be shut down by software to avoid **Catastrophic failure**.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **RAS**.

I_{LFCG}

For an **Unrecoverable** error, the application cannot continue and must be isolated by software methods.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **RAS**.

I_{PTXC}

For a **Recoverable** error, if software cannot locate and repair the error, the application must be isolated by software methods.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **RAS**.

I_{RSJP}

For a **Restartable** error, software might take action to locate and repair the error before it is consumed. The PE can be restarted by software without software taking any action to locate and repair the error.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - **RAS**.

R_{SJKL}

On each error exception, it is IMPLEMENTATION DEFINED whether the error has been contained or whether it is **Uncontainable**. If the error has been contained, it is further IMPLEMENTATION DEFINED whether the state of the PE on taking the error exception is **Unrecoverable**, **Recoverable**, or **Restartable**:

Uncontainable error (UC): The error is **Uncontainable** if it has been, or might have been, **silently propagated**. This is also referred to as an Uncontained error.

Unrecoverable error (UEU): The state of the PE is **Unrecoverable** if all of the following are true:

- The error has not been **silently propagated**.
- The PE cannot recover execution from the return address of the exception. This might be because of one of the following:
 - The error has been architecturally consumed by the PE and infected the state of the PE general-purpose registers, the Floating-point Extension register file, and System registers.
 - The exception is asynchronous.

Recoverable error (UER): The state of the PE is **Recoverable** if all of the following are true:

- The error has not been **silently propagated**.
- The error has not been architecturally consumed by the PE (the architectural state of the PE is not infected).
- The exception is synchronous and the PE can recover execution from the return address of the exception.

The PE cannot make correct progress without either consuming the error or otherwise making the error **Unrecoverable**. The error remains latent in the system.

Restartable error (UEO): The state of the PE is **Restartable** if all of the following are true:

- The error has not been **silently propagated**.
- The error has not been architecturally consumed by the PE (the architectural state of the PE is not infected).
- The exception is synchronous and the PE can recover execution from the return address of the exception.

The PE can make progress. However, the error might remain latent in the system.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

R_{KZPT}

The set of error types that can be reported by an implementation is IMPLEMENTATION DEFINED. An implementation can report:

- Any **Restartable** error as any of **Recoverable**, **Unrecoverable**, or **Uncontainable**.
- Any **Recoverable** error as either **Unrecoverable** or **Uncontainable**.
- Any **Unrecoverable** error as **Uncontainable**.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

I_{XCSK}

If the state of the PE is reported as **Recoverable**, this does not mean that the error can be recovered from. For example, because the error in memory might be one which does not allow software to recover the operation. Rather, software *might* be able to recover if it can repair the error and continue.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

R_{SGHS}

A bus error might be raised in response to:

- An architectural memory read, or reads from instruction fetches.
- An architectural write to memory, or cache maintenance operation.
- A read from memory because of hardware speculation, prefetching, or other non-architectural mechanisms.

The events that trigger bus errors are IMPLEMENTATION DEFINED.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

R_{QLCQ}

It is IMPLEMENTATION DEFINED whether a RAS error that is detected by the consumer of a write from a PE:

- Is deferred to the consumer.
- Is returned to the PE as a bus error.
- Generates an error recovery interrupt.

The behavior might vary by physical address or memory type.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

R_{TFLL} The method by which the error is deferred depends on the component and is implementation-specific.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

I_{RDQV} A common mechanism to defer the error is to create **Poisoned** state, which subsequently generates an error when that state is accessed.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

R_{CXSW} The size of the **Protection granule** for any implemented error detection mechanism is IMPLEMENTATION DEFINED, and a system might implement multiple error detection mechanisms with different **Protection granule** sizes.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

R_{GBDR} The mechanisms for clearing an error or poison from a **Protection granule** is IMPLEMENTATION DEFINED, and it is IMPLEMENTATION DEFINED whether any such mechanism exists.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

R_{PXMR} **BFSR** is populated for every BusFault that is generated by a RAS error. **RFSR** is populated for every RAS BusFault exception, and it is IMPLEMENTATION DEFINED whether it is also updated for other asynchronous BusFault exceptions. If **RFSR** is updated, **RFSR.V** is set.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

R_{FZKC} The severity of the error and the state of the PE are reported when the error exception is pended.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

I_{ZKNJ} If an error exception occurs while the BusFault exception handler is handling a previous error exception, the existing exception nesting behaviors apply. This might cause the exception to escalate to a HardFault or Lockup.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

R_{VWPS} A DAP request that causes a synchronous RAS error returns an error to the debugger, but does not update **RFSR**, **BFSR**, and **BFAR**.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

R_{DCKC} A DAP access that causes an asynchronous RAS error does not return an error to the debugger, but for a read access UNKNOWN data is returned. A BusFault might be generated and update **RFSR**, **BFSR**, and **BFAR**.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

B16.3.1 Error correction and deferment

R_{WXZD} Hardware corrects or defers an error if it can do so. The error is logged, and a fault handling interrupt is generated for fault handling purposes if the node is configured to do so.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.

B16.4 Error Synchronization Barrier (ESB)

I _{JLPC}	The RAS Extension introduces a new instruction, the Error Synchronization Barrier (ESB). <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{DNTZ}	ESB acts as a NOP when the system cannot synchronize RAS errors. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{DLKL}	ESB does not itself update any registers, but any ensuing RAS BusFault exception will update at least BFSR and RFSR. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{LNRD}	ESB might update syndrome bits, for example if IESBs are disabled. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{ZLCQ}	An ESB acts as a Data Synchronization Barrier when either: • There is a pending BusFault exception, which might have been forced to be recognized as a direct result of the call to SynchronizeBusFault(). • There is a pending HardFault exception, which might have been caused by an asynchronous BusFault which escalated due to the BusFault exception being disabled. This behavior guarantees that a subsequent load of the memory mapped syndrome registers and pending bits in BFSR, RFSR and SHCSR return the correct values. If a BusFault exception is recognized or escalated as a result of the ESB instruction affects the current Security state, then the values in the syndrome registers must be congruent with the current Security state. When determining whether the ESB instruction also acts as an implicit Data Synchronization Barrier, only the corresponding exception bits associated with the current Security state must be checked. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>

See also:

[B16.5 Implicit Error Synchronization \(IESB\)](#).

B16.4.1 ESB and Unrecoverable errors

R _{VDCF}	An ESB acts as a barrier to all unrecoverable (RAS-related and non-RAS related) bus errors and causes any Latent faults to be recognized synchronously with the instruction. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{MSMM}	Depending on the current state of the PE, recognizing any latent bus errors might simply result in a BusFault being pending, or the BusFault might be taken or escalated. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{GCBZ}	The Error Synchronization Barrier operation contains the error for the current software context. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>

B16.4.2 ESB and other containable errors

- I_{PHSB} For other types of [Containable](#) error:
- A [Recoverable](#) error has not yet been consumed by the PE.
 - [Restartable](#) and [Corrected](#) errors, and BusFault exceptions from reads by hardware speculation that do not corrupt the state of the PE, have not been consumed by the PE.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

- R_{WKMP} An unconsumed RAS error might be taken at the [ESB](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

B16.4.3 ESB and other errors

- R_{PHJW} Synchronous BusFault exceptions are not synchronized by an Error Synchronization Barrier.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

- R_{TXBQ} Interrupts are not synchronized by an [ESB](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

- R_{VSDB} An [ESB](#) instruction will synchronize asynchronous BusFaults.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

- R_{WRJG} An [ESB](#) always synchronizes [Containable](#) errors, but it is IMPLEMENTATION DEFINED whether IMPLEMENTATION DEFINED and uncategorized BusFault exceptions are [Uncontainable](#), and whether they can be synchronized by an Error Synchronization Barrier.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

- I_{TZTR} [Uncontainable](#) errors might not have been contained, and Uncontainable BusFault exceptions might be asynchronous. An [Uncontainable](#) error might be taken at the [ESB](#) but this is not architecturally required.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

- R_{TSGR} It is IMPLEMENTATION DEFINED whether IMPLEMENTATION DEFINED and uncategorized interrupts are [Containable](#) or [Uncontainable](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

- I_{HGSQ} An [Uncontainable](#) error might be taken at an Error Synchronization Barrier or recorded in [RFSR.IS](#) by an [ESB](#) instruction.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

- R_{SBFD} When an [ESB](#) instruction is executed the return address from any taken BusFault exception is the [ESB](#) instruction.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

B16.5 Implicit Error Synchronization (IESB)

I _{SSXC}	To ensure that faults arise in the appropriate PE state, an implicit error synchronization event (IESB) can optionally be inserted on every exception entry, exception return, and on lazy state stacking operations. Neither entry to, nor exit from, Debug state insert an IESB. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{ZGNM}	IESB only updates at least one of BFSR or RFSR if there was a latent asynchronous bus error that was synchronized. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{VTKM}	An IESB acts as a barrier to all Unrecoverable (RAS-related and non-RAS related) bus errors. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{GRKC}	Enabling IESBs causes all asynchronous BusFaults to escalate as if they were synchronous BusFaults, regardless of whether they were asynchronously recognized, or forced to be recognized by an ESB or IESB. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{NGQC}	Asynchronous BusFaults that escalate synchronously because IESBs are enabled are still reported as asynchronous faults. This means that the exception return address does not point to the instruction that caused the fault. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{WRDB}	AIRCR.IESB determines whether an exception entry or an exception return behaves as an implicit error synchronization event and requires any outstanding RAS exceptions to be acknowledged: • On exception entry, only errors relating to the background code and register stacking are affected by AIRCR.IESB. Any resulting BusFault from exception entry will be subject to the rules on derived exceptions. • On exception return, only errors relating to the handler code are affected by AIRCR.IESB. Register unstacking relates to the background code and is not affected by the implicit error synchronization barrier and are handled on return to Thread mode subject to the rules on exception priority. Any BusFault that occurs during exception return, before the transition to Thread mode, when AIRCR.IESB is one will be treated as a synchronous exception to the Handler. This ensures that the fault is associated with the context that triggered it. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{PDXH}	Arm recommends that implementations support the implicit error synchronization behavior when asynchronous exceptions and associated timings create the possibility of errors being associated with a context that is different to the one that caused the error. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{WJNK}	Writes by software to AIRCR.IESB can be ignored by the PE if it does not support configurable implicit ESB insertion. The value that is read back determines whether the feature is supported. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>

R_{LZWF}

When IESBs are enabled:

- An IESB occurs prior to stacking of the lazy floating point state, and any RAS errors are associated with foreground code.
- An IESB occurs after the stacking of the lazy floating point state, and any RAS errors are associated with the register stacking.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS** && (**FP** || **MVE**).*

R_{JMXC}

RAS faults set the first applicable syndrome information from the following list:

- **BFSR.LSPERR** if the error is attributable to lazy state preservation stacking.
- **BFSR.STKERR** if the error is attributable to exception register stacking.
- **BFSR.UNSTKERR** if the error is attributable to exception register unstacking.
- **BFSR.IBUSERR** if the error is caused by an instruction fetch or prefetch.
- **BFSR.PRECISERR** if the error occurred synchronously and is attributable to instruction execution.
- **BFSR.IMPRESERR** otherwise.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

B16.6 Fault handling

R _{QRPV}	There are four forms of RAS error reporting: • A bus error in response to an action from the PE. • An error recovery interrupt. • A Fault handling interrupt. • A critical error interrupt. Depending on the system configuration, the interrupts can be routed through a system interrupt controller or the NVIC. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{WPNL}	An error recovery interrupt can notify software of RAS faults that are detected in the system. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{LVWP}	A fault handling interrupt can notify software of RAS events that are detected in the system. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{JLPR}	When an error is detected by a node that supports fault reporting, the node records the error in Error record registers and generates a RAS fault handling interrupt, if configured to do so. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{XKVL}	The RAS fault handling interrupt might be routed at the system level to a PE that is not directly affected by the fault. Conversely, the PE might receive fault handling interrupts relating to other devices in the system. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{JTXP}	RAS fault handling interrupts might be sent to a dedicated fault handling PE by IMPLEMENTATION DEFINED means. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{SMXP}	The error recovery interrupts, fault handling interrupts, and critical error interrupts, are level sensitive or pulse sensitive in the same way as other interrupts. It is IMPLEMENTATION DEFINED whether a node employs level sensitive or pulse sensitive interrupts. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{GTMF}	The error recovery interrupts, fault handling interrupts, and critical error interrupts are pended in finite times after changes to the error conditions or to the fault being observed or corrected. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{NHGH}	Support for critical error conditions and critical error interrupts at a node is IMPLEMENTATION DEFINED. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{KLPK}	An example of a critical error is one where the node has entered a failure mode, which means that the primary error recovery mechanisms cannot be used. For example, if a memory controller enters a failure mode and stops handling memory requests from application processors, and application processors host the primary error recovery software, then the error is signaled to a secondary error controller that has its own private resources to log the error. The RAS Extension allows for a dedicated interrupt, called the critical error interrupt, to be generated by a node when such an error occurs. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{FWDD}	For a given node, the critical error interrupt is implemented if ERRFRn.CI != 0b00. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>

R _{HJHC}	For a given node, if the critical error interrupt is implemented, then the error recovery interrupt is also implemented. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{TCPP}	The critical error interrupt is enabled when ERRCTRLn.CI is set to 1. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{LLPF}	If the critical error interrupt is implemented, then when a critical error condition is recorded, the node sets ERRSTATUSn.CI to 1, regardless of whether the critical error interrupt is enabled or disabled. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{WDPT}	If the critical error interrupt is implemented and disabled, then when a critical error condition is detected, the node records the critical error as an Uncontainable error. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{XPHJ}	Classifying the critical error condition as an Uncontainable error if the critical error interrupt is not enabled has the effect of causing the node to generate an error recovery interrupt. The node also sets ERRSTATUSn.CI to 1. If the critical error interrupt is enabled, it is IMPLEMENTATION DEFINED how the error is classified at the node. The critical error flag is set to 1 in addition to the other syndrome information for the error, which is handled in the normal way. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>

See also:

[Chapter B12 Nested Vectored Interrupt Controller](#).

B16.7 RAS error records

I_{ZFWK} On encountering an error, a **node** writes to the RAS **Error records**. These records can then be analyzed by software to determine whether there are any systematic problems to be dealt with.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

R_{QDZN} Each **node** provides at least one **Error record** to control error reporting and expose status.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

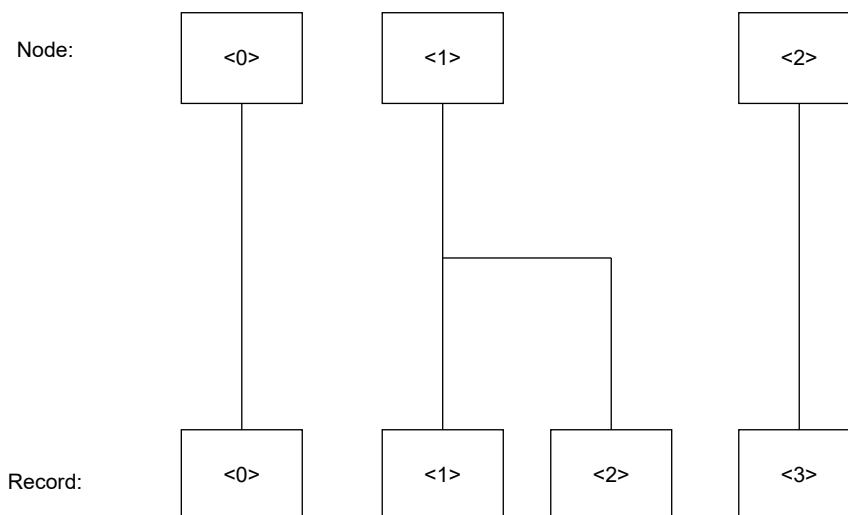
R_{FDFQ} A **node** can provide multiple **Error records** if it offers multiple, logically distinct functions.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

R_{STJR} If a **node** provides multiple **Error records**, these are serially indexed. Each record, other than the first record, has an **ERRFR_n** register that is RAZ/WI and the **ERRCTRL_n** register is RES0.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

I_{SDBZ} An example of a group containing four error records owned by three nodes is shown below:



In the diagram:

- Node<0> owns one error record: <0>.
- Node<1> owns two error records: <1> and <2>.
- Node<2> owns one error record: <3>.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

R_{JSWF} The standard **Error records** contain:

- Controls for common features, and an identification mechanism for these controls. For each **node** it is IMPLEMENTATION DEFINED whether the fault and error reporting mechanisms apply to both reads and writes, or whether they can be individually controlled for reads and writes.
- A status register for common status fields, such as the type and coarse characterization of the error.
- An address register, if applicable.
- IMPLEMENTATION DEFINED controls and identification registers.
- IMPLEMENTATION DEFINED status registers.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **RAS**.*

I _{WSCV}	Arm recommends that the IMPLEMENTATION DEFINED status registers in the standard Error record are used for: - Identifying a Field Replaceable Unit (FRU). - Locating the error within the FRU. - Optional Corrected Error counters for software to poll the rate of Corrected errors. The architecture provides optional formats for the counters. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{QDFJ}	The content and format of the Error records is flexible to allow implementations to select an appropriate amount of reporting. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{GVGT}	A group of Error records can be sparsely populated, which means that not all registers in the group might contain valid information. Error record registers that are not implemented have an associated ERRFRn register field that reads as zero. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{PJGD}	The number of Error records that can be accessed through the memory-mapped registers is IMPLEMENTATION DEFINED, and might be zero. ERRDEVID.NUM indicates the highest numbered index of the Error records that can be accessed, plus one. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{JVDC}	The content of the Error record registers is preserved over Warm reset. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
I _{SVLS}	Arm recommends that all Error records are remotely accessible for access by all PEs in a system, or by a Baseboard Management Controller (BMC) or System Control Processor (SCP) or debugger. The remote access mechanism is IMPLEMENTATION DEFINED but might use CoreSight-like interfaces. Arm recommends that remote access is possible when the rest of the system is in a fail state, for example when the system has locked up. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{PRWQ}	When a new error is detected, the node: - Sets or modifies ERRSTATUSn.{CE, DE, CI, UE, UET} to indicate the type of the new Detected error. - The node either: - Overwrites the Error record with the syndrome for the new error, if it has a higher priority than the previous highest priority recorded error. - Keeps the syndrome for the previous error, if the new error has a lower or the same priority as the previous highest priority recorded error. - Counts the error if it is a Corrected error and a counter is implemented. - Sets ERRSTATUSn.V to 1. - Generates an interrupt as required. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{XPXH}	A counter for Corrected errors is OPTIONAL. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>

R_{LJTP}

The overwriting of errors depends on the type of the previous highest priority error and the type of the newly recorded error. This is shown in the table below. The table uses the following abbreviations:

- **CE:** [Corrected error](#).
- **CO:** Count and overflow. Keep the previous error syndrome and count the error. If counting the error causes an unsigned overflow of the counter set [ERRSTATUSn.OF](#) to 1.
- **CW:** Count and overwrite. Count CE if a counter is implemented and overwrite. If a counter is implemented and overflows, [ERRSTATUSn.OF](#) is set to an UNKNOWN value. Otherwise, it is IMPLEMENTATION DEFINED whether [ERRSTATUSn.OF](#) is set to 0 or unchanged.
- **CWO:** Count and overwrite or keep. The behavior is IMPLEMENTATION DEFINED and described by the value of [ERRFRn.CEO](#):
 - 0:** Count CE if a counter is implemented and keep the previous error syndrome.
 - 1:** Count CE. If [ERRSTATUSn.OF](#) == 1 before the CE is counted, keep the previous syndrome. Otherwise record the new error syndrome.

If the counter overflows or if no counter is implemented [ERRSTATUSn.OF](#) is set to 1.

- **DE:** [Detected error](#).
- **O:** Overflow. Keep the previous error syndrome and set [ERRSTATUSn.OF](#) to 1.
- **UEO:** [Restartable error](#).
- **UER:** [Recoverable error](#).
- **UEU:** [Unrecoverable error](#).
- **UC:** [Uncontainable error](#).
- **WO:** Overwrite and overflow. [ERRSTATUSn.OF](#) is set to 1.

If no counter is implemented, CW behaves the same as W, and CWO and CO behave the same as O.

Previous error type	New detected error type					
	CE	DE	UEO	UER	UEU	UC
	CW	W	W	W	W	W
CE	CWO	WO	WO	WO	WO	WO
DE	CO	O	WO	WO	WO	WO
UEO	CO	O	O	WO	WO	WO
UER	CO	O	O	O	WO	WO
UEU	CO	O	O	O	O	WO
UC	CO	O	O	O	O	O

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [RAS](#).

R_{GPFK}

When a [node](#) generates an interrupt or exception, it ensures that any subsequent reads to the [Error records](#) return the updated values.

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [RAS](#).

R_{ZCKJ}

[ERRGSRn](#) contains a read-only copy of [ERRSTATUSn.V](#).

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - [RAS](#).

B16.8 Multiple BusFault exceptions

I _{RDHF}	Asynchronous BusFaults can be generated by multiple exception conditions. The architecture does not define relative priorities. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{SCCP}	It is IMPLEMENTATION DEFINED whether bus errors that were generated by multiple exception conditions are taken as a single BusFault exception. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{SWRS}	On taking a BusFault exception, whether for one or more BusFault exception conditions, the effects of the BusFault exception or exceptions on the state of the PE is reported in RFSR.UET as any one of the following: • Uncontained error. • Unrecoverable error. • Restartable error. • Recoverable error. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>
R _{JJCW}	An Error Synchronization Barrier operation requires that all Unrecoverable errors are synchronized. If there are multiple requests outstanding, they are all synchronized by a single Error Synchronization Barrier operation. <i>Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - RAS.</i>

B16.9 Minimal RAS implementation

I_{QJVV} The minimal RAS Extension is mandatory in an Armv8.1-M implementation. All additional RAS features are optional.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

I_{NPMM} The RAS Extension is designed to provide a low implementation cost for devices that have limited RAS support.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

I_{DHMH} The minimal implementation of the RAS Extension is:

- The [ESB](#) instruction. This might be implemented as a NOP for implementations that do not offer RAS-induced BusFaults.
- [RFSR](#). This might be implemented as RAZ/WI.
- [ERRDEVID](#). This might be implemented as RAZ/WI.
- [AIRC.RIESB](#) This might be implemented as RAZ/WI.
- [ID_PFR0.RAS](#) reads as a 0b0010.

This provides the necessary architectural support while offering no actual RAS reporting functionality.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

I_{GRQK} [Error record](#) registers that are not implemented are RES0. In a minimal implementation, there might be no [Error records](#).

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - [RAS](#).

Part C

Armv8-M Instruction Set

Chapter C1

Instruction Set Overview

This chapter provides a definition of the *instruction descriptions* contained in [Chapter C2 Instruction Specification](#). It contains the following sections:

- [C1.1 *Instruction set.*](#)
- [C1.2 *Format of instruction descriptions.*](#)
- [C1.3 *Conditional execution.*](#)
- [C1.4 *Instruction set encoding information.*](#)
- [C1.5 *Modified immediate constants.*](#)
- [C1.6 *NOP-compatible hint instructions.*](#)

C1.1 Instruction set

R_{NPFK}

There is one instruction set, called T32.

*Applies to an implementation of the architecture **Armv8.0-M** onward.*

See also:

[C1.4 Instruction set encoding information.](#)

[Chapter C2 Instruction Specification.](#)

C1.2 Format of instruction descriptions

I_{XQQV} Each instruction description in [Chapter C2 Instruction Specification](#) has the following content:

1. A title.
2. A short description.
3. The instruction encoding or encodings.
4. Any alias conditions, if applicable.
5. A list of the assembler symbols for the instruction.
6. Pseudocode describing how the instruction operates.
7. Notes, if applicable.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.2.1 The title

I_{RFFL} The title of an instruction description includes the base mnemonic or mnemonics for the instruction. This is part of the assembler syntax, for example SUB.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{BSWN} If different forms of an instruction use the same base mnemonic, each form has its own description. In this case, the title is the mnemonic followed by a short description of the instruction form in parentheses. This is most often used when an operand is an immediate value in one instruction form, but is a register in another form.

For example, in [Chapter C2 Instruction Specification](#) the Armv8-M Instruction Set there are the following titles for different forms of the ADD instruction:

- [ADD \(SP plus immediate\)](#)
- [ADD \(SP plus register\)](#)
- [ADD \(immediate\)](#)
- [ADD \(immediate to PC\)](#)
- [ADD \(register\)](#)

Applies to an implementation of the architecture Armv8.0-M onward.

I_{RKXC} Where an instruction has more than one variant, the descriptions might be combined, for example for [CDP](#) and [CDP2](#).

Applies to an implementation of the architecture Armv8.0-M onward.

C1.2.2 A short description

I_{QNXW} This briefly describes the function of the instruction. The short description is not a complete description of the instruction and must be read in conjunction with the instruction encoding, mnemonic, alias conditions, assembler symbols, pseudocode and any applicable notes.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.2.3 The instruction encoding or encodings

R_{LTJB} Instruction descriptions in this manual contain:

- An encoding section, containing one or more encoding diagrams, each followed by some decode pseudocode that:

1. Picks out any encoding-specific special cases.
 2. Translates the fields of the encoding into inputs for the common pseudocode of the instruction
- An operation section, containing common pseudocode that applies to all of the encodings being described. The Operation section pseudocode contains a call to the `EncodingSpecificOperations()` function which triggers the decode pseudocode, either at its start or only after a **Condition code check** performed by `if ConditionPassed() then`.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BDDV}

An encoding diagram specifies each bit of the instruction encoding as one of the following:

- A mandatory 0 or 1, represented in the diagram as 0 or 1. If the PE attempts to decode and execute the instruction and a bit does not have a mandatory value, the encoding corresponds to a different instruction.
- A *should be* 0 or *should be* 1, represented in the diagram as (0) or (1). If the PE attempts to decode and execute the instruction and a bit does not have the *should be* value, the instruction is CONSTRAINED UNPREDICTABLE.
- A named single bit or a bit in a named multi-bit field.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WHJJ}

If the instruction bit pattern of an instruction is executed with these fields not having the *should-be* values, one or more of the following must occur:

- The instruction is UNDEFINED.
- The instruction executes as a NOP.
- The instruction operates as if the bit had the *should-be* value.
- Any destination registers of the instruction become UNKNOWN.
- For an MVE instruction that includes a 3-bit field representing the MSBs of a 4-bit general-purpose source register specifier, that is immediately followed by a SBZ or SBO field, the register reads as an UNKNOWN value.
- For an MVE instruction that includes a 3-bit field representing the MSBs of a 4-bit general-purpose destination register specifier, that is immediately followed by a SBZ or SBO field, it is UNKNOWN whether the LSB selects the even or odd general-purpose destination register corresponding to the top 3-bit MSBs.
- For an instruction that is part of the PACBTI extension, any hard-coded operand register reads as an UNKNOWN value.

The exceptions to this rule are:

- The following instructions treat the *should-be* value hw2 bit 13 differently than the above:
 - LDM, LDMIA, LDMFD.
 - LDMDB, LDMEA.
 - POP (multiple registers).
 - PUSH (multiple registers).
 - STM, STMIA, STMEA.
 - STMDB, STMFD.

The behavior is CONSTRAINED UNPREDICTABLE and one of the following must occur:

- The instruction is UNDEFINED.
 - The instruction executes as NOP.
 - The instruction performs all the loads or stores using the specified addressing mode, but SP is UNKNOWN.
- In the SDIV and UDIV instructions, the *should-be* value is treated as a register field. If the *should-be* value is not 0b1111 then the behavior is CONSTRAINED UNPREDICTABLE and one of the following must occur:
 - The instruction is UNDEFINED.
 - The instruction executes as a NOP.
 - The instruction operates as if the field had the *should-be* value.
 - The instruction performs the divide operation but the register specified by the *should-be* value becomes UNKNOWN.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{RTFM} In certain versions of the architecture some instruction encodings contain SBO or SBZ fields or bits. If these encoding fields or bits are changed to hard bits, 1 or 0, the field or bit in the diagram will be enclosed in square brackets. The architecture version being implemented will dictate whether or not the implementation follows hard encodings or SBZ or SBO rules.
For example in [MOV \(register\)](#) bit 15 of the second halfword of the T3 encoding has changed from SBZ in Armv8.0-M to 0 in Armv8.1-M, and is represented as [0].

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BBZT} An encoding diagram matches an instruction if all mandatory bits are identical in the encoding diagram and the instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DKWV} Between each encoding diagram and its T <n> heading, there is an italicized statement that describes which *Armv8-M variant* the encoding is present in. For example, *Armv8-M Main Extension only*.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{JSBT} The instruction description shows the instruction encoding diagram, or, if the instruction has multiple encodings, shows all of the encoding diagrams. The heading for each encoding is the letter T followed by an arbitrary number, usually between 1 and 5.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{FQDP} Below each encoding diagram is the *assembler syntax prototype* for that encoding, written in typewriter font. The assembler syntax prototype describes the syntax that can be used in the assembler to select this encoding, and also the syntax that is used when disassembling this encoding.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{BLJR} In some cases an encoding has multiple variants of *assembler syntax prototype*, when the prototype differs depending on the value in one or more of the encoding fields. In these cases, the correct variant to use can be identified by either:

- Its subheading.
- An annotation to the syntax.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B7.5 Endianness.](#)

[C1.2.6 Pseudocode describing how the instruction operates.](#)

C1.2.4 Any alias conditions, if applicable

I_{BMHC} Alias conditions are an optional part of an instruction description. If included, it describes the set of conditions for which an alternative mnemonic and its associated assembler syntax prototypes are preferred for disassembly by a disassembler. It includes a link to the alias instruction description that defines the alternative syntax. The alias syntax and the original syntax can be used interchangeably in the assembler source code.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{RBCM} Arm recommends that if a disassembler outputs the alias syntax, it consistently outputs the alias syntax.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{ZJKQ} Arm recommends that where possible, the alias is used.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.2.5 Standard assembler syntax fields

I_{RHCC} This manual uses the Arm *Unified Assembler Language* (UAL). This assembly language syntax provides a canonical form for all T32 instructions.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{LBNB} **UAL** describes the syntax for the mnemonic and the operands of each instruction. Operands can also be referred to as *Assembler symbols*. In addition, **UAL** assumes that instructions and data items can be given labels. It does not specify the syntax to be used for labels, see the assembler documentation for these details.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{DPLM} The *Assembler symbols* subsection of an instruction description contains a list of the symbols that the assembler syntax prototype or prototypes use.

The following conventions are used:

< >: Angle brackets. Any symbol enclosed by these is mandatory. For each symbol, there is a description of what the symbol represents. The description usually also specifies which encoding field or fields encodes the symbol.

{ }: Brace brackets. Any symbol enclosed by these is optional. For each optional symbol, there is a description of what the symbol represents and how its presence or absence is encoded.

In some assembler syntax prototypes, some brace brackets are mandatory, for example if they surround a register list. When the use of brace brackets is mandatory, they are separated from other syntax items by one or more spaces.

: Usually precedes a numeric constant. All uses of # are optional in assembler source code. Arm recommends that disassemblers output the # where the assembler syntax prototype includes it.

+/-: Indicates an optional + or – sign. If neither is coded, + is assumed.

! : Indicates that the result address is written back to the base register.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MBQS} Single spaces are used for clarity, to separate syntax items. Where a space is mandatory, the assembler syntax prototype shows two or more consecutive spaces.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{SXWN} Any characters not shown in this conventions list must be coded exactly as shown in the assembler syntax prototype. Apart from brace brackets, these characters are used as part of a meta-language to define the architectural assembler syntax prototype for an instruction encoding, but have no architecturally defined significance in the input to an assembler or in the output from a disassembler.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{QZDB} **UAL** includes *instruction selection* rules that specify which instruction encoding is selected when more than one can provide the required functionality. The following assembler syntax prototype fields are standard across all or most instructions:

<c>: Specifies the condition under which the instruction is executed. If <c> is omitted, it defaults to *always* (AL).

<q>: Specifies one of the following optional assembler qualifiers on the instruction:

.N

Meaning narrow. The assembler must select a 16-bit encoding for the instruction. If this is not possible, an assembler error is produced.

.W

Meaning wide. The assembler must select a 32-bit encoding for the instruction. If this is not possible, an assembler error is produced.

If neither `.W` nor `.N` is specified, the assembler can select either a 16-bit or 32-bit encoding. If both encoding lengths are available, it must select a 16-bit encoding. In the few cases where more than one encoding of the same length is available for an instruction, the rules for selecting between them are instruction-specific and are part of the instruction description.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{HGFS}

The following assembler syntax prototype field is standard across MVE instructions subject to VPT prediction:

`<v>`: Specifies a VPT predication block. This field is only available in implementations that include MVE, and it is only available inside a VPT block. Inside a VPT block, `<v>` can have one of the following values:

T

Indicates that the instruction is in the THEN section of a VPT block.

E

Indicates that the instruction is in the ELSE section of a VPT block.

`<v>` does not affect the encoding of the instruction, and only highlights to the programmer that some of the vector lanes might be masked because of VPT predication. The use of T or E outside a VPT block produces an assembler error.

Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - MVE.

R_{BGFV}

The `<dt>` field normally contains one data type specifier. Unless the assembler syntax description for the instruction indicates otherwise, `<dt>` indicates the data type contained in:

- The second operand, if any.
- The operand, if there is no second operand.
- The result, if there are no operand registers.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

R_{NKRJ}

The `<dt>` field is usually specified as a single field. However, it can and might be specified as a concatenation of two or more fields.

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - FP || MVE.

R_{JSCH}

There is some flexibility in the data type specifier syntax:

- Three data types can be specified, the results and both operands data types.
`VMULL.P16.P8.P8, Q0, Q1, Q3` instead of `VMULL.P8, Q0, Q1, Q3`.
- Two data types can be specified, the data types of the two operands. In this case the result data type of the result is implied by the shape of the instruction.
`VMULL.P8.P8, Q0, Q1, Q3` instead of `VMULL.P8, Q0, Q1, Q3`.
- Two data types can be specified, the data types of the single operand and the destination.
`VMOVN.I16.I32 Q0, Q1` instead of `VMOVN.I32 Q0, Q1`
- Where an instruction does not specify a data type a data type can be specified for the instruction.
- A more specific data type can be used than the one specified by the instruction.
- The F32 data type can be abbreviated to F.
- The F64 data type can be abbreviated to D.

Specified data type		Permitted more specific data types			
None		Any			
.I<size>	-	.S<size>	.U<size>	-	-
.8	.I8	.S8	.U8	.P8	-
.16	.I16	.S16	.U16	.P16	.F16
.32	.I32	.S32	.U32	-	.F32 or .F
.64	.I64	.S64	.U64	-	.F64 or .D

Applies to an implementation of the architecture *Armv8.0-M* onward. The extension requirements are - **FP** || **MVE**.

I_{BWNR}

Syntax options exist to override the normal instruction selection rules and ensure that a particular encoding is selected. These are useful when disassembling code, to ensure that subsequent assembly produces the original code, and in some other situations.

Applies to an implementation of the architecture *Armv8.0-M* onward.

See also:

[B5.5.2 VPT predication](#).

Applies to an implementation of the architecture *Armv8.1-M* onward.

C1.2.6 Pseudocode describing how the instruction operates

I_{RTDZ}

Each instruction description includes pseudocode that provides a precise description of what the instruction does.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{LRFZ}

In the instruction pseudocode, instruction fields are referred to by the names shown in the encoding diagram for the instruction.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{NLPM}

Where the pseudocode describes UNPREDICTABLE behavior the constraints on that behavior are described in the Operation section.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{BNVW}

Pseudocode does not describe the ordering requirements when an instruction generates multiple memory accesses.

Applies to an implementation of the architecture *Armv8.0-M* onward.

R_{CRWM}

Pseudocode describes the exact rules when an UNDEFINED instruction fails its [Condition code check](#).

In such cases, the UNDEFINED pseudocode statement lies inside the if `ConditionPassed()` then ... structure, either directly or in the `EncodingSpecificOperations()` function call, and so the pseudocode indicates that the instruction executes as a NOP.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{MZKZ}

Pseudocode does not describe the exact ordering requirements when a single floating-point instruction generates more than one floating-point exception and one or more of those floating-point exceptions is trapped.

Applies to an implementation of the architecture *Armv8.0-M* onward.

I_{JMFG}

An exception can be taken during execution of the pseudocode for an instruction, either explicitly as a result of the execution of a pseudocode function, or implicitly, for example if an interrupt is taken during execution of an [LDM](#) instruction. If this happens, the pseudocode does not describe the extent to which the normal behavior of the instruction occurs.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[Chapter E1 Arm Pseudocode Definition.](#)

[B7.2.3 Ordering and observability.](#)

[E1.1.1 General limitations of Arm pseudocode.](#)

[C1.3.3 Conditional execution of undefined instructions.](#)

[B4.12 Priority of Floating-point exceptions relative to other Floating-point exceptions.](#)

[B3.18 Exception handling.](#)

[B3.22 Exception return.](#)

C1.2.7 Use of labels in UAL instruction syntax

I_{BFJV}

The [UAL](#) syntax for some instructions includes the label of an instruction or a literal data item that is at a fixed offset from the instruction being specified. The assembler must:

1. Calculate the `PC` or `Align(PC, 4)` value of the instruction. The `PC` value of an instruction is its address plus 4 for a T32 instruction. The `Align(PC, 4)` value of an instruction is its `PC` value ANDed with `0xFFFFFFFFC` to force it to be word-aligned.
2. Calculate the offset from the `PC` or `Align(PC, 4)` value of the instruction to the address of the labeled instruction or literal data item.
3. Assemble a *PC-relative* encoding of the instruction, that is, one that reads its `PC` or `Align(PC, 4)` value and adds the calculated offset to form the required address.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{TCVF}

For instructions that encode a subtraction operation, if the instruction cannot encode the calculated offset, but can encode minus the calculated offset, the instruction encoding specifies a subtraction of minus the calculated offset.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WDSH}

The following special handling of the `<label>` encoding applies for the instruction types listed below:

For Load instructions, and their aliases, that use the `PC` as a base register:

- When the assembler calculates an offset of zero for the normal syntax of these instructions, it must assemble an encoding that adds zero to the `Align(PC, 4)` value of the instruction. Encodings that subtract zero from the `Align(PC, 4)` value cannot be specified by the normal syntax.
- There is an alternative syntax for these instructions that specifies the addition or subtraction and the immediate offset explicitly. In this syntax, the label is replaced by `[PC, #+/-<imm>]`, where:
 - `+/-`: If `+` or omitted is used to specify that the immediate offset is to be added to the `Align(PC, 4)` value, or `-` if the immediate offset is to be subtracted from the `Align(PC, 4)` value.
 - `<imm>`: Is the immediate offset.
- This alternative syntax makes it possible to assemble the encodings that subtract zero from the `Align(PC, 4)` value, and to disassemble them to a syntax that can be re-assembled correctly.
- [ADR](#):

- When the assembler calculates an offset of zero for the normal syntax of this instruction, it must assemble the encoding that adds zero to the `Align(PC, 4)` value of the instruction. The encoding that subtracts from the `Align(PC, 4)` value cannot be specified by the normal syntax.
- There is an alternative syntax for this instruction that specifies the addition or subtraction and the **immediate value** explicitly, by writing them as additions `ADD <Rd>, PC, #<imm>` or subtractions `SUB <Rd>, PC, #<imm>`. This alternative syntax makes it possible to assemble the encoding that subtracts zero from the `Align(PC, 4)` value, and to disassemble it to a syntax that can be re-assembled correctly.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.2.8 Using syntax information

`IBJGX`

For a particular encoding:

- There is usually more than one assembler syntax prototype variant that assembles to it.
- The exact set of prototype variants that assemble to it usually depends on the operands to the instruction, for example the register numbers or immediate constants. As an example, for the `AND (register)` instruction, the syntax `AND R0, R0, R8` selects a 32-bit encoding, but `AND R0, R0, R1` selects a 16-bit encoding.

Applies to an implementation of the architecture Armv8.0-M onward.

`IHQSS`

For each instruction encoding that belongs to a target instruction set, an assembler can use the information in the encoding to determine whether it can use that particular encoding to encode the instruction requested by the `UAL` source. If multiple encodings can encode the instruction, then:

- If both a 16-bit encoding and a 32-bit encoding can encode the instruction, the architecturally preferred encoding is the 16-bit encoding. This means that the assembler must use the 16-bit encoding instead of the 32-bit encoding.
- If multiple encodings of the same width can encode the instruction, the assembler syntax indicates the preferred encoding, and how software can select other encodings if required. Each encoding also documents `UAL` syntax that selects it in preference to any other encoding. If no encodings of the target instruction set can encode the instruction requested by the `UAL` source, the assembler normally generates an error that indicates that the instruction is not available in that instruction set.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.3 Conditional execution

I_{XDMQ} *Conditionally executed* means that the instruction only has its normal effect on the programmers' model operation, memory and coprocessors if the N, Z, C, and V flags in the [APSR](#) satisfy a condition specified in the instruction. If the flags do not satisfy this condition, the instruction acts as a NOP, that is, execution advances to the next instruction as normal, including any relevant checks for exceptions being taken, but has no other effect.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{SPPQ} Most T32 instructions are unconditional. Conditional execution in T32 code can be achieved using any of the following instructions:

- A 16-bit conditional branch instruction, with a branch range of -256 to +254 bytes. See [B](#) for details.
- A 32-bit conditional branch instruction, with a branch range of approximately $\pm 1\text{MB}$. See [B](#) for details.
- 16-bit Compare and Branch on Zero and Compare and Branch on Nonzero instructions, with a branch range of +4 to +130 bytes. See [CBNZ](#), [CBZ](#) for details.
- A 16-bit If-Then instruction that makes up to four following instructions conditional. See [IT](#) for details. The instructions that are made conditional by an IT instruction are called its *IT block*. Instructions in an IT block must either all have the same condition, or some can have one condition, and others can have the inverse condition.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{FNBQ} In T32 instructions, the condition (if it is not [AL](#)) is encoded in a preceding IT instruction, other than [B](#), [CBNZ](#) and [CBZ](#). Some conditional branch instructions do not require a preceding IT instruction, and include a condition code in their encoding.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{BDMC} The following table shows the conditions that are available for conditionally executed instructions.

cond	Mnemonic extension	Meaning, integer arithmetic	Meaning, Floating-point arithmetic	APSR condition flags
0000	EQ	Equal	Equal	Z == 1
0001	NE	Not equal	Not equal, or unordered	Z == 0
0010	CS	Carry set	Greater than, equal or unordered	C == 1
0011	CC	Carry clear	Less than	C == 0
0100	MI	Minus, negative	Less than	N == 1
0101	PL	Plus, positive or zero	Greater than, equal or unordered	N == 0
0110	VS	Overflow	Unordered	V == 1
0111	VC	No overflow	Not unordered	V == 0
1000	HI	Unsigned higher	Greater than or unordered	C == 1 and Z == 0
1001	LS	Unsigned lower or same	Less than or equal	C == 0 or Z == 1
1010	GE	Signed greater than or equal	Greater than or equal	N == V
1011	LT	Signed less than	Less than or unordered	N != V
1100	GT	Signed greater than	Greater than	Z == 0 and N == V
1101	LE	Signed less than or equal	Less than, equal or unordered	Z == 1 or N != V
1110	None (AL)	Always (unconditional)	Always (unconditional)	Any

Unordered means at least one NaN operand.

HS (unsigned higher or same) is a synonym for CS.

LO (unsigned lower) is a synonym for CC.

AL is an optional mnemonic extension for always, except in IT instructions. See IT for details.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.3.1 Conditional instructions

R_{WRJS} The instructions that are made conditional by an IT instruction must be written with a condition after the mnemonic. These conditions must match the conditions imposed by the IT instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{WVXC} An example of R_{WRJS} is:

```
1      ITTEE EQ
2      ADDEQ R0, R1
3      SUBEQ R2, R3
4      ADDNE R4, R5
5      SUBNE R6, R7
```

Applies to an implementation of the architecture Armv8.0-M onward.

R_{THGJ} Some instructions cannot be made conditional by an IT instruction. Some instructions can be conditional if they are the last instruction in the IT block, but not otherwise, see the individual instruction descriptions for details.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{TGXF} If the assembler syntax indicates a conditional branch that correctly matches a preceding IT instruction, it is assembled using a branch instruction encoding that does not include a condition field.

Applies to an implementation of the architecture Armv8.0-M onward.

See also

- IT instruction

C1.3.2 Pseudocode details of conditional execution

R_{NMVJ} The `CurrentCond()` pseudocode function prototype returns a 4-bit condition specifier as follows:

- For the T1 and T3 encodings of the Branch instruction, it returns the 4-bit `cond` field of the encoding.
- For all other T32 instructions:
 - If `ITSTATE.IT[3:0] != '0000'` it returns `ITSTATE.IT[7:4]`
 - If `ITSTATE.IT[7:0] == '00000000'` it returns `'1110'`
 - Otherwise, execution of the instruction is UNPREDICTABLE.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{LTPQ} The `ConditionPassed()` function calls the `ConditionHolds()` function to determine whether the instruction must be executed.

Applies to an implementation of the architecture Armv8.0-M onward.

See also

[C1.3.5 ITSTATE.](#)

[B.](#)

C1.3.3 Conditional execution of undefined instructions

R_{NPNF} The conditional execution applies to all instructions. This includes undefined instructions and other instructions that would cause entry to the UsageFault or the UNDEFINSTR UsageFault.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{PCJZ} If such an instruction fails its [condition code check](#) the instruction behaves as a NOP and does not cause an UsageFault.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.3.4 Interaction of undefined instruction behavior with UNPREDICTABLE or CONSTRAINED_UNPREDICTABLE instruction behavior

R_{NZWQ} If this manual describes an instruction as both:

- UNPREDICTABLE and UNDEFINED, then the instruction is UNPREDICTABLE.
- CONSTRAINED UNPREDICTABLE and UNDEFINED, then the instruction is CONSTRAINED UNPREDICTABLE.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.3.5 ITSTATE

I_{RGFT} ITSTATE is held in [EPSR.IT](#).

This register holds the If-Then Execution state bits for the T32 [IT](#) instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{QKPG} [EPSR.IT](#) and ITSTATE divide into two subfields:

IT[7:5]

Holds the *base condition* for the current IT block. The base condition is the top 3 bits of the condition specified by the IT instruction.

This subfield is 0b000 when no IT block is active.

IT[4:0]

Encodes:

- * The size of the IT block. This is the number of instructions that are to be conditionally executed. The size of the block is indicated by the position of the least significant 1 in this field which is bit [4-size of the block].
- * The value of the least significant bit, bit[0], of the condition code for each instruction in the block.
- * Changing the value of the least significant bit of a condition code from 0 to 1 inverts the condition code. For example cond 0000 is EQ, and cond 0001 is NE.

This subfield is 0b00000 when no IT block is active.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{FPNH} When an IT instruction is executed, IT bits[7:0] are set according to the condition in the instruction, and the *Then* and *Else* (T and E) parameters in the instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{XKGZ} An instruction in an IT block is conditional. The condition used is the current value of IT[7:4]. When an instruction in an IT block completes its execution normally, ITSTATE is advanced by shifting IT bits[4:0] left by 1 bit.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{VQJM} For example:

	IT[7:5]	IT[4:0]
ITTEE EQ	000	00111
ADDEQ R0, R1	000	01110
SUBEQ R2, R3	000	11100
ADDNE R4, R5	000	11000
SUBNE R6, R7	000	00000

Applies to an implementation of the architecture Armv8.0-M onward.

I_{KQBQ} Instructions that can complete their normal execution by branching are only permitted in an IT block as its last instruction, and so always result in ITSTATE advancing to normal execution.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{FJLN} In the following table, *P* represents the base condition or the inverse of the base condition.

IT Bits						
[7:5]	[4]	[3]	[2]	[1]	[0]	
cond_base	P1	P2	P3	P4	1	Entry point for 4-instruction IT block
cond_base	P1	P2	P3	1	0	Entry point for 3-instruction IT block
cond_base	P1	P2	1	0	0	Entry point for 2-instruction IT block
cond_base	P1	1	0	0	0	Entry point for 1-instruction IT block
000	0	0	0	0	0	Normal execution, not in an IT block

Combinations of the IT bits not shown in this table are reserved.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.3.6 Pseudocode details of ITSTATE operation

I_{JLKP} `ITAdvance()` describes how ITSTATE advances after normal execution.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{ZGZL} `InITBlock()` and `LastInITBlock()` test whether the current instruction is in an IT block, and whether it is the last instruction of an IT block.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.3.7 SVC and ITSTATE

R_{ZRFL} The ReturnAddress for an **SVC** instruction must point to the instruction directly after the **SVC** instruction, that is the address of the **SVC** instruction plus two. The execution of an **SVC** causes ITSTATE to advance.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{QWZW} When an **SVC** instruction is escalated to HardFault resulting in **lookup** the ReturnAddress is 0xEFFFFFFE.

Applies to an implementation of the architecture Armv8.0-M onward.

See also:

[B3.33 Lookup](#).

C1.3.8 CONSTRAINED_UNPREDICTABLE behavior and IT blocks

R_{WWVX} Branching into an IT block, other than by way of exception return or exit from Debug state, leads to CONSTRAINED UNPREDICTABLE behavior. Execution starts from the address that is determined by the branch, but each instruction in the IT block is:

- Executed as if the instruction is not in an IT block, meaning that the instruction is executed unconditionally.
- Executed as if the instruction had passed its [Condition code check](#) within an IT block.
- Executed as a NOP. That is, the instruction **behaves as if** it had failed the Condition code check.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CPDC} For exception returns or Debug state exits that cause **EPSR.IT** to be set to a reserved value with a nonzero value in **EPSR.IT**, the **EPSR.IT** bits are forced to 0b00000000.

Applies to an implementation of the architecture Armv8.0-M onward. Note, Debug state requires HDBG.

R_{HVNS} Exception returns or Debug state exits that set **EPSR.IT** to a non-reserved value can occur when the flow of execution returns to a point:

- Outside an IT block, but with the **EPSR.IT** bits set to a value other than 0b00000000.
- Inside an IT block, but with a different value of the **EPSR.IT** bits than if the IT block had been executed without an exception return or Debug state exit.

In this case the instructions at the target of the exception return or Debug state exit does one of the following:

- Execute as if they passed the Condition code check for the remaining iterations of the **EPSR.IT** state machine.
- Execute as NOPs. That is, they **behave as if** they failed the Condition code check for the remaining iterations of the **EPSR.IT** state machine.

Applies to an implementation of the architecture Armv8.0-M onward. Note, Debug state requires HDBG.

R_{LLDK} A number of instructions in the architecture are described as being CONSTRAINED UNPREDICTABLE either:

- Anywhere within an IT block.
- As an instruction within an IT block, other than the last instruction within an IT block.

Unless otherwise stated in this reference manual, when these instructions are committed for execution, one of the following occurs:

- An UNDEFINED exception is taken.
- The instructions are executed as if they had passed the [condition code check](#).
- The instructions execute as NOPs, as if they had failed the [condition code check](#).

Applies to an implementation of the architecture Armv8.0-M onward.

I_{NJKE} The behavior might in some implementations vary from instruction to instruction, or between different instances of the same instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{BWMN} Branch instructions or other non-sequential instructions that change the **PC** are CONSTRAINED UNPREDICTABLE in an IT block. Where these instructions are not treated as UNDEFINED within an IT block, the remaining iterations of the **EPSR.IT** state machine is treated in one of the following ways:

- **EPSR.IT** is cleared to 0.
- **EPSR.IT** advances for either a sequential or a nonsequential change of the **PC** in the same way as it does for instructions that are not CONSTRAINED UNPREDICTABLE that cause a sequential change of the **PC**.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{XHBL} This behavior does not apply to an instruction that is the last instruction in an IT block.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{TMWN} The instructions that are addressed by the updated **PC** does one of the following:

- Execute as if they had passed the **condition code check** for the remaining iterations of the **EPSR.IT** state machine.
- Execute as **NOPS**. That is, they behave as if they had failed the **condition code check** for the remaining iterations of the **EPSR.IT** state machine.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{KVXD} The remaining iterations of the **EPSR.IT** state machine behave in one of the following ways:

- The **EPSR.IT** state machine advances as if it were in an IT block.
- The **EPSR.IT** bits are ignored.
- The **EPSR.IT** bits are forced to 0b00000000.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GZBX} Execution of an instruction inside an IT block with ITSTATE set to zero, an ICI value, or a value that is inconsistent with the IT block is UNPREDICTABLE.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{VBCG} In the **VIWDUP** and **VDWDUP** instructions the following conditions result in CONSTRAINED UNPREDICTABLE behavior:

- Rn is not a multiple of imm.
- Rm is not a multiple of imm.
- Rn >= Rm.

The CONSTRAINED UNPREDICTABLE behavior is that the resulting values of Rn and Qd become UNKNOWN.

*Applies to an implementation of the architecture Armv8.1-M onward. The extension requirements are - **MVE**.*

See also:

[B3.5 XPSR, APSR, IPSR, and EPSR.](#)

[B3.5.2 Execution Program Status Register \(EPSR\).](#)

C1.4 Instruction set encoding information

C1.4.1 UNDEFINED and UNPREDICTABLE instruction set space

I_{FLRZ}	An attempt to execute an unallocated instruction results in either: • UNPREDICTABLE behavior. The instruction is described as UNPREDICTABLE. • An UNDEFINSTR UsageFault. The instruction is described as UNDEFINED. • Unallocated instructions in the NOP hint space behave as NOPs. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{KDXB}	An instruction is UNDEFINED if it is declared as UNDEFINED in an instruction description. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{JMVZ}	An instruction is UNPREDICTABLE or CONSTRAINED UNPREDICTABLE if: • A bit marked (0) or (1) in the encoding diagram of an instruction is not 0 or 1, respectively, and the pseudocode for that encoding does not indicate that a different special case applies. • It is declared as UNPREDICTABLE or CONSTRAINED UNPREDICTABLE in an instruction description. Details of the constraints on the behavior of an instruction, if it is defined as CONSTRAINED UNPREDICTABLE, are contained in the relevant instruction descriptions. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{TRHK}	Unless otherwise specified, a T32 instruction that is provided by one or more of the architecture extensions is either UNPREDICTABLE or UNDEFINED in an implementation that does not include those extensions. See the individual instruction descriptions for details. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>

C1.4.2 Pseudocode descriptions of operations on general-purpose registers and the PC

R_{HRGP}	In pseudocode, the uses of the R[] function are: • Reading or writing R0-R12, SP, and LR, using n = 0-12, 13, and 14 respectively. • Reading the PC, using n = 15. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{CHTM}	The use of RSPCheck() returns the value of the current SP <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R_{GXHR}	The function RZ returns zeros if the PC is called. <i>Applies to an implementation of the architecture Armv8.1-M onward.</i> See also: R[], RSPCheck() RZ <i>Applies to an implementation of the architecture Armv8.1-M onward.</i>

C1.4.3 Use of 0b1111 as a register specifier

R _{WMVJ}	All use of the PC as a named register specifier for a source register that is described as CONSTRAINED UNPREDICTABLE in the pseudocode or in other places in this reference manual does one of the following: • Cause the instruction to be treated as UNDEFINED. • Cause the instruction to be executed as a NOP. • Read an UNKNOWN value for the source register that is specified as the PC. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{BGJG}	All use of the PC as a named register specifier for a destination register that is described as CONSTRAINED UNPREDICTABLE in the pseudocode or in other places in this reference manual does one of the following: • Cause the instruction to be treated as UNDEFINED. • Cause the instruction to be executed as a NOP. • Ignore the write. • Branch to an UNKNOWN location. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{QVWL}	The choice between the behavior of the PC as a source or destination register might vary in some implementations from instruction to instruction, or between different instances of the same instruction. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{LXPR}	For instructions that specify two destination registers and if one is specified as the PC, then the other destination register of the pair is UNKNOWN. The CONSTRAINED UNPREDICTABLE behavior for the write to the PC is either to ignore the write or to branch to an UNKNOWN location. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{DRSS}	An instruction that specifies the PC as a Base register and specifies a base register write-back is CONSTRAINED UNPREDICTABLE and behaves as if the PC is both the source and destination register. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XLVX}	For instructions that affect any or all of APSR.{N, Z, C, V} or APSR.GE when the register specifier is not the PC, any flags that are affected by an instruction that is CONSTRAINED UNPREDICTABLE become UNKNOWN. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{JFGT}	For MRC instructions that use the PC as the destination register descriptor (and therefore target APSR.{N, Z, C, V}) and where these instructions are described as being CONSTRAINED UNPREDICTABLE the status of the flags becomes UNKNOWN. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{XPBT}	Multi-access instructions that load the PC from Device memory are CONSTRAINED UNPREDICTABLE and one of the following behaviors occurs: • The instruction loads the PC from the memory location as if the memory location had the Normal Non-cacheable attribute • The instruction generates a IACCVIOL MemManage fault. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
I _{QXNP}	When a value of 0b1111 is permitted as a register specifier, as indicated in the individual instruction descriptions, a variety of meanings is possible. For register reads, these meanings are: • Read the PC value, that is, the address of the current instruction + 4. The base register of the table branch instructions TBB and TBH can be the PC. This enables branch tables to be placed in memory immediately after the instruction. (Some instructions read the PC value implicitly, without the use of a register specifier, for example the conditional branch instruction B<cond>.)

- Read the word-aligned **PC** value, that is, the address of the current instruction + 4, with bits [1:0] forced to zero. The base register of **LDC**, **LDR**, **LDRB**, **LDRD** (pre-indexed, no write-back), **LDRH**, **LDRSB**, and **LDRSH** instructions can be the word-aligned **PC**. This enables PC-relative data addressing. In addition, some encodings of the **ADD** and **SUB** instructions permit their source registers to be **0b1111** for the same purpose.
- Read zero. This is done in some cases when one instruction is a special case of another, more general instruction, but with one operand zero. In these cases, the instructions are listed on separate pages, with a special case in the pseudocode for the more general instruction cross-referencing the other page. An example of this is the descriptions of **MOV (register)** and **ORR (register)**.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{KVHQ}

When a value of **0b1111** is permitted as a register specifier, as indicated in the individual instruction descriptions, a variety of meanings is possible. For register writes, these meanings are:

- The **PC** can be specified as the destination register of an **LDR** instruction. This is done by encoding **Rt** as **0b1111**. The loaded value is treated as an address, and the effect of execution is a branch to that address. bit[0] of the loaded value selects the Execution state after the branch and must have the value 1.
- Discard the result of a calculation. This is done in some cases when one instruction is a special case of another, more general instruction, but with the result discarded. In these cases, the instructions are listed on separate pages, with a special case in the pseudocode for the more general instruction cross-referencing the other page. An example of this is the descriptions of **TST (register)** and **AND (register)**.
- If the destination register specifier of an **LDRB**, **LDRH**, **LDRSB**, or **LDRSH** instruction is **0b1111**, the instruction is a memory hint instead of a load operation.
- If the destination register specifier of an **MRC** instruction is **0b1111**, bits[31:28] of the value transferred from the coprocessor are written to the **N**, **Z**, **C**, and **V** flags in the **APSR**, and bits[27:0] are discarded.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{PLFG}

Some instructions permit the use of a *zero register* (**ZR**) as a scalar source operand, as indicated in the individual instruction descriptions. **ZR** is encoded as the value **0b1111** when a 4-bit register specifier is used. **ZR** is **RAZ/WI**.

Applies to an implementation of the architecture Armv8.1-M onward.

C1.4.4 Use of 0b1101 as a register specifier

SP[1:0] definition

R_{DSDB}

Bits [1:0] of **SP** must be treated as **SBZP** (Should Be Zero or Preserved). Writing a nonzero value to bits [1:0] results in UNPREDICTABLE behavior. Reading bits [1:0] returns zero.

Applies to an implementation of the architecture Armv8.0-M onward.

32-bit T32 instruction support for SP

R_{SKNR}

Use of the **SP** in T32 instructions and 16-bit data processing instructions is restricted to the following cases:

- **SP** as the source or destination register of a **MOV** instruction. Only register to register transfers without shifts are supported, with no flag setting:

1	MOV	SP , Rm
2	MOV	Rn , SP

- Adjusting **SP** up or down by a multiple of its alignment:

1	ADD { W }	SP , SP , #N	; For N a multiple of 4
2	SUB { W }	SP , SP , #N	; For N a multiple of 4
3	ADD	SP , SP , Rm , LSL #shft	; For shft=0,1,2,3
4	SUB	SP , SP , Rm , LSL #shft	; For shft=0,1,2,3

- **SP** as a base register, R_n , of any load or store instruction. This supports **SP**-based addressing for load, store, or memory hint instructions, with positive or negative offsets, with and without write-back.
- **SP** as the first operand, R_n , in any `ADD{S}`, `CMN`, `CMP`, or `SUB{S}` instruction. The add and subtract instructions support **SP**-based address generation, with the address going into a general-purpose register. `CMN` and `CMP` can check the stack pointer.
- **SP** as the transferred register, R_t , in any `LDR` or `STR` instruction.
- **SP** as the address in a `POP` or `PUSH` instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{MRNT}

Where an instruction states that the **SP** is UNPREDICTABLE and **SP** is used:

- The value that is read or written from or to the **SP** is UNKNOWN.
- The instruction is permitted to be treated as UNDEFINED.
- If the **SP** is being written, it is UNKNOWN whether a stack limit check is applied.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.4.5 16-bit T32 instruction support for SP

R_{STHZ}

Arm deprecates any other use of the **SP** in 16-bit T32 instructions. This affects the high register forms of `CMP` and `ADD`, where Arm deprecates the use of **SP** as R_m .

Applies to an implementation of the architecture Armv8.0-M onward.

C1.4.6 Branching

I_{PVGL}

Writing an address to the **PC** causes either a simple branch to that address or an *interworking* branch.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{RHYZ}

A simple branch is performed by `BranchTo()`.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{DNWW}

An interworking branch is performed by `BranchReturn()`.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{CWSL}

Branching can occur in cases where `0b1111` is not a register specifier. In these cases, instructions write the **PC** either:

- Implicitly, for example, `b<cond>`.
- By using a register mask rather than a register specifier, for example `LDM`.

Applies to an implementation of the architecture Armv8.0-M onward.

I_{FLZZ}

The address to branch to can be:

- A loaded value, for example `LDM`.
- A register value, for example `BX`.
- The result of a calculation, for example `TBB` or `TBH`.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WQBX} The following table summarizes the branch instructions in the T32 instruction set.

Instruction	See	Range, T32
Branch to target address	B	±16MB
Compare and Branch on Nonzero, Compare and Branch on Zero	CBNZ , CBZ	0-126 bytes
Call a subroutine	BL	±16MB
Call a subroutine, optionally change Security state	BLX , BLXNS	Any
Branch to target address, change to Non-secure state	BX , BXNS	Any
Table Branch (byte offsets)	TBB , TBH	0-510 bytes
Table Branch (halfword offsets)		0-31070 bytes

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GJML} Branches to loaded and calculated addresses can be performed by [LDR](#), [LDM](#) and data-processing instructions.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{TPTF} A load instruction that targets the [PC](#) behaves as a branch instruction.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.4.7 Instruction set, interworking and interstating support

R_{LBQC} The following instructions are [Interworking](#) branches:

- [BX](#) and [BLX](#).
- [POP \(multiple registers\)](#)
and all forms of [LDM](#), when the register list includes the [PC](#).
- [LDR \(immediate\)](#), [LDR \(literal\)](#), and [LDR \(register\)](#), with <Rt> equal to the [PC](#).

Applies to an implementation of the architecture Armv8.0-M onward.

R_{WRZR} The value of bit[0] of an interworking branch instruction is not stored in the [PC](#). Bit[0] of an interworking branch instruction sets [EPSR.T](#). If [EPSR.T](#) is cleared to 0 an INVSTATE UsageFault is generated on the next instruction the PE attempts to execute.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{GLPL} The following instructions are *interstating branches*:

- [BXNS](#) and [BLXNS](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{GJMJ} When an interstating branch is executed in Secure state, bit[0] of the target address indicates the target Security state:

- 0: The target Security state is Non-secure state.
- 1: The target Security state is Secure state.

The value of bit[0] of an interstating branch instruction is not stored in the [PC](#).

Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - [S](#).

R_{WNSX}

Interstating branches are UNDEFINED when executing in Non-secure state.

*Applies to an implementation of the architecture **Armv8.0-M** onward. The extension requirements are - **S**.*

See also:

[C1.1 Instruction set.](#)

[B3.15 Security state transitions.](#)

C1.5 Modified immediate constants

R_{JVCL} The encoding of modified immediate constants in T32 instructions is:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
										i						imm3								a	b	c	d	e	f	g	h

Applies to an implementation of the architecture Armv8.0-M onward.

R_{TCLZ} The table shows the range of modified immediate constants available in T32 data processing instructions, and how they are encoded in the a, b, c, d, e, f, g, h, i, and imm3 fields in the instruction.

i:imm3:a	<const>					Carry flag set
0000x	00000000	00000000	00000000	abcdefgh		No
0001x	00000000	abcdefgh	00000000	abcdefgh		No
0010x	abcdefgh	00000000	abcdefgh	00000000		No
0011x	abcdefgh	abcdefgh	abcdefgh	abcdefgh		No
01000	1bcdefgh	00000000	00000000	00000000		Yes, to 1
01001	01bcdefg	h0000000	00000000	00000000		Yes, to 0
01010	001bcdef	gh000000	00000000	00000000		Yes, to 0
01011	0001bcde	fgh00000	00000000	00000000		Yes, to 0
-						Yes, to 0
-	8-bit values shifted to other positions					
-						
11101	00000000	00000000	000001bc	defgh000		Yes, to 0
11110	00000000	00000000	0000001b	cdefgh00		Yes, to 0
11111	00000000	00000000	00000001	bcdefgh0		Yes, to 0

This table shows the immediate constant value in binary form, to relate `abcdefgh` to the encoding diagram. In assembly syntax, the **immediate value** is specified as a decimal integer by default.

The setting of the Carry flag will only apply if a logical operation with a modified immediate constant can set the flags.

Where `i:imm3:a` is `0001x`, `0010x` or `0011x` the instruction will be UNPREDICTABLE if `abcdefgh == 0b00000000`.

Applies to an implementation of the architecture Armv8.0-M onward.

C1.5.1 Operation of modified immediate constants

R_{TLFG} `T32ExpandImm()` and `T32ExpandImm_C()` describe the operation of modified immediate constants.

Applies to an implementation of the architecture Armv8.0-M onward.

R_{PHBG} The operation of modified immediate constants are UNPREDICTABLE where both:

- `hw2[7:0] == 0b00000000`.

- $hw1[10] == 0$ and either:
 - $hw2[14:12] == 0b001$.
 - $hw2[14:12] == 0b010$.
 - $hw2[14:12] == 0b011$.

Applies to an implementation of the architecture *Armv8.0-M* onward.

C1.5.2 Modified immediate values for MVE instructions

R_{TBDZ}

The modified immediate values for MVE instructions are as follows:

op	cmode	Constant ^a								<dt> ^b	Notes
-	000x	00000000	00000000	00000000	abcdefgh	00000000	00000000	00000000	abcdefgh	I32	c
-	001x	00000000	00000000	abcdefgh	00000000	00000000	00000000	abcdefgh	00000000	I32	c
-	010x	00000000	abcdefgh	00000000	00000000	00000000	abcdefgh	00000000	00000000	I32	c
-	011x	abcdefgh	00000000	00000000	00000000	abcdefgh	00000000	00000000	00000000	I32	c
-	100x	00000000	abcdefgh	00000000	abcdefgh	00000000	abcdefgh	00000000	abcdefgh	I16	c
-	101x	abcdefgh	00000000	abcdefgh	00000000	abcdefgh	00000000	abcdefgh	00000000	I16	c
-	1100	00000000	00000000	abcdefgh	11111111	00000000	00000000	abcdefgh	11111111	I32	d
-	1101	00000000	abcdefgh	11111111	11111111	00000000	abcdefgh	11111111	11111111	I32	d
0	1110	abcdefgh	abcdefgh	abcdefgh	abcdefgh	abcdefgh	abcdefgh	abcdefgh	abcdefgh	I8	e
0	1111	aBbbbbb	defgh000	00000000	00000000	aBbbbbb	defgh000	00000000	00000000	F32	e, f
1	1110	aaaaaaaa	bbbbbbbb	cccccccc	dddddddd	eeeeeeee	ffffff	gggggg	hhhhhh	I64	e
1	1111	UNDEFINED								-	-

- In this table, the immediate value is shown in binary form, to relate `abcdefgh` to the encoding diagram. In assembler syntax, the constant is specified by a data type and a value of that type. That value is specified in the normal way (a decimal number by default) and is replicated enough times to fill the 64-bit immediate. For example, a data type of I32 and a value of 10 specify the 64-bit constant `0x0000000A0000000A`.
- This specifies the data type used when the instruction is disassembled. Where possible, on assembly the data type must be matched in the table. Other data types are permitted as pseudo-instructions when a program is assembled, provided the 64-bit constant specified by the data type and value is available for the instruction. If a constant is available in more than one way, the first entry in this table that can produce it is used. For example, `VMOV.I64 Q0, #0x8000000080000000` does not specify a 64-bit constant that is available from the I64 line of the table, but does specify one that is available from the fourth I32 line or the F32 line. It is assembled to the first of these, and therefore is disassembled as `VMOV.I32 Q0, #0x80000000`.
- This constant is available for the `VBIC (immediate)`, `VMOV (immediate) (vector)`, `VMVN (immediate)`, and `VORR (immediate)` instructions.
- This constant is available for the `VMOV (immediate) (vector)`, and `VMVN (immediate)` instructions only.
- This constant is available for the `VMOV` instruction only.
- In this entry, $B = \text{NOT}(b)$. The bit pattern represents the floating-point number:

$$(-1)^S \cdot 2^{\text{exp}} \cdot \text{mantissa}$$

where:

- $S = \text{UInt}(a)$
- $\text{exp} = \text{UInt}(\text{NOT}(b) : c : d) - 3$
- $\text{mantissa} = (16 + (\text{UInt}(e : f : g : h)) / 16)$

Applies to an implementation of the architecture *Armv8.1-M* onward. The extension requirements are - *MVE*.

C1.5.3 Modified immediate constants for floating-point instructions

R_{MZKF}

The modified immediates for floating-point instructions are:

size	Constant								<D:Vd or Vd:D >
01	aBbbcdef	gh00000	-	-	-	-	-	-	F16
10	aBbbbbbbc	defgh000	00000000	00000000	-	-	-	-	F32
11	aBbbbbbbc	bbcddefgh	00000000	00000000	00000000	00000000	00000000	00000000	F64

In this table, B = NOT (b) . The bit pattern represents the floating-point number:

$(-1)^S \cdot 2^{exp} \cdot mantissa$

where:

- S = UInt (a)
- exp = UInt (NOT (b) : c : d) - 3
- mantissa = (16 + (UInt (e : f : g : h)) / 16)

Applies to an implementation of the architecture **Armv8.0-M** onward. The extension requirements are - **FP**.

See also:

[T32ExpandImm\(\)](#)

[T32ExpandImm_C\(\)](#)

[AdvSIMDExpandImm\(\)](#)

[VFPExpandImm\(\)](#)

C1.6 NOP-compatible hint instructions

I _{BJRT}	A hint instruction only provides an indication to the PE. It is not required that the PE perform an operation on a hint instruction. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{VXQV}	A NOP-compatible hint instruction either: • Acts as a <code>NOP</code> (No Operation) instruction. • Performs some IMPLEMENTATION DEFINED behavior. <i>Applies to an implementation of the architecture Armv8.0-M onward.</i>
R _{DBNQ}	A PE without the Main Extension only supports the 16-bit encodings of the Armv8-M NOP-compatible hint instructions. The 32-bit NOP-compatible instructions are UNDEFINED if the Main extension is not implemented. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - !M.</i>
R _{DJQL}	A PE with the Main Extension supports both the 16-bit and the 32-bit encodings of the Armv8-M NOP-compatible hint instructions. <i>Applies to an implementation of the architecture Armv8.0-M onward. The extension requirements are - M.</i>

Chapter C2

Instruction Specification

This chapter specifies the Armv8-M instruction set. It contains the following sections:

[Top level T32 instruction set encoding](#)

[32-bit T32 instruction encoding](#)

[16-bit T32 instruction encoding](#)

[Alphabetical list of instructions](#)

C2.1 Top level T32 instruction set encoding

The T32 instruction stream is a sequence of halfword-aligned halfwords. Each T32 instruction is either a single 16-bit halfword in that stream, or a 32-bit instruction consisting of two consecutive halfwords in that stream.

If the value of bits[15:11] of the halfword being decoded is one of the following, the halfword is the first halfword of a 32-bit instruction:

- 0b11101.
- 0b11110.
- 0b11111.

Otherwise, the halfword is a 16-bit instruction.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op0					op1																										

This table shows the decode field values and the associated subgroups:

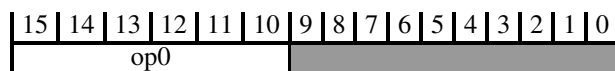
op0	op1	Subgroup
!= 111	-	16-bit T32 instruction encoding
111	!= 00	32-bit T32 instruction encoding

This table shows the decode field values and the associated instructions:

op0	op1	Instruction
111	00	B T2

C2.2 16-bit T32 instruction encoding

This section describes the encoding of the 16-bit T32 instruction encoding. This section is decoded from [Top level T32 instruction set encoding](#).



This table shows the decode field values and the associated subgroups:

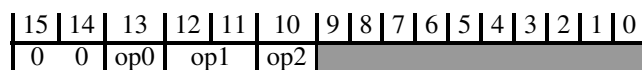
op0	Subgroup
00xxxx	Shift (immediate), add, subtract, move, and compare
010000	Data-processing (two low registers)
010001	Special data instructions and branch and exchange
0101xx	Load/store (register offset)
011xxx	Load/store word/byte (immediate offset)
1000xx	Load/store halfword (immediate offset)
1001xx	Load/store (SP-relative)
1010xx	Add PC/SP (immediate)
1011xx	Miscellaneous 16-bit instructions
1100xx	Load/store multiple
1101xx	Conditional branch, and Supervisor Call

This table shows the decode field values and the associated instructions:

op0	Instruction
01001x	LDR (literal) T1

C2.2.1 Shift (immediate), add, subtract, move, and compare

This section describes the encoding of the Shift (immediate), add, subtract, move, and compare. This section is decoded from [16-bit T32 instruction encoding](#).



This table shows the decode field values and the associated subgroups:

op0	op1	op2	Subgroup
0	11	0	Add, subtract (three low registers)
0	11	1	Add, subtract (two low registers and immediate)
1	-	-	Add, subtract, compare, move (one low register and immediate)

This table shows the decode field values and the associated instructions:

op0	op1	op2	Instruction
	Alias		ASRS (immediate) T2
	Alias		ASR (immediate) T2
	Alias		LSLS (immediate) T2
	Alias		LSL (immediate) T2
	Alias		LSRS (immediate) T2
	Alias		LSR (immediate) T2
0	!= 11	-	MOV (register) T2

C2.2.1.1 Add, subtract (three low registers)

This section describes the encoding of the Add, subtract (three low registers). This section is decoded from [Shift \(immediate\)](#), [add](#), [subtract](#), [move](#), and [compare](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	0	S	Rm			Rn			Rd		

This table shows the decode field values and the associated instructions:

S	Instruction
0	ADD (register) T1
1	SUB (register) T1

C2.2.1.2 Add, subtract (two low registers and immediate)

This section describes the encoding of the Add, subtract (two low registers and immediate). This section is decoded from [Shift \(immediate\)](#), [add](#), [subtract](#), [move](#), and [compare](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	S	imm3			Rn			Rd		

This table shows the decode field values and the associated instructions:

S	Instruction
0	ADD (immediate) T1
1	SUB (immediate) T1

C2.2.1.3 Add, subtract, compare, move (one low register and immediate)

This section describes the encoding of the Add, subtract, compare, move (one low register and immediate). This section is decoded from [Shift \(immediate\)](#), [add](#), [subtract](#), [move](#), and [compare](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	op		reg			imm8							

This table shows the decode field values and the associated instructions:

op	Instruction
00	MOV (immediate) T1
01	CMP (immediate) T1
10	ADD (immediate) T2
11	SUB (immediate) T2

C2.2.2 Data-processing (two low registers)

This section describes the encoding of the Data-processing (two low registers). This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op				Rs		reg			

This table shows the decode field values and the associated instructions:

op	Instruction
0000	AND (register) T1
0001	EOR (register) T1
Alias	LSL (register) T1
Alias	LSLS (register) T1
Alias	LSR (register) T1
Alias	LSRS (register) T1
Alias	ASR (register) T1
Alias	ASRS (register) T1
0101	ADC (register) T1
0110	SBC (register) T1
Alias	ROR (register) T1
Alias	RORS (register) T1
1000	TST (register) T1
1001	RSB (immediate) T1
1010	CMP (register) T1
1011	CMN (register) T1
1100	ORR (register) T1
1101	MUL T1
1110	BIC (register) T1
1111	MVN (register) T1
0010	MOV, MOVS (register-shifted register) T1
0011	
0100	
0111	

C2.2.3 Special data instructions and branch and exchange

This section describes the encoding of the Special data instructions and branch and exchange. This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	op0									

This table shows the decode field values and the associated subgroups:

op0	Subgroup
11	Branch and exchange
!= 11	Add, subtract, compare, move (two high registers)

C2.2.3.1 Branch and exchange

This section describes the encoding of the Branch and exchange. This section is decoded from [Special data instructions and branch and exchange](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	1	1	L	Rm			NS	(0)(0)		

This table shows the decode field values and the associated instructions:

L	Instruction
0	BX, BXNS T1
1	BLX, BLXNS T1

C2.2.3.2 Add, subtract, compare, move (two high registers)

This section describes the encoding of the Add, subtract, compare, move (two high registers). This section is decoded from [Special data instructions and branch and exchange](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	op		D	Rs				Rd		

This table shows the decode field values and the associated instructions:

D:Rd	Rs	op	Instruction
!= 1101	!= 1101	00	ADD (register) T2
-	1101	00	ADD (SP plus register) T1
1101	!= 1101	00	ADD (SP plus register) T2
-	-	01	CMP (register) T2
-	-	10	MOV (register) T1

C2.2.4 Load/store (register offset)

This section describes the encoding of the Load/store (register offset). This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	L	B	H	Rm			Rn			Rt		

This table shows the decode field values and the associated instructions:

B	H	L	Instruction
0	0	0	STR (register) T1
0	1	0	STRH (register) T1
1	0	0	STRB (register) T1
1	1	0	LDRSB (register) T1
0	0	1	LDR (register) T1
0	1	1	LDRH (register) T1
1	0	1	LDRB (register) T1
1	1	1	LDRSH (register) T1

C2.2.5 Load/store word/byte (immediate offset)

This section describes the encoding of the Load/store word/byte (immediate offset). This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	B	L	imm5			Rn			Rt				

This table shows the decode field values and the associated instructions:

B	L	Instruction
0	0	STR (immediate) T1
0	1	LDR (immediate) T1
1	0	STRB (immediate) T1
1	1	LDRB (immediate) T1

C2.2.6 Load/store halfword (immediate offset)

This section describes the encoding of the Load/store halfword (immediate offset). This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	L	imm5					Rn			Rt		

This table shows the decode field values and the associated instructions:

L	Instruction
0	STRH (immediate) T1
1	LDRH (immediate) T1

C2.2.7 Load/store (SP-relative)

This section describes the encoding of the Load/store (SP-relative). This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	L	Rt			imm8							

This table shows the decode field values and the associated instructions:

L	Instruction
0	STR (immediate) T2
1	LDR (immediate) T2

C2.2.8 Add PC/SP (immediate)

This section describes the encoding of the Add PC/SP (immediate). This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	op	Rd			imm8							

This table shows the decode field values and the associated instructions:

op	Instruction
Alias	ADD (immediate, to PC) T1
0	ADR T1
1	ADD (SP plus immediate) T1

C2.2.9 Miscellaneous 16-bit instructions

This section describes the encoding of the Miscellaneous 16-bit instructions. This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	op0				op1		op2		op3			

This table shows the decode field values and the associated subgroups:

op0	op1	op2	op3	Subgroup
0000	-	-	-	Adjust SP (immediate)
0010	-	-	-	Extend
1010	!= 10	-	-	Reverse bytes
1111	-	-	0000	Hints
x10x	-	-	-	Push and Pop

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	Instruction
0110	01	1	-	CPS T1
1110	-	-	-	BKPT T1
1111	-	-	!= 0000	IT T1
x0x1	-	-	-	CBNZ, CBZ T1

C2.2.9.1 Adjust SP (immediate)

This section describes the encoding of the Adjust SP (immediate). This section is decoded from [Miscellaneous 16-bit instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	0	0	S	imm7						

This table shows the decode field values and the associated instructions:

S	Instruction
0	ADD (SP plus immediate) T2
1	SUB (SP minus immediate) T1

C2.2.9.2 Extend

This section describes the encoding of the Extend. This section is decoded from [Miscellaneous 16-bit instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	1	0	U	B	Rm			Rd		

This table shows the decode field values and the associated instructions:

B	U	Instruction
0	0	SXTH T1
1	0	SXTB T1
0	1	UXTH T1
1	1	UXTB T1

C2.2.9.3 Reverse bytes

This section describes the encoding of the Reverse bytes. This section is decoded from [Miscellaneous 16-bit instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	0	1	0	op		Rm			Rd		

This table shows the decode field values and the associated instructions:

op	Instruction
00	REV T1
01	REV16 T1
11	REVSH T1

C2.2.9.4 Hints

This section describes the encoding of the Hints. This section is decoded from [Miscellaneous 16-bit instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	hint				0	0	0	0

This table shows the decode field values and the associated instructions:

hint	Instruction
0000	NOP T1
0001	YIELD T1
0010	WFE T1
0011	WFI T1
0100	SEV T1
0101	Reserved hint, it behaves as NOP
011x	Reserved hint, it behaves as NOP
1xxx	Reserved hint, it behaves as NOP

C2.2.9.5 Push and Pop

This section describes the encoding of the Push and Pop. This section is decoded from [Miscellaneous 16-bit instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	L	1	0	P	register_list							

This table shows the decode field values and the associated instructions:

L	Instruction
1	LDM, LDMIA, LDMFD T3
Alias	POP (multiple registers) T3
Alias	PUSH (multiple registers) T2
0	STMDB, STMFD T2

C2.2.10 Load/store multiple

This section describes the encoding of the Load/store multiple. This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	L	Rn			register_list							

This table shows the decode field values and the associated instructions:

L	Instruction
1	LDM, LDMIA, LDMFD T1
0	STM, STMIA, STMEA T1

C2.2.11 Conditional branch, and Supervisor Call

This section describes the encoding of the Conditional branch, and Supervisor Call. This section is decoded from [16-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	op0											

This table shows the decode field values and the associated subgroups:

op0	Subgroup
-----	----------

111x	Exception generation
------	--------------------------------------

This table shows the decode field values and the associated instructions:

op0	Instruction
!= 111x	B T1

C2.2.11.1 Exception generation

This section describes the encoding of the Exception generation. This section is decoded from [Conditional branch](#), [and Supervisor Call](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	1	1	1	S	imm8							

This table shows the decode field values and the associated instructions:

S	imm8	Instruction
0	-	UDF T1
1	-	SVC T1

C2.3 32-bit T32 instruction encoding

This section describes the encoding of the 32-bit T32 instruction encoding. This section is decoded from [Top level T32 instruction set encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0				op1			op2						op3								op4							

This table shows the decode field values and the associated subgroups:

op0	op1	op1:op2	op2	op3	op4	Subgroup
x11x	-	-	-	-	-	Coprocessor, floating-point, and vector instructions
0100	-	-	-	-	-	Load/store (multiple, dual, exclusive, acquire-release)
0101	-	-	-	-	-	Data-processing (shifted register)
10xx	-	-	-	1xxx	-	Branches and miscellaneous control
10x0	-	-	-	0xxx	-	Data-processing (modified immediate)
10x1	-	-	0	0xxx	-	Data-processing (plain binary immediate)
1100	-	!= 1x0	-	-	-	Load/store single
1101	0x	-	-	1111	-	Data-processing (register)
1101	10	-	-	-	00	Multiply, multiply accumulate, and absolute difference
1101	11	-	-	-	-	Long multiply and divide

C2.3.1 Coprocessor, floating-point, and vector instructions

This section describes the encoding of the Coprocessor, floating-point, and vector instructions. This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1		1	1	op0								op1								op2									

This table shows the decode field values and the associated subgroups:

op0	op1	op2	Subgroup
10	-	-	Floating-point and vector miscellaneous instructions
0x	-	-	Floating-point and vector load/store, move, and coprocessor instructions
11	0	1	Miscellaneous vector arithmetic instructions

C2.3.1.1 Floating-point and vector miscellaneous instructions

This section describes the encoding of the Floating-point and vector miscellaneous instructions. This section is decoded from [Coprocessor, floating-point, and vector instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1		1	1	1	0													op0						op1					

This table shows the decode field values and the associated subgroups:

op0	op1	Subgroup
0xx	-	Coproprocessor data-processing instructions
100	1	Vector move instructions
10x	0	Floating-point data-processing, minNum/maxNum, and convert
101	1	Floating-point and vector move (register)
110	-	Architected coprocessor data-processing instructions
111	0	Vector immediate and register, and coprocessor data-processing instructions

C2.3.1.1.1 Coprocessor data-processing instructions

This section describes the encoding of the Coprocessor data-processing instructions. This section is decoded from [Floating-point and vector miscellaneous instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1		1	1	1	0															0	coproc									

This table shows the decode field values and the associated subgroups:

CoprocType(UInt(coproc))	Subgroup
CP_GCP	Architected coprocessor data-processing instructions
CP_CDEv1	Custom general-purpose register instructions

Architected coprocessor data-processing instructions

This section describes the encoding of the Architected coprocessor data-processing instructions. This section is decoded from [Coproprocessor data-processing instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	1	0	opc1	op1	CRn			reg			coproc			opc2	op2	CRm										

This table shows the decode field values and the associated instructions:

op0	op1	op2	Instruction
0	-	0	CDP, CDP2 T1
1	-	0	CDP, CDP2 T2
0	0	1	MCR, MCR2 T1
1	0	1	MCR, MCR2 T2
0	1	1	MRC, MRC2 T1
1	1	1	MRC, MRC2 T2

Custom general-purpose register instructions

This section describes the encoding of the Custom general-purpose register instructions. This section is decoded from [Coprorocessor data-processing instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
1	1	1	A	1	1	1	0	op0	op1												0												op2							

This table shows the decode field values and the associated instructions:

op0	op1	op2	Instruction
0	0	0	CX1 T1
0	0	1	CX1D T1
0	1	0	CX2 T1
0	1	1	CX2D T1
1	-	0	CX3 T1
1	-	1	CX3D T1

C2.3.1.1.2 Vector move instructions

This section describes the encoding of the Vector move instructions. This section is decoded from [Floating-point and vector miscellaneous instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	op0	1	1	1	0	op1	op	Vn						Rt												op2					

This table shows the decode field values and the associated instructions:

op0	op1	op2	Instruction
0	000	1	VMOV (between general-purpose register and half-precision register) T1

C2.3.1.1.3 Floating-point data-processing, minNum/maxNum, and convert

This section describes the encoding of the Floating-point data-processing, minNum/maxNum, and convert. This section is decoded from [Floating-point and vector miscellaneous instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0									
1	1	1	op0	1	1	1	0	op1	D	op2						op3						Vd						1	0	op4	op5	op6	M	0	Vm					

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	op4	op5	op6	Instruction
-----	-----	-----	-----	-----	-----	-----	-------------

1	1	00x	-	-	-	1	VMINNM T2
1	1	111	110	-	-	1	VCVTP T1
0	1	00x	-	-	-	0	VDIV T1
0	0	11x	-	-	-	0	VADD T2
0	0	01x	-	-	-	0	VNMLS T1
1	1	111	010	-	0	1	VRINTP T1
0	1	110	000	1x	0	1	VMOV (register) T2
0	1	111	000	-	-	1	VCVT (integer to floating-point) T1
0	1	10x	-	-	-	1	VFMS T2
0	1	01x	-	-	-	0	VFNMS T1
0	1	110	100	-	1	1	VCMPE T1
0	1	110	101	-	1	1	VCMPE T2
0	1	110	01x	1x	1	1	VCVTT T1
1	1	111	011	-	0	1	VRINTM T1
1	1	111	000	-	0	1	VRINTA T1
0	1	110	001	-	0	1	VNEG T2
0	1	110	111	-	0	1	VRINTX T1
1	1	111	111	-	-	1	VCVTM T1
0	1	110	01x	1x	0	1	VCVTB T1
1	1	00x	-	-	-	0	VMAXNM T2
0	0	00x	-	-	-	0	VMLA T2
0	0	00x	-	-	-	1	VMLS T2
0	1	111	10x	-	1	1	VCVT (floating-point to integer) T1
0	1	01x	-	-	-	1	VFNMA T1
0	1	10x	-	-	-	0	VFMA T2
0	1	110	000	-	1	1	VABS T2
0	1	110	110	-	0	1	VRINTR T1
0	1	11x	-	-	-	0	VMOV (immediate) T2
0	1	110	100	-	0	1	VCMP T1
1	1	110	000	10	0	1	VMOVX T1
0	1	110	101	-	0	1	VCMP T2
0	1	110	111	1x	1	1	VCVT (between double-precision and single-precision) T1
0	1	111	x1x	-	-	1	VCVT (between floating-point and fixed-point) T1
1	1	110	000	10	1	1	VINS T1
1	1	111	001	-	0	1	VRINTN T1
1	0	-	-	-	-	0	VSEL T1
0	0	01x	-	-	-	1	VNMLA T1
0	1	110	110	-	1	1	VRINTZ T1
1	1	111	101	-	-	1	VCVTN T1
0	0	10x	-	-	-	0	VMUL T2
1	1	111	100	-	-	1	VCVTA T1
0	1	110	001	-	1	1	VSQRT T1
0	1	111	10x	-	0	1	VCVTR T1
0	0	10x	-	-	-	1	VNMUL T2
0	0	11x	-	-	-	1	VSUB T2

C2.3.1.1.4 Floating-point and vector move (register)

This section describes the encoding of the Floating-point and vector move (register). This section is decoded from [Floating-point and vector miscellaneous instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	1	0									Rt								op1					1	(0)(0)(0)(0)	

This table shows the decode field values and the associated subgroups:

op0	op1	Subgroup
0	1	Move between Floating-point and vector registers
0	0	Move between general-purpose Register and Floating-point Special register

Move between Floating-point and vector registers

This section describes the encoding of the Move between Floating-point and vector registers. This section is decoded from [Floating-point and vector move \(register\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
1	1	1	0	1	1	1	0	op0								op1					1	0	1	1	op2								1	(0)(0)(0)(0)

This table shows the decode field values and the associated instructions:

op0	op1	op2	Instruction
1x10	0	0	VDUP T1
0xx0	-	-	VMOV (general-purpose register to vector lane) T1
xxx1	-	-	VMOV (vector lane to general-purpose register) T1
Alias			VMOV (single general-purpose register to half of doubleword register) T1
Alias			VMOV (half of doubleword register to single general-purpose register) T1

Move between general-purpose Register and Floating-point Special register

This section describes the encoding of the Move between general-purpose Register and Floating-point Special register. This section is decoded from [Floating-point and vector move \(register\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	op0				reg				Rt				1	0	1	0	N	(0)(0)		1	(0)(0)(0)(0)			

This table shows the decode field values and the associated instructions:

op0	Instruction
000x	VMOV (between general-purpose register and single-precision register) T1
1111	VMRS T1
1110	VMSR T1

C2.3.1.1.5 Architected coprocessor data-processing instructions

This section describes the encoding of the Architected coprocessor data-processing instructions. This section is decoded from [Floating-point and vector miscellaneous instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	1	0	op1	op1	CRn	reg	coproc	opc2	op2	CRm																

This table shows the decode field values and the associated instructions:

op0	op1	op2	Instruction
0	-	0	CDP, CDP2 T1
1	-	0	CDP, CDP2 T2
0	0	1	MCR, MCR2 T1
1	0	1	MCR, MCR2 T2
0	1	1	MRC, MRC2 T1
1	1	1	MRC, MRC2 T2

C2.3.1.1.6 Vector immediate and register, and coprocessor data-processing instructions

This section describes the encoding of the Vector immediate and register, and coprocessor data-processing instructions. This section is decoded from [Floating-point and vector miscellaneous instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1		1	1	1	0	op0								1	1	1	op1							0					

This table shows the decode field values and the associated subgroups:

op0	op1	Subgroup
0	0	Miscellaneous vector register instructions
1	0	Miscellaneous vector register and immediate instructions
0	1	Vector register, immediate, and predication instructions
1	1	Vector arithmetic, minimum, maximum, and shift instructions

Miscellaneous vector register instructions

This section describes the encoding of the Miscellaneous vector register instructions. This section is decoded from [Vector immediate and register, and coprocessor data-processing instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	1	0	0	D	op1	op2	op3	Qd	op4	1	1	1	0	op5	0											op6

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	op4	op5	op6	Instruction
-----	-----	-----	-----	-----	-----	-----	-------------

0	11	!= 11	01	-	10x	1	VQMOVUN T1
-	!= 11	-	x1	1	x10	-	VMLAS (vector by vector plus scalar) T1
-	11	!= 11	11	1	10x	1	VMIN, VMINA T2
1	!= 11	-	x0	-	x0x	0	VQDMLSDH, VQRDMLSDH T1
-	11	!= 11	01	1	111	-	VQSHL, VQSHLU T1
-	11	11	11	1	10x	1	VMINNM, VMINNMA (floating-point) T2
-	11	!= 11	11	0	10x	1	VMAX, VMAXA T2
-	11	-	x1	-	x0x	0	VMULL (polynomial) T1
1	11	!= 11	01	-	10x	1	VMOVN T1
-	!= 11	-	x1	0	x0x	1	VMULH, VRMULH T1
-	!= 11	-	x1	1	x0x	1	VMULH, VRMULH T2
0	!= 11	-	x1	1	x11	-	VMUL (vector) T2
-	!= 11	-	x0	0	x11	-	VQDMLAH, VQRDMLAH (vector by scalar plus vector) T1
-	!= 11	-	x0	0	x10	-	VQDMLAH, VQRDMLAH (vector by scalar plus vector) T2
-	11	-	x1	0	x10	-	VFMA (vector by scalar plus vector, floating-point) T1
0	!= 11	-	x0	-	x0x	1	VQDMLADH, VQRDMLADH T2
-	11	-	x1	0	x11	-	VMUL (floating-point) T2
-	11	-	x1	1	x10	-	VFMA (vector by vector plus scalar, floating-point) T1
0	!= 11	-	x1	0	x11	-	VQDMULH, VQRDMULH T3
-	11	-	x0	-	x0x	-	VCML (floating-point) T1
0	!= 11	-	x0	-	x0x	0	VQDMLADH, VQRDMLADH T1
1	!= 11	-	x1	0	x11	-	VQDMULH, VQRDMULH T4
1	!= 11	-	x0	-	x0x	1	VQDMLSDH, VQRDMLSDH T2
-	!= 11	-	x0	1	x11	-	VQDMLASH, VQRDMLASH (vector by vector plus scalar) T1
-	!= 11	-	x0	1	x10	-	VQDMLASH, VQRDMLASH (vector by vector plus scalar) T2
-	11	!= 11	01	-	00x	1	VSHLL T2
-	11	!= 11	11	1	011	-	VRSHL T2
-	11	!= 11	01	1	011	-	VSHL T2
-	11	11	11	-	00x	1	VCVT (between single and half-precision floating-point) T1
-	11	!= 11	11	-	00x	1	VQMOVN T1
-	!= 11	-	x1	-	x0x	0	VMULL (integer) T1
1	!= 11	-	x1	1	x11	-	VBRSR T1
-	11	11	11	0	10x	1	VMAXNM, VMAXNMA (floating-point) T2
-	11	!= 11	11	1	111	-	VQRSHL T2
-	!= 11	-	x1	0	x10	-	VMLA (vector by scalar plus vector) T1

Miscellaneous vector register and immediate instructions

This section describes the encoding of the Miscellaneous vector register and immediate instructions. This section is decoded from [Vector immediate and register, and coprocessor data-processing instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	1	0	1	op1	Qn	op2	Rda	X	1	1	1	0	N	op3	A	0	Qm	op4								

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	op4	Instruction
-	111	-	0	0	VMLADAV T1
Alias					VMLAV T1
Alias					VMLALV T1
-	!= 111	-	0	0	VMLALDAV T1
0	!= 111	-	0	1	VMLS LDAV T1

0	111	-	0	1	VMLSDAV T1
1	111	0	0	1	VMLSDAV T2
1	!= 111	0	0	1	VRMLS LDAVH T1

Vector register, immediate, and predication instructions

This section describes the encoding of the Vector register, immediate, and predication instructions. This section is decoded from [Vector immediate and register, and coprocessor data-processing instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	1	0	0	op1	op2	reg	op3	op4			op5	1	1	1	1	T	op6	0	op7		op8					

This table shows the decode field values and the associated instructions:

op0	op1:op4	op2	op3:op4:op5	op5:op8	op6	op7	op7:op8	op8	Instruction
0	-	!= 11	1xxx0	-	11	111	-	-	VIDUP, VIWDUP T2
0	-	!= 11	1xxx0	-	11	!= 111	-	-	VIDUP, VIWDUP T1
-	-	11	0xxxx	-	0x	-	-	1	VQDMULL T1
-	-	!= 11	0xxx0	-	11	-	-	-	VQADD T2
-	-	11	0xxx0	-	10	-	-	-	VADD (floating-point) T2
0	-	!= 11	1xxx1	-	10	-	-	-	VSUB (vector) T2
0	-	!= 11	1xxx0	-	10	-	-	-	VADD (vector) T2
0	-	!= 11	0xxxx	-	0x	-	-	0	VHCADD T1
0	-	!= 11	1xxx1	-	11	111	-	-	VDDUP, VDWDUP T2
0	-	!= 11	1xxx1	-	11	!= 111	-	-	VDDUP, VDWDUP T1
1	-	!= 11	0xxxx	-	0x	-	-	0	VCADD T1
-	-	!= 11	0xxx0	-	10	-	-	-	VHADD T2
-	-	11	0xxx1	-	10	-	-	-	VSUB (floating-point) T2
-	-	!= 11	0xxx1	-	10	-	-	-	VHSUB T2
1	-	11	1xxx0	01	0x	-	-	-	VPSEL T1
1	0000	11	1xxxx	-	1x	-	1101	-	VPNOT T1
1	0000	!= 11	10000	-	0x	-	-	0	VCMP (vector) T1
1	0000	!= 11	10000	-	0x	-	-	1	VCMP (vector) T2
1	0000	!= 11	10001	-	0x	-	-	-	VCMP (vector) T3
1	0000	!= 11	10000	-	10	-	-	-	VCMP (vector) T4
1	0000	!= 11	10000	-	11	-	-	-	VCMP (vector) T5
1	0000	!= 11	10001	-	1x	-	-	-	VCMP (vector) T6
-	0000	11	1000x	-	1x	-	!= 1101	-	VCMP (floating-point) T2
-	0000	11	1000x	!= 01	0x	-	-	-	VCMP (floating-point) T1
1	!= 0000	11	1xxxx	-	1x	-	1101	-	VPST T1
1	!= 0000	!= 11	1xxx0	-	0x	-	-	0	VPT T1
1	!= 0000	!= 11	1xxx0	-	0x	-	-	1	VPT T2
1	!= 0000	!= 11	1xxx1	-	0x	-	-	-	VPT T3
1	!= 0000	!= 11	1xxx0	-	10	-	-	-	VPT T4
1	!= 0000	!= 11	1xxx0	-	11	-	-	-	VPT T5
1	!= 0000	!= 11	1xxx1	-	1x	-	-	-	VPT T6
-	!= 0000	11	1xxxx	-	1x	-	!= 1101	-	VPT (floating-point) T2
-	!= 0000	11	1xxxx	!= 01	0x	-	-	-	VPT (floating-point) T1
0	-	11	0xxxx	-	0x	-	-	0	VADC T1
-	-	!= 11	0xxx1	-	11	-	-	-	VQSUB T2
-	-	11	0xxxx	-	11	-	-	-	VQDMULL T2
1	-	11	0xxxx	-	0x	-	-	0	VSBC T1

Vector arithmetic, minimum, maximum, and shift instructions

This section describes the encoding of the Vector arithmetic, minimum, maximum, and shift instructions. This section is decoded from [Vector immediate and register, and coprocessor data-processing instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	1	0	1	op1	op2	op3	op4	op5	reg			op6	1	1	1	1	op7	0	Qm			op8				

This table shows the decode field values and the associated instructions:

op0	op1:op2	op2:op3:op4:op5	op3:op4	op5	op6	op7	op8	Instruction
-	110	-	!= 11	00	-	10x	0	VMINV, VMINAV T2
0	x0x	-	-	-	-	11x	1	VSHRN T1
-	110	-	!= 11	10	-	10x	0	VMINV, VMINAV T1
1	x0x	-	-	-	-	11x	0	VQRSHRUN T1
-	110	-	11	10	-	10x	0	VMINNMV, VMINNMAV (floating-point) T1
-	111	-	-	x0	-	x0x	0	VMLADAV T2
-	!= 111	-	-	01	-	00x	0	VADDLV T1
-	111	-	!= 11	01	-	00x	0	VADDV T1
-	110	-	11	00	-	10x	0	VMINNMV, VMINNMAV (floating-point) T2
0	x1x	-	-	-	0	110	-	VSHLC T1
0	x0x	-	-	-	-	11x	0	VQSHRUN T1
-	x0x	-	-	-	-	01x	0	VQSHRN T1
1	x0x	-	-	-	-	11x	1	VRSHRN T1
-	110	-	11	10	-	00x	0	VMAXNMV, VMAXNMAV (floating-point) T1
-	110	-	!= 11	00	-	00x	0	VMAXV, VMAXAV T2
-	110	-	!= 11	10	-	00x	0	VMAXV, VMAXAV T1
-	110	-	11	00	-	00x	0	VMAXNMV, VMAXNMAV (floating-point) T2
-	!= 11x	-	-	x0	-	x0x	0	VRMLALDAH T1
Alias								VRMLALVH T1
-	x0x	-	-	-	-	01x	1	VQRSHRN T1
-	00x 010	-	-	x0	-	x0x	1	VABAV T1
-	x1x	00000 11000 xx001 xx01x xx1xx	-	-	-	01x	0	VSHLL T1
-	x1x	01000 10000	-	-	-	01x	0	VMOVL T1

C2.3.1.2 Floating-point and vector load/store, move, and coprocessor instructions

This section describes the encoding of the Floating-point and vector load/store, move, and coprocessor instructions. This section is decoded from [Coprocessor, floating-point, and vector instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1		1	1	0					op1											op0									

This table shows the decode field values and the associated subgroups:

op0	op1	Subgroup
0xx	-	Coprocessor load, store, and move instructions
100	-	Floating-point and vector load/store and, complex arithmetic instructions
101	-	Coprocessor and Floating-point load/store, move, and security
110	-	Architected coprocessor load, store, and move instructions
111	1	Vector load instructions
111	0	Vector store instructions

C2.3.1.2.1 Coprocessor load, store, and move instructions

This section describes the encoding of the Coprocessor load, store, and move instructions. This section is decoded from [Floating-point and vector load/store, move, and coprocessor instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1		1	1	0																0	coproc								

This table shows the decode field values and the associated subgroups:

CoprocType(UInt(coproc))	Subgroup
CP_GCP	Architected coprocessor load, store, and move instructions
CP_CDEv1	Custom Floating-point and vector instructions

Architected coprocessor load, store, and move instructions

This section describes the encoding of the Architected coprocessor load, store, and move instructions. This section is decoded from [Coprocessor load, store, and move instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	0					op1	op2	op3																		

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	Instruction
0	!= 0010	1	!= 1111	LDC, LDC2 (immediate) T1
1	!= 0010	1	!= 1111	LDC, LDC2 (immediate) T2
0	!= 0010	1	1111	LDC, LDC2 (literal) T1
1	!= 0010	1	1111	LDC, LDC2 (literal) T2
0	!= 0010	0	-	STC, STC2 T1
1	!= 0010	0	-	STC, STC2 T2

0	0010	1	-	MRRC, MRRC2 T1
1	0010	1	-	MRRC, MRRC2 T2
0	0010	0	-	MCRR, MCRR2 T1
1	0010	0	-	MCRR, MCRR2 T2

Custom Floating-point and vector instructions

This section describes the encoding of the Custom Floating-point and vector instructions. This section is decoded from [Coprorocessor load, store, and move instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	A	1	1	0		op0	D	op1							Vd			0		coproc			op2						

This table shows the decode field values and the associated instructions:

op0	op1	op2	Instruction
0	10	1	VCX1 (vector) T1
0	10	0	VCX1 T1
0	11	1	VCX2 (vector) T1
0	11	0	VCX2 T1
1	-	1	VCX3 (vector) T1
1	-	0	VCX3 T1

C2.3.1.2.2 Floating-point and vector load/store and, complex arithmetic instructions

This section describes the encoding of the Floating-point and vector load/store and, complex arithmetic instructions. This section is decoded from [Floating-point and vector load/store, move, and coprocessor instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1																										

This table shows the decode field values and the associated subgroups:

op0	Subgroup
1	Floating-point and vector complex arithmetic instructions
0	Floating-point and vector load/store instructions

Floating-point and vector complex arithmetic instructions

This section describes the encoding of the Floating-point and vector complex arithmetic instructions. This section is decoded from [Floating-point and vector load/store and, complex arithmetic instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0		op0		op1			Qn	op2		Qd		op3	1	0	0	op4		op5		op6		Qm	op7		

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	op4	op5	op6	op7	Instruction
1	0	0	0	0	1	0	0	VCADD (floating-point) T1
-	1	0	0	0	1	0	0	VCMLA (floating-point) T1

Floating-point and vector load/store instructions

This section describes the encoding of the Floating-point and vector load/store instructions. This section is decoded from [Floating-point and vector load/store and, complex arithmetic instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	op0	U	D	op1			Rn			Vd			1	0	0	op2									

This table shows the decode field values and the associated instructions:

op0	op1	op2	Instruction
1	01	1	VLDR T3
1	00	1	VSTR T3

C2.3.1.2.3 Coprocessor and Floating-point load/store, move, and security

This section describes the encoding of the Coprocessor and Floating-point load/store, move, and security. This section is decoded from [Floating-point and vector load/store, move, and coprocessor instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	0		op1		op2		op3											op4			op5					op6

This table shows the decode field values and the associated instructions:

op0	op1	op1:op3	op2	op3	op4	op5	op6	Instruction
0	00x1 01xx 1xx1	!= 01x01111	1	-	1xx	-	1	FLDMDBX, FLDMIAX T1
0	00x1 01xx 1xx1	!= 01x01111	1	-	1xx	-	0	VLDM T1
0	01xx 1xx1	!= 01x01111	1	-	0xx	-	-	VLDM T2
0	1xx0	-	1	-	1xx	-	-	VLDR T1
0	1xx0	-	1	-	0xx	-	-	VLDR T2

0	00x1	-	1	-	00x	-	-	VLLDM T1
0	00x1	-	1	-	01x	-	-	VLLDM T2
Alias								VPOP T1
Alias								VPOP T2
0	00x1 01xx 1xx1	-	0	-	1xx	-	1	FSTMDBX, FSTMIAx T1
0	00x1 01xx 1xx1	-	0	-	1xx	-	0	VSTM T1
0	01xx 1xx1	-	0	-	0xx	-	-	VSTM T2
0	1xx0	-	0	-	1xx	-	-	VSTR T1
0	1xx0	-	0	-	0xx	-	-	VSTR T2
0	00x1	-	0	-	00x	-	-	VLSTM T1
0	00x1	-	0	-	01x	-	-	VLSTM T2
Alias								VPUSH T1
Alias								VPUSH T2
0	0010	-	-	-	100	1	-	VMOV (two GP registers and a doubleword register) T1
0	0010	-	-	-	000	1	-	VMOV (two GP registers and two SP registers) T1
0	01x0	-	1	1111	1xx	-	0	VSCCLRM T1
0	01x0	-	1	1111	0xx	-	-	VSCCLRM T2

C2.3.1.2.4 Architected coprocessor load, store, and move instructions

This section describes the encoding of the Architected coprocessor load, store, and move instructions. This section is decoded from [Floating-point and vector load/store, move, and coprocessor instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	0	op1			op2	op3																				

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	Instruction
0	!= 0010	1	!= 1111	LDC, LDC2 (immediate) T1
1	!= 0010	1	!= 1111	LDC, LDC2 (immediate) T2
0	!= 0010	1	1111	LDC, LDC2 (literal) T1
1	!= 0010	1	1111	LDC, LDC2 (literal) T2
0	!= 0010	0	-	STC, STC2 T1
1	!= 0010	0	-	STC, STC2 T2
0	0010	1	-	MRRC, MRRC2 T1
1	0010	1	-	MRRC, MRRC2 T2
0	0010	0	-	MCRR, MCRR2 T1
1	0010	0	-	MCRR, MCRR2 T2

C2.3.1.2.5 Vector load instructions

This section describes the encoding of the Vector load instructions. This section is decoded from [Floating-point and vector load/store, move, and coprocessor instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	0	op1			1	op2	Rn0	reg			op3	1	1	1	op4	op5		op6				op7				

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	op4	op5	op6	op7	Instruction
1	01xx	-	1	!= 11	-	-	0	VLD2 T1
1	01xx	-	1	!= 11	-	-	1	VLD4 T1
0	1xxx 0xx1	-	1	00	-	-	-	VLDRB, VLDRH, VLDRW T5
0	1xxx 0xx1	-	1	01	-	-	-	VLDRB, VLDRH, VLDRW T6
0	1xxx 0xx1	-	1	10	-	-	-	VLDRB, VLDRH, VLDRW T7
1	1xxx	-	1	0x	-	-	-	VLDRB, VLDRH, VLDRW, VLDRD (vector) T5
1	1xxx	-	1	1x	-	-	-	VLDRB, VLDRH, VLDRW, VLDRD (vector) T6
-	1x0x 0x01	0	0	!= 11	-	-	-	VLDRB, VLDRH, VLDRW T1
-	1x0x 0x01	1	0	!= 11	-	-	-	VLDRB, VLDRH, VLDRW T2
-	1xxx 0xx1	-	0	11	-	-	-	VLDR (System Register) T1
-	01x0	-	0	-	0	0	-	VLDRB, VLDRH, VLDRW, VLDRD (vector) T1
-	01x0	-	0	-	0	1	-	VLDRB, VLDRH, VLDRW, VLDRD (vector) T2
-	01x0	-	0	-	1	0	-	VLDRB, VLDRH, VLDRW, VLDRD (vector) T3
-	01x0	-	0	-	1	1	-	VLDRB, VLDRH, VLDRW, VLDRD (vector) T4
0	00x0	-	0	1x	-	-	-	VMOV (two general-purpose registers to two 32-bit vector lanes) T1

C2.3.1.2.6 Vector store instructions

This section describes the encoding of the Vector store instructions. This section is decoded from [Floating-point and vector load/store, move, and coprocessor instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	0	op1			0	op2	Rn0	reg			op3	1	1	1	op4	op5		op6				op7				

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	op4	op5	op6	op7	Instruction
1	01xx	-	1	!= 11	-	-	0	VST2 T1
1	01xx	-	1	!= 11	-	-	1	VST4 T1
0	1xxx 0xx1	-	1	00	-	-	-	VSTRB, VSTRH, VSTRW T5
0	1xxx 0xx1	-	1	01	-	-	-	VSTRB, VSTRH, VSTRW T6
0	1xxx 0xx1	-	1	10	-	-	-	VSTRB, VSTRH, VSTRW T7
1	1xxx	-	1	0x	-	-	-	VSTRB, VSTRH, VSTRW, VSTRD (vector) T5
1	1xxx	-	1	1x	-	-	-	VSTRB, VSTRH, VSTRW, VSTRD (vector) T6

-	1x0x 0x01	0	0	!= 11	-	-	-	VSTRB, VSTRH, VSTRW T1
-	1x0x 0x01	1	0	!= 11	-	-	-	VSTRB, VSTRH, VSTRW T2
-	1xxx 0xx1	-	0	11	-	-	-	VSTR (System Register) T1
-	01x0	-	0	-	0	0	-	VSTRB, VSTRH, VSTRW, VSTRD (vector) T1
-	01x0	-	0	-	0	1	-	VSTRB, VSTRH, VSTRW, VSTRD (vector) T2
-	01x0	-	0	-	1	0	-	VSTRB, VSTRH, VSTRW, VSTRD (vector) T3
-	01x0	-	0	-	1	1	-	VSTRB, VSTRH, VSTRW, VSTRD (vector) T4
0	00x0	-	0	1x	-	-	-	VMOV (two 32-bit vector lanes to two general-purpose registers) T1

C2.3.1.3 Miscellaneous vector arithmetic instructions

This section describes the encoding of the Miscellaneous vector arithmetic instructions. This section is decoded from [Coprocessor, floating-point, and vector instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op0	1	1	1	1	op1	D	op2	op3	op4				Qd			0			op5	op6	1	op7	op8					op9

This table shows the decode field values and the associated instructions:

op0	op1	op2	op2:op3	op4	op5	op6	op7:op5	op7:op8	op9	Instruction
1	1	11	-	xx0	0100	0	-	x0	0	VCLS T1
0	0	00	-	xx0	0001	-	-	x1	0	VAND T1
-	1	-	000	-	-	0	1111x 0xxx0 011x1	x1	-	VMOV (immediate) (vector) T1
-	0	-	-	xx0	0000	-	-	x1	0	VQADD T1
Alias										VORN (immediate) T1
-	0	!= 11	-	xx0	0100	-	-	x0	0	VSHL T3
-	1	-	000	-	0xx1 10x1	0	-	11	-	VBIC (immediate) T1
-	0	!= 11	-	xx0	0110	-	-	x1	0	VMIN, VMINA T1
0	0	0x	-	xx0	1101	-	-	x0	0	VADD (floating-point) T1
1	0	1x	-	xx0	1111	-	-	x1	0	VMINNM, VMINNMA (floating-point) T1
-	1	-	!= 000	-	0111	0	-	x1	0	VQSHL, VQSHLU T2
-	0	!= 11	-	xx0	0110	-	-	x0	0	VMAX, VMAXA T1
0	0	-	-	xx0	1000	-	-	x0	0	VADD (vector) T1
-	1	-	!= 000	-	0010	0	-	x1	0	VRSHR T1
-	0	-	-	xx0	0111	-	-	x0	0	VABD T1
-	1	-	!= 000	-	0000	0	-	x1	0	VSHR T1
-	1	-	000	-	0xx1 10x1	0	-	01	-	VORR (immediate) T1
Alias										VMOV (register) (vector) T1
1	0	-	-	xx0	1000	-	-	x0	0	VSUB (vector) T1
0	0	1x	-	xx0	1100	-	-	x1	0	VFMA, VFMS (floating-point) T2
0	0	0x	-	xx0	1100	-	-	x1	0	VFMA, VFMS (floating-point) T1
-	1	-	!= 000	-	11xx	0	-	x1	0	VCVT (between floating-point and fixed-point) (vector) T1
1	1	11	-	xx0	0000	0	-	x0	0	VREV64 T1
0	0	-	-	xx0	1001	-	-	x1	0	VMUL (vector) T1
0	0	1x	-	xx0	1101	-	-	x0	0	VSUB (floating-point) T1

-	0	-	-	xx0	0000	-	-	x0	0	VHADD T1
1	0	0x	-	xx0	1101	-	-	x1	0	VMUL (floating-point) T1
1	1	11	-	x01	0111	0	-	x0	0	VABS (floating-point) T1
1	1	11	-	x00	0111	1	-	x0	0	VQNEG T1
-	0	-	-	xx0	0010	-	-	x0	0	VHSUB T1
1	1	-	110	000	0101	1	-	x0	0	VMVN (register) T1
-	0	!= 11	-	xx0	0101	-	-	x0	0	VRSHL T1
1	0	-	-	xx0	1011	-	-	x0	0	VQDMULH, VQORDMULH T2
Alias										VAND (immediate) T1
1	1	11	-	x00	0111	0	-	x0	0	VQABS T1
1	1	11	-	x01	0011	0	-	x0	0	VABS (vector) T1
1	1	11	-	x10	01xx	-	-	x0	0	VRINT (floating-point) T1
1	0	1x	-	xx0	1101	-	-	x0	0	VABD (floating-point) T1
1	1	11	-	x11	011x	-	-	x0	0	VCVT (between floating-point and integer) T1
0	1	-	!= 000	-	0101	0	-	x1	0	VSHL T1
-	0	-	-	xx0	0001	-	-	x0	0	VRHADD T1
0	0	11	-	xx0	0001	-	-	x1	0	VORN T1
1	1	-	!= 000	-	0101	0	-	x1	0	VSLI T1
1	1	-	!= 000	-	0100	0	-	x1	0	VSRI T1
0	0	10	-	xx0	0001	-	-	x1	0	VORR T1
1	0	00	-	xx0	0001	-	-	x1	0	VEOR T1
0	0	-	-	xx0	1011	-	-	x0	0	VQDMULH, VQORDMULH T1
1	1	11	-	x11	00xx	-	-	x0	0	VCVT (from floating-point to integer) T1
1	1	11	-	x01	0011	1	-	x0	0	VNEG (vector) T1
-	1	-	000	-	0xx0 10x0 110x	0	-	11	-	VMVN (immediate) T1
-	0	!= 11	-	xx0	0100	-	-	x1	0	VQSHL, VQSHLU T4
1	1	11	-	x01	0111	1	-	x0	0	VNEG (floating-point) T1
1	1	11	-	x00	0001	0	-	x0	0	VREV16 T1
1	0	0x	-	xx0	1111	-	-	x1	0	VMAXNM, VMAXNMA (floating-point) T1
0	0	01	-	xx0	0001	-	-	x1	0	VBIC (register) T1
1	1	-	!= 000	-	0110	0	-	x1	0	VQSHL, VQSHLU T3
1	1	11	-	x00	0000	1	-	x0	0	VREV32 T1
-	0	-	-	xx0	0010	-	-	x1	0	VQSUB T1
1	1	11	-	x00	0100	1	-	x0	0	VCLZ T1
-	0	!= 11	-	xx0	0101	-	-	x1	0	VQRSHL T1

C2.3.2 Load/store (multiple, dual, exclusive, acquire-release)

This section describes the encoding of the Load/store (multiple, dual, exclusive, acquire-release). This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	op0			op1																					

This table shows the decode field values and the associated subgroups:

op0	op1	Subgroup
-	0x	Load/store multiple
0	10	Load/store exclusive, load-acquire/store-release

0	11	Load/store dual (post-indexed)
1	10	Load/store dual (literal and immediate)
1	11	Load/store dual (pre-indexed), secure gateway

C2.3.2.1 Load/store multiple

This section describes the encoding of the Load/store multiple. This section is decoded from [Load/store \(multiple, dual, exclusive, acquire-release\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	0	1	0	0	opc	0	W	L					Rn	P	M	(0)	register_list														

This table shows the decode field values and the associated instructions:

L	Rn	opc	Instruction
0	-	01	STM, STMIA, STMEA T2
1	!= 1111	01	LDM, LDMIA, LDMFD T2
1	1111	01	CLRM T1
0	-	10	STMDB, STMFD T1
1	-	10	LDMDB, LDMEA T1
Alias			PUSH (multiple registers) T1
Alias			POP (multiple registers) T2

C2.3.2.2 Load/store exclusive, load-acquire/store-release

This section describes the encoding of the Load/store exclusive, load-acquire/store-release. This section is decoded from [Load/store \(multiple, dual, exclusive, acquire-release\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
1	1	1	0	1	0	0	0	op0	1	0	op1												op2											

This table shows the decode field values and the associated subgroups:

op0	op1	op2	Subgroup
0	!= 0xxxx1111	-	Load/store exclusive
1	-	01x	Load/store exclusive byte/half/dual
1	-	1xx	Load-acquire / Store-release

This table shows the decode field values and the associated instructions:

op0	op1	op2	Instruction
0	0xxxx1111	-	TT, TTT, TTA, TTAT T1
1	1xxxxxxx	000	TBB, TBH T1

C2.3.2.2.1 Load/store exclusive

This section describes the encoding of the Load/store exclusive. This section is decoded from [Load/store exclusive](#), [load-acquire/store-release](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	0	1	0	L	Rn				Rt				(1)(1)(1)(1)				imm8							

This table shows the decode field values and the associated instructions:

L	Instruction
0	STREX T1
1	LDREX T1

C2.3.2.2.2 Load/store exclusive byte/half/dual

This section describes the encoding of the Load/store exclusive byte/half/dual. This section is decoded from [Load/store exclusive](#), [load-acquire/store-release](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	L	Rn				Rt			(1)(1)(1)(1)				0	1	sz	Rd					

This table shows the decode field values and the associated instructions:

L	sz	Instruction
0	00	STREXB T1
0	01	STREXH T1
1	00	LDREXB T1
1	01	LDREXH T1

C2.3.2.2.3 Load-acquire / Store-release

This section describes the encoding of the Load-acquire / Store-release. This section is decoded from [Load/store exclusive](#), [load-acquire/store-release](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	L	Rn			Rt			(1)(1)(1)(1)				1	op	sz	Rd						

This table shows the decode field values and the associated instructions:

L	op	sz	Instruction
0	0	00	STLB T1

0	0	01	STLH T1
0	0	10	STL T1
0	1	00	STLEXB T1
0	1	01	STLEXH T1
0	1	10	STLEX T1
1	0	00	LDAB T1
1	0	01	LDAH T1
1	0	10	LDA T1
1	1	00	LDAEXB T1
1	1	01	LDAEXH T1
1	1	10	LDAEX T1

C2.3.2.3 Load/store dual (post-indexed)

This section describes the encoding of the Load/store dual (post-indexed). This section is decoded from [Load/store \(multiple, dual, exclusive, acquire-release\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	P	U	1	W	L	Rn				Rt				Rt2				imm8							

This table shows the decode field values and the associated instructions:

L	Rn	Instruction
0	!= 1111	STRD (immediate) T1
1	!= 1111	LDRD (immediate) T1
-	1111	UNPREDICTABLE

C2.3.2.4 Load/store dual (literal and immediate)

This section describes the encoding of the Load/store dual (literal and immediate). This section is decoded from [Load/store \(multiple, dual, exclusive, acquire-release\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	P	U	1	W	L	Rn				Rt				Rt2				imm8							

This table shows the decode field values and the associated instructions:

L	Rn	Instruction
1	1111	LDRD (literal) T1
0	!= 1111	STRD (immediate) T1
1	!= 1111	LDRD (immediate) T1

C2.3.2.5 Load/store dual (pre-indexed), secure gateway

This section describes the encoding of the Load/store dual (pre-indexed), secure gateway. This section is decoded from [Load/store \(multiple, dual, exclusive, acquire-release\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	1	op0	1	1	op1	op2		op3																	

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	Instruction
0	0	1111	-	UNPREDICTABLE
0	1	1111	!= 1110100101111111	UNPREDICTABLE
0	1	1111	1110100101111111	SG T1
-	0	!= 1111	-	STRD (immediate) T1
1	-	1111	-	UNPREDICTABLE
-	1	!= 1111	-	LDRD (immediate) T1

C2.3.3 Data-processing (shifted register)

This section describes the encoding of the Data-processing (shifted register). This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	1	0	1	op0				S	Rn				L	imm3				Rd				op1				Rm			

This table shows the decode field values and the associated subgroups:

op0	Subgroup
0010	Wide shift, shift, and conditional instructions
!= 0010	Shifted register instructions

C2.3.3.1 Wide shift, shift, and conditional instructions

This section describes the encoding of the Wide shift, shift, and conditional instructions. This section is decoded from [Data-processing \(shifted register\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	S	Rn		op0		op1	op2						op3		op4						

This table shows the decode field values and the associated instructions:

Rn	S	op0	op1	op2	op3	op4	Instruction
xxx0	1	0	-	!= 111	10	1111	ASRL (immediate) T1

xxx0	1	0	-	!= 111	00	1111	LSLL (immediate) T1
xxx0	1	0	-	!= 111	01	1111	LSRL (immediate) T1
-	1	0	-	!= 111	11	1111	SQSHLL (immediate) T1
xxx1	1	0	-	!= 111	10	1111	SRSHRL (immediate) T1
xxx1	1	0	-	!= 111	00	1111	UQSHLL (immediate) T1
xxx1	1	0	-	!= 111	01	1111	URSHRL (immediate) T1
-	1	0	-	111	11	1111	SQSHL (immediate) T1
-	1	0	-	111	10	1111	SRSHR (immediate) T1
-	1	0	-	111	00	1111	UQSHL (immediate) T1
-	1	0	-	111	01	1111	URSHR (immediate) T1
xxx0	1	-	-	!= 111	10	1101	ASRL (register) T1
xxx0	1	-	-	!= 111	00	1101	LSLL (register) T1
-	1	-	-	111	10	1101	SQRSHR (register) T1
xxx1	1	-	-	!= 111	10	1101	SQRSHRL (register) T1
-	1	-	-	111	00	1101	UQRSHL (register) T1
xxx1	1	-	-	!= 111	00	1101	UQRSHLL (register) T1
!= 1111	0	0	-	-	-	-	ORR (register) T2
!= 1111	1	0	-	-	-	!= 11x1	ORRS (register) T2
1111	0	0	-	-	-	-	MOV (register) T3
1111	1	0	-	-	-	!= 11x1	MOVS (register) T3
Alias							ASR (immediate) T3
Alias							ASRS (immediate) T3
Alias							LSL (immediate) T3
Alias							LSLS (immediate) T3
Alias							LSR (immediate) T3
Alias							LSRS (immediate) T3
Alias							ROR (immediate) T3
Alias							RORS (immediate) T3
Alias							RRX T3
Alias							RRXS T3
-	1	1	00	-	-	!= 1101	CSEL T1
-	1	1	01	-	-	!= 1101	CSINC T1
-	1	1	10	-	-	!= 1101	CSINV T1
-	1	1	11	-	-	!= 1101	CSNEG T1
Alias							CINC T1
Alias							CINV T1
Alias							CNEG T1
Alias							CSET T1
Alias							CSETM T1

C2.3.3.2 Shifted register instructions

This section describes the encoding of the Shifted register instructions. This section is decoded from [Data-processing \(shifted register\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	op0			S	Rn			(0)	imm3			Rd			imm2			sr_type		Rm					

This table shows the decode field values and the associated instructions:

Rd	Rn	S	imm3:imm2:sr_type	op0	Instruction
-	-	0	!= 0000011	0000	AND (register) T2

-	-	0	0000011	0000	AND extend (register) T2
!= 1111	-	1	!= 0000011	0000	ANDS (register) T2
!= 1111	-	1	0000011	0000	ANDS extend (register) T2
1111	-	1	!= 0000011	0000	TST (register) T2
1111	-	1	0000011	0000	TST extend (register) T2
-	-	-	-	0001	BIC (register) T2
-	!= 1111	0	-	0011	ORN (register) T1
-	!= 1111	1	-	0011	ORNS (register) T1
-	1111	0	-	0011	MVN (register) T2
-	1111	1	-	0011	MVNS (register) T2
-	-	0	!= 0000011	0100	EOR (register) T2
-	-	0	0000011	0100	EOR extend (register) T2
!= 1111	-	1	!= 0000011	0100	EORS (register) T2
!= 1111	-	1	0000011	0100	EORS extend (register) T2
1111	-	1	!= 0000011	0100	TEQ (register) T1
1111	-	1	0000011	0100	TEQ extend (register) T1
-	-	0	xxxxxx0	0110	PKHBT, PKHTB T1
-	!= 1101	0	-	1000	ADD (register) T3
!= 1111	!= 1101	1	-	1000	ADDS (register) T3
-	1101	0	-	1000	ADD (SP plus register) T3
!= 1111	1101	1	-	1000	ADDS (SP plus register) T3
1111	-	1	-	1000	CMN (register) T2
-	-	-	-	1010	ADC (register) T2
-	-	-	-	1011	SBC (register) T2
-	!= 1101	0	-	1101	SUB (register) T2
!= 1111	!= 1101	1	-	1101	SUBS (register) T2
-	1101	0	-	1101	SUB (SP minus register) T1
!= 1111	1101	1	-	1101	SUBS (SP minus register) T1
1111	-	1	-	1101	CMP (register) T3
-	-	-	-	1110	RSB (register) T1

C2.3.4 Branches and miscellaneous control

This section describes the encoding of the Branches and miscellaneous control. This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	op0		op1		op2						1	op3		op4			op5										op6

This table shows the decode field values and the associated subgroups:

op0	op1	op2	op3	op4	op5	op6	Subgroup
0	1110	10	0	0	000	-	Hints
0	1110	11	0	0	-	-	Miscellaneous system
1	1111	1x	0	0	-	-	Exception generation
-	-	-	1	0	-	1	Loop and branch instructions

This table shows the decode field values and the associated instructions:

op0	op1	op2	op3	op4	op5	op6	Instruction
0	1110	0x	0	0	-	-	MSR (register) T1
0	1111	1x	0	0	-	-	MRS T1
-	!= 111x	-	0	0	-	-	B T3
-	-	-	0	1	-	-	B T4
-	-	-	1	1	-	-	BL T1

C2.3.4.1 Hints

This section describes the encoding of the Hints. This section is decoded from [Branches and miscellaneous control](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	hint				option			

This table shows the decode field values and the associated instructions:

hint	option	Instruction
0000	0000	NOP T2
0000	0001	YIELD T2
0000	0010	WFE T2
0000	0011	WFI T2
0000	0100	SEV T2
0000	0101	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0000	011x	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0000	10xx	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0000	11x0	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0000	1101	PACBTI T1
0000	1111	BTI T1
0001	0000	ESB T1
0001	001x	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0001	0100	CSDB T1
0001	0x01	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0001	011x	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0001	10xx	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0001	11x0	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0001	1101	PAC T1
0001	1111	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0010	0xxx	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0010	10xx	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0010	11x0	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0010	1101	AUT T1
0010	1111	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
0011	-	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
01xx	-	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
10xx	-	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
110x	-	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
1110	-	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED
1111	-	DBG T1

C2.3.4.2 Miscellaneous system

This section describes the encoding of the Miscellaneous system. This section is decoded from [Branches and miscellaneous control](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	1	(1)	(1)	(1)	(1)	1	0	(0)	0	(1)	(1)	(1)	(1)	opc				option			

This table shows the decode field values and the associated instructions:

opc	option	Instruction
0010	-	CLREX T1
0100	!= 0x00	DSB T1
0100	0000	SSBB T1
0100	0100	PSSBB T1
0101	-	DMB T1
0110	-	ISB T1

C2.3.4.3 Exception generation

This section describes the encoding of the Exception generation. This section is decoded from [Branches and miscellaneous control](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	1	1	1	1	1	o1	imm4				1	0	o2	0	imm12											

This table shows the decode field values and the associated instructions:

o1	o2	Instruction
1	1	UDF T2

C2.3.4.4 Loop and branch instructions

This section describes the encoding of the Loop and branch instructions. This section is decoded from [Branches and miscellaneous control](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	op0											1	1			0											1

This table shows the decode field values and the associated subgroups:

op0	Subgroup
!= 0000	Branch future instructions
0000	Loop instructions

C2.3.4.4.1 Branch future instructions

This section describes the encoding of the Branch future instructions. This section is decoded from [Loop and branch instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	boff				op0								1	1	op1		0	immC		immB								1

This table shows the decode field values and the associated instructions:

op0	op1	Instruction
-	0	BF, BFX, BFL, BFLX, BFCSEL T4
0xx	1	BF, BFX, BFL, BFLX, BFCSEL T2
10x	1	BF, BFX, BFL, BFLX, BFCSEL T1
110	1	BF, BFX, BFL, BFLX, BFCSEL T3
111	1	BF, BFX, BFL, BFLX, BFCSEL T5

C2.3.4.4.2 Loop instructions

This section describes the encoding of the Loop instructions. This section is decoded from [Loop and branch instructions](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	i	op0				op1				1	1	op2	0	imml	immh								1	

This table shows the decode field values and the associated instructions:

imml	op0	op1	op2	Instruction
-	0xx	1111	1	LCTP T1
-	000	1111	0	LE, LETP T1
-	01x	1111	0	LE, LETP T2
-	001	1111	0	LE, LETP T3
-	1xx	-	0	WLS, DLS, WLSTP, DLSTP T1
-	1xx	-	1	WLS, DLS, WLSTP, DLSTP T2
-	0xx	!= 1111	0	WLS, DLS, WLSTP, DLSTP T3
0	0xx	!= 1111	1	WLS, DLS, WLSTP, DLSTP T4
1	0xx	!= 1111	1	VCTP T1

C2.3.5 Data-processing (modified immediate)

This section describes the encoding of the Data-processing (modified immediate). This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	op1				S	Rn				0	imm3				Rd				imm8							

This table shows the decode field values and the associated instructions:

Rd	Rn	S	op1	Instruction
-	-	0	0000	AND (immediate) T1
!= 1111	-	1	0000	ANDS (immediate) T1
1111	-	1	0000	TST (immediate) T1
-	-	-	0001	BIC (immediate) T1
-	!= 1111	-	0010	ORR (immediate) T1
-	1111	-	0010	MOV (immediate) T2
-	!= 1111	-	0011	ORN (immediate) T1
-	1111	-	0011	MVN (immediate) T1
-	-	0	0100	EOR (immediate) T1
!= 1111	-	1	0100	EORS (immediate) T1
1111	-	1	0100	TEQ (immediate) T1
-	!= 1101	0	1000	ADD (immediate) T3
!= 1111	!= 1101	1	1000	ADDS (immediate) T3
-	1101	0	1000	ADD (SP plus immediate) T3
!= 1111	1101	1	1000	ADDS (SP plus immediate) T3
1111	-	1	1000	CMN (immediate) T1
-	-	-	1010	ADC (immediate) T1
-	-	-	1011	SBC (immediate) T1
-	!= 1101	0	1101	SUB (immediate) T3
!= 1111	!= 1101	1	1101	SUBS (immediate) T3
-	1101	0	1101	SUB (SP minus immediate) T2
!= 1111	1101	1	1101	SUBS (SP minus immediate) T2
1111	-	1	1101	CMP (immediate) T2
-	-	-	1110	RSB (immediate) T2

C2.3.6 Data-processing (plain binary immediate)

This section describes the encoding of the Data-processing (plain binary immediate). This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0		1	op0		op1							0															

This table shows the decode field values and the associated subgroups:

op0	op1	Subgroup
0	0x	Data-processing (simple immediate)
0	10	Move Wide (16-bit immediate)
1	-	Saturate, bitfield

C2.3.6.1 Data-processing (simple immediate)

This section describes the encoding of the Data-processing (simple immediate). This section is decoded from [Data-processing \(plain binary immediate\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	1	0	o1	0	o2	0	Rn				0	imm3			Rd			imm8								

This table shows the decode field values and the associated instructions:

Rn	o1	o2	Instruction
Alias			ADD (immediate, to PC) T3
!= 11x1	0	0	ADD (immediate) T4
1101	0	0	ADD (SP plus immediate) T4
1111	0	0	ADR T3
!= 11x1	1	1	SUB (immediate) T4
1101	1	1	SUB (SP minus immediate) T3
1111	1	1	ADR T2
Alias			SUB (immediate, from PC) T2

C2.3.6.2 Move Wide (16-bit immediate)

This section describes the encoding of the Move Wide (16-bit immediate). This section is decoded from [Data-processing \(plain binary immediate\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	1	0	o1	1	0	0	imm4				0	imm3			Rd			imm8								

This table shows the decode field values and the associated instructions:

o1	Instruction
0	MOV (immediate) T3
1	MOVT T1

C2.3.6.3 Saturate, bitfield

This section describes the encoding of the Saturate, bitfield. This section is decoded from [Data-processing \(plain binary immediate\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	(0)	1	1	op1	0	Rn				0	imm3			Rd			imm2		(0)	imm5							

This table shows the decode field values and the associated instructions:

Rn	imm3:imm2	op1	Instruction
-	-	000	SSAT LSL T1
-	!= 00000	001	SSAT ASR T1
-	00000	001	SSAT16 T1
-	-	010	SBFX T1
!= 1111	-	011	BFI T1
1111	-	011	BFC T1
-	-	100	USAT LSL T1
-	!= 00000	101	USAT ASR T1
-	00000	101	USAT16 T1
-	-	110	UBFX T1

C2.3.7 Load/store single

This section describes the encoding of the Load/store single. This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	op0					op1		op2						op3											

This table shows the decode field values and the associated subgroups:

op0	op1	op2	op3	Subgroup
00	-	!= 1111	000000	Load/store, unsigned (register offset)
00	-	!= 1111	10xxxx	Load/store, unsigned (immediate, post-indexed)
00	-	!= 1111	1100xx	Load/store, unsigned (negative immediate)
00	-	!= 1111	1110xx	Load/store, unsigned (unprivileged)
00	-	!= 1111	11x1xx	Load/store, unsigned (immediate, pre-indexed)
01	-	!= 1111	-	Load/store, unsigned (positive immediate)
0x	-	1111	-	Load, unsigned (literal)
10	1	!= 1111	000000	Load/store, signed (register offset)
10	1	!= 1111	10xxxx	Load/store, signed (immediate, post-indexed)
10	1	!= 1111	1100xx	Load/store, signed (negative immediate)
10	1	!= 1111	1110xx	Load/store, signed (unprivileged)
10	1	!= 1111	11x1xx	Load/store, signed (immediate, pre-indexed)
11	1	!= 1111	-	Load/store, signed (positive immediate)
1x	1	1111	-	Load, signed (literal)

C2.3.7.1 Load/store, unsigned (register offset)

This section describes the encoding of the Load/store, unsigned (register offset). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	size	L	Rn				Rt				0	0	0	0	0	0	imm2	Rm					

This table shows the decode field values and the associated instructions:

L	Rt	size	Instruction
0	-	00	STRB (register) T2
1	!= 1111	00	LDRB (register) T2
1	1111	0x	PLD, PLDW (register) T1
0	-	01	STRH (register) T2
1	!= 1111	01	LDRH (register) T2
0	-	10	STR (register) T2
1	-	10	LDR (register) T2

C2.3.7.2 Load/store, unsigned (immediate, post-indexed)

This section describes the encoding of the Load/store, unsigned (immediate, post-indexed). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	1	1	1	0	0	0	0	size	L											P	U	W												
																Rn				Rt				1				imm8							

This table shows the decode field values and the associated instructions:

L	size	Instruction
0	00	STRB (immediate) T3
1	00	LDRB (immediate) T3
0	01	STRH (immediate) T3
1	01	LDRH (immediate) T3
0	10	STR (immediate) T4
	Alias	PUSH (single register) T4
1	10	LDR (immediate) T4
	Alias	POP (single register) T4

C2.3.7.3 Load/store, unsigned (negative immediate)

This section describes the encoding of the Load/store, unsigned (negative immediate). This section is decoded from [Load/store single](#).

[illegible]

This table shows the decode field values and the associated instructions:

L	Rt	size	Instruction
0	-	00	STRB (immediate) T3
1	!= 1111	00	LDRB (immediate) T3
1	1111	0x	PLD, PLDW (immediate) T2
0	-	01	STRH (immediate) T3
1	!= 1111	01	LDRH (immediate) T3

0	-	10	STR (immediate) T4
Alias			PUSH (single register) T4
1	-	10	LDR (immediate) T4
Alias			POP (single register) T4

C2.3.7.4 Load/store, unsigned (unprivileged)

This section describes the encoding of the Load/store, unsigned (unprivileged). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	size	L	Rn			Rt			1	1	1	0	imm8										

This table shows the decode field values and the associated instructions:

L	size	Instruction
0	00	STRBT T1
1	00	LDRBT T1
0	01	STRHT T1
1	01	LDRHT T1
0	10	STRT T1
1	10	LDRT T1

C2.3.7.5 Load/store, unsigned (immediate, pre-indexed)

This section describes the encoding of the Load/store, unsigned (immediate, pre-indexed). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	size	L	Rn			Rt			1	P	U	W	imm8										

This table shows the decode field values and the associated instructions:

L	size	Instruction
0	00	STRB (immediate) T3
1	00	LDRB (immediate) T3
0	01	STRH (immediate) T3
1	01	LDRH (immediate) T3
0	10	STR (immediate) T4
Alias		PUSH (single register) T4
1	10	LDR (immediate) T4
Alias		POP (single register) T4

C2.3.7.6 Load/store, unsigned (positive immediate)

This section describes the encoding of the Load/store, unsigned (positive immediate). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	1	size	L						Rt															

This table shows the decode field values and the associated instructions:

L	Rt	size	Instruction
0	-	00	STRB (immediate) T2
1	!= 1111	00	LDRB (immediate) T2
1	1111	0x	PLD, PLDW (immediate) T1
0	-	01	STRH (immediate) T2
1	!= 1111	01	LDRH (immediate) T2
0	-	10	STR (immediate) T3
1	-	10	LDR (immediate) T3

C2.3.7.7 Load, unsigned (literal)

This section describes the encoding of the Load, unsigned (literal). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	U	size	L	1	1	1	1		Rt															

This table shows the decode field values and the associated instructions:

L	Rt	size	Instruction
1	!= 1111	00	LDRB (literal) T1
1	1111	0x	PLD (literal) T1
1	!= 1111	01	LDRH (literal) T1
1	-	10	LDR (literal) T2

C2.3.7.8 Load/store, signed (register offset)

This section describes the encoding of the Load/store, signed (register offset). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	0	size	1						Rt															

This table shows the decode field values and the associated instructions:

Rt	size	Instruction
!= 1111	00	LDRSB (register) T2
1111	00	PLI (register) T1
!= 1111	01	LDRSH (register) T2
1111	01	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED

C2.3.7.9 Load/store, signed (immediate, post-indexed)

This section describes the encoding of the Load/store, signed (immediate, post-indexed). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
1	1	1	1	1	0	0	1	0	size	1		Rn					Rt					1	P	U	W		imm8							

This table shows the decode field values and the associated instructions:

size	Instruction
00	LDRSB (immediate) T2
01	LDRSH (immediate) T2

C2.3.7.10 Load/store, signed (negative immediate)

This section describes the encoding of the Load/store, signed (negative immediate). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	0	size	1					Rn																
																Rt															imm12

This table shows the decode field values and the associated instructions:

Rt	size	Instruction
!= 1111	00	LDRSB (immediate) T2
1111	00	PLI (immediate, literal) T2
!= 1111	01	LDRSH (immediate) T2
1111	01	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED

C2.3.7.11 Load/store, signed (unprivileged)

This section describes the encoding of the Load/store, signed (unprivileged). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	0	1	0	size	1	Rn					Rt					1	1	1	0	imm8							

This table shows the decode field values and the associated instructions:

size	Instruction
00	LDRSBT T1
01	LDRSHT T1

C2.3.7.12 Load/store, signed (immediate, pre-indexed)

This section describes the encoding of the Load/store, signed (immediate, pre-indexed). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	0	1	0	size	1	Rn					Rt					1	P	U	W	imm8							

This table shows the decode field values and the associated instructions:

size	Instruction
00	LDRSB (immediate) T2
01	LDRSH (immediate) T2

C2.3.7.13 Load/store, signed (positive immediate)

This section describes the encoding of the Load/store, signed (positive immediate). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	0	1	1	size	1	Rn					Rt					imm12											

This table shows the decode field values and the associated instructions:

Rt	size	Instruction
!= 1111	00	LDRSB (immediate) T1
1111	00	PLI (immediate, literal) T1
!= 1111	01	LDRSH (immediate) T1
1111	01	Reserved hint, if the Main extension is implemented it behaves as NOP, else UNDEFINED

C2.3.7.14 Load, signed (literal)

This section describes the encoding of the Load, signed (literal). This section is decoded from [Load/store single](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	0	1	U	size	1	1	1	1	1		Rt					imm12											

This table shows the decode field values and the associated instructions:

Rt	size	Instruction
!= 1111	00	LDRSB (literal) T1
1111	00	PLI (immediate, literal) T3
!= 1111	01	LDRSH (literal) T1
1111	01	Reserved hint, if the Main extension is implemented it behaves as NOP , else UNDEFINED

C2.3.8 Data-processing (register)

This section describes the encoding of the Data-processing (register). This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	1	0	op0																	op1							

This table shows the decode field values and the associated subgroups:

op0	op1	Subgroup
0	1xxx	Register extends
1	0xxx	Parallel add-subtract
1	10xx	Data-processing (two source registers)

This table shows the decode field values and the associated instructions:

op0	op1	Instruction
Alias		ASRS (register) T2
Alias		ASR (register) T2
Alias		LSLS (register) T2
Alias		LSL (register) T2
Alias		LSRS (register) T2
Alias		LSR (register) T2
0	0000	MOV, MOVS (register-shifted register) T2
Alias		RORS (register) T2
Alias		ROR (register) T2

C2.3.8.1 Register extends

This section describes the encoding of the Register extends. This section is decoded from [Data-processing \(register\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0	op1	U	Rn					1	1	1	1	Rd				1	(0)	rot	Rm				

This table shows the decode field values and the associated instructions:

Rn	U	op1	Instruction
!= 1111	0	00	SXTAH T1
1111	0	00	SXTH T2
!= 1111	1	00	UXTAH T1
1111	1	00	UXTH T2
!= 1111	0	01	SXTAB16 T1
1111	0	01	SXTB16 T1
!= 1111	1	01	UXTAB16 T1
1111	1	01	UXTB16 T1
!= 1111	0	10	SXTAB T1
1111	0	10	SXTB T2
!= 1111	1	10	UXTAB T1
1111	1	10	UXTB T2

C2.3.8.2 Parallel add-subtract

This section describes the encoding of the Parallel add-subtract. This section is decoded from [Data-processing \(register\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	op1	Rn						1	1	1	1	Rd				0	U	H	S	Rm			

This table shows the decode field values and the associated instructions:

H	S	U	op1	Instruction
0	0	0	000	SADD8 T1
0	1	0	000	QADD8 T1
1	0	0	000	SHADD8 T1
0	0	1	000	UADD8 T1
0	1	1	000	UQADD8 T1
1	0	1	000	UHADD8 T1
0	0	0	001	SADD16 T1
0	1	0	001	QADD16 T1
1	0	0	001	SHADD16 T1
0	0	1	001	UADD16 T1
0	1	1	001	UQADD16 T1
1	0	1	001	UHADD16 T1
0	0	0	010	SASX T1
0	1	0	010	QASX T1
1	0	0	010	SHASX T1

0	0	1	010	UASX T1
0	1	1	010	UQASX T1
1	0	1	010	UHASX T1
0	0	0	100	SSUB8 T1
0	1	0	100	QSUB8 T1
1	0	0	100	SHSUB8 T1
0	0	1	100	USUB8 T1
0	1	1	100	UQSUB8 T1
1	0	1	100	UHSUB8 T1
0	0	0	101	SSUB16 T1
0	1	0	101	QSUB16 T1
1	0	0	101	SHSUB16 T1
0	0	1	101	USUB16 T1
0	1	1	101	UQSUB16 T1
1	0	1	101	UHSUB16 T1
0	0	0	110	SSAX T1
0	1	0	110	QSAX T1
1	0	0	110	SHSAX T1
0	0	1	110	USAX T1
0	1	1	110	UQSAX T1
1	0	1	110	UHSAX T1

C2.3.8.3 Data-processing (two source registers)

This section describes the encoding of the Data-processing (two source registers). This section is decoded from [Data-processing \(register\)](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	op0			Rn				1	1	1	1	Rd				1	0	op1		Rm			

This table shows the decode field values and the associated instructions:

op0	op1	Instruction
000	00	QADD T1
000	01	QDADD T1
000	10	QSUB T1
000	11	QDSUB T1
001	00	REV T2
001	01	REV16 T2
001	10	RBIT T1
001	11	REVSH T2
010	00	SEL T1
011	00	CLZ T1

C2.3.9 Multiply, multiply accumulate, and absolute difference

This section describes the encoding of the Multiply, multiply accumulate, and absolute difference. This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	op1			Rn			Ra			Rd			0	0	op2			Rm					

This table shows the decode field values and the associated instructions:

Ra	Rd	op1	op2	Instruction
!= 1111	-	000	00	MLA T1
-	-	000	01	MLS T1
1111	-	000	00	MUL T2
!= 1111	-	001	-	SMLABB, SMLABT, SMLATB, SMLATT T1
1111	-	001	-	SMULBB, SMULBT, SMULTB, SMULTT T1
!= 1111	-	010	0x	SMLAD, SMLADX T1
1111	-	010	0x	SMUAD, SMUADX T1
!= 1111	-	011	0x	SMLAWB, SMLAWT T1
1111	-	011	0x	SMULWB, SMULWT T1
!= 1111	-	100	0x	SMLSD, SMLSDX T1
1111	-	100	0x	SMUSD, SMUSDX T1
!= 1111	!= 1111	101	0x	SMMLA, SMMLAR T1
!= 1111	1111	101	00	AUTG T1
!= 1111	1111	101	01	BXAUT T1
1111	-	101	0x	SMMUL, SMMULR T1
!= 1111	-	110	0x	SMMLS, SMMLSR T1
1111	-	110	00	PACG T1
!= 1111	-	111	00	USADA8 T1
1111	-	111	00	USAD8 T1

C2.3.10 Long multiply and divide

This section describes the encoding of the Long multiply and divide. This section is decoded from [32-bit T32 instruction encoding](#).

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	op1			Rn			RdLo			RdHi			op2			Rm							

This table shows the decode field values and the associated instructions:

op1	op2	Instruction
000	0000	SMULL T1
001	1111	SDIV T1
010	0000	UMULL T1
011	1111	UDIV T1
100	0000	SMLAL T1
100	10xx	SMLALBB, SMLALBT, SMLALTB, SMLALTT T1
100	110x	SMLALD, SMLALDX T1
101	110x	SMLS LD, SMLS LDx T1
110	0000	UMLAL T1
110	0110	UMAAL T1

C2.4 Alphabetical list of instructions

Every Armv8-M instruction is listed in this section. See Instruction Set Overview for the format of the instruction descriptions.

C2.4.1 ADC (immediate)

Add with Carry (immediate). Add with Carry (immediate) adds an immediate value and the carry flag value to a register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	1	0	1	0	S	Rn			0	imm3			Rd			imm8									

ADC variant

Applies when **S** == 0.

ADC{<c>}{<q>} {<Rd>}, {<Rn>, #<const>

ADCS variant

Applies when **S** == 1.

ADCS{<c>}{<q>} {<Rd>}, {<Rn>, #<const>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1'); imm32 = T32ExpandImm(i:imm3:imm8);
3 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (result, carry, overflow) = AddWithCarry(R[n], imm32, APSR.C);
4   R[d] = result;
5   if setflags then
6     APSR.N = result[31];
7     APSR.Z = IsZeroBit(result);
8     APSR.C = carry;
9     APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:

- The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.2 ADC (register)

Add with Carry (register). Add with Carry (register) adds a register value, the carry flag value, and an optionally-shifted register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	1	0	1	Rm			Rdn		

T1 variant

```
ADC<c>{<q>} {<Rdn>,,} <Rdn>, <Rm>
    // Inside IT block
ADCS{<q>} {<Rdn>,,} <Rdn>, <Rm>
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rdn); n = UInt(Rdn); m = UInt(Rm); setflags = !InITBlock();
2 (shift_t, shift_n) = (SRTType_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	1	0	1	0	1	1	0	1	0	S	Rn			(0)	imm3			Rd			imm2						Rm							
																											sr type								

ADC, rotate right with extend variant

Applies when **S == 0** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
ADC{<c>}{<q>} {<Rd>,,} <Rn>, <Rm>, RRX
```

ADC, shift or rotate by value variant

Applies when **S == 0** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
ADC<c>.W {<Rd>,,} <Rn>, <Rm>
    // Inside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
ADC{<c>}{<q>} {<Rd>,,} <Rn>, <Rm> {, <shift> #<amount>}
```

ADCS, rotate right with extend variant

Applies when **S == 1** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
ADCS{<c>}{<q>} {<Rd>,,} <Rn>, <Rm>, RRX
```

ADCS, shift or rotate by value variant

Applies when **S == 1** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
ADCS.W {<Rd>}, <Rn>, <Rm>
// Outside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
ADCS{<c>}{<q>} {<Rd>}, <Rn>, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
3 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdn>	Is the first general-purpose source register and the destination register, encoded in the "Rdn" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   shifted = Shift(R[m], shift_t, shift_n, APSR.C);
4   (result, carry, overflow) = AddWithCarry(R[n], shifted, APSR.C);
5   R[d] = result;
6   if setflags then
7     APSR.N = result[31];
8     APSR.Z = IsZeroBit(result);
9     APSR.C = carry;
10    APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.3 ADD (SP plus immediate)

Add to SP (immediate). ADD (SP plus immediate) adds an immediate value to the SP value, and writes the result to the destination register.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	1	Rd			imm8							

T1 variant

ADD{<c>}{<q>} <Rd>, SP, #<imm8>

Decode for this encoding

```
1 d = UInt(Rd); setflags = FALSE; imm32 = ZeroExtend(imm8:'00', 32);
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	0	0	0	imm7						

T2 variant

ADD{<c>}{<q>} {SP,} SP, #<imm7>

Decode for this encoding

```
1 d = 13; setflags = FALSE; imm32 = ZeroExtend(imm7:'00', 32);
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	1	0	0	0	S	1	1	0	1	0	imm3			Rd			imm8								

ADD variant

Applies when **S == 0**.

```
ADD{<c>}.W {<Rd>,} SP, #<const>
// <Rd>, <const> can be represented in T1 or T2
ADD{<c>}{<q>} {<Rd>,} SP, #<const>
```

ADDS variant

Applies when **S == 1 && Rd != 1111**.

```
ADDS{<c>}{<q>} {<Rd>,} SP, #<const>
```

Decode for this encoding


```
1 if Rd == '1111' && S == '1' then SEE "CMN (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); setflags = (S == '1'); imm32 = T32ExpandImm(i:imm3:imm8);
4 if d == 15 && S == '0' then UNPREDICTABLE;
```

T4

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	1	0	0	0	0	0	1	1	0	1	0	imm3															imm8

T4 variant

```
ADD{<c>}{<q>} {<Rd>}, SP, #<imm12>
    // <imm12> cannot be represented in T1, T2, or T3
ADDW{<c>}{<q>} {<Rd>}, SP, #<imm12>
    // <imm12> can be represented in T1, T2, or T3
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); setflags = FALSE; imm32 = ZeroExtend(i:imm3:imm8, 32);
3 if d == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<imm7>	Is an unsigned immediate, a multiple of 4 in the range 0 to 508, encoded in the "imm7" field as <imm7>/4.
<Rd>	For encoding T1: is the general-purpose destination register, encoded in the "Rd" field. For encoding T3 and T4: is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the SP.
<imm8>	Is an unsigned immediate, a multiple of 4, in the range 0 to 1020, encoded in the "imm8" field as <imm8>/4.
<imm12>	Is a 12-bit unsigned immediate, in the range 0 to 4095, encoded in the "i:imm3:imm8" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     (result, carry, overflow) = AddWithCarry(SP, imm32, '0');
4     RSPCheck[d] = result;
5     if setflags then
6         APSR.N = result[31];
7         APSR.Z = IsZeroBit(result);
8         APSR.C = carry;
9         APSR.V = overflow;
```

C2.4.4 ADD (SP plus register)

Add to SP (register). ADD (SP plus register) adds an optionally-shifted register value to the SP value, and writes the result to the destination register.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	DM	1	1	0	1	Rdm		

T1 variant

ADD{<c>}{<q>} {<Rdm>}, SP, <Rdm>

Decode for this encoding

```
1 d = UInt(DM:Rdm); m = UInt(DM:Rdm); setflags = FALSE;
2 if d == 15 && InITBlock() && !LastInITBlock() then UNPREDICTABLE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	1	Rm			1	0	1	

T2 variant

ADD{<c>}{<q>} {SP}, SP, <Rm>

Decode for this encoding

```
1 if Rm == '1101' then SEE "encoding T1";
2 d = 13; m = UInt(Rm); setflags = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	1	0	0	0	S	1	1	0	1	(0)	imm3			Rd			imm2			Rm					

sr type

sr_type

ADD, rotate right with extend variant

Applies when S == 0 && imm3 == 000 && imm2 == 00 && sr_type == 11.

ADD{<c>}{<q>} {<Rd>}, SP, <Rm>, RRX

ADD, shift or rotate by value variant

Applies when S == 0 && !(imm3 == 000 && imm2 == 00 && sr_type == 11).

```
ADD{<c>}.W {<Rd>}, {<Rm>}
    // <Rd>, <Rm> can be represented in T1 or T2
ADD{<c>}{<q>} {<Rd>}, {<Rm>} {, <shift> #<amount>}
```

ADDS, rotate right with extend variant

Applies when `S == 1 && imm3 == 000 && Rd != 1111 && imm2 == 00 && sr_type == 11`.

```
ADDS{<c>}{<q>} {<Rd>}, {<Rm>} SP, <Rm>, RRX
```

ADDS, shift or rotate by value variant

Applies when `S == 1 && !(imm3 == 000 && imm2 == 00 && sr_type == 11) && Rd != 1111`.

```
ADDS{<c>}{<q>} {<Rd>}, {<Rm>} {, <shift> #<amount>}
```

Decode for this encoding

```
1 if Rd == '1111' && S == '1' then SEE "CMN (register)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); m = UInt(Rm); setflags = (S == '1');
4 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
5 if d == 13 && (shift_t != SRType_LSL || shift_n > 3) then UNPREDICTABLE;
6 if (d == 15 && S == '0') || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the general-purpose destination and second source register, encoded in the "Rdm" field. If omitted, this register is the SP. Arm deprecates using the PC as the destination register, but if the PC is used, the instruction is a simple branch to the address calculated by the operation.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the SP.
<Rm>	For encoding T2: is the second general-purpose source register, encoded in the "Rm" field. The PC can be used, but this is deprecated. For encoding T3: is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     shifted = Shift(R[m], shift_t, shift_n, APSR.C);
4     (result, carry, overflow) = AddWithCarry(SP, shifted, '0');
5     if d == 15 then
6         BranchTo(result); // setflags is always FALSE here
7     else
8         RSPCheck[d] = result;
9         if setflags then
10             APSR.N = result[31];
11             APSR.Z = IsZeroBit(result);
12             APSR.C = carry;
```

13 `APSR.V = overflow;`

C2.4.5 ADD (immediate)

Add (immediate). Add (immediate) adds an immediate value to a register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	0	imm3			Rn			Rd		

T1 variant

```
ADD<c>{<q>} <Rd>, <Rn>, #<imm3>
    // Inside IT block
ADDS{<q>} <Rd>, <Rn>, #<imm3>
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rd); n = UInt(Rn); setflags = !InITBlock(); imm32 = ZeroExtend(imm3, 32);
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	0	Rdn			imm8							

T2 variant

```
ADD<c>{<q>} <Rdn>, #<imm8>
    // Inside IT block, and <Rdn>, <imm8> can be represented in T1
ADD<c>{<q>} {<Rdn>, } <Rdn>, #<imm8>
    // Inside IT block, and <Rdn>, <imm8> cannot be represented in T1
ADDS{<q>} <Rdn>, #<imm8>
    // Outside IT block, and <Rdn>, <imm8> can be represented in T1
ADDS{<q>} {<Rdn>, } <Rdn>, #<imm8>
    // Outside IT block, and <Rdn>, <imm8> cannot be represented in T1
```

Decode for this encoding

```
1 d = UInt(Rdn); n = UInt(Rdn); setflags = !InITBlock(); imm32 = ZeroExtend(imm8, 32);
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	1	0	0	0	S	Rn			0	imm3			Rd			imm8									

ADD variant

Applies when S == 0.

```
ADD<c>.W {<Rd>, } <Rn>, #<const>
    // Inside IT block, and <Rd>, <Rn>, <const> can be represented in T1 or T2
ADD{<c>}{<q>} {<Rd>, } <Rn>, #<const>
```

ADDS variant

Applies when S == 1 && Rd != 1111.

```

ADDS.W {<Rd>, } <Rn>, #<const>
    // Outside IT block, and <Rd>, <Rn>, <const> can be represented in T1 or T2
ADDS{<c>}{<q>} {<Rd>, } <Rn>, #<const>

```

Decode for this encoding

```

1 if Rd == '1111' && S == '1' then SEE "CMN (immediate)";
2 if Rn == '1101' then SEE "ADD (SP plus immediate)";
3 if !HaveMainExt() then UNDEFINED;
4 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1'); imm32 = T32ExpandImm(i:imm3:imm8);
5 if d == 13 || (d == 15 && S == '0') || n == 15 then UNPREDICTABLE;

```

T4

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	1	0	0	0	0	0	Rn			0	imm3			Rd			imm8									

T4 variant

```
ADD{<c>}{<q>} {<Rd>,> <Rn>, #<imm12>
    // <imm12> cannot be represented in T1, T2, or T3
ADDW{<c>}{<q>} {<Rd>,> <Rn>, #<imm12>
    // <imm12> can be represented in T1, T2, or T3
```

Decode for this encoding

```
1 if Rn == '1111' then SEE ADR;
2 if Rn == '1101' then SEE "ADD (SP plus immediate)";
3 if !HaveMainExt() then UNDEFINED;
4 d = UInt(Rd); n = UInt(Rn); setflags = FALSE; imm32 = ZeroExtend(i:imm3:imm8, 32);
5 if d IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdn>	Is the general-purpose source and destination register, encoded in the "Rdn" field.
<imm8>	Is an 8-bit unsigned immediate, in the range 0 to 255, encoded in the "imm8" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	For encoding T1: is the general-purpose source register, encoded in the "Rn" field. For encoding T3: is the general-purpose source register, encoded in the "Rn" field. If the SP is used, see C2.4.3 ADD (SP plus immediate) . For encoding T4: is the general-purpose source register, encoded in the "Rn" field. If the SP is used, see C2.4.3 ADD (SP plus immediate) . If the PC is used, see C2.4.8 ADR .
<imm3>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "imm3" field.
<imm12>	Is a 12-bit unsigned immediate, in the range 0 to 4095, encoded in the "i:imm3:imm8" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
```

```

3      (result, carry, overflow) = AddWithCarry(R[n], imm32, '0');
4      R[d] = result;
5      if setflags then
6          APSR.N = result[31];
7          APSR.Z = IsZeroBit(result);
8          APSR.C = carry;
9          APSR.V = overflow;

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.6 ADD (immediate, to PC)

Add to PC. Add to PC adds an immediate value to the Align(PC, 4) value to form a PC-relative address, and writes the result to the destination register. Arm recommends that, where possible, software avoids using this alias.

This instruction is a pseudo-instruction of the [ADR](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [ADR](#).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of [ADR](#) gives the operational pseudocode for this instruction.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	Rd			imm8							

T1 variant

ADD{<c>}{<q>} <Rd>, PC, #<imm8>

is equivalent to

ADR{<c>}{<q>} <Rd>, <label>

and is never the preferred disassembly.

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	1	0	0	0	0	0	1	1	1	1	0	imm3			Rd			imm8								

T3 variant

ADDW{<c>}{<q>} <Rd>, PC, #<imm12>
// <Rd>, <imm12> can be represented in T1

is equivalent to

ADR{<c>}{<q>} <Rd>, <label>

and is never the preferred disassembly.

T3 variant

ADD{<c>}{<q>} <Rd>, PC, #<imm12>

is equivalent to

ADR{<c>}{<q>} <Rd>, <label>

and is never the preferred disassembly.

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
<q> See [C1.2.5 Standard assembler syntax fields](#).
<Rd> Is the general-purpose destination register, encoded in the "Rd" field.

<label>	For encoding T1: the label of an instruction or literal data item whose address is to be loaded into <Rd>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the ADR instruction to this label. Permitted values of the size of the offset are multiples of 4 in the range 0 to 1020. For encoding T2 and T3: the label of an instruction or literal data item whose address is to be loaded into <Rd>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the ADR instruction to this label. If the offset is zero or positive, encoding T3 is used, with <code>imm32</code> equal to the offset. If the offset is negative, encoding T2 is used, with <code>imm32</code> equal to the size of the offset. That is, the use of encoding T2 indicates that the required offset is minus the value of <code>imm32</code>. Permitted values of the size of the offset are 0-4095.
<imm8>	Is an unsigned immediate, a multiple of 4, in the range 0 to 1020, encoded in the "imm8" field as <imm8>/4.
<imm12>	Is a 12-bit unsigned immediate, in the range 0 to 4095, encoded in the "i:imm3:imm8" field.

Operation for all encodings

The description of [ADR](#) gives the operational pseudocode for this instruction.

C2.4.7 ADD (register)

Add (register). ADD (register) adds a register value and an optionally-shifted register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	0	0	Rm			Rn			Rd		

T1 variant

```
ADD<c>{<q>} <Rd>, <Rn>, <Rm>
    // Inside IT block
ADDS{<q>} {<Rd>}, <Rn>, <Rm>
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = !InITBlock();
2 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	DN	Rm			Rdn			

T2 variant

```
ADD<c>{<q>} <Rdn>, <Rm>
    // Preferred syntax, Inside IT block
ADD{<c>}{<q>} {<Rdn>}, <Rdn>, <Rm>
```

Decode for this encoding

```
1 if (DN:Rdn) == '1101' || Rm == '1101' then SEE "ADD (SP plus register)";
2 d = UInt(DN:Rdn); n = UInt(DN:Rdn); m = UInt(Rm); setflags = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
4 if d == 15 && InITBlock() && !LastInITBlock() then UNPREDICTABLE;
5 if d == 15 && m == 15 then UNPREDICTABLE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	1	0	0	0	S	Rn			(0)	imm3			Rd			imm2						Rm			
																											sr type				

ADD, rotate right with extend variant

Applies when S == 0 && imm3 == 000 && imm2 == 00 && sr_type == 11.

```
ADD{<c>}{<q>} {<Rd>}, <Rn>, <Rm>, RRX
```

ADD, shift or rotate by value variant

Applies when **S == 0 && !(imm3 == 000 && imm2 == 00 && sr_type == 11).**

```
ADD<c>.W {<Rd>}, {<Rn>}, <Rm>
    // Inside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1 or T2
ADD{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm> {, <shift> #<amount>}
```

ADDS, rotate right with extend variant

Applies when **S == 1 && imm3 == 000 && Rd != 1111 && imm2 == 00 && sr_type == 11.**

```
ADDS{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>, RRX
```

ADDS, shift or rotate by value variant

Applies when **S == 1 && !(imm3 == 000 && imm2 == 00 && sr_type == 11) && Rd != 1111.**

```
ADDS.W {<Rd>}, {<Rn>}, <Rm>
    // Outside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1 or T2
ADDS{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if Rd == '1111' && S == '1' then SEE "CMN (register)";
2 if Rn == '1101' then SEE "ADD (SP plus register)";
3 if !HaveMainExt() then UNDEFINED;
4 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
5 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
6 if d == 13 || (d == 15 && S == '0') || n == 15 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdn>	Is the general-purpose source and destination register, encoded in the "DN:Rdn" field. If the PC is used, the instruction is a branch to the address calculated by the operation. This is a simple branch. The assembler language allows <Rdn> to be specified once or twice in the assembler syntax. When used inside an IT block, and <Rdn> and <Rm> are in the range R0 to R7, <Rdn> must be specified once so that encoding T2 is preferred to encoding T1. In all other cases there is no difference in behavior when <Rdn> is specified once or twice.
<Rd>	For encoding T1: is the general-purpose destination register, encoded in the "Rd" field. When used inside an IT block, <Rd> must be specified. When used outside an IT block, <Rd> is optional, and: - If omitted, this register is the same as <Rn>. - If present, encoding T1 is preferred to encoding T2. For encoding T3: is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	For encoding T1: is the first general-purpose source register, encoded in the "Rn" field. For encoding T3: is the first general-purpose source register, encoded in the "Rn" field. If the SP is used, see C2.4.4 ADD (SP plus register) .
<Rm>	For encoding T1 and T3: is the second general-purpose source register, encoded in the "Rm" field. For encoding T2: is the second general-purpose source register, encoded in the "Rm" field. The PC can be used.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00

LSR when `sr_type = 01`
 ASR when `sr_type = 10`
 ROR when `sr_type = 11`
 <amount> Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      shifted = Shift(R[m], shift_t, shift_n, APSR.C);
4      (result, carry, overflow) = AddWithCarry(R[n], shifted, '0');
5      if d == 15 then
6          BranchTo(result); // setflags is always FALSE here
7      else
8          R[d] = result;
9          if setflags then
10             APSR.N = result[31];
11             APSR.Z = IsZeroBit(result);
12             APSR.C = carry;
13             APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.8 ADR

Form PC-relative address. Address to Register adds an immediate value to the PC value, and writes the result to the destination register.

This instruction is used by the pseudo-instructions [ADD \(immediate, to PC\)](#) and [SUB \(immediate, from PC\)](#).

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	Rd			imm8							

T1 variant

ADR{<c>}{<q>} <Rd>, <label>

Decode for this encoding

```
1 d = UInt(Rd); imm32 = ZeroExtend(imm8:'00', 32); add = TRUE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	1	0	1	0	1	0	1	1	1	1	0	imm3			Rd			imm8								

T2 variant

ADR{<c>}{<q>} <Rd>, <label>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); imm32 = ZeroExtend(i:imm3:imm8, 32); add = FALSE;
3 if d IN {13,15} then UNPREDICTABLE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	1	0	0	0	0	0	0	1	1	1	1	0	imm3			Rd			imm8							

T3 variant

ADR{<c>}.W <Rd>, <label>

// <Rd>, <label> can be presented in T1

ADR{<c>}{<q>} <Rd>, <label>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); imm32 = ZeroExtend(i:imm3:imm8, 32); add = TRUE;
3 if d IN {13,15} then UNPREDICTABLE;
```

Alias conditions

Alias or pseudo-instruction	preferred when
ADD (immediate, to PC)	Never
SUB (immediate, from PC)	<code>i:imm3:imm8 == '000000000000'</code>

Assembler symbols for all encodings

<code><c></code>	See C1.2.5 Standard assembler syntax fields .
<code><q></code>	See C1.2.5 Standard assembler syntax fields .
<code><Rd></code>	Is the general-purpose destination register, encoded in the "Rd" field.
<code><label></code>	For encoding T1: the label of an instruction or literal data item whose address is to be loaded into <code><Rd></code> . The assembler calculates the required value of the offset from the Align (PC, 4) value of the ADR instruction to this label. Permitted values of the size of the offset are multiples of 4 in the range 0 to 1020. For encoding T2 and T3: the label of an instruction or literal data item whose address is to be loaded into <code><Rd></code> . The assembler calculates the required value of the offset from the Align (PC, 4) value of the ADR instruction to this label. If the offset is zero or positive, encoding T3 is used, with <code>imm32</code> equal to the offset. If the offset is negative, encoding T2 is used, with <code>imm32</code> equal to the size of the offset. That is, the use of encoding T2 indicates that the required offset is minus the value of <code>imm32</code> . Permitted values of the size of the offset are 0-4095.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   result = if add then (Align(PC,4) + imm32) else (Align(PC,4) - imm32);
4   R[d] = result;
```

C2.4.9 AND (immediate)

Bitwise AND (immediate). AND (immediate) performs a bitwise AND of a register value and an immediate value, and writes the result to the destination register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	0	0	0	0	S	Rn				0	imm3				Rd				imm8							

AND variant

Applies when **S == 0**.

AND{<c>}{<q>} {<Rd>}, {<Rn>, #<const>

ANDS variant

Applies when **S == 1 && Rd != 1111**.

ANDS{<c>}{<q>} {<Rd>}, {<Rn>, #<const>

Decode for this encoding

```

1 if Rd == '1111' && S == '1' then SEE "TST (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1');
4 (imm32, carry) = T32ExpandImm_C(i:imm3:imm8, APSR.C);
5 if d == 13 || (d == 15 && S == '0') || n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = R[n] AND imm32;
4     R[d] = result;
5     if setflags then
6         APSR.N = result[31];
7         APSR.Z = IsZeroBit(result);
8         APSR.C = carry;
9         // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:

- The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.10 AND (register)

Bitwise AND (register). AND (register) performs a bitwise AND of a register value and an optionally-shifted register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	0	0	0	Rm					Rdn

T1 variant

```
AND<c>{<q>} {<Rdn>, } <Rdn>, <Rm>
    // Inside IT block
ANDS{<q>} {<Rdn>, } <Rdn>, <Rm>
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rdn); n = UInt(Rdn); m = UInt(Rm); setflags = !InITBlock();
2 (shift_t, shift_n) = (SRTType_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
1	1	1	0	1	0	1	0	0	0	0	S	Rn			(0)	imm3	Rd			imm2				Rm												
																				sr type																

AND, rotate right with extend variant

Applies when **S == 0 && imm3 == 000 && imm2 == 00 && sr_type == 11**.

```
AND{<c>}{<q>} {<Rd>, } <Rn>, <Rm>, RRX
```

AND, shift or rotate by value variant

Applies when **S == 0 && !(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
AND<c>.W {<Rd>, } <Rn>, <Rm>
    // Inside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
AND{<c>}{<q>} {<Rd>, } <Rn>, <Rm> {, <shift> #<amount>}
```

ANDS, rotate right with extend variant

Applies when **S == 1 && imm3 == 000 && Rd != 1111 && imm2 == 00 && sr_type == 11**.

```
ANDS{<c>}{<q>} {<Rd>, } <Rn>, <Rm>, RRX
```

ANDS, shift or rotate by value variant

Applies when **S == 1 && !(imm3 == 000 && imm2 == 00 && sr_type == 11) && Rd != 1111**.

```
ANDS.W {<Rd>}, {<Rn>}, <Rm>
// Outside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
ANDS{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if Rd == '1111' && S == '1' then SEE "TST (register)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
4 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
5 if d == 13 || (d == 15 && S == '0') || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdn>	Is the first general-purpose source register and the destination register, encoded in the "Rdn" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (shifted, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);
4   result = R[n] AND shifted;
5   R[d] = result;
6   if setflags then
7     APSR.N = result[31];
8     APSR.Z = IsZeroBit(result);
9     APSR.C = carry;
10    // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.11 ASR (immediate)

Arithmetic Shift Right (immediate). Arithmetic Shift Right (immediate) shifts a register value right by an immediate number of bits, shifting in copies of its sign bit, and writes the result to the destination register.

This instruction is an alias of the [MOV \(register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV \(register\)](#).
- The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	op = 10	imm5				Rm				Rd			

T2 variant

```
ASR<c>{<q>} {<Rd>}, <Rm>, #<imm>
// Inside IT block
```

is equivalent to

```
MOV<c>{<q>} <Rd>, <Rm>, ASR #<imm>
```

and is the preferred disassembly when `InITBlock()`.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	1	0	1	0	0	1	0		1	1	1	1	[0]	imm3				Rd				imm2				Rm != 11x1			
S = 0												sr_type = 10																				

MOV, shift or rotate by value variant

```
ASR<c>.W {<Rd>}, <Rm>, #<imm>
// Inside IT block, and <Rd>, <Rm>, <imm> can be represented in T2
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, ASR #<imm>
```

and is always the preferred disassembly.

MOV, shift or rotate by value variant

```
ASR{<c>}{<q>} {<Rd>}, <Rm>, #<imm>
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, ASR #<imm>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.
<imm>	For encoding T2: is the shift amount, in the range 1 to 32, encoded in the "imm5" field as <imm> modulo 32. For encoding T3: is the shift amount, in the range 1 to 32, encoded in the "imm3:imm2" field as <imm> modulo 32.

Operation for all encodings

The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.12 ASR (register)

Arithmetic Shift Right (register). Arithmetic Shift Right (register) shifts a register value right by a variable number of bits, shifting in copies of its sign bit, and writes the result to the destination registers. The variable number of bits is read from the bottom byte of a register.

This instruction is an alias of the [MOV, MOVS \(register-shifted register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV, MOVS \(register-shifted register\)](#).
- The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op = 0100				Rs			Rdm		

Arithmetic shift right variant

```
ASR<c>{<q>} {<Rdm>, } <Rdm>, <Rs>
// Inside IT block
```

is equivalent to

```
MOV<c>{<q>} <Rdm>, <Rdm>, ASR <Rs>
```

and is the preferred disassembly when `InITBlock()`.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0							Rm	1	1	1	1												Rs

sr_type = 10

S = 0

Non flag setting variant

```
ASR<c>.W {<Rd>, } <Rm>, <Rs>
// Inside IT block, and <Rd>, <Rm>, <sr_type>, <Rs> can be represented in T1
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, ASR <Rs>
```

and is always the preferred disassembly.

Non flag setting variant

```
ASR{<c>}{<q>} {<Rd>, } <Rm>, <Rs>
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, ASR <Rs>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<C>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the first general-purpose source register and the destination register, encoded in the "Rdm" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rs>	Is the second general-purpose source register holding a shift amount in its bottom 8 bits, encoded in the "Rs" field.

Operation for all encodings

The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.13 ASRL (immediate)

Arithmetic Shift Right Long. Arithmetic shift right by 1 to 32 bits of a 64-bit value stored in two general-purpose registers.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	0	0	immh	RdaHi	(1)	imml	1	0	1	1	1	1	1	1	1	1	1	1	

T1: ASRL variant

ASRL<c><q> RdaLo, RdaHi, #<imm>

Decode for this encoding

```

1 if RdaHi == '111' then SEE "SRSHR (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 (-, amount) = DecodeImmShift('10', immh:imml);
8 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <RdaLo> General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
 <RdaHi> General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
 <imm> The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   op1 = SInt(R[dah]:R[dal]);
5   result = (op1 >> amount)[63:0];
6   R[dah] = result[63:32];
7   R[dal] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.14 ASRL (register)

Arithmetic Shift Right Long. Arithmetic shift right by 0 to 64 bits of a 64-bit value stored in two general-purpose registers. The shift amount is read in as the bottom byte of Rm. If the shift amount is negative, the shift direction is reversed.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	0			Rm		RdaHi	(1)	(0)	(0)	1	0	1	1	0	1				

T1: ASRL variant

ASRL<c><q> RdaLo, RdaHi, Rm

Decode for this encoding

```

1 if RdaHi == '111' then SEE "SQRSHR (register)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 m = UInt(Rm);
8 if RdaHi == '110' || Rm == '11x1' || Rm == RdaHi:'1' then CONSTRAINED_UNPREDICTABLE;
9 if Rm == RdaLo:'0' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See C1.2.5 *Standard assembler syntax fields*.
 <RdaLo> General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
 <RdaHi> General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
 <Rm> General-purpose source register holding a shift amount in its bottom 8 bits.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     amount = SInt(R[m][7:0]);
5     opl = SInt(R[dah]:R[dal]);
6     result = (opl >> amount)[63:0];
7     R[dah] = result[63:32];
8     R[dal] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If AIRCR.DIT is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.15 ASRS (immediate)

Arithmetic Shift Right, Setting flags (immediate). Arithmetic Shift Right, Setting flags (immediate) shifts a register value right by an immediate number of bits, shifting in copies of its sign bit, writes the result to the destination register, and updates the condition flags based on the result.

This instruction is an alias of the [MOV \(register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV \(register\)](#).
- The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	op = 10	imm5				Rm				Rd			

T2 variant

```
ASRS{<q>} {<Rd>}, <Rm>, #<imm>
// Outside IT block
```

is equivalent to

```
MOVS{<q>} <Rd>, <Rm>, ASR #<imm>
```

and is the preferred disassembly when **!InITBlock()**.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	1	0	1	0					1	1	1	1	[0]	imm3				Rd				imm2				Rm			
S = 1												sr type = 10																				

MOVS, shift or rotate by value variant

```
ASRS.W {<Rd>}, <Rm>, #<imm>
// Outside IT block, and <Rd>, <Rm>, <imm> can be represented in T2
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, ASR #<imm>
```

and is always the preferred disassembly.

MOVS, shift or rotate by value variant

```
ASRS{<c>}{<q>} {<Rd>}, <Rm>, #<imm>
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, ASR #<imm>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.
<imm>	For encoding T2: is the shift amount, in the range 1 to 32, encoded in the "imm5" field as <imm> modulo 32. For encoding T3: is the shift amount, in the range 1 to 32, encoded in the "imm3:imm2" field as <imm> modulo 32.

Operation for all encodings

The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.16 ASRS (register)

Arithmetic Shift Right, Setting flags (register). Arithmetic Shift Right, Setting flags (register) shifts a register value right by a variable number of bits, shifting in copies of its sign bit, writes the result to the destination register, and updates the condition flags based on the result. The variable number of bits is read from the bottom byte of a register.

This instruction is an alias of the [MOV, MOVS \(register-shifted register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV, MOVS \(register-shifted register\)](#).
- The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op = 0100				Rs			Rdm		

Arithmetic shift right variant

```
ASRS{<q>} {<Rdm>, } <Rdm>, <Rs>
// Outside IT block
```

is equivalent to

```
MOVS{<q>} <Rdm>, <Rdm>, ASR <Rs>
```

and is the preferred disassembly when `!InITBlock()`.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0				Rm				1	1	1	1	Rd			0	0	0	0	Rs				
								sr_type = 10																							
								S = 1																							

Flag setting variant

```
ASRS.W {<Rd>, } <Rm>, <Rs>
// Outside IT block, and <Rd>, <Rm>, <sr_type>, <Rs> can be represented in T1
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, ASR <Rs>
```

and is always the preferred disassembly.

Flag setting variant

```
ASRS{<c>}{<q>} {<Rd>, } <Rm>, <Rs>
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, ASR <Rs>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<C>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the first general-purpose source register and the destination register, encoded in the "Rdm" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rs>	Is the second general-purpose source register holding a shift amount in its bottom 8 bits, encoded in the "Rs" field.

Operation for all encodings

The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.17 AUT

Authenticate link register, using key. This instruction computes a pointer authentication code using LR as the address, SP as the modifier and, a key. The computed PAC is compared against the PAC in R12. If the values do not match an INVSTATE UsageFault is generated.

This instruction is not permitted in an IT block.

This is a NOP-compatible hint.

T1

Armv8.1-M PACBTI

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	1	0	1	1	0	1

T1 variant

AUT R12, LR, SP

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes unconditionally.
- The instruction executes conditionally.

Assembler symbols for all encodings

LR LR register, this instruction must always use this register. This register holds the address.
SP SP register, this instruction must always use this register. This register holds the modifier to apply during the PAC code generation.
R12 R12 register, this instruction must always use this register. This register holds PAC for comparison.

Operation for all encodings

```
1 EncodingSpecificOperations();
2
3 if PACEnabled() then
4     if !ValidatePAC(LR, SP, R[12]) then
5         UFSR.INVSTATE = '1';
6         excInfo = CreateException(UsageFault);
7         HandleException(excInfo);
```

Restricted behavior

Data Independent Timing behavior and the PACBTI extension

The timing of this instruction is insensitive to the value of the data being loaded or stored regardless of the value of [AIRCR.DIT](#) when accessing any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

The point at which an asynchronous exception is taken does not vary based on any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

C2.4.18 AUTG

Authenticate general value, using key. This instruction computes a pointer authentication code using two input registers and a key. The computed PAC is compared against the PAC in the specified register. If the values do not match an INVSTATE UsageFault is generated.

T1

Armv8.1-M PACBTI

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	1	0	1	Rn				Ra				1	1	1	1	0	0	0	0	Rm			

T1 variant

AUTG{<c>} <Ra>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HavePACBTIExt() then
2     if HaveDSPExt() then UNPREDICTABLE; else UNDEFINED;
3 if Ra == '1111' then SEE SMMUL;
4 a = UInt(Ra); n = UInt(Rn); m = UInt(Rm);
5 if n == 15 || m == 15 || a == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose source register containing the value to be encrypted, encoded in the "Rn" field.
<Rm>	Is the general-purpose source register containing the modifier to be used in the encryption operation, encoded in the "Rm" field.
<Ra>	Is the general-purpose register containing the encrypted value, encoded in the "Ra" field. The value in this register will be compared against the value produced by the encryption operation.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if PACEnabled() then
4         if !ValidatePAC(R[n], R[m], R[a]) then
5             UFSR.INVSTATE = '1';
6             excInfo = CreateException(UsageFault);
7             HandleException(excInfo);
```

Restricted behavior

Data Independent Timing behavior and the PACBTI extension

The timing of this instruction is insensitive to the value of the data being loaded or stored regardless of the value of [AIRCR.DIT](#) when accessing any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

The point at which an asynchronous exception is taken does not vary based on any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

C2.4.19 B

Branch. Branch causes a branch to a target address.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	cond				imm8							

T1 variant

```
B<c>{<q>} <label>
// Not permitted in IT block
```

Decode for this encoding

```
1 if cond == '1110' then SEE UDF;
2 if cond == '1111' then SEE SVC;
3 imm32 = SignExtend(imm8:'0', 32);
4 if InITBlock() then UNPREDICTABLE;
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	0	imm11										

T2 variant

```
B{<c>}{<q>} <label>
// Outside or last in IT block
```

Decode for this encoding

```
1 imm32 = SignExtend(imm11:'0', 32);
2 cond = CurrentCond();
3 if InITBlock() && !LastInITBlock() then UNPREDICTABLE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	S	cond		imm6				1	0	J1	0	J2	imm11														

T3 variant

```
B<c>.W <label>
// Not permitted in IT block, and <label> can be represented in T1
B<c>{<q>} <label>
// Not permitted in IT block
```

Decode for this encoding

```
1 if cond[3:1] == '111' then SEE "Related encodings";
2 if !HaveMainExt() then UNDEFINED;
3 imm32 = SignExtend(S:J2:J1:imm6:imm11:'0', 32);
4 if InitBlock() then UNPREDICTABLE;
```

T4

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	S	imm10										1	0	J1	1	J2	imm11										

T4 variant

B{<c>}.W <label>
 // <label> can be represented in T2
 B{<c>}{<q>} <label>

Decode for this encoding

```
1 I1 = NOT(J1 EOR S); I2 = NOT(J2 EOR S); imm32 = SignExtend(S:I1:I2:imm10:imm11:'0', 32);
2 cond = CurrentCond();
3 if InitBlock() && !LastInitBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> For encoding T1: see [C1.2.5 Standard assembler syntax fields](#). Must not be AL or omitted.
 For encoding T2 and T4: see [C1.2.5 Standard assembler syntax fields](#).
 For encoding T3: see [C1.2.5 Standard assembler syntax fields](#). <c> must not be AL or omitted.

<q> See [C1.2.5 Standard assembler syntax fields](#).

<label> For encoding T1: the label of the instruction that is to be branched to. The assembler calculates the required value of the offset from the PC value of the B instruction to this label, then selects an encoding that sets imm32 to that offset. Permitted offsets are even numbers in the range -256 to 254.
 For encoding T2: the label of the instruction that is to be branched to. The assembler calculates the required value of the offset from the PC value of the B instruction to this label, then selects an encoding that sets imm32 to that offset. Permitted offsets are even numbers in the range -2048 to 2046.
 For encoding T3: the label of the instruction that is to be branched to. The assembler calculates the required value of the offset from the PC value of the B instruction to this label, then selects an encoding that sets imm32 to that offset. Permitted offsets are even numbers in the range -1048576 to 1048574.
 For encoding T4: the label of the instruction that is to be branched to. The assembler calculates the required value of the offset from the PC value of the B instruction to this label, then selects an encoding that sets imm32 to that offset. Permitted offsets are even numbers in the range -16777216 to 16777214.

Operation for all encodings

```
1 if ConditionPassed(cond) then
2     EncodingSpecificOperations();
3     BranchTo(PC + imm32);
```

C2.4.20 BF, BFX, BFL, BFLX, BFCSEL

Branch Future, Branch Future and Exchange, Branch Future with Link, Branch Future with Link and Exchange, Branch Future Conditional Select. These instructions notify the PE about an upcoming branch to `<label>`, so that the branch will be taken instead of fetching and executing the instruction at `<b_label>`. This allows the PE to reduce or eliminate any associated performance penalty that might have been caused by a branch that the PE was not notified about. It is IMPLEMENTATION DEFINED whether this instruction is treated as a NOP. To ensure correct program flow behavior in these cases, fallback code should be present at the point specified by `<b_label>`.

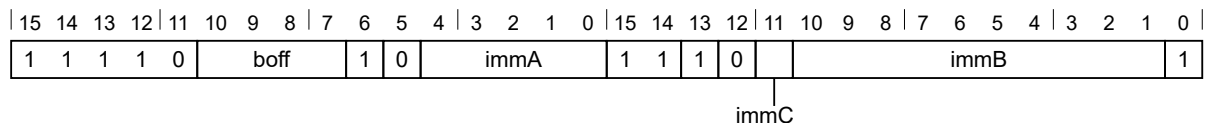
For the Branch Future with Link variant, the link register is updated when the branch is performed. The value written is offset from the branch point by 4 bytes, which corresponds to the length of the BL (immediate) instruction that would usually follow the branch point as part of the fallback code.

For the Branch Future with Link and Exchange variant, the link register is updated when the branch is performed. The value written is offset from the branch point by 2 bytes, which corresponds to the length of the BLX (register) instruction that would usually follow the branch point as part of the fallback code. If the PACBTI Extension is implemented this instruction will set LO_BRANCH_INFO.BTI to one (this bit will not be set if LR register is used as an argument in BFX). When the branch is taken and BTI is enabled then the EPSR.B is set to one.

The Branch Future Conditional Select variant creates a future branch to `<label>` if the condition code passes. If the explicit condition code fails, this variant does not behave as a NOP. Instead it creates a future branch to the instruction specified by `<ba_label>`, if there is no other active BF entry in the loop and branch cache.

T1

Armv8.1-M Low Overhead Branch Extension



T1: BF variant

BF<c><q> <b_label>, <label>

Decode for this encoding

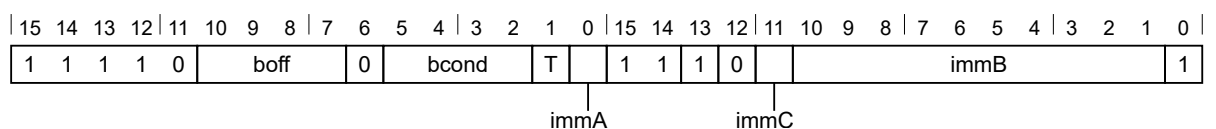
```

1 if boff == '0000' then SEE "Related encodings";
2 if !HaveLOBExt() then UNDEFINED;
3
4 endOffset = ZeroExtend(boff:'0', 32);
5 offset    = SignExtend(immA:immB:immC:'1', 32);
6 bOffset   = bits(32) UNKNOWN;
7 n         = integer UNKNOWN;
8 regAddr   = FALSE;
9 link      = FALSE;
10 setBti    = FALSE;
11 bcond     = CondAL;

```

T2

Armv8.1-M Low Overhead Branch Extension



T2: BFCSEL variant

BFCSEL <b_label>, <label>, <ba_label>, <bcond>

Decode for this encoding

```

1 if boff == '0000' then SEE "Related encodings";
2 if !HaveLOBExt() then UNDEFINED;
3
4 endOffset = ZeroExtend(boff:'0', 32);
5 offset    = SignExtend(immA:immB:immC:'1', 32);
6 bOffset   = (if T == '1' then 5 else 3)[31:0];
7 n         = integer UNKNOWN;
8 regAddr   = FALSE;
9 link      = FALSE;
10 setBti    = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
12 if bcond == '111x' then CONSTRAINED_UNPREDICTABLE;

```

T3

Armv8.1-M Low Overhead Branch Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	boff				1	1	0	Rn				1	1	1	0	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	1

T3: BFX variant

BFX<c><q> <b_label>, Rn

Decode for this encoding

```

1 if boff == '0000' then SEE "Related encodings";
2 if !HaveLOBExt() then UNDEFINED;
3
4 endOffset = ZeroExtend(boff:'0', 32);
5 offset    = Zeros(32);
6 bOffset   = bits(32) UNKNOWN;
7 n         = UInt(Rn);
8 regAddr   = TRUE;
9 link      = FALSE;
10 setBti    = if n != 14 then TRUE else FALSE;
11 bcond     = CondAL;
12 if Rn == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

T4

Armv8.1-M Low Overhead Branch Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	boff				immA				1	1	0	0	immB										1				
																immC															

T4: BFL variant

BFL<c><q> <b_label>, <label>

Decode for this encoding

```

1 if boff == '0000' then SEE "Related encodings";
2 if !HaveLOBExt() then UNDEFINED;
3
4 endOffset = ZeroExtend(boff:'0', 32);
5 offset    = SignExtend(immA:immB:immC:'1', 32);
6 bOffset   = bits(32) UNKNOWN;
7 n         = integer UNKNOWN;
8 regAddr   = FALSE;
9 link      = TRUE;
10 setBti    = FALSE;
11 bcond     = CondAL;

```

T5

Armv8.1-M Low Overhead Branch Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	boff				1	1	1	Rn			1	1	1	0	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	1

T5: BFLX variant

BFLX<c><q> <b_label>, Rn

Decode for this encoding

```

1 if boff == '0000' then SEE "Related encodings";
2 if !HaveLOBExt() then UNDEFINED;
3
4 endOffset = ZeroExtend(boff:'0', 32);
5 offset    = Zeros(32);
6 bOffset   = bits(32) UNKNOWN;
7 n         = UInt(Rn);
8 regAddr   = TRUE;
9 link      = TRUE;
10 setBti    = TRUE;
11 bcond     = CondAL;
12 if Rn == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<b_label>	The PC relative offset of the first instruction in the fallback code, that will not be executed if the future branch is taken.
<Rn>	The address to branch to.
<ba_label>	The PC relative offset of the address to branch to in case the associated BFCSEL condition code check fails and no other branch future is pending. The range of this address allows branching over a 2-byte or 4-byte instruction located at <b_label>.
<T>	Selects whether the instruction at <b_label> is a 2-byte (T = 0) or 4-byte (T = 1) instruction to be branched around, as specified by <ba_label>.
<bcond>	The comparison condition to use. The evaluation of this comparison is performed when this instruction is executed and not at the point the branch is performed. This parameter must be one of the following values:
EQ	Encoded as bcond = 0000
NE	Encoded as bcond = 0001
CS	Encoded as bcond = 0010
CC	Encoded as bcond = 0011
MI	Encoded as bcond = 0100
PL	Encoded as bcond = 0101
VS	Encoded as bcond = 0110
VC	Encoded as bcond = 0111
HI	Encoded as bcond = 1000

	LS	Encoded as	bcond = 1001
	GE	Encoded as	bcond = 1010
	LT	Encoded as	bcond = 1011
	GT	Encoded as	bcond = 1100
	LE	Encoded as	bcond = 1101

<label> The PC relative offset of the address to branch to.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3
4      // BF can be implemented as a NOP
5      allow_bf = boolean IMPLEMENTATION_DEFINED "BF allowed";
6
7      if ConditionPassed(bcond) then
8          jump_addr = if regAddr then R[n] else PC + offset;
9      else
10         jump_addr = PC + endOffset + bOffset;
11         // If the condition fails, capture a BF entry only if it does not override
12         // an existing BF entry.
13         if LO_BRANCH_INFO.VALID == '1' && LO_BRANCH_INFO.BF == '1' then
14             allow_bf = FALSE;
15         // A branch that results in a transition to a different instruction set
16         // causes the BF instruction to be treated as a NOP.
17         allow_bf = allow_bf && (jump_addr[0] == '1');
18         // Branches that would cause a FNC_RETURN or EXC_RETURN aren't supported,
19         // and also cause BF to be treated as a NOP.
20         allow_bf = allow_bf && (!regAddr || IsReturn(jump_addr) == AddrType_NORMAL);
21         // Check if the branch cache info is enabled.
22         allow_bf = allow_bf && CCR.LOB == '1';
23         // Set up the branch info cache if allowed
24         if allow_bf then
25             LO_BRANCH_INFO.VALID = '1';
26             LO_BRANCH_INFO.BF = '1';
27             LO_BRANCH_INFO.LF = if link then '1' else '0';
28             LO_BRANCH_INFO.T16IND = if regAddr then '1' else '0';
29             LO_BRANCH_INFO.BTI = if setBti then '1' else '0';
30             LO_BRANCH_INFO.END_ADDR = (PC + endOffset)[31:1];
31             LO_BRANCH_INFO.JUMP_ADDR = jump_addr[31:1];

```

C2.4.21 BFC

Bit Field Clear. Bit Field Clear clears any number of adjacent bits at any position in a register, without affecting the other bits in the register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	(0)	1	1	0	1	1	0	1	1	1	1	0	imm3		Rd		imm2	(0)									msb

T1 variant

BFC{<c>}{<q>} <Rd>, #<lsb>, #<width>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); msbit = UInt(msb); lsbit = UInt(imm3:imm2);
3 if msbit < lsbit then UNPREDICTABLE;
4 if d IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `msbit < lsbit`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<lsb>	Is the least significant bit that is to be cleared, in the range 0 to 31, encoded in the "imm3:imm2" field.
<width>	Is the number of bits to be cleared, in the range 1 to 32-<lsb>, encoded in the "msb" field as <lsb>+<width>-1.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if msbit >= lsbit then
4         R[d][msbit:lsbit] = Replicate('0', msbit-lsbit+1);
5         // Other bits of R[d] are unchanged
6     else
7         R[d] = bits(32) UNKNOWN;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.

- The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.22 BFI

Bit Field Insert. Bit Field Insert copies any number of low order bits from a register into the same number of adjacent bits at any position in the destination register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	(0)	1	1	0	1	1	0	Rn				0	imm3				Rd				imm2		(0)	msb			

T1 variant

BFI{<c>}{<q>} <Rd>, <Rn>, #<lsb>, #<width>

Decode for this encoding

```

1 if Rn == '1111' then SEE BFC;
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); msbit = UInt(msb); lsbit = UInt(imm3:imm2);
4 if msbit < lsbit then UNPREDICTABLE;
5 if d IN {13,15} || n == 13 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `msbit < lsbit`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<lsb>	Is the least significant destination bit, in the range 0 to 31, encoded in the "imm3:imm2" field.
<width>	Is the number of bits to be copied, in the range 1 to 32-<lsb>, encoded in the "msb" field as <lsb>+<width>-1.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if msbit >= lsbit then
4         R[d][msbit:lsbit] = R[n][(msbit-lsbit):0];
5         // Other bits of R[d] are unchanged
6     else
7         R[d] = bits(32) UNKNOWN;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:

- The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.23 BIC (immediate)

Bit Clear (immediate). Bit Clear (immediate) performs a bitwise AND of a register value and the complement of an immediate value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	0	0	0	1	S	Rn				0	imm3				Rd				imm8							

BIC variant

Applies when **S** == 0.

BIC{<c>}{<q>} {<Rd>}, {<Rn>, #<const>

BICS variant

Applies when **S** == 1.

BICS{<c>}{<q>} {<Rd>}, {<Rn>, #<const>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1');
3 (imm32, carry) = T32ExpandImm_C(i:imm3:imm8, APSR.C);
4 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = R[n] AND NOT(imm32);
4     R[d] = result;
5     if setflags then
6         APSR.N = result[31];
7         APSR.Z = IsZeroBit(result);
8         APSR.C = carry;
9         // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:

- The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.24 BIC (register)

Bit Clear (register). Bit Clear (register) performs a bitwise AND of a register value and the complement of an optionally-shifted register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	1	1	1	0	Rm			Rdn		

T1 variant

```
BIC{<c>}{<q>} {<Rdn>,} <Rdn>, <Rm>
    // Inside IT block
BICS{<c>}{<q>} {<Rdn>,} <Rdn>, <Rm>
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rdn); n = UInt(Rdn); m = UInt(Rm); setflags = !InITBlock();
2 (shift_t, shift_n) = (SRTType_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	0	1	S	Rn			(0)	imm3			Rd			imm2						Rm			
																											sr type				

BIC, rotate right with extend variant

Applies when **S == 0** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
BIC{<c>}{<q>} {<Rd>,} <Rn>, <Rm>, RRX
```

BIC, shift or rotate by value variant

Applies when **S == 0** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
BIC{<c>}.W {<Rd>,} <Rn>, <Rm>
    // Inside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
BIC{<c>}{<q>} {<Rd>,} <Rn>, <Rm> {, <shift> #<amount>}
```

BICS, rotate right with extend variant

Applies when **S == 1** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
BICS{<c>}{<q>} {<Rd>,} <Rn>, <Rm>, RRX
```

BICS, shift or rotate by value variant

Applies when **S == 1** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
BICS.W {<Rd>}, <Rn>, <Rm>
// Outside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
BICS{<c>}{<q>} {<Rd>}, <Rn>, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
3 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdn>	Is the first general-purpose source register and the destination register, encoded in the "Rdn" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (shifted, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);
4   result = R[n] AND NOT(shifted);
5   R[d] = result;
6   if setflags then
7     APSR.N = result[31];
8     APSR.Z = IsZeroBit(result);
9     APSR.C = carry;
10    // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.25 BKPT

Breakpoint. Breakpoint causes a DebugMonitor exception or a debug halt to occur depending on the configuration of the debug support.

BKPT is an unconditional instruction and executes as such both inside and outside an IT instruction block.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	0	1	1	1	1	1	0	imm8								

T1 variant

BKPT{<q>} {#}<imm>

Decode for this encoding

```
1 imm32 = ZeroExtend(imm8, 32);
2 // imm32 is for assembly/disassembly only and is ignored by hardware.
```

Assembler symbols for all encodings

<q>

See C1.2.5 *Standard assembler syntax fields*. A BKPT instruction must be unconditional.

<imm>

Is an 8-bit unsigned immediate, in the range 0 to 255, encoded in the "imm8" field. The PE ignores this value, but a debugger might use it to store additional information about the breakpoint.

Operation for all encodings

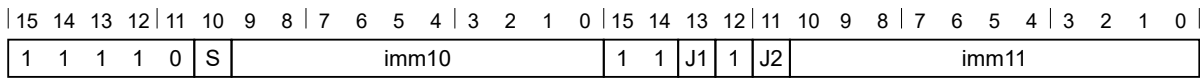
```
1 EncodingSpecificOperations();
2 GenerateDebugEventResponse(DebugEventCause_BKPT);
```

C2.4.26 BL

Branch with Link (immediate). Branch with Link (immediate) calls a subroutine at a PC-relative address.

T1

Armv8-M



T1 variant

BL{<c>}{<q>} <label>

Decode for this encoding

```
1 I1 = NOT(J1 EOR S); I2 = NOT(J2 EOR S); imm32 = SignExtend(S:I1:I2:imm10:imm11:'0', 32);
2 if InITBlock() && !LastInITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<label>	The label of the instruction that is to be branched to. The assembler calculates the required value of the offset from the PC value of the BL instruction to this label, then selects an encoding with imm32 set to that offset. Permitted offsets are even numbers in the range -16777216 to 16777214.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   LR = (ThisInstrAddr()[31:1] : '1') + ThisInstrLength();
4   BranchTo(PC + imm32);
```


C2.4.27 BLX, BLXNS

Branch with Link and Exchange (Non-secure). Branch with Link and Exchange calls a subroutine at an address, with the address and instruction set specified by a register. Bit[0] complies with the Arm architecture interworking rules for switching between the A32 and T32 instruction sets. However, Armv8-M only supports the T32 instruction set, so bit[0] must be 1. If bit[0] is 0 the PE takes an INVSTATE UsageFault exception on the instruction at the target address.

Branch with Link and Exchange Non-secure calls a subroutine at an address specified by a register, and if bit[0] of the target address is 0 then the instruction causes a transition from Secure to Non-secure state. This variant of the instruction must only be used when the additional steps required to make such a transition safe have been taken.

BLXNS is UNDEFINED if executed in Non-secure state, or if the Security Extension is not implemented.

If the PACBTI Extension is implemented and BTI is enabled for the destination Security state and privilege mode then this instruction is a BTI setting instruction. When this instruction is executed it will set EPSR.B to one.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	1	1	1		Rm		NS	(0)	(0)	

BLX variant

Applies when **NS** == 0.

BLX{<c>} {<q>} <Rm>

BLXNS variant

Applies when **NS** == 1.

BLXNS{<c>} {<q>} <Rm>

Decode for this encoding

```

1 m = UInt(Rm); allowNonSecure = NS == '1';
2 if !IsSecure() && allowNonSecure then UNDEFINED;
3 if m IN {13,15} then UNPREDICTABLE;
4 if InITBlock() && !LastInITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rm> Is the general-purpose register holding the address to be branched to, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     target = R[m];
5     nextInstrAddr = (ThisInstrAddr()[31:1] : '1') + ThisInstrLength();
6
7     if allowNonSecure && (target[0] == '0') then
8         targetIsSecure = FALSE;
```

```

9      if !IsAligned(SP, 8) then UNPREDICTABLE;
10     address = SP - 8;
11     RETPSR_Type savedPSR = Zeros();
12     // EPSR.B is not preserved on the stack
13     savedPSR.Exception = IPSR.Exception;
14     savedPSR.SFPA = CONTROL_S.SFPA;
15     // Only the stack locations, not the store order, are architected
16     spName = LookUpSP();
17     mode = CurrentMode();
18     exc = Stack(address, 0, spName, mode, nextInstrAddr);
19     if exc.fault == NoFault then exc = Stack(address, 4, spName, mode, savedPSR);
20     HandleException(exc);
21     // Stack pointer update will raise a fault if limit violated
22     SP = address;
23     LR = 0xFFFFFFFF[31:0];
24     // If in handler mode, IPSR must be nonzero. To prevent revealing which
25     // Secure handler is calling Non-secure code, IPSR is set to an invalid but
26     // nonzero value (ie the reset exception number).
27     if mode == PMode_Handler then
28         IPSR = 0x1[31:0];
29     else
30         targetIsSecure = IsSecure();
31         LR = nextInstrAddr;
32
33     setBti = BTIEnabled(targetIsSecure);
34     BranchCall(target, allowNonSecure, setBti);

```

CONSTRAINED UNPREDICTABLE behavior

If !IsAligned(SP, 8), then one of the following behaviors must occur:

- The instruction uses the current value of the stack pointer.
- The instruction behaves as though bits[2:0] of the stack pointer are 0b000.

C2.4.28 BTI

Branch target identification. When the PACBTI Extension is enabled this instruction is treated as a valid landing pad for jumps and calls and resets EPSR.B to zero.

This instruction is not permitted in an IT block.

This is a NOP-compatible hint.

T1

Armv8.1-M PACBTI

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	0	0	1	1	1	1

T1 variant

BTI

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes unconditionally.
- The instruction executes conditionally.

Assembler symbols for all encodings

<none>

Operation for all encodings

```
1 EncodingSpecificOperations();
2
3 // This instruction is an explicit BTI landing pad instruction.
4 // The EPSR.B bit is reset to zero.
5 EPSR.B = '0';
```

C2.4.29 BX, BXNS

Branch and Exchange (Non-secure). Branch and Exchange causes a branch to an address, with the address and instruction set specified by a register. Bit[0] complies with the Arm architecture interworking rules for switching between the A32 and T32 instruction sets. However, Armv8-M only supports the T32 instruction set, so bit[0] must be 1. If bit[0] is 0 the PE takes an INVSTATE UsageFault exception on the instruction at the target address.

Branch and Exchange Non-secure causes a branch to an address specified by a register. If bit[0] of the target address is 0, and the target address is not FNC_RETURN or EXC_RETURN, then the instruction causes a transition from Secure to Non-secure state. This variant of the instruction must only be used when the additional steps required to make such a transition safe have been taken.

BX can also be used for an exception return.

BXNS is UNDEFINED if executed in Non-secure state, or if the Security Extension is not implemented.

If the PACBTI Extension is implemented, BTI is enabled for the destination security and privilege state, the operation is not a FNC_RETURN or an EXC_RETURN, and <Rm> is not LR, then this instruction is a BTI setting instruction, and when this instruction is executed it will set EPSR.B to one.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	1	1	0	Rm				NS	(0)	(0)

BX variant

Applies when NS == 0.

BX{<c>}{<q>} <Rm>

BXNS variant

Applies when NS == 1.

BXNS{<c>}{<q>} <Rm>

Decode for this encoding

```
1 m = UInt(Rm); allowNonSecure = NS == '1';
2 if !IsSecure() && allowNonSecure then UNDEFINED;
3 if m IN {13,15} then UNPREDICTABLE;
4 if InITBlock() && !LastInITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See C1.2.5 Standard assembler syntax fields.
 <q> See C1.2.5 Standard assembler syntax fields.
 <Rm> Is the general-purpose register holding the address to be branched to, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     // If BTI is enabled, this branch sets the EPSR.B flag to ensure that the target
4     // instruction is BTI clearing. If register LR is used, then a BTI clearing
5     // instruction is not required at the branch target.
6     target = R[m];
7     targetIsSecure = IsSecure() && !(allowNonSecure && target[0] == '0');
```

```
8      setBti      = BTIEnabled(targetIsSecure) && m != 14;
9      exc        = BranchReturn(target, allowNonSecure, setBti);
10     HandleException(exc);
```

C2.4.30 BXAUT

Branch Exchange after Authenticating the address, using key. This instruction computes a pointer authentication code using the address, a modifier and, a key. The computed PAC is compared against the PAC in the specified register. If the values do not match an INVSTATE UsageFault is generated. Once the address has been authenticated the PE branches to that address.

Bit[0] complies with the Arm architecture interworking rules for switching between the A32 and T32 instruction sets. However, Armv8-M only supports the T32 instruction set, so bit[0] must be 1. If bit[0] is 0 the PE takes an INVSTATE UsageFault exception on the instruction at the target address.

BXAUT can be used for an exception return.

The instruction must be either outside any IT block, or the last instruction in an IT block.

T1

Armv8.1-M PACBTI

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	1	0	1	Rn				Ra				1	1	1	1	0	0	0	1	Rm			

T1 variant

BXAUT{<c>} <Ra>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HavePACBTIExt() then
2     if HaveDSPEExt() then UNPREDICTABLE; else UNDEFINED;
3 if Ra == '1111' then SEE SMMUL;
4 a = UInt(Ra); n = UInt(Rn); m = UInt(Rm);
5 if n IN {13,15} || m == 15 || a == 13 then UNPREDICTABLE;
6 if InITBlock() && !LastInITBlock() then CONSTRAINED_UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock() && !LastInITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes unconditionally.
- The instruction executes conditionally.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose source register containing the address to be encrypted, encoded in the "Rn" field.
<Rm>	Is the general-purpose source register containing the modifier to be used in the encryption operation, encoded in the "Rm" field.
<Ra>	Is the general-purpose register containing the encrypted value, encoded in the "Ra" field. The value in this register will be compared against the value produced by the encryption operation.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if PACEEnabled() then
```

```
4      if !ValidatePAC(R[n], R[m], R[a]) then
5          UFSR.INVSTATE = '1';
6          excInfo = CreateException(UsageFault);
7          HandleException(excInfo);
8      exc = BranchReturn(R[n], FALSE);
9      HandleException(exc);
```

Restricted behavior

Data Independent Timing behavior and the PACBTI extension

The timing of this instruction is insensitive to the value of the data being loaded or stored regardless of the value of [AIRCR.DIT](#) when accessing any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

The point at which an asynchronous exception is taken does not vary based on any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

C2.4.31 CBNZ, CBZ

Compare and Branch on Nonzero or Zero. Compare and Branch on Nonzero and Compare and Branch on Zero compare the value in a register with zero, and conditionally branch forward a constant value. They do not affect the condition flags.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	op	0	i	1	imm5					Rn		

CBNZ variant

Applies when `op == 1`.

CBNZ{<q>} <Rn>, <label>

CBZ variant

Applies when `op == 0`.

CBZ{<q>} <Rn>, <label>

Decode for this encoding

```
1 n = UInt(Rn); imm32 = ZeroExtend(i:imm5:'0', 32); nonzero = (op == '1');
2 if InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<q>	See C1.2.5 <i>Standard assembler syntax fields</i> .
<Rn>	Is the general-purpose register to be tested, encoded in the "Rn" field.
<label>	Is the program label to be conditionally branched to. Its offset from the PC, a multiple of 2 in the range 0 to 126, is encoded as "i:imm5" times 4.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 if nonzero != IsZero(R[n]) then
3     BranchTo(PC + imm32);
```


C2.4.32 CDP, CDP2

Coprocessor Data Processing. Coprocessor Data Processing tells a coprocessor to perform an operation.

If no coprocessor can execute the instruction, a UsageFault exception is generated.

Arm reserves coprocessors CP8 to CP15, and this manual defines the valid instructions when coproc is in this range.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	1	1	1	0	opc1				CRn				CRd				coproc				opc2				0	CRm			

T1 variant

CDP{<c>}{<q>} <coproc>, {#}<opc1>, <CRd>, <CRn>, <CRm> {, {#}<opc2>}

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	1	1	0	opc1				CRn				CRd				coproc				opc2				0	CRm			

T2 variant

CDP2{<c>}{<q>} <coproc>, {#}<opc1>, <CRd>, <CRn>, <CRm> {, {#}<opc2>}

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoded in the "coproc" field. The generic coprocessor names are p0 to p15.
<opc1>	Is a coprocessor-specific opcode, in the range 0 to 15, encoded in the "opc1" field.
<CRd>	Is the destination coprocessor register, encoded in the "CRd" field.
<CRn>	Is the coprocessor register that contains the first operand, encoded in the "CRn" field.
<CRm>	Is the coprocessor register that contains the second operand, encoded in the "CRm" field.
<opc2>	Is a coprocessor-specific opcode in the range 0 to 7, defaulting to 0 and encoded in the "opc2" field.

Operation for all encodings

```
1 if ConditionPassed() then  
2     EncodingSpecificOperations();  
3  
4     if !Coprocc_Accepted(cp, ThisInstr()) then  
5         GenerateCoproccException();  
6     else  
7         Coproc_InternalOperation(cp, ThisInstr());
```

C2.4.33 CINC

Conditional Increment. Returns, in the destination register, the value of the source register incremented by 1, if the condition is TRUE. Otherwise returns the value of the source register.

This is an alias of [CSINC](#) with the following condition satisfied: $Rn == Rm \ \&\& \ Rn \neq 15$.

This alias is the preferred disassembly.

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	Rn				1	(0)	0	1	Rd				fcond				Rn			

CINC variant

CINC<q> Rd, Rn, <fcond>

is equivalent to

CSINC<q> Rd, Rn, Rn, invert (<cond>)

and is the preferred disassembly when $Rn == Rm \ \&\& \ Rn \neq 15$

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.34 CINV

Conditional Invert. Returns, in the destination register, the bitwise inversion of the value of the source register, if the condition is TRUE. Otherwise returns the value of the source register.

This is an alias of [CSINV](#) with the following condition satisfied: $Rn == Rm \ \&\& \ Rn \neq 15$.

This alias is the preferred disassembly.

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	Rn				1	(0)	1	0	Rd				fcond				Rn			

T1: CINV variant

CINV<q> Rd, Rn, <fcond>

is equivalent to

CSINV<q> Rd, Rn, Rn, invert (<cond>)

and is the preferred disassembly when $Rn == Rm \ \&\& \ Rn \neq 15$

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.35 CLREX

Clear Exclusive. Clear Exclusive clears the local record of the executing PE that an address has had a request for an exclusive access.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	1	(1)	(1)	(1)	(1)	1	0	(0)	0	(1)	(1)	(1)	(1)	0	0	1	0	(1)	(1)	(1)	(1)

T1 variant

CLREX{<c>}{<q>}

Decode for this encoding

```
1 // No additional decoding required
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
<q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     ClearExclusiveLocal(ProcessorID());
```

C2.4.36 CLRM

Clear multiple. Zeros the specified general-purpose registers. It is IMPLEMENTATION DEFINED whether this instruction is interrupt-continuable. See EPSR.ICI. If an exception returns to this instruction with nonzero EPSR.ICI bits, and the PE does not support interrupt-continuable behavior, the instruction restarts from the beginning.

T1

Armv8.1-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	0	(0)	1	1	1	1	1	A	M	(0)	register_list												

T1: CLRM variant

CLRM<c><q> <registers>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 if !HasArchVersion(Armv8p1) then CONSTRAINED_UNPREDICTABLE;
3
4 registers = A:M:'0':register_list;
5 if BitCount(registers) < 1 then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <registers> A list of the registers to clear, separated by commas and surrounded by { and }. The valid registers are APSR, LR/R14, and R12-R0, and are encoded as a bitmask in the A, M and register_list fields.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   for i = 0 to 14
5     if registers[i] == '1' then
6       R[i] = Zeros(32);
7
8   if registers[15] == '1' then
9     APSR = Zeros(32);
```

C2.4.37 CLZ

Count Leading Zeros. Count Leading Zeros returns the number of binary zero bits before the first binary one bit in a value.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	1	1	Rm				1	1	1	1	Rd				1	0	0	0	Rm2			

T1 variant

CLZ{<c>}{<q>} <Rd>, <Rm>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 if Rm != Rm2 then UNPREDICTABLE;
3 d = UInt(Rd); m = UInt(Rm); m2 = UInt(Rm2);
4 if d IN {13,15} || m IN {13,15} || m2 IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $Rm \neq Rm2$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as described, with no change to its behavior and no additional side effects.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field. Its number must be encoded twice.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = CountLeadingZeroBits(R[m]);
4     R[d] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.

- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.38 CMN (immediate)

Compare Negative (immediate). Compare Negative (immediate) adds a register value and an immediate value. It updates the condition flags based on the result, and discards the result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	1	0	0	0	1	Rn				0	imm3				1	1	1	1	imm8							

T1 variant

CMN{<c>}{<q>} <Rn>, #<const>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); imm32 = T32ExpandImm(i:imm3:imm8);
3 if n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rn> Is the general-purpose source register, encoded in the "Rn" field.
 <const> Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See [C1.5 Modified immediate constants](#) for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     (result, carry, overflow) = AddWithCarry(R[n], imm32, '0');
4     APSR.N = result[31];
5     APSR.Z = IsZeroBit(result);
6     APSR.C = carry;
7     APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.39 CMN (register)

Compare Negative (register). Compare Negative (register) adds a register value and an optionally-shifted register value. It updates the condition flags based on the result, and discards the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	1	0	1	1	Rm			Rn		

T1 variant

CMN{<c>}{<q>} <Rn>, <Rm>

Decode for this encoding

```
1 n = UInt(Rn); m = UInt(Rm);
2 (shift_t, shift_n) = (SRTType_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	0	1	0	1	1	0	0	0	1	Rn				(0)	imm3			1	1	1	1	imm2						Rm			
																						sr_type											

Rotate right with extend variant

Applies when `imm3 == 000 && imm2 == 00 && sr_type == 11`.

CMN{<c>}{<q>} <Rn>, <Rm>, RRX

Shift or rotate by value variant

Applies when `!(imm3 == 000 && imm2 == 00 && sr_type == 11)`.

```
CMN{<c>}.W <Rn>, <Rm>
// <Rn>, <Rm> can be represented in T1
CMN{<c>}{<q>} <Rn>, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); m = UInt(Rm);
3 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
4 if n == 15 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when <code>sr_type = 00</code>

LSR when sr_type = 01
ASR when sr_type = 10
ROR when sr_type = 11
<amount> Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   shifted = Shift(R[m], shift_t, shift_n, APSR.C);
4   (result, carry, overflow) = AddWithCarry(R[n], shifted, '0');
5   APSR.N = result[31];
6   APSR.Z = IsZeroBit(result);
7   APSR.C = carry;
8   APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.40 CMP (immediate)

Compare (immediate). Compare (immediate) subtracts an immediate value from a register value. It updates the condition flags based on the result, and discards the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	1	Rn				imm8						

T1 variant

CMP{<c>}{<q>} <Rn>, #<imm8>

Decode for this encoding

```
1 n = UInt(Rn); imm32 = ZeroExtend(imm8, 32);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	1	1	0	1	1	Rn				0	imm3				1	1	1	1	imm8						

T2 variant

CMP{<c>}.W <Rn>, #<const>
// <Rn>, <const> can be represented in T1
CMP{<c>}{<q>} <Rn>, #<const>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); imm32 = T32ExpandImm(i:imm3:imm8);
3 if n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	For encoding T1: is a general-purpose source register, encoded in the "Rn" field. For encoding T2: is the general-purpose source register, encoded in the "Rn" field.
<imm8>	Is an 8-bit unsigned immediate, in the range 0 to 255, encoded in the "imm8" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (result, carry, overflow) = AddWithCarry(R[n], NOT(imm32), '1');
4   APSR.N = result[31];
5   APSR.Z = IsZeroBit(result);
6   APSR.C = carry;
7   APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.41 CMP (register)

Compare (register). Compare (register) subtracts an optionally-shifted register value from a register value. It updates the condition flags based on the result, and discards the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	1	0	1	0	Rm			Rn		

T1 variant

```
CMP{<c>}{<q>} <Rn>, <Rm>
// <Rn> and <Rm> both from R0-R7
```

Decode for this encoding

```
1 n = UInt(Rn); m = UInt(Rm);
2 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	N	Rm			Rn			

T2 variant

```
CMP{<c>}{<q>} <Rn>, <Rm>
// <Rn> and <Rm> not both from R0-R7
```

Decode for this encoding

```
1 n = UInt(N:Rn); m = UInt(Rm);
2 (shift_t, shift_n) = (SRTYPE_LSL, 0);
3 if n < 8 && m < 8 then UNPREDICTABLE;
4 if n == 15 || m == 15 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $n < 8$ && $m < 8$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as described, with no change to its behavior and no additional side effects.
- The condition flags become UNKNOWN.

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	1	1	0	1	1	Rn			(0)	imm3			1	1	1	1	imm2						Rm		
																														sr type	

Rotate right with extend variant

Applies when `imm3 == 000 && imm2 == 00 && sr_type == 11`.

`CMP{<c>}{<q>} <Rn>, <Rm>, RRX`

Shift or rotate by value variant

Applies when `!(imm3 == 000 && imm2 == 00 && sr_type == 11)`.

`CMP{<c>}.W <Rn>, <Rm>`
// <Rn>, <Rm> can be represented in T1 or T2
`CMP{<c>}{<q>} <Rn>, <Rm>, <shift> #<amount>`

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); m = UInt(Rm);
3 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
4 if n == 15 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<code><c></code>	See C1.2.5 Standard assembler syntax fields .
<code><q></code>	See C1.2.5 Standard assembler syntax fields .
<code><Rn></code>	For encoding T1 and T3: is the first general-purpose source register, encoded in the "Rn" field. For encoding T2: is the first general-purpose source register, encoded in the "N:Rn" field.
<code><Rm></code>	Is the second general-purpose source register, encoded in the "Rm" field.
<code><shift></code>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: <code>LSL</code> when <code>sr_type = 00</code> <code>LSR</code> when <code>sr_type = 01</code> <code>ASR</code> when <code>sr_type = 10</code> <code>ROR</code> when <code>sr_type = 11</code>
<code><amount></code>	Is the shift amount, in the range 1 to 31 (when <code><shift> = LSL</code> or <code>ROR</code>) or 1 to 32 (when <code><shift> = LSR</code> or <code>ASR</code>) encoded in the "imm3:imm2" field as <code><amount></code> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   shifted = Shift(R[m], shift_t, shift_n, APSR.C);
4   (result, carry, overflow) = AddWithCarry(R[n], NOT(shifted), '1');
5   APSR.N = result[31];
6   APSR.Z = IsZeroBit(result);
7   APSR.C = carry;
8   APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.

- The values of the N, Z, C, V flags.

C2.4.42 CNEG

Conditional Negate. Returns, in the destination register, the negated value of the source register if the condition is TRUE, and otherwise returns the value of the source register.

This is an alias of CSNEG with the following condition satisfied: Rn == Rm.

This alias is the preferred disassembly.

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	Rn				1	(0)	1	1	Rd				fcond				Rn			

CNEG variant

CNEG<q> Rd, Rn, <fcond>

is equivalent to

CSNEG<q> Rd, Rn, Rn, invert (<cond>)

and is the preferred disassembly when Rn == Rm

Restricted behavior

Data Independent Timing behavior

If AIRCR.DIT is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.43 CPS

Change PE State. Change PE State. The instruction modifies the PRIMASK and FAULTMASK special-purpose register values.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	1	1	0	0	1	1	im	(0)	(0)	I	F

CPSID variant

Applies when **im** == 1.

CPSID{<q>} <iflags>

CPSIE variant

Applies when **im** == 0.

CPSIE{<q>} <iflags>

Decode for this encoding

```

1 enable = (im == '0'); disable = (im == '1');
2 if InITBlock() then UNPREDICTABLE;
3 if (I == '0' && F == '0') then UNPREDICTABLE;
4 affectPRI = (I == '1'); affectFAULT = (F == '1');
5 if !HaveMainExt() then
6     if (I == '0') then UNPREDICTABLE;
7     if (F == '1') then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **I** == '0' && **F** == '0', then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If **!HaveMainExt()** && (**I** == '0' || **F** == '1'), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

Assembler symbols for all encodings

<q> See [C1.2.5 Standard assembler syntax fields](#).
 <iflags> Is a sequence of one or more of the following, specifying which interrupt mask bits are affected:
 f FAULTMASK. When set to 1, raises the execution priority to -1, the same priority as HardFault. This is a 1-bit register, that can be updated only by privileged software. The register clears to 0 on return from any exception other than NMI.
 i PRIMASK. When set to 1, raises the execution priority to 0. This is a 1-bit register, that can be updated only by privileged software.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 if CurrentModeIsPrivileged() then
3     if enable then
4         if affectPRI then
5             PRIMASK.PM = '0';
6         if affectFAULT then
7             FAULTMASK.FM = '0';
8     if disable then
9         if affectPRI then
10            PRIMASK.PM = '1';
11        if affectFAULT && ExecutionPriority() > -1 then
12            FAULTMASK.FM = '1';
```

C2.4.44 CSDB

Consumption of Speculative Data Barrier. Consumption of Speculative Data Barrier is a memory barrier that controls speculative execution and data value prediction.

No instruction other than branch instructions and instructions that write to the PC appearing in program order after the CSDB can be speculatively executed using the results of any:

- Data value predictions of any instructions.
- APSR.N, Z, C, V predictions of any instructions other than conditional branch instructions and conditional instructions that write to the PC appearing in program order before the CSDB that have not been architecturally resolved.

APSR.N, Z, C, V is not considered a data value. This instruction permits:

- Control flow speculation before and after the CSDB.
- Speculative execution of conditional data processing instructions after the CSDB, unless they use the results of data value or APSR.N, Z, C, V predictions of instructions appearing in program order before the CSDB that have not been architecturally resolved.

This is a NOP-compatible hint.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	1	0	1	0	0

T1 variant

CSDB{<c>} .W

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes unconditionally.
- The instruction executes conditionally.

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ConsumptionOfSpeculativeDataBarrier();
```

C2.4.45 CSEL

Conditional Select. Returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the value of the second source register.

This instruction is not permitted in an IT block.

T1

Armv8.1-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	Rn				1	(0)	0	0	Rd				fcond				Rm			

T1: CSEL variant

CSEL<q> Rd, Rn, Rm, <fcond>

Decode for this encoding

```

1 if Rm == '1101' then SEE "Related encodings";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then CONSTRAINED_UNPREDICTABLE;
4 d = UInt(Rd);
5 m = UInt(Rm);
6 n = UInt(Rn);
7 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
8 if Rd == '11x1' || Rn == '1101' then CONSTRAINED_UNPREDICTABLE;
9 if fcond == '111x' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Destination general-purpose register.
 <Rn> First source general-purpose register (ZR is permitted, PC is not).
 <Rm> Second source general-purpose register (ZR is permitted, PC is not).
 <fcond> The comparison condition to use. This is in the format of a standard Arm condition code.
 This parameter must be one of the following values:

EQ	Encoded as	fcond = 0000
NE	Encoded as	fcond = 0001
CS	Encoded as	fcond = 0010
CC	Encoded as	fcond = 0011
MI	Encoded as	fcond = 0100
PL	Encoded as	fcond = 0101
VS	Encoded as	fcond = 0110
VC	Encoded as	fcond = 0111
HI	Encoded as	fcond = 1000
LS	Encoded as	fcond = 1001
GE	Encoded as	fcond = 1010
LT	Encoded as	fcond = 1011
GT	Encoded as	fcond = 1100
LE	Encoded as	fcond = 1101

Operation for all encodings

```

1 EncodingSpecificOperations();
2
3 bits(32) result;
4 if ConditionHolds(fcond) then
```

```
5     result = RZ[n];  
6 else  
7     result = RZ[m];  
8 R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.46 CSET

Conditional Set. Sets the destination register to 1 if the condition is TRUE, and otherwise set it to 0.

This is an alias of [CSINC](#) with the following condition satisfied: Rn==15 && Rm==15.

This alias is the preferred disassembly.

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	0	1	0	1	1	1	1	1	(0)	0	1	Rd				fcond				1	1	1	1

CSET variant

CSET<q> Rd, <fcond>

is equivalent to

CSINC<q> Rd, Zr, Zr, invert (<cond>)

and is the preferred disassembly when Rn == 15 && Rm == 15

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.47 CSETM

Conditional Set Mask. Sets all bits of the destination register to 1 if the condition is TRUE. Otherwise sets all bits to 0.

This is an alias of [CSINV](#) with the following condition satisfied: $Rn == 15 \ \&\& \ Rm == 15$.

This alias is the preferred disassembly.

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	1	1	1	1	1	(0)	1	0	Rd				fcond				1	1	1	1

T1: CSETM variant

CSETM<q> Rd, <fcond>

is equivalent to

CSINV<q> Rd, Zr, Zr, invert (<cond>)

and is the preferred disassembly when $Rn == 15 \ \&\& \ Rm == 15$

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.48 CSINC

Conditional Select Increment. Returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the value of the second source register incremented by 1.

This instruction is not permitted in an IT block.

T1

Armv8.1-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	Rn				1	(0)	0	1	Rd				fcond				Rm			

T1: CSINC variant

CSINC<q> Rd, Rn, Rm, <fcond>

Decode for this encoding

```

1 if Rm == '1101' then SEE "Related encodings";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then CONSTRAINED_UNPREDICTABLE;
4 d = UInt(Rd);
5 m = UInt(Rm);
6 n = UInt(Rn);
7 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
8 if Rd == '11x1' || Rn == '1101' then CONSTRAINED_UNPREDICTABLE;
9 if fcond == '111x' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Destination general-purpose register.
 <Rn> First source general-purpose register (ZR is permitted, PC is not).
 <Rm> Second source general-purpose register (ZR is permitted, PC is not).
 <fcond> The comparison condition to use. This is in the format of a standard Arm condition code.
 This parameter must be one of the following values:

EQ	Encoded as	fcond = 0000
NE	Encoded as	fcond = 0001
CS	Encoded as	fcond = 0010
CC	Encoded as	fcond = 0011
MI	Encoded as	fcond = 0100
PL	Encoded as	fcond = 0101
VS	Encoded as	fcond = 0110
VC	Encoded as	fcond = 0111
HI	Encoded as	fcond = 1000
LS	Encoded as	fcond = 1001
GE	Encoded as	fcond = 1010
LT	Encoded as	fcond = 1011
GT	Encoded as	fcond = 1100
LE	Encoded as	fcond = 1101

Operation for all encodings

```

1 EncodingSpecificOperations();
2
3 bits(32) result;
4 if ConditionHolds(fcond) then
```

```
5     result = RZ[n];  
6 else  
7     result = RZ[m] + 1;  
8 R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.49 CSINV

Conditional Select Invert. Returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the value of the second source register, bitwise inverted.

This instruction is not permitted in an IT block.

T1

Armv8.1-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	Rn				1	(0)	1	0	Rd				fcond				Rm			

T1: CSINV variant

CSINV<q> Rd, Rn, Rm, <fcond>

Decode for this encoding

```

1 if Rm == '1101' then SEE "Related encodings";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then CONSTRAINED_UNPREDICTABLE;
4 d = UInt(Rd);
5 m = UInt(Rm);
6 n = UInt(Rn);
7 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
8 if Rd == '11x1' || Rn == '1101' then CONSTRAINED_UNPREDICTABLE;
9 if fcond == '111x' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<q> See C1.2.5 *Standard assembler syntax fields*.
 <Rd> Destination general-purpose register.
 <Rn> First source general-purpose register (ZR is permitted, PC is not).
 <Rm> Second source general-purpose register (ZR is permitted, PC is not).
 <fcond> The comparison condition to use. This is in the format of a standard Arm condition code.
 This parameter must be one of the following values:

EQ	Encoded as	fcond = 0000
NE	Encoded as	fcond = 0001
CS	Encoded as	fcond = 0010
CC	Encoded as	fcond = 0011
MI	Encoded as	fcond = 0100
PL	Encoded as	fcond = 0101
VS	Encoded as	fcond = 0110
VC	Encoded as	fcond = 0111
HI	Encoded as	fcond = 1000
LS	Encoded as	fcond = 1001
GE	Encoded as	fcond = 1010
LT	Encoded as	fcond = 1011
GT	Encoded as	fcond = 1100
LE	Encoded as	fcond = 1101

Operation for all encodings

```

1 EncodingSpecificOperations();
2
3 bits(32) result;
4 if ConditionHolds(fcond) then
```

```
5     result = RZ[n];  
6 else  
7     result = NOT(RZ[m]);  
8 R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.50 CSNEG

Conditional Select Negation. Returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the value of the second source register negated.

This instruction is not permitted in an IT block.

T1

Armv8.1-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	Rn				1	(0)	1	1	Rd				fcond				Rm			

T1: CSNEG variant

CSNEG<q> Rd, Rn, Rm, <fcond>

Decode for this encoding

```

1 if Rm == '1101' then SEE "Related encodings";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then CONSTRAINED_UNPREDICTABLE;
4 d = UInt(Rd);
5 m = UInt(Rm);
6 n = UInt(Rn);
7 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
8 if Rd == '11x1' || Rn == '1101' then CONSTRAINED_UNPREDICTABLE;
9 if fcond == '111x' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Destination general-purpose register.
 <Rn> First source general-purpose register (ZR is permitted, PC is not).
 <Rm> Second source general-purpose register (ZR is permitted, PC is not).
 <fcond> The comparison condition to use. This is in the format of a standard Arm condition code.
 This parameter must be one of the following values:

EQ	Encoded as	fcond = 0000
NE	Encoded as	fcond = 0001
CS	Encoded as	fcond = 0010
CC	Encoded as	fcond = 0011
MI	Encoded as	fcond = 0100
PL	Encoded as	fcond = 0101
VS	Encoded as	fcond = 0110
VC	Encoded as	fcond = 0111
HI	Encoded as	fcond = 1000
LS	Encoded as	fcond = 1001
GE	Encoded as	fcond = 1010
LT	Encoded as	fcond = 1011
GT	Encoded as	fcond = 1100
LE	Encoded as	fcond = 1101

Operation for all encodings

```

1 EncodingSpecificOperations();
2
3 bits(32) result;
4 if ConditionHolds(fcond) then
```

```
5     result = RZ[n];  
6 else  
7     result = NOT(RZ[m]);  
8     result = result + 1;  
9 R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.51 CX1

Custom Instruction Class 1. Custom instruction class 1 computes a value based on an immediate, and optionally the destination value, and writes the result to the destination register.

The source and destination registers can be either general-purpose registers or the Condition flags, specified by use of APSR_nzcv.

T1

Armv8-M Custom Datapath Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	A	1	1	1	0	0	0	op1				Rd				0	coproc				op2	0	op3						

Accumulator variant

Applies when **A == 1**

CX1A<c>, <coproc>, <Rd>, #<imm>

Non-accumulator variant

Applies when **A == 0**

CX1 <coproc>, <Rd>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocessor instructions";
2 if !HaveMainExt() then UNDEFINED;
3 cp = UInt(coproc);
4 d = UInt(Rd);
5 imm = op1:op2:op3;
6 acc = (A == '1');
7 CheckCPDecodeFaults(cp);
8 if d == 13 then UNPREDICTABLE;
9 if !acc && InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **d == 13**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns an UNKNOWN value in SP and it is UNKNOWN whether a stack limit check is performed.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If **!acc && InITBlock()**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 - A Encoded as A = 1
<c>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<Rd>	Is the general-purpose R0 – R14 or APSR_nzcv destination register, encoded in the "Rd" field. For accumulator variants <Rd> also specifies the source register. APSR_nzcv is encoded by the "Rd" field value 0b1111.
<imm>	Is the immediate encoded in op1:op2:op3.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if (!Coprocc_Accepted(cp, ThisInstr())) then
4         GenerateCoproccorException();
5     elseif acc then
6         RF[d] = CX_op1(ThisInstr(), RF[d], 32);
7     else
8         RF[d] = CX_op0(ThisInstr(), 32);
```


C2.4.52 CX1D

Custom Instruction Class 1. Custom instruction class 1 dual computes a value based on an immediate, and optionally the destination register pair value, and writes the result to a destination register pair.

The destination registers are a consecutive pair of general-purpose registers.

The significance of the words in each pair is consistent with the current data endianness.

T1

Armv8-M Custom Datapath Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	A	1	1	1	0	0	0	op1				Rd				0	coproc				op2	1	op3						

Accumulator variant

Applies when **A == 1**

CX1DA<c>, <coproc>, <Rd>, <Rd+1>, #<imm>

Non-accumulator variant

Applies when **A == 0**

CX1D <coproc>, <Rd>, <Rd+1>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocessor instructions";
2 if !HaveMainExt() then UNDEFINED;
3 cp = UInt(coproc);
4 d = UInt(Rd);
5 d2 = d + 1;
6 imm = op1:op2:op3;
7 acc = (A == '1');
8 CheckCPDecodeFaults(cp);
9 // Register pairs containing SP or PC are UNPREDICTABLE.
10 if d > 10 then UNPREDICTABLE;
11 if Rd[0] == '1' then UNPREDICTABLE;
12 if !acc && InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **d** is odd, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns and UNKNOWN value in **Rd**, **Rd + 1** and **Rd - 1**.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If **d == 12**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns and UNKNOWN value in **R12** and **SP**.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If `d == 12 || d == 13`, then one of the following behaviors must occur:

- It is UNKNOWN whether a stack limit check is performed.

CONSTRAINED UNPREDICTABLE behavior

If `d == 14`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If `!acc && InITBlock()`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

<code><A></code>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as <code>A = 0</code> <code>A</code> Encoded as <code>A = 1</code>
<code><c></code>	See C1.2.5 Standard assembler syntax fields .
<code><coproc></code>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<code><Rd></code>	Is the general-purpose register R0 – R10 specifying the first of destination register pair, encoded in the "Rd" field. For accumulator variants, <code><Rd></code> also specifies the source register.
<code><imm></code>	Is the immediate encoded in <code>op1:op2:op3</code> .

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if (!Coprocc_Accepted(cp, ThisInstr())) then
4         GenerateCoprocc_Exception();
5     elseif acc then
6         RFD[d] = CX_op1(ThisInstr(), RFD[d], 64);
7     else
8         RFD[d] = CX_op0(ThisInstr(), 64);
```

C2.4.53 CX2

Custom Instruction Class 2. Custom instruction class 2 computes a value based on a source register, an immediate, and optionally the destination value, and writes the result to the destination register. The source and destination registers can be either general-purpose registers or the Condition flags, specified by use of APSR_nzcv.

T1

Armv8-M Custom Datapath Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	A	1	1	1	0	0	1	op1			Rn			Rd			0			coproc			op2		0	op3				

Accumulator variant

Applies when **A == 1**

CX2A<c>, <coproc>, <Rd>, <Rn>, #<imm>

Non-accumulator variant

Applies when **A == 0**

CX2 <coproc>, <Rd>, <Rn>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocessor instructions";
2 if !HaveMainExt() then UNDEFINED;
3 cp = UInt(coproc);
4 d = UInt(Rd);
5 n = UInt(Rn);
6 imm = op1:op2:op3;
7 acc = (A == '1');
8 CheckCPDecodeFaults(cp);
9 if d == 13 || n == 13 then UNPREDICTABLE;
10 if !acc && InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **d == 13**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns an UNKNOWN value in SP and it is UNKNOWN whether a stack limit check is performed.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If **n == 13**, then one of the following behaviors must occur:

- The instruction executes as NOP.
- The instruction returns an UNKNOWN value.
- The instruction is UNDEFINED.

CONSTRAINED UNPREDICTABLE behavior

If **!acc && InITBlock()**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 - A Encoded as A = 1
<c>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<Rd>	Is the general-purpose R0 – R14 or APSR_nzcv destination register, encoded in the "Rd" field. For accumulator variants <Rd> also specifies the source register. APSR_nzcv is encoded by the "Rd" field value 0b1111.
<Rn>	Is the general-purpose R0 – R14 or APSR_nzcv source register, encoded in the "Rn" field. APSR_nzcv is encoded by the "Rn" field value 0b1111.
<imm>	Is the immediate encoded in op1:op2:op3.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      if (!Coprocc_Accepted(cp, ThisInstr())) then
4          GenerateCoproccorException();
5      elseif acc then
6          RF[d] = CX_op2(ThisInstr(), RF[d], RF[n], 32);
7      else
8          RF[d] = CX_op1(ThisInstr(), RF[n], 32);

```

C2.4.54 CX2D

Custom Instruction Class 2. Custom instruction class 2 dual computes a value based on a source register, an immediate, and optionally the destination register pair value, and writes the result to the destination register pair.

The destination registers are a consecutive pair of general-purpose registers.

The source registers can be either general-purpose registers or the Condition flags, specified by use of APSR_nzcv.

The significance of the words in each pair is consistent with the current data endianness.

T1

Armv8-M Custom Datapath Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	A	1	1	1	0	0	1	op1			Rn			Rd			0			coproc			op2		1	op3			

Accumulator variant

Applies when **A == 1**

CX2DA<c>, <coproc>, <Rd>, <Rd+1>, <Rn>, #<imm>

Non-accumulator variant

Applies when **A == 0**

CX2D <coproc>, <Rd>, <Rd+1>, <Rn>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocessor instructions";
2 if !HaveMainExt() then UNDEFINED;
3 cp = UInt(coproc);
4 d = UInt(Rd);
5 n = UInt(Rn);
6 imm = op1:op2:op3;
7 acc = (A == '1');
8 CheckCPDecodeFaults(cp);
9 // Register pairs containing SP or PC are UNPREDICTABLE.
10 if d > 10 then UNPREDICTABLE;
11 if n == 13 then UNPREDICTABLE;
12 if Rd[0] == '1' then UNPREDICTABLE;
13 if !acc && InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **d** is odd, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns and UNKNOWN value in **Rd**, **Rd + 1** and **Rd - 1**.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If **d == 12**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns and UNKNOWN value in **R12** and **SP**.

- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If `d == 12 || d == 13`, then one of the following behaviors must occur:

- It is UNKNOWN whether a stack limit check is performed.

CONSTRAINED UNPREDICTABLE behavior

If `d == 14`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If `n == 13`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The value in the destination register is UNKNOWN.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If `!acc && InITBlock()`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

<code><A></code>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as <code>A = 0</code> A Encoded as <code>A = 1</code>
<code><c></code>	See C1.2.5 Standard assembler syntax fields .
<code><coproc></code>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<code><Rd></code>	Is the general-purpose register R0 – R10 specifying the first of destination register pair, encoded in the "Rd" field. For accumulator variants, <code><Rd></code> also specifies the source register.
<code><Rn></code>	Is the general-purpose R0 – R14 or APSR_nzcv source register, encoded in the "Rn" field. APSR_nzcv is encoded by the "Rn" field value 0b1111.
<code><imm></code>	Is the immediate encoded in <code>op1:op2:op3</code> .

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if (!Coprocc_Accepted(cp, ThisInstr())) then
4         GenerateCoproccorException();
5     elseif acc then
6         RFD[d] = CX_op2(ThisInstr(), RFD[d], RF[n], 64);
7     else
8         RFD[d] = CX_op1(ThisInstr(), RF[n], 64);

```

C2.4.55 CX3

Custom Instruction Class 3. Custom instruction class 3 computes a value based two source registers, an immediate and optionally the destination value, and writes the result to the destination register.

The source and destination registers can be either general-purpose registers or the Condition flags, specified by use of APSR_nzcv.

T1

Armv8-M Custom Datapath Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	1	A	1	1	1	0	1	op1				Rn				Rm				0	coproc				op2	0	op3				Rd			

Accumulator variant

Applies when **A == 1**

CX3A<c>, <coproc>, <Rd>, <Rn>, <Rm>, #<imm>

Non-accumulator variant

Applies when **A == 0**

CX3 <coproc>, <Rd>, <Rn>, <Rm>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocessor instructions";
2 if !HaveMainExt() then UNDEFINED;
3 cp = UInt(coproc);
4 d = UInt(Rd);
5 n = UInt(Rn);
6 m = UInt(Rm);
7 imm = op1:op2:op3;
8 acc = (A == '1');
9 CheckCPDecodeFaults(cp);
10 if d == 13 || n == 13 || m == 13 then UNPREDICTABLE;
11 if !acc && InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **d == 13**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns an UNKNOWN value in SP and it is UNKNOWN whether a stack limit check is performed.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If **n == 13 or m == 13**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns an UNKNOWN value in SP.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If `!acc && InITBlock()`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 - A Encoded as A = 1
<c>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<Rd>	Is the general-purpose R0 – R14 or APSR_nzcv destination register, encoded in the "Rd" field. For accumulator variants <Rd> also specifies the source register. APSR_nzcv is encoded by the "Rd" field value 0b1111.
<Rn>	Is the general-purpose R0 – R14 or APSR_nzcv destination register, encoded in the "Rn" field. APSR_nzcv is encoded by the "Rn" field value 0b1111.
<Rm>	Is the general-purpose R0 – R14 or APSR_nzcv destination register, encoded in the "Rm" field. APSR_nzcv is encoded by the "Rm" field value 0b1111.
<imm>	Is the immediate encoded in op1:op2:op3.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if (!CoproccAccepted(cp, ThisInstr())) then
4         GenerateCoproccorException();
5     elseif acc then
6         RF[d] = CX_op3(ThisInstr(), RF[d], RF[n], RF[m], 32);
7     else
8         RF[d] = CX_op2(ThisInstr(), RF[n], RF[m], 32);

```


C2.4.56 CX3D

Custom Instruction Class 3. Custom instruction class 3 dual computes a value based on two source registers, an immediate, and optionally the destination register pair value, and writes the result to the destination register pair.

The source registers can be either general-purpose registers or the Condition flags, specified by use of APSR_nzcv.

The destination registers are a consecutive pair of general-purpose registers.

The significance of the words in each pair is consistent with the current data endianness.

T1

Armv8-M Custom Datapath Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	1	A	1	1	1	0	1	op1				Rn				Rm				0	coproc				op2	1	op3				Rd			

Accumulator variant

Applies when **A == 1**

CX3DA<c>, <coproc>, <Rd>, <Rd+1>, <Rn>, <Rm>, #<imm>

Non-accumulator variant

Applies when **A == 0**

CX3D <coproc>, <Rd>, <Rd+1>, <Rn>, <Rm>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocessor instructions";
2 if !HaveMainExt() then UNDEFINED;
3 cp = UInt(coproc);
4 d = UInt(Rd);
5 n = UInt(Rn);
6 m = UInt(Rm);
7 imm = op1:op2:op3;
8 acc = (A == '1');
9 CheckCPDecodeFaults(cp);
10 // Register pairs containing SP or PC are UNPREDICTABLE.
11 if d > 10 then UNPREDICTABLE;
12 if n == 13 || m == 13 then UNPREDICTABLE;
13 if Rd[0] == '1' then UNPREDICTABLE;
14 if !acc && InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **d** is odd, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns and UNKNOWN value in **Rd**, **Rd + 1** and **Rd - 1**.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If **d == 12**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction returns and UNKNOWN value in **SP**.

- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If `d == 12 || d == 13`, then one of the following behaviors must occur:

- It is UNKNOWN whether a stack limit check is performed.

CONSTRAINED UNPREDICTABLE behavior

If `d == 14`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If `n == 13 || m == 13`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The value in the destination register is UNKNOWN.
- The instruction executes as NOP.

CONSTRAINED UNPREDICTABLE behavior

If `!acc && InITBlock()`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as <code>A = 0</code> <code>A</code> Encoded as <code>A = 1</code>
<c>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<Rd>	Is the general-purpose register R0 – R10 specifying the first of destination register pair, encoded in the "Rd" field. For accumulator variants, <Rd> also specifies the source register.
<Rn>	Is the general-purpose R0 – R14 or APSR_nzcv source register, encoded in the "Rn" field. APSR_nzcv is encoded by the "Rn" field value 0b1111.
<Rm>	Is the general-purpose R0 – R14 or APSR_nzcv source register, encoded in the "Rm" field. APSR_nzcv is encoded by the "Rm" field value 0b1111.
<imm>	Is the immediate encoded in <code>op1:op2:op3</code> .

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   if (!Coprocc_Accepted(cp, ThisInstr())) then
4     GenerateCoproccorException();
5   elseif acc then
6     RFD[d] = CX_op3(ThisInstr(), RFD[d], RF[n], RF[m], 64);
7   else
8     RFD[d] = CX_op2(ThisInstr(), RF[n], RF[m], 64);

```

C2.4.57 DBG

Debug hint. Debug Hint provides a hint to debug trace support and related debug systems. See debug architecture documentation for what use (if any) is made of this instruction.

DBG is a NOP-compatible hint.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	1	1	1	1	option			

T1 variant

DBG{<c>}{<q>} #<option>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 // Any decoding of 'option' is specified by the debug system
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <option> Is a 4-bit unsigned immediate, in the range 0 to 15, encoded in the "option" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     Hint_Debug(option);
```

C2.4.58 DMB

Data Memory Barrier. Data Memory Barrier acts as a memory barrier. It ensures that all explicit memory accesses that appear in program order before the DMB instruction are observed before any explicit memory accesses that appear in program order after the DMB instruction. It does not affect the ordering of any other instructions executing on the PE.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	0	1	1	1	0	1	1	(1)	(1)	(1)	(1)	1	0	(0)	0	(1)	(1)	(1)	(1)	0	1	0	1	option				

T1 variant

DMB{<c>}{<q>} {<option>}

Decode for this encoding

```
1 // No additional decoding required
```

Assembler symbols for all encodings

- <c> See C1.2.5 Standard assembler syntax fields.
- <q> See C1.2.5 Standard assembler syntax fields.
- <option> Specifies an optional limitation on the barrier operation. Values are:
 - SY Full system barrier operation, encoded as option = 0b1111. Can be omitted.All other encodings of option are reserved. The corresponding instructions execute as full system barrier operations, but must not be relied upon by software.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   DataMemoryBarrier(option);
```

C2.4.59 DSB

Data Synchronization Barrier. Data Synchronization Barrier acts as a special kind of memory barrier. No instruction in program order after this instruction can execute until this instruction completes. This instruction completes only when both:

- Any explicit memory access made before this instruction is complete.
- The side effects of any SCS access that performs a context-altering operation are visible.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	1	(1)	(1)	(1)	(1)	1	0	(0)	0	(1)	(1)	(1)	(1)	0	1	0	0	option			

T1 variant

DSB{<c>}{<q>} {<option>}

Decode for this encoding

```
1 if option == '0000' then SEE "SSBB";
2 if option == '0100' then SEE "PSSBB";
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <option> Specifies an optional limitation on the barrier operation. Values are:
 option Encoded as 0b1111 in bits[3:0] of the instruction encoding.
 SY Full system barrier operation, encoded as option = 0b1111. Can be omitted.
 When option = 0b0000, this instruction encoding corresponds to SSBB. When option = 0b0100, this instruction encoding corresponds to PSSBB. All other encodings of option are reserved and execute as full system barrier operations, but must not be relied upon by software.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     // SY encoded as option == '1111'
4     DataSynchronizationBarrier('1111');
```

C2.4.60 EOR (immediate)

Exclusive OR (immediate). Exclusive OR (immediate) performs a bitwise Exclusive OR of a register value and an immediate value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	0	1	0	0	S	Rn				0	imm3				Rd				imm8							

EOR variant

Applies when **S == 0**.

EOR{<c>}{<q>} {<Rd>}, {<Rn>}, #<const>

EORS variant

Applies when **S == 1 && Rd != 1111**.

EORS{<c>}{<q>} {<Rd>}, {<Rn>}, #<const>

Decode for this encoding

```

1 if Rd == '1111' && S == '1' then SEE "TEQ (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1');
4 (imm32, carry) = T32ExpandImm_C(i:imm3:imm8, APSR.C);
5 if d == 13 || (d == 15 && S == '0') || n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = R[n] EOR imm32;
4     R[d] = result;
5     if setflags then
6         APSR.N = result[31];
7         APSR.Z = IsZeroBit(result);
8         APSR.C = carry;
9         // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.61 EOR (register)

Exclusive OR (register). Exclusive OR (register) performs a bitwise Exclusive OR of a register value and an optionally-shifted register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Arm v8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	0	0	1	Rm			Rdn		

T1 variant

```
EOR<c>{<q>} {<Rdn>,} <Rdn>, <Rm>
    // Inside IT block
EORS{<q>} {<Rdn>,} <Rdn>, <Rm>
    // Outside IT block
```

Decode for this encoding

```
1  d = UInt(Rdn);  n = UInt(Rdn);  m = UInt(Rm);  setflags = !InITBlock();
2  (shift_t, shift_n) = (SRTypL_SLL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	1	0	1	0	1	0	1	0	0	S	Rn			(0)	imm3	Rd			imm2				Rm											
																								sr type											

EOR, rotate right with extend variant

Applies when S == 0 && imm3 == 000 && imm2 == 00 && sr_type == 11.

EOR { <c> } { <q> } { <Rd> , } <Rn> , <Rm> , RRX

EOR, shift or rotate by value variant

Applies when S == 0 && !(imm3 == 000 && imm2 == 00 && sr_type == 11).

```

EOR<c>.W {<Rd>, } <Rn>, <Rm>
    // Inside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
EOR<c>{<q> } {<Rd>, } <Rn>, <Rm> {, <shift> #<amount>}

```

EORS, rotate right with extend variant

```
Applies when S == 1 && imm3 == 000 && Rd != 1111 && imm2 == 00 && sr_type == 11.
```

$$\text{EORS} \{ \langle c \rangle \} \{ \langle q \rangle \} \{ \langle Rd \rangle, \langle Rn \rangle, \langle Rm \rangle, \text{RRX} \}$$

EORS, shift or rotate by value variant

```
Applies when S == 1 && !(imm3 == 000 && imm2 == 00 && sr_type == 11) && Rd != 1111.
```



```
EORS.W {<Rd>}, {<Rn>}, <Rm>
// Outside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
EORS{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if Rd == '1111' && S == '1' then SEE "TEQ (register)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
4 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
5 if d == 13 || (d == 15 && S == '0') || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdn>	Is the first general-purpose source register and the destination register, encoded in the "Rdn" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (shifted, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);
4   result = R[n] EOR shifted;
5   R[d] = result;
6   if setflags then
7     APSR.N = result[31];
8     APSR.Z = IsZeroBit(result);
9     APSR.C = carry;
10    // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.62 ESB

Error Synchronization Barrier. Error Synchronization Barrier is used to synchronize any asynchronous RAS exceptions. That is, RAS errors notified to the PE will not silently propagate past this instruction. This is a NOP-compatible hint.

T1

Armv8.1-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	1	0	0	0	0

T1: ESB variant

ESB<c><q>

Decode for this encoding

```
1
2 if !HaveMainExt() then UNDEFINED;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     supported = boolean IMPLEMENTATION_DEFINED "Detection of Unrecoverable errors supported";
5     if HasArchVersion(Armv8p1) && supported then
6         exc = SynchronizeBusFault();
7         // When an ESB is executed, the return address for any resulting exception is the
8         // ESB instruction. This is accomplished by indicating that any exception that will
9         // preempt should terminate the execution of the ESB.
10        if exc.fault != NoFault then
11            if ExceptionPriority(exc.fault, exc.isSecure, TRUE) < ExecutionPriority() then
12                exc.termInst = TRUE;
13        HandleException(exc);
14        // An ESB acts as a Data Synchronization Barrier when either:
15        // - There is a pending BusFault (which might have been forced to be recognised as
16        //   a direct result of the call to SynchronizeBusFault() above).
17        // - There is a pending HardFault (which might have been caused by an asynchronous
18        //   BusFault which escalated due to the BusFault exception being disabled).
19        //
20        // This behavior guarantees that a subsequent load of the memory mapped syndrome
21        // registers and pending bits (BFSR, RFSR, and SHCSR) return the correct values.
22        //
23        // If a BusFault exception is recognized or escalated as a result of the ESB
24        // instruction affects the current Security state, then the values in the syndrome
25        // registers must be congruent with the current Security state.
26        // When determining whether the ESB instruction also acts as an implicit DSB, only
27        // the corresponding exception bits associated with the current Security state must
28        // be checked.
29        if SHCSR.BUSFAULTPENDE == '1' || SHCSR.HARDFFAULTPENDE == '1' then
30            DataSynchronizationBarrier('1111');
```

C2.4.63 FLDMDBX, FLDMIAX

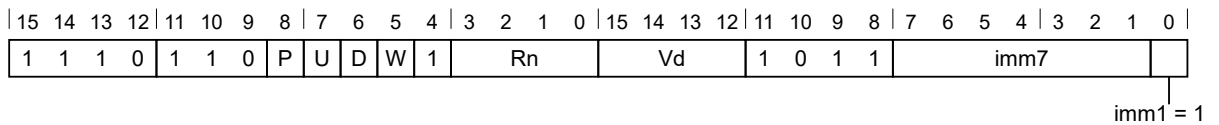
FLDMX (Decrement Before, Increment After). FLDMX (Decrement Before, Increment After) loads multiple extension registers from consecutive memory locations using an address from a general-purpose register.

Arm deprecates use of FLDMDBX and FLDMIAX, except for disassembly purposes, and reassembly of disassembled code.

This instruction is subject to stack limit checking.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE



Decrement Before variant

Applies when **P == 1 && U == 0 && W == 1**.

FLDMDBX{<c>}{<q>} <Rn>{!}, <dreglist>

Increment After variant

Applies when **P == 0 && U == 1**.

FLDMIAX{<c>}{<q>} <Rn>{!}, <dreglist>

Decode for this encoding

```

1 if P == '0' && U == '0' && W == '0' then SEE "Related encodings";
2 if P == '0' && U == '1' && W == '0' && Rn == '1111' then SEE "VSCCLRM";
3 if P == '1' && W == '0' then SEE VLDR;
4 CheckDecodeFaults(ExtType_MveOrFp);
5 if P == U && W == '1' then UNDEFINED;
6 // Remaining combinations are PUW = 010 (IA without !), 011 (IA with !), 101 (DB with !)
7 single_regs = FALSE; add = (U == '1'); wback = (W == '1'); imm8 = imm7 : imm1;
8 d = UInt(D:Vd); n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
9 regs = UInt(imm8) DIV 2;
10 if n == 15 then UNPREDICTABLE;
11 if regs == 0 || (d+regs) > 32 then UNPREDICTABLE;
12 if !HasArchVersion(Armv8pl) then
13     if VFPSmallRegisterBank() && (d+regs) > 16 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **regs == 0**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as a FLDMX with the same addressing mode but loads no registers.

CONSTRAINED UNPREDICTABLE behavior

If **(d+regs) > 32**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

- One or more of the floating-point registers are UNKNOWN. If the instruction specifies write-back, the base register becomes UNKNOWN. This behavior does not affect any general-purpose registers.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
!	Specifies base register write-back. Encoded in the "W" field as 1 if present, otherwise 0.
<dreglist>	Is the list of consecutively numbered 64-bit floating-point registers to be transferred. The first register in the list is encoded in "D:Vd", and "imm8" is set to twice the number of registers in the list plus one. The list must contain at least one register.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      address = if add then R[n]           else R[n]-imm32;
5      regval  = if add then R[n]+imm32 else R[n]-imm32;
6
7      // Determine if the stack pointer limit must be checked
8      if n == 13 && wback then
9          // If memory operation is not performed as a result of a stack limit violation,
10         // and the write-back of the SP itself does not raise a stack limit violation, it
11         // is "IMPLEMENTATION_DEFINED" whether a SPLIM exception is raised.
12         // Arm recommends that any instruction which discards a memory access as
13         // a result of a stack limit violation, and where the write-back of the SP itself
14         // does not raise a stack limit violation, generates an SPLIM exception.
15         if boolean IMPLEMENTATION_DEFINED "SPLIM exception on invalid memory access" then
16             if ViolatesSPLim(LookUpSP(), address) then
17                 if HaveMainExt() then
18                     UFSR.STKOF = '1';
19                     // If the Main Extension is not implemented the fault always escalates to
20                     // a HardFault
21                     excInfo = CreateException(UsageFault);
22                     HandleException(excInfo);
23             applylimit = TRUE;
24         else
25             applylimit = FALSE;
26
27         // Memory operation only performed if limit not violated
28         if !(applylimit && ViolatesSPLim(LookUpSP(), regval)) then
29             for r = 0 to regs-1
30                 if single_regs then
31                     if (d+r) < 32 || !VFPSmallRegisterBank() then
32                         S[d+r] = MemA[address, 4];
33                         address = address+4;
34                 else
35                     if (d+r) < 16 || !VFPSmallRegisterBank() then
36                         word1 = MemA[address, 4]; word2 = MemA[address+4, 4];
37                         // Combine the word-aligned words in the correct order for
38                         // current endianness.
39                         D[d+r] = if BigEndian(address, 8) then word1:word2 else word2:word1;
40                     elseif boolean UNKNOWN then
41                         - = MemA[address, 4]; - = MemA[address+4, 4];
42                         address = address+8;
43
44         // If the stack pointer is being updated a fault will be raised if
45         // the limit is violated
46         if wback then RSPCheck[n] = regval;

```

C2.4.64 FSTMDBX, FSTMIAX

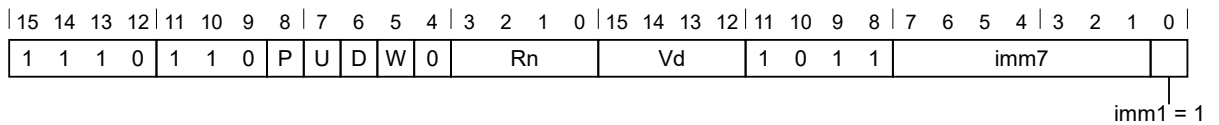
FSTMX (Decrement Before, Increment After). FSTMX (Decrement Before, Increment After) stores multiple extension registers to consecutive memory locations using an address from a general-purpose register.

Arm deprecates use of FSTMDBX and FSTMIAX, except for disassembly purposes, and reassembly of disassembled code.

This instruction is subject to stack limit checking.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE



Decrement Before variant

Applies when **P == 1** && **U == 0** && **W == 1**.

FSTMDBX{<c>}{<q>} <Rn>{!}, <dreglist>

Increment After variant

Applies when **P == 0** && **U == 1**.

FSTMIAX{<c>}{<q>} <Rn>{!}, <dreglist>

Decode for this encoding

```

1 if P == '0' && U == '0' && W == '0' then SEE "Related encodings";
2 if P == '1' && W == '0' then SEE VSTR;
3 CheckDecodeFaults(ExtType_MveOrFp);
4 if P == U && W == '1' then UNDEFINED;
5 // Remaining combinations are PUW = 010 (IA without !), 011 (IA with !), 101 (DB with !)
6 single_regs = FALSE; add = (U == '1'); wback = (W == '1'); imm8 = imm7 : imm1;
7 d = UInt(D:Vd); n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
8 regs = UInt(imm8) DIV 2;
9 if n == 15 then UNPREDICTABLE;
10 if regs == 0 || (d+regs) > 32 then UNPREDICTABLE;
11 if !HasArchVersion(Armv8pl) then
12     if VFPSmallRegisterBank() && (d+regs) > 16 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **regs == 0**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as a FSTMX with the same addressing mode but stores no registers.

CONSTRAINED UNPREDICTABLE behavior

If **(d+regs) > 32**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

- The memory locations specified by the instruction and the number of registers specified by the instruction if the register list had not gone out of range, become UNKNOWN. If the instruction specifies write-back, then that register becomes UNKNOWN. This behavior does not affect any other memory locations.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
!	Specifies base register write-back. Encoded in the "W" field as 1 if present, otherwise 0.
<dreglist>	Is the list of consecutively numbered 64-bit floating-point registers to be transferred. The first register in the list is encoded in "D:Vd", and "imm8" is set to twice the number of registers in the list plus one. The list must contain at least one register.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      address = if add then R[n]          else R[n]-imm32;
5      regval  = if add then R[n]+imm32 else R[n]-imm32;
6
7      // Determine if the stack pointer limit should be checked
8      if n == 13 && wback then
9          violatesLimit = ViolatesSPLim(LookUpSP(), regval);
10     else
11         violatesLimit = FALSE;
12
13     // Memory operation only performed if limit not violated
14     if !violatesLimit then
15         for r = 0 to regs-1
16             if single_regs then
17                 if (d+r) < 32 || !VFPSmallRegisterBank() then
18                     MemA[address, 4] = S[d+r];
19                     address = address+4;
20             else
21                 // Store as two word-aligned words in the correct order for current
22                 // endianness.
23                 if (d+r) < 16 || !VFPSmallRegisterBank() then
24                     bigEndian = BigEndian(address, 8);
25                     MemA[address, 4] = if bigEndian then D[d+r][63:32] else D[d+r][31:0];
26                     MemA[address+4, 4] = if bigEndian then D[d+r][31:0] else D[d+r][63:32];
27                 elsif boolean UNKNOWN then
28                     MemA[address, 4] = bits(32) UNKNOWN;
29                     MemA[address+4, 4] = bits(32) UNKNOWN;
30                     address = address+8;
31
32     // If the stack pointer is being updated a fault will be raised if
33     // the limit is violated
34     if wback then RSPCheck[n] = regval;

```

C2.4.65 ISB

Instruction Synchronization Barrier. Instruction Synchronization Barrier flushes the pipeline in the PE and is a context synchronization event.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	0	1	1	1	0	1	1	(1)	(1)	(1)	(1)	1	0	(0)	0	(1)	(1)	(1)	(1)	0	1	1	0	option				

T1 variant

ISB{<c>}{<q>} {<option>}

Decode for this encoding

```
1 // No additional decoding required
```

Assembler symbols for all encodings

- <c> See C1.2.5 Standard assembler syntax fields.
- <q> See C1.2.5 Standard assembler syntax fields.
- <option> Specifies an optional limitation on the barrier operation. Values are:
 - SY Full system barrier operation, encoded as option = 0b1111. Can be omitted.All other encodings of option are reserved. The corresponding instructions execute as full system barrier operations, but must not be relied upon by software.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   InstructionSynchronizationBarrier(option);
```

C2.4.66 IT

If-Then. If Then makes up to four following instructions (the IT block) conditional. The conditions for the instructions in the IT block can be the same, or some of them can be the inverse of others.

IT does not affect the condition code flags. Branches to any instruction in the IT block are not permitted, apart from those performed by exception returns.

16-bit instructions in the IT block, other than **CMP (register)**, **CMN (register)**, and **TST (register)**, do not set the condition code flags. The AL condition can be specified to get this changed behavior without conditional execution.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	firstcond				mask			

T1 variant

IT{<x>{<y>{<z>}}} {<q>} <cond>

Decode for this encoding

```
1 if mask == '0000' then SEE "Related encodings";
2 if !HaveMainExt() then UNDEFINED;
3 if firstcond == '1111' || (firstcond == '1110' && BitCount(mask) != 1) then UNPREDICTABLE;
4 if InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `firstcond == '1111' || (firstcond == '1110' && BitCount(mask) != 1)`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The '1111' condition is treated as being the same as the '1110' condition, meaning always, and the ITSTATE state machine is progressed in the same way as for any other cond_base value.

Assembler symbols for all encodings

<x>	The condition for the second instruction in the IT block. If omitted, the "mask" field is set to 0b1000. If present it is encoded in the "mask[3]" field: E NOT firstcond[0] T firstcond[0]
<y>	The condition for the third instruction in the IT block. If omitted and <x> is present, the "mask[2:0]" field is set to 0b100. If <y> is present it is encoded in the "mask[2]" field: E NOT firstcond[0] T firstcond[0]
<z>	The condition for the fourth instruction in the IT block. If omitted and <y> is present, the "mask[1:0]" field is set to 0b10. If <z> is present, the "mask[0]" field is set to 1, and it is encoded in the "mask[1]" field: E NOT firstcond[0] T firstcond[0]
<q>	See C1.2.5 Standard assembler syntax fields .
<cond>	The condition for the first instruction in the IT block, encoded in the "firstcond" field. See C1.3 Conditional execution for the range of conditions available, and the encodings.

Operation for all encodings

```
1 EncodingSpecificOperations();  
2 ITSTATE[7:0] = firstcond;mask;
```

C2.4.67 LCTP

Loop Clear with Tail Predication. Exits loop mode by invalidating LO_BRANCH_INFO and clears any tail predication being applied.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	(0)	(0)	1	1	1	1	1	1	0	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	1

T1: LCTP variant

LCTP<c><q>

Decode for this encoding

```
1 if !HaveLOBExt() then UNDEFINED;
2 if !HaveMve() then UNDEFINED;
3 HandleException(CheckCPEnabled(10));
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     ExecuteFPCheck();
5     FPSCR.LTPSIZE = 4[2:0]; // Disable loop predication
6     if LO_BRANCH_INFO.BF == '0' then
7         LO_BRANCH_INFO.VALID = '0';
```

C2.4.68 LDA

Load-Acquire Word. Load-Acquire Word loads a word from memory and writes it to a register. The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	1		Rn				Rt		(1)	(1)	(1)	(1)	1	0	1	0	(1)	(1)	(1)	(1)	

T1 variant

LDA{<c>}{<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n];
4     R[t] = MemO[address, 4];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.69 LDAB

Load-Acquire Byte. Load-Acquire Byte loads a byte from memory, zero-extends it to form a 32-bit word and writes it to a register. The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	1	Rn				Rt				(1) (1) (1) (1)				1 0 0 0				(1) (1) (1) (1)			

T1 variant

LDAB{<c>}{<q>} <Rt>, [*<Rn>*]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n];
4     R[t] = ZeroExtend(MemO[address, 1], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.70 LDAEX

Load-Acquire Exclusive Word. Load-Acquire Exclusive Word loads a word from memory, writes it to a register, and:

- If the address has the Shareable memory attribute, marks the physical address as exclusive access for the executing PE in a global monitor.
- Causes the executing PE to indicate an active exclusive access in the local monitor.

The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	1	Rn				Rt				(1) (1) (1) (1)				1	1	1	0	(1) (1) (1) (1)			

T1 variant

LDAEX{<c>}{<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n];
4   SetExclusiveMonitors(address, 4);
5   R[t] = MemO(address, 4);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.71 LDAEXB

Load-Acquire Exclusive Byte. Load-Acquire Exclusive Byte loads a byte from memory, zero-extends it to form a 32-bit word, writes it to a register, and:

- If the address has the Shareable memory attribute, marks the physical address as exclusive access for the executing PE in a global monitor.
- Causes the executing PE to indicate an active exclusive access in the local monitor.

The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	1	Rn				Rt				(1) (1) (1) (1)				1 1 0 0				(1) (1) (1) (1)			

T1 variant

LDAEXB{<c>} {<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n];
4   SetExclusiveMonitors(address, 1);
5   R[t] = ZeroExtend(MemO[address, 1], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.72 LDAEXH

Load-Acquire Exclusive Halfword. Load-Acquire Exclusive Halfword loads a halfword from memory, zero-extends it to form a 32-bit word, writes it to a register, and:

- If the address has the Shareable memory attribute, marks the physical address as exclusive access for the executing PE in a global monitor.
- Causes the executing PE to indicate an active exclusive access in the local monitor.

The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	1																				

T1 variant

LDAEXH{<c>} {<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n];
4   SetExclusiveMonitors(address, 2);
5   R[t] = ZeroExtend(MemO[address, 2], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.73 LDAH

Load-Acquire Halfword. Load-Acquire Halfword loads a halfword from memory, zero-extends it to form a 32-bit word and writes it to a register. The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	1	Rn				Rt				(1) (1) (1) (1)				1	0	0	1	(1) (1) (1) (1)			

T1 variant

LDAH{<c>}{<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n];
4     R[t] = ZeroExtend(MemO[address, 2], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.74 LDC, LDC2 (immediate)

Load Coprocessor (immediate). Load Coprocessor loads memory data from a sequence of consecutive memory addresses to a coprocessor. If no coprocessor can execute the instruction, a UsageFault exception is generated.

This is a generic coprocessor instruction. Some of the fields have no functionality defined by the architecture and are free for use by the coprocessor instruction set designer. These fields are the D bit, the CRd field, and in the Unindexed addressing mode only, the imm8 field.

Arm reserves coprocessors CP8 to CP15, and this manual defines the valid instructions when coproc is in this range.

This instruction is subject to stack limit checking.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	P	U	D	W	1	Rn				CRd				coproc				imm8							

Offset variant

Applies when **P == 1 && W == 0**.

LDC{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>{, #+/->}<imm>]

Post-indexed variant

Applies when **P == 0 && W == 1**.

LDC{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>], #+/->imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

LDC{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>, #+/->imm>]!

Unindexed variant

Applies when **P == 0 && U == 1 && W == 0**.

LDC{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>], <option>

Decode for this encoding

```

1 if Rn == '1111' then SEE "LDC (literal)";
2 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
3 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
4 if P == '0' && U == '0' && D == '1' && W == '0' then SEE "MRRC, MRRC2";
5 cp = UInt(coproc);
6 CheckCPDecodeFaults(cp);
7 if P == '0' && U == '0' && D == '0' && W == '0' then UNDEFINED;
8 if !HaveMainExt() then UNDEFINED;
9 n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
10 index = (P == '1'); add = (U == '1'); wback = (W == '1');
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	P	U	D	W	1	Rn				CRd				coproc				imm8							

Offset variant

Applies when **P == 1 && W == 0**.

LDC2{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>{, #+/-}<imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

LDC2{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>], #+/-<imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

LDC2{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>, #+/-<imm>]!

Unindexed variant

Applies when **P == 0 && U == 1 && W == 0**.

LDC2{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>], <option>

Decode for this encoding

```

1 if Rn == '1111' then SEE "LDC (literal)";
2 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
3 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
4 if P == '0' && U == '0' && D == '1' && W == '0' then SEE "MRRC, MRRC2";
5 cp = UInt(coproc);
6 CheckCPDecodeFaults(cp);
7 if P == '0' && U == '0' && D == '0' && W == '0' then UNDEFINED;
8 if !HaveMainExt() then UNDEFINED;
9 n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
10 index = (P == '1'); add = (U == '1'); wback = (W == '1');
```

Assembler symbols for all encodings

L	If specified, selects the D == 1 form of the encoding. If omitted, selects the D == 0 form.
<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoded in the "coproc" field. The generic coprocessor names are p0 to p15.
<CRd>	Is the coprocessor register to be transferred, encoded in the "CRd" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field. If the PC is used, see C2.4.75 LDC, LDC2 (literal) .
<option>	Is a coprocessor option, in the range 0 to 255 enclosed in { }, encoded in the "imm8" field.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	Is the immediate offset used for forming the address, a multiple of 4 in the range 0-1020, defaulting to 0 and encoded in the "imm8" field, as <imm>/4.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
```

```

3
4  thisInstr = ThisInstr();
5  if !Coprocc_Accepted(cp, thisInstr) then
6      GenerateCoprocc_Exception();
7  else
8      offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
9      address = if index then offset_addr else R[n];
10
11     // Determine if the stack pointer limit check should be performed
12     if wback && n == 13 then
13         violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
14     else
15         violatesLimit = FALSE;
16
17     // Memory operation only performed if limit not violated
18     if !violatesLimit then
19         repeat
20             Coproc_SendLoadedWord(MemA[address, 4], cp, thisInstr);
21             address = address + 4;
22         until Coproc_DoneLoading(cp, thisInstr);
23
24     // If the stack pointer is being updated a fault will be raised
25     // if the limit is violated
26     if wback then RSPCheck[n] = offset_addr;

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.75 LDC, LDC2 (literal)

Load Coprocessor (literal). Load Coprocessor loads memory data from a sequence of consecutive memory addresses to a coprocessor. If no coprocessor can execute the instruction, a UsageFault exception is generated.

This is a generic coprocessor instruction. The D bit and the CRd field have no functionality defined by the architecture and are free for use by the coprocessor instruction set designer.

Arm reserves coprocessors CP8 to CP15, and this manual defines the valid instructions when coproc is in this range.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	P	U	D	W	1	1	1	1	1	CRd				coproc				imm8							

T1 variant

LDC{L}{<c>}{<q>} <coproc>, <CRd>, <label>
LDC{L}{<c>}{<q>} <coproc>, <CRd>, [PC, #{+/-}<imm>]

Decode for this encoding

```
1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 if P == '0' && U == '0' && D == '1' && W == '0' then SEE "MRRC, MRRC2";
4 cp = UInt(coproc);
5 CheckCPDecodeFaults(cp);
6 if P == '0' && U == '0' && D == '0' && W == '0' then UNDEFINED;
7 if !HaveMainExt() then UNDEFINED;
8 index = (P == '1'); // Always TRUE in the T32 instruction set
9 add = (U == '1'); imm32 = ZeroExtend(imm8:'00', 32);
10 if W == '1' || P == '0' then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $W == '1' \parallel P == '0'$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes without write-back of the base address.
- The instruction executes as LDC with write-back to the PC.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	P	U	D	W	1	1	1	1	1	CRd				coproc				imm8							

T2 variant

LDC2{L}{<c>}{<q>} <coproc>, <CRd>, <label>
LDC2{L}{<c>}{<q>} <coproc>, <CRd>, [PC, #{+/-}<imm>]

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 if P == '0' && U == '0' && D == '1' && W == '0' then SEE "MRRC, MRRC2";
4 cp = UInt(coproc);
5 CheckCPDecodeFaults(cp);
6 if P == '0' && U == '0' && D == '0' && W == '0' then UNDEFINED;
7 if !HaveMainExt() then UNDEFINED;
8 index = (P == '1'); // Always TRUE in the T32 instruction set
9 add = (U == '1'); imm32 = ZeroExtend(imm8:'00', 32);
10 if W == '1' || P == '0' then UNPREDICTABLE;

```

CONSTRAINED UNPREDICTABLE behavior

If $W == '1' \mid \mid P == '0'$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes without write-back of the base address.
- The instruction executes as LDC with write-back to the PC.

Assembler symbols for all encodings

L	If specified, selects the $D == 1$ form of the encoding. If omitted, selects the $D == 0$ form.
<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoded in the "coproc" field. The generic coprocessor names are p0 to p15.
<CRd>	Is the coprocessor register to be transferred, encoded in the "CRd" field.
<label>	The label of the literal data item that is to be loaded into <Rt>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. Permitted values of the offset are multiples of 4 in the range -1020 to 1020. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE (encoded as $U == 1$). If the offset is negative, imm32 is equal to minus the offset and add == FALSE (encoded as $U == 0$).
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when $U = 0$ + when $U = 1$
<imm>	Is the immediate offset used for forming the address, a multiple of 4 in the range 0-1020, defaulting to 0 and encoded in the "imm8" field, as <imm>/4.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     thisInstr = ThisInstr();
5     if !Coproc_Accepted(cp, thisInstr) then
6         GenerateCoproprocessorException();
7     else
8         offset_addr = if add then (Align(PC,4) + imm32) else (Align(PC,4) - imm32);
9         address = if index then offset_addr else Align(PC,4);
10        repeat
11            Coproc_SendLoadedWord(MemA[address, 4], cp, thisInstr); address = address + 4;
12        until Coproc_DoneLoading(cp, thisInstr);

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.76 LDM, LDMIA, LDMFD

Load Multiple (Increment After, Full Descending). Load Multiple loads multiple registers from consecutive memory locations using an address from a base register. The sequential memory locations start at this address, and the address just above the last of those locations can optionally be written back to the base register.

The registers loaded can include the PC. If they do, the word loaded for the PC is treated as a branch address, a function return value, or an exception return value. Bit[0] of the address in the PC complies with the Arm architecture interworking rules for switching between the A32 and T32 instruction sets. However, Armv8-M only supports the T32 instruction set, so bit[0] must be 1. If bit[0] of the target address is 0, and the target address is not FNC_RETURN or EXC_RETURN, the PE takes an INVSTATE UsageFault exception on the instruction at the target address.

This instruction is used by the alias [POP \(multiple registers\)](#).

This instruction is a BTI setting instruction when all of the following are true:

- The PACBTI Extension is implemented.
- BTI is enabled for the destination security and privilege state.
- The operation is not a FNC_RETURN or an EXC_RETURN.
- PC is used.
- The base register is either:
 - Not SP.
 - The SP and a write-back does not occur.

This instruction is interrupt-continuable.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	Rn			register_list							

T1 variant

```
LDM{IA}{<c>}{<q>} <Rn>{!}, <registers>
// Preferred syntax
LDMFD{<c>}{<q>} <Rn>{!}, <registers>
// Alternate syntax, Full Descending stack
```

Decode for this encoding

```
1 n = UInt(Rn); registers = '0000000':register_list; wback = (registers[n] == '0');
2 if BitCount(registers) < 1 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) < 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

- The instruction executes as LDM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15. If the instruction specifies write-back, the modification to the base address on write-back might differ from the number of registers loaded.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	0	W	1	Rn				P	M	(0)	register_list												

T2 variant

```
LDM{IA}{<c>}.W <Rn>{!}, <registers>
// Preferred syntax
// if <Rn>, '!' and <registers> can be represented in T1
LDMFD{<c>}.W <Rn>{!}, <registers>
// Alternate syntax
// Full Descending stack, if <Rn>, '!' and
// <registers> can be represented in T1
LDM{IA}{<c>}{<q>} <Rn>{!}, <registers>
// Preferred syntax
LDMFD{<c>}{<q>} <Rn>{!}, <registers>
// Alternate syntax, Full Descending stack
```

Decode for this encoding

```
1 if Rn == '1111' && HasArchVersion(Armv8p1) then SEE "CLRM";
2 if !HaveMainExt() then UNDEFINED;
3 n = UInt(Rn); registers = P:M:'0':register_list; wback = (W == '1');
4 if n == 15 || BitCount(registers) < 2 || (P == '1' && M == '1') then UNPREDICTABLE;
5 if registers[15] == '1' && !LastInITBlock() && !LastInITBlock() then UNPREDICTABLE;
6 if wback && registers[n] == '1' then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) < 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as LDM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15. If the instruction specifies write-back, the modification to the base address on write-back might differ from the number of registers loaded.

CONSTRAINED UNPREDICTABLE behavior

If `wback && registers<n> == '1'`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs all of the loads using the specified addressing mode and the content of the register that is written back is UNKNOWN. In addition, if an exception occurs during such an instruction, the base address might be corrupted so that the instruction cannot be repeated.

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) == 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction loads a single register using the specified addressing modes.
- The instruction executes as LDM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15.

CONSTRAINED UNPREDICTABLE behavior

If $P == '1'$ && $M == '1'$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction loads the register list and either R14 or R15, both R14 and R15, or neither of these registers.

T3

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	0	P	register_list							

T3 variant

LDM{<c>}{<q>} SP!, <registers>

Decode for this encoding

```

1 n = 13; wback = TRUE;
2 registers = P:'000000':register_list;
3 if BitCount(registers) < 1 then UNPREDICTABLE;
4 if registers[15] == '1' && InitBlock() && !LastInitBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $\text{BitCount}(\text{registers}) < 1$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as LDM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15. If the instruction specifies write-back, the modification to the base address on write-back might differ from the number of registers loaded.

Assembler symbols for all encodings

IA	Is an optional suffix for the Increment After form.
<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
!	For encoding T1: the address adjusted by the size of the data loaded is written back to the base register. It is omitted if <Rn> is included in <registers>, otherwise it must be present. For encoding T2: the address adjusted by the size of the data loaded is written back to the base register. If specified, it is encoded in the "W" field as 1, otherwise this field defaults to 0.

<registers> For encoding T1: is a list of one or more registers to be loaded, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R7, encoded in the "register_list" field. For encoding T2: is a list of one or more registers to be loaded, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R12, encoded in the "register_list" field, and can optionally contain one of the LR or the PC. If the LR is in the list, the "M" field is set to 1, otherwise it defaults to 0. If the PC is in the list, the "P" field is set to 1, otherwise it defaults to 0. If the PC is in the list:

- The LR must not be in the list.
- The instruction must be either outside any IT block, or the last instruction in an IT block.

For encoding T3: is a list of one or more registers to be loaded, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R7, encoded in the "register_list" field, and can optionally include the PC. If the PC is in the list, the "P" field is set to 1, otherwise this field defaults to 0. If the PC is in the list, the instruction must be either outside any IT block, or the last instruction in an IT block.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      address = R[n];
4      if n == 13 && wback then
5          // If memory operation is not performed as a result of a stack limit violation,
6          // and the write-back of the SP itself does not raise a stack limit violation, it
7          // is "IMPLEMENTATION_DEFINED" whether a SPLIM exception is raised.
8          // Arm recommends that any instruction which discards a memory access as
9          // a result of a stack limit violation, and where the write-back of the SP itself
10         // does not raise a stack limit violation, generates an SPLIM exception.
11         if boolean IMPLEMENTATION_DEFINED "SPLIM exception on invalid memory access" then
12             if ViolatesSPLim(LookUpSP(), address) then
13                 if HaveMainExt() then
14                     UFSR.STKOF = '1';
15                     // If the Main Extension is not implemented the fault always escalates to
16                     // a HardFault
17                     excInfo = CreateException(UsageFault);
18                     HandleException(excInfo);
19             applylimit = TRUE;
20         else
21             applylimit = FALSE;
22
23         for i = 0 to 14
24             // If R[n] is the SP, memory operation only performed if limit not violated
25             if registers[i] == '1' && !(applylimit && ViolatesSPLim(LookUpSP(), address)) then
26                 if i != n then
27                     R[i] = MemA[address, 4];
28                 else
29                     newBaseVal = MemA[address, 4];
30                     address = address + 4;
31             setPC = registers[15] == '1' && !(applylimit && ViolatesSPLim(LookUpSP(), address));
32             if setPC then
33                 newPCVal = MemA[address, 4];
34
35             // If the register list contains the register that holds the base address it
36             // must be updated after all memory reads have been performed. This prevents
37             // the base address being overwritten if one of the memory reads generates a
38             // fault.
39             if registers[n] == '1' then
40                 wback = TRUE;
41             else
42                 newBaseVal = R[n] + 4*BitCount(registers);
43             // If the PC is in the register list update that now, which might raise a fault
44             // Likewise if R[n] is the SP writing back might raise a fault due to SP limit violation
45             if setPC then
46                 // If BTI is enabled, this branch sets the EPSR.B flag to ensure that the target
47                 // instruction is BTI clearing.
48                 setBti = BTIEnabled() && (n != 13 || (n == 13 && !wback));
49                 LoadWritePC(newPCVal, n, newBaseVal, wback, FALSE, setBti);

```

```
50     elsif wback then  
51         RSPCheck[n] = newBaseVal;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to values being loaded into R15, but does apply to all values being loaded into any other specified register even if R15 is being used.

The timing invariance does not apply to the addresses being accessed.

C2.4.77 LDMDB, LDMEA

Load Multiple Decrement Before (Empty Ascending). Load Multiple Decrement Before (Load Multiple Empty Ascending) loads multiple registers from sequential memory locations using an address from a base register. The sequential memory locations end just below this address, and the address of the first of those locations can optionally be written back to the base register.

The registers loaded can include the PC. If they do, the word loaded for the PC is treated as a branch address or an exception return value. Bit[0] complies with the Arm architecture interworking rules for switching between the A32 and T32 instruction sets. However, Armv8-M only supports the T32 instruction set, so bit[0] must be 1. If bit[0] is 0 the PE takes an INVSTATE UsageFault exception on the instruction at the target address.

This instruction is a BTI setting instruction when all of the following are true:

- The PACBTI Extension is implemented.
- BTI is enabled for the destination security and privilege state.
- The operation is not a FNC_RETURN or an EXC_RETURN.
- PC is used.
- The base register is either:
 - Not SP.
 - The SP and a write-back does not occur.

This instruction is interrupt-continuable.

This instruction is subject to stack limit checking.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	1	0	0	W	1				Rn	P	M	(0)													register_list

T1 variant

```
LDMDB{<c>}{<q>} <Rn>{!}, <registers>
// Preferred syntax
LDMEA{<c>}{<q>} <Rn>{!}, <registers>
// Alternate syntax, Empty Ascending stack
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); registers = P:M:'0':register_list; wback = (W == '1');
3 if n == 15 || BitCount(registers) < 2 || (P == '1' && M == '1') then UNPREDICTABLE;
4 if registers[15] == '1' && InITBlock() && !LastInITBlock() then UNPREDICTABLE;
5 if wback && registers[n] == '1' then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If wback && registers<n> == '1', then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

- The instruction performs all of the loads using the specified addressing mode and the content of the register that is written back is UNKNOWN. In addition, if an exception occurs during such an instruction, the base address might be corrupted so that the instruction cannot be repeated.

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) < 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as LDM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15. If the instruction specifies write-back, the modification to the base address on write-back might differ from the number of registers loaded.

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) == 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction loads a single register using the specified addressing modes.
- The instruction executes as LDM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15.

CONSTRAINED UNPREDICTABLE behavior

If `P == '1' && M == '1'`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction loads the register list and either R14 or R15, both R14 and R15, or neither of these registers.

Assembler symbols for all encodings

<code><c></code>	See C1.2.5 Standard assembler syntax fields .
<code><q></code>	See C1.2.5 Standard assembler syntax fields .
<code><Rn></code>	Is the general-purpose base register, encoded in the "Rn" field.
<code>!</code>	The address adjusted by the size of the data loaded is written back to the base register. If specified, it is encoded in the "W" field as 1, otherwise this field defaults to 0.
<code><registers></code>	Is a list of one or more registers to be loaded, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R12, encoded in the "register_list" field, and can optionally contain one of the LR or the PC. If the LR is in the list, the "M" field is set to 1, otherwise it defaults to 0. If the PC is in the list, the "P" field is set to 1, otherwise it defaults to 0. If the PC is in the list: - The LR must not be in the list. - The instruction must be either outside any IT block, or the last instruction in an IT block.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n] - 4*BitCount(registers);
4
5     // Determine if the stack pointer limit should be checked
6     if n == 13 && wback && registers[n] == '0' then
7         violatesLimit = ViolatesSPLim(LookUpSP(), address);
8     else

```

```

9      violatesLimit = FALSE;
10
11     for i = 0 to 15
12         // Memory operation only performed if limit not violated
13         if registers[i] == '1' && !violatesLimit then
14             data = MemA[address, 4];
15             address = address + 4;
16             if i == 15 then
17                 newPCVal = data;
18                 setPC = TRUE;
19             elseif i == n then
20                 newBaseVal = data;
21             else
22                 R[i] = data;
23
24         // If the register list contains the register that holds the base address it
25         // must be updated after all memory reads have been performed. This prevents
26         // the base address being overwritten if one of the memory reads generates a
27         // fault.
28         if registers[n] == '1' then
29             wback = TRUE;
30         else
31             newBaseVal = R[n] - 4*BitCount(registers);
32         // If the PC is in the register list update that now, which may raise a fault
33         if setPC then
34             // If BTI is enabled, this branch sets the EPSR.B flag to ensure that the target
35             // instruction is BTI clearing.
36             setBti = BTIEnabled() && (n != 13 || (n == 13 && !wback));
37             LoadWritePC(newPCVal, n, newBaseVal, wback, TRUE, setBti);
38         elseif wback then
39             RSPCheck[n] = newBaseVal;

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to values being loaded into R15, but does apply to all values being loaded into any other specified register even if R15 is being used.

The timing invariance does not apply to the addresses being accessed.

C2.4.78 LDR (immediate)

Load Register (immediate). Load Register (immediate) calculates an address from a base register value and an immediate offset, loads a word from memory, and writes it to a register. It can use offset, post-indexed, or pre-indexed addressing.

The register loaded can be the PC. If it is, the word loaded for the PC is treated as a branch address or an exception return value. Bit[0] complies with the Arm architecture interworking rules for switching between the A32 and T32 instruction sets. However, Armv8-M only supports the T32 instruction set, so bit[0] must be 1. If bit[0] is 0 the PE takes an INVSTATE UsageFault exception on the instruction at the target address.

This instruction is used by the alias [POP \(single register\)](#).

This instruction is a BTI setting instruction when all of the following are true:

- The PACBTI Extension is implemented.
- BTI is enabled for the destination security and privilege state.
- The operation is not a FNC_RETURN or an EXC_RETURN.
- PC is used.
- The base register is either:
 - Not SP.
 - The SP and a write-back does not occur.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	1	imm5					Rn			Rt		

T1 variant

LDR{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm5:'00', 32);
2 index = TRUE; add = TRUE; wback = FALSE;
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	1	Rt			imm8							

T2 variant

LDR{<c>}{<q>} <Rt>, [SP{, #<+><imm>}]

Decode for this encoding

```
1 t = UInt(Rt); n = 13; imm32 = ZeroExtend(imm8:'00', 32);
2 index = TRUE; add = TRUE; wback = FALSE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	1	1	0	1	Rn			Rt			imm12													

T3 variant

```
LDR{<c>}.W <Rt>, [<Rn> {, #{+}<imm>}]
// <Rt>, <Rn>, <imm> can be represented in T1 or T2
LDR{<c>}{<q>} <Rt>, [<Rn> {, #{+}<imm>}]
```

Decode for this encoding

```
1 if Rn == '1111' then SEE "LDR (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm12, 32); index = TRUE; add = TRUE;
4 wback = FALSE; if t == 15 && InITBlock() && !LastInITBlock() then UNPREDICTABLE;
```

T4

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	1	0	1	Rn			Rt			1	P	U	W	imm8									

Offset variant

Applies when **P == 1 && U == 0 && W == 0**.

```
LDR{<c>}{<q>} <Rt>, [<Rn> {, #-<imm>}]
```

Post-indexed variant

Applies when **P == 0 && W == 1**.

```
LDR{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>
```

Pre-indexed variant

Applies when **P == 1 && W == 1**.

```
LDR{<c>}{<q>} <Rt>, [<Rn>, #{+/-}<imm>]!
```

Decode for this encoding

```
1 if Rn == '1111' then SEE "LDR (literal)";
2 if P == '1' && U == '1' && W == '0' then SEE LDRT;
3 if P == '0' && W == '0' then UNDEFINED;
4 if !HaveMainExt() then UNDEFINED;
5 t = UInt(Rt); n = UInt(Rn);
6 imm32 = ZeroExtend(imm8, 32); index = (P == '1'); add = (U == '1'); wback = (W == '1');
7 if (wback && n == t) || (t == 15 && InITBlock() && !LastInITBlock()) then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **wback && n == t**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

- The instruction performs all of the loads using the specified addressing mode and the content of the register that is written back is UNKNOWN. In addition, if an exception occurs during such an instruction, the base address might be corrupted so that the instruction cannot be repeated.

Alias conditions

Alias	preferred when
POP (single register)	Rn == `1101` && P == `0` && U == `1` && W == `1` && imm8 == `00000100`

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	For encoding T1 and T2: is the general-purpose register to be transferred, encoded in the "Rt" field. For encoding T3: is the general-purpose register to be transferred, encoded in the "Rt" field. The SP can be used. The PC can be used, provided the instruction is either outside an IT block or the last instruction of an IT block. If the PC is used, the instruction branches to the address (data) loaded to the PC. For encoding T4: is the general-purpose register to be transferred, encoded in the "Rt" field. The PC can be used, provided the instruction is either outside an IT block or the last instruction of an IT block. If the PC is used, the instruction branches to the address (data) loaded to the PC.
<Rn>	For encoding T1: is the general-purpose base register, encoded in the "Rn" field. For encoding T3 and T4: is the general-purpose base register, encoded in the "Rn" field. For PC use see C2.4.79 LDR (literal) .
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
+	Specifies the offset is added to the base register.
<imm>	For the post-indexed or pre-indexed variant: is an 8-bit unsigned immediate byte offset, in the range 0 to 255, encoded in the "imm8" field. For encoding T1: is the optional positive unsigned immediate byte offset, a multiple of 4 in the range 0 to 124, defaulting to 0 and encoded in the "imm5" field as <imm>/4. For encoding T2: is the optional positive unsigned immediate byte offset, a multiple of 4, in the range 0 to 1020, defaulting to 0 and encoded in the "imm8" field as <imm>/4. For encoding T3: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T4: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
4      address = if index then offset_addr else R[n];
5
6      // Determine if the stack pointer limit should be checked
7      if n == 13 && wback then
8          violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
9      else
10         violatesLimit = FALSE;
11     // Memory operation only performed if limit not violated
12     if !violatesLimit then

```

```

13     data = MemU[address, 4];
14
15     // If the stack pointer is being updated a fault will be raised if
16     // the limit is violated
17     if t == 15 then
18         if address[1:0] == '00' then
19             // If BTI is enabled, this branch sets the EPSR.B flag to ensure that the target
20             // instruction is BTI clearing.
21             setBti = BTIEnabled() && (n != 13 || (n == 13 && !wback));
22             LoadWritePC(data, n, offset_addr, wback, TRUE, setBti);
23         else
24             UNPREDICTABLE;
25     else
26         if wback then RSPCheck[n] = offset_addr;
27         R[t] = data;

```

CONSTRAINED UNPREDICTABLE behavior

If `t == 15 && address<1:0> != '00'`, then one of the following behaviors must occur:

- The instruction executes as described, with no change to its behavior and no additional side effects.
- The instruction generates an UNALIGNED UsageFault.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1, and R15 is not being used as the destination register:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.79 LDR (literal)

Load Register (literal). Load Register (literal) calculates an address from the PC value and an immediate offset, loads a word from memory, and writes it to a register.

The register loaded can be the PC. If it is, the word loaded for the PC is treated as a branch address or an exception return value. Bit[0] complies with the Arm architecture interworking rules for switching between the A32 and T32 instruction sets. However, Armv8-M only supports the T32 instruction set, so bit[0] must be 1. If bit[0] is 0 the PE takes an INVSTATE UsageFault exception on the instruction at the target address.

If the PACBTI Extension is implemented, BTI is enabled for the destination security and privilege state, the operation is not a FNC_RETURN or an EXC_RETURN, and the destination register is PC, then this instruction is a BTI setting instruction, and when this instruction is executed it will set EPSR.B to one.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	Rt			imm8							

T1 variant

```
LDR{<c>}{<q>} <Rt>, <label>
// Normal form
```

Decode for this encoding

```
1 t = UInt(Rt); imm32 = ZeroExtend(imm8:'00', 32); add = TRUE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	U	1	0	1	1	1	1	1	Rt			imm12												

T2 variant

```
LDR{<c>}.W <Rt>, <label>
// Preferred syntax, and <Rt>, <label> can be represented in T1
LDR{<c>}{<q>} <Rt>, <label>
// Preferred syntax
LDR{<c>}{<q>} <Rt>, [PC, #{+/-}<imm>]
// Alternative syntax
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 t = UInt(Rt); imm32 = ZeroExtend(imm12, 32); add = (U == '1');
3 if t == 15 && InITBlock() && !LastInITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> For encoding T1: is the general-purpose register to be transferred, encoded in the "Rt" field.

	For encoding T2: is the general-purpose register to be transferred, encoded in the "Rt" field. The SP can be used. The PC can be used, provided the instruction is either outside an IT block or the last instruction of an IT block. If the PC is used, the instruction branches to the address (data) loaded to the PC.
<label>	For encoding T1: the label of the literal data item that is to be loaded into <Rt>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. Permitted values of the offset are Multiples of four in the range 0 to 1020. For encoding T2: the label of the literal data item that is to be loaded into <Rt>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. Permitted values of the offset are -4095 to 4095. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE, encoded as U == 1. If the offset is negative, imm32 is equal to minus the offset and add == FALSE, encoded as U == 0.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	Is a 12-bit unsigned immediate byte offset, in the range 0 to 4095, encoded in the "imm12" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      base = Align(PC, 4);
4      address = if add then (base + imm32) else (base - imm32);
5      data = MemU[address, 4];
6      if t == 15 then
7          if address[1:0] == '00' then
8              // If BTI is enabled, this branch sets the EPSR.B flag to ensure that the target
9              // instruction is BTI clearing.
10             LoadWritePC(data, 0, Zeros(32), FALSE, FALSE, BTIEnabled());
11         else
12             UNPREDICTABLE;
13     else
14         R[t] = data;
```

CONSTRAINED UNPREDICTABLE behavior

If `t == 15` && `address<1:0> != '00'`, then one of the following behaviors must occur:

- The instruction executes as described, with no change to its behavior and no additional side effects.
- The instruction generates an UNALIGNED UsageFault.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1, and R15 is not being used as the destination register:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.80 LDR (register)

Load Register (register). Load Register (register) calculates an address from a base register value and an offset register value, loads a word from memory, and writes it to a register. The offset register value can be shifted left by 0, 1, 2, or 3 bits.

The register loaded can be the PC. If it is, the word loaded for the PC is treated as a branch address or an exception return value. Bit[0] complies with the Arm architecture interworking rules for switching between the A32 and T32 instruction sets. However, Armv8-M only supports the T32 instruction set, so bit[0] must be 1. If bit[0] is 0 the PE takes an INVSTATE UsageFault exception on the instruction at the target address.

If the PACBTI Extension is implemented, BTI is enabled for the destination security and privilege state, the operation is not a FNC_RETURN or an EXC_RETURN, and the destination register is PC, then this instruction is a BTI setting instruction, and when this instruction is executed it will set EPSR.B to one.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	0	0	Rm			Rn			Rt		

T1 variant

LDR{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
2 index = TRUE; add = TRUE; wback = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	1	1	0	0	0	0	1	0	1	Rn				Rt				0	0	0	0	0	0	imm2				Rm			

T2 variant

LDR{<c>}.W <Rt>, [<Rn>, {+}<Rm>]
// <Rt>, <Rn>, <Rm> can be represented in T1
LDR{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>{, LSL #<imm>}]

Decode for this encoding

```
1 if Rn == '1111' then SEE "LDR (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
4 index = TRUE; add = TRUE; wback = FALSE;
5 (shift_t, shift_n) = (SRTYPE_LSL, UInt(imm2));
6 if m IN {13,15} then UNPREDICTABLE;
7 if t == 15 && InITBlock() && !LastInITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).

<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	For encoding T1: is the general-purpose register to be transferred, encoded in the "Rt" field. For encoding T2: is the general-purpose register to be transferred, encoded in the "Rt" field. The SP can be used. The PC can be used, provided the instruction is either outside an IT block or the last instruction of an IT block. If the PC is used, the instruction branches to the address (data) loaded to the PC.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register is added to the base register.
<Rm>	Is the general-purpose index register, encoded in the "Rm" field.
<imm>	If present, the size of the left shift to apply to the value from <Rm>, in the range 1-3. <imm> is encoded in imm2. If absent, no shift is specified and imm2 is encoded as 0b00.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      offset = Shift(R[m], shift_t, shift_n, APSR.C);
4      offset_addr = if add then (R[n] + offset) else (R[n] - offset);
5      address = if index then offset_addr else R[n];
6
7      // Determine if the stack pointer limit should be checked
8      if n == 13 && wback then
9          violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
10     else
11         violatesLimit = FALSE;
12     // Memory operation only performed if limit not violated
13     if !violatesLimit then
14         data = MemU[address, 4];
15
16     // If the stack pointer is being updated a fault will be raised if
17     // the limit is violated
18     if t == 15 then
19         if address[1:0] == '00' then
20             // If BTI is enabled, this branch sets the EPSR.B flag to ensure that the target
21             // instruction is BTI clearing.
22             LoadWritePC(data, n, offset_addr, wback, TRUE, BTIEnabled());
23         else
24             UNPREDICTABLE;
25     else
26         if wback then RSPCheck[n] = offset_addr;
27         R[t] = data;

```

CONSTRAINED UNPREDICTABLE behavior

If `t == 15 && address<1:0> != '00'`, then one of the following behaviors must occur:

- The instruction executes as described, with no change to its behavior and no additional side effects.
- The instruction generates an UNALIGNED UsageFault.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1, and R15 is not being used as the destination register:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.81 LDRB (immediate)

Load Register Byte (immediate). Load Register Byte (immediate) calculates an address from a base register value and an immediate offset, loads a byte from memory, zero-extends it to form a 32-bit word, and writes it to a register. It can use offset, post-indexed, or pre-indexed addressing.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	1	imm5					Rn			Rt		

T1 variant

LDRB{<c>}{<q>} <Rt>, [<Rn> {, #(<+>)<imm>}]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm5, 32);
2 index = TRUE; add = TRUE; wback = FALSE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	1	0	0	1	Rn				Rt				imm12											

T2 variant

LDRB{<c>}.W <Rt>, [<Rn> {, #(<+>)<imm>}]
 // <Rt>, <Rn>, <imm> can be represented in T1
 LDRB{<c>}{<q>} <Rt>, [<Rn> {, #(<+>)<imm>}]

Decode for this encoding

```
1 if Rt == '1111' then SEE "PLD (immediate)";
2 if Rn == '1111' then SEE "LDRB (literal)";
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm12, 32);
5 index = TRUE; add = TRUE; wback = FALSE;
6 if t == 13 then UNPREDICTABLE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	0	1	Rn				Rt				1	P	U	W	imm8							

Offset variant

Applies when Rt != 1111 && P == 1 && U == 0 && W == 0.

LDRB{<c>}{<q>} <Rt>, [<Rn> {, #-<imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

`LDRB{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>`

Pre-indexed variant

Applies when **P == 1 && W == 1**.

`LDRB{<c>}{<q>} <Rt>, [<Rn>, #{+/-}<imm>]!`

Decode for this encoding

```
1 if Rt == '1111' && P == '1' && U == '0' && W == '0' then SEE "PLD (immediate)";
2 if Rn == '1111' then SEE "LDRB (literal)";
3 if P == '1' && U == '1' && W == '0' then SEE LDRBT;
4 if P == '0' && W == '0' then UNDEFINED;
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm8, 32);
7 index = (P == '1'); add = (U == '1'); wback = (W == '1');
8 if t == 13 || (wback && n == t) then UNPREDICTABLE;
9 if t == 15 && W == '1' then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `wback && n == t`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs all of the loads using the specified addressing mode and the content of the register that is written back is UNKNOWN. In addition, if an exception occurs during such an instruction, the base address might be corrupted so that the instruction cannot be repeated.

Assembler symbols for all encodings

<code><c></code>	See C1.2.5 Standard assembler syntax fields .
<code><q></code>	See C1.2.5 Standard assembler syntax fields .
<code><Rt></code>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<code><Rn></code>	For encoding T1: is the general-purpose base register, encoded in the "Rn" field. For encoding T2 and T3: is the general-purpose base register, encoded in the "Rn" field. For PC use see C2.4.82 LDRB (literal) .
<code>+/-</code>	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when <code>U = 0</code> + when <code>U = 1</code>
<code>+</code>	Specifies the offset is added to the base register.
<code><imm></code>	For the post-indexed or pre-indexed variant: is an 8-bit unsigned immediate byte offset, in the range 0 to 255, encoded in the "imm8" field. For encoding T1: is an optional 5-bit unsigned immediate byte offset, in the range 0 to 31, defaulting to 0 and encoded in the "imm5" field. For encoding T2: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T3: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
```



```
3  offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
4  address = if index then offset_addr else R[n];
5
6  // Determine if the stack pointer limit should be checked
7  if n == 13 && wback then
8      violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
9  else
10     violatesLimit = FALSE;
11
12 // Memory operation only performed if limit not violated
13 if !violatesLimit then
14     R[t] = ZeroExtend(MemU[address, 1], 32);
15
16 // If the stack pointer is being updated a fault will be raised if
17 // the limit is violated
18 if wback then RSPCheck[n] = offset_addr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1, and R15 is not being used as the destination register:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.82 LDRB (literal)

Load Register Byte (literal). Load Register Byte (literal) calculates an address from the PC value and an immediate offset, loads a byte from memory, zero-extends it to form a 32-bit word, and writes it to a register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	U	0	0	1	1	1	1	1	Rt				imm12											

T1 variant

```
LDRB{<c>}{<q>} <Rt>, <label>
// Preferred syntax
LDRB{<c>}{<q>} <Rt>, [PC, #{+/-}<imm>]
// Alternative syntax
```

Decode for this encoding

```
1 if Rt == '1111' then SEE "PLD (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); imm32 = ZeroExtend(imm12, 32); add = (U == '1');
4 if t == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<label>	The label of the literal data item that is to be loaded into <Rt>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. Permitted values of the offset are -4095 to 4095. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE, encoded as U == 1. If the offset is negative, imm32 is equal to minus the offset and add == FALSE, encoded as U == 0.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	Is a 12-bit unsigned immediate byte offset, in the range 0 to 4095, encoded in the "imm12" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   base = Align(PC, 4);
4   address = if add then (base + imm32) else (base - imm32);
5   R[t] = ZeroExtend(MemU[address, 1], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.

- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.83 LDRB (register)

Load Register Byte (register). Load Register Byte (register) calculates an address from a base register value and an offset register value, loads a byte from memory, zero-extends it to form a 32-bit word, and writes it to a register. The offset register value can be shifted left by 0, 1, 2, or 3 bits.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	1	0	Rm			Rn			Rt		

T1 variant

LDRB{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
2 index = TRUE; add = TRUE; wback = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	0	0	0	0	0	1	Rn				Rt				0	0	0	0	0	0	imm2			Rm			

T2 variant

LDRB{<c>}.W <Rt>, [<Rn>, {+}<Rm>]
// <Rt>, <Rn>, <Rm> can be represented in T1
LDRB{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>{, LSL #<imm>}]

Decode for this encoding

```
1 if Rt == '1111' then SEE "PLD (register)";
2 if Rn == '1111' then SEE "LDRB (literal)";
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
5 index = TRUE; add = TRUE; wback = FALSE;
6 (shift_t, shift_n) = (SRTYPE_LSL, UInt(imm2));
7 if t == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register is added to the base register.
<Rm>	Is the general-purpose index register, encoded in the "Rm" field.
<imm>	If present, the size of the left shift to apply to the value from <Rm>, in the range 1-3. <imm> is encoded in imm2. If absent, no shift is specified and imm2 is encoded as 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```

2      EncodingSpecificOperations();
3      offset = Shift(R[m], shift_t, shift_n, APSR.C);
4      offset_addr = if add then (R[n] + offset) else (R[n] - offset);
5      address = if index then offset_addr else R[n];
6      R[t] = ZeroExtend(MemU[address, 1], 32);

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.84 LDRBT

Load Register Byte Unprivileged. Load Register Byte Unprivileged calculates an address from a base register value and an immediate offset, loads a byte from memory, zero-extends it to form a 32-bit word, and writes it to a register.

When privileged software uses an [LDRBT](#) instruction, the memory access is restricted as if the software was unprivileged.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	0	1	Rn				Rt				1	1	1	0	imm8							

T1 variant

LDRBT{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```

1 if Rn == '1111' then SEE "LDRB (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); postindex = FALSE; add = TRUE;
4 register_form = FALSE; imm32 = ZeroExtend(imm8, 32);
5 if t IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the offset is added to the base register.
<imm>	Is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n] + imm32;
4     R[t] = ZeroExtend(MemU_unpriv[address, 1], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.85 LDRD (immediate)

Load Register Dual (immediate). Load Register Dual (immediate) calculates an address from a base register value and an immediate offset, loads two words from memory, and writes them to two registers. It can use offset, post-indexed, or pre-indexed addressing.

This instruction is subject to stack limit checking.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	P	U	1	W	1	Rn				Rt				Rt2				imm8							

Offset variant

Applies when **P == 1** && **W == 0**.

LDRD{<c>}{<q>} <Rt>, <Rt2>, [<Rn> {, # {+/-}<imm>}]

Post-indexed variant

Applies when **P == 0** && **W == 1**.

LDRD{<c>}{<q>} <Rt>, <Rt2>, [<Rn>], # {+/-}<imm>

Pre-indexed variant

Applies when **P == 1** && **W == 1**.

LDRD{<c>}{<q>} <Rt>, <Rt2>, [<Rn>, # {+/-}<imm>]!

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 if Rn == '1111' then SEE "LDRD (literal)";
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); t2 = UInt(Rt2); n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
5 index = (P == '1'); add = (U == '1'); wback = (W == '1');
6 if wback && (n == t || n == t2) then UNPREDICTABLE;
7 if t IN {13,15} || t2 IN {13,15} || t == t2 then UNPREDICTABLE;

```

CONSTRAINED UNPREDICTABLE behavior

If wback && (n == t || n == t2), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs all of the loads using the specified addressing mode and the content of the register that is written back is UNKNOWN. In addition, if an exception occurs during such an instruction, the base address might be corrupted so that the instruction cannot be repeated.

CONSTRAINED UNPREDICTABLE behavior

If t == t2, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

- The load instruction executes but the destination register takes an UNKNOWN value.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the first general-purpose register to be transferred, encoded in the "Rt" field.
<Rt2>	Is the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field. For PC use see C2.4.86 LDRD (literal) .
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	For the offset variant: is the optional unsigned immediate byte offset, a multiple of 4, in the range 0 to 1020, defaulting to 0 and encoded in the "imm8" field as <imm>/4. For the post-indexed and pre-indexed variant: is the unsigned immediate byte offset, a multiple of 4, in the range 0 to 1020, encoded in the "imm8" field as <imm>/4.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
4      address = if index then offset_addr else R[n];
5
6      // Determine if the stack pointer limit should be checked
7      if n == 13 && wback then
8          violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
9      else
10         violatesLimit = FALSE;
11         // Memory operation only performed if limit not violated
12         if !violatesLimit then
13             R[t] = MemA[address, 4];
14             R[t2] = MemA[address+4, 4];
15
16         // If the stack pointer is being updated a fault will be raised if
17         // the limit is violated
18         if wback then RSPCheck[n] = offset_addr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.86 LDRD (literal)

Load Register Dual (literal). Load Register Dual (literal) calculates an address from the PC value and an immediate offset, loads two words from memory, and writes them to two registers.

For the M profile, the PC value must be word-aligned, otherwise the behavior of the instruction is UNPREDICTABLE.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	P	U	1	W	1	1	1	1	1	Rt															

T1 variant

```

LDRD{<c>}{<q>} <Rt>, <Rt2>, <label>
    // Normal form
LDRD{<c>}{<q>} <Rt>, <Rt2>, [PC, #{+/-}<imm>]
    // Alternative form

```

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); t2 = UInt(Rt2);
4 imm32 = ZeroExtend(imm8:'00', 32); add = (U == '1');
5 if t IN {13,15} || t2 IN {13,15} || t == t2 then UNPREDICTABLE;
6 if W == '1' then UNPREDICTABLE;

```

CONSTRAINED UNPREDICTABLE behavior

If $t == t2$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The load instruction executes but the destination register takes an UNKNOWN value.

CONSTRAINED UNPREDICTABLE behavior

If $W == '1'$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes without write-back of the base address.
- The instruction uses post-indexed addressing when $P == '0'$ and uses pre-indexed addressing otherwise.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the first general-purpose register to be transferred, encoded in the "Rt" field.
<Rt2>	Is the second general-purpose register to be transferred, encoded in the "Rt2" field.
<label>	The label of the literal data item that is to be loaded into <Rt>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. Permitted values of the offset are multiples of 4 in the range -1020 to 1020. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE, encoded as U == 1. If the offset is negative, imm32 is equal to minus the offset and add == FALSE, encoded as U == 0.

<code>+/-</code>	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<code><imm></code>	Is the optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1 // Since the SG instruction is not subject to IT conditions, the SEE check for that
2 // instruction must be done before the condition code check, and not inside the decode
3 // pseudocode (called by EncodingSpecificOperations()).
4 if P == '1' && W == '1' && U == '0' then SEE SG;
5
6 if ConditionPassed() then
7     EncodingSpecificOperations();
8     if PC[1:0] != '00' then UNPREDICTABLE;
9     address = if add then (PC + imm32) else (PC - imm32);
10    R[t] = MemA[address, 4];
11    R[t2] = MemA[address+4, 4];

```

CONSTRAINED UNPREDICTABLE behavior

If `PC<1:0> != '00'`, then one of the following behaviors must occur:

- The instruction executes as described, with no change to its behavior and no additional side effects.
- The instruction generates an UNALIGNED UsageFault.

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.87 LDREX

Load Register Exclusive. Load Register Exclusive calculates an address from a base register value and an immediate offset, loads a word from memory, writes it to a register, and:

- If the address has the Shareable memory attribute, marks the physical address as exclusive access for the executing PE in a global monitor.
- Causes the executing PE to indicate an active exclusive access in the local monitor.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	0	1	0	1	Rn				Rt				(1)	(1)	(1)	(1)	imm8							

T1 variant

LDREX{<c>}{<q>} <Rt>, [<Rn> {, #<imm>}]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
<imm>	The immediate offset added to the value of <Rn> to calculate the address. <imm> can be omitted, meaning an offset of 0. Values are multiples of 4 in the range 0-1020.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n] + imm32;
4   SetExclusiveMonitors(address,4);
5   R[t] = MemA(address, 4);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.88 LDREXB

Load Register Exclusive Byte. Load Register Exclusive Byte derives an address from a base register value, loads a byte from memory, zero-extends it to form a 32-bit word, writes it to a register, and:

- If the address has the Shareable memory attribute, marks the physical address as exclusive access for the executing PE in a global monitor.
- Causes the executing PE to indicate an active exclusive access in the local monitor.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	1	Rn				Rt				(1) (1) (1) (1)				0	1	0	0	(1) (1) (1) (1)			

T1 variant

LDREXB{<c>}{<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n];
4     SetExclusiveMonitors(address,1);
5     R[t] = ZeroExtend(MemA[address, 1], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.89 LDREXH

Load Register Exclusive Halfword. Load Register Exclusive Halfword derives an address from a base register value, loads a halfword from memory, zero-extends it to form a 32-bit word, writes it to a register, and:

- If the address has the Shareable memory attribute, marks the physical address as exclusive access for the executing PE in a global monitor.
- Causes the executing PE to indicate an active exclusive access in the local monitor.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	1	Rn				Rt				(1) (1) (1) (1)				0	1	0	1	(1) (1) (1) (1)			

T1 variant

LDREXH{<c>}{<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n];
4   SetExclusiveMonitors(address,2);
5   R[t] = ZeroExtend(MemA[address, 2], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.90 LDRH (immediate)

Load Register Halfword (immediate). Load Register Halfword (immediate) calculates an address from a base register value and an immediate offset, loads a halfword from memory, zero-extends it to form a 32-bit word, and writes it to a register. It can use offset, post-indexed, or pre-indexed addressing.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	1	imm5					Rn			Rt		

T1 variant

LDRH{<c>}{<q>} <Rt>, [<Rn> {, #(<+>)<imm>}]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm5:'0', 32);
2 index = TRUE; add = TRUE; wback = FALSE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	1	0	1	1	Rn				Rt				imm12											

T2 variant

LDRH{<c>}.W <Rt>, [<Rn> {, #(<+>)<imm>}]
 // <Rt>, <Rn>, <imm> can be represented in T1
 LDRH{<c>}{<q>} <Rt>, [<Rn> {, #(<+>)<imm>}]

Decode for this encoding

```
1 if Rt == '1111' then SEE "Related encodings";
2 if Rn == '1111' then SEE "LDRH (literal)";
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm12, 32);
5 index = TRUE; add = TRUE; wback = FALSE;
6 if t == 13 then UNPREDICTABLE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	1	1	Rn				Rt				1	P	U	W	imm8							

Offset variant

Applies when Rt != 1111 && P == 1 && U == 0 && W == 0.

LDRH{<c>}{<q>} <Rt>, [<Rn> {, #-<imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

LDRH{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

LDRH{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>!

Decode for this encoding

```
1 if Rn == '1111' then SEE "LDRH (literal)";
2 if Rt == '1111' && P == '1' && U == '0' && W == '0' then SEE "Related encodings";
3 if P == '1' && U == '1' && W == '0' then SEE LDRHT;
4 if P == '0' && W == '0' then UNDEFINED;
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm8, 32);
7 index = (P == '1'); add = (U == '1'); wback = (W == '1');
8 if t == 13 || (t == 15 && W == '1') || (wback && n == t) then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If wback && n == t, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs all of the loads using the specified addressing mode and the content of the register that is written back is UNKNOWN. In addition, if an exception occurs during such an instruction, the base address might be corrupted so that the instruction cannot be repeated.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	For encoding T1: is the general-purpose base register, encoded in the "Rn" field. For encoding T2 and T3: is the general-purpose base register, encoded in the "Rn" field. For PC use see C2.4.91 LDRH (literal) .
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
+	Specifies the offset is added to the base register.
<imm>	For the post-indexed or pre-indexed variant: is an 8-bit unsigned immediate byte offset, in the range 0 to 255, encoded in the "imm8" field. For encoding T1: is the optional positive unsigned immediate byte offset, a multiple of 2 in the range 0 to 62, defaulting to 0 and encoded in the "imm5" field as <imm>/2. For encoding T2: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T3: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
```

```
4      address = if index then offset_addr else R[n];
5
6      // Determine if the stack pointer limit should be checked
7      if n == 13 && wback then
8          violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
9      else
10         violatesLimit = FALSE;
11     // Memory operation only performed if limit not violated
12     if !violatesLimit then
13         R[t] = ZeroExtend(MemU[address, 2], 32);
14
15     // If the stack pointer is being updated a fault will be raised if
16     // the limit is violated
17     if wback then RSPCheck[n] = offset_addr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1, and R15 is not being used as the destination register:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.91 LDRH (literal)

Load Register Halfword (literal). Load Register Halfword (literal) calculates an address from the PC value and an immediate offset, loads a halfword from memory, zero-extends it to form a 32-bit word, and writes it to a register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	U	0	1	1	1	1	1	1	Rt				imm12											

T1 variant

```
LDRH{<c>}{<q>} <Rt>, <label>
// Preferred syntax
LDRH{<c>}{<q>} <Rt>, [PC, #{+/-}<imm>]
// Alternative syntax
```

Decode for this encoding

```
1 if Rt == '1111' then SEE "PLD (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); imm32 = ZeroExtend(imm12, 32); add = (U == '1');
4 if t == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<label>	The label of the literal data item that is to be loaded into <Rt>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. Permitted values of the offset are -4095 to 4095. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE, encoded as U == 1. If the offset is negative, imm32 is equal to minus the offset and add == FALSE, encoded as U == 0.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	Is a 12-bit unsigned immediate byte offset, in the range 0 to 4095, encoded in the "imm12" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   base = Align(PC, 4);
4   address = if add then (base + imm32) else (base - imm32);
5   data = MemU[address, 2];
6   R[t] = ZeroExtend(data, 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.

- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.92 LDRH (register)

Load Register Halfword (register). Load Register Halfword (register) calculates an address from a base register value and an offset register value, loads a halfword from memory, zero-extends it to form a 32-bit word, and writes it to a register. The offset register value can be shifted left by 0, 1, 2, or 3 bits.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	0	1	Rm			Rn			Rt		

T1 variant

LDRH{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
2 index = TRUE; add = TRUE; wback = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	0	0	0	0	1	1	Rn				Rt				0	0	0	0	0	0	imm2			Rm			

T2 variant

LDRH{<c>}.W <Rt>, [<Rn>, {+}<Rm>]
 // <Rt>, <Rn>, <Rm> can be represented in T1
 LDRH{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>{, LSL #<imm>}]

Decode for this encoding

```
1 if Rn == '1111' then SEE "LDRH (literal)";
2 if Rt == '1111' then SEE "Related encodings";
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
5 index = TRUE; add = TRUE; wback = FALSE;
6 (shift_t, shift_n) = (SRTYPE_LSL, UInt(imm2));
7 if t == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register is added to the base register.
<Rm>	Is the general-purpose index register, encoded in the "Rm" field.
<imm>	If present, the size of the left shift to apply to the value from <Rm>, in the range 1-3. <imm> is encoded in imm2. If absent, no shift is specified and imm2 is encoded as 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```

2      EncodingSpecificOperations();
3      offset = Shift(R[m], shift_t, shift_n, APSR.C);
4      offset_addr = if add then (R[n] + offset) else (R[n] - offset);
5      address = if index then offset_addr else R[n];
6      data = MemU(address, 2);
7      if wback then R[n] = offset_addr;
8      R[t] = ZeroExtend(data, 32);

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.93 LDRHT

Load Register Halfword Unprivileged. Load Register Halfword Unprivileged calculates an address from a base register value and an immediate offset, loads a halfword from memory, zero-extends it to form a 32-bit word, and writes it to a register.

When privileged software uses an [LDRHT](#) instruction, the memory access is restricted as if the software was unprivileged.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	1	1	Rn				Rt				1	1	1	0	imm8							

T1 variant

LDRHT{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```

1 if Rn == '1111' then SEE "LDRH (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); postindex = FALSE; add = TRUE;
4 register_form = FALSE; imm32 = ZeroExtend(imm8, 32);
5 if t IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the offset is added to the base register.
<imm>	Is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n] + imm32;
4     data = MemU_unpriv(address, 2);
5     R[t] = ZeroExtend(data, 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.94 LDRSB (immediate)

Load Register Signed Byte (immediate). Load Register Signed Byte (immediate) calculates an address from a base register value and an immediate offset, loads a byte from memory, sign-extends it to form a 32-bit word, and writes it to a register. It can use offset, post-indexed, or pre-indexed addressing.

This instruction is subject to stack limit checking.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	1	0	0	1	Rn				Rt				imm12											

T1 variant

LDRSB{<c>}{<q>} <Rt>, [<Rn> {, #{+}<imm>}]

Decode for this encoding

```

1 if Rt == '1111' then SEE "PLI (immediate, literal)";
2 if Rn == '1111' then SEE "LDRSB (literal)";
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm12, 32);
5 index = TRUE; add = TRUE; wback = FALSE;
6 if t == 13 then UNPREDICTABLE;

```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	0	0	0	1	Rn				Rt				1	P	U	W	imm8							

Offset variant

Applies when **P == 1 && U == 0 && W == 0**.

LDRSB{<c>}{<q>} <Rt>, [<Rn> {, #-<imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

LDRSB{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

LDRSB{<c>}{<q>} <Rt>, [<Rn>, #{+/-}<imm>]!

Decode for this encoding

```

1 if Rt == '1111' && P == '1' && U == '0' && W == '0' then SEE "PLI (immediate, literal)";
2 if Rn == '1111' then SEE "LDRSB (literal)";
3 if P == '1' && U == '1' && W == '0' then SEE LDRSBT;
4 if P == '0' && W == '0' then UNDEFINED;
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm8, 32);
7 index = (P == '1'); add = (U == '1'); wback = (W == '1');
8 if t == 13 || (t == 15 && W == '1') || (wback && n == t) then UNPREDICTABLE;

```

CONSTRAINED UNPREDICTABLE behavior

If `wback && n == t`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs all of the loads using the specified addressing mode and the content of the register that is written back is UNKNOWN. In addition, if an exception occurs during such an instruction, the base address might be corrupted so that the instruction cannot be repeated.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field. For PC use see C2.4.95 LDRSB (literal) .
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
+	Specifies the offset is added to the base register.
<imm>	For the post-indexed or pre-indexed variant: is an 8-bit unsigned immediate byte offset, in the range 0 to 255, encoded in the "imm8" field. For encoding T1: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T2: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
4     address = if index then offset_addr else R[n];
5
6     // Determine if the stack pointer limit should be checked
7     if n == 13 && wback then
8         violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
9     else
10        violatesLimit = FALSE;
11    // Memory operation only performed if limit not violated
12    if !violatesLimit then
13        R[t] = SignExtend(MemU(address, 1), 32);
14
15    // If the stack pointer is being updated a fault will be raised if
16    // the limit is violated
17    if wback then RSPCheck[n] = offset_addr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1, and R15 is not being used as the destination register:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.95 LDRSB (literal)

Load Register Signed Byte (literal). Load Register Signed Byte (literal) calculates an address from the PC value and an immediate offset, loads a byte from memory, sign-extends it to form a 32-bit word, and writes it to a register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	U	0	0	1	1	1	1	1	Rt				imm12											

T1 variant

```
LDRSB{<c>}{<q>} <Rt>, <label>
// Preferred syntax
LDRSB{<c>}{<q>} <Rt>, [PC, #{+/-}<imm>]
// Alternative syntax
```

Decode for this encoding

```
1 if Rt == '1111' then SEE "PLI (immediate, literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); imm32 = ZeroExtend(imm12, 32); add = (U == '1');
4 if t == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<label>	The label of the literal data item that is to be loaded into <Rt>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. Permitted values of the offset are -4095 to 4095. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE, encoded as U == 1. If the offset is negative, imm32 is equal to minus the offset and add == FALSE, encoded as U == 0.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	Is a 12-bit unsigned immediate byte offset, in the range 0 to 4095, encoded in the "imm12" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   base = Align(PC, 4);
4   address = if add then (base + imm32) else (base - imm32);
5   R[t] = SignExtend(MemU[address, 1], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.

- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.96 LDRSB (register)

Load Register Signed Byte (register). Load Register Signed Byte (register) calculates an address from a base register value and an offset register value, loads a byte from memory, sign-extends it to form a 32-bit word, and writes it to a register. The offset register value can be shifted left by 0, 1, 2, or 3 bits.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	Rm			Rn			Rt		

T1 variant

LDRSB{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
2 index = TRUE; add = TRUE; wback = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	0	1	0	0	0	1	Rn				Rt				0	0	0	0	0	imm2				Rm			

T2 variant

LDRSB{<c>}.W <Rt>, [<Rn>, {+}<Rm>]
 // <Rt>, <Rn>, <Rm> can be represented in T1
 LDRSB{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>{, LSL #<imm>}]

Decode for this encoding

```
1 if Rt == '1111' then SEE "PLI (register)";
2 if Rn == '1111' then SEE "LDRSB (literal)";
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
5 index = TRUE; add = TRUE; wback = FALSE;
6 (shift_t, shift_n) = (SRTYPE_LSL, UInt(imm2));
7 if t == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register is added to the base register.
<Rm>	Is the general-purpose index register, encoded in the "Rm" field.
<imm>	If present, the size of the left shift to apply to the value from <Rm>, in the range 1-3. <imm> is encoded in imm2. If absent, no shift is specified and imm2 is encoded as 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```

2      EncodingSpecificOperations();
3      offset = Shift(R[m], shift_t, shift_n, APSR.C);
4      offset_addr = if add then (R[n] + offset) else (R[n] - offset);
5      address = if index then offset_addr else R[n];
6      R[t] = SignExtend(MemU[address, 1], 32);

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.97 LDRSBT

Load Register Signed Byte Unprivileged. Load Register Signed Byte Unprivileged calculates an address from a base register value and an immediate offset, loads a byte from memory, sign-extends it to form a 32-bit word, and writes it to a register.

When privileged software uses an **LDRSBT** instruction, the memory access is restricted as if the software was unprivileged.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	0	0	0	1	Rn				Rt				1	1	1	0	imm8							

T1 variant

LDRSBT{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```

1 if Rn == '1111' then SEE "LDRSB (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); postindex = FALSE; add = TRUE;
4 register_form = FALSE; imm32 = ZeroExtend(imm8, 32);
5 if t IN {13,15} then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the offset is added to the base register.
<imm>	Is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n] + imm32;
4     R[t] = SignExtend(MemU_unpriv[address, 1], 32);

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.98 LDRSH (immediate)

Load Register Signed Halfword (immediate). Load Register Signed Halfword (immediate) calculates an address from a base register value and an immediate offset, loads a halfword from memory, sign-extends it to form a 32-bit word, and writes it to a register. It can use offset, post-indexed, or pre-indexed addressing.

This instruction is subject to stack limit checking.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	1	0	1	1	Rn				Rt				imm12											

T1 variant

LDRSH{<c>}{<q>} <Rt>, [<Rn> {, #{+}<imm>}]

Decode for this encoding

```
1 if Rn == '1111' then SEE "LDRSH (literal)";
2 if Rt == '1111' then SEE "Related encodings";
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm12, 32);
5 index = TRUE; add = TRUE; wback = FALSE;
6 if t == 13 then UNPREDICTABLE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	0	0	1	1	Rn				Rt				1	P	U	W	imm8							

Offset variant

Applies when Rt != 1111 && P == 1 && U == 0 && W == 0.

LDRSH{<c>}{<q>} <Rt>, [<Rn> {, #-<imm>}]

Post-indexed variant

Applies when P == 0 && W == 1.

LDRSH{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>

Pre-indexed variant

Applies when P == 1 && W == 1.

LDRSH{<c>}{<q>} <Rt>, [<Rn>, #{+/-}<imm>]!

Decode for this encoding

```
1 if Rn == '1111' then SEE "LDRSH (literal)";
2 if Rt == '1111' && P == '1' && U == '0' && W == '0' then SEE "Related encodings";
3 if P == '1' && U == '1' && W == '0' then SEE LDRSHT;
4 if P == '0' && W == '0' then UNDEFINED;
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm8, 32);
7 index = (P == '1'); add = (U == '1'); wback = (W == '1');
8 if t == 13 || (t == 15 && W == '1') || (wback && n == t) then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `wback && n == t`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs all of the loads using the specified addressing mode and the content of the register that is written back is UNKNOWN. In addition, if an exception occurs during such an instruction, the base address might be corrupted so that the instruction cannot be repeated.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field. For PC use see C2.4.99 LDRSH (literal) .
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
+	Specifies the offset is added to the base register.
<imm>	For the post-indexed or pre-indexed variant: is an 8-bit unsigned immediate byte offset, in the range 0 to 255, encoded in the "imm8" field. For encoding T1: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T2: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
4      address = if index then offset_addr else R[n];
5      // Determine if the stack pointer limit should be checked
6      if n == 13 && wback then
7          violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
8      else
9          violatesLimit = FALSE;
10     // Memory operation only performed if limit not violated
11     if !violatesLimit then
12         R[t] = SignExtend(MemU(address, 2), 32);
13
14     // If the stack pointer is being updated a fault will be raised if
15     // the limit is violated
16     if wback then RSPCheck[n] = offset_addr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1, and R15 is not being used as the destination register:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.99 LDRSH (literal)

Load Register Signed Halfword (literal). Load Register Signed Halfword (literal) calculates an address from the PC value and an immediate offset, loads a halfword from memory, sign-extends it to form a 32-bit word, and writes it to a register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	U	0	1	1	1	1	1	1	Rt				imm12											

T1 variant

```
LDRSH{<c>}{<q>} <Rt>, <label>
// Preferred syntax
LDRSH{<c>}{<q>} <Rt>, [PC, #{+/-}<imm>]
// Alternative syntax
```

Decode for this encoding

```
1 if Rt == '1111' then SEE "Related encodings";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); imm32 = ZeroExtend(imm12, 32); add = (U == '1');
4 if t == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<label>	The label of the literal data item that is to be loaded into <Rt>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. Permitted values of the offset are -4095 to 4095. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE, encoded as U == 1. If the offset is negative, imm32 is equal to minus the offset and add == FALSE, encoded as U == 0.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	Is a 12-bit unsigned immediate byte offset, in the range 0 to 4095, encoded in the "imm12" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   base = Align(PC, 4);
4   address = if add then (base + imm32) else (base - imm32);
5   data = MemU(address, 2);
6   R[t] = SignExtend(data, 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.

- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.100 LDRSH (register)

Load Register Signed Halfword (register). Load Register Signed Halfword (register) calculates an address from a base register value and an offset register value, loads a halfword from memory, sign-extends it to form a 32-bit word, and writes it to a register. The offset register value can be shifted left by 0, 1, 2, or 3 bits.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	1	1	Rm	Rn	Rt						

T1 variant

LDRSH{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
2 index = TRUE; add = TRUE; wback = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	1	1	0	0	1	0	0	1	1	Rn				Rt				0	0	0	0	0	0	imm2				Rm			

T2 variant

LDRSH{<c>}.W <Rt>, [<Rn>, {+}<Rm>]
// <Rt>, <Rn>, <Rm> can be represented in T1
LDRSH{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>{, LSL #<imm>}]

Decode for this encoding

```
1 if Rn == '1111' then SEE "LDRSH (literal)";
2 if Rt == '1111' then SEE "Related encodings";
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
5 index = TRUE; add = TRUE; wback = FALSE;
6 (shift_t, shift_n) = (SRTYPE_LSL, UInt(imm2));
7 if t == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register is added to the base register.
<Rm>	Is the general-purpose index register, encoded in the "Rm" field.
<imm>	If present, the size of the left shift to apply to the value from <Rm>, in the range 1-3. <imm> is encoded in imm2. If absent, no shift is specified and imm2 is encoded as 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```

2      EncodingSpecificOperations();
3      offset = Shift(R[m], shift_t, shift_n, APSR.C);
4      offset_addr = if add then (R[n] + offset) else (R[n] - offset);
5      address = if index then offset_addr else R[n];
6      data = MemU(address, 2);
7      if wback then R[n] = offset_addr;
8      R[t] = SignExtend(data, 32);

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.101 LDRSHT

Load Register Signed Halfword Unprivileged. Load Register Signed Halfword Unprivileged calculates an address from a base register value and an immediate offset, loads a halfword from memory, sign-extends it to form a 32-bit word, and writes it to a register.

When privileged software uses an **LDRSHT** instruction, the memory access is restricted as if the software was unprivileged.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	0	0	1	1	Rn				Rt				1	1	1	0	imm8							

T1 variant

LDRSHT{<c>}{<q>} <Rt>, [<Rn> {, #<+>}<imm>}]

Decode for this encoding

```

1 if Rn == '1111' then SEE "LDRSH (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); postindex = FALSE; add = TRUE;
4 register_form = FALSE; imm32 = ZeroExtend(imm8, 32);
5 if t IN {13,15} then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the offset is added to the base register.
<imm>	Is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n] + imm32;
4   data = MemU_unpriv[address, 2];
5   R[t] = SignExtend(data, 32);

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.102 LDRT

Load Register Unprivileged. Load Register Unprivileged calculates an address from a base register value and an immediate offset, loads a word from memory, and writes it to a register.

When privileged software uses an [LDRT](#) instruction, the memory access is restricted as if the software was unprivileged.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	1	0	1	Rn				Rt				1	1	1	0	imm8							

T1 variant

LDRT{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```

1 if Rn == '1111' then SEE "LDR (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); postindex = FALSE; add = TRUE;
4 register_form = FALSE; imm32 = ZeroExtend(imm8, 32);
5 if t IN {13,15} then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the offset is added to the base register.
<imm>	Is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n] + imm32;
4     data = MemU_unpriv[address, 4];
5     R[t] = data;

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.103 LE, LETP

Loop End, Loop End with Tail Predication. If additional iterations of a loop are required this instruction branches back to the <label>. It also stores the loop information in the loop info cache so that future iterations of the loop will branch back to the start just before the LE instruction is encountered. The first variant of the instruction checks a loop iteration counter (stored in LR) to determine if additional iterations are required. It also decrements the counter ready for the next iteration.

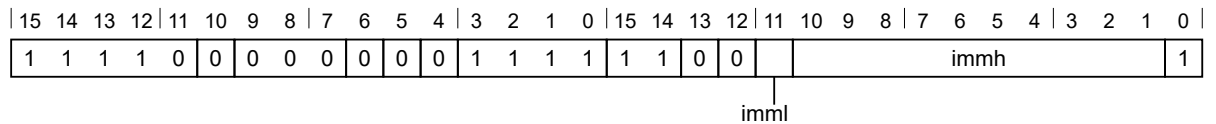
The second variant does not use an iteration count and always triggers another iteration of the loop.

The third (TP) variant also checks the loop iteration counter to determine if additional iterations are required. However the counter is decremented by the number of elements of a vector (as indicated by the FPSCR.LTPSIZE field). On the last iteration of the loop, this variant disables tail predication.

This instruction is not permitted in an IT block.

T1

Armv8.1-M Low Overhead Branch Extension



T1: LE variant

LE<q> LR, <label>

Decode for this encoding

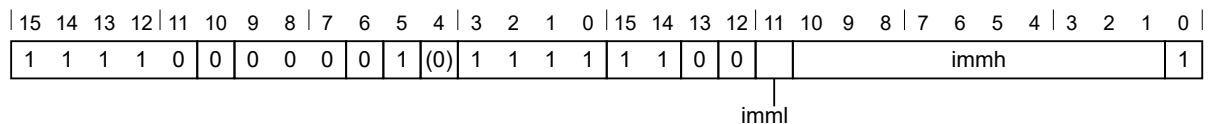
```

1 if !HaveLOBExt() then UNDEFINED;
2
3 forever = FALSE;
4 tp      = FALSE;
5 imm32   = ZeroExtend(immh:imm1:'0', 32);
6 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M Low Overhead Branch Extension



T2: LE variant

LE<q> <label>

Decode for this encoding

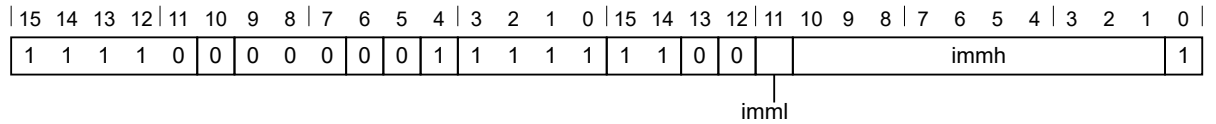
```

1 if !HaveLOBExt() then UNDEFINED;
2
3 forever = TRUE;
4 tp      = FALSE;
5 imm32   = ZeroExtend(immh:imm1:'0', 32);
6 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

T3

Armv8.1-M Low Overhead Branch Extension and MVE



T3: LETP variant

LETP<q> LR, <label>

Decode for this encoding

```

1 if !HaveLOBExt() then UNDEFINED;
2 if !HaveMve() then UNDEFINED;
3 HandleException(CheckCPEnabled(10));
4
5 forever = FALSE;
6 tp      = TRUE;
7 imm32   = ZeroExtend(immh:immh:'0', 32);
8 if InitTBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<q> See [C1.2.5 Standard assembler syntax fields](#).
 <LR> LR is used to hold the iteration counter of the loop, this instruction must always use this register.
 <label> Specifies the label of the first instruction in the loop body.

Operation for all encodings

```

1 EncodingSpecificOperations();
2
3 if tp then
4     ExecuteFPCheck();
5 elseif LTPSIZE != 4 then
6     // Tail predicated loop starts should be paired with an LETP loop end.
7     // Using a LE instruction in this case is a programming error.
8     UFSR.INVSTATE = '1';
9     HandleException(CreateException(UsageFault));
10 if !forever && IsLastLowOverheadLoop() then
11     if tp then
12         FPSR.LTPSIZE = 4[2:0]; // Disable loop predication
13 else
14     // Decrement the loop counter
15     if !forever then
16         LR = LR - (1 << (4 - LTPSIZE))[31:0];
17     // Set up the branch cache info
18     jumpAddr = PC - imm32;
19     if CCR.LOB == '1' then
20         LO_BRANCH_INFO.VALID = '1';
21         LO_BRANCH_INFO.BF = '0';
22         LO_BRANCH_INFO.LF = if forever then '1' else '0';
23         LO_BRANCH_INFO.T16IND = '0';
24         LO_BRANCH_INFO.BTI = '0';
25         LO_BRANCH_INFO.JUMP_ADDR = jumpAddr[31:1];
26         LO_BRANCH_INFO.END_ADDR = ThisInstrAddr()[31:1];
27     // Branch to the start of the loop
28     BranchTo(jumpAddr);

```

C2.4.104 LSL (immediate)

Logical Shift Left (immediate). Logical Shift Left (immediate) shifts a register value left by an immediate number of bits, shifting in zeros, and writes the result to the destination register.

This instruction is an alias of the [MOV \(register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV \(register\)](#).
- The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	op = 00	imm5 != 00000				Rm			Rd			

T2 variant

```
LSL<c>{<q>} {<Rd>}, <Rm>, #<imm>
// Inside IT block
```

is equivalent to

```
MOV<c>{<q>} <Rd>, <Rm>, LSL #<imm>
```

and is the preferred disassembly when `InITBlock()`.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0		1	1	1	1	[0]	imm3			Rd			imm2			Rm != 11x1					
S = 0												sr_type = 00																			

MOV, shift or rotate by value variant

```
LSL<c>.W {<Rd>}, <Rm>, #<imm>
// Inside IT block, and <Rd>, <Rm>, <imm> can be represented in T2
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, LSL #<imm>
```

and is always the preferred disassembly.

MOV, shift or rotate by value variant

```
LSL{<c>}{<q>} {<Rd>}, <Rm>, #<imm>
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, LSL #<imm>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.
<imm>	For encoding T2: is the shift amount, in the range 1 to 31, encoded in the "imm5" field. For encoding T3: is the shift amount, in the range 1 to 31, encoded in the "imm3:imm2" field.

Operation for all encodings

The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.105 LSL (register)

Logical Shift Left (register). Logical Shift Left (register) shifts a register value left by a variable number of bits, shifting in zeros, and writes the result to the destination register. The variable number of bits is read from the bottom byte of a register.

This instruction is an alias of the [MOV, MOVS \(register-shifted register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV, MOVS \(register-shifted register\)](#).
- The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op = 0010				Rs			Rdm		

Logical shift left variant

```
LSL<c>{<q>} {<Rdm>, } <Rdm>, <Rs>
// Inside IT block
```

is equivalent to

```
MOV<c>{<q>} <Rdm>, <Rdm>, LSL <Rs>
```

and is the preferred disassembly when `InITBlock()`.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0				Rm				1	1	1	1	Rd			0	0	0	0	Rs				
								sr_type = 00																							
								S = 0																							

Non flag setting variant

```
LSL<c>.W {<Rd>, } <Rm>, <Rs>
// Inside IT block, and <Rd>, <Rm>, <sr_type>, <Rs> can be represented in T1
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, LSL <Rs>
```

and is always the preferred disassembly.

Non flag setting variant

```
LSL{<c>}{<q>} {<Rd>, } <Rm>, <Rs>
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, LSL <Rs>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<C>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the first general-purpose source register and the destination register, encoded in the "Rdm" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rs>	Is the second general-purpose source register holding a shift amount in its bottom 8 bits, encoded in the "Rs" field.

Operation for all encodings

The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.106 LSL (immediate)

Logical Shift Left Long. Logical shift left by 1 to 32 bits of a 64-bit value stored in two general-purpose registers.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	0	0	immh	RdaHi	(1)	imml	0	0	1	1	1	1							

T1: LSL variant

LSL<c><q> RdaLo, RdaHi, #<imm>

Decode for this encoding

```

1 if RdaHi == '111' then SEE "UQSHL (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 (-, amount) = DecodeImmShift('10', immh:immh);
8 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
<imm>	The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     op1 = UInt(R[dah]:R[dal]);
5     result = (op1 << amount)[63:0];
6     R[dah] = result[63:32];
7     R[dal] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.107 LSL (register)

Logical Shift Left Long. Logical shift left by 0 to 64 bits of a 64-bit value stored in two general-purpose registers. The shift amount is read in as the bottom byte of Rm. If the shift amount is negative, the shift direction is reversed.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	0	Rm	RdaHi	(1)	(0)	(0)	0	0	1	1	0	1							

T1: LSL variant

LSLL<c><q> RdaLo, RdaHi, Rm

Decode for this encoding

```

1 if RdaHi == '111' then SEE "UQRSHL (register)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 m = UInt(Rm);
8 if RdaHi == '110' || Rm == '11x1' || Rm == RdaHi:'1' then CONSTRAINED_UNPREDICTABLE;
9 if Rm == RdaLo:'0' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
<Rm>	General-purpose source register holding a shift amount in its bottom 8 bits.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     amount = SInt(R[m][7:0]);
5     opl = UInt(R[dah]:R[dal]);
6     result = (opl << amount)[63:0];
7     R[dah] = result[63:32];
8     R[dal] = result[31:0];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.108 LSLS (immediate)

Logical Shift Left, Setting flags (immediate). Logical Shift Left, Setting flags (immediate) shifts a register value left by an immediate number of bits, shifting in zeros, writes the result to the destination register, and updates the condition flags based on the result.

This instruction is an alias of the [MOV \(register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV \(register\)](#).
- The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
op = 00				imm5 != 00000				Rm				Rd			

T2 variant

```
LSLS{<q>} {<Rd>}, <Rm>, #<imm>
// Outside IT block
```

is equivalent to

```
MOVS{<q>} <Rd>, <Rm>, LSL #<imm>
```

and is the preferred disassembly when **!InITBlock()**.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0					1	1	1	1	[0]	imm3				Rd				imm2		Rm				
S = 1												sr type = 00																			

MOVS, shift or rotate by value variant

```
LSLS.W {<Rd>}, <Rm>, #<imm>
// Outside IT block, and <Rd>, <Rm>, <imm> can be represented in T2
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, LSL #<imm>
```

and is always the preferred disassembly.

MOVS, shift or rotate by value variant

```
LSLS{<c>}{<q>} {<Rd>}, <Rm>, #<imm>
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, LSL #<imm>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.
<imm>	For encoding T2: is the shift amount, in the range 1 to 31, encoded in the "imm5" field. For encoding T3: is the shift amount, in the range 1 to 31, encoded in the "imm3:imm2" field.

Operation for all encodings

The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.109 LSLS (register)

Logical Shift Left, Setting flags (register). Logical Shift Left, Setting flags (register) shifts a register value left by a variable number of bits, shifting in zeros, writes the result to the destination register, and updates the condition flags based on the result. The variable number of bits is read from the bottom byte of a register.

This instruction is an alias of the [MOV, MOVS \(register-shifted register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV, MOVS \(register-shifted register\)](#).
- The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op = 0010				Rs			Rdm		

Logical shift left variant

```
LSLS{<q>} {<Rdm>, } <Rdm>, <Rs>
// Outside IT block
```

is equivalent to

```
MOVS{<q>} <Rdm>, <Rdm>, LSL <Rs>
```

and is the preferred disassembly when `!InITBlock()`.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0								1	1	1	1					0	0	0	0				

sr_type = 00

S = 1

Flag setting variant

```
LSLS.W {<Rd>, } <Rm>, <Rs>
// Outside IT block, and <Rd>, <Rm>, <sr_type>, <Rs> can be represented in T1
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, LSL <Rs>
```

and is always the preferred disassembly.

Flag setting variant

```
LSLS{<c>}{<q>} {<Rd>, } <Rm>, <Rs>
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, LSL <Rs>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<C>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the first general-purpose source register and the destination register, encoded in the "Rdm" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rs>	Is the second general-purpose source register holding a shift amount in its bottom 8 bits, encoded in the "Rs" field.

Operation for all encodings

The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.110 LSR (immediate)

Logical Shift Right (immediate). Logical Shift Right (immediate) shifts a register value right by an immediate number of bits, shifting in zeros, and writes the result to the destination register.

This instruction is an alias of the [MOV \(register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV \(register\)](#).
- The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	pp = 01	imm5				Rm			Rd			

T2 variant

```
LSR<c>{<q>} {<Rd>}, <Rm>, #<imm>
// Inside IT block
```

is equivalent to

```
MOV<c>{<q>} <Rd>, <Rm>, LSR #<imm>
```

and is the preferred disassembly when `InITBlock()`.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0		1	1	1	1	[0]	imm3				Rd				imm2		Rm != 11x1				
S = 0												sr_type = 01																			

MOV, shift or rotate by value variant

```
LSR<c>.W {<Rd>}, <Rm>, #<imm>
// Inside IT block, and <Rd>, <Rm>, <imm> can be represented in T2
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, LSR #<imm>
```

and is always the preferred disassembly.

MOV, shift or rotate by value variant

```
LSR{<c>}{<q>} {<Rd>}, <Rm>, #<imm>
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, LSR #<imm>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.
<imm>	For encoding T2: is the shift amount, in the range 1 to 32, encoded in the "imm5" field as <imm> modulo 32. For encoding T3: is the shift amount, in the range 1 to 32, encoded in the "imm3:imm2" field as <imm> modulo 32.

Operation for all encodings

The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.111 LSR (register)

Logical Shift Right (register). Logical Shift Right (register) shifts a register value right by a variable number of bits, shifting in zeros, and writes the result to the destination register. The variable number of bits is read from the bottom byte of a register.

This instruction is an alias of the [MOV, MOVS \(register-shifted register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV, MOVS \(register-shifted register\)](#).
- The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op = 0011				Rs			Rdm		

Logical shift right variant

```
LSR<c>{<q>} {<Rdm>, } <Rdm>, <Rs>
// Inside IT block
```

is equivalent to

```
MOV<c>{<q>} <Rdm>, <Rdm>, LSR <Rs>
```

and is the preferred disassembly when `InITBlock()`.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0				Rm				1	1	1	1	Rd			0	0	0	0	Rs				
								sr_type																							
								S = 0																							

Non flag setting variant

```
LSR<c>.W {<Rd>, } <Rm>, <Rs>
// Inside IT block, and <Rd>, <Rm>, <type>, <Rs> can be represented in T1
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, LSR <Rs>
```

and is always the preferred disassembly.

Non flag setting variant

```
LSR{<c>}{<q>} {<Rd>, } <Rm>, <Rs>
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, LSR <Rs>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<C>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the first general-purpose source register and the destination register, encoded in the "Rdm" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rs>	Is the second general-purpose source register holding a shift amount in its bottom 8 bits, encoded in the "Rs" field.

Operation for all encodings

The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.112 LSRL (immediate)

Logical Shift Right Long. Logical shift right by 1 to 32 bits of a 64-bit value stored in two general-purpose registers.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	0	0	immh	RdaHi	(1)	imml	0	1	1	1	1	1	1	1	1	1	1	1	

T1: LSRL variant

LSRL<c><q> RdaLo, RdaHi, #<imm>

Decode for this encoding

```

1 if RdaHi == '111' then SEE "URSHR (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 (-, amount) = DecodeImmShift('10', immh:imml);
8 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <RdaLo> General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
 <RdaHi> General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
 <imm> The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   op1 = UInt(R[dah]:R[dal]);
5   result = (op1 >> amount)[63:0];
6   R[dah] = result[63:32];
7   R[dal] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.113 LSRS (immediate)

Logical Shift Right, Setting flags (immediate). Logical Shift Right, Setting flags (immediate) shifts a register value right by an immediate number of bits, shifting in zeros, writes the result to the destination register, and updates the condition flags based on the result.

This instruction is an alias of the [MOV \(register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV \(register\)](#).
- The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	imm5					Rm			Rd		

T2 variant

```
LSRS{<q>} {<Rd>}, <Rm>, #<imm>
// Outside IT block
```

is equivalent to

```
MOVS{<q>} <Rd>, <Rm>, LSR #<imm>
```

and is the preferred disassembly when **!InITBlock()**.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	1	0	1	0	1	0					1	1	1	1	[0]	imm3			Rd			imm2						Rm						
											S = 1																				sr type = 01				

MOVS, shift or rotate by value variant

```
LSRS.W {<Rd>}, <Rm>, #<imm>
// Outside IT block, and <Rd>, <Rm>, <imm> can be represented in T2
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, LSR #<imm>
```

and is always the preferred disassembly.

MOVS, shift or rotate by value variant

```
LSRS{<c>}{<q>} {<Rd>}, <Rm>, #<imm>
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, LSR #<imm>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.
<imm>	For encoding T2: is the shift amount, in the range 1 to 32, encoded in the "imm5" field as <imm> modulo 32. For encoding T3: is the shift amount, in the range 1 to 32, encoded in the "imm3:imm2" field as <imm> modulo 32.

Operation for all encodings

The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.114 LSRS (register)

Logical Shift Right, Setting flags (register). Logical Shift Right, Setting flags (register) shifts a register value right by a variable number of bits, shifting in zeros, writes the result to the destination register, and updates the condition flags based on the result. The variable number of bits is read from the bottom byte of a register.

This instruction is an alias of the [MOV, MOVS \(register-shifted register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV, MOVS \(register-shifted register\)](#).
- The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op = 0011				Rs			Rdm		

Logical shift right variant

```
LSRS{<q>} {<Rdm>, } <Rdm>, <Rs>
// Outside IT block
```

is equivalent to

```
MOVS{<q>} <Rdm>, <Rdm>, LSR <Rs>
```

and is the preferred disassembly when `!InITBlock()`.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0				Rm				1	1	1	1	Rd				0	0	0	0	Rs			
								sr_type = 01																							
								S = 1																							

Flag setting variant

```
LSRS.W {<Rd>, } <Rm>, <Rs>
// Outside IT block, and <Rd>, <Rm>, <sr_type>, <Rs> can be represented in T1
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, LSR <Rs>
```

and is always the preferred disassembly.

Flag setting variant

```
LSRS{<c>}{<q>} {<Rd>, } <Rm>, <Rs>
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, LSR <Rs>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<C>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the first general-purpose source register and the destination register, encoded in the "Rdm" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rs>	Is the second general-purpose source register holding a shift amount in its bottom 8 bits, encoded in the "Rs" field.

Operation for all encodings

The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.115 MCR, MCR2

Move to Coprocessor from Register. Move to Coprocessor from Register passes the value of a general-purpose register to a coprocessor.

If no coprocessor can execute the instruction, a UsageFault exception is generated.

Arm reserves coprocessors CP8 to CP15, and this manual defines the valid instructions when coproc is in this range.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	opc1	0	CRn					Rt	coproc					opc2	1	CRm								

T1 variant

MCR{<c>}{<q>} <coproc>, {#}<opc1>, <Rt>, <CRn>, <CRm>{, {#}<opc2>}

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt);
7 if t == 15 || t == 13 then UNPREDICTABLE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	opc1	0	CRn					Rt	coproc					opc2	1	CRm								

T2 variant

MCR2{<c>}{<q>} <coproc>, {#}<opc1>, <Rt>, <CRn>, <CRm>{, {#}<opc2>}

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt);
7 if t == 15 || t == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields.
<q>	See C1.2.5 Standard assembler syntax fields.
<coproc>	Is the name of the coprocessor, encoded in the "coproc" field. The generic coprocessor names are p0 to p15.
<opc1>	Is a coprocessor-specific opcode in the range 0 to 7, encoded in the "opc1" field.
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<CRn>	Is the first coprocessor register, encoded in the "CRn" field.

<CRm>	Is the second coprocessor register, encoded in the "CRm" field.
<opc2>	Is a coprocessor-specific opcode in the range 0 to 7, defaulting to 0 and encoded in the "opc2" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     if !Coprocc_Accepted(cp, ThisInstr()) then
5         GenerateCoproccorException();
6     else
7         Coproc_SendOneWord(R[t], cp, ThisInstr());

```

C2.4.116 MCRR, MCRR2

Move to Coprocessor from two Registers. Move to Coprocessor from two Registers passes the values of two general-purpose registers to a coprocessor.

If no coprocessor can execute the instruction, a UsageFault exception is generated.

Arm reserves coprocessors CP8 to CP15, and this manual defines the valid instructions when coproc is in this range.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	1	0	0	Rt2				Rt				coproc				opc1				CRm			

T1 variant

MCRR{<c>}{<q>} <coproc>, {#}<opc1>, <Rt>, <Rt2>, <CRm>

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt); t2 = UInt(Rt2);
7 if t == 15 || t2 == 15 then UNPREDICTABLE;
8 if t == 13 || t2 == 13 then UNPREDICTABLE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	0	0	1	0	0	Rt2				Rt				coproc				opc1				CRm			

T2 variant

MCRR2{<c>}{<q>} <coproc>, {#}<opc1>, <Rt>, <Rt2>, <CRm>

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt); t2 = UInt(Rt2);
7 if t == 15 || t2 == 15 then UNPREDICTABLE;
8 if t == 13 || t2 == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoded in the "coproc" field. The generic coprocessor names are p0 to p15.
<opc1>	Is a coprocessor-specific opcode in the range 0 to 15, encoded in the "opc1" field.

<Rt>	Is the first general-purpose register to be transferred, encoded in the "Rt" field.
<Rt2>	Is the second general-purpose register to be transferred, encoded in the "Rt2" field.
<CRm>	Is a coprocessor register, encoded in the "CRm" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3
4      if !Coprocc_Accepted(cp, ThisInstr()) then
5          GenerateCoproccorException();
6      else
7          Coproc_SendTwoWords(R[t2], R[t], cp, ThisInstr());

```

C2.4.117 MLA

Multiply Accumulate. Multiply Accumulate multiplies two register values, and adds a third register value. The least significant 32 bits of the result are written to the destination register. These 32 bits do not depend on whether signed or unsigned calculations are performed.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	0	0	0	Rn				Ra				Rd				0 0 0 0				Rm			

T1 variant

MLA{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

Decode for this encoding

```
1 if Ra == '1111' then SEE MUL;
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); a = UInt(Ra); setflags = FALSE;
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} || a == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
 <Ra> Is the third general-purpose source register holding the addend, encoded in the "Ra" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   operand1 = SInt(R[n]); // operand1 = UInt(R[n]) produces the same final results
4   operand2 = SInt(R[m]); // operand2 = UInt(R[m]) produces the same final results
5   addend = SInt(R[a]); // addend = UInt(R[a]) produces the same final results
6   result = operand1 * operand2 + addend;
7   R[d] = result[31:0];
8   if setflags then
9     APSR.N = result[31];
10    APSR.Z = IsZeroBit(result[31:0]);
11    // APSR.C unchanged
12    // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.RDIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:

- The values of the data supplied in any of the registers being used in this instruction.
- The values of the N, Z, C, V flags.

C2.4.118 MLS

Multiply and Subtract. Multiply and Subtract multiplies two register values, and subtracts the least significant 32 bits of the result from a third register value. These 32 bits do not depend on whether signed or unsigned calculations are performed. The result is written to the destination register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	0	0	0	Rn				Ra				Rd				0	0	0	1	Rm			

T1 variant

MLS {<c>} {<q>} <Rd>, <Rn>, <Rm>, <Ra>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); a = UInt(Ra);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} || a IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Ra>	Is the third general-purpose source register holding the minuend, encoded in the "Ra" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   operand1 = SInt(R[n]); // operand1 = UInt(R[n]) produces the same final results
4   operand2 = SInt(R[m]); // operand2 = UInt(R[m]) produces the same final results
5   addend = SInt(R[a]); // addend = UInt(R[a]) produces the same final results
6   result = addend - operand1 * operand2;
7   R[d] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.119 MOV (immediate)

Move (immediate). Move (immediate) writes an immediate value to the destination register. It can optionally update the condition flags based on the value.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	Rd			imm8							

T1 variant

```
MOV<c>{<q>} <Rd>, #<imm8>
// Inside IT block
MOVS{<q>} <Rd>, #<imm8>
// Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rd); setflags = !InITBlock(); imm32 = ZeroExtend(imm8, 32); carry = APSR.C;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	0	0	1	0	S	1	1	1	1	0	imm3			Rd			imm8								

MOV variant

Applies when **S == 0**.

```
MOV<c>.W <Rd>, #<const>
// Inside IT block, and <Rd>, <const> can be represented in T1
MOV{<c>}{<q>} <Rd>, #<const>
```

MOVS variant

Applies when **S == 1**.

```
MOVS.W <Rd>, #<const>
// Outside IT block, and <Rd>, <const> can be represented in T1
MOVS{<c>}{<q>} <Rd>, #<const>
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); setflags = (S == '1'); (imm32, carry) = T32ExpandImm_C(i:imm3:imm8, APSR.C);
3 if d IN {13,15} then UNPREDICTABLE;
```

T3

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	1	0	0	1	0	0	imm4			0	imm3			Rd			imm8									

T3 variant

```
MOV{<c>}{<q>} <Rd>, #<imm16>
    // <imm16> cannot be represented in T1 or T2
MOVW{<c>}{<q>} <Rd>, #<imm16>
    // <imm16> can be represented in T1 or T2
```

Decode for this encoding

```
1 d = UInt(Rd); setflags = FALSE; imm32 = ZeroExtend(imm4:i:imm3:imm8, 32);
2 carry = bit UNKNOWN;
3 if d IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<imm8>	Is an 8-bit unsigned immediate, in the range 0 to 255, encoded in the "imm8" field.
<imm16>	Is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm4:i:imm3:imm8" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = imm32;
4     R[d] = result;
5     if setflags then
6         APSR.N = result[31];
7         APSR.Z = IsZeroBit(result);
8         APSR.C = carry;
9         // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.120 MOV (register)

Move (register). Move (register) copies a value from a register to the destination register. It can optionally update the condition flags based on the value.

This instruction is subject to stack limit checking.

This instruction is used by the aliases [ASRS \(immediate\)](#), [ASR \(immediate\)](#), [LSLS \(immediate\)](#), [LSL \(immediate\)](#), [LSRS \(immediate\)](#), [LSR \(immediate\)](#), [RORS \(immediate\)](#), [ROR \(immediate\)](#), [RRXS](#), [RRX](#).

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	1	0	D			Rm				Rd

T1 variant

MOV{<c>}{<q>} <Rd>, <Rm>

Decode for this encoding

```
1 d = UInt(D:Rd); m = UInt(Rm); setflags = FALSE;
2 (shift_t, shift_n) = (SRTYPE_LSL, 0);
3 if HaveMainExt() then
4     if d == 15 && InITBlock() && !LastInITBlock() then UNPREDICTABLE;
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	op != 11									Rm			Rd

T2 variant

MOV<c>{<q>} <Rd>, <Rm> {, <shift> #<amount>}
 // Inside IT block
 MOVS{<q>} <Rd>, <Rm> {, <shift> #<amount>}
 // Outside IT block

Decode for this encoding

```
1 if op == '11' then SEE "Related encodings";
2 d = UInt(Rd); m = UInt(Rm); setflags = !InITBlock();
3 (shift_t, shift_n) = DecodeImmShift(op, imm5);
4 if op == '00' && imm5 == '00000' && InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

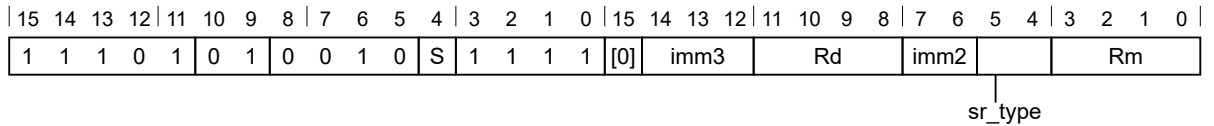
If `op == '00' && imm5 == '00000' && InITBlock()`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passed its condition code check.
- The instruction executes as [NOP](#), as if it failed its condition code check.
- The instruction executes as MOV Rd, Rm.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.



MOV, rotate right with extend variant

Applies when **S == 0** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

MOV{<c>}{<q>} <Rd>, <Rm>, RRX

MOV, shift or rotate by value variant

Applies when **S == 0** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
MOV<c>.W <Rd>, <Rm> {, <shift> #<amount>}
// Inside IT block
// and <Rd>, <Rm>, <shift>, <amount> can be represented in T1 or T2
MOV{<c>}{<q>} <Rd>, <Rm> {, <shift> #<amount>}
```

MOVS, rotate right with extend variant

Applies when **S == 1** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

MOVS{<c>}{<q>} <Rd>, <Rm>, RRX

MOVS, shift or rotate by value variant

Applies when **S == 1** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
MOVS.W <Rd>, <Rm> {, <shift> #<amount>}
// Outside IT block
// and <Rd>, <Rm>, <shift>, <amount> can be represented in T1 or T2
MOVS{<c>}{<q>} <Rd>, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if HasArchVersion(Armv8p1) then
2     if S == '1' && Rm == '11x1' then SEE "Wide shift instructions";
3     if !HaveMainExt() then UNDEFINED;
4     d = UInt(Rd); m = UInt(Rm); setflags = (S == '1');
5     (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
6     if !setflags && (imm3:imm2:sr_type == '0000000') then
7         if (d == 15 || m == 15 || (d == 13 && m == 13)) then UNPREDICTABLE;
8     else
9         if (d IN {13,15} || m IN {13,15}) then UNPREDICTABLE;
```

Alias conditions

Alias	variant	preferred when
ASRS (immediate)	T3	$S == '1' \ \&\& \ sr_type == '10'$
ASRS (immediate)	T2	$op == '10' \ \&\& \ !InITBlock()$
ASR (immediate)	T3	$S == '0' \ \&\& \ sr_type == '10'$
ASR (immediate)	T2	$op == '10' \ \&\& \ InITBlock()$
LSLS (immediate)	T3	$S == '1' \ \&\& \ imm3:Rd:imm2 != '000xxxx00' \ \&\& \ sr_type == '00'$
LSLS (immediate)	T2	$op == '00' \ \&\& \ imm5 != '00000' \ \&\& \ !InITBlock()$
LSL (immediate)	T3	$S == '0' \ \&\& \ imm3:Rd:imm2 != '000xxxx00' \ \&\& \ sr_type == '00'$
LSL (immediate)	T2	$op == '00' \ \&\& \ imm5 != '00000' \ \&\& \ InITBlock()$
LSRS (immediate)	T3	$S == '1' \ \&\& \ sr_type == '01'$
LSRS (immediate)	T2	$op == '01' \ \&\& \ !InITBlock()$
LSR (immediate)	T3	$S == '0' \ \&\& \ sr_type == '01'$
LSR (immediate)	T2	$op == '01' \ \&\& \ InITBlock()$
RORS (immediate)	-	$S == '1' \ \&\& \ imm3:Rd:imm2 != '000xxxx00' \ \&\& \ sr_type == '11'$
ROR (immediate)	-	$S == '0' \ \&\& \ imm3:Rd:imm2 != '000xxxx00' \ \&\& \ sr_type == '11'$
RRXS	-	$S == '1' \ \&\& \ imm3 == '000' \ \&\& \ imm2 == '00' \ \&\& \ sr_type == '11'$
RRX	-	$S == '0' \ \&\& \ imm3 == '000' \ \&\& \ imm2 == '00' \ \&\& \ sr_type == '11'$

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	For encoding T1: is the general-purpose destination register, encoded in the "D:Rd" field. If the PC is used: - The instruction causes a simple branch to the address moved to the PC. - The instruction must either be outside an IT block or the last instruction of an IT block.
<Rm>	For encoding T2 and T3: is the general-purpose destination register, encoded in the "Rd" field. For encoding T1: is the general-purpose source register, encoded in the "Rm" field. The PC can be used.
<shift>	For encoding T2 and T3: is the general-purpose source register, encoded in the "Rm" field. For encoding T2: is the type of shift to be applied to the source register, encoded in the "op" field. It can have the following values: - LSL when $sr_type = 00$ - LSR when $sr_type = 01$ - ASR when $sr_type = 10$ For encoding T3: is the type of shift to be applied to the source register, encoded in the "sr_type" field. It can have the following values: - LSL when $sr_type = 00$ - LSR when $sr_type = 01$ - ASR when $sr_type = 10$ - ROR when $sr_type = 11$
<amount>	For encoding T2: is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm5" field as <amount> modulo 32. For encoding T3: is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     (result, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);
4     if d == 15 then
5         BranchTo(result); // setflags is always FALSE here
6     else
7         RSPCheck[d] = result;
8         if setflags then
9             APSR.N = result[31];
10            APSR.Z = IsZeroBit(result);
11            APSR.C = carry;
12            // APSR.V unchanged

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.121 MOV, MOVS (register-shifted register)

Move (register-shifted register). Move (register-shifted register) copies a register-shifted register value to the destination register. It can optionally update the condition flags based on the value.

This instruction is used by the aliases [ASRS \(register\)](#), [ASR \(register\)](#), [LSLS \(register\)](#), [LSL \(register\)](#), [LSRS \(register\)](#), [LSR \(register\)](#), [RORS \(register\)](#), [ROR \(register\)](#).

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op				Rs			Rdm		

Arithmetic shift right variant

Applies when **op == 0100**.

```
MOV<c>{<q>} <Rdm>, <Rdm>, ASR <Rs>
// Inside IT block
MOVS{<q>} <Rdm>, <Rdm>, ASR <Rs>
// Outside IT block
```

Logical shift left variant

Applies when **op == 0010**.

```
MOV<c>{<q>} <Rdm>, <Rdm>, LSL <Rs>
// Inside IT block
MOVS{<q>} <Rdm>, <Rdm>, LSL <Rs>
// Outside IT block
```

Logical shift right variant

Applies when **op == 0011**.

```
MOV<c>{<q>} <Rdm>, <Rdm>, LSR <Rs>
// Inside IT block
MOVS{<q>} <Rdm>, <Rdm>, LSR <Rs>
// Outside IT block
```

Rotate right variant

Applies when **op == 0111**.

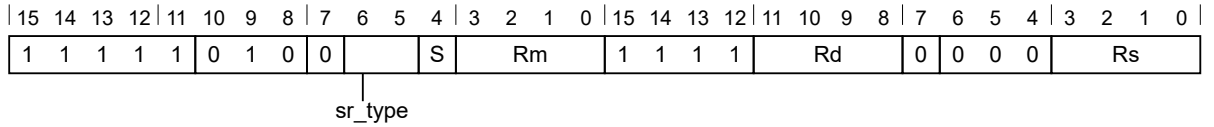
```
MOV<c>{<q>} <Rdm>, <Rdm>, ROR <Rs>
// Inside IT block
MOVS{<q>} <Rdm>, <Rdm>, ROR <Rs>
// Outside IT block
```

Decode for this encoding

```
1 if !(op IN {'0010', '0011', '0100', '0111'}) then SEE "Related encodings";
2 d = UInt(Rdm); m = UInt(Rdm); s = UInt(Rs);
3 setflags = !InITBlock(); shift_t = DecodeRegShift(op[2]:op[0]);
```

T2

Armv8-M Main Extension only



Flag setting variant

Applies when **S == 1**.

```
MOVS.W <Rd>, <Rm>, <sr_type> <Rs>
// Outside IT block, and <Rd>, <Rm>, <sr_type>, <Rs> can be represented in T1
MOVS{<c>}{<q>} <Rd>, <Rm>, <sr_type> <Rs>
```

Non flag setting variant

Applies when **S == 0**.

```
MOV<c>.W <Rd>, <Rm>, <sr_type> <Rs>
// Inside IT block, and <Rd>, <Rm>, <sr_type>, <Rs> can be represented in T1
MOV{<c>}{<q>} <Rd>, <Rm>, <sr_type> <Rs>
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); m = UInt(Rm); s = UInt(Rs);
3 setflags = (S == '1'); shift_t = DecodeRegShift(sr_type);
4 if d IN {13,15} || m IN {13,15} || s IN {13,15} then UNPREDICTABLE;
```

Alias conditions

Alias	variant	preferred when
ASRS (register)	T1 (arithmetic shift right)	op == '0100' && !InITBlock()
ASRS (register)	T2 (flag setting)	sr_type == '10' && S == '1'
ASR (register)	T1 (arithmetic shift right)	op == '0100' && InITBlock()
ASR (register)	T2 (non flag setting)	sr_type == '10' && S == '0'
LSLS (register)	T1 (logical shift left)	op == '0010' && !InITBlock()
LSLS (register)	T2 (flag setting)	sr_type == '00' && S == '1'
LSL (register)	T1 (logical shift left)	op == '0010' && InITBlock()
LSL (register)	T2 (non flag setting)	sr_type == '00' && S == '0'
LSRS (register)	T1 (logical shift right)	op == '0011' && !InITBlock()
LSRS (register)	T2 (flag setting)	sr_type == '01' && S == '1'
LSR (register)	T1 (logical shift right)	op == '0011' && InITBlock()
LSR (register)	T2 (non flag setting)	sr_type == '01' && S == '0'
RORS (register)	T1 (rotate right)	op == '0111' && !InITBlock()
RORS (register)	T2 (flag setting)	sr_type == '11' && S == '1'
ROR (register)	T1 (rotate right)	op == '0111' && InITBlock()
ROR (register)	T2 (non flag setting)	sr_type == '11' && S == '0'

Assembler symbols for all encodings

<c> <q> <Rdm> <Rd> <Rm> <sr_type>	See C1.2.5 Standard assembler syntax fields . See C1.2.5 Standard assembler syntax fields . Is the general-purpose source register and the destination register, encoded in the "Rdm" field. Is the general-purpose destination register, encoded in the "Rd" field. Is the general-purpose source register, encoded in the "Rm" field. Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01
--	--

ASR when `sr_type = 10`
ROR when `sr_type = 11`
<Rs> Is the general-purpose source register holding a shift amount in its bottom 8 bits, encoded in the "Rs" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      shift_n = UInt(R[s][7:0]);
4      (result, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);
5      R[d] = result;
6      if setflags then
7          APSR.N = result[31];
8          APSR.Z = IsZeroBit(result);
9          APSR.C = carry;
10         // APSR.V unchanged

```

Restricted behavior

Data Independent Timing behavior

If `AIRCR.DIT` is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.122 MOV_T

Move Top. Move Top writes an immediate value to the top halfword of the destination register. It does not affect the contents of the bottom halfword.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	1	0	1	1	0	0	imm4				0	imm3				Rd				imm8							

T1 variant

MOV_T{<c>}{<q>} <Rd>, #<imm16>

Decode for this encoding

```
1 d = UInt(Rd); imm16 = imm4:i:imm3:imm8;
2 if d IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <imm16> Is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm4:i:imm3:imm8" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     R[d][31:16] = imm16;
4     // R[d][15:0] unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.123 MRC, MRC2

Move to Register from Coprocessor. Move to Register from Coprocessor causes a coprocessor to transfer a value to a general-purpose register or to the condition flags.

Arm reserves coprocessors CP8 to CP15, and this manual defines the valid instructions when coproc is in this range.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	opc1				1	CRn			Rt			coproc			opc2			1	CRm					

T1 variant

MRC{<c>}{<q>} <coproc>, {#}<opc1>, <Rt>, <CRn>, <CRm>{, {#}<opc2>}

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt);
7 if t == 13 then UNPREDICTABLE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	opc1				1	CRn			Rt			coproc			opc2			1	CRm					

T2 variant

MRC2{<c>}{<q>} <coproc>, {#}<opc1>, <Rt>, <CRn>, <CRm>{, {#}<opc2>}

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt);
7 if t == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoded in the "coproc" field. The generic coprocessor names are p0 to p15.
<opc1>	Is a coprocessor-specific opcode in the range 0 to 7, encoded in the "opc1" field.
<Rt>	Is the general-purpose register to be transferred or APSR_nzcv (encoded as 0b1111), encoded in the "Rt" field. If APSR_nzcv is used, bits [31:28] of the transferred value are written to the APSR condition flags.

<CRn>	Is the first coprocessor register, encoded in the "CRn" field.
<CRm>	Is the second coprocessor register, encoded in the "CRm" field.
<opc2>	Is a coprocessor-specific opcode in the range 0 to 7, defaulting to 0 and encoded in the "opc2" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3
4      if !Coprocc_Accepted(cp, ThisInstr()) then
5          GenerateCoprocc_Exception();
6      else
7          value = Coproc_GetOneWord(cp, ThisInstr());
8          if t != 15 then
9              R[t] = value;
10         else
11             APSR.N = value[31];
12             APSR.Z = value[30];
13             APSR.C = value[29];
14             APSR.V = value[28];
15             // value[27:0] are not used.

```

C2.4.124 MRRC, MRRC2

Move to two Registers from Coprocessor. Move to two Registers from Coprocessor causes a coprocessor to transfer values to two general-purpose registers.

If no coprocessor can execute the instruction, a UsageFault exception is generated.

Arm reserves coprocessors CP8 to CP15, and this manual defines the valid instructions when coproc is in this range.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	1	0	1	Rt2				Rt				coproc				opc1				CRm			

T1 variant

MRRC{<c>}{<q>} <coproc>, {#}<opc1>, <Rt>, <Rt2>, <CRm>

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt); t2 = UInt(Rt2);
7 if t == 15 || t2 == 15 || t == t2 then UNPREDICTABLE;
8 if t == 13 || t2 == 13 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $t == t2$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	0	0	1	0	1	Rt2				Rt				coproc				opc1				CRm			

T2 variant

MRRC2{<c>}{<q>} <coproc>, {#}<opc1>, <Rt>, <Rt2>, <CRm>

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 cp = UInt(coproc);
4 CheckCPDecodeFaults(cp);
5 if !HaveMainExt() then UNDEFINED;
6 t = UInt(Rt); t2 = UInt(Rt2);
7 if t == 15 || t2 == 15 || t == t2 then UNPREDICTABLE;
8 if t == 13 || t2 == 13 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $t == t2$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoded in the "coproc" field. The generic coprocessor names are p0 to p15.
<opc1>	Is a coprocessor-specific opcode in the range 0 to 15, encoded in the "opc1" field.
<Rt>	Is the first general-purpose register to be transferred, encoded in the "Rt" field.
<Rt2>	Is the second general-purpose register to be transferred, encoded in the "Rt2" field.
<CRm>	Is a coprocessor register, encoded in the "CRm" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3
4      if !Coprocc_Accepted(cp, ThisInstr()) then
5          GenerateCoproccorException();
6      else
7          (R[t2], R[t]) = Coproc_GetTwoWords(cp, ThisInstr());

```

C2.4.125 MRS

Move to Register from Special register. Move to Register from Special register moves the value from the selected special-purpose register into a general-purpose register.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	1	1	(0)	(1)	(1)	(1)	(1)	1	0	(0)	0	Rd						SYSm					

T1 variant

MRS{<c>}{<q>} <Rd>, <spec_reg>

Decode for this encoding

```
1 d = UInt (Rd);
2 if d IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<spec_reg>	Is the special register to be accessed, encoded in the "SYSm" field. It can have the following values:
APSR	when SYSm = 00000000
IAPSR	when SYSm = 00000001
EAPSR	when SYSm = 00000010
XPSR	when SYSm = 00000011
IPSR	when SYSm = 00000101
EPSR	when SYSm = 00000110
IEPSR	when SYSm = 00000111
MSP	when SYSm = 00001000
PSP	when SYSm = 00001001
MSPLIM	when SYSm = 00001010
PSPLIM	when SYSm = 00001011
PRIMASK	when SYSm = 00010000
BASEPRI	when SYSm = 00010001
BASEPRI_MAX	when SYSm = 00010010
FAULTMASK	when SYSm = 00010011
CONTROL	when SYSm = 00010100
PAC_KEY_P_0	when SYSm = 00100000
PAC_KEY_P_1	when SYSm = 00100001
PAC_KEY_P_2	when SYSm = 00100010
PAC_KEY_P_3	when SYSm = 00100011
PAC_KEY_U_0	when SYSm = 00100100
PAC_KEY_U_1	when SYSm = 00100101
PAC_KEY_U_2	when SYSm = 00100110
PAC_KEY_U_3	when SYSm = 00100111
MSP_NS	when SYSm = 10001000
PSP_NS	when SYSm = 10001001
MSPLIM_NS	when SYSm = 10001010
PSPLIM_NS	when SYSm = 10001011
PRIMASK_NS	when SYSm = 10010000
BASEPRI_NS	when SYSm = 10010001
FAULTMASK_NS	when SYSm = 10010011
CONTROL_NS	when SYSm = 10010100

SP_NS	when SYSm = 10011000
PAC_KEY_P_0_NS	when SYSm = 10100000
PAC_KEY_P_1_NS	when SYSm = 10100001
PAC_KEY_P_2_NS	when SYSm = 10100010
PAC_KEY_P_3_NS	when SYSm = 10100011
PAC_KEY_U_0_NS	when SYSm = 10100100
PAC_KEY_U_1_NS	when SYSm = 10100101
PAC_KEY_U_2_NS	when SYSm = 10100110
PAC_KEY_U_3_NS	when SYSm = 10100111

The following encodings are UNPREDICTABLE:

- SYSm = 00000100
- SYSm = 000011xx
- SYSm = 00010101
- SYSm = 0001011x
- SYSm = 00011xxx
- SYSm = 00101xxx
- SYSm = 0011xxxx
- SYSm = 01xxxxxx
- SYSm = 10000xxx
- SYSm = 100011xx
- SYSm = 10010010
- SYSm = 10010101
- SYSm = 1001011x
- SYSm = 10011001
- SYSm = 1001101x
- SYSm = 100111xx
- SYSm = 10101xxx
- SYSm = 1011xxxx
- SYSm = 11xxxxxx

An access to a register not ending in _NS returns the register associated with the current Security state. Access to a register ending in _NS in Secure state returns the Non-secure register. Access to a register ending in _NS in Non-secure state is RAZ/WI. Access to BASEPRI_MAX returns the contents of BASEPRI.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      R[d] = Zeros(32);
4
5      // NOTE: the MSB of SYSm is used to select between either the current
6      // domains view of the registers and other domains view of the register.
7      // This is required so that the Secure state can access the Non-secure
8      // versions of banked registers. For security reasons the Secure versions of
9      // the registers are not accessible from the Non-secure state.
10     case SYSm[7:3] of
11         when '0000' // XPSR accesses
12             if UInt(SYSm) == 4 then UNPREDICTABLE;
13             if CurrentModeIsPrivileged() && SYSm[0] == '1' then
14                 R[d][8:0] = IPSR.Exception;
15             if SYSm[1] == '1' then
16                 R[d][26:24] = '000'; // EPSR reads as zero
17                 R[d][21] = '0'; // EPSR.B reads as zero
18                 R[d][15:10] = '000000';
19             if SYSm[2] == '0' then
20                 R[d][31:27] = APSR[31:27];
21                 if HaveDSPExt() then
22                     R[d][19:16] = APSR[19:16];
23         when '00001' // SP access
24             if CurrentModeIsPrivileged() then
25                 case SYSm[2:0] of
26                     when '000'
27                         R[d] = SP_Main;
28                     when '001'
```



```

29         R[d] = SP_Process;
30     when '010'
31         if IsSecure() then
32             R[d] = MSPLIM_S.LIMIT:'000';
33         else
34             if HaveMainExt() then
35                 R[d] = MSPLIM_NS.LIMIT:'000';
36             else
37                 UNPREDICTABLE;
38     when '011'
39         if IsSecure() then
40             R[d] = PSPLIM_S.LIMIT:'000';
41         else
42             if HaveMainExt() then
43                 R[d] = PSPLIM_NS.LIMIT:'000';
44             else
45                 UNPREDICTABLE;
46     otherwise
47         UNPREDICTABLE;
48 when '10001' // SP access - alt domain
49     if !HaveSecurityExt() then UNPREDICTABLE;
50     if CurrentModeIsPrivileged() && CurrentState == SecurityState_Secure then
51         case SYSm[2:0] of
52             when '000'
53                 R[d] = SP_Main_NonSecure;
54             when '001'
55                 R[d] = SP_Process_NonSecure;
56             when '010'
57                 if HaveMainExt() then
58                     R[d] = MSPLIM_NS.LIMIT:'000';
59                 else
60                     UNPREDICTABLE;
61             when '011'
62                 if HaveMainExt() then
63                     R[d] = PSPLIM_NS.LIMIT:'000';
64                 else
65                     UNPREDICTABLE;
66             otherwise
67                 UNPREDICTABLE;
68 when '00010' // Priority mask or CONTROL access
69     case SYSm[2:0] of
70         when '000'
71             if CurrentModeIsPrivileged() then
72                 R[d][0] = PRIMASK.PM;
73         when '001'
74             if HaveMainExt() then
75                 if CurrentModeIsPrivileged() then
76                     R[d][7:0] = BASEPRI[7:0];
77             else
78                 UNPREDICTABLE;
79         when '010'
80             if HaveMainExt() then
81                 if CurrentModeIsPrivileged() then
82                     R[d][7:0] = BASEPRI[7:0];
83             else
84                 UNPREDICTABLE;
85         when '011'
86             if HaveMainExt() then
87                 if CurrentModeIsPrivileged() then
88                     R[d][0] = FAULTMASK.FM;
89             else
90                 UNPREDICTABLE;
91         when '100'
92             if HaveMveOrFPEExt() && IsSecure() then
93                 R[d][3:0] = CONTROL[3:0];
94             elsif HaveMveOrFPEExt() then
95                 R[d][2:0] = CONTROL[2:0];
96             else
97                 R[d][1:0] = CONTROL[1:0];

```

```

98         // PACBTI Extension required for accessing the PAC and BTI enables
99         if HavePACBTIExt() then
100             if CurrentModeIsPrivileged() then
101                 R[d][7:4] = CONTROL[7:4]; // UPAC_EN, PAC_EN, UBTI_EN, BTI_EN
102             else
103                 R[d][7] = CONTROL[7]; // UPAC_EN
104                 R[d][5] = CONTROL[5]; // UBTI_EN
105             otherwise
106                 UNPREDICTABLE;
107     when '00100' // Access to PAC keys
108         if !HavePACBTIExt() then UNPREDICTABLE; // PACBTI Extension required
109         if CurrentModeIsPrivileged() then
110             R[d] = PACKeyReg[SYSm[2:0], IsSecure()];
111     when '10010' // Priority mask or CONTROL access - alt domain
112         if !HaveSecurityExt() then UNPREDICTABLE;
113         if CurrentState == SecurityState_Secure then
114             case SYSm[2:0] of
115                 when '000'
116                     if CurrentModeIsPrivileged() then
117                         R[d][0] = PRIMASK_NS.PM;
118                 when '001'
119                     if HaveMainExt() then
120                         if CurrentModeIsPrivileged() then
121                             R[d][7:0] = BASEPRI_NS[7:0];
122                     else
123                         UNPREDICTABLE;
124                 when '011'
125                     if HaveMainExt() then
126                         if CurrentModeIsPrivileged() then
127                             R[d][0] = FAULTMASK_NS.FM;
128                     else
129                         UNPREDICTABLE;
130                 when '100'
131                     if HaveMveOrFPEExt() then
132                         R[d][2:0] = CONTROL_NS[2:0];
133                     else
134                         R[d][1:0] = CONTROL_NS[1:0];
135                     // PACBTI Extension required for accessing the PAC and BTI enables
136                     if HavePACBTIExt() then
137                         if CurrentModeIsPrivileged() then
138                             R[d][7:4] = CONTROL_NS[7:4]; // UPAC_EN, PAC_EN,
139                                                         // UBTI_EN, BTI_EN
140                         else
141                             R[d][7] = CONTROL_NS[7]; // UPAC_EN
142                             R[d][5] = CONTROL_NS[5]; // UBTI_EN
143                     otherwise
144                         UNPREDICTABLE;
145     when '10011' // SP_NS - Non-secure stack pointer
146         if !HaveSecurityExt() then UNPREDICTABLE;
147         if CurrentState == SecurityState_Secure then
148             case SYSm[2:0] of
149                 when '000'
150                     R[d] = _SP(LookUpSP_with_security_mode(FALSE, CurrentMode()));
151                 otherwise
152                     UNPREDICTABLE;
153     when '10100' // Access to Non-secure PAC keys
154         if !HaveSecurityExt() || !HavePACBTIExt() then UNPREDICTABLE;
155         if CurrentModeIsPrivileged() && CurrentState == SecurityState_Secure then
156             R[d] = PACKeyReg[SYSm[2:0], FALSE];
157     otherwise
158         UNPREDICTABLE;

```

CONSTRAINED UNPREDICTABLE behavior

If SYSm not valid special register, then one of the following behaviors must occur:

- The instruction is UNDEFINED.

- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Restricted behavior

Data Independent Timing behavior and the PACBTI extension

If the PACBTI extension is implemented the timing of this instruction is insensitive to the value of the data being loaded or stored regardless of the value of [AIRCR.DIT](#) when accessing any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).

C2.4.126 MSR (register)

Move to Special register from Register. Move to Special register from Register moves the value of a general-purpose register to the specified special-purpose register.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	0	1	1	1	0	0	(0)	Rn	1	0	(0)	0	mask	(0)	(0)													SYSm

T1 variant

MSR{<c>}{<q>} <spec_reg>, <Rn>

Decode for this encoding

```

1 n = UInt(Rn);
2 if HaveMainExt() then
3     if mask == '00' || (mask != '10' && !(UInt(SYSm) IN {0..3})) then UNPREDICTABLE;
4 else
5     if mask != '10' then UNPREDICTABLE;
6 if n IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If combination of SYSm and mask not supported, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction treats mask and SYSm as UNKNOWN.

Encoding conditions

<bits>	Effect	mask encoding	Notes
_nzcvcq	Write the N, Z, C, V, Q bits, APSR[31:27]	0b10	Always supported
_g	Write the GE[3:0] bits, APSR[19:16]	0b01	Supported only if the PE
_nzcvcqg	Write the N, Z, C, V, Q, GE[3:0] bits	0b11	implements the DSP extension.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .	
<q>	See C1.2.5 Standard assembler syntax fields .	
<spec_reg>	Is the special register to be accessed, encoded in the "SYSm" field. It can have the following values:	
	APSR	when SYSm = 00000000
	IAPSR	when SYSm = 00000001
	EAPSR	when SYSm = 00000010
	XPSR	when SYSm = 00000011
	IPSR	when SYSm = 00000101
	EPSR	when SYSm = 00000110
	IEPSR	when SYSm = 00000111
	MSP	when SYSm = 00001000
	PSP	when SYSm = 00001001
	MSPLIM	when SYSm = 00001010
	PSPLIM	when SYSm = 00001011
	PRIMASK	when SYSm = 00010000

BASEPRI	when SYSm = 00010001
BASEPRI_MAX	when SYSm = 00010010
FAULTMASK	when SYSm = 00010011
CONTROL	when SYSm = 00010100
PAC_KEY_P_0	when SYSm = 00100000
PAC_KEY_P_1	when SYSm = 00100001
PAC_KEY_P_2	when SYSm = 00100010
PAC_KEY_P_3	when SYSm = 00100011
PAC_KEY_U_0	when SYSm = 00100100
PAC_KEY_U_1	when SYSm = 00100101
PAC_KEY_U_2	when SYSm = 00100110
PAC_KEY_U_3	when SYSm = 00100111
MSP_NS	when SYSm = 10001000
PSP_NS	when SYSm = 10001001
MSPLIM_NS	when SYSm = 10001010
PSPLIM_NS	when SYSm = 10001011
PRIMASK_NS	when SYSm = 10010000
BASEPRI_NS	when SYSm = 10010001
FAULTMASK_NS	when SYSm = 10010011
CONTROL_NS	when SYSm = 10010100
SP_NS	when SYSm = 10011000
PAC_KEY_P_0_NS	when SYSm = 10100000
PAC_KEY_P_1_NS	when SYSm = 10100001
PAC_KEY_P_2_NS	when SYSm = 10100010
PAC_KEY_P_3_NS	when SYSm = 10100011
PAC_KEY_U_0_NS	when SYSm = 10100100
PAC_KEY_U_1_NS	when SYSm = 10100101
PAC_KEY_U_2_NS	when SYSm = 10100110
PAC_KEY_U_3_NS	when SYSm = 10100111

The following encodings are UNPREDICTABLE:

- SYSm = 00000100
- SYSm = 000011xx
- SYSm = 00010101
- SYSm = 0001011x
- SYSm = 00011xxx
- SYSm = 00101xxx
- SYSm = 0011xxxx
- SYSm = 01xxxxxx
- SYSm = 10000xxx
- SYSm = 100011xx
- SYSm = 10010010
- SYSm = 10010101
- SYSm = 1001011x
- SYSm = 10011001
- SYSm = 1001101x
- SYSm = 100111xx
- SYSm = 10101xxx
- SYSm = 1011xxxx
- SYSm = 11xxxxxx

An access to a register not ending in _NS returns the register associated with the current Security state. Access to a register ending in _NS in Secure state returns the Non-secure register. Access to a register ending in _NS in Non-secure state is RAZ/WI. Access to BASEPRI_MAX writes to BASEPRI if the priority that is written is higher than the existing priority in BASEPRI. Otherwise, the access is ignored.

<Rn>

Is the general-purpose source register, encoded in the "Rn" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     // NOTE: the MSB of SYSm is used to select between either the current

```

```

5 // domains view of the registers and other domains view of the register.
6 // This is required to that the Secure state can access the Non-secure
7 // versions of banked registers. For security reasons the Secure versions of
8 // the registers are not accessible from the Non-secure state.
9 case SYSm[7:3] of
10     when '00000' // XPSR accesses
11         if UInt(SYSm) == 4 then UNPREDICTABLE;
12         if SYSm[2] == '0' then // Include APSR
13             if mask[0] == '1' then // GE[3:0] bits
14                 if !HaveDSPEExt() then
15                     UNPREDICTABLE;
16                 else
17                     APSR[19:16] = R[n][19:16];
18             if mask[1] == '1' then // N, Z, C, V, Q bits
19                 APSR[31:27] = R[n][31:27];
20     when '00001' // SP access
21         if CurrentModeIsPrivileged() then
22             case SYSm[2:0] of
23                 when '000'
24                     // MSR not subject to SP limit, write directly to register.
25                     if IsSecure() then
26                         exc = _SP(RNamesSP_Main_Secure, FALSE, TRUE, R[n][31:2]:'00');
27                     else
28                         exc = _SP(RNamesSP_Main_NonSecure, FALSE, TRUE, R[n][31:2]:'00');
29                     assert exc.fault == NoFault;
30                 when '001'
31                     // MSR not subject to SP limit, write directly to register.
32                     if IsSecure() then
33                         exc = _SP(RNamesSP_Process_Secure, FALSE, TRUE, R[n][31:2]:'00');
34                     else
35                         exc = _SP(RNamesSP_Process_NonSecure, FALSE, TRUE,
36                             R[n][31:2]:'00');
37                     assert exc.fault == NoFault;
38                 when '010'
39                     if IsSecure() then
40                         MSPLIM_S.LIMIT = R[n][31:3];
41                     else
42                         if HaveMainExt() then
43                             MSPLIM_NS.LIMIT = R[n][31:3];
44                         else
45                             UNPREDICTABLE;
46                 when '011'
47                     if IsSecure() then
48                         PSPLIM_S.LIMIT = R[n][31:3];
49                     else
50                         if HaveMainExt() then
51                             PSPLIM_NS.LIMIT = R[n][31:3];
52                         else
53                             UNPREDICTABLE;
54             otherwise
55                 UNPREDICTABLE;
56     when '10001' // SP access - alt domain
57         if !HaveSecurityExt() then UNPREDICTABLE;
58         if CurrentModeIsPrivileged() && CurrentState == SecurityState_Secure then
59             case SYSm[2:0] of
60                 when '000'
61                     // MSR not subject to SP limit, write directly to register.
62                     exc = _SP(RNamesSP_Main_NonSecure, FALSE, TRUE, R[n][31:2]:'00');
63                     assert exc.fault == NoFault;
64                 when '001'
65                     // MSR not subject to SP limit, write directly to register.
66                     exc = _SP(RNamesSP_Process_NonSecure, FALSE, TRUE, R[n][31:2]:'00');
67                     assert exc.fault == NoFault;
68                 when '010'
69                     if HaveMainExt() then
70                         MSPLIM_NS.LIMIT = R[n][31:3];
71                     else
72                         UNPREDICTABLE;
73                 when '011'

```

```

74         if HaveMainExt() then
75             PSPLIM_NS.LIMIT = R[n][31:3];
76         else
77             UNPREDICTABLE;
78         otherwise
79             UNPREDICTABLE;
80     when '00010' // Priority mask or CONTROL access
81     case SYSm[2:0] of
82         when '000'
83             if CurrentModeIsPrivileged() then
84                 PRIMASK.PM = R[n][0];
85         when '001'
86             if CurrentModeIsPrivileged() then
87                 if HaveMainExt() then
88                     BASEPRI[7:0] = R[n][7:0];
89                 else
90                     UNPREDICTABLE;
91         when '010'
92             if CurrentModeIsPrivileged() then
93                 if HaveMainExt() then
94                     if ((R[n][7:0] != '00000000') &&
95                        (UInt(R[n][7:0]) < UInt(BASEPRI[7:0]) ||
96                         BASEPRI[7:0] == '00000000')) then
97                         BASEPRI[7:0] = R[n][7:0];
98                     else
99                         UNPREDICTABLE;
100        when '011'
101            if CurrentModeIsPrivileged() then
102                if HaveMainExt() then
103                    if ExecutionPriority() > -1 || R[n][0] == '0' then
104                        FAULTMASK.FM = R[n][0];
105                else
106                    UNPREDICTABLE;
107        when '100'
108            if CurrentModeIsPrivileged() then
109                CONTROL.nPRIV = R[n][0];
110                CONTROL.SPSEL = R[n][1];
111                if HaveMveOrFPEExt() && (IsSecure() || NSACR.CP10 == '1') then
112                    CONTROL.FPCA = R[n][2];
113                    // PACBTI Extension required for accessing the PAC and BTI enables
114                    if HavePACBTIExt() then
115                        CONTROL[7:4] = R[n][7:4]; // UPAC_EN, PAC_EN, UBTI_EN, BTI_EN
116                if HaveMveOrFPEExt() && IsSecure() then
117                    CONTROL_S.SFPA = R[n][3];
118            otherwise
119                UNPREDICTABLE;
120    when '00100' // Access to PAC keys
121        if !HavePACBTIExt() then UNPREDICTABLE; // PACBTI Extension required
122        if CurrentModeIsPrivileged() then
123            PACKeyReg[SYSm[2:0], IsSecure()] = R[n];
124    when '10010' // Priority mask or CONTROL access - alt domain
125        if !HaveSecurityExt() then UNPREDICTABLE;
126        if CurrentModeIsPrivileged() && CurrentState == SecurityState_Secure then
127            case SYSm[2:0] of
128                when '000'
129                    PRIMASK_NS.PM = R[n][0];
130                when '001'
131                    if HaveMainExt() then
132                        BASEPRI_NS[7:0] = R[n][7:0];
133                    else
134                        UNPREDICTABLE;
135                when '011'
136                    if HaveMainExt() then
137                        if ExecutionPriority() > -1 || R[n][0] == '0' then
138                            FAULTMASK_NS.FM = R[n][0];
139                    else
140                        UNPREDICTABLE;
141                when '100'
142                    CONTROL_NS.nPRIV = R[n][0];

```

```

143         CONTROL_NS.SPSEL = R[n][1];
144         if HaveMveOrFPExt() then CONTROL_NS.FPCA = R[n][2];
145         // PACBTI Extension required for accessing the PAC and BTI enables
146         if HavePACBTIExt() then
147             CONTROL_NS[7:4] = R[n][7:4]; // UPAC_EN, PAC_EN, UBTI_EN, BTI_EN
148         otherwise
149             UNPREDICTABLE;
150     when '10011' // SP_NS - Non-secure stack pointer
151         if !HaveSecurityExt() then UNPREDICTABLE;
152         if CurrentState == SecurityState_Secure then
153             case SYSm[2:0] of
154                 when '000'
155                     spName = LookUpSP_with_security_mode(FALSE, CurrentMode());
156                     // MSR SP_NS is subject to SP limit check. If a STKOF UsageFault is
157                     // generated then it can target either the Secure or Non-secure state
158                     - = _SP(spName, FALSE, FALSE, R[n]);
159                 otherwise
160                     UNPREDICTABLE;
161     when '10100' // Access to Non-secure PAC keys
162         if !HaveSecurityExt() || !HavePACBTIExt() then UNPREDICTABLE;
163         if CurrentModeIsPrivileged() && CurrentState == SecurityState_Secure then
164             PACKeyReg[SYSm[2:0], FALSE] = R[n];
165     otherwise
166         UNPREDICTABLE;

```

CONSTRAINED UNPREDICTABLE behavior

If SYSm not valid special register, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction treats SYSm as UNKNOWN.

Restricted behavior

Data Independent Timing behavior and the PACBTI extension

If the PACBTI extension is implemented the timing of this instruction is insensitive to the value of the data being loaded or stored regardless of the value of [AIRCR.DIT](#) when accessing any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).

C2.4.127 MUL

Multiply. Multiply multiplies two register values. The least significant 32 bits of the result are written to the destination register. These 32 bits do not depend on whether signed or unsigned calculations are performed.

It can optionally update the condition flags based on the result. In the T32 instruction set, this option is limited to only a few forms of the instruction. Use of this option adversely affects performance on many implementations.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	1	1	0	1	Rn			Rdm		

T1 variant

```
MUL<c>{<q>} <Rdm>, <Rn>{, <Rdm>}
    // Inside IT block
MULS{<q>} <Rdm>, <Rn>{, <Rdm>}
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rdm); n = UInt(Rn); m = UInt(Rdm); setflags = !InITBlock();
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	0	0	0	Rn				1	1	1	1	Rd				0	0	0	0	Rm			

T2 variant

```
MUL<c>.W <Rd>, <Rn>{, <Rm>}
    // Inside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
MUL{<c>}{<q>} <Rd>, <Rn>{, <Rm>}
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = FALSE;
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the second general-purpose source register holding the multiplier and the destination register, encoded in the "Rdm" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
	If omitted, <Rd> is used.

Operation for all encodings

```
1 if ConditionPassed() then
```

```

2      EncodingSpecificOperations();
3      operand1 = SInt(R[n]); // operand1 = UInt(R[n]) produces the same final results
4      operand2 = SInt(R[m]); // operand2 = UInt(R[m]) produces the same final results
5      result = operand1 * operand2;
6      R[d] = result[31:0];
7      if setflags then
8          APSR.N = result[31];
9          APSR.Z = IsZeroBit(result[31:0]);
10         // APSR.C unchanged
11         // APSR.V unchanged

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.128 MVN (immediate)

Bitwise NOT (immediate). Bitwise NOT (immediate) writes the bitwise inverse of an immediate value to the destination register. It can optionally update the condition flags based on the value.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	0	0	1	1	S	1	1	1	1	0	imm3				Rd				imm8							

MVN variant

Applies when **S == 0**.

MVN{<c>}{<q>} <Rd>, #<const>

MVNS variant

Applies when **S == 1**.

MVNS{<c>}{<q>} <Rd>, #<const>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); setflags = (S == '1');
3 (imm32, carry) = T32ExpandImm_C(i:imm3:imm8, APSR.C);
4 if d IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = NOT(imm32);
4     R[d] = result;
5     if setflags then
6         APSR.N = result[31];
7         APSR.Z = IsZeroBit(result);
8         APSR.C = carry;
9         // APSR.V unchanged
```

C2.4.129 MVN (register)

Bitwise NOT (register). Bitwise NOT (register) writes the bitwise inverse of a register value to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	1	1	1	1	Rm					Rd

T1 variant

```
MVN<c>{<q>} <Rd>, <Rm>
    // Inside IT block
MVNS{<q>} <Rd>, <Rm>
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rd); m = UInt(Rm); setflags = !InITBlock();
2 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	1	S	1	1	1	1	(0)	imm3														Rm

sr_type

MVN, rotate right with extend variant

Applies when **S == 0** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
MVN{<c>}{<q>} <Rd>, <Rm>, RRX
```

MVN, shift or rotate by value variant

Applies when **S == 0** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
MVN<c>.W <Rd>, <Rm>
    // Inside IT block, and <Rd>, <Rm> can be represented in T1
MVN{<c>}{<q>} <Rd>, <Rm> {, <shift> #<amount>}
```

MVNS, rotate right with extend variant

Applies when **S == 1** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
MVNS{<c>}{<q>} <Rd>, <Rm>, RRX
```

MVNS, shift or rotate by value variant

Applies when **S == 1** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
MVNS.W <Rd>, <Rm>
    // Outside IT block, and <Rd>, <Rm> can be represented in T1
MVNS{<c>}{<q>} <Rd>, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); m = UInt(Rm); setflags = (S == '1');
3 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
4 if d IN {13,15} || m IN {13,15} then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> Is the general-purpose source register, encoded in the "Rm" field.
 <shift> Is the type of shift to be applied to the source register, encoded in the "sr_type" field. It can have the following values:
 LSL when sr_type = 00
 LSR when sr_type = 01
 ASR when sr_type = 10
 ROR when sr_type = 11
 <amount> Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     (shifted, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);
4     result = NOT(shifted);
5     R[d] = result;
6     if setflags then
7         APSR.N = result[31];
8         APSR.Z = IsZeroBit(result);
9         APSR.C = carry;
10        // APSR.V unchanged

```

C2.4.130 NOP

No Operation. No Operation does nothing.

The architecture makes no guarantees about any timing effects of including a NOP instruction.

This is a NOP-compatible hint.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	0	0	0	0	0	0	0

T1 variant

NOP {<c>} {<q>}

Decode for this encoding

```
1 // No additional decoding required
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	0	0	0	0	0

T2 variant

NOP {<c>} .W

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 // No additional decoding required
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).

<q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     // Do nothing
```

C2.4.131 ORN (immediate)

Logical OR NOT (immediate). Logical OR NOT (immediate) performs a bitwise (inclusive) OR of a register value and the complement of an immediate value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	0	0	1	1	S	Rn				0	imm3				Rd				imm8							

Flag setting variant

Applies when **S == 1**.

ORNS{<c>}{<q>} {<Rd>}, {<Rn>}, #<const>

Non flag setting variant

Applies when **S == 0**.

ORN{<c>}{<q>} {<Rd>}, {<Rn>}, #<const>

Decode for this encoding

```

1 if Rn == '1111' then SEE "MVN (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1');
4 (imm32, carry) = T32ExpandImm_C(i:imm3:imm8, APSR.C);
5 if d IN {13,15} || n == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = R[n] OR NOT(imm32);
4     R[d] = result;
5     if setflags then
6         APSR.N = result[31];
7         APSR.Z = IsZeroBit(result);
8         APSR.C = carry;
9         // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

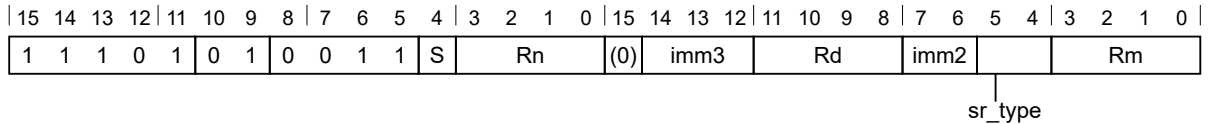
- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.132 ORN (register)

Logical OR NOT (register). Logical OR NOT (register) performs a bitwise (inclusive) OR of a register value and the complement of an optionally-shifted register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M Main Extension only



ORN, rotate right with extend variant

Applies when **S == 0** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

ORN{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>, RRX

ORN, shift or rotate by value variant

Applies when **S == 0** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

ORN{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm> {, <shift> #<amount>}

ORNS, rotate right with extend variant

Applies when **S == 1** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

ORNS{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>, RRX

ORNS, shift or rotate by value variant

Applies when **S == 1** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

ORNS{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm> {, <shift> #<amount>}

Decode for this encoding

```

1 if Rn == '1111' then SEE "MVN (register)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
4 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
5 if d IN {13,15} || n == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then  
2     EncodingSpecificOperations();  
3     (shifted, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);  
4     result = R[n] OR NOT(shifted);  
5     R[d] = result;  
6     if setflags then  
7         APSR.N = result[31];  
8         APSR.Z = IsZeroBit(result);  
9         APSR.C = carry;  
10        // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.133 ORR (immediate)

Logical OR (immediate). Logical OR (immediate) performs a bitwise (inclusive) OR of a register value and an immediate value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	0	0	1	0	S	Rn				0	imm3				Rd				imm8							

ORR variant

Applies when **S** == 0.

ORR{<c>}{<q>} {<Rd>}, {<Rn>, #<const>

ORRS variant

Applies when **S** == 1.

ORRS{<c>}{<q>} {<Rd>}, {<Rn>, #<const>

Decode for this encoding

```

1 if Rn == '1111' then SEE "MOV (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1');
4 (imm32, carry) = T32ExpandImm_C(i:imm3:imm8, APSR.C);
5 if d IN {13,15} || n == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = R[n] OR imm32;
4     R[d] = result;
5     if setflags then
6         APSR.N = result[31];
7         APSR.Z = IsZeroBit(result);
8         APSR.C = carry;
9         // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.134 ORR (register)

Logical OR (register). Logical OR (register) performs a bitwise (inclusive) OR of a register value and an optionally-shifted register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	1	1	0	0	Rm					Rdn

T1 variant

```
ORR<c>{<q>} {<Rdn>}, <Rdn>, <Rm>
// Inside IT block
ORRS{<q>} {<Rdn>}, <Rdn>, <Rm>
// Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rdn); n = UInt(Rdn); m = UInt(Rm); setflags = !InITBlock();
2 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	S		Rn		[0]	imm3		Rd		imm2									Rm		

sr type

ORR, rotate right with extend variant

Applies when **S == 0** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
ORR{<c>}{<q>} {<Rd>}, <Rn>, <Rm>, RRX
```

ORR, shift or rotate by value variant

Applies when **S == 0** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
ORR<c>.W {<Rd>}, <Rn>, <Rm>
// Inside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
ORR{<c>}{<q>} {<Rd>}, <Rn>, <Rm> {, <shift> #<amount>}
```

ORRS, rotate right with extend variant

Applies when **S == 1** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
ORRS{<c>}{<q>} {<Rd>}, <Rn>, <Rm>, RRX
```

ORRS, shift or rotate by value variant

Applies when **S == 1** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
ORRS.W {<Rd>,} <Rn>, <Rm>
// Outside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
ORRS{<c>}{<q>} {<Rd>,} <Rn>, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if HasArchVersion(Armv8p1) then
2   if S == '1' && Rm == '11x1' then SEE "Wide shift instructions";
3   if Rn == '1111' then SEE "MOV (register)";
4   if !HaveMainExt() then UNDEFINED;
5   d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
6   (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
7   if d IN {13,15} || n == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdn>	Is the first general-purpose source register and the destination register, encoded in the "Rdn" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (shifted, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);
4   result = R[n] OR shifted;
5   R[d] = result;
6   if setflags then
7     APSR.N = result[31];
8     APSR.Z = IsZeroBit(result);
9     APSR.C = carry;
10    // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.135 PAC

Pointer Authentication Code for the link register, using key. This instruction computes a pointer authentication code using LR as the address, SP as the modifier and, a key. The computed PAC is written to R12. The intended use of this instruction is for signing return addresses where the link register value should match the address used for function return.

This instruction is not permitted in an IT block.

This is a NOP-compatible hint.

T1

Armv8.1-M PACBTI

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	1	1	1	0	1

T1 variant

PAC R12, LR, SP

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes unconditionally.
- The instruction executes conditionally.

Assembler symbols for all encodings

LR	LR register, this instruction must always use this register. This register holds the address.
SP	SP register, this instruction must always use this register. This register holds the modifier to apply during the PAC code generation.
R12	R12 register, this instruction must always use this register. This register holds PAC for comparison.

Operation for all encodings

```
1 EncodingSpecificOperations();
2
3 if PACEnabled() then
4     R[12] = CreatePAC(LR, SP);
```

Restricted behavior

Data Independent Timing behavior and the PACBTI extension

The timing of this instruction is insensitive to the value of the data being loaded or stored regardless of the value of AIRCR.DIT when accessing any of the following:

- PAC_KEY_P_n.

- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

The point at which an asynchronous exception is taken does not vary based on any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

C2.4.136 PACBTI

Pointer Authentication Code for the link register with BTI clearing, using key. This instruction computes a pointer authentication code using LR as the address, SP as the modifier and, a key. The computed PAC is written to R12. The intended use of this instruction is for signing return addresses where the link register value should match the address used for function return.

This instruction is a valid BTI landing pad when the PACBTI Extension is implemented and EPSR.B is one.

This instruction is not permitted in an IT block.

This is a NOP-compatible hint.

T1

Armv8.1-M PACBTI

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	0	1	1	0	1

T1 variant

PACBTI R12, LR, SP

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes unconditionally.
- The instruction executes conditionally.

Assembler symbols for all encodings

LR	LR register, this instruction must always use this register. This register holds the address.
SP	SP register, this instruction must always use this register. This register holds the modifier to apply during the PAC code generation.
R12	R12 register, this instruction must always use this register. This register holds PAC for comparison.

Operation for all encodings

```
1 EncodingSpecificOperations();
2
3 // This instruction can be used as a BTI landing pad.
4 // The EPSR.B bit is reset to zero.
5 EPSR.B = '0';
6
7 if PACEnabled() then
8     R[12] = CreatePAC(LR, SP);
```

Restricted behavior

Data Independent Timing behavior and the PACBTI extension

The timing of this instruction is insensitive to the value of the data being loaded or stored regardless of the value of [AIRCR.DIT](#) when accessing any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

The point at which an asynchronous exception is taken does not vary based on any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

C2.4.137 PACG

Pointer Authentication Code for a general value, using key. This instruction computes a pointer authentication code using two input registers and a key. The computed PAC is written to the specified register.

T1

Armv8.1-M PACBTI

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	1	1	0	Rn	1	1	1	1	Rd	0	0	0	0	Rm									

T1 variant

PACG{<c>} <Rd>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HavePACBTIExt() then
2     if HaveDSPEExt() then UNPREDICTABLE; else UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
4 if n == 15 || m == 15 || d IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose source register containing the value to be encrypted, encoded in the "Rn" field.
<Rm>	Is the general-purpose source register containing the modifier to be used in the encryption operation, encoded in the "Rm" field.
<Rd>	Is the general-purpose destination register which will contain the encrypted value, encoded in the "Rd" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     // Create PAC and store it in the selected register.
4     if PACEnabled() then
5         R[d] = CreatePAC(R[n], R[m]);
```

Restricted behavior

Data Independent Timing behavior and the PACBTI extension

The timing of this instruction is insensitive to the value of the data being loaded or stored regardless of the value of [AIRCR.DIT](#) when accessing any of the following:

- [PAC_KEY_P_n](#).
- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

The point at which an asynchronous exception is taken does not vary based on any of the following:

- [PAC_KEY_P_n](#).

- [PAC_KEY_U_n](#).
- The pointer value.
- The modifier value.
- The PAC value.

C2.4.138 PKHBT, PKHTB

Pack Halfword. Pack Halfword combines one halfword of its first operand with the other halfword of its shifted second operand.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	1	1	0	0	Rn				(0)	imm3				Rd				imm2	tb	0	Rm			

PKHBT variant

Applies when **tb** == 0.

```
PKHBT{<c>}{<q>} {<Rd>}, <Rn>, <Rm> {, LSL #<imm>}
// tbform == FALSE
```

PKHTB variant

Applies when **tb** == 1.

```
PKHTB{<c>}{<q>} {<Rd>}, <Rn>, <Rm> {, ASR #<imm>}
// tbform == TRUE
```

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); tbform = (tb == '1');
3 (shift_t, shift_n) = DecodeImmShift(tb:'0', imm3:imm2);
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<imm>	The shift to apply to the value read from <Rm>, encoded in imm3:imm2. For PKHBT, it is one of: 1-31 Left shift by specified number of bits, encoded as a binary number. - omitted No shift, encoded as 0b00000.
<imm>	The shift to apply to the value read from <Rm>, encoded in imm3:imm2. For PKHTB, it is one of: 1-32 Arithmetic right shift by specified number of bits. A shift by 32 bits is encoded as 0b00000. Other shift amounts are encoded as binary numbers. - omitted Instruction is a pseudo-instruction and is assembled as though PKHBT<c><q><Rd>, <Rm>, <Rn> had been written. For both variants an assembler can permit <imm> = 0 to mean the same thing as omitting the shift, but this is not standard UAL and must not be used for disassembly.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   operand2 = Shift(R[m], shift_t, shift_n, APSR.C); // APSR.C ignored
4   bits(32) result;
5   result[15:0] = if tbform then operand2[15:0] else R[n][15:0];
```

```
6      result[31:16] = if tbform then R[n][31:16]      else operand2[31:16];  
7      R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.139 PLD (literal)

Preload Data (literal). Preload Data signals the memory system that data memory accesses from a specified address are likely in the near future. The memory system can respond by taking actions that are expected to speed up the memory accesses when they do occur, such as pre-loading the cache line containing the specified address into the data cache.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	U	0	(0)	1	1	1	1	1	1	1	1	1	imm12											

T1 variant

```
PLD{<c>}{<q>} <label>
    // Preferred syntax
PLD{<c>}{<q>} [PC, #<+/-><imm>]
    // Alternative syntax
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 imm32 = ZeroExtend(imm12, 32); add = (U == '1');
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<label>	The label of the literal data item that is likely to be accessed in the near future. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. The offset must be in the range -4095 to 4095. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE. If the offset is negative, imm32 is equal to minus the offset and add == FALSE.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	Is a 12-bit unsigned immediate byte offset, in the range 0 to 4095, encoded in the "imm12" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = if add then (Align(PC, 4) + imm32) else (Align(PC, 4) - imm32);
4     Hint_PreloadData(address);
```

C2.4.140 PLD, PLDW (immediate)

Preload Data (immediate). Preload Data signals the memory system that data memory accesses from a specified address are likely in the near future. The memory system can respond by taking actions that are expected to speed up the memory accesses when they do occur, such as pre-loading the cache line containing the specified address into the data cache.

The PLD instruction signals that the likely memory access is a read, and the PLDW instruction signals that it is a write.

The effect of a PLD or PLDW is IMPLEMENTATION DEFINED.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	1	0	W	1	Rn				1	1	1	1	imm12											

Preload read variant

Applies when **W** == 0.

PLD{<c>}{<q>} [<Rn> {, #<+><imm>}]

Preload write variant

Applies when **W** == 1.

PLDW{<c>}{<q>} [<Rn> {, #<+><imm>}]

Decode for this encoding

```
1 if Rn == '1111' then SEE "PLD (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 n = UInt(Rn); imm32 = ZeroExtend(imm12, 32); add = TRUE; is_pldw = (W == '1');
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	W	1	Rn				1	1	1	1	1	1	0	0	imm8							

Preload read variant

Applies when **W** == 0.

PLD{<c>}{<q>} [<Rn> {, #-<imm>}]

Preload write variant

Applies when **W** == 1.

PLDW{<c>}{<q>} [<Rn> {, #-<imm>}]

Decode for this encoding

```
1 if Rn == '1111' then SEE "PLD (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 n = UInt(Rn); imm32 = ZeroExtend(imm8, 32); add = FALSE; is_pldw = (W == '1');
```


Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field. If the PC is used, see C2.4.139 PLD (literal) .
+	Specifies the offset is added to the base register.
<imm>	For encoding T1: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T2: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = if add then (R[n] + imm32) else (R[n] - imm32);
4     if is_pldw then
5         Hint_PreloadDataForWrite(address);
6     else
7         Hint_PreloadData(address);
```

C2.4.141 PLD, PLDW (register)

Preload Data (register). Preload Data is a memory hint instruction that can signal the memory system that data memory accesses from a specified address are likely in the near future. The memory system can respond by taking actions that are expected to speed up the memory accesses when they do occur, such as pre-loading the cache line containing the specified address into the data cache.

The PLD instruction signals that the likely memory access is a read, and the PLDW instruction signals that it is a write.

The effect of a PLD or PLDW is IMPLEMENTATION DEFINED.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	1	1	0	0	0	0	0	W	1	Rn				1	1	1	1	0	0	0	0	0	0	shift				Rm			

Preload read variant

Applies when **W** == 0.

PLD{<c>}{<q>} [<Rn>, {+}<Rm> {, LSL #<amount>}]

Preload write variant

Applies when **W** == 1.

PLDW{<c>}{<q>} [<Rn>, {+}<Rm> {, LSL #<amount>}]

Decode for this encoding

```

1 if Rn == '1111' then SEE "PLD (literal)";
2 if !HaveMainExt() then UNDEFINED;
3 n = UInt(Rn); m = UInt(Rm); add = TRUE; is_pldw = (W == '1');
4 (shift_t, shift_n) = (SRTYPE_LSL, UInt(shift));
5 if m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register that is added to the base register.
<Rm>	Is the general purpose index register, encoded in the "Rm" field.
<amount>	Is the shift amount, in the range 0-3, defaulting to 0 and encoded in the "imm2" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     offset = Shift(R[m], shift_t, shift_n, APSR.C);
4     address = if add then (R[n] + offset) else (R[n] - offset);
5     if is_pldw then
6         Hint_PreloadDataForWrite(address);
7     else
8         Hint_PreloadData(address);
```

C2.4.142 PLI (immediate, literal)

Preload Instruction (immediate, literal). Preload Instruction is a memory hint instruction that can signal the memory system that instruction memory accesses from a specified address are likely in the near future. The memory system can respond by taking actions that are expected to speed up the memory accesses when they do occur, such as pre-loading the cache line containing the specified address into the instruction cache.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	1	0	0	1	Rn				1	1	1	1	imm12											

T1 variant

PLI{<c>}{<q>} [<Rn> {, #<+><imm>}]

Decode for this encoding

```
1 if Rn == '1111' then SEE "encoding T3";
2 if !HaveMainExt() then UNDEFINED;
3 n = UInt(Rn); imm32 = ZeroExtend(imm12, 32); add = TRUE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	0	0	0	1	Rn				1	1	1	1	1	1	0	0	imm8							

T2 variant

PLI{<c>}{<q>} [<Rn> {, #-<imm>}]

Decode for this encoding

```
1 if Rn == '1111' then SEE "encoding T3";
2 if !HaveMainExt() then UNDEFINED;
3 n = UInt(Rn); imm32 = ZeroExtend(imm8, 32); add = FALSE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	1	U	0	0	1	1	1	1	1	1	1	1	1	imm12											

T3 variant

```
PLI{<c>}{<q>} <label>
    // Preferred syntax
PLI{<c>}{<q>} [PC, #<+/-><imm>]
    // Alternative syntax
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = 15; imm32 = ZeroExtend(imm12, 32); add = (U == '1');
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<label>	The label of the instruction that is likely to be accessed in the near future. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. The offset must be in the range -4095 to 4095. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE. If the offset is negative, imm32 is equal to minus the offset and add == FALSE.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the offset is added to the base register.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	For encoding T1: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T2: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field. For encoding T3: is a 12-bit unsigned immediate byte offset, in the range 0 to 4095, encoded in the "imm12" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     base = if n == 15 then Align(PC, 4) else R[n];
4     address = if add then (base + imm32) else (base - imm32);
5     Hint_PreloadInstr(address);

```

C2.4.143 PLI (register)

Preload Instruction (register). Preload Instruction is a memory hint instruction that can signal the memory system that instruction memory accesses from a specified address are likely in the near future. The memory system can respond by taking actions that are expected to speed up the memory accesses when they do occur, such as pre-loading the cache line containing the specified address into the instruction cache.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	1	1	0	0	1	0	0	0	1	Rn				1	1	1	1	0	0	0	0	0	0	imm2				Rm			

T1 variant

PLI{<c>}{<q>} [<Rn>, {+}<Rm> {, LSL #<amount>}]

Decode for this encoding

```

1 if Rn == '1111' then SEE "PLI (immediate, literal)";
2 if !HaveMainExt() then UNDEFINED;
3 n = UInt(Rn); m = UInt(Rm); add = TRUE;
4 (shift_t, shift_n) = (SRTYPE_LSL, UInt(imm2));
5 if m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register is added to the base register.
<Rm>	Is the general-purpose index register, encoded in the "Rm" field.
<amount>	Is the shift amount, in the range 0 to 3, defaulting to 0 and encoded in the "imm2" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   offset = Shift(R[m], shift_t, shift_n, APSR.C);
4   address = if add then (R[n] + offset) else (R[n] - offset);
5   Hint_PreloadInstr(address);
```

C2.4.144 POP (multiple registers)

Pop multiple registers from stack. Pop multiple registers from stack loads multiple general-purpose registers from the stack, loading from consecutive memory locations starting at the address in SP, and updates SP to point above the loaded data.

If the registers loaded include the PC, the word loaded for the PC is treated as a branch address or an exception return value. Bit[0] complies with the Arm architecture interworking rules for switching between the A32 and T32 instruction sets. However, Armv8-M only supports the T32 instruction set, so bit[0] must be 1. If bit[0] is 0 the PE takes an INVSTATE UsageFault exception on the instruction at the target address.

This instruction is interrupt-continuable.

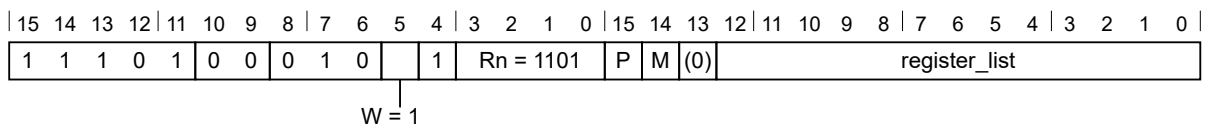
This instruction is subject to stack limit checking.

This instruction is an alias of the [LDM](#), [LDMIA](#), [LDMFD](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [LDM](#), [LDMIA](#), [LDMFD](#).
- The description of [LDM](#), [LDMIA](#), [LDMFD](#) gives the operational pseudocode for this instruction.

T2

Armv8-M Main Extension only



T2 variant

POP{<c>}{<q>} <registers>

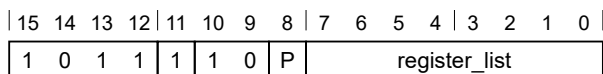
is equivalent to

LDM{<c>}{<q>} SP!, <registers>

and is the preferred disassembly when BitCount (register_list) > 1.

T3

Armv8-M



T3 variant

POP{<c>}{<q>} <registers>

is equivalent to

LDM{<c>}{<q>} SP!, <registers>

and is always the preferred disassembly.

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
<q> See [C1.2.5 Standard assembler syntax fields](#).

<registers> For encoding T2: is a list of two or more registers to be loaded, separated by commas and surrounded by { and }. The lowest-numbered register is loaded from the lowest memory address, through to the highest-numbered register from the highest memory address. The registers in the list must be in the range R0-R12, encoded in the "register_list" field, and can optionally contain one of the LR or the PC. If the LR is in the list, the "M" field is set to 1, otherwise it defaults to 0. If the PC is in the list, the "P" field is set to 1, otherwise it defaults to 0. The PC can be in the list. If it is, the instruction branches to the address loaded to the PC, and: If the PC is in the list:

- The LR must not be in the list.
- The instruction must be either outside any IT block, or the last instruction in an IT block.

For encoding T3: is a list of one or more registers to be loaded, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R7, encoded in the "register_list" field, and can optionally include the PC. If the PC is in the list, the "P" field is set to 1, otherwise this field defaults to 0. If the PC is in the list, the instruction must be either outside any IT block, or the last instruction in an IT block.

Operation for all encodings

The description of [LDM](#), [LDMIA](#), [LDMFD](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to values being loaded into R15, but does apply to all values being loaded into any other specified register even if R15 is being used.

The timing invariance does not apply to the addresses being accessed.

C2.4.145 POP (single register)

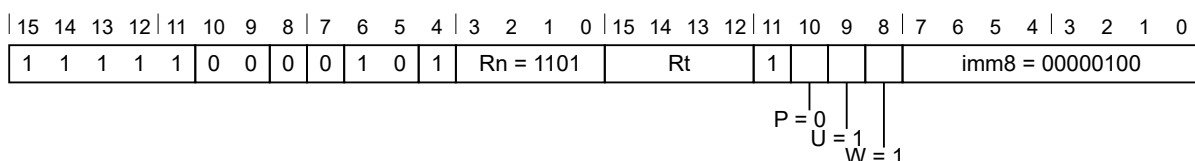
Pop single register from stack. Pop single register from stack loads a single general-purpose register from the stack, loading from the address in SP, and updates SP to point above the loaded data.

This instruction is an alias of the [LDR \(immediate\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [LDR \(immediate\)](#).
- The description of [LDR \(immediate\)](#) gives the operational pseudocode for this instruction.

T4

Armv8-M Main Extension only



Post-indexed variant

POP{<c>}{<q>} <register>

is equivalent to

LDR{<c>}{<q>} <Rt>, [SP], #4

and is always the preferred disassembly.

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <register> Is the general-purpose register <Rt> to be loaded surrounded by { and }.
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field. The PC can be used, provided the instruction is either outside an IT block or the last instruction of an IT block. If the PC is used, the instruction branches to the address (data) loaded to the PC.

Operation for all encodings

The description of [LDR \(immediate\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.146 PSSBB

Physical Speculative Store Bypass Barrier. Physical Speculative Store Bypass Barrier is a memory barrier which prevents speculative loads from bypassing earlier stores to the same physical address.

The semantics of the Physical Speculative Store Bypass Barrier are:

- When a load to a location appears in program order after the PSSBB, then the load does not speculatively read an entry earlier in the coherence order for that location than the entry generated by the latest store satisfying all of the following conditions:
 - The store is to the same location as the load.
 - The store appears in program order before the PSSBB.
 - The memory attributes of the store are the same as the memory attributes of the load.
- When a load to a location appears in program order before the PSSBB, then the load does not speculatively read data from any store satisfying all of the following conditions:
 - The store is to the same location as the load.
 - The store appears in program order after the PSSBB.
 - The memory attributes of the store are the same as the memory attributes of the load.

PSSBB does not have any impact on the rules associated with mismatched attributes.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	1	(1)	(1)	(1)	(1)	1	0	(0)	0	(1)	(1)	(1)	(1)	0	1	0	0	0	1	0	0

T1 variant

PSSBB{<q>}

Decode for this encoding

```
1 if InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 EncodingSpecificOperations();
2 SpeculativeSynchronizationBarrier();
```

C2.4.147 PUSH (multiple registers)

Push multiple registers to stack. Push multiple registers to stack stores multiple general-purpose registers to the stack, storing to consecutive memory locations ending below the address in SP, and updates SP to point to the start of the stored data.

This instruction is interrupt-continuable.

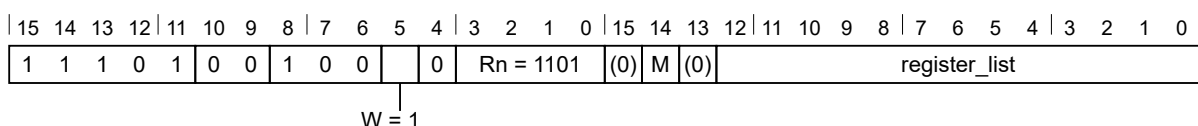
This instruction is subject to stack limit checking.

This instruction is an alias of the [STMDB](#), [STMTD](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [STMDB](#), [STMTD](#).
- The description of [STMDB](#), [STMTD](#) gives the operational pseudocode for this instruction.

T1

Armv8-M Main Extension only



T1 variant

PUSH{<c>}{<q>} <registers>

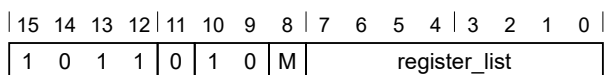
is equivalent to

STMDB{<c>}{<q>} SP!, <registers>

and is the preferred disassembly when `BitCount (register_list) > 1`.

T2

Armv8-M



T2 variant

PUSH{<c>}{<q>} <registers>

is equivalent to

STMDB{<c>}{<q>} SP!, <registers>

and is always the preferred disassembly.

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <registers> For encoding T1: is a list of one or more registers to be stored, separated by commas and surrounded by { and }. The lowest-numbered register is stored to the lowest memory address, through to the highest-numbered register to the highest memory address. The registers in the list must be in the range R0-R12, encoded in the "register_list" field, and can optionally contain the LR. If the LR is in the list, the "M" field is set to 1, otherwise it defaults to 0.

For encoding T2: is a list of one or more registers to be stored, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R7, encoded in the "register_list" field, and can optionally include the LR. If the LR is in the list, the "M" field is set to 1, otherwise this field defaults to 0.

Operation for all encodings

The description of [STMDB](#), [STMFD](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value or values being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.148 PUSH (single register)

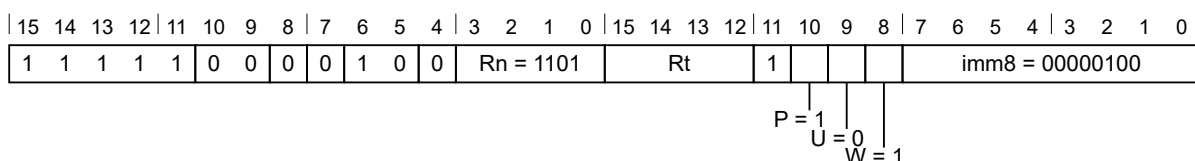
Push single register to stack. Push single register to stack stores a single general-purpose register to the stack, storing to the 32-bit word below the address in SP, and updates SP to point to the start of the stored data.

This instruction is an alias of the [STR \(immediate\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [STR \(immediate\)](#).
- The description of [STR \(immediate\)](#) gives the operational pseudocode for this instruction.

T4

Armv8-M Main Extension only



Pre-indexed variant

```
PUSH{<c>}{<q>} <register>
// Standard syntax
```

is equivalent to

```
STR{<c>}{<q>} <Rt>, [SP, #-4]!
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<code><c></code>	See C1.2.5 Standard assembler syntax fields .
<code><q></code>	See C1.2.5 Standard assembler syntax fields .
<code><register></code>	Is the general-purpose register <code><Rt></code> to be stored surrounded by { and }.
<code><Rt></code>	Is the general-purpose register to be transferred, encoded in the "Rt" field.

Operation for all encodings

The description of [STR \(immediate\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.149 QADD

Saturating Add. Saturating Add adds two register values, saturates the result to the 32-bit signed integer range -2^{31} to $2^{31}-1$, and writes the result to the destination register. If saturation occurs, it sets the Q flag in the APSR.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				1	0	0	0	Rm			

T1 variant

QADD{<c>}{<q>}{<Rd>,<Rm>,<Rn>}

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rn>	Is the second general-purpose source register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (R[d], sat) = SignedSatQ(SInt(R[m]) + SInt(R[n]), 32);
4   if sat then
5     APSR.Q = '1';
```

C2.4.150 QADD16

Saturating Add 16. Saturating Add 16 performs two 16-bit integer additions, saturates the results to the 16-bit signed integer range -2^{15} to $2^{15}-1$, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rn				1	1	1	1	Rd				0	0	0	1	Rm			

T1 variant

QADD16{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum1 = SInt(R[n][15:0]) + SInt(R[m][15:0]);
4   sum2 = SInt(R[n][31:16]) + SInt(R[m][31:16]);
5   bits(32) result;
6   result[15:0] = SignedSat(sum1, 16);
7   result[31:16] = SignedSat(sum2, 16);
8   R[d] = result;
```

C2.4.151 QADD8

Saturating Add 8. Saturating Add 8 performs four 8-bit integer additions, saturates the results to the 8-bit signed integer range -2^7 to 2^7-1 , and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				0	0	0	1	Rm			

T1 variant

QADD8 {<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     sum1 = SInt(R[n][7:0]) + SInt(R[m][7:0]);
4     sum2 = SInt(R[n][15:8]) + SInt(R[m][15:8]);
5     sum3 = SInt(R[n][23:16]) + SInt(R[m][23:16]);
6     sum4 = SInt(R[n][31:24]) + SInt(R[m][31:24]);
7     bits(32) result;
8     result[7:0] = SignedSat(sum1, 8);
9     result[15:8] = SignedSat(sum2, 8);
10    result[23:16] = SignedSat(sum3, 8);
11    result[31:24] = SignedSat(sum4, 8);
12    R[d] = result;

```

C2.4.152 QASX

Saturating Add and Subtract with Exchange. Saturating Add and Subtract with Exchange exchanges the two halfwords of the second operand, performs one 16-bit integer addition and one 16-bit subtraction, saturates the results to the 16-bit signed integer range -2^{15} to $2^{15}-1$, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	1	0	Rn				1	1	1	1	Rd				0	0	0	1	Rm			

T1 variant

QASX{<c>} {<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   diff = SInt(R[n][15:0]) - SInt(R[m][31:16]);
4   sum = SInt(R[n][31:16]) + SInt(R[m][15:0]);
5   bits(32) result;
6   result[15:0] = SignedSat(diff, 16);
7   result[31:16] = SignedSat(sum, 16);
8   R[d] = result;
```


C2.4.153 QDADD

Saturating Double and Add. Saturating Double and Add adds a doubled register value to another register value, and writes the result to the destination register. Both the doubling and the addition have their results saturated to the 32-bit signed integer range -2^{31} to $2^{31}-1$. If saturation occurs in either operation, it sets the Q flag in the APSR.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				1	0	0	1	Rm			

T1 variant

QDADD {<c>} {<q>} {<Rd>}, {<Rm>}, <Rn>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> Is the first general-purpose source register, encoded in the "Rm" field.
 <Rn> Is the second general-purpose source register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     (doubled, sat1) = SignedSatQ(2 * SInt(R[n]), 32);
4     (R[d], sat2) = SignedSatQ(SInt(R[m]) + SInt(doubled), 32);
5     if sat1 || sat2 then
6         APSR.Q = '1';
```

C2.4.154 QDSUB

Saturating Double and Subtract. Saturating Double and Subtract subtracts a doubled register value from another register value, and writes the result to the destination register. Both the doubling and the subtraction have their results saturated to the 32-bit signed integer range -2^{31} to $2^{31}-1$. If saturation occurs in either operation, it sets the Q flag in the APSR.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				1	0	1	1	Rm			

T1 variant

QDSUB{<c>}{<q>} {<Rd>}, {<Rm>}, <Rn>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rn>	Is the second general-purpose source register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (doubled, sat1) = SignedSatQ(2 * SInt(R[n]), 32);
4   (R[d], sat2) = SignedSatQ(SInt(R[m]) - SInt(doubled), 32);
5   if sat1 || sat2 then
6     APSR.Q = '1';
```

C2.4.155 QSAX

Saturating Subtract and Add with Exchange. Saturating Subtract and Add with Exchange exchanges the two halfwords of the second operand, performs one 16-bit integer subtraction and one 16-bit addition, saturates the results to the 16-bit signed integer range -2^{15} to $2^{15}-1$, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	1	0	Rn				1	1	1	1	Rd				0	0	0	1	Rm			

T1 variant

QSAX{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     sum = SInt(R[n][15:0]) + SInt(R[m][31:16]);
4     diff = SInt(R[n][31:16]) - SInt(R[m][15:0]);
5     bits(32) result;
6     result[15:0] = SignedSat(sum, 16);
7     result[31:16] = SignedSat(diff, 16);
8     R[d] = result;
```

C2.4.156 QSUB

Saturating Subtract. Saturating Subtract subtracts one register value from another register value, saturates the result to the 32-bit signed integer range -2^{31} to $2^{31}-1$, and writes the result to the destination register. If saturation occurs, it sets the Q flag in the APSR.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				1	0	1	0	Rm			

T1 variant

QSUB{<c>}{<q>} {<Rd>}, {<Rm>}, <Rn>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rn>	Is the second general-purpose source register, encoded in the "Rn" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     (R[d], sat) = SignedSatQ(SInt(R[m]) - SInt(R[n]), 32);
4     if sat then
5         APSR.Q = '1';
```

C2.4.157 QSUB16

Saturating Subtract 16. Saturating Subtract 16 performs two 16-bit integer subtractions, saturates the results to the 16-bit signed integer range -2^{15} to $2^{15}-1$, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	1	Rn				1	1	1	1	Rd				0	0	0	1	Rm			

T1 variant

QSUB16{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff1 = SInt(R[n][15:0]) - SInt(R[m][15:0]);
4     diff2 = SInt(R[n][31:16]) - SInt(R[m][31:16]);
5     bits(32) result;
6     result[15:0] = SignedSat(diff1, 16);
7     result[31:16] = SignedSat(diff2, 16);
8     R[d] = result;
```

C2.4.158 QSUB8

Saturating Subtract 8. Saturating Subtract 8 performs four 8-bit integer subtractions, saturates the results to the 8-bit signed integer range -2^7 to 2^7-1 , and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	0	Rn				1	1	1	1	Rd				0	0	0	1	Rm			

T1 variant

QSUB8 {<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff1 = SInt(R[n][7:0]) - SInt(R[m][7:0]);
4     diff2 = SInt(R[n][15:8]) - SInt(R[m][15:8]);
5     diff3 = SInt(R[n][23:16]) - SInt(R[m][23:16]);
6     diff4 = SInt(R[n][31:24]) - SInt(R[m][31:24]);
7     R[d][7:0] = SignedSat(diff1, 8);
8     R[d][15:8] = SignedSat(diff2, 8);
9     R[d][23:16] = SignedSat(diff3, 8);
10    R[d][31:24] = SignedSat(diff4, 8);
```

C2.4.159 RBIT

Reverse Bits. Reverse Bits reverses the bit order in a 32-bit register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rm				1	1	1	1	Rd				1	0	1	0	Rm2			

T1 variant

RBIT{<c>} {<q>} <Rd>, <Rm>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if Rm != Rm2 then UNPREDICTABLE;
3 d = UInt(Rd); m = UInt(Rm); m2 = UInt(Rm2);
4 if d IN {13,15} || m IN {13,15} || m2 IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $Rm \neq Rm2$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as described, with no change to its behavior and no additional side effects.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field. Its number must be encoded twice.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   bits(32) result;
4   for i = 0 to 31
5     result[31-i] = R[m][i];
6   R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.160 REV

Byte-Reverse Word. Byte-Reverse Word reverses the byte order in a 32-bit register.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	0	1	0	0	0	Rm			Rd		

T1 variant

REV{<c>}{<q>} <Rd>, <Rm>

Decode for this encoding

```
1 d = UInt(Rd); m = UInt(Rm);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rm				1	1	1	1	Rd				1	0	0	0	Rm2			

T2 variant

REV{<c>}.W <Rd>, <Rm>
// <Rd>, <Rm> can be represented in T1
REV{<c>}{<q>} <Rd>, <Rm>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if Rm != Rm2 then UNPREDICTABLE;
3 d = UInt(Rd); m = UInt(Rm); m2 = UInt(Rm2);
4 if d IN {13,15} || m IN {13,15} || m2 IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $Rm \neq Rm2$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as described, with no change to its behavior and no additional side effects.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> For encoding T1: is the general-purpose source register, encoded in the "Rm" field.
 For encoding T2: is the general-purpose source register, encoded in the "Rm" field. Its number must be encoded twice.

Operation for all encodings

```
1 if ConditionPassed() then  
2     EncodingSpecificOperations();  
3     bits(32) result;  
4     result[31:24] = R[m][7:0];  
5     result[23:16] = R[m][15:8];  
6     result[15:8]  = R[m][23:16];  
7     result[7:0]   = R[m][31:24];  
8     R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.161 REV16

Byte-Reverse Packed Halfword. Byte-Reverse Packed Halfword reverses the byte order in each 16-bit halfword of a 32-bit register.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	0	1	0	0	1	Rm			Rd		

T1 variant

REV16{<c>}{<q>} <Rd>, <Rm>

Decode for this encoding

```
1 d = UInt(Rd); m = UInt(Rm);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rm				1	1	1	1	Rd				1	0	0	1	Rm2			

T2 variant

REV16{<c>}.W <Rd>, <Rm>
// <Rd>, <Rm> can be represented in T1
REV16{<c>}{<q>} <Rd>, <Rm>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if Rm != Rm2 then UNPREDICTABLE;
3 d = UInt(Rd); m = UInt(Rm); m2 = UInt(Rm2);
4 if d IN {13,15} || m IN {13,15} || m2 IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $Rm \neq Rm2$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as described, with no change to its behavior and no additional side effects.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> For encoding T1: is the general-purpose source register, encoded in the "Rm" field.
 For encoding T2: is the general-purpose source register, encoded in the "Rm" field. Its number must be encoded twice.

Operation for all encodings

```
1 if ConditionPassed() then  
2     EncodingSpecificOperations();  
3     bits(32) result;  
4     result[31:24] = R[m][23:16];  
5     result[23:16] = R[m][31:24];  
6     result[15:8]  = R[m][7:0];  
7     result[7:0]   = R[m][15:8];  
8     R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.162 REVSH

Byte-Reverse Signed Halfword. Byte-Reverse Signed Halfword reverses the byte order in the lower 16-bit halfword of a 32-bit register, and sign extends the result to 32 bits.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	0	1	0	1	1	Rm			Rd		

T1 variant

REVSH{<c>}{<q>} <Rd>, <Rm>

Decode for this encoding

```
1 d = UInt(Rd); m = UInt(Rm);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rm				1	1	1	1	Rd				1	0	1	1	Rm2			

T2 variant

REVSH{<c>}.W <Rd>, <Rm>
// <Rd>, <Rm> can be represented in T1
REVSH{<c>}{<q>} <Rd>, <Rm>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if Rm != Rm2 then UNPREDICTABLE;
3 d = UInt(Rd); m = UInt(Rm); m2 = UInt(Rm2);
4 if d IN {13,15} || m IN {13,15} || m2 IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $Rm \neq Rm2$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as described, with no change to its behavior and no additional side effects.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> For encoding T1: is the general-purpose source register, encoded in the "Rm" field.
 For encoding T2: is the general-purpose source register, encoded in the "Rm" field. Its number must be encoded twice.

Operation for all encodings

```
1 if ConditionPassed() then  
2     EncodingSpecificOperations();  
3     bits(32) result;  
4     result[31:8] = SignExtend(R[m][7:0], 24);  
5     result[7:0]  = R[m][15:8];  
6     R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.163 ROR (immediate)

Rotate Right (immediate). Rotate Right (immediate) rotates a register value by a constant number of bits, inserting the bits that are rotated off the right end into the vacated bit positions on the left, and writes the result to the destination register.

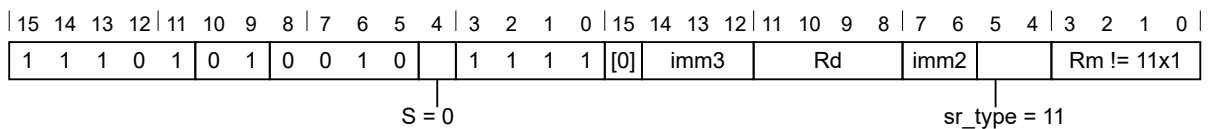
This instruction is an alias of the [MOV \(register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV \(register\)](#).
- The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.



MOV, shift or rotate by value variant

Applies when ! (imm3 == 000 && imm2 == 00).

ROR{<c>} {<q>} {<Rd>}, <Rm>, #<imm>

is equivalent to

MOV{<c>} {<q>} <Rd>, <Rm>, ROR #<imm>

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.
<imm>	Is the shift amount, in the range 1 to 31, encoded in the "imm3:imm2" field.

Operation for all encodings

The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.

- The values of the N, Z, C, V flags.

C2.4.164 ROR (register)

Rotate Right (register). Rotate Right (register) rotates a register value by a variable number of bits, inserting the bits that are rotated off the right end into the vacated bit positions on the left, and writes the result to the destination register. The variable number of bits is read from the bottom byte of a register.

This instruction is an alias of the [MOV, MOVS \(register-shifted register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV, MOVS \(register-shifted register\)](#).
- The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op = 0111				Rs			Rdm		

Rotate right variant

```
ROR<c>{<q>} {<Rdm>, } <Rdm>, <Rs>
// Inside IT block
```

is equivalent to

```
MOV<c>{<q>} <Rdm>, <Rdm>, ROR <Rs>
```

and is the preferred disassembly when `InITBlock()`.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0							Rm	1	1	1	1				Rd	0	0	0	0				Rs
										sr_type = 11																					
										S = 0																					

Non flag setting variant

```
ROR<c>.W {<Rd>, } <Rm>, <Rs>
// Inside IT block, and <Rd>, <Rm>, <sr_type>, <Rs> can be represented in T1
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, ROR <Rs>
```

and is always the preferred disassembly.

Non flag setting variant

```
ROR{<c>}{<q>} {<Rd>, } <Rm>, <Rs>
```

is equivalent to

```
MOV{<c>}{<q>} <Rd>, <Rm>, ROR <Rs>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<C>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the first general-purpose source register and the destination register, encoded in the "Rdm" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rs>	Is the second general-purpose source register holding a rotate amount in its bottom 8 bits, encoded in the "Rs" field.

Operation for all encodings

The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.165 RORS (immediate)

Rotate Right, Setting flags (immediate). Rotate Right, Setting flags (immediate) rotates a register value by a constant number of bits, inserting the bits that are rotated off the right end into the vacated bit positions on the left, writes the result to the destination register, and updates the condition flags based on the result.

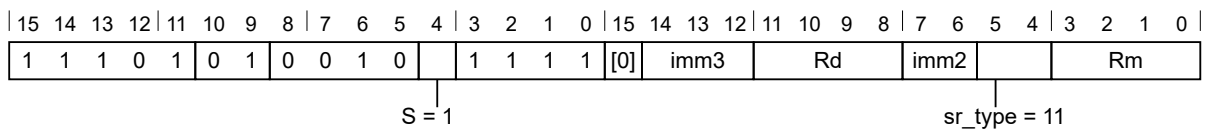
This instruction is an alias of the [MOV \(register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV \(register\)](#).
- The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.



MOVS, shift or rotate by value variant

Applies when ! (imm3 == 000 && imm2 == 00).

RORS{<c>}{<q>} {<Rd>, } <Rm>, #<imm>

is equivalent to

MOVS{<c>}{<q>} <Rd>, <Rm>, ROR #<imm>

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.
<imm>	Is the shift amount, in the range 1 to 31, encoded in the "imm3:imm2" field.

Operation for all encodings

The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.

- The values of the N, Z, C, V flags.

C2.4.166 RORS (register)

Rotate Right, Setting flags (register). Rotate Right, Setting flags (register) rotates a register value by a variable number of bits, inserting the bits that are rotated off the right end into the vacated bit positions on the left, writes the result to the destination register, and updates the condition flags based on the result. The variable number of bits is read from the bottom byte of a register.

This instruction is an alias of the [MOV, MOVS \(register-shifted register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV, MOVS \(register-shifted register\)](#).
- The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	op = 0111				Rs		Rdm			

Rotate right variant

```
RORS{<q>} {<Rdm>, } <Rdm>, <Rs>
// Outside IT block
```

is equivalent to

```
MOVS{<q>} <Rdm>, <Rdm>, ROR <Rs>
```

and is the preferred disassembly when !InITBlock().

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0				Rm				1	1	1	1	Rd				0	0	0	0	Rs			

$\begin{array}{c} \text{sr_type} = 11 \\ \text{S} = 1 \end{array}$

Flag setting variant

```
RORS.W {<Rd>, } <Rm>, <Rs>
// Outside IT block, and <Rd>, <Rm>, <sr_type>, <Rs> can be represented in T1
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, ROR <Rs>
```

and is always the preferred disassembly.

Flag setting variant

```
RORS{<c>}{<q>} {<Rd>, } <Rm>, <Rs>
```

is equivalent to

```
MOVS{<c>}{<q>} <Rd>, <Rm>, ROR <Rs>
```

and is always the preferred disassembly.

Assembler symbols for all encodings

<C>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdm>	Is the first general-purpose source register and the destination register, encoded in the "Rdm" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the first general-purpose source register, encoded in the "Rm" field.
<Rs>	Is the second general-purpose source register holding a rotate amount in its bottom 8 bits, encoded in the "Rs" field.

Operation for all encodings

The description of [MOV, MOVS \(register-shifted register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.167 RRX

Rotate Right with Extend. Rotate Right with Extend shifts a register value right by one bit, shifting the Carry flag into bit[31], and writes the result to the destination register.

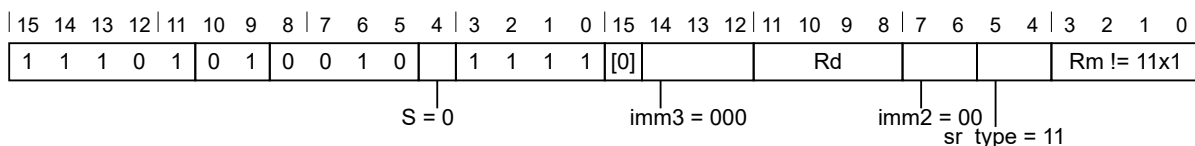
This instruction is an alias of the [MOV \(register\)](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [MOV \(register\)](#).
- The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.



MOV, rotate right with extend variant

RRX{<c>}{<q>}{<Rd>}, <Rm>

is equivalent to

MOV{<c>}{<q>}<Rd>, <Rm>, RRX

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

The description of [MOV \(register\)](#) gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.168 RRXS

Rotate Right with Extend, Setting flags. Rotate Right with Extend, Setting flags shifts a register value right by one bit, shifting the Carry flag into bit[31] and bit[0] into the Carry flag, writes the result to the destination register and updates the condition flags (other than Carry) based on the result.

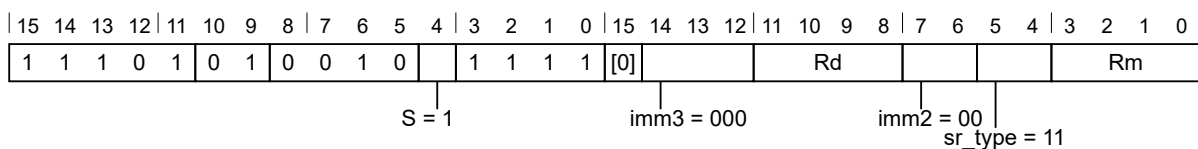
This instruction is an alias of the **MOV (register)** instruction. This means that:

- The encodings in this description are named to match the encodings of **MOV (register)**.
- The description of **MOV (register)** gives the operational pseudocode for this instruction.

T3

Armv8-M Main Extension only

- In Armv8.0-M, bit[15] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[15] (lower 16 bits of the encoding) [0] represents 0.



MOVS, rotate right with extend variant

$$\text{RRXS}\{\langle c \rangle\}\{\langle q \rangle\}\{\langle Rd \rangle, \} \langle Rm \rangle$$

is equivalent to

MOVS {<c>} {<q>} <Rd>, <Rm>, RRX

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

The description of **MOV (register)** gives the operational pseudocode for this instruction.

Restricted behavior

Data Independent Timing behavior

If AIRCR.DIT is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.169 RSB (immediate)

Reverse Subtract (immediate). Reverse Subtract (immediate) subtracts a register value from an immediate value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	1	0	0	1	Rn			Rd		

T1 variant

```
RSB<c>{<q>} {<Rd>, }<Rn>, #0
    // Inside IT block
RSBS{<q>} {<Rd>, }<Rn>, #0
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rd); n = UInt(Rn); setflags = !InITBlock(); imm32 = Zeros(32); // immediate = #0
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	1	1	1	0	S	Rn			0	imm3			Rd			imm8									

RSB variant

Applies when **S** == 0.

```
RSB<c>.W {<Rd>, }<Rn>, #0
    // Inside IT block
RSB{<c>}{<q>} {<Rd>, }<Rn>, #<const>
```

RSBS variant

Applies when **S** == 1.

```
RSBS.W {<Rd>, }<Rn>, #0
    // Outside IT block
RSBS{<c>}{<q>} {<Rd>, }<Rn>, #<const>
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1'); imm32 = T32ExpandImm(i:imm3:imm8);
3 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.

<const> Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See [C1.5 Modified immediate constants](#) for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (result, carry, overflow) = AddWithCarry(NOT(R[n]), imm32, '1');
4   R[d] = result;
5   if setflags then
6     APSR.N = result[31];
7     APSR.Z = IsZeroBit(result);
8     APSR.C = carry;
9     APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

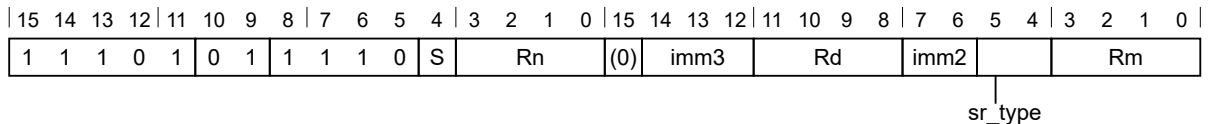
- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.170 RSB (register)

Reverse Subtract (register). Reverse Subtract (register) subtracts a register value from an optionally-shifted register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M Main Extension only



RSB, rotate right with extend variant

Applies when **S == 0** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

RSB{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>, RRX

RSB, shift or rotate by value variant

Applies when **S == 0** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

RSB{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm> {, <shift> #<amount>}

RSBS, rotate right with extend variant

Applies when **S == 1** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

RSBS{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>, RRX

RSBS, shift or rotate by value variant

Applies when **S == 1** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

RSBS{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm> {, <shift> #<amount>}

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
3 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then  
2     EncodingSpecificOperations();  
3     shifted = Shift(R[m], shift_t, shift_n, APSR.C);  
4     (result, carry, overflow) = AddWithCarry(NOT(R[n]), shifted, '1');  
5     R[d] = result;  
6     if setflags then  
7         APSR.Z = IsZeroBit(result);  
8         APSR.N = result[31];  
9         APSR.C = carry;  
10        APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.171 SADD16

Signed Add 16. Signed Add 16 performs two 16-bit signed integer additions, and writes the results to the destination register. It sets the APSR.GE bits according to the results of the additions.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rn				1	1	1	1	Rd				0	0	0	0	Rm			

T1 variant

SADD16{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum1 = SInt(R[n][15:0]) + SInt(R[m][15:0]);
4   sum2 = SInt(R[n][31:16]) + SInt(R[m][31:16]);
5   R[d] = sum2[15:0] : sum1[15:0];
6   APSR.GE[1:0] = if sum1 >= 0 then '11' else '00';
7   APSR.GE[3:2] = if sum2 >= 0 then '11' else '00';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.172 SADD8

Signed Add 8. Signed Add 8 performs four 8-bit signed integer additions, and writes the results to the destination register. It sets the APSR.GE bits according to the results of the additions.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				0	0	0	0	Rm			

T1 variant

SADD8{<c>}{<q>} {<Rd>}, {<Rn>}, {<Rm>}

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum1 = SInt(R[n][7:0]) + SInt(R[m][7:0]);
4   sum2 = SInt(R[n][15:8]) + SInt(R[m][15:8]);
5   sum3 = SInt(R[n][23:16]) + SInt(R[m][23:16]);
6   sum4 = SInt(R[n][31:24]) + SInt(R[m][31:24]);
7   R[d] = sum4[7:0] : sum3[7:0] : sum2[7:0] : sum1[7:0];
8   APSR.GE[0] = if sum1 >= 0 then '1' else '0';
9   APSR.GE[1] = if sum2 >= 0 then '1' else '0';
10  APSR.GE[2] = if sum3 >= 0 then '1' else '0';
11  APSR.GE[3] = if sum4 >= 0 then '1' else '0';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.173 SASX

Signed Add and Subtract with Exchange. Signed Add and Subtract with Exchange exchanges the two halfwords of the second operand, performs one 16-bit integer addition and one 16-bit subtraction, and writes the results to the destination register. It sets the APSR.GE bits according to the results.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	1	0	Rn				1	1	1	1	Rd				0	0	0	0	Rm			

T1 variant

SASX{<c>}{<q>}{<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff = SInt(R[n][15:0]) - SInt(R[m][31:16]);
4     sum = SInt(R[n][31:16]) + SInt(R[m][15:0]);
5     R[d] = sum[15:0] : diff[15:0];
6     APSR.GE[1:0] = if diff >= 0 then '11' else '00';
7     APSR.GE[3:2] = if sum >= 0 then '11' else '00';
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.174 SBC (immediate)

Subtract with Carry (immediate). Subtract with Carry (immediate) subtracts an immediate value and the value of NOT(Carry flag) from a register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	1	0	1	1	S	Rn			0	imm3			Rd			imm8									

SBC variant

Applies when **S** == 0.

SBC{<c>}{<q>} {<Rd>}, {<Rn>}, #<const>

SBCS variant

Applies when **S** == 1.

SBCS{<c>}{<q>} {<Rd>}, {<Rn>}, #<const>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1'); imm32 = T32ExpandImm(i:imm3:imm8);
3 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (result, carry, overflow) = AddWithCarry(R[n], NOT(imm32), APSR.C);
4   R[d] = result;
5   if setflags then
6     APSR.N = result[31];
7     APSR.Z = IsZeroBit(result);
8     APSR.C = carry;
9     APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:

- The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.175 SBC (register)

Subtract with Carry (register). Subtract with Carry (register) subtracts an optionally-shifted register value and the value of NOT(Carry flag) from a register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	1	1	0	Rm			Rdn		

T1 variant

```
SBC<c>{<q>} {<Rdn>}, <Rdn>, <Rm>
    // Inside IT block
SBCS{<q>} {<Rdn>}, <Rdn>, <Rm>
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rdn); n = UInt(Rdn); m = UInt(Rm); setflags = !InITBlock();
2 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	1	S	Rn			(0)	imm3			Rd			imm2						Rm						

sr type

SBC, rotate right with extend variant

Applies when **S == 0** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
SBC{<c>}{<q>} {<Rd>}, <Rn>, <Rm>, RRX
```

SBC, shift or rotate by value variant

Applies when **S == 0** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
SBC<c>.W {<Rd>}, <Rn>, <Rm>
    // Inside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
SBC{<c>}{<q>} {<Rd>}, <Rn>, <Rm> {, <shift> #<amount>}
```

SBCS, rotate right with extend variant

Applies when **S == 1** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

```
SBCS{<c>}{<q>} {<Rd>}, <Rn>, <Rm>, RRX
```

SBCS, shift or rotate by value variant

Applies when **S == 1** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
SBCS.W {<Rd>}, <Rn>, <Rm>
// Outside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
SBCS{<c>}{<q>} {<Rd>}, <Rn>, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
3 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdn>	Is the first general-purpose source register and the destination register, encoded in the "Rdn" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   shifted = Shift(R[m], shift_t, shift_n, APSR.C);
4   (result, carry, overflow) = AddWithCarry(R[n], NOT(shifted), APSR.C);
5   R[d] = result;
6   if setflags then
7     APSR.N = result[31];
8     APSR.Z = IsZeroBit(result);
9     APSR.C = carry;
10    APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.176 SBFX

Signed Bit Field Extract. Signed Bit Field Extract extracts any number of adjacent bits at any position from one register, sign extends them to 32 bits, and writes the result to the destination register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	(0)	1	1	0	1	0	0	Rn				0	imm3				Rd				imm2	(0)	widthm1				

T1 variant

SBFX{<c>}{<q>} <Rd>, <Rn>, #<lsb>, #<width>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn);
3 lsbit = UInt(imm3:imm2); widthminus1 = UInt(widthm1);
4 msbit = lsbit + widthminus1;
5 if msbit > 31 then UNPREDICTABLE;
6 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `msbit > 31`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<lsb>	Is the bit number of the least significant bit in the field, in the range 0 to 31, encoded in the "imm3:imm2" field.
<width>	Is the width of the field, in the range 1 to 32-<lsb>, encoded in the "widthm1" field as <width>-1.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if msbit <= 31 then
4         R[d] = SignExtend(R[n][msbit:lsbit], 32);
5     else
6         R[d] = bits(32) UNKNOWN;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.177 SDIV

Signed Divide. Signed Divide divides a 32-bit signed integer register value by a 32-bit signed integer register value and writes the result to the destination register. The condition code flags are not affected.

If $R[n] == 0 \times 80000000$ (-2^{31}) and $R[m] == 0 \times \text{FFFFFFFF}$ (-1), the result of the division is 0×80000000 .

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	0	0	1	Rn				(1)	(1)	(1)	(1)	Rd				1	1	1	1	Rm			

T1 variant

SDIV{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
2 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the dividend, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the divisor, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   if SInt(R[m]) == 0 then
4     if IntegerZeroDivideTrappingEnabled() then
5       GenerateIntegerZeroDivide();
6     else
7       result = 0;
8   else
9     result = RoundTowardsZero(Real(SInt(R[n])) / Real(SInt(R[m])));
10    R[d] = result[31:0];
```

C2.4.178 SEL

Select Bytes. Select Bytes selects each byte of its result from either its first operand or its second operand, according to the values of the GE flags.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	1	0	Rn				1	1	1	1	Rd				1	0	0	0	Rm			

T1 variant

SEL{<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     bits(32) result;
4     result[7:0] = if APSR.GE[0] == '1' then R[n][7:0] else R[m][7:0];
5     result[15:8] = if APSR.GE[1] == '1' then R[n][15:8] else R[m][15:8];
6     result[23:16] = if APSR.GE[2] == '1' then R[n][23:16] else R[m][23:16];
7     result[31:24] = if APSR.GE[3] == '1' then R[n][31:24] else R[m][31:24];
8     R[d] = result;
```

C2.4.179 SEV

Send Event. Send Event is a hint instruction. It causes an event to be signaled to all PEs within the multiprocessor system.

This is a NOP-compatible hint.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0

T1 variant

SEV{<c>}{<q>}

Decode for this encoding

```
1 // No additional decoding required
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	0	0	1	0	0

T2 variant

SEV{<c>}.W

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 // No additional decoding required
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
<q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     SendEvent();
```

C2.4.180 SG

Secure Gateway. Secure Gateway marks a valid branch target for branches from Non-secure code that call Secure code.

This instruction sets the Security state to Secure if its address is in Secure memory. If the address of this instruction is in Non-secure memory the instruction behaves as a NOP.

If the PACTBTI Extension is implemented, this instruction is always a valid BTI landing pad regardless of whether or not the instruction behaves as a NOP.

If the PE was previously in Non-secure state:

- This instruction sets bit[0] of LR to 0, to indicate that the return address will cause a transition from Secure to Non-secure state.
- If the Floating-point Extension is implemented, this instruction marks Secure floating-point state as inactive, by setting CONTROL_S.SFPA to 0. This indicates that the floating-point registers do not contain active state that belongs to the Secure state.

An INVEP SecureFault is generated if the PE attempts to reenter Thread mode when CCR_S.TRD is set to 1 and either or both of the following are true:

- CONTROL_S.SPSEL is 0.
- The current Stack Pointer of the Secure state points to an address that contains the value 0xFEFA125A[31:1].

SG is an unconditional instruction and executes as such both inside and outside an **IT** instruction block. Arm recommends that software does not place SG inside an **IT** instruction block.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	0	1	0	0	1	0	1	1	1	1	1	1	1

T1 variant

SG{<q>}

Decode for this encoding

```
1 // No encoding specific operations
```

Assembler symbols for all encodings

<q> See C1.2.5 Standard assembler syntax fields.

Operation for all encodings

```
1 EncodingSpecificOperations();
2
3 if HaveSecurityExt() then
4     sAttributes = SecurityCheck(ThisInstrAddr(), TRUE, IsSecure());
5     if !sAttributes.ns then
6         if !IsSecure() then
7             if HasArchVersion(Armv8p1) && CurrentMode() == PEmode_Thread then
8                 // The access to the Secure stack should be performed with the privilege
9                 // level of the current mode in the Secure state, and not the current state.
10                // NOTE: The load below is performed, and any faults handled even if thread
11                // mode re-entrancy checking is disabled.
12                secIsPriv = CurrentModeIsPrivileged(TRUE);
```



```

13         secSp          = LookUpSP_with_security_mode(TRUE, PEMode_Thread);
14         (exc, spData) = MemA_with_priv_security(_SP(secSp), 4, AccType_NORMAL,
15                                           FindMemPriv(secIsPriv), TRUE, TRUE);
16         HandleException(exc);
17         // Check for an exception stack frame if thread mode re-entrancy is disabled.
18         if CCR_S.TRD == '1' && (spData[31:1] == 0xFEFA125A[31:1] ||
19                               secSp          == RNamesSP_Main_Secure) then
20             SFSR.INVEP = '1';
21             HandleException(CreateException(SecureFault));
22
23         // Set up the security meta data flags and change to the Secure state
24         LR[0] = '0';
25         if HaveMveOrFPEExt() then
26             CONTROL_S.SFPA = '0';
27         // LOB data cleared to prevent Non-secure code from interfering
28         // with Secure execution
29         if HaveLOBExt() then
30             LO_BRANCH_INFO.VALID = '0';
31         CurrentState = SecurityState_Secure;
32         // IT/ICI/ECI data cleared to prevent Non-secure code from interfering
33         // with Secure execution
34         if HaveMainExt() then
35             ITSTATE = Zeros(8);
36
37         // This instruction can be used as a BTI landing pad.
38         // The EPSR.B bit is reset to zero.
39         EPSR.B = '0';

```

C2.4.181 SHADD16

Signed Halving Add 16. Signed Halving Add 16 performs two signed 16-bit integer additions, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rn				1	1	1	1	Rd				0	0	1	0	Rm			

T1 variant

SHADD16{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     sum1 = SInt(R[n][15:0]) + SInt(R[m][15:0]);
4     sum2 = SInt(R[n][31:16]) + SInt(R[m][31:16]);
5     R[d] = sum2[16:1] : sum1[16:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.182 SHADD8

Signed Halving Add 8. Signed Halving Add 8 performs four signed 8-bit integer additions, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				0	0	1	0	Rm			

T1 variant

SHADD8{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum1 = SInt(R[n][7:0]) + SInt(R[m][7:0]);
4   sum2 = SInt(R[n][15:8]) + SInt(R[m][15:8]);
5   sum3 = SInt(R[n][23:16]) + SInt(R[m][23:16]);
6   sum4 = SInt(R[n][31:24]) + SInt(R[m][31:24]);
7   R[d] = sum4[8:1] : sum3[8:1] : sum2[8:1] : sum1[8:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.183 SHASX

Signed Halving Add and Subtract with Exchange. Signed Halving Add and Subtract with Exchange exchanges the two halfwords of the second operand, performs one signed 16-bit integer addition and one signed 16-bit subtraction, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	1	0	Rn				1	1	1	1	Rd				0	0	1	0	Rm			

T1 variant

SHASX{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   diff = SInt(R[n][15:0]) - SInt(R[m][31:16]);
4   sum = SInt(R[n][31:16]) + SInt(R[m][15:0]);
5   R[d] = sum[16:1] : diff[16:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.184 SHSAX

Signed Halving Subtract and Add with Exchange. Signed Halving Subtract and Add with Exchange exchanges the two halfwords of the second operand, performs one signed 16-bit integer subtraction and one signed 16-bit addition, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	1	0	Rn				1	1	1	1	Rd				0	0	1	0	Rm			

T1 variant

SHSAX{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum = SInt(R[n][15:0]) + SInt(R[m][31:16]);
4   diff = SInt(R[n][31:16]) - SInt(R[m][15:0]);
5   R[d] = diff[16:1] : sum[16:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.185 SHSUB16

Signed Halving Subtract 16. Signed Halving Subtract 16 performs two signed 16-bit integer subtractions, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	1	Rn				1	1	1	1	Rd				0	0	1	0	Rm			

T1 variant

SHSUB16{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff1 = SInt(R[n][15:0]) - SInt(R[m][15:0]);
4     diff2 = SInt(R[n][31:16]) - SInt(R[m][31:16]);
5     R[d] = diff2[16:1] : diff1[16:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.186 SHSUB8

Signed Halving Subtract 8. Signed Halving Subtract 8 performs four signed 8-bit integer subtractions, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	0	Rn				1	1	1	1	Rd				0	0	1	0	Rm			

T1 variant

SHSUB8{<c>}{<q>}{<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff1 = SInt(R[n][7:0]) - SInt(R[m][7:0]);
4     diff2 = SInt(R[n][15:8]) - SInt(R[m][15:8]);
5     diff3 = SInt(R[n][23:16]) - SInt(R[m][23:16]);
6     diff4 = SInt(R[n][31:24]) - SInt(R[m][31:24]);
7     R[d] = diff4[8:1] : diff3[8:1] : diff2[8:1] : diff1[8:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.187 SMLABB, SMLABT, SMLATB, SMLATT

Signed Multiply Accumulate (halfwords). Signed Multiply Accumulate (halfwords) performs a signed multiply accumulate operation. The multiply acts on two signed 16-bit quantities, taken from either the bottom or the top half of their respective source registers. The other halves of these source registers are ignored. The 32-bit product is added to a 32-bit accumulate value and the result is written to the destination register.

If overflow occurs during the addition of the accumulate value, the instruction sets the Q flag in the APSR. It is not possible for overflow to occur during the multiplication.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	0	0	1	Rn				Ra				Rd				0	0	N	M	Rm			

SMLABB variant

Applies when N == 0 && M == 0.

SMLABB{<c>} {<q>} <Rd>, <Rn>, <Rm>, <Ra>

SMLABT variant

Applies when N == 0 && M == 1.

SMLABT{<c>} {<q>} <Rd>, <Rn>, <Rm>, <Ra>

SMLATB variant

Applies when N == 1 && M == 0.

SMLATB{<c>} {<q>} <Rd>, <Rn>, <Rm>, <Ra>

SMLATT variant

Applies when N == 1 && M == 1.

SMLATT{<c>} {<q>} <Rd>, <Rn>, <Rm>, <Ra>

Decode for this encoding

```

1 if Ra == '1111' then SEE "SMULBB, SMULBT, SMULTB, SMULTT";
2 if !HaveDSPExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); a = UInt(Ra);
4 n_high = (N == '1'); m_high = (M == '1');
5 if d IN {13,15} || n IN {13,15} || m IN {13,15} || a == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the multiplicand in the bottom or top half (selected by <x>), encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier in the bottom or top half (selected by <y>), encoded in the "Rm" field.
<Ra>	Is the third general-purpose source register holding the addend, encoded in the "Ra" field.

Operation for all encodings

```
1 if ConditionPassed() then  
2   EncodingSpecificOperations();  
3   operand1 = if n_high then R[n][31:16] else R[n][15:0];  
4   operand2 = if m_high then R[m][31:16] else R[m][15:0];  
5   result = SInt(operand1) * SInt(operand2) + SInt(R[a]);  
6   R[d] = result[31:0];  
7   if result != SInt(result[31:0]) then // Signed overflow  
8     APSR.Q = '1';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.188 SMLAD, SMLADX

Signed Multiply Accumulate Dual. Signed Multiply Accumulate Dual performs two signed 16-bit by 16-bit multiplications. It adds the products to a 32-bit accumulate operand.

Optionally, the instruction can exchange the halfwords of the second operand before performing the arithmetic.

This instruction sets the Q flag if the accumulate operation overflows. Overflow cannot occur during the multiplications.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	0	1	0	Rn				Ra				Rd				0 0 0 M				Rm			

SMLAD variant

Applies when **M == 0**.

SMLAD{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

SMLADX variant

Applies when **M == 1**.

SMLADX{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

Decode for this encoding

```

1 if Ra == '1111' then SEE SMUAD;
2 if !HaveDSPExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); a = UInt(Ra);
4 m_swap = (M == '1');
5 if d IN {13,15} || n IN {13,15} || m IN {13,15} || a == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<Ra>	Is the third general-purpose source register holding the addend, encoded in the "Ra" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     operand2 = if m_swap then ROR(R[m],16) else R[m];
4     product1 = SInt(R[n][15:0]) * SInt(operand2[15:0]);
5     product2 = SInt(R[n][31:16]) * SInt(operand2[31:16]);
6     result = product1 + product2 + SInt(R[a]);
7     R[d] = result[31:0];
8     if result != SInt(result[31:0]) then // Signed overflow
9         APSR.Q = '1';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.189 SMLAL

Signed Multiply Accumulate Long. Signed Multiply Accumulate Long multiplies two signed 32-bit values to produce a 64-bit value, and accumulates this with a 64-bit value.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	1	0	0	Rn				RdLo				RdHi				0	0	0	0	Rm			

T1 variant

SMLAL{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 dLo = UInt(RdLo); dHi = UInt(RdHi); n = UInt(Rn); m = UInt(Rm); setflags = FALSE;
3 if dLo IN {13,15} || dHi IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
4 if dHi == dLo then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `dHi == dLo`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<RdLo>	Is the general-purpose source register holding the lower 32 bits of the addend, and the destination register for the lower 32 bits of the result, encoded in the "RdLo" field.
<RdHi>	Is the general-purpose source register holding the upper 32 bits of the addend, and the destination register for the upper 32 bits of the result, encoded in the "RdHi" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = SInt(R[n]) * SInt(R[m]) + SInt(R[dHi]:R[dLo]);
4     R[dHi] = result[63:32];
5     R[dLo] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.190 SMLALBB, SMLALBT, SMLALTB, SMLALTT

Signed Multiply Accumulate Long (halfwords). Signed Multiply Accumulate Long (halfwords) multiplies two signed 16-bit values to produce a 32-bit value, and accumulates this with a 64-bit value. The multiply acts on two signed 16-bit quantities, taken from either the bottom or the top half of their respective source registers. The other halves of these source registers are ignored. The 32-bit product is sign-extended and accumulated with a 64-bit accumulate value.

Overflow is possible during this instruction, but only as a result of the 64-bit addition. This overflow is not detected if it occurs. Instead, the result wraps around modulo 2^{64} .

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	1	0	0	Rn				RdLo				RdHi				1	0	N	M	Rm			

SMLALBB variant

Applies when **N == 0** && **M == 0**.

SMLALBB{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

SMLALBT variant

Applies when **N == 0** && **M == 1**.

SMLALBT{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

SMLALTB variant

Applies when **N == 1** && **M == 0**.

SMLALTB{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

SMLALTT variant

Applies when **N == 1** && **M == 1**.

SMLALTT{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 dLo = UInt(RdLo); dHi = UInt(RdHi); n = UInt(Rn); m = UInt(Rm);
3 n_high = (N == '1'); m_high = (M == '1');
4 if dLo IN {13,15} || dHi IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
5 if dHi == dLo then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **dHi == dLo**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<RdLo>	Is the general-purpose source register holding the lower 32 bits of the addend, and the destination register for the lower 32 bits of the result, encoded in the "RdLo" field.
<RdHi>	Is the general-purpose source register holding the upper 32 bits of the addend, and the destination register for the upper 32 bits of the result, encoded in the "RdHi" field.
<Rn>	Is the first general-purpose source register holding the multiplicand in the bottom or top half (selected by <x>), encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier in the bottom or top half (selected by <x>), encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     operand1 = if n_high then R[n][31:16] else R[n][15:0];
4     operand2 = if m_high then R[m][31:16] else R[m][15:0];
5     result = SInt(operand1) * SInt(operand2) + SInt(R[dHi]:R[dLo]);
6     R[dHi] = result[63:32];
7     R[dLo] = result[31:0];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.191 SMLALD, SMLALDX

Signed Multiply Accumulate Long Dual. Signed Multiply Accumulate Long Dual performs two signed 16-bit by 16-bit multiplications. It adds the products to a 64-bit accumulate operand.

Optionally, the instruction can exchange the halfwords of the second operand before performing the arithmetic.

Overflow is possible during this instruction, but only as a result of the 64-bit addition. This overflow is not detected if it occurs. Instead, the result wraps around modulo 2^{64} .

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	1	1	0	1	1	1	1	0	0	Rn				RdLo				RdHi				1			1	0	M	Rm			

SMLALD variant

Applies when **M** == 0.

SMLALD{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

SMLALDX variant

Applies when **M** == 1.

SMLALDX{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 dLo = UInt(RdLo); dHi = UInt(RdHi); n = UInt(Rn); m = UInt(Rm); m_swap = (M == '1');
3 if dLo IN {13,15} || dHi IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
4 if dHi == dLo then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **dHi** == **dLo**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<RdLo>	Is the general-purpose source register holding the lower 32 bits of the addend, and the destination register for the lower 32 bits of the result, encoded in the "RdLo" field.
<RdHi>	Is the general-purpose source register holding the upper 32 bits of the addend, and the destination register for the upper 32 bits of the result, encoded in the "RdHi" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     operand2 = if m_swap then ROR(R[m], 16) else R[m];
```



```
4     product1 = SInt (R[n][15:0]) * SInt (operand2[15:0]);
5     product2 = SInt (R[n][31:16]) * SInt (operand2[31:16]);
6     result = product1 + product2 + SInt (R[dHi]:R[dLo]);
7     R[dHi] = result[63:32];
8     R[dLo] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.192 SMLAWB, SMLAWT

Signed Multiply Accumulate (word by halfword). Signed Multiply Accumulate (word by halfword) performs a signed multiply accumulate operation. The multiply acts on a signed 32-bit quantity and a signed 16-bit quantity. The signed 16-bit quantity is taken from either the bottom or the top half of its source register. The other half of the second source register is ignored. The top 32 bits of the 48-bit product are added to a 32-bit accumulate value and the result is written to the destination register. The bottom 16 bits of the 48-bit product are ignored.

If overflow occurs during the addition of the accumulate value, the instruction sets the Q flag in the APSR. No overflow can occur during the multiplication.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	0	1	1	Rn	Ra	Rd	0	0	0	M	Rm												

SMLAWB variant

Applies when **M == 0**.

SMLAWB{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

SMLAWT variant

Applies when **M == 1**.

SMLAWT{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

Decode for this encoding

```

1 if Ra == '1111' then SEE "SMULWB, SMULWT";
2 if !HaveDSPExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); a = UInt(Ra); m_high = (M == '1');
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} || a == 13 then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier in the bottom or top half (selected by <y>), encoded in the "Rm" field.
<Ra>	Is the third general-purpose source register holding the addend, encoded in the "Ra" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   operand2 = if m_high then R[m][31:16] else R[m][15:0];
4   result = SInt(R[n]) * SInt(operand2) + (SInt(R[a]) << 16);
5   R[d] = result[47:16];
6   if (result >> 16) != SInt(R[d]) then // Signed overflow
7     APSR.Q = '1';

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.193 SMLSD, SMLSDX

Signed Multiply Subtract Dual. Signed Multiply Subtract Dual performs two signed 16-bit by 16-bit multiplications. It adds the difference of the products to a 32-bit accumulate operand.

Optionally, the instruction can exchange the halfwords of the second operand before performing the arithmetic.

This instruction sets the Q flag if the accumulate operation overflows. Overflow cannot occur during the multiplications or subtraction.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	1	1	0	1	0	0	Rn				Ra				Rd				0 0 0				M	Rm			

SMLSD variant

Applies when **M == 0**.

SMLSD{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

SMLSDX variant

Applies when **M == 1**.

SMLSDX{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

Decode for this encoding

```

1 if Ra == '1111' then SEE SMUSD;
2 if !HaveDSPEExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); a = UInt(Ra); m_swap = (M == '1');
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} || a == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<Ra>	Is the third general-purpose source register holding the addend, encoded in the "Ra" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     operand2 = if m_swap then ROR(R[m],16) else R[m];
4     product1 = SInt(R[n][15:0]) * SInt(operand2[15:0]);
5     product2 = SInt(R[n][31:16]) * SInt(operand2[31:16]);
6     result = product1 - product2 + SInt(R[a]);
7     R[d] = result[31:0];
8     if result != SInt(result[31:0]) then // Signed overflow
9         APSR.Q = '1';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.194 SMLS LD, SMLS LDx

Signed Multiply Subtract Long Dual. Signed Multiply Subtract Long Dual performs two signed 16-bit by 16-bit multiplications. It adds the difference of the products to a 64-bit accumulate operand.

Optionally, the instruction can exchange the halfwords of the second operand before performing the arithmetic.

Overflow is possible during this instruction, but only as a result of the 64-bit addition. This overflow is not detected if it occurs. Instead, the result wraps around modulo 2^{64} .

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	1	0	1	Rn			RdLo			RdHi			1			1	0	M	Rm				

SMLS LD variant

Applies when **M** == 0.

SMLS LD{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

SMLS LDx variant

Applies when **M** == 1.

SMLS LDx{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 dLo = UInt(RdLo); dHi = UInt(RdHi); n = UInt(Rn); m = UInt(Rm); m_swap = (M == '1');
3 if dLo IN {13,15} || dHi IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
4 if dHi == dLo then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **dHi** == **dLo**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<RdLo>	Is the general-purpose source register holding the lower 32 bits of the addend, and the destination register for the lower 32 bits of the result, encoded in the "RdLo" field.
<RdHi>	Is the general-purpose source register holding the upper 32 bits of the addend, and the destination register for the upper 32 bits of the result, encoded in the "RdHi" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     operand2 = if m_swap then ROR(R[m], 16) else R[m];
```

```
4    product1 = SInt(R[n][15:0]) * SInt(operand2[15:0]);
5    product2 = SInt(R[n][31:16]) * SInt(operand2[31:16]);
6    result = product1 - product2 + SInt(R[dHi]:R[dLo]);
7    R[dHi] = result[63:32];
8    R[dLo] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.195 SMMLA, SMMLAR

Signed Most Significant Word Multiply Accumulate. Signed Most Significant Word Multiply Accumulate multiplies two signed 32-bit values, extracts the most significant 32 bits of the result, and adds an accumulate value.

Optionally, the instruction can specify that the result is rounded instead of being truncated. In this case, the constant 0x80000000 is added to the product before the high word is extracted.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	1	0	1	Rn				Ra				Rd				0 0 0 R				Rm			

SMMLA variant

Applies when **R == 0**.

SMMLA{<c>} {<q>} <Rd>, <Rn>, <Rm>, <Ra>

SMMLAR variant

Applies when **R == 1**.

SMMLAR{<c>} {<q>} <Rd>, <Rn>, <Rm>, <Ra>

Decode for this encoding

```

1 if Ra == '1111' then SEE SMMUL;
2 if Rd == '1111' then SEE "AUTG and BXAUT";
3 if !HaveDSPExt() then UNDEFINED;
4 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); a = UInt(Ra); round = (R == '1');
5 if d == 13 || n IN {13,15} || m IN {13,15} || a == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Ra>	Is the third general-purpose source register holding the addend, encoded in the "Ra" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = (SInt(R[a]) << 32) + SInt(R[n]) * SInt(R[m]);
4     if round then result = result + 0x80000000;
5     R[d] = result[63:32];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.196 SMMLS, SMMLSR

Signed Most Significant Word Multiply Subtract. Signed Most Significant Word Multiply Subtract multiplies two signed 32-bit values, subtracts the result from a 32-bit accumulate value that is shifted left by 32 bits, and extracts the most significant 32 bits of the result of that subtraction.

Optionally, the instruction can specify that the result of the instruction is rounded instead of being truncated. In this case, the constant 0x80000000 is added to the result of the subtraction before the high word is extracted.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	1	1	0	Rn				Ra				Rd				0	0	0	R	Rm			

SMMLS variant

Applies when **R == 0**.

SMMLS{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

SMMLSR variant

Applies when **R == 1**.

SMMLSR{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

Decode for this encoding

```

1 if Ra == '1111' && R == '0' then SEE PACG;
2 if !HaveDSPEXT() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); a = UInt(Ra); round = (R == '1');
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} || a IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Ra>	Is the third general-purpose source register holding the addend, encoded in the "Ra" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = (SInt(R[a]) << 32) - SInt(R[n]) * SInt(R[m]);
4     if round then result = result + 0x80000000;
5     R[d] = result[63:32];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.197 SMMUL, SMMULR

Signed Most Significant Word Multiply. Signed Most Significant Word Multiply multiplies two signed 32-bit values, extracts the most significant 32 bits of the result, and writes those bits to the destination register.

Optionally, the instruction can specify that the result is rounded instead of being truncated. In this case, the constant 0x80000000 is added to the product before the high word is extracted.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	1	0	1	Rn				1	1	1	1	Rd				0	0	0	R	Rm			

SMMUL variant

Applies when R == 0.

SMMUL{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

SMMULR variant

Applies when R == 1.

SMMULR{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); round = (R == '1');
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = SInt(R[n]) * SInt(R[m]);
4     if round then result = result + 0x80000000;
5     R[d] = result[63:32];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.198 SMUAD, SMUADX

Signed Dual Multiply Add. Signed Dual Multiply Add performs two signed 16-bit by 16-bit multiplications. It adds the products together, and writes the result to the destination register.

Optionally, the instruction can exchange the halfwords of the second operand before performing the arithmetic.

This instruction sets the Q flag if the addition overflows. The multiplications cannot overflow.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	0	1	0	Rn				1	1	1	1	Rd				0	0	0	M	Rm			

SMUAD variant

Applies when **M == 0**.

SMUAD{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

SMUADX variant

Applies when **M == 1**.

SMUADX{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); m_swap = (M == '1');
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   operand2 = if m_swap then ROR(R[m],16) else R[m];
4   product1 = SInt(R[n][15:0]) * SInt(operand2[15:0]);
5   product2 = SInt(R[n][31:16]) * SInt(operand2[31:16]);
6   result = product1 + product2;
7   R[d] = result[31:0];
8   if result != SInt(result[31:0]) then // Signed overflow
9     APSR.Q = '1';
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.199 SMULBB, SMULBT, SMULTB, SMULTT

Signed Multiply (halfwords). Signed Multiply (halfwords) multiplies two signed 16-bit quantities, taken from either the bottom or the top half of their respective source registers. The other halves of these source registers are ignored. The 32-bit product is written to the destination register. No overflow is possible during this instruction.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	0	0	1	Rn				1	1	1	1	Rd				0	0	N	M	Rm			

SMULBB variant

Applies when **N == 0** && **M == 0**.

SMULBB{<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

SMULBT variant

Applies when **N == 0** && **M == 1**.

SMULBT{<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

SMULTB variant

Applies when **N == 1** && **M == 0**.

SMULTB{<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

SMULTT variant

Applies when **N == 1** && **M == 1**.

SMULTT{<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 n_high = (N == '1'); m_high = (M == '1');
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the multiplicand in the bottom or top half (selected by <x>), encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier in the bottom or top half (selected by <y>), encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   operand1 = if n_high then R[n][31:16] else R[n][15:0];
4   operand2 = if m_high then R[m][31:16] else R[m][15:0];
5   result = SInt(operand1) * SInt(operand2);
```

```
6   R[d] = result[31:0];  
7   // Signed overflow cannot occur
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.200 SMULL

Signed Multiply Long. Signed Multiply Long multiplies two 32-bit signed values to produce a 64-bit result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	0	0	0	Rn				RdLo				RdHi				0	0	0	0	Rm			

T1 variant

SMULL{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 dLo = UInt(RdLo); dHi = UInt(RdHi); n = UInt(Rn); m = UInt(Rm); setflags = FALSE;
3 if dLo IN {13,15} || dHi IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
4 if dHi == dLo then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `dHi == dLo`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<RdLo>	Is the general-purpose destination register for the lower 32 bits of the result, encoded in the "RdLo" field.
<RdHi>	Is the general-purpose destination register for the upper 32 bits of the result, encoded in the "RdHi" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = SInt(R[n]) * SInt(R[m]);
4     R[dHi] = result[63:32];
5     R[dLo] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.201 SMULWB, SMULWT

Signed Multiply (word by halfword). Signed Multiply (word by halfword) multiplies a signed 32-bit quantity and a signed 16-bit quantity. The signed 16-bit quantity is taken from either the bottom or the top half of its source register. The other half of the second source register is ignored. The top 32 bits of the 48-bit product are written to the destination register. The bottom 16 bits of the 48-bit product are ignored. No overflow is possible during this instruction.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	0	1	1	Rn				1	1	1	1	Rd				0	0	0	M	Rm			

SMULWB variant

Applies when **M == 0**.

SMULWB{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

SMULWT variant

Applies when **M == 1**.

SMULWT{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); m_high = (M == '1');
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier in the bottom or top half (selected by <y>), encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     operand2 = if m_high then R[m][31:16] else R[m][15:0];
4     product = SInt(R[n]) * SInt(operand2);
5     R[d] = product[47:16];
6     // Signed overflow cannot occur
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.202 SMUSD, SMUSDX

Signed Dual Multiply Subtract. Signed Dual Multiply Subtract performs two signed 16-bit by 16-bit multiplications. It subtracts one of the products from the other, and writes the result to the destination register.

Optionally, the instruction can exchange the halfwords of the second operand before performing the arithmetic.

Overflow cannot occur.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	1	0	0	Rn				1	1	1	1	Rd				0	0	0	M	Rm			

SMUSD variant

Applies when **M == 0**.

SMUSD{<c>}{<q>}{<Rd>, } <Rn>, <Rm>

SMUSDX variant

Applies when **M == 1**.

SMUSDX{<c>}{<q>}{<Rd>, } <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); m_swap = (M == '1');
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     operand2 = if m_swap then ROR(R[m],16) else R[m];
4     product1 = SInt(R[n][15:0]) * SInt(operand2[15:0]);
5     product2 = SInt(R[n][31:16]) * SInt(operand2[31:16]);
6     result = product1 - product2;
7     R[d] = result[31:0];
8     // Signed overflow cannot occur
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.203 SQRSHR (register)

Signed Saturating Rounding Shift Right. Signed saturating rounding shift right by 0 to 32 bits of a 32-bit value stored in a general-purpose register. The shift amount is read in as the bottom byte of Rm. If the shift amount is negative, the shift direction is reversed.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1																				

T1: SQRSHR variant

SQRSHR<c><q> Rda, Rm

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 if !HasArchVersion(Armv8pl) then SEE "Related encodings";
3 if !HaveMve() then UNDEFINED;
4 da = UInt(Rda);
5 m = UInt(Rm);
6 if Rda == '11x1' || Rm == '11x1' || Rm == Rda then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rda> General-purpose source and destination register, containing the value to be shifted.
 <Rm> General-purpose source register holding a shift amount in its bottom 8 bits.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   amount = SInt(R[m][7:0]);
5   opl = SInt(R[da]);
6   opl = opl + (1 << (amount - 1));
7   (result, sat) = SignedSatQ((opl >> amount), 32);
8   if sat then APSR.Q = '1';
9   R[da] = result[31:0];
```

C2.4.204 SQRSHRL (register)

Signed Saturating Rounding Shift Right Long. Signed saturating rounding shift right by 0 to 64 bits of a 64-bit value stored in two general-purpose registers. The shift amount is read in as the bottom byte of Rm. If the shift amount is negative, the shift direction is reversed.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	1			Rm			RdaHi	(1)	sat	(0)	1	0	1	1	0	1			

T1: SQRSHRL variant

SQRSHRL<c><q> RdaLo, RdaHi, #<saturate>, Rm

Decode for this encoding

```

1 if RdaHi == '111' then SEE "SQRSHR (register)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 m = UInt(Rm);
8 saturateTo = if sat == '0' then 64 else 48;
9 if RdaHi == '110' || Rm == '11x1' || Rm == RdaHi:'1' then CONSTRAINED_UNPREDICTABLE;
10 if Rm == RdaLo:'0' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
<saturate>	The bit position for saturation. This parameter must be one of the following values: #64 Encoded as sat = 0 #48 Encoded as sat = 1
<Rm>	General-purpose source register holding a shift amount in its bottom 8 bits.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   amount = SInt(R[m][7:0]);
5   op1 = SInt(R[dah]:R[dal]);
6   op1 = op1 + (1 << (amount - 1));
7   (shiftedOp, didSat) = SignedSatQ((op1 >> amount), saturateTo);
8   result = SignExtend(shiftedOp, 64);
9   if didSat then APSR.Q = '1';
10  R[dah] = result[63:32];
11  R[dal] = result[31:0];

```

C2.4.205 SQSHL (immediate)

Signed Saturating Shift Left. Signed saturating shift left by 1 to 32 bits of a 32-bit value stored in a general-purpose register.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	1	0	1	0	0	1	0	1	Rda				0	immh				1	1	1	(1)	imml				1	1	1	1

T1: SQSHL variant

SQSHL<c><q> Rda, #<imm>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if !HasArchVersion(Armv8pl) then SEE "Related encodings";
3 if !HaveMve() then UNDEFINED;
4 da = UInt(Rda);
5 (-, amount) = DecodeImmShift('10', immh:imml);
6 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

- <c, q> See C1.2.5 Standard assembler syntax fields.
- <Rda> General-purpose source and destination register, containing the value to be shifted.
- <imm> The number of bits to shift by, in the range 1-32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   op1 = SInt(R[da]);
5   (result, sat) = SignedSatQ((op1 << amount), 32);
6   if sat then APSR.Q = '1';
7   R[da] = result[31:0];
```

C2.4.206 SQSHLL (immediate)

Signed Saturating Shift Left Long. Signed saturating shift left by 1 to 32 bits of a 64-bit value stored in two general-purpose registers.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	(1)	0	immh	RdaHi	(1)	imml	1	1	1	1	1	1	1	1	1	1	1	1	

T1: SQSHLL variant

SQSHLL<c><q> RdaLo, RdaHi, #<imm>

Decode for this encoding

```

1 if RdaHi == '111' then SEE "SQSHL (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 (-, amount) = DecodeImmShift('10', immh:immh);
8 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
<imm>	The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     op1 = SInt(R[dah]:R[dal]);
5     (result, sat) = SignedSatQ((op1 << amount), 64);
6     if sat then APSR.Q = '1';
7     R[dah] = result[63:32];
8     R[dal] = result[31:0];
```


C2.4.207 SRRSHR (immediate)

Signed Rounding Shift Right. Signed rounding shift right by 1 to 32 bits of a 32-bit value stored in a general-purpose register.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	1	0	1	0	0	1	0	1		Rda		0		immh		1	1	1	(1)		imml		1	0		1	1	1	1

T1: SRRSHR variant

SRRSHR<c><q> Rda, #<imm>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 if !HasArchVersion(Armv8pl) then SEE "Related encodings";
3 if !HaveMve() then UNDEFINED;
4 da = UInt(Rda);
5 (-, amount) = DecodeImmShift('10', immh:imml);
6 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields.
<Rda>	General-purpose source and destination register, containing the value to be shifted.
<imm>	The number of bits to shift by, in the range 1-32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   opl = SInt(R[da]);
5   opl = opl + (1 << (amount - 1));
6   result = (opl >> amount)[31:0];
7   R[da] = result[31:0];
```

C2.4.208 SRSHRL (immediate)

Signed Rounding Shift Right Long. Signed rounding shift right by 1 to 32 bits of a 64-bit value stored in two general-purpose registers.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	1	0	immh	RdaHi	(1)	imml	1	0	1	1	1	1	1	1	1	1	1	1	

T1: SRSHRL variant

SRSHRL<c><q> RdaLo, RdaHi, #<imm>

Decode for this encoding

```

1 if RdaHi == '111' then SEE "SRSHR (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 (-, amount) = DecodeImmShift('10', immh:immh);
8 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
<imm>	The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   opl = SInt(R[dah]:R[dal]);
5   opl = opl + (1 << (amount - 1));
6   result = (opl >> amount)[63:0];
7   R[dah] = result[63:32];
8   R[dal] = result[31:0];
```

C2.4.209 SSAT

Signed Saturate. Signed Saturate saturates an optionally-shifted signed value to a selectable signed range.

The APSR.Q flag is set to 1 if the operation saturates.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	(0)	1	1	0	0	sh	0	Rn	0	imm3	Rd	imm2	(0)	sat_imm													

Arithmetic shift right variant

Applies when **sh == 1 && !(imm3 == 000 && imm2 == 00)**.

SSAT{<c>}{<q>} <Rd>, #<imm>, <Rn>, ASR #<amount>

Logical shift left variant

Applies when **sh == 0**.

SSAT{<c>}{<q>} <Rd>, #<imm>, <Rn> {, LSL #<amount>}

Decode for this encoding

```

1 if sh == '1' && (imm3:imm2) == '0000' then
2     if HaveDSPExt() then
3         SEE SSAT16;
4     else
5         UNDEFINED;
6 if !HaveMainExt() then UNDEFINED;
7 d = UInt(Rd); n = UInt(Rn); saturate_to = UInt(sat_imm)+1;
8 (shift_t, shift_n) = DecodeImmShift(sh:'0', imm3:imm2);
9 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<imm>	Is the bit position for saturation, in the range 1 to 32, encoded in the "sat_imm" field as <imm>-1.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<amount>	For the arithmetic shift right variant: is the shift amount, in the range 1 to 31 encoded in the "imm3:imm2" field as <amount>. For the logical shift left variant: is the optional shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm3:imm2" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     operand = Shift(R[n], shift_t, shift_n, APSR.C); // APSR.C ignored
4     (result, sat) = SignedSatQ(SInt(operand), saturate_to);
5     R[d] = SignExtend(result, 32);
6     if sat then
7         APSR.Q = '1';
```

C2.4.210 SSAT16

Signed Saturate 16. Signed Saturate 16 saturates two signed 16-bit values to a selected signed range.

The APSR.Q flag is set to 1 if the operation saturates.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	(0)	1	1	0	0	1	0	Rn				0	0	0	0	Rd				0	0	(0)	(0)	sat_imm			

T1 variant

SSAT16{<c>}{<q>} <Rd>, #<imm>, <Rn>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); saturate_to = UInt(sat_imm)+1;
3 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<imm>	Is the bit position for saturation, in the range 1 to 16, encoded in the "sat_imm" field as <imm>-1.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     (result1, sat1) = SignedSatQ(SInt(R[n][15:0]), saturate_to);
4     (result2, sat2) = SignedSatQ(SInt(R[n][31:16]), saturate_to);
5     bits(32) result;
6     result[15:0] = SignExtend(result1, 16);
7     result[31:16] = SignExtend(result2, 16);
8     R[d] = result;
9     if sat1 || sat2 then
10         APSR.Q = '1';

```

C2.4.211 SSAX

Signed Subtract and Add with Exchange. Signed Subtract and Add with Exchange exchanges the two halfwords of the second operand, performs one 16-bit integer subtraction and one 16-bit addition, and writes the results to the destination register. It sets the APSR.GE bits according to the results.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	1	0	Rn				1	1	1	1	Rd				0	0	0	0	Rm			

T1 variant

SSAX{<c>}{<q>}{<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum = SInt(R[n][15:0]) + SInt(R[m][31:16]);
4   diff = SInt(R[n][31:16]) - SInt(R[m][15:0]);
5   R[d] = diff[15:0] : sum[15:0];
6   APSR.GE[1:0] = if sum >= 0 then '11' else '00';
7   APSR.GE[3:2] = if diff >= 0 then '11' else '00';
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.212 SSBB

Speculative Store Bypass Barrier. Speculative Store Bypass Barrier is a memory barrier which prevents speculative loads from bypassing earlier stores to the same address under certain conditions.

The semantics of the Speculative Store Bypass Barrier are:

- When a load to a location appears in program order after the SSBB, then the load does not speculatively read an entry earlier in the coherence order for that location than the entry generated by the latest store satisfying all of the following conditions:
 - The store is to the same location as the load.
 - The store appears in program order before the SSBB.
- When a load to a location appears in program order before the SSBB, then the load does not speculatively read data from any store satisfying all of the following conditions:
 - The store is to the same location as the load.
 - The store appears in program order after the SSBB.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	1	(1)	(1)	(1)	(1)	1	0	(0)	0	(1)	(1)	(1)	(1)	0	1	0	0	0	0	0	0

T1 variant

SSBB{<q>}

Decode for this encoding

```
1 if InitBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<q> See C1.2.5 Standard assembler syntax fields.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 SpeculativeSynchronizationBarrier();
```

C2.4.213 SSUB16

Signed Subtract 16. Signed Subtract 16 performs two 16-bit signed integer subtractions, and writes the results to the destination register. It sets the APSR.GE bits according to the results of the subtractions.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	1	Rn				1	1	1	1	Rd				0	0	0	0	Rm			

T1 variant

SSUB16{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   diff1 = SInt(R[n][15:0]) - SInt(R[m][15:0]);
4   diff2 = SInt(R[n][31:16]) - SInt(R[m][31:16]);
5   R[d] = diff2[15:0] : diff1[15:0];
6   APSR.GE[1:0] = if diff1 >= 0 then '11' else '00';
7   APSR.GE[3:2] = if diff2 >= 0 then '11' else '00';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.214 SSUB8

Signed Subtract 8. Signed Subtract 8 performs four 8-bit signed integer subtractions, and writes the results to the destination register. It sets the APSR.GE bits according to the results of the subtractions.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	0	Rn				1	1	1	1	Rd				0	0	0	0	Rm			

T1 variant

SSUB8{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff1 = SInt(R[n][7:0]) - SInt(R[m][7:0]);
4     diff2 = SInt(R[n][15:8]) - SInt(R[m][15:8]);
5     diff3 = SInt(R[n][23:16]) - SInt(R[m][23:16]);
6     diff4 = SInt(R[n][31:24]) - SInt(R[m][31:24]);
7     R[d] = diff4[7:0] : diff3[7:0] : diff2[7:0] : diff1[7:0];
8     APSR.GE[0] = if diff1 >= 0 then '1' else '0';
9     APSR.GE[1] = if diff2 >= 0 then '1' else '0';
10    APSR.GE[2] = if diff3 >= 0 then '1' else '0';
11    APSR.GE[3] = if diff4 >= 0 then '1' else '0';
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.215 STC, STC2

Store Coprocessor. Store Coprocessor stores data from a coprocessor to a sequence of consecutive memory addresses.

If no coprocessor can execute the instruction, a UsageFault exception is generated.

Arm reserves coprocessors CP8 to CP15, and this manual defines the valid instructions when coproc is in this range.

This instruction is subject to stack limit checking.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	P	U	D	W	0	Rn				CRd				coproc				imm8							

Offset variant

Applies when **P == 1 && W == 0**.

STC{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>{, #<+/-><imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

STC{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>], #<+/-><imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

STC{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>, #<+/-><imm>]!

Unindexed variant

Applies when **P == 0 && U == 1 && W == 0**.

STC{L}{<c>}{<q>} <coproc>, <CRd>, [<Rn>], <option>

Decode for this encoding

```

1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 if P == '0' && U == '0' && D == '1' && W == '0' then SEE "MCRR, MCRR2";
4 cp = UInt(coproc);
5 CheckCPDecodeFaults(cp);
6 if P == '0' && U == '0' && D == '0' && W == '0' then UNDEFINED;
7 if !HaveMainExt() then UNDEFINED;
8 n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
9 index = (P == '1'); add = (U == '1'); wback = (W == '1');
10 if n == 15 then UNPREDICTABLE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	P	U	D	W	0	Rn				CRd				coproc				imm8							

Offset variant

Applies when **P == 1 && W == 0**.

STC2{L}{<c>}{<q>}<coproc>, <CRd>, [<Rn>{, # {+/-}<imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

STC2{L}{<c>}{<q>}<coproc>, <CRd>, [<Rn>], # {+/-}<imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

STC2{L}{<c>}{<q>}<coproc>, <CRd>, [<Rn>, # {+/-}<imm>]!

Unindexed variant

Applies when **P == 0 && U == 1 && W == 0**.

STC2{L}{<c>}{<q>}<coproc>, <CRd>, [<Rn>], <option>

Decode for this encoding

```
1 if coproc IN {'100x', '101x', '111x'} then SEE "Floating-point and MVE";
2 if CoprocType(UInt(coproc)) == CP_CDEv1 then SEE "CDE instructions";
3 if P == '0' && U == '0' && D == '1' && W == '0' then SEE "MCRr, MCRR2";
4 cp = UInt(coproc);
5 CheckCPDecodeFaults(cp);
6 if P == '0' && U == '0' && D == '0' && W == '0' then UNDEFINED;
7 if !HaveMainExt() then UNDEFINED;
8 n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
9 index = (P == '1'); add = (U == '1'); wback = (W == '1');
10 if n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

L	If specified, selects the D == 1 form of the encoding. If omitted, selects the D == 0 form.
<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoded in the "coproc" field. The generic coprocessor names are p0 to p15.
<CRd>	Is the coprocessor register to be transferred, encoded in the "CRd" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
<option>	Is a coprocessor option, in the range 0 to 255 enclosed in { }, encoded in the "imm8" field.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	Is the immediate offset used for forming the address, a multiple of 4 in the range 0-1020, defaulting to 0 and encoded in the "imm8" field, as <imm>/4.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     if !Coproc_Accepted(cp, ThisInstr()) then
5         GenerateCoproprocessorException();
6     else
7         offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
```

```
8      address = if index then offset_addr else R[n];
9
10     // Determine if the stack pointer limit check should be performed
11     if wback && n == 13 then
12         violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
13     else
14         violatesLimit = FALSE;
15
16     // Memory operation only performed if limit not violated
17     if !violatesLimit then
18         repeat
19             MemA[address, 4] = Coproc_GetWordToStore(cp, ThisInstr());
20             address = address + 4;
21         until Coproc_DoneStoring(cp, ThisInstr());
22
23     // If the stack pointer is being updated a fault will be raised
24     // if the limit is violated
25     if wback then RSPCheck[n] = offset_addr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.216 STL

Store-Release Word. Store Release Word stores a word from a register to memory. The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	0	Rn				Rt				(1)	(1)	(1)	(1)	1	0	1	0	(1)	(1)	(1)	(1)

T1 variant

STL{<c>}{<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n];
4     MemO[address, 4] = R[t];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.217 STL

Store-Release Byte. Store Release Byte stores a byte from a register to memory. The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	0	Rn				Rt				(1)	(1)	(1)	(1)	1	0	0	0	(1)	(1)	(1)	(1)

T1 variant

STLB{<c>}{<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n];
4     MemO[address, 1] = R[t][7:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.218 STLEX

Store-Release Exclusive Word. Store Release Exclusive Word stores a word from a register to memory if the executing PE has exclusive access to the memory addressed. The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	0	Rn				Rt				(1)	(1)	(1)	(1)	1	1	1	0	Rd			

T1 variant

STLEX{<c>}{<q>} <Rd>, <Rt>, [<Rn>]

Decode for this encoding

```
1 d = UInt(Rd); t = UInt(Rt); n = UInt(Rn);
2 if d IN {13,15} || t IN {13,15} || n == 15 then UNPREDICTABLE;
3 if d == n || d == t then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $d == t$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

CONSTRAINED UNPREDICTABLE behavior

If $d == n$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs the store to an UNKNOWN address.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the destination general-purpose register into which the status result of the store exclusive is written, encoded in the "Rd" field. The value returned is: 1 If the operation fails to update memory. 0 If the operation updates memory.
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n];
4   if ExclusiveMonitorsPass(address,4) then
5     MemO[address, 4] = R[t];
6     R[d] = ZeroExtend('0');
7   else
8     R[d] = ZeroExtend('1');
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.219 STLEXB

Store-Release Exclusive Byte. Store Release Exclusive Byte stores a byte from a register to memory if the executing PE has exclusive access to the memory addressed. The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	0	Rn				Rt				(1)	(1)	(1)	(1)	1	1	0	0	Rd			

T1 variant

STLEXB{<c>}{<q>} <Rd>, <Rt>, [<Rn>]

Decode for this encoding

```

1 d = UInt(Rd); t = UInt(Rt); n = UInt(Rn);
2 if d IN {13,15} || t IN {13,15} || n == 15 then UNPREDICTABLE;
3 if d == n || d == t then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $d == t$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

CONSTRAINED UNPREDICTABLE behavior

If $d == n$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs the store to an UNKNOWN address.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the destination general-purpose register into which the status result of the store exclusive is written, encoded in the "Rd" field. The value returned is: 1 If the operation fails to update memory. 0 If the operation updates memory.
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n];
4   if ExclusiveMonitorsPass(address,1) then
5     MemO[address, 1] = R[t][7:0];
6     R[d] = ZeroExtend('0');
7   else
8     R[d] = ZeroExtend('1');
```


Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.220 STLEXH

Store-Release Exclusive Halfword. Store Release Exclusive Halfword stores a halfword from a register to memory if the executing PE has exclusive access to the memory addressed. The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	0	Rn				Rt				(1)	(1)	(1)	(1)	1	1	0	1	Rd			

T1 variant

STLEXH{<c>}{<q>} <Rd>, <Rt>, [<Rn>]

Decode for this encoding

```

1 d = UInt(Rd); t = UInt(Rt); n = UInt(Rn);
2 if d IN {13,15} || t IN {13,15} || n == 15 then UNPREDICTABLE;
3 if d == n || d == t then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $d == t$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

CONSTRAINED UNPREDICTABLE behavior

If $d == n$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs the store to an UNKNOWN address.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the destination general-purpose register into which the status result of the store exclusive is written, encoded in the "Rd" field. The value returned is: 1 If the operation fails to update memory. 0 If the operation updates memory.
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n];
4   if ExclusiveMonitorsPass(address, 2) then
5     MemO[address, 2] = R[t][15:0];
6     R[d] = ZeroExtend('0');
```

```
7     else  
8         R[d] = ZeroExtend('1');
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.221 STLH

Store-Release Halfword. Store Release Halfword stores a halfword from a register to memory. The instruction also has memory ordering semantics.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	0	Rn				Rt				(1)	(1)	(1)	(1)	1	0	0	1	(1)	(1)	(1)	(1)

T1 variant

STLH{<c>}{<q>} <Rt>, [<Rn>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn);
2 if t IN {13,15} || n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Is the general-purpose register to be transferred, encoded in the "Rt" field.
 <Rn> Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n];
4     MemO[address, 2] = R[t][15:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.222 STM, STMIA, STMEA

Store Multiple. Store Multiple stores multiple registers to consecutive memory locations using an address from a base register. The consecutive memory locations start at this address, and the address just above the last of those locations can optionally be written back to the base register.

This instruction is interrupt-continuable.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	Rn			register_list							

T1 variant

```
STM{IA}{<c>}{<q>} <Rn>!, <registers>
    // Preferred syntax
STMEA{<c>}{<q>} <Rn>!, <registers>
    // Alternate syntax, Empty Ascending stack
```

Decode for this encoding

```
1 n = UInt(Rn); registers = '00000000':register_list; wback = TRUE;
2 if BitCount(registers) < 1 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) < 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as an STM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15. If the instruction specifies write-back, the modification to the base address on write-back might differ from the number of registers stored.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	0	W	0	Rn			(0)	M	(0)	register_list													

T2 variant

```
STM{IA}{<c>}.W <Rn>{!}, <registers>
    // Preferred syntax
    // if <Rn>, '!' and <registers> can be represented in T1
STMEA{<c>}.W <Rn>{!}, <registers>
    // Alternate syntax
    // Empty Ascending stack
    // if <Rn>, '!' and <registers> can be represented in T1
STM{IA}{<c>}{<q>} <Rn>{!}, <registers>
    // Preferred syntax
```

```
STMEA{<c>}{<q>} <Rn>{!}, <registers>
// Alternate syntax, Empty Ascending stack
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); registers = '0':M:'0':register_list; wback = (W == '1');
3 if n == 15 || BitCount(registers) < 2 then UNPREDICTABLE;
4 if wback && registers[n] == '1' then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) < 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as an STM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15. If the instruction specifies write-back, the modification to the base address on write-back might differ from the number of registers stored.

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) == 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction executes as described, with no change to its behavior and no additional side effects.
- The instruction operates as an STM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15.

CONSTRAINED UNPREDICTABLE behavior

If `wback && registers<n> == '1'`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored for the base register is UNKNOWN.

Assembler symbols for all encodings

IA	Is an optional suffix for the Increment After form.
<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
!	The address adjusted by the size of the data loaded is written back to the base register. If specified, it is encoded in the "W" field as 1, otherwise this field defaults to 0.
<registers>	For encoding T1: is a list of one or more registers to be stored, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R7, encoded in the "register_list" field. If the base register is not the lowest-numbered register in the list, such an instruction stores an UNKNOWN value for the base register. For encoding T2: is a list of one or more registers to be stored, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R12, encoded in the "register_list" field, and can optionally contain the LR. If the LR is in the list, the "M" field is set to 1, otherwise it defaults to 0.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      address      = R[n];
4      endAddress = R[n] + 4*BitCount(registers);
5
6      // Determine if the stack pointer limit should be checked
7      if n == 13 && wback && registers[n] == '0' then
8          violatesLimit = ViolatesSPLim(LookUpSP(), endAddress);
9      else
10         violatesLimit = FALSE;
11
12     for i = 0 to 14
13         // Memory operation only performed if limit not violated
14         if registers[i] == '1' && !violatesLimit then
15             if i == n && wback && i != LowestSetBit(registers) then
16                 MemA[address, 4] = bits(32) UNKNOWN; // encoding T1 only
17             else
18                 MemA[address, 4] = R[i];
19                 address = address + 4;
20
21         // If the stack pointer is being updated a fault will be raised if
22         // the limit is violated
23         if wback then RSPCheck[n] = endAddress;

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value or values being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.223 STMDB, STMFD

Store Multiple Decrement Before (Full Descending). Store Multiple Decrement Before stores multiple registers to consecutive memory locations using an address from a base register. The consecutive memory locations end just below this address, and the address of the first of those locations can optionally be written back to the base register.

This instruction is used by the alias [PUSH \(multiple registers\)](#).

This instruction is interrupt-continuable.

This instruction is subject to stack limit checking.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	1	0	0	W	0	Rn				(0)	M	(0)	register_list												

T1 variant

```
STMDB{<c>}{<q>} <Rn>{!}, <registers>
// Preferred syntax
STMFD{<c>}{<q>} <Rn>{!}, <registers>
// Alternate syntax, Full Descending stack
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); registers = '0':M:'0':register_list; wback = (W == '1');
3 if n == 15 || BitCount(registers) < 2 then UNPREDICTABLE;
4 if wback && registers[n] == '1' then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) < 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as an STM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15. If the instruction specifies write-back, the modification to the base address on write-back might differ from the number of registers stored.

CONSTRAINED UNPREDICTABLE behavior

If `wback && registers<n> == '1'`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored for the base register is UNKNOWN.

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) == 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.

- The instruction executes as described, with no change to its behavior and no additional side effects.
- The instruction operates as an STM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15.

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	1	0	M	register_list							

T2 variant

STMDB{<c>}{<q>} SP!, <registers>

Decode for this encoding

```
1 n = 13; wback = TRUE;
2 registers = '0':M:'00000':register_list;
3 if BitCount(registers) < 1 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `BitCount(registers) < 1`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as an STM with the same addressing mode but targeting an unspecified set of registers. These registers might include R15. If the instruction specifies write-back, the modification to the base address on write-back might differ from the number of registers stored.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
!	The address adjusted by the size of the data loaded is written back to the base register. If specified, it is encoded in the "W" field as 1, otherwise this field defaults to 0.
<registers>	For encoding T1: is a list of one or more registers to be stored, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R12, encoded in the "register_list" field, and can optionally contain the LR. If the LR is in the list, the "M" field is set to 1, otherwise it defaults to 0. For encoding T2: is a list of one or more registers to be stored, separated by commas and surrounded by { and }. The registers in the list must be in the range R0-R7, encoded in the "register_list" field, and can optionally include the LR. If the LR is in the list, the "M" field is set to 1, otherwise this field defaults to 0.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n] - 4*BitCount(registers);
4   applyLimit = n == 13 && wback;
5
6   for i = 0 to 14
7     // If R[n] is the SP, memory operation only performed if limit not violated
8     if registers[i] == '1' && !(applyLimit && ViolatesSPLim(LookUpSP(), address)) then
9       MemA[address, 4] = R[i];
10      address = address + 4;
```

```
11
12 // If R[n] is the SP, stack pointer update will raise a fault if limit violated
13 if wback then RSPCheck[n] = R[n] - 4*BitCount(registers);
```

Restricted behavior

Data Independent Timing behavior

If AIRCR.DIT is 1:

- The execution time of this instruction is insensitive to the value or values of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value or values being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.224 STR (immediate)

Store Register (immediate). Store Register (immediate) calculates an address from a base register value and an immediate offset, and stores a word from a register to memory. It can use offset, post-indexed, or pre-indexed addressing.

This instruction is subject to stack limit checking.

This instruction is used by the alias [PUSH \(single register\)](#).

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	imm5					Rn			Rt		

T1 variant

STR{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm5:'00', 32);
2 index = TRUE; add = TRUE; wback = FALSE;
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	0	Rt			imm8							

T2 variant

STR{<c>}{<q>} <Rt>, [SP{, #<+><imm>}]

Decode for this encoding

```
1 t = UInt(Rt); n = 13; imm32 = ZeroExtend(imm8:'00', 32);
2 index = TRUE; add = TRUE; wback = FALSE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	1	1	0	0	Rn				Rt				imm12											

T3 variant

STR{<c>}.W <Rt>, [<Rn> {, #<+><imm>}]
 // <Rt>, <Rn>, <imm> can be represented in T1 or T2
 STR{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```

1 if Rn == '1111' then UNDEFINED;
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm12, 32);
4 index = TRUE; add = TRUE; wback = FALSE;
5 if t == 15 then UNPREDICTABLE;

```

T4

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	1	0	0	Rn				Rt				1	P	U	W	imm8							

Offset variant

Applies when **P == 1 && U == 0 && W == 0**.

STR{<c>}{<q>} <Rt>, [<Rn> {, #-<imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

STR{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

STR{<c>}{<q>} <Rt>, [<Rn>, #{+/-}<imm>]!

Decode for this encoding

```

1 if P == '1' && U == '1' && W == '0' then SEE STRT;
2 if Rn == '1111' || (P == '0' && W == '0') then UNDEFINED;
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm8, 32);
5 index = (P == '1'); add = (U == '1'); wback = (W == '1');
6 if t == 15 || (wback && n == t) then UNPREDICTABLE;

```

CONSTRAINED UNPREDICTABLE behavior

If wback && n == t, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

Alias conditions

Alias	preferred when
PUSH (single register)	Rn == '1101' && P == '1' && U == '0' && W == '1' && imm8 == '00000100'

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
+	Specifies the offset is added to the base register.
<imm>	For the post-indexed or pre-indexed variant: is an 8-bit unsigned immediate byte offset, in the range 0 to 255, encoded in the "imm8" field. For encoding T1: is the optional positive unsigned immediate byte offset, a multiple of 4 in the range 0 to 124, defaulting to 0 and encoded in the "imm5" field as <imm>/4. For encoding T2: is the optional positive unsigned immediate byte offset, a multiple of 4, in the range 0 to 1020, defaulting to 0 and encoded in the "imm8" field as <imm>/4. For encoding T3: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T4: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
4      address = if index then offset_addr else R[n];
5
6      // Determine if the stack pointer limit should be checked
7      if n == 13 && wback then
8          violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
9      else
10         violatesLimit = FALSE;
11     // Memory operation only performed if limit not violated
12     if !violatesLimit then
13         MemU[address, 4] = R[t];
14
15     // If the stack pointer is being updated a fault will be raised if
16     // the limit is violated
17     if wback then RSPCheck[n] = offset_addr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.225 STR (register)

Store Register (register). Store Register (register) calculates an address from a base register value and an offset register value, stores a word from a register to memory. The offset register value can be shifted left by 0, 1, 2, or 3 bits.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	0	0	Rm			Rn			Rt		

T1 variant

STR{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
2 index = TRUE; add = TRUE; wback = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	0	0	0	0	1	0	0	Rn				Rt				0	0	0	0	0	0	imm2			Rm			

T2 variant

STR{<c>}.W <Rt>, [<Rn>, {+}<Rm>]
// <Rt>, <Rn>, <Rm> can be represented in T1
STR{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>{, LSL #<imm>}]

Decode for this encoding

```
1 if Rn == '1111' then UNDEFINED;
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
4 index = TRUE; add = TRUE; wback = FALSE;
5 (shift_t, shift_n) = (SRTYPE_LSL, UInt(imm2));
6 if t == 15 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields.
<q>	See C1.2.5 Standard assembler syntax fields.
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register is added to the base register.
<Rm>	Is the general-purpose index register, encoded in the "Rm" field.
<imm>	If present, the size of the left shift to apply to the value from <Rm>, in the range 1-3. <imm> is encoded in imm2. If absent, no shift is specified and imm2 is encoded as 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```
2      EncodingSpecificOperations();
3      offset = Shift(R[m], shift_t, shift_n, APSR.C);
4      address = R[n] + offset;
5      MemU[address, 4] = R[t];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.226 STRB (immediate)

Store Register Byte (immediate). Store Register Byte (immediate) calculates an address from a base register value and an immediate offset, and stores a byte from a register to memory. It can use offset, post-indexed, or pre-indexed addressing.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	0	imm5					Rn			Rt		

T1 variant

STRB{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm5, 32);
2 index = TRUE; add = TRUE; wback = FALSE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	1	0	0	0	Rn != 1111				Rt				imm12											

T2 variant

STRB{<c>}.W <Rt>, [<Rn> {, #<+><imm>}]
// <Rt>, <Rn>, <imm> can be represented in T1
STRB{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```
1 if Rn == '1111' then UNDEFINED;
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm12, 32);
4 index = TRUE; add = TRUE; wback = FALSE;
5 if t IN {13,15} then UNPREDICTABLE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	0	0	Rn != 1111				Rt				1	P	U	W	imm8							

Offset variant

Applies when P == 1 && U == 0 && W == 0.

STRB{<c>}{<q>} <Rt>, [<Rn> {, #-<imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

STRB{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

STRB{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>!

Decode for this encoding

```
1 if P == '1' && U == '1' && W == '0' then SEE STRBT;
2 if Rn == '1111' || (P == '0' && W == '0') then UNDEFINED;
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm8, 32);
5 index = (P == '1'); add = (U == '1'); wback = (W == '1');
6 if t IN {13,15} || (wback && n == t) then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If wback && n == t, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
+	Specifies the offset is added to the base register.
<imm>	For the post-indexed or pre-indexed variant: is an 8-bit unsigned immediate byte offset, in the range 0 to 255, encoded in the "imm8" field. For encoding T1: is an optional 5-bit unsigned immediate byte offset, in the range 0 to 31, defaulting to 0 and encoded in the "imm5" field. For encoding T2: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T3: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
4   address = if index then offset_addr else R[n];
5
6   // Determine if the stack pointer limit should be checked
7   if n == 13 && wback then
8     violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
9   else
10    violatesLimit = FALSE;
```

```
11 // Memory operation only performed if limit not violated
12 if !violatesLimit then
13     MemU[address, 1] = R[t][7:0];
14
15 // If the stack pointer is being updated a fault will be raised if
16 // the limit is violated
17 if wback then RSPCheck[n] = offset_addr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.227 STRB (register)

Store Register Byte (register). Store Register Byte (register) calculates an address from a base register value and an offset register value, and stores a byte from a register to memory. The offset register value can be shifted left by 0, 1, 2, or 3 bits.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	0	Rm			Rn			Rt		

T1 variant

STRB{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
2 index = TRUE; add = TRUE; wback = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	1	1	0	0	0	0	0	0	0	Rn != 1111				Rt				0	0	0	0	0	0	imm2				Rm			

T2 variant

STRB{<c>}.W <Rt>, [<Rn>, {+}<Rm>]
 // <Rt>, <Rn>, <Rm> can be represented in T1
 STRB{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>{, LSL #<imm>}]

Decode for this encoding

```
1 if Rn == '1111' then UNDEFINED;
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
4 index = TRUE; add = TRUE; wback = FALSE;
5 (shift_t, shift_n) = (SRTYPE_LSL, UInt(imm2));
6 if t IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register is added to the base register.
<Rm>	Is the general-purpose index register, encoded in the "Rm" field.
<imm>	If present, the size of the left shift to apply to the value from <Rm>, in the range 1-3. <imm> is encoded in imm2. If absent, no shift is specified and imm2 is encoded as 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```
2   EncodingSpecificOperations();  
3   offset = Shift(R[m], shift_t, shift_n, APSR.C);  
4   address = R[n] + offset;  
5   MemU[address, 1] = R[t][7:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.228 STRBT

Store Register Byte Unprivileged. Store Register Byte Unprivileged calculates an address from a base register value and an immediate offset, and stores a byte from a register to memory. When privileged software uses an [STRBT](#) instruction, the memory access is restricted as if the software was unprivileged.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	0	0	Rn != 1111		Rt		1	1	1	0												

T1 variant

STRBT{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```

1 if Rn == '1111' then UNDEFINED;
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); postindex = FALSE; add = TRUE;
4 register_form = FALSE; imm32 = ZeroExtend(imm8, 32);
5 if t IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the offset is added to the base register.
<imm>	Is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n] + imm32;
4     MemU_unpriv[address, 1] = R[t][7:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.229 STRD (immediate)

Store Register Dual (immediate). Store Register Dual (immediate) calculates an address from a base register value and an immediate offset, and stores two words from two registers to memory. It can use offset, post-indexed, or pre-indexed addressing.

This instruction is subject to stack limit checking.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	P	U	1	W	0	Rn				Rt				Rt2				imm8							

Offset variant

Applies when **P == 1 && W == 0**.

STRD{<c>}{<q>} <Rt>, <Rt2>, [<Rn> {, # {+/-}<imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

STRD{<c>}{<q>} <Rt>, <Rt2>, [<Rn>], # {+/-}<imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

STRD{<c>}{<q>} <Rt>, <Rt2>, [<Rn>, # {+/-}<imm>]!

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); t2 = UInt(Rt2); n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
4 index = (P == '1'); add = (U == '1'); wback = (W == '1');
5 if wback && (n == t || n == t2) then UNPREDICTABLE;
6 if n == 15 || t IN {13,15} || t2 IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If wback && (n == t || n == t2), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the first general-purpose register to be transferred, encoded in the "Rt" field.
<Rt2>	Is the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0

+ when $U = 1$
 $\langle \text{imm} \rangle$ For the offset variant: is the optional unsigned immediate byte offset, a multiple of 4, in the range 0 to 1020, defaulting to 0 and encoded in the "imm8" field as $\langle \text{imm} \rangle / 4$.
For the post-indexed and pre-indexed variant: is the unsigned immediate byte offset, a multiple of 4, in the range 0 to 1020, encoded in the "imm8" field as $\langle \text{imm} \rangle / 4$.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
4      address = if index then offset_addr else R[n];
5
6      // Determine if the stack pointer limit should be checked
7      if n == 13 && wback then
8          violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
9      else
10         violatesLimit = FALSE;
11     // Memory operation only performed if limit not violated
12     if !violatesLimit then
13         MemA[address, 4] = R[t];
14         MemA[address+4, 4] = R[t2];
15
16     // If the stack pointer is being updated a fault will be raised if
17     // the limit is violated
18     if wback then RSPCheck[n] = offset_addr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.230 STREX

Store Register Exclusive. Store Register Exclusive calculates an address from a base register value and an immediate offset, and stores a word from a register to memory if the executing PE has exclusive access to the memory addressed.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	0	1	0	0	Rn				Rt				Rd				imm8							

T1 variant

STREX{<c>}{<q>} <Rd>, <Rt>, [<Rn> {, #<imm>}]

Decode for this encoding

```

1 d = UInt(Rd); t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
2 if t == 15 then SEE "TT";
3 if d IN {13,15} || t == 13 || n == 15 then UNPREDICTABLE;
4 if d == n || d == t then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $d == t$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

CONSTRAINED UNPREDICTABLE behavior

If $d == n$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs the store to an UNKNOWN address.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the destination general-purpose register into which the status result of the store exclusive is written, encoded in the "Rd" field. The value returned is: 1 If the operation fails to update memory. 0 If the operation updates memory.
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
<imm>	The immediate offset added to the value of <Rn> to calculate the address. <imm> can be omitted, meaning an offset of 0. Values are multiples of 4 in the range 0-1020.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n] + imm32;
```



```
4      if ExclusiveMonitorsPass(address, 4) then
5          MemA[address, 4] = R[t];
6          R[d] = ZeroExtend('0');
7      else
8          R[d] = ZeroExtend('1');
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.231 STREXB

Store Register Exclusive Byte. Store Register Exclusive Byte derives an address from a base register value, and stores a byte from a register to memory if the executing PE has exclusive access to the memory addressed.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	0				Rn				Rt		(1)	(1)	(1)	(1)	0	1	0	0			Rd

T1 variant

STREXB{<c>}{<q>} <Rd>, <Rt>, [<Rn>]

Decode for this encoding

```

1 d = UInt(Rd); t = UInt(Rt); n = UInt(Rn);
2 if d IN {13,15} || t IN {13,15} || n == 15 then UNPREDICTABLE;
3 if d == n || d == t then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $d == t$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

CONSTRAINED UNPREDICTABLE behavior

If $d == n$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs the store to an UNKNOWN address.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the destination general-purpose register into which the status result of the store exclusive is written, encoded in the "Rd" field. The value returned is: 1 If the operation fails to update memory. 0 If the operation updates memory.
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n];
4   if ExclusiveMonitorsPass(address,1) then
5     MemA[address, 1] = R[t][7:0];
6     R[d] = ZeroExtend('0');
7   else
8     R[d] = ZeroExtend('1');
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.232 STREXH

Store Register Exclusive Halfword. Store Register Exclusive Halfword derives an address from a base register value, and stores a halfword from a register to memory if the executing PE has exclusive access to the memory addressed.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	0	Rn				Rt				(1) (1) (1) (1)				0	1	0	1	Rd			

T1 variant

STREXH{<c>}{<q>} <Rd>, <Rt>, [<Rn>]

Decode for this encoding

```

1 d = UInt(Rd); t = UInt(Rt); n = UInt(Rn);
2 if d IN {13,15} || t IN {13,15} || n == 15 then UNPREDICTABLE;
3 if d == n || d == t then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If $d == t$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

CONSTRAINED UNPREDICTABLE behavior

If $d == n$, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction performs the store to an UNKNOWN address.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the destination general-purpose register into which the status result of the store exclusive is written, encoded in the "Rd" field. The value returned is: 1 If the operation fails to update memory. 0 If the operation updates memory.
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   address = R[n];
4   if ExclusiveMonitorsPass(address, 2) then
5     MemA[address, 2] = R[t][15:0];
6     R[d] = ZeroExtend('0');
```

```
7     else  
8         R[d] = ZeroExtend('1');
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.233 STRH (immediate)

Store Register Halfword (immediate). Store Register Halfword (immediate) calculates an address from a base register value and an immediate offset, and stores a halfword from a register to memory. It can use offset, post-indexed, or pre-indexed addressing.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	imm5					Rn			Rt		

T1 variant

STRH{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm5:'0', 32);
2 index = TRUE; add = TRUE; wback = FALSE;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	1	0	1	0	Rn				Rt				imm12											

T2 variant

STRH{<c>}.W <Rt>, [<Rn> {, #<+><imm>}]
 // <Rt>, <Rn>, <imm> can be represented in T1
 STRH{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```
1 if Rn == '1111' then UNDEFINED;
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm12, 32);
4 index = TRUE; add = TRUE; wback = FALSE;
5 if t IN {13,15} then UNPREDICTABLE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	1	0	Rn				Rt				1	P	U	W	imm8							

Offset variant

Applies when P == 1 && U == 0 && W == 0.

STRH{<c>}{<q>} <Rt>, [<Rn> {, #-<imm>}]

Post-indexed variant

Applies when **P == 0 && W == 1**.

STRH{<c>}{<q>} <Rt>, [<Rn>], #{+/-}<imm>

Pre-indexed variant

Applies when **P == 1 && W == 1**.

STRH{<c>}{<q>} <Rt>, [<Rn>, #{+/-}<imm>]!

Decode for this encoding

```
1 if P == '1' && U == '1' && W == '0' then SEE STRHT;
2 if Rn == '1111' || (P == '0' && W == '0') then UNDEFINED;
3 if !HaveMainExt() then UNDEFINED;
4 t = UInt(Rt); n = UInt(Rn); imm32 = ZeroExtend(imm8, 32);
5 index = (P == '1'); add = (U == '1'); wback = (W == '1');
6 if t IN {13,15} || (wback && n == t) then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **wback && n == t**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The store instruction executes but the value stored is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
+	Specifies the offset is added to the base register.
<imm>	For the post-indexed or pre-indexed variant: is an 8-bit unsigned immediate byte offset, in the range 0 to 255, encoded in the "imm8" field. For encoding T1: is the optional positive unsigned immediate byte offset, a multiple of 2 in the range 0 to 62, defaulting to 0 and encoded in the "imm5" field as <imm>/2. For encoding T2: is an optional 12-bit unsigned immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field. For encoding T3: is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   offset_addr = if add then (R[n] + imm32) else (R[n] - imm32);
4   address = if index then offset_addr else R[n];
5
6   // Determine if the stack pointer limit should be checked
7   if n == 13 && wback then
8     violatesLimit = ViolatesSPLim(LookUpSP(), offset_addr);
9   else
10    violatesLimit = FALSE;
```

```

11      // Memory operation only performed if limit not violated
12      if !violatesLimit then
13          MemU[address, 2] = R[t][15:0];
14
15      // If the stack pointer is being updated a fault will be raised if
16      // the limit is violated
17      if wback then RSPCheck[n] = offset_addr;

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.234 STRH (register)

Store Register Halfword (register). Store Register Halfword (register) calculates an address from a base register value and an offset register value, and stores a halfword from a register to memory. The offset register value can be shifted left by 0, 1, 2, or 3 bits.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	0	1	Rm			Rn			Rt		

T1 variant

STRH{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>]

Decode for this encoding

```
1 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
2 index = TRUE; add = TRUE; wback = FALSE;
3 (shift_t, shift_n) = (SRTYPE_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	1	0	Rn			Rt			0	0	0	0	0	0	imm2			Rm				

T2 variant

STRH{<c>}.W <Rt>, [<Rn>, {+}<Rm>]
// <Rt>, <Rn>, <Rm> can be represented in T1
STRH{<c>}{<q>} <Rt>, [<Rn>, {+}<Rm>{, LSL #<imm>}]

Decode for this encoding

```
1 if Rn == '1111' then UNDEFINED;
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); m = UInt(Rm);
4 index = TRUE; add = TRUE; wback = FALSE;
5 (shift_t, shift_n) = (SRTYPE_LSL, UInt(imm2));
6 if t IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields.
<q>	See C1.2.5 Standard assembler syntax fields.
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the index register is added to the base register.
<Rm>	Is the general-purpose index register, encoded in the "Rm" field.
<imm>	If present, the size of the left shift to apply to the value from <Rm>, in the range 1-3. <imm> is encoded in imm2. If absent, no shift is specified and imm2 is encoded as 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```
2   EncodingSpecificOperations();  
3   offset = Shift(R[m], shift_t, shift_n, APSR.C);  
4   address = R[n] + offset;  
5   MemU[address, 2] = R[t][15:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.235 STRHT

Store Register Halfword Unprivileged. Store Register Halfword Unprivileged calculates an address from a base register value and an immediate offset, and stores a halfword from a register to memory.

When privileged software uses an [STRHT](#) instruction, the memory access is restricted as if the software was unprivileged.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	1	0	Rn != 1111				Rt				1	1	1	0	imm8							

T1 variant

STRHT{<c>}{<q>} <Rt>, [<Rn> {, #<+><imm>}]

Decode for this encoding

```

1 if Rn == '1111' then UNDEFINED;
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); postindex = FALSE; add = TRUE;
4 register_form = FALSE; imm32 = ZeroExtend(imm8, 32);
5 if t IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the offset is added to the base register.
<imm>	Is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n] + imm32;
4     MemU_unpriv[address, 2] = R[t][15:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.236 STRT

Store Register Unprivileged. Store Register Unprivileged calculates an address from a base register value and an immediate offset, and stores a word from a register to memory.

When privileged software uses an [STRT](#) instruction, the memory access is restricted as if the software was unprivileged.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	1	0	0	Rn				Rt				1	1	1	0	imm8							

T1 variant

STRT{<c>}{<q>} <Rt>, [<Rn> {, #{+}<imm>}]

Decode for this encoding

```

1 if Rn == '1111' then UNDEFINED;
2 if !HaveMainExt() then UNDEFINED;
3 t = UInt(Rt); n = UInt(Rn); postindex = FALSE; add = TRUE;
4 register_form = FALSE; imm32 = ZeroExtend(imm8, 32);
5 if t IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose register to be transferred, encoded in the "Rt" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+	Specifies the offset is added to the base register.
<imm>	Is an optional 8-bit unsigned immediate byte offset, in the range 0 to 255, defaulting to 0 and encoded in the "imm8" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     address = R[n] + imm32;
4     data = R[t];
5     MemU_unpriv[address, 4] = data;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.237 SUB (SP minus immediate)

Subtract from SP (immediate). Subtract (SP minus immediate) subtracts an immediate value from the SP value, and writes the result to the destination register.

This instruction is subject to stack limit checking.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	0	0	1	imm7						

T1 variant

SUB{<c>}{<q>} {SP,} SP, #<imm7>

Decode for this encoding

```
1 d = 13; setflags = FALSE; imm32 = ZeroExtend(imm7:'00', 32);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	0	1	1	0	1	S	1	1	0	1	0	imm3	Rd				imm8									

SUB variant

Applies when **S == 0**.

```
SUB{<c>}.W {<Rd>,} SP, #<const>
// <Rd>, <const> can be represented in T1
SUB{<c>}{<q>} {<Rd>,} SP, #<const>
```

SUBS variant

Applies when **S == 1 && Rd != 1111**.

SUBS{<c>}{<q>} {<Rd>,} SP, #<const>

Decode for this encoding

```
1 if Rd == '1111' && S == '1' then SEE "CMP (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); setflags = (S == '1'); imm32 = T32ExpandImm(i:imm3:imm8);
4 if d == 15 && S == '0' then UNPREDICTABLE;
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	1	0	1	0	1	0	1	1	0	1	0	imm3	Rd				imm8									

T3 variant

```
SUB{<c>}{<q>} {<Rd>}, SP, #<imm12>
    // <imm12> cannot be represented in T1, T2, or T3
SUBW{<c>}{<q>} {<Rd>}, SP, #<imm12>
    // <imm12> can be represented in T1, T2, or T3
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); setflags = FALSE; imm32 = ZeroExtend(i:imm3:imm8, 32);
3 if d == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<imm7>	Is an unsigned immediate, a multiple of 4 in the range 0 to 508, encoded in the "imm7" field as <imm7>/4.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the SP.
<imm12>	Is a 12-bit unsigned immediate, in the range 0 to 4095, encoded in the "i:imm3:imm8" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     (result, carry, overflow) = AddWithCarry(SP, NOT(imm32), '1');
4     RSPCheck[d] = result;
5     if setflags then
6         APSR.N = result[31];
7         APSR.Z = IsZeroBit(result);
8         APSR.C = carry;
9         APSR.V = overflow;
```

C2.4.238 SUB (SP minus register)

Subtract from SP (register). Subtract (SP minus register) subtracts an optionally-shifted register value from the SP value, and writes the result to the destination register.

This instruction is subject to stack limit checking.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	1	1	0	1	S	1	1	0	1	(0)	imm3														
																				Rd				imm2				Rm			

sr_type

SUB, rotate right with extend variant

Applies when **S == 0** && **imm3 == 000** && **imm2 == 00** && **sr_type == 11**.

SUB{<c>}{<q>} {<Rd>}, SP, <Rm>, RRX

SUB, shift or rotate by value variant

Applies when **S == 0** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

SUB{<c>}.W {<Rd>}, SP, <Rm>

// <Rd>, <Rm> can be represented in T1 or T2

SUB{<c>}{<q>} {<Rd>}, SP, <Rm> {, <shift> #<amount>}

SUBS, rotate right with extend variant

Applies when **S == 1** && **imm3 == 000** && **Rd != 1111** && **imm2 == 00** && **sr_type == 11**.

SUBS{<c>}{<q>} {<Rd>}, SP, <Rm>, RRX

SUBS, shift or rotate by value variant

Applies when **S == 1** && **!(imm3 == 000 && imm2 == 00 && sr_type == 11) && Rd != 1111**.

SUBS{<c>}{<q>} {<Rd>}, SP, <Rm> {, <shift> #<amount>}

Decode for this encoding

```

1 if Rd == '1111' && S == '1' then SEE "CMP (register)";
2 if !HaveMainExt() then UNDEFINED;
3 d = UInt(Rd); m = UInt(Rm); setflags = (S == '1');
4 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
5 if d == 13 && (shift_t != SRTYPE_LSL || shift_n > 3) then UNPREDICTABLE;
6 if (d == 15 && S == '0') || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See C1.2.5 Standard assembler syntax fields.

<q> See C1.2.5 Standard assembler syntax fields.

<Rd> Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the SP.

<Rm> Is the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values:

- LSL when sr_type = 00
- LSR when sr_type = 01
- ASR when sr_type = 10
- ROR when sr_type = 11

<amount> Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     shifted = Shift(R[m], shift_t, shift_n, APSR.C);
4     (result, carry, overflow) = AddWithCarry(SP, NOT(shifted), '1');
5     RSPCheck[d] = result;
6     if setflags then
7         APSR.N = result[31];
8         APSR.Z = IsZeroBit(result);
9         APSR.C = carry;
10        APSR.V = overflow;
```


C2.4.239 SUB (immediate)

Subtract (immediate). Subtract (immediate) subtracts an immediate value from a register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	1	imm3				Rn		Rd		

T1 variant

```
SUB<c>{<q>} <Rd>, <Rn>, #<imm3>
// Inside IT block
SUBS{<q>} <Rd>, <Rn>, #<imm3>
// Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rd); n = UInt(Rn); setflags = !InITBlock(); imm32 = ZeroExtend(imm3, 32);
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	1	Rdn			imm8							

T2 variant

```
SUB<c>{<q>} <Rdn>, #<imm8>
// Inside IT block, and <Rdn>, <imm8> can be represented in T1
SUB<c>{<q>} {<Rdn>, } <Rdn>, #<imm8>
// Inside IT block, and <Rdn>, <imm8> cannot be represented in T1
SUBS{<q>} <Rdn>, #<imm8>
// Outside IT block, and <Rdn>, <imm8> can be represented in T1
SUBS{<q>} {<Rdn>, } <Rdn>, #<imm8>
// Outside IT block, and <Rdn>, <imm8> cannot be represented in T1
```

Decode for this encoding

```
1 d = UInt(Rdn); n = UInt(Rdn); setflags = !InITBlock(); imm32 = ZeroExtend(imm8, 32);
```

T3

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	1	1	0	1	S	Rn				0	imm3				Rd				imm8							

SUB variant

Applies when S == 0.

```
SUB<c>.W {<Rd>, } <Rn>, #<const>
// Inside IT block, and <Rd>, <Rn>, <const> can be represented in T1 or T2
SUB{<c>}{<q>} {<Rd>, } <Rn>, #<const>
```

SUBS variant

Applies when **S == 1 && Rd != 1111**.

SUBS.W {<Rd>}, {<Rn>}, #<const>
// Outside IT block, and <Rd>, <Rn>, <const> can be represented in T1 or T2
SUBS{<c>}{<q>} {<Rd>}, {<Rn>}, #<const>

Decode for this encoding

```
1 if Rd == '1111' && S == '1' then SEE "CMP (immediate)";
2 if Rn == '1101' then SEE "SUB (SP minus immediate)";
3 if !HaveMainExt() then UNDEFINED;
4 d = UInt(Rd); n = UInt(Rn); setflags = (S == '1'); imm32 = T32ExpandImm(i:imm3:imm8);
5 if d == 13 || (d == 15 && S == '0') || n == 15 then UNPREDICTABLE;
```

T4

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	i	1	0	1	0	1	0	Rn				0	imm3			Rd				imm8							

T4 variant

SUB{<c>}{<q>} {<Rd>}, {<Rn>}, #<imm12>
// <imm12> cannot be represented in T1, T2, or T3
SUBW{<c>}{<q>} {<Rd>}, {<Rn>}, #<imm12>
// <imm12> can be represented in T1, T2, or T3

Decode for this encoding

```
1 if Rn == '1111' then SEE ADR;
2 if Rn == '1101' then SEE "SUB (SP minus immediate)";
3 if !HaveMainExt() then UNDEFINED;
4 d = UInt(Rd); n = UInt(Rn); setflags = FALSE; imm32 = ZeroExtend(i:imm3:imm8, 32);
5 if d IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rdn>	Is the general-purpose source and destination register, encoded in the "Rdn" field.
<imm8>	Is an 8-bit unsigned immediate, in the range 0 to 255, encoded in the "imm8" field.
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	For encoding T1: is the general-purpose source register, encoded in the "Rn" field. For encoding T3: is the general-purpose source register, encoded in the "Rn" field. If the SP is used, see C2.4.237 SUB (SP minus immediate) . For encoding T4: is the general-purpose source register, encoded in the "Rn" field. If the SP is used, see C2.4.237 SUB (SP minus immediate) . If the PC is used, see C2.4.8 ADR .
<imm3>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "imm3" field.
<imm12>	Is a 12-bit unsigned immediate, in the range 0 to 4095, encoded in the "i:imm3:imm8" field.
<const>	Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See C1.5 Modified immediate constants for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
```

```

3      (result, carry, overflow) = AddWithCarry(R[n], NOT(imm32), '1');
4      R[d] = result;
5      if setflags then
6          APSR.N = result[31];
7          APSR.Z = IsZeroBit(result);
8          APSR.C = carry;
9          APSR.V = overflow;

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.240 SUB (immediate, from PC)

Subtract from PC. Subtract from PC subtracts an immediate value from the `Align(PC, 4)` value to form a PC-relative address, and writes the result to the destination register. Arm recommends that, where possible, software avoids using this alias.

This instruction is an alias of the [ADR](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [ADR](#).
- The description of [ADR](#) gives the operational pseudocode for this instruction.

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	1	0	1	0	1	0	1	1	1	1	0	imm3				Rd				imm8							

T2 variant

`SUB{<c>}{<q>} <Rd>, PC, #<imm12>`

is equivalent to

`ADR{<c>}{<q>} <Rd>, <label>`

and is the preferred disassembly when `i:imm3:imm8 == '000000000000'`.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<label>	For encoding T1: the label of an instruction or literal data item whose address is to be loaded into <Rd>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the ADR instruction to this label. Permitted values of the size of the offset are multiples of 4 in the range 0 to 1020. For encoding T2 and T3: the label of an instruction or literal data item whose address is to be loaded into <Rd>. The assembler calculates the required value of the offset from the Align (PC, 4) value of the ADR instruction to this label. If the offset is zero or positive, encoding T3 is used, with <code>imm32</code> equal to the offset. If the offset is negative, encoding T2 is used, with <code>imm32</code> equal to the size of the offset. That is, the use of encoding T2 indicates that the required offset is minus the value of <code>imm32</code> . Permitted values of the size of the offset are 0-4095.
<imm12>	Is a 12-bit unsigned immediate, in the range 0 to 4095, encoded in the "i:imm3:imm8" field.

Operation for all encodings

The description of [ADR](#) gives the operational pseudocode for this instruction.

C2.4.241 SUB (register)

Subtract (register). Subtract (register) subtracts an optionally-shifted register value from a register value, and writes the result to the destination register. It can optionally update the condition flags based on the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	0	1		Rm		Rn					Rd

T1 variant

```
SUB<c>{<q>} <Rd>, <Rn>, <Rm>
    // Inside IT block
SUBS{<q>} {<Rd>}, <Rn>, <Rm>
    // Outside IT block
```

Decode for this encoding

```
1 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = !InITBlock();
2 (shift_t, shift_n) = (SRTType_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	1	1	0	1	S	Rn			(0)	imm3	Rd			imm2				Rm							
																												sr_type			

SUB, rotate right with extend variant

Applies when **S == 0 && imm3 == 000 && imm2 == 00 && sr_type == 11**.

```
SUB{<c>}{<q>} {<Rd>}, <Rn>, <Rm>, RRX
```

SUB, shift or rotate by value variant

Applies when **S == 0 && !(imm3 == 000 && imm2 == 00 && sr_type == 11)**.

```
SUB<c>.W {<Rd>}, <Rn>, <Rm>
    // Inside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
SUB{<c>}{<q>} {<Rd>}, <Rn>, <Rm> {, <shift> #<amount>}
```

SUBS, rotate right with extend variant

Applies when **S == 1 && imm3 == 000 && Rd != 1111 && imm2 == 00 && sr_type == 11**.

```
SUBS{<c>}{<q>} {<Rd>}, <Rn>, <Rm>, RRX
```

SUBS, shift or rotate by value variant

Applies when **S == 1 && !(imm3 == 000 && imm2 == 00 && sr_type == 11) && Rd != 1111**.

```
SUBS.W {<Rd>,} <Rn>, <Rm>
// Outside IT block, and <Rd>, <Rn>, <Rm> can be represented in T1
SUBS{<c>}{<q>} {<Rd>,} <Rn>, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if Rd == '1111' && S == '1' then SEE "CMP (register)";
2 if Rn == '1101' then SEE "SUB (SP minus register)";
3 if !HaveMainExt() then UNDEFINED;
4 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); setflags = (S == '1');
5 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
6 if d == 13 || (d == 15 && S == '0') || n == 15 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field. If omitted, this register is the same as <Rn>.
<Rn>	For encoding T1: is the first general-purpose source register, encoded in the "Rn" field. For encoding T2: is the first general-purpose source register, encoded in the "Rn" field. If the SP is used, see C2.4.238 SUB (SP minus register) .
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   shifted = Shift(R[m], shift_t, shift_n, APSR.C);
4   (result, carry, overflow) = AddWithCarry(R[n], NOT(shifted), '1');
5   R[d] = result;
6   if setflags then
7     APSR.N = result[31];
8     APSR.Z = IsZeroBit(result);
9     APSR.C = carry;
10    APSR.V = overflow;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.242 SVC

Supervisor Call. The Supervisor Call instruction generates a call to a system supervisor.

Use it as a call to an operating system to provide a service.

In older versions of the Arm architecture, SVC was called SWI, Software Interrupt.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	0	1	1	1	1	1	imm8								

T1 variant

SVC{<c>}{<q>} {#}<imm>

Decode for this encoding

```
1 imm32 = ZeroExtend(imm8, 32);
2 // imm32 is for assembly/disassembly. SVC handlers in some
3 // systems interpret imm8 in software, for example to determine the required service.
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <imm> Is an 8-bit unsigned immediate, in the range 0 to 255, encoded in the "imm8" field.

Operation for all encodings

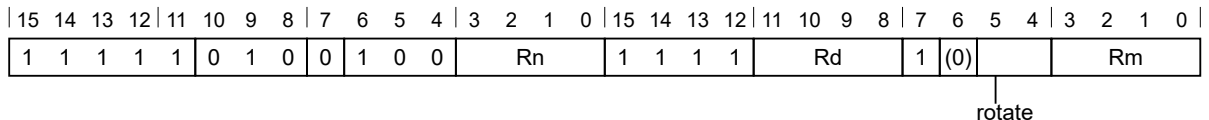
```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     CallSupervisor();
```

C2.4.243 SXTAB

Signed Extend and Add Byte. Signed Extend and Add Byte extracts an 8-bit value from a register, sign-extends it to 32 bits, adds the result to the value in another register, and writes the final result to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 8-bit value.

T1

Armv8-M DSP Extension only



T1 variant

SXTAB{<c>}{<q>} {<Rd>,<Rn>,<Rm> {,<ROR #<amount>}}

Decode for this encoding

```
1 if Rn == '1111' then SEE SXTB;
2 if !HaveDSPExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); rotation = UInt(rotate:'000');
4 if d IN {13,15} || n == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<amount>	Is the rotate amount, encoded in the "rotate" field. It can have the following values: 0 when rotate = 00 8 when rotate = 01 16 when rotate = 10 24 when rotate = 11 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     rotated = ROR(R[m], rotation);
4     R[d] = R[n] + SignExtend(rotated[7:0], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

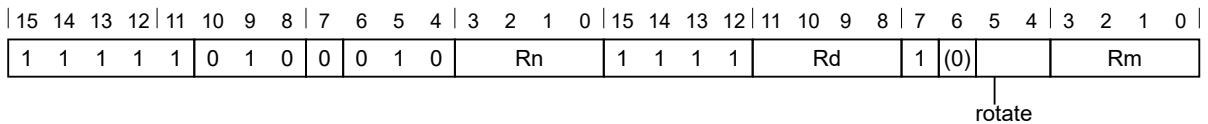
- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.244 SXTAB16

Signed Extend and Add Byte 16. Signed Extend and Add Byte 16 extracts two 8-bit values from a register, sign-extends them to 16 bits each, adds the results to two 16-bit values from another register, and writes the final results to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 8-bit values.

T1

Armv8-M DSP Extension only



T1 variant

SXTAB16{<c>}{<q>} {<Rd>,<Rn>,<Rm> {,<ROR #<amount>}}

Decode for this encoding

```
1 if Rn == '1111' then SEE SXTB16;
2 if !HaveDSPExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); rotation = UInt(rotate:'000');
4 if d IN {13,15} || n == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<amount>	Is the rotate amount, encoded in the "rotate" field. It can have the following values: 0 when rotate = 00 8 when rotate = 01 16 when rotate = 10 24 when rotate = 11 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   rotated = ROR(R[m], rotation);
4   bits(32) result;
5   result[15:0] = R[n][15:0] + SignExtend(rotated[7:0], 16);
6   result[31:16] = R[n][31:16] + SignExtend(rotated[23:16], 16);
7   R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

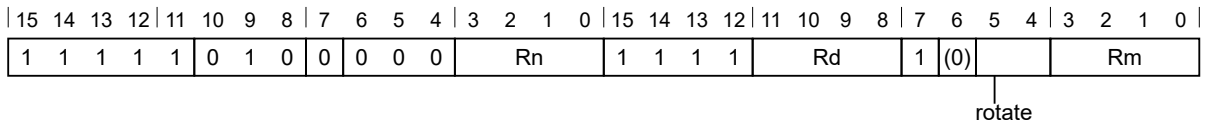
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.245 SXTAH

Signed Extend and Add Halfword. Signed Extend and Add Halfword extracts a 16-bit value from a register, sign-extends it to 32 bits, adds the result to a value from another register, and writes the final result to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 16-bit value.

T1

Armv8-M DSP Extension only



T1 variant

SXTAH{<c>}{<q>} {<Rd>,<Rn>,<Rm> {,<ROR #<amount>}}

Decode for this encoding

```
1 if Rn == '1111' then SEE SXTB;
2 if !HaveDSPEXT() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); rotation = UInt(rotate:'000');
4 if d IN {13,15} || n == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<amount>	Is the rotate amount, encoded in the "rotate" field. It can have the following values: 0 when rotate = 00 8 when rotate = 01 16 when rotate = 10 24 when rotate = 11 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   rotated = ROR(R[m], rotation);
4   R[d] = R[n] + SignExtend(rotated[15:0], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.246 SXTB

Signed Extend Byte. Signed Extend Byte extracts an 8-bit value from a register, sign extends it to 32 bits, and writes the result to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 8-bit value.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	1	0	0	1	Rm			Rd		

T1 variant

SXTB{<c>}{<q>} {<Rd>}, {<Rm>}

Decode for this encoding

```
1 d = UInt(Rd); m = UInt(Rm); rotation = 0;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0	1	0	0	1	1	1	1	1	1	1	1	1	Rd			1	(0)				Rm		

rotate

T2 variant

SXTB{<c>}.W {<Rd>}, {<Rm>
 // <Rd>, <Rm> can be represented in T1
 SXTB{<c>}{<q>} {<Rd>}, {<Rm> {, ROR #<amount>}}

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); m = UInt(Rm); rotation = UInt(rotate: '000');
3 if d IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> Is the general-purpose source register, encoded in the "Rm" field.
 <amount> Is the rotate amount, encoded in the "rotate" field. It can have the following values:
 0 when rotate = 00
 8 when rotate = 01
 16 when rotate = 10
 24 when rotate = 11
 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```
2   EncodingSpecificOperations();  
3   rotated = ROR(R[m], rotation);  
4   R[d] = SignExtend(rotated[7:0], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

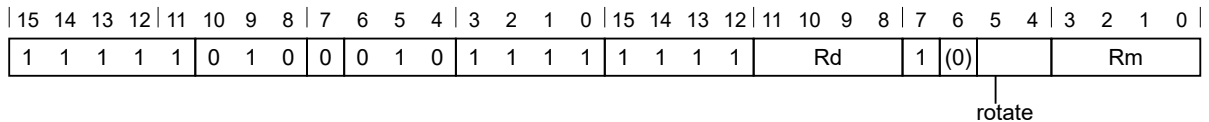
- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.247 SXTB16

Signed Extend Byte 16. Signed Extend Byte 16 extracts two 8-bit values from a register, sign-extends them to 16 bits each, and writes the results to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 8-bit values.

T1

Armv8-M DSP Extension only



T1 variant

SXTB16{<c>}{<q>} {<Rd>, } <Rm> {, ROR #<amount>}

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); m = UInt(Rm); rotation = UInt(rotate: '000');
3 if d IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> Is the general-purpose source register, encoded in the "Rm" field.
 <amount> Is the rotate amount, encoded in the "rotate" field. It can have the following values:

0	when rotate = 00
8	when rotate = 01
16	when rotate = 10
24	when rotate = 11

ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     rotated = ROR(R[m], rotation);
4     bits(32) result;
5     result[15:0] = SignExtend(rotated[7:0], 16);
6     result[31:16] = SignExtend(rotated[23:16], 16);
7     R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.248 SXTB

Signed Extend Halfword. Signed Extend Halfword extracts a 16-bit value from a register, sign extends it to 32 bits, and writes the result to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 16-bit value.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	1	0	0	0	Rm			Rd		

T1 variant

SXTB{<c>}{<q>} {<Rd>}, {<Rm>}

Decode for this encoding

```
1 d = UInt(Rd); m = UInt(Rm); rotation = 0;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
1	1	1	1	1	0	1	0	0	0	0	0	1	1	1	1	1	1	1	1	Rd			1	(0)			Rm							
																												rotate						

T2 variant

SXTB{<c>}.W {<Rd>}, {<Rm>
 // <Rd>, <Rm> can be represented in T1
 SXTB{<c>}{<q>} {<Rd>}, {<Rm> {, ROR #<amount>}}

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); m = UInt(Rm); rotation = UInt(rotate: '000');
3 if d IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See C1.2.5 Standard assembler syntax fields.
 <q> See C1.2.5 Standard assembler syntax fields.
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> Is the general-purpose source register, encoded in the "Rm" field.
 <amount> Is the rotate amount, encoded in the "rotate" field. It can have the following values:
 0 when rotate = 00
 8 when rotate = 01
 16 when rotate = 10
 24 when rotate = 11
 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```
2   EncodingSpecificOperations();  
3   rotated = ROR(R[m], rotation);  
4   R[d] = SignExtend(rotated[15:0], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.249 TBB, TBH

Table Branch Byte or Halfword. Table Branch Byte causes a PC-relative forward branch using a table of single byte offsets. A base register provides a pointer to the table, and a second register supplies an index into the table. The branch length is twice the value of the byte returned from the table.

Table Branch Halfword causes a PC-relative forward branch using a table of single halfword offsets. A base register provides a pointer to the table, and a second register supplies an index into the table. The branch length is twice the value of the halfword returned from the table.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	1	1	0	1	Rn				(1)	(1)	(1)	(1)	(0)	(0)	(0)	(0)	0	0	0	H	Rm			

Byte variant

Applies when **H == 0**.

```
TBB{<c>}{<q>} [<Rn>, <Rm>]
// Outside or last in IT block
```

Halfword variant

Applies when **H == 1**.

```
TBH{<c>}{<q>} [<Rn>, <Rm>, LSL #1]
// Outside or last in IT block
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); m = UInt(Rm); is_tbh = (H == '1');
3 if n == 13 || m IN {13,15} then UNPREDICTABLE;
4 if InITBlock() && !LastInITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register holding the address of the table of branch lengths, encoded in the "Rn" field. The PC can be used. If it is, the table immediately follows this instruction.
<Rm>	For the byte variant: is the general-purpose index register, encoded in the "Rm" field. This register contains an integer pointing to a single byte in the table. The offset in the table is the value of the index. For the halfword variant: is the general-purpose index register, encoded in the "Rm" field. This register contains an integer pointing to a halfword in the table. The offset in the table is twice the value of the index.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   if is_tbh then
4     halfwords = UInt(MemU[R[n]+LSL(R[m],1), 2]);
5   else
6     halfwords = UInt(MemU[R[n]+R[m], 1]);
7   BranchTo(PC + 2*halfwords);
```

C2.4.250 TEQ (immediate)

Test Equivalence (immediate). Test Equivalence (immediate) performs an exclusive OR operation on a register value and an immediate value. It updates the condition flags based on the result, and discards the result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	0	1	0	0	1	Rn				0	imm3				1	1	1	1	imm8							

T1 variant

TEQ{<c>}{<q>} <Rn>, #<const>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn);
3 (imm32, carry) = T32ExpandImm_C(i:imm3:imm8, APSR.C);
4 if n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rn> Is the general-purpose source register, encoded in the "Rn" field.
 <const> Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See [C1.5 Modified immediate constants](#) for the range of values.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   result = R[n] EOR imm32;
4   APSR.N = result[31];
5   APSR.Z = IsZeroBit(result);
6   APSR.C = carry;
7   // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

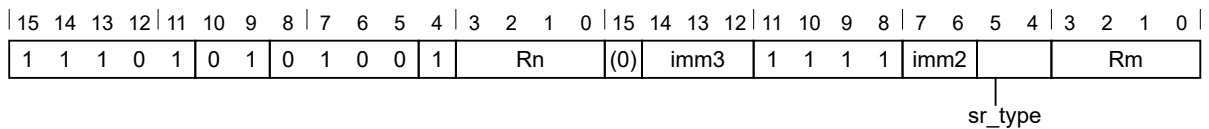
- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.251 TEQ (register)

Test Equivalence (register). Test Equivalence (register) performs an exclusive OR operation on a register value and an optionally-shifted register value. It updates the condition flags based on the result, and discards the result.

T1

Armv8-M Main Extension only



Rotate right with extend variant

Applies when `imm3 == 000 && imm2 == 00 && sr_type == 11`.

TEQ{<c>}{<q>} <Rn>, <Rm>, RRX

Shift or rotate by value variant

Applies when `!(imm3 == 000 && imm2 == 00 && sr_type == 11)`.

TEQ{<c>}{<q>} <Rn>, <Rm> {, <shift> #<amount>}

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); m = UInt(Rm);
3 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
4 if n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when sr_type = 00 LSR when sr_type = 01 ASR when sr_type = 10 ROR when sr_type = 11
<amount>	Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (shifted, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);
4   result = R[n] EOR shifted;
5   APSR.N = result[31];
6   APSR.Z = IsZeroBit(result);
7   APSR.C = carry;
8   // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.252 TST (immediate)

Test (immediate). Test (immediate) performs a bitwise AND operation on a register value and an immediate value. It updates the condition flags based on the result, and discards the result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	0	i	0	0	0	0	0	1	Rn				0	imm3				1	1	1	1	imm8							

T1 variant

TST{<c>}{<q>} <Rn>, #<const>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn);
3 (imm32, carry) = T32ExpandImm_C(i:imm3:imm8, APSR.C);
4 if n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rn> Is the general-purpose source register, encoded in the "Rn" field.
 <const> Is an immediate value derived from the 12-bit immediate that is encoded in the "i:imm3:imm8" field. See [C1.5 Modified immediate constants](#) for the range of values.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   result = R[n] AND imm32;
4   APSR.N = result[31];
5   APSR.Z = IsZeroBit(result);
6   APSR.C = carry;
7   // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.253 TST (register)

Test (register). Test (register) performs a bitwise AND operation on a register value and an optionally-shifted register value. It updates the condition flags based on the result, and discards the result.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	1	0	0	0	Rm					Rn

T1 variant

TST{<c>}{<q>} <Rn>, <Rm>

Decode for this encoding

```
1 n = UInt(Rn); m = UInt(Rm);
2 (shift_t, shift_n) = (SRTType_LSL, 0);
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	1	0	1	0	0	0	0	1	Rn				(0)	imm3				1	1	1	1	imm2				Rm			

sr_type

Rotate right with extend variant

Applies when `imm3 == 000 && imm2 == 00 && sr_type == 11`.

TST{<c>}{<q>} <Rn>, <Rm>, RRX

Shift or rotate by value variant

Applies when `!(imm3 == 000 && imm2 == 00 && sr_type == 11)`.

```
TST{<c>}.W <Rn>, <Rm>
// <Rn>, <Rm> can be represented in T1
TST{<c>}{<q>} <Rn>, <Rm> {, <shift> #<amount>}
```

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn); m = UInt(Rm);
3 (shift_t, shift_n) = DecodeImmShift(sr_type, imm3:imm2);
4 if n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the type of shift to be applied to the second source register, encoded in the "sr_type" field. It can have the following values: LSL when <code>sr_type = 00</code>

LSR when `sr_type = 01`
 ASR when `sr_type = 10`
 ROR when `sr_type = 11`
 <amount> Is the shift amount, in the range 1 to 31 (when <shift> = LSL or ROR) or 1 to 32 (when <shift> = LSR or ASR) encoded in the "imm3:imm2" field as <amount> modulo 32.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (shifted, carry) = Shift_C(R[m], shift_t, shift_n, APSR.C);
4   result = R[n] AND shifted;
5   APSR.N = result[31];
6   APSR.Z = IsZeroBit(result);
7   APSR.C = carry;
8   // APSR.V unchanged
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V flags.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of the registers being used in this instruction.
 - The values of the N, Z, C, V flags.

C2.4.254 TT, TTT, TTA, TTAT

Test Target (Alternate Domain, Unprivileged). Test Target (TT) queries the Security state and access permissions of a memory location.

Test Target Unprivileged (TTT) queries the Security state and access permissions of a memory location for an unprivileged access to that location.

Test Target Alternate Domain (TTA) and Test Target Alternate Domain Unprivileged (TTAT) query the Security state and access permissions of a memory location for a Non-secure access to that location. These instructions are only valid when executing in Secure state, and are UNDEFINED if used from Non-secure state.

These instructions return the Security state and access permissions in the destination register. See TT_RESP for the format of the destination register.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	0	0	1	0	0	Rn				1	1	1	1	Rd				A	T	(0)	(0)	(0)	(0)	(0)	(0)

TT variant

Applies when **A == 0** && **T == 0**.

TT{<c>}{<q>} <Rd>, <Rn>

TTA variant

Applies when **A == 1** && **T == 0**.

TTA{<c>}{<q>} <Rd>, <Rn>

TTAT variant

Applies when **A == 1** && **T == 1**.

TTAT{<c>}{<q>} <Rd>, <Rn>

TTT variant

Applies when **A == 0** && **T == 1**.

TTT{<c>}{<q>} <Rd>, <Rn>

Decode for this encoding

```
1 d = UInt(Rd); n = UInt(Rn); alt = (A == '1'); forceunpriv = (T == '1');
2 if d IN {13,15} || n == 15 then UNPREDICTABLE;
3 if alt && !IsSecure() then UNDEFINED;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the destination general-purpose register into which the status result of the target test is written, encoded in the "Rd" field.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.

Operation for all encodings


```
1  if ConditionPassed() then  
2      EncodingSpecificOperations();  
3      addr = R[n];  
4      R[d] = TTResp(addr, alt, forceunpriv);
```

C2.4.255 UADD16

Unsigned Add 16. Unsigned Add 16 performs two 16-bit unsigned integer additions, and writes the results to the destination register. It sets the APSR.GE bits according to the results of the additions.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rn				1	1	1	1	Rd				0	1	0	0	Rm			

T1 variant

UADD16{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum1 = UInt(R[n][15:0]) + UInt(R[m][15:0]);
4   sum2 = UInt(R[n][31:16]) + UInt(R[m][31:16]);
5   R[d] = sum2[15:0] : sum1[15:0];
6   APSR.GE[1:0] = if sum1 >= 0x10000 then '11' else '00';
7   APSR.GE[3:2] = if sum2 >= 0x10000 then '11' else '00';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.256 UADD8

Unsigned Add 8. Unsigned Add 8 performs four unsigned 8-bit integer additions, and writes the results to the destination register. It sets the APSR.GE bits according to the results of the additions.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				0	1	0	0	Rm			

T1 variant

UADD8 {<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     sum1 = UInt(R[n][7:0]) + UInt(R[m][7:0]);
4     sum2 = UInt(R[n][15:8]) + UInt(R[m][15:8]);
5     sum3 = UInt(R[n][23:16]) + UInt(R[m][23:16]);
6     sum4 = UInt(R[n][31:24]) + UInt(R[m][31:24]);
7     R[d] = sum4[7:0] : sum3[7:0] : sum2[7:0] : sum1[7:0];
8     APSR.GE[0] = if sum1 >= 0x100 then '1' else '0';
9     APSR.GE[1] = if sum2 >= 0x100 then '1' else '0';
10    APSR.GE[2] = if sum3 >= 0x100 then '1' else '0';
11    APSR.GE[3] = if sum4 >= 0x100 then '1' else '0';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.257 UASX

Unsigned Add and Subtract with Exchange. Unsigned Add and Subtract with Exchange exchanges the two halfwords of the second operand, performs one unsigned 16-bit integer addition and one unsigned 16-bit subtraction, and writes the results to the destination register. It sets the APSR.GE bits according to the results.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	1	0	Rn				1	1	1	1	Rd				0	1	0	0	Rm			

T1 variant

UASX{<c>} {<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   diff = UInt(R[n][15:0]) - UInt(R[m][31:16]);
4   sum = UInt(R[n][31:16]) + UInt(R[m][15:0]);
5   R[d] = sum[15:0] : diff[15:0];
6   APSR.GE[1:0] = if diff >= 0 then '11' else '00';
7   APSR.GE[3:2] = if sum >= 0x10000 then '11' else '00';
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.258 UBFX

Unsigned Bit Field Extract. Unsigned Bit Field Extract extracts any number of adjacent bits at any position from one register, zero extends them to 32 bits, and writes the result to the destination register.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	(0)	1	1	1	1	0	0	Rn				0	imm3				Rd				imm2	(0)	widthm1				

T1 variant

UBFX{<c>}{<q>} <Rd>, <Rn>, #<lsb>, #<width>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn);
3 lsbit = UInt(imm3:imm2); widthminus1 = UInt(widthm1);
4 msbit = lsbit + widthminus1;
5 if msbit > 31 then UNPREDICTABLE;
6 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `msbit > 31`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the general-purpose source register, encoded in the "Rn" field.
<lsb>	Is the bit number of the least significant bit in the field, in the range 0 to 31, encoded in the "imm3:imm2" field.
<width>	Is the width of the field, in the range 1 to 32-<lsb>, encoded in the "widthm1" field as <width>-1.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if msbit <= 31 then
4         R[d] = ZeroExtend(R[n][msbit:lsbit], 32);
5     else
6         R[d] = bits(32) UNKNOWN;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.259 UDF

Permanently Undefined. Permanently Undefined generates an Undefined Instruction exception.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	1	1	1	0	imm8							

T1 variant

UDF{<c>}{<q>} {#}<imm>

Decode for this encoding

```
1 imm32 = ZeroExtend(imm8, 32);
2 // imm32 is for assembly and disassembly only, and is ignored by hardware.
```

T2

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	1	1	1	1	1	1	1	imm4				1	0	1	0	imm12											

T2 variant

UDF{<c>}.W {#}<imm>
// <imm> can be represented in T1
UDF{<c>}{<q>} {#}<imm>

Decode for this encoding

```
1 imm32 = ZeroExtend(imm4:imm12, 32);
2 // imm32 is for assembly and disassembly only, and is ignored by hardware.
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#). Arm deprecates using any <c> value other than AL.

<q> See [C1.2.5 Standard assembler syntax fields](#).

<imm> For encoding T1: is an 8-bit unsigned immediate, in the range 0 to 255, encoded in the "imm8" field. The PE ignores the value of this constant.
For encoding T2: is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm4:imm12" field. The PE ignores the value of this constant.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   UNDEFINED;
```

C2.4.260 UDIV

Unsigned Divide. Unsigned Divide divides a 32-bit unsigned integer register value by a 32-bit unsigned integer register value, and writes the result to the destination register. The condition flags are not affected.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	0	1	1				Rn	(1)	(1)	(1)	(1)					Rd	1	1	1	1			Rm

T1 variant

UDIV{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
2 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register holding the dividend, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the divisor, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     if UInt(R[m]) == 0 then
4         if IntegerZeroDivideTrappingEnabled() then
5             GenerateIntegerZeroDivide();
6         else
7             result = 0;
8     else
9         result = RoundTowardsZero(Real(UInt(R[n])) / Real(UInt(R[m])));
10    R[d] = result[31:0];
```


C2.4.261 UHADD16

Unsigned Halving Add 16. Unsigned Halving Add 16 performs two unsigned 16-bit integer additions, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rn				1	1	1	1	Rd				0	1	1	0	Rm			

T1 variant

UHADD16{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum1 = UInt(R[n][15:0]) + UInt(R[m][15:0]);
4   sum2 = UInt(R[n][31:16]) + UInt(R[m][31:16]);
5   R[d] = sum2[16:1] : sum1[16:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.262 UHADD8

Unsigned Halving Add 8. Unsigned Halving Add 8 performs four unsigned 8-bit integer additions, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				0	1	1	0	Rm			

T1 variant

UHADD8{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum1 = UInt(R[n][7:0]) + UInt(R[m][7:0]);
4   sum2 = UInt(R[n][15:8]) + UInt(R[m][15:8]);
5   sum3 = UInt(R[n][23:16]) + UInt(R[m][23:16]);
6   sum4 = UInt(R[n][31:24]) + UInt(R[m][31:24]);
7   R[d] = sum4[8:1] : sum3[8:1] : sum2[8:1] : sum1[8:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.263 UHASX

Unsigned Halving Add and Subtract with Exchange. Unsigned Halving Add and Subtract with Exchange exchanges the two halfwords of the second operand, performs one unsigned 16-bit integer addition and one unsigned 16-bit subtraction, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	1	0	Rn				1	1	1	1	Rd				0	1	1	0	Rm			

T1 variant

UHASX{<c>} {<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   diff = UInt(R[n][15:0]) - UInt(R[m][31:16]);
4   sum = UInt(R[n][31:16]) + UInt(R[m][15:0]);
5   R[d] = sum[16:1] : diff[16:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.264 UHSAX

Unsigned Halving Subtract and Add with Exchange. Unsigned Halving Subtract and Add with Exchange exchanges the two halfwords of the second operand, performs one unsigned 16-bit integer subtraction and one unsigned 16-bit addition, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	1	0	Rn				1	1	1	1	Rd				0	1	1	0	Rm			

T1 variant

UHSAX{<c>} {<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum = UInt(R[n][15:0]) + UInt(R[m][31:16]);
4   diff = UInt(R[n][31:16]) - UInt(R[m][15:0]);
5   R[d] = diff[16:1] : sum[16:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.265 UHSUB16

Unsigned Halving Subtract 16. Unsigned Halving Subtract 16 performs two unsigned 16-bit integer subtractions, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	1	Rn				1	1	1	1	Rd				0	1	1	0	Rm			

T1 variant

UHSUB16{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   diff1 = UInt(R[n][15:0]) - UInt(R[m][15:0]);
4   diff2 = UInt(R[n][31:16]) - UInt(R[m][31:16]);
5   R[d] = diff2[16:1] : diff1[16:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.266 UHSUB8

Unsigned Halving Subtract 8. Unsigned Halving Subtract 8 performs four unsigned 8-bit integer subtractions, halves the results, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	0	Rn				1	1	1	1	Rd				0	1	1	0	Rm			

T1 variant

UHSUB8{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff1 = UInt(R[n][7:0]) - UInt(R[m][7:0]);
4     diff2 = UInt(R[n][15:8]) - UInt(R[m][15:8]);
5     diff3 = UInt(R[n][23:16]) - UInt(R[m][23:16]);
6     diff4 = UInt(R[n][31:24]) - UInt(R[m][31:24]);
7     R[d] = diff4[8:1] : diff3[8:1] : diff2[8:1] : diff1[8:1];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.267 UMAAL

Unsigned Multiply Accumulate Accumulate Long. Unsigned Multiply Accumulate Accumulate Long multiplies two unsigned 32-bit values to produce a 64-bit value, adds two unsigned 32-bit values, and writes the 64-bit result to two registers.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	1	1	0	Rn				RdLo				RdHi				0	1	1	0	Rm			

T1 variant

UMAAL{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 dLo = UInt(RdLo); dHi = UInt(RdHi); n = UInt(Rn); m = UInt(Rm);
3 if dLo IN {13,15} || dHi IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
4 if dHi == dLo then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `dHi == dLo`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<RdLo>	Is the general-purpose source register holding the first addend and the destination register for the lower 32 bits of the result, encoded in the "RdLo" field.
<RdHi>	Is the general-purpose source register holding the second addend and the destination register for the upper 32 bits of the result, encoded in the "RdHi" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = UInt(R[n]) * UInt(R[m]) + UInt(R[dHi]) + UInt(R[dLo]);
4     R[dHi] = result[63:32];
5     R[dLo] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.268 UMLAL

Unsigned Multiply Accumulate Long. Unsigned Multiply Accumulate Long multiplies two unsigned 32-bit values to produce a 64-bit value, and accumulates this with a 64-bit value.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	1	1	0	Rn				RdLo				RdHi				0 0 0 0				Rm			

T1 variant

UMLAL{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 dLo = UInt(RdLo); dHi = UInt(RdHi); n = UInt(Rn); m = UInt(Rm); setflags = FALSE;
3 if dLo IN {13,15} || dHi IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
4 if dHi == dLo then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `dHi == dLo`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<RdLo>	Is the general-purpose source register holding the lower 32 bits of the addend, and the destination register for the lower 32 bits of the result, encoded in the "RdLo" field.
<RdHi>	Is the general-purpose source register holding the upper 32 bits of the addend, and the destination register for the upper 32 bits of the result, encoded in the "RdHi" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = UInt(R[n]) * UInt(R[m]) + UInt(R[dHi]:R[dLo]);
4     R[dHi] = result[63:32];
5     R[dLo] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.269 UMULL

Unsigned Multiply Long. Unsigned Multiply Long multiplies two 32-bit unsigned values to produce a 64-bit result.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	1	0	1	0	Rn				RdLo				RdHi				0 0 0 0				Rm			

T1 variant

UMULL{<c>}{<q>} <RdLo>, <RdHi>, <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 dLo = UInt(RdLo); dHi = UInt(RdHi); n = UInt(Rn); m = UInt(Rm); setflags = FALSE;
3 if dLo IN {13,15} || dHi IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
4 if dHi == dLo then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `dHi == dLo`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<RdLo>	Is the general-purpose destination register for the lower 32 bits of the result, encoded in the "RdLo" field.
<RdHi>	Is the general-purpose destination register for the upper 32 bits of the result, encoded in the "RdHi" field.
<Rn>	Is the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     result = UInt(R[n]) * UInt(R[m]);
4     R[dHi] = result[63:32];
5     R[dLo] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.270 UQADD16

Unsigned Saturating Add 16. Unsigned Saturating Add 16 performs two unsigned 16-bit integer additions, saturates the results to the 16-bit unsigned integer range 0 to $2^{16}-1$, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	1	Rn				1	1	1	1	Rd				0	1	0	1	Rm			

T1 variant

UQADD16{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     sum1 = UInt(R[n][15:0]) + UInt(R[m][15:0]);
4     sum2 = UInt(R[n][31:16]) + UInt(R[m][31:16]);
5     bits(32) result;
6     result[15:0] = UnsignedSat(sum1, 16);
7     result[31:16] = UnsignedSat(sum2, 16);
8     R[d] = result;
```

C2.4.271 UQADD8

Unsigned Saturating Add 8. Unsigned Saturating Add 8 performs four unsigned 8-bit integer additions, saturates the results to the 8-bit unsigned integer range 0 to 2^8-1 , and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	0	0	Rn				1	1	1	1	Rd				0	1	0	1	Rm			

T1 variant

UQADD8{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     sum1 = UInt(R[n][7:0]) + UInt(R[m][7:0]);
4     sum2 = UInt(R[n][15:8]) + UInt(R[m][15:8]);
5     sum3 = UInt(R[n][23:16]) + UInt(R[m][23:16]);
6     sum4 = UInt(R[n][31:24]) + UInt(R[m][31:24]);
7     bits(32) result;
8     result[7:0] = UnsignedSat(sum1, 8);
9     result[15:8] = UnsignedSat(sum2, 8);
10    result[23:16] = UnsignedSat(sum3, 8);
11    result[31:24] = UnsignedSat(sum4, 8);
12    R[d] = result;

```

C2.4.272 UQASX

Unsigned Saturating Add and Subtract with Exchange. Unsigned Saturating Add and Subtract with Exchange exchanges the two halfwords of the second operand, performs one unsigned 16-bit integer addition and one unsigned 16-bit subtraction, saturates the results to the 16-bit unsigned integer range 0 to $2^{16}-1$, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	0	1	0	Rn				1	1	1	1	Rd				0	1	0	1	Rm			

T1 variant

UQASX{<c>} {<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   diff = UInt(R[n][15:0]) - UInt(R[m][31:16]);
4   sum = UInt(R[n][31:16]) + UInt(R[m][15:0]);
5   bits(32) result;
6   result[15:0] = UnsignedSat(diff, 16);
7   result[31:16] = UnsignedSat(sum, 16);
8   R[d] = result;
```

C2.4.273 UQRSHL (register)

Unsigned Saturating Rounding Shift Left. Unsigned saturating rounding shift left by 0 to 32 bits of a 32-bit value stored in a general-purpose register. The shift amount is read in as the bottom byte of Rm. If the shift amount is negative, the shift direction is reversed.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1																				

T1: UQRSHL variant

UQRSHL<c><q> Rda, Rm

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 if !HasArchVersion(Armv8pl) then SEE "Related encodings";
3 if !HaveMve() then UNDEFINED;
4 da = UInt(Rda);
5 m = UInt(Rm);
6 if Rda == '11x1' || Rm == '11x1' || Rm == Rda then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rda> General-purpose source and destination register, containing the value to be shifted.
 <Rm> General-purpose source register holding a shift amount in its bottom 8 bits.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   amount = SInt(R[m][7:0]);
5   opl = UInt(R[da]);
6   opl = opl + (1 << (- 1 - amount));
7   (result, sat) = UnsignedSatQ((opl << amount), 32);
8   if sat then APSR.Q = '1';
9   R[da] = result[31:0];
```


C2.4.274 UQRSHLL (register)

Unsigned Saturating Rounding Shift Left Long. Unsigned saturating rounding shift left by 0 to 64 bits of a 64-bit value stored in two general-purpose registers. The shift amount is read in as the bottom byte of Rm. If the shift amount is negative, the shift direction is reversed.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	1	Rm	RdaHi	(1)	sat	(0)	0	0	1	1	0	1							

T1: UQRSHLL variant

UQRSHLL<c><q> RdaLo, RdaHi, #<saturate>, Rm

Decode for this encoding

```

1 if RdaHi == '111' then SEE "UQRSHL (register)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8pl) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 m = UInt(Rm);
8 saturateTo = if sat == '0' then 64 else 48;
9 if RdaHi == '110' || Rm == '11x1' || Rm == RdaHi:'1' then CONSTRAINED_UNPREDICTABLE;
10 if Rm == RdaLo:'0' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
<saturate>	The bit position for saturation. This parameter must be one of the following values: #64 Encoded as sat = 0 #48 Encoded as sat = 1
<Rm>	General-purpose source register holding a shift amount in its bottom 8 bits.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   amount = SInt(R[m][7:0]);
5   opl = UInt(R[dah]:R[dal]);
6   opl = opl + (1 << (- 1 - amount));
7   (shiftedOp, didSat) = UnsignedSatQ((opl << amount), saturateTo);
8   result = ZeroExtend(shiftedOp, 64);
9   if didSat then APSR.Q = '1';
10  R[dah] = result[63:32];
11  R[dal] = result[31:0];

```

C2.4.275 UQSAX

Unsigned Saturating Subtract and Add with Exchange. Unsigned Saturating Subtract and Add with Exchange exchanges the two halfwords of the second operand, performs one unsigned 16-bit integer subtraction and one unsigned 16-bit addition, saturates the results to the 16-bit unsigned integer range 0 to $2^{16}-1$, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	1	0	Rn				1	1	1	1	Rd				0	1	0	1	Rm			

T1 variant

UQSAX{<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   sum = UInt(R[n][15:0]) + UInt(R[m][31:16]);
4   diff = UInt(R[n][31:16]) - UInt(R[m][15:0]);
5   bits(32) result;
6   result[15:0] = UnsignedSat(sum, 16);
7   result[31:16] = UnsignedSat(diff, 16);
8   R[d] = result;
```

C2.4.276 UQSHL (immediate)

Unsigned Saturating Shift Left. Unsigned saturating shift left by 1 to 32 bits of a 32-bit value stored in a general-purpose register.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1					0															

T1: UQSHL variant

UQSHL<c><q> Rda, #<imm>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 if !HasArchVersion(Armv8pl) then SEE "Related encodings";
3 if !HaveMve() then UNDEFINED;
4 da = UInt(Rda);
5 (-, amount) = DecodeImmShift('10', immh:imml);
6 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rda> General-purpose source and destination register, containing the value to be shifted.
 <imm> The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   op1 = UInt(R[da]);
5   (result, sat) = UnsignedSatQ((op1 << amount), 32);
6   if sat then APSR.Q = '1';
7   R[da] = result[31:0];

```

C2.4.277 UQSHLL (immediate)

Unsigned Saturating Shift Left Long. Unsigned saturating shift left by 1 to 32 bits of a 64-bit value stored in two general-purpose registers.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	1	0	immh	RdaHi	(1)	imml	0	0	1	1	1	1							

T1: UQSHLL variant

UQSHLL<c><q> RdaLo, RdaHi, #<imm>

Decode for this encoding

```

1 if RdaHi == '111' then SEE "UQSHL (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 (-, amount) = DecodeImmShift('10', immh:imml);
8 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
<imm>	The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     op1 = UInt(R[dah]:R[dal]);
5     (result, sat) = UnsignedSatQ((op1 << amount), 64);
6     if sat then APSR.Q = '1';
7     R[dah] = result[63:32];
8     R[dal] = result[31:0];
```

C2.4.278 UQSUB16

Unsigned Saturating Subtract 16. Unsigned Saturating Subtract 16 performs two unsigned 16-bit integer subtractions, saturates the results to the 16-bit unsigned integer range 0 to $2^{16}-1$, and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	1	Rn				1	1	1	1	Rd				0	1	0	1	Rm			

T1 variant

UQSUB16{<c>}{<q>} {<Rd>, } <Rn>, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff1 = UInt(R[n][15:0]) - UInt(R[m][15:0]);
4     diff2 = UInt(R[n][31:16]) - UInt(R[m][31:16]);
5     bits(32) result;
6     result[15:0] = UnsignedSat(diff1, 16);
7     result[31:16] = UnsignedSat(diff2, 16);
8     R[d] = result;
```

C2.4.279 UQSUB8

Unsigned Saturating Subtract 8. Unsigned Saturating Subtract 8 performs four unsigned 8-bit integer subtractions, saturates the results to the 8-bit unsigned integer range 0 to 2^8-1 , and writes the results to the destination register.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	0	Rn				1	1	1	1	Rd				0	1	0	1	Rm			

T1 variant

UQSUB8{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff1 = UInt(R[n][7:0]) - UInt(R[m][7:0]);
4     diff2 = UInt(R[n][15:8]) - UInt(R[m][15:8]);
5     diff3 = UInt(R[n][23:16]) - UInt(R[m][23:16]);
6     diff4 = UInt(R[n][31:24]) - UInt(R[m][31:24]);
7     bits(32) result;
8     result[7:0] = UnsignedSat(diff1, 8);
9     result[15:8] = UnsignedSat(diff2, 8);
10    result[23:16] = UnsignedSat(diff3, 8);
11    result[31:24] = UnsignedSat(diff4, 8);
12    R[d] = result;
```

C2.4.280 URSHR (immediate)

Unsigned Rounding Shift Right. Unsigned rounding shift right by 1 to 32 bits of a 32-bit value stored in a general-purpose register.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	1	0	1	0	0	1	0	1		Rda		0		immh		1	1	1	(1)		imml		0	1		1	1	1	1

T1: URSHR variant

URSHR<c><q> Rda, #<imm>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 if !HasArchVersion(Armv8pl) then SEE "Related encodings";
3 if !HaveMve() then UNDEFINED;
4 da = UInt(Rda);
5 (-, amount) = DecodeImmShift('10', immh:imml);
6 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rda> General-purpose source and destination register, containing the value to be shifted.
 <imm> The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   op1 = UInt(R[da]);
5   op1 = op1 + (1 << (amount - 1));
6   result = (op1 >> amount)[31:0];
7   R[da] = result[31:0];
```

C2.4.281 URSHRL (immediate)

Unsigned Rounding Shift Right Long. Unsigned rounding shift right by 1 to 32 bits of a 64-bit value stored in two general-purpose registers.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	0	0	1	0	1	RdaLo	1	0	immh	RdaHi	(1)	imml	0	1	1	1	1	1	1	1	1	1	1	1	

T1: URSHRL variant

URSHRL<c><q> RdaLo, RdaHi, #<imm>

Decode for this encoding

```

1 if RdaHi == '111' then SEE "URSHR (immediate)";
2 if !HaveMainExt() then UNDEFINED;
3 if !HasArchVersion(Armv8p1) then SEE "Related encodings";
4 if !HaveMve() then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 (-, amount) = DecodeImmShift('10', immh:immh);
8 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination, containing the value to be shifted. This must be an even numbered register.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination, containing the value to be shifted. This must be an odd numbered register.
<imm>	The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     opl = UInt(R[dah]:R[dal]);
5     opl = opl + (1 << (amount - 1));
6     result = (opl >> amount)[63:0];
7     R[dah] = result[63:32];
8     R[dal] = result[31:0];
```


C2.4.282 USAD8

Unsigned Sum of Absolute Differences. Unsigned Sum of Absolute Differences performs four unsigned 8-bit subtractions, and adds the absolute values of the differences together.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	1	1	1	Rn				1	1	1	1	Rd				0	0	0	0	Rm			

T1 variant

USAD8 {<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

Decode for this encoding

```

1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     absdiff1 = Abs(UInt(R[n][7:0]) - UInt(R[m][7:0]));
4     absdiff2 = Abs(UInt(R[n][15:8]) - UInt(R[m][15:8]));
5     absdiff3 = Abs(UInt(R[n][23:16]) - UInt(R[m][23:16]));
6     absdiff4 = Abs(UInt(R[n][31:24]) - UInt(R[m][31:24]));
7     result = absdiff1 + absdiff2 + absdiff3 + absdiff4;
8     R[d] = result[31:0];
```

C2.4.283 USADA8

Unsigned Sum of Absolute Differences and Accumulate. Unsigned Sum of Absolute Differences and Accumulate performs four unsigned 8-bit subtractions, and adds the absolute values of the differences to a 32-bit accumulate operand.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1	0	1	1	1	Rn				Ra				Rd				0 0 0 0				Rm			

T1 variant

USADA8{<c>}{<q>} <Rd>, <Rn>, <Rm>, <Ra>

Decode for this encoding

```
1 if Ra == '1111' then SEE USAD8;
2 if !HaveDSPExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); a = UInt(Ra);
4 if d IN {13,15} || n IN {13,15} || m IN {13,15} || a == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<Ra>	Is the third general-purpose source register holding the addend, encoded in the "Ra" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   absdiff1 = Abs(UInt(R[n][7:0]) - UInt(R[m][7:0]));
4   absdiff2 = Abs(UInt(R[n][15:8]) - UInt(R[m][15:8]));
5   absdiff3 = Abs(UInt(R[n][23:16]) - UInt(R[m][23:16]));
6   absdiff4 = Abs(UInt(R[n][31:24]) - UInt(R[m][31:24]));
7   result = UInt(R[a]) + absdiff1 + absdiff2 + absdiff3 + absdiff4;
8   R[d] = result[31:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.284 USAT

Unsigned Saturate. Unsigned Saturate saturates an optionally-shifted signed value to a selected unsigned range. The Q flag is set to 1 if the operation saturates.

T1

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	(0)	1	1	1	0	sh	0	Rn	0	imm3	Rd	imm2	(0)	sat_imm													

Arithmetic shift right variant

Applies when **sh** == 1 && !(imm3 == 000 && imm2 == 00).

USAT{<c>}{<q>} <Rd>, #<imm>, <Rn>, ASR #<amount>

Logical shift left variant

Applies when **sh** == 0.

USAT{<c>}{<q>} <Rd>, #<imm>, <Rn> {, LSL #<amount>}

Decode for this encoding

```

1 if sh == '1' && (imm3:imm2) == '0000' then
2   if HaveDSPExt() then
3     SEE USAT16;
4   else
5     UNDEFINED;
6 if !HaveMainExt() then UNDEFINED;
7 d = UInt(Rd); n = UInt(Rn); saturate_to = UInt(sat_imm);
8 (shift_t, shift_n) = DecodeImmShift(sh:'0', imm3:imm2);
9 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <imm> Is the bit position for saturation, in the range 0 to 31, encoded in the "sat_imm" field.
 <Rn> Is the general-purpose source register, encoded in the "Rn" field.
 <amount> For the arithmetic shift right variant: is the shift amount, in the range 1 to 31 encoded in the "imm3:imm2" field as <amount>.
 For the logical shift left variant: is the optional shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm3:imm2" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   operand = Shift(R[n], shift_t, shift_n, APSR.C); // APSR.C ignored
4   (result, sat) = UnsignedSatQ(SInt(operand), saturate_to);
5   R[d] = ZeroExtend(result, 32);
6   if sat then
7     APSR.Q = '1';
```

C2.4.285 USAT16

Unsigned Saturate 16. Unsigned Saturate 16 saturates two signed 16-bit values to a selected unsigned range.

The Q flag is set to 1 if the operation saturates.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	(0)	1	1	1	0	1	0	Rn				0	0	0	0	Rd				0	0	(0)	(0)	sat_imm			

T1 variant

USAT16{<c>}{<q>} <Rd>, #<imm>, <Rn>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); saturate_to = UInt(sat_imm);
3 if d IN {13,15} || n IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <imm> Is the bit position for saturation, in the range 0 to 15, encoded in the "sat_imm" field.
 <Rn> Is the general-purpose source register, encoded in the "Rn" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   (result1, sat1) = UnsignedSatQ(SInt(R[n][15:0]), saturate_to);
4   (result2, sat2) = UnsignedSatQ(SInt(R[n][31:16]), saturate_to);
5   bits(32) result;
6   result[15:0] = ZeroExtend(result1, 16);
7   result[31:16] = ZeroExtend(result2, 16);
8   R[d] = result;
9   if sat1 || sat2 then
10     APSR.Q = '1';
```

C2.4.286 USAX

Unsigned Subtract and Add with Exchange. Unsigned Subtract and Add with Exchange exchanges the two halfwords of the second operand, performs one unsigned 16-bit integer subtraction and one unsigned 16-bit addition, and writes the results to the destination register. It sets the APSR.GE bits according to the results.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	1	0	Rn				1	1	1	1	Rd				0	1	0	0	Rm			

T1 variant

USAX{<c>} {<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     sum = UInt(R[n][15:0]) + UInt(R[m][31:16]);
4     diff = UInt(R[n][31:16]) - UInt(R[m][15:0]);
5     R[d] = diff[15:0] : sum[15:0];
6     APSR.GE[1:0] = if sum >= 0x10000 then '11' else '00';
7     APSR.GE[3:2] = if diff >= 0 then '11' else '00';
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.287 USUB16

Unsigned Subtract 16. Unsigned Subtract 16 performs two 16-bit unsigned integer subtractions, and writes the results to the destination register. It sets the APSR.GE bits according to the results of the subtractions.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	1	Rn				1	1	1	1	Rd				0	1	0	0	Rm			

T1 variant

USUB16{<c>}{<q>} {<Rd>}, {<Rn>}, <Rm>

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   diff1 = UInt(R[n][15:0]) - UInt(R[m][15:0]);
4   diff2 = UInt(R[n][31:16]) - UInt(R[m][31:16]);
5   R[d] = diff2[15:0] : diff1[15:0];
6   APSR.GE[1:0] = if diff1 >= 0 then '11' else '00';
7   APSR.GE[3:2] = if diff2 >= 0 then '11' else '00';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.288 USUB8

Unsigned Subtract 8. Unsigned Subtract 8 performs four 8-bit unsigned integer subtractions, and writes the results to the destination register. It sets the APSR.GE bits according to the results of the subtractions.

T1

Armv8-M DSP Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	1	1	0	0	Rn				1	1	1	1	Rd				0	1	0	0	Rm			

T1 variant

USUB8 {<c>} {<q>} {<Rd>}, {<Rn>}, {<Rm>}

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm);
3 if d IN {13,15} || n IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See C1.2.5 *Standard assembler syntax fields*.
 <q> See C1.2.5 *Standard assembler syntax fields*.
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rn> Is the first general-purpose source register, encoded in the "Rn" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     diff1 = UInt(R[n][7:0]) - UInt(R[m][7:0]);
4     diff2 = UInt(R[n][15:8]) - UInt(R[m][15:8]);
5     diff3 = UInt(R[n][23:16]) - UInt(R[m][23:16]);
6     diff4 = UInt(R[n][31:24]) - UInt(R[m][31:24]);
7     R[d] = diff4[7:0] : diff3[7:0] : diff2[7:0] : diff1[7:0];
8     APSR.GE[0] = if diff1 >= 0 then '1' else '0';
9     APSR.GE[1] = if diff2 >= 0 then '1' else '0';
10    APSR.GE[2] = if diff3 >= 0 then '1' else '0';
11    APSR.GE[3] = if diff4 >= 0 then '1' else '0';
```

Restricted behavior

Data Independent Timing behavior

If AIRCR.DIT is 1:

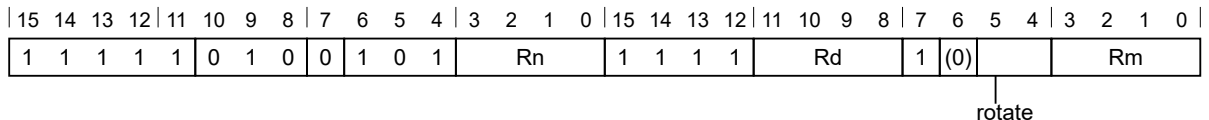
- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.289 UXTAB

Unsigned Extend and Add Byte. Unsigned Extend and Add Byte extracts an 8-bit value from a register, zero-extends it to 32 bits, adds the result to the value in another register, and writes the final result to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 8-bit value.

T1

Armv8-M DSP Extension only



T1 variant

UXTAB{<c>}{<q>} {<Rd>,<Rn>,<Rm> {,<ROR #<amount>}}

Decode for this encoding

```
1 if Rn == '1111' then SEE UXTB;
2 if !HaveDSPExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); rotation = UInt(rotate:'000');
4 if d IN {13,15} || n == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<amount>	Is the rotate amount, encoded in the "rotate" field. It can have the following values: 0 when rotate = 00 8 when rotate = 01 16 when rotate = 10 24 when rotate = 11 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   rotated = ROR(R[m], rotation);
4   R[d] = R[n] + ZeroExtend(rotated[7:0], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

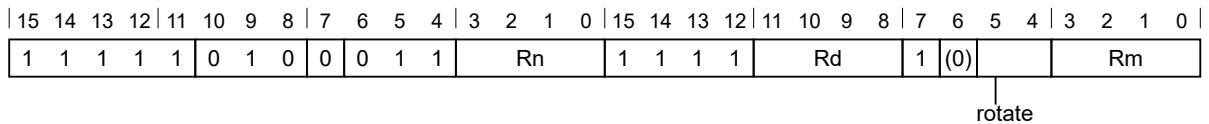
- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.290 UXTAB16

Unsigned Extend and Add Byte 16. Unsigned Extend and Add Byte 16 extracts two 8-bit values from a register, zero-extends them to 16 bits each, adds the results to two 16-bit values from another register, and writes the final results to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 8-bit values.

T1

Armv8-M DSP Extension only



T1 variant

UXTAB16{<c>}{<q>} {<Rd>}, <Rn>, <Rm> {, ROR #<amount>}

Decode for this encoding

```
1 if Rn == '1111' then SEE UXTB16;
2 if !HaveDSPExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); rotation = UInt(rotate:'000');
4 if d IN {13,15} || n == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<amount>	Is the rotate amount, encoded in the "rotate" field. It can have the following values: 0 when rotate = 00 8 when rotate = 01 16 when rotate = 10 24 when rotate = 11 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   rotated = ROR(R[m], rotation);
4   bits(32) result;
5   result[15:0] = R[n][15:0] + ZeroExtend(rotated[7:0], 16);
6   result[31:16] = R[n][31:16] + ZeroExtend(rotated[23:16], 16);
7   R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

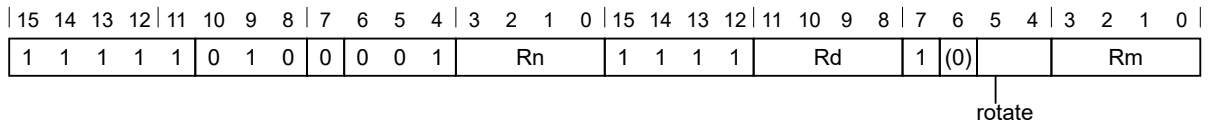
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.291 UXTAH

Unsigned Extend and Add Halfword. Unsigned Extend and Add Halfword extracts a 16-bit value from a register, zero-extends it to 32 bits, adds the result to a value from another register, and writes the final result to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 16-bit value.

T1

Armv8-M DSP Extension only



T1 variant

UXTAH{<c>}{<q>} {<Rd>, } <Rn>, <Rm> {, ROR #<amount>}

Decode for this encoding

```
1 if Rn == '1111' then SEE UXTH;
2 if !HaveDSPExt() then UNDEFINED;
3 d = UInt(Rd); n = UInt(Rn); m = UInt(Rm); rotation = UInt(rotate:'000');
4 if d IN {13,15} || n == 13 || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rn>	Is the first general-purpose source register, encoded in the "Rn" field.
<Rm>	Is the second general-purpose source register, encoded in the "Rm" field.
<amount>	Is the rotate amount, encoded in the "rotate" field. It can have the following values: 0 when rotate = 00 8 when rotate = 01 16 when rotate = 10 24 when rotate = 11 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     rotated = ROR(R[m], rotation);
4     R[d] = R[n] + ZeroExtend(rotated[15:0], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.292 UXTB

Unsigned Extend Byte. Unsigned Extend Byte extracts an 8-bit value from a register, zero extends it to 32 bits, and writes the result to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 8-bit value.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	1	0	1	1	Rm			Rd		

T1 variant

UXTB{<c>}{<q>} {<Rd>}, {<Rm>}

Decode for this encoding

```
1 d = UInt(Rd); m = UInt(Rm); rotation = 0;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0	1	0	1	1	1	1	1	Rd			1	(0)										Rm	
																								rotate							

T2 variant

UXTB{<c>}.W {<Rd>}, {<Rm>
// <Rd>, <Rm> can be represented in T1
UXTB{<c>}{<q>} {<Rd>}, {<Rm> {, ROR #<amount>}}

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); m = UInt(Rm); rotation = UInt(rotate: '000');
3 if d IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> Is the general-purpose source register, encoded in the "Rm" field.
 <amount> Is the rotate amount, encoded in the "rotate" field. It can have the following values:
 0 when rotate = 00
 8 when rotate = 01
 16 when rotate = 10
 24 when rotate = 11
 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```
2   EncodingSpecificOperations();  
3   rotated = ROR(R[m], rotation);  
4   R[d] = ZeroExtend(rotated[7:0], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

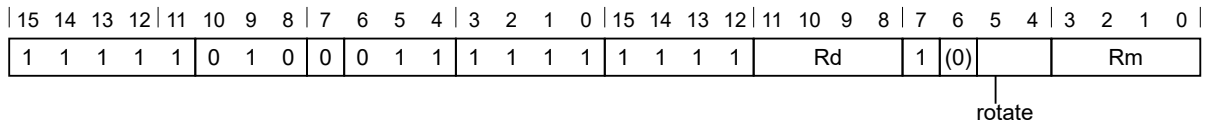
- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.293 UXTB16

Unsigned Extend Byte 16. Unsigned Extend Byte 16 extracts two 8-bit values from a register, zero-extends them to 16 bits each, and writes the results to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 8-bit values.

T1

Armv8-M DSP Extension only



T1 variant

UXTB16{<c>}{<q>} {<Rd>, } <Rm> {, ROR #<amount>}

Decode for this encoding

```
1 if !HaveDSPExt() then UNDEFINED;
2 d = UInt(Rd); m = UInt(Rm); rotation = UInt(rotate: '000');
3 if d IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rd> Is the general-purpose destination register, encoded in the "Rd" field.
 <Rm> Is the second general-purpose source register, encoded in the "Rm" field.
 <amount> Is the rotate amount, encoded in the "rotate" field. It can have the following values:
 0 when rotate = 00
 8 when rotate = 01
 16 when rotate = 10
 24 when rotate = 11
 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     rotated = ROR(R[m], rotation);
4     bits(32) result;
5     result[15:0] = ZeroExtend(rotated[7:0], 16);
6     result[31:16] = ZeroExtend(rotated[23:16], 16);
7     R[d] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.294 UXTH

Unsigned Extend Halfword. Unsigned Extend Halfword extracts a 16-bit value from a register, zero extends it to 32 bits, and writes the result to the destination register. The instruction can specify a rotation by 0, 8, 16, or 24 bits before extracting the 16-bit value.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	0	0	1	0	1	0	Rm			Rd		

T1 variant

UXTH{<c>}{<q>} {<Rd>}, {<Rm>}

Decode for this encoding

```
1 d = UInt(Rd); m = UInt(Rm); rotation = 0;
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	0	0	0	0	1	1	1	1	1	1	1	1	1	Rd			1	(0)							Rm

rotate

T2 variant

UXTH{<c>}.W {<Rd>}, {<Rm>}

// <Rd>, <Rm> can be represented in T1

UXTH{<c>}{<q>} {<Rd>}, {<Rm>}, {, ROR #<amount>}

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 d = UInt(Rd); m = UInt(Rm); rotation = UInt(rotate: '000');
3 if d IN {13,15} || m IN {13,15} then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rd>	Is the general-purpose destination register, encoded in the "Rd" field.
<Rm>	Is the general-purpose source register, encoded in the "Rm" field.
<amount>	Is the rotate amount, encoded in the "rotate" field. It can have the following values: 0 when rotate = 00 8 when rotate = 01 16 when rotate = 10 24 when rotate = 11 ROR #<amount> can be omitted, meaning a rotate amount of 0. This is the preferred disassembly for rotate == 0b00.

Operation for all encodings

```
1 if ConditionPassed() then
```

```
2   EncodingSpecificOperations();  
3   rotated = ROR(R[m], rotation);  
4   R[d] = ZeroExtend(rotated[15:0], 32);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.295 VABAV

Vector Absolute Difference and Accumulate Across Vector. Subtract the elements of the second source vector register from the corresponding elements of the first source vector and accumulate the absolute values of the results. The initial value of the general-purpose destination register is added to the result.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	0	size		Qn		0		Rda		1	1	1	1	N	0	M	0		Qm		1		

T1: VABAV variant

VABAV<v><q>.<dt> Rda, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if M == '1' || N == '1' then UNDEFINED;
4 da      = UInt(Rda);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 esize   = 8 << UInt(size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> This parameter determines the following values:
 – Unsigned flag: S indicates signed, U indicates unsigned.
 – Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

<Rda> General-purpose source and destination register.
 <Qn> First source vector register.
 <Qm> Second source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
```

```
5
6 op1    = Q[n, curBeat];
7 op2    = Q[m, curBeat];
8 result = UInt(R[da]);
9 for e = 0 to elements-1
10     if elmtMask[e*(esize>>3)] == '1' then
11         result = result + Abs(Int(Elem[op1, e, esize], unsigned) -
12                               Int(Elem[op2, e, esize], unsigned));
13
14 R[da] = result[31:0];
```

C2.4.296 VABD

Vector Absolute Difference. Subtract the elements of the second source vector register from the corresponding elements of the first source vector register and place the absolute values of the results in the elements of the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn	0	Qd	0	0	1	1	1	N	1	M	0		Qm	0					

T1: VABD variant

VABD<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(N:Qn);
7 esize = 8 << UInt(size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> This parameter determines the following values:
 – Unsigned flag: S indicates signed, U indicates unsigned.
 – Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

<Qd> Destination vector register.
 <Qn> First source vector register.
 <Qm> Second source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
```

```

6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 op2    = Q[m, curBeat];
9 for e = 0 to elements-1
10     value = Abs(Int(Elem[op1, e, esize], unsigned) - Int(Elem[op2, e, esize], unsigned));
11     Elem[result, e, esize] = value[esize-1:0];
12
13 for e = 0 to 3
14     if elmtMask[e] == '1' then
15         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.297 VABD (floating-point)

Vector Absolute Difference. Subtract the elements of the second source vector from the corresponding elements of the first source vector and place the absolute values of the results in the elements of the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	0	D	1	sz	Qn	0	Qd	0	1	1	0	1	N	1	M	0	Qm	0						

T1: VABD variant

VABD<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_MveFp);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt(D:Qd);
4 m      = UInt(M:Qm);
5 n      = UInt(N:Qn);
6 esize  = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> Size: indicates the floating-point format used.
 This parameter must be one of the following values:
 F32 Encoded as sz = 0
 F16 Encoded as sz = 1
 <Qd> Destination vector register.
 <Qn> First source vector register.
 <Qm> Second source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 op2    = Q[m, curBeat];
9 for e = 0 to elements-1
10     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
11     pred = (elmtMask[e*(esize>>3)] == '0');
12     value = FPAbs(FPSub(Elem[op1, e, esize], Elem[op2, e, esize], FALSE, pred));
13     Elem[result, e, esize] = value;
14
15 for e = 0 to 3

```

```
16     if elmtMask[e] == '1' then  
17         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.298 VABS

Floating-point Absolute. Floating-point Absolute takes the absolute value of a half-precision or single-precision or double-precision register, and places the result in the destination register.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	0	0	0	Vd	1	0	size	1	1	M	0	Vm							

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VABS{<c>}{<q>}.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VABS{<c>}{<q>}.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VABS{<c>}{<q>}.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
6 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   case size of
5     when '01' S[d] = Zeros(16) : FPAbs(S[m][15:0]);
6     when '10' S[d] = FPAbs(S[m]);
7     when '11' D[d] = FPAbs(D[m]);

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.299 VABS (floating-point)

Vector Absolute. Compute the absolute value of each element of a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	1		Qd	0	0	1	1	1	0	1	M	0		Qm	0		0	

T1: VABS variant

VABS<v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size IN {'11', '00'} then UNDEFINED;
4 d      = UInt (D:Qd);
5 m      = UInt (M:Qm);
6 esize  = 8 << UInt (size);
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the floating-point format used.
 This parameter must be one of the following values:
 F16 Encoded as size = 01
 F32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[m, curBeat];
8 for e = 0 to elements-1
9     value = FPAbs (Elem[op1, e, esize]);
10    Elem[result, e, esize] = value;
11
12 for e = 0 to 3
13     if elmtMask[e] == '1' then
14         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.300 VABS (vector)

Vector Absolute. Compute the absolute value of each element of a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	D	1	1	size	0	1	Qd	0	0	0	1	1	0	1	M	0	Qm	0						

T1: VABS variant

VABS<v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 esize = 8 << UInt(size);
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 S8 Encoded as size = 00
 S16 Encoded as size = 01
 S32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[m, curBeat];
8 for e = 0 to elements-1
9     value = Abs(SInt(Elm[op1, e, esize]));
10    Elm[result, e, esize] = value[esize-1:0];
11
12 for e = 0 to 3
13     if elmtMask[e] == '1' then
14         Elm[Q[d, curBeat], e, 8] = Elm[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.301 VADC

Whole Vector Add With Carry. Add with carry across beats, with carry in from and out to FPSCR.C. Initial value of FPSCR.C can be overridden by using the I variant. FPSCR.C is not updated for beats disabled because of predication. FPSCR.N, .V and .Z are zeroed whenever FPSCR.C is updated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	1	1	Qn	0	Qd	I	1	1	1	1	N	0	M	0	Qm	0						

T1: VADC variant

VADC{I}<v><q>.I32 Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt(D:Qd);
4 m      = UInt(M:Qm);
5 n      = UInt(N:Qn);
6 carryInit = (I == '1');
7 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<I>	Specifies where the initial carry in for wide arithmetic comes from. This parameter must be one of the following values: – Encoded as I = 0 Indicates carry input comes from FPSCR.C. I Encoded as I = 1 Indicates carry input is 0.
<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1 = Q[n, curBeat];
7 op2 = Q[m, curBeat];
8 if carryInit && IsFirstBeat() then
9     (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = ('0', '0', '0', '0');
10 (result, carryOut, -) = AddWithCarry(op1, op2, FPSCR.C);
11 if elmtMask[0] == '1' then
12     (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = ('0', '0', carryOut, '0');
13

```

```
14 for e = 0 to 3
15     if elmtMask[e] == '1' then
16         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.302 VADD

Floating-point Add. Floating-point Add adds two half-precision or single-precision or double-precision registers, and places the result in the destination register.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	1	1	Vn				Vd				1	0	size		N	0	M	0	Vm			

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VADD{<c>}{<q>}.F16 {<Sd>, } <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VADD{<c>}{<q>}.F32 {<Sd>, } <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VADD{<c>}{<q>}.F64 {<Dd>, } <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
7 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
8 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();

```

```
3      ExecuteFPCheck();
4      case size of
5          when '01' S[d] = Zeros(16) : FPAdd(S[n][15:0], S[m][15:0], TRUE);
6          when '10' S[d] = FPAdd(S[n], S[m], TRUE);
7          when '11' D[d] = FPAdd(D[n], D[m], TRUE);
```


C2.4.303 VADD (floating-point)

Vector Add. Add the value of the elements in the first source vector register to either the respective elements in the second source vector register or a general-purpose register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	0	sz	Qn	0	Qd	0	1	1	0	1	N	1	M	0	Qm	0						

T1: VADD variant

VADD<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (M:Qm);
5 n      = UInt (N:Qn);
6 esize  = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 withScalar = FALSE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	D	1	1	Qn	0	Qd	0	1	1	1	1	N	1	0	0	Rm							

T2: VADD variant

VADD<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (Rm);
5 n      = UInt (N:Qn);
6 esize  = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 withScalar = TRUE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F32 Encoded as sz = 0 F16 Encoded as sz = 1
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.
<Rm>	Source general-purpose register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[n, curBeat];
8 if withScalar then
9     for e = 0 to elements-1
10         // Avoid Floating-point exceptions on a predicated lane by checking the element mask
11         pred = (elmtMask[e*(esize>>3)] == '0');
12         value = FPAdd(Elem[op1, e, esize], R[m][esize-1:0], FALSE, pred);
13         Elem[result, e, esize] = value;
14 else
15     for e = 0 to elements-1
16         // Avoid Floating-point exceptions on a predicated lane by checking the element mask
17         pred = (elmtMask[e*(esize>>3)] == '0');
18         op2 = Q[m, curBeat];
19         value = FPAdd(Elem[op1, e, esize], Elem[op2, e, esize], FALSE, pred);
20         Elem[result, e, esize] = value;
21
22 for e = 0 to 3
23     if elmtMask[e] == '1' then
24         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.304 VADD (vector)

Vector Add. Add the value of the elements in the first source vector register to either the respective elements in the second source vector register or a general-purpose register. The result is then written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	size		Qn		0		Qd		0	1	0	0	0	N	1	M	0		Qm		0	

T1: VADD variant

VADD<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(N:Qn);
7 withScalar = FALSE;
8 esize = 8 << UInt(size);
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	size		Qn		1		Qd		0	1	1	1	1	N	1	0	0		Rm			

T2: VADD variant

VADD<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || N == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(Rm);
6 n = UInt(N:Qn);
7 withScalar = TRUE;
8 esize = 8 << UInt(size);
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: I8 Encoded as size = 00 I16 Encoded as size = 01 I32 Encoded as size = 10
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.
<Rm>	Source general-purpose register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1 = Q[n, curBeat];
8  if withScalar then
9      for e = 0 to elements-1
10         value = Elem[op1, e, esize] + R[m][esize-1:0];
11         Elem[result, e, esize] = value;
12 else
13     op2 = Q[m, curBeat];
14     for e = 0 to elements-1
15         value = Elem[op1, e, esize] + Elem[op2, e, esize];
16         Elem[result, e, esize] = value;
17
18 for e = 0 to 3
19     if elmtMask[e] == '1' then
20         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.305 VADDLV

Vector Add Long Across Vector. Add across the elements of a vector accumulating the result into a scalar. The 64-bit result is stored across two registers, the upper-half is stored in an odd-numbered register and the lower half is stored in an even-numbered register. The initial value of the general-purpose destination registers can optionally be added to the result.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	U	1	1	1	0	1	RdaHi				(1)	(0)	0	1	RdaLo				(0)	1	1	1	1	0	0	A	0	Qm		0

T1: VADDLV variant

VADDLV{A}<v><q>.<dt> RdaLo, RdaHi, Qm

Decode for this encoding

```

1 if RdaHi == '111' then SEE "VADDV";
2 CheckDecodeFaults(ExtType_Mve);
3 dah      = UInt(RdaHi:'1');
4 dal      = UInt(RdaLo:'0');
5 m        = UInt(Qm);
6 accumulate = (A == '1');
7 esize     = 32;
8 elements  = 32 DIV esize;
9 unsigned  = (U == '1');
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 A Encoded as A = 1
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Unsigned flag: S indicates signed, U indicates unsigned. This parameter must be one of the following values: S32 Encoded as U = 0 U32 Encoded as U = 1
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination. This must be an even numbered register. The value RdaLo » 1 is encoded in the RdaLo field.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination. This must be an odd numbered register. The value RdaHi » 1 is encoded in the RdaHi field.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
```

```
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = if accumulate || !IsFirstBeat() then Int(R[dah]:R[dal], unsigned) else 0;
7 op      = Q[m, curBeat];
8
9 for e = 0 to elements-1
10     if elmtMask[e*(esize>>3)] == '1' then
11         result = result + Int(Elem[op, e, esize], unsigned);
12
13 R[dah] = result[63:32];
14 R[dal] = result[31:0];
```

C2.4.306 VADDV

Vector Add Across Vector. Add across the elements of a vector accumulating the result into a scalar. The initial value of the general-purpose destination register can optionally be added to the result.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	1	1	1	size	0	1		Rda	(0)	1	1	1	1	0	0	A	0		Qm		0		

T1: VADDV variant

VADDV{A}<v><q>.<dt> Rda, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 da      = UInt(Rda:'0');
4 m       = UInt(Qm);
5 accumulate = (A == '1');
6 esize    = 8 << UInt(size);
7 elements  = 32 DIV esize;
8 unsigned  = (U == '1');
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 A Encoded as A = 1
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	This parameter determines the following values: – Unsigned flag: S indicates signed, U indicates unsigned. – Size: indicates the size of the elements in the vector. This parameter must be one of the following values: S8 Encoded as size = 00, U = 0 U8 Encoded as size = 00, U = 1 S16 Encoded as size = 01, U = 0 U16 Encoded as size = 01, U = 1 S32 Encoded as size = 10, U = 0 U32 Encoded as size = 10, U = 1
<Rda>	General-purpose source and destination register. This must be an even numbered register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();

```

```
5
6 result = if accumulate || !IsFirstBeat() then Int(R[da], unsigned) else 0;
7 op      = Q[m, curBeat];
8
9 for e = 0 to elements-1
10     if elmtMask[e*(esize>>3)] == '1' then
11         result = result + Int(Elem[op, e, esize], unsigned);
12
13 R[da] = result[31:0];
```


C2.4.307 VAND

Vector Bitwise And. Compute a bitwise AND of a vector register with another vector register. The result is written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	0	0	Qn	0	Qd	0	0	0	0	1	N	1	M	1	Qm	0						

T1: VAND variant

VAND<v><q>{.<dt>} Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 n = UInt(N:Qn);
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	An optional data type. It is ignored by assemblers and does not affect the encoding. This can be one of the following: S8, S16, S32, U8, U16, U32, I8, I16, I32, F16, F32.
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Q[n, curBeat] AND Q[m, curBeat];
7
8 for e = 0 to 3
9     if elmtMask[e] == '1' then
10         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.308 VAND (immediate)

Vector Bitwise AND. This is a pseudo-instruction, equivalent to a VBIC (immediate) instruction with the immediate value bitwise inverted.

This is an alias of [VBIC \(immediate\)](#).

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	i	1	1	1	1	1	Da	0	0	0	imm3			Qda	0	cmode			0	1	1	1	imm4						

VAND variant

VAND<v><q>.<dt> Qda, #<imm>

is equivalent to

VBIC<v><q>.<dt> Qda, #~<imm>

and is never the preferred disassembly

C2.4.309 VBIC (immediate)

Vector Bitwise Clear. Compute a bitwise AND of a vector register and the complement of an immediate value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	i	1	1	1	1	1	Da	0	0	0	imm3	Qda	0	cmode	0	1	1	1	imm4										

T1: VBIC variant

VBIC<v><q>.<dt> Qda, #<imm>

Decode for this encoding

```

1 if cmode == '111x' then SEE "VMOV (immediate) (vector)";
2 if cmode IN {'0xx0', '110x', '10x0'} then SEE "VMVN (immediate)";
3 CheckDecodeFaults(ExtType_Mve);
4 if Da == '1' then UNDEFINED;
5 da = UInt(Da:Qda);
6 imm64 = AdvSIMDEExpandImm('1', cmode, i:imm3:imm4);
7 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
<dt> Size: indicates the size of the elements in the vector, for use with the AdvSIMDEExpandImm() function.
This parameter must be one of the following values:

I32	Encoded as:
	cmode = 0001
	cmode = 0011
	cmode = 0101
	cmode = 0111
I16	Encoded as:
	cmode = 1001
	cmode = 1011

<Qda> Source and destination vector register.
<imm> The immediate value to load in to each element. This must be an immediate that can be encoded for use with the AdvSIMDEExpandImm() function.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 opd = Q[da, curBeat];
7 imm32 = if curBeat[0] == '0' then imm64[31:0] else imm64[63:32];
8 result = opd AND NOT(imm32);
9

```

```
10 for e = 0 to 3
11     if elmtMask[e] == '1' then
12         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.310 VBIC (register)

Vector Bitwise Clear. Compute a bitwise AND of a vector register and the complement of a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	0	1	Qn	0	Qd	0	0	0	0	1	N	1	M	1	Qm	0						

T1: VBIC variant

VBIC<v><q>{.<dt>} Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 n = UInt(N:Qn);
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	An optional data type. It is ignored by assemblers and does not affect the encoding. This can be one of the following: S8, S16, S32, U8, U16, U32, I8, I16, I32, F16, F32.
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 opm = Q[m, curBeat];
7 opn = Q[n, curBeat];
8 result = opn AND NOT(opm);
9
10 for e = 0 to 3
11     if elmtMask[e] == '1' then
12         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.311 VBRSR

Vector Bit Reverse and Shift Right. Reverse the specified number of LSB bits in each element of a vector register and set the other bits to zero. The number of bits to reverse is read in from the bottom byte of Rm and clamped to the range [0, <dt>].

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	D	size		Qn	1		Qd	1	1	1	1	0	N	1	1	0						Rm	

T1: VBRSR variant

VBRSR<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(Rm);
6 n      = UInt(N:Qn);
7 esize  = 8 << UInt(size);
8 elements = 32 DIV esize;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 8 Encoded as size = 00
 16 Encoded as size = 01
 32 Encoded as size = 10

<Qd> Destination vector register.
 <Qn> Source vector register.
 <Rm> General-purpose register containing the number of LSB bits to reverse in its bottom 8 bits.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 revBit = UInt(R[m][7:0]);
9 for e = 0 to elements-1
10     Elem[result, e, esize] = BitReverseShiftRight(Elem[op1, e, esize], revBit);
11
```



```
12 for e = 0 to 3
13     if elmtMask[e] == '1' then
14         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.312 VCADD

Vector Complex Add with Rotate. This instruction performs a complex addition of the first operand with the second operand rotated in the complex plane by the specified amount. A 90 degree rotation of this operand corresponds to a multiplication by a positive imaginary unit, while a 270 degree rotation corresponds to a multiplication by a negative imaginary unit. Even and odd elements of the source vectors are interpreted to be the real and imaginary components, respectively, of a complex number. The result is then written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	D	size		Qn		0		Qd	rot	1	1	1	1	N	0	M	0		Qm		0		

T1: VCADD variant

VCADD<v><q>.<dt> Qd, Qn, Qm, #<rotate>

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 esize  = 8 << UInt(size);
8 elements = 32 DIV esize;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if D:Qd == M:Qm && size == '10' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 I8 Encoded as size = 00
 I16 Encoded as size = 01
 I32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qn> First source vector register.
 <Qm> Second source vector register.
 <rotate> The rotation amount.
 This parameter must be one of the following values:
 #90 Encoded as rot = 0
 #270 Encoded as rot = 1

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
```

```

5
6 result = Zeros(32);
7 // 32 bit operations are handled differently as they perform cross beat
8 // register accesses
9 if esize == 32 then
10     case rot:curBeat[0] of
11         when '00' result = (Q[n, curBeat] - Q[m, curBeat+1])[31:0];
12         when '01' result = (Q[n, curBeat] + Q[m, curBeat-1])[31:0];
13         when '10' result = (Q[n, curBeat] + Q[m, curBeat+1])[31:0];
14         when '11' result = (Q[n, curBeat] - Q[m, curBeat-1])[31:0];
15     else
16         op1 = Q[n, curBeat];
17         op2 = Q[m, curBeat];
18         for e = 0 to elements-1
19             case rot:e[0] of
20                 when '00' value = Elem[op1, e, esize] - Elem[op2, e+1, esize];
21                 when '01' value = Elem[op1, e, esize] + Elem[op2, e-1, esize];
22                 when '10' value = Elem[op1, e, esize] + Elem[op2, e+1, esize];
23                 when '11' value = Elem[op1, e, esize] - Elem[op2, e-1, esize];
24             Elem[result, e, esize] = value[esize-1:0];
25
26 for e = 0 to 3
27     if elmtMask[e] == '1' then
28         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.313 VCADD (floating-point)

Vector Complex Add with Rotate. This instruction performs a complex addition of the first operand with the second operand rotated in the complex plane by the specified amount. A 90 degree rotation of this operand corresponds to a multiplication by a positive imaginary unit, while a 270 degree rotation corresponds to a multiplication by a negative imaginary unit. Even and odd elements of the source vectors are interpreted to be the real and imaginary components, respectively, of a complex number. The results are written into the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	rot	1	D	0	sz	Qn	0	Qd	0	1	0	0	0	N	1	M	0	Qm	0						

T1: VCADD variant

VCADD<v><q>.<dt> Qd, Qn, Qm, #<rotate>

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d = UInt (D:Qd);
4 m = UInt (M:Qm);
5 n = UInt (N:Qn);
6 esize = if sz == '0' then 16 else 32;
7 elements = 32 DIV esize;
8 if InitITBlock() then CONSTRAINED_UNPREDICTABLE;
9 if D:Qd == M:Qm && sz == '1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the floating-point format used.
 This parameter must be one of the following values:
 F16 Encoded as sz = 0
 F32 Encoded as sz = 1
 <Qd> Destination vector register.
 <Qn> First source vector register.
 <Qm> Second source vector register.
 <rotate> The rotation amount.
 This parameter must be one of the following values:
 #90 Encoded as rot = 0
 #270 Encoded as rot = 1

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
```

```

7  if esize == 32 then
8      // Avoid Floating-point exceptions on a predicated lane by checking the element mask
9      pred = (elmtMask[0*(esize>>3)] == '0');
10     case rot:curBeat[0] of
11         when '00' result = FPSub(Q[n, curBeat], Q[m, curBeat+1], FALSE, pred);
12         when '01' result = FPAdd(Q[n, curBeat], Q[m, curBeat-1], FALSE, pred);
13         when '10' result = FPAdd(Q[n, curBeat], Q[m, curBeat+1], FALSE, pred);
14         when '11' result = FPSub(Q[n, curBeat], Q[m, curBeat-1], FALSE, pred);
15     else
16         op1 = Q[n, curBeat];
17         op2 = Q[m, curBeat];
18         for e = 0 to elements-1
19             // Avoid Floating-point exceptions on a predicated lane by checking the element mask
20             pred = (elmtMask[e*(esize>>3)] == '0');
21             case rot:e[0] of
22                 when '00' value = FPSub(Elem[op1, e, esize], Elem[op2, e+1, esize], FALSE, pred);
23                 when '01' value = FPAdd(Elem[op1, e, esize], Elem[op2, e-1, esize], FALSE, pred);
24                 when '10' value = FPAdd(Elem[op1, e, esize], Elem[op2, e+1, esize], FALSE, pred);
25                 when '11' value = FPSub(Elem[op1, e, esize], Elem[op2, e-1, esize], FALSE, pred);
26             Elem[result, e, esize] = value;
27
28     for e = 0 to 3
29         if elmtMask[e] == '1' then
30             Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.314 VCLS

Vector Count Leading Sign-bits. Count the leading sign bits of each element of a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	0		Qd	0	0	1	0	0	0	0	1	M	0		Qm	0		

T1: VCLS variant

VCLS<v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 esize = 8 << UInt(size);
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 S8 Encoded as size = 00
 S16 Encoded as size = 01
 S32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[m, curBeat];
8 for e = 0 to elements-1
9     value = CountLeadingSignBits(Elm[op1, e, esize]);
10    Elm[result, e, esize] = value[esize-1:0];
11
12 for e = 0 to 3
13     if elmtMask[e] == '1' then
14         Elm[Q[d, curBeat], e, 8] = Elm[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.315 VCLZ

Vector Count Leading Zeros. Count the leading zeros of each element of a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	0		Qd	0	0	1	0	0	1	1	M	0	Qm	0				

T1: VCLZ variant

VCLZ<v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 esize = 8 << UInt(size);
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 I8 Encoded as size = 00
 I16 Encoded as size = 01
 I32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[m, curBeat];
8 for e = 0 to elements-1
9     value = CountLeadingZeroBits(Elm[op1, e, esize]);
10    Elem[result, e, esize] = value[esize-1:0];
11
12 for e = 0 to 3
13     if elmtMask[e] == '1' then
14         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.316 VCMLA (floating-point)

Vector Complex Multiply Accumulate. This instruction operates on complex numbers that are represented in registers as pairs of elements. Each element holds a floating-point value. The odd element holds the imaginary part of the number, and the even element holds the real part of the number. The instruction performs the computation on the corresponding complex number element pairs from the two source registers and the destination register. Considering the complex number from the second source register on an Argand diagram, the number is rotated counterclockwise by 0, 90, 180, or 270 degrees. If the transformation was a rotation by 0 or 180 degrees, the two elements of the transformed complex number are multiplied by the real element of the first source register. If the transformation was a rotation by 90 or 270 degrees, the two elements are multiplied by the imaginary element of the complex number from the first source register. The result of the multiplication is added on to the existing value in the destination vector register. The multiplication and addition operations are fused and the result is not rounded.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0		rot	Da	1	sz		Qn		0	Qda		0	1	0	0	0	N	1	M	0		Qm		0	

T1: VCMLA variant

VCMLA<v><q>.<dt> Qda, Qn, Qm, #<rotate>

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if Da == '1' || M == '1' || N == '1' then UNDEFINED;
3 da      = UInt (Da:Qda);
4 m      = UInt (M:Qm);
5 n      = UInt (N:Qn);
6 esize   = if sz == '0' then 16 else 32;
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
9 if sz == '1' && (Da:Qda == M:Qm || Da:Qda == N:Qn) then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the floating-point format used.
 This parameter must be one of the following values:
 F16 Encoded as sz = 0
 F32 Encoded as sz = 1

<Qda> Source and destination vector register.
 <Qn> First source vector register.
 <Qm> Second source vector register.
 <rotate> The rotation amount.
 This parameter must be one of the following values:
 #0 Encoded as rot = 00
 #90 Encoded as rot = 01
 #180 Encoded as rot = 10
 #270 Encoded as rot = 11

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 dest = Q[da, curBeat];
8 if esize == 32 then
9     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
10    pred = (elmtMask[0*(esize>>3)] == '0');
11    if (curBeat[0] == '0') then
12        case rot of
13            when '00'
14                element1 = Q[m, curBeat];
15                element2 = Q[n, curBeat];
16            when '01'
17                element1 = FPNeg(Q[m, curBeat+1]);
18                element2 = Q[n, curBeat+1];
19            when '10'
20                element1 = FPNeg(Q[m, curBeat]);
21                element2 = Q[n, curBeat];
22            when '11'
23                element1 = Q[m, curBeat+1];
24                element2 = Q[n, curBeat+1];
25        else
26            case rot of
27                when '00'
28                    element1 = Q[m, curBeat];
29                    element2 = Q[n, curBeat-1];
30                when '01'
31                    element1 = Q[m, curBeat-1];
32                    element2 = Q[n, curBeat];
33                when '10'
34                    element1 = FPNeg(Q[m, curBeat]);
35                    element2 = Q[n, curBeat-1];
36                when '11'
37                    element1 = FPNeg(Q[m, curBeat-1]);
38                    element2 = Q[n, curBeat];
39            result = FPMulAdd(dest, element2, element1, FALSE, pred);
40    else
41        // Avoid Floating-point exceptions on a predicated lane by checking the element mask
42        pred0 = (elmtMask[0*(esize>>3)] == '0');
43        pred1 = (elmtMask[1*(esize>>3)] == '0');
44        op1 = Q[m, curBeat];
45        op2 = Q[n, curBeat];
46        case rot of
47            when '00'
48                elem1 = Elem[op1, 0, esize];
49                elem2 = Elem[op2, 0, esize];
50                elem3 = Elem[op1, 1, esize];
51                elem4 = Elem[op2, 0, esize];
52            when '01'
53                elem1 = FPNeg(Elem[op1, 1, esize]);
54                elem2 = Elem[op2, 1, esize];
55                elem3 = Elem[op1, 0, esize];
56                elem4 = Elem[op2, 1, esize];
57            when '10'
58                elem1 = FPNeg(Elem[op1, 0, esize]);
59                elem2 = Elem[op2, 0, esize];
60                elem3 = FPNeg(Elem[op1, 1, esize]);
61                elem4 = Elem[op2, 0, esize];
62            when '11'
63                elem1 = Elem[op1, 1, esize];
64                elem2 = Elem[op2, 1, esize];
65                elem3 = FPNeg(Elem[op1, 0, esize]);
66                elem4 = Elem[op2, 1, esize];
67        Elem[result, 0, esize] = FPMulAdd(Elem[dest, 0, esize], elem2, elem1, FALSE, pred0);

```

```
68     Elem[result, 1, esize] = FPMulAdd(Elem[dest, 1, esize], elem4, elem3, FALSE, pred1);
69
70 for e = 0 to 3
71     if elmtMask[e] == '1' then
72         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.317 VCMF

Floating-point Compare. Floating-point Compare compares two registers, or one register and zero. It writes the result to FPSCR condition flags. These are normally transferred to the APSR condition flags by a subsequent [VMRS](#) instruction.

It raises an Invalid Operation exception only if either operand is a signaling NaN.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	1	0	0	Vd		1	0	size		1	M	0	Vm						

E = 0

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

`VCMP {<c>} {<q>}.F16 <Sd>, <Sm>`

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

`VCMP {<c>} {<q>}.F32 <Sd>, <Sm>`

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

`VCMP {<c>} {<q>}.F64 <Dd>, <Dm>`

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 quiet_nan_exc = (E == '1'); with_zero = FALSE;
6 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
7 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	1	Vd		1	0	size		1	(0)	0	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	

E = 0

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VCMP{<c>}{<q>}.F16 <Sd>, #0.0

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VCMP{<c>}{<q>}.F32 <Sd>, #0.0

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VCMP{<c>}{<q>}.F64 <Dd>, #0.0

Decode for this encoding

```
1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 quiet_nan_exc = (E == '1'); with_zero = TRUE;
6 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
7 m = integer UNKNOWN;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   case size of
5     when '01'
6       op16 = if with_zero then FPZero('0',16) else S[m][15:0];
7       (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = FPCompare(S[d][15:0], op16, quiet_nan_exc,
8                                                         TRUE);
9     when '10'
10      op32 = if with_zero then FPZero('0',32) else S[m];
11      (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = FPCompare(S[d], op32, quiet_nan_exc,
12                                                         TRUE);
13     when '11'
14      op64 = if with_zero then FPZero('0',64) else D[m];
15      (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = FPCompare(D[d], op64, quiet_nan_exc,
16                                                         TRUE);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.RDIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.318 VCMP (floating-point)

Vector Compare. Perform a lane-wise comparison between each element in the first source vector register and either the respective elements in the second source vector register or the value of a general-purpose register. The resulting boolean conditions are placed in VPR.P0. The VPR.P0 flags for predicated lanes are zeroed.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	0	1	1	Qn	1	0	0	0	fcA	1	1	1	1	fcC	0	M	0	Qm	fcB				

T1: VCMP variant

VCMP<v><q>.<dt> <fc>, Qn, Qm

Decode for this encoding

```

1 if fcA == '0' && fcB == '1' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_MveFp);
3 if M == '1' then UNDEFINED;
4 m      = UInt(M:Qm);
5 n      = UInt(Qn);
6 f_cond = fcA:fcB:fcC;
7 withScalar = FALSE;
8 esize     = 8 << UInt(if sz == '1' then '01' else '10');
9 elements  = 32 DIV esize;
10 ebytes    = esize DIV 8;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	0	1	1	Qn	1	0	0	0	fcA	1	1	1	1	fcC	1	fcB	0	Rm					

T2: VCMP variant

VCMP<v><q>.<dt> <fc>, Qn, Rm

Decode for this encoding

```

1 if Rm == '1101' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_MveFp);
3 m      = UInt(Rm);
4 n      = UInt(Qn);
5 f_cond = fcA:fcB:fcC;
6 withScalar = TRUE;
7 esize     = 8 << UInt(if sz == '1' then '01' else '10');
8 elements  = 32 DIV esize;
9 ebytes    = esize DIV 8;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if fcA == '0' && fcB == '1' then CONSTRAINED_UNPREDICTABLE;

```


Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F32 Encoded as sz = 0 F16 Encoded as sz = 1
<fc>	The comparison condition to use. This parameter must be one of the following values: EQ Encoded as fcA = 0, fcB = 0, fcC = 0 NE Encoded as fcA = 0, fcB = 0, fcC = 1 GE Encoded as fcA = 1, fcB = 0, fcC = 0 LT Encoded as fcA = 1, fcB = 0, fcC = 1 GT Encoded as fcA = 1, fcB = 1, fcC = 0 LE Encoded as fcA = 1, fcB = 1, fcC = 1
<Qn>	First source vector register
<Qm>	Source vector register.
<Rm>	Source general-purpose register (ZR is permitted, PC is not).

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  op1      = Q[n, curBeat];
7  beatPred = Zeros(4);
8  if withScalar then
9      op2 = RZ[m][esize-1:0];
10 else
11     opm = Q[m, curBeat];
12     for e = 0 to elements-1
13         if !withScalar then
14             op2 = Elem[opm, e, esize];
15             // Avoid Floating-point exceptions on a predicated lane by checking the element mask
16             predicated = (elmtMask[e*(esize>>3)] == '0');
17             (flN, flZ, flC, flV) = FPCompare(Elem[op1, e, esize], op2, TRUE, FALSE, predicated);
18             pred = ConditionHolds(f_cond, flN, flZ, flC, flV);
19             Elem[beatPred, e, ebytes] = Replicate(if pred then '1' else '0');
20
21 Elem[VPR.P0, curBeat, 4] = beatPred AND elmtMask;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.319 VCMP (vector)

Vector Compare. Perform a lane-wise comparison between each element in the first source vector register and either the respective elements in the second source vector register or the value of a general-purpose register. The resulting boolean conditions are placed in VPR.P0. The VPR.P0 flags for predicated lanes are zeroed.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	0	size		Qn	1	0	0	0	0	1	1	1	1	fc	0	M	0	Qm	0				

T1: VCMP variant

VCMP<v><q>.<dt> <fc>, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if M == '1' then UNDEFINED;
4 m      = UInt(M:Qm);
5 n      = UInt(Qn);
6 f_cond = '00':fc;
7 withScalar = FALSE;
8 esize      = 8 << UInt(size);
9 elements   = 32 DIV esize;
10 ebytes     = esize DIV 8;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	0	size		Qn	1	0	0	0	0	1	1	1	1	fc	0	M	0	Qm	1				

T2: VCMP variant

VCMP<v><q>.<dt> <fc>, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if M == '1' then UNDEFINED;
4 m      = UInt(M:Qm);
5 n      = UInt(Qn);
6 f_cond = '01':fc;
7 withScalar = FALSE;
8 esize      = 8 << UInt(size);
9 elements   = 32 DIV esize;
10 ebytes     = esize DIV 8;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T3

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	0	size		Qn	1	0	0	0	1	1	1	1	1	fcl	0	M	0		Qm	fch			

T3: VCMP variant

VCMP<v><q>.<dt> <fc>, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if M == '1' then UNDEFINED;
4 m      = UInt(M:Qm);
5 n      = UInt(Qn);
6 f_cond = '1':fch:fcl;
7 withScalar = FALSE;
8 esize      = 8 << UInt(size);
9 elements   = 32 DIV esize;
10 ebytes     = esize DIV 8;
11 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
```

T4

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	0	size		Qn	1	0	0	0	0	1	1	1	1	fc	1	0	0		Rm				

T4: VCMP variant

VCMP<v><q>.<dt> <fc>, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 m      = UInt(Rm);
4 n      = UInt(Qn);
5 f_cond = '00':fc;
6 withScalar = TRUE;
7 esize      = 8 << UInt(size);
8 elements   = 32 DIV esize;
9 ebytes     = esize DIV 8;
10 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '1101' then CONSTRAINED_UNPREDICTABLE;
```

T5

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	0	size		Qn	1	0	0	0	0	1	1	1	1	fc	1	1	0		Rm				

T5: VCMP variant

VCMP<v><q>.<dt> <fc>, Qn, Rm

Decode for this encoding

```
1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 m = UInt(Rm);
4 n = UInt(Qn);
5 f_cond = '01':fc;
6 withScalar = TRUE;
7 esize = 8 << UInt(size);
8 elements = 32 DIV esize;
9 ebytes = esize DIV 8;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '1101' then CONSTRAINED_UNPREDICTABLE;
```

T6

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	0	size		Qn		1	0	0	0	1	1	1	1	1	fcl	1	fch	0			Rm		

T6: VCMP variant

VCMP<v><q>.<dt> <fc>, Qn, Rm

Decode for this encoding

```
1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 m = UInt(Rm);
4 n = UInt(Qn);
5 f_cond = '1':fch:fcl;
6 withScalar = TRUE;
7 esize = 8 << UInt(size);
8 elements = 32 DIV esize;
9 ebytes = esize DIV 8;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '1101' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for T1 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
I8 Encoded as size = 00
I16 Encoded as size = 01
I32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
EQ Encoded as fc = 0
NE Encoded as fc = 1

Assembler symbols for T2 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
U8 Encoded as size = 00
U16 Encoded as size = 01
U32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
CS Encoded as fc = 0
HI Encoded as fc = 1

Assembler symbols for T3 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
S8 Encoded as size = 00
S16 Encoded as size = 01
S32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
GE Encoded as fch = 0, fcl = 0
LT Encoded as fch = 0, fcl = 1
GT Encoded as fch = 1, fcl = 0
LE Encoded as fch = 1, fcl = 1

Assembler symbols for T4 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
I8 Encoded as size = 00
I16 Encoded as size = 01
I32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
EQ Encoded as fc = 0
NE Encoded as fc = 1

Assembler symbols for T5 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
U8 Encoded as size = 00
U16 Encoded as size = 01
U32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
CS Encoded as fc = 0
HI Encoded as fc = 1

Assembler symbols for T6 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
S8 Encoded as size = 00
S16 Encoded as size = 01
S32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
GE Encoded as fch = 0, fcl = 0
LT Encoded as fch = 0, fcl = 1
GT Encoded as fch = 1, fcl = 0
LE Encoded as fch = 1, fcl = 1

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qn>	First source vector register
<Qm>	Second source vector register
<Rm>	Source general-purpose register (ZR is permitted, PC is not).

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1      = Q[n, curBeat];
7 beatPred = Zeros(4);
8 if withScalar then
9     op2 = RZ[m][esize-1:0];
10 else
11     opm = Q[m, curBeat];
12 for e = 0 to elements-1
13     if !withScalar then
14         op2 = Elem[opm, e, esize];
15         (result, flC, flV) = AddWithCarry(Elem[op1, e, esize], NOT(op2), '1');
16         flN                = result[esize-1];
17         flZ                = IsZeroBit(result);
18         pred = ConditionHolds(f_cond, flN, flZ, flC, flV);
19         Elem[beatPred, e, ebytes] = Replicate(if pred then '1' else '0');
20
21 Elem[VPR.P0, curBeat, 4] = beatPred AND elmtMask;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.320 VCMPE

Floating-point Compare, raising Invalid Operation on NaN. Floating-point Compare, raising Invalid Operation on NaN compares two registers, or one register and zero. It writes the result to FPSCR condition flags. These are normally transferred to the APSR condition flags by a subsequent [VMRS](#) instruction.

It raises an Invalid Operation exception if either operand is any type of NaN.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	1	0	0	Vd		1	0	size		1	M	0	Vm						

E = 1

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VCMPE{<c>}{<q>}.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VCMPE{<c>}{<q>}.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VCMPE{<c>}{<q>}.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 quiet_nan_exc = (E == '1'); with_zero = FALSE;
6 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
7 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	1	0	1	Vd		1	0	size		1	(0)	0	(0)	(0)	(0)	(0)	(0)	(0)	

E = 1

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VCMPE{<c>}{<q>}.F16 <Sd>, #0.0

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VCMPE{<c>}{<q>}.F32 <Sd>, #0.0

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VCMPE{<c>}{<q>}.F64 <Dd>, #0.0

Decode for this encoding

```
1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 quiet_nan_exc = (E == '1'); with_zero = TRUE;
6 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
7 m = integer UNKNOWN;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   case size of
5     when '01'
6       op16 = if with_zero then FPZero('0',16) else S[m][15:0];
7       (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = FPCompare(S[d][15:0], op16, quiet_nan_exc,
8                                                         TRUE);
9     when '10'
10      op32 = if with_zero then FPZero('0',32) else S[m];
11      (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = FPCompare(S[d], op32, quiet_nan_exc,
12                                                         TRUE);
13    when '11'
14      op64 = if with_zero then FPZero('0',64) else D[m];
15      (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = FPCompare(D[d], op64, quiet_nan_exc,
16                                                         TRUE);
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.RDIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.321 VCMUL (floating-point)

Vector Complex Multiply. This instruction operates on complex numbers that are represented in registers as pairs of elements. Each element holds a floating-point value. The odd element holds the imaginary part of the number, and the even element holds the real part of the number. The instruction performs the computation on the corresponding complex number element pairs from the two source registers and the destination register. Considering the complex number from the second source register on an Argand diagram, the number is rotated counterclockwise by 0, 90, 180, or 270 degrees. If the transformation was a rotation by 0 or 180 degrees, the two elements of the transformed complex number are multiplied by the real element of the first source register. If the transformation was a rotation by 90 or 270 degrees, the two elements are multiplied by the imaginary element of the complex number from the first source register. The results are written into the destination vector register.

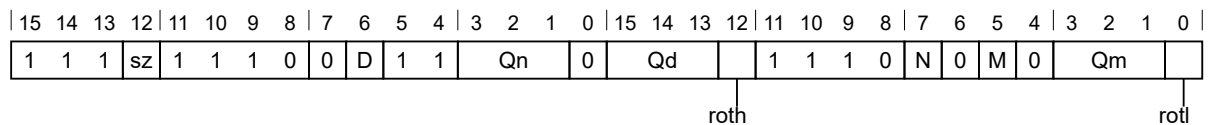
This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension



T1: VCMUL variant

VCMUL<v><q>.<dt> Qd, Qn, Qm, #<rotate>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_MveFp);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt(D:Qd);
4 m      = UInt(M:Qm);
5 n      = UInt(N:Qn);
6 esize  = if sz == '0' then 16 else 32;
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
9 if sz == '1' && (D:Qd == M:Qm || D:Qd == N:Qn) then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F16 Encoded as sz = 0 F32 Encoded as sz = 1
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.
<rotate>	The rotation amount. This parameter must be one of the following values: #0 Encoded as roth = 0, rotl = 0 #90 Encoded as roth = 0, rotl = 1 #180 Encoded as roth = 1, rotl = 0 #270 Encoded as roth = 1, rotl = 1

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  if esize == 32 then
8      // Avoid Floating-point exceptions on a predicated lane by checking the element mask
9      pred = (elmtMask[0*(esize>>3)] == '0');
10     if (curBeat[0] == '0') then
11         case roth:rotl of
12             when '00'
13                 element1 = Q[m, curBeat];
14                 element2 = Q[n, curBeat];
15             when '01'
16                 element1 = FPNeg(Q[m, curBeat+1]);
17                 element2 = Q[n, curBeat+1];
18             when '10'
19                 element1 = FPNeg(Q[m, curBeat]);
20                 element2 = Q[n, curBeat];
21             when '11'
22                 element1 = Q[m, curBeat+1];
23                 element2 = Q[n, curBeat+1];
24         else
25             case roth:rotl of
26                 when '00'
27                     element1 = Q[m, curBeat];
28                     element2 = Q[n, curBeat-1];
29                 when '01'
30                     element1 = Q[m, curBeat-1];
31                     element2 = Q[n, curBeat];
32                 when '10'
33                     element1 = FPNeg(Q[m, curBeat]);
34                     element2 = Q[n, curBeat-1];
35                 when '11'
36                     element1 = FPNeg(Q[m, curBeat-1]);
37                     element2 = Q[n, curBeat];
38             result = FPMul(element2, element1, FALSE, pred);
39     else
40         // Avoid Floating-point exceptions on a predicated lane by checking the element mask
41         pred0 = (elmtMask[0*(esize>>3)] == '0');
42         pred1 = (elmtMask[1*(esize>>3)] == '0');
43         op1 = Q[m, curBeat];
44         op2 = Q[n, curBeat];
45         case roth:rotl of
46             when '00'
47                 elem1 = Elem[op1, 0, esize];
48                 elem2 = Elem[op2, 0, esize];
49                 elem3 = Elem[op1, 1, esize];
50                 elem4 = Elem[op2, 0, esize];
51             when '01'
52                 elem1 = FPNeg(Elem[op1, 1, esize]);
53                 elem2 = Elem[op2, 1, esize];
54                 elem3 = Elem[op1, 0, esize];
55                 elem4 = Elem[op2, 1, esize];
56             when '10'
57                 elem1 = FPNeg(Elem[op1, 0, esize]);
58                 elem2 = Elem[op2, 0, esize];
59                 elem3 = FPNeg(Elem[op1, 1, esize]);
60                 elem4 = Elem[op2, 0, esize];
61             when '11'
62                 elem1 = Elem[op1, 1, esize];
63                 elem2 = Elem[op2, 1, esize];
64                 elem3 = FPNeg(Elem[op1, 0, esize]);
65                 elem4 = Elem[op2, 1, esize];
66         Elem[result, 0, esize] = FPMul(elem2, elem1, FALSE, pred0);
67         Elem[result, 1, esize] = FPMul(elem4, elem3, FALSE, pred1);

```

```
68
69 for e = 0 to 3
70     if elmtMask[e] == '1' then
71         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.322 VCTP

Create Vector Tail Predicate. Creates a predicate pattern in VPR.P0 such that any element numbered the value of Rn or greater is predicated. Any element numbered lower than the value of Rn is not predicated. If placed within a VPT block and a lane is predicated, the corresponding VPR.P0 pattern will also be predicated. The generated VPR.P0 pattern can be used by an ensuing predication instruction to apply tail predication on a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	size		Rn				1	1	1	0	1	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	1

T1: VCTP variant

VCTP<v><q>.<dt> Rn

Decode for this encoding

```

1 if Rn == '1111' then SEE "Related encodings";
2 if !HaveMve() then UNDEFINED;
3 HandleException(CheckCPEnabled(10));
4 n = UInt(Rn);
5 predSize = UInt(size);
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
7 if Rn == '1101' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> The size of the elements in the vector to process.
 This parameter must be one of the following values:
 8 Encoded as size = 00
 16 Encoded as size = 01
 32 Encoded as size = 10
 64 Encoded as size = 11
 <Rn> The register containing the number of elements that need to be processed.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 loopCount = R[n];
7 if UInt(loopCount) <= (1 << (4 - predSize)) then
8     fullMask = ZeroExtend(Ones(UInt(loopCount[4-predSize:0] : Zeros(predSize))), 16);
9 else
10     fullMask = Ones(16);
11
12 Elem[VPR.P0, curBeat, 4] = elmtMask AND Elem[fullMask, curBeat, 4];
```

C2.4.323 VCVT (between double-precision and single-precision)

Convert between double-precision and single-precision. This instruction does one of the following:

- Converts the value in a double-precision register to single-precision and writes the result to a single-precision register.
- Converts the value in a single-precision register to double-precision and writes the result to a double-precision register.

T1

Armv8-M Floating-point Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	1	1	1	Vd				1	0	1	sz	1	1	M	0	Vm			

Single-precision to double-precision variant

Applies when **sz == 0**.

VCVT{<c>}{<q>}.F64.F32 <Dd>, <Sm>

Double-precision to single-precision variant

Applies when **sz == 1**.

VCVT{<c>}{<q>}.F32.F64 <Sd>, <Dm>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_DpFp);
2 double_to_single = (sz == '1');
3 if double_to_single then
4     if VFPSmallRegisterBank() && (M == '1') then UNDEFINED;
5     d = UInt(Vd:D);
6     m = UInt(M:Vm);
7 else
8     if VFPSmallRegisterBank() && (D == '1') then UNDEFINED;
9     d = UInt(D:Vd);
10    m = UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations(); ExecuteFPCheck();
3     if double_to_single then
4         S[d] = FPDoubleToSingle(D[m], TRUE);
5     else
6         D[d] = FPSingleToDouble(S[m], TRUE);

```

C2.4.324 VCVT (between floating-point and fixed-point)

Floating-point Convert (between floating-point and fixed-point). Floating-point Convert (between floating-point and fixed-point) converts a value in a register from floating-point to fixed-point, or from fixed-point to floating-point, and places the result in the destination register. Software can specify the fixed-point value as either signed or unsigned.

The fixed-point value can be 16-bit or 32-bit. Conversions from fixed-point values take their operand from the low-order bits of the source register and ignore any remaining bits. Signed conversions to fixed-point values sign-extend the result value to the destination register width. Unsigned conversions to fixed-point values zero-extend the result value to the destination register width.

The floating-point to fixed-point operation uses the Round towards Zero rounding mode. The fixed-point to floating-point operation uses the Round to Nearest rounding mode.

T1

Armv8-M Floating-point Extension only, *sf == 11* UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	1	op	1	U	Vd				1	0	sf		sx	1	i	0	imm4			

Fixed-point to half-precision variant

Armv8.1-M Floating-point Extension only.

Applies when **op == 0 && sf == 01**.

VCVT{<c>}{<q>}.F16.<dt> <Sdm>, <Sdm>, #<fbits>

Half-precision to fixed-point variant

Armv8.1-M Floating-point Extension only.

Applies when **op == 1 && sf == 01**.

VCVT{<c>}{<q>}.<dt>.F16 <Sdm>, <Sdm>, #<fbits>

Fixed-point to single-precision variant

Armv8-M Floating-point Extension only.

Applies when **op == 0 && sf == 10**.

VCVT{<c>}{<q>}.F32.<dt> <Sdm>, <Sdm>, #<fbits>

Single-precision to fixed-point variant

Armv8-M Floating-point Extension only.

Applies when **op == 1 && sf == 10**.

VCVT{<c>}{<q>}.<dt>.F32 <Sdm>, <Sdm>, #<fbits>

Fixed-point to double-precision variant

Armv8-M Floating-point Extension only.

Applies when **op == 0 && sf == 11**.

VCVT{<c>}{<q>}.F64.<dt> <Ddm>, <Ddm>, #<fbits>

Double-precision to fixed-point variant

Armv8-M Floating-point Extension only.

Applies when **op == 1 && sf == 11**.

VCVT{<c>}{<q>}.<dt>.F64 <Ddm>, <Ddm>, #<fbits>

Decode for this encoding

```
1 dp_operation = (sf == '11');
2 CheckFPDecodeFaults(sf);
3 if VFPSmallRegisterBank() && dp_operation && (D == '1') then UNDEFINED;
4 if sf == '01' && InITBlock() then UNPREDICTABLE;
5 to_fixed = (op == '1'); unsigned = (U == '1');
6 esize = if sx == '0' then 16 else 32;
7 fract = esize - UInt(imm4:i);
8 if to_fixed then
9     round_zero = TRUE;
10 else
11     round_nearest = TRUE;
12 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
13 if fract < 0 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **fract < 0**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Is the data type for the fixed-point number, encoded in the "U:sx" field. It can have the following values: S16 when U = 0, sx = 0 S32 when U = 0, sx = 1 U16 when U = 1, sx = 0 U32 when U = 1, sx = 1
<Sdm>	Is the 32-bit name of the floating-point destination and source register, encoded in the "Vd:D" field.
<Ddm>	Is the 64-bit name of the floating-point destination and source register, encoded in the "D:Vd" field.
<fbits>	The number of fraction bits in the fixed-point number: - If <dt> is S16 or U16, <fbits> must be in the range 0-16. (16 - <fbits>) is encoded in [imm4, i] - If <dt> is S32 or U32, <fbits> must be in the range 1-32. (32 - <fbits>) is encoded in [imm4, i].

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     ExecuteFPCheck();
4     if to_fixed then
5         case sf of
6             when '01'
7                 result = FPToFixed(S[d][15:0], esize, fract, unsigned, round_zero, TRUE);
8                 S[d] = if unsigned then ZeroExtend(result, 32) else SignExtend(result, 32);
```



```

9      when '10'
10         result = FPToFixed(S[d], esize, fract, unsigned, round_zero, TRUE);
11         S[d] = if unsigned then ZeroExtend(result, 32) else SignExtend(result, 32);
12     when '11'
13         result = FPToFixed(D[d], esize, fract, unsigned, round_zero, TRUE);
14         D[d] = if unsigned then ZeroExtend(result, 64) else SignExtend(result, 64);
15 else
16     case sf of
17     when '01'
18         fp16 = FixedToFP(S[d][esize-1:0], 16, fract, unsigned, round_nearest, TRUE);
19         S[d] = Zeros(16):fp16;
20     when '10'
21         S[d] = FixedToFP(S[d][esize-1:0], 32, fract, unsigned, round_nearest, TRUE);
22     when '11'
23         D[d] = FixedToFP(D[d][esize-1:0], 64, fract, unsigned, round_nearest, TRUE);

```

C2.4.325 VCVT (between floating-point and fixed-point) (vector)

Vector Convert between floating-point and fixed-point. Convert between floating-point and fixed-point values in elements of a vector register. The number of fractional bits in the fixed-point value is specified by an immediate. Fixed-point values can be specified as signed or unsigned. The floating-point to fixed-point operation uses the Round towards Zero rounding mode. The fixed-point to floating-point operation uses the Round to Nearest rounding mode. For floating-point to fixed-point operation, if the source value is outside the range of the target fixed-point type, the result is saturated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	1	D	imm6				Qd		0	1	1	fsi	op	0	1	M	1	Qm		0				

T1: VCVT variant

VCVT<v><q>.<dt> Qd, Qm, #<fbits>

Decode for this encoding

```

1 if imm6 == '000xxx' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_MveFp);
3 if D == '1' || M == '1' then UNDEFINED;
4 if imm6 == '0xxxxx' || (fsi == '0' && imm6 == '10xxxx') then UNDEFINED;
5 d = UInt (D:Qd);
6 m = UInt (M:Qm);
7 esize = 16 << UInt (fsi);
8 elements = 32 DIV esize;
9 toFixed = (op == '1');
10 unsigned = (U == '1');
11 fracBits = 64 - UInt (imm6);
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>
<dt>

See [C1.2.5 Standard assembler syntax fields](#).

This parameter must be one of the following values:

F16.S16	Encoded as	fsi = 0,	op = 0,	U = 0	Convert signed 16-bit integer to half-precision floating-point
F16.U16	Encoded as	fsi = 0,	op = 0,	U = 1	Convert unsigned 16-bit integer to half-precision floating-point
S16.F16	Encoded as	fsi = 0,	op = 1,	U = 0	Convert half-precision floating-point to signed 16-bit integer
U16.F16	Encoded as	fsi = 0,	op = 1,	U = 1	Convert half-precision floating-point to unsigned 16-bit integer
F32.S32	Encoded as	fsi = 1,	op = 0,	U = 0	Convert signed 32-bit integer to single-precision floating-point
F32.U32	Encoded as	fsi = 1,	op = 0,	U = 1	Convert unsigned 32-bit integer to single-precision floating-point
S32.F32	Encoded as	fsi = 1,	op = 1,	U = 0	Convert single-precision floating-point to signed 32-bit integer
U32.F32	Encoded as	fsi = 1,	op = 1,	U = 1	Convert single-precision floating-point to unsigned 32-bit integer

Convert single-precision floating-point to unsigned 32-bit integer

<Qd> Destination vector register.
<Qm> Source vector register.
<fbits> The number of fraction bits in the fixed-point number. For 16-bit fixed-point, this number must be in the range 1-16. For 32-bit fixed-point, this number must be in the range 1-32. The value of (64 - <fbits>) is encoded in imm6.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[m, curBeat];
8 for e = 0 to elements-1
9     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
10    pred = (elmtMask[e*(esize>>3)] == '0');
11    if toFixed then
12        // Round to zero
13        value = FPToFixed(Elem[op1, e, esize], esize, fracBits, unsigned, TRUE, FALSE, pred);
14    else
15        // Round nearest
16        value = FixedToFP(Elem[op1, e, esize], esize, fracBits, unsigned, TRUE, FALSE, pred);
17    Elem[result, e, esize] = value[esize-1:0];
18
19 for e = 0 to 3
20     if elmtMask[e] == '1' then
21         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.326 VCVT (between floating-point and integer)

Vector Convert between floating-point and integer. Convert between floating-point and integer values in elements of a vector register. When converting to integer the value is rounded towards zero, when converting to floating-point the value is rounded to nearest. For floating-point to integer operation, if the source value is outside the range of the target integer type, the result is saturated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	D	1	1	size	1	1	Qd	0	0	1	1	op	1	M	0	Qm	0							

T1: VCVT variant

VCVT<v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size IN {'11', '00'} then UNDEFINED;
4 d      = UInt (D:Qd);
5 m      = UInt (M:Qm);
6 toInteger = (op[1] == '1');
7 unsigned  = (op[0] == '1');
8 esize     = 8 << UInt (size);
9 elements  = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	This parameter must be one of the following values: F16.S16 Encoded as op = 00, size = 01 Convert signed 16-bit integer to half-precision floating-point F32.S32 Encoded as op = 00, size = 10 Convert signed 32-bit integer to single-precision floating-point F16.U16 Encoded as op = 01, size = 01 Convert unsigned 16-bit integer to half-precision floating-point F32.U32 Encoded as op = 01, size = 10 Convert unsigned 32-bit integer to single-precision floating-point S16.F16 Encoded as op = 10, size = 01 Convert half-precision floating-point to signed 16-bit integer S32.F32 Encoded as op = 10, size = 10 Convert single-precision floating-point to signed 32-bit integer U16.F16 Encoded as op = 11, size = 01 Convert half-precision floating-point to unsigned 16-bit integer U32.F32 Encoded as op = 11, size = 10 Convert single-precision floating-point to unsigned 32-bit integer
<Qd>	Destination vector register.
<Qm>	Source vector register.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1     = Q[m, curBeat];
8 for e = 0 to elements-1
9     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
10    pred = (elmtMask[e*(esize>>3)] == '0');
11    if toInteger then
12        // Round to zero
13        value = FPToFixed(Elm[op1, e, esize], esize, 0, unsigned, TRUE, FALSE, pred);
14    else
15        // Round to nearest
16        value = FixedToFP(Elm[op1, e, esize], esize, 0, unsigned, TRUE, FALSE, pred);
17    Elem[result, e, esize] = value[esize-1:0];
18
19 for e = 0 to 3
20     if elmtMask[e] == '1' then
21         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.327 VCVT (between single and half-precision floating-point)

Vector Convert between half-precision and single-precision. Convert between half-precision and single-precision floating-point values in elements of a vector register. For half-precision to single-precision operation, the top half (T variant) or bottom half (B variant) of the source vector register is selected. For single-precision to half-precision operation, the top half (T variant) or bottom half (B variant) of the destination vector register is selected and the other half retains its previous value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	op	1	1	1	0	0	D	1	1	1	1	1	1	Qd	T	1	1	1	0	0	0	M	0	Qm	1				

T1: VCVT variant

VCVT<T><v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' then UNDEFINED;
3 d = UInt (D:Qd);
4 m = UInt (M:Qm);
5 elements = 4;
6 halfToSingle = (op == '1');
7 esize = if halfToSingle then 16 else 32;
8 top = UInt (T);
9 if InITBlock () then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T>	Specifies that the FP16 value read from or written to the top (T) or bottom (B) half of the FP32 vector register element. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	This parameter must be one of the following values: F16.F32 Encoded as op = 0 Convert single-precision to half-precision F32.F16 Encoded as op = 1 Convert half-precision to single-precision
<Qd>	Destination vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
```

```

3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 opl    = Q[m, curBeat];
8 if halfToSingle then
9     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
10    pred = (elmtMask[top*(esize>>3)] == '0');
11    result = FPHalfToSingle(Elem[opl, top, 16], FALSE, pred);
12 else
13     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
14    pred = (elmtMask[0*(esize>>3)] == '0');
15    // Write to the selected half of the destination element
16    Elem[result, top, 16] = FPSingleToHalf(opl, FALSE, pred);
17    // Do not overwrite the other half
18    Elem[result, 1-top, 16] = Elem[Q[d, curBeat], 1-top, 16];
19
20 for e = 0 to 3
21     if elmtMask[e] == '1' then
22         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

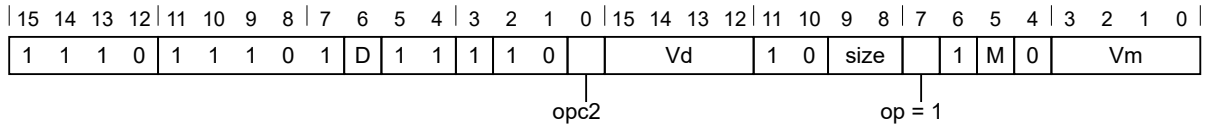
```

C2.4.328 VCVT (floating-point to integer)

Convert floating-point to integer with Round towards Zero. Convert floating-point to integer with Round towards Zero converts a value in a register from floating-point to a 32-bit integer, using the Round towards Zero rounding mode, and places the result in the destination register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.



Half-precision to unsigned single-precision variant

Armv8.1-M Floating-point Extension only.

Applies when **opc2 == 0** && **size == 01**.

VCVT{<c>}{<q>}.U32.F16 <Sd>, <Sm>

Half-precision to signed single-precision variant

Armv8.1-M Floating-point Extension only.

Applies when **opc2 == 1** && **size == 01**.

VCVT{<c>}{<q>}.S32.F16 <Sd>, <Sm>

Single-precision to unsigned single-precision variant

Armv8-M Floating-point Extension only.

Applies when **opc2 == 0** && **size == 10**.

VCVT{<c>}{<q>}.U32.F32 <Sd>, <Sm>

Single-precision to signed single-precision variant

Armv8-M Floating-point Extension only.

Applies when **opc2 == 1** && **size == 10**.

VCVT{<c>}{<q>}.S32.F32 <Sd>, <Sm>

Double-precision to unsigned single-precision variant

Armv8-M Floating-point Extension only.

Applies when **opc2 == 0** && **size == 11**.

VCVT{<c>}{<q>}.U32.F64 <Sd>, <Dm>

Double-precision to signed single-precision variant

Armv8-M Floating-point Extension only.

Applies when **opc2 == 1** && **size == 11**.

VCVT{<c>}{<q>}.S32.F64 <Sd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1') then UNDEFINED;
4 unsigned = (opc2 == '0'); round_zero = (op == '1');
5 d = UInt(Vd:D); m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
6 if size == '01' && InITBlock() then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Sd> Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
 <Sm> Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
 <Dm> Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     ExecuteFPCheck();
4     case size of
5         when '01'
6             S[d] = FPToFixed(S[m][15:0], 32, 0, unsigned, round_zero, TRUE);
7         when '10'
8             S[d] = FPToFixed(S[m], 32, 0, unsigned, round_zero, TRUE);
9         when '11'
10            S[d] = FPToFixed(D[m], 32, 0, unsigned, round_zero, TRUE);

```

C2.4.329 VCVT (from floating-point to integer)

Vector Convert from floating-point to integer. Convert each element of a vector from floating-point to integer using the specified rounding mode and place the results in a second vector. If a source element is outside the range of the target integer type, the result element is saturated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	1	1		Qd	0	0	0	RM	op	1	M	0		Qm	0				

T1: VCVT variant

VCVT<ANPM><v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size IN {'11', '00'} then UNDEFINED;
4 d      = UInt (D:Qd);
5 m      = UInt (M:Qm);
6 unsigned = (op == '1');
7 esize   = 8 << UInt (size);
8 elements = 32 DIV esize;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<ANPM>	The rounding mode. This parameter must be one of the following values: A Encoded as RM = 00 Round to nearest with ties to away N Encoded as RM = 01 Round to nearest with ties to even P Encoded as RM = 10 Round towards plus infinity M Encoded as RM = 11 Round towards minus infinity
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	This parameter must be one of the following values: S16.F16 Encoded as op = 0, size = 01 Convert half-precision floating-point to signed 16-bit integer S32.F32 Encoded as op = 0, size = 10 Convert single-precision floating-point to signed 32-bit integer U16.F16 Encoded as op = 1, size = 01 Convert half-precision floating-point to unsigned 16-bit integer U32.F32 Encoded as op = 1, size = 10 Convert single-precision floating-point to unsigned 32-bit integer
<Qd>	Destination vector register.
<Qm>	Source vector register.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1     = Q[m, curBeat];
8 for e = 0 to elements-1
9     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
10    pred = (elmtMask[e*(esize>>3)] == '0');
11    U     = unsigned;
12    Elem[result, e, esize] = FPToFixedDirected(Elem[op1, e, esize], 0, U, RM, FALSE, pred);
13
14 for e = 0 to 3
15     if elmtMask[e] == '1' then
16         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.330 VCVT (integer to floating-point)

Convert integer to floating-point. Convert integer to floating-point converts a value in a register from a 32-bit integer to floating-point, using the rounding mode specified by FPSCR, and places the result in the destination register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	1	0	0	0	Vd	1	0	size	op	1	M	0	Vm							

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VCVT{<c>}{<q>}.F16.<dt> <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VCVT{<c>}{<q>}.F32.<dt> <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VCVT{<c>}{<q>}.F64.<dt> <Dd>, <Sm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (D == '1') then UNDEFINED;
4 unsigned = (op == '0'); round_nearest = FALSE; // FALSE selects FPSCR rounding
5 m = UInt(Vm:M); d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
6 if size == '01' && InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Is the data type for the operand, encoded in the "op" field. It can have the following values:
 U32 when op = 0
 S32 when op = 1
 <Sd> Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
 <Dd> Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
 <Sm> Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     ExecuteFPCheck();
```

```
4      case size of
5          when '01'
6              fp16 = FixedToFP(S[m], 16, 0, unsigned, round_nearest, TRUE);
7              S[d] = Zeros(16):fp16;
8          when '10'
9              S[d] = FixedToFP(S[m], 32, 0, unsigned, round_nearest, TRUE);
10         when '11'
11             D[d] = FixedToFP(S[m], 64, 0, unsigned, round_nearest, TRUE);
```

C2.4.331 VCVTA

Convert floating-point to integer with Round to Nearest with Ties to Away. Convert floating-point to integer with Round to Nearest with Ties to Away converts a value in a register from floating-point to a 32-bit integer using the Round to Nearest with Ties to Away rounding mode, and places the result in the destination register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	1	1	RM = 00		Vd				1	0	size	op	1	M	0			Vm		

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VCVTA{<q>}.<dt>.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VCVTA{<q>}.<dt>.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VCVTA{<q>}.<dt>.F64 <Sd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1') then UNDEFINED;
4 if InitBlock() then UNPREDICTABLE;
5 unsigned = (op == '0');
6 round_mode = RM;
7 d = UInt(Vd:D);
8 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Is the data type for the elements of the destination, encoded in the "op" field. It can have the following values: U32 when op = 0 S32 when op = 1
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 EncodingSpecificOperations();

```

```
2  ExecuteFPCheck();
3
4  case size of
5      when '01'
6          S[d] = FPToFixedDirected(S[m][15:0], 0, unsigned, round_mode, TRUE);
7      when '10'
8          S[d] = FPToFixedDirected(S[m], 0, unsigned, round_mode, TRUE);
9      when '11'
10         S[d] = FPToFixedDirected(D[m], 0, unsigned, round_mode, TRUE);
```

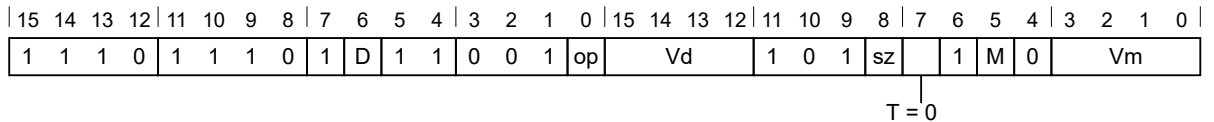
C2.4.332 VCVTB

Floating-point Convert Bottom. Floating-point Convert Bottom does one of the following:

- Converts the half-precision value in the bottom half of a single-precision register to single-precision and writes the result to a single-precision register.
- Converts the value in a single-precision register to half-precision and writes the result into the bottom half of a single-precision register, preserving the other half of the target register.
- Converts the half-precision value in the bottom half of a single-precision register to double-precision and writes the result to a double-precision register, without intermediate rounding.
- Converts the value in the double-precision register to half-precision and writes the result into the bottom half of a single-precision register, preserving the other half of the target register, without intermediate rounding.

T1

Armv8-M Floating-point Extension only, $sz == 1$ UNDEFINED in single-precision only implementations.



Half-precision to single-precision variant

Armv8-M Floating-point Extension only.

Applies when $op == 0$ && $sz == 0$.

VCVTB{<c>}{<q>}.F32.F16 <Sd>, <Sm>

Single-precision to half-precision variant

Armv8-M Floating-point Extension only.

Applies when $op == 1$ && $sz == 0$.

VCVTB{<c>}{<q>}.F16.F32 <Sd>, <Sm>

Half-precision to double-precision variant

Armv8-M Floating-point Extension only.

Applies when $op == 0$ && $sz == 1$.

VCVTB{<c>}{<q>}.F64.F16 <Dd>, <Sm>

Double-precision to half-precision variant

Armv8-M Floating-point Extension only.

Applies when $op == 1$ && $sz == 1$.

VCVTB{<c>}{<q>}.F16.F64 <Sd>, <Dm>

Decode for this encoding

```

1 dp_operation = (sz == '1');
2 CheckDecodeFaults(if dp_operation then ExtType_DpFp else ExtType_SpFp);
3 if (VFPSmallRegisterBank() && dp_operation &&
4    ((M == '1' && op == '1') || (D == '1' && op == '0'))) then
5     UNDEFINED;
```



```

6 convert_from_half = (op == '0');
7 lowbit = if T == '1' then 16 else 0;
8 if dp_operation then
9     if convert_from_half then
10         d = UInt(D:Vd); m = UInt(Vm:M);
11     else
12         d = UInt(Vd:D); m = UInt(M:Vm);
13 else
14     d = UInt(Vd:D); m = UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     ExecuteFPCheck();
4
5     if convert_from_half then
6         if dp_operation then
7             D[d] = FPHalfToDouble(S[m][lowbit+15:lowbit], TRUE);
8         else
9             S[d] = FPHalfToSingle(S[m][lowbit+15:lowbit], TRUE);
10    else
11        if dp_operation then
12            S[d][lowbit+15:lowbit] = FPDDoubleToHalf(D[m], TRUE);
13        else
14            S[d][lowbit+15:lowbit] = FPSingleToHalf(S[m], TRUE);

```

C2.4.333 VCVTM

Convert floating-point to integer with Round towards -Infinity. Convert floating-point to integer with Round towards -Infinity converts a value in a register from floating-point to a 32-bit integer using the Round towards -Infinity rounding mode, and places the result in the destination register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	1	1	RM = 11		Vd				1	0	size	op	1	M	0				Vm	

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VCVTM{<q>}.<dt>.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VCVTM{<q>}.<dt>.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VCVTM{<q>}.<dt>.F64 <Sd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1') then UNDEFINED;
4 if InitBlock() then UNPREDICTABLE;
5 unsigned = (op == '0');
6 round_mode = RM;
7 d = UInt(Vd:D);
8 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Is the data type for the elements of the destination, encoded in the "op" field. It can have the following values: U32 when op = 0 S32 when op = 1
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 EncodingSpecificOperations();

```

```
2 ExecuteFPCheck();
3
4 case size of
5     when '01'
6         S[d] = FPToFixedDirected(S[m][15:0], 0, unsigned, round_mode, TRUE);
7     when '10'
8         S[d] = FPToFixedDirected(S[m], 0, unsigned, round_mode, TRUE);
9     when '11'
10        S[d] = FPToFixedDirected(D[m], 0, unsigned, round_mode, TRUE);
```

C2.4.334 VCVTN

Convert floating-point to integer with Round to Nearest. Convert floating-point to integer with Round to Nearest converts a value in a register from floating-point to a 32-bit integer using the Round to Nearest rounding mode, and places the result in the destination register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	1	1	RM = 0		Vd				1	0	size	op	1	M	0				Vm	

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VCVTN{<q>}.<dt>.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VCVTN{<q>}.<dt>.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VCVTN{<q>}.<dt>.F64 <Sd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1') then UNDEFINED;
4 if InitBlock() then UNPREDICTABLE;
5 unsigned = (op == '0');
6 round_mode = RM;
7 d = UInt(Vd:D);
8 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Is the data type for the elements of the destination, encoded in the "op" field. It can have the following values: U32 when op = 0 S32 when op = 1
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 EncodingSpecificOperations();

```

```
2 ExecuteFPCheck();
3
4 case size of
5     when '01'
6         S[d] = FPToFixedDirected(S[m][15:0], 0, unsigned, round_mode, TRUE);
7     when '10'
8         S[d] = FPToFixedDirected(S[m], 0, unsigned, round_mode, TRUE);
9     when '11'
10        S[d] = FPToFixedDirected(D[m], 0, unsigned, round_mode, TRUE);
```

C2.4.335 VCVTP

Convert floating-point to integer with Round towards +Infinity. Convert floating-point to integer with Round towards +Infinity converts a value in a register from floating-point to a 32-bit integer using the Round towards +Infinity rounding mode, and places the result in the destination register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	1	1	RM = 10		Vd				1	0	size	op	1	M	0				Vm	

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VCVTP{<q>}.<dt>.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VCVTP{<q>}.<dt>.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VCVTP{<q>}.<dt>.F64 <Sd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1') then UNDEFINED;
4 if InitBlock() then UNPREDICTABLE;
5 unsigned = (op == '0');
6 round_mode = RM;
7 d = UInt(Vd:D);
8 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Is the data type for the elements of the destination, encoded in the "op" field. It can have the following values: U32 when op = 0 S32 when op = 1
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 EncodingSpecificOperations();

```

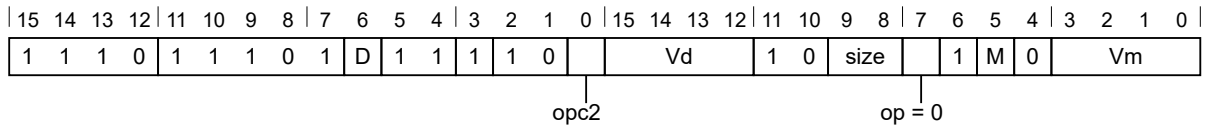
```
2 ExecuteFPCheck();
3
4 case size of
5     when '01'
6         S[d] = FPToFixedDirected(S[m][15:0], 0, unsigned, round_mode, TRUE);
7     when '10'
8         S[d] = FPToFixedDirected(S[m], 0, unsigned, round_mode, TRUE);
9     when '11'
10        S[d] = FPToFixedDirected(D[m], 0, unsigned, round_mode, TRUE);
```

C2.4.336 VCVTR

Convert floating-point to integer. Convert floating-point to integer converts a value in a register from floating-point to a 32-bit integer, using the rounding mode specified by FPSCR, and places the result in the destination register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.



Half-precision to unsigned single-precision variant

Armv8.1-M Floating-point Extension only.

Applies when **opc2 == 0** && **size == 01**.

VCVTR{<c>}{<q>}.U32.F16 <Sd>, <Sm>

Half-precision to signed single-precision variant

Armv8.1-M Floating-point Extension only.

Applies when **opc2 == 1** && **size == 01**.

VCVTR{<c>}{<q>}.S32.F16 <Sd>, <Sm>

Single-precision to unsigned single-precision variant

Armv8-M Floating-point Extension only.

Applies when **opc2 == 0** && **size == 10**.

VCVTR{<c>}{<q>}.U32.F32 <Sd>, <Sm>

Single-precision to signed single-precision variant

Armv8-M Floating-point Extension only.

Applies when **opc2 == 1** && **size == 10**.

VCVTR{<c>}{<q>}.S32.F32 <Sd>, <Sm>

Double-precision to unsigned single-precision variant

Armv8.1-M Floating-point Extension only.

Applies when **opc2 == 0** && **size == 11**.

VCVTR{<c>}{<q>}.U32.F64 <Sd>, <Dm>

Double-precision to signed single-precision variant

Armv8.1-M Floating-point Extension only.

Applies when **opc2 == 1** && **size == 11**.

VCVTR{<c>}{<q>}.S32.F64 <Sd>, <Dm>

Decode for this encoding


```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1') then UNDEFINED;
4 unsigned = (opc2 == '0'); round_zero = (op == '1');
5 d = UInt(Vd:D); m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
6 if size == '01' && InITBlock() then UNPREDICTABLE;

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   case size of
5     when '01'
6       S[d] = FPToFixed(S[m][15:0], 32, 0, unsigned, round_zero, TRUE);
7     when '10'
8       S[d] = FPToFixed(S[m], 32, 0, unsigned, round_zero, TRUE);
9     when '11'
10      S[d] = FPToFixed(D[m], 32, 0, unsigned, round_zero, TRUE);

```

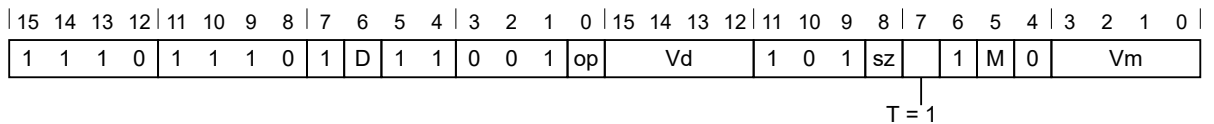
C2.4.337 VCVTT

Floating-point Convert Top. Floating-point Convert Top does one of the following:

- Converts the half-precision value in the top half of a single-precision register to single-precision and writes the result to a single-precision register.
- Converts the value in a single-precision register to half-precision and writes the result into the top half of a single-precision register, preserving the other half of the target register.
- Converts the half-precision value in the top half of a single-precision register to double-precision and writes the result to a double-precision register, without intermediate rounding.
- Converts the value in the double-precision register to half-precision and writes the result into the top half of a double-precision register, preserving the other half of the target register, without intermediate rounding.

T1

Armv8-M Floating-point Extension only, `sz == 1` UNDEFINED in single-precision only implementations.



Half-precision to single-precision variant

Armv8-M Floating-point Extension only.

Applies when `op == 0` && `sz == 0`.

`VCVTT{<c>}{<q>}.F32.F16 <Sd>, <Sm>`

Single-precision to half-precision variant

Armv8-M Floating-point Extension only.

Applies when `op == 1` && `sz == 0`.

`VCVTT{<c>}{<q>}.F16.F32 <Sd>, <Sm>`

Half-precision to double-precision variant

Armv8-M Floating-point Extension only.

Applies when `op == 0` && `sz == 1`.

`VCVTT{<c>}{<q>}.F64.F16 <Dd>, <Sm>`

Double-precision to half-precision variant

Armv8-M Floating-point Extension only.

Applies when `op == 1` && `sz == 1`.

`VCVTT{<c>}{<q>}.F16.F64 <Sd>, <Dm>`

Decode for this encoding

```

1 dp_operation = (sz == '1');
2 CheckDecodeFaults(if dp_operation then ExtType_DpFp else ExtType_SpFp);
3 if (VFPSmallRegisterBank() && dp_operation &&
4   ((M == '1' && op == '1') || (D == '1' && op == '0'))) then
5   UNDEFINED;
```

```

6 convert_from_half = (op == '0');
7 lowbit = if T == '1' then 16 else 0;
8 if dp_operation then
9     if convert_from_half then
10         d = UInt(D:Vd); m = UInt(Vm:M);
11     else
12         d = UInt(Vd:D); m = UInt(M:Vm);
13 else
14     d = UInt(Vd:D); m = UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     ExecuteFPCheck();
4
5     if convert_from_half then
6         if dp_operation then
7             D[d] = FPHalfToDouble(S[m][lowbit+15:lowbit], TRUE);
8         else
9             S[d] = FPHalfToSingle(S[m][lowbit+15:lowbit], TRUE);
10    else
11        if dp_operation then
12            S[d][lowbit+15:lowbit] = FPDDoubleToHalf(D[m], TRUE);
13        else
14            S[d][lowbit+15:lowbit] = FPSingleToHalf(S[m], TRUE);

```

C2.4.338 VCX1

Custom Extension Instruction Class 1. Custom extension register instruction class 1 computes a value based on an immediate, and optionally the destination value, and writes the result to the destination register. The source and destination registers are within the Floating-point register file, and require the current execution state to have access to these registers.

T1

Armv8-M Custom Datapath Extension with Armv8.1-M MVE or Armv8-M Floating-point Extension.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	A	1	1	0	sz	0	D	1	0	op1			Vd			0	coproc			op2	0	op3							

Single-register accumulator variant

Applies when **A == 1** and **sz == 0**

VCX1A <coproc>, <Sd>, #<imm>

Double-register accumulator variant

Applies when **A == 1** and **sz == 1**

VCX1A <coproc>, <Dd>, #<imm>

Single-register non-accumulator variant

Applies when **A == 0** && **sz == 0**

VCX1 <coproc>, <Sd>, #<imm>

Double-register non-accumulator variant

Applies when **A == 0** && **sz == 1**

VCX1 <coproc>, <Dd>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coproprocessor instructions";
2 cp = UInt(coproc);
3 CheckCDEDecodeFaults(cp, FALSE);
4 dp_operation = (sz == '1');
5 imm = op1:op2:op3;
6 acc = (A == '1');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 if VFPSmallRegisterBank() && dp_operation && D == '1' then UNDEFINED;
9 if InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `InITBlock()`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 A Encoded as A = 1
<coproc>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<Dd>	Is the 64-bit name of the floating-point source and destination register D0 – D15 encoded in the "D:Vd" fields.
<Sd>	Is the 32-bit name of the floating-point source and destination register S0 – S31 encoded in the "Vd:D" fields.
<imm>	Is the immediate encoded in "op1:op2:op3".

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3  if (!Coprocc_Accepted(cp, ThisInstr())) then
4      GenerateCoproccorException();
5  elseif dp_operation then
6      if acc then
7          D[d] = VCX_op1(ThisInstr(), D[d], 64);
8      else
9          D[d] = VCX_op0(ThisInstr(), 64);
10 else
11     if acc then
12         S[d] = VCX_op1(ThisInstr(), S[d], 32);
13     else
14         S[d] = VCX_op0(ThisInstr(), 32);

```

C2.4.339 VCX1 (vector)

Custom Extension Instruction Class 1 Vector. Custom extension register instruction class 1 vector computes a value based on an immediate, and optionally the destination value, and writes the result to the destination register. The source and destination registers are within the Floating-point and SIMD register file, and require the current execution state to have access to these registers.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8-M Custom Datapath Extension with Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	A	1	1	0	op1	0	D	1	0	op2				Vd				0	coproc				op3	1	op4					

Accumulator variant

Applies when **A** == 1

VCX1A<v> <coproc>, <Qd>, #<imm>

Non-accumulator variant

Applies when **A** == 0

VCX1<v> <coproc>, <Qd>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocessor instructions";
2 cp = UInt(coproc);
3 CheckCDEDecodeFaults(cp, TRUE);
4 if D == '1' || Vd[0] == '1' then UNDEFINED;
5 imm = op1:op2:op3:op4;
6 acc = (A == '1');
7 d = UInt(D:Vd[3:1]);
8 if InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

A Accumulate with existing register contents. This parameter must be one of the following values:

- Encoded as **A** = 0
- Encoded as **A** = 1

<v> See [C1.2.5 Standard assembler syntax fields](#).

<coproc> Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.

<Qd> Is the source and destination vector register Q0 – Q7, encoded in the "D:Vd" fields as <Qd>*2.
<imm> Is the immediate encoded in "op1:op2:op3:op4".

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3 if (!Coprocc_Accepted(cp, ThisInstr())) then GenerateCoproccException();
4
5 // Get current beat number and predication mask
6 (curBeat, elmtMask) = GetCurInstrBeat();
7
8 result = Zeros(32);
9 if acc then
10     // If the accumulator variant is used, then the 32-bit value from the vector
11     // source-destination register is used as an input to the custom operation
12     result = VCX_op1(ThisInstr(), Q[d, curBeat], 32, TRUE, curBeat, elmtMask);
13 else
14     result = VCX_op0(ThisInstr(), 32, TRUE, curBeat, elmtMask);
15
16 for e = 0 to 3
17     // If the vector lane is not predicated
18     if elmtMask[e] == '1' then
19         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.340 VCX2

Custom Extension Instruction Class 2. Custom extension register instruction class 2 computes a value based on a source register, an immediate, and optionally the destination value, and writes the result to the destination register. The source and destination registers are within the Floating-point register file, and require the current execution state to have access to these registers.

T1

Armv8-M Custom Datapath Extension with Armv8.1-M MVE or Armv8-M Floating-point Extension.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	A	1	1	0	sz	0	D	1	1	op1			Vd			0	coproc			op2	0	M	op3	Vm					

Single-register accumulator variant

Applies when **A == 1** && **sz == 0**

VCX2A <coproc>, <Sd>, <Sm>, #<imm>

Double-register accumulator variant

Applies when **A == 1** && **sz == 1**

VCX2A <coproc>, <Dd>, <Dm>, #<imm>

Single-register non-accumulator variant

Applies when **A == 0** and **sz == 0**

VCX2 <coproc>, <Sd>, <Sm>, #<imm>

Double-register non-accumulator variant

Applies when **A == 0** and **sz == 1**

VCX2 <coproc>, <Dd>, <Dm>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coproprocessor instructions";
2 cp = UInt(coproc);
3 CheckCDEDecodeFaults(cp, FALSE);
4 dp_operation = (sz == '1');
5 imm = op1:op2:op3;
6 acc = (A == '1');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
9 if VFPSmallRegisterBank() && dp_operation && (D == '1' || M == '1') then UNDEFINED;
10 if InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 A Encoded as A = 1
<coproc>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<Dd>	Is the 64-bit name of the floating-point source and destination register D0 – D15 encoded in the "D:Vd" fields.
<Dm>	Is the 64-bit name of the floating-point source register D0 – D15, encoded in the "M:Vm" fields.
<Sd>	Is the 32-bit name of the floating-point source and destination register S0 – S31 encoded in the "Vd:D" fields.
<Sm>	Is the 32-bit name of the floating-point source register S0 – S31, encoded in the "Vm:M" fields
<imm>	Is the immediate encoded in "op1:op2:op3".

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3  if (!Coprocc_Accepted(cp, ThisInstr())) then
4      GenerateCoproccorException();
5  elseif dp_operation then
6      if acc then
7          D[d] = VCX_op2(ThisInstr(), D[d], D[m], 64);
8      else
9          D[d] = VCX_op1(ThisInstr(), D[m], 64);
10 else
11     if acc then
12         S[d] = VCX_op2(ThisInstr(), S[d], S[m], 32);
13     else
14         S[d] = VCX_op1(ThisInstr(), S[m], 32);

```

C2.4.341 VCX2 (vector)

Custom Extension Instruction Class 2 Vector. Custom extension register instruction class 2 vector computes a value based on a source register, an immediate, and optionally the destination value, and writes the result to the destination register. The source and destination registers are within the Floating-point and SIMD register file, and require the current execution state to have access to these registers.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8-M Custom Datapath Extension with Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	A	1	1	0	op1	0	D	1	1	op2				Vd				0	coproc				op3	1	M	op4	Vm			

Accumulator variant

Applies when **A** == 1

VCX2A<v> <coproc>, <Qd>, <Qm>, #<imm>

Non-accumulator variant

Applies when **A** == 0

VCX2<v> <coproc>, <Qd>, <Qm>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocessor instructions";
2 cp = UInt(coproc);
3 CheckCDEDecodeFaults(cp, TRUE);
4 if D == '1' || M == '1' || Vd[0] == '1' || Vm[0] == '1' then UNDEFINED;
5 imm = op1:op2:op3:op4;
6 acc = (A == '1');
7 d = UInt(D:Vd[3:1]);
8 m = UInt(M:Vm[3:1]);
9 if InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

A	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 A Encoded as A = 1
<v>	See C1.2.5 Standard assembler syntax fields .
<coproc>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.

<Qd>	Is the source and destination vector register Q0 – Q7, encoded in the "D:Vd" fields as <Qd>*2.
<Qm>	Is the source vector register Q0 – Q7, encoded in the "M:Vm" fields as <Qm>*2.
<imm>	Is the immediate encoded in "op1:op2:op3:op4".

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3  if (!CoproccAccepted(cp, ThisInstr())) then GenerateCoproccException();
4
5  // Get current beat number and predication mask
6  (curBeat, elmtMask) = GetCurInstrBeat();
7
8  result = Zeros(32);
9  if acc then
10     // If the accumulator variant is used, then the 32-bit value from the vector
11     // source-destination register is used as an input to the custom operation
12     result = VCX_op2(ThisInstr(), Q[d, curBeat], Q[m, curBeat], 32, TRUE,
13                     curBeat, elmtMask);
14 else
15     result = VCX_op1(ThisInstr(), Q[m, curBeat], 32, TRUE, curBeat, elmtMask);
16
17 for e = 0 to 3
18     // If the vector lane is not predicated
19     if elmtMask[e] == '1' then
20         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.342 VCX3

Custom Extension Instruction Class 3. Custom extension register instruction class 3 computes a value based on two source registers, an immediate, and optionally the destination value, and writes the result to the destination register. The source and destination registers are within the Floating-point register file, and require the current execution state to have access to these registers.

T1

Armv8-M Custom Datapath Extension with Armv8.1-M MVE or Armv8-M Floating-point Extension.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	A	1	1	0	sz	1	D	op1	Vn				Vd				0	coproc				N	0	M	op2	Vm			

Single-register accumulator variant

Applies when **A == 1** && **sz == 0**

VCX3A <coproc>, <Sd>, <Sn>, <Sm>, #<imm>

Double-register accumulator variant

Applies when **A == 1** && **sz == 1**

VCX3A <coproc>, <Dd>, <Dn>, <Dm>, #<imm>

Single-register non-accumulator variant

Applies when **A == 0** and **sz == 0**

VCX3 <coproc>, <Sd>, <Sn>, <Sm>, #<imm>

Double-register non-accumulator variant

Applies when **A == 0** and **sz == 1**

VCX3 <coproc>, <Dd>, <Dn>, <Dm>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocesor instructions";
2 cp = UInt(coproc);
3 CheckCDEDecodeFaults(cp, FALSE);
4 dp_operation = (sz == '1');
5 imm = op1:op2;
6 acc = (A == '1');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
10 if VFPSmallRegisterBank() && dp_operation && (D == '1' || M == '1' || N == '1') then
11     UNDEFINED;
12 if InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `InITBlock()`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 A Encoded as A = 1
<coproc>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<Dd>	Is the 64-bit name of the floating-point source and destination register D0 – D15 encoded in the "D:Vd" fields.
<Dm>	Is the 64-bit name of the floating-point source register D0 – D15, encoded in the "M:Vm" fields.
<Dn>	Is the 64-bit name of the floating-point source register D0 – D15, encoded in the "N:Vn" fields.
<Sd>	Is the 32-bit name of the floating-point source and destination register S0 – S31 encoded in the "Vd:D" fields.
<Sm>	Is the 32-bit name of the floating-point source register S0 – S31, encoded in the "Vm:M" fields.
<Sn>	Is the 32-bit name of the floating-point source register S0 – S31, encoded in the "Vn:N" fields.
<imm>	Is the immediate encoded in "op1:op2".

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3  if (!CoproccAccepted(cp, ThisInstr())) then
4      GenerateCoproccException();
5  elseif dp_operation then
6      if acc then
7          D[d] = VCX_op3(ThisInstr(), D[d], D[n], D[m], 64);
8      else
9          D[d] = VCX_op2(ThisInstr(), D[n], D[m], 64);
10 else
11     if acc then
12         S[d] = VCX_op3(ThisInstr(), S[d], S[n], S[m], 32);
13     else
14         S[d] = VCX_op2(ThisInstr(), S[n], S[m], 32);

```

C2.4.343 VCX3 (vector)

Custom Extension Instruction Class 3 Vector. Custom extension register instruction class 3 vector computes a value based on two source registers, an immediate, and optionally the destination value, and writes the result to the destination register. The source and destination registers are within the Floating-point and SIMD register file, and require the current execution state to have access to these registers.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8-M Custom Datapath Extension with Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	A	1	1	0	op1	1	D	op2	Vn			Vd			0	coproc			N	1	M	op3	Vm						

Accumulator variant

Applies when **A == 1**

VCX3A<v> <coproc>, <Qd>, <Qn>, <Qm>, #<imm>

Non-accumulator variant

Applies when **A == 0**

VCX3<v> <coproc>, <Qd>, <Qn>, <Qm>, #<imm>

Decode for this encoding

```

1 if CoprocType(UInt(coproc)) != CP_CDEv1 then SEE "Coprocessor instructions";
2 cp = UInt(coproc);
3 CheckCDEDecodeFaults(cp, TRUE);
4 if D == '1' || M == '1' || N == '1' || Vd[0] == '1' || Vm[0] == '1' || Vn[0] == '1' then
5     UNDEFINED;
6 imm = op1:op2:op3;
7 acc = (A == '1');
8 d = UInt(D:Vd[3:1]);
9 n = UInt(N:Vn[3:1]);
10 m = UInt(M:Vm[3:1]);
11 if InITBlock() then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If InITBlock(), then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as if it passes the Condition code check.
- The instruction executes as NOP.

Assembler symbols for all encodings

A Accumulate with existing register contents. This parameter must be one of the following values:

- Encoded as A = 0
- A Encoded as A = 1

<v> See [C1.2.5 Standard assembler syntax fields](#).

<coproc>	Is the name of the coprocessor, encoding in the "coproc" field. Valid names are p0 to p7 inclusive.
<Qm>	Is the source vector register Q0 – Q7, encoded in the "M:Vm" fields as <Qm>*2.
<Qd>	Is the source and destination vector register Q0 – Q7, encoded in the "D:Vd" fields as <Qd>*2.
<Qn>	Is the source vector register Q0 – Q7, encoded in the "N:Vn" fields as <Qn>*2.
<imm>	Is the immediate encoded in "op1:op2:op3".

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3  if (!CoproccAccepted(cp, ThisInstr())) then GenerateCoproccorException();
4
5  // Get current beat number and predication mask
6  (curBeat, elmtMask) = GetCurInstrBeat();
7
8  result = Zeros(32);
9  if acc then
10     // If the accumulator variant is used, then the 32-bit value from the vector
11     // source-destination register is used as an input to the custom operation
12     result = VCX_op3(ThisInstr(), Q[d, curBeat], Q[n, curBeat], Q[m, curBeat], 32,
13                     TRUE, curBeat, elmtMask);
14 else
15     result = VCX_op2(ThisInstr(), Q[n, curBeat], Q[m, curBeat], 32,
16                     TRUE, curBeat, elmtMask);
17
18 for e = 0 to 3
19     // If the vector lane is not predicated
20     if elmtMask[e] == '1' then
21         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.344 VDDUP, VDWDUP

Vector Decrement and Duplicate, Vector Decrement with Wrap and Duplicate. Creates a vector with elements of successively decrementing values, starting at an offset specified by Rn. The value is decremented by the specified immediate value, which can take the following values: 1, 2, 4, 8. For all variants, the updated start offset is written back to Rn. For the wrapping variant, the operation wraps so that the values written to the vector register elements are in the range [0, Rm). However, if Rn and Rm are not a multiple of imm, or if Rn >= Rm, the operation is CONSTRAINED UNPREDICTABLE, with the resulting values of Rn and Qd UNKNOWN.

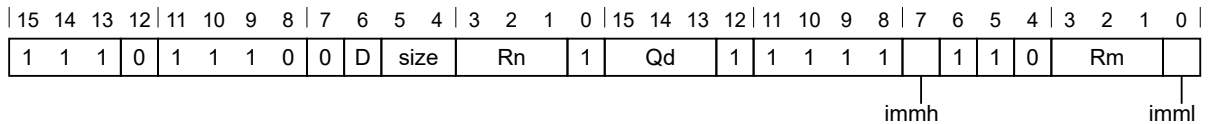
This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE



T1: VDWDUP variant

VDWDUP<v><q>.<dt> Qd, Rn, Rm, #<imm>

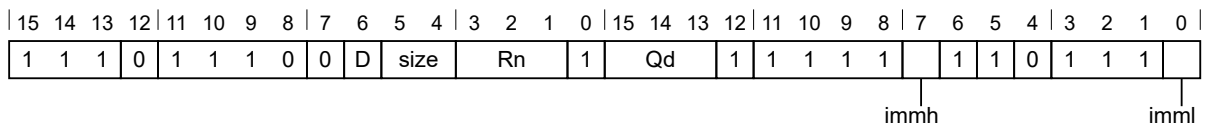
Decode for this encoding

```

1 if Rm == '111' then SEE "VDDUP";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 if D == '1' then UNDEFINED;
5 d = UInt(D:Qd);
6 m = UInt(Rm:'1');
7 n = UInt(Rn:'0');
8 wrap = TRUE;
9 imm2i = 1 << UInt(immh:imml);
10 esize = 8 << UInt(size);
11 elements = 32 DIV esize;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
13 if Rm == '110' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE



T2: VDDUP variant

VDDUP<v><q>.<dt> Qd, Rn, #<imm>

Decode for this encoding


```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 n      = UInt(Rn:'0');
6 m      = integer UNKNOWN;
7 wrap   = FALSE;
8 imm2i  = 1 << UInt(immh:imm1);
9 esize  = 8 << UInt(size);
10 elements = 32 DIV esize;
11 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<Rn> Current offset to start writing into Qd. Must be a multiple of imm. This must be an even numbered register.

Assembler symbols for T2 encodings

<Rn> Current offset to start writing into Qd. This must be an even numbered register.

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 U8 Encoded as size = 00
 U16 Encoded as size = 01
 U32 Encoded as size = 10
 <Qd> Destination vector register.
 <Rm> Size of the range. Must be a multiple of imm. This must be an odd numbered register.
 <imm> The increment between successive element values.
 This parameter must be one of the following values:
 #1 Encoded as immh = 0, imm1 = 0
 #2 Encoded as immh = 0, imm1 = 1
 #4 Encoded as immh = 1, imm1 = 0
 #8 Encoded as immh = 1, imm1 = 1

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result      = Zeros(32);
7 curOffset = UInt(R[n]);
8 if wrap then
9     bufSize = UInt(R[m]);
10     if bufSize MOD imm2i != 0 then CONSTRAINED_UNPREDICTABLE;
11     if curOffset MOD imm2i != 0 then CONSTRAINED_UNPREDICTABLE;
12     if curOffset >= bufSize then CONSTRAINED_UNPREDICTABLE;
13 for e = 0 to elements - 1
14     Elem[result, e, esize] = curOffset[esize-1:0];
15     if wrap && curOffset == 0 then
16         curOffset = bufSize - imm2i;
17     else
18         curOffset = curOffset - imm2i;
19 R[n] = curOffset[31:0];
20
21 for e = 0 to 3

```

```
22     if elmtMask[e] == '1' then  
23         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.345 VDIV

Floating-point Divide. Floating-point Divide divides one floating-point value by another floating-point value and writes the result to a third floating-point register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	0	0	Vn			Vd			1	0	size		N	0	M	0	Vm					

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VDIV{<c>}{<q>}.F16 {<Sd>, } <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VDIV{<c>}{<q>}.F32 {<Sd>, } <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VDIV{<c>}{<q>}.F64 {<Dd>, } <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
7 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
8 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();

```

```
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              S[d] = Zeros(16) : FPDIV(S[n][15:0], S[m][15:0], TRUE);
7          when '10'
8              S[d] = FPDIV(S[n], S[m], TRUE);
9          when '11'
10         D[d] = FPDIV(D[n], D[m], TRUE);
```

C2.4.346 VDUP

Vector Duplicate. Set each element of a vector register to the value of a general-purpose register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	B	1	0	Qd	0	Rt	1	0	1	1	D	0	E	1	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)

T1: VDUP variant

VDUP<v><q>.<size> Qd, Rt

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if B:E == '11' then UNDEFINED;
3 if D == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 t = UInt(Rt);
6 case B:E of
7   when '00' esize = 32; elements = 1;
8   when '01' esize = 16; elements = 2;
9   when '10' esize = 8; elements = 4;
10  otherwise
11    UNDEFINED;
12 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
13 if Rt == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <size> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 32 Encoded as B = 0, E = 0
 16 Encoded as B = 0, E = 1
 8 Encoded as B = 1, E = 0
 <Qd> Destination vector register.
 <Rt> Source general-purpose register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 for e = 0 to elements-1
8   Elem[result, e, esize] = R[t][esize-1:0];
9
10 for e = 0 to 3
11   if elmtMask[e] == '1' then
12     Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.347 VEOR

Vector Bitwise Exclusive Or. Compute a bitwise EOR of a vector register with another vector register. The result is written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	0	D	0	0	Qn	0	Qd	0	0	0	0	1	N	1	M	1	Qm	0						

T1: VEOR variant

VEOR<v><q>{.<dt>} Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 n = UInt(N:Qn);
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	An optional data type. It is ignored by assemblers and does not affect the encoding. This can be one of the following: S8, S16, S32, U8, U16, U32, I8, I16, I32, F16, F32.
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Q[n, curBeat] EOR Q[m, curBeat];
7
8 for e = 0 to 3
9     if elmtMask[e] == '1' then
10         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.348 VFMA

Floating-point Fused Multiply Accumulate. Floating-point Fused Multiply Accumulate multiplies two registers, adds the product to the destination register, and places the result in the destination register. The result of the multiply is not rounded before the addition.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	0	Vn			Vd			1	0	size		N		M	0	Vm					
																								op = 0							

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VFMA{<c>}{<q>}.F16 <Sd>, <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VFMA{<c>}{<q>}.F32 <Sd>, <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VFMA{<c>}{<q>}.F64 <Dd>, <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 opl_neg = (op == '1');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              op16 = if op1_neg then FPNeg(S[n][15:0]) else S[n][15:0];
7              S[d] = Zeros(16) : FPMulAdd(S[d][15:0], op16, S[m][15:0], TRUE);
8          when '10'
9              op32 = if op1_neg then FPNeg(S[n]) else S[n];
10             S[d] = FPMulAdd(S[d], op32, S[m], TRUE);
11          when '11'
12             op64 = if op1_neg then FPNeg(D[n]) else D[n];
13             D[d] = FPMulAdd(D[d], op64, D[m], TRUE);
```

C2.4.349 VFMA (vector by scalar plus vector, floating-point)

Vector Fused Multiply Accumulate. Multiply each element of a vector register by a general-purpose register value to produce a vector of results. Each result is then added to its respective element in the destination register. The result of each multiply is not rounded before the addition.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	Da	1	1	Qn	1	Qda	0	1	1	1	0	N	1	0	0	Rm							

T1: VFMA variant

VFMA<v><q>.<dt> Qda, Qn, Rm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if Da == '1' || N == '1' then UNDEFINED;
3 da      = UInt (Da:Qda);
4 m       = UInt (Rm);
5 n       = UInt (N:Qn);
6 esize   = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
9 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the floating-point format used.
 This parameter must be one of the following values:
 F32 Encoded as sz = 0
 F16 Encoded as sz = 1
 <Qda> Accumulator vector register.
 <Qn> Source vector register.
 <Rm> Source general-purpose register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 element2 = R[m][esize-1:0];
9 op3     = Q[da, curBeat];
10 for e = 0 to elements-1
11     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
12     pred = (elmtMask[e*(esize>>3)] == '0');
13     element1 = Elem[op1, e, esize];
```

```
14     element3 = Elem[op3, e, esize];
15     Elem[result, e, esize] = FPMulAdd(element3, element1, element2, FALSE, pred);
16
17 for e = 0 to 3
18     if elmtMask[e] == '1' then
19         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.350 VFMA, VFMS (floating-point)

Vector Fused Multiply Accumulate, Vector Fused Multiply Subtract. Multiply each element of the first source vector register by its respective element in the second vector register. Each result is then added to or subtracted from its respective element in the destination register. The result of each multiply is not rounded before the addition or subtraction.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	Da	0	sz	Qn	0	Qda	0	1	1	0	0	N	1	M	1	Qm	0						

T1: VFMA variant

VFMA<v><q>.<dt> Qda, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if Da == '1' || M == '1' || N == '1' then UNDEFINED;
3 da      = UInt (Da:Qda);
4 m       = UInt (M:Qm);
5 n       = UInt (N:Qn);
6 esize   = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 add     = TRUE;
9 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	Da	1	sz	Qn	0	Qda	0	1	1	0	0	N	1	M	1	Qm	0						

T2: VFMS variant

VFMS<v><q>.<dt> Qda, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if Da == '1' || M == '1' || N == '1' then UNDEFINED;
3 da      = UInt (Da:Qda);
4 m       = UInt (M:Qm);
5 n       = UInt (N:Qn);
6 esize   = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 add     = FALSE;
9 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F32 Encoded as sz = 0 F16 Encoded as sz = 1
<Qda>	Source and destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1    = Q[n, curBeat];
8  op2    = Q[m, curBeat];
9  op3    = Q[da, curBeat];
10 for e = 0 to elements-1
11     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
12     pred    = (elmtMask[e*(esize>>3)] == '0');
13     element1 = Elem[op1, e, esize];
14     element2 = Elem[op2, e, esize];
15     element3 = Elem[op3, e, esize];
16     if !add then
17         element1 = FPNeg(element1);
18     Elem[result, e, esize] = FPMulAdd(element3, element1, element2, FALSE, pred);
19
20 for e = 0 to 3
21     if elmtMask[e] == '1' then
22         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.351 VFMA (vector by vector plus scalar, floating-point)

Vector Fused Multiply Accumulate Scalar. Multiply each element in the source vector by the respective element from the destination vector and add to a scalar value. The resulting values are stored in the destination vector register. The result of each multiply is not rounded before the addition.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	Da	1	1	Qn	1	Qda	1	1	1	1	0	N	1	0	0							Rm	

T1: VFMA variant

VFMA<v><q>.<dt> Qda, Qn, Rm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if Da == '1' || N == '1' then UNDEFINED;
3 da      = UInt (Da:Qda);
4 m       = UInt (Rm);
5 n       = UInt (N:Qn);
6 esize   = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 add     = TRUE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> Size: indicates the floating-point format used.
 This parameter must be one of the following values:
 F32 Encoded as sz = 0
 F16 Encoded as sz = 1
 <Qda> Source and destination vector register.
 <Qn> Source vector register.
 <Rm> Source general-purpose register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 op2    = Q[da, curBeat];
9 element3 = R[m][esize-1:0];
10 for e = 0 to elements-1
11     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
12     pred = (elmtMask[e*(esize>>3)] == '0');
```

```
13     element1 = Elem[op1, e, esize];
14     element2 = Elem[op2, e, esize];
15     if !add then
16         element1 = FPNeg(element1);
17     Elem[result, e, esize] = FPMulAdd(element3, element1, element2, FALSE, pred);
18
19 for e = 0 to 3
20     if elmtMask[e] == '1' then
21         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];
```


C2.4.352 VFMS

Floating-point Fused Multiply Subtract. Floating-point Fused Multiply Subtract negates one register and multiplies it with another register, adds the product to the destination register, and places the result in the destination register. The result of the multiply is not rounded before the addition.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15 14 13 12				11 10 9 8				7 6 5 4				3 2 1 0				15 14 13 12				11 10 9 8				7 6 5 4				3 2 1 0															
1 1 1 0				1 1 1 0 1				D 1 0				Vn				Vd				1 0				size				N				M				0				Vm			

op = 1

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when `size == 01`.

VFMS {<c>} {<q>}.F16 <Sd>, <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when `size == 10`.

VFMS {<c>} {<q>} .F32 <Sd>, <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when `size == 11`.

VFMS {<c>} {<q>} .F64 <Dd>, <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InitTBlock() then UNPREDICTABLE;
6 opl_neg = (op == '1');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   case size of
5     when '01'
6       op16 = if op1_neg then FPNeg(S[n][15:0]) else S[n][15:0];
7       S[d] = Zeros(16) : FPMulAdd(S[d][15:0], op16, S[m][15:0], TRUE);
8     when '10'
9       op32 = if op1_neg then FPNeg(S[n]) else S[n];
10      S[d] = FPMulAdd(S[d], op32, S[m], TRUE);
11    when '11'
12      op64 = if op1_neg then FPNeg(D[n]) else D[n];
13      D[d] = FPMulAdd(D[d], op64, D[m], TRUE);
```

C2.4.353 VFNMA

Floating-point Fused Negate Multiply Accumulate. Floating-point Fused Negate Multiply Accumulate negates one floating-point register value and multiplies it by another floating-point register value, adds the negation of the floating-point value in the destination register to the product, and writes the result back to the destination register. The result of the multiply is not rounded before the addition.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	0	1	Vn			Vd			1	0	size		N	M		0	Vm					
																								op = 1							

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VFNMA{<c>}{<q>}.F16 <Sd>, <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VFNMA{<c>}{<q>}.F32 <Sd>, <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VFNMA{<c>}{<q>}.F64 <Dd>, <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 opl_neg = (op == '1');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     ExecuteFPCheck();
4     case size of
5         when '01'
6             op16 = if op1_neg then FPNeg(S[n][15:0]) else S[n][15:0];
7             S[d] = Zeros(16) : FPMulAdd(FPNeg(S[d][15:0]), op16, S[m][15:0], TRUE);
8         when '10'
9             op32 = if op1_neg then FPNeg(S[n]) else S[n];
10            S[d] = FPMulAdd(FPNeg(S[d]), op32, S[m], TRUE);
11        when '11'
12            op64 = if op1_neg then FPNeg(D[n]) else D[n];
13            D[d] = FPMulAdd(FPNeg(D[d]), op64, D[m], TRUE);
```

C2.4.354 VFNMS

Floating-point Fused Negate Multiply Subtract. Floating-point Fused Negate Multiply Subtract multiplies together two floating-point register values, adds the negation of the floating-point value in the destination register to the product, and writes the result back to the destination register. The result of the multiply is not rounded before the addition.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	0	1	Vn			Vd			1	0	size		N	M		0	Vm					
																										op = 0					

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VFNMS{<c>}{<q>}.F16 <Sd>, <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VFNMS{<c>}{<q>}.F32 <Sd>, <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VFNMS{<c>}{<q>}.F64 <Dd>, <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 opl_neg = (op == '1');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              op16 = if op1_neg then FPNeg(S[n][15:0]) else S[n][15:0];
7              S[d] = Zeros(16) : FPMulAdd(FPNeg(S[d][15:0]), op16, S[m][15:0], TRUE);
8          when '10'
9              op32 = if op1_neg then FPNeg(S[n]) else S[n];
10             S[d] = FPMulAdd(FPNeg(S[d]), op32, S[m], TRUE);
11          when '11'
12             op64 = if op1_neg then FPNeg(D[n]) else D[n];
13             D[d] = FPMulAdd(FPNeg(D[d]), op64, D[m], TRUE);
```

C2.4.355 VHADD

Vector Halving Add. Add the value of the elements in the first source vector register to either the respective elements in the second source vector register or a general-purpose register. The result is halved before being written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn		0		Qd		0	0	0	0	0	0	N	1	M	0		Qm	0	

T1: VHADD variant

VHADD<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (M:Qm);
6 n = UInt (N:Qn);
7 esize = 8 << UInt (size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 withScalar = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	D	size		Qn		0		Qd		0	1	1	1	1	N	1	0	0			Rm		

T2: VHADD variant

VHADD<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_Mve);
3 if D == '1' || N == '1' then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (Rm);
6 n = UInt (N:Qn);
7 esize = 8 << UInt (size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 withScalar = TRUE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
12 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .																								
<dt>	This parameter determines the following values: – Size: indicates the size of the elements in the vector. – Unsigned flag: S indicates signed, U indicates unsigned. This parameter must be one of the following values: <table><tr><td>S8</td><td>Encoded as</td><td>size = 00,</td><td>U = 0</td></tr><tr><td>U8</td><td>Encoded as</td><td>size = 00,</td><td>U = 1</td></tr><tr><td>S16</td><td>Encoded as</td><td>size = 01,</td><td>U = 0</td></tr><tr><td>U16</td><td>Encoded as</td><td>size = 01,</td><td>U = 1</td></tr><tr><td>S32</td><td>Encoded as</td><td>size = 10,</td><td>U = 0</td></tr><tr><td>U32</td><td>Encoded as</td><td>size = 10,</td><td>U = 1</td></tr></table>	S8	Encoded as	size = 00,	U = 0	U8	Encoded as	size = 00,	U = 1	S16	Encoded as	size = 01,	U = 0	U16	Encoded as	size = 01,	U = 1	S32	Encoded as	size = 10,	U = 0	U32	Encoded as	size = 10,	U = 1
S8	Encoded as	size = 00,	U = 0																						
U8	Encoded as	size = 00,	U = 1																						
S16	Encoded as	size = 01,	U = 0																						
U16	Encoded as	size = 01,	U = 1																						
S32	Encoded as	size = 10,	U = 0																						
U32	Encoded as	size = 10,	U = 1																						
<Qd>	Destination vector register.																								
<Qn>	First source vector register.																								
<Qm>	Second source vector register.																								
<Rm>	Source general-purpose register.																								

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1    = Q[n, curBeat];
8  if !withScalar then
9      op2 = Q[m, curBeat];
10 for e = 0 to elements-1
11     if withScalar then
12         value = Int(Elem[op1, e, esize], unsigned) + Int(R[m][esize-1:0], unsigned);
13     else
14         value = Int(Elem[op1, e, esize], unsigned) + Int(Elem[op2, e, esize], unsigned);
15     Elem[result, e, esize] = value[esize:1];
16
17 for e = 0 to 3
18     if elmtMask[e] == '1' then
19         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.356 VHCADD

Vector Halving Complex Add with Rotate. This instruction performs a complex addition of the first operand with the second operand rotated in the complex plane by the specified amount. A 90 degree rotation of this operand corresponds to a multiplication by a positive imaginary unit, while a 270 degree rotation corresponds to a multiplication by a negative imaginary unit. Even and odd elements of the source vectors are interpreted to be the real and imaginary components, respectively, of a complex number. The result is halved before being written to the destination register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	size		Qn	0	Qd	rot	1	1	1	1	N	0	M	0	Qm	0						

T1: VHCADD variant

VHCADD<v><q>.<dt> Qd, Qn, Qm, #<rotate>

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 esize  = 8 << UInt(size);
8 elements = 32 DIV esize;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if D:Qd == M:Qm && size == '10' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 S8 Encoded as size = 00
 S16 Encoded as size = 01
 S32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qn> First source vector register.
 <Qm> Second source vector register.
 <rotate> The rotation amount.
 This parameter must be one of the following values:
 #90 Encoded as rot = 0
 #270 Encoded as rot = 1

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
```

```

4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  // 32 bit operations are handled differently as they perform cross beat
8  // register accesses
9  if esize == 32 then
10     case rot:curBeat[0] of
11         when '00' result = (SInt(Q[n, curBeat]) - SInt(Q[m, curBeat+1]))[32:1];
12         when '01' result = (SInt(Q[n, curBeat]) + SInt(Q[m, curBeat-1]))[32:1];
13         when '10' result = (SInt(Q[n, curBeat]) + SInt(Q[m, curBeat+1]))[32:1];
14         when '11' result = (SInt(Q[n, curBeat]) - SInt(Q[m, curBeat-1]))[32:1];
15     else
16         op1 = Q[n, curBeat];
17         op2 = Q[m, curBeat];
18         for e = 0 to elements-1
19             case rot:e[0] of
20                 when '00' value = SInt(Elem[op1, e, esize]) - SInt(Elem[op2, e+1, esize]);
21                 when '01' value = SInt(Elem[op1, e, esize]) + SInt(Elem[op2, e-1, esize]);
22                 when '10' value = SInt(Elem[op1, e, esize]) + SInt(Elem[op2, e+1, esize]);
23                 when '11' value = SInt(Elem[op1, e, esize]) - SInt(Elem[op2, e-1, esize]);
24                 Elem[result, e, esize] = value[esize:1];
25
26 for e = 0 to 3
27     if elmtMask[e] == '1' then
28         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.357 VHSUB

Vector Halving Subtract. Subtract the value of the elements in the second source vector register from either the respective elements in the first source vector register or a general-purpose register. The result is halved before being written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn		0		Qd		0	0	0	1	0	N	1	M	0		Qm		0	

T1: VHSUB variant

VHSUB<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (M:Qm);
6 n = UInt (N:Qn);
7 esize = 8 << UInt (size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 withScalar = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	D	size		Qn		0		Qd		1	1	1	1	1	N	1	0	0		Rm			

T2: VHSUB variant

VHSUB<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_Mve);
3 if D == '1' || N == '1' then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (Rm);
6 n = UInt (N:Qn);
7 esize = 8 << UInt (size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 withScalar = TRUE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
12 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .																								
<dt>	This parameter determines the following values: – Size: indicates the size of the elements in the vector. – Unsigned flag: S indicates signed, U indicates unsigned. This parameter must be one of the following values: <table><tr><td>S8</td><td>Encoded as</td><td>size = 00,</td><td>U = 0</td></tr><tr><td>U8</td><td>Encoded as</td><td>size = 00,</td><td>U = 1</td></tr><tr><td>S16</td><td>Encoded as</td><td>size = 01,</td><td>U = 0</td></tr><tr><td>U16</td><td>Encoded as</td><td>size = 01,</td><td>U = 1</td></tr><tr><td>S32</td><td>Encoded as</td><td>size = 10,</td><td>U = 0</td></tr><tr><td>U32</td><td>Encoded as</td><td>size = 10,</td><td>U = 1</td></tr></table>	S8	Encoded as	size = 00,	U = 0	U8	Encoded as	size = 00,	U = 1	S16	Encoded as	size = 01,	U = 0	U16	Encoded as	size = 01,	U = 1	S32	Encoded as	size = 10,	U = 0	U32	Encoded as	size = 10,	U = 1
S8	Encoded as	size = 00,	U = 0																						
U8	Encoded as	size = 00,	U = 1																						
S16	Encoded as	size = 01,	U = 0																						
U16	Encoded as	size = 01,	U = 1																						
S32	Encoded as	size = 10,	U = 0																						
U32	Encoded as	size = 10,	U = 1																						
<Qd>	Destination vector register.																								
<Qn>	First source vector register.																								
<Qm>	Second source vector register.																								
<Rm>	Source general-purpose register.																								

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1    = Q[n, curBeat];
8  if !withScalar then
9      op2 = Q[m, curBeat];
10 for e = 0 to elements-1
11     if withScalar then
12         value = Int(Elem[op1, e, esize], unsigned) - Int(R[m][esize-1:0], unsigned);
13     else
14         value = Int(Elem[op1, e, esize], unsigned) - Int(Elem[op2, e, esize], unsigned);
15     Elem[result, e, esize] = value[esize:1];
16
17 for e = 0 to 3
18     if elmtMask[e] == '1' then
19         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.358 VIDUP, VIWDUP

Vector Increment and Duplicate, Vector Increment with Wrap and Duplicate. Creates a vector with elements of successively incrementing values, starting at an offset specified by Rn. The value is incremented by the specified immediate value, which can take the following values: 1, 2, 4, 8. For all variants, the updated start offset is written back to Rn. For the wrapping variant, the operation wraps so that the values written to the vector register elements are in the range [0, Rm). However, if Rn and Rm are not a multiple of imm, or if Rn >= Rm, the operation is CONSTRAINED UNPREDICTABLE, with the resulting values of Rn and Qd UNKNOWN.

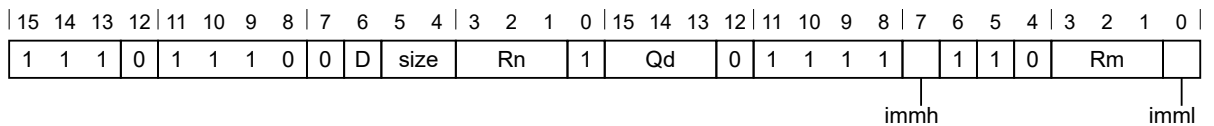
This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE



T1: VIWDUP variant

VIWDUP<v><q>.<dt> Qd, Rn, Rm, #<imm>

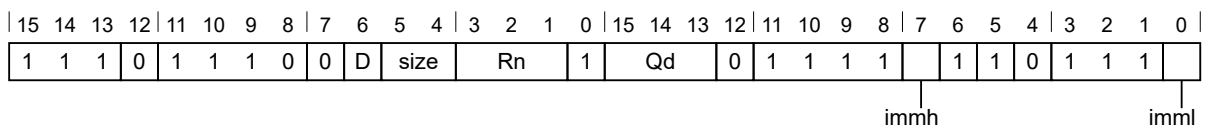
Decode for this encoding

```

1 if Rm == '111' then SEE "VIDUP";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 if D == '1' then UNDEFINED;
5 d = UInt(D:Qd);
6 m = UInt(Rm:'1');
7 n = UInt(Rn:'0');
8 wrap = TRUE;
9 imm2i = 1 << UInt(immh:imml);
10 esize = 8 << UInt(size);
11 elements = 32 DIV esize;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
13 if Rm == '110' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE



T2: VIDUP variant

VIDUP<v><q>.<dt> Qd, Rn, #<imm>

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 n      = UInt(Rn:'0');
6 m      = integer UNKNOWN;
7 wrap   = FALSE;
8 imm2i  = 1 << UInt(immh:imm1);
9 esize  = 8 << UInt(size);
10 elements = 32 DIV esize;
11 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<Rn> Current offset to start writing into Qd. Must be a multiple of imm. This must be an even numbered register.

Assembler symbols for T2 encodings

<Rn> Current offset to start writing into Qd. This must be an even numbered register.

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 U8 Encoded as size = 00
 U16 Encoded as size = 01
 U32 Encoded as size = 10
 <Qd> Destination vector register.
 <Rm> Size of the range. Must be a multiple of imm. This must be an odd numbered register.
 <imm> The increment between successive element values.
 This parameter must be one of the following values:
 #1 Encoded as immh = 0, imm1 = 0
 #2 Encoded as immh = 0, imm1 = 1
 #4 Encoded as immh = 1, imm1 = 0
 #8 Encoded as immh = 1, imm1 = 1

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result      = Zeros(32);
7 curOffset = UInt(R[n]);
8 if wrap then
9     bufSize = UInt(R[m]);
10     if bufSize MOD imm2i != 0 then CONSTRAINED_UNPREDICTABLE;
11     if curOffset MOD imm2i != 0 then CONSTRAINED_UNPREDICTABLE;
12     if curOffset >= bufSize then CONSTRAINED_UNPREDICTABLE;
13 for e = 0 to elements - 1
14     Elem[result, e, esize] = curOffset[esize-1:0];
15     curOffset = curOffset + imm2i;
16     if wrap && curOffset == bufSize then
17         curOffset = 0;
18 R[n] = curOffset[31:0];
19
20 for e = 0 to 3
21     if elmtMask[e] == '1' then

```

```
22      Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.359 VINS

Floating-point move Insertion. Floating-point move Insertion copies the lower 16 bits of the 32-bit source Floating-point Extension register into the upper 16 bits of the 32-bit destination Floating-point Extension register, while preserving the values in the remaining bits.

T1

Armv8.1-M Floating-point Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	0	0	0	0	Vd	1	0	1	0	1	1	M	0	Vm						

T1 variant

VINS<q>,F16 <Sd>, <Sm>

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_HpFp);
2 d = UInt(Vd:D); m = UInt(Vm:M);
3 if InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

- <q> See C1.2.5 Standard assembler syntax fields.
- <Sd> Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
- <Sm> Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3 S[d][31:16] = S[m][15:0];
```


C2.4.360 VLD2

Vector Deinterleaving Load - Stride 2. Loads two 64-bit contiguous blocks of data from memory and writes them to parts of 2 destination registers. The parts of the destination registers written to, and the offsets from the base address register, are determined by the pat parameter. If the instruction is executed 2 times with the same base address and destination registers, but with different pat values, the effect is to load data from memory and to deinterleave it into the specified registers with a stride of 2. The base address register can optionally be incremented by 32.

This instruction is subject to beat-wise execution.

This instruction is not VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	1	0	0	1	D	W	1	Rn				Qd				1	1	1	1	size		(0)	pat	(0)	(0)	(0)	(0)	0

T1: VLD2 variant (Non write-back: W=0)

VLD2<pat><q>.<size> {Qd, Qd+1}, [Rn]

T1: VLD2 variant (Writeback: W=1)

VLD2<pat><q>.<size> {Qd, Qd+1}, [Rn]!

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 n      = UInt(Rn);
6 pattern = UInt(pat);
7 esize  = 8 << UInt(size);
8 elements = 32 DIV esize;
9 wback  = (W == '1');
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
12 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
13 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;
14 if UInt(D:Qd) > 6 then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<pat>	Specifies the pattern of register elements and memory addresses to access. This parameter must be one of the following values: 0 Encoded as pat = 0 1 Encoded as pat = 1
<q>	See C1.2.5 Standard assembler syntax fields .
<size>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: 8 Encoded as size = 00 16 Encoded as size = 01 32 Encoded as size = 10
<Qd>	Destination vector register.
<Rn>	The base register for the target address.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, -) = GetCurInstrBeat();
5
6 // Pre-calculate variables for memory / register access patterns
7 addrWordOffset = curBeat[1] : (UInt(curBeat[1]) + pattern)[0] : curBeat[0];
8 baseAddress    = R[n] + ZeroExtend(addrWordOffset:'00', 32);
9 xBeat         = UInt(curBeat[1] : (pattern[0] EOR curBeat[1]));
10
11 for e = 0 to elements-1
12     address = baseAddress + (e * (esize DIV 8));
13     case esize of
14         when 8
15             y = UInt(e[0]);
16             xE = UInt(curBeat[0] : e[1]);
17         when 16
18             y = UInt(e[0]);
19             xE = UInt(curBeat[0]);
20         when 32
21             y = UInt(curBeat[0]);
22             xE = 0;
23     Elem[Q[d + y, xBeat], xE, esize] = MemA_MVE[address, esize DIV 8];
24
25 // The optional write back to the base register is only performed on the
26 // last beat of the instruction.
27 if wback && IsLastBeat() then
28     R[n] = R[n] + 32;

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.361 VLD4

Vector Deinterleaving Load - Stride 4. Loads two 64-bit contiguous blocks of data from memory and writes them to parts of 4 destination registers. The parts of the destination registers written to, and the offsets from the base address register, are determined by the pat parameter. If the instruction is executed 4 times with the same base address and destination registers, but with different pat values, the effect is to load data from memory and to deinterleave it into the specified registers with a stride of 4. The base address register can optionally be incremented by 64.

This instruction is subject to beat-wise execution.

This instruction is not VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	1	0	0	1	D	W	1	Rn				Qd				1	1	1	1	size		pat		(0)	(0)	(0)	(0)	1

T1: VLD4 variant (Non write-back: W=0)

VLD4<pat><q>.<size> {Qd, Qd+1, Qd+2, Qd+3}, [Rn]

T1: VLD4 variant (Writeback: W=1)

VLD4<pat><q>.<size> {Qd, Qd+1, Qd+2, Qd+3}, [Rn]!

Decode for this encoding

```

1  if size == '11' then SEE "Related encodings";
2  CheckDecodeFaults(ExtType_Mve);
3  if D == '1' then UNDEFINED;
4  d      = UInt(D:Qd);
5  n      = UInt(Rn);
6  pattern = UInt(pat);
7  esize  = 8 << UInt(size);
8  elements = 32 DIV esize;
9  wback  = (W == '1');
10 if InITBlock()          then CONSTRAINED_UNPREDICTABLE;
11 if InVPTBlock()         then CONSTRAINED_UNPREDICTABLE;
12 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
13 if Rn == '1111'         then CONSTRAINED_UNPREDICTABLE;
14 if UInt(D:Qd) > 4       then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<pat>	Specifies the pattern of register elements and memory addresses to access. This parameter must be one of the following values:
	0 Encoded as pat = 00
	1 Encoded as pat = 01
	2 Encoded as pat = 10
	3 Encoded as pat = 11
<q>	See C1.2.5 Standard assembler syntax fields .
<size>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values:
	8 Encoded as size = 00
	16 Encoded as size = 01
	32 Encoded as size = 10

<Qd> Destination vector register.
<Rn> The base register for the target address.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, -) = GetCurInstrBeat();
5
6 // Pre-calculate variables for memory / register access patterns
7 addrWordOffset = curBeat[1] : (UInt(curBeat[1]) + pattern)[1:0] : curBeat[0];
8 baseAddress    = R[n] + ZeroExtend(addrWordOffset:'00', 32);
9 xBeat         = UInt(curBeat[1] : (pattern[1] EOR (pattern[0] AND curBeat[1])));
10
11 for e = 0 to elements-1
12     address = baseAddress + (e * (esize DIV 8));
13     case esize of
14         when 8
15             y = UInt(e[1:0]);
16             xE = UInt((pattern[0] EOR curBeat[1]) : curBeat[0]);
17         when 16
18             y = UInt(curBeat[0] : e[0]);
19             xE = UInt(pattern[0] EOR curBeat[1]);
20         when 32
21             y = UInt((pattern[0] EOR curBeat[1]) : curBeat[0]);
22             xE = 0;
23     Elem[Q[d + y, xBeat], xE, esize] = MemA_MVE[address, esize DIV 8];
24
25 // The optional write back to the base register is only performed on the
26 // last beat of the instruction.
27 if wback && IsLastBeat() then
28     R[n] = R[n] + 64;

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.362 VLDM

Floating-point Load Multiple. Floating-point Load Multiple loads multiple extension registers from consecutive memory locations using an address from a general-purpose register.

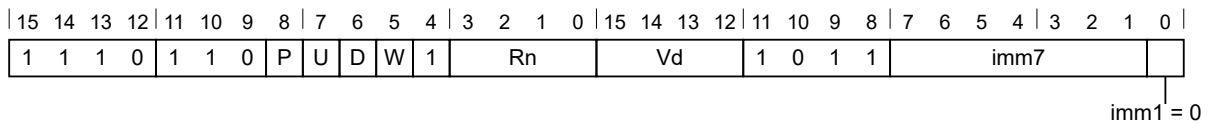
This instruction is subject to stack limit checking.

This instruction is used by the alias [VPOP](#).

This instruction is interrupt-continuable.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE



Decrement Before variant

Applies when **P == 1 && U == 0 && W == 1**.

`VLDMDB{<c>}{<q>}{.<size>} <Rn>!, <dreglist>`

Increment After variant

Applies when **P == 0 && U == 1 && Rn != 1111**.

`VLDM{<c>}{<q>}{.<size>} <Rn>{!}, <dreglist>`

`VLDMIA{<c>}{<q>}{.<size>} <Rn>{!}, <dreglist>`

Decode for this encoding

```

1 if P == '0' && U == '0' && W == '0' then SEE "Related encodings";
2 if P == '0' && U == '1' && W == '0' && Rn == '1111' then SEE "VSCCLRM";
3 if P == '1' && W == '0' then SEE VLDR;
4 CheckDecodeFaults(ExtType_MveOrFp);
5 if P == U && W == '1' then UNDEFINED;
6 // Remaining combinations are PUW = 010 (IA without !), 011 (IA with !), 101 (DB with !)
7 single_regs = FALSE; add = (U == '1'); wback = (W == '1'); imm8 = imm7 : imm1;
8 d = UInt(D:Vd); n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
9 regs = UInt(imm8) DIV 2;
10 if n == 15 then UNPREDICTABLE;
11 if regs == 0 || (d+regs) > 32 then UNPREDICTABLE;
12 if !HasArchVersion(Armv8pl) then
13     if VFPSmallRegisterBank() && (d+regs) > 16 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `regs == 0`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as a [VLDM](#) with the same addressing mode but loads no registers.

CONSTRAINED UNPREDICTABLE behavior

If `(d+regs) > 32`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.

- The instruction executes as NOP.
- One or more of the floating-point registers are UNKNOWN. If the instruction specifies write-back, the base register becomes UNKNOWN. This behavior does not affect any general-purpose registers.

T2

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	P	U	D	W	1	Rn				Vd				1	0	1	0	imm8							

Decrement Before variant

Applies when **P == 1 && U == 0 && W == 1**.

VLDMDB{<c>}{<q>}{.<size>} <Rn>!, <sreglist>

Increment After variant

Applies when **P == 0 && U == 1 && Rn != 1111**.

VLDM{<c>}{<q>}{.<size>} <Rn>{!}, <sreglist>

VLDMIA{<c>}{<q>}{.<size>} <Rn>{!}, <sreglist>

Decode for this encoding

```

1 if P == '0' && U == '0' then SEE "Related encodings";
2 if P == '0' && U == '1' && W == '0' && Rn == '1111' then SEE "VSCCLRM";
3 if P == '1' && W == '0' then SEE VLDR;
4 CheckDecodeFaults(ExtType_MveOrFp);
5 if P == '1' && U == '1' && W == '1' then UNDEFINED;
6 // Remaining combinations are PUW = 010 (IA without !), 011 (IA with !), 101 (DB with !)
7 single_regs = TRUE; add = (U == '1'); wback = (W == '1');
8 d = UInt(Vd:D); n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
9 regs = UInt(imm8);
10 if n == 15 then UNPREDICTABLE;
11 if regs == 0 || (d+regs) > 32 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **regs == 0**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as a **VLDM** with the same addressing mode but loads no registers.

CONSTRAINED UNPREDICTABLE behavior

If **(d+regs) > 32**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- One or more of the floating-point registers are UNKNOWN. If the instruction specifies write-back, the base register becomes UNKNOWN. This behavior does not affect any general-purpose registers.

Alias conditions

Alias	preferred when
VPOP	P == '0' && U == '1' && W == '1' && Rn == '1101'

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<size>	An optional data size specifier. If present, it must be equal to the size in bits, 32 or 64, of the registers being transferred.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
!	Specifies base register write-back. Encoded in the "W" field as 1 if present, otherwise 0.
<sreglist>	Is the list of consecutively numbered 32-bit floating-point registers to be transferred. The first register in the list is encoded in "Vd:D", and "imm8" is set to the number of registers in the list. The list must contain at least one register.
<dreglist>	Is the list of consecutively numbered 64-bit floating-point registers to be transferred. The first register in the list is encoded in "D:D", and "imm8" is set to twice the number of registers in the list. The list must contain at least one register.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      address = if add then R[n]          else R[n]-imm32;
5      regval  = if add then R[n]+imm32 else R[n]-imm32;
6
7      // Determine if the stack pointer limit must be checked
8      if n == 13 && wback then
9          // If memory operation is not performed as a result of a stack limit violation,
10         // and the write-back of the SP itself does not raise a stack limit violation, it
11         // is "IMPLEMENTATION_DEFINED" whether a SPLIM exception is raised.
12         // Arm recommends that any instruction which discards a memory access as
13         // a result of a stack limit violation, and where the write-back of the SP itself
14         // does not raise a stack limit violation, generates an SPLIM exception.
15         if boolean IMPLEMENTATION_DEFINED "SPLIM exception on invalid memory access" then
16             if ViolatesSPLim(LookUpSP(), address) then
17                 if HaveMainExt() then
18                     UFSR.STKOF = '1';
19                     // If the Main Extension is not implemented the fault always escalates to
20                     // a HardFault
21                     excInfo = CreateException(UsageFault);
22                     HandleException(excInfo);
23             applylimit = TRUE;
24         else
25             applylimit = FALSE;
26
27         // Memory operation only performed if limit not violated
28         if !(applylimit && ViolatesSPLim(LookUpSP(), regval)) then
29             for r = 0 to regs-1
30                 if single_regs then
31                     if (d+r) < 32 || !VFPSmallRegisterBank() then
32                         S[d+r] = MemA[address, 4];
33                         address = address+4;
34                 else
35                     if (d+r) < 16 || !VFPSmallRegisterBank() then
36                         word1 = MemA[address, 4]; word2 = MemA[address+4, 4];
37                         // Combine the word-aligned words in the correct order for
38                         // current endianness.
39                         D[d+r] = if BigEndian(address, 8) then word1:word2 else word2:word1;
40                     elseif boolean UNKNOWN then
41                         - = MemA[address, 4]; - = MemA[address+4, 4];
42                         address = address+8;
43 
```

```
44 // If the stack pointer is being updated a fault will be raised if
45 // the limit is violated
46 if wback then RSPCheck[n] = regval;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.363 VLDR

Floating-point Load Register. Floating-point Load Register loads a Floating-point Extension register from memory, using an address from a general-purpose register, with an optional offset.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	1	U	D	0	1	Rn				Vd				1	0	1	1	imm8							

Literal variant

Applies when Rn == 1111.

```
VLDR{<c>}{<q>}{.64} <Dd>, <label>
VLDR{<c>}{<q>}{.64} <Dd>, [PC, #{+/-}<imm>]
```

Offset variant

Applies when Rn != 1111.

```
VLDR{<c>}{<q>}{.64} <Dd>, [<Rn> {, #{+/-}<imm>}]
```

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_MveOrFp);
2 if VFPSmallRegisterBank() && (D == '1') then UNDEFINED;
3 fp_size = 64; add = (U == '1'); imm32 = ZeroExtend(imm8:'00', 32);
4 d = UInt(D:Vd); n = UInt(Rn);
```

T2

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	1	U	D	0	1	Rn				Vd				1	0	1	0	imm8							

Literal variant

Applies when Rn == 1111.

```
VLDR{<c>}{<q>}{.32} <Sd>, <label>
VLDR{<c>}{<q>}{.32} <Sd>, [PC, #{+/-}<imm>]
```

Offset variant

Applies when Rn != 1111.

```
VLDR{<c>}{<q>}{.32} <Sd>, [<Rn> {, #{+/-}<imm>}]
```

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_MveOrFp);
2 fp_size = 32; add = (U == '1'); imm32 = ZeroExtend(imm8:'00', 32);
3 d = UInt(Vd:D); n = UInt(Rn);
```

T3

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	1	U	D	0	1	Rn				Vd				1	0	0	1	imm8							

Literal variant

Applies when **Rn == 1111**.

VLDL{<c>}{<q>}.16 <Sd>, <label>
VLDL{<c>}{<q>}.16 <Sd>, [PC, #{+/-}<imm>]

Offset variant

Applies when **Rn != 1111**.

VLDL{<c>}{<q>}.16 <Sd>, [<Rn> {, #{+/-}<imm>}]

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_MveOrFp);
2 fp_size = 16; add = (U == '1'); imm32 = ZeroExtend(imm8:'0', 32);
3 d = UInt(Vd:D); n = UInt(Rn);
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
.64	Optional data size specifiers.
<Dd>	The destination register for a doubleword load.
.32	Optional data size specifiers.
<Sd>	The destination register for a singleword load.
<label>	The label of the literal data item to be loaded. The assembler calculates the required value of the offset from the Align (PC, 4) value of the instruction to this label. Permitted values are multiples of 4 in the range -1020 to 1020. If the offset is zero or positive, imm32 is equal to the offset and add == TRUE. If the offset is negative, imm32 is equal to minus the offset and add == FALSE.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	The immediate offset used for forming the address. For the immediate forms of the syntax, <imm> can be omitted, in which case the #0 form of the instruction is assembled. Permitted values are multiples of 4 in the range 0 to 1020.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   base = if n == 15 then Align(PC,4) else R[n];
5   address = if add then (base + imm32) else (base - imm32);
6   case fp_size of
7     when 16
8       S[d] = Zeros(16) : MemA[address, 2];
9     when 32
10      S[d] = MemA[address, 4];
11    when 64
12      word1 = MemA[address, 4]; word2 = MemA[address+4, 4];
13      // Combine the word-aligned words in the correct order for current endianness.
14      D[d] = if BigEndian(address, 8) then word1:word2 else word2:word1;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

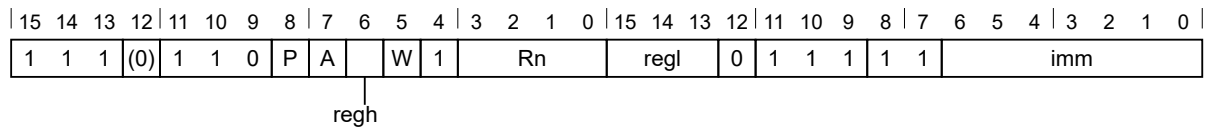
C2.4.364 VLDR (System Register)

Load System Register. Load a system register from memory. The target address is calculated from a base register plus an immediate offset. Access to the FPCXT payloads generates an UNDEFINED exception if the instruction is executed from Non-secure state. If CP10 is not enabled and either the Main extension is not implemented or the Floating-point context is active, access to FPCXT_NS will generate a NOCP UsageFault. Accesses to FPCXT_NS will not trigger lazy state preservation if there is no active Floating-point context. Accesses to FPCXT_NS do not trigger Floating-point context creation regardless of the value of FPCCR.ASPEN. FPSCR_nzcvqc allows access to FPSCR condition and saturation flags. The VPR register can only be accessed from privileged mode. FPCXT_NS, enables saving and restoration of the Non-secure floating-point context. If neither the Floating-point extension nor MVE are implemented and Floating-point context is active then the current FPSCR value is accessed. FPCXT_S, enables saving and restoration of the Secure floating-point context.

This instruction is subject to stack limit checking.

T1

Armv8.1-M Floating-point Extension or Armv8.1-M MVE



T1: VLDR variant (Offset: P=1, W=0)

VLDR<c><q> <reg>, [Rn{, #+/-<imm>}]

T1: VLDR variant (Pre-indexed: P=1, W=1)

VLDR<c><q> <reg>, [Rn, #+/-<imm>]!

T1: VLDR variant (Post-indexed: P=0, W=1)

VLDR<c><q> <reg>, [Rn], #+/-<imm>

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 if !HasArchVersion(Armv8pl) then
3     UFSR.NOCP = '1';
4     HandleException(CreateException(UsageFault));
5 fpCxtAnyAccess = (regh:regl == '111x');
6 fpCxtNSAccess = (regh:regl == '1110');
7 fpInactive = !HaveMveOrFPExt() || (FPCCR_NS.ASPEN == '1' && CONTROL.FPCA == '0');
8 if fpCxtAnyAccess && !IsSecure() then UNDEFINED;
9 if fpCxtNSAccess then
10     if !HaveMainExt() || !fpInactive then
11         HandleException(CheckCPEnabled(10));
12 else
13     CheckDecodeFaults(ExtType_MveOrFp);
14 n = UInt(Rn);
15 index = (P == '1');
16 add = (A == '1');
17 wback = (W == '1');
18 r = regh:regl;
19 imm32 = ZeroExtend(imm:'00', 32);
20 if (regh:regl) IN {'10xx', '01xx', '0011', '0000'} then CONSTRAINED_UNPREDICTABLE;
21 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<reg>	The system register to access This parameter must be one of the following values: FPSCR Encoded as regh = 0, regl = 001 FPSCR_nzcvqc Encoded as regh = 0, regl = 010 VPR Encoded as regh = 1, regl = 100 P0 Encoded as regh = 1, regl = 101 FPCXT_NS Encoded as regh = 1, regl = 110 FPCXT_S Encoded as regh = 1, regl = 111
<Rn>	The base register for the target address.
<imm>	The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 4.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      if !fpCxtNSAccess then
4          ExecuteFPCheck();
5      elseif !fpInactive then
6          PreserveFPState();
7      SerializeVFP();
8      VFPExcBarrier();
9
10     offsetAddr = if add then (R[n] + imm32) else (R[n] - imm32);
11     address    = if index then offsetAddr else R[n];
12
13     // Determine if the stack pointer limit should be checked
14     if n == 13 && wback then
15         violatesLimit = ViolatesSPLim(LookUpSP(), offsetAddr);
16     else
17         violatesLimit = FALSE;
18     // Memory operation only performed if limit not violated
19     if !violatesLimit then
20         case r of
21             when '0001'
22                 FPSCR = MemA[address, 4];
23             when '0010'
24                 // Only update the N, Z, C, V, and QC flags
25                 FPSCR[31:27] = MemA[address, 4][31:27];
26             when '1100'
27                 if HaveMve() then
28                     if CurrentModeIsPrivileged() then
29                         VPR = MemA[address, 4];
30                     elseif boolean IMPLEMENTATION_DEFINED "No MVE VPR mem access" then
31                         - = MemA[address, 4];
32                 else
33                     UNPREDICTABLE;
34             when '1101'
35                 if HaveMve() then
36                     VPR.P0 = MemA[address, 4][15:0];
37                 else
38                     UNPREDICTABLE;
39             when '1110'
40                 if (HaveFPEExt() || HaveMve()) && !fpInactive then
41                     FPCXT_Type cxt = MemA[address, 4];
42                     CONTROL_S.SFPA = cxt.SFPA;
43                     FPSCR          = Zeros(4):cxt[27:0];
44             when '1111'
45                 FPCXT_Type cxt = MemA[address, 4];
46                 CONTROL_S.SFPA = cxt.SFPA;
47                 FPSCR          = Zeros(4):cxt[27:0];
48             otherwise
49                 UNPREDICTABLE;

```

```
50
51 // If the stack pointer is being updated a fault will be raised if
52 // the limit is violated
53 if wback then
54     RSPCheck[n] = offsetAddr;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.365 VLDRB, VLDRH, VLDRW

Vector Load Register. Load consecutive elements from memory into a destination vector register. Each element loaded will be the zero or sign-extended representation of the value in memory. In indexed mode, the target address is calculated from a base register offset by an immediate value. Otherwise, the base register address is used directly. The sum of the base register and the immediate value can optionally be written back to the base register. Predicated lanes are zeroed instead of retaining their previous values.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	0	P	A	0	W	1	0	Rn	Qd	0	1	1	1	size	imm											

T1: VLDRB variant (Offset: P=1, W=0)

VLDRB<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T1: VLDRB variant (Pre-indexed: P=1, W=1)

VLDRB<v><q>.<dt> Qd, [Rn, #+/-<imm>]!

T1: VLDRB variant (Post-indexed: P=0, W=1)

VLDRB<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 if size == '00' then UNDEFINED;
5 d = UInt(Qd);
6 n = UInt(Rn);
7 msize = 8;
8 mbytes = msize DIV 8;
9 esize = 8 << UInt(size);
10 elements = 32 DIV esize;
11 imm32 = ZeroExtend(imm, 32);
12 index = (P == '1');
13 add = (A == '1');
14 wback = (W == '1');
15 unsigned = (U == '1');
16 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	0	P	A	0	W	1	1	Rn	Qd	0	1	1	1	size	imm											

T2: VLDRH variant (Offset: P=1, W=0)

VLDHRH<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T2: VLDHRH variant (Pre-indexed: P=1, W=1)

VLDHRH<v><q>.<dt> Qd, [Rn, #+/-<imm>]!

T2: VLDHRH variant (Post-indexed: P=0, W=1)

VLDHRH<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```
1 if P == '0' && W == '0' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 if size == '0x' then UNDEFINED;
5 d = UInt(Qd);
6 n = UInt(Rn);
7 msize = 16;
8 mbytes = msize DIV 8;
9 esize = 8 << UInt(size);
10 elements = 32 DIV esize;
11 imm32 = ZeroExtend(imm:'0', 32);
12 index = (P == '1');
13 add = (A == '1');
14 wback = (W == '1');
15 unsigned = (U == '1');
16 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

T5

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	P	A	D	W	1	Rn				Qd				1	1	1	1	0	0	imm					

T5: VLDRB variant (Offset: P=1, W=0)

VLDRB<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T5: VLDRB variant (Pre-indexed: P=1, W=1)

VLDRB<v><q>.<dt> Qd, [Rn, #+/-<imm>]!

T5: VLDRB variant (Post-indexed: P=0, W=1)

VLDRB<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```
1 if P == '0' && W == '0' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 n = UInt(Rn);
6 msize = 8;
7 mbytes = msize DIV 8;
8 esize = msize;
9 elements = 32 DIV esize;
10 imm32 = ZeroExtend(imm, 32);
11 index = (P == '1');
12 add = (A == '1');
```



```

13 wback    = (W == '1');
14 unsigned = TRUE;
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
16 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
17 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;

```

T6

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	0	1	1	0	P	A	D	W	1	Rn				Qd				1	1	1	1	0	1	imm							

T6: VLDRH variant (Offset: P=1, W=0)

VLDRH<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T6: VLDRH variant (Pre-indexed: P=1, W=1)

VLDRH<v><q>.<dt> Qd, [Rn, #+/-<imm>]!

T6: VLDRH variant (Post-indexed: P=0, W=1)

VLDRH<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 n      = UInt(Rn);
6 msize  = 16;
7 mbytes = msize DIV 8;
8 esize  = msize;
9 elements = 32 DIV esize;
10 imm32  = ZeroExtend(imm:'0', 32);
11 index  = (P == '1');
12 add    = (A == '1');
13 wback  = (W == '1');
14 unsigned = TRUE;
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
16 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
17 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;

```

T7

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	1	1	0	1	1	0	P	A	D	W	1	Rn				Qd				1	1	1	1	1	0	imm							

T7: VLDRW variant (Offset: P=1, W=0)

VLDRW<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T7: VLDRW variant (Pre-indexed: P=1, W=1)

VLDRW<v><q>.<dt> Qd, [Rn, #+/-<imm>]!

T7: VLDRW variant (Post-indexed: P=0, W=1)

VLDRW<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 n      = UInt(Rn);
6 msize  = 32;
7 mbytes  = msize DIV 8;
8 esize   = msize;
9 elements = 32 DIV esize;
10 imm32   = ZeroExtend(imm:'00', 32);
11 index   = (P == '1');
12 add     = (A == '1');
13 wback   = (W == '1');
14 unsigned = TRUE;
15 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
16 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
17 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> This parameter determines the following values:
 – Unsigned flag: S indicates signed, U indicates unsigned.
 – Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

<imm> The signed immediate value that is added to base register to calculate the target address.

Assembler symbols for T2 encodings

<dt> This parameter determines the following values:
 – Unsigned flag: S indicates signed, U indicates unsigned.
 – Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

<imm> The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 2.

Assembler symbols for T5 encodings

<dt> Data size. This parameter is ".8".
 <imm> The signed immediate value that is added to base register to calculate the target address.

Assembler symbols for T6 encodings

<dt> Data size. This parameter is ".16".
 <imm> The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 2.

Assembler symbols for T7 encodings

<dt>	Data size. This parameter is ".32".
<imm>	The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 4.

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Destination vector register.
<Rn>	The base register for the target address.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result      = Zeros(32);
7  offsetAddr = if add then (R[n] + imm32) else (R[n] - imm32);
8  address    = if index then offsetAddr else R[n];
9  address    = address + (curBeat * mbytes * elements);
10
11 for e = 0 to elements-1
12     if elmtMask[e*(esize >> 3)] == '1' then
13         Elem[result, e, esize] = Extend(MemA_MVE[address + (e * mbytes)], mbytes, unsigned);
14
15 // The optional write back to the base register is only performed on the
16 // last beat of the instruction.
17 if wback && IsLastBeat() then
18     R[n] = offsetAddr;
19
20 Q[d, curBeat] = result;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.366 VLDRB, VLDRH, VLDRW, VLDRD (vector)

Vector Gather Load. Load a byte, halfword, word, or doubleword from memory at the address contained in either:

- a) A base register R[n] plus an offset contained in each element of Q[m], optionally shifted by the element size, or
- b) Each element of Q[m] plus an immediate offset. The base element can optionally be written back, irrespective of predication, with that value incremented by the immediate or by the immediate scaled by the memory element size.

Each element loaded will be the zero or sign-extended representation of the value in memory. The result is written back into the corresponding element in the destination vector register Q[d]. Predicated lanes are zeroed instead of retaining their previous values.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	0	0	1	D	0	1	Rn				Qd				0	1	1	1	size		0	M	0	Qm		os

T1: VLDRB variant

VLDRB<v><q>.<dt> Qd, [Rn, Qm]

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' || (U == '0' && size == '00') then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (M:Qm);
6 n = UInt (Rn);
7 msize = '00';
8 esize = 8 << UInt (size);
9 elements = 32 DIV esize;
10 mesize = 8;
11 add = TRUE;
12 useReg = TRUE;
13 scaleOffset = (os == '1');
14 unsigned = (U == '1');
15 wback = FALSE;
16 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
17 if D:Qd == M:Qm then CONSTRAINED_UNPREDICTABLE;
18 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;
19 if os == '1' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	0	0	1	D	0	1	Rn				Qd				0	1	1	1	size		0	M	1	Qm		os

T2: VLDRH variant

VLDRH<v><q>.<dt> Qd, [Rn, Qm{, UXTW #os}]

Decode for this encoding

```
1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' || size == '00' || (U == '0' && size == '01') then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (M:Qm);
6 n = UInt (Rn);
7 msize = '01';
8 esize = 8 << UInt (size);
9 elements = 32 DIV esize;
10 mesize = 16;
11 add = TRUE;
12 useReg = TRUE;
13 scaleOffset = (os == '1');
14 unsigned = (U == '1');
15 wback = FALSE;
16 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
17 if D:Qd == M:Qm then CONSTRAINED_UNPREDICTABLE;
18 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;
```

T3

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	1	U	1	1	0	0	1	D	0	1	Rn				Qd				0	1	1	1	size				1	M	0	Qm				os

T3: VLDRW variant

VLDRW<v><q>.<dt> Qd, [Rn, Qm{, UXTW #os}]

Decode for this encoding

```
1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if U == '0' || size != '10' then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (M:Qm);
6 n = UInt (Rn);
7 msize = '10';
8 esize = 8 << UInt (size);
9 elements = 32 DIV esize;
10 mesize = 32;
11 add = TRUE;
12 useReg = TRUE;
13 scaleOffset = (os == '1');
14 unsigned = (U == '1');
15 wback = FALSE;
16 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
17 if D:Qd == M:Qm then CONSTRAINED_UNPREDICTABLE;
18 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;
```

T4

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	0	0	1	D	0	1	Rn				Qd				0	1	1	1	size		1	M	1	Qm		os

T4: VLDRD variant

VLDRD<v><q>.<dt> Qd, [Rn, Qm{, UXTW #os}]

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if U == '0' || size != '11' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(Rn);
7 msize  = '11';
8 esize  = 8 << UInt(size);
9 elements = 32 DIV esize;
10 mesize = 64;
11 add    = TRUE;
12 useReg  = TRUE;
13 scaleOffset = (os == '1');
14 unsigned = (U == '1');
15 wback   = FALSE;
16 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
17 if D:Qd == M:Qm then CONSTRAINED_UNPREDICTABLE;
18 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;
```

T5

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	A	D	W	1		Qm	(0)	Qd	1	1	1	1	0	M										

T5: VLDRW variant (Non write-back: W=0)

VLDRW<v><q>.<dt> Qd, [Qm{, #+/-<imm>}]

T5: VLDRW variant (Write-back: W=1)

VLDRW<v><q>.<dt> Qd, [Qm{, #+/-<imm>}]!

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d      = UInt(D:Qd);
4 m      = UInt(M:Qm);
5 n      = integer UNKNOWN;
6 size   = '10';
7 msize  = size;
8 offset = ZeroExtend(imm:Zeros(UInt(size)), 32);
9 esize  = 8 << UInt(size);
10 elements = 32 DIV esize;
11 mesize = 32;
12 add    = (A == '1');
13 useReg  = FALSE;
14 scaleOffset = FALSE;
15 unsigned = TRUE;
16 wback   = (W == '1');
17 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
18 if D:Qd == M:Qm then CONSTRAINED_UNPREDICTABLE;
```

T6

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	A	D	W	1	Qm	(0)	Qd	1	1	1	1	1	M	imm										

T6: VLDRD variant (Non write-back: W=0)

VLDRD<v><q>.<dt> Qd, [Qm{, #+/-<imm>}]

T6: VLDRD variant (Write-back: W=1)

VLDRD<v><q>.<dt> Qd, [Qm{, #+/-<imm>}]!

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (M:Qm);
5 n      = integer UNKNOWN;
6 size   = '11';
7 msize  = size;
8 offset = ZeroExtend(imm:Zeros(UInt(size)), 32);
9 esize  = 8 << UInt(size);
10 elements = 32 DIV esize;
11 mesize = 64;
12 add    = (A == '1');
13 useReg  = FALSE;
14 scaleOffset = FALSE;
15 unsigned = TRUE;
16 wback   = (W == '1');
17 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
18 if D:Qd == M:Qm then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for T1 encodings

<dt>

This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned. Operations that do not perform widening are always unsigned (encoded with U=1), the equivalent sized floating and signless datatypes are allowed but are an alias for the unsigned version.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

<Qm>

Vector offset register. The elements of this register contain the unsigned offsets to add to the base address.

Assembler symbols for T2 encodings

<dt>

This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned. Operations that do not perform widening are always unsigned (encoded with U=1), the equivalent sized floating and signless datatypes are allowed but are an alias for the unsigned version.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0

`<Qm>` U32 Encoded as `size = 10, U = 1`
Vector offset register. The elements of this register contain the unsigned offsets to add to the base address.

Assembler symbols for T3 encodings

`<dt>` Unsigned flag: S indicates signed, U indicates unsigned. Operations that do not perform widening are always unsigned (encoded with U=1), the equivalent sized floating and signless datatypes are allowed but are an alias for the unsigned version.
This parameter must be the following value:
U32 Encoded as `size = 10, U = 1`
`<Qm>` Vector offset register. The elements of this register contain the unsigned offsets to add to the base address.

Assembler symbols for T4 encodings

`<dt>` Unsigned flag: S indicates signed, U indicates unsigned. Operations that do not perform widening are always unsigned (encoded with U=1), the equivalent sized floating and signless datatypes are allowed but are an alias for the unsigned version.
This parameter must be the following value:
U64 Encoded as `size = 11, U = 1`
`<Qm>` Vector offset register. The elements of this register contain the unsigned offsets to add to the base address.

Assembler symbols for T5 encodings

`<dt>` Data size. This parameter is ".32".
`<Qm>` The base register for the target address.
`<imm>` The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 4.

Assembler symbols for T6 encodings

`<dt>` Data size. This parameter is ".64".
`<Qm>` The base register for the target address.
`<imm>` The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 8.

Assembler symbols for all encodings

`<v, q>` See [C1.2.5 Standard assembler syntax fields](#).
`<Qd>` Destination vector register.
`<Rn>` The base register for the target address.
`<os>` The amount by which the vector offset is left shifted by before being added to the general-purpose base address. If the value is present it must correspond to memory transfer size (1=half word, 2=word, 3=double word).
This parameter must be one of the following values:
`<omitted>` Encoded as `os = 0`
`<Offset scaled>` Encoded as `os = 1`

Operation for all encodings

```
1 EncodingSpecificOperations();
```



```

2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  if esize == 64 then
8      // 64 bit accesses read their base address or offset from the first element
9      // in each pair of 32 bit elements.
10     if useReg then
11         baseAddr = R[n];
12         offset = Q[m, UInt(curBeat[1]:'0')];
13         if scaleOffset then
14             offset = LSL(offset, UInt(msize));
15         else
16             baseAddr = Q[m, UInt(curBeat[1]:'0')];
17         offsetAddress = if add then baseAddr + offset else baseAddr - offset;
18         bigEndian = BigEndian(offsetAddress, 8);
19         address = (if (curBeat[0] == '0') == bigEndian then offsetAddress + 4
20                                                         else offsetAddress);
21         if elmtMask[0] == '1' then
22             result = MemA_MVE(address, 4);
23         // Address write-back is not predicated
24         if wback && (curBeat[0] == '1') then
25             Q[m, curBeat-1] = offsetAddress[31:0];
26     else
27         // 32, 16, or 8 bit accesses
28         for e = 0 to (elements - 1)
29             if useReg then
30                 baseAddr = R[n];
31                 offset = ZeroExtend(Elem[Q[m, curBeat], e, esize], 32);
32                 if scaleOffset then
33                     offset = LSL(offset, UInt(msize));
34             else
35                 // 16 / 8 bit vector+immediate accesses are not supported
36                 baseAddr = Q[m, curBeat];
37                 address = if add then baseAddr + offset else baseAddr - offset;
38                 if elmtMask[e*(esize>>3)] == '1' then
39                     memValue = MemA_MVE(address, mesize DIV 8);
40                     Elem[result, e, esize] = Extend(memValue, esize, unsigned);
41                 // Address write-back is not predicated
42                 if wback then
43                     Elem[Q[m, curBeat], e, esize] = address[esize-1:0];
44
45
46  Q[d, curBeat] = result;

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.367 VLLDM

Floating-point Lazy Load Multiple. Floating-point Lazy Load Multiple restores the contents of the Secure Floating-point registers that were protected by a VLSTM instruction, and marks the Floating-point context as active.

If the lazy state preservation set up by a previous VLSTM instruction is active (FPCCR.LSPACT == 1), this instruction deactivates lazy state preservation and enables access to the Secure Floating-point registers.

If lazy state preservation is inactive (FPCCR.LSPACT == 0), either because lazy state preservation was not enabled (FPCCR.LSPEN == 0) or because a Floating-point instruction caused the Secure Floating-point register contents to be stored to memory, this instruction loads the stored Secure Floating-point register contents back into the Floating-point registers.

If Secure Floating-point is not in use (CONTROL_S.SFPA == 0), this instruction behaves as a NOP.

This instruction is only available in Secure state, and is UNDEFINED in Non-secure state.

If the Floating-point Extension and MVE are not implemented, this instruction is available in Secure state, but behaves as a NOP.

T1

Armv8-M Main Extension only

- In Armv8.0-M, bit[7] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[7] (lower 16 bits of the encoding) [0] represents 0.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	(0)	1	1	Rn				(0)	(0)	(0)	(0)	1	0	1	0	[0]	(0)	(0)	(0)	(0)	(0)	(0)	(0)

T1 variant

VLLDM{<c>}{<q>} <Rn> {, <reglist>}

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn);
3 if !IsSecure() || !VFPSmallRegisterBank() then UNDEFINED;
4 lowRegsOnly = TRUE;
5 if n == 15 then UNPREDICTABLE;
```

T2

Armv8.1-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	(0)	1	1	Rn				(0)	(0)	(0)	(0)	1	0	1	0	1	(0)	(0)	(0)	(0)	(0)	(0)	(0)

T2 variant

VLLDM{<c>}{<q>} <Rn>, <reglist>

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn);
3 if !IsSecure() then UNDEFINED;
4 lowRegsOnly = FALSE;
5 if n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
<reglist>	For encoding T1: Optional register list of {D0-D15}, having the register list is the preferred disassembly For encoding T2: Mandatory register list of {D0-D31}

Operation for all encodings

```
1 if ConditionPassed() then
2     EncodingSpecificOperations();
3
4     if CONTROL.SFPA == '1' then
5         // Check access to the co-processor is permitted
6         exc = CheckCPEnabled(10);
7         HandleException(exc);
8
9         if FPCCR.S.LSPACT == '1' then // state in floating-point is still valid
10             FPCCR.S.LSPACT = '0';
11     else
12         if !IsAligned(R[n],8) then
13             UFSR.UNALIGNED = '1';
14             exc = CreateException(UsageFault);
15             HandleException(exc);
16
17         for i = 0 to 15
18             if exc.fault == NoFault then
19                 (exc, S[i]) = MemA_with_priv_security(R[n] + (4*i), 4, AccType_NORMAL,
20                                                         FindMemPriv(), IsSecure(), TRUE);
21
22             if exc.fault == NoFault then
23                 (exc, FPSCR) = MemA_with_priv_security(R[n] + 0x40, 4, AccType_NORMAL,
24                                                         FindMemPriv(), IsSecure(), TRUE);
25
26             if exc.fault == NoFault then
27                 if HaveMve() then
28                     (exc, VPR) = MemA_with_priv_security(R[n] + 0x44, 4, AccType_NORMAL,
29                                                         FindMemPriv(), IsSecure(), TRUE);
30                 elseif boolean IMPLEMENTATION_DEFINED "No MVE VPR mem access" then
31                     (exc, -) = MemA_with_priv_security(R[n] + 0x44, 4, AccType_NORMAL,
32                                                         FindMemPriv(), IsSecure(), TRUE);
33
34             if FPCCR.S.TS == '1' then
35                 for i = 0 to 15
36                     if exc.fault == NoFault then
37                         (exc, S[i+16]) = MemA_with_priv_security(R[n] + 0x48 + (4*i), 4,
38                                                                     AccType_NORMAL,
39                                                                     FindMemPriv(),
40                                                                     IsSecure(), TRUE);
41
42             if !lowRegsOnly && boolean UNKNOWN then
43                 for i = 0 to 31
44                     if exc.fault == NoFault then
45                         (exc, -) = MemA_with_priv_security(R[n] + 0x88 + (4*i), 4,
46                                                                     AccType_NORMAL,
47                                                                     FindMemPriv(),
48                                                                     IsSecure(), TRUE);
49
50             // If an interrupt or a memory fault causes this instruction to abort,
51             // then all the registers that would have been loaded must be cleared.
52             if exc.fault != NoFault then
53                 InvalidateFPRegs(TRUE, FPCCR.S.TS == '1');
54
55             HandleException(exc);
56
57     CONTROL.FPCA = '1';
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data being loaded.
- The point at which an asynchronous exception is taken does not vary based on the value or values being loaded by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.368 VLSTM

Floating-point Lazy Store Multiple. Floating-point Lazy Store Multiple stores the contents of Secure Floating-point registers to a prepared stack frame, and clears the Secure Floating-point registers.

If Floating-point lazy preservation is enabled (FPCCR.LSPEN == 1), then the next time a Floating-point instruction other than VLSTM or VLLDM is executed:

- The contents of Secure Floating-point registers are stored to memory.
- The Secure Floating-point registers are cleared.

If Secure Floating-point is not in use (CONTROL_S.SFPA == 0), this instruction behaves as a NOP.

This instruction is only available in Secure state, and is UNDEFINED in Non-secure state.

If the Floating-point Extension and MVE are not implemented, this instruction is available in Secure state, but behaves as a NOP.

T1

Armv8-M Main Extension only

- In Armv8.0-M, bit[7] (lower 16 bits of the encoding) [0] represents (0).
- In Armv8.1-M, bit[7] (lower 16 bits of the encoding) [0] represents 0.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	(0)	1	0	Rn				(0)	(0)	(0)	(0)	1	0	1	0	[0]	(0)	(0)	(0)	(0)	(0)	(0)	(0)

T1 variant

VLSTM{<c>}{<q>} <Rn> {, <reglist>}

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn);
3 if !IsSecure() || !VFPSmallRegisterBank() then UNDEFINED;
4 lowRegsOnly = TRUE;
5 if n == 15 then UNPREDICTABLE;
```

T2

Armv8.1-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	(0)	1	0	Rn				(0)	(0)	(0)	(0)	1	0	1	0	1	(0)	(0)	(0)	(0)	(0)	(0)	(0)

T2 variant

VLSTM{<c>}{<q>} <Rn>, <reglist>

Decode for this encoding

```

1 if !HaveMainExt() then UNDEFINED;
2 n = UInt(Rn);
3 if !IsSecure() then UNDEFINED;
4 lowRegsOnly = FALSE;
5 if n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
<reglist>	For encoding T1: Optional register list of {D0-D15}, having the register list is the preferred disassembly For encoding T2: Mandatory register list of {D0-D31}

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3
4      if CONTROL_S.SFPA == '1' then
5          // Check access to the co-processor is permitted
6          exc = CheckCPEnabled(10);
7          HandleException(exc);
8
9          // LSPACT should not be active at the same time as there is active
10         // floating-point state. This is a possible attack scenario so raise
11         // a SecureFault.
12         lspact = if FPCCR_S.S == '1' then FPCCR_S.LSPACT else FPCCR_NS.LSPACT;
13         if lspact == '1' then
14             SFSR.LSERR = '1';
15             exc = CreateException(SecureFault);
16             HandleException(exc);
17         else
18             if !IsAligned(R[n], 8) then
19                 UFSR.UNALIGNED = '1';
20                 exc = CreateException(UsageFault);
21                 HandleException(exc);
22
23             if FPCCR.LSPEN == '0' then
24                 for i = 0 to 15
25                     MemA[R[n] + (4*i), 4] = S[i];
26                     MemA[R[n] + 0x40, 4] = FPSCR;
27                     if HaveMve() then
28                         MemA[R[n] + 0x44, 4] = VPR;
29                     elseif boolean IMPLEMENTATION_DEFINED "No MVE VPR mem access" then
30                         MemA[R[n] + 0x44, 4] = bits(32) UNKNOWN;
31
32                 pushFPCalleeFrame = FPCCR.TS == '1';
33                 if pushFPCalleeFrame then
34                     for i = 0 to 15
35                         MemA[R[n] + 0x48 + (4*i), 4] = S[i+16];
36
37                     InvalidateFPCregs(pushFPCalleeFrame, pushFPCalleeFrame);
38
39                 if !lowRegsOnly && boolean UNKNOWN then
40                     for i = 0 to 31
41                         MemA[R[n] + 0x88 + (4*i), 4] = bits(32) UNKNOWN;
42             else
43                 UpdateFPCCR(R[n], FALSE);
44
45         CONTROL.FPCA = '0';

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.369 VMAX, VMAXA

Vector Maximum, Vector Maximum Absolute. Find the maximum value of the elements in the source operands, and store the result in the corresponding destination elements.

The absolute variant takes the elements from the destination vector, treating them as unsigned, and compares them to the absolute values of the corresponding elements in the source vector. The larger values are stored back into the destination vector.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn	0		Qd	0	0	1	1	0	N	1	M	0		Qm		0			

T1: VMAX variant

VMAX<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 absolute = FALSE;
8 unsigned = U == '1';
9 esize    = 8 << UInt(size);
10 elements = 32 DIV esize;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	0	Da	1	1	size	1	1		Qda	0	1	1	1	0	1	0	M	0		Qm		1		

T2: VMAXA variant

VMAXA<v><q>.<dt> Qda, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' || M == '1' then UNDEFINED;
4 da     = UInt(Da:Qda);
5 m      = UInt(M:Qm);
6 d      = da;
7 n      = da;
8 absolute = TRUE;

```



```

9 unsigned = FALSE;
10 esize    = 8 << UInt(size);
11 elements = 32 DIV esize;
12 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

Assembler symbols for T2 encodings

<dt> Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00
S16	Encoded as	size = 01
S32	Encoded as	size = 10

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Destination vector register.
<Qda>	Source and destination vector register.
<Qm>	Source vector register.
<Qn>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 op2    = Q[m, curBeat];
9 for e = 0 to elements-1
10     value1 = Int(Elem[op1, e, esize], unsigned || absolute);
11     value2 = Int(Elem[op2, e, esize], unsigned);
12     if absolute then
13         value2 = Abs(value2);
14     Elem[result, e, esize] = Max(value1, value2)[esize-1:0];
15
16 for e = 0 to 3
17     if elmtMask[e] == '1' then
18         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1, and the T1 variant of this instruction is being executed:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.370 VMAXNM

Floating-point Maximum Number. Floating-point Maximum Number determines the floating-point maximum number.

NaN handling is specified by IEEE754-2008.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	0	0	Vn			Vd			1	0	size		N		M	0	Vm					
																										op = 0					

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

```
VMAXNM{<q>}.F16 <Sd>, <Sn>, <Sm>
// Not permitted in IT block
```

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

```
VMAXNM{<q>}.F32 <Sd>, <Sn>, <Sm>
// Not permitted in IT block
```

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

```
VMAXNM{<q>}.F64 <Dd>, <Dn>, <Dm>
// Not permitted in IT block
```

Decode for this encoding

```
1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if InITBlock() then UNPREDICTABLE;
6 maximum = (op == '0');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.

<Dm> Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3 case size of
4     when '01'
5         if maximum then
6             S[d] = Zeros(16) : FPMaxNum(S[n][15:0], S[m][15:0], TRUE);
7         else
8             S[d] = Zeros(16) : FPMinNum(S[n][15:0], S[m][15:0], TRUE);
9     when '10'
10        if maximum then
11            S[d] = FPMaxNum(S[n], S[m], TRUE);
12        else
13            S[d] = FPMinNum(S[n], S[m], TRUE);
14    when '11'
15        if maximum then
16            D[d] = FPMaxNum(D[n], D[m], TRUE);
17        else
18            D[d] = FPMinNum(D[n], D[m], TRUE);
```

C2.4.371 VMAXNM, VMAXNMA (floating-point)

Vector Maximum, Vector Maximum Absolute. Find the floating-point maximum number of the elements in the source operands, and store the result in the corresponding destination elements. It handles NaNs in consistence with the IEEE754-2008 specification, and returns the numerical operand when one operand is numerical and the other is a quiet NaN.

The absolute variant takes the absolute values of the elements from the destination vector and compares them to the absolute values of the corresponding elements in the source vector. The larger values are stored back into the destination vector.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	0	D	0	sz	Qn	0	Qd	0	1	1	1	1	N	1	M	1	Qm	0						

T1: VMAXNM variant

VMAXNM<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (M:Qm);
5 n      = UInt (N:Qn);
6 absolute = FALSE;
7 esize   = if sz == '1' then 16 else 32;
8 elements = 32 DIV esize;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	Da	1	1	1	1	1	1	Qda	0	1	1	1	0	1	0	M	0	Qm	1				

T2: VMAXNMA variant

VMAXNMA<v><q>.<dt> Qda, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if Da == '1' || M == '1' then UNDEFINED;
3 da     = UInt (Da:Qda);
4 m      = UInt (M:Qm);
5 d      = da;
6 n      = da;
7 absolute = TRUE;

```

```

8  esize      = if sz == '1' then 16 else 32;
9  elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F32 Encoded as sz = 0 F16 Encoded as sz = 1
<Qd>	Destination vector register.
<Qda>	Source and destination vector register.
<Qm>	Source vector register.
<Qn>	Source vector register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1    = Q[n, curBeat];
8  op2    = Q[m, curBeat];
9  for e = 0 to elements-1
10     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
11     predicated = (elmtMask[e*(esize>>3)] == '0');
12     value1     = Elem[op1, e, esize];
13     value2     = Elem[op2, e, esize];
14     if absolute then
15         value1 = FPAbs(value1);
16         value2 = FPAbs(value2);
17     Elem[result, e, esize] = FPMaxNum(value1, value2, FALSE, predicated);
18
19 for e = 0 to 3
20     if elmtMask[e] == '1' then
21         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.372 VMAXNMV, VMAXNMAV (floating-point)

Vector Maximum Across Vector, Vector Maximum Absolute Across Vector. Find the maximum value of the elements of a vector register. Store the maximum value in the general-purpose destination register only if it is larger than the starting value of the general-purpose destination register. The general-purpose register is read as the same width as the vector elements. For half-precision the upper half of the general-purpose register is cleared on write-back. This instruction handles NaNs in consistence with the IEEE754-2008 specification, and returns the numerical operand when one operand is numerical and the other is a quiet NaN.

The absolute variant of the instruction compares the absolute value of vector elements.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	1	1	1	0	1	1	1	0	Rda	1	1	1	1	0	0	M	0	Qm	0					

T1: VMAXNMV variant

VMAXNMV<v><q>.<dt> Rda, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if M == '1' then UNDEFINED;
3 da      = UInt (Rda);
4 m       = UInt (M:Qm);
5 absolute = FALSE;
6 esize   = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
9 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	1	1	1	0	1	1	0	0	Rda	1	1	1	1	0	0	M	0	Qm	0					

T2: VMAXNMAV variant

VMAXNMAV<v><q>.<dt> Rda, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if M == '1' then UNDEFINED;
3 da      = UInt (Rda);
4 m       = UInt (M:Qm);
5 absolute = TRUE;
6 esize   = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
```

```
8 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
9 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the floating-point format used.
 This parameter must be one of the following values:
 F32 Encoded as sz = 0
 F16 Encoded as sz = 1
 <Rda> General-purpose source and destination register.
 <Qm> Source vector register.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1 = Q[m, curBeat];
7 result = Elem[R[da], 0, esize];
8 for e = 0 to elements-1
9     if elmtMask[e*(esize>>3)] == '1' then
10         value = Elem[op1, e, esize];
11         result = FPConvertNaN(result, FALSE);
12         value = FPConvertNaN(value, FALSE);
13         if absolute then
14             value = FPAbs(value);
15         result = FPMaxNum(value, result, FALSE);
16
17 R[da] = ZeroExtend(result);
```


C2.4.373 VMAXV, VMAXAV

Vector Maximum Across Vector, Vector Maximum Absolute Across Vector. Find the maximum value of the elements of a vector register. Store the maximum value in the general-purpose destination register only if it is larger than the starting value of the general-purpose destination register. The general-purpose register is read as the same width as the vector elements. The result of the operation is sign-extended to 32 bits before being stored back.

The absolute variant of the instruction compares the absolute value of signed vector elements and treats the value in the general-purpose register as unsigned.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	1	1	0	size	1	0		Rda				1	1	1	1	0	0	M	0		Qm	0	

T1: VMAXV variant

VMAXV<v><q>.<dt> Rda, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if M == '1' then UNDEFINED;
4 da      = UInt(Rda);
5 m       = UInt(M:Qm);
6 absolute = FALSE;
7 unsigned = U == '1';
8 esize    = 8 << UInt(size);
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	1	1	1	0	size	0	0		Rda				1	1	1	1	0	0	M	0		Qm	0	

T2: VMAXAV variant

VMAXAV<v><q>.<dt> Rda, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if M == '1' then UNDEFINED;
4 da      = UInt(Rda);
5 m       = UInt(M:Qm);
6 absolute = TRUE;
```

```

7 unsigned = FALSE;
8 esize    = 8 << UInt(size);
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

Assembler symbols for T2 encodings

<dt> Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00
S16	Encoded as	size = 01
S32	Encoded as	size = 10

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).

<Rda> General-purpose source and destination register.

<Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1    = Q[m, curBeat];
7 result = Int(Elem[R[da], 0, esize], absolute || unsigned);
8 for e = 0 to elements-1
9     if elmtMask[e*(esize>>3)] == '1' then
10         value = Int(Elem[op1, e, esize], unsigned);
11         if absolute then
12             value = Abs(value);
13         result = Max(value, result);
14
15 R[da] = result[31:0];

```

C2.4.374 VMIN, VMINA

Vector Minimum, Vector Minimum Absolute. Find the minimum value of the elements in the source operands, and store the result in the corresponding destination elements.

The absolute variant takes the elements from the destination vector, treating them as unsigned, and compares them to the absolute values of the corresponding elements in the source vector. The smaller values are stored back into the destination vector.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn	0		Qd	0	0	1	1	0	N	1	M	1		Qm	0				

T1: VMIN variant

VMIN<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(N:Qn);
7 absolute = FALSE;
8 unsigned = U == '1';
9 esize = 8 << UInt(size);
10 elements = 32 DIV esize;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	0	Da	1	1	size	1	1	Qda	1	1	1	1	0	1	0	M	0		Qm	1				

T2: VMINA variant

VMINA<v><q>.<dt> Qda, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' || M == '1' then UNDEFINED;
4 da = UInt(Da:Qda);
5 m = UInt(M:Qm);
6 d = da;
7 n = da;
8 absolute = TRUE;

```

```

9 unsigned = FALSE;
10 esize    = 8 << UInt(size);
11 elements = 32 DIV esize;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

Assembler symbols for T2 encodings

<dt> Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00
S16	Encoded as	size = 01
S32	Encoded as	size = 10

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Destination vector register.
<Qda>	Source and destination vector register.
<Qm>	Source vector register.
<Qn>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 op2    = Q[m, curBeat];
9 for e = 0 to elements-1
10     value1 = Int(Elem[op1, e, esize], unsigned || absolute);
11     value2 = Int(Elem[op2, e, esize], unsigned);
12     if absolute then
13         value2 = Abs(value2);
14     Elem[result, e, esize] = Min(value1, value2)[esize-1:0];
15
16 for e = 0 to 3
17     if elmtMask[e] == '1' then
18         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1, and the T1 variant of this instruction is being executed:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.375 VMINNM

Floating-point Minimum Number. Floating-point Minimum Number determines the floating-point minimum number.

NaN handling is specified by IEEE754-2008.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	0	0	Vn			Vd			1	0	size		N		M	0	Vm					
																								op = 1							

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

```
VMINNM{<q>}.F16 <Sd>, <Sn>, <Sm>
// Not permitted in IT block
```

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

```
VMINNM{<q>}.F32 <Sd>, <Sn>, <Sm>
// Not permitted in IT block
```

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

```
VMINNM{<q>}.F64 <Dd>, <Dn>, <Dm>
// Not permitted in IT block
```

Decode for this encoding

```
1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if InITBlock() then UNPREDICTABLE;
6 maximum = (op == '0');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.

<Dm> Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3 case size of
4     when '01'
5         if maximum then
6             S[d] = Zeros(16) : FPMaxNum(S[n][15:0], S[m][15:0], TRUE);
7         else
8             S[d] = Zeros(16) : FPMinNum(S[n][15:0], S[m][15:0], TRUE);
9     when '10'
10        if maximum then
11            S[d] = FPMaxNum(S[n], S[m], TRUE);
12        else
13            S[d] = FPMinNum(S[n], S[m], TRUE);
14    when '11'
15        if maximum then
16            D[d] = FPMaxNum(D[n], D[m], TRUE);
17        else
18            D[d] = FPMinNum(D[n], D[m], TRUE);
```

C2.4.376 VMINNM, VMINNMA (floating-point)

Vector Minimum, Vector Minimum Absolute. Find the floating-point minimum number of the elements in the source operands, and store the result in the corresponding destination elements. It handles NaNs in consistence with the IEEE754-2008 specification, and returns the numerical operand when one operand is numerical and the other is a quiet NaN.

The absolute variant takes the absolute values of the elements from the destination vector and compares them to the absolute values of the corresponding elements in the source vector. The smaller values are stored back into the destination vector.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	0	D	1	sz	Qn	0	Qd	0	1	1	1	1	N	1	M	1	Qm	0						

T1: VMINNM variant

VMINNM<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (M:Qm);
5 n      = UInt (N:Qn);
6 absolute = FALSE;
7 esize   = if sz == '1' then 16 else 32;
8 elements = 32 DIV esize;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	Da	1	1	1	1	1	1	Qda	1	1	1	1	0	1	0	M	0	Qm	1				

T2: VMINNMA variant

VMINNMA<v><q>.<dt> Qda, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if Da == '1' || M == '1' then UNDEFINED;
3 da     = UInt (Da:Qda);
4 m      = UInt (M:Qm);
5 d      = da;
6 n      = da;
7 absolute = TRUE;

```



```

8  esize      = if sz == '1' then 16 else 32;
9  elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F32 Encoded as sz = 0 F16 Encoded as sz = 1
<Qd>	Destination vector register.
<Qda>	Source and destination vector register.
<Qm>	Source vector register.
<Qn>	Source vector register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1     = Q[n, curBeat];
8  op2     = Q[m, curBeat];
9  for e = 0 to elements-1
10     // Avoid Floating-point exceptions on a predicated lane by checking the element mask
11     predicated = (elmtMask[e*(esize>>3)] == '0');
12     value1     = Elem[op1, e, esize];
13     value2     = Elem[op2, e, esize];
14     if absolute then
15         value1 = FPAbs(value1);
16         value2 = FPAbs(value2);
17     Elem[result, e, esize] = FPMinNum(value1, value2, FALSE, predicated);
18
19 for e = 0 to 3
20     if elmtMask[e] == '1' then
21         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.377 VMINNMV, VMINNMAV (floating-point)

Vector Minimum Across Vector, Vector Minimum Absolute Across Vector. Find the minimum value of the elements of a vector register. Store the minimum value in the general-purpose destination register only if it is smaller than the starting value of the general-purpose destination register. The general-purpose register is read as the same width as the vector elements. For half-precision the upper half of the general-purpose register is cleared on write-back. This instruction handles NaNs in consistence with the IEEE754-2008 specification, and returns the numerical operand when one operand is numerical and the other is a quiet NaN.

The absolute variant of the instruction compares the absolute value of vector elements.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	1	1	1	0	1	1	1	0	Rda	1	1	1	1	1	0	M	0	Qm	0					

T1: VMINNMV variant

VMINNMV<v><q>.<dt> Rda, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if M == '1' then UNDEFINED;
3 da      = UInt (Rda);
4 m       = UInt (M:Qm);
5 absolute = FALSE;
6 esize   = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
9 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	1	1	1	0	1	1	0	0	Rda	1	1	1	1	1	0	M	0	Qm	0					

T2: VMINNMAV variant

VMINNMAV<v><q>.<dt> Rda, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if M == '1' then UNDEFINED;
3 da      = UInt (Rda);
4 m       = UInt (M:Qm);
5 absolute = TRUE;
6 esize   = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
```

```
8 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
9 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the floating-point format used.
 This parameter must be one of the following values:
 F32 Encoded as sz = 0
 F16 Encoded as sz = 1
 <Rda> General-purpose source and destination register.
 <Qm> Source vector register.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1 = Q[m, curBeat];
7 result = Elem[R[da], 0, esize];
8 for e = 0 to elements-1
9     if elmtMask[e*(esize>>3)] == '1' then
10         value = Elem[op1, e, esize];
11         result = FPConvertNaN(result, FALSE);
12         value = FPConvertNaN(value, FALSE);
13         if absolute then
14             value = FPAbs(value);
15         result = FPMinNum(value, result, FALSE);
16
17 R[da] = ZeroExtend(result);
```

C2.4.378 VMINV, VMINAV

Vector Minimum Across Vector, Vector Minimum Absolute Across Vector. Find the minimum value of the elements of a vector register. Store the minimum value in the general-purpose destination register only if it is smaller than the starting value of the general-purpose destination register. The general-purpose register is read as the same width as the vector elements. The result of the operation is sign-extended to 32 bits before being stored back.

The absolute variant of the instruction compares the absolute value of signed vector elements and treats the value in the general-purpose register as unsigned.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	1	1	0	size	1	0		Rda	1	1	1	1	1	0	M	0		Qm	0				

T1: VMINV variant

VMINV<v><q>.<dt> Rda, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_Mve);
3 if M == '1' then UNDEFINED;
4 da      = UInt (Rda);
5 m      = UInt (M:Qm);
6 absolute = FALSE;
7 unsigned = U == '1';
8 esize   = 8 << UInt (size);
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	1	1	1	0	size	0	0		Rda					1	1	1	1	1	0	M	0		Qm	0

T2: VMINAV variant

VMINAV<v><q>.<dt> Rda, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_Mve);
3 if M == '1' then UNDEFINED;
4 da      = UInt (Rda);
5 m      = UInt (M:Qm);
```

```

6 absolute = TRUE;
7 unsigned = FALSE;
8 esize    = 8 << UInt(size);
9 elements = 32 DIV esize;
10 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rda == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

Assembler symbols for T2 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:

S8	Encoded as	size = 00
S16	Encoded as	size = 01
S32	Encoded as	size = 10

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rda> General-purpose source and destination register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1    = Q[m, curBeat];
7 result = Int(Elem[R[da], 0, esize], absolute || unsigned);
8 for e = 0 to elements-1
9     if elmtMask[e*(esize>>3)] == '1' then
10         value = Int(Elem[op1, e, esize], unsigned);
11         if absolute then
12             value = Abs(value);
13         result = Min(value, result);
14
15 R[da] = result[31:0];

```

C2.4.379 VMLA

Floating-point Multiply Accumulate. Floating-point Multiply Accumulate multiplies two floating-point registers, adds the product to the destination register, and places the result in the destination register.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	0	0	Vn			Vd			1	0	size	N		M	0	Vm						
																							op = 0								

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

`VMLA{<c>}{<q>}.F16 <Sd>, <Sn>, <Sm>`

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

`VMLA{<c>}{<q>}.F32 <Sd>, <Sn>, <Sm>`

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

`VMLA{<c>}{<q>}.F64 <Dd>, <Dn>, <Dm>`

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 add = (op == '0');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              if add then
7                  addend16 = FPMul(S[n][15:0], S[m][15:0], TRUE);
8              else
9                  addend16 = FPNeg(FPMul(S[n][15:0], S[m][15:0], TRUE));
10             S[d] = Zeros(16) : FPAdd(S[d][15:0], addend16, TRUE);
11         when '10'
12             if add then
13                 addend32 = FPMul(S[n], S[m], TRUE);
14             else
15                 addend32 = FPNeg(FPMul(S[n], S[m], TRUE));
16             S[d] = FPAdd(S[d], addend32, TRUE);
17         when '11'
18             if add then
19                 addend64 = FPMul(D[n], D[m], TRUE);
20             else
21                 addend64 = FPNeg(FPMul(D[n], D[m], TRUE));
22             D[d] = FPAdd(D[d], addend64, TRUE);
```

C2.4.380 VMLA (vector by scalar plus vector)

Vector Multiply Accumulate. Multiply each element in the source vector by a scalar value and add to the respective element from the destination vector. Store the result in the destination register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	0	Da	size		Qn	1		Qda	0	1	1	1	0	N	1	0	0					Rm		

T1: VMLA variant

VMLA<v><q>.<dt> Qda, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' || N == '1' then UNDEFINED;
4 da      = UInt(Da:Qda);
5 m       = UInt(Rm);
6 n       = UInt(N:Qn);
7 esize   = 8 << UInt(size);
8 elements = 32 DIV esize;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 I8 Encoded as size = 00
 I16 Encoded as size = 01
 I32 Encoded as size = 10
 <Qda> Accumulator vector register.
 <Qn> Source vector register.
 <Rm> Source general-purpose register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 element2 = SInt(R[m][esize-1:0]);
9 op3     = Q[da, curBeat];
10 for e = 0 to elements-1
11     element1 = SInt(Elm[op1, e, esize]);
12     element3 = SInt(Elm[op3, e, esize]);
```



```

13     value      = (element1 * element2) + element3;
14     Elem[result, e, esize] = value[esize-1:0];
15
16 for e = 0 to 3
17     if elmtMask[e] == '1' then
18         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.381 VMLADAV

Vector Multiply Add Dual Accumulate Across Vector. The elements of the vector registers are handled in pairs. In the base variant, corresponding elements from the two source registers are multiplied together, whereas the exchange variant swaps the values in each pair of values read from the first source register, before multiplying them with the values from the second source register. The results of the pairs of multiply operations are combined by adding them together. At the end of each beat these results are accumulated and the lower 32 bits written back to the general-purpose destination register. The initial value of the general-purpose destination register can optionally be added to the result.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	1	1	1	Qn	sz	Rda	X	1	1	1	0	N	0	A	0	Qm	0						

T1: VMLADAV variant

VMLADAV{A}{X}<v><q>.<dt> Rda, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if N == '1' then UNDEFINED;
3 if U == '1' && X == '1' then UNDEFINED;
4 da = UInt(Rda:'0');
5 m = UInt(Qm);
6 n = UInt(N:Qn);
7 exchange = (X == '1');
8 accumulate = (A == '1');
9 esize = if sz == '0' then 16 else 32;
10 elements = 32 DIV esize;
11 unsigned = (U == '1');
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	1	1	1	Qn	0	Rda	X	1	1	1	1	N	0	A	0	Qm	0						

T2: VMLADAV variant

VMLADAV{A}{X}<v><q>.<dt> Rda, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if N == '1' then UNDEFINED;
3 if U == '1' && X == '1' then UNDEFINED;
4 da = UInt(Rda:'0');
5 m = UInt(Qm);

```

```

6  n          = UInt(N:Qn);
7  exchange   = (X == '1');
8  accumulate = (A == '1');
9  esize      = 8;
10 elements   = 32 DIV esize;
11 unsigned   = (U == '1');
12 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S16	Encoded as	sz = 0,	U = 0
U16	Encoded as	sz = 0,	U = 1
S32	Encoded as	sz = 1,	U = 0
U32	Encoded as	sz = 1,	U = 1

Assembler symbols for T2 encodings

<dt> Unsigned flag: S indicates signed, U indicates unsigned.
This parameter must be one of the following values:

S8	Encoded as	U = 0
U8	Encoded as	U = 1

Assembler symbols for all encodings

<A> Accumulate with existing register contents.
This parameter must be one of the following values:

–	Encoded as	A = 0
A	Encoded as	A = 1

<X> Exchange adjacent pairs of values in Qm.
This parameter must be one of the following values:

–	Encoded as	X = 0
X	Encoded as	X = 1

<v, q> See [C1.2.5 Standard assembler syntax fields](#).

<Rda> General-purpose source and destination register. This must be an even numbered register.

<Qn> First source vector register.

<Qm> Second source vector register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = if accumulate || !IsFirstBeat() then Int(R[da], unsigned) else 0;
7  // 32 bit operations are handled differently as they perform cross beat
8  // register accesses
9  if esize == 32 then
10     if elmtMask[0] == '1' then
11         if exchange then
12             if curBeat[0] == '0' then
13                 mul = Int(Q[n, curBeat+1], unsigned) * Int(Q[m, curBeat], unsigned);
14             else
15                 mul = Int(Q[n, curBeat-1], unsigned) * Int(Q[m, curBeat], unsigned);
16         else
17             mul = Int(Q[n, curBeat], unsigned) * Int(Q[m, curBeat], unsigned);

```

```

18         result = result + mul;
19     else
20         op1 = Q[n, curBeat];
21         op2 = Q[m, curBeat];
22         for e = 0 to elements-1
23             if elmtMask[e*(esize>>3)] == '1' then
24                 if exchange then
25                     if e[0] == '0' then
26                         mul = (Int(Elem[op1, e+1, esize], unsigned) *
27                             Int(Elem[op2, e, esize], unsigned));
28                     else
29                         mul = (Int(Elem[op1, e-1, esize], unsigned) *
30                             Int(Elem[op2, e, esize], unsigned));
31                 else
32                     elem1 = Int(Elem[op1, e, esize], unsigned);
33                     elem2 = Int(Elem[op2, e, esize], unsigned);
34                     mul = elem1 * elem2;
35                 result = result + mul;
36     R[da] = result[31:0];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.382 VMLALDAV

Vector Multiply Add Long Dual Accumulate Across Vector. The elements of the vector registers are handled in pairs. In the base variant, corresponding elements from the two source registers are multiplied together, whereas the exchange variant swaps the values in each pair of values read from the first source register, before multiplying them with the values from the second source register. The results of the pairs of multiply operations are combined by adding them together. At the end of each beat these results are accumulated. The 64-bit result is stored across two registers, the upper-half is stored in an odd-numbered register and the lower half is stored in an even-numbered register. The initial value of the general-purpose destination registers can optionally be added to the result.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	RdaHi			Qn	sz	RdaLo	X	1	1	1	0	N	0	A	0			Qm					0

T1: VMLALDAV variant

VMLALDAV{A}{X}<v><q>.<dt> RdaLo, RdaHi, Qn, Qm

Decode for this encoding

```

1 if RdaHi == '111' then SEE "VMLADAV";
2 CheckDecodeFaults(ExtType_Mve);
3 if N == '1' then UNDEFINED;
4 if U == '1' && X == '1' then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 m = UInt(Qm);
8 n = UInt(N:Qn);
9 exchange = (X == '1');
10 accumulate = (A == '1');
11 esize = if sz == '0' then 16 else 32;
12 elements = 32 DIV esize;
13 unsigned = (U == '1');
14 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
15 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 A Encoded as A = 1
<X>	Exchange adjacent pairs of values in Qm. This parameter must be one of the following values: – Encoded as X = 0 X Encoded as X = 1
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	This parameter determines the following values: – Unsigned flag: S indicates signed, U indicates unsigned. – Size: indicates the size of the elements in the vector. This parameter must be one of the following values: S16 Encoded as sz = 0, U = 0

	U16	Encoded as	sz = 0, U = 1
	S32	Encoded as	sz = 1, U = 0
	U32	Encoded as	sz = 1, U = 1
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination. This must be an even numbered register. The value RdaLo » 1 is encoded in the RdaLo field.		
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination. This must be an odd numbered register. The value RdaHi » 1 is encoded in the RdaHi field.		
<Qn>	First source vector register.		
<Qm>	Second source vector register.		

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = if accumulate || !IsFirstBeat() then Int(R[dah]:R[dal], unsigned) else 0;
7  // 32 bit operations are handled differently as they perform cross beat
8  // register accesses
9  if esize == 32 then
10     if elmtMask[0] == '1' then
11         if exchange then
12             if curBeat[0] == '0' then
13                 mul = Int(Q[n, curBeat+1], unsigned) * Int(Q[m, curBeat], unsigned);
14             else
15                 mul = Int(Q[n, curBeat-1], unsigned) * Int(Q[m, curBeat], unsigned);
16         else
17             mul = Int(Q[n, curBeat], unsigned) * Int(Q[m, curBeat], unsigned);
18         result = result + mul;
19     else
20         op1 = Q[n, curBeat];
21         op2 = Q[m, curBeat];
22         for e = 0 to elements-1
23             if elmtMask[e*(esize>>3)] == '1' then
24                 if exchange then
25                     if e[0] == '0' then
26                         mul = (Int(Elem[op1, e+1, esize], unsigned) *
27                             Int(Elem[op2, e, esize], unsigned));
28                     else
29                         mul = (Int(Elem[op1, e-1, esize], unsigned) *
30                             Int(Elem[op2, e, esize], unsigned));
31                 else
32                     elem1 = Int(Elem[op1, e, esize], unsigned);
33                     elem2 = Int(Elem[op2, e, esize], unsigned);
34                     mul = elem1 * elem2;
35                     result = result + mul;
36 R[dah] = result[63:32];
37 R[dal] = result[31:0];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.383 VMLALV

Vector Multiply Accumulate Long Across Vector. This is an alias of VMLALDAV without exchange.

This is an alias of [VMLALDAV](#) with the following condition satisfied: X==0.

This alias is the preferred disassembly.

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	RdaHi			Qn		sz		RdaLo	0	1	1	1	0	N	0	A	0	Qm		0			

VMLALV variant

VMLALV{A}<v><q>.<dt> RdaLo, RdaHi, Qn, Qm

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.384 VMLAS (vector by vector plus scalar)

Vector Multiply Accumulate Scalar. Multiply each element in the source vector by the respective element from the destination vector and add to a scalar value. Store the result in the destination register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	0	Da	size		Qn	1			Qda	1	1	1	1	0	N	1	0	0					Rm	

T1: VMLAS variant

VMLAS<v><q>.<dt> Qda, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' || N == '1' then UNDEFINED;
4 da      = UInt(Da:Qda);
5 m       = UInt(Rm);
6 n       = UInt(N:Qn);
7 esize   = 8 << UInt(size);
8 elements = 32 DIV esize;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 I8 Encoded as size = 00
 I16 Encoded as size = 01
 I32 Encoded as size = 10
 <Qda> Source and destination vector register.
 <Qn> Source vector register.
 <Rm> Source general-purpose register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 op2    = Q[da, curBeat];
9 element3 = SInt(R[m][esize-1:0]);
10 for e = 0 to elements-1
11     element1 = SInt(Elm[op1, e, esize]);
12     element2 = SInt(Elm[op2, e, esize]);
```



```

13     value      = (element1 * element2) + element3;
14     Elem[result, e, esize] = value[esize-1:0];
15
16 for e = 0 to 3
17     if elmtMask[e] == '1' then
18         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.385 VMLAV

Vector Multiply Accumulate Across Vector. This is an alias of VMLADAV without exchange.

This is an alias of [VMLADAV](#) with the following condition satisfied: X==0.

This alias is the preferred disassembly.

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	1	1	1	Qn	sz	Rda	0	1	1	1	0	N	0	A	0	Qm	0						

VMLAV variant

VMLAV{A}<v><q>.<dt> Rda, Qn, Qm

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.386 VMLS

Floating-point Multiply Subtract. Floating-point Multiply Subtract multiplies two floating-point registers, subtracts the product from the destination floating-point register, and places the result in the destination floating-point register.

Arm recommends that software does not use the **VMLS** instruction in the Round towards +Infinity and Round towards -Infinity rounding modes, because the rounding of the product and of the sum can change the result of the instruction in opposite directions, defeating the purpose of these rounding modes.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	0	0	Vn				Vd				1	0	size	N		M	0	Vm				
																							op = 1								

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VMLS{<c>}{<q>}.F16 <Sd>, <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VMLS{<c>}{<q>}.F32 <Sd>, <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VMLS{<c>}{<q>}.F64 <Dd>, <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 add = (op == '0');
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.

<Dn> Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm> Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              if add then
7                  addend16 = FPMul(S[n][15:0], S[m][15:0], TRUE);
8              else
9                  addend16 = FPNeg(FPMul(S[n][15:0], S[m][15:0], TRUE));
10             S[d] = Zeros(16) : FPAdd(S[d][15:0], addend16, TRUE);
11         when '10'
12             if add then
13                 addend32 = FPMul(S[n], S[m], TRUE);
14             else
15                 addend32 = FPNeg(FPMul(S[n], S[m], TRUE));
16             S[d] = FPAdd(S[d], addend32, TRUE);
17         when '11'
18             if add then
19                 addend64 = FPMul(D[n], D[m], TRUE);
20             else
21                 addend64 = FPNeg(FPMul(D[n], D[m], TRUE));
22             D[d] = FPAdd(D[d], addend64, TRUE);

```

C2.4.387 VMLSDAV

Vector Multiply Subtract Dual Accumulate Across Vector. The elements of the vector registers are handled in pairs. In the base variant, corresponding elements from the two source registers are multiplied together, whereas the exchange variant swaps the values in each pair of values read from the first source register, before multiplying them with the values from the second source register. The results of the pairs of multiply operations are combined by subtracting one from the other. At the end of each beat these results are accumulated and the lower 32 bits written back to the general-purpose destination register. The initial value of the general-purpose destination register can optionally be added to the result.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	1	1	1	Qn	sz	Rda	X	1	1	1	0	N	0	A	0	Qm	1						

T1: VMLSDAV variant

VMLSDAV{A}{X}<v><q>.<dt> Rda, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if N == '1' then UNDEFINED;
3 da = UInt(Rda:'0');
4 m = UInt(Qm);
5 n = UInt(N:Qn);
6 exchange = (X == '1');
7 accumulate = (A == '1');
8 esize = if sz == '0' then 16 else 32;
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	1	1	1	Qn	0	Rda	X	1	1	1	0	N	0	A	0	Qm	1						

T2: VMLSDAV variant

VMLSDAV{A}{X}<v><q>.S8 Rda, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if N == '1' then UNDEFINED;
3 da = UInt(Rda:'0');
4 m = UInt(Qm);
5 n = UInt(N:Qn);
6 exchange = (X == '1');
7 accumulate = (A == '1');

```

```

8  esize      = 8;
9  elements   = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
 S16 Encoded as sz = 0
 S32 Encoded as sz = 1

Assembler symbols for all encodings

<A> Accumulate with existing register contents.
This parameter must be one of the following values:
 - Encoded as A = 0
 A Encoded as A = 1

<X> Exchange adjacent pairs of values in Qm.
This parameter must be one of the following values:
 - Encoded as X = 0
 X Encoded as X = 1

<v, q> See [C1.2.5 Standard assembler syntax fields](#).

<Rda> General-purpose source and destination register. This must be an even numbered register.

<Qn> First source vector register.

<Qm> Second source vector register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = if accumulate || !IsFirstBeat() then SInt(R[da]) else 0;
7  // 32 bit operations are handled differently as they perform cross beat
8  // register accesses
9  if esize == 32 then
10     if elmtMask[0] == '1' then
11         if exchange then
12             if curBeat[0] == '0' then
13                 mul = SInt(Q[n, curBeat+1]) * SInt(Q[m, curBeat]);
14             else
15                 mul = SInt(Q[n, curBeat-1]) * SInt(Q[m, curBeat]);
16         else
17             mul = SInt(Q[n, curBeat]) * SInt(Q[m, curBeat]);
18         if curBeat[0] == '0' then
19             result = result + mul;
20         else
21             result = result - mul;
22     else
23         op1 = Q[n, curBeat];
24         op2 = Q[m, curBeat];
25         for e = 0 to elements-1
26             if elmtMask[e*(esize>>3)] == '1' then
27                 if exchange then
28                     if e[0] == '0' then
29                         mul = (SInt(Elem[op1, e+1, esize]) *
30                             SInt(Elem[op2, e, esize]));
31                     else
32                         mul = (SInt(Elem[op1, e-1, esize]) *
33                             SInt(Elem[op2, e, esize]));
34                 else
35                     elem1 = SInt(Elem[op1, e, esize]);

```

```

36         elem2 = SInt(Elem[op2, e, esize]);
37         mul    = elem1 * elem2;
38         if e[0] == '0' then
39             result = result + mul;
40         else
41             result = result - mul;
42 R[da] = result[31:0];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.388 VMLS�DAV

Vector Multiply Subtract Long Dual Accumulate Across Vector. The elements of the vector registers are handled in pairs. In the base variant, corresponding elements from the two source registers are multiplied together, whereas the exchange variant swaps the values in each pair of values read from the first source register, before multiplying them with the values from the second source register. The results of the pairs of multiply operations are combined by subtracting one from the other. At the end of each beat these results are accumulated. The 64-bit result is stored across two registers, the upper-half is stored in an odd-numbered register and the lower half is stored in an even-numbered register. The initial value of the general-purpose destination registers can optionally be added to the result.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	RdaHi			Qn	sz	RdaLo	X	1	1	1	0	N	0	A	0			Qm				1	

T1: VMLS�DAV variant

VMLS�DAV{A}{X}<v><q>.<dt> RdaLo, RdaHi, Qn, Qm

Decode for this encoding

```

1 if RdaHi == '111' then SEE "VMLSĐAV";
2 CheckDecodeFaults(ExtType_Mve);
3 if N == '1' then UNDEFINED;
4 dah      = UInt(RdaHi:'1');
5 dal      = UInt(RdaLo:'0');
6 m        = UInt(Qm);
7 n        = UInt(N:Qn);
8 exchange = (X == '1');
9 accumulate = (A == '1');
10 esize    = if sz == '0' then 16 else 32;
11 elements = 32 DIV esize;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
13 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 A Encoded as A = 1
<X>	Exchange adjacent pairs of values in Qm. This parameter must be one of the following values: – Encoded as X = 0 X Encoded as X = 1
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: S16 Encoded as sz = 0 S32 Encoded as sz = 1
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination. This must be an even numbered register. The value RdaLo » 1 is encoded in the RdaLo field.

<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination. This must be an odd numbered register. The value $\text{RdaHi} \gg 1$ is encoded in the RdaHi field.
<Qn>	First source vector register.
<Qm>	Second source vector register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = if accumulate || !IsFirstBeat() then SInt(R[dah]:R[dal]) else 0;
7  // 32 bit operations are handled differently as they perform cross beat
8  // register accesses
9  if esize == 32 then
10     if elmtMask[0] == '1' then
11         if exchange then
12             if curBeat[0] == '0' then
13                 mul = SInt(Q[n, curBeat+1]) * SInt(Q[m, curBeat]);
14             else
15                 mul = SInt(Q[n, curBeat-1]) * SInt(Q[m, curBeat]);
16         else
17             mul = SInt(Q[n, curBeat]) * SInt(Q[m, curBeat]);
18         if curBeat[0] == '0' then
19             result = result + mul;
20         else
21             result = result - mul;
22     else
23         op1 = Q[n, curBeat];
24         op2 = Q[m, curBeat];
25         for e = 0 to elements-1
26             if elmtMask[e*(esize>>3)] == '1' then
27                 if exchange then
28                     if e[0] == '0' then
29                         mul = (SInt(Elem[op1, e+1, esize]) *
30                             SInt(Elem[op2, e, esize]));
31                     else
32                         mul = (SInt(Elem[op1, e-1, esize]) *
33                             SInt(Elem[op2, e, esize]));
34                 else
35                     elem1 = SInt(Elem[op1, e, esize]);
36                     elem2 = SInt(Elem[op2, e, esize]);
37                     mul = elem1 * elem2;
38                     if e[0] == '0' then
39                         result = result + mul;
40                     else
41                         result = result - mul;
42 R[dah] = result[63:32];
43 R[dal] = result[31:0];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.389 VMOV (between general-purpose register and half-precision register)

Floating-point Move (between general-purpose register and half-precision register). Floating-point Move (between general-purpose register and half-precision register) transfers the contents of a half-precision register to a general-purpose register, or the contents of a general-purpose register to a half-precision register

T1

Armv8.1-M Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	0	0	op	Vn				Rt				1	0	0	1	N	(0)	(0)	1	(0)	(0)	(0)	(0)

General-purpose register to half-precision register variant

Applies when **op == 0**.

VMOV{<c>}{<q>}.F16, <Sn>, <Rt>

Half-precision register to general-purpose register variant

Applies when **op == 1**.

VMOV{<c>}{<q>}.F16, <Rt>, <Sn>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_HpFp);
2 to_arm_register = (op == '1');
3 t = UInt(Rt); n = UInt(Vn:N);
4 if t == 15 || t == 13 then UNPREDICTABLE;
5 if InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sn>	Is the 32-bit name of the floating-point source register, encoded in the "Vn:N" field.
<Rt>	Is the general-purpose register that <Sn> will be transferred to or from, encoded in the "Rt" field.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3 if to_arm_register then
4     R[t] = Zeros(16) : S[n][15:0];
5 else
6     S[n] = Zeros(16) : R[t][15:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.390 VMOV (between general-purpose register and single-precision register)

Floating-point Move (between general-purpose register and single-precision register). Floating-point Move (between general-purpose register and single-precision register) transfers the contents of a single-precision register to a general-purpose register, or the contents of a general-purpose register to a single-precision register.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	0	0	op	Vn				Rt				1	0	1	0	N	(0)	(0)	1	(0)	(0)	(0)	(0)

General-purpose register to single-precision register variant

Applies when **op** == 0.

VMOV{<c>}{<q>} <Sn>, <Rt>

Single-precision register to general-purpose register variant

Applies when **op** == 1.

VMOV{<c>}{<q>} <Rt>, <Sn>

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_MveOrFp);
2 to_arm_register = (op == '1'); t = UInt(Rt); n = UInt(Vn:N);
3 if t == 15 || t == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<Rt>	Is the general-purpose register that <Sn> will be transferred to or from, encoded in the "Rt" field.
<Sn>	Is the 32-bit name of the floating-point register to be transferred, encoded in the "Vn:N" field.
<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   if to_arm_register then
5     R[t] = S[n];
6   else
7     S[n] = R[t];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.391 VMOV (between two general-purpose registers and a doubleword register)

Floating-point Move (between two general-purpose registers and a doubleword register). Floating-point Move (between two general-purpose registers and a doubleword register) transfers two words from two general-purpose registers to a doubleword register, or from a doubleword register to two general-purpose registers.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	1	0	op																				

Doubleword register to general-purpose registers variant

Applies when **op** == 1.

VMOV{<c>}{<q>} <Rt>, <Rt2>, <Dm>

General-purpose registers to doubleword register variant

Applies when **op** == 0.

VMOV{<c>}{<q>} <Dm>, <Rt>, <Rt2>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_MveOrFp);
2 if VFPSmallRegisterBank() && (M == '1') then UNDEFINED;
3 to_arm_registers = (op == '1'); t = UInt(Rt); t2 = UInt(Rt2); m = UInt(M:Vm);
4 if t == 15 || t2 == 15 then UNPREDICTABLE;
5 if t == 13 || t2 == 13 then UNPREDICTABLE;
6 if to_arm_registers && t == t2 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If to_arm_registers && t == t2, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

Assembler symbols for all encodings

<Dm>	Is the 64-bit name of the floating-point register to be transferred, encoded in the "M:Vm" field.
<Rt2>	Is the first general-purpose register that <Dm>[63:32] will be transferred to or from, encoded in the "Rt" field.
<Rt>	Is the first general-purpose register that <Dm>[31:0] will be transferred to or from, encoded in the "Rt" field.
<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   if to_arm_registers then
5     R[t] = D[m][31:0];
6     R[t2] = D[m][63:32];
```

```
7     else  
8         D[m][31:0] = R[t];  
9         D[m][63:32] = R[t2];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.392 VMOV (between two general-purpose registers and two single-precision registers)

Floating-point Move (between two general-purpose registers and two single-precision registers). Floating-point Move (between two general-purpose registers and two single-precision registers) transfers the contents of two consecutively numbered single-precision registers to two general-purpose registers, or the contents of two general-purpose registers to a pair of single-precision registers. The general-purpose registers do not have to be contiguous.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	1	0	op	Rt2				Rt				1	0	1	0	0	0	M	1	Vm			

Single-precision register to general-purpose register variant

Applies when **op** == 1.

VMOV{<c>}{<q>} <Rt>, <Rt2>, <Sm>, <Sm1>

General-purpose register to single-precision register variant

Applies when **op** == 0.

VMOV{<c>}{<q>} <Sm>, <Sm1>, <Rt>, <Rt2>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_MveOrFp);
2 to_arm_registers = (op == '1'); t = UInt(Rt); t2 = UInt(Rt2); m = UInt(Vm:M);
3 if t == 15 || t2 == 15 || m == 31 then UNPREDICTABLE;
4 if t == 13 || t2 == 13 then UNPREDICTABLE;
5 if to_arm_registers && t == t2 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If `to_arm_registers && t == t2`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The value in the destination register is UNKNOWN.

CONSTRAINED UNPREDICTABLE behavior

If `m == 31`, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- One or more of the single-precision registers become UNKNOWN for a move to the single-precision register. The general-purpose registers listed in the instruction become UNKNOWN for a move from the single-precision registers. This behavior does not affect any other general-purpose registers.

Assembler symbols for all encodings

<Rt2> Is the second general-purpose register that <Sm1> will be transferred to or from, encoded in the "Rt" field.

<Rt>	Is the first general-purpose register that <Sm> will be transferred to or from, encoded in the "Rt" field.
<Sm1>	Is the 32-bit name of the second floating-point register to be transferred. This is the next floating-point register after <Sm>.
<Sm>	Is the 32-bit name of the first floating-point register to be transferred, encoded in the "Vm:M" field.
<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   if to_arm_registers then
5     R[t] = S[m];
6     R[t2] = S[m+1];
7   else
8     S[m] = R[t];
9     S[m+1] = R[t2];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.393 VMOV (general-purpose register to vector lane)

Vector Move (general-purpose register to vector lane). Copy the value of a general-purpose register to a vector lane.

This instruction is subject to beat-wise execution if it is not in an IT block.

This instruction is not VPT compatible.

T1

Armv8.1-M Floating-point Extension or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	op1	0	Qd	h			Rt	1	0	1	1	D	op2	1	(0)	(0)	(0)	(0)	(0)	(0)	(0)	(0)	

T1: VMOV variant

VMOV<c><q>.<dt> Qd[idx], Rt

Decode for this encoding

```

1 d = UInt(D:Qd);
2 t = UInt(Rt);
3 esize = integer UNKNOWN; elemIdx = integer UNKNOWN;
4 case (h:op1:op2) of
5     when 'x1xxx' instType = ExtType_Mve;      esize = 8;  elemIdx = UInt((h:op1:op2)[1:0]);
6     when 'x0xx1' instType = ExtType_Mve;      esize = 16; elemIdx = UInt((h:op1:op2)[1]);
7     when 'x0x00' instType = ExtType_MveOrFp; esize = 32; elemIdx = 0;
8     otherwise    instType = ExtType_Unknown;
9 CheckDecodeFaults(instType);
10 targetBeat = UInt((h:op1:op2)[4]:(h:op1:op2)[2]);
11 if D == '1' then UNDEFINED;
12 if op1 == '0x' && op2 == '10' then UNDEFINED;
13 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
14 if Rt == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 32 Encoded as op1 = 0x, op2 = 00
 16 Encoded as op1 = 0x, op2 = x1
 8 Encoded as op1 = 1x, op2 = xx
 <Qd> Destination vector register.
 <idx> Element index to select in the vector register, must be in the range 0 to ((128/dt)-1). This value is encoded into the bits of h:op1:op2 which are not used to encode dt.
 <Rt> Source general-purpose register.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     ExecuteFPCheck();
4
5     if InITBlock() || !HaveMve() then
6         Elem[Q[d, targetBeat],elemIdx,esize] = R[t][esize-1:0];
7     else
8         (curBeat, -) = GetCurInstrBeat();
9         if curBeat == targetBeat then
```



```
10      Elem[Q[d, curBeat],elemIdx,esize] = R[t][esize-1:0];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.394 VMOV (half of doubleword register to single general-purpose register)

Floating-point Move (half of doubleword register to single general-purpose register). Floating-point Move (half of doubleword register to single general-purpose register) transfers one word from the upper or lower half of a doubleword register to a general-purpose register.

This instruction is an alias of [VMOV \(vector lane to general-purpose register\)](#)

T1

Armv8.0-M Floating-point Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	0	H	1	Vn				Rt				1	0	1	1	N	0	0	1	(0)	(0)	(0)	(0)

VMOV variant

VMOV<c><q>.<dt> Rt, Dn[x]

is equivalent to

VMOV<c>.<dt> Rt, Qn[idx]

where Dn[x] is expressed as Qn[idx]

Operation for all encodings

The description of [VMOV \(vector lane to general-purpose register\)](#) gives the operational pseudocode for this instruction.

C2.4.395 VMOV (immediate)

Floating-point Move (immediate). Floating-point Move (immediate) places an immediate constant into the destination floating-point register.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	imm4H			Vd			1	0	size		(0)	0	(0)	0	imm4L					

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VMOV{<c>}{<q>}.F16 <Sd>, #<imm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VMOV{<c>}{<q>}.F32 <Sd>, #<imm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VMOV{<c>}{<q>}.F64 <Dd>, #<imm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 case size of
6     when '01'
7         d = UInt(Vd:D);
8         imm16 = VFPEExpandImm(imm4H:imm4L, 16);
9         imm32 = Zeros(16) : imm16;
10    when '10'
11        d = UInt(Vd:D);
12        imm32 = VFPEExpandImm(imm4H:imm4L, 32);
13    when '11'
14        d = UInt(D:Vd);
15        imm64 = VFPEExpandImm(imm4H:imm4L, 64);

```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <Sd> Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
 <Dd> Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
 <imm> Is a floating-point constant. For details of the range of constants available and the encoding of <imm>, see the definition of [VFPEExpandImm\(\)](#).

Operation for all encodings

```
1 if ConditionPassed() then  
2     EncodingSpecificOperations();  
3     ExecuteFPCheck();  
4     if dp_operation then  
5         D[d] = imm64;  
6     else  
7         S[d] = imm32;
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.396 VMOV (immediate) (vector)

Vector Move (immediate). Set each element of a vector register to the immediate operand value. The immediate is generated by the AdvSIMDEExpandImm() function based on the requested data type and immediate value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	i	1	1	1	1	1	D	0	0	0	imm3	Qd	0	cmode	0	1	op	1	imm4										

T1: VMOV variant

VMOV<v><q>.<dt> Qd, #<imm>

Decode for this encoding

```

1 if op == '0' && cmode IN {'0xx1', '10x1'} then SEE "VORR (immediate)";
2 if op == '1' && cmode IN {'0xx0', '110x', '10x0'} then SEE "VMVN (immediate)";
3 if op == '1' && cmode IN {'0xx1', '10x1'} then SEE "VBIC (immediate)";
4 CheckDecodeFaults(ExtType_Mve);
5 if D == '1' then UNDEFINED;
6 if cmode == '1111' && op == '1' then UNDEFINED;
7 d = UInt(D:Qd);
8 imm64 = AdvSIMDEExpandImm(op, cmode, i:imm3:imm4);
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
<dt> Size: indicates the size of the elements in the vector, for use with the AdvSIMDEExpandImm() function.
This parameter must be one of the following values:

- I32 Encoded as:
 - cmode = 0000, op = 0
 - cmode = 0010, op = 0
 - cmode = 0100, op = 0
 - cmode = 0110, op = 0
 - cmode = 1100, op = 0
 - cmode = 1101, op = 0
- I16 Encoded as:
 - cmode = 1000, op = 0
 - cmode = 1010, op = 0
- I8 Encoded as:
 - cmode = 1110, op = 0
- I64 Encoded as:
 - cmode = 1110, op = 1
- F32 Encoded as:
 - cmode = 1111, op = 0

<Qd> Destination vector register.
<imm> The immediate value to load in to each element. This must be an immediate that can be encoded for use with the AdvSIMDEExpandImm() function.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 if curBeat[0] == '0' then
7     result = imm64[31:0];
8 else
9     result = imm64[63:32];
10
11 for e = 0 to 3
12     if elmtMask[e] == '1' then
13         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.397 VMOV (register)

Floating-point Move (register). Floating-point Move (register) copies the contents of one register to another.

T2

Armv8-M Floating-point Extension, or Armv8.1-M MVE. `sz == 1` UNDEFINED in implementations that do not include either MVE or Double-precision support.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	0	0	0	Vd	1	0	1	sz	0	1	M	0	Vm						

Single-precision scalar variant

Armv8-M Floating-point Extension, or Armv8.1-M MVE.

Applies when `sz == 0`.

VMOV{<c>}{<q>}.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension with Double-precision support, or Armv8.1-M MVE.

Applies when `sz == 1`.

VMOV{<c>}{<q>}.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (sz == '1');
2 CheckDecodeFaults(if dp_operation then ExtType_MveOrDpFp else ExtType_MveOrFp);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
5 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();
3     ExecuteFPCheck();
4     if dp_operation then
5         D[d] = D[m];
6     else
7         S[d] = S[m];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.398 VMOV (register) (vector)

Vector Move (register). Copy the value of one vector register to another vector register.

This is an alias of [VORR](#) with the following condition satisfied: $Qm == Qn$.

This alias is the preferred disassembly.

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	1	0	Qm	0	Qd	0	0	0	0	1	M	1	M	1	Qm	0						

VMOV variant

VMOV<v><q> Qd, Qm

is equivalent to

VORR<v><q> Qd, Qm, Qm

and is the preferred disassembly when $Qm == Qn$

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.399 **VMOV (single general-purpose register to half of doubleword register)**

Floating-point Move (single general-purpose register to half of doubleword register). Floating-point Move (single general-purpose register to half of doubleword register) transfers one word from a general-purpose register to the upper or lower half of a doubleword register.

This instruction is an alias of [VMOV \(general-purpose register to vector lane\)](#)

T1

Armv8.0-M Floating-point Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	0	H	0	Vd				Rt				1	0	1	1	D	0	0	1	(0)	(0)	(0)	(0)

VMOV variant

VMOV<c><q>.<size> Dd[x], Rt

is equivalent to

VMOV<c>.<dt> Qd[idx], Rt

where Dd[x] is expressed as Qd[idx]

Operation for all encodings

The description of [VMOV \(general-purpose register to vector lane\)](#) gives the operational pseudocode for this instruction.

C2.4.400 VMOV (two 32-bit vector lanes to two general-purpose registers)

Vector Move (two 32-bit vector lanes to two general-purpose registers). Copy two 32-bit vector lanes to two general-purpose registers.

This instruction is subject to beat-wise execution if it is not in an IT block.

This instruction is not VPT compatible.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	0	1	1	0	0	0	D	0	0	Rt2				Qd				0	1	1	1	1	(0)	(0)	(0)	idx	Rt			

T1: VMOV variant

VMOV<c><q> Rt, Rt2, Qd[idx], Qd[idx2]

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 t = UInt(Rt);
5 t2 = UInt(Rt2);
6 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
7 if Rt == '11x1' then CONSTRAINED_UNPREDICTABLE;
8 if Rt2 == '11x1' || Rt == Rt2 then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <Rt> Destination general-purpose register
 <Rt2> Destination general-purpose register
 <idx> The first index for the vector register.
 This parameter must be one of the following values:
 2 Encoded as idx = 0
 3 Encoded as idx = 1
 <Qd> Source vector register.
 <idx2> The second index for the vector register. This must be two less than the first index.
 This parameter must be one of the following values:
 0 Encoded as idx = 0
 1 Encoded as idx = 1

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4
5   if InITBlock() then
6     R[t] = Q[d, UInt('0':idx)];
7     R[t2] = Q[d, UInt('1':idx)];
8   else
9     (curBeat, -) = GetCurInstrBeat();
10    if curBeat[0] == idx then
11      tReg = if curBeat[1] == '0' then t else t2;
12      R[tReg] = Q[d, curBeat];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.401 VMOV (two general-purpose registers to two 32-bit vector lanes)

Vector Move (two general-purpose registers to two 32-bit vector lanes). Copy two general-purpose registers to two 32-bit vector lanes.

This instruction is subject to beat-wise execution if it is not in an IT block.

This instruction is not VPT compatible.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	0	D	0	1	Rt2				Qd		0	1	1	1	1	(0)	(0)	(0)	idx	Rt				

T1: VMOV variant

VMOV<c><q> Qd[idx], Qd[idx2], Rt, Rt2

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 t = UInt(Rt);
5 t2 = UInt(Rt2);
6 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
7 if Rt == '11x1' then CONSTRAINED_UNPREDICTABLE;
8 if Rt2 == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Destination vector register.
<idx>	The first index for the vector register. This parameter must be one of the following values: 2 Encoded as idx = 0 3 Encoded as idx = 1
<idx2>	The second index for the vector register. This must be two less than the first index. This parameter must be one of the following values: 0 Encoded as idx = 0 1 Encoded as idx = 1
<Rt>	Source general-purpose register to be written to Qd[idx2].
<Rt2>	Source general-purpose register to be written to Qd[idx].

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4
5   if InITBlock() then
6     Q[d, UInt('0':idx)] = R[t];
7     Q[d, UInt('1':idx)] = R[t2];
8   else
9     (curBeat, -) = GetCurInstrBeat();
10    if curBeat[0] == idx then
11      tReg = if curBeat[1] == '0' then t else t2;
12      Q[d, curBeat] = R[tReg];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.402 VMOV (vector lane to general-purpose register)

Vector Move (vector lane to general-purpose register). Copy the value of a vector lane to a general-purpose register.

This instruction is subject to beat-wise execution if it is not in an IT block.

This instruction is not VPT compatible.

T1

Armv8.1-M Floating-point Extension or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	U	op1	1		Qn		h				Rt		1	0	1	1	N		op2	1	(0)	(0)	(0)	(0)

T1: VMOV variant

VMOV<c><q>.<dt> Rt, Qn[idx]

Decode for this encoding

```

1 n = UInt(N:Qn);
2 t = UInt(Rt);
3 esize = integer UNKNOWN; elemIdx = integer UNKNOWN;
4 case (U:h:op1:op2) of
5   when 'xxlxxx' instType = ExtType_Mve;      esize = 8;  elemIdx = UInt((h:op1:op2)[1:0]);
6   when 'xx0xx1' instType = ExtType_Mve;      esize = 16; elemIdx = UInt((h:op1:op2)[1]);
7   when '0x0x00' instType = ExtType_MveOrFp; esize = 32; elemIdx = 0;
8   otherwise    instType = ExtType_Unknown;
9 CheckDecodeFaults(instType);
10 targetBeat = UInt((h:op1:op2)[4]:(h:op1:op2)[2]);
11 unsigned   = (U == '1');
12 if N == '1' then UNDEFINED;
13 if U == '1' && op1 == '0x' && op2 == '00' then UNDEFINED;
14 if op1 == '0x' && op2 == '10' then UNDEFINED;
15 if InvPTBlock() then CONSTRAINED_UNPREDICTABLE;
16 if Rt == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> This parameter determines the following values:
 – Unsigned flag: S indicates signed, U indicates unsigned.
 – Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

32	Encoded as	op1 = 0x,	op2 = 00,	U = 0
S16	Encoded as	op1 = 0x,	op2 = x1,	U = 0
U16	Encoded as	op1 = 0x,	op2 = x1,	U = 1
S8	Encoded as	op1 = 1x,	op2 = xx,	U = 0
U8	Encoded as	op1 = 1x,	op2 = xx,	U = 1

<Rt> Destination general-purpose register.
 <Qn> Source vector register.
 <idx> Element index to select in the vector register, must be in the range 0 to ((128/dt)-1). This value is encoded into the bits of h:op1:op2 which are not used to encode dt.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();

```

```

4
5     if InITBlock() || !HaveMve() then
6         R[t] = Extend(Elem[Q[n, targetBeat],elemIdx,esize], unsigned);
7     else
8         (curBeat, -) = GetCurInstrBeat();
9         if curBeat == targetBeat then
10            R[t] = Extend(Elem[Q[n, curBeat],elemIdx,esize], unsigned);

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.403 VMOVL

Vector Move Long. Selects an element of 8 or 16-bits from either the top half (T variant) or bottom half (B variant) of each source element, sign- or zero-extends, and places the 16 or 32-bit results in the destination vector.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	D	1	sz	0	0	0	Qd	T	1	1	1	1	0	1	M	0	Qm	0					

T1: VMOVL variant

VMOVL<T><v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 if sz IN {'11', '00'} then SEE "VSHLL";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 unsigned = (U == '1');
7 esize   = 8 * UInt(sz);
8 elements = 16 DIV esize;
9 top     = UInt(T);
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T>	Specifies which half of the source element is used. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q> <dt>	See C1.2.5 Standard assembler syntax fields . This parameter determines the following values: – Unsigned flag: S indicates signed, U indicates unsigned. – The size of the elements in the vector. This parameter must be one of the following values: S8 Encoded as sz = 01, U = 0 U8 Encoded as sz = 01, U = 1 S16 Encoded as sz = 10, U = 0 U16 Encoded as sz = 10, U = 1
<Qd>	Destination vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
```

```

4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  opl    = Q[m, curBeat];
8  for e = 0 to elements-1
9      operand = Int(Elem[opl, 2*e + top, esize], unsigned);
10     Elem[result, e, 2*esize] = operand[(2*esize)-1:0];
11
12  for e = 0 to 3
13      if elmtMask[e] == '1' then
14          Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.404 VMOVN

Vector Move and Narrow. Performs an element-wise narrowing to half-width, writing the result to either the top half (T variant) or bottom half (B variant) of the result element. The other half of the destination vector element retains its previous value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	D	1	1	size	0	1		Qd	T	1	1	1	0	1	0	M	0		Qm			1	

T1: VMOVN variant

VMOVN<T><v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' then UNDEFINED;
4 if size == '10' then UNDEFINED;
5 d = UInt(D:Qd);
6 m = UInt(M:Qm);
7 esize = 8 << UInt(size);
8 elements = 16 DIV esize;
9 top = UInt(T);
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T>	Specifies which half of the result element the result is written to. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: I16 Encoded as size = 00 I32 Encoded as size = 01
<Qd>	Destination vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1 = Q[m, curBeat];
```

```

7 result = Q[d, curBeat];
8 for e = 0 to elements-1
9     operand = UInt(Elem[op1, e, 2*esize]);
10    Elem[result, 2*e + top, esize] = operand[esize-1:0];
11
12 for e = 0 to 3
13     if elmtMask[e] == '1' then
14         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.405 VMOVX

Floating-point Move extraction. Floating-point Move extraction copies the upper 16 bits of the 32-bit source floating-point register into the lower 16 bits of the 32-bit destination floating-point register, while clearing the remaining bits to zero.

T1

Armv8.1-M Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	0	0	0	0	Vd	1	0	1	0	0	1	M	0	Vm						

T1 variant

VMOVX<q>,F16 <Sd>, <Sm>

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_HpFp);
2 d = UInt(Vd:D); m = UInt(Vm:M);
3 if InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<q> See [C1.2.5 Standard assembler syntax fields](#).
 <Sd> Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
 <Sm> Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3 S[d] = Zeros(16) : S[m][31:16];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.406 VMRS

Move to general-purpose Register from Floating-point Special register. Move to general-purpose Register from Floating-point Special register moves the value of FPSCR, FPCXT_NS, FPCXT_S, VPR, or VPR.P0 to a general-purpose register, or the values of FPSCR condition flags to the APSR condition flags.

Access to the FPCXT payloads generates an UNDEFINED exception if the instruction is executed from Non-secure state.

If CP10 is not enabled and either the Main extension is not implemented or the Floating-point context is active, access to FPCXT_NS will generate a NOCP UsageFault. Accesses to FPCXT_NS will not trigger lazy state preservation if there is no active Floating-point context. Accesses to FPCXT_NS do not trigger Floating-point context creation regardless of the value of FPCCR.ASPEN.

T1

Armv8-M Floating-point Extension, or Armv8.1-M MVE.

- For Armv8.1-M, FPCXT access instructions are required. The `reg` field is configurable, and all of the listed registers can be accessed.
- For Armv8.0-M, only access to FPSCR is permitted, and the `reg` field is not configurable.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	1	1	1	reg				Rt				1	0	1	0	(0)	(0)	(0)	1	(0)	(0)	(0)	(0)

T1 variant

VMRS{<c>}{<q>} <Rt>, <spec_reg>

Decode for this encoding

```

1 fpCxtAnyAccess = HasArchVersion(Armv8p1) && (reg == '111x');
2 fpCxtNSAccess  = HasArchVersion(Armv8p1) && (reg == '1110');
3 fpInactive     = !HaveMveOrFPExt() || (FPCCR_NS.ASPEN == '1' && CONTROL.FPCA == '0');
4 if fpCxtAnyAccess && !IsSecure() then UNDEFINED;
5 if fpCxtNSAccess then
6     if !HaveMainExt() || !fpInactive then
7         HandleException(CheckCPEnabled(10));
8     else
9         CheckDecodeFaults(ExtType_MveOrFp);
10 t = UInt(Rt);
11 if t == 13 || (t == 15 && reg != '0001') then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Rt>	Is the general-purpose destination register, encoded in the "Rt" field. Is one of: APSR_nzcv Encoded as 0b1111. This instruction transfers the FPSCR.N, Z, C, V condition flags to the APSR.N, Z, C, V condition flags. R0–R14 General-purpose register.
<spec_reg>	Is the special register to be accessed, encoded in the "reg" field. The permitted values are: FPSCR when reg = 0b0001 FPSCR_nzcvq when reg = 0b0010, access to FPSCR condition and saturation flags. If Armv8.1-M is not implemented, access to this register is UNPREDICTABLE. VPR when reg = 0b1100. If MVE is not implemented, access to this register is UNPREDICTABLE. The VPR register can only be accessed from privileged mode. P0 when reg = 0b1101. If MVE is implemented access to VPR.P0 predicate field is permitted, otherwise the access is UNPREDICTABLE.

FPCCR_NS when reg = 0b1110. Enables saving and restoration of the Non-secure floating-point context. If the Floating-point context is active then the current FPSCR value is accessed and the default value in FPDSCR_NS is written into FPSCR, otherwise the default value in FPDSCR_NS is accessed. If neither the Floating-point extension nor MVE are implemented then access to this payload behaves as a NOP. If Armv8.1-M is not implemented, access to this register is UNPREDICTABLE.

FPCXT_S when reg = 0b1111. Enables saving and restoration of the Secure floating-point context. If Armv8.1-M is not implemented, access to this register is UNPREDICTABLE.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      if !fpCxtNSAccess then
4          ExecuteFPCheck();
5      elsif !fpInactive then
6          PreserveFPState();
7          SerializeVFP();
8          VFPExcBarrier();
9
10     case reg of
11         when '0001'
12             if t == 15 then
13                 APSR.N = FPSCR.N;
14                 APSR.Z = FPSCR.Z;
15                 APSR.C = FPSCR.C;
16                 APSR.V = FPSCR.V;
17             else
18                 R[t] = FPSCR;
19         when '0010'
20             if HasArchVersion(Armv8p1) then
21                 // Only read the N, Z, C, V, and QC flags
22                 R[t] = FPSCR[31:27]:Zeros(27);
23             else
24                 UNPREDICTABLE;
25         when '1100'
26             if HaveMve() then
27                 if CurrentModeIsPrivileged() then
28                     R[t] = VPR;
29                 elsif boolean IMPLEMENTATION_DEFINED "No MVE VPR mem access" then
30                     R[t] = Zeros(32);
31             else
32                 UNPREDICTABLE;
33         when '1101'
34             if HaveMve() then
35                 R[t] = Zeros(16):VPR.P0;
36             else
37                 UNPREDICTABLE;
38         when '1110'
39             if HasArchVersion(Armv8p1) then
40                 if HaveFPExt() || HaveMve() then
41                     FPCXT_Type cxt = Zeros(32);
42                     if !fpInactive then
43                         cxt.SFPA = CONTROL_S.SFPA;
44                         cxt[27:0] = FPSCR[27:0];
45                     else
46                         cxt[27:0] = FPDSCR_NS[27:0];
47                     R[t] = cxt;
48                     // If the floating-point context isn't Secure the FPSCR
49                     // value is set to the Non-secure default so any
50                     // Non-secure functions that are called before a
51                     // floating-point instruction is executed in the secure
52                     // state will get the same FPSCR value as functions
53                     // called after a Secure floating-point instruction
54                     // (which is the value of FPDSCR_NS).
55                     if !fpInactive && CONTROL_S.SFPA == '0' then

```

```
56         FPSCR = FPDSCR_NS[31:0];  
57     else  
58         UNPREDICTABLE;  
59     when '1111'  
60     if HasArchVersion(Armv8p1) then  
61         FPCXT_Type cxt = Zeros(32);  
62         cxt.SFPA      = CONTROL_S.SFPA;  
63         cxt[27:0]     = FPSCR[27:0];  
64         R[t]          = cxt;  
65         FPSCR         = FPDSCR_NS[31:0];  
66         CONTROL_S.SFPA = '0';  
67     else  
68         UNPREDICTABLE;  
69     otherwise  
70         UNPREDICTABLE;
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.DIT](#) is 1 and this instruction accesses the FPSCR.N, Z, C, V, Q flags or VPR.P0:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the FPSCR.N, Z, C, V, Q flags or VPR.P0.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of its registers.
 - The values of the FPSCR.N, Z, C, V, Q flags or VPR.P0.

C2.4.407 VMSR

Move to Floating-point Special register from general-purpose Register. Move to Floating-point Special register from general-purpose Register moves the value of a general-purpose register to FPSCR, FPCXT_NS, FPCXT_S, VPR, or VPR.P0.

Access to the FPCXT payloads generates an UNDEFINED exception if the instruction is executed from Non-secure state.

If CP10 is not enabled and either the Main extension is not implemented or the Floating-point context is active, access to FPCXT_NS will generate a NOCP UsageFault. Accesses to FPCXT_NS will not trigger lazy state preservation if there is no active Floating-point context. Accesses to FPCXT_NS do not trigger Floating-point context creation regardless of the value of FPCCR.ASPEN.

T1

Armv8-M Floating-point Extension, or Armv8.1-M MVE.

- For Armv8.1-M, FPCXT access instructions are required. The `reg` field is configurable, and all of the listed registers can be accessed.
- For Armv8.0-M, only access to FPSCR is permitted, and the `reg` field is not configurable.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	1	1	0	reg				Rt				1	0	1	0	(0)	(0)	(0)	1	(0)	(0)	(0)	(0)

T1 variant

VMSR{<c>}{<q>} <spec_reg>, <Rt>

Decode for this encoding

```

1 fpCxtAnyAccess = HasArchVersion(Armv8p1) && (reg == '111x');
2 fpCxtNSAccess  = HasArchVersion(Armv8p1) && (reg == '1110');
3 fpInactive     = !HaveMveOrFPExt() || (FPCCR_NS.ASPEN == '1' && CONTROL.FPCA == '0');
4 if fpCxtAnyAccess && !IsSecure() then UNDEFINED;
5 if fpCxtNSAccess then
6     if !HaveMainExt() || !fpInactive then
7         HandleException(CheckCPEnabled(10));
8     else
9         CheckDecodeFaults(ExtType_MveOrFp);
10 t = UInt(Rt);
11 if t == 15 || t == 13 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
 <q> See [C1.2.5 Standard assembler syntax fields](#).
 <spec_reg> Is the special register to be accessed, encoded in the "reg" field. The permitted values are:
 FPSCR when reg = 0b0001
 FPSCR_nzcvq when reg = 0b0010, access to FPSCR condition and saturation flags.
 If Armv8.1-M is not implemented, access to this register is UNPREDICTABLE.
 VPR when reg = 0b1100. If MVE is not implemented, access to this register is UNPREDICTABLE. The VPR register can only be accessed from privileged mode.
 P0 when reg = 0b1101. If MVE is implemented access to VPR.P0 predicate field is permitted, otherwise the access is UNPREDICTABLE.
 FPCCR_NS when reg = 0b1110. Enables saving and restoration of the Non-secure floating-point context. If the Floating-point context is active then the current FPSCR value is accessed and the default value in FPDSCR_NS is written into FPSCR, otherwise the default value in FPDSCR_NS is accessed. If neither the Floating-point extension nor MVE are implemented then access to this payload behaves as a NOP. If Armv8.1-M is not implemented, access to this register is UNPREDICTABLE.

FPCXT_S when reg = 0b1111. Enables saving and restoration of the Secure floating-point context. If Armv8.1-M is not implemented, access to this register is UNPREDICTABLE.

<Rt> Is the general-purpose source register to be transferred to <spec_reg>, encoded in the "Rt" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      if !fpCxtNSAccess then
4          ExecuteFPCheck();
5      elseif !fpInactive then
6          PreserveFPState();
7          SerializeVFP();
8          VFPExcBarrier();
9
10     case reg of
11         when '0001'
12             FPSCR = R[t];
13         when '0010'
14             if HasArchVersion(Armv8p1) then
15                 // Only update the N, Z, C, V, and QC flags
16                 FPSCR[31:27] = R[t][31:27];
17             else
18                 UNPREDICTABLE;
19         when '1100'
20             if HaveMve() then
21                 if CurrentModeIsPrivileged() then
22                     VPR = R[t];
23             else
24                 UNPREDICTABLE;
25         when '1101'
26             if HaveMve() then
27                 VPR.P0 = R[t][15:0];
28             else
29                 UNPREDICTABLE;
30         when '1110'
31             if HasArchVersion(Armv8p1) then
32                 if (HaveFPEExt() || HaveMve()) && !fpInactive then
33                     FPCXT_Type cxt = R[t];
34                     CONTROL_S.SFPA = cxt.SFPA;
35                     FPSCR = Zeros(4):cxt[27:0];
36             else
37                 UNPREDICTABLE;
38         when '1111'
39             if HasArchVersion(Armv8p1) then
40                 FPCXT_Type cxt = R[t];
41                 CONTROL_S.SFPA = cxt.SFPA;
42                 FPSCR = Zeros(4):cxt[27:0];
43             else
44                 UNPREDICTABLE;
45         otherwise
46             UNPREDICTABLE;

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.DIT](#) is 1 and this instruction accesses the FPSCR.N, Z, C, V, Q flags or VPR.P0:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the FPSCR.N, Z, C, V, Q flags or VPR.P0.

- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of its registers.
 - The values of the FPSCR.N, Z, C, V, Q flags or VPR.P0.

C2.4.408 VMUL

Floating-point Multiply. Floating-point Multiply multiplies two floating-point register values, and places the result in the destination floating-point register.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	1	0	Vn				Vd				1	0	size		N	0	M	0	Vm			

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VMUL{<c>}{<q>}.F16 {<Sd>, } <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VMUL{<c>}{<q>}.F32 {<Sd>, } <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VMUL{<c>}{<q>}.F64 {<Dd>, } <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
7 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
8 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();

```

```
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              S[d] = Zeros(16) : FPMul(S[n][15:0], S[m][15:0], TRUE);
7          when '10'
8              S[d] = FPMul(S[n], S[m], TRUE);
9          when '11'
10         D[d] = FPMul(D[n], D[m], TRUE);
```

C2.4.409 VMUL (floating-point)

Vector Multiply. Multiply the value of the elements in the first source vector register by either the respective elements in the second source vector register or a general-purpose register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	0	D	0	sz	Qn	0	Qd	0	1	1	0	1	N	1	M	1	Qm	0						

T1: VMUL variant

VMUL<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (M:Qm);
5 n      = UInt (N:Qn);
6 esize  = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 withScalar = FALSE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	D	1	1	Qn	1	Qd	0	1	1	1	0	N	1	1	0	Rm							

T2: VMUL variant

VMUL<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (Rm);
5 n      = UInt (N:Qn);
6 esize  = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 withScalar = TRUE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F32 Encoded as sz = 0 F16 Encoded as sz = 1
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.
<Rm>	Source general-purpose register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[n, curBeat];
8 if withScalar then
9     for e = 0 to elements-1
10         // Avoid Floating-point exceptions on a predicated lane by checking the element mask
11         pred = (elmtMask[e*(esize>>3)] == '0');
12         value = FPMul(Elem[op1, e, esize], R[m][esize-1:0], FALSE, pred);
13         Elem[result, e, esize] = value;
14 else
15     for e = 0 to elements-1
16         // Avoid Floating-point exceptions on a predicated lane by checking the element mask
17         pred = (elmtMask[e*(esize>>3)] == '0');
18         op2 = Q[m, curBeat];
19         value = FPMul(Elem[op1, e, esize], Elem[op2, e, esize], FALSE, pred);
20         Elem[result, e, esize] = value;
21
22 for e = 0 to 3
23     if elmtMask[e] == '1' then
24         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.410 VMUL (vector)

Vector Multiply. Multiply the value of the elements in the first source vector register by either the respective elements in the second source vector register or a general-purpose register. The result is then written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	size		Qn		0		Qd		0	1	0	0	1	N	1	M	1		Qm		0	

T1: VMUL variant

VMUL<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(N:Qn);
7 withScalar = FALSE;
8 esize = 8 << UInt(size);
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	size		Qn		1		Qd		1	1	1	1	0	N	1	1	0			Rm		

T2: VMUL variant

VMUL<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || N == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(Rm);
6 n = UInt(N:Qn);
7 withScalar = TRUE;
8 esize = 8 << UInt(size);
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```


Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: I8 Encoded as size = 00 I16 Encoded as size = 01 I32 Encoded as size = 10
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.
<Rm>	Source general-purpose register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1 = Q[n, curBeat];
8  if withScalar then
9      for e = 0 to elements-1
10         value = SInt(Elem[op1, e, esize]) * SInt(R[m][esize-1:0]);
11         Elem[result, e, esize] = value[esize-1:0];
12 else
13     op2 = Q[m, curBeat];
14     for e = 0 to elements-1
15         value = SInt(Elem[op1, e, esize]) * SInt(Elem[op2, e, esize]);
16         Elem[result, e, esize] = value[esize-1:0];
17
18 for e = 0 to 3
19     if elmtMask[e] == '1' then
20         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.411 VMULH, VRMULH

Vector Multiply Returning High Half, Vector Rounding Multiply Returning High Half. Multiply each element of a vector register by its respective element in another vector register and return the high half of the result. The result is optionally rounded before the high half is selected.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	D	size		Qn	1		Qd	0	1	1	1	0	N	0	M	0		Qm				1	

T1: VMULH variant

VMULH<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 esize  = 8 << UInt(size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 round  = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	D	size		Qn	1		Qd	1	1	1	1	0	N	0	M	0		Qm				1	

T2: VRMULH variant

VRMULH<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 esize  = 8 << UInt(size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 round  = TRUE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .																								
<dt>	This parameter determines the following values: – Unsigned flag: S indicates signed, U indicates unsigned. – Size: indicates the size of the elements in the vector. This parameter must be one of the following values: <table><tr><td>S8</td><td>Encoded as</td><td>size = 00,</td><td>U = 0</td></tr><tr><td>U8</td><td>Encoded as</td><td>size = 00,</td><td>U = 1</td></tr><tr><td>S16</td><td>Encoded as</td><td>size = 01,</td><td>U = 0</td></tr><tr><td>U16</td><td>Encoded as</td><td>size = 01,</td><td>U = 1</td></tr><tr><td>S32</td><td>Encoded as</td><td>size = 10,</td><td>U = 0</td></tr><tr><td>U32</td><td>Encoded as</td><td>size = 10,</td><td>U = 1</td></tr></table>	S8	Encoded as	size = 00,	U = 0	U8	Encoded as	size = 00,	U = 1	S16	Encoded as	size = 01,	U = 0	U16	Encoded as	size = 01,	U = 1	S32	Encoded as	size = 10,	U = 0	U32	Encoded as	size = 10,	U = 1
S8	Encoded as	size = 00,	U = 0																						
U8	Encoded as	size = 00,	U = 1																						
S16	Encoded as	size = 01,	U = 0																						
U16	Encoded as	size = 01,	U = 1																						
S32	Encoded as	size = 10,	U = 0																						
U32	Encoded as	size = 10,	U = 1																						
<Qd>	Destination vector register.																								
<Qn>	Source vector register.																								
<Qm>	Source vector register.																								

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 op2    = Q[m, curBeat];
9 rVal   = if round then 1 << (esize-1) else 0;
10 for e = 0 to elements-1
11     value = (Int(Elem[op1, e, esize], unsigned) * Int(Elem[op2, e, esize], unsigned)) + rVal;
12     Elem[result, e, esize] = value[(2*esize)-1:esize];
13
14 for e = 0 to 3
15     if elmtMask[e] == '1' then
16         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.412 VMULL (integer)

Vector Multiply Long. Performs an element-wise integer multiplication of two single-width source operand elements. These are selected from either the top half (T variant) or bottom half (B variant) of double-width source vector register elements. The operation produces a double-width result.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	D	size		Qn	1		Qd	T	1	1	1	0	N	0	M	0		Qm	0				

T1: VMULL variant

VMULL<T><v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 esize  = 8 << UInt(size);
8 unsigned = (U == '1');
9 top    = UInt(T);
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if size == '10' && (D:Qd == M:Qm || D:Qd == N:Qn) then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<T>	Specifies which half of the source element is used. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q> <dt>	See C1.2.5 <i>Standard assembler syntax fields</i> . This parameter determines the following values: – Unsigned flag: S indicates signed, U indicates unsigned. – Size: indicates the size of the elements in the vector. This parameter must be one of the following values: S8 Encoded as size = 00, U = 0 U8 Encoded as size = 00, U = 1 S16 Encoded as size = 01, U = 0 U16 Encoded as size = 01, U = 1 S32 Encoded as size = 10, U = 0 U32 Encoded as size = 10, U = 1
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  if esize == 32 then
8      op1    = Q[n, UInt(curBeat[1]:T)];
9      op2    = Q[m, UInt(curBeat[1]:T)];
10     mul     = Int(op1, unsigned) * Int(op2, unsigned);
11     result  = if curBeat[0] == '1' then mul[63:32] else mul[31:0];
12 else
13     op1      = Q[n, curBeat];
14     op2      = Q[m, curBeat];
15     elements = 16 DIV esize;
16     for e = 0 to elements-1
17         element1 = Elem[op1, e * 2 + top, esize];
18         element2 = Elem[op2, e * 2 + top, esize];
19         Elem[result, e, esize * 2] = (Int(element1, unsigned) *
20                                     Int(element2, unsigned)) [2*esize-1:0];
21
22 for e = 0 to 3
23     if elmtMask[e] == '1' then
24         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.413 VMULL (polynomial)

Vector Multiply Long. Performs an element-wise polynomial multiplication of two single-width source operand elements. These are selected from either the top half (T variant) or bottom half (B variant) of double-width source vector register elements. The operation produces a double-width result.

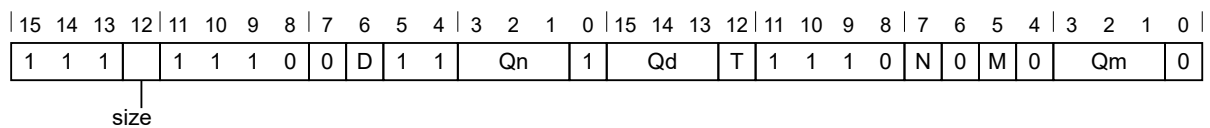
This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE



T1: VMULL variant

VMULL<T><v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt(D:Qd);
4 m      = UInt(M:Qm);
5 n      = UInt(N:Qn);
6 esize  = 8 << UInt(size);
7 elements = 16 DIV esize;
8 top    = UInt(T);
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<T>	Specifies which half of the source element is used. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Specifies whether to do 8x8->16 or 16x16->32 polynomial multiplications. This parameter must be one of the following values: P8 Encoded as size = 0 Indicates 8x8->16 P16 Encoded as size = 1 Indicates 16x16->32
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();

```

```
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[m, curBeat];
8 op2    = Q[n, curBeat];
9 for e = 0 to elements-1
10     element1 = Elem[op1, e * 2 + top, esize];
11     element2 = Elem[op2, e * 2 + top, esize];
12     Elem[result, e, esize*2] = PolynomialMult(element1, element2)[esize*2-1:0];
13
14 for e = 0 to 3
15     if elmtMask[e] == '1' then
16         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.414 VMVN (immediate)

Vector Bitwise NOT. Set each element of a vector register to the bitwise inverse of the immediate operand value. The immediate is generated by the AdvSIMDExpandImm() function based on the requested data type and immediate value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	i	1	1	1	1	1	D	0	0	0	imm3			Qd		0	cmode			0	1	1	1	imm4					

T1: VMVN variant

VMVN<v><q>.<dt> Qd, #<imm>

Decode for this encoding

```

1 if cmode == '111x' then SEE "VMOV (immediate) (vector)";
2 if cmode IN {'0xx1', '10x1'} then SEE "VBIC (immediate)";
3 CheckDecodeFaults(ExtType_Mve);
4 if D == '1' then UNDEFINED;
5 d = UInt(D:Qd);
6 imm64 = AdvSIMDExpandImm('1', cmode, i:imm3:imm4);
7 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector, for use with the AdvSIMDExpandImm() function.
 This parameter must be one of the following values:
 I32 Encoded as:
 cmode = 0000
 cmode = 0010
 cmode = 0100
 cmode = 0110
 cmode = 1100
 cmode = 1101
 I16 Encoded as:
 cmode = 1000
 cmode = 1010
 <Qd> Destination vector register.
 <imm> The immediate value to load in to each element. This must be an immediate that can be encoded for use with the AdvSIMDExpandImm() function.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5

```



```

6  if curBeat[0] == '0' then
7      result = NOT(imm64[31:0]);
8  else
9      result = NOT(imm64[63:32]);
10
11 for e = 0 to 3
12     if elmtMask[e] == '1' then
13         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.415 VMVN (register)

Vector Bitwise Not. Bitwise invert the value of a vector register and place the result in another vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	0	0	0	0	Qd	0	0	1	0	1	1	1	M	0	Qm	0				

T1: VMVN variant

VMVN<v><q> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = NOT(Q[m, curBeat]);
7
8 for e = 0 to 3
9     if elmtMask[e] == '1' then
10         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.416 VNEG

Floating-point Negate. Floating-point Negate inverts the sign bit of a half-precision or single-precision or double-precision register, and places the result in the destination register.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	0	0	1	Vd				1	0	size		0	1	M	0	Vm			

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VNEG{<c>}{<q>}.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VNEG{<c>}{<q>}.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VNEG{<c>}{<q>}.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
6 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   case size of
5     when '01' S[d] = Zeros(16) : FPNeg(S[m][15:0]);
6     when '10' S[d] = FPNeg(S[m]);
7     when '11' D[d] = FPNeg(D[m]);

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.417 VNEG (floating-point)

Vector Negate. Negate the value of each element of a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	1		Qd	0	0	1	1	1	1	1	1	1	M	0		Qm	0	

T1: VNEG variant

VNEG<v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size IN {'11', '00'} then UNDEFINED;
4 d      = UInt (D:Qd);
5 m      = UInt (M:Qm);
6 esize  = 8 << UInt (size);
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <dt> Size: indicates the floating-point format used.
 This parameter must be one of the following values:
 F16 Encoded as size = 01
 F32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[m, curBeat];
8 for e = 0 to elements-1
9     value = FPNeg(Elem[op1, e, esize]);
10    Elem[result, e, esize] = value;
11
12 for e = 0 to 3
13     if elmtMask[e] == '1' then
14         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.418 VNEG (vector)

Vector Negate. Negate the value of each element of a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	1		Qd	0	0	0	1	1	1	1	M	0		Qm	0			

T1: VNEG variant

VNEG<v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 esize = 8 << UInt(size);
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 S8 Encoded as size = 00
 S16 Encoded as size = 01
 S32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[m, curBeat];
8 for e = 0 to elements-1
9     value = -SInt(Elem[op1, e, esize]);
10    Elem[result, e, esize] = value[esize-1:0];
11
12 for e = 0 to 3
13     if elmtMask[e] == '1' then
14         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.419 VNMLA

Floating-point Multiply Accumulate and Negate. Floating-point Multiply Accumulate and Negate multiplies two floating-point register values, adds the negation of the floating-point value in the destination register to the negation of the product, and writes the result back to the destination register.

Arm recommends that software does not use the [VNMLA](#) instruction in the Round towards +Infinity and Round towards -Infinity rounding modes, because the rounding of the product and of the sum can change the result of the instruction in opposite directions, defeating the purpose of these rounding modes.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	0	1	Vn				Vd				1	0	size	N		M	0	Vm				
																							op = 1								

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VNMLA{<c>}{<q>}.F16 <Sd>, <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VNMLA{<c>}{<q>}.F32 <Sd>, <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VNMLA{<c>}{<q>}.F64 <Dd>, <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 operation = if op == '1' then VFPNegMul_VNMLA else VFPNegMul_VNMULS;
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.

<Dn> Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm> Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              product16 = FPMul(S[n][15:0], S[m][15:0], TRUE);
7              case operation of
8                  when VFPNegMul_VNMLA
9                      S[d] = Zeros(16) : FPAdd(FPNeg(S[d][15:0]), FPNeg(product16), TRUE);
10                 when VFPNegMul_VNMMLS
11                     S[d] = Zeros(16) : FPAdd(FPNeg(S[d][15:0]), product16, TRUE);
12                 when VFPNegMul_VNMUL
13                     S[d] = Zeros(16) : FPNeg(product16);
14          when '10'
15              product32 = FPMul(S[n], S[m], TRUE);
16              case operation of
17                  when VFPNegMul_VNMLA
18                      S[d] = FPAdd(FPNeg(S[d]), FPNeg(product32), TRUE);
19                  when VFPNegMul_VNMMLS
20                      S[d] = FPAdd(FPNeg(S[d]), product32, TRUE);
21                  when VFPNegMul_VNMUL
22                      S[d] = FPNeg(product32);
23          when '11'
24              product64 = FPMul(D[n], D[m], TRUE);
25              case operation of
26                  when VFPNegMul_VNMLA
27                      D[d] = FPAdd(FPNeg(D[d]), FPNeg(product64), TRUE);
28                  when VFPNegMul_VNMMLS
29                      D[d] = FPAdd(FPNeg(D[d]), product64, TRUE);
30                  when VFPNegMul_VNMUL
31                      D[d] = FPNeg(product64);

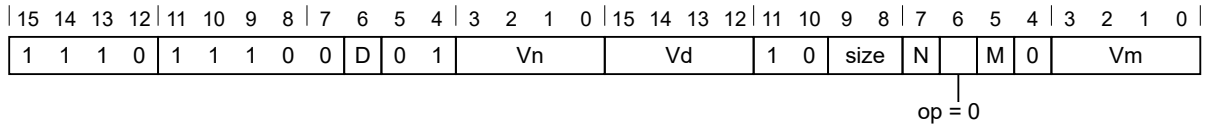
```

C2.4.420 VNMLS

Floating-point Multiply Subtract and Negate. Floating-point Multiply Subtract and Negate multiplies two floating-point register values, adds the negation of the floating-point value in the destination register to the product, and writes the result back to the destination register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.



Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VNMLS{<c>}{<q>}.F16 <Sd>, <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VNMLS{<c>}{<q>}.F32 <Sd>, <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VNMLS{<c>}{<q>}.F64 <Dd>, <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 operation = if op == '1' then VFPNegMul_VNMLA else VFPNegMul_VNMLS;
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              product16 = FPMul(S[n][15:0], S[m][15:0], TRUE);
7              case operation of
8                  when VFPNegMul_VNMLA
9                      S[d] = Zeros(16) : FPAdd(FPNeg(S[d][15:0]), FPNeg(product16), TRUE);
10                 when VFPNegMul_VNMMLS
11                     S[d] = Zeros(16) : FPAdd(FPNeg(S[d][15:0]), product16, TRUE);
12                 when VFPNegMul_VNMUL
13                     S[d] = Zeros(16) : FPNeg(product16);
14             when '10'
15                 product32 = FPMul(S[n], S[m], TRUE);
16                 case operation of
17                     when VFPNegMul_VNMLA
18                         S[d] = FPAdd(FPNeg(S[d]), FPNeg(product32), TRUE);
19                     when VFPNegMul_VNMMLS
20                         S[d] = FPAdd(FPNeg(S[d]), product32, TRUE);
21                     when VFPNegMul_VNMUL
22                         S[d] = FPNeg(product32);
23             when '11'
24                 product64 = FPMul(D[n], D[m], TRUE);
25                 case operation of
26                     when VFPNegMul_VNMLA
27                         D[d] = FPAdd(FPNeg(D[d]), FPNeg(product64), TRUE);
28                     when VFPNegMul_VNMMLS
29                         D[d] = FPAdd(FPNeg(D[d]), product64, TRUE);
30                     when VFPNegMul_VNMUL
31                         D[d] = FPNeg(product64);

```

C2.4.421 VNMUL

Floating-point Multiply and Negate. Floating-point Multiply and Negate multiplies two floating-point register values, and writes the negation of the result to the destination register.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	1	0	Vn				Vd				1	0	size		N	1	M	0	Vm			

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VNMUL{<c>}{<q>}.F16 <Sd>, <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VNMUL{<c>}{<q>}.F32 {<Sd>, } <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VNMUL{<c>}{<q>}.F64 {<Dd>, } <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 operation = VFPNegMul_VNMUL;
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then

```

```

2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              product16 = FPMul(S[n][15:0], S[m][15:0], TRUE);
7              case operation of
8                  when VFPNegMul_VNMLA
9                      S[d] = Zeros(16) : FPAdd(FPNeg(S[d][15:0]), FPNeg(product16), TRUE);
10                 when VFPNegMul_VNMMLS
11                     S[d] = Zeros(16) : FPAdd(FPNeg(S[d][15:0]), product16, TRUE);
12                 when VFPNegMul_VNMUL
13                     S[d] = Zeros(16) : FPNeg(product16);
14             when '10'
15                 product32 = FPMul(S[n], S[m], TRUE);
16                 case operation of
17                     when VFPNegMul_VNMLA
18                         S[d] = FPAdd(FPNeg(S[d]), FPNeg(product32), TRUE);
19                     when VFPNegMul_VNMMLS
20                         S[d] = FPAdd(FPNeg(S[d]), product32, TRUE);
21                     when VFPNegMul_VNMUL
22                         S[d] = FPNeg(product32);
23             when '11'
24                 product64 = FPMul(D[n], D[m], TRUE);
25                 case operation of
26                     when VFPNegMul_VNMLA
27                         D[d] = FPAdd(FPNeg(D[d]), FPNeg(product64), TRUE);
28                     when VFPNegMul_VNMMLS
29                         D[d] = FPAdd(FPNeg(D[d]), product64, TRUE);
30                     when VFPNegMul_VNMUL
31                         D[d] = FPNeg(product64);

```

C2.4.422 VORN

Vector Bitwise Or Not. Compute a bitwise OR NOT of a vector register with another vector register. The result is written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	1	1	Qn	0	Qd	0	0	0	0	1	N	1	M	1	Qm	0						

T1: VORN variant

VORN<v><q>{.<dt>} Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 n = UInt(N:Qn);
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	An optional data type. It is ignored by assemblers and does not affect the encoding. This can be one of the following: S8, S16, S32, U8, U16, U32, I8, I16, I32, F16, F32.
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Q[n, curBeat] OR NOT(Q[m, curBeat]);
7
8 for e = 0 to 3
9     if elmtMask[e] == '1' then
10         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.423 VORN (immediate)

Vector Bitwise OR NOT. This is a pseudo-instruction, equivalent to a VORR (immediate) instruction with the immediate value bitwise inverted.

This is an alias of [VORR \(immediate\)](#).

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	i	1	1	1	1	1	Da	0	0	0	imm3			Qda	0	cmode			0	1	0	1	imm4						

VORN variant

VORN<v><q>.<dt> Qda, #<imm>

is equivalent to

VORR<v><q>.<dt> Qda, #~<imm>

and is never the preferred disassembly

C2.4.424 VORR

Vector Bitwise Or. Compute a bitwise OR of a vector register with another vector register. The result is written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	1	0	Qn	0	Qd	0	0	0	0	1	N	1	M	1	Qm	0						

T1: VORR variant

VORR<v><q>{.<dt>} Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 n = UInt(N:Qn);
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	An optional data type. It is ignored by assemblers and does not affect the encoding. This can be one of the following: S8, S16, S32, U8, U16, U32, I8, I16, I32, F16, F32.
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Q[n, curBeat] OR Q[m, curBeat];
7
8 for e = 0 to 3
9     if elmtMask[e] == '1' then
10         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.

- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.425 VORR (immediate)

Vector Bitwise OR. OR the value of a vector register with the immediate operand value. The immediate is generated by the AdvSIMDExpandImm() function based on the requested data type and immediate value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	i	1	1	1	1	1	Da	0	0	0	imm3	Qda	0	cmode	0	1	0	1	imm4										

T1: VORR variant

VORR<v><q>.<dt> Qda, #<imm>

Decode for this encoding

```

1 if cmode IN {'xxx0', '11x1'} then SEE "VMOV (immediate) (vector)";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' then UNDEFINED;
4 da = UInt(Da:Qda);
5 imm64 = AdvSIMDExpandImm('1', cmode, i:imm3:imm4);
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector, for use with the AdvSIMDExpandImm() function. This parameter must be one of the following values: I32 Encoded as: cmode = 0001 cmode = 0011 cmode = 0101 cmode = 0111 I16 Encoded as: cmode = 1001 cmode = 1011
<Qda>	Source and destination vector register.
<imm>	The immediate value to load in to each element. This must be an immediate that can be encoded for use with the AdvSIMDExpandImm() function.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 opd = Q[da, curBeat];
7 imm32 = if curBeat[0] == '0' then imm64[31:0] else imm64[63:32];
8 result = opd OR imm32;
9
```

```
10 for e = 0 to 3
11     if elmtMask[e] == '1' then
12         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.426 VPNOT

Vector Predicate NOT. Inverts the predicate condition in VPR.P0. The VPR.P0 flags for predicated lanes are zeroed.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	0	1	1	(0)	(0)	(0)	1	0	0	0	(0)	1	1	1	1	(0)	1	(0)	0	1	1	0	1

T1: VPNOT variant

VPNOT<v><q>

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_Mve);
2
3 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 Elem[VPR.P0, curBeat, 4] = (NOT Elem[VPR.P0, curBeat, 4]) AND elmtMask;
```

C2.4.427 VPOP

Pop Floating-point registers from stack. Pop Floating-point registers from stack loads multiple consecutive Floating-point registers from the stack.

This instruction is interrupt-continuable.

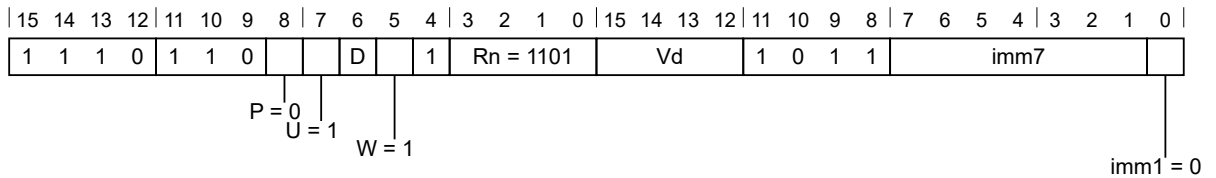
This instruction is subject to stack limit checking.

This instruction is an alias of the [VLDM](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [VLDM](#).
- The description of [VLDM](#) gives the operational pseudocode for this instruction.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE



Increment After variant

VPOP{<c>}{<q>}{.<size>} <dreglist>

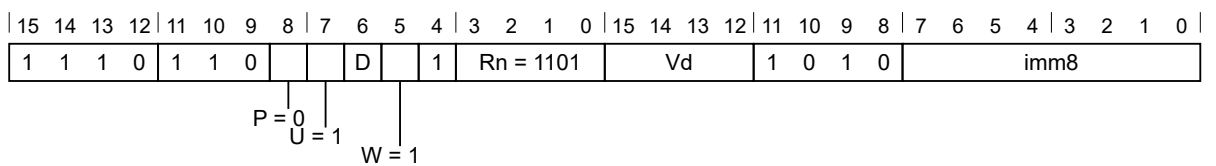
is equivalent to

VLDM{<c>}{<q>}{.<size>} SP!, <dreglist>

and is always the preferred disassembly.

T2

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE



Increment After variant

VPOP{<c>}{<q>}{.<size>} <sreglist>

is equivalent to

VLDM{<c>}{<q>}{.<size>} SP!, <sreglist>

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<size>	An optional data size specifier. If present, it must be equal to the size in bits, 32 or 64, of the registers being transferred.

- `<sreglist>` Is the list of consecutively numbered 32-bit floating-point registers to be transferred. The first register in the list is encoded in "Vd:D", and "imm8" is set to the number of registers in the list. The list must contain at least one register.
- `<dreglist>` Is the list of consecutively numbered 64-bit floating-point registers to be transferred. The first register in the list is encoded in "D:Vd", and "imm8" is set to twice the number of registers in the list. The list must contain at least one register.

Operation for all encodings

The description of [VLDM](#) gives the operational pseudocode for this instruction.

C2.4.428 VPSEL

Vector Predicated Select. Compute a bitwise conditional select of a vector register with another vector register, based on the VPR predicate bits

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	D	1	1	Qn	1	Qd	0	1	1	1	1	N	0	M	0	Qm	1						

T1: VPSEL variant

VPSEL<v><q>{.<dt>} Qd, Qn, Qm

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 n = UInt(N:Qn);
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	An optional data type. It is ignored by assemblers and does not affect the encoding. This can be one of the following: S8, S16, S32, U8, U16, U32, I8, I16, I32, F16, F32.
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 opm = Q[m, curBeat];
8 opn = Q[n, curBeat];
9 vpr = Elem[VPR.P0, curBeat, 4];
10 for e = 0 to 3
11     Elem[result, e, 8] = Elem[if vpr[e] == '1' then opn else opm, e, 8];
12
13 for e = 0 to 3
14     if elmtMask[e] == '1' then
15         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1 and this instruction sets the FPSCR.N, Z, C, V, Q flags or VPR.P0:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V, Q flags or VPR.P0.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V, Q flags or VPR.P0.

C2.4.429 VPST

Vector Predicate Set Then. Predicates the following instructions, up to a maximum of four instructions. This instruction is similar to VPT. However no comparison is performed and instead the current value of VPR.P0 is used as the predicate condition.

This instruction is subject to beat-wise execution.

This instruction is not VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	Mkh	1	1	(0)	(0)	(0)	1	Mkl	(0)	1	1	1	1	(0)	1	(0)	0	1	1	0	1		

T1: VPST variant

VPST{x{y{z}}}

Decode for this encoding

```

1 if Mkh:Mkl == '0000' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3
4 mask = Mkh:Mkl;
5 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
6 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<x>	Specifies the condition for an optional second instruction in the VPT block, and whether the condition is the same as for the first instruction (T) or its inverse (E). This is encoded in the mask field in a similar way to the IT instruction, except that rather than encoding T and E directly into f_cond[0], a 1 in the corresponding mask bit indicates that the previous predicate value in VPR.P0 should be inverted
<y>	Specifies the condition for an optional third instruction in the VPT block. It is encoded in the mask field in the same way as the x field.
<z>	Specifies the condition for an optional fourth instruction in the VPT block. It is encoded in the mask field in the same way as the x field.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, -) = GetCurInstrBeat();
5
6 // Only one mask field per pair of beats, so the mask is only updated on odd beats.
7 if curBeat[0] == '1' then
8     SetVPTMask(curBeat, mask);
```

C2.4.430 VPT

Vector Predicate Then. Predicates the following instructions, up to a maximum of four instructions, by masking the operation of instructions on a per-lane basis based on the VPR.P0 predicate values. The predicated instructions are referred to as the Vector Predication Block or simply the VPT Block. The VPR.P0 predicate values may be inverted after each instruction in the VPT block based on the mask fields (see x, y, and z).

This instruction is subject to beat-wise execution.

This instruction is not VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	Mkh	size		Qn	1		Mkl	0	1	1	1	1	fc	0	M	0		Qm				0	

T1: VPT variant

VPT{x{y{z}}}.<dt> <fc>, Qn, Qm

Decode for this encoding

```

1 if Mkh:Mkl == '0000' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 if M == '1' then UNDEFINED;
5 m = UInt(M:Qm);
6 n = UInt(Qn);
7 mask = Mkh:Mkl;
8 f_cond = '00':fc;
9 withScalar = FALSE;
10 esize = 8 << UInt(size);
11 elements = 32 DIV esize;
12 ebytes = esize DIV 8;
13 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
14 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	Mkh	size		Qn	1		Mkl	0	1	1	1	1	fc	0	M	0		Qm				1	

T2: VPT variant

VPT{x{y{z}}}.<dt> <fc>, Qn, Qm

Decode for this encoding

```

1 if Mkh:Mkl == '0000' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 if M == '1' then UNDEFINED;
5 m = UInt(M:Qm);
6 n = UInt(Qn);
```

```

7 mask      = Mkh:Mkl;
8 f_cond    = '01':fc;
9 withScalar = FALSE;
10 esize     = 8 << UInt(size);
11 elements  = 32 DIV esize;
12 ebytes    = esize DIV 8;
13 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
14 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;

```

T3

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	Mkh	size		Qn	1		Mkl	1	1	1	1	1	fcl	0	M	0		Qm	fch				

T3: VPT variant

VPT{x{y{z}}}.<dt> <fc>, Qn, Qm

Decode for this encoding

```

1 if Mkh:Mkl == '0000' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 if M == '1' then UNDEFINED;
5 m = UInt(M:Qm);
6 n = UInt(Qn);
7 mask = Mkh:Mkl;
8 f_cond = '1':fch:fcl;
9 withScalar = FALSE;
10 esize = 8 << UInt(size);
11 elements = 32 DIV esize;
12 ebytes = esize DIV 8;
13 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
14 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;

```

T4

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	Mkh	size		Qn	1		Mkl	0	1	1	1	1	fc	1	0	0		Rm					

T4: VPT variant

VPT{x{y{z}}}.<dt> <fc>, Qn, Rm

Decode for this encoding

```

1 if Mkh:Mkl == '0000' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 m = UInt(Rm);
5 n = UInt(Qn);
6 mask = Mkh:Mkl;
7 f_cond = '00':fc;
8 withScalar = TRUE;
9 esize = 8 << UInt(size);
10 elements = 32 DIV esize;
11 ebytes = esize DIV 8;

```

```
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
13 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
14 if Rm == '1101' then CONSTRAINED_UNPREDICTABLE;
```

T5

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	Mkh	size	Qn		1	Mkl	0	1	1	1	1	fc	1	1	0	Rm								

T5: VPT variant

VPT{x{y{z}}}.<dt> <fc>, Qn, Rm

Decode for this encoding

```
1 if Mkh:Mkl == '0000' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 m = UInt(Rm);
5 n = UInt(Qn);
6 mask = Mkh:Mkl;
7 f_cond = '01':fc;
8 withScalar = TRUE;
9 esize = 8 << UInt(size);
10 elements = 32 DIV esize;
11 ebytes = esize DIV 8;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
13 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
14 if Rm == '1101' then CONSTRAINED_UNPREDICTABLE;
```

T6

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	Mkh	size		Qn	1		Mkl	1	1	1	1	1	fcl	1	fch	0					Rm		

T6: VPT variant

VPT{x{y{z}}}.<dt> <fc>, Qn, Rm

Decode for this encoding

```
1 if Mkh:Mkl == '0000' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 m = UInt(Rm);
5 n = UInt(Qn);
6 mask = Mkh:Mkl;
7 f_cond = '1':fch:fcl;
8 withScalar = TRUE;
9 esize = 8 << UInt(size);
10 elements = 32 DIV esize;
11 ebytes = esize DIV 8;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
13 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
14 if Rm == '1101' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for T1 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
I8 Encoded as size = 00
I16 Encoded as size = 01
I32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
EQ Encoded as fc = 0
NE Encoded as fc = 1

Assembler symbols for T2 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
U8 Encoded as size = 00
U16 Encoded as size = 01
U32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
CS Encoded as fc = 0
HI Encoded as fc = 1

Assembler symbols for T3 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
S8 Encoded as size = 00
S16 Encoded as size = 01
S32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
GE Encoded as fch = 0, fcl = 0
LT Encoded as fch = 0, fcl = 1
GT Encoded as fch = 1, fcl = 0
LE Encoded as fch = 1, fcl = 1

Assembler symbols for T4 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
I8 Encoded as size = 00
I16 Encoded as size = 01
I32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:
EQ Encoded as fc = 0
NE Encoded as fc = 1

Assembler symbols for T5 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
U8 Encoded as size = 00
U16 Encoded as size = 01
U32 Encoded as size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:

CS	Encoded as	fc = 0
HI	Encoded as	fc = 1

Assembler symbols for T6 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:

S8	Encoded as	size = 00
S16	Encoded as	size = 01
S32	Encoded as	size = 10

<fc> The comparison condition to use.
This parameter must be one of the following values:

GE	Encoded as	fch = 0, fcl = 0
LT	Encoded as	fch = 0, fcl = 1
GT	Encoded as	fch = 1, fcl = 0
LE	Encoded as	fch = 1, fcl = 1

Assembler symbols for all encodings

<q> See [C1.2.5 Standard assembler syntax fields](#).
<Qn> Source vector register.
<Qm> Source vector register.
<Rm> Source general-purpose register (ZR is permitted, PC is not).
<x> Specifies the condition for an optional second instruction in the VPT block, and whether the condition is the same as for the first instruction (T) or its inverse (E). This is encoded in the mask field in a similar way to the IT instruction, except that rather than encoding T and E directly into f_cond[0], a 1 in the corresponding mask bit indicates that the previous predicate value in VPR.P0 should be inverted
<y> Specifies the condition for an optional third instruction in the VPT block. It is encoded in the mask field in the same way as the x field.
<z> Specifies the condition for an optional fourth instruction in the VPT block. It is encoded in the mask field in the same way as the x field.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1 = Q[n, curBeat];
7 beatPred = Zeros(4);
8 if withScalar then
9     op2 = RZ[m][esize-1:0];
10 else
11     opm = Q[m, curBeat];
12 for e = 0 to elements-1
13     if !withScalar then
14         op2 = Elem[opm, e, esize];
15     (result, flC, flV) = AddWithCarry(Elem[op1, e, esize], NOT(op2), '1');
16     flN = result[esize-1];
17     flZ = IsZeroBit(result);
18     pred = ConditionHolds(f_cond, flN, flZ, flC, flV);
19     Elem[beatPred, e, ebytes] = Replicate(if pred then '1' else '0');
20
21 Elem[VPR.P0, curBeat, 4] = beatPred AND elmtMask;
22 // Only one mask field per pair of beats, so the mask is only updated on odd beats.
23 if curBeat[0] == '1' then

```


24 `SetVPTMask(curBeat, mask);`

C2.4.431 VPT (floating-point)

Vector Predicate Then. Predicates the following instructions, up to a maximum of four instructions, by masking the operation of instructions on a per-lane basis based on the VPR.P0 predicate values. The predicated instructions are referred to as the Vector Predication Block or simply the VPT Block. The VPR.P0 predicate values may be inverted after each instruction in the VPT block based on the mask fields (see x, y, and z).

This instruction is subject to beat-wise execution.

This instruction is not VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	Mkh	1	1	Qn	1	Mkl	fcA	1	1	1	1	fcC	0	M	0	Qm	fcB						

T1: VPT variant

VPT{x{y{z}}}.<dt> <fc>, Qn, Qm

Decode for this encoding

```

1 if (fcA == '0' && fcB == '1') || Mkh:Mkl == '0000' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_MveFp);
3 if M == '1' then UNDEFINED;
4 m      = UInt (M:Qm);
5 n      = UInt (Qn);
6 mask   = Mkh:Mkl;
7 f_cond = fcA:fcB:fcC;
8 withScalar = FALSE;
9 esize   = 8 << UInt (if sz == '1' then '01' else '10');
10 elements = 32 DIV esize;
11 ebytes  = esize DIV 8;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
13 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	Mkh	1	1	Qn	1	Mkl	fcA	1	1	1	1	fcC	1	fcB	0	Rm							

T2: VPT variant

VPT{x{y{z}}}.<dt> <fc>, Qn, Rm

Decode for this encoding

```

1 if Mkh:Mkl == '0000' then SEE "Related encodings";
2 if Rm      == '1101' then SEE "Related encodings";
3 CheckDecodeFaults (ExtType_MveFp);
4 m      = UInt (Rm);
5 n      = UInt (Qn);
6 mask   = Mkh:Mkl;
7 f_cond = fcA:fcB:fcC;
```

```

8 withScalar = TRUE;
9 esize      = 8 << UInt(if sz == '1' then '01' else '10');
10 elements  = 32 DIV esize;
11 ebytes     = esize DIV 8;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
13 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
14 if fcA == '0' && fcB == '1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F32 Encoded as sz = 0 F16 Encoded as sz = 1
<fc>	The comparison condition to use. This parameter must be one of the following values: EQ Encoded as fcA = 0, fcB = 0, fcC = 0 NE Encoded as fcA = 0, fcB = 0, fcC = 1 GE Encoded as fcA = 1, fcB = 0, fcC = 0 LT Encoded as fcA = 1, fcB = 0, fcC = 1 GT Encoded as fcA = 1, fcB = 1, fcC = 0 LE Encoded as fcA = 1, fcB = 1, fcC = 1
<Qn>	Source vector register.
<Qm>	Source vector register.
<Rm>	Source general-purpose register (ZR is permitted, PC is not).
<x>	Specifies the condition for an optional second instruction in the VPT block, and whether the condition is the same as for the first instruction (T) or its inverse (E). This is encoded in the mask field in a similar way to the IT instruction, except that rather than encoding T and E directly into f_cond[0], a 1 in the corresponding mask bit indicates that the previous predicate value in VPR.P0 should be inverted
<y>	Specifies the condition for an optional third instruction in the VPT block. It is encoded in the mask field in the same way as the x field.
<z>	Specifies the condition for an optional fourth instruction in the VPT block. It is encoded in the mask field in the same way as the x field.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1 = Q[n, curBeat];
7 beatPred = Zeros(4);
8 if withScalar then
9     op2 = RZ[m][esize-1:0];
10 else
11     opm = Q[m, curBeat];
12 for e = 0 to elements-1
13     if !withScalar then
14         op2 = Elem[opm, e, esize];
15         // Avoid Floating-point exceptions on a predicated lane by checking the element mask
16         predicated = (elmtMask[e*(esize>>3)] == '0');
17         (flN, flZ, flC, flV) = FPCompare(Elem[op1, e, esize], op2, TRUE, FALSE, predicated);
18         pred = ConditionHolds(f_cond, flN, flZ, flC, flV);
19         Elem[beatPred, e, ebytes] = Replicate(if pred then '1' else '0');
20
21 Elem[VPR.P0, curBeat, 4] = beatPred AND elmtMask;
22 // Only one mask field per pair of beats, so the mask is only updated on odd beats.
23 if curBeat[0] == '1' then
24     SetVPTMask(curBeat, mask);

```

C2.4.432 VPUSH

Push Floating-point registers to stack. Push Floating-point registers to stack stores multiple consecutive registers from the Floating-point register file to the stack.

This instruction is interrupt-continuable.

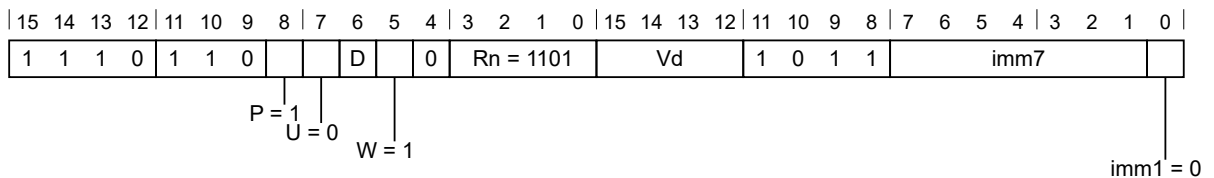
This instruction is subject to stack limit checking.

This instruction is an alias of the [VSTM](#) instruction. This means that:

- The encodings in this description are named to match the encodings of [VSTM](#).
- The description of [VSTM](#) gives the operational pseudocode for this instruction.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE



Decrement Before variant

VPUSH{<c>}{<q>}{.<size>} <dreglist>

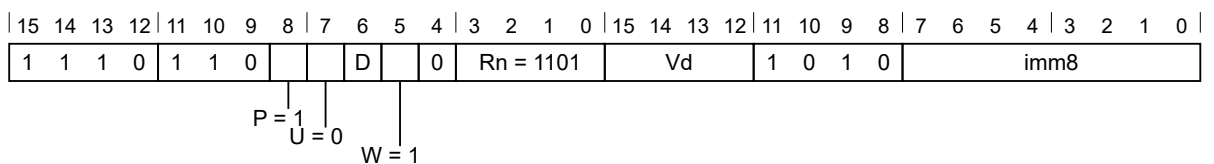
is equivalent to

VSTMDB{<c>}{<q>}{.<size>} SP!, <dreglist>

and is always the preferred disassembly.

T2

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE



Decrement Before variant

VPUSH{<c>}{<q>}{.<size>} <sreglist>

is equivalent to

VSTMDB{<c>}{<q>}{.<size>} SP!, <sreglist>

and is always the preferred disassembly.

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<size>	An optional data size specifier. If present, it must be equal to the size in bits, 32 or 64, of the registers being transferred.

- `<sreglist>` Is the list of consecutively numbered 32-bit floating-point registers to be transferred. The first register in the list is encoded in "Vd:D", and "imm8" is set to the number of registers in the list. The list must contain at least one register.
- `<dreglist>` Is the list of consecutively numbered 64-bit floating-point registers to be transferred. The first register in the list is encoded in "D:Vd", and "imm8" is set to twice the number of registers in the list. The list must contain at least one register.

Operation for all encodings

The description of [VSTM](#) gives the operational pseudocode for this instruction.

C2.4.433 VQABS

Vector Saturating Absolute. Compute the absolute value of and saturate each element of a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	0		Qd	0	0	1	1	1	0	1	M	0	Qm	0				

T1: VQABS variant

VQABS<v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 esize = 8 << UInt(size);
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 S8 Encoded as size = 00
 S16 Encoded as size = 01
 S32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[m, curBeat];
8 for e = 0 to elements-1
9     value = Abs(SInt(Elm[op1, e, esize]));
10    (Elm[result, e, esize], sat) = SignedSatQ(value, esize);
11    if sat && elmtMask[e*(esize>>3)] == '1' then
12        FPSCR.QC = '1';
13
14 for e = 0 to 3
15     if elmtMask[e] == '1' then
16         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.434 VQADD

Vector Saturating Add. Add the value of the elements in the first source vector register to either the respective elements in the second source vector register or a general-purpose register. The result is saturated before being written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn	0		Qd	0	0	0	0	0	0	0	N	1	M	1		Qm	0		

T1: VQADD variant

VQADD<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (M:Qm);
6 n = UInt (N:Qn);
7 esize = 8 << UInt (size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 withScalar = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	D	size		Qn	0		Qd	0	1	1	1	1	N	1	1	1	0				Rm		

T2: VQADD variant

VQADD<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_Mve);
3 if D == '1' || N == '1' then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (Rm);
6 n = UInt (N:Qn);
7 esize = 8 << UInt (size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 withScalar = TRUE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
12 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	This parameter determines the following values: – Size: indicates the size of the elements in the vector. – Unsigned flag: S indicates signed, U indicates unsigned. This parameter must be one of the following values: S8 Encoded as size = 00, U = 0 U8 Encoded as size = 00, U = 1 S16 Encoded as size = 01, U = 0 U16 Encoded as size = 01, U = 1 S32 Encoded as size = 10, U = 0 U32 Encoded as size = 10, U = 1
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.
<Rm>	Source general-purpose register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1    = Q[n, curBeat];
8  if !withScalar then
9      op2 = Q[m, curBeat];
10 for e = 0 to elements-1
11     if withScalar then
12         value = Int(Elem[op1, e, esize], unsigned) + Int(R[m][esize-1:0], unsigned);
13     else
14         value = Int(Elem[op1, e, esize], unsigned) + Int(Elem[op2, e, esize], unsigned);
15     (Elem[result, e, esize], sat) = SatQ(value, esize, unsigned);
16     if sat && elmtMask[e*(esize>>3)] == '1' then
17         FPSCR.QC = '1';
18
19 for e = 0 to 3
20     if elmtMask[e] == '1' then
21         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```


C2.4.435 VQDMLADH, VQRDMLADH

Vector Saturating Doubling Multiply Add Dual Returning High Half, Vector Saturating Rounding Doubling Multiply Add Dual Returning High Half. The elements of the vector registers are handled in pairs. In the base variant, corresponding elements from the two source registers are multiplied together, whereas the exchange variant swaps the values in each pair of values read from the first source register, before multiplying them with the values from the second source register. The results of the pairs of multiply operations are combined by adding them together and doubling the result. The high halves of the resulting values are selected as the final results. The base variant writes the results into the lower element of each pair of elements in the destination register, whereas the exchange variant writes to the upper element in each pair. The results are optionally rounded before the high half is selected and saturated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	size		Qn	0	Qd	X	1	1	1	0	N	0	M	0					Qm	0		

T1: VQDMLADH variant

VQDMLADH{X}<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 exchange = (X == '1');
8 esize   = 8 << UInt(size);
9 elements = 32 DIV esize;
10 round   = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	size		Qn	0	Qd	X	1	1	1	0	N	0	M	0		Qm					1	

T2: VQRDMLADH variant

VQRDMLADH{X}<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;

```

```

4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 exchange = (X == '1');
8 esize   = 8 << UInt(size);
9 elements = 32 DIV esize;
10 round   = TRUE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<X>	Exchange adjacent pairs of values in Qm. This parameter must be one of the following values: – Encoded as X = 0 X Encoded as X = 1
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: S8 Encoded as size = 00 S16 Encoded as size = 01 S32 Encoded as size = 10
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 rVal = if round then 1 << (esize-1) else 0;
8 // 32 bit operations are handled differently as they perform cross beat
9 // register accesses
10 if esize == 32 then
11     if (curBeat[0] == '1') == exchange then
12         if exchange then
13             mul1 = SInt(Q[n, curBeat]) * SInt(Q[m, curBeat-1]);
14             mul2 = SInt(Q[n, curBeat-1]) * SInt(Q[m, curBeat]);
15         else
16             mul1 = SInt(Q[n, curBeat]) * SInt(Q[m, curBeat]);
17             mul2 = SInt(Q[n, curBeat+1]) * SInt(Q[m, curBeat+1]);
18             (value, sat) = SignedSatQ(2 * (mul1 + mul2) + rVal, esize*2);
19             result = value[63:32];
20             if sat && elmtMask[0] == '1' then
21                 FPSCR.QC = '1';
22         else
23             // No computation on this beat, so don't write to the dest register.
24             elmtMask = Zeros();
25     else
26         op1 = Q[n, curBeat];
27         op2 = Q[m, curBeat];
28         for e = 0 to elements-1
29             if (e[0] == '1') == exchange then
30                 if exchange then
31                     mul1 = SInt(Elem[op1, e, esize]) * SInt(Elem[op2, e-1, esize]);
32                     mul2 = SInt(Elem[op1, e-1, esize]) * SInt(Elem[op2, e, esize]);
33                 else
34                     mul1 = SInt(Elem[op1, e, esize]) * SInt(Elem[op2, e, esize]);
35                     mul2 = SInt(Elem[op1, e+1, esize]) * SInt(Elem[op2, e+1, esize]);
36                     (value, sat) = SignedSatQ(2 * (mul1 + mul2) + rVal, esize*2);
37                     Elem[result, e, esize] = value[esize+esize-1:esize];
38                     if sat && elmtMask[e*(esize>>3)] == '1' then

```

```
39         FPSCR.QC = '1';
40     else
41         // No computation on this lane, so assign original value.
42         Elem[result, e, esize] = Elem[Q[d, curBeat], e, esize];
43
44 for e = 0 to 3
45     if elmtMask[e] == '1' then
46         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.436 VQDMLAH, VQRDMLAH (vector by scalar plus vector)

Vector Saturating Doubling Multiply Accumulate, Vector Saturating Rounding Doubling Multiply Accumulate. Multiply each element in the source vector by a scalar value, double the result and add to the respective element from the destination vector High Half. Store the high half of each result in the destination register. The result is optionally rounded before the high half is selected and saturated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	0	Da	size		Qn	0			Qda	0	1	1	1	0	N	1	1	0					Rm	

T1: VQDMLAH variant

VQDMLAH<v><q>.<dt> Qda, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_Mve);
3 if Da == '1' || N == '1' then UNDEFINED;
4 da      = UInt (Da:Qda);
5 m      = UInt (Rm);
6 n      = UInt (N:Qn);
7 esize  = 8 << UInt (size);
8 elements = 32 DIV esize;
9 round  = FALSE;
10 if InITBlock () then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	0	Da	size		Qn	0			Qda	0	1	1	1	0	N	1	0	0					Rm	

T2: VQRDMLAH variant

VQRDMLAH<v><q>.<dt> Qda, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_Mve);
3 if Da == '1' || N == '1' then UNDEFINED;
4 da      = UInt (Da:Qda);
5 m      = UInt (Rm);
6 n      = UInt (N:Qn);
7 esize  = 8 << UInt (size);
8 elements = 32 DIV esize;
9 round  = TRUE;
```

```
10 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 S8 Encoded as size = 00
 S16 Encoded as size = 01
 S32 Encoded as size = 10
 <Qda> Accumulator vector register.
 <Qn> Source vector register.
 <Rm> Source general-purpose register.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[n, curBeat];
8 element2 = SInt(R[m][esize-1:0]);
9 op3 = Q[da, curBeat];
10 rVal = if round then 1 << (esize-1) else 0;
11 for e = 0 to elements-1
12     element1 = SInt(Elem[op1, e, esize]);
13     element3 = SInt(Elem[op3, e, esize]) << esize;
14     (value, sat) = SignedSatQ((2 * element1 * element2) + element3 + rVal, esize*2);
15     Elem[result, e, esize] = value[esize+esize-1:esize];
16     if sat && elmtMask[e*(esize>>3)] == '1' then
17         FPSCR.QC = '1';
18
19 for e = 0 to 3
20     if elmtMask[e] == '1' then
21         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1 and this instruction sets the FPSCR.N, Z, C, V, Q flags or VPR.P0:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V, Q flags or VPR.P0.
- The point at which an asynchronous exception is taken does not vary based on any of the following:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V, Q flags or VPR.P0.

C2.4.437 VQDMLASH, VQRDMLASH (vector by vector plus scalar)

Vector Saturating Doubling Multiply Accumulate Scalar High Half, Vector Saturating Rounding Doubling Multiply Accumulate Scalar High Half. Multiply each element in the source vector by the respective element from the destination vector, double the result and add to a scalar value. Store the high half of each result in the destination register. The result is optionally rounded before the high half is selected and saturated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	0	Da	size		Qn	0			Qda	1	1	1	1	0	N	1	1	0					Rm	

T1: VQDMLASH variant

VQDMLASH<v><q>.<dt> Qda, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' || N == '1' then UNDEFINED;
4 da      = UInt(Da:Qda);
5 m      = UInt(Rm);
6 n      = UInt(N:Qn);
7 esize  = 8 << UInt(size);
8 elements = 32 DIV esize;
9 round  = FALSE;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	1	0	0	Da	size		Qn	0			Qda	1	1	1	1	0	N	1	0	0					Rm	

T2: VQRDMLASH variant

VQRDMLASH<v><q>.<dt> Qda, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' || N == '1' then UNDEFINED;
4 da      = UInt(Da:Qda);
5 m      = UInt(Rm);
6 n      = UInt(N:Qn);
7 esize  = 8 << UInt(size);
8 elements = 32 DIV esize;
9 round  = TRUE;

```

```
10 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 S8 Encoded as size = 00
 S16 Encoded as size = 01
 S32 Encoded as size = 10
 <Qda> Source and destination vector register.
 <Qn> Source vector register.
 <Rm> Source general-purpose register.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[n, curBeat];
8 op2 = Q[da, curBeat];
9 element3 = SInt(R[m][esize-1:0]) << esize;
10 rVal = if round then 1 << (esize-1) else 0;
11 for e = 0 to elements-1
12     element1 = SInt(Elem[op1, e, esize]);
13     element2 = SInt(Elem[op2, e, esize]);
14     (value, sat) = SignedSatQ((2 * element1 * element2) + element3 + rVal, esize*2);
15     Elem[result, e, esize] = value[esize+esize-1:esize];
16     if sat && elmtMask[e*(esize>>3)] == '1' then
17         FPSCR.QC = '1';
18
19 for e = 0 to 3
20     if elmtMask[e] == '1' then
21         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.438 VQDMLSDH, VQRDMLSDH

Vector Saturating Doubling Multiply Subtract Dual Returning High Half, Vector Saturating Rounding Doubling Multiply Subtract Dual Returning High Half. The elements of the vector registers are handled in pairs. In the base variant, corresponding elements from the two source registers are multiplied together, whereas the exchange variant swaps the values in each pair of values read from the first source register, before multiplying them with the values from the second source register. The results of the pairs of multiply operations are combined by subtracting one from the other and doubling the result. The high halves of the resulting values are selected as the final results. The base variant writes the results into the lower element of each pair of elements in the destination register, whereas the exchange variant writes to the upper element in each pair. The results are optionally rounded before the high half is selected and saturated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	D	size		Qn		0		Qd	X	1	1	1	0	N	0	M	0		Qm		0		

T1: VQDMLSDH variant

VQDMLSDH{X}<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 exchange = (X == '1');
8 esize   = 8 << UInt(size);
9 elements = 32 DIV esize;
10 round   = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	D	size		Qn	0		Qd	X	1	1	1	0	N	0	M	0		Qm		1			

T2: VQRDMLSDH variant

VQRDMLSDH{X}<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;

```



```

4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 exchange = (X == '1');
8 esize   = 8 << UInt(size);
9 elements = 32 DIV esize;
10 round  = TRUE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<X>	Exchange adjacent pairs of values in Qm. This parameter must be one of the following values: – Encoded as X = 0 X Encoded as X = 1
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: S8 Encoded as size = 00 S16 Encoded as size = 01 S32 Encoded as size = 10
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 rVal = if round then 1 << (esize-1) else 0;
8 // 32 bit operations are handled differently as they perform cross beat
9 // register accesses
10 if esize == 32 then
11     if (curBeat[0] == '1') == exchange then
12         if exchange then
13             mul1 = SInt(Q[n, curBeat]) * SInt(Q[m, curBeat-1]);
14             mul2 = SInt(Q[n, curBeat-1]) * SInt(Q[m, curBeat]);
15         else
16             mul1 = SInt(Q[n, curBeat]) * SInt(Q[m, curBeat]);
17             mul2 = SInt(Q[n, curBeat+1]) * SInt(Q[m, curBeat+1]);
18             (value, sat) = SignedSatQ(2 * (mul1 - mul2) + rVal, esize*2);
19             result = value[63:32];
20             if sat && elmtMask[0] == '1' then
21                 FPSCR.QC = '1';
22         else
23             // No computation on this beat, so don't write to the dest register.
24             elmtMask = Zeros();
25     else
26         op1 = Q[n, curBeat];
27         op2 = Q[m, curBeat];
28         for e = 0 to elements-1
29             if (e[0] == '1') == exchange then
30                 if exchange then
31                     mul1 = SInt(Elem[op1, e, esize]) * SInt(Elem[op2, e-1, esize]);
32                     mul2 = SInt(Elem[op1, e-1, esize]) * SInt(Elem[op2, e, esize]);
33                 else
34                     mul1 = SInt(Elem[op1, e, esize]) * SInt(Elem[op2, e, esize]);
35                     mul2 = SInt(Elem[op1, e+1, esize]) * SInt(Elem[op2, e+1, esize]);
36                     (value, sat) = SignedSatQ(2 * (mul1 - mul2) + rVal, esize*2);
37                     Elem[result, e, esize] = value[esize+esize-1:esize];
38                     if sat && elmtMask[e*(esize>>3)] == '1' then

```

```
39         FPSCR.QC = '1';
40     else
41         // No computation on this lane, so assign original value.
42         Elem[result, e, esize] = Elem[Q[d, curBeat], e, esize];
43
44 for e = 0 to 3
45     if elmtMask[e] == '1' then
46         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.439 VQDMULH, VQRDMULH

Vector Saturating Doubling Multiply Returning High Half, Vector Saturating Rounding Doubling Multiply Returning High Half. Multiply a general-purpose register value by each element of a vector register to produce a vector of results or multiply each element of a vector register by its corresponding element in another vector register, double the results, and place the most significant half of the final results in the destination vector. The results are optionally rounded before being saturated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	size		Qn	0		Qd	0	1	0	1	1	N	1	M	0		Qm	0				

T1: VQDMULH variant

VQDMULH<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(N:Qn);
7 esize = 8 << UInt(size);
8 elements = 32 DIV esize;
9 withScalar = FALSE;
10 round = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	0	D	size		Qn	0		Qd	0	1	0	1	1	N	1	M	0		Qm	0				

T2: VQRDMULH variant

VQRDMULH<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(N:Qn);
7 esize = 8 << UInt(size);
8 elements = 32 DIV esize;

```

```

9  withScalar = FALSE;
10 round      = TRUE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T3

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	size		Qn	1		Qd	0	1	1	1	0	N	1	1	0					Rm		

T3: VQDMULH variant

VQDMULH<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1  if size == '11' then SEE "Related encodings";
2  CheckDecodeFaults(ExtType_Mve);
3  if D == '1' || N == '1' then UNDEFINED;
4  d      = UInt(D:Qd);
5  m      = UInt(Rm);
6  n      = UInt(N:Qn);
7  esize   = 8 << UInt(size);
8  elements = 32 DIV esize;
9  withScalar = TRUE;
10 round    = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
12 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

T4

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	D	size		Qn	1		Qd	0	1	1	1	0	N	1	1	0					Rm		

T4: VQRDMULH variant

VQRDMULH<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1  if size == '11' then SEE "Related encodings";
2  CheckDecodeFaults(ExtType_Mve);
3  if D == '1' || N == '1' then UNDEFINED;
4  d      = UInt(D:Qd);
5  m      = UInt(Rm);
6  n      = UInt(N:Qn);
7  esize   = 8 << UInt(size);
8  elements = 32 DIV esize;
9  withScalar = TRUE;
10 round    = TRUE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
12 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<Qn> First source vector register.

Assembler symbols for T2 encodings

<Qn> First source vector register.

Assembler symbols for T3 encodings

<Qn> Source vector register.

Assembler symbols for T4 encodings

<Qn> Source vector register.

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
S8 Encoded as size = 00
S16 Encoded as size = 01
S32 Encoded as size = 10
<Qd> Destination vector register.
<Qm> Second source vector register.
<Rm> Source general-purpose register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[n, curBeat];
8 op2 = if withScalar then R[m] else Q[m, curBeat];
9 rVal = if round then 1 << (esize-1) else 0;
10 for e = 0 to elements-1
11     opm = if withScalar then op2[esize-1:0] else Elem[op2, e, esize];
12     value = ((2 * SInt(Elem[op1, e, esize]) * SInt(opm)) + rVal) >> esize;
13     (Elem[result, e, esize], sat) = SignedSatQ(value, esize);
14     if sat && elmtMask[e*(esize>>3)] == '1' then
15         FPSCR.QC = '1';
16
17 for e = 0 to 3
18     if elmtMask[e] == '1' then
19         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1 and this instruction sets the FPSCR.N, Z, C, V, Q flags or VPR.P0:

- The execution time of this instruction is independent of:
 - The values of the data supplied in any of its registers.
 - The values of the N, Z, C, V, Q flags or VPR.P0.
- The point at which an asynchronous exception is taken does not vary based on any of the following:

- The values of the data supplied in any of its registers.
- The values of the N, Z, C, V, Q flags or VPR.P0.

C2.4.440 VQDMULL

Vector Multiply Long. Performs an element-wise integer multiplication of two single-width source operand elements. These are selected from either the top half (T variant) or bottom half (B variant) of double-width source vector register elements or the lower single-width portion of the general-purpose register. The product of the multiplication is doubled and saturated to produce a double-width product that is written back to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	D	1	1	Qn	0	Qd	T	1	1	1	1	N	0	M	0	Qm	1						

T1: VQDMULL variant

VQDMULL<T><v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (M:Qm);
5 n      = UInt (N:Qn);
6 esize  = 16 << UInt (sz);
7 top    = UInt (T);
8 withScalar = FALSE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if sz == '1' && (D:Qd == M:Qm || D:Qd == N:Qn) then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	D	1	1	Qn	0	Qd	T	1	1	1	1	N	1	1	0	Rm							

T2: VQDMULL variant

VQDMULL<T><v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (Rm);
5 n      = UInt (N:Qn);
6 esize  = 16 << UInt (sz);
7 top    = UInt (T);
8 withScalar = TRUE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

```
10 if D:Qd == N:Qn && sz == '1' then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T>	Specifies which half of the source element is used. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: S16 Encoded as sz = 0 S32 Encoded as sz = 1
<Qd>	Destination vector register.
<Qn>	Source vector register.
<Qm>	Source vector register.
<Rm>	Source general-purpose register.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 if esize == 32 then
8     op1 = Q[n, UInt(curBeat[1]:T)];
9     if withScalar then
10         op2 = R[m];
11     else
12         op2 = Q[m, UInt(curBeat[1]:T)];
13     (mul, sat) = SignedSatQ(2 * SInt(op1) * SInt(op2), 64);
14     result = if curBeat[0] == '1' then mul[63:32] else mul[31:0];
15     if sat && elmtMask[0] == '1' then
16         FPSCR.QC = '1';
17 else
18     op1 = Q[n, curBeat];
19     if withScalar then
20         op2 = R[m];
21     else
22         op2 = Q[m, curBeat];
23     elements = 16 DIV esize;
24     for e = 0 to elements-1
25         element1 = Elem[op1, e * 2 + top, esize];
26         if withScalar then
27             element2 = Elem[op2, 0, esize];
28         else
29             element2 = Elem[op2, e * 2 + top, esize];
30         value = 2 * SInt(element1) * SInt(element2);
31         (Elem[result, e, esize * 2], sat) = SignedSatQ(value, 2 * esize);
32         if sat && elmtMask[(e*2 + top) * (esize>>3)] == '1' then
33             FPSCR.QC = '1';
34
35 for e = 0 to 3
36     if elmtMask[e] == '1' then
37         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```


C2.4.441 VQMOVN

Vector Saturating Move and Narrow. Performs an element-wise saturation to half-width, writing the result to either the top half (T variant) or bottom half (B variant) of the result element. The other half of the destination vector element retains its previous value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	D	1	1	size	1	1		Qd	T	1	1	1	0	0	0	M	0		Qm			1	

T1: VQMOVN variant

VQMOVN<T><v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' then UNDEFINED;
4 if size == '10' then UNDEFINED;
5 d = UInt(D:Qd);
6 m = UInt(M:Qm);
7 unsigned = (U == '1');
8 esize = 8 << UInt(size);
9 elements = 16 DIV esize;
10 top = UInt(T);
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<T>	Specifies which half of the result element the result is written to. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q> <dt>	See C1.2.5 Standard assembler syntax fields . This parameter determines the following values: – Unsigned flag: S indicates signed, U indicates unsigned. – Size: indicates the size of the elements in the vector. This parameter must be one of the following values: S16 Encoded as size = 00, U = 0 U16 Encoded as size = 00, U = 1 S32 Encoded as size = 01, U = 0 U32 Encoded as size = 01, U = 1
<Qd>	Destination vector register.
<Qm>	Source vector register.

Operation for all encodings

```
1 EncodingSpecificOperations();
```

```

2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  op1    = Q[m, curBeat];
7  result = Q[d, curBeat];
8  for e = 0 to elements-1
9      operand = Int(Elem[op1, e, 2*esize], unsigned);
10     (value, sat) = SatQ(operand, esize, unsigned);
11     Elem[result, 2*e + top, esize] = value;
12     if sat && elmtMask[(e*2 + top) * (esize>>3)] == '1' then
13         FPSCR.QC = '1';
14
15 for e = 0 to 3
16     if elmtMask[e] == '1' then
17         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.442 VQMOVUN

Vector Saturating Move Unsigned and Narrow. Performs an element-wise saturation to half-width, writing the result to either the top half (T variant) or bottom half (B variant) of the result element. The other half of the destination vector element retains its previous value. The result is always saturated to an unsigned value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	1	1	size	0	1		Qd	T	1	1	1	0	1	0	M	0		Qm		1		

T1: VQMOVUN variant

VQMOVUN<T><v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' then UNDEFINED;
4 if size == '10' then UNDEFINED;
5 d = UInt(D:Qd);
6 m = UInt(M:Qm);
7 unsigned = FALSE;
8 destUnsigned = TRUE;
9 esize = 8 << UInt(size);
10 elements = 16 DIV esize;
11 top = UInt(T);
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T>	Specifies which half of the result element the result is written to. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: S16 Encoded as size = 00 S32 Encoded as size = 01
<Qd>	Destination vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
```

```

5
6 op1    = Q[m, curBeat];
7 result = Q[d, curBeat];
8 for e = 0 to elements-1
9     operand = Int(Elem[op1, e, 2*esize], unsigned);
10    (value, sat) = SatQ(operand, esize, destUnsigned);
11    Elem[result, 2*e + top, esize] = value;
12    if sat && elmtMask[(e*2 + top) * (esize>>3)] == '1' then
13        FPSCR.QC = '1';
14
15 for e = 0 to 3
16     if elmtMask[e] == '1' then
17         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.443 VQNEG

Vector Saturating Negate. Negate the value and saturate each element of a vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	0		Qd	0	0	1	1	1	1	1	1	1	M	0		Qm	0	

T1: VQNEG variant

VQNEG<v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 esize = 8 << UInt(size);
7 elements = 32 DIV esize;
8 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 S8 Encoded as size = 00
 S16 Encoded as size = 01
 S32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 opl = Q[m, curBeat];
8 for e = 0 to elements-1
9     value = -SInt(Elm[opl, e, esize]);
10    (Elm[result, e, esize], sat) = SignedSatQ(value, esize);
11    if sat && elmtMask[e*(esize>>3)] == '1' then
12        FPSCR.QC = '1';
13
14 for e = 0 to 3
15     if elmtMask[e] == '1' then
16         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.444 VQRSHL

Vector Saturating Rounding Shift Left. The vector variant shifts each element of the first vector by a value from the least significant byte of the corresponding element of the second vector and places the results in the destination vector.

The register variants shift each element of a vector register by the value specified in a source register. The direction of the shift depends on the sign of the element from the second vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn	0		Qd	0	0	1	0	1	N	1	M	1		Qm	0				

T1: VQRSHL variant

VQRSHL<v><q>.<dt> Qd, Qm, Qn

Decode for this encoding

```
1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 unsigned = (U == '1');
8 withScalar = FALSE;
9 withVector = TRUE;
10 esize     = 8 << UInt(size);
11 elements  = 32 DIV esize;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	Da	1	1	size	1	1	Qda	1	1	1	1	0	1	1	1	0					Rm		

T2: VQRSHL variant

VQRSHL<v><q>.<dt> Qda, Rm

Decode for this encoding

```
1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' then UNDEFINED;
4 da      = UInt(Da:Qda);
5 m       = UInt(Rm);
6 d       = da;
7 n       = m;
```

```

8 m      = da;
9 unsigned = (U == '1');
10 withScalar = TRUE;
11 withVector = FALSE;
12 esize      = 8 << UInt(size);
13 elements   = 32 DIV esize;
14 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
15 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> This parameter determines the following values:
 – Unsigned flag: S indicates signed, U indicates unsigned.
 – Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

<Qd> Destination vector register.
 <Qda> Source and destination vector register.
 <Qm> Source vector register.
 <Qn> Source vector register, the elements of which containing the amount to shift by.
 <Rm> Source general-purpose register containing the amount to shift by.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1    = Q[m, curBeat];
8 if withVector then
9     op2 = Q[n, curBeat];
10 if withScalar then
11     shiftAmount = SInt(R[n][7:0]);
12 for e = 0 to elements-1
13     if withVector then
14         shiftAmount = SInt(Elem[op2, e, esize][7:0]);
15         // 0 for left shift, 2^(n-1) for right shift
16         roundConst  = 1 << (-1-shiftAmount);
17         operand      = Int(Elem[op1, e, esize], unsigned);
18         (value, sat) = SatQ((operand + roundConst) << shiftAmount, esize, unsigned);
19         Elem[result, e, esize] = value;
20         if sat && elmtMask[e*(esize>>3)] == '1' then
21             FPSCR.QC = '1';
22
23 for e = 0 to 3
24     if elmtMask[e] == '1' then
25         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.445 VQRSHRN

Vector Saturating Rounding Shift Right and Narrow. Performs an element-wise saturation to half-width, with shift, writing the rounded result to either the top half (T variant) or bottom half (B variant) of the result element. The other half of the destination vector element retains its previous value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	D	0	sz	imm			Qd		T	1	1	1	1	0	1	M	0	Qm		1			

T1: VQRSHRN variant

VQRSHRN<T><v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d      = UInt(D:Qd);
4 m      = UInt(M:Qm);
5 unsigned = (U == '1');
6 imm5   = sz:imm;
7 case sz of
8   when '01' size = '00'; shiftAmount = 16 - UInt(imm5);
9   when '1x' size = '01'; shiftAmount = 32 - UInt(imm5);
10  otherwise UNDEFINED;
11 esize  = 8 << UInt(size);
12 elements = 16 DIV esize;
13 top     = UInt(T);
14 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T> Specifies which half of the result element the result is written to.
This parameter must be one of the following values:

B Encoded as T = 0
Indicates bottom half
T Encoded as T = 1
Indicates top half

<v, q> See [C1.2.5 Standard assembler syntax fields](#).

<dt> This parameter determines the following values:

– Unsigned flag: S indicates signed, U indicates unsigned.
– The size of the elements in the vector.

This parameter must be one of the following values:

S16 Encoded as sz = 01, U = 0
U16 Encoded as sz = 01, U = 1
S32 Encoded as sz = 1x, U = 0
U32 Encoded as sz = 1x, U = 1

<Qd> Destination vector register.

<Qm> Source vector register.

<imm> The number of bits to shift by, in the range 1 to <dt>/2. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1    = Q[m, curBeat];
7 result = Q[d, curBeat];
8 for e = 0 to elements-1
9     operand = Int(Elem[op1, e, 2*esize], unsigned);
10    // 0 for left shift, 2^(n-1) for right shift
11    operand = operand + (1 << (shiftAmount-1));
12    operand = operand >> shiftAmount;
13    (value, sat) = SatQ(operand, esize, unsigned);
14    Elem[result, 2*e + top, esize] = value;
15    if sat && elmtMask[(e*2 + top) * (esize>>3)] == '1' then
16        FPSCR.QC = '1';
17
18 for e = 0 to 3
19     if elmtMask[e] == '1' then
20         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.446 VQRSHRUN

Vector Saturating Rounding Shift Right Unsigned and Narrow. Performs an element-wise saturation to half-width, with shift, writing the rounded result to either the top half (T variant) or bottom half (B variant) of the result element. The other half of the destination vector element retains its previous value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	0	sz	imm		Qd		T	1	1	1	1	1	1	M	0	Qm		0				

T1: VQRSHRUN variant

VQRSHRUN<T><v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 unsigned      = FALSE;
6 destUnsigned = TRUE;
7 imm5          = sz:imm;
8 case sz of
9   when '01' size = '00'; shiftAmount = 16 - UInt(imm5);
10  when '1x' size = '01'; shiftAmount = 32 - UInt(imm5);
11  otherwise UNDEFINED;
12 esize      = 8 << UInt(size);
13 elements   = 16 DIV esize;
14 top        = UInt(T);
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T>	Specifies which half of the result element the result is written to. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	The size of the elements in the vector. This parameter must be one of the following values: S16 Encoded as sz = 01 S32 Encoded as sz = 1x
<Qd>	Destination vector register.
<Qm>	Source vector register.
<imm>	The number of bits to shift by, in the range 1 to <dt>/2. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1    = Q[m, curBeat];
7 result = Q[d, curBeat];
8 for e = 0 to elements-1
9     operand = Int(Elem[op1, e, 2*esize], unsigned);
10    // 0 for left shift, 2^(n-1) for right shift
11    operand = operand + (1 << (shiftAmount-1));
12    operand = operand >> shiftAmount;
13    (value, sat) = SatQ(operand, esize, destUnsigned);
14    Elem[result, 2*e + top, esize] = value;
15    if sat && elmtMask[(e*2 + top) * (esize>>3)] == '1' then
16        FPSCR.QC = '1';
17
18 for e = 0 to 3
19     if elmtMask[e] == '1' then
20         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.447 VQSHL, VQSHLU

Vector Saturating Shift Left, Vector Saturating Shift Left Unsigned. The register variants shift each element of a vector register by the value specified in a source register. The direction of the shift depends on the sign of the element from the second vector register.

The immediate variant shifts each element of a vector register to the left by the immediate value.

The vector variant shifts each element of the first vector by a value from the least significant byte of the corresponding element of the second vector and places the results in the destination vector.

The unsigned variant produces unsigned results, although the operands are signed.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	Da	1	1	size	0	1	Qda	1	1	1	1	0	1	1	1	0	1	1	1	0	Rm		

T1: VQSHL variant

VQSHL<v><q>.<dt> Qda, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' then UNDEFINED;
4 da = UInt(Da:Qda);
5 m = UInt(Rm);
6 d = da;
7 n = m;
8 m = da;
9 unsigned = (U == '1');
10 destUnsigned = unsigned;
11 withScalar = TRUE;
12 withVector = FALSE;
13 esize = 8 << UInt(size);
14 elements = 32 DIV esize;
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
16 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	1	D	sz			imm		Qd		0	0	1	1	1	0	1	M	1	Qm		0			

T2: VQSHL variant

VQSHL<v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1  if sz == '000' then SEE "VMOV";
2  CheckDecodeFaults(ExtType_Mve);
3  if D == '1' || M == '1' then UNDEFINED;
4  d = UInt(D:Qd);
5  m = UInt(M:Qm);
6  unsigned      = (U == '1');
7  destUnsigned = unsigned;
8  n = integer UNKNOWN;
9  imm6          = sz:imm;
10 case sz of
11   when '001' size = '00'; shiftAmount = UInt(imm6) - 8;
12   when '01x' size = '01'; shiftAmount = UInt(imm6) - 16;
13   when '1xx' size = '10'; shiftAmount = UInt(imm6) - 32;
14   otherwise UNDEFINED;
15 withScalar = FALSE;
16 withVector = FALSE;
17 esize      = 8 << UInt(size);
18 elements   = 32 DIV esize;
19 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

T3*Armv8.1-M MVE*

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D		sz			imm		Qd	0	0	1	1	0	0	1	M	1		Qm		0		

T3: VQSHLU variant

VQSHLU<v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1  if sz == '000' then SEE "VMOV";
2  CheckDecodeFaults(ExtType_Mve);
3  if D == '1' || M == '1' then UNDEFINED;
4  d = UInt(D:Qd);
5  m = UInt(M:Qm);
6  unsigned      = FALSE;
7  destUnsigned = TRUE;
8  n = integer UNKNOWN;
9  imm6          = sz:imm;
10 case sz of
11   when '001' size = '00'; shiftAmount = UInt(imm6) - 8;
12   when '01x' size = '01'; shiftAmount = UInt(imm6) - 16;
13   when '1xx' size = '10'; shiftAmount = UInt(imm6) - 32;
14   otherwise UNDEFINED;
15 withScalar = FALSE;
16 withVector = FALSE;
17 esize      = 8 << UInt(size);
18 elements   = 32 DIV esize;
19 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

T4*Armv8.1-M MVE*

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn	0		Qd	0	0	1	0	0	N	1	M	1		Qm		0			

T4: VQSHL variant

VQSHL<v><q>.<dt> Qd, Qm, Qn

Decode for this encoding

```
1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(N:Qn);
7 unsigned = (U == '1');
8 destUnsigned = unsigned;
9 withScalar = FALSE;
10 withVector = TRUE;
11 esize = 8 << UInt(size);
12 elements = 32 DIV esize;
13 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for T1 encodings

<dt>

This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

Assembler symbols for T2 encodings

<dt>

This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- The size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	sz = 001,	U = 0
U8	Encoded as	sz = 001,	U = 1
S16	Encoded as	sz = 01x,	U = 0
U16	Encoded as	sz = 01x,	U = 1
S32	Encoded as	sz = 1xx,	U = 0
U32	Encoded as	sz = 1xx,	U = 1

Assembler symbols for T3 encodings

<dt>

The size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	sz = 001
S16	Encoded as	sz = 01x
S32	Encoded as	sz = 1xx

Assembler symbols for T4 encodings

<dt>

This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
----	------------	------------	-------

U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Destination vector register.
<Qda>	Source and destination vector register.
<Qm>	Source vector register.
<Qn>	Source vector register, the elements of which containing the amount to shift by.
<Rm>	Source general-purpose register containing the amount to shift by.
<imm>	The number of bits to shift by, in the range 0 to <dt>-1. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1 = Q[m, curBeat];
8  if withVector then
9      op2 = Q[n, curBeat];
10 if withScalar then
11     shiftAmount = SInt(R[n][7:0]);
12 for e = 0 to elements-1
13     if withVector then
14         shiftAmount = SInt(Elem[op2, e, esize][7:0]);
15         operand = Int(Elem[op1, e, esize], unsigned);
16         (value, sat) = SatQ(operand << shiftAmount, esize, destUnsigned);
17         Elem[result, e, esize] = value;
18         if sat && elmtMask[e*(esize>>3)] == '1' then
19             FPSCR.QC = '1';
20
21 for e = 0 to 3
22     if elmtMask[e] == '1' then
23         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.448 VQSHRN

Vector Saturating Shift Right and Narrow. Performs an element-wise saturation to half-width, with shift, writing the result to either the top half (T variant) or bottom half (B variant) of the result element. The other half of the destination vector element retains its previous value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	D	0	sz	imm			Qd			T	1	1	1	1	0	1	M	0	Qm			0	

T1: VQSHRN variant

VQSHRN<T><v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d      = UInt(D:Qd);
4 m      = UInt(M:Qm);
5 unsigned = (U == '1');
6 imm5    = sz:imm;
7 case sz of
8   when '01' size = '00'; shiftAmount = 16 - UInt(imm5);
9   when '1x' size = '01'; shiftAmount = 32 - UInt(imm5);
10  otherwise UNDEFINED;
11 esize    = 8 << UInt(size);
12 elements = 16 DIV esize;
13 top      = UInt(T);
14 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T> Specifies which half of the result element the result is written to.
This parameter must be one of the following values:

B Encoded as T = 0
Indicates bottom half
T Encoded as T = 1
Indicates top half

<v, q> See [C1.2.5 Standard assembler syntax fields](#).

<dt> This parameter determines the following values:

– Unsigned flag: S indicates signed, U indicates unsigned.
– The size of the elements in the vector.

This parameter must be one of the following values:

S16 Encoded as sz = 01, U = 0
U16 Encoded as sz = 01, U = 1
S32 Encoded as sz = 1x, U = 0
U32 Encoded as sz = 1x, U = 1

<Qd> Destination vector register.

<Qm> Source vector register.

<imm> The number of bits to shift by, in the range 1 to <dt>/2. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1      = Q[m, curBeat];
7 result = Q[d, curBeat];
8 for e = 0 to elements-1
9     operand = Int(Elem[op1, e, 2*esize], unsigned);
10    operand = operand >> shiftAmount;
11    (value, sat) = SatQ(operand, esize, unsigned);
12    Elem[result, 2*e + top, esize] = value;
13    if sat && elmtMask[(e*2 + top) * (esize>>3)] == '1' then
14        FPSCR.QC = '1';
15
16 for e = 0 to 3
17     if elmtMask[e] == '1' then
18         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.449 VQSHRUN

Vector Saturating Shift Right Unsigned and Narrow. Performs an element-wise saturation to half-width, with shift, writing the result to either the top half (T variant) or bottom half (B variant) of the result element. The other half of the destination vector element retains its previous value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	0	sz		imm			Qd	T	1	1	1	1	1	1	M	0		Qm		0		

T1: VQSHRUN variant

VQSHRUN<T><v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 unsigned      = FALSE;
6 destUnsigned = TRUE;
7 imm5         = sz:imm;
8 case sz of
9   when '01' size = '00'; shiftAmount = 16 - UInt(imm5);
10  when '1x' size = '01'; shiftAmount = 32 - UInt(imm5);
11  otherwise UNDEFINED;
12 esize      = 8 << UInt(size);
13 elements   = 16 DIV esize;
14 top        = UInt(T);
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T>	Specifies which half of the result element the result is written to. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	The size of the elements in the vector. This parameter must be one of the following values: S16 Encoded as sz = 01 S32 Encoded as sz = 1x
<Qd>	Destination vector register.
<Qm>	Source vector register.
<imm>	The number of bits to shift by, in the range 1 to <dt>/2. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1      = Q[m, curBeat];
7 result = Q[d, curBeat];
8 for e = 0 to elements-1
9     operand = Int(Elem[op1, e, 2*esize], unsigned);
10    operand = operand >> shiftAmount;
11    (value, sat) = SatQ(operand, esize, destUnsigned);
12    Elem[result, 2*e + top, esize] = value;
13    if sat && elmtMask[(e*2 + top) * (esize>>3)] == '1' then
14        FPSCR.QC = '1';
15
16 for e = 0 to 3
17     if elmtMask[e] == '1' then
18         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.450 VQSUB

Vector Saturating Subtract. Subtract the value of the elements in the second source vector register from either the respective elements in the first source vector register or a general-purpose register. The result is saturated before being written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn	0		Qd	0	0	0	1	0	N	1	M	1		Qm	0				

T1: VQSUB variant

VQSUB<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (M:Qm);
6 n = UInt (N:Qn);
7 esize = 8 << UInt (size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 withScalar = FALSE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	D	size		Qn	0		Qd	1	1	1	1	1	N	1	1	1	0			Rm			

T2: VQSUB variant

VQSUB<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults (ExtType_Mve);
3 if D == '1' || N == '1' then UNDEFINED;
4 d = UInt (D:Qd);
5 m = UInt (Rm);
6 n = UInt (N:Qn);
7 esize = 8 << UInt (size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 withScalar = TRUE;
11 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
12 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .																								
<dt>	This parameter determines the following values: – Size: indicates the size of the elements in the vector. – Unsigned flag: S indicates signed, U indicates unsigned. This parameter must be one of the following values: <table><tr><td>S8</td><td>Encoded as</td><td>size = 00,</td><td>U = 0</td></tr><tr><td>U8</td><td>Encoded as</td><td>size = 00,</td><td>U = 1</td></tr><tr><td>S16</td><td>Encoded as</td><td>size = 01,</td><td>U = 0</td></tr><tr><td>U16</td><td>Encoded as</td><td>size = 01,</td><td>U = 1</td></tr><tr><td>S32</td><td>Encoded as</td><td>size = 10,</td><td>U = 0</td></tr><tr><td>U32</td><td>Encoded as</td><td>size = 10,</td><td>U = 1</td></tr></table>	S8	Encoded as	size = 00,	U = 0	U8	Encoded as	size = 00,	U = 1	S16	Encoded as	size = 01,	U = 0	U16	Encoded as	size = 01,	U = 1	S32	Encoded as	size = 10,	U = 0	U32	Encoded as	size = 10,	U = 1
S8	Encoded as	size = 00,	U = 0																						
U8	Encoded as	size = 00,	U = 1																						
S16	Encoded as	size = 01,	U = 0																						
U16	Encoded as	size = 01,	U = 1																						
S32	Encoded as	size = 10,	U = 0																						
U32	Encoded as	size = 10,	U = 1																						
<Qd>	Destination vector register.																								
<Qn>	First source vector register.																								
<Qm>	Second source vector register.																								
<Rm>	Source general-purpose register.																								

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1    = Q[n, curBeat];
8  if !withScalar then
9      op2 = Q[m, curBeat];
10 for e = 0 to elements-1
11     if withScalar then
12         value = Int(Elem[op1, e, esize], unsigned) - Int(R[m][esize-1:0], unsigned);
13     else
14         value = Int(Elem[op1, e, esize], unsigned) - Int(Elem[op2, e, esize], unsigned);
15     (Elem[result, e, esize], sat) = SatQ(value, esize, unsigned);
16     if sat && elmtMask[e*(esize>>3)] == '1' then
17         FPSCR.QC = '1';
18
19 for e = 0 to 3
20     if elmtMask[e] == '1' then
21         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.451 VREV16

Vector Reverse. Reverse the order of 8-bit elements within each halfword of the source vector register and places the result in the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	0		Qd	0	0	0	0	1	0	1	M	0		Qm	0			

T1: VREV16 variant

VREV16<v><q>.<size> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size != '00' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 esize = 8 << UInt(size);
7 elements = 64 DIV esize;
8 width = 2;
9 groupsize = (1 << (3-width-UInt(size))); // elements per reversing group: 2
10 reverse_mask = (groupsize-1)[esize-1:0]; // EORing mask used for index calculations
11 groups_per_beat = width;
12 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See C1.2.5 *Standard assembler syntax fields*.
 <size> Size: indicates the size of the elements in the vector.
 This parameter must be the following value:
 8 Encoded as size = 00
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 // 64 bit wide operations are handled differently as they perform cross beat
8 // register accesses
9 if width == 0 then
10     op1 = Q[m, UInt(curBeat[1:0] EOR 1[1:0])];
11     for e = 0 to (groupsize >> 1)-1
12         // Calculate destination element index by bitwise EOR on source element index:
13         elemIdx = UInt((e[esize-1:0] EOR (UInt(reverse_mask) >> 1)[esize-1:0]));
14         Elem[result, elemIdx, esize] = Elem[op1, e, esize];
```

```
15 else
16     opl = Q[m, curBeat];
17     for g = 0 to groups_per_beat-1
18         for e = 0 to groupsize -1
19             elemIdx = (g*groupsize) + UInt(e[esize-1:0] EOR (reverse_mask));
20             Elem[result, elemIdx, esize] = Elem[opl, (g*groupsize)+e, esize];
21
22 for e = 0 to 3
23     if elmtMask[e] == '1' then
24         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.452 VREV32

Vector Reverse. Reverse the order of 8-bit or 16-bit elements within each word of the source vector register and places the result in the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	0		Qd	0	0	0	0	0	0	1	1	M	0		Qm	0		

T1: VREV32 variant

VREV32<v><q>.<size> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '1x' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 esize = 8 << UInt(size);
7 elements = 64 DIV esize;
8 width = 1;
9 groupsize = (1 << (3-width-UInt(size))); // elements per reversing group: 2 or 4
10 reverse_mask = (groupsize-1)[esize-1:0]; // EORing mask used for index calculations
11 groups_per_beat = width;
12 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <size> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 8 Encoded as size = 00
 16 Encoded as size = 01
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 // 64 bit wide operations are handled differently as they perform cross beat
8 // register accesses
9 if width == 0 then
10   op1 = Q[m, UInt(curBeat[1:0] EOR 1[1:0])];
11   for e = 0 to (groupsize >> 1)-1
12     // Calculate destination element index by bitwise EOR on source element index:
13     elemIdx = UInt((e[esize-1:0] EOR (UInt(reverse_mask) >> 1)[esize-1:0]));

```



```

14     Elem[result, elemIdx, esize] = Elem[op1, e, esize];
15 else
16     op1 = Q[m, curBeat];
17     for g = 0 to groups_per_beat-1
18         for e = 0 to groupsize -1
19             elemIdx = (g*groupsize) + UInt(e[esize-1:0] EOR (reverse_mask));
20             Elem[result, elemIdx, esize] = Elem[op1, (g*groupsize)+e, esize];
21
22 for e = 0 to 3
23     if elmtMask[e] == '1' then
24         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.453 VREV64

Vector Reverse. Reverse the order of 8-bit, 16-bit or 32-bit elements within each doubleword of the source vector register and places the result in the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	0	0		Qd	0	0	0	0	0	0	0	0	1	M	0		Qm	0	

T1: VREV64 variant

VREV64<v><q>.<size> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 esize = 8 << UInt(size);
7 elements = 64 DIV esize;
8 width = 0;
9 groupsize = (1 << (3-width-UInt(size))); // elements per reversing group: 2, 4 or 8
10 reverse_mask = (groupsize-1)[esize-1:0]; // EORing mask used for index calculations
11 groups_per_beat = width;
12 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
13 if D:Qd == M:Qm then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <size> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:
 8 Encoded as size = 00
 16 Encoded as size = 01
 32 Encoded as size = 10
 <Qd> Destination vector register.
 <Qm> Source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 // 64 bit wide operations are handled differently as they perform cross beat
8 // register accesses
9 if width == 0 then
10     op1 = Q[m, UInt(curBeat[1:0] EOR 1[1:0])];
```

```
11     for e = 0 to (groupsize >> 1)-1
12         // Calculate destination element index by bitwise EOR on source element index:
13         elemIdx = UInt((e[esize-1:0] EOR (UInt(reverse_mask) >> 1)[esize-1:0]));
14         Elem[result, elemIdx, esize] = Elem[op1, e, esize];
15     else
16         op1 = Q[m, curBeat];
17         for g = 0 to groups_per_beat-1
18             for e = 0 to groupsize -1
19                 elemIdx = (g*groupsize) + UInt(e[esize-1:0] EOR (reverse_mask));
20                 Elem[result, elemIdx, esize] = Elem[op1, (g*groupsize)+e, esize];
21
22     for e = 0 to 3
23         if elmtMask[e] == '1' then
24             Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.454 VRHADD

Vector Rounding Halving Add. Add the value of the elements in the first source vector register to the respective elements in the second source vector register. The result is halved and rounded before being written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn		0		Qd		0	0	0	0	1	N	1	M	0		Qm		0	

T1: VRHADD variant

VRHADD<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(N:Qn);
7 esize = 8 << UInt(size);
8 elements = 32 DIV esize;
9 unsigned = (U == '1');
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q> See [C1.2.5 Standard assembler syntax fields](#).
 <dt> This parameter determines the following values:
 – Unsigned flag: S indicates signed, U indicates unsigned.
 – Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

<Qd> Destination vector register.
 <Qn> First source vector register.
 <Qm> Second source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5

```

```

6 result = Zeros(32);
7 op1    = Q[n, curBeat];
8 op2    = Q[m, curBeat];
9 for e = 0 to elements-1
10     value = Int(Elem[op1, e, esize], unsigned) + Int(Elem[op2, e, esize], unsigned);
11     Elem[result, e, esize] = (value + 1)[esize:1];
12
13 for e = 0 to 3
14     if elmtMask[e] == '1' then
15         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.455 VRINT (floating-point)

Vector Round Integer. Round a floating-point value to an integer value. The result remains in floating-point format. It is not converted to an integer.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D	1	1	size	1	0		Qd	0	0	1		op	1	M	0		Qm	0				

T1: VRINT variant

VRINT<op><v><q>.<dt> Qd, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' then UNDEFINED;
3 if op == '1x0' then UNDEFINED;
4 if size IN {'11', '00'} then UNDEFINED;
5 d = UInt (D:Qd);
6 m = UInt (M:Qm);
7 esize = 8 << UInt (size);
8 elements = 32 DIV esize;
9 case op of
10   when '010' // A, Round to nearest, with ties away
11     rmode = '01'; away = TRUE; exact = FALSE;
12   when '000' // N, Round to nearest, with ties to even
13     rmode = '00'; away = FALSE; exact = FALSE;
14   when '111' // P, Round towards Plus Infinity
15     rmode = '01'; away = FALSE; exact = FALSE;
16   when '101' // M, Round towards Minus Infinity
17     rmode = '10'; away = FALSE; exact = FALSE;
18   when '001' // X, Round to nearest with ties to even, raising inexact
19     // exception if result not numerically equal to input
20     rmode = '00'; away = FALSE; exact = TRUE;
21   when '011' // Z, Round towards zero
22     rmode = '11'; away = FALSE; exact = FALSE;
23   otherwise
24     UNDEFINED;
25 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<op> The rounding mode.
This parameter must be one of the following values:

- N Encoded as op = 000
Round to nearest with ties to even
- X Encoded as op = 001
Round to nearest with ties to even, raising inexact exception if result not numerically equal to input
- A Encoded as op = 010
Round to nearest with ties to away
- Z Encoded as op = 011

	Round towards zero
M	Encoded as op = 101
	Round towards minus infinity
P	Encoded as op = 111
	Round towards plus infinity
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F16 Encoded as size = 01 F32 Encoded as size = 10
<Qd>	Destination vector register.
<Qm>	Source vector register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  opl    = Q[m, curBeat];
8  for e = 0 to elements-1
9      // Avoid Floating-point exceptions on a predicated lane by checking the element mask
10     pred = (elmtMask[e*(esize>>3)] == '0');
11     conv = FPRoundInt(Elem[opl, e, esize], rmode, away, exact, FALSE, pred);
12     Elem[result, e, esize] = conv;
13
14  for e = 0 to 3
15     if elmtMask[e] == '1' then
16         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.456 VRINTA

Floating-point Round to Nearest Integer with Ties to Away. Floating-point Round to Nearest Integer with Ties to Away rounds a floating-point value to an integral floating-point value of the same size using the Round to Nearest with Ties to Away rounding mode. A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	1	0	RM = 00		Vd				1	0	size	0	1	M	0				Vm	

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VRINTA{<q>}.F16.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VRINTA{<q>}.F32.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VRINTA{<q>}.F64.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 case RM of
5     when '00' // Round to nearest, with ties away
6         rmode = '01'; away = TRUE;
7     when '01' // Round to nearest, with ties to even
8         rmode = '00'; away = FALSE;
9     when '10' // Round towards Plus Infinity
10        rmode = '01'; away = FALSE;
11    when '11' // Round towards Minus Infinity
12        rmode = '10'; away = FALSE;
13 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
14 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
15 if InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 exact = FALSE;
5
6 case size of
7     when '01'
8         S[d] = Zeros(16) : FPRoundInt(S[m][15:0], rmode, away, exact, TRUE);
9     when '10'
10        S[d] = FPRoundInt(S[m], rmode, away, exact, TRUE);
11    when '11'
12        D[d] = FPRoundInt(D[m], rmode, away, exact, TRUE);
```

C2.4.457 VRINTM

Floating-point Round to Integer towards -Infinity. Floating-point Round to Integer towards -Infinity rounds a floating-point value to an integral floating-point value of the same size using the Round towards -Infinity rounding mode. A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	1	0	RM = 1		Vd				1	0	size	0	1	M	0				Vm	

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VRINTM{<q>}.F16.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VRINTM{<q>}.F32.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VRINTM{<q>}.F64.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 case RM of
5     when '00' // Round to nearest, with ties away
6         rmode = '01'; away = TRUE;
7     when '01' // Round to nearest, with ties to even
8         rmode = '00'; away = FALSE;
9     when '10' // Round towards Plus Infinity
10        rmode = '01'; away = FALSE;
11    when '11' // Round towards Minus Infinity
12        rmode = '10'; away = FALSE;
13 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
14 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
15 if InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 exact = FALSE;
5
6 case size of
7     when '01'
8         S[d] = Zeros(16) : FPRoundInt(S[m][15:0], rmode, away, exact, TRUE);
9     when '10'
10        S[d] = FPRoundInt(S[m], rmode, away, exact, TRUE);
11    when '11'
12        D[d] = FPRoundInt(D[m], rmode, away, exact, TRUE);
```

C2.4.458 VRINTN

Floating-point Round to Nearest Integer with Ties to Even. Floating-point Round to Nearest Integer with Ties to Even rounds a floating-point value to an integral floating-point value of the same size using the Round to Nearest rounding mode. A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	1	0	RM = 0		Vd				1	0	size	0	1	M	0				Vm	

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VRINTN{<q>}.F16.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VRINTN{<q>}.F32.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VRINTN{<q>}.F64.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 case RM of
5     when '00' // Round to nearest, with ties away
6         rmode = '01'; away = TRUE;
7     when '01' // Round to nearest, with ties to even
8         rmode = '00'; away = FALSE;
9     when '10' // Round towards Plus Infinity
10        rmode = '01'; away = FALSE;
11    when '11' // Round towards Minus Infinity
12        rmode = '10'; away = FALSE;
13 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
14 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
15 if InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 exact = FALSE;
5
6 case size of
7     when '01'
8         S[d] = Zeros(16) : FPRoundInt(S[m][15:0], rmode, away, exact, TRUE);
9     when '10'
10        S[d] = FPRoundInt(S[m], rmode, away, exact, TRUE);
11    when '11'
12        D[d] = FPRoundInt(D[m], rmode, away, exact, TRUE);
```

C2.4.459 VRINTP

Floating-point Round to Integer towards +Infinity. Floating-point Round to Integer towards +Infinity rounds a floating-point value to an integral floating-point value of the same size using the Round towards +Infinity rounding mode. A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	1	1	1	0	RM = 10		Vd				1	0	size	0	1	M	0				Vm	

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VRINTP{<q>}.F16.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VRINTP{<q>}.F32.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VRINTP{<q>}.F64.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 case RM of
5     when '00' // Round to nearest, with ties away
6         rmode = '01'; away = TRUE;
7     when '01' // Round to nearest, with ties to even
8         rmode = '00'; away = FALSE;
9     when '10' // Round towards Plus Infinity
10        rmode = '01'; away = FALSE;
11    when '11' // Round towards Minus Infinity
12        rmode = '10'; away = FALSE;
13 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
14 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);
15 if InITBlock() then UNPREDICTABLE;
```

Assembler symbols for all encodings

<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```
1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 exact = FALSE;
5
6 case size of
7     when '01'
8         S[d] = Zeros(16) : FPRoundInt(S[m][15:0], rmode, away, exact, TRUE);
9     when '10'
10        S[d] = FPRoundInt(S[m], rmode, away, exact, TRUE);
11    when '11'
12        D[d] = FPRoundInt(D[m], rmode, away, exact, TRUE);
```

C2.4.460 VRINTR

Floating-point Round to Integer. Floating-point Round to Integer rounds a floating-point value to an integral floating-point value of the same size using the rounding mode specified in FPSCR. A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	1	1	0	Vd		1	0	size		1	M	0	Vm						
																op = 0															

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VRINTR{<c>}{<q>}.F16.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VRINTR{<c>}{<q>}.F32.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VRINTR{<c>}{<q>}.F64.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
6 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();

```



```
3      ExecuteFPCheck();
4
5      rmode = if op == '1' then '11' else FPSCR.RMode;
6      exact = FALSE;
7      away  = FALSE;
8
9      case size of
10         when '01'
11             S[d] = Zeros(16) : FPRoundInt(S[m][15:0], rmode, away, exact, TRUE);
12         when '10'
13             S[d] = FPRoundInt(S[m], rmode, away, exact, TRUE);
14         when '11'
15             D[d] = FPRoundInt(D[m], rmode, away, exact, TRUE);
```

C2.4.461 VRINTX

Floating-point Round to Integer, raising Inexact exception. This instruction rounds a floating-point value to an integral floating-point value of the same size. A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

VRINTX uses the rounding mode specified in FPSCR, and raises an Inexact exception when the result value is not numerically equal to the input value.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	1	1	1	Vd	1	0	size	0	1	M	0	Vm							

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VRINTX{<c>}{<q>}.F16.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VRINTX{<c>}{<q>}.F32.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VRINTX{<c>}{<q>}.F64.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
6 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();

```

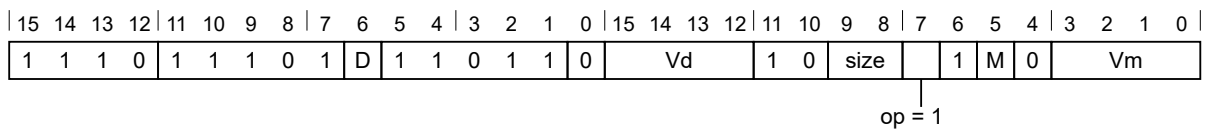
```
3      ExecuteFPCheck();
4
5      rmode = FPSCR[23:22];
6      away  = FALSE;
7      exact = TRUE;
8
9      case size of
10         when '01'
11             S[d] = Zeros(16) : FPRoundInt(S[m][15:0], rmode, away, exact, TRUE);
12         when '10'
13             S[d] = FPRoundInt(S[m], rmode, away, exact, TRUE);
14         when '11'
15             D[d] = FPRoundInt(D[m], rmode, away, exact, TRUE);
```

C2.4.462 VRINTZ

Floating-point Round to Integer towards Zero. Floating-point Round to Integer towards Zero rounds a floating-point value to an integral floating-point value of the same size, using the Round towards Zero rounding mode. A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.



Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VRINTZ{<c>}{<q>}.F16.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VRINTZ{<c>}{<q>}.F32.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VRINTZ{<c>}{<q>}.F64.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
6 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();

```

```
3      ExecuteFPCheck();
4
5      rmode = if op == '1' then '11' else FPSCR.RMode;
6      exact = FALSE;
7      away  = FALSE;
8
9      case size of
10         when '01'
11             S[d] = Zeros(16) : FPRoundInt(S[m][15:0], rmode, away, exact, TRUE);
12         when '10'
13             S[d] = FPRoundInt(S[m], rmode, away, exact, TRUE);
14         when '11'
15             D[d] = FPRoundInt(D[m], rmode, away, exact, TRUE);
```

C2.4.463 VRMLALDAVH

Vector Rounding Multiply Add Long Dual Accumulate Across Vector Returning High 64 bits. The elements of the vector registers are handled in pairs. In the base variant, corresponding elements from the two source registers are multiplied together, whereas the exchange variant swaps the values in each pair of values read from the first source register, before multiplying them with the values from the second source register. The results of the pairs of multiply operations are combined by adding them together. At the end of each beat these results are accumulated. The upper 64 bits of a 72-bit accumulator value is selected and stored across two registers, the top 32 bits are stored in an odd-numbered register and the lower 32 bits are stored in an even-numbered register. The initial value of the general-purpose destination registers can optionally be shifted up by 8 bits and added to the result. The result is rounded before the top 64 bits are selected.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	RdaHi			Qn		0	RdaLo	X	1	1	1	1	N	0	A	0		Qm		0			

T1: VRMLALDAVH variant

VRMLALDAVH{A}{X}<v><q>.<dt> RdaLo, RdaHi, Qn, Qm

Decode for this encoding

```

1 if RdaHi == '11x' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if N == '1' then UNDEFINED;
4 if U == '1' && X == '1' then UNDEFINED;
5 dah = UInt(RdaHi:'1');
6 dal = UInt(RdaLo:'0');
7 m = UInt(Qm);
8 n = UInt(N:Qn);
9 exchange = (X == '1');
10 accumulate = (A == '1');
11 esize = 32;
12 elements = 32 DIV esize;
13 unsigned = (U == '1');
14 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<A> Accumulate with existing register contents.
This parameter must be one of the following values:
– Encoded as A = 0
A Encoded as A = 1

<X> Exchange adjacent pairs of values in Qm.
This parameter must be one of the following values:
– Encoded as X = 0
X Encoded as X = 1

<v, q> See C1.2.5 *Standard assembler syntax fields*.

<dt> Unsigned flag: S indicates signed, U indicates unsigned.
This parameter must be one of the following values:
S32 Encoded as U = 0

	U32 Encoded as U = 1
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination. This must be an even numbered register. The value $RdaLo \gg 1$ is encoded in the RdaLo field.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination. This must be an odd numbered register. The value $RdaHi \gg 1$ is encoded in the RdaHi field.
<Qn>	First source vector register.
<Qm>	Second source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = if accumulate || !IsFirstBeat() then Int(R[dah]:R[dal], unsigned) << 8 else 0;
7 if elmtMask[0] == '1' then
8     if exchange then
9         if curBeat[0] == '0' then
10             mul = Int(Q[n, curBeat+1], unsigned) * Int(Q[m, curBeat], unsigned);
11         else
12             mul = Int(Q[n, curBeat-1], unsigned) * Int(Q[m, curBeat], unsigned);
13     else
14         mul = Int(Q[n, curBeat], unsigned) * Int(Q[m, curBeat], unsigned);
15     result = result + mul + (1 << 7);
16 R[dah] = result[71:40];
17 R[dal] = result[39:8];

```

C2.4.464 VRMLALVH

Vector Multiply Accumulate Long Across Vector Returning High 64 bits. This is an alias of VRMLALDAVH without exchange.

This is an alias of VRMLALDAVH with the following condition satisfied: X==0.

This alias is the preferred disassembly.

T1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	RdaHi			Qn		0		RdaLo		0	1	1	1	1	N	0	A	0		Qm		0	

VRMLALVH variant

VRMLALVH{A}<v><q>.<dt> RdaLo, RdaHi, Qn, Qm

C2.4.465 VRMLSLDAVH

Vector Rounding Multiply Subtract Long Dual Accumulate Across Vector Returning High 64 bits. The elements of the vector registers are handled in pairs. In the base variant, corresponding elements from the two source registers are multiplied together, whereas the exchange variant swaps the values in each pair of values read from the first source register, before multiplying them with the values from the second source register. The results of the pairs of multiply operations are combined by subtracting one from the other. At the end of each beat these results are accumulated. The upper 64 bits of a 72-bit accumulator value is selected and stored across two registers, the top 32 bits are stored in an odd-numbered register and the lower 32 bits are stored in an even-numbered register. The initial value of the general-purpose destination registers can optionally be shifted up by 8 bits and added to the result. The result is rounded before the top 64 bits are selected.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	RdaHi			Qn		0	RdaLo	X	1	1	1	0	N	0	A	0		Qm			1		

T1: VRMLSLDAVH variant

VRMLSLDAVH{A}{X}<v><q>.S32 RdaLo, RdaHi, Qn, Qm

Decode for this encoding

```

1 if RdaHi == '111' then SEE "VMSLDAV";
2 CheckDecodeFaults(ExtType_Mve);
3 if N == '1' then UNDEFINED;
4 dah      = UInt(RdaHi:'1');
5 dal      = UInt(RdaLo:'0');
6 m        = UInt(Qm);
7 n        = UInt(N:Qn);
8 exchange = (X == '1');
9 accumulate = (A == '1');
10 esize    = 32;
11 elements = 32 DIV esize;
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
13 if RdaHi == '110' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<A>	Accumulate with existing register contents. This parameter must be one of the following values: – Encoded as A = 0 A Encoded as A = 1
<X>	Exchange adjacent pairs of values in Qm. This parameter must be one of the following values: – Encoded as X = 0 X Encoded as X = 1
<v, q>	See C1.2.5 Standard assembler syntax fields .
<RdaLo>	General-purpose register for the low-half of the 64-bit source and destination. This must be an even numbered register. The value RdaLo » 1 is encoded in the RdaLo field.
<RdaHi>	General-purpose register for the high-half of the 64-bit source and destination. This must be an odd numbered register. The value RdaHi » 1 is encoded in the RdaHi field.

<Qn> First source vector register.
<Qm> Second source vector register.

Operation for all encodings

```
1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = if accumulate || !IsFirstBeat() then SInt(R[dah]:R[dal]) << 8 else 0;
7  if elmtMask[0] == '1' then
8      if exchange then
9          if curBeat[0] == '0' then
10             mul = SInt(Q[n, curBeat+1]) * SInt(Q[m, curBeat]);
11             else
12                 mul = SInt(Q[n, curBeat-1]) * SInt(Q[m, curBeat]);
13             else
14                 mul = SInt(Q[n, curBeat]) * SInt(Q[m, curBeat]);
15             if curBeat[0] == '0' then
16                 result = result + mul + (1 << 7);
17             else
18                 result = result - mul + (1 << 7);
19  R[dah] = result[71:40];
20  R[dal] = result[39:8];
```

C2.4.466 VRSHL

Vector Rounding Shift Left. The vector variant shifts each element of the first vector by a value from the least significant byte of the corresponding element of the second vector and places the results in the destination vector.

The register variants shift each element of a vector register by the value specified in a source register. The direction of the shift depends on the sign of the element from the second vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn	0		Qd	0	0	1	0	1	N	1	M	0		Qm	0				

T1: VRSHL variant

VRSHL<v><q>.<dt> Qd, Qm, Qn

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 unsigned = (U == '1');
8 withScalar = FALSE;
9 withVector = TRUE;
10 esize     = 8 << UInt(size);
11 elements  = 32 DIV esize;
12 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	Da	1	1	size	1	1		Qda	1	1	1	1	0	0	1	1	0				Rm		

T2: VRSHL variant

VRSHL<v><q>.<dt> Qda, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' then UNDEFINED;
4 da      = UInt(Da:Qda);
5 m       = UInt(Rm);
6 d       = da;
7 n       = m;
8 m       = da;

```

```

9 unsigned = (U == '1');
10 withScalar = TRUE;
11 withVector = FALSE;
12 esize = 8 << UInt(size);
13 elements = 32 DIV esize;
14 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
15 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .																								
<dt>	This parameter determines the following values: – Unsigned flag: S indicates signed, U indicates unsigned. – Size: indicates the size of the elements in the vector. This parameter must be one of the following values: <table><tr><td>S8</td><td>Encoded as</td><td>size = 00,</td><td>U = 0</td></tr><tr><td>U8</td><td>Encoded as</td><td>size = 00,</td><td>U = 1</td></tr><tr><td>S16</td><td>Encoded as</td><td>size = 01,</td><td>U = 0</td></tr><tr><td>U16</td><td>Encoded as</td><td>size = 01,</td><td>U = 1</td></tr><tr><td>S32</td><td>Encoded as</td><td>size = 10,</td><td>U = 0</td></tr><tr><td>U32</td><td>Encoded as</td><td>size = 10,</td><td>U = 1</td></tr></table>	S8	Encoded as	size = 00,	U = 0	U8	Encoded as	size = 00,	U = 1	S16	Encoded as	size = 01,	U = 0	U16	Encoded as	size = 01,	U = 1	S32	Encoded as	size = 10,	U = 0	U32	Encoded as	size = 10,	U = 1
S8	Encoded as	size = 00,	U = 0																						
U8	Encoded as	size = 00,	U = 1																						
S16	Encoded as	size = 01,	U = 0																						
U16	Encoded as	size = 01,	U = 1																						
S32	Encoded as	size = 10,	U = 0																						
U32	Encoded as	size = 10,	U = 1																						
<Qd>	Destination vector register.																								
<Qda>	Source and destination vector register.																								
<Qm>	Source vector register.																								
<Qn>	Source vector register, the elements of which containing the amount to shift by.																								
<Rm>	Source general-purpose register containing the amount to shift by.																								

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[m, curBeat];
8 if withVector then
9     op2 = Q[n, curBeat];
10 if withScalar then
11     shiftAmount = SInt(R[n][7:0]);
12 for e = 0 to elements-1
13     if withVector then
14         shiftAmount = SInt(Elm[op2, e, esize][7:0]);
15         // 0 for left shift, if applicable, 2^(n-1) for right shift
16         roundConst = 1 << (-1-shiftAmount);
17         operand = Int(Elm[op1, e, esize], unsigned);
18         Elem[result, e, esize] = ((operand + roundConst) << shiftAmount)[esize-1:0];
19
20 for e = 0 to 3
21     if elmtMask[e] == '1' then
22         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1     = Q[m, curBeat];
8 if withVector then
9     op2 = Q[n, curBeat];
10 if withScalar then
11     shiftAmount = SInt(R[n][7:0]);
12 for e = 0 to elements-1
13     if withVector then
14         shiftAmount = SInt(Elem[op2, e, esize][7:0]);
15         // 0 for left shift, if applicable, 2^(n-1) for right shift
16         roundConst = 1 << (shiftAmount-1);
17         operand     = Int(Elem[op1, e, esize], unsigned);
18         Elem[result, e, esize] = (operand + roundConst) >> shiftAmount[esize-1:0];
19
20 for e = 0 to 3
21     if elmtMask[e] == '1' then
22         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.468 VRSHRN

Vector Rounding Shift Right and Narrow. Performs an element-wise narrowing to half-width, with shift, writing the rounded result to either the top half (T variant) or bottom half (B variant) of the result element. The other half of the destination vector element retains its previous value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	1	D	0	sz	imm			Qd			T	1	1	1	1	1	1	M	0	Qm			1	

T1: VRSHRN variant

VRSHRN<T><v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 imm5 = sz:imm;
6 case sz of
7   when '01' size = '00'; shiftAmount = 16 - UInt(imm5);
8   when '1x' size = '01'; shiftAmount = 32 - UInt(imm5);
9   otherwise UNDEFINED;
10 esize = 8 << UInt(size);
11 elements = 16 DIV esize;
12 top = UInt(T);
13 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T>	Specifies which half of the result element the result is written to. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	The size of the elements in the vector. This parameter must be one of the following values: I16 Encoded as sz = 01 I32 Encoded as sz = 1x
<Qd>	Destination vector register.
<Qm>	Source vector register.
<imm>	The number of bits to shift by, in the range 1 to <dt>/2. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```
1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  op1      = Q[m, curBeat];
7  result = Q[d, curBeat];
8  for e = 0 to elements-1
9      operand = UInt(Elem[op1, e, 2*esize]);
10     // 0 for left shift, 2^(n-1) for right shift
11     operand = operand + (1 << (shiftAmount-1));
12     operand = operand >> shiftAmount;
13     Elem[result, 2*e + top, esize] = operand[esize-1:0];
14
15 for e = 0 to 3
16     if elmtMask[e] == '1' then
17         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.469 VSBC

Whole Vector Subtract With Carry. Beat-wise subtracts the value of the elements in the second source vector register and the value of NOT(Carry flag) from the respective elements in the first source vector register, the carry flag being FPSCR.C. The initial value of FPSCR.C can be overridden by using the I variant. FPSCR.C is not updated for beats disabled due to predication. FPSCR.N, .V and .Z are zeroed whenever FPSCR.C is updated.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	D	1	1	Qn	0	Qd	I	1	1	1	1	N	0	M	0	Qm	0						

T1: VSBC variant

VSBC{I}<v><q>.I32 Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt(D:Qd);
4 m      = UInt(M:Qm);
5 n      = UInt(N:Qn);
6 carryInit = (I == '1');
7 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<I>	Specifies where the initial carry in for wide arithmetic comes from. This parameter must be one of the following values: – Encoded as I = 0 Indicates carry input comes from FPSCR.C. I Encoded as I = 1 Indicates carry input is 1.
<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 op1 = Q[n, curBeat];
7 op2 = NOT(Q[m, curBeat]);
8 if carryInit && IsFirstBeat() then
9     (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = ('0', '0', '1', '0');
10 (result, carryOut, -) = AddWithCarry(op1, op2, FPSCR.C);
11 if elmtMask[0] == '1' then
12     (FPSCR.N, FPSCR.Z, FPSCR.C, FPSCR.V) = ('0', '0', carryOut, '0');

```

```
13  
14 for e = 0 to 3  
15     if elmtMask[e] == '1' then  
16         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.470 VSCCLRM

Floating-point Secure Context Clear Multiple. Zeros VPR and the specified floating-point registers if there is an active floating-point context. This instruction is UNDEFINED if executed in Non-secure state. This instruction is present on all PEs that implement the Armv8.1-M architecture, even if the Floating-point Extension or the MVE integer variant is not present. It is IMPLEMENTATION DEFINED whether this instruction is interrupt-continuable. See EPSR.ICI. If an exception returns to this instruction with nonzero EPSR.ICI bits, and the PE does not support interrupt-continuable behavior, the instruction restarts from the beginning. It is permissible to specify any or none of the floating-point registers to be cleared, but the VPR must always be specified. If the Floating-point and MVE are not implemented, this instruction is available in Secure state, but behaves as a NOP.

T1

Armv8-M Security Extension and Armv8.1-M Main Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	1	D	0	1	1	1	1	1	Vd	1	0	1	1	imm7						0				

T1: VSCCLRM variant

VSCCLRM<c><q> <dreglist>

Decode for this encoding

```

1
2 if !HaveMainExt ()           then UNDEFINED;
3 if !HasArchVersion (Armv8p1) then UNPREDICTABLE;
4 if !IsSecure ()              then UNDEFINED;
5 if HaveMveOrFPExt () && (FPCCR.ASPEN == '0' || CONTROL_S.SFPA == '1') then
6     HandleException(CheckCPEnabled(10));
7 singleRegs = FALSE;
8 d           = UInt (D:Vd);
9 regs        = UInt (imm7);
10 topReg      = d+regs-1;
11 if topReg > 31 then UNPREDICTABLE;

```

T2

Armv8-M Security Extension and Armv8.1-M Main Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	0	1	D	0	1	1	1	1		Vd				1	0	1	0	imm8							

T2: VSCCLRM variant

VSCCLRM<c><q> <sreglist>

Decode for this encoding

```

1
2 if !HaveMainExt ()           then UNDEFINED;
3 if !HasArchVersion (Armv8p1) then UNPREDICTABLE;
4 if !IsSecure ()              then UNDEFINED;
5 if HaveMveOrFPExt () && (FPCCR.ASPEN == '0' || CONTROL_S.SFPA == '1') then
6     HandleException(CheckCPEnabled(10));
7 singleRegs = TRUE;
8 d           = UInt (Vd:D);
9 regs        = UInt (imm8);
10 topReg      = d+regs-1;

```

```
11 if topReg > 63 then UNPREDICTABLE;
12 if topReg > 31 && topReg[0] == '0' then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c, q> See [C1.2.5 Standard assembler syntax fields](#).

<dreglist> Is the list of consecutively numbered 64-bit floating-point registers to be cleared, separated by commas and surrounded by { and }. The first register in the list is encoded in "D:Vd", and "imm7" is set to the number of registers in the list. Because this instruction always clears the VPR register, it is mandatory to have VPR in the register list.

<sreglist> Is the list of consecutively numbered 32-bit floating-point registers to be cleared, separated by commas and surrounded by { and }. The first register in the list is encoded in "Vd:D", and "imm8" is set to the number of registers in the list. Registers above S31 are specified by using D registers in the register list. Because this instruction always clears the VPR register, it is mandatory to have VPR in the register list.

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3
4   if HaveMveOrFPExt() && (FPCCR.ASPEN == '0' || CONTROL_S.SFPA == '1') then
5     ExecuteFPCheck();
6
7     for r = 0 to regs-1
8       if singleRegs then
9         if (d+r) < 32 || !VFPSmallRegisterBank() then
10          S[d+r] = Zeros();
11       else
12         if (d+r) < 16 || !VFPSmallRegisterBank() then
13          D[d+r] = Zeros();
14       VPR = Zeros(32);
```

C2.4.471 VSEL

Floating-point Conditional Select. Floating-point Conditional Select allows the destination register to take the value from either one or the other of two source registers according to the condition codes in the APSR.

The condition codes for VSEL are limited to GE, GT, EQ, and VS. The effect of LT, LE, NE, and VC can be achieved by exchanging the source operands.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	0	D	cc		Vn				Vd				1	0	size	N	0	M	0			Vm		

VSELEQ, Double-precision variant

Armv8-M Floating-point Extension only.

Applies when cc == 00 && size == 11.

```
VSELEQ.F64 <Dd>, <Dn>, <Dm>
// Not permitted in IT block
```

VSELEQ, Half-precision variant

Armv8.1-M Floating-point Extension only.

Applies when cc == 00 && size == 01.

```
VSELEQ.F16 <Sd>, <Sn>, <Sm>
// Not permitted in IT block
```

VSELEQ, Single-precision variant

Armv8-M Floating-point Extension only.

Applies when cc == 00 && size == 10.

```
VSELEQ.F32 <Sd>, <Sn>, <Sm>
// Not permitted in IT block
```

VSELGE, Double-precision variant

Armv8-M Floating-point Extension only.

Applies when cc == 10 && size == 11.

```
VSELGE.F64 <Dd>, <Dn>, <Dm>
// Not permitted in IT block
```

VSELGE, Half-precision variant

Armv8.1-M Floating-point Extension only.

Applies when cc == 10 && size == 01.

```
VSELGE.F16 <Sd>, <Sn>, <Sm>
// Not permitted in IT block
```

VSELGE, Single-precision variant

Armv8-M Floating-point Extension only.

Applies when `cc == 10` && `size == 10`.

```
VSELGE.F32 <Sd>, <Sn>, <Sm>  
    // Not permitted in IT block
```

VSELGT, Double-precision variant

Armv8-M Floating-point Extension only.

Applies when `cc == 11` && `size == 11`.

```
VSELGT.F64 <Dd>, <Dn>, <Dm>  
    // Not permitted in IT block
```

VSELGT, Half-precision variant

Armv8.1-M Floating-point Extension only.

Applies when `cc == 11` && `size == 01`.

```
VSELGT.F16 <Sd>, <Sn>, <Sm>  
    // Not permitted in IT block
```

VSELGT, Single-precision variant

Armv8-M Floating-point Extension only.

Applies when `cc == 11` && `size == 10`.

```
VSELGT.F32 <Sd>, <Sn>, <Sm>  
    // Not permitted in IT block
```

VSELVS, Double-precision variant

Armv8-M Floating-point Extension only.

Applies when `cc == 01` && `size == 11`.

```
VSELVS.F64 <Dd>, <Dn>, <Dm>  
    // Not permitted in IT block
```

VSELVS, Half-precision variant

Armv8.1-M Floating-point Extension only.

Applies when `cc == 01` && `size == 01`.

```
VSELVS.F16 <Sd>, <Sn>, <Sm>  
    // Not permitted in IT block
```

VSELVS, Single-precision variant

Armv8-M Floating-point Extension only.

Applies when `cc == 01` && `size == 10`.

```
VSELVS.F32 <Sd>, <Sn>, <Sm>  
    // Not permitted in IT block
```

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if InITBlock() then UNPREDICTABLE;
6 cond = cc:(cc[1] EOR cc[0]):'0';
7 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
8 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
9 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 case size of
5     when '01'
6         S[d] = Zeros(16) : (if ConditionHolds(cond) then S[n] else S[m])[15:0];
7     when '10'
8         S[d] = if ConditionHolds(cond) then S[n] else S[m];
9     when '11'
10        D[d] = if ConditionHolds(cond) then D[n] else D[m];

```

C2.4.472 VSHL

Vector Shift Left. The immediate variant shifts each element of a vector register to the left by the immediate value.

The register variants shift each element of a vector register by the value specified in a source register. The direction of the shift depends on the sign of the element from the second vector register.

The vector variant shifts each element of the first vector by a value from the least significant byte of the corresponding element of the second vector and places the results in the destination vector.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	1	D	sz			imm		Qd		0	0	1	0	1	0	1	0	1	M	1	Qm		0	

T1: VSHL variant

VSHL<v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 if sz == '000' then SEE "VMOV";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 unsigned = TRUE;
7 n      = integer UNKNOWN;
8 imm6   = sz:imm;
9 case sz of
10   when '001' size = '00'; shiftAmount = UInt(imm6) - 8;
11   when '01x' size = '01'; shiftAmount = UInt(imm6) - 16;
12   when '1xx' size = '10'; shiftAmount = UInt(imm6) - 32;
13   otherwise UNDEFINED;
14 withScalar = FALSE;
15 withVector = FALSE;
16 esize      = 8 << UInt(size);
17 elements   = 32 DIV esize;
18 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	Da	1	1	size	0	1		Qda	1	1	1	1	0	0	1	1	0				Rm		

T2: VSHL variant

VSHL<v><q>.<dt> Qda, Rm

Decode for this encoding


```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if Da == '1' then UNDEFINED;
4 da      = UInt(Da:Qda);
5 m      = UInt(Rm);
6 d      = da;
7 n      = m;
8 m      = da;
9 unsigned = (U == '1');
10 withScalar = TRUE;
11 withVector = FALSE;
12 esize      = 8 << UInt(size);
13 elements   = 32 DIV esize;
14 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
15 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

T3

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	1	0	D	size		Qn	0	Qd	0	0	1	0	0	N	1	M	0		Qm	0					

T3: VSHL variant

VSHL<v><q>.<dt> Qd, Qm, Qn

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' || N == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 n      = UInt(N:Qn);
7 unsigned = (U == '1');
8 withScalar = FALSE;
9 withVector = TRUE;
10 esize      = 8 << UInt(size);
11 elements   = 32 DIV esize;
12 if InitBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> The size of the elements in the vector.
This parameter must be one of the following values:

I8	Encoded as	sz = 001
I16	Encoded as	sz = 01x
I32	Encoded as	sz = 1xx

Assembler symbols for T2 encodings

<dt> This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0

U32 Encoded as size = 10, U = 1

Assembler symbols for T3 encodings

<dt> This parameter determines the following values:
 – Unsigned flag: S indicates signed, U indicates unsigned.
 – Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1
S32	Encoded as	size = 10,	U = 0
U32	Encoded as	size = 10,	U = 1

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Destination vector register.
<Qda>	Source and destination vector register.
<Qm>	Source vector register.
<Qn>	Source vector register, the elements of which containing the amount to shift by.
<Rm>	Source general-purpose register containing the amount to shift by.
<imm>	The number of bits to shift by, in the range 0 to <dt>-1. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[m, curBeat];
8 if withVector then
9     op2 = Q[n, curBeat];
10 if withScalar then
11     shiftAmount = SInt(R[n][7:0]);
12 for e = 0 to elements-1
13     if withVector then
14         shiftAmount = SInt(Elem[op2, e, esize][7:0]);
15     operand = Int(Elem[op1, e, esize], unsigned);
16     Elem[result, e, esize] = (operand << shiftAmount)[esize-1:0];
17
18 for e = 0 to 3
19     if elmtMask[e] == '1' then
20         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
  
```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.473 VSHLC

Whole Vector Left Shift with Carry. Logical shift left by 1-32 bits, with carry across beats, carry in from general-purpose register, and carry out to the same general-purpose register. Permits treating a vector register as a single 128-bit scalar. The carry in is from the lower <imm> bits of the general-purpose register, not the upper bits.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	Da	1																					

T1: VSHLC variant

VSHLC<v><q> Qda, Rdm, #<imm>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if Da == '1' then UNDEFINED;
3 da = UInt(Da:Qda);
4 dm = UInt(Rdm);
5 (-, amount) = DecodeImmShift('10', imm);
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
7 if Rdm == '11x1' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qda>	Source and destination vector register.
<Rdm>	Source and destination general-purpose register for carry in and out.
<imm>	The number of bits to shift by, in the range 1-32.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 (result, carry) = LSL_C(Q[da, curBeat], R[dm], amount);
7 if elmtMask[0] == '1' then
8     R[dm] = carry;
9
10 for e = 0 to 3
11     if elmtMask[e] == '1' then
12         Elem[Q[da, curBeat], e, 8] = Elem[result, e, 8];
```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.474 VSHLL

Vector Shift Left Long. Selects an element of 8 or 16-bits from either the top half (T variant) or bottom half (B variant) of each source element, performs a signed or unsigned left shift by an immediate value, and places the 16 or 32-bit results in the destination vector.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	1	D	1	sz	imm			Qd		T	1	1	1	1	0	1	M	0	Qm		0			

T1: VSHLL variant

VSHLL<T><v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 if imm == '000' && sz IN {'10', '01'} then SEE "VMOVL";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 unsigned = (U == '1');
7 imm5   = sz:imm;
8 case sz of
9   when '01' size = '00'; shiftAmount = UInt(imm5) - 8;
10  when '1x' size = '01'; shiftAmount = UInt(imm5) - 16;
11  otherwise UNDEFINED;
12 esize  = 8 << UInt(size);
13 elements = 16 DIV esize;
14 top     = UInt(T);
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	U	1	1	1	0	0	D	1	1	size	0	1		Qd	T	1	1	1	0	0	0	M	0		Qm		1		

T2: VSHLL variant

VSHLL<T><v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' then UNDEFINED;
4 if size == '10' then UNDEFINED;
5 d      = UInt(D:Qd);
6 m      = UInt(M:Qm);
7 unsigned = (U == '1');

```

```

8  esize      = 8 << UInt(size);
9  elements   = 16 DIV esize;
10 shiftAmount = esize;
11 top        = UInt(T);
12 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- The size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	sz = 01,	U = 0
U8	Encoded as	sz = 01,	U = 1
S16	Encoded as	sz = 1x,	U = 0
U16	Encoded as	sz = 1x,	U = 1

Assembler symbols for T2 encodings

<dt> This parameter determines the following values:

- Unsigned flag: S indicates signed, U indicates unsigned.
- Size: indicates the size of the elements in the vector.

This parameter must be one of the following values:

S8	Encoded as	size = 00,	U = 0
U8	Encoded as	size = 00,	U = 1
S16	Encoded as	size = 01,	U = 0
U16	Encoded as	size = 01,	U = 1

Assembler symbols for all encodings

<T> Specifies which half of the source element is used.
This parameter must be one of the following values:

B	Encoded as	T = 0
	Indicates bottom half	
T	Encoded as	T = 1
	Indicates top half	

<v, q> See [C1.2.5 Standard assembler syntax fields](#).

<Qd> Destination vector register.

<Qm> Source vector register.

<imm> The number of bits to shift by, in the range 1 to <dt>. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz). If <imm> == <dt> the encoding is T2, otherwise the encoding is T1

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1    = Q[m, curBeat];
8  for e = 0 to elements-1
9      operand = Int(Elem[op1, 2*e + top, esize], unsigned);
10     operand = operand << shiftAmount;
11     Elem[result, e, 2*esize] = operand[(2*esize)-1:0];
12
13 for e = 0 to 3
14     if elmtMask[e] == '1' then

```

```
15      Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```



```

2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1    = Q[m, curBeat];
8  for e = 0 to elements-1
9      operand = Int(Elem[op1, e, esize], unsigned);
10     Elem[result, e, esize] = (operand >> shiftAmount)[esize-1:0];
11
12 for e = 0 to 3
13     if elmtMask[e] == '1' then
14         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.476 VSHRN

Vector Shift Right and Narrow. Performs an element-wise narrowing to half-width, with shift, writing the result to either the top half (T variant) or bottom half (B variant) of the result element. The other half of the destination vector element retains its previous value.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	0	sz		imm			Qd	T	1	1	1	1	1	1	M	0		Qm			1	

T1: VSHRN variant

VSHRN<T><v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d = UInt(D:Qd);
4 m = UInt(M:Qm);
5 imm5 = sz:imm;
6 case sz of
7   when '01' size = '00'; shiftAmount = 16 - UInt(imm5);
8   when '1x' size = '01'; shiftAmount = 32 - UInt(imm5);
9   otherwise UNDEFINED;
10 esize = 8 << UInt(size);
11 elements = 16 DIV esize;
12 top = UInt(T);
13 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<T>	Specifies which half of the result element the result is written to. This parameter must be one of the following values: B Encoded as T = 0 Indicates bottom half T Encoded as T = 1 Indicates top half
<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	The size of the elements in the vector. This parameter must be one of the following values: I16 Encoded as sz = 01 I32 Encoded as sz = 1x
<Qd>	Destination vector register.
<Qm>	Source vector register.
<imm>	The number of bits to shift by, in the range 1 to <dt>/2. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```
1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  op1    = Q[m, curBeat];
7  result = Q[d, curBeat];
8  for e = 0 to elements-1
9      operand = UInt(Elem[op1, e, 2*esize]);
10     operand = operand >> shiftAmount;
11     Elem[result, 2*e + top, esize] = operand[esize-1:0];
12
13 for e = 0 to 3
14     if elmtMask[e] == '1' then
15         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];
```

C2.4.477 VSLI

Vector Shift Left and Insert. Takes each element in the operand vector, left shifts them by an immediate value, and inserts the results in the destination vector. Bits shifted out of the left of each element are lost.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D		sz			imm		Qd	0	0	1	0	1	0	1	0	1	M	1		Qm	0	

T1: VSLI variant

VSLI<v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 if sz == '000' then SEE "VMOV";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 unsigned = TRUE;
7 imm6    = sz:imm;
8 case sz of
9     when '001' size = '00'; shiftAmount = UInt(imm6) - 8;
10    when '01x' size = '01'; shiftAmount = UInt(imm6) - 16;
11    when '1xx' size = '10'; shiftAmount = UInt(imm6) - 32;
12    otherwise UNDEFINED;
13 esize   = 8 << UInt(size);
14 elements = 32 DIV esize;
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	The size of the elements in the vector. This parameter must be one of the following values: 8 Encoded as sz = 001 16 Encoded as sz = 01x 32 Encoded as sz = 1xx
<Qd>	Destination vector register.
<Qm>	Source vector register.
<imm>	The number of bits to shift by, in the range 0 to <dt>-1. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
```

```

6  op1    = Q[m, curBeat];
7  result = Q[d, curBeat];
8  mask   = LSL(Ones(esize), shiftAmount);
9  for e = 0 to elements-1
10     shiftedOp      = (LSL(Elem[op1, e, esize], shiftAmount))[esize-1:0];
11     Elem[result, e, esize] = (Elem[result, e, esize] AND NOT(mask)) OR shiftedOp;
12
13 for e = 0 to 3
14     if elmtMask[e] == '1' then
15         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.478 VSQRT

Floating-point Square Root. Floating-point Square Root calculates the square root of a floating-point register value and writes the result to another floating-point register.

T1

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	1	D	1	1	0	0	0	1	Vd	1	0	size	1	1	M	0								Vm

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VSQRT{<c>}{<q>}.F16 <Sd>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VSQRT{<c>}{<q>}.F32 <Sd>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VSQRT{<c>}{<q>}.F64 <Dd>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || D == '1') then UNDEFINED;
4 if size == '01' && InITBlock() then UNPREDICTABLE;
5 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
6 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sm>	Is the 32-bit name of the floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dm>	Is the 64-bit name of the floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   ExecuteFPCheck();
4   case size of
5     when '01' S[d] = Zeros(16) : FPSqrt(S[m][15:0]);
6     when '10' S[d] = FPSqrt(S[m]);
7     when '11' D[d] = FPSqrt(D[m]);

```

C2.4.479 VSRI

Vector Shift Right and Insert. Takes each element in the operand vector, right shifts them by an immediate value, and inserts the results in the destination vector. Bits shifted out of the right of each element are lost.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	D		sz			imm		Qd		0	0	1	0	0	0	0	1	M	1		Qm		0

T1: VSRI variant

VSRI<v><q>.<dt> Qd, Qm, #<imm>

Decode for this encoding

```

1 if sz == '000' then SEE "VMOV";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || M == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 m      = UInt(M:Qm);
6 unsigned = TRUE;
7 imm6    = sz:imm;
8 case sz of
9     when '001' size = '00'; shiftAmount = 16 - UInt(imm6);
10    when '01x' size = '01'; shiftAmount = 32 - UInt(imm6);
11    when '1xx' size = '10'; shiftAmount = 64 - UInt(imm6);
12    otherwise UNDEFINED;
13 esize    = 8 << UInt(size);
14 elements = 32 DIV esize;
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	The size of the elements in the vector. This parameter must be one of the following values: 8 Encoded as sz = 001 16 Encoded as sz = 01x 32 Encoded as sz = 1xx
<Qd>	Destination vector register.
<Qm>	Source vector register.
<imm>	The number of bits to shift by, in the range 1 to <dt>. The encoding of this field is a logical OR of the most significant bits of the imm parameter and the least significant bits of the size field (sz).

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
```

```

6  op1    = Q[m, curBeat];
7  result = Q[d, curBeat];
8  mask   = LSR(Ones(esize), shiftAmount);
9  for e = 0 to elements-1
10     shiftedOp      = (LSR(Elem[op1, e, esize], shiftAmount))[esize-1:0];
11     Elem[result, e, esize] = (Elem[result, e, esize] AND NOT(mask)) OR shiftedOp;
12
13 for e = 0 to 3
14     if elmtMask[e] == '1' then
15         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.480 VST2

Vector Interleaving Store - Stride 2. Saves two 64-bit contiguous blocks of data to memory made up of multiple parts of 2 source registers. The parts of the source registers written to, and the offsets from the base address register, are determined by the pat parameter. If the instruction is executed 2 times with the same base address and source registers, but with different pat values, the effect is to interleave the specified registers with a stride of 2 and to save the data to memory. The base address register can optionally be incremented by 32.

This instruction is subject to beat-wise execution.

This instruction is not VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	1	0	0	1	D	W	0	Rn				Qd				1	1	1	1	size		(0)	pat	(0)	(0)	(0)	(0)	0

T1: VST2 variant (Non write-back: W=0)

VST2<pat><q>.<size> {Qd, Qd+1}, [Rn]

T1: VST2 variant (Writeback: W=1)

VST2<pat><q>.<size> {Qd, Qd+1}, [Rn]!

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 n = UInt(Rn);
6 pattern = UInt(pat);
7 esize = 8 << UInt(size);
8 elements = 32 DIV esize;
9 wback = (W == '1');
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if InVPTBlock() then CONSTRAINED_UNPREDICTABLE;
12 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
13 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;
14 if UInt(D:Qd) > 6 then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<pat>	Specifies the pattern of register elements and memory addresses to access. This parameter must be one of the following values: 0 Encoded as pat = 0 1 Encoded as pat = 1
<q>	See C1.2.5 Standard assembler syntax fields .
<size>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: 8 Encoded as size = 00 16 Encoded as size = 01 32 Encoded as size = 10
<Qd>	Source vector register.
<Rn>	The base register for the target address.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, -) = GetCurInstrBeat();
5
6 // Pre-calculate variables for memory / register access patterns
7 addrWordOffset = curBeat[1] : (UInt(curBeat[1]) + pattern)[0] : curBeat[0];
8 baseAddress    = R[n] + ZeroExtend(addrWordOffset:'00', 32);
9 xBeat         = UInt(curBeat[1] : (pattern[0] EOR curBeat[1]));
10
11 for e = 0 to elements-1
12     address = baseAddress + (e * (esize DIV 8));
13     case esize of
14         when 8
15             y = UInt(e[0]);
16             xE = UInt(curBeat[0] : e[1]);
17         when 16
18             y = UInt(e[0]);
19             xE = UInt(curBeat[0]);
20         when 32
21             y = UInt(curBeat[0]);
22             xE = 0;
23     MemA_MVE[address, esize DIV 8] = Elem[Q[d + y, xBeat], xE, esize];
24
25 // The optional write back to the base register is only performed on the
26 // last beat of the instruction.
27 if wback && IsLastBeat() then
28     R[n] = R[n] + 32;

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.481 VST4

Vector Interleaving Store - Stride 4. Saves two 64-bit contiguous blocks of data to memory made up of multiple parts of 4 source registers. The parts of the source registers written to, and the offsets from the base address register, are determined by the pat parameter. If the instruction is executed 4 times with the same base address and source registers, but with different pat values, the effect is to interleave the specified registers with a stride of 4 and to save the data to memory. The base address register can optionally be incremented by 64.

This instruction is subject to beat-wise execution.

This instruction is not VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	1	1	1	1	1	0	0	1	D	W	0	Rn				Qd				1	1	1	1	size		pat		(0)	(0)	(0)	(0)	1

T1: VST4 variant (Non write-back: W=0)

VST4<pat><q>.<size> {Qd, Qd+1, Qd+2, Qd+3}, [Rn]

T1: VST4 variant (Write-back: W=1)

VST4<pat><q>.<size> {Qd, Qd+1, Qd+2, Qd+3}, [Rn]!

Decode for this encoding

```

1  if size == '11' then SEE "Related encodings";
2  CheckDecodeFaults(ExtType_Mve);
3  if D == '1' then UNDEFINED;
4  d      = UInt(D:Qd);
5  n      = UInt(Rn);
6  pattern = UInt(pat);
7  esize   = 8 << UInt(size);
8  elements = 32 DIV esize;
9  wback   = (W == '1');
10 if InITBlock()          then CONSTRAINED_UNPREDICTABLE;
11 if InVPTBlock()         then CONSTRAINED_UNPREDICTABLE;
12 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
13 if Rn == '1111'         then CONSTRAINED_UNPREDICTABLE;
14 if UInt(D:Qd) > 4       then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for all encodings

<pat>	Specifies the pattern of register elements and memory addresses to access. This parameter must be one of the following values:
0	Encoded as pat = 00
1	Encoded as pat = 01
2	Encoded as pat = 10
3	Encoded as pat = 11
<q>	See C1.2.5 Standard assembler syntax fields .
<size>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values:
8	Encoded as size = 00
16	Encoded as size = 01
32	Encoded as size = 10
<Qd>	Source vector register.
<Rn>	The base register for the target address.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, -) = GetCurInstrBeat();
5
6 // Pre-calculate variables for memory / register access patterns
7 addrWordOffset = curBeat[1] : (UInt(curBeat[1]) + pattern)[1:0] : curBeat[0];
8 baseAddress    = R[n] + ZeroExtend(addrWordOffset:'00', 32);
9 xBeat         = UInt(curBeat[1] : (pattern[1] EOR (pattern[0] AND curBeat[1])));
10
11 for e = 0 to elements-1
12     address = baseAddress + (e * (esize DIV 8));
13     case esize of
14         when 8
15             y = UInt(e[1:0]);
16             xE = UInt((pattern[0] EOR curBeat[1]) : curBeat[0]);
17         when 16
18             y = UInt(curBeat[0] : e[0]);
19             xE = UInt(pattern[0] EOR curBeat[1]);
20         when 32
21             y = UInt((pattern[0] EOR curBeat[1]) : curBeat[0]);
22             xE = 0;
23     MemA_MVE[address, esize DIV 8] = Elem[Q[d + y, xBeat], xE, esize];
24
25 // The optional write back to the base register is only performed on the
26 // last beat of the instruction.
27 if wback && IsLastBeat() then
28     R[n] = R[n] + 64;

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.482 VSTM

Floating-point Store Multiple. Floating-point Store Multiple stores multiple extension registers to consecutive memory locations using an address from a general-purpose register.

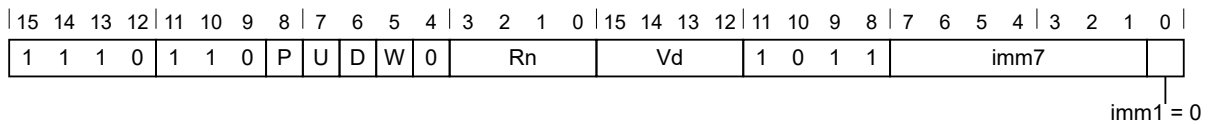
This instruction is subject to stack limit checking.

This instruction is used by the alias [VPUSH](#).

This instruction is interrupt-continuable.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE



Decrement Before variant

Applies when **P == 1 && U == 0 && W == 1**.

VSTMDB{<c>}{<q>}{.<size>} <Rn>!, <dreglist>

Increment After variant

Applies when **P == 0 && U == 1**.

VSTM{<c>}{<q>}{.<size>} <Rn>{!}, <dreglist>

VSTMIA{<c>}{<q>}{.<size>} <Rn>{!}, <dreglist>

Decode for this encoding

```

1 if P == '0' && U == '0' && W == '0' then SEE "Related encodings";
2 if P == '1' && W == '0' then SEE VSTR;
3 CheckDecodeFaults(ExtType_MveOrFp);
4 if P == U && W == '1' then UNDEFINED;
5 // Remaining combinations are PUW = 010 (IA without !), 011 (IA with !), 101 (DB with !)
6 single_regs = FALSE; add = (U == '1'); wback = (W == '1'); imm8 = imm7 : imm1;
7 d = UInt(D:Vd); n = UInt(Rn); imm32 = ZeroExtend(imm8:'00', 32);
8 regs = UInt(imm8) DIV 2;
9 if n == 15 then UNPREDICTABLE;
10 if regs == 0 || (d+regs) > 32 then UNPREDICTABLE;
11 if !HasArchVersion(Armv8pl) then
12     if VFPSmallRegisterBank() && (d+regs) > 16 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **regs == 0**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as a [VSTM](#) with the same addressing mode but stores no registers.

CONSTRAINED UNPREDICTABLE behavior

If **(d+regs) > 32**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.

- The instruction executes as NOP.
- The memory locations specified by the instruction and the number of registers specified by the instruction if the register list had not gone out of range, become UNKNOWN. If the instruction specifies write-back, then that register becomes UNKNOWN. This behavior does not affect any other memory locations.

T2

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	P	U	D	W	0	Rn				Vd				1	0	1	0	imm8							

Decrement Before variant

Applies when **P == 1 && U == 0 && W == 1**.

VSTMDB{<c>}{<q>}{.<size>} <Rn>!, <sreglist>

Increment After variant

Applies when **P == 0 && U == 1**.

VSTM{<c>}{<q>}{.<size>} <Rn>{!}, <sreglist>

VSTMIA{<c>}{<q>}{.<size>} <Rn>{!}, <sreglist>

Decode for this encoding

```

1 if P == '0' && U == '0' then SEE "Related encodings";
2 if P == '1' && W == '0' then SEE VSTR;
3 CheckDecodeFaults(ExtType_MveOrFp);
4 if P == '1' && U == '1' && W == '1' then UNDEFINED;
5 // Remaining combinations are PUW = 010 (IA without !), 011 (IA with !), 101 (DB with !)
6 single_regs = TRUE; add = (U == '1'); wback = (W == '1'); d = UInt(Vd:D); n = UInt(Rn);
7 imm32 = ZeroExtend(imm8:'00', 32); regs = UInt(imm8);
8 if n == 15 then UNPREDICTABLE;
9 if regs == 0 || (d+regs) > 32 then UNPREDICTABLE;
```

CONSTRAINED UNPREDICTABLE behavior

If **regs == 0**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The instruction operates as a **VSTM** with the same addressing mode but stores no registers.

CONSTRAINED UNPREDICTABLE behavior

If **(d+regs) > 32**, then one of the following behaviors must occur:

- The instruction is UNDEFINED.
- The instruction executes as NOP.
- The memory locations specified by the instruction and the number of registers specified by the instruction if the register list had not gone out of range, become UNKNOWN. If the instruction specifies write-back, then that register becomes UNKNOWN. This behavior does not affect any other memory locations.

Alias conditions

Alias	is preferred when
VPUSH	P == '1' && U == '0' && W == '1' && RN == '1101'

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<size>	An optional data size specifier. If present, it must be equal to the size in bits, 32 or 64, of the registers being transferred.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
!	Specifies base register write-back. Encoded in the "W" field as 1 if present, otherwise 0.
<sreglist>	Is the list of consecutively numbered 32-bit floating-point registers to be transferred. The first register in the list is encoded in "Vd:D", and "imm8" is set to the number of registers in the list. The list must contain at least one register.
<dreglist>	Is the list of consecutively numbered 64-bit floating-point registers to be transferred. The first register in the list is encoded in "D:Vd", and "imm8" is set to twice the number of registers in the list. The list must contain at least one register.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      address = if add then R[n]          else R[n]-imm32;
5      regval  = if add then R[n]+imm32 else R[n]-imm32;
6
7      // Determine if the stack pointer limit should be checked
8      if n == 13 && wback then
9          violatesLimit = ViolatesSPLim(LookUpSP(), regval);
10     else
11         violatesLimit = FALSE;
12
13     // Memory operation only performed if limit not violated
14     if !violatesLimit then
15         for r = 0 to regs-1
16             if single_regs then
17                 if (d+r) < 32 || !VFPSmallRegisterBank() then
18                     MemA[address, 4] = S[d+r];
19                     address = address+4;
20             else
21                 // Store as two word-aligned words in the correct order for current
22                 // endianness.
23                 if (d+r) < 16 || !VFPSmallRegisterBank() then
24                     bigEndian = BigEndian(address, 8);
25                     MemA[address, 4] = if bigEndian then D[d+r][63:32] else D[d+r][31:0];
26                     MemA[address+4, 4] = if bigEndian then D[d+r][31:0] else D[d+r][63:32];
27                 elsif boolean UNKNOWN then
28                     MemA[address, 4] = bits(32) UNKNOWN;
29                     MemA[address+4, 4] = bits(32) UNKNOWN;
30                     address = address+8;
31
32     // If the stack pointer is being updated a fault will be raised if
33     // the limit is violated
34     if wback then RSPCheck[n] = regval;

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.

- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.483 VSTR

Floating-point Store Register. Floating-point Store Register stores a single Floating-point Extension register to memory, using an address from a general-purpose register, with an optional offset.

T1

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	1	U	D	0	0	Rn				Vd				1	0	1	1	imm8							

T1 variant

VSTR{<c>}{<q>}{.64} <Dd>, [<Rn>{, # {+/-}<imm>}]

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_MveOrFp);
2 if VFPSmallRegisterBank() && (D == '1') then UNDEFINED;
3 fp_size = 64; add = (U == '1'); imm32 = ZeroExtend(imm8:'00', 32);
4 d = UInt(D:Vd); n = UInt(Rn);
5 if n == 15 then UNPREDICTABLE;
```

T2

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	1	U	D	0	0	Rn				Vd				1	0	1	0	imm8							

T2 variant

VSTR{<c>}{<q>}{.32} <Sd>, [<Rn>{, # {+/-}<imm>}]

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_MveOrFp);
2 fp_size = 32; add = (U == '1'); imm32 = ZeroExtend(imm8:'00', 32);
3 d = UInt(Vd:D); n = UInt(Rn);
4 if n == 15 then UNPREDICTABLE;
```

T3

Armv8-M Floating-point Extension, Armv8.1-M Floating-point Extension, or Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	1	U	D	0	0	Rn				Vd				1	0	0	1	imm8							

T3 variant

VSTR{<c>}{<q>}.16 <Sd>, [<Rn> {, # {+/-}<imm>}]

Decode for this encoding

```
1 CheckDecodeFaults(ExtType_MveOrFp);
2 fp_size = 16; add = (U == '1'); imm32 = ZeroExtend(imm8:'0', 32);
3 d = UInt(Vd:D); n = UInt(Rn);
4 if n == 15 then UNPREDICTABLE;
```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
.64	Optional data size specifiers.
<Dd>	The source register for a doubleword store.
.32	Optional data size specifiers.
<Sd>	The source register for a singleword store.
<Rn>	Is the general-purpose base register, encoded in the "Rn" field.
+/-	Specifies the offset is added to or subtracted from the base register, defaulting to + if omitted and encoded in the "U" field. It can have the following values: - when U = 0 + when U = 1
<imm>	The immediate offset used for forming the address. Values are multiples of 4 in the range 0-1020. <imm> can be omitted, meaning an offset of +0.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      ExecuteFPCheck();
4      address = if add then (R[n] + imm32) else (R[n] - imm32);
5      case fp_size of
6          when 16
7              MemA[address, 2] = S[d][15:0];
8          when 32
9              MemA[address, 4] = S[d];
10         when 64
11             // Store as two word-aligned words in the correct order for current endianness.
12             bigEndian = BigEndian(address, 8);
13             MemA[address, 4] = if bigEndian then D[d][63:32] else D[d][31:0];
14             MemA[address+4, 4] = if bigEndian then D[d][31:0] else D[d][63:32];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

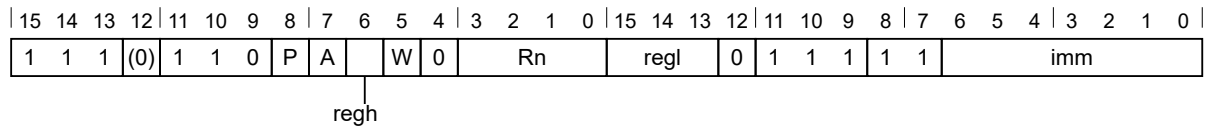
C2.4.484 VSTR (System Register)

Store System Register. Store a system register in memory. The target address is calculated from a base register plus an immediate offset. Access to the FPCXT payloads generates an UNDEFINED exception if the instruction is executed from Non-secure state. If CP10 is not enabled and either the Main extension is not implemented or the Floating-point context is active, access to FPCXT_NS will generate a NOCP UsageFault. Accesses to FPCXT_NS will not trigger lazy state preservation if there is no active Floating-point context. Accesses to FPCXT_NS do not trigger Floating-point context creation regardless of the value of FPCCR.ASPEN. FPSCR_nzcvqc allows access to FPSCR condition and saturation flags. The VPR register can only be accessed from privileged mode. FPCXT_NS, enables saving and restoration of the Non-secure floating-point context. If the Floating-point context is active then the current FPSCR value is accessed and the default value in FPDSCR_NS is written into FPSCR, otherwise the default value in FPDSCR_NS is accessed. The payloads cannot be accessed if neither the Floating-point extension nor MVE are implemented. FPCXT_S, enables saving and restoration of the Secure floating-point context.

This instruction is subject to stack limit checking.

T1

Armv8.1-M Floating-point Extension or Armv8.1-M MVE



T1: VSTR variant (Offset: P=1, W=0)

VSTR<c><q> <reg>, [Rn{, #+/-<imm>}]

T1: VSTR variant (Pre-indexed: P=1, W=1)

VSTR<c><q> <reg>, [Rn, #+/-<imm>]!

T1: VSTR variant (Post-indexed: P=0, W=1)

VSTR<c><q> <reg>, [Rn], #+/-<imm>

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 if !HasArchVersion(Armv8p1) then
3     UFSR.NOCP = '1';
4     HandleException(CreateException(UsageFault));
5 fpCxtAnyAccess = (regh:regl == '111x');
6 fpCxtNSAccess = (regh:regl == '1110');
7 fpInactive = !HaveMveOrFPExt() || (FPCCR_NS.ASPEN == '1' && CONTROL.FPCA == '0');
8 if fpCxtAnyAccess && !IsSecure() then UNDEFINED;
9 if fpCxtNSAccess then
10     if !HaveMainExt() || !fpInactive then
11         HandleException(CheckCPEnabled(10));
12 else
13     CheckDecodeFaults(ExtType_MveOrFp);
14 n = UInt(Rn);
15 index = (P == '1');
16 add = (A == '1');
17 wback = (W == '1');
18 r = regh:regl;
19 imm32 = ZeroExtend(imm:'00', 32);
20 if (regh:regl) IN {'10xx', '01xx', '0011', '0000'} then CONSTRAINED_UNPREDICTABLE;
21 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<c, q>	See C1.2.5 Standard assembler syntax fields .
<reg>	The system register to access This parameter must be one of the following values: FPSCR Encoded as regh = 0, regl = 001 FPSCR_nzcvqc Encoded as regh = 0, regl = 010 VPR Encoded as regh = 1, regl = 100 P0 Encoded as regh = 1, regl = 101 FPCXT_NS Encoded as regh = 1, regl = 110 FPCXT_S Encoded as regh = 1, regl = 111
<Rn>	The base register for the target address.
<imm>	The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 4.

Operation for all encodings

```

1  if ConditionPassed() then
2      EncodingSpecificOperations();
3      if !fpCxtNSAccess then
4          ExecuteFPCheck();
5      elseif !fpInactive then
6          PreserveFPState();
7      SerializeVFP();
8      VFPExcBarrier();
9
10     offsetAddr = if add then (R[n] + imm32) else (R[n] - imm32);
11     address    = if index then offsetAddr else R[n];
12
13     // Determine if the stack pointer limit should be checked
14     if n == 13 && wback then
15         violatesLimit = ViolatesSPLim(LookUpSP(), offsetAddr);
16     else
17         violatesLimit = FALSE;
18     // Memory operation only performed if limit not violated
19     if !violatesLimit then
20         case r of
21             when '0001'
22                 MemA[address, 4] = FPSCR;
23             when '0010'
24                 // Only read the N, Z, C, V, and QC flags
25                 MemA[address, 4] = FPSCR[31:27]:Zeros(27);
26             when '1100'
27                 if HaveMve() then
28                     if CurrentModeIsPrivileged() then
29                         MemA[address, 4] = VPR;
30                     elseif boolean IMPLEMENTATION_DEFINED "No MVE VPR mem access" then
31                         MemA[address, 4] = bits(32) UNKNOWN;
32                 else
33                     UNPREDICTABLE;
34             when '1101'
35                 if HaveMve() then
36                     MemA[address, 4] = Zeros(16):VPR.P0;
37                 else
38                     UNPREDICTABLE;
39             when '1110'
40                 if HaveFPEExt() || HaveMve() then
41                     FPCXT_Type cxt = Zeros(32);
42                     if !fpInactive then
43                         cxt.SFPA = CONTROL_S.SFPA;
44                         cxt[27:0] = FPSCR[27:0];
45                     else
46                         cxt[27:0] = FPDSCR_NS[27:0];
47                     MemA[address, 4] = cxt;
48                     // If the floating-point context isn't secure the FPSCR
49                     // value is set to the NS default so any NS functions

```

```

50         // that are called before a floating-point instruction
51         // is executed in the secure state will get the same
52         // FPSCR value as functions called after a secure
53         // floating-point instruction (which is the value
54         // of FPDSCR_NS).
55         if !fpInactive && CONTROL_S.SFPA == '0' then
56             // Ensuring that the memory access has succeeded before
57             // updating the value of FPSCR.
58             FPSCR = FPDSCR_NS[31:0];
59         when '1111'
60             FPCXT_Type cxt    = Zeros(32);
61             cxt.SFPA          = CONTROL_S.SFPA;
62             cxt[27:0]         = FPSCR[27:0];
63             MemA[address, 4] = cxt;
64             // Ensuring that the memory access has succeeded before
65             // updating the value of FPSCR.
66             FPSCR            = FPDSCR_NS[31:0];
67             CONTROL_S.SFPA   = '0';
68         otherwise
69             UNPREDICTABLE;
70
71         // If the stack pointer is being updated a fault will be raised if
72         // the limit is violated
73         if wback then
74             RSPCheck[n] = offsetAddr;

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.485 VSTRB, VSTRH, VSTRW

Vector Store Register. Store consecutive elements to memory from a vector register. In indexed mode, the target address is calculated from a base register offset by an immediate value. Otherwise, the base register address is used directly. The sum of the base register and the immediate value can optionally be written back to the base register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	0	P	A	0	W	0	0	Rn	Qd	0	1	1	1	size	imm											

T1: VSTRB variant (Offset: P=1, W=0)

VSTRB<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T1: VSTRB variant (Pre-indexed: P=1, W=1)

VSTRB<v><q>.<dt> Qd, [Rn, #+/-<imm>]!

T1: VSTRB variant (Post-indexed: P=0, W=1)

VSTRB<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 if size == '00' then UNDEFINED;
5 d = UInt(Qd);
6 n = UInt(Rn);
7 msize = 8;
8 mbytes = msize DIV 8;
9 esize = 8 << UInt(size);
10 elements = 32 DIV esize;
11 imm32 = ZeroExtend(imm, 32);
12 index = (P == '1');
13 add = (A == '1');
14 wback = (W == '1');
15 unsigned = TRUE;
16 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	0	P	A	0	W	0	1	Rn	Qd	0	1	1	1	size	imm											

T2: VSTRH variant (Offset: P=1, W=0)

VSTRH<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T2: VSTRH variant (Pre-indexed: P=1, W=1)

VSTRH<v><q>.<dt> Qd, [Rn, #+/-<imm>] !

T2: VSTRH variant (Post-indexed: P=0, W=1)

VSTRH<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```
1 if P == '0' && W == '0' then SEE "Related encodings";
2 if size == '11' then SEE "Related encodings";
3 CheckDecodeFaults(ExtType_Mve);
4 if size == '0x' then UNDEFINED;
5 d = UInt(Qd);
6 n = UInt(Rn);
7 msize = 16;
8 mbytes = msize DIV 8;
9 esize = 8 << UInt(size);
10 elements = 32 DIV esize;
11 imm32 = ZeroExtend(imm:'0', 32);
12 index = (P == '1');
13 add = (A == '1');
14 wback = (W == '1');
15 unsigned = TRUE;
16 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

T5

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	P	A	D	W	0	Rn				Qd				1	1	1	1	0	0	imm					

T5: VSTRB variant (Offset: P=1, W=0)

VSTRB<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T5: VSTRB variant (Pre-indexed: P=1, W=1)

VSTRB<v><q>.<dt> Qd, [Rn, #+/-<imm>] !

T5: VSTRB variant (Post-indexed: P=0, W=1)

VSTRB<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```
1 if P == '0' && W == '0' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 n = UInt(Rn);
6 msize = 8;
7 mbytes = msize DIV 8;
8 esize = msize;
9 elements = 32 DIV esize;
10 imm32 = ZeroExtend(imm, 32);
11 index = (P == '1');
12 add = (A == '1');
13 wback = (W == '1');
14 unsigned = TRUE;
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

```
16 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
17 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;
```

T6

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	P	A	D	W	0	Rn				Qd				1	1	1	1	0	1	imm					

T6: VSTRH variant (Offset: P=1, W=0)

VSTRH<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T6: VSTRH variant (Pre-indexed: P=1, W=1)

VSTRH<v><q>.<dt> Qd, [Rn, #+/-<imm>]!

T6: VSTRH variant (Post-indexed: P=0, W=1)

VSTRH<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```
1 if P == '0' && W == '0' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 n = UInt(Rn);
6 msize = 16;
7 mbytes = msize DIV 8;
8 esize = msize;
9 elements = 32 DIV esize;
10 imm32 = ZeroExtend(imm:'0', 32);
11 index = (P == '1');
12 add = (A == '1');
13 wback = (W == '1');
14 unsigned = TRUE;
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
16 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
17 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;
```

T7

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	0	P	A	D	W	0	Rn				Qd				1	1	1	1	1	0	imm					

T7: VSTRW variant (Offset: P=1, W=0)

VSTRW<v><q>.<dt> Qd, [Rn{, #+/-<imm>}]

T7: VSTRW variant (Pre-indexed: P=1, W=1)

VSTRW<v><q>.<dt> Qd, [Rn, #+/-<imm>]!

T7: VSTRW variant (Post-indexed: P=0, W=1)

VSTRW<v><q>.<dt> Qd, [Rn], #+/-<imm>

Decode for this encoding

```

1 if P == '0' && W == '0' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' then UNDEFINED;
4 d      = UInt(D:Qd);
5 n      = UInt(Rn);
6 msize  = 32;
7 mbytes = msize DIV 8;
8 esize  = msize;
9 elements = 32 DIV esize;
10 imm32  = ZeroExtend(imm:'00', 32);
11 index  = (P == '1');
12 add    = (A == '1');
13 wback  = (W == '1');
14 unsigned = TRUE;
15 if InitBlock() then CONSTRAINED_UNPREDICTABLE;
16 if Rn == '1101' && W == '1' then CONSTRAINED_UNPREDICTABLE;
17 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for T1 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be one of the following values:
16 Encoded as size = 01
32 Encoded as size = 10

<imm> The signed immediate value that is added to base register to calculate the target address.

Assembler symbols for T2 encodings

<dt> Size: indicates the size of the elements in the vector.
This parameter must be the following value:
32 Encoded as size = 10

<imm> The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 2.

Assembler symbols for T5 encodings

<dt> Data size. This parameter is ".8".

<imm> The signed immediate value that is added to base register to calculate the target address.

Assembler symbols for T6 encodings

<dt> Data size. This parameter is ".16".

<imm> The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 2.

Assembler symbols for T7 encodings

<dt> Data size. This parameter is ".32".

<imm> The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 4.

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Source vector register.
<Rn>	The base register for the target address.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 offsetAddr = if add then (R[n] + imm32) else (R[n] - imm32);
7 address    = if index then offsetAddr else R[n];
8 address    = address + (curBeat * mbytes * elements);
9
10 for e = 0 to elements-1
11     if elmtMask[e*(esize >> 3)] == '1' then
12         elem = Elem[Q[d, curBeat], e, esize][(mbytes*8)-1:0];
13         MemA_MVE[address + (e * mbytes), mbytes] = elem;
14
15 // The optional write back to the base register is only performed on the
16 // last beat of the instruction.
17 if wback && IsLastBeat() then
18     R[n] = offsetAddr;

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.486 VSTRB, VSTRH, VSTRW, VSTRD (vector)

Vector Scatter Store. Store data from elements of $Q[d]$ into a memory byte, halfword, word, or doubleword at the address contained in either:

- a) A base register $R[n]$ plus an offset contained in each element of $Q[m]$, optionally shifted by the element size, or
- b) Each element of $Q[m]$ plus an immediate offset. The base element can optionally be written back, irrespective of predication, with that value incremented by the immediate or by the immediate scaled by the memory element size.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	0	0	1	D	0	0	Rn				Qd		0	1	1	1	size	0	M	0	Qm			os		

T1: VSTRB variant

VSTRB<v><q>.<dt> Qd, [Rn, Qm]

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(Rn);
7 msize = '00';
8 esize = 8 << UInt(size);
9 elements = 32 DIV esize;
10 mesize = 8;
11 add = TRUE;
12 useReg = TRUE;
13 scaleOffset = (os == '1');
14 wback = FALSE;
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
16 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;
17 if os == '1' then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	0	0	1	D	0	0	Rn				Qd		0	1	1	1	size		0	M	1	Qm		os		

T2: VSTRH variant

VSTRH<v><q>.<dt> Qd, [Rn, Qm{, UXTW #os}]

Decode for this encoding

T3

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
1	1	1	(0)	1	1	0	0	1	D	0	0	Rn				Qd				0	1	1	1	size				1	M	0	Qm				os

```
VSTRW<v><q>.<dt> Qd, [Rn, Qm{, UXTW #os}]
```

T4

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	(0)	1	1	0	0	1	D	0	0	Rn			Qd			0	1	1	1	size			1	M	1	Qm			os

VSTRD<v><q>.<dt> Qd, [Rn, Qm{, UXTW #os}]

1424

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 if size != '11' then UNDEFINED;
4 d      = UInt (D:Qd);
5 m      = UInt (M:Qm);
6 n      = UInt (Rn);
7 msize  = '11';
8 esize  = 8 << UInt (size);
9 elements = 32 DIV esize;
10 mesize = 64;
11 add    = TRUE;
12 useReg  = TRUE;
13 scaleOffset = (os == '1');
14 wback   = FALSE;
15 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
16 if Rn == '1111' then CONSTRAINED_UNPREDICTABLE;

```

T5

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	A	D	W	0	Qm	(0)	Qd	1	1	1	1	0	M	imm										

T5: VSTRW variant (Non write-back: W=0)

VSTRW<v><q>.<dt> Qd, [Qm{, #+/-<imm>}]

T5: VSTRW variant (Write-back: W=1)

VSTRW<v><q>.<dt> Qd, [Qm{, #+/-<imm>}]!

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (M:Qm);
5 n      = integer UNKNOWN;
6 size   = '10';
7 msize  = size;
8 offset = ZeroExtend(imm:Zeros(UInt (size)), 32);
9 esize  = 8 << UInt (size);
10 elements = 32 DIV esize;
11 mesize = 32;
12 add    = (A == '1');
13 useReg  = FALSE;
14 scaleOffset = FALSE;
15 wback   = (W == '1');
16 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T6

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	0	1	A	D	W	0	Qm	(0)	Qd	1	1	1	1	1	M	imm										

T6: VSTRD variant (Non write-back: W=0)

VSTRD<v><q>.<dt> Qd, [Qm{, #+/-<imm>}]

T6: VSTRD variant (Write-back: W=1)

VSTRD<v><q>.<dt> Qd, [Qm{, #+/-<imm>}] !

Decode for this encoding

```
1 CheckDecodeFaults (ExtType_Mve);
2 if D == '1' || M == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (M:Qm);
5 n      = integer UNKNOWN;
6 size   = '11';
7 msize  = size;
8 offset = ZeroExtend(imm:Zeros(UInt(size)), 32);
9 esize  = 8 << UInt(size);
10 elements = 32 DIV esize;
11 mesize = 64;
12 add    = (A == '1');
13 useReg = FALSE;
14 scaleOffset = FALSE;
15 wback  = (W == '1');
16 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for T1 encodings

<dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

8	Encoded as	size = 00
16	Encoded as	size = 01
32	Encoded as	size = 10

<Qm> Vector offset register. The elements of this register contain the unsigned offsets to add to the base address.

Assembler symbols for T2 encodings

<dt> Size: indicates the size of the elements in the vector.
 This parameter must be one of the following values:

16	Encoded as	size = 01
32	Encoded as	size = 10

<Qm> Vector offset register. The elements of this register contain the unsigned offsets to add to the base address.

Assembler symbols for T3 encodings

<dt> This parameter must be the following value:

32	Encoded as	size = 10
----	------------	-----------

<Qm> Vector offset register. The elements of this register contain the unsigned offsets to add to the base address.

Assembler symbols for T4 encodings

<dt> This parameter must be the following value:

64	Encoded as	size = 11
----	------------	-----------

<Qm> Vector offset register. The elements of this register contain the unsigned offsets to add to the base address.

Assembler symbols for T5 encodings

<dt>	Data size. This parameter is ".32".
<Qm>	The base register for the target address.
<imm>	The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 4.

Assembler symbols for T6 encodings

<dt>	Data size. This parameter is ".64".
<Qm>	The base register for the target address.
<imm>	The signed immediate value that is added to base register to calculate the target address. This value must be a multiple of 8.

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<Qd>	Source vector register.
<Rn>	The base register for the target address.
<os>	The amount by which the vector offset is left shifted by before being added to the general-purpose base address. If the value is present it must correspond to memory transfer size (1=half word, 2=word, 3=double word). This parameter must be one of the following values: <omitted> Encoded as os = 0 <Offset scaled> Encoded as os = 1

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  if esize == 64 then
7      // 64 bit accesses read their base address or offset from the first element
8      // in each pair of 32 bit elements.
9      if useReg then
10         baseAddr = R[n];
11         offset = Q[m, UInt(curBeat[1]:'0')];
12         if scaleOffset then
13             offset = LSL(offset, UInt(msize));
14         else
15             baseAddr = Q[m, UInt(curBeat[1]:'0')];
16             offsetAddress = if add then baseAddr + offset else baseAddr - offset;
17             bigEndian = BigEndian(offsetAddress, 8);
18             address = (if (curBeat[0] == '0') == bigEndian then offsetAddress + 4
19                        else offsetAddress);
20             if elmtMask[0] == '1' then
21                 MemA_MVE[address, 4] = Q[d, curBeat];
22                 // Address write-back is not predicated
23                 if wback && (curBeat[0] == '1') then
24                     Q[m, curBeat-1] = offsetAddress[31:0];
25             else
26                 // 32, 16, or 8 bit accesses
27                 for e = 0 to (elements - 1)
28                     if useReg then
29                         baseAddr = R[n];
30                         offset = ZeroExtend(Elem[Q[m, curBeat], e, esize], 32);
31                         if scaleOffset then
32                             offset = LSL(offset, UInt(msize));
33                     else
34                         // 16 / 8 bit vector+immediate accesses are not supported
35                         baseAddr = Q[m, curBeat];

```

```

36     address = if add then baseAddr + offset else baseAddr - offset;
37     if elmtMask[e*(esize>>3)] == '1' then
38         memValue = Elem[Q[d, curBeat], e, esize][mesize-1:0];
39         MemA_MVE[address, mesize DIV 8] = memValue;
40         // Address write-back is not predicated
41     if wback then
42         Elem[Q[m, curBeat], e, esize] = address[esize-1:0];

```

Restricted behavior

Data Independent Timing behavior

If [AIRCR.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value of the data being stored.
- The point at which an asynchronous exception is taken does not vary based on the value being stored by this instruction.

The timing invariance does not apply to the addresses being accessed.

C2.4.487 VSUB

Floating-point Subtract. Floating-point Subtract subtracts one floating-point register value from another floating-point register value, and places the results in the destination floating-point register.

T2

Armv8-M Floating-point Extension only, size == 11 UNDEFINED in single-precision only implementations.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	1	1	Vn	Vd	1	0	size	N	1	M	0	Vm										

Half-precision scalar variant

Armv8.1-M Floating-point Extension only.

Applies when **size** == 01.

VSUB{<c>}{<q>}.F16 {<Sd>, } <Sn>, <Sm>

Single-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 10.

VSUB{<c>}{<q>}.F32 {<Sd>, } <Sn>, <Sm>

Double-precision scalar variant

Armv8-M Floating-point Extension only.

Applies when **size** == 11.

VSUB{<c>}{<q>}.F64 {<Dd>, } <Dn>, <Dm>

Decode for this encoding

```

1 dp_operation = (size == '11');
2 CheckFPDecodeFaults(size);
3 if VFPSmallRegisterBank() && dp_operation && (M == '1' || N == '1' || D == '1') then
4     UNDEFINED;
5 if size == '01' && InITBlock() then UNPREDICTABLE;
6 d = if dp_operation then UInt(D:Vd) else UInt(Vd:D);
7 n = if dp_operation then UInt(N:Vn) else UInt(Vn:N);
8 m = if dp_operation then UInt(M:Vm) else UInt(Vm:M);

```

Assembler symbols for all encodings

<c>	See C1.2.5 Standard assembler syntax fields .
<q>	See C1.2.5 Standard assembler syntax fields .
<Sd>	Is the 32-bit name of the floating-point destination register, encoded in the "Vd:D" field.
<Sn>	Is the 32-bit name of the first floating-point source register, encoded in the "Vn:N" field.
<Sm>	Is the 32-bit name of the second floating-point source register, encoded in the "Vm:M" field.
<Dd>	Is the 64-bit name of the floating-point destination register, encoded in the "D:Vd" field.
<Dn>	Is the 64-bit name of the first floating-point source register, encoded in the "N:Vn" field.
<Dm>	Is the 64-bit name of the second floating-point source register, encoded in the "M:Vm" field.

Operation for all encodings

```

1 if ConditionPassed() then
2     EncodingSpecificOperations();

```

```
3      ExecuteFPCheck();
4      case size of
5          when '01'
6              S[d] = Zeros(16) : FPSub(S[n][15:0], S[m][15:0], TRUE);
7          when '10'
8              S[d] = FPSub(S[n], S[m], TRUE);
9          when '11'
10         D[d] = FPSub(D[n], D[m], TRUE);
```

C2.4.488 VSUB (floating-point)

Vector Subtract. Subtract the value of the elements in the second source vector register from either the respective elements in the first source vector register or a general-purpose register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	1	0	D	1	sz	Qn	0	Qd	0	1	1	0	1	N	1	M	0	Qm	0						

T1: VSUB variant

VSUB<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (M:Qm);
5 n      = UInt (N:Qn);
6 esize  = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 withScalar = FALSE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE with Half-precision and Single-precision Floating-point Extension

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	sz	1	1	1	0	0	D	1	1	Qn	0	Qd	1	1	1	1	1	N	1	0	0								Rm

T2: VSUB variant

VSUB<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 CheckDecodeFaults (ExtType_MveFp);
2 if D == '1' || N == '1' then UNDEFINED;
3 d      = UInt (D:Qd);
4 m      = UInt (Rm);
5 n      = UInt (N:Qn);
6 esize  = if sz == '1' then 16 else 32;
7 elements = 32 DIV esize;
8 withScalar = TRUE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the floating-point format used. This parameter must be one of the following values: F32 Encoded as sz = 0 F16 Encoded as sz = 1
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.
<Rm>	Source general-purpose register.

Operation for all encodings

```

1 EncodingSpecificOperations();
2 ExecuteFPCheck();
3
4 (curBeat, elmtMask) = GetCurInstrBeat();
5
6 result = Zeros(32);
7 op1 = Q[n, curBeat];
8 if withScalar then
9     for e = 0 to elements-1
10         // Avoid Floating-point exceptions on a predicated lane by checking the element mask
11         pred = (elmtMask[e*(esize>>3)] == '0');
12         value = FPSub(Elem[op1, e, esize], R[m][esize-1:0], FALSE, pred);
13         Elem[result, e, esize] = value;
14 else
15     for e = 0 to elements-1
16         // Avoid Floating-point exceptions on a predicated lane by checking the element mask
17         pred = (elmtMask[e*(esize>>3)] == '0');
18         op2 = Q[m, curBeat];
19         value = FPSub(Elem[op1, e, esize], Elem[op2, e, esize], FALSE, pred);
20         Elem[result, e, esize] = value;
21
22 for e = 0 to 3
23     if elmtMask[e] == '1' then
24         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

C2.4.489 VSUB (vector)

Vector Subtract. Subtract the value of the elements in the second source vector register from either the respective elements in the first source vector register or a general-purpose register. The result is then written to the destination vector register.

This instruction is subject to beat-wise execution.

This instruction is VPT compatible.

This instruction is not permitted in an IT block.

T1

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	0	D	size		Qn		0		Qd		0	1	0	0	0	N	1	M	0		Qm		0	

T1: VSUB variant

VSUB<v><q>.<dt> Qd, Qn, Qm

Decode for this encoding

```

1 CheckDecodeFaults(ExtType_Mve);
2 if D == '1' || M == '1' || N == '1' then UNDEFINED;
3 if size == '11' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(M:Qm);
6 n = UInt(N:Qn);
7 withScalar = FALSE;
8 esize = 8 << UInt(size);
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;

```

T2

Armv8.1-M MVE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	0	D	size		Qn	1		Qd	1	1	1	1	1	N	1	0	0					Rm		

T2: VSUB variant

VSUB<v><q>.<dt> Qd, Qn, Rm

Decode for this encoding

```

1 if size == '11' then SEE "Related encodings";
2 CheckDecodeFaults(ExtType_Mve);
3 if D == '1' || N == '1' then UNDEFINED;
4 d = UInt(D:Qd);
5 m = UInt(Rm);
6 n = UInt(N:Qn);
7 withScalar = TRUE;
8 esize = 8 << UInt(size);
9 elements = 32 DIV esize;
10 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
11 if Rm == '11x1' then CONSTRAINED_UNPREDICTABLE;

```

Assembler symbols for all encodings

<v, q>	See C1.2.5 Standard assembler syntax fields .
<dt>	Size: indicates the size of the elements in the vector. This parameter must be one of the following values: I8 Encoded as size = 00 I16 Encoded as size = 01 I32 Encoded as size = 10
<Qd>	Destination vector register.
<Qn>	First source vector register.
<Qm>	Second source vector register.
<Rm>	Source general-purpose register.

Operation for all encodings

```

1  EncodingSpecificOperations();
2  ExecuteFPCheck();
3
4  (curBeat, elmtMask) = GetCurInstrBeat();
5
6  result = Zeros(32);
7  op1 = Q[n, curBeat];
8  if withScalar then
9      for e = 0 to elements-1
10         value = Elem[op1, e, esize] - R[m][esize-1:0];
11         Elem[result, e, esize] = value;
12 else
13     op2 = Q[m, curBeat];
14     for e = 0 to elements-1
15         value = Elem[op1, e, esize] - Elem[op2, e, esize];
16         Elem[result, e, esize] = value;
17
18 for e = 0 to 3
19     if elmtMask[e] == '1' then
20         Elem[Q[d, curBeat], e, 8] = Elem[result, e, 8];

```

Restricted behavior

Data Independent Timing behavior

If [AIRC.R.DIT](#) is 1:

- The execution time of this instruction is insensitive to the value or values of the data subject to this instruction.
- The point at which an asynchronous exception is taken does not vary based on the value or values being used by this instruction.

C2.4.490 WFE

Wait For Event. Wait For Event is a hint instruction. If the Event Register is clear, it suspends execution in the lowest power state available consistent with a fast wakeup without the need for software restoration, until a reset, exception or other event occurs.

This is a NOP-compatible hint.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	0	1	0	0	0	0	0

T1 variant

WFE{<c>}{<q>}

Decode for this encoding

```
1 // No additional decoding required
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	0	0	0	1	0

T2 variant

WFE{<C>}.W

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 // No additional decoding required
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
<q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   if EventRegistered() then
4     ClearEventRegister();
5   else
6     WaitForEvent();
```

C2.4.491 WFI

Wait For Interrupt. Wait For Interrupt is a hint instruction. It suspends execution, in the lowest power state available consistent with a fast wakeup without the need for software restoration, until a reset, asynchronous exception or other event occurs.

This is a NOP-compatible hint.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	0	1	1	0	0	0	0

T1 variant

WFI {<c>} {<q>}

Decode for this encoding

```
1 // No additional decoding required
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	0	0	0	1	1

T2 variant

WFI {<c>} .W

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 // No additional decoding required
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
<q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   WaitForInterrupt();
```

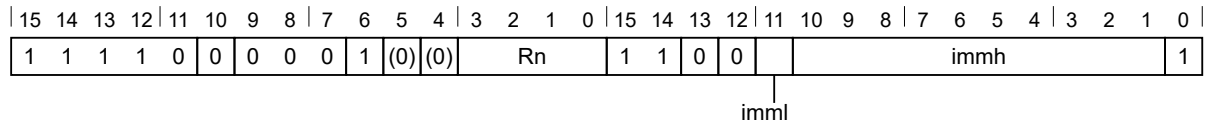

C2.4.492 WLS, DLS, WLSTP, DLSTP

While Loop Start, Do Loop Start, While Loop Start with Tail Predication, Do Loop Start with Tail Predication. This instruction partially sets up a loop. A LE or LETP (Loop End) instruction completes the setup. The base variants of this instruction (WLS and DLS) set LR to the number of loop iterations to be performed, whereas the TP variants of this instruction set LR to the number of vector-elements that must be processed. For the TP variants, if the number of elements required is not a multiple of the vector length then the appropriate number of vector elements will be predicated on the last iteration of the loop. When using WLS or WLSTP, if the number of iterations required is zero, then these instructions branch to the label specified. Each loop start instruction is normally used with a matching LE or LETP instruction.

This instruction is not permitted in an IT block.

T1

Armv8.1-M Low Overhead Branch Extension



T1: WLS variant

WLS<q> LR, Rn, <label>

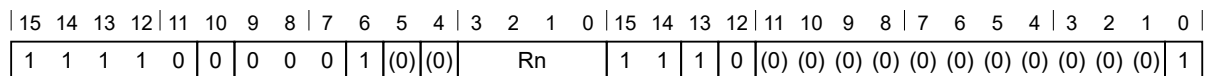
Decode for this encoding

```

1 if !HaveLOBExt() then UNDEFINED;
2 n      = UInt(Rn);
3 tSize  = 4[2:0]; // No truncation. Set size to full vector length
4 imm32  = ZeroExtend(immh:imml:'0', 32);
5 isWhileLoop = TRUE;
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
7 if Rn == 'l1x1' then CONSTRAINED_UNPREDICTABLE;
```

T2

Armv8.1-M Low Overhead Branch Extension



T2: DLS variant

DLS<q> LR, Rn

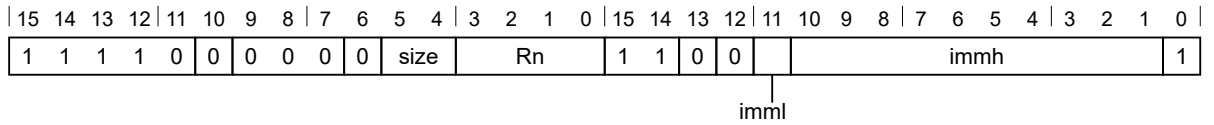
Decode for this encoding

```

1 if !HaveLOBExt() then UNDEFINED;
2 n      = UInt(Rn);
3 tSize  = 4[2:0]; // No truncation. Set size to full vector length
4 imm32  = Zeros(32);
5 isWhileLoop = FALSE;
6 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
7 if Rn == 'l1x1' then CONSTRAINED_UNPREDICTABLE;
```

T3

Armv8.1-M Low Overhead Branch Extension and MVE



T3: WLSTP variant

WLSTP<q>.<size> LR, Rn, <label>

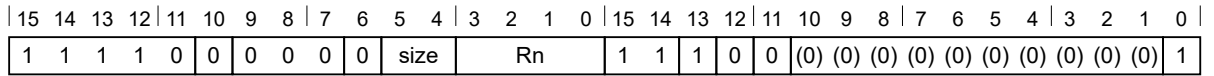
Decode for this encoding

```

1 if Rn == '1111' then SEE "Related encodings";
2 if !HaveLOBExt() then UNDEFINED;
3 if !HaveMve() then UNDEFINED;
4 HandleException(CheckCPEnabled(10));
5 n = UInt(Rn);
6 tSize = '0':size;
7 imm32 = ZeroExtend(immh:imm1:'0', 32);
8 isWhileLoop = TRUE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if Rn == '1101' then CONSTRAINED_UNPREDICTABLE;
```

T4

Armv8.1-M Low Overhead Branch Extension and MVE



T4: DLSTP variant

DLSTP<q>.<size> LR, Rn

Decode for this encoding

```

1 if Rn == '1111' then SEE "Related encodings";
2 if !HaveLOBExt() then UNDEFINED;
3 if !HaveMve() then UNDEFINED;
4 HandleException(CheckCPEnabled(10));
5 n = UInt(Rn);
6 tSize = '0':size;
7 imm32 = Zeros(32);
8 isWhileLoop = FALSE;
9 if InITBlock() then CONSTRAINED_UNPREDICTABLE;
10 if Rn == '1101' then CONSTRAINED_UNPREDICTABLE;
```

Assembler symbols for T1 encodings

<LR> LR is used to hold the iteration counter of the loop, this instruction must always use this register.

<Rn> The register holding the number of loop iterations to perform.

<label> Specifies the label of the instruction to branch to if no loop iterations are required.

Assembler symbols for T2 encodings

<LR> LR is used to hold the iteration counter of the loop, this instruction must always use this register.

<Rn> The register holding the number of loop iterations to perform.

Assembler symbols for T3 encodings

<LR> LR is used to hold the number of elements to process, this instruction must always use this register.
<Rn> The register holding the number of elements to process.
<label> Specifies the label of the instruction after the loop (the first instruction after the LE).

Assembler symbols for T4 encodings

<LR> LR is used to hold the number of elements to process, this instruction must always use this register.
<Rn> The register holding the number of elements to process.

Assembler symbols for all encodings

<q> See [C1.2.5 Standard assembler syntax fields](#).
<size> The size of the elements in the vector to process. This value is stored in the FPSCR.LTPSIZE field, and causes tail predication to be applied on the last iteration of the loop. This parameter must be one of the following values:

8	Encoded as	size = 00
16	Encoded as	size = 01
32	Encoded as	size = 10
64	Encoded as	size = 11

Operation for all encodings

```

1 EncodingSpecificOperations();
2
3 count = R[n];
4 if isWhileLoop && count == Zeros(32) then
5     BranchTo(PC + imm32);
6 else
7     // To avoid creating unnecessary floating-point context, the
8     // LTPSIZE is only set if tail predication is being used.
9     if tSize != 4[2:0] then
10         ExecuteFPCheck();
11         FPSCR.LTPSIZE = tSize;
12     // Set up the new iteration count
13     LR = count;
```

C2.4.493 YIELD

Yield hint. Yield is a hint instruction. It enables software with a multithreading capability to indicate to the hardware that it is performing a task, for example a spinlock, that could be swapped out to improve overall system performance. Hardware can use this hint to suspend and resume multiple code threads if it supports the capability.

This is a NOP-compatible hint.

T1

Armv8-M

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	0	0	1	0	0	0	0

T1 variant

YIELD{<c>}{<q>}

Decode for this encoding

```
1 // No additional decoding required
```

T2

Armv8-M Main Extension only

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	1	1	1	0	1	0	(1)	(1)	(1)	(1)	1	0	(0)	0	(0)	0	0	0	0	0	0	0	0	0	0	1

T2 variant

YIELD{<c>}.W

Decode for this encoding

```
1 if !HaveMainExt() then UNDEFINED;
2 // No additional decoding required
```

Assembler symbols for all encodings

<c> See [C1.2.5 Standard assembler syntax fields](#).
<q> See [C1.2.5 Standard assembler syntax fields](#).

Operation for all encodings

```
1 if ConditionPassed() then
2   EncodingSpecificOperations();
3   Hint_Yield();
```

Part D

Armv8-M Registers and Payload Specification

Chapter D1

Register and Payload Specification

This chapter specifies the Armv8-M registers and payloads. It contains the following sections:

[Register Index](#)

[Alphabetical list of registers](#)

D1.1 Register index

Address	Component
–	Special and general-purpose registers
–	Payloads
0xE0000000	Instrumentation Macrocell
0xE0001000	Data Watchpoint and Trace
0xE0002000	Flash Patch and Breakpoint
0xE0003000	Performance Monitoring Unit
0xE0005000	Reliability, Availability and Serviceability Extension Fault Status Register
0xE000E004	Implementation Control Block
0xE000E010	SysTick Timer
0xE000E100	Nested Vectored Interrupt Controller
0xE000ECFC	System Control Block
0xE000ED90	Memory Protection Unit
0xE000EDD0	Security Attribution Unit
0xE000EDF0	Debug Control Block
0xE000EF00	Software Interrupt Generation
0xE000EF04	Reliability, Availability and Serviceability Extension Fault Status Register
0xE000EF34	Floating-Point Extension
0xE000EF50	Cache Maintenance Operations
0xE000EFB0	Debug Identification Block
0xE002E004	Implementation Control Block (NS alias)
0xE002E010	SysTick Timer (NS alias)
0xE002E100	Nested Vectored Interrupt Controller (NS alias)
0xE002ECFC	System Control Block (NS alias)
0xE002ED90	Memory Protection Unit (NS alias)
0xE002EDF0	Debug Control Block (NS alias)
0xE002EF00	Software Interrupt Generation (NS alias)
0xE002EF04	Reliability, Availability and Serviceability Extension Fault Status Register (NS Alias)
0xE002EF34	Floating-Point Extension (NS alias)
0xE002EF50	Cache Maintenance Operations (NS alias)
0xE002EFB0	Debug Identification Block (NS alias)
0xE0040000	Trace Port Interface Unit

D1.1.1 Special and general-purpose registers

Name	Description
APSR	Application Program Status Register
BASEPRI	Base Priority Mask Register
CONTROL	Control Register
Dn	Floating-point Double-precision register <i>n</i>
EPSR	Execution Program Status Register
FAULTMASK	Fault Mask Register
FPSCR	Floating-point Status and Control Register
IPSR	Interrupt Program Status Register
LO_BRANCH_INFO	Loop and branch tracking information
LR	Link Register
MSPLIM	Main Stack Pointer Limit Register
PAC_KEY_P_n	Pointer Authentication Key Privileged <i>n</i>

Name	Description
PAC_KEY_U_n	Pointer Authentication Key Unprivileged n
PC	Program Counter
PRIMASK	Exception Mask Register
PSPLIM	Process Stack Pointer Limit Register
Qn	Vector register n
Rn	General-Purpose Register n
SP	Current Stack Pointer Register
SP_NS	Current Stack Pointer register (Non-secure)
Sn	Floating-point Single-precision register n
VPR	Vector Predication Status and Control Register
XPSR	Combined Program Status Registers

D1.1.2 Payloads

Name	Description
EXC_RETURN	Exception Return Payload
FNC_RETURN	Function Return Payload
FPCXT	Floating-point context payload
MAIR_ATTR	Memory Attribute Indirection Register Attributes
RETPSR	Combined Exception Return Program Status Registers
TT_RESP	Test Target Response Payload

D1.1.3 Instrumentation Macrocell

Address	Register	Description
0xE0000000	ITM_STIMn	ITM Stimulus Port Register n
0xE0000E00	ITM_TERn	ITM Trace Enable Register n
0xE0000E40	ITM_TPR	ITM Trace Privilege Register
0xE0000E80	ITM_TCR	ITM Trace Control Register
0xE0000FB0	ITM_LAR	ITM Software Lock Access Register
0xE0000FB4	ITM_LSR	ITM Software Lock Status Register
0xE0000FBC	ITM_DEVARCH	ITM Device Architecture Register
0xE0000FCC	ITM_DEVTYPE	ITM Device Type Register
0xE0000FD0	ITM_PIDR4	ITM Peripheral Identification Register 4
0xE0000FD4	ITM_PIDR5	ITM Peripheral Identification Register 5
0xE0000FD8	ITM_PIDR6	ITM Peripheral Identification Register 6
0xE0000FDC	ITM_PIDR7	ITM Peripheral Identification Register 7
0xE0000FE0	ITM_PIDR0	ITM Peripheral Identification Register 0
0xE0000FE4	ITM_PIDR1	ITM Peripheral Identification Register 1
0xE0000FE8	ITM_PIDR2	ITM Peripheral Identification Register 2
0xE0000FEC	ITM_PIDR3	ITM Peripheral Identification Register 3
0xE0000FF0	ITM_CIDR0	ITM Component Identification Register 0
0xE0000FF4	ITM_CIDR1	ITM Component Identification Register 1
0xE0000FF8	ITM_CIDR2	ITM Component Identification Register 2
0xE0000FFC	ITM_CIDR3	ITM Component Identification Register 3

D1.1.4 Data Watchpoint and Trace

Address	Register	Description
0xE0001000	DWT_CTRL	DWT Control Register
0xE0001004	DWT_CYCCNT	DWT Cycle Count Register
0xE0001008	DWT_CPICNT	DWT CPI Count Register
0xE000100C	DWT_EXCCNT	DWT Exception Overhead Count Register
0xE0001010	DWT_SLEEPCNT	DWT Sleep Count Register
0xE0001014	DWT_LSUCNT	DWT LSU Count Register
0xE0001018	DWT_FOLDCNT	DWT Folded Instruction Count Register
0xE000101C	DWT_PCSR	DWT Program Counter Sample Register
0xE0001020	DWT_COMPn	DWT Comparator Register <i>n</i>
0xE0001028	DWT_FUNCTIONn	DWT Comparator Function Register <i>n</i>
0xE000102C	DWT_VMASKn	DWT Comparator Value Mask Register <i>n</i>
0xE0001FB0	DWT_LAR	DWT Software Lock Access Register
0xE0001FB4	DWT_LSR	DWT Software Lock Status Register
0xE0001FBC	DWT_DEVARCH	DWT Device Architecture Register
0xE0001FCC	DWT_DEVTYPE	DWT Device Type Register
0xE0001FD0	DWT_PIDR4	DWT Peripheral Identification Register 4
0xE0001FD4	DWT_PIDR5	DWT Peripheral Identification Register 5
0xE0001FD8	DWT_PIDR6	DWT Peripheral Identification Register 6
0xE0001FDC	DWT_PIDR7	DWT Peripheral Identification Register 7
0xE0001FE0	DWT_PIDR0	DWT Peripheral Identification Register 0
0xE0001FE4	DWT_PIDR1	DWT Peripheral Identification Register 1
0xE0001FE8	DWT_PIDR2	DWT Peripheral Identification Register 2
0xE0001FEC	DWT_PIDR3	DWT Peripheral Identification Register 3
0xE0001FF0	DWT_CIDR0	DWT Component Identification Register 0
0xE0001FF4	DWT_CIDR1	DWT Component Identification Register 1
0xE0001FF8	DWT_CIDR2	DWT Component Identification Register 2
0xE0001FFC	DWT_CIDR3	DWT Component Identification Register 3

D1.1.5 Flash Patch and Breakpoint

Address	Register	Description
0xE0002000	FP_CTRL	Flash Patch Control Register
0xE0002004	FP_REMAP	Flash Patch Remap Register
0xE0002008	FP_COMPn	Flash Patch Comparator Register <i>n</i>
0xE0002FB0	FP_LAR	FPB Software Lock Access Register
0xE0002FB4	FP_LSR	FPB Software Lock Status Register
0xE0002FBC	FP_DEVARCH	FPB Device Architecture Register
0xE0002FCC	FP_DEVTYPE	FPB Device Type Register
0xE0002FD0	FP_PIDR4	FP Peripheral Identification Register 4
0xE0002FD4	FP_PIDR5	FP Peripheral Identification Register 5
0xE0002FD8	FP_PIDR6	FP Peripheral Identification Register 6
0xE0002FDC	FP_PIDR7	FP Peripheral Identification Register 7
0xE0002FE0	FP_PIDR0	FP Peripheral Identification Register 0
0xE0002FE4	FP_PIDR1	FP Peripheral Identification Register 1
0xE0002FE8	FP_PIDR2	FP Peripheral Identification Register 2
0xE0002FEC	FP_PIDR3	FP Peripheral Identification Register 3
0xE0002FF0	FP_CIDR0	FP Component Identification Register 0
0xE0002FF4	FP_CIDR1	FP Component Identification Register 1

Address	Register	Description
0xE0002FF8	FP_CIDR2	FP Component Identification Register 2
0xE0002FFC	FP_CIDR3	FP Component Identification Register 3

D1.1.6 Performance Monitoring Unit

Address	Register	Description
0xE0003000	PMU_EVCNTRn	Performance Monitoring Unit Event Counter Register
0xE000307C	PMU_CCNTR	Performance Monitoring Unit Cycle Counter Register
0xE0003400	PMU_EVTYPERn	Performance Monitoring Unit Event Type and Filter Register
0xE000347C	PMU_CCFILTR	Performance Monitoring Unit Cycle Counter Filter Register
0xE0003C00	PMU_CNTENSET	Performance Monitoring Unit Count Enable Set Register
0xE0003C20	PMU_CNTENCLR	Performance Monitoring Unit Count Enable Clear Register
0xE0003C40	PMU_INTENSET	Performance Monitoring Unit Interrupt Enable Set Register
0xE0003C60	PMU_INTENCLR	Performance Monitoring Unit Interrupt Enable Clear Register
0xE0003C80	PMU_OVSCLR	Performance Monitoring Unit Overflow Flag Status Clear Register
0xE0003CA0	PMU_SWINC	Performance Monitoring Unit Software Increment Register
0xE0003CC0	PMU_OVSSET	Performance Monitoring Unit Overflow Flag Status Set Register
0xE0003E00	PMU_TYPE	Performance Monitoring Unit Type Register
0xE0003E04	PMU_CTRL	Performance Monitoring Unit Control Register
0xE0003FB8	PMU_AUTHSTATUS	Performance Monitoring Unit Authentication Status Register
0xE0003FBC	PMU_DEVARCH	Performance Monitoring Unit Device Architecture Register
0xE0003FCC	PMU_DEVTYPE	Performance Monitoring Unit Device Type Register
0xE0003FD0	PMU_PIDR4	Performance Monitoring Unit Peripheral Identification Register 4
0xE0003FE0	PMU_PIDR0	Performance Monitoring Unit Peripheral Identification Register 0
0xE0003FE4	PMU_PIDR1	Performance Monitoring Unit Peripheral Identification Register 1
0xE0003FE8	PMU_PIDR2	Performance Monitoring Unit Peripheral Identification Register 2
0xE0003FEC	PMU_PIDR3	Performance Monitoring Unit Peripheral Identification Register 3
0xE0003FF0	PMU_CIDR0	Performance Monitoring Unit Component Identification Register 0
0xE0003FF4	PMU_CIDR1	Performance Monitoring Unit Component Identification Register 1
0xE0003FF8	PMU_CIDR2	Performance Monitoring Unit Component Identification Register 2
0xE0003FFC	PMU_CIDR3	Performance Monitoring Unit Component Identification Register 3

D1.1.7 Reliability, Availability and Serviceability Extension Fault Status Register

Address	Register	Description
0xE0005000	ERRFRn	Error Record Feature Register <i>n</i>
0xE0005008	ERRCTRLn	Error Record Control Register <i>n</i>
0xE0005010	ERRSTATUSn	Error Record Primary Status Register <i>n</i>
0xE0005018	ERRADDRn	Error Record Address Register <i>n</i>
0xE000501C	ERRADDR2n	Error Record Address 2 Register <i>n</i>
0xE0005020	ERRMISC0n	Error Record Miscellaneous 0 Register <i>n</i>
0xE0005024	ERRMISC1n	Error Record Miscellaneous 1 Register <i>n</i>
0xE0005028	ERRMISC2n	Error Record Miscellaneous 2 Register <i>n</i>
0xE000502C	ERRMISC3n	Error Record Miscellaneous 3 Register <i>n</i>
0xE0005030	ERRMISC4n	Error Record Miscellaneous 4 Register <i>n</i>
0xE0005034	ERRMISC5n	Error Record Miscellaneous 5 Register <i>n</i>
0xE0005038	ERRMISC6n	Error Record Miscellaneous 6 Register <i>n</i>
0xE000503C	ERRMISC7n	Error Record Miscellaneous 7 Register <i>n</i>

Address	Register	Description
0xE0005E00	ERRGSRn	RAS Fault Group Status Register
0xE0005E10	ERRIIDR	Error Implementer ID Register
0xE0005FC8	ERRDEVID	Error Record Device ID Register

D1.1.8 Implementation Control Block

Address	Register	Description
0xE000E004	ICTR	Interrupt Controller Type Register
0xE000E008	ACTLR	Auxiliary Control Register
0xE000E00C	CPPWR	Coprocessor Power Control Register

D1.1.9 SysTick Timer

Address	Register	Description
0xE000E010	SYST_CSR	SysTick Control and Status Register
0xE000E014	SYST_RVR	SysTick Reload Value Register
0xE000E018	SYST_CVR	SysTick Current Value Register
0xE000E01C	SYST_CALIB	SysTick Calibration Value Register

D1.1.10 Nested Vectored Interrupt Controller

Address	Register	Description
0xE000E100	NVIC_ISERn	Interrupt Set Enable Register <i>n</i>
0xE000E180	NVIC_ICERn	Interrupt Clear Enable Register <i>n</i>
0xE000E200	NVIC_ISPRn	Interrupt Set Pending Register <i>n</i>
0xE000E280	NVIC_ICPRn	Interrupt Clear Pending Register <i>n</i>
0xE000E300	NVIC_IABRn	Interrupt Active Bit Register <i>n</i>
0xE000E380	NVIC_ITNSn	Interrupt Target Non-secure Register <i>n</i>
0xE000E400	NVIC_IPRn	Interrupt Priority Register <i>n</i>

D1.1.11 System Control Block

Address	Register	Description
0xE000ECFC	REVIDR	Revision ID Register
0xE000ED00	CPUID	CPUID Base Register
0xE000ED04	ICSR	Interrupt Control and State Register
0xE000ED08	VTOR	Vector Table Offset Register
0xE000ED0C	AIRCR	Application Interrupt and Reset Control Register
0xE000ED10	SCR	System Control Register
0xE000ED14	CCR	Configuration and Control Register
0xE000ED18	SHPR1	System Handler Priority Register 1
0xE000ED1C	SHPR2	System Handler Priority Register 2
0xE000ED20	SHPR3	System Handler Priority Register 3

Address	Register	Description
0xE000ED24	SHCSR	System Handler Control and State Register
0xE000ED28	MMFSR	MemManage Fault Status Register
0xE000ED28	CFSR	Configurable Fault Status Register
0xE000ED29	BFSR	BusFault Status Register
0xE000ED2A	UFSR	UsageFault Status Register
0xE000ED2C	HFSR	HardFault Status Register
0xE000ED30	DFSR	Debug Fault Status Register
0xE000ED34	MMFAR	MemManage Fault Address Register
0xE000ED38	BFAR	BusFault Address Register
0xE000ED3C	AFSR	Auxiliary Fault Status Register
0xE000ED40	ID_PFR0	Processor Feature Register 0
0xE000ED44	ID_PFR1	Processor Feature Register 1
0xE000ED48	ID_DFR0	Debug Feature Register 0
0xE000ED4C	ID_AFR0	Auxiliary Feature Register 0
0xE000ED50	ID_MMFR0	Memory Model Feature Register 0
0xE000ED54	ID_MMFR1	Memory Model Feature Register 1
0xE000ED58	ID_MMFR2	Memory Model Feature Register 2
0xE000ED5C	ID_MMFR3	Memory Model Feature Register 3
0xE000ED60	ID_ISAR0	Instruction Set Attribute Register 0
0xE000ED64	ID_ISAR1	Instruction Set Attribute Register 1
0xE000ED68	ID_ISAR2	Instruction Set Attribute Register 2
0xE000ED6C	ID_ISAR3	Instruction Set Attribute Register 3
0xE000ED70	ID_ISAR4	Instruction Set Attribute Register 4
0xE000ED74	ID_ISAR5	Instruction Set Attribute Register 5
0xE000ED78	CLIDR	Cache Level ID Register
0xE000ED7C	CTR	Cache Type Register
0xE000ED80	CCSIDR	Current Cache Size ID register
0xE000ED84	CSSELR	Cache Size Selection Register
0xE000ED88	CPACR	Coprocessor Access Control Register
0xE000ED8C	NSACR	Non-secure Access Control Register

D1.1.12 Memory Protection Unit

Address	Register	Description
0xE000ED90	MPU_TYPE	MPU Type Register
0xE000ED94	MPU_CTRL	MPU Control Register
0xE000ED98	MPU_RNR	MPU Region Number Register
0xE000ED9C	MPU_RBAR	MPU Region Base Address Register
0xE000EDA0	MPU_RLAR	MPU Region Limit Address Register
0xE000EDA4	MPU_RBAR_An	MPU Region Base Address Register Alias <i>n</i>
0xE000EDA8	MPU_RLAR_An	MPU Region Limit Address Register Alias <i>n</i>
0xE000EDC0	MPU_MAIR0	MPU Memory Attribute Indirection Register 0
0xE000EDC4	MPU_MAIR1	MPU Memory Attribute Indirection Register 1

D1.1.13 Security Attribution Unit

Address	Register	Description
0xE000EDD0	SAU_CTRL	SAU Control Register

Address	Register	Description
0xE000EDD4	SAU_TYPE	SAU Type Register
0xE000EDD8	SAU_RNR	SAU Region Number Register
0xE000EDDC	SAU_RBAR	SAU Region Base Address Register
0xE000EDE0	SAU_RLAR	SAU Region Limit Address Register
0xE000EDE4	SFSR	Secure Fault Status Register
0xE000EDE8	SFAR	Secure Fault Address Register

D1.1.14 Debug Control Block

Address	Register	Description
0xE000EDF0	DHCSR	Debug Halting Control and Status Register
0xE000EDF4	DCRSR	Debug Core Register Select Register
0xE000EDF8	DCRDR	Debug Core Register Data Register
0xE000EDFC	DEMCR	Debug Exception and Monitor Control Register
0xE000EE00	DSCEMCR	Debug Set Clear Exception and Monitor Control Register
0xE000EE04	DAUTHCTRL	Debug Authentication Control Register
0xE000EE08	DSCSR	Debug Security Control and Status Register

D1.1.15 Software Interrupt Generation

Address	Register	Description
0xE000EF00	STIR	Software Triggered Interrupt Register

D1.1.16 Reliability, Availability and Serviceability Extension Fault Status Register

Address	Register	Description
0xE000EF04	RFSR	RAS Fault Status Register

D1.1.17 Floating-Point Extension

Address	Register	Description
0xE000EF34	FPCCR	Floating-Point Context Control Register
0xE000EF38	FPCAR	Floating-Point Context Address Register
0xE000EF3C	FPDSCR	Floating-Point Default Status Control Register
0xE000EF40	MVFR0	Media and VFP Feature Register 0
0xE000EF44	MVFR1	Media and VFP Feature Register 1
0xE000EF48	MVFR2	Media and VFP Feature Register 2

D1.1.18 Cache Maintenance Operations

Address	Register	Description
0xE000EF50	ICIALLU	Instruction Cache Invalidate All to PoU
0xE000EF58	ICIMVAU	Instruction Cache line Invalidate by Address to PoU
0xE000EF5C	DCIMVAC	Data Cache line Invalidate by Address to PoC
0xE000EF60	DCISW	Data Cache line Invalidate by Set/Way
0xE000EF64	DCCMVAU	Data Cache line Clean by address to PoU
0xE000EF68	DCCMVAC	Data Cache line Clean by Address to PoC
0xE000EF6C	DCCSW	Data Cache Clean line by Set/Way
0xE000EF70	DCCIMVAC	Data Cache line Clean and Invalidate by Address to PoC
0xE000EF74	DCCISW	Data Cache line Clean and Invalidate by Set/Way
0xE000EF78	BPIALL	Branch Predictor Invalidate All

D1.1.19 Debug Identification Block

Address	Register	Description
0xE000EFB0	DLAR	SCS Software Lock Access Register
0xE000EFB4	DLSR	SCS Software Lock Status Register
0xE000EFB8	DAUTHSTATUS	Debug Authentication Status Register
0xE000EFBC	DDEVARCH	SCS Device Architecture Register
0xE000EFCC	DDEVTYPE	SCS Device Type Register
0xE000EFD0	DPIDR4	SCS Peripheral Identification Register 4
0xE000EFD4	DPIDR5	SCS Peripheral Identification Register 5
0xE000EFD8	DPIDR6	SCS Peripheral Identification Register 6
0xE000EFD8	DPIDR7	SCS Peripheral Identification Register 7
0xE000EFE0	DPIDR0	SCS Peripheral Identification Register 0
0xE000EFE4	DPIDR1	SCS Peripheral Identification Register 1
0xE000EFE8	DPIDR2	SCS Peripheral Identification Register 2
0xE000EFEC	DPIDR3	SCS Peripheral Identification Register 3
0xE000EFF0	DCIDR0	SCS Component Identification Register 0
0xE000EFF4	DCIDR1	SCS Component Identification Register 1
0xE000EFF8	DCIDR2	SCS Component Identification Register 2
0xE000EFFC	DCIDR3	SCS Component Identification Register 3

D1.1.20 Implementation Control Block (NS alias)

Address	Register	Description
0xE002E004	ICTR	Interrupt Controller Type Register (NS)
0xE002E008	ACTLR	Auxiliary Control Register (NS)
0xE002E00C	CPPWR	Coprocessor Power Control Register (NS)

D1.1.21 SysTick Timer (NS alias)

Address	Register	Description
0xE002E010	SYST_CSR	SysTick Control and Status Register (NS)
0xE002E014	SYST_RVR	SysTick Reload Value Register (NS)
0xE002E018	SYST_CVR	SysTick Current Value Register (NS)
0xE002E01C	SYST_CALIB	SysTick Calibration Value Register (NS)

D1.1.22 Nested Vectored Interrupt Controller (NS alias)

Address	Register	Description
0xE002E100	NVIC_ISERn	Interrupt Set Enable Register <i>n</i> (NS)
0xE002E180	NVIC_ICERn	Interrupt Clear Enable Register <i>n</i> (NS)
0xE002E200	NVIC_ISPRn	Interrupt Set Pending Register <i>n</i> (NS)
0xE002E280	NVIC_ICPRn	Interrupt Clear Pending Register <i>n</i> (NS)
0xE002E300	NVIC_IABRn	Interrupt Active Bit Register <i>n</i> (NS)
0xE002E400	NVIC_IPRn	Interrupt Priority Register <i>n</i> (NS)

D1.1.23 System Control Block (NS alias)

Address	Register	Description
0xE002ECFC	REVIDR	Revision ID Register (NS)
0xE002ED00	CPUID	CPUID Base Register (NS)
0xE002ED04	ICSR	Interrupt Control and State Register (NS)
0xE002ED08	VTOR	Vector Table Offset Register (NS)
0xE002ED0C	AIRCR	Application Interrupt and Reset Control Register (NS)
0xE002ED10	SCR	System Control Register (NS)
0xE002ED14	CCR	Configuration and Control Register (NS)
0xE002ED18	SHPR1	System Handler Priority Register 1 (NS)
0xE002ED1C	SHPR2	System Handler Priority Register 2 (NS)
0xE002ED20	SHPR3	System Handler Priority Register 3 (NS)
0xE002ED24	SHCSR	System Handler Control and State Register (NS)
0xE002ED28	CFSR	Configurable Fault Status Register (NS)
0xE002ED28	MMFSR	MemManage Fault Status Register (NS)
0xE002ED29	BFSR	BusFault Status Register (NS)
0xE002ED2A	UFSR	UsageFault Status Register (NS)
0xE002ED2C	HFSR	HardFault Status Register (NS)
0xE002ED30	DFSR	Debug Fault Status Register (NS)
0xE002ED34	MMFAR	MemManage Fault Address Register (NS)
0xE002ED38	BFAR	BusFault Address Register (NS)
0xE002ED3C	AFSR	Auxiliary Fault Status Register (NS)
0xE002ED40	ID_PFR0	Processor Feature Register 0 (NS)
0xE002ED44	ID_PFR1	Processor Feature Register 1 (NS)
0xE002ED48	ID_DFR0	Debug Feature Register 0 (NS)
0xE002ED4C	ID_AFR0	Auxiliary Feature Register 0 (NS)
0xE002ED50	ID_MMFR0	Memory Model Feature Register 0 (NS)
0xE002ED54	ID_MMFR1	Memory Model Feature Register 1 (NS)
0xE002ED58	ID_MMFR2	Memory Model Feature Register 2 (NS)
0xE002ED5C	ID_MMFR3	Memory Model Feature Register 3 (NS)
0xE002ED60	ID_ISAR0	Instruction Set Attribute Register 0 (NS)
0xE002ED64	ID_ISAR1	Instruction Set Attribute Register 1 (NS)
0xE002ED68	ID_ISAR2	Instruction Set Attribute Register 2 (NS)
0xE002ED6C	ID_ISAR3	Instruction Set Attribute Register 3 (NS)
0xE002ED70	ID_ISAR4	Instruction Set Attribute Register 4 (NS)
0xE002ED74	ID_ISAR5	Instruction Set Attribute Register 5 (NS)
0xE002ED78	CLIDR	Cache Level ID Register (NS)
0xE002ED7C	CTR	Cache Type Register (NS)
0xE002ED80	CCSIDR	Current Cache Size ID register (NS)
0xE002ED84	CSSELR	Cache Size Selection Register (NS)

Address	Register	Description
0xE002ED88	CPACR	Coprocessor Access Control Register (NS)

D1.1.24 Memory Protection Unit (NS alias)

Address	Register	Description
0xE002ED90	MPU_TYPE	MPU Type Register (NS)
0xE002ED94	MPU_CTRL	MPU Control Register (NS)
0xE002ED98	MPU_RNR	MPU Region Number Register (NS)
0xE002ED9C	MPU_RBAR	MPU Region Base Address Register (NS)
0xE002EDA0	MPU_RLAR	MPU Region Limit Address Register (NS)
0xE002EDA4	MPU_RBAR_An	MPU Region Base Address Register Alias <i>n</i> (NS)
0xE002EDA8	MPU_RLAR_An	MPU Region Limit Address Register Alias <i>n</i> (NS)
0xE002EDC0	MPU_MAIRO	MPU Memory Attribute Indirection Register 0 (NS)
0xE002EDC4	MPU_MAIR1	MPU Memory Attribute Indirection Register 1 (NS)

D1.1.25 Debug Control Block (NS alias)

Address	Register	Description
0xE002EDF0	DHCSR	Debug Halting Control and Status Register (NS)
0xE002EDF8	DCRDR	Debug Core Register Data Register (NS)
0xE002EDFC	DEMCR	Debug Exception and Monitor Control Register (NS)
0xE002EE00	DSCEMCR	Debug Set Clear Exception and Monitor Control Register (NS)
0xE002EE04	DAUTHCTRL	Debug Authentication Control Register (NS)

D1.1.26 Software Interrupt Generation (NS alias)

Address	Register	Description
0xE002EF00	STIR	Software Triggered Interrupt Register (NS)

D1.1.27 Reliability, Availability and Serviceability Extension Fault Status Register (NS Alias)

Address	Register	Description
0xE002EF04	RFSR	RAS Fault Status Register (NS)

D1.1.28 Floating-Point Extension (NS alias)

Address	Register	Description
0xE002EF34	FPCCR	Floating-Point Context Control Register (NS)
0xE002EF38	FPCAR	Floating-Point Context Address Register (NS)
0xE002EF3C	FPDSCR	Floating-Point Default Status Control Register (NS)
0xE002EF40	MVFR0	Media and VFP Feature Register 0 (NS)

Address	Register	Description
0xE002EF44	MVFR1	Media and VFP Feature Register 1 (NS)
0xE002EF48	MVFR2	Media and VFP Feature Register 2 (NS)

D1.1.29 Cache Maintenance Operations (NS alias)

Address	Register	Description
0xE002EF50	ICIALLU	Instruction Cache Invalidate All to PoU (NS)
0xE002EF58	ICIMVAU	Instruction Cache line Invalidate by Address to PoU (NS)
0xE002EF5C	DCIMVAC	Data Cache line Invalidate by Address to PoC (NS)
0xE002EF60	DCISW	Data Cache line Invalidate by Set/Way (NS)
0xE002EF64	DCCMVAU	Data Cache line Clean by address to PoU (NS)
0xE002EF68	DCCMVAC	Data Cache line Clean by Address to PoC (NS)
0xE002EF6C	DCCSW	Data Cache Clean line by Set/Way (NS)
0xE002EF70	DCCIMVAC	Data Cache line Clean and Invalidate by Address to PoC (NS)
0xE002EF74	DCCISW	Data Cache line Clean and Invalidate by Set/Way (NS)
0xE002EF78	BPIALL	Branch Predictor Invalidate All (NS)

D1.1.30 Debug Identification Block (NS alias)

Address	Register	Description
0xE002EFB0	DLAR	SCS Software Lock Access Register (NS)
0xE002EFB4	DLSR	SCS Software Lock Status Register (NS)
0xE002EFB8	DAUTHSTATUS	Debug Authentication Status Register (NS)
0xE002EFBC	DDEVARCH	SCS Device Architecture Register (NS)
0xE002EFCC	DDEVTYPE	SCS Device Type Register (NS)
0xE002EFD0	DPIDR4	SCS Peripheral Identification Register 4 (NS)
0xE002EFD4	DPIDR5	SCS Peripheral Identification Register 5 (NS)
0xE002EFD8	DPIDR6	SCS Peripheral Identification Register 6 (NS)
0xE002EFD8	DPIDR7	SCS Peripheral Identification Register 7 (NS)
0xE002EFE0	DPIDR0	SCS Peripheral Identification Register 0 (NS)
0xE002EFE4	DPIDR1	SCS Peripheral Identification Register 1 (NS)
0xE002EFE8	DPIDR2	SCS Peripheral Identification Register 2 (NS)
0xE002EFEC	DPIDR3	SCS Peripheral Identification Register 3 (NS)
0xE002EFF0	DCIDR0	SCS Component Identification Register 0 (NS)
0xE002EFF4	DCIDR1	SCS Component Identification Register 1 (NS)
0xE002EFF8	DCIDR2	SCS Component Identification Register 2 (NS)
0xE002EFFC	DCIDR3	SCS Component Identification Register 3 (NS)

D1.1.31 Trace Port Interface Unit

Address	Register	Description
0xE0040000	TPIU_SSPSR	TPIU Supported Parallel Port Sizes Register
0xE0040004	TPIU_CSPPSR	TPIU Current Parallel Port Sizes Register
0xE0040010	TPIU_ACPR	TPIU Asynchronous Clock Prescaler Register
0xE00400F0	TPIU_SPPR	TPIU Selected Pin Protocol Register
0xE0040300	TPIU_FFSR	TPIU Formatter and Flush Status Register

Address	Register	Description
0xE0040304	TPIU_FFCR	TPIU Formatter and Flush Control Register
0xE0040308	TPIU_PSCR	TPIU Periodic Synchronization Control Register
0xE0040FA0	TPIU_CLAIMSET	TPIU Claim Tag Set Register
0xE0040FA4	TPIU_CLAIMCLR	TPIU Claim Tag Clear Register
0xE0040FB0	TPIU_LAR	TPIU Software Lock Access Register
0xE0040FB4	TPIU_LSR	TPIU Software Lock Status Register
0xE0040FC8	TPIU_DEVID	TPIU Device Identifier Register
0xE0040FCC	TPIU_DEVTYPE	TPIU Device Type Register
0xE0040FD0	TPIU_PIDR4	TPIU Peripheral Identification Register 4
0xE0040FD4	TPIU_PIDR5	TPIU Peripheral Identification Register 5
0xE0040FD8	TPIU_PIDR6	TPIU Peripheral Identification Register 6
0xE0040FDC	TPIU_PIDR7	TPIU Peripheral Identification Register 7
0xE0040FE0	TPIU_PIDR0	TPIU Peripheral Identification Register 0
0xE0040FE4	TPIU_PIDR1	TPIU Peripheral Identification Register 1
0xE0040FE8	TPIU_PIDR2	TPIU Peripheral Identification Register 2
0xE0040FEC	TPIU_PIDR3	TPIU Peripheral Identification Register 3
0xE0040FF0	TPIU_CIDR0	TPIU Component Identification Register 0
0xE0040FF4	TPIU_CIDR1	TPIU Component Identification Register 1
0xE0040FF8	TPIU_CIDR2	TPIU Component Identification Register 2
0xE0040FFC	TPIU_CIDR3	TPIU Component Identification Register 3

D1.2 Alphabetical list of registers

D1.2.1 ACTLR, Auxiliary Control Register

The ACTLR characteristics are:

Purpose

Provides IMPLEMENTATION DEFINED configuration and control options.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000E008.

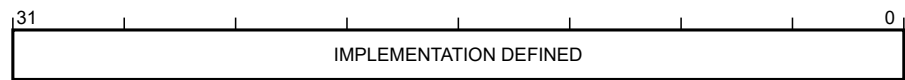
Secure software can access the Non-secure version of this register via ACTLR_NS located at 0xE002E008. The location 0xE002E008 is RES0 to software executing in Non-secure state and the debugger.

It is IMPLEMENTATION DEFINED whether this register is banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The ACTLR bit assignments are:



IMPLEMENTATION DEFINED, bits [31:0]

IMPLEMENTATION DEFINED. The contents of this field are IMPLEMENTATION DEFINED.

D1.2.2 AFSR, Auxiliary Fault Status Register

The AFSR characteristics are:

Purpose

Provides **IMPLEMENTATION DEFINED** fault status information.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED3C.

Secure software can access the Non-secure version of this register via AFSR_NS located at 0xE002ED3C. The location 0xE002ED3C is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The AFSR bit assignments are:



IMPLEMENTATION DEFINED, bits [31:0]

IMPLEMENTATION DEFINED. The contents of this field are **IMPLEMENTATION DEFINED**.

D1.2.3 AIRCR, Application Interrupt and Reset Control Register

The AIRCR characteristics are:

Purpose

Sets or returns interrupt control and reset configuration.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED0C.

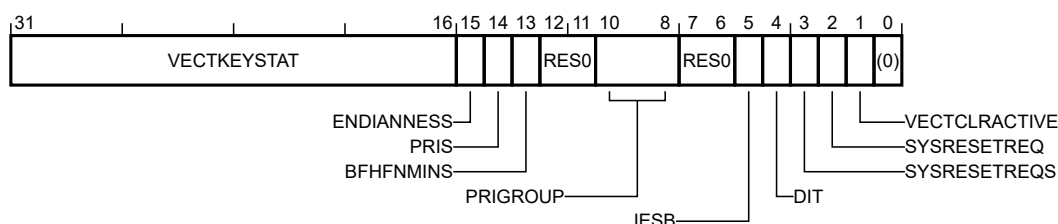
Secure software can access the Non-secure version of this register via AIRCR_NS located at 0xE002ED0C. The location 0xE002ED0C is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

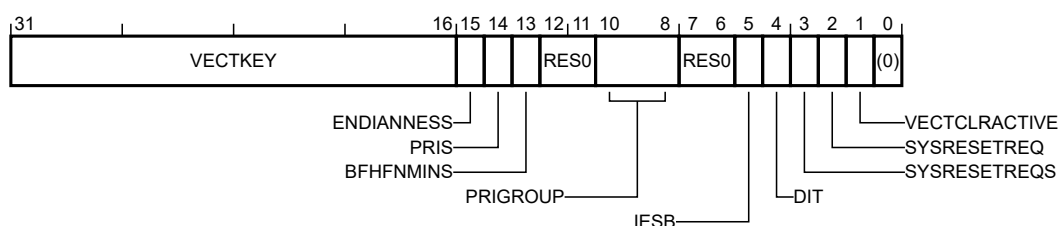
Field descriptions

The AIRCR bit assignments are:

On a read:



On a write:



VECTKEY, bits [31:16], on a write

Vector key. Writes to the AIRCR must be accompanied by a write of the value 0x05FA to this field. Writes to the AIRCR fields that are not accompanied by this value are ignored for the purpose of updating any of the AIRCR values or initiating any AIRCR functionality.

This field is not banked between Security states.

The possible values of this field are:

0x05FA

Permit write to AIRCR fields.

Not 0x05FA

Accompanying write to AIRCR fields ignored.

VECTKEYSTAT, bits [31:16], on a read

Vector key status. Returns the bitwise inverse of the value required to be written to VECTKEY.

This field is not banked between Security states.

This field reads as 0xFA05.

ENDIANNESS, bit [15]

Data endianness. Indicates how the PE interprets the memory system data endianness.

This bit is not banked between Security states.

The possible values of this bit are:

0

Little-endian.

1

Big-endian.

This bit is read-only.

This bit reads as an **IMPLEMENTATION DEFINED** value.

PRIS, bit [14]

Prioritize Secure exceptions. The value of this bit defines whether Secure exception priority boosting is enabled.

This bit is not banked between Security states.

The possible values of this bit are:

0

Priority ranges of Secure and Non-secure exceptions are identical.

1

Non-secure exceptions are de-prioritized.

To allow lock down of this bit, it is **IMPLEMENTATION DEFINED** whether this bit is writable.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

BFHFNMINs, bit [13]

BusFault, HardFault, and NMI Non-secure enable. The value of this bit defines whether BusFault and NMI exceptions are Non-secure, and whether exceptions target the Non-secure HardFault exception.

This bit is not banked between Security states.

The possible values of this bit are:

0

BusFault, HardFault, and NMI are Secure.

1

BusFault and NMI are Non-secure and exceptions can target Non-secure HardFault.

If an implementation resets into Secure state, this bit resets to zero. If an implementation does not support Secure state, this bit is RAO/WI. To allow lock down of this field it is **IMPLEMENTATION DEFINED** whether this bit is writable. The effect of setting both BFHFNMINs and PRIS to 1 is **UNPREDICTABLE**.

From Armv8.1-M the clearing of this field to zero, except by reset, is deprecated.

This bit is read-only from Non-secure state.

This bit resets to zero on a Warm reset.

Bits [12:11]

Reserved, [RES0](#).

PRIGROUP, bits [10:8]

Priority grouping. The value of this field defines the exception priority binary point position for the selected Security state.

This field is banked between Security states.

The possible values of this field are:

0b000

Group priority [7:1], subpriority [0].

0b001

Group priority [7:2], subpriority [1:0].

0b010

Group priority [7:3], subpriority [2:0].

0b011

Group priority [7:4], subpriority [3:0].

0b100

Group priority [7:5], subpriority [4:0].

0b101

Group priority [7:6], subpriority [5:0].

0b110

Group priority [7], subpriority [6:0].

0b111

No group priority, subpriority [7:0].

If the Main Extension is not implemented, this field is [RES0](#).

This field resets to zero on a Warm reset.

Bits [7:6]

Reserved, [RES0](#).

IESB, bit [5]

Implicit ESB Enable. This bit indicates and allows modification of whether an implicit Error Synchronization Barriers occurs around lazy Floating-point state preservation, and on every exception entry and return.

This bit is not banked between Security states.

The possible values of this bit are:

0

No Implicit ESB.

1

Implicit ESB are enabled.

Enabling implicit ESB's also causes imprecise BusFault exceptions to escalate as if they were precise, however because the address of the instruction that caused the fault is not known they are still reported as an IMPRECISERR in BFSR.

If a PE does not allow configuring implicit ESB insertion, this bit is WI and the value read indicates whether the PE never or always inserts an implicit ESB around lazy Floating-point state preservation, and on exception entry and return.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

If RAS is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

DIT, bit [4]

Data Independent Timing. This bit indicates and allows modification of whether for the selected Security state data independent timing operations are guaranteed to be timing invariant with respect to the data values being operated on.

This bit is banked between Security states.

The possible values of this bit are:

0

The architecture makes no statement about the timing properties of any instructions.

1

The architecture requires that the timing of every load and store instruction is insensitive to the value of the data being loaded or stored.

For certain data processing instructions, the instruction takes a time which is independent of:

- The values of the data supplied in any of its registers.
- The values of the NZCV flags.

For certain data processing instructions, the response of the instruction to asynchronous exceptions does not vary based on:

- The values of the data supplied in any of its registers.
- The values of the NZCV flags.

If configuration of timing invariance is not supported this bit is WI, and the value read indicates whether the PE always or never exhibits timing invariant behavior.

This bit resets to zero on a Warm reset.

SYSRESETREQS, bit [3]

System reset request Secure only. The value of this bit defines whether the SYSRESETREQ bit is functional for Non-secure use.

This bit is not banked between Security states.

The possible values of this bit are:

0

SYSRESETREQ functionality is available to both Security states.

1

SYSRESETREQ functionality is only available to Secure state.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

SYSRESETREQ, bit [2]

System reset request. This bit allows software or a debugger to request a system reset.

This bit is not banked between Security states.

The possible values of this bit are:

0

Do not request a system reset.

1

Request a system reset.

When SYSRESETREQS is set to 1, the Non-secure view of this bit is RAZ/WI.

This bit resets to zero on a Warm reset.

VECTCLRACTIVE, bit [1]

Clear active state.

A debugger write of one to this bit when the PE is halted in Debug state:

- IPSR is cleared to zero.
- If DHCSR.S_NSUIDE==0, the active state for all Non-secure exceptions is cleared.
- If DHCSR.S_SUIDE==0 and DHCSR.S_SDE==1, the active state for all Secure exceptions is cleared.

This bit is not banked between Security states.

The possible values of this bit are:

0

Do not clear active state.

1

Clear active state.

Writes to this bit while the PE is in Non-debug state are ignored.

This bit reads as zero.

Bit [0]

Reserved, [RES0](#).

D1.2.4 APSR, Application Program Status Register

The APSR characteristics are:

Purpose

Provides privileged and unprivileged access to the PE Execution state fields.

Usage constraints

Privileged and unprivileged access permitted.

Configurations

This register is always implemented.

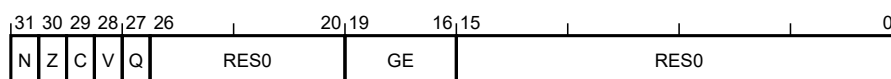
Attributes

32-bit read/write special-purpose register.

This register is not banked between Security states.

Field descriptions

The APSR bit assignments are:



N, bit [31]

Negative condition flag. When updated by a flag setting instruction this bit indicates whether the result of the operation when treated as a two's complement signed integer is negative.

The possible values of this bit are:

0

Result is positive or zero.

1

Result is negative.

See individual instruction pages for details.

This bit resets to an **UNKNOWN** value on a Warm reset.

Z, bit [30]

Zero condition flag. When updated by a flag setting instruction this bit indicates whether the result of the operation was zero.

The possible values of this bit are:

0

Result is nonzero.

1

Result is zero.

See individual instruction pages for details.

This bit resets to an **UNKNOWN** value on a Warm reset.

C, bit [29]

Carry condition flag. When updated by a flag setting instruction this bit indicates whether the operation resulted in an unsigned overflow or whether the last bit shifted out of the result was set.

The possible values of this bit are:

0
No carry occurred, or last bit shifted was clear.

1
Carry occurred, or last bit shifted was set.

See individual instruction pages for details.

This bit resets to an **UNKNOWN** value on a Warm reset.

V, bit [28]

Overflow condition flag. When updated by a flag setting instruction this bit indicates whether a signed overflow occurred.

The possible values of this bit are:

0
Signed overflow did not occur.

1
Signed overflow occurred.

See individual instruction pages for details.

This bit resets to an **UNKNOWN** value on a Warm reset.

Q, bit [27]

Sticky saturation flag. When updated by certain instructions this bit indicates either that an overflow occurred or that the result was saturated. This bit is cumulative and can only be cleared to zero by software.

The possible values of this bit are:

0
Saturation or overflow has not occurred since bit was last cleared.

1
Saturation or overflow has occurred since bit was last cleared.

See individual instruction pages for details.

If the Main Extension is not implemented, this bit is **RES0**.

This bit resets to an **UNKNOWN** value on a Warm reset.

Bits [26:20]

Reserved, **RES0**.

GE, bits [19:16]

Greater than or equal flags. When updated by parallel addition and subtraction instructions these bits record whether the result was greater than or equal to zero. SEL instructions use these bits to determine which register to select a particular byte from.

See individual instruction pages for details.

If the DSP Extension is not implemented, this field is **RES0**.

This field resets to an **UNKNOWN** value on a Warm reset.

Bits [15:0]

Reserved, **RES0**.

D1.2.5 BASEPRI, Base Priority Mask Register

The BASEPRI characteristics are:

Purpose

Changes the priority level required for exception preemption.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

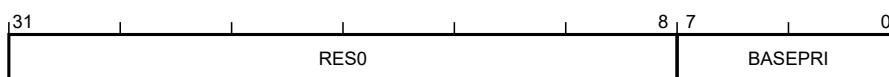
Attributes

32-bit read/write special-purpose register.

This register is banked between Security states.

Field descriptions

The BASEPRI bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

BASEPRI, bits [7:0]

Base priority mask. BASEPRI changes the priority level required for exception preemption. It has an effect only when BASEPRI has a lower value than the unmasked priority level of the currently executing software.

The possible values of this field are:

0

Disables masking by BASEPRI.

1-255

Priority value.

The number of implemented bits in BASEPRI is the same as the number of implemented bits in each field of the priority registers, and BASEPRI has the same format as those fields.

If the PE implements fewer than eight bits of priority, the least significant bits of this field are RAZ/WI.

This field resets to zero on a Warm reset.

D1.2.6 BFAR, BusFault Address Register

The BFAR characteristics are:

Purpose

Shows the address associated with a precise data access BusFault.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read/write register located at 0xE000ED38.

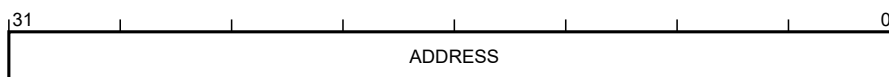
Secure software can access the Non-secure version of this register via BFAR_NS located at 0xE002ED38. The location 0xE002ED38 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The BFAR bit assignments are:



ADDRESS, bits [31:0]

Data address for a precise BusFault. This register is updated with the address of a location that produced a BusFault. BFSR shows the reason for the fault. This field is valid only when BFSR.BFARVALID is set, otherwise it is **UNKNOWN**.

In implementations without unique SFAR, BFAR and MMFAR registers, the value of this register is **UNKNOWN** if MMFSR.MMFARVALID or SFSR.SFARVALID is set.

If AIRCR.BFHFNMIN is zero this field is RAZ/WI from Non-secure state.

This field resets to an **UNKNOWN** value on a Warm reset.

Note

If an implementation shares a common SFAR, BFAR and MMFAR it must not leak Secure state information to the Non-secure state. One possible implementation is that BFAR shares resource with the Secure MMFAR if AIRCR.BFHFNMIN is zero, and with the Non-secure MMFAR if AIRCR.BFHFNMIN is set.

D1.2.7 BFSR, BusFault Status Register

The BFSR characteristics are:

Purpose

Shows the status of bus errors resulting from instruction fetches and data accesses. This includes errors resulting from RAS errors.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If the Main Extension is implemented, this register is word, halfword, and byte accessible.

If the Main Extension is not implemented, this register is word accessible only, halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

8-bit read/write-one-to-clear register located at 0xE000ED29.

Secure software can access the Non-secure version of this register via BFSR_NS located at 0xE002ED29. The location 0xE002ED29 is [RES0](#) to software executing in Non-secure state and the debugger.

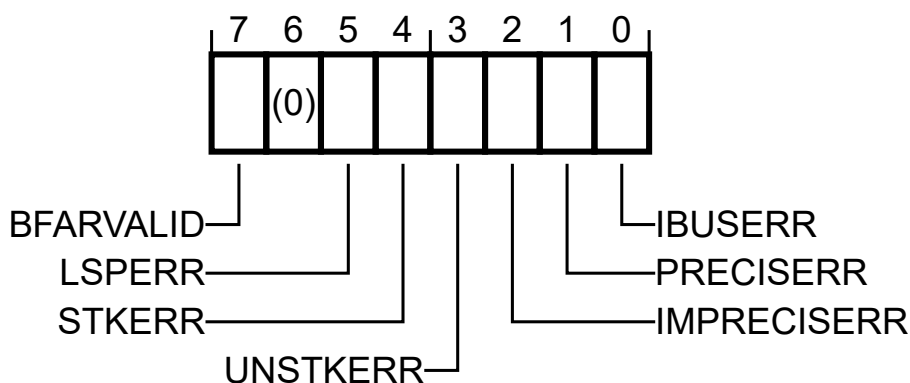
This register is not banked between Security states.

This register is part of CFSR.

From Armv8.1-M onwards it is [IMPLEMENTATION DEFINED](#) whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The BFSR bit assignments are:



BFARVALID, bit [7]

BFAR valid. Indicates validity of the contents of the BFAR register.

The possible values of this bit are:

0

BFAR content not valid.

1

BFAR content valid.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

Bit [6]

Reserved, [RES0](#).

LSPERR, bit [5]

Lazy state preservation error. Records whether a precise BusFault occurred during floating-point lazy Floating-point state preservation.

The possible values of this bit are:

0

No BusFault occurred.

1

BusFault occurred.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

STKERR, bit [4]

Stack error. Records whether a precise derived BusFault occurred during exception entry stacking.

The possible values of this bit are:

0

No derived BusFault occurred.

1

Derived BusFault occurred during exception entry.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

UNSTKERR, bit [3]

Unstack error. Records whether a precise derived BusFault occurred during exception return unstacking.

The possible values of this bit are:

0

No derived BusFault occurred.

1

Derived BusFault occurred during exception return.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

IMPRECISERR, bit [2]

Imprecise error. Records whether an imprecise data access error has occurred.

The possible values of this bit are:

0

No imprecise data access error has occurred.

1

Imprecise data access error has occurred.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

PRECISERR, bit [1]

Precise error. Records whether a precise data access error has occurred.

The possible values of this bit are:

0

No precise data access error has occurred.

1

Precise data access error has occurred.

When a precise error is recorded, the associated address is written to the BFAR and BFSR.BFARVALID bit is set.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

IBUSERR, bit [0]

Instruction bus error. Records whether a precise BusFault on an instruction prefetch has occurred.

The possible values of this bit are:

0

No BusFault on instruction prefetch has occurred.

1

A BusFault on an instruction prefetch has occurred.

An IBUSERR is only recorded if the instruction is issued for execution.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

D1.2.8 BPIALL, Branch Predictor Invalidate All

The BPIALL characteristics are:

Purpose

Invalidate all entries from branch predictors.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

32-bit write-only register located at 0xE000EF78.

Secure software can access the Non-secure version of this register via BPIALL_NS located at 0xE002EF78.

The location 0xE002EF78 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The BPIALL bit assignments are:



Ignored, bits [31:0]

Ignored. The value written to this field is ignored.

D1.2.9 CCR, Configuration and Control Register

The CCR characteristics are:

Purpose

Sets or returns configuration and control data.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible to read accesses through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set. Write accesses through unprivileged DAP requests are not permitted.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED14.

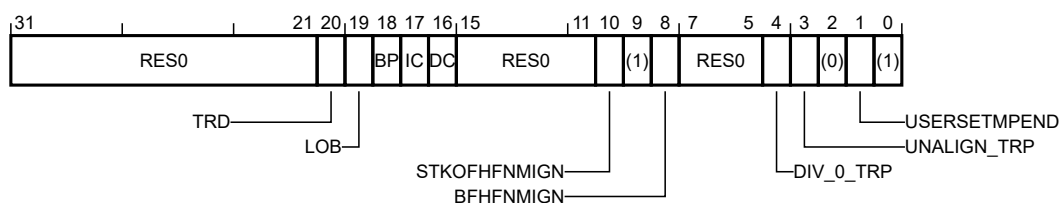
Secure software can access the Non-secure version of this register via CCR_NS located at 0xE002ED14. The location 0xE002ED14 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The CCR bit assignments are:



Bits [31:21]

Reserved, **RES0**.

TRD, bit [20]

Thread reentrancy disabled. Enables checking for exception stack frame integrity signatures on SG instructions. If enabled this check causes a fault to be raised if an attempt is made to re-enter Secure Thread mode while a call to Secure Thread mode is already in progress.

This bit is not banked between Security states.

The possible values of this bit are:

0

Integrity signature checking not performed.

1

Integrity signature checking performed.

This bit is RAZ/WI from Non-secure state.

If version Armv8.1-M of the architecture is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

LOB, bit [19]

Loop and branch info cache enable. Enables the branch cache used by loop and branch future instructions for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

Branch cache disabled for the selected Security state.

1

Branch cache enabled for the selected Security state.

If version Armv8.1-M of the architecture is not implemented, this bit is [RES0](#).

If the branch cache is not supported, this bit is RAZ/WI.

This bit resets to zero on a Warm reset.

BP, bit [18]

Branch prediction enable. Enables program flow prediction for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

Program flow prediction disabled for the selected Security state.

1

Program flow prediction enabled for the selected Security state.

If program flow prediction cannot be disabled, this bit is RAO/WI. If the program flow prediction is not supported, this bit is RAZ/WI. Arm recommends that the RAO/WI behavior should only be used where the branch predictor is static and does not display any dynamic behavior.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

IC, bit [17]

Instruction cache enable. This is a global enable bit for instruction caches in the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

Instruction caches disabled for the selected Security state.

1

Instruction caches enabled for the selected Security state.

If the PE does not implement instruction caches, this bit is RAZ/WI.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

DC, bit [16]

Data cache enable. Enables data caching of all data accesses to Normal memory.

This bit is banked between Security states.

The possible values of this bit are:

0

Data caching disabled.

1

Data caching enabled.

The secure version of this bit controls the Cacheability of accesses to secure memory.

The Non-secure version of this bit controls the Cacheability of accesses to Non-secure memory.

If the PE does not implement data caches, this bit is RAZ/WI.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

Bits [15:11]

Reserved, [RES0](#).

STKOFHFNIGN, bit [10]

Stack overflow in HardFault and NMI ignore. Controls the effect of a stack limit violation while executing at a requested priority less than 0 for the Security state with which the stack limit register is associated.

This bit is banked between Security states.

The possible values of this bit are:

0

Stack limit faults not ignored.

1

Stack limit faults at requested priorities of less than 0 ignored.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

Bit [9]

Reserved, [RES1](#).

BFHFNIGN, bit [8]

BusFault in HardFault or NMI ignore. Determines the effect of precise BusFaults arising from data accesses performed by code running at a requested priority less than 0.

This bit is not banked between Security states.

The possible values of this bit are:

0

Precise BusFaults not ignored.

1

Precise BusFaults at requested priorities of less than 0 ignored.

If AIRCR.BFHFNMIN is 0, this bit is read-only from Non-secure state.

Setting this bit to one is deprecated from Armv8.1-M onwards.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

Bits [7:5]

Reserved, [RES0](#).

DIV_0_TRP, bit [4]

Divide by zero trap. Controls the generation of a DIVBYZERO UsageFault when attempting to perform integer division by zero.

This bit is banked between Security states.

The possible values of this bit are:

0

DIVBYZERO UsageFault generation disabled.

1

DIVBYZERO UsageFault generation enabled.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

UNALIGN_TRP, bit [3]

Unaligned trap. Controls the trapping of unaligned word or halfword accesses.

This bit is banked between Security states.

The possible values of this bit are:

0

Unaligned accesses permitted from LDR, LDRH, STR, and STRH.

1

Any unaligned transaction generates an UNALIGNED UsageFault.

Unaligned load/store multiples and atomic/exclusive accesses always generate an UNALIGNED UsageFault.

If the Main Extension is not implemented, this bit is [RES1](#).

This bit resets to zero on a Warm reset if the Main Extension is implemented.

Bit [2]

Reserved, [RES0](#).

USERSETMPEND, bit [1]

User set main pending. Determines whether unprivileged accesses are permitted to pend interrupts via the STIR.

This bit is banked between Security states.

The possible values of this bit are:

0

Unprivileged accesses to the STIR generate a fault.

1

Unprivileged accesses to the STIR are permitted.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

Bit [0]

Reserved, [RES1](#).

D1.2.10 CCSIDR, Current Cache Size ID register

The CCSIDR characteristics are:

Purpose

The CCSIDR provides information about the architecture of the currently selected cache.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If CSSELR points to an unimplemented cache, the value of this register is **UNKNOWN**.

Configurations

This register is always implemented.

Attributes

32-bit read-only register located at 0xE000ED80.

Secure software can access the Non-secure version of this register via CCSIDR_NS located at 0xE002ED80.

The location 0xE002ED80 is **RES0** to software executing in Non-secure state and the debugger.

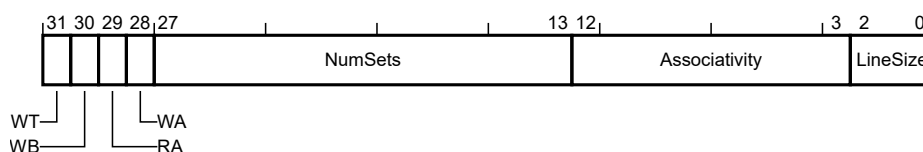
This register is banked between Security states.

Preface

Provides indirect read access to the architecture of the cache currently selected by CSSELR. The parameters NumSets, Associativity, and LineSize in these registers define the architecturally visible parameters that are required for the cache maintenance by Set/Way instructions. They are not guaranteed to represent the actual microarchitectural features of a design. You cannot make any inference about the actual sizes of caches based on these parameters.

Field descriptions

The CCSIDR bit assignments are:



WT, bit [31]

Write-Through. Indicates whether the currently selected cache level supports Write-Through.

The possible values of this bit are:

0
Not supported.

1
Supported.

This bit reads as an **IMPLEMENTATION DEFINED** value.

WB, bit [30]

Writeback. Indicates whether the currently selected cache level supports Write-Back.

The possible values of this bit are:

0

Not supported.

1

Supported.

This bit reads as an **IMPLEMENTATION DEFINED** value.

RA, bit [29]

Read-allocate. Indicates whether the currently selected cache level supports read-allocation.

The possible values of this bit are:

0

Not supported.

1

Supported.

This bit reads as an **IMPLEMENTATION DEFINED** value.

WA, bit [28]

Write-Allocate. Indicates whether the currently selected cache level supports write-allocation.

The possible values of this bit are:

0

Not supported.

1

Supported.

This bit reads as an **IMPLEMENTATION DEFINED** value.

NumSets, bits [27:13]

Number of sets. Indicates (Number of sets in the currently selected cache) - 1. Therefore, a value of 0 indicates that 1 is set in the cache. The number of sets does not have to be a power of 2.

This field reads as an **IMPLEMENTATION DEFINED** value.

Associativity, bits [12:3]

Associativity. Indicates (Associativity of cache) - 1. A value of 0 indicates an associativity of 1. The associativity does not have to be a power of 2.

This field reads as an **IMPLEMENTATION DEFINED** value.

LineSize, bits [2:0]

Line size. Indicates ($\log_2(\text{Number of words per line in the currently selected cache})$) - 2.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.11 CFSR, Configurable Fault Status Register

The CFSR characteristics are:

Purpose

Contains the three Configurable Fault Status Registers.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If the Main Extension is implemented, this register is word, halfword, and byte accessible.

If the Main Extension is not implemented, this register is word accessible only, halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read/write-one-to-clear register located at 0xE000ED28.

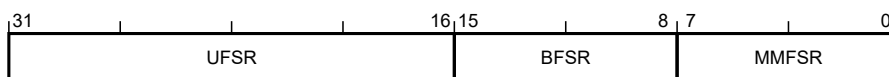
Secure software can access the Non-secure version of this register via CFSR_NS located at 0xE002ED28. The location 0xE002ED28 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The CFSR bit assignments are:



UFSR, bits [31:16]

UsageFault Status Register. Provides information on UsageFault exceptions.

This field is banked between Security states.

See UFSR.

This field resets to zero on a Warm reset.

BFSR, bits [15:8]

BusFault Status Register. Provides information on BusFault exceptions.

This field is not banked between Security states.

See BFSR.

This field resets to zero on a Warm reset.

MMFSR, bits [7:0]

MemManage Fault Status Register. Provides information on MemManage exceptions.

This field is banked between Security states.

See MMFSR.

This field resets to zero on a Warm reset.

D1.2.12 CLIDR, Cache Level ID Register

The CLIDR characteristics are:

Purpose

Identifies the type of caches implemented and the level of coherency and unification.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

32-bit read-only register located at 0xE000ED78.

Secure software can access the Non-secure version of this register via CLIDR_NS located at 0xE002ED78.

The location 0xE002ED78 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The CLIDR bit assignments are:

31	30	29	27	26	24	23	21	20	18	17	15	14	12	11	9	8	6	5	3	2	0
ICB	LoUU	LoC	LoUIS	Ctype7	Ctype6	Ctype5	Ctype4	Ctype3	Ctype2	Ctype1											

ICB, bits [31:30]

Inner cache boundary. This field indicates the boundary between inner and outer domain.

The possible values of this field are:

0b00

Not disclosed in this mechanism.

0b01

L1 cache is the highest inner level.

0b10

L2 cache is the highest inner level.

0b11

L3 cache is the highest inner level.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

LoUU, bits [29:27]

Level of Unification Uniprocessor. This field indicates the Level of Unification Uniprocessor for the cache hierarchy.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

LoC, bits [26:24]

Level of Coherence. This field indicates the Level of Coherence for the cache hierarchy.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

LoUIS, bits [23:21]

Level of Unification Inner Shareable. This field indicates the Level of Unification Shareable for the cache hierarchy.

This field reads as an **IMPLEMENTATION DEFINED** value.

Ctypem, bits [3(m-1)+2:3(m-1)], for m = 1 to 7

Cache type field *m*. Indicates the type of cache implemented at level *m*.

The possible values of this field are:

0b000

No cache.

0b001

Instruction cache only.

0b010

Data cache only.

0b011

Separate instruction and data caches.

0b100

Unified cache.

All other values are reserved.

If Ctype<*m*> is set to 0b000, and *m* < 7, then all of the following apply.

Level *m* represents the last level of software-visible cache.

Ctype<*m*=1> through to Ctype7 must read as zero.

Software must treat Ctype<*m*+1> through Ctype7 as if they are invalid and read as an **UNKNOWN** value.

This field reads as an **IMPLEMENTATION DEFINED** value.

0
Pointer authentication is disabled for privileged accesses.

1
Pointer authentication is enabled for privileged accesses.
If the PACBTI Extension is not implemented, this bit is [RES0](#).
This bit resets to zero on a Warm reset.

UBTI_EN, bit [5]

Unprivileged branch target identification enable. Unless otherwise stated, when this feature is enabled the branch target for BTI setting instructions in unprivileged mode must be a BTI clearing instruction otherwise an INVSTATE UsageFault is generated.

This bit is banked between Security states.

The possible values of this bit are:

0
Branch target identification disabled for unprivileged accesses.

1
Branch target identification enabled for unprivileged accesses.

If the PACBTI Extension is not implemented, this bit is [RES0](#).
This bit resets to zero on a Warm reset.

BTI_EN, bit [4]

Privileged branch target identification enable. Unless otherwise stated, when this feature is enabled the branch target for BTI setting instructions in privileged mode must be a BTI clearing instruction, otherwise an INVSTATE UsageFault is generated.

This bit is banked between Security states.

The possible values of this bit are:

0
Branch target identification disabled for privileged accesses.

1
Branch target identification enabled for privileged accesses.

If the PACBTI Extension is not implemented, this bit is [RES0](#).
This bit resets to zero on a Warm reset.

SFPA, bit [3]

Secure Floating-point active. Indicates that the Floating-point registers contain active state that belongs to the Secure state.

This bit is not banked between Security states.

The possible values of this bit are:

0
The Floating-point registers do not contain state that belongs to the Secure state.

1
The Floating-point registers contain state that belongs to the Secure state.

This bit accessible from both privileged and unprivileged modes using the MRS or MSR instructions.

This bit is RAZ/WI from Non-secure state.

If the Security Extension is not implemented, this bit is [RES0](#).

If neither the Floating-point Extension or MVE are implemented, this bit is RAZ/WI.

This bit resets to zero on a Warm reset.

FPCA, bit [2]

Floating-point context active. Defines whether the Floating-point Extension is active in the current context.

This bit is not banked between Security states.

The possible values of this bit are:

0

Floating-point Extension is not active.

1

Floating-point Extension is active.

When NSACR.CP10 is set to zero, the Non-secure view of this bit is read-only. If FPCCR.ASPEN is set to 1, enabling automatic Floating-point state preservation, then the PE sets this bit to 1 on successful completion of any Floating-point instruction.

If neither the Floating-point Extension or MVE are implemented, this bit is RAZ/WI.

This bit resets to zero on a Warm reset.

SPSEL, bit [1]

Stack-pointer select. Defines the stack pointer to be used.

This bit is banked between Security states.

The possible values of this bit are:

0

Use SP_main as the current stack.

1

In Thread mode use SP_process as the current stack.

This bit resets to zero on a Warm reset.

nPRIV, bit [0]

Not privileged. Defines the execution privilege in Thread mode.

This bit is banked between Security states.

The possible values of this bit are:

0

Thread mode has privileged access.

1

Thread mode has unprivileged access only.

If the Main Extension is not implemented, it is **IMPLEMENTATION DEFINED** whether this field is RW or RAZ/WI.

This bit resets to zero on a Warm reset.

D1.2.14 CPACR, Coprocessor Access Control Register

The CPACR characteristics are:

Purpose

Specifies the access privileges for coprocessors and the Floating-point Extension. If MVE is implemented, this register specifies the access privileges for MVE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read/write register located at 0xE000ED88.

Secure software can access the Non-secure version of this register via CPACR_NS located at 0xE002ED88. The location 0xE002ED88 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The CPACR bit assignments are:

31																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					
----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Bits [31:24]

Reserved, RES0.

CP11, bits [23:22]

CP11 Privilege. The value in this field is ignored. If the CP10 field is RAZ/WI this field is also RAZ/WI. If the value of this field is not programmed to the same value as the CP10 field, then the value is **UNKNOWN**.

This field resets to an **UNKNOWN** value on a Warm reset.

CP10, bits [21:20]

CP10 Privilege. Defines the access rights for the Floating-point functionality.

The possible values of this field are:

0b00

All accesses to the Floating-point Extension and MVE result in NOCP UsageFault.

0b01

Unprivileged accesses to the Floating-point Extension and MVE result in NOCP UsageFault.

0b11

Full access to the Floating-point Extension and MVE.

All other values are reserved.

The features controlled by this field are:

- Unless otherwise specified, the execution of any instructions within the encoding space defined by IsCPIInstruction() for CP10.
- Access to any Floating-point registers in the range D0-D16.

See individual instruction pages for details.

If neither the Floating-point Extension or MVE are implemented, this field is RAZ/WI.

This field resets to an UNKNOWN value on a Warm reset.

Bits [19:16]

Reserved, RES0.

CP m , bits [2 m +1:2 m], for $m = 0$ to 7

Coprocessor m privilege. Controls access privileges for coprocessor m .

The possible values of this field are:

0b00

Access denied. Any attempted access generates a NOCP UsageFault.

0b01

Privileged access only. An unprivileged access generates a NOCP UsageFault.

0b10

Reserved.

0b11

Full access.

If coprocessor m is not implemented, this field is RAZ/WI.

This field resets to an UNKNOWN value on a Warm reset.

D1.2.15 CPPWR, Coprocessor Power Control Register

The CPPWR characteristics are:

Purpose

Specifies whether coprocessors are permitted to enter a non-retentive power state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read/write register located at 0xE000E00C.

Secure software can access the Non-secure version of this register via CPPWR_NS located at 0xE002E00C. The location 0xE002E00C is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

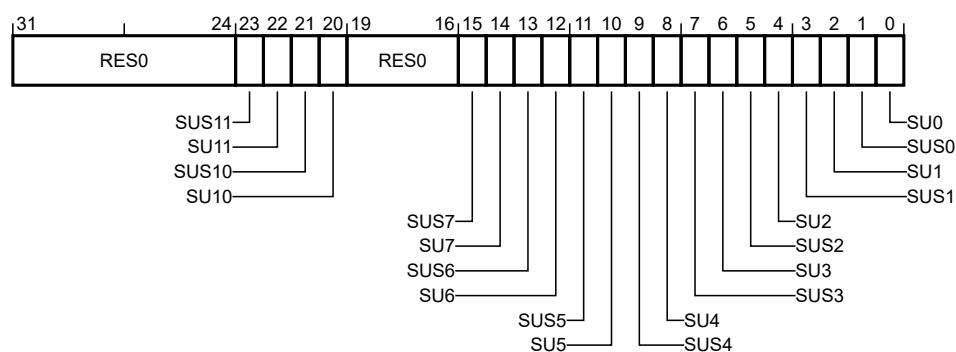
From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

If no coprocessors are implemented this register is RAZ/WI.

Field descriptions

The CPPWR bit assignments are:



Bits [31:24]

Reserved, **RES0**.

SUS11, bit [23]

State **UNKNOWN** Secure only 11. The value in this field is ignored. If the value of this bit is not programmed to the same value as the SUS10 field, then the value is **UNKNOWN**.

If SU10 is always RAZ/WI this field is also RAZ/WI.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

SU11, bit [22]

State **UNKNOWN** 11. The value in this field is ignored. If the value of this bit is not programmed to the same value as the SU10 field, then the value is **UNKNOWN**.

When SUS10 is set to 1, the Non-secure view of this bit is RAZ/WI. If SU10 is always RAZ/WI this field is also RAZ/WI.

This bit resets to zero on a Warm reset.

SUS10, bit [21]

State **UNKNOWN** Secure only 10. This bit indicates and allows modification of whether the SU10 field can be modified from Non-secure state.

The possible values of this bit are:

0

The SU10 field is accessible from both Security states.

1

The SU10 field is only accessible from the Secure state.

If SU10 is always RAZ/WI this field is also RAZ/WI.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

SU10, bit [20]

State **UNKNOWN** 10. This bit indicates and allows modification of whether the state associated with the Floating-point and MVE units is permitted to become **UNKNOWN**. This can be used as a hint to power control logic that these units might be powered down.

The possible values of this bit are:

0

The Floating-point and MVE state is not permitted to become **UNKNOWN**.

1

The Floating-point and MVE state is permitted to become **UNKNOWN**.

When SUS10 is set to 1, the Non-secure view of this bit is RAZ/WI. It is **IMPLEMENTATION DEFINED** whether this bit is always RAZ/WI.

This bit resets to zero on a Warm reset.

Bits [19:16]

Reserved, **RES0**.

SUS m , bit [2 m +1], for $m = 0$ to 7

State **UNKNOWN** Secure only m . This field indicates and allows modification of whether the SUM field can be modified from Non-secure state.

The possible values of this field are:

0

The SUM field is accessible from both Security states.

1

The SUM field is only accessible from the Secure state.

If SUM is always RAZ/WI this field is also RAZ/WI.

This field is RAZ/WI from Non-secure state.

This field resets to zero on a Warm reset.

SU m , bit [2 m], for $m = 0$ to 7

State **UNKNOWN** m . This field indicates and allows modification of whether the state associated with coprocessor m is permitted to become **UNKNOWN**. This can be used as a hint to power control logic that the coprocessor might be powered down.

The possible values of this field are:

0

The coprocessor state is not permitted to become **UNKNOWN**.

1

The coprocessor state is permitted to become **UNKNOWN**.

When SUS m is set to 1, the Non-secure view of this bit is RAZ/WI. It is **IMPLEMENTATION DEFINED** whether this bit is always RAZ/WI.

This field resets to zero on a Warm reset.

D1.2.16 CPUID, CPUID Base Register

The CPUID characteristics are:

Purpose

Provides identification information for the PE, including an implementer code for the device and a device ID number.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

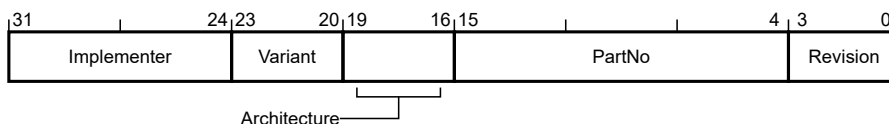
32-bit read-only register located at 0xE000ED00.

Secure software can access the Non-secure version of this register via CPUID_NS located at 0xE002ED00. The location 0xE002ED00 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The CPUID bit assignments are:



Implementer, bits [31:24]

Implementer code. This field must hold an implementer code that has been assigned by Arm.

The possible values of this field are:

0x41

'A': Arm Limited.

Not 0x41

Implementer other than Arm Limited.

Arm can assign codes that are not published in this manual. All values not assigned by Arm are reserved and must not be used.

This field reads as an **IMPLEMENTATION DEFINED** value.

Variant, bits [23:20]

Variant number. **IMPLEMENTATION DEFINED** variant number. Typically, this field is used to distinguish between different product variants, or major revisions of a product.

This field reads as an **IMPLEMENTATION DEFINED** value.

Architecture, bits [19:16]

Architecture version. Defines the Architecture implemented by the PE.

The possible values of this field are:

0b1100

Armv8-M architecture without Main Extension.

0b1111

Armv8-M architecture with Main Extension.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

PartNo, bits [15:4]

Part number. **IMPLEMENTATION DEFINED** primary part number for the device.

This field reads as an **IMPLEMENTATION DEFINED** value.

Revision, bits [3:0]

Revision number. **IMPLEMENTATION DEFINED** revision number for the device.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.17 CSSELR, Cache Size Selection Register

The CSSELR characteristics are:

Purpose

Selects the current Cache Size ID Register, CCSIDR, by specifying the required cache level and the cache type (either instruction or data cache)

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED84.

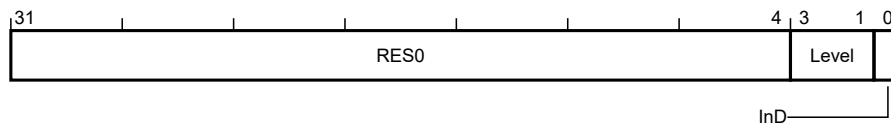
Secure software can access the Non-secure version of this register via CSSELR_NS located at 0xE002ED84. The location 0xE002ED84 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The CSSELR bit assignments are:

**Bits [31:4]**

Reserved, RES0.

Level, bits [3:1]

Cache level. Selects which cache level is to be identified. Permitted values are from 0b000, indicating Level 1 cache, to 0b110 indicating Level 7 cache.

The possible values of this field are:

0b000

Level 1 cache.

0b001

Level 2 cache.

0b010

Level 3 cache.

0b011

Level 4 cache.

0b100

Level 5 cache.

0b101

Level 6 cache.

0b110

Level 7 cache.

All other values are reserved.

Writing a reserved value or value corresponding to an unimplemented level of cache is **CONSTRAINED UNPREDICTABLE**.

This field resets to an **UNKNOWN** value on a Warm reset.

InD, bit [0]

Instruction not data. Selects whether the instruction or the data cache is to be identified.

The possible values of this bit are:

0

Data or unified cache.

1

Instruction cache.

This bit resets to an **UNKNOWN** value on a Warm reset.

D1.2.18 CTR, Cache Type Register

The CTR characteristics are:

Purpose

Provides information about the architecture of the caches.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

32-bit read-only register located at 0xE000ED7C.

Secure software can access the Non-secure version of this register via CTR_NS located at 0xE002ED7C.

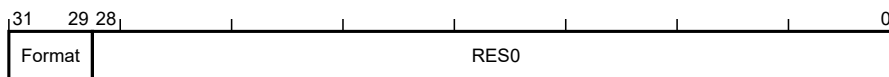
The location 0xE002ED7C is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

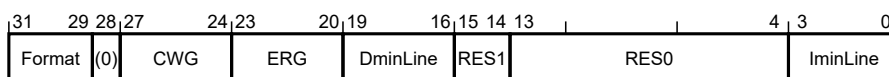
Field descriptions

The CTR bit assignments are:

When Format!='0b100':



When Format=='0b100':

**Format, bits [31:29]**

Cache Type Register format. Indicates whether cache type information is provided.

The possible values of this field are:

0b000

No cache type information is provided.

0b100

Cache type information is provided.

All other values are reserved.

The value of this field is an **IMPLEMENTATION DEFINED** choice of either 0b000 or 0b100.

If CLIDR is nonzero then this field must read as 0b100.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [28:0], when Format!=’0b100’

Reserved, **RES0**.

Bit [28], when Format==‘0b100’

Reserved, [RES0](#).

CWG, bits [27:24], when Format==‘0b100’

Cache Write-Back Granule. $\log_2$ of the number of words of the maximum size of memory that can be overwritten as a result of the eviction of a cache entry that has had a memory location in it modified.

The possible values of this field are:

0b0000

Indicates that this register does not provide Cache Write-Back Granule information and either the architectural maximum of 512 words (2KB) must be assumed, or the Cache Write-Back Granule can be determined from maximum cache line size encoded in the Cache Size ID Registers.

0b0001-0b1000

$\log_2$ of the number of words.

All other values are reserved.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

ERG, bits [23:20], when Format==‘0b100’

Exclusives Reservation Granule. $\log_2$ of the number of words of the maximum size of the reservation granule that has been implemented for the Load-Exclusive and Store-Exclusive instructions.

The possible values of this field are:

0b0000

Indicates that this register does not provide Exclusives Reservation Granule information and the architectural maximum of 512 words (2KB) must be assumed.

0b0001-0b1000

$\log_2$ of the number of words.

All other values are reserved.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

DminLine, bits [19:16], when Format==‘0b100’

Data cache minimum line length. $\log_2$ of the number of words in the smallest cache line of all the data caches and unified caches that are controlled by the PE.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

Bits [15:14], when Format==‘0b100’

Reserved, [RES1](#).

Bits [13:4], when Format==‘0b100’

Reserved, [RES0](#).

IminLine, bits [3:0], when Format==‘0b100’

Instruction cache minimum line length. $\log_2$ of the number of words in the smallest cache line of all the instruction caches that are controlled by the PE.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

D1.2.19 Dn, Floating-point Double-precision register, n = 0 - 15

The D{0..15} characteristics are:

Purpose

Provides access to the Floating-point Double-precision register file. These registers are an alias of the Floating-point register file.

Usage constraints

Privileged and unprivileged access permitted.

Configurations

Present if the Floating-point Extension, or MVE, or both are implemented.

This register is [RES0](#) if neither the Floating-point Extension nor MVE are implemented.

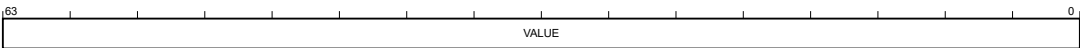
Attributes

64-bit read/write special-purpose register.

This register is not banked between Security states.

Field descriptions

The D{0..15} bit assignments are:



VALUE, bits [63:0]

Floating-point double-precision register value.

D1.2.20 DAUTHCTRL, Debug Authentication Control Register

The DAUTHCTRL characteristics are:

Purpose

This register allows the **IMPLEMENTATION DEFINED** authentication interface to be overridden from software.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If Armv8.1-M is implemented, this register is word, halfword and byte accessible.

This register is **RES0** if accessed via the debugger.

Configurations

Present if Halting debug or the Main Extension is implemented.

This register is **RES0** if both Halting debug and Main Extension are not implemented.

Attributes

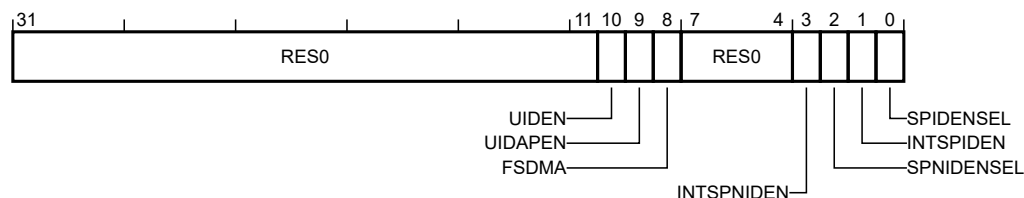
32-bit read/write register located at 0xE000EE04.

Secure software can access the Non-secure version of this register via DAUTHCTRL_NS located at 0xE002EE04. The location 0xE002EE04 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

Field descriptions

The DAUTHCTRL bit assignments are:



Bits [31:11]

Reserved, **RES0**.

UIDEN, bit [10]

Unprivileged Invasive Debug Enable. Enables halting debug for unprivileged modes, regardless of the state of other debug controls.

This bit is banked between Security states.

The possible values of this bit are:

0

Halting debug operates as normal.

1

Unprivileged halting debug allowed.

See UnprivHaltingDebugEnabled() and UpdateDebugEnabled().

If version Armv8.1 of the architecture and UDE are not implemented, this bit is **RES0**.

This bit resets to zero on a Warm reset.

UIDAPEN, bit [9]

Unprivileged Invasive DAP Access Enable. Enables unprivileged DAP access to specific PPB registers.

This bit is banked between Security states.

The possible values of this bit are:

0

No unprivileged DAP access.

1

Unprivileged DAP access allowed.

Unprivileged DAP access is allowed if either of the banked fields is set.

If version Armv8.1 of the architecture and UDE are not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

FSDMA, bit [8]

Force Secure DebugMonitor Allowed. Allows Secure DebugMonitor to be enabled without having to enable secure halting debug.

This bit is not banked between Security states.

The possible values of this bit are:

0

Secure DebugMonitor determined by other means.

1

Secure DebugMonitor allowed.

This bit is RAZ/WI from Non-secure state.

If version Armv8.1-M of the architecture is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

Bits [7:4]

Reserved, [RES0](#).

INTSPNIDEN, bit [3]

Internal Secure non-invasive debug enable. Overrides the external Secure non-invasive debug authentication interface.

This bit is not banked between Security states.

The possible values of this bit are:

0

Secure Non-invasive debug prohibited.

1

Secure Non-invasive debug allowed.

Ignored if DAUTHCTRL.SPNIDENSEL == 0. See SecureNoninvasiveDebugEnabled().

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Cold reset.

SPNIDENSEL, bit [2]

Secure non-invasive debug enable select. Selects between DAUTHCTRL and the [IMPLEMENTATION DEFINED](#) external authentication interface for control of Secure non-invasive debug.

This bit is not banked between Security states.

The possible values of this bit are:

0

Secure non-invasive debug controlled by the **IMPLEMENTATION DEFINED** external authentication interface. In the CoreSight authentication interface, this is controlled by the SPNIDEN signal.

1

Secure non-invasive debug controlled by DAUTHCTRL.INTSPNIDEN.

The PE ignores the value of this bit and Secure non-invasive debug is allowed if DHCSR.S_SDE == 1. See SecureNoninvasiveDebugAllowed().

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Cold reset.

INTSPIDEN, bit [1]

Internal Secure invasive debug enable. Overrides the external Secure invasive debug authentication interfaces.

This bit is not banked between Security states.

The possible values of this bit are:

0

Secure halting and self-hosted debug prohibited.

1

Secure halting and self-hosted debug allowed.

Ignored if DAUTHCTRL.SPIDENSEL == 0. See SecureHaltingDebugEnabled() and SecureDebugMonitorAllowed().

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Cold reset.

SPIDENSEL, bit [0]

Secure invasive debug enable select. Selects between DAUTHCTRL and the **IMPLEMENTATION DEFINED** external authentication interface for control of Secure invasive debug.

This bit is not banked between Security states.

The possible values of this bit are:

0

Secure halting and self-hosted debug controlled by the **IMPLEMENTATION DEFINED** external authentication interface. In the CoreSight authentication interface, both are controlled by the SPIDEN signal.

1

Secure halting and self-hosted debug controlled by DAUTHCTRL.INTSPIDEN.

See SecureHaltingDebugEnabled() and SecureDebugMonitorAllowed().

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Cold reset.

D1.2.21 DAUTHSTATUS, Debug Authentication Status Register

The DAUTHSTATUS characteristics are:

Purpose

Provides information about the state of the **IMPLEMENTATION DEFINED** authentication interface for debug.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

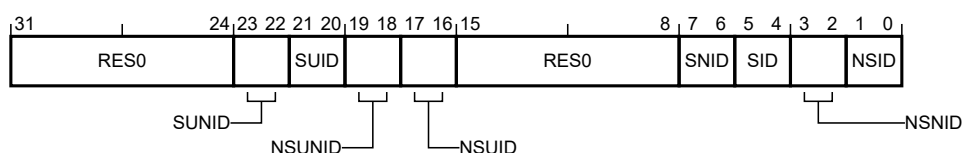
32-bit read-only register located at 0xE000EFB8.

Secure software can access the Non-secure version of this register via DAUTHSTATUS_NS located at 0xE002EFB8. The location 0xE002EFB8 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DAUTHSTATUS bit assignments are:



Bits [31:24]

Reserved, **RES0**.

SUNID, bits [23:22]

Secure Unprivileged Non-invasive Debug Allowed. Indicates that Unprivileged Non-invasive debug is allowed for the Secure state.

The possible values of this field are:

0b00

Security Extension or Unprivileged Non-invasive Debug not implemented.

0b01

Reserved.

0b10

Secure Non-invasive debug prohibited.

0b11

Secure Non-invasive debug allowed in unprivileged mode.

If version Armv8.1 of the architecture and UDE are not implemented, this field is [RES0](#).

SUID, bits [21:20]

Secure Unprivileged Invasive Debug Allowed. Indicates that Unprivileged Halting Debug is allowed for the Secure state.

The possible values of this field are:

0b00

Security Extension or Unprivileged Debug not implemented.

0b01

Reserved.

0b10

Secure halting debug prohibited.

0b11

Secure halting debug allowed in unprivileged mode.

If version Armv8.1 of the architecture and UDE are not implemented, this field is [RES0](#).

NSUNID, bits [19:18]

Non-secure Unprivileged Non-invasive Debug Allowed. Indicates that Unprivileged Non-invasive Debug is allowed for the Non-secure state.

The possible values of this field are:

0b00

Unprivileged Non-invasive debug not implemented.

0b01

Reserved.

0b10

Non-secure Non-invasive debug prohibited.

0b11

Non-secure Non-invasive debug allowed in unprivileged mode.

If version Armv8.1 of the architecture and UDE are not implemented, this field is [RES0](#).

NSUID, bits [17:16]

Non-secure Unprivileged Invasive Debug Allowed. Indicates that Unprivileged Halting Debug is allowed for the Non-secure state.

The possible values of this field are:

0b00

Unprivileged halting debug not implemented.

0b01

Reserved.

0b10

Non-secure halting debug prohibited.

0b11

Non-secure halting debug allowed in unprivileged mode.

If version Armv8.1 of the architecture and UDE are not implemented, this field is [RES0](#).

Bits [15:8]

Reserved, [RES0](#).

SNID, bits [7:6]

Secure Non-invasive Debug. Indicates whether Secure non-invasive debug is implemented and allowed.

The possible values of this field are:

0b00

Security Extension not implemented.

0b01

Reserved.

0b10

Security Extension implemented and Secure non-invasive debug prohibited.

0b11

Security Extension implemented and Secure non-invasive debug allowed.

SID, bits [5:4]

Secure Invasive Debug. Indicates whether Secure invasive debug is implemented and allowed.

The possible values of this field are:

0b00

Security Extension not implemented.

0b01

Reserved.

0b10

Security Extension implemented and Secure invasive debug prohibited.

0b11

Security Extension implemented and Secure invasive debug allowed.

NSNID, bits [3:2]

Non-secure Non-invasive Debug. Indicates whether Non-secure non-invasive debug is allowed.

The possible values of this field are:

0b0x

Reserved.

0b10

Non-secure non-invasive debug prohibited.

0b11

Non-secure non-invasive debug allowed.

NSID, bits [1:0]

Non-secure Invasive Debug. Indicates whether Non-secure invasive debug is allowed.

The possible values of this field are:

0b0x

Reserved.

0b10

Non-secure invasive debug prohibited.

0b11

Non-secure invasive debug allowed.

D1.2.22 DCCIMVAC, Data Cache line Clean and Invalidate by Address to PoC

The DCCIMVAC characteristics are:

Purpose

Clean and invalidate data or unified cache line by address to PoC.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

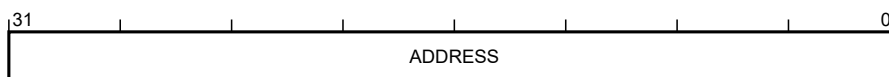
32-bit write-only register located at 0xE000EF70.

Secure software can access the Non-secure version of this register via DCCIMVAC_NS located at 0xE002EF70. The location 0xE002EF70 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCCIMVAC bit assignments are:



ADDRESS, bits [31:0]

Address. Writing to this field initiates the maintenance operation for the address written.

D1.2.23 DCCISW, Data Cache line Clean and Invalidate by Set/Way

The DCCISW characteristics are:

Purpose

Clean and invalidate data or unified cache line by set/way.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

32-bit write-only register located at 0xE000EF74.

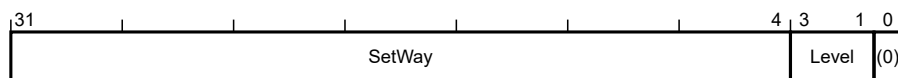
Secure software can access the Non-secure version of this register via DCCISW_NS located at 0xE002EF74.

The location 0xE002EF74 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCCISW bit assignments are:

**SetWay, bits [31:4]**

Cache set/way. Contains two fields: Way, bits[31:32-A], the number of the way to operate on. Set, bits[B-1:L], the number of the set to operate on. Bits[L-1:4] are [RES0](#). A = $\text{Log}_2(\text{ASSOCIATIVITY})$, L = $\text{Log}_2(\text{LINELEN})$, B = (L + S), S = $\text{Log}_2(\text{NSETS})$. ASSOCIATIVITY, LINELEN (line length, in bytes), and NSETS (number of sets) have their usual meanings and are the values for the cache level being operated on. The values of A and S are rounded up to the next integer.

Level, bits [3:1]

Cache level. Cache level to operate on, minus 1. For example, this field is 0 for operations on L1 cache, or 1 for operations on L2 cache.

Bit [0]

Reserved, RES0.

D1.2.24 DCCMVAC, Data Cache line Clean by Address to PoC

The DCCMVAC characteristics are:

Purpose

Clean data or unified cache line by address to PoC.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

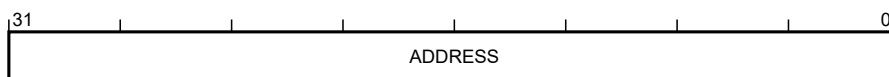
32-bit write-only register located at 0xE000EF68.

Secure software can access the Non-secure version of this register via DCCMVAC_NS located at 0xE002EF68. The location 0xE002EF68 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCCMVAC bit assignments are:



ADDRESS, bits [31:0]

Address. Writing to this field initiates the maintenance operation for the address written.

D1.2.25 DCCMVAU, Data Cache line Clean by address to PoU

The DCCMVAU characteristics are:

Purpose

Clean data or unified cache line by address to PoU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

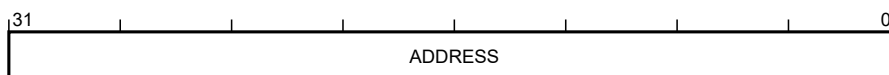
32-bit write-only register located at 0xE000EF64.

Secure software can access the Non-secure version of this register via DCCMVAU_NS located at 0xE002EF64. The location 0xE002EF64 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCCMVAU bit assignments are:



ADDRESS, bits [31:0]

Address. Writing to this field initiates the maintenance operation for the address written.

D1.2.27 DCIDR0, SCS Component Identification Register 0

The DCIDR0 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

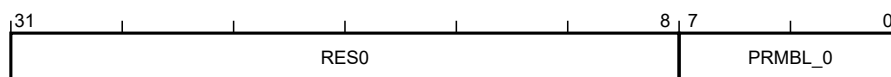
32-bit read-only register located at 0xE000EFF0.

Secure software can access the Non-secure version of this register via DCIDR0_NS located at 0xE002EFF0. The location 0xE002EFF0 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCIDR0 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

PRMBL_0, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0D.

D1.2.28 DCIDR1, SCS Component Identification Register 1

The DCIDR1 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

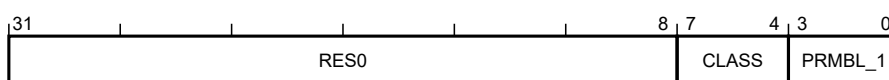
32-bit read-only register located at 0xE000EFF4.

Secure software can access the Non-secure version of this register via DCIDR1_NS located at 0xE002EFF4. The location 0xE002EFF4 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCIDR1 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

CLASS, bits [7:4]

CoreSight component class. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x9.

PRMBL_1, bits [3:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0.

D1.2.29 DCIDR2, SCS Component Identification Register 2

The DCIDR2 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

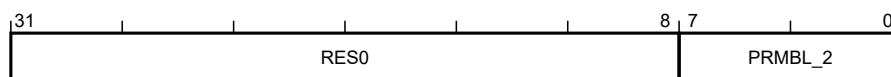
32-bit read-only register located at 0xE000EFF8.

Secure software can access the Non-secure version of this register via DCIDR2_NS located at 0xE002EFF8. The location 0xE002EFF8 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCIDR2 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

PRMBL_2, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x05.

D1.2.30 DCIDR3, SCS Component Identification Register 3

The DCIDR3 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

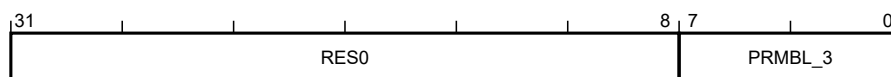
32-bit read-only register located at 0xE000EFC.

Secure software can access the Non-secure version of this register via DCIDR3_NS located at 0xE002EFFC. The location 0xE002EFFC is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCIDR3 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

PRMBL_3, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0xB1.

D1.2.31 DCIMVAC, Data Cache line Invalidate by Address to PoC

The DCIMVAC characteristics are:

Purpose

Invalidate data or unified cache line by address to PoC.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via an unprivileged debugger. It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

This register is always implemented.

Attributes

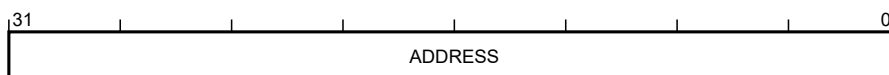
32-bit write-only register located at 0xE000EF5C.

Secure software can access the Non-secure version of this register via DCIMVAC_NS located at 0xE002EF5C. The location 0xE002EF5C is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCIMVAC bit assignments are:



ADDRESS, bits [31:0]

Address. Writing to this field initiates the maintenance operation for the address written.

D1.2.32 DCISW, Data Cache line Invalidate by Set/Way

The DCISW characteristics are:

Purpose

Invalidate data or unified cache line by set/way.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via an unprivileged debugger. It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

This register is always implemented.

Attributes

32-bit write-only register located at 0xE000EF60.

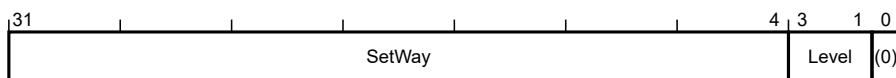
Secure software can access the Non-secure version of this register via DCISW_NS located at 0xE002EF60.

The location 0xE002EF60 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCISW bit assignments are:

**SetWay, bits [31:4]**

Cache set/way. Contains two fields: Way, bits[31:32-A], the number of the way to operate on. Set, bits[B-1:L], the number of the set to operate on. Bits[L-1:4] are [RES0](#). A = $\text{Log}_2(\text{ASSOCIATIVITY})$, L = $\text{Log}_2(\text{LINELEN})$, B = (L + S), S = $\text{Log}_2(\text{NSETS})$. ASSOCIATIVITY, LINELEN (line length, in bytes), and NSETS (number of sets) have their usual meanings and are the values for the cache level being operated on. The values of A and S are rounded up to the next integer.

Level, bits [3:1]

Cache level. Cache level to operate on, minus 1. For example, this field is 0 for operations on L1 cache, or 1 for operations on L2 cache.

Bit [0]

Reserved, RES0.

D1.2.33 DCRDR, Debug Core Register Data Register

The DCRDR characteristics are:

Purpose

With the DCRSR, provides debug access to the general-purpose registers, special-purpose registers, and the Floating-point Extension registers. If the Main Extension is implemented, it can also be used for message passing between an external debugger and a debug agent running on the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then this register is accessible only to the debugger and **UNKNOWN** to software.

Configurations

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

32-bit read/write register located at 0xE000EDF8.

Secure software can access the Non-secure version of this register via DCRDR_NS located at 0xE002EDF8. The location 0xE002EDF8 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DCRDR bit assignments are:



DBGTMP, bits [31:0]

Data temporary buffer. Provides debug access for reading and writing the general-purpose registers, special-purpose registers, and Floating-point Extension registers.

The value of this register is **UNKNOWN** if the PE is in Debug state, the debugger has written to DCRSR since entering Debug state and DHCSR.S_REGRDY is set to 0. The value of this register is **UNKNOWN** if the Main Extension is not implemented and the PE is in Non-debug state.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.34 DCRSR, Debug Core Register Select Register

The DCRSR characteristics are:

Purpose

With the DCRDR, provides debug access to the general-purpose registers, special-purpose registers, and the Floating-point Extension registers. A write to the DCRSR specifies the register to transfer, whether the transfer is a read or write, and starts the transfer.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Writes to this register while the PE is in Non-debug state are ignored.

This register is accessible only to the debugger and [RES0](#) to software.

Configurations

Present only if Halting debug is implemented.

This register is [RES0](#) if Halting debug is not implemented.

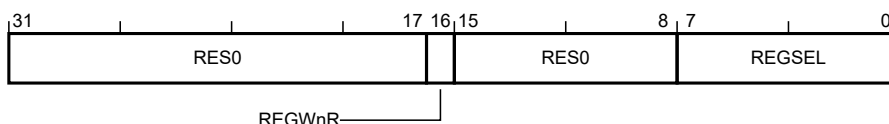
Attributes

32-bit write-only register located at 0xE000EDF4.

This register is not banked between Security states.

Field descriptions

The DCRSR bit assignments are:



Bits [31:17]

Reserved, [RES0](#).

REGWnR, bit [16]

Register write/not-read. Specifies the access type for the transfer.

The possible values of this bit are:

0

Read.

1

Write.

Bits [15:8]

Reserved, [RES0](#).

REGSEL, bits [7:0]

Register selector. Specifies the general-purpose register, special-purpose register, or Floating-point Extension register to transfer.

The possible values of this field are:

0b00000000-0b00001100

General-purpose registers R0-R12.

0b00001101

Current stack pointer, SP.

0b00001110

LR.

0b00001111

DebugReturnAddress.

0b00010000

XPSR / EAPSR, on a read accesses XPSR, otherwise EAPSR is accessed.

0b00010001

Current state main stack pointer, SP_main.

Accessible only when privileged debug is permitted for the current state.

0b00010010

Current state process stack pointer, SP_process.

0b00010100

Current state. The value corresponds to {CONTROL[7:0], FAULTMASK[7:0], BASEPRI[7:0], PRIMASK[7:0]}.

All bits are accessible except when one or more of the following restrictions apply.

If the PE is in Non-secure state the bit corresponding to CONTROL.SFPA is RAZ/WI.

When only unprivileged debug is permitted for the current Security state, the bits corresponding to the following fields are [RES0](#):

- All fields corresponding to the Control register except CONTROL.FPCA and CONTROL.SFPA.
- FAULTMASK[7:0].
- BASEPRI[7:0].
- PRIMASK[7:0].

If the Main Extension is not implemented FAULTMASK[7:0] and BASEPRI[7:0] are [RES0](#).

0b00011000

Non-secure main stack pointer, MSP_NS.

If the Security Extension is not implemented, this value is reserved.

Accessible only when DHCSR.S_NSUIDE == 0.

0b00011001

Non-secure process stack pointer, PSP_NS.

If the Security Extension is not implemented, this value is reserved.

0b00011010

Secure main stack pointer, MSP_S.

If the Security Extension is not implemented, this value is reserved.

Accessible only when DHCSR.S_SDE == 1 and DHCSR.S_SUIDE == 0.

0b00011011

Secure process stack pointer, PSP_S.

If the Security Extension is not implemented, this value is reserved.

Accessible only when `DHCSR.S_SDE == 1`.

0b00011100

Secure main stack limit, `MSPLIM_S`.

If the Security Extension is not implemented, this value is reserved.

Accessible only when `DHCSR.S_SDE == 1` and `DHCSR.S_SUIDE == 0`.

0b00011101

Secure process stack limit, `PSPLIM_S`.

If the Security Extension is not implemented, this value is reserved.

Accessible only when `DHCSR.S_SDE == 1`.

0b00011110

Non-secure main stack limit, `MSPLIM_NS`.

If the Main Extension is not implemented, this value is reserved.

Accessible only when `DHCSR.S_NSUIDE == 0`.

0b00011111

Non-secure process stack limit, `PSPLIM_NS`.

If the Main Extension is not implemented, this value is reserved.

0b00100001

`FPSCR`.

If MVE and the Floating-point Extension are not implemented, this value is reserved.

0b00100010

Secure state. The value corresponds to `{CONTROL_S[7:0], FAULTMASK_S[7:0], BASEPRI_S[7:0], PRIMASK_S[7:0]}`.

All bits are **RES0** when `DHCSR.S_SDE == 0`.

All bits are accessible when `DHCSR.S_SDE == 1` and `DHCSR.S_SUIDE == 0`.

When `DHCSR.S_SDE == 1` and `DHCSR.S_SUIDE == 1` the bits corresponding to the following fields are **RES0**:

- All fields corresponding to the `CONTROL_S` register except for `CONTROL_S.SFPA` and `CONTROL_S.FPCA`.
- `FAULTMASK_S[7:0]`.
- `BASEPRI_S[7:0]`.
- `PRIMASK_S[7:0]`.

If the Main Extension is not implemented, `FAULTMASK_S[7:0]` and `BASEPRI_S[7:0]` are **RES0**.

If the Security Extension is not implemented, this value is reserved.

0b00100011

Non-secure state. The value corresponds to `{CONTROL_NS[7:0], FAULTMASK_NS[7:0], BASEPRI_NS[7:0], PRIMASK_NS[7:0]}`. All bits are accessible when `DHCSR.S_NSUIDE == 0`.

When `DHCSR.S_NSUIDE == 1` the bits corresponding to following fields are **RES0**:

- All fields corresponding to the `CONTROL_NS` register except for `CONTROL_NS.FPCA`.
- `FAULTMASK_NS[7:0]`.
- `BASEPRI_NS[7:0]`.

- PRIMASK_NS[7:0].

If the Main Extension is not implemented, FAULTMASK_NS[7:0] and BASEPRI_NS[7:0] are [RES0](#).

If the Security Extension is not implemented, this value is reserved.

0b00100100

VPR.

If MVE is not implemented, this value is reserved.

0b01000000-0b01011111

Floating-point registers, S0-S31.

If MVE and the Floating-point Extension are not implemented, these values are reserved.

0b10000000-0b10000011

Current state PAC_KEY_P, word 0 to 3.

If the PACBTI Extension is not implemented, this value is reserved.

Accessible only when privileged debug is permitted for the current state.

0b10000100-0b10000111

Current state PAC_KEY_U, word 0 to 3.

If the PACBTI Extension is not implemented, this value is reserved.

Accessible only when privileged debug is permitted for the current state.

0b10001000-0b10001011

Secure PAC_KEY_P_S, word 0 to 3.

Accessible only when DHCSR.S_SDE == 1.

If the PACBTI or Security Extensions are not implemented, this value is reserved.

Accessible only when privileged debug is permitted for the Secure state.

0b10001100-0b10001111

Secure PAC_KEY_U_S, word 0 to 3.

Accessible only when DHCSR.S_SDE == 1.

If the PACBTI or Security Extensions are not implemented, this value is reserved.

Accessible only when privileged debug is permitted for the Secure state.

0b10010000-0b10010011

Non-secure PAC_KEY_P_NS, word 0 to 3.

If the PACBTI or Security Extensions are not implemented, this value is reserved.

Accessible only when privileged debug is permitted for the Non-secure state.

0b10010100-0b10010111

Non-secure PAC_KEY_U_NS, word 0 to 3.

If the PACBTI or Security Extensions are not implemented, this value is reserved.

Accessible only when privileged debug is permitted for the Non-secure state.

All other values are reserved.

If the Security Extension and either MVE or the Floating-point Extension are implemented, then FPSCR, VPR and S0-S31 are not accessible:

- From Non-secure state if DHCSR.S_SDE == 0 and FPCCR indicates the registers contain values from Secure state.

- From Non-secure state if DHCSR.S_SDE == 0 and NSACR prevents Non-secure access to the registers.
- FPCCR.LSPACT, the banked variant associated with the Floating-point context as indicated by FPCCR.S.S, is set and only unprivileged debug is permitted for the security state associated with the Floating-point state (as indicated by FPCCR.S).

Registers that are not accessible are RAZ/WI.

If this field is written with a reserved value, the PE might behave as if a defined value was written, or ignore the value written, and the value of DCRDR becomes **UNKNOWN**.

D1.2.35 DDEVARCH, SCS Device Architecture Register

The DDEVARCH characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

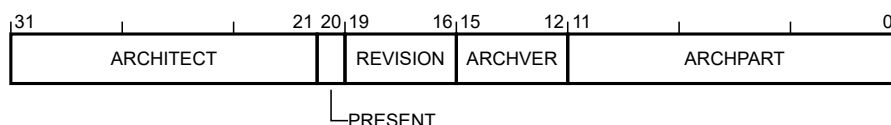
32-bit read-only register located at 0xE000EFBC.

Secure software can access the Non-secure version of this register via DDEVARCH_NS located at 0xE002EFBC. The location 0xE002EFBC is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DDEVARCH bit assignments are:



ARCHITECT, bits [31:21]

Architect. Defines the architect of the component. Bits [31:28] are the JEP106 continuation code (JEP106 bank ID, minus 1) and bits [27:21] are the JEP106 ID code.

The possible values of this field are:

0x23B

JEP106 continuation code 0x4, ID code 0x3B. Arm Limited.

Other values are defined by the JEDEC JEP106 standard.

This field reads as 0x23B.

PRESENT, bit [20]

DEVARCH Present. Defines that the DEVARCH register is present.

The possible values of this bit are:

1

DEVARCH information present.

This bit reads as one.

REVISION, bits [19:16]

Revision. Defines the architecture revision of the component.

The possible values of this field are:

0b0000

M-Profile debug architecture v3.0.

This field reads as 0b0000.

ARCHVER, bits [15:12]

Architecture Version. Defines the architecture version of the component.

The possible values of this field are:

0b0010

M-Profile debug architecture v3.

ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHVER is ARCHID[15:12].

This field reads as 0b0010.

ARCHPART, bits [11:0]

Architecture Part. Defines the architecture of the component.

The possible values of this field are:

0xA04

M-Profile debug architecture.

ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHPART is ARCHID[11:0].

This field reads as 0xA04.

D1.2.36 DDEVTYPE, SCS Device Type Register

The DDEVTYPE characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

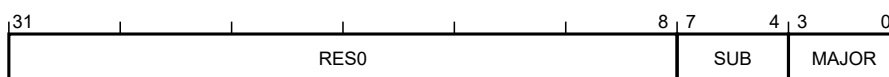
32-bit read-only register located at 0xE000EFCC.

Secure software can access the Non-secure version of this register via DDEVTYPE_NS located at 0xE002EFCC. The location 0xE002EFCC is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DDEVTYPE bit assignments are:



Bits [31:8]

Reserved, **RES0**.

SUB, bits [7:4]

Sub-type. Component sub-type.

The possible values of this field are:

0x0

Other.

This field reads as 0b0000.

MAJOR, bits [3:0]

Major type. CoreSight major type.

The possible values of this field are:

0x0

Miscellaneous.

This field reads as 0b0000.

D1.2.37 DEMCR, Debug Exception and Monitor Control Register

The DEMCR characteristics are:

Purpose

Manages vector catch behavior and DebugMonitor handling when debugging.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If Armv8.1-M is implemented, this register is word, halfword and byte accessible.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present if Halting debug or the Main Extension is implemented.

This register is **RES0** if both Halting debug and Main Extension are not implemented.

Attributes

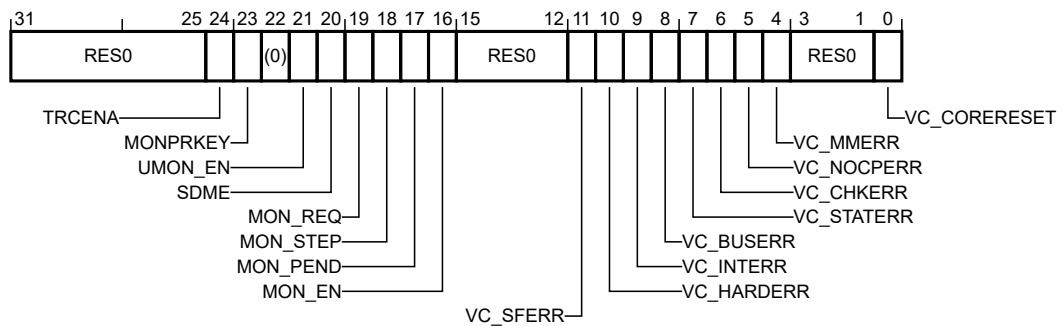
32-bit read/write register located at 0xE000EDFC.

Secure software can access the Non-secure version of this register via DEMCR_NS located at 0xE002EDFC. The location 0xE002EDFC is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DEMCR bit assignments are:



Bits [31:25]

Reserved, **RES0**.

TRCENA, bit [24]

Trace enable. Global enable for all DWT, PMU, and ITM features.

The possible values of this bit are:

0

DWT, PMU, and ITM features disabled.

1

DWT, PMU, and ITM features enabled.

If the DWT, PMU, and ITM units are not implemented, this bit is [RES0](#). See the descriptions of DWT, PMU, and ITM for details of which features this bit controls.

Setting this bit to 0 might not stop all events. To ensure that all events are stopped, software must set all DWT, PMU, and ITM feature enable bits to 0, and ensure that all trace generated by the DWT, PMU, and ITM has been flushed, before setting this bit to 0.

It is [IMPLEMENTATION DEFINED](#) whether this bit affects how the system processes trace.

Arm recommends that this bit is set to 1 when using an ETM even if any implemented DWT, PMU, and ITM are not being used.

This bit resets to zero on a Cold reset.

MONPRKEY, bit [23]

Monitor pend req key. Writes to the MON_PEND and MON_REQ fields are ignored unless MONPRKEY is concurrently written to zero.

The possible values of this bit are:

0

Concurrent write to MON_PEND and MON_REQ are not ignored.

1

Concurrent write to MON_PEND and MON_REQ are ignored.

This bit reads-as-zero.

If version Armv8.1-M of the architecture is not implemented, this bit is [RES0](#).

Bit [22]

Reserved, [RES0](#).

UMON_EN, bit [21]

Unprivileged monitor enable. DebugMonitor pend enable when the PE is in an unprivileged mode.

The possible values of this bit are:

0

DebugMonitor exception controlled by DEMCR.MON_EN.

1

DebugMonitor exception can be pended for unprivileged execution.

Writes to this field by unprivileged DAP accesses are ignored.

If DEMCR.SDME is one this bit is RAZ/WI from Non-secure state

If version Armv8.1 of the architecture and UDE are not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

SDME, bit [20]

Secure DebugMonitor enable. Indicates whether the DebugMonitor targets the Secure or the Non-secure state and whether debug events are allowed in Secure state.

The possible values of this bit are:

0

Debug events prohibited in Secure state and the DebugMonitor exception targets Non-secure state.

1

Debug events allowed in Secure state and the DebugMonitor exception targets Secure state.

When DebugMonitor exception is not pending or active, this bit reflects the value of SecureDebugMonitorAllowed(), otherwise, the previous value is retained.

This bit is read-only.

If the Security Extension is not implemented, this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

MON_REQ, bit [19]

Monitor request. DebugMonitor semaphore bit.

The PE does not use this bit. The monitor software defines the meaning and use of this bit.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

MON_STEP, bit [18]

Monitor step. Enable DebugMonitor exception stepping.

The possible values of this bit are:

0

Stepping disabled.

1

Stepping enabled.

The effect of changing this bit at an execution priority that is lower than the priority of the DebugMonitor exception is [UNPREDICTABLE](#).

The effect of a write to DEMCR.MON_STEP is [CONSTRAINED UNPREDICTABLE](#) unless one of:

- The PE is executing at a priority that is greater than or the same as the priority of the DebugMonitor exception.
- Before the write, DEMCR.MON_STEP == 0 and the write writes 0 to DEMCR.MON_STEP.

The [CONSTRAINED UNPREDICTABLE](#) write does one of the following:

- The write is ignored.
- DEMCR.MON_STEP is set to 1. The effect of the subsequent instruction is [UNPREDICTABLE](#), and the PE might generate a DebugMonitor exception at any time, if enabled and permitted.
- DEMCR.MON_STEP is set to 0. It is [UNPREDICTABLE](#) whether an 'in progress' step completes.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

MON_PEND, bit [17]

Monitor pend. Sets or clears the pending state of the DebugMonitor exception.

The possible values of this bit are:

0

Clear the status of the DebugMonitor exception to not pending.

1

Set the status of the DebugMonitor exception to pending.

When the DebugMonitor exception is pending it becomes active subject to the exception priority rules. The effect of setting this bit to 1 is not affected by the value of the MON_EN and UMON_EN bits. This means that software or a debugger can set MON_PEND to 1 and pend a DebugMonitor exception, even when MON_EN and UMON_EN are set to 0.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

MON_EN, bit [16]

Monitor enable. Enable the DebugMonitor exception.

The possible values of this bit are:

0

DebugMonitor exception disabled.

1

DebugMonitor exception enabled.

If a debug event halts the PE, the PE ignores the value of this bit. Unprivileged writes to this bit from the DAP are ignored.

If DEMCR.SDME is one this bit is RAZ/WI from Non-secure state

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

Bits [15:12]

Reserved, [RES0](#).

VC_SFERR, bit [11]

Vector Catch SecureFault. SecureFault exception Halting debug vector catch enable.

The possible values of this bit are:

0

Halting debug trap on SecureFault disabled.

1

Halting debug trap on SecureFault enabled.

The PE ignores the value of this bit if any of the following are true:

- DHCSR.C_DEBUGEN == 0.
- DHCSR.S_NSUIDE == 1.
- HaltingDebugAllowed() == FALSE.
- DHCSR.S_SDE == 0.
- DHCSR.S_SUIDE == 1.

If the Security Extension is not implemented, this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

VC_HARDERR, bit [10]

Vector Catch HardFault errors. HardFault exception Halting debug vector catch enable.

The possible values of this bit are:

0

Halting debug trap on HardFault disabled.

1

Halting debug trap on HardFault enabled.

The PE ignores the value of this bit if any of the following are true:

- DHCSR.C_DEBUGEN == 0.

- `DHCSR.S_NSUIDE == 1`.
- `HaltingDebugAllowed() == FALSE`.
- The Security Extension is implemented, either `DHCSR.S_SDE == 0` or `DHCSR.S_SUIDE == 1`, and the exception targets Secure state.

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

VC_INTERR, bit [9]

Vector Catch interrupt errors. Enable Halting debug vector catch for faults arising from lazy Floating-point state preservation, stack violations and context stacking or unstacking during exception entry or return.

The possible values of this bit are:

0

Halting debug trap on faults disabled.

1

Halting debug trap on faults enabled.

The PE ignores the value of this bit if any of the following are true:

- `DHCSR.C_DEBUGEN == 0`.
- `DHCSR.S_NSUIDE == 1`.
- `HaltingDebugAllowed() == FALSE`.
- The Security Extension is implemented, `DHCSR.S_SDE == 0` or `DHCSR.S_SUIDE == 1`, and the exception targets Secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

VC_BUSERR, bit [8]

Vector Catch BusFault errors. BusFault exception Halting debug vector catch enable.

The possible values of this bit are:

0

Halting debug trap on BusFault disabled.

1

Halting debug trap on BusFault enabled.

The PE ignores the value of this bit if any of the following are true:

- `DHCSR.C_DEBUGEN == 0`.
- `DHCSR.S_NSUIDE == 1`.
- `HaltingDebugAllowed() == FALSE`.
- The Security Extension is implemented, `DHCSR.S_SDE == 0` or `DHCSR.S_SUIDE == 1`, and the exception targets Secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

VC_STATERR, bit [7]

Vector Catch state errors. Enable Halting debug trap on a UsageFault exception caused by a state information error, for example an Undefined Instruction exception.

The possible values of this bit are:

0

Halting debug trap on UsageFault caused by state information error disabled.

1

Halting debug trap on UsageFault caused by state information error enabled.

The PE ignores the value of this bit if any of the following are true:

- DHCSR.C_DEBUGEN == 0.
- DHCSR.S_NSUIDE == 1.
- HaltingDebugAllowed() == FALSE.
- The Security Extension is implemented, DHCSR.S_SDE == 0 or DHCSR.S_SUIDE == 1, and the exception targets Secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

VC_CHKERR, bit [6]

Vector Catch check errors. Enable Halting debug trap on a UsageFault exception caused by an alignment check error or divide-by-zero trap.

The possible values of this bit are:

0

Halting debug trap on UsageFault caused by checking error disabled.

1

Halting debug trap on UsageFault caused by checking error enabled.

The PE ignores the value of this bit if any of the following are true:

- DHCSR.C_DEBUGEN == 0.
- DHCSR.S_NSUIDE == 1.
- HaltingDebugAllowed() == FALSE.
- The Security Extension is implemented, DHCSR.S_SDE == 0 or DHCSR.S_SUIDE == 1, and the exception targets Secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

VC_NOCPPERR, bit [5]

Vector Catch NOCP errors. Enable Halting debug trap on a UsageFault caused by an access to a coprocessor.

The possible values of this bit are:

0

Halting debug trap on UsageFault caused by access to a coprocessor disabled.

1

Halting debug trap on UsageFault caused by access to a coprocessor enabled.

The PE ignores the value of this bit if any of the following are true:

- `DHCSR.C_DEBUGEN == 0`.
- `DHCSR.S_NSUIDE == 1`.
- `HaltingDebugAllowed() == FALSE`.
- The Security Extension is implemented, `DHCSR.S_SDE == 0` or `DHCSR.S_SUIDE == 1`, and the exception targets Secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

VC_MMERR, bit [4]

Vector Catch MemManage errors. Enable Halting debug trap on a MemManage exception.

The possible values of this bit are:

- 0**
Halting debug trap on MemManage disabled.
- 1**
Halting debug trap on MemManage enabled.

The PE ignores the value of this bit if any of the following are true:

- `DHCSR.C_DEBUGEN == 0`.
- `DHCSR.S_NSUIDE == 1`.
- `HaltingDebugAllowed() == FALSE`.
- The Security Extension is implemented, `DHCSR.S_SDE == 0` or `DHCSR.S_SUIDE == 1`, and the exception targets Secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

Bits [3:1]

Reserved, [RES0](#).

VC_CORERESET, bit [0]

Vector Catch Core reset. Enable Reset Vector Catch. This causes a Warm reset to halt a running system.

The possible values of this bit are:

- 0**
Halting debug trap on reset disabled.
- 1**
Halting debug trap on reset enabled.

If `DHCSR.C_DEBUGEN == 0`, the PE ignores the value of this bit. Otherwise, when this bit is set to 1 a Warm reset will pend a Vector Catch debug event. The debug event is pended regardless of debug permissions or the security state of the PE, and the PE will halt when it enters a mode or state where debug is enabled.

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

D1.2.38 DFSR, Debug Fault Status Register

The DFSSR characteristics are:

Purpose

Shows which debug event occurred.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present if Halting debug or the Main Extension is implemented.

This register is **RES0** if both Halting debug and Main Extension are not implemented.

Attributes

32-bit read/write-one-to-clear register located at 0xE00ED30.

Secure software can access the Non-secure version of this register via DFSR_NS located at 0xE002ED30. The location 0xE002ED30 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DFSSR bit assignments are:

**Bits [31:6]**

Reserved, RES0.

PMU, bit [5]

PMU event. Sticky flag indicating whether a PMU counter overflow event has occurred.

The possible values of this bit are:

0

PMU event has not occurred.

1

PMU event has occurred.

If version Armv8.1 of the architecture and PMU are not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

EXTERNAL, bit [4]

External event. Sticky flag indicating whether an External debug request debug event has occurred.

The possible values of this bit are:

0

Debug event has not occurred.

1

Debug event has occurred.

This bit resets to zero on a Cold reset.

VCATCH, bit [3]

Vector Catch event. Sticky flag indicating whether a Vector catch debug event has occurred.

The possible values of this bit are:

0

Debug event has not occurred.

1

Debug event has occurred.

If Halting debug is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

DWTTRAP, bit [2]

Watchpoint event. Sticky flag indicating whether a Watchpoint debug event has occurred.

The possible values of this bit are:

0

Debug event has not occurred.

1

Debug event has occurred.

If the DWT is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

BKPT, bit [1]

Breakpoint event. Sticky flag indicating whether a Breakpoint debug event has occurred.

The possible values of this bit are:

0

Debug event has not occurred.

1

Debug event has occurred.

This bit resets to zero on a Cold reset.

HALTED, bit [0]

Halt or step event. Sticky flag indicating that a Halt request debug event or Step debug event has occurred.

The possible values of this bit are:

0

Debug event has not occurred.

1

Debug event has occurred.

This bit resets to zero on a Cold reset.

D1.2.39 DHCSR, Debug Halting Control and Status Register

The DHCSR characteristics are:

Purpose

Controls Halting debug.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

It is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software.

Configurations

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

32-bit read/write register located at 0xE000EDF0.

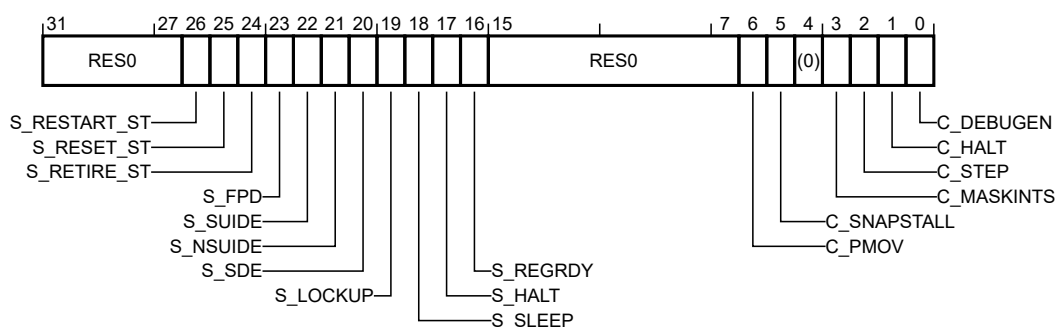
Secure software can access the Non-secure version of this register via DHCSR_NS located at 0xE002EDF0. The location 0xE002EDF0 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

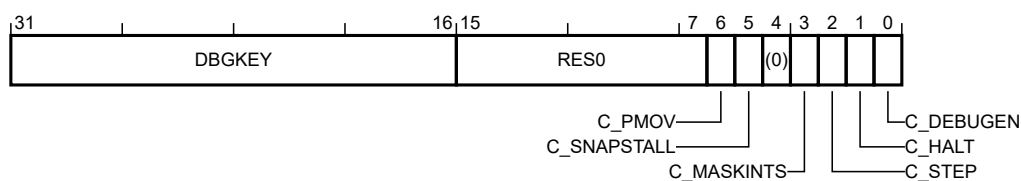
Field descriptions

The DHCSR bit assignments are:

On a read:



On a write:



DBGKEY, bits [31:16], on a write

Debug key. A debugger must write 0xA05F to this field to enable write access to the remaining bits, otherwise the PE ignores the write access.

The possible values of this field are:

0xA05F

Writes accompanied by this value update bits[15:0].

Not 0xA05F

Write ignored.

Bits [31:27], on a read

Reserved, [RES0](#).

S_RESTART_ST, bit [26], on a read

Restart sticky status. Indicates the PE has processed a request to clear DHCSR.C_HALT to 0. That is, either a write to DHCSR that clears DHCSR.C_HALT from 1 to 0, or an External Restart Request.

The possible values of this bit are:

0

PE has not left Debug state since the last read of DHCSR.

1

PE has left Debug state since the last read of DHCSR.

If the PE is not halted when C_HALT is cleared to zero, it is [UNPREDICTABLE](#) whether this bit is set to 1. If DHCSR.C_DEBUGEN == 0 this bit reads as an [UNKNOWN](#) value.

This bit clears to zero when read.

Note

If the request to clear C_HALT is made simultaneously with a request to set C_HALT, for example a restart request and external debug request occur together, then the PE notionally leaves Debug state and immediately halts again and S_RESTART_ST is set to 1.

S_RESET_ST, bit [25], on a read

Reset sticky status. Indicates whether the PE has been reset since the last read of the DHCSR.

The possible values of this bit are:

0

No reset since last DHCSR read.

1

At least one reset since last DHCSR read.

This bit clears to zero when read.

This bit resets to one on a Warm reset.

S_RETIRE_ST, bit [24], on a read

Retire sticky status. Set to 1 every time the PE retires one or more instructions.

The possible values of this bit are:

0

No instruction retired since last DHCSR read.

1

At least one instruction retired since last DHCSR read.

This bit clears to zero when read.

This bit resets to an [UNKNOWN](#) value on a Warm reset.

S_FPD, bit [23], on a read

Floating-point registers Debuggable. Indicates that FPSCR, VPR, and the Floating-point registers are RAZ/WI in the current PE state when accessed via DCSR. This reflects !CanDebugAccessFP().

The possible values of this bit are:

0

Floating-point registers accessible.

1

Floating-point registers are RAZ/WI.

If version Armv8.1-M of the architecture is not implemented, this bit is [RES0](#).

S_SUIDE, bit [22], on a read

Secure unprivileged halting debug enabled. Indicates whether Secure unprivileged-only halting debug is allowed or active.

The possible values of this bit are:

0

Secure invasive halting debug prohibited or not restricted to an unprivileged mode.

1

Unprivileged Secure invasive halting debug enabled.

If the PE is in Non-debug state, this bit reflects the value of `UnprivHaltingDebugAllowed(TRUE) && !SecureHaltingDebugAllowed()`.

The value of this bit does not change whilst the PE remains in Debug state.

If the Security Extension is not implemented, this bit is [RES0](#).

If version Armv8.1 of the architecture and UDE are not implemented, this bit is [RES0](#).

S_NSUIDE, bit [21], on a read

Non-secure unprivileged halting debug enabled. Indicates whether Non-secure unprivileged-only halting debug is allowed or active.

The possible values of this bit are:

0

Non-secure invasive halting debug prohibited or not restricted to an unprivileged mode.

1

Unprivileged Non-secure invasive halting debug enabled.

If the PE is in Non-debug state, this bit reflects the value of `UnprivHaltingDebugAllowed(FALSE) && !HaltingDebugAllowed()`.

The value of this bit does not change whilst the PE remains in Debug state.

If version Armv8.1 of the architecture and UDE are not implemented, this bit is [RES0](#).

S_SDE, bit [20], on a read

Secure debug enabled. Indicates whether Secure invasive debug is allowed.

The possible values of this bit are:

0

Secure invasive debug prohibited.

1

Secure invasive debug allowed.

If the PE is in Non-debug state, this bit reflects the value of `SecureHaltingDebugAllowed()` or `UnprivHaltingDebugAllowed(TRUE)`.

The value of this bit does not change while the PE remains in Debug state.

If the Security Extension is not implemented, this bit is [RES0](#).

S_LOCKUP, bit [19], on a read

Lockup status. Indicates whether the PE is in Lockup state.

The possible values of this bit are:

- 0**
Not locked up.
- 1**
Locked up.

This bit can only be read as 1 by a remote debugger, using the DAP. The value of 1 indicates that the PE is running but locked up. The bit clears to 0 when the PE enters Debug state.

S_SLEEP, bit [18], on a read

Sleeping status. Indicates whether the PE is sleeping.

The possible values of this bit are:

- 0**
Not sleeping.
- 1**
Sleeping.

The debugger must set the C_HALT bit to 1 to gain control, or wait for an interrupt or other wakeup event to wakeup the system.

S_HALT, bit [17], on a read

Halted status. Indicates whether the PE is in Debug state.

The possible values of this bit are:

- 0**
In Non-debug state.
- 1**
In Debug state.

S_REGRDY, bit [16], on a read

Register ready status. Handshake flag to transfers through the DCRDR.

The possible values of this bit are:

- 0**
Write to DCRSR performed, but transfer not yet complete.
- 1**
Transfer complete, or no outstanding transfer.

This bit is valid only when the PE is in Debug state, otherwise this bit is **UNKNOWN**.

This bit resets to an **UNKNOWN** value on a Warm reset.

Bits [15:7]

Reserved, **RES0**.

C_PMOV, bit [6]

Halt on PMU overflow control. Request entry to Debug state when a PMU counter overflows.

The possible values of this bit are:

- 0**
No action.

1

If C_DEBUGEN is set to 1, then when a PMU counter is configured to generate an interrupt overflows, the PE sets DHCSR.C_HALT to 1 and DFSR.PMU to 1.

PMU_OVSSET and PMU_OVSCLR indicate which counter or counters triggered the halt.

If the Main Extension is not implemented, this bit is [RES0](#).

If version Armv8.1 of the architecture and PMU are not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

C_SNAPSTALL, bit [5]

Snap stall control. Allow imprecise entry to Debug state.

The possible values of this bit are:

0

No action.

1

Allows imprecise entry to Debug state, for example by forcing any stalled load or store instruction to be abandoned.

Setting this bit to 1 allows a debugger to request an imprecise entry to Debug state. Writing 1 to this bit makes the state of the memory system [UNPREDICTABLE](#). Therefore if a debugger writes 1 to this bit it must reset the system before leaving Debug state.

The effect of setting this bit to 1 is [UNPREDICTABLE](#) unless the DHCSR write also sets C_DEBUGEN and C_HALT to 1. This means that if the PE is not already in Debug state, it enters Debug state when the stalled instruction completes.

If the Security Extension is implemented, then writes to this bit are ignored when DHCSR.S_SDE == 0 or DHCSR.S_SUIDE == 1.

If DHCSR.C_DEBUGEN == 0, HaltingDebugAllowed() == FALSE, or DHCSR.S_NSUIDE == 1, the PE ignores this bit and behaves as if it is set to 0.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

Note

A debugger can write to the DHCSR to clear this bit to 0. However, this does not remove the [UNPREDICTABLE](#) state of the memory system caused by setting C_SNAPSTALL to 1. The architecture does not guarantee that setting this bit to 1 will force an entry to Debug State. Arm strongly recommends that a value of 1 is never written to C_SNAPSTALL when the PE is in Debug state. It is [IMPLEMENTATION DEFINED](#) whether this bit behaves as RAZ/WI.

Bit [4]

Reserved, [RES0](#).

C_MASKINTS, bit [3]

Mask interrupts control. When debug is enabled, the debugger can write to this bit to mask PendSV, SysTick and external interrupts.

The possible values of this bit are:

0

Do not mask.

1

Mask PendSV, SysTick and external interrupts.

The effect of any single write to DHCSR that changes the value of this bit is [UNPREDICTABLE](#) unless one of:

- Before the write, DHCSR.{S_HALT, C_HALT} are both set to 1 and the write also writes 1 to DHCSR.C_HALT.
- Before the write, DHCSR.C_DEBUGEN == 0 or HaltingDebugAllowed() == FALSE, and the write writes 0 to DHCSR.C_MASKINTS.

This means that a single write to DHCSR must not clear DHCSR.C_HALT to 0 and change the value of the C_MASKINTS bit.

If the Security Extension is implemented and either DHCSR.S_SDE == 0 or DHCSR.S_SUIDE == 1, this bit does not affect interrupts targeting Secure state.

If DHCSR.C_DEBUGEN == 0, HaltingDebugAllowed() == FALSE, or DHCSR.S_NSUIDE == 1, the PE ignores this bit and behaves as if it is set to 0.

If DHCSR.C_DEBUGEN == 0 this bit reads as an UNKNOWN value.

This bit resets to an UNKNOWN value on a Cold reset.

Note

This bit does not affect NMI.

C_STEP, bit [2]

Step control. Enable single instruction step.

The possible values of this bit are:

0

No effect.

1

Single step enabled.

The effect of a single write to DHCSR that changes the value of this bit is UNPREDICTABLE unless one of:

- Before the write, DHCSR.{S_HALT, C_HALT} are both set to 1.
- Before the write, DHCSR.C_DEBUGEN == 0 or HaltingDebugAllowed() == FALSE or DHCSR.C_STEP == 0, and the write writes 0 to DHCSR.C_STEP.

The PE ignores this bit and behaves as if it set to 0 if any of:

- UnprivHaltingDebugAllowed(IsSecure()) == FALSE and either DHCSR.C_DEBUGEN == 0 or HaltingDebugAllowed() == FALSE.
- The Security Extension is implemented, DHCSR.S_SDE == 0 and the PE is in Secure state.

If DHCSR.C_DEBUGEN == 0 this bit reads as an UNKNOWN value.

This bit resets to an UNKNOWN value on a Cold reset.

C_HALT, bit [1]

Halt control. PE to enter Debug state halt request.

The possible values of this bit are:

0

Causes the PE to leave Debug state, if the PE is in Debug state.

1

Halt the PE.

The PE sets C_HALT to 1 when a debug event pends an entry to Debug state.

The PE ignores this bit and behaves as if it is set to 0 if any of:

- `UnprivHaltingDebugAllowed(IsSecure()) == FALSE` and either `DHCSR.C_DEBUGEN == 0` or `HaltingDebugAllowed() == FALSE`.
- The Security Extension is implemented, `DHCSR.S_SDE == 0` and the PE is in Secure state.

If `DHCSR.C_DEBUGEN == 0` this bit reads as an **UNKNOWN** value.

This bit resets to zero on a Warm reset.

C_DEBUGEN, bit [0]

Debug enable control. Enable Halting debug.

The possible values of this bit are:

0

Disabled.

1

Enabled.

If a debugger writes to `DHCSR` to change the value of this bit from 0 to 1, it must also write 0 to the `C_MASKINTS` bit, otherwise behavior is **UNPREDICTABLE**.

If this bit is set to 0:

- The PE behaves as if `DHCSR.{C_MASKINTS, C_STEP, C_HALT}` are all set to 0.
- `DHCSR.{S_RESTART_ST, C_MASKINTS, C_STEP, C_HALT}` are **UNKNOWN** on reads of `DHCSR`.

This bit is read/write to the debugger. Writes from software are ignored.

This bit resets to zero on a Cold reset.

D1.2.40 DLAR, SCS Software Lock Access Register

The DLAR characteristics are:

Purpose

Provides CoreSight Software Lock control for the SCS, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank). It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

Attributes

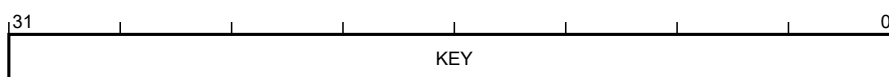
32-bit write-only register located at 0xE000EFB0.

Secure software can access the Non-secure version of this register via DLAR_NS located at 0xE002EFB0. The location 0xE002EFB0 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DLAR bit assignments are:



KEY, bits [31:0]

Lock Access control.

Writing the key value 0xC5ACCE55 to this field unlocks the lock, enabling write accesses to the registers of this component through a memory-mapped interface.

Writing any other value to this register locks the lock, disabling write accesses to the registers of this component through a memory mapped interface.

D1.2.41 DLSR, SCS Software Lock Status Register

The DLSR characteristics are:

Purpose

Provides CoreSight Software Lock status information for the SCS, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank). It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

Attributes

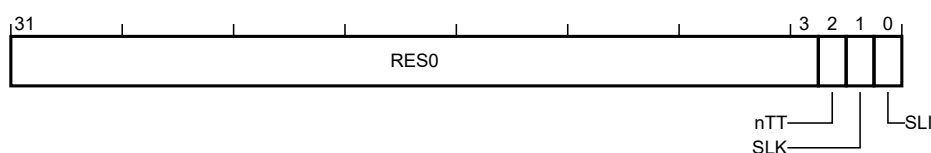
32-bit read-only register located at 0xE000EFB4.

Secure software can access the Non-secure version of this register via DLSR_NS located at 0xE002EFB4. The location 0xE002EFB4 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DLSR bit assignments are:



Bits [31:3]

Reserved, **RES0**.

nTT, bit [2]

Not thirty-two bit. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as zero.

SLK, bit [1]

Software Lock status. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

0

Lock clear. Software writes are permitted to the registers of the component.

1

Lock set. Software writes to the registers of this component are ignored, and reads have no side effects.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is [RES0](#).

This bit resets to one on a Warm reset.

SLI, bit [0]

Software Lock implemented. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

0

Software Lock not implemented or debugger access.

1

Software Lock is implemented and software access.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is RAZ.

This bit reads as an [IMPLEMENTATION DEFINED](#) value.

D1.2.42 DPIDR0, SCS Peripheral Identification Register 0

The DPIDR0 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

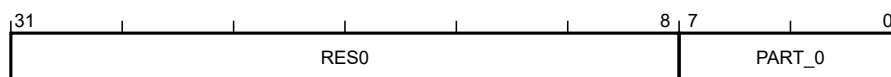
32-bit read-only register located at 0xE000EFE0.

Secure software can access the Non-secure version of this register via DPIDR0_NS located at 0xE002EFE0. The location 0xE002EFE0 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DPIDR0 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

PART_0, bits [7:0]

Part number bits [7:0]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.44 DPIDR2, SCS Peripheral Identification Register 2

The DPIDR2 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is [IMPLEMENTATION DEFINED](#) whether this register is accessible only to the debugger and [RES0](#) for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is [RES0](#) if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is [RES0](#) if Halting debug is not implemented.

Attributes

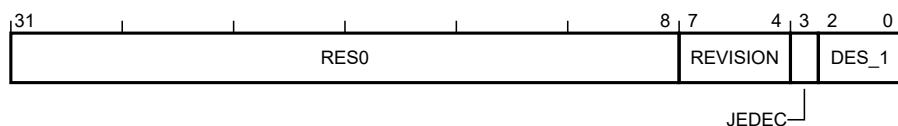
32-bit read-only register located at 0xE000EFE8.

Secure software can access the Non-secure version of this register via DPIDR2_NS located at 0xE002EFE8. The location 0xE002EFE8 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DPIDR2 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

REVISION, bits [7:4]

Component revision. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

JEDEC, bit [3]

JEDEC assignee value is used. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as one.

DES_1, bits [2:0]

JEP106 identification code bits [6:4]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

D1.2.45 DPIDR3, SCS Peripheral Identification Register 3

The DPIDR3 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

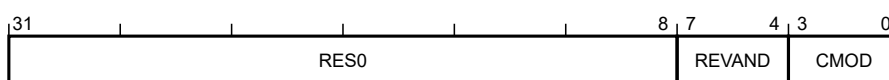
32-bit read-only register located at 0xE000EFEC.

Secure software can access the Non-secure version of this register via DPIDR3_NS located at 0xE002EFEC. The location 0xE002EFEC is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DPIDR3 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REVAND, bits [7:4]

RevAnd. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

CMOD, bits [3:0]

Customer Modified. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.46 DPIDR4, SCS Peripheral Identification Register 4

The DPIDR4 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is [RES0](#) if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

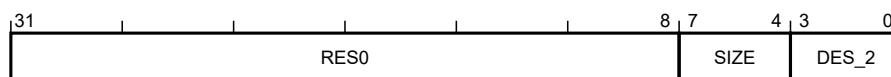
32-bit read-only register located at 0xE000EFD0.

Secure software can access the Non-secure version of this register via DPIDR4_NS located at 0xE002EFD0. The location 0xE002EFD0 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DPIDR4 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

SIZE, bits [7:4]

4KB count. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as zero.

DES 2, bits [3:0]

JEP106 continuation code. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.47 DPIDR5, SCS Peripheral Identification Register 5

The DPIDR5 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

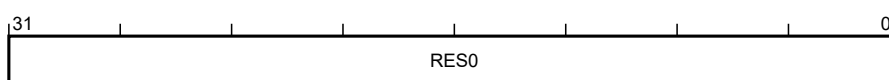
32-bit read-only register located at 0xE000EFD4.

Secure software can access the Non-secure version of this register via DPIDR5_NS located at 0xE002EFD4. The location 0xE002EFD4 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DPIDR5 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.48 DPIDR6, SCS Peripheral Identification Register 6

The DPIDR6 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

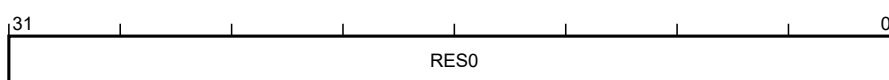
32-bit read-only register located at 0xE000EFD8.

Secure software can access the Non-secure version of this register via DPIDR6_NS located at 0xE002EFD8. The location 0xE002EFD8 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DPIDR6 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.49 DPIDR7, SCS Peripheral Identification Register 7

The DPIDR7 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the SCS.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

Attributes

32-bit read-only register located at 0xE000EFDC.

Secure software can access the Non-secure version of this register via DPIDR7_NS located at 0xE002EFDC. The location 0xE002EFDC is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DPIDR7 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.50 DSCEMCR, Debug Set Clear Exception and Monitor Control Register

The DSCEMCR characteristics are:

Purpose

Atomically sets or clears selected fields in the DEMCR register.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present if Halting debug or the Main Extension is implemented.

This register is [RES0](#) if both Halting debug and Main Extension are not implemented.

Present only if version Armv8.1-M of the architecture is implemented.

This register is [RES0](#) if Armv8.1-M is not implemented.

Attributes

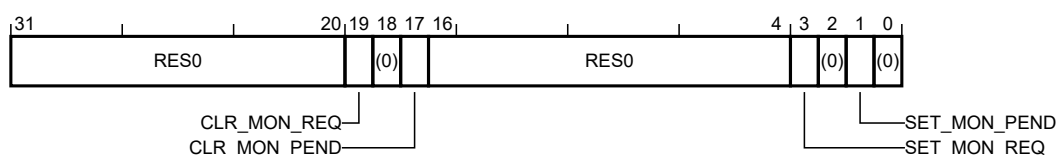
32-bit write-only register located at 0xE000EE00.

Secure software can access the Non-secure version of this register via DSCEMCR_NS located at 0xE002EE00. The location 0xE002EE00 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The DSCEMCR bit assignments are:



Bits [31:20]

Reserved, [RES0](#).

CLR_MON_REQ, bit [19]

Clear monitor request. Atomically clears the DEMCR.MON_REQ field.

The possible values of this bit are:

0

No effect.

1

Clear DEMCR.MON_REQ.

A write to this register with both SET_MON_REQ and CLR_MON_REQ set to 1 causes DEMCR.MON_REQ to become [UNKNOWN](#).

Bit [18]

Reserved, [RES0](#).

CLR_MON_PEND, bit [17]

Clear monitor pend. Atomically clears the DEMCR.MON_PEND field.

The possible values of this bit are:

0

No effect.

1

Clear DEMCR.MON_PEND.

A write to this register with both SET_MON_PEND and CLR_MON_PEND set to 1 causes DEMCR.MON_PEND to become **UNKNOWN**.

Bits [16:4]

Reserved, **RES0**.

SET_MON_REQ, bit [3]

Set monitor request. Atomically sets the DEMCR.MON_REQ field.

The possible values of this bit are:

0

No effect.

1

Sets DEMCR.MON_REQ.

A write to this register with both SET_MON_REQ and CLR_MON_REQ set to 1 causes DEMCR.MON_REQ to become **UNKNOWN**.

Bit [2]

Reserved, **RES0**.

SET_MON_PEND, bit [1]

Set monitor pend. Atomically sets the DEMCR.MON_PEND field.

The possible values of this bit are:

0

No effect.

1

Sets DEMCR.MON_PEND.

A write to this register with both SET_MON_PEND and CLR_MON_PEND set to 1 causes DEMCR.MON_PEND to become **UNKNOWN**.

Bit [0]

Reserved, **RES0**.

D1.2.51 DSCSR, Debug Security Control and Status Register

The DSCSR characteristics are:

Purpose

Provides control and status information for Secure debug.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

This register is accessible only to the debugger and **RES0** to software.

Configurations

Present only if the Security Extension is implemented.

This register is **RES0** if the Security Extension is not implemented.

Present only if Halting debug is implemented.

This register is **RES0** if Halting debug is not implemented.

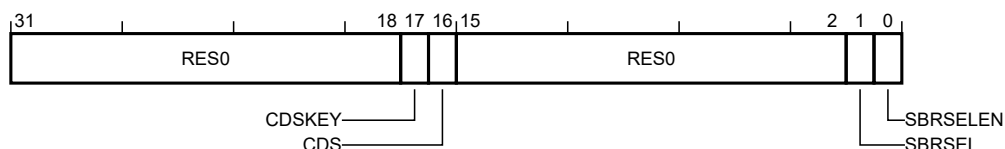
Attributes

32-bit read/write register located at 0xE000EE08.

This register is not banked between Security states.

Field descriptions

The DSCSR bit assignments are:



Bits [31:18]

Reserved, **RES0**.

CDSKEY, bit [17]

CDS write-enable key. Writes to the CDS bit are ignored unless CDSKEY is concurrently written to zero.

The possible values of this bit are:

0

Concurrent write to CDS not ignored.

1

Concurrent write to CDS ignored.

This bit reads-as-one.

CDS, bit [16]

Current domain Secure. This field indicates the current Security state of the processor.

The possible values of this bit are:

0

PE is in Non-secure state.

1

PE is in Secure state.

This bit is only writable if all of the following are true:

- DHCSR.S_SDE is 1.
- The processor is in Secure state and either DHCSR.S_NSUIDE == 0 or the current mode for the Non-secure state is unprivileged, or the processor is in Non-secure state and either DHCSR.S_SUIDE == 0 or the current mode for the Secure state is unprivileged.
- The access to the register originates from the debugger.
- The PE is halted in Debug state.
- CDSKEY is concurrently written to zero.

Bits [15:2]

Reserved, [RES0](#).

SBRSEL, bit [1]

Secure banked register select. If SBRSELEN is 1 this bit selects whether the Non-secure or the Secure versions of the memory-mapped banked registers are accessible to the debugger.

The possible values of this bit are:

0

Selects the Non-secure versions.

1

Selects the Secure versions.

This bit behaves as RAZ/WI if DHCSR.S_SDE is 0.

This bit resets to zero on a Cold reset.

SBRSELEN, bit [0]

Secure banked register select enable. Controls whether the SBRSEL field or the current Security state of the processor selects which version of the memory-mapped banked registers are accessible to the debugger.

The possible values of this bit are:

0

The current Security state of the PE determines which memory-mapped Banked registers are accessed by the debugger.

1

DHCSR.SBRSEL selects which memory-mapped Banked registers are accessed by the debugger.

This bit behaves as RAO/WI if DHCSR.S_SDE is 0.

This bit resets to zero on a Cold reset.

Note

This method of banked register selection means that the register aliasing is not used for accesses from the debugger. Accesses to the aliased addresses from the debugger have the same behavior as reserved addresses.

D1.2.52 DWT_CIDR0, DWT Component Identification Register 0

The DWT_CIDR0 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

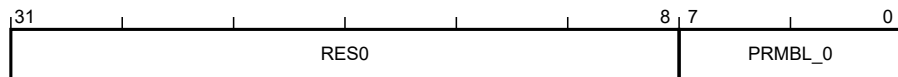
Attributes

32-bit read-only register located at 0xE0001FF0.

This register is not banked between Security states.

Field descriptions

The DWT_CIDR0 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

PRMBL_0, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0D.

D1.2.53 DWT_CIDR1, DWT Component Identification Register 1

The DWT_CIDR1 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

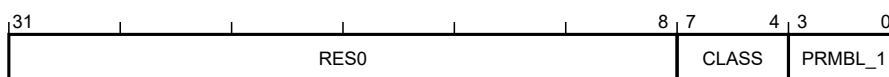
Attributes

32-bit read-only register located at 0xE0001FF4.

This register is not banked between Security states.

Field descriptions

The DWT_CIDR1 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

CLASS, bits [7:4]

CoreSight component class. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x9.

PRMBL_1, bits [3:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0.

D1.2.56 DWT_COMPn, DWT Comparator Register, n = 0 - 14

The DWT_COMP{0..14} characteristics are:

Purpose

Provides a reference value for use by watchpoint comparator *n*.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Attributes

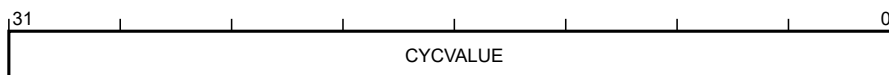
32-bit read/write register located at $0xE0001020 + 16n$.

This register is not banked between Security states.

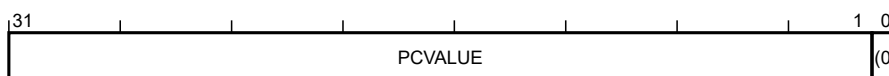
Field descriptions

The DWT_COMP{0..14} bit assignments are:

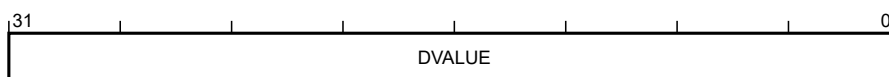
When DWT_FUNCTIONn.MATCH == 0b0001:



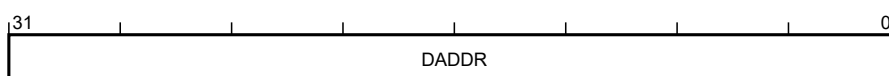
When DWT_FUNCTIONn.MATCH == 0b001x:



When DWT_FUNCTIONn.MATCH == 0b10xx:



When DWT_FUNCTIONn.MATCH == 0bx1xx:



CYCVALUE, bits [31:0], when DWT_FUNCTIONn.MATCH == 0b0001

Cycle value. Reference value for comparison with cycle count.

This field resets to an **UNKNOWN** value on a Cold reset.

PCVALUE, bits [31:1], when DWT_FUNCTIONn.MATCH == 0b001x

PC value. Reference value for comparison with Program Counter.

This field resets to an **UNKNOWN** value on a Cold reset.

Bit [0], when DWT_FUNCTIONn.MATCH == 0b001x

Reserved, **RES0**.

DADDR, bits [31:0], when DWT_FUNCTIONn.MATCH == 0bx1xx

Data address. Reference value for comparison with load or store address.

For halfword address comparisons, DADDR[0] is **RES0**. For byte address comparisons, DADDR[1:0] are **RES0**.

This field resets to an **UNKNOWN** value on a Cold reset.

DVALUE, bits [31:0], when DWT_FUNCTIONn.MATCH == 0b10xx

Data value. Reference value for comparison with load or store data.

For halfword or word comparisons, the data value is in little-endian order. That is, the least significant byte of this register is compared with the byte targeting the lowest address in memory.

For byte or halfword comparisons, if the value of the byte or halfword is not replicated across all byte or halfword lanes, the value used for the comparison is **UNKNOWN**.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.57 DWT_CPICNT, DWT CPI Count Register

The DWT_CPICNT characteristics are:

Purpose

Counts additional cycles required to execute multicycle instructions and instruction fetch stalls.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Present only if DWT_CTRL.NOPRFCNT == 0.

This register is **RES0** if DWT_CTRL.NOPRFCNT == 1.

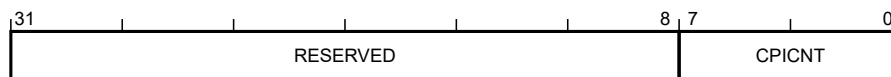
Attributes

32-bit read/write register located at 0xE0001008.

This register is not banked between Security states.

Field descriptions

The DWT_CPICNT bit assignments are:

**RESERVED, bits [31:8]**

Reserved, RAZ/WI

CPICNT, bits [7:0]

Base instruction overhead counter.

Counts one on each cycle when all of the following are true:

- `DWT_CTRL.CPIEVTENA == 1` and `DEMCR.TRCENA == 1`.
- No instruction is executed.
- No load-store operation is in progress, see `DWT_LSUCNT`.
- No exception-entry or exception-exit operation is in progress, see `DWT_EXCCNT`.
- The PE is not in a power-saving mode, see `DWT_SLEPCNT`.
- Either `SecureNoninvasiveDebugAllowed() == TRUE`, or the PE is in Non-secure state and `NoninvasiveDebugAllowed() == TRUE`.

The definition of "no instruction is executed" is **IMPLEMENTATION DEFINED**. Arm recommends that this counts each cycle on which no instruction is retired.

Initialized to zero when the counter is disabled and DWT_CTRL.CPIEVTENA is written with 1. An Event Counter packet is emitted on counter overflow.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.58 DWT_CTRL, DWT Control Register

The DWT_CTRL characteristics are:

Purpose

Provides configuration and status information for the DWT unit, and used to control features of the unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

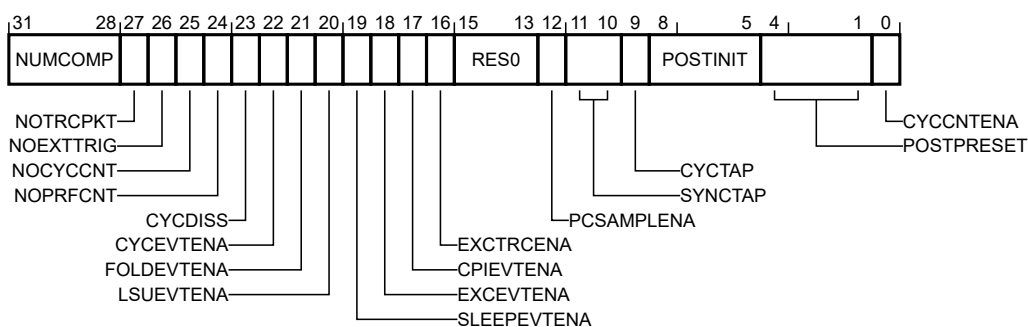
Attributes

32-bit read/write register located at 0xE0001000.

This register is not banked between Security states.

Field descriptions

The DWT_CTRL bit assignments are:



NUMCOMP, bits [31:28]

Number of comparators. Number of DWT comparators implemented.

A value of zero indicates no comparator support.

This field reads as an **IMPLEMENTATION DEFINED** value.

NOTRCPKT, bit [27]

No trace packets. Indicates whether the implementation does not support trace.

The possible values of this bit are:

0

Trace supported.

1

Trace not supported.

If this bit is RAZ, the NOCYCCNT bit must also RAZ.

If the Main Extension is not implemented, this bit is RES1.

This bit reads as an IMPLEMENTATION DEFINED value.

NOEXTTRIG, bit [26]

No External Triggers. Shows whether the implementation does not support external triggers.

Reserved, RES0.

NOCYCCNT, bit [25]

No cycle count. Indicates whether the implementation does not include a cycle counter.

The possible values of this bit are:

0

Cycle counter implemented.

1

Cycle counter not implemented.

If the Main Extension is not implemented, this bit is RES1.

This bit reads as an IMPLEMENTATION DEFINED value.

NOPRFCNT, bit [24]

No profile counters. Indicates whether the implementation does not include the profiling counters.

The possible values of this bit are:

0

Profiling counters implemented.

1

Profiling counters not implemented.

If the Main Extension is not implemented, this bit is RES1.

This bit reads as an IMPLEMENTATION DEFINED value.

CYCDISS, bit [23]

Cycle counter disabled secure. Controls whether the cycle counter is disabled in Secure state.

The possible values of this bit are:

0

No effect.

1

Disable incrementing of the cycle counter when the PE is in Secure state.

If the NOCYCCNT bit is RAO this bit is RES0.

If the Main Extension is not implemented, this bit is RES0.

This bit resets to zero on a Cold reset.

CYCEVTENA, bit [22]

Cycle event enable. Enables Event Counter packet generation on POSTCNT underflow.

The possible values of this bit are:

0

No Event Counter packets generated when POSTCNT underflows.

1

If PCSAMPLENA set to 0, an Event Counter packet is generated when POSTCNT underflows.

If the NOTRCPKT bit is RAO or the NOCYCCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

FOLDEVTENA, bit [21]

Fold event enable. Enables DWT_FOLDCNT counter.

The possible values of this bit are:

0

DWT_FOLDCNT disabled.

1

DWT_FOLDCNT enabled.

If the NOPRFCNT bit is RAO this bit is [RES0](#). The reset value is 0.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

LSUEVTENA, bit [20]

LSU event enable. Enables DWT_LSUCNT counter.

The possible values of this bit are:

0

DWT_LSUCNT disabled.

1

DWT_LSUCNT enabled.

If the NOPRFCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

SLEEPEVTENA, bit [19]

Sleep event enable. Enable DWT_SLEEPCNT counter.

The possible values of this bit are:

0

DWT_SLEEPCNT disabled.

1

DWT_SLEEPCNT enabled.

If the NOPRFCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

EXCEVTENA, bit [18]

Exception event enable. Enables DWT_EXCCNT counter.

The possible values of this bit are:

0

DWT_EXCCNT disabled.

1

DWT_EXCCNT enabled.

If the NOPRFCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

CPIEVTENA, bit [17]

CPI event enable. Enables DWT_CPICNT counter.

The possible values of this bit are:

0

DWT_CPICNT disabled.

1

DWT_CPICNT enabled.

If the NOPRFCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

EXCTRCENA, bit [16]

Exception trace enable. Enables generation of Exception Trace packets.

The possible values of this bit are:

0

Exception Trace packet generation disabled.

1

Exception Trace packet generation enabled.

If the NOTRCPKT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

Bits [15:13]

Reserved, [RES0](#).

PCSAMPLENA, bit [12]

PC sample enable. Enables use of POSTCNT counter as a timer for Periodic PC Sample packet generation.

The possible values of this bit are:

0

Periodic PC Sample packet generation disabled.

1

Periodic PC Sample packet generated on POSTCNT underflow.

If the NOTRCPKT bit is RAO or the NOCYCCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

SYNCTAP, bits [11:10]

Synchronization tap. Selects the position of the synchronization packet request counter tap on the CYCCNT counter. This determines the rate of Synchronization packet requests made by the DWT.

The possible values of this field are:

0b00

Synchronization packet request disabled.

0b01

Synchronization counter tap at CYCCNT[24].

0b10

Synchronization counter tap at CYCCNT[26].

0b11

Synchronization counter tap at CYCCNT[28].

If the NOCYCCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this field is [RES0](#).

This field resets to an [UNKNOWN](#) value on a Cold reset.

CYCTAP, bit [9]

Cycle count tap. Selects the position of the POSTCNT tap on the CYCCNT counter.

The possible values of this bit are:

0

POSTCNT tap at CYCCNT[6].

1

POSTCNT tap at CYCCNT[10].

If the NOCYCCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to an [UNKNOWN](#) value on a Cold reset.

POSTINIT, bits [8:5]

POSTCNT initial. Initial value for the POSTCNT counter.

If the NOCYCCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this field is [RES0](#).

This field resets to an [UNKNOWN](#) value on a Cold reset.

POSTPRESET, bits [4:1]

POSTCNT preset. Reload value for the POSTCNT counter.

If the NOCYCCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this field is [RES0](#).

This field resets to an [UNKNOWN](#) value on a Cold reset.

CYCCNTENA, bit [0]

CYCCNT enable. Enables CYCCNT.

The possible values of this bit are:

0

CYCCNT disabled.

1

CYCCNT enabled.

If the NOCYCCNT bit is RAO this bit is [RES0](#).

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Cold reset.

D1.2.59 DWT_CYCCNT, DWT Cycle Count Register

The DWT_CYCCNT characteristics are:

Purpose

Shows or sets the value of the processor cycle counter, CYCCNT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if the DWT is implemented.

This register is [RES0](#) if the DWT is not implemented.

Present only if DWT_CTRL.NOCYCCNT == 0.

This register is [RES0](#) if DWT_CTRL.NOCYCCNT == 1.

Attributes

32-bit read/write register located at 0xE0001004.

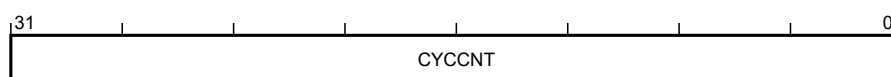
This register is not banked between Security states.

Preface

The PMU_CCNTR register, if implemented, is an alias of this register.

Field descriptions

The DWT_CYCCNT bit assignments are:



CYCCNT, bits [31:0]

Incrementing cycle counter value. Increments one on each processor clock cycle when DWT_CTRL.CYCCNTENA == 1 and DEMCR.TRCENA == 1. On overflow, CYCCNT wraps to zero.

This field resets to an [UNKNOWN](#) value on a Cold reset.

D1.2.60 DWT_DEVARCH, DWT Device Architecture Register

The DWT_DEVARCH characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

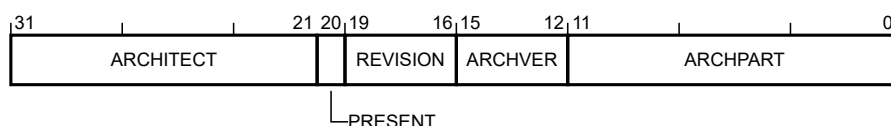
Attributes

32-bit read-only register located at 0xE0001FBC.

This register is not banked between Security states.

Field descriptions

The DWT_DEVARCH bit assignments are:



ARCHITECT, bits [31:21]

Architect. Defines the architect of the component. Bits [31:28] are the JEP106 continuation code (JEP106 bank ID, minus 1) and bits [27:21] are the JEP106 ID code.

The possible values of this field are:

0x23B

JEP106 continuation code 0x4, ID code 0x3B. Arm Limited.

Other values are defined by the JEDEC JEP106 standard.

This field reads as 0x23B.

PRESENT, bit [20]

DEVARCH Present. Defines that the DEVARCH register is present.

The possible values of this bit are:

1

DEVARCH information present.

This bit reads as one.

REVISION, bits [19:16]

Revision. Defines the architecture revision of the component.

The possible values of this field are:

0b0000

DWT architecture v2.0.

0b0001

DWT architecture v2.1. The DWT_VMASKn registers are implemented.

DWT architecture v2.1 is mandatory for a DWT implementation that includes data value comparators in a PE that implements v8.1-M and MVE. If the DWT implementation does not include data value comparators, it is IMPDEF whether it is v2.0 or v2.1.

This field reads as an **IMPLEMENTATION DEFINED** value.

ARCHVER, bits [15:12]

Architecture Version. Defines the architecture version of the component.

The possible values of this field are:

0b0001

DWT architecture v2.

ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHVER is ARCHID[15:12].

This field reads as 0b0001.

ARCHPART, bits [11:0]

Architecture Part. Defines the architecture of the component.

The possible values of this field are:

0xA02

DWT architecture.

ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHPART is ARCHID[11:0].

This field reads as 0xA02.

D1.2.61 DWT_DEVTYPE, DWT Device Type Register

The DWT_DEVTYPE characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

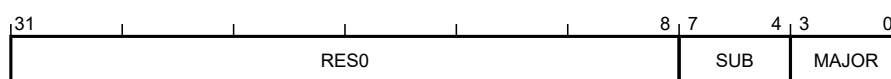
Attributes

32-bit read-only register located at 0xE0001FCC.

This register is not banked between Security states.

Field descriptions

The DWT_DEVTYPE bit assignments are:



Bits [31:8]

Reserved, **RES0**.

SUB, bits [7:4]

Sub-type. Component sub-type.

The possible values of this field are:

0x0

Other.

This field reads as 0b0000.

MAJOR, bits [3:0]

Major type. Component major type.

The possible values of this field are:

0x0

Miscellaneous.

This field reads as 0b0000.

D1.2.62 DWT_EXCCNT, DWT Exception Overhead Count Register

The DWT_EXCCNT characteristics are:

Purpose

Counts the total cycles spent in exception processing.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Present only if DWT_CTRL.NOPRFCNT == 0.

This register is **RES0** if DWT_CTRL.NOPRFCNT == 1.

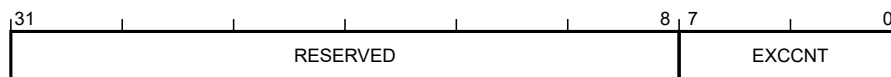
Attributes

32-bit read/write register located at 0xE000100C.

This register is not banked between Security states.

Field descriptions

The DWT_EXCCNT bit assignments are:

**RESERVED, bits [31:8]**

Reserved, RAZ/WI

EXCCNT, bits [7:0]

The exception overhead counter.

Counts one on each cycle when all of the following are true:

- DWT_CTRL.EXCEVTENA == 1 and DEMCR.TRCENA == 1.
- No instruction is executed, see DWT_CPICNT.
- An exception-entry or exception-exit related operation is in progress.
- Either SecureNoninvasiveDebugAllowed() == TRUE, or NS-Req for the operation is set to Non-secure and NoninvasiveDebugAllowed() == TRUE.

Exception-entry or exception-exit related operations include the stacking of registers on exception entry, lazy Floating-point state preservation, unstacking of registers on exception exit, and preemption.

Initialized to zero when the counter is disabled and DWT_CTRL.EXCEVTENA is written with 1. An Event Counter packet is emitted on counter overflow.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.63 DWT_FOLDCNT, DWT Folded Instruction Count Register

The DWT_FOLDCNT characteristics are:

Purpose

Increments for each additional instruction executed in the current cycle.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Present only if DWT_CTRL.NOPRFCNT == 0.

This register is **RES0** if DWT_CTRL.NOPRFCNT == 1.

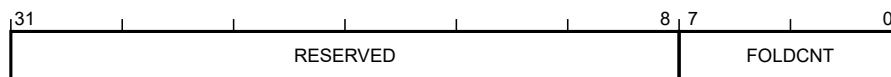
Attributes

32-bit read/write register located at 0xE0001018.

This register is not banked between Security states.

Field descriptions

The DWT_FOLDCNT bit assignments are:

**RESERVED, bits [31:8]**

Reserved, RAZ/WI

FOLDCNT, bits [7:0]

Folded instruction counter.

Counts on each cycle when all of the following are true:

- DWT_CTRL.FOLDEVTENA == 1 and DEMCR.TRCENA == 1.
- At least two instructions are executed, see DWT_CPICNT.
- Either SecureNoninvasiveDebugAllowed() == TRUE, or the PE is in Non-secure state and NoninvasiveDebugAllowed() == TRUE.

The counter is incremented by the number of instructions executed, minus one.

Initialized to zero when the counter is disabled and DWT_CTRL.FOLDEVTENA is written with 1. An event is emitted on counter overflow.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.64 DWT_FUNCTIONn, DWT Comparator Function Register, n = 0 - 14

The DWT_FUNCTION{0..14} characteristics are:

Purpose

Controls the operation of watchpoint comparator *n*.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

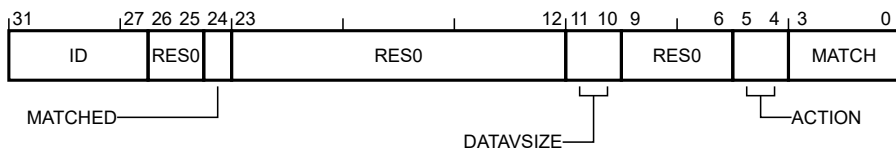
Attributes

32-bit read/write register located at $0xE0001028 + 16n$.

This register is not banked between Security states.

Field descriptions

The DWT_FUNCTION{0..14} bit assignments are:



ID, bits [31:27]

Identify capability. Identifies the capabilities for MATCH for comparator *n*.

The possible values of this field are:

0b00000

Reserved.

0b01000

Data Address, and Data Address With Value.

0b01001

Cycle Counter, Data Address, and Data Address With Value.

0b01010

Instruction Address, Data Address, and Data Address With Value.

0b01011

Cycle Counter, Instruction Address, Data Address and Data Address With Value.

0b11000

Data Address, Data Address Limit, and Data Address With Value.

0b11010

Instruction Address, Instruction Address Limit, Data Address, Data Address Limit, and Data Address With Value.

0b11100

Data Address, Data Address Limit, Data Value, Linked Data Value, and Data Address With Value.

0b11110

Instruction Address, Instruction Address Limit, Data Address, Data Address Limit, Data value, Linked Data Value, and Data Address With Value.

All other values are reserved.

Comparator 0 never supports linking. If more than one comparator is implemented, then at least one comparator must support linking. Arm recommends that odd-numbered comparators support linking.

Cycle Counter matching is only supported if the Main Extension is implemented and DWT_CTRL.NOCYCCNT == 0, meaning the cycle counter is implemented. Comparator 0 must support Cycle Counter matching if the cycle counter is implemented.

Data Address With Value is supported for the first four comparators only, and only if the Main Extension and ITM are implemented, and DWT_CTRL.NOTRCPKT == 0. Data Value and Linked Data Value not supported if the Main Extension is not implemented.

This field is read-only.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [26:25]

Reserved, **RES0**.

MATCHED, bit [24]

Comparator matched. Set to 1 when the comparator matches.

The possible values of this bit are:

0

No match.

1

Match. The comparator has matched since the last read of this register.

For an Instruction Address Limit or Data Address Limit comparator, this bit is **UNKNOWN** on reads.

This bit is read-only.

This bit clears to zero when read.

This bit resets to an **UNKNOWN** value on a Cold reset.

Bits [23:12]

Reserved, **RES0**.

DATAVSIZE, bits [11:10]

Data value size. Defines the size of the object being watched for by Data Value and Data Address comparators.

The possible values of this field are:

0b00

1 byte.

0b01

2 bytes.

0b10

4 bytes.

All other values are reserved.

For an Instruction Address or Instruction Address Limit comparator, DATAVSIZ must be 0b01 (2 bytes). If this comparator is part of an data address range pair, DATAVSIZ must be 0b00 (1 byte).

For a Data Address comparator, DWT_COMP n must be aligned to the size specified by DATAVSIZ. For a Data Value or Linked Data Value comparator:

- For halfword comparisons, DWT_COMP n [31:16] must be equal to DWT_COMP n [15:0].
- For byte comparisons, DWT_COMP n [31:24], DWT_COMP n [23:16], and DWT_COMP n [15:8] must be equal to DWT_COMP n [7:0].

This field resets to an UNKNOWN value on a Cold reset.

Bits [9:6]

Reserved, RES0.

ACTION, bits [5:4]

Action on match. Defines the action on a match. This field is ignored and the comparator generates no actions if it is disabled by MATCH.

The possible values of this field are:

0b00

Trigger only.

0b01

Generate debug event.

0b10

For a Cycle Counter, Instruction Address, Data Address, Data Value or Linked Data Value comparator, generate a Data Trace Match packet.

For a Data Address With Value comparator, generate a Data Trace Data Value packet.

0b11

For a Data Address Limit comparator, generate a Data Trace Data Address packet.

For a Cycle Counter, Instruction Address Limit, or Data Address comparator, generate a Data Trace PC Value packet.

For a Data Address With Value comparator, generate both a Data Trace PC Value packet and a Data Trace Data Value packet.

If the Main Extension is not implemented, the values 0b10 and 0b11 are reserved.

This field resets to an UNKNOWN value on a Cold reset.

MATCH, bits [3:0]

Match type. Controls the type of match generated by this comparator.

The possible values of this field are:

0b0000

Disabled. Never generates a match.

0b0001

Cycle Counter. Matches if DWT_CYCCNT equals the comparator value. The comparator is checked each time DWT_CYCCNT is written to, directly or indirectly.

Only supported if the Main Extension is implemented, DWT_FUNCTION< n >.ID<0> == 1 and DWT_CTRL.NOCYCCNT == 0.

0b0010

Instruction Address. If not linked to, an instruction matches if the address of the first byte of the instruction matches the comparator address.

Only supported if DWT_FUNCTION<n>.ID<1> == 1.

0b0011

Instruction Address Limit. When comparator <n-1> is configured as Instruction Address (0b0010) an access will generate a match if the first byte of the instruction lies between the lower address (specified by the comparator <n-1>) and the limit address (specified by this comparator, <n>). Both the lower address and the limit address are inclusive to the address range.

It is **UNPREDICTABLE** whether or not a match is generated on an access by any comparator when either:

- Comparator <n-1> is configured as a comparator type which is neither Instruction Address (0b0010) nor Disabled (0b0000).
- The lower address is unsigned greater-than-or-equal-to the limit address.

If comparator <n-1> is Disabled (0b0000) no access will generate a match for this comparator.

Only supported if DWT_FUNCTION<n>.ID<4> == 1 and DWT_FUNCTION<n>.ID<1> == 1.

0b0100

Data Address. If not linked to by a Data Address Limit comparator, an access matches if any accessed byte lies between the comparator address and a limit defined by the DATAVSIZE field. Supported for all comparators.

0b0101

Data Address, writes. As 0b0100, except that only write accesses generate a match.

0b0110

Data Address, reads. As 0b0100, except that only read accesses generate a match.

0b0111

Data Address Limit. An access will generate a match when all of the following are true:

- Comparator <n-1> is configured as one of Data Address (0b01xx, not 0b0111) or Data Address With Value (0b11xx, not 0b1111).
- Any accessed byte lies between the lower address (specified by comparator <n-1>) and the limit address (specified by this comparator <n>).
- Both the lower address and limit address are inclusive to the range.

It is **UNPREDICTABLE** whether or not a match is generated on an access by any comparator when either:

- Comparator <n-1> is not configured as one of Data Address (0b01xx, not 0b0111) nor Data Address With Value (0b11xx, not 0b1111).
- The lower address is unsigned greater-than-or-equal-to the limit address.

If comparator <n-1> is configured as Disabled (0b0000) no access will generate a match for this comparator.

Only supported if DWT_FUNCTION<n>.ID<4> == 1.

0b1000

Data Value. An access matches if the value accessed matches the comparator value.

Only supported if the Main Extension is implemented and DWT_FUNCTION<n>.ID<2> == 1.

0b1001

Data Value, writes. As 0b1000, except that only write accesses generate a match.

0b1010

Data Value, reads. As 0b1000, except that only read accesses generate a match.

0b1011

Linked Data Value. An access will generate a match when all of the following are true:

- Comparator $\langle n-1 \rangle$ is configured as Data Address (0b01xx, not 0b0111), or Data Address With Value (0b11xx, not 0b1111).
- DATAVSIZE for the two comparators $\langle n \rangle$ and $\langle n-1 \rangle$ are equal.
- The value accessed matches the comparator value specified by comparator $\langle n \rangle$.
- Any accessed byte lies between the lower address (specified by comparator $\langle n-1 \rangle$) and limit defined by the DATAVSIZE field.
- Both the lower address and the limit address are inclusive to the range.

It is UNPREDICTABLE whether or not any comparator generates a match if any of the following are true:

- Comparator $\langle n-1 \rangle$ is not configured as one of Data Address (0b01xx, not 0b0111), Data Address With Value (0b11xx, not 0b1111), or Disabled (0b0000).
- DATAVSIZE for the two comparators $\langle n-1 \rangle$ and $\langle n \rangle$ are not equal.

If comparator $\langle n-1 \rangle$ is programmed to Disabled (0b0000) no access will generate a match for this comparator.

Only supported if the Main Extension is implemented and $\text{DWT_FUNCTION}\langle n \rangle.\text{ID}\langle 4 \rangle == 1$ and $\text{DWT_FUNCTION}\langle n \rangle.\text{ID}\langle 2 \rangle == 1$.

0b1100

Data Address With Value. As 0b01xx, except that the data value is traced.

Supported for the first four comparators only, and only if $\text{DWT_CTRL}.\text{NOTRCPKT} == 0$ and ITM is also implemented.

0b1101

Data Address With Value, writes. As 0b1100, except that only write accesses generate a match.

0b1110

Data Address With Value, reads. As 0b1100, except that only read accesses generate a match.

Any value not supported by the comparator is reserved. For a pair of linked comparators, $\langle n \rangle$ and $\langle n-1 \rangle$, $\text{DWT_FUNCTION}\langle n-1 \rangle.\text{MATCH}[1:0]$ determines the matching access types.

This field resets to zero on a Cold reset.

D1.2.65 DWT_LAR, DWT Software Lock Access Register

The DWT_LAR characteristics are:

Purpose

Provides CoreSight Software Lock control for the DWT, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank). It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

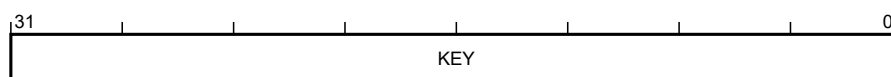
Attributes

32-bit write-only register located at 0xE0001FB0.

This register is not banked between Security states.

Field descriptions

The DWT_LAR bit assignments are:



KEY, bits [31:0]

Lock Access control.

Writing the key value 0xC5ACCE55 to this field unlocks the lock, enabling write accesses to the registers of this component through a memory-mapped interface.

Writing any other value to this register locks the lock, disabling write accesses to the registers of this component through a memory mapped interface.

D1.2.66 DWT_LSR, DWT Software Lock Status Register

The DWT_LSR characteristics are:

Purpose

Provides CoreSight Software Lock status information for the DWT, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank). It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

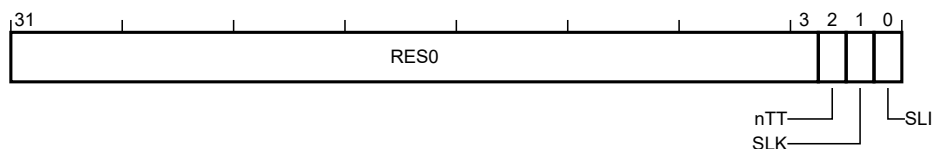
Attributes

32-bit read-only register located at 0xE0001FB4.

This register is not banked between Security states.

Field descriptions

The DWT_LSR bit assignments are:

**Bits [31:3]**

Reserved, RES0.

nTT, bit [2]

Not thirty-two bit. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as zero.

SLK, bit [1]

Software Lock status. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

O

Lock clear. Software writes are permitted to the registers of this component.

1

Lock set. Software writes to the registers of this component are ignored, and reads have no side effects.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is [RES0](#).

This bit resets to one on a Cold reset.

SLI, bit [0]

Software Lock implemented. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

0

Software Lock not implemented or debugger access.

1

Software Lock is implemented and software access.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is RAZ.

This bit reads as an [IMPLEMENTATION DEFINED](#) value.

D1.2.67 DWT_LSUCNT, DWT LSU Count Register

The DWT_LSUCNT characteristics are:

Purpose

Increments on the additional cycles required to execute all load or store instructions.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Present only if DWT_CTRL.NOPRFCNT == 0.

This register is **RES0** if DWT_CTRL.NOPRFCNT == 1.

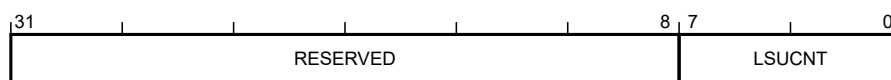
Attributes

32-bit read/write register located at 0xE0001014.

This register is not banked between Security states.

Field descriptions

The DWT_LSUCNT bit assignments are:

**RESERVED, bits [31:8]**

Reserved, RAZ/WI

LSUCNT, bits [7:0]

Load-store overhead counter.

Counts one on each cycle when all of the following are true:

- `DWT_CTRL.LSUEVTENA == 1` and `DEMCR.TRCENA == 1`.
- No instruction is executed, see `DWT_CPICNT`.
- No exception-entry or exception-exit operation is in progress, see `DWT_EXCCNT`.
- A load-store operation is in progress.
- Either `SecureNoninvasiveDebugAllowed() == TRUE`, or NS-Req for the operation is set to Non-secure and `NoninvasiveDebugAllowed() == TRUE`.

Initialized to zero when the counter is disabled and DWT_CTRL.LSUEVTENA is written with 1. An Event Counter packet is emitted on counter overflow.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.68 DWT_PCSR, DWT Program Counter Sample Register

The DWT_PCSR characteristics are:

Purpose

Samples the current value of the Program Counter.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

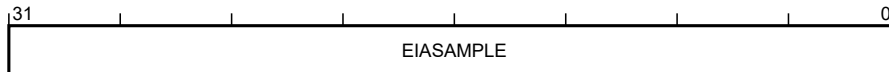
Attributes

32-bit read-only register located at 0xE000101C.

This register is not banked between Security states.

Field descriptions

The DWT_PCSR bit assignments are:



EIASAMPLE, bits [31:0]

Executed instruction address sample. Recently executed instruction address sample value.

The possible values of this field are:

0xFFFFFFFF

One of the following is true:

- The PE is halted in Debug state.
- The address of the recently executed instruction is not available.
- NoninvasiveDebugAllowed(FALSE) == FALSE.
- UDE is implemented, NoninvasiveDebugAllowed(TRUE) == FALSE and the instruction was executed in a privileged mode.
- The Security Extension is implemented, the sampled instruction was executed in Secure state, and SecureNoninvasiveDebugAllowed(FALSE) == FALSE.
- The Security Extension and UDE are implemented and SecureNoninvasiveDebugAllowed(TRUE) == FALSE, and the instruction was executed in Secure state and privileged mode.
- DEMCR.TRCENA == 0.
- The address of a recently-executed instruction is not available.

Not 0xFFFFFFFF

Instruction address of a recently executed instruction. Bit [0] of the sample instruction address is 0.

The conditions when the address of a recently-executed instruction is not available are [IMPLEMENTATION DEFINED](#).

D1.2.70 DWT_PIDR1, DWT Peripheral Identification Register 1

The DWT_PIDR1 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

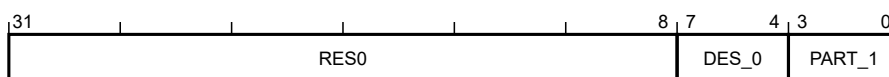
Attributes

32-bit read-only register located at 0xE0001FE4.

This register is not banked between Security states.

Field descriptions

The DWT_PIDR1 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

DES_0, bits [7:4]

JEP106 identification code bits [3:0]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

PART_1, bits [3:0]

Part number bits [11:8]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.71 DWT_PIDR2, DWT Peripheral Identification Register 2

The DWT_PIDR2 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

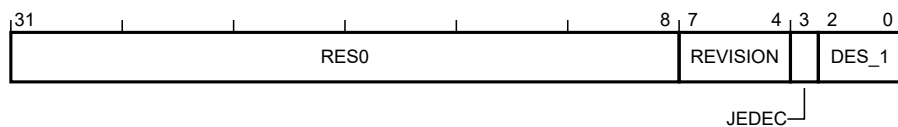
Attributes

32-bit read-only register located at 0xE0001FE8.

This register is not banked between Security states.

Field descriptions

The DWT_PIDR2 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REVISION, bits [7:4]

Component revision. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

JEDEC, bit [3]

JEDEC assignee value is used. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as one.

DES_1, bits [2:0]

JEP106 identification code bits [6:4]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.72 DWT_PIDR3, DWT Peripheral Identification Register 3

The DWT_PIDR3 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

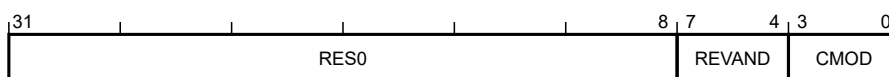
Attributes

32-bit read-only register located at 0xE0001FEC.

This register is not banked between Security states.

Field descriptions

The DWT_PIDR3 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REVAND, bits [7:4]

RevAnd. See the Arm® CoreSight™ Architecture Specification.

This field reads as an **IMPLEMENTATION DEFINED** value.

CMOD, bits [3:0]

Customer Modified. See the Arm® CoreSight™ Architecture Specification.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.73 DWT_PIDR4, DWT Peripheral Identification Register 4

The DWT_PIDR4 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

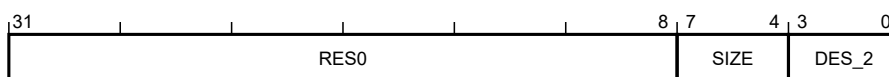
Attributes

32-bit read-only register located at 0xE0001FD0.

This register is not banked between Security states.

Field descriptions

The DWT_PIDR4 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

SIZE, bits [7:4]

4KB count. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as zero.

DES_2, bits [3:0]

JEP106 continuation code. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.74 DWT_PIDR5, DWT Peripheral Identification Register 5

The DWT_PIDR5 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Attributes

32-bit read-only register located at 0xE0001FD4.

This register is not banked between Security states.

Field descriptions

The DWT_PIDR5 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.75 DWT_PIDR6, DWT Peripheral Identification Register 6

The DWT_PIDR6 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Attributes

32-bit read-only register located at 0xE0001FD8.

This register is not banked between Security states.

Field descriptions

The DWT_PIDR6 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.76 DWT_PIDR7, DWT Peripheral Identification Register 7

The DWT_PIDR7 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the DWT.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Attributes

32-bit read-only register located at 0xE0001FDC.

This register is not banked between Security states.

Field descriptions

The DWT_PIDR7 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.77 DWT_SLEEPCNT, DWT Sleep Count Register

The DWT_SLEEPCNT characteristics are:

Purpose

Counts the total number of cycles that the processor is sleeping.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Present only if DWT_CTRL.NOPRFCNT == 0.

This register is **RES0** if DWT_CTRL.NOPRFCNT == 1.

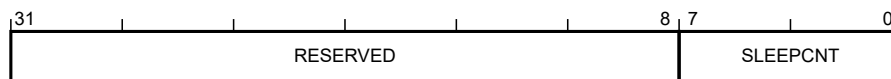
Attributes

32-bit read/write register located at 0xE0001010.

This register is not banked between Security states.

Field descriptions

The DWT_SLEEPCNT bit assignments are:



RESERVED, bits [31:8]

Reserved, RAZ/WI

SLEEPCNT, bits [7:0]

Sleep counter.

Counts one on each cycle when all of the following are true:

- DWT_CTRL.SLEEPEVTENA == 1 and DEMCR.TRCENA == 1.
- No instruction is executed, see DWT_CPICNT.
- No load-store operation is in progress, see DWT_LSUCNT.
- No exception-entry or exception-exit operation is in progress, see DWT_EXCCNT.
- The PE is in a power-saving mode.
- Either SecureNoninvasiveDebugAllowed() == TRUE, or the PE is in Non-secure state and NoninvasiveDebugAllowed() == TRUE.

Power-saving modes include WFI, WFE, and Sleep-on-exit.

All power-saving features are **IMPLEMENTATION DEFINED** and therefore when this counter counts is **IMPLEMENTATION DEFINED**. In particular, it is **IMPLEMENTATION DEFINED** whether the counter increments if the PE is in a power-saving mode and SCR.SLEEPDEEP is set.

Initialized to zero when the counter is disabled and DWT_CTRL.SLEEPEVTENA is written with 1. An Event Counter packet is emitted on counter overflow.

This field resets to an **UNKNOWN** value on a Cold reset.

Note

Arm recommends that this counter counts all cycles when the PE is sleeping and SCR.SLEEPDEEP is clear, regardless of whether a WFI or WFE instruction, or Sleep-on-exit, caused the entry to the power-saving mode.

D1.2.78 DWT_VMASKn, DWT Comparator Value Mask Register, n = 0 - 14

The DWT_VMASK{0..14} characteristics are:

Purpose

Provides a mask value for use by watchpoint comparator *n* when comparing data values.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the DWT is implemented.

This register is **RES0** if the DWT is not implemented.

Present only if the DWT implements DWT architecture version 2.1 or later.

This register is **RES0** if the DWT implements DWT architecture version 2.0 or earlier.

Attributes

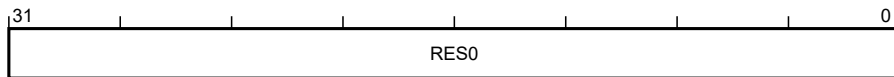
32-bit read/write register located at $0xE000102C + 16n$.

This register is not banked between Security states.

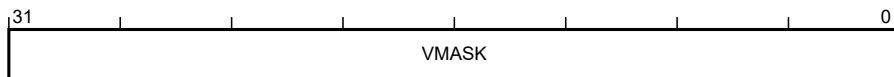
Field descriptions

The DWT_VMASK{0..14} bit assignments are:

When DWT_FUNCTIONn.MATCH != 0b10xx:



When DWT_FUNCTIONn.MATCH == 0b10xx:



Bits [31:0], when DWT_FUNCTIONn.MATCH != 0b10xx

Reserved, **RES0**.

VMASK, bits [31:0], when DWT_FUNCTIONn.MATCH == 0b10xx

Data value mask. Mask value for use in the comparison with load or store data.

The possible values of each bit are:

0

The comparison matches only if DWT_COMPn[m] matches bit [m] of the candidate data value.

1

The comparison ignores bit [m] of the candidate data value. If DWT_COMPn[m] is not set to zero, the result of the comparison is **UNPREDICTABLE**.

For halfword or word comparisons, the mask is in little-endian order. That is, the least significant byte of this register masks the byte targeting the lowest address in memory.

For byte or halfword comparisons, if the value of the byte or halfword is not replicated across all byte or halfword lanes, the value used for the comparison is **UNKNOWN**.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.79 EPSR, Execution Program Status Register

The EPSR characteristics are:

Purpose

Holds Execution state bits.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

This register is always implemented.

Attributes

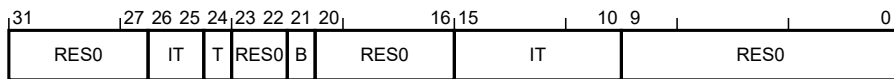
32-bit read/write special-purpose register.

This register is not banked between Security states.

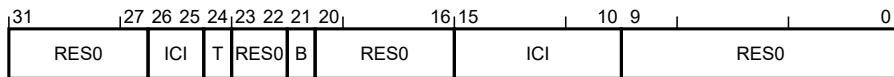
Field descriptions

The EPSR bit assignments are:

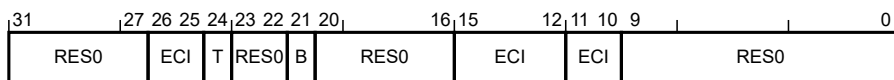
When {EPSR[26:25], EPSR[11:10]} != 0:



When {EPSR[26:25], EPSR[11:10]} == 0, and a multicycle load or store instruction is in progress:



When {EPSR[26:25], EPSR[11:10]} == 0, and beat-wise vector instructions are in progress:



Bits [31:27]

Reserved, [RES0](#).

T, bit [24]

T32 state bit. Determines the current instruction set state.

The possible values of this bit are:

0

Execution of any instruction generates an INVSTATE UsageFault.

1

Instructions decoded as T32 instructions.

This bit resets to an [UNKNOWN](#) value on a Warm reset.

Bits [23:22]

Reserved, [RES0](#).

B, bit [21]

Branch target identification active. Indicates whether a BTI clearing instruction is expected.

The possible values of this bit are:

0

Branch target identification inactive.

1

Branch target identification active.

If the PACBTI Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

Bits [20:16]

Reserved, [RES0](#).

IT, bits [15:10, 26:25] , when [{EPSR[26:25], EPSR[11:10]} != 0]

If-then flags. This field encodes the current condition and position in an IT block sequence.

The field IT[7:0] is equivalent to EPSR[15:10,26:25].

If the Main Extension is not implemented, this field is [RES0](#).

This field resets to zero on a Warm reset.

ICI, bits [26:25, 15:10] , when [{EPSR[26:25], EPSR[11:10]} == 0, and a multicycle load or store instruction is in progress]

Interrupt continuation flags. This field encodes information on the outstanding register list for an interrupted exception-continuable multicycle load or store instruction.

The field ICI[7:0] is equivalent to EPSR[26:25,15:10].

If the Main Extension is not implemented, this field is [RES0](#).

This field resets to zero on a Warm reset.

ECI, bits [26:25, 11:10, 15:12] , when [{EPSR[26:25], EPSR[11:10]} == 0, and beat-wise vector instructions are in progress]

Exception continuation flags for beat-wise vector instructions. This field encodes which beats of the in-flight instructions have completed.

The possible values of this field are:

0b00000000

No completed beats.

0b00000001

Completed beats: A0.

0b00000010

Completed beats: A0 A1.

0b00000011

Reserved.

0b00000100

Completed beats: A0 A1 A2.

0b00000101

Completed beats: A0 A1 A2 B0.

0b0000011X

Reserved.

0b00001XXX

Reserved.

In the enumeration above the letters correspond to the instructions at the return address and beyond, whilst the numbers correspond to the beats of those instructions that have been completed. For example, the sequence A0 A1 A2 B0 means the first three beats of the instruction at the return address, plus the first beat of the instruction at the return address +4 have been completed. The field ECI[7:0] is equivalent to EPSR[26:25,11:10,15:12].

This field resets to zero on a Warm reset.

Bits [9:0]

Reserved, [RES0](#).

D1.2.80 ERRADDRn, Error Record Address Register, n = 0 - 55

The ERRADDR{0..55} characteristics are:

Purpose

If an error has an associated address, this must be written to the address register when the error is recorded. It is **IMPLEMENTATION DEFINED** how the recorded addresses map to the software-visible physical addresses. Software might have to reconstruct the actual physical addresses using the identity of the node and knowledge of the system. Ignores writes if ERRSTATUS<n>.AV is set to 1.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is **RES0** if RAS is not implemented.

Attributes

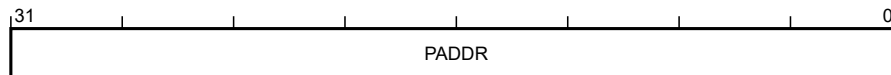
32-bit read/write register located at $0xE0005018 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The ERRADDR{0..55} bit assignments are:



PADDR, bits [31:0]

Address, bits [31:0]. Unimplemented bits are **RES0**.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.81 ERRADDR2n, Error Record Address 2 Register, n = 0 - 55

The ERRADDR2{0..55} characteristics are:

Purpose

If an error has an associated address, this must be written to the address register when the error is recorded. It is **IMPLEMENTATION DEFINED** how the recorded addresses map to the software-visible physical addresses. Software might have to reconstruct the actual physical addresses using the identity of the node and knowledge of the system. Ignores writes if `ERRSTATUS<n>.AV` is set to 1.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is **RES0** if RAS is not implemented.

Attributes

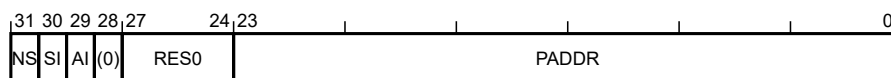
32-bit read/write register located at 0xE000501C + 64n.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The ERRADDR2{0..55} bit assignments are:

**NS, bit [31]**

Non-secure attribute.

The possible values of this bit are:

0

The address is Secure.

1

The address is Non-secure.

It is **IMPLEMENTATION DEFINED** whether this bit is read-only or read/write.

This bit resets to an **UNKNOWN** value on a Cold reset.

SI, bit [30]

Secure Incorrect. Indicates whether the NS bit is valid.

The possible values of this bit are:

O

The NS bit is correct. That is, it matches the programmers' view of the Non-secure attribute for this recorded location.

1

The NS bit might not be correct, and might not match the programmers' view of the Non-secure attribute for the recorded location.

It is **IMPLEMENTATION DEFINED** whether this bit is read-only or read/write.

This bit resets to an **UNKNOWN** value on a Cold reset.

AI, bit [29]

Address Incomplete or incorrect. Indicates whether the PADDR field in ERRADDR<n> and ERRADDR2<n> is a valid physical address.

The possible values of this bit are:

0

The PADDR field is a valid physical address. That is, it matches the programmers' view of the physical address for this recorded location.

1

The PADDR field might not be a valid physical address, and might not match the programmers' view of the physical address for the recorded location.

It is **IMPLEMENTATION DEFINED** whether this bit is read-only or read/write.

This bit resets to an **UNKNOWN** value on a Cold reset.

Bit [28]

Reserved, **RES0**.

Bits [27:24]

Reserved, **RES0**.

PADDR, bits [23:0]

Address, bits [55:32]. Unimplemented bits are **RES0**.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.82 ERRCTRLn, Error Record Control Register, n = 0 - 55

The ERRCTRL{0..55} characteristics are:

Purpose

The error control register contains enable bits for the node that writes to this record.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is **RES0** if RAS is not implemented.

Attributes

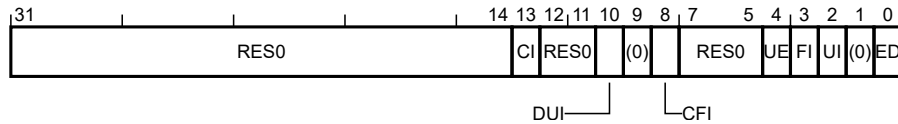
32-bit read/write register located at $0xE0005008 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The ERRCTRL{0..55} bit assignments are:



Bits [31:14]

Reserved, **RES0**.

CI, bit [13]

Critical error interrupt enable. When enabled the critical error interrupt is generated for a critical error condition.

The possible values of this bit are:

0

Critical error interrupt not generated for critical errors. Critical errors are treated as Uncontained errors.

1

Critical error interrupt generated for critical errors.

This bit is **RES0** if the node does not support this control.

This bit resets to an **UNKNOWN** value on a Cold reset.

Bits [12:11]

Reserved, **RES0**.

DUI, bit [10]

Enable error recovery interrupt enable for deferred errors. When enabled the error recovery interrupt is generated for all detected Deferred errors.

The possible values of this bit are:

0
Error recovery interrupt not generated for deferred errors.

1
Error recovery interrupt generated for deferred errors.

The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error. This bit is **RES0** if the node does not support this control.

This bit resets to an **UNKNOWN** value on a Cold reset.

Bit [9]
Reserved, **RES0**.

CFI, bit [8]
Enable fault handling interrupt for corrected errors. When enabled, if the node implements a Corrected error counter, then the fault handling interrupt is generated when the counter overflows and the overflow bit is set. Otherwise the fault handling interrupt is also generated for all detected Corrected errors.

The possible values of this bit are:

0
Fault handling interrupt not generated for corrected errors.

1
Fault handling interrupt generated for corrected errors.

The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error. This bit is **RES0** if the node does not support this control.

This bit resets to an **UNKNOWN** value on a Cold reset.

Bits [7:5]
Reserved, **RES0**.

UE, bit [4]
Enable in-band uncorrected error reporting. When enabled, responses to transactions that detect an uncorrected error that cannot be deferred are signaled as a detected error (external abort).

The possible values of this bit are:

0
External abort response for uncorrected errors disabled.

1
External abort response for uncorrected errors enabled.

This bit is **RES0** if the node does not support this control.

This bit resets to an **UNKNOWN** value on a Cold reset.

FI, bit [3]
Enable fault handling interrupt.

When enabled, the fault handling interrupt is generated for all detected Deferred errors and Uncorrected errors.

If the fault handling interrupt for corrected errors control is not implemented then:

- If the node implements a Corrected error counter then the fault handling interrupt is also generated when the counter overflows and the overflow bit is set.
- Otherwise the fault handling interrupt is also generated for all detected Corrected errors.

The possible values of this bit are:

0
Fault handling interrupt disabled.

1
Fault handling interrupt enabled.

This bit is **RES0** if the node does not support this control.

This bit resets to an **UNKNOWN** value on a Cold reset.

UI, bit [2]

Enable error recovery interrupt. Uncorrected error recovery interrupt enable. When enabled, the error recovery interrupt is generated for all detected Uncorrected errors that are not deferred.

The possible values of this bit are:

0
Error recovery interrupt disabled.

1
Error recovery interrupt enabled.

The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error. This bit is **RES0** if the node does not support this control.

This bit resets to an **UNKNOWN** value on a Cold reset.

Bit [1]

Reserved for **IMPLEMENTATION DEFINED** controls. Must permit SBZP write policy for software. This bit reads as an **IMPLEMENTATION DEFINED** value and writes to this bit have **IMPLEMENTATION DEFINED** behavior.

Reserved, **RES0**.

ED, bit [0]

Error reporting and logging enable. When disabled, the node behaves as if error detection and correction are disabled, and no errors are recorded or signaled by the node. Arm recommends that, when disabled, correct error detection and correction codes are written for writes, unless disabled by an **IMPLEMENTATION DEFINED** control for error injection.

The possible values of this bit are:

0
Error reporting disabled.

1
Error reporting enabled.

It is **IMPLEMENTATION DEFINED** whether the node fully disables error detection and correction when reporting is disabled. That is, even with error reporting disabled, the node might continue to silently correct errors. Uncorrectable errors might result in corrupt data being silently propagated by the node. This bit is **RES1** if the node does not support this control.

This bit resets to zero on a Cold reset.

D1.2.83 ERRDEVID, Error Record Device ID Register

The ERRDEVID characteristics are:

Purpose

Defines the number of error records that can be accessed through the Memory-mapped registers.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is **RES0** if RAS is not implemented.

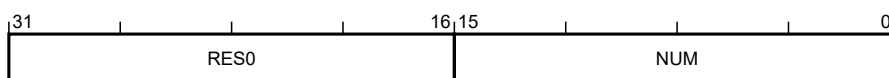
Attributes

32-bit read-only register located at 0xE0005FC8.

This register is not banked between Security states.

Field descriptions

The ERRDEVID bit assignments are:



Bits [31:16]

Reserved, **RES0**.

NUM, bits [15:0]

Number of implemented error record indexes.

Highest numbered index of the error records in this group, plus one. Each implemented record is owned by a node. A node might own multiple records.

This manual describes the memory-mapped view of a group with up to 56 records, the most that can be contained in a 4KB component, meaning the highest value for this field is 56.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.84 ERRFRn, Error Record Feature Register, n = 0 - 55

The ERRFR{0..55} characteristics are:

Purpose

Identifies the features implemented by the associated record n , and of those features that are software programmable.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

If AIRCR.BFHFNMIN is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is **RES0** if RAS is not implemented.

Attributes

32-bit read-only register located at $0xE0005000 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The ERRFR{0..55} bit assignments are:

31																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																												
----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Bits [31:24]

Reserved, RES0.

CI, bits [23:22]

Critical Error Interrupt. Indicates whether the critical error interrupt and associated controls are implemented.

The possible values of this field are:

0b00

Does not support feature.

0b01

Feature always enabled.

0b10

Feature is controllable.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [21:20]

Reserved, RES0.

CEO, bits [19:18]

Corrected Error overwrite. Indicates the behavior when a second Corrected error is detected after a first Corrected error has been recorded by the node.

The possible values of this field are:

0b00

Count Corrected error if a counter is implemented. Keep the previous error syndrome. If the counter overflows, or no counter is implemented then ERRSTATUS<n>.OF is set to 1.

0b01

Count Corrected error. If ERRSTATUS<n>.OF == 1 before the Corrected error is counted then keep the previous syndrome. Otherwise the previous syndrome is overwritten. If the counter overflows then ERRSTATUS<n>.OF is set to 1.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

DUI, bits [17:16]

Error recovery interrupt for deferred errors. Indicates whether the node implements a control for enabling error recovery interrupts on deferred errors.

The possible values of this field are:

0b00

Does not support feature. ERRCTL<n>.DUI is **RES0**.

0b10

Feature is controllable using ERRCTL<n>.DUI.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

RP, bit [15]

Repeat counter. Indicates whether the node implements a repeat Corrected error counter in ERRMISC0<n>.

The possible values of this bit are:

0b0

A single CE counter is implemented.

0b1

A first (repeat) counter and a second (other) counter are implemented. The repeat counter is the same size as the primary error counter.

This bit is **RES0** if ERRFR<n>.CEC == 0b000.

This bit reads as an **IMPLEMENTATION DEFINED** value.

CEC, bits [14:12]

Corrected Error Counter. Indicates whether the node implements a standard Corrected error (CE) counter mechanism in ERRMISC0<n>.

The possible values of this field are:

0b000

Does not implement the standard Corrected error counter model.

0b010

Implements an 8-bit Corrected error counter in ERRMISC0<n>[7:0].

0b100

Implements a 16-bit Corrected error counter in ERRMISC0<n>[15:0].

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

CFI, bits [11:10]

Fault handling for Corrected errors. Indicates whether the node implements a control for enabling fault handling interrupts on Corrected errors.

The possible values of this field are:

0b00

Does not support feature, ERRCTLR<n>.CFI is [RES0](#).

0b10

Feature is controllable using ERRCTLR<n>.CFI.

All other values are reserved.

This bit is '[RES0](#)' if ERRFR<n>.FI == 0b00.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

UE, bits [9:8]

In-band uncorrected error reporting. Indicates whether the node implements in-band uncorrected error reporting (external aborts), and, if so, whether it implements controls for enabling and disabling in-band uncorrected error reporting.

The possible values of this field are:

0b00

Does not support feature, ERRCTLR<n>.UE is [RES0](#).

0b01

Feature always enabled, ERRCTLR<n>.UE is [RES0](#).

0b10

Feature is controllable using ERRCTLR<n>.UE.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

FI, bits [7:6]

Fault handling interrupt. Indicates whether the node implements a fault handling interrupt, and, if so, whether it implements controls for enabling and disabling the fault handling interrupt.

The possible values of this field are:

0b00

Does not support feature, ERRCTLR<n>.FI is [RES0](#).

0b01

Feature always enabled, ERRCTLR<n>.FI is [RES0](#).

0b10

Feature is controllable using ERRCTLR<n>.FI.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

UI, bits [5:4]

Error recovery interrupt for uncorrected errors. Indicates whether the node implements an error recovery interrupt, and, if so, whether it implements controls for enabling and disabling the error recovery interrupt.

The possible values of this field are:

0b00

Does not support feature, ERRCTLR<n>.UI is [RES0](#).

0b01

Feature always enabled, ERRCTLR<n>.UI is [RES0](#).

0b10

Feature is controllable using ERRCTLR<n>.UI.

This field reads as an IMPLEMENTATION DEFINED value.

Bits [3:2]

Reserved, RES0.

ED, bits [1:0]

Error reporting and logging. Indicates whether the node implements controls for enabling and disabling error reporting and logging.

The possible values of this field are:

0b01

Feature always enabled, ERRCTLR<n>.ED is RES1.

0b10

Feature is controllable using ERRCTLR<n>.ED.

All other values are reserved.

This field reads as an IMPLEMENTATION DEFINED value.

D1.2.85 ERRGSRn, RAS Fault Group Status Register

The ERRGSR{0..1} characteristics are:

Purpose

Each ERRGSR<n> register shows the status for up to 32 records in the group. <n> selects the set of 32 records from the records in the group.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

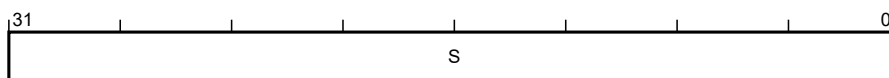
Attributes

32-bit read-only register located at $0 \times E0005E00 + 4n$.

This register is not banked between Security states.

Field descriptions

The ERRGSR{0..1} bit assignments are:



S, bits [31:0]

Status for Error Record <m>. Each bit is a read-only copy of the corresponding ERRSTATUS<m>.V field.

The possible values of this field are:

0

No error.

1

One or more errors.

This bit is RES0 if <m> is greater than or equal to the number of implemented records, or if the record does not support this type of reporting.

D1.2.86 ERRIIDR, Error Implementer ID Register

The ERRIIDR characteristics are:

Purpose

Defines the implementer of the component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is **RES0** if RAS is not implemented.

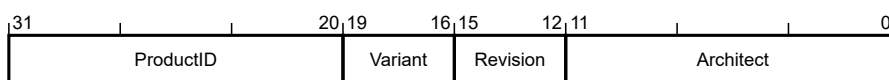
Attributes

32-bit read-only register located at 0xE0005E10.

This register is not banked between Security states.

Field descriptions

The ERRIIDR bit assignments are:



ProductID, bits [31:20]

This field reads as an **IMPLEMENTATION DEFINED** value.

Variant, bits [19:16]

Component Major Revision. This field distinguishes between variants or major revisions of the product.

This field reads as an **IMPLEMENTATION DEFINED** value.

Revision, bits [15:12]

Component minor revision. This field distinguishes between minor revisions of the product.

This field reads as an **IMPLEMENTATION DEFINED** value.

Architect, bits [11:0]

The possible values of this field are:

0x43B

JEP106 continuation code 0x4, ID code 0x3B. Arm Limited.

Other values are defined by the JEDEC JEP106 standard.

This field reads as 0x43B.

D1.2.87 ERRMISC0n, Error Record Miscellaneous 0 Register, n = 0 - 55

The ERRMISC0{0..55} characteristics are:

Purpose

IMPLEMENTATION DEFINED error syndrome register.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

Attributes

32-bit read/write register located at $0xE0005020 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

The miscellaneous syndrome registers contain:

- Corrected error counter or counters, if the node supports the counting of Corrected errors.
- Information to identify the Field Replaceable Unit (FRU) in which the error was detected, and might contain enough information to locate error within that FRU.
- Other state information not present in the corresponding status and address registers.

If the node supports the architecturally-defined error counter then it is implemented in ERRMISC0<n>.

If ERRSTATUS<n>.MV is set to 1 then it is IMPLEMENTATION DEFINED whether fields of ERRMISC0<n> ignores writes. Arm recommends that miscellaneous syndrome for multiple errors, such as a Corrected error counter, is read/write. This allows a counter to be reset in the presence of a persistent error. Miscellaneous syndrome for the most recently recorded error, such as information locating a FRU for that error, must ignore writes. This prevents the loss of information if an error is detected whilst the previous error is being logged.

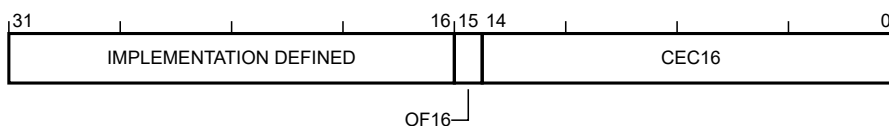
Field descriptions

The ERRMISC0{0..55} bit assignments are:

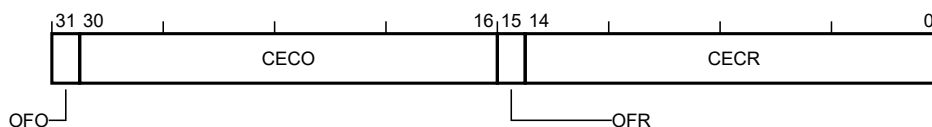
When Contents are IMPLEMENTATION DEFINED:



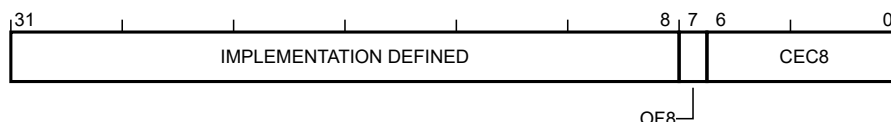
When Standard 16-bit CE counter:



When Standard 16-bit CE counter pair:



When Standard 8-bit CE counter:



Bits [31:0], when Contents are IMPLEMENTATION DEFINED

IMPLEMENTATION DEFINED.

Bits [31:8], when Standard 8-bit CE counter

IMPLEMENTATION DEFINED.

OF8, bit [7], when Standard 8-bit CE counter

Overflow. Sticky overflow bit.

The possible values of this bit are:

0

Counter has not overflowed.

1

Counter has overflowed.

Set to 1 when the Corrected error count field is incremented and wraps through zero.

A direct write that modifies this bit might indirectly set ERRSTATUS<n>.OF to an UNKNOWN value and a direct write to ERRSTATUS<n>.OF that clears it to zero might indirectly set this bit to an UNKNOWN value.

This bit resets to an UNKNOWN value on a Cold reset.

CEC8, bits [6:0], when Standard 8-bit CE counter

Corrected error count.

This field resets to an UNKNOWN value on a Cold reset.

Bits [31:16], when Standard 16-bit CE counter

IMPLEMENTATION DEFINED.

OF16, bit [15], when Standard 16-bit CE counter

Overflow. Sticky overflow bit.

The possible values of this bit are:

0

Counter has not overflowed.

1

Counter has overflowed.

Set to 1 when the Corrected error count field is incremented and wraps through zero.

A direct write that modifies this bit might indirectly set ERRSTATUS<n>.OF to an UNKNOWN value and a direct write to ERRSTATUS<n>.OF that clears it to zero might indirectly set this bit to an UNKNOWN value.

This bit resets to an UNKNOWN value on a Cold reset.

CEC16, bits [14:0], when Standard 16-bit CE counter

Corrected error count.

This field resets to an **UNKNOWN** value on a Cold reset.

OFO, bit [31], when Standard 16-bit CE counter pair

Overflow Other. Sticky overflow bit, other.

The possible values of this bit are:

0

Other Counter has not overflowed.

1

Other Counter has overflowed.

Set to 1 when the Corrected error count, other, field is incremented and wraps through zero.

A direct write that modifies this bit might indirectly set ERRSTATUS<n>.OF to an **UNKNOWN** value and a direct write to ERRSTATUS<n>.OF that clears it to zero might indirectly set this bit to an **UNKNOWN** value.

This bit resets to an **UNKNOWN** value on a Cold reset.

CECO, bits [30:16], when Standard 16-bit CE counter pair

Corrected error count, other. Incremented for each countable error that is not accounted for by incrementing CECR.

This field resets to an **UNKNOWN** value on a Cold reset.

OFR, bit [15], when Standard 16-bit CE counter pair

Overflow Repeat. Sticky overflow bit, repeat.

The possible values of this bit are:

0

Repeat Counter has not overflowed.

1

Repeat Counter has overflowed.

Set to 1 when the Corrected error count, repeat, field is incremented and wraps through zero.

A direct write that modifies this bit might indirectly set ERRSTATUS<n>.OF to an **UNKNOWN** value and a direct write to ERRSTATUS<n>.OF that clears it to zero might indirectly set this bit to an **UNKNOWN** value.

This bit resets to an **UNKNOWN** value on a Cold reset.

CECR, bits [14:0], when Standard 16-bit CE counter pair

Corrected error count. Incremented for the first detected countable error, which also records other syndrome for the error, and subsequently for each countable error that matches the recorded other syndrome. Corrected errors are countable errors. It is **IMPLEMENTATION DEFINED** and might be **UNPREDICTABLE** whether Deferred and Uncorrected errors are countable errors.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.88 ERRMISC1n, Error Record Miscellaneous 1 Register, n = 0 - 55

The ERRMISC1{0..55} characteristics are:

Purpose

IMPLEMENTATION DEFINED error syndrome register.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

Attributes

32-bit read/write register located at $0xE0005024 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

The miscellaneous syndrome registers contain:

- Corrected error counter or counters, if the node supports the counting of Corrected errors.
- Information to identify the Field Replaceable Unit (FRU) in which the error was detected, and might contain enough information to locate error within that FRU.
- Other state information not present in the corresponding status and address registers.

If ERRSTATUS<n>.MV is set to 1 then it is IMPLEMENTATION DEFINED whether fields of ERRMISC1<n> ignores writes. Arm recommends that miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write. This allows a counter to be reset in the presence of a persistent error. Miscellaneous syndrome for the most recently recorded error, such as information locating a FRU for that error, should ignore writes. This prevents information being lost if an error is detected whilst the previous error is being logged.

Field descriptions

The ERRMISC1{0..55} bit assignments are:



Bits [31:0]

IMPLEMENTATION DEFINED.

D1.2.89 ERRMISC2n, Error Record Miscellaneous 2 Register, n = 0 - 55

The ERRMISC2{0..55} characteristics are:

Purpose

IMPLEMENTATION DEFINED error syndrome register.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

Attributes

32-bit read/write register located at $0xE0005028 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

The miscellaneous syndrome registers contain:

- Corrected error counter or counters, if the node supports the counting of Corrected errors.
- Information to identify the Field Replaceable Unit (FRU) in which the error was detected, and might contain enough information to locate error within that FRU.
- Other state information not present in the corresponding status and address registers.

If ERRSTATUS<n>.MV is set to 1 then it is IMPLEMENTATION DEFINED whether fields of ERRMISC2<n> ignores writes. Arm recommends that miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write. This allows a counter to be reset in the presence of a persistent error. Miscellaneous syndrome for the most recently recorded error, such as information locating a FRU for that error, should ignore writes. This prevents information being lost if an error is detected whilst the previous error is being logged.

Field descriptions

The ERRMISC2{0..55} bit assignments are:



Bits [31:0]

IMPLEMENTATION DEFINED.

D1.2.90 ERRMISC3n, Error Record Miscellaneous 3 Register, n = 0 - 55

The ERRMISC3{0..55} characteristics are:

Purpose

IMPLEMENTATION DEFINED error syndrome register.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

Attributes

32-bit read/write register located at $0xE000502C + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

The miscellaneous syndrome registers contain:

- Corrected error counter or counters, if the node supports the counting of Corrected errors.
- Information to identify the Field Replaceable Unit (FRU) in which the error was detected, and might contain enough information to locate error within that FRU.
- Other state information not present in the corresponding status and address registers.

If ERRSTATUS<n>.MV is set to 1 then it is IMPLEMENTATION DEFINED whether fields of ERRMISC3<n> ignores writes. Arm recommends that miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write. This allows a counter to be reset in the presence of a persistent error. Miscellaneous syndrome for the most recently recorded error, such as information locating a FRU for that error, should ignore writes. This prevents information being lost if an error is detected whilst the previous error is being logged.

Field descriptions

The ERRMISC3{0..55} bit assignments are:



Bits [31:0]

IMPLEMENTATION DEFINED.

D1.2.91 ERRMISC4n, Error Record Miscellaneous 4 Register, n = 0 - 55

The ERRMISC4{0..55} characteristics are:

Purpose

IMPLEMENTATION DEFINED error syndrome register.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

Attributes

32-bit read/write register located at $0xE0005030 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

The miscellaneous syndrome registers contain:

- Corrected error counter or counters, if the node supports the counting of Corrected errors.
- Information to identify the Field Replaceable Unit (FRU) in which the error was detected, and might contain enough information to locate error within that FRU.
- Other state information not present in the corresponding status and address registers.

If ERRSTATUS<n>.MV is set to 1 then it is IMPLEMENTATION DEFINED whether fields of ERRMISC4<n> ignores writes. Arm recommends that miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write. This allows a counter to be reset in the presence of a persistent error. Miscellaneous syndrome for the most recently recorded error, such as information locating a FRU for that error, should ignore writes. This prevents information being lost if an error is detected whilst the previous error is being logged.

Field descriptions

The ERRMISC4{0..55} bit assignments are:



Bits [31:0]

IMPLEMENTATION DEFINED.

D1.2.92 ERRMISC5n, Error Record Miscellaneous 5 Register, n = 0 - 55

The ERRMISC5{0..55} characteristics are:

Purpose

IMPLEMENTATION DEFINED error syndrome register.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

Attributes

32-bit read/write register located at $0xE0005034 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

The miscellaneous syndrome registers contain:

- Corrected error counter or counters, if the node supports the counting of Corrected errors.
- Information to identify the Field Replaceable Unit (FRU) in which the error was detected, and might contain enough information to locate error within that FRU.
- Other state information not present in the corresponding status and address registers.

If ERRSTATUS<n>.MV is set to 1 then it is IMPLEMENTATION DEFINED whether fields of ERRMISC5<n> ignores writes. Arm recommends that miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write. This allows a counter to be reset in the presence of a persistent error. Miscellaneous syndrome for the most recently recorded error, such as information locating a FRU for that error, should ignore writes. This prevents information being lost if an error is detected whilst the previous error is being logged.

Field descriptions

The ERRMISC5{0..55} bit assignments are:



Bits [31:0]

IMPLEMENTATION DEFINED.

D1.2.93 ERRMISC6n, Error Record Miscellaneous 6 Register, n = 0 - 55

The ERRMISC6{0..55} characteristics are:

Purpose

IMPLEMENTATION DEFINED error syndrome register.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

Attributes

32-bit read/write register located at $0xE0005038 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

The miscellaneous syndrome registers contain:

- Corrected error counter or counters, if the node supports the counting of Corrected errors.
- Information to identify the Field Replaceable Unit (FRU) in which the error was detected, and might contain enough information to locate error within that FRU.
- Other state information not present in the corresponding status and address registers.

If ERRSTATUS<n>.MV is set to 1 then it is IMPLEMENTATION DEFINED whether fields of ERRMISC6<n> ignores writes. Arm recommends that miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write. This allows a counter to be reset in the presence of a persistent error. Miscellaneous syndrome for the most recently recorded error, such as information locating a FRU for that error, should ignore writes. This prevents information being lost if an error is detected whilst the previous error is being logged.

Field descriptions

The ERRMISC6{0..55} bit assignments are:



Bits [31:0]

IMPLEMENTATION DEFINED.

D1.2.94 ERRMISC7n, Error Record Miscellaneous 7 Register, n = 0 - 55

The ERRMISC7{0..55} characteristics are:

Purpose

IMPLEMENTATION DEFINED error syndrome register.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

Attributes

32-bit read/write register located at $0xE000503C + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

The miscellaneous syndrome registers contain:

- Corrected error counter or counters, if the node supports the counting of Corrected errors.
- Information to identify the Field Replaceable Unit (FRU) in which the error was detected, and might contain enough information to locate error within that FRU.
- Other state information not present in the corresponding status and address registers.

If ERRSTATUS<n>.MV is set to 1 then it is IMPLEMENTATION DEFINED whether fields of ERRMISC7<n> ignores writes. Arm recommends that miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write. This allows a counter to be reset in the presence of a persistent error. Miscellaneous syndrome for the most recently recorded error, such as information locating a FRU for that error, should ignore writes. This prevents information being lost if an error is detected whilst the previous error is being logged.

Field descriptions

The ERRMISC7{0..55} bit assignments are:



Bits [31:0]

IMPLEMENTATION DEFINED.

D1.2.95 ERRSTATUS_n, Error Record Primary Status Register, n = 0 - 55

The ERRSTATUS{0..55} characteristics are:

Purpose

Contains status information for the error record.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is RES0 if RAS is not implemented.

Attributes

32-bit read/write register located at $0xE0005010 + 64n$.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

This register contains the following information:

- Whether any error has been detected (valid).
- Whether any detected error was not corrected, and returned to a Requester.
- Whether any detected error was not corrected and deferred.
- Whether an error record has been discarded because additional errors have been detected before the first error was handled by software (overflow).
- Whether any error has been reported.
- Whether the other error record registers contain valid information.
- Whether the error was reported because poison data was detected or because a corrupt value was detected by an error detection code.
- A primary error code.
- An IMPLEMENTATION DEFINED extended error code.

Within this register:

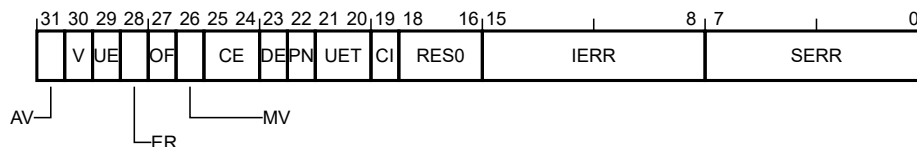
- The {AV, V, MV} bits are valid bits that define whether the error record registers are valid.
- The {UE, OF, CE, DE, UET} bits encode the type of error or errors recorded.
- The {CI, ER, PN, IERR, SERR} fields are syndrome fields.

After reading the status register, software must clear the valid bits to allow new errors to be recorded. Between reading the register and clearing the valid bits, a new error might have overwritten the register. To prevent this error being lost, a write to ERRSTATUS<n> is ignored if all of:

- Any of the ERRSTATUS<n>.{V, UE, OF, CE, DE} fields are nonzero before the write.
- The write does not clear the nonzero ERRSTATUS<n>.{V, UE, OF, CE, DE} field(s) to zero by writing one(s) to the applicable field(s).

Field descriptions

The ERRSTATUS{0..55} bit assignments are:



AV, bit [31]

Address Valid.

The possible values of this bit are:

0

ERRADDR<n> and ERRADDR2<n> not valid.

1

ERRADDR<n> and ERRADDR2<n> contain an address associated with the highest priority error recorded by this record.

This bit is read/write-one-to-clear.

This bit resets to zero on a Cold reset.

V, bit [30]

Status Register valid.

The possible values of this bit are:

0

ERRSTATUS<n> not valid.

1

ERRSTATUS<n> valid. At least one error has been recorded.

This bit is read/write-one-to-clear.

This bit resets to zero on a Cold reset.

UE, bit [29]

Uncorrected error or errors.

The possible values of this bit are:

0

No errors that could neither be corrected nor deferred.

1

At least one error that has neither been corrected nor deferred.

This bit reads as **UNKNOWN** if ERRSTATUS<n>.V is set to 0. This bit is read/write-one-to-clear.

This bit resets to an **UNKNOWN** value on a Cold reset.

ER, bit [28]

Error Reported.

The possible values of this bit are:

0

No in-band error (external abort) reported.

1

An external abort was signaled by the node to the Requester making the access or other transaction. This can be because any of:

- ERRCTL<n>.UE is implemented and was set to 1 when an Uncorrected error was detected.
- ERRCTL<n>.UE is not implemented and the node always reports errors.

It is **IMPLEMENTATION DEFINED** whether this bit can be set to 1 by a Deferred error. This bit is not valid and reads **UNKNOWN** if any of:

- ERRSTATUS<n>.UE is set to 0 and this bit is only set to 1 by Uncorrected errors.
- ERRSTATUS<n>.{UE, DE} are both set to 0 and this bit can be set to 1 by Deferred errors.
- ERRSTATUS<n>.V is set to 0.

This bit is read/write-one-to-clear.

This bit resets to an **UNKNOWN** value on a Cold reset.

OF, bit [27]

Overflow.

Indicates that multiple errors have been detected. This bit is set to 1 when one of the following occurs:

- A corrected error counter is implemented, an error is counted, and the counter overflows.
- A corrected error counter is not implemented, a corrected error is recorded, and ERRSTATUS<n>.V was previously set to 1.

A type of error other than a corrected error is recorded and ERRSTATUS<n>.V was previously set to 1.

Otherwise, this bit is unchanged when an error is recorded.

If a corrected error counter is implemented:

- A direct write that modifies the counter overflow flag indirectly might set this bit to an **UNKNOWN** value.
- A direct write to this bit that clears this bit to zero might indirectly set the counter overflow flag to an **UNKNOWN** value.

The possible values of this bit are:

0

No error syndrome has been discarded and, if a Corrected error counter is implemented, it has not overflowed since this bit was last cleared to zero.

1

At least one error syndrome has been discarded or, if a Corrected error counter is implemented, it might have overflowed, since this bit was last cleared to zero.

This bit is not valid and reads **UNKNOWN** if ERRSTATUS<n>.V is set to 0.

This bit is read/write-one-to-clear.

This bit resets to an **UNKNOWN** value on a Cold reset.

MV, bit [26]

Miscellaneous Registers Valid.

The possible values of this bit are:

0

ERRMISC*<n> not valid.

1

The **IMPLEMENTATION DEFINED** contents of the **ERRMISC* $\langle n \rangle$** registers contains additional information for an error recorded by this record.

This bit is not valid and reads **UNKNOWN** if **ERRSTATUS $\langle n \rangle$.V** is set to 0.

This bit is read/write-one-to-clear.

This bit resets to an **UNKNOWN** value on a Cold reset.

CE, bits [25:24]

Corrected error or errors.

The possible values of this field are:

0b00

No errors were corrected.

0b01

At least one transient error was corrected.

0b10

At least one error was corrected.

0b11

At least one persistent error was corrected.

The mechanism by which a node detects whether a correctable error is transient or persistent is **IMPLEMENTATION DEFINED**. If no such mechanism is implemented then the node sets this field to **0b10** when an error is corrected.

This field is not valid and reads **UNKNOWN** if **ERRSTATUS $\langle n \rangle$.V** is set to 0.

This field is read/write-ones-to-clear. Writing a value other than all-zeros or all-ones sets this field to an **UNKNOWN** value.

This field resets to an **UNKNOWN** value on a Cold reset.

DE, bit [23]

Deferred error or errors.

The possible values of this bit are:

0

No errors were deferred.

1

At least one error was not corrected and deferred.

Support for deferring errors is **IMPLEMENTATION DEFINED**.

This bit is not valid and reads **UNKNOWN** if **ERRSTATUS $\langle n \rangle$.V** is set to 0.

This bit is read/write-one-to-clear.

This bit resets to an **UNKNOWN** value on a Cold reset.

PN, bit [22]

Poison.

The possible values of this bit are:

0

Uncorrected or deferred error from a corrupted value.

1

Uncorrected error or Deferred error from a poisoned value. Indicates that an error occurred because of the detection of a poison value rather because of the detection of a corrupted value.

It is **IMPLEMENTATION DEFINED** whether a node can distinguish a poisoned value from a corrupted value.

This bit is not valid and reads **UNKNOWN** if any of:

- ERRSTATUS<n>.V is set to 0.
- ERRSTATUS<n>.{CE, UE} are both set to 0.

This bit is read/write-one-to-clear.

This bit resets to an **UNKNOWN** value on a Cold reset.

Note

If a node detects a corrupted value and defers the error by producing poison then this bit is set to 0b0 at the producer node. The value 0b1 might only be an indication of a poisoned value. As in some EDC schemes, it is possible to mistake a corrupted value for a poisoned value.

UET, bits [21:20]

Uncorrected Error Type. Describes the state of the component after detecting or consuming an Uncorrected error.

The possible values of this field are:

0b00

Uncorrected error, Uncontainable error (UC).

0b01

Uncorrected error, Unrecoverable error (UEU).

0b10

Uncorrected error, Latent or Restartable error (UEO).

0b11

Uncorrected error, Signaled or Recoverable error (UER).

This field is not valid and reads **UNKNOWN** if any of:

- ERRSTATUS<n>.V is set to 0.
- ERRSTATUS<n>.UE is set to 0.

This field is read/write-ones-to-clear. Writing a value other than all-zeros or all-ones sets this field to an **UNKNOWN** value.

Note: Software might use the information in the error record registers to determine what recovery is necessary.

This field resets to an **UNKNOWN** value on a Cold reset.

CI, bit [19]

Critical error. Indicates whether a critical error condition has been recorded.

The possible values of this bit are:

0

No critical error condition.

1

Critical error condition.

This bit is not valid and reads **UNKNOWN** if ERRSTATUS<n>.V is set to 0.

This bit is read/write-one-to-clear.

This bit resets to an **UNKNOWN** value on a Cold reset.

Bits [18:16]

Reserved, **RES0**.

IERR, bits [15:8]

IMPLEMENTATION DEFINED error code.

Used with any primary error code **SERR** value. Further **IMPLEMENTATION DEFINED** information can be placed in the **MISC** registers.

The subset of architecturally-defined values that this field can take is **IMPLEMENTATION DEFINED**. If any value not in this set is written to this register then the value read back from this field is **UNKNOWN**.

This field is not valid and reads **UNKNOWN** if **ERRSTATUS**<*n*>.V is set to 0.

This field resets to an **UNKNOWN** value on a Cold reset.

Note

One or more bits of this field might be implemented as fixed read-as-zero or read-as-one values.

SERR, bits [7:0]

Architecturally-defined primary error code. The primary error code might be used by a fault handling agent to triage an error without requiring device-specific code. For example, to count and threshold corrected errors in software, or generate a short log entry.

The possible values of this field are:

0

No error.

1

IMPLEMENTATION DEFINED error.

2

Data value from (non-associative) internal memory. For example, ECC from on-chip SRAM or buffer.

3

IMPLEMENTATION DEFINED pin.

4

Assertion failure. For example, consistency failure.

5

Internal data path. For example, parity on ALU result.

6

Data value from associative memory. For example, ECC error on cache data.

7

Address/control value or values from associative memory. For example, ECC error on cache tag.

10

Data value from producer. For example, parity error on write data bus.

11

Address/control value or values from producer. For example, parity error on address bus.

12

Data value from (non-associative) external memory. For example, ECC error in SDRAM.

13

Illegal address (software fault). For example, access to unpopulated memory.

- 14** Illegal access (software fault). For example, byte write to word register.
- 15** Illegal state (software fault). For example, device not ready.
- 16** Internal data register. For example, parity on a floating-point or MVE register. For a PE, all general-purpose, stack pointer, and floating-point or MVE registers are data registers.
- 17** Internal control register. For example, parity on a system register. For a PE, all registers other than general-purpose, stack pointer, and floating-point or MVE registers are control registers.
- 18** Error response from Completer of access. For example, error response from cache write-back.
- 19** External timeout. For example, timeout on interaction with another node.
- 20** Internal timeout. For example, timeout on interface within the node.
- 21** Deferred error from Completer not supported at Requester. For example, poisoned data received from a Completer of an access by a Requester that cannot defer the error further.

All other values are reserved.

The subset of architecturally-defined values that this field can take is **IMPLEMENTATION DEFINED**. If any value not in this set is written to this register then the value read back from this field is **UNKNOWN**.

This field is not valid and reads **UNKNOWN** if **ERRSTATUS<n>.V** is set to 0.

Note: one or more bits of this field might be implemented as fixed read-as-zero or read-as-one values.

This field resets to an **UNKNOWN** value on a Cold reset.

D1.2.96 EXC_RETURN, Exception Return Payload

The EXC_RETURN characteristics are:

Purpose
Value provided in LR on entry to an exception, and used with a BX or load to PC to perform an exception return.

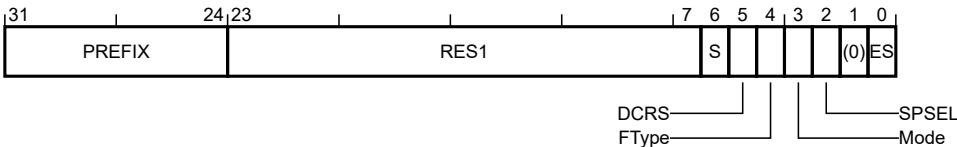
Usage constraints
None.

Configurations
All.

Attributes
32-bit payload.

Field descriptions

The EXC_RETURN bit assignments are:



PREFIX, bits [31:24]
Prefix. Indicates that this is an EXC_RETURN value.
This field reads as 0b11111111.

Bits [23:7]
Reserved, [RES1](#).

S, bit [6]
Secure or Non-secure stack. Indicates whether a Secure or Non-secure stack is used to restore stack frame on exception return.
The possible values of this bit are:
0
Non-secure stack used.
1
Secure stack used.
Behavior is [UNPREDICTABLE](#) if the Security Extension is not implemented and this field is not zero.
If the Security Extension is not implemented, this bit is [RES0](#).

DCRS, bit [5]
Default callee register stacking. Indicates whether the default stacking rules apply, or whether the Additional state context, also known as callee registers, are already on the stack.
The possible values of this bit are:
0
Stacking of the Additional state context registers skipped.
1
Default rules for stacking the Additional state context registers followed.

Behavior is **UNPREDICTABLE** if the Security Extension is not implemented and this field is not one.

If the Security Extension is not implemented, this bit is **RES1**.

FType, bit [4]

Stack frame type. Indicates whether the stack frame is a standard integer only stack frame or an extended Floating-point stack frame.

The possible values of this bit are:

0

Extended stack frame.

1

Standard stack frame.

Behavior is **UNPREDICTABLE** if neither the Floating-point Extension or MVE are implemented and this field is not one.

If neither the Floating-point Extension or MVE are implemented, this bit is **RES1**.

Mode, bit [3]

Mode. Indicates the Mode that was stacked from.

The possible values of this bit are:

0

Handler mode.

1

Thread mode.

SPSEL, bit [2]

Stack pointer selection. The value of this bit indicates the transitory value of the CONTROL.SPSEL bit associated with the Security state of the exception as indicated by EXC_RETURN.ES.

The possible values of this bit are:

0

Main stack pointer.

1

Process stack pointer.

Bit [1]

Reserved, **RES0**.

ES, bit [0]

Exception Secure. The security domain the exception was taken to.

The possible values of this bit are:

0

Non-secure.

1

Secure.

Behavior is **UNPREDICTABLE** if the Security Extension is not implemented and this field is not zero.

If the Security Extension is not implemented, this bit is **RES0**.

D1.2.97 FAULTMASK, Fault Mask Register

The FAULTMASK characteristics are:

Purpose

Provides access to the PE FAULTMASK register.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

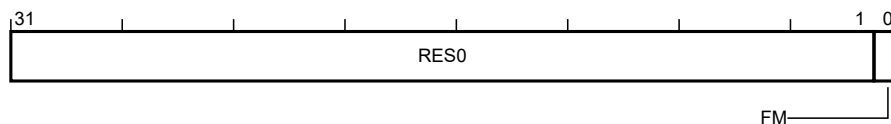
Attributes

32-bit read/write special-purpose register.

This register is banked between Security states.

Field descriptions

The FAULTMASK bit assignments are:

**Bits [31:1]**

Reserved, RES0.

FM, bit [0]

Fault mask enable.

The Secure and Non-secure FAULTMASK registers individually boost the current execution priority based on the settings of AIRCR.PRIS and AIRCR.BFHFNMINS.

If AIRCR.BFHFNMINS is zero, AIRCR.PRIS is zero, and FAULTMASK_NS.FM is one, the execution priority is boosted to 0.

If AIRCR.BFHFNMINS is zero, AIRCR.PRIS is one, and FAULTMASK_NS.FM is one, the execution priority is boosted to 0×80 .

If AIRCR.BFHFNMINS is zero and FAULTMASK_S is one, the execution priority is boosted to -1.

If AIRCR.BFHFNMINS is one and FAULTMASK_NS is one, the execution priority is boosted to -1.

If AIRCR.BFHFNMINS is one and FAULTMASK_S is one, the execution priority is boosted to -3.

The possible values of this bit are:

O

No effect.

1

Boost priority.

On an exception return from a raw execution priority greater or equal to zero, the FM bit corresponding to EXC_RETURN.ES is cleared.

This bit resets to zero on a Warm reset.

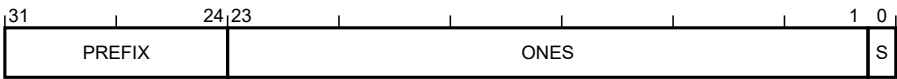
D1.2.98 FNC_RETURN, Function Return Payload

The FNC_RETURN characteristics are:

- Purpose**
Value provided in LR on entry to Non-secure state from a Secure BLXNS.
- Usage constraints**
None.
- Configurations**
All.
- Attributes**
32-bit payload.

Field descriptions

The FNC_RETURN bit assignments are:



- PREFIX, bits [31:24]**
This field reads as 0b11111110.
- ONES, bits [23:1]**
This field reads as 0b111111111111111111111111.
- S, bit [0]**

Secure. Indicates whether the function call was from the Non-secure or Secure state. Because FNC_RETURN is only used when calling from the Secure state, this bit is always set to 1. However, some function chaining cases can result in an SG instruction clearing this bit, so the architecture ignores the state of this bit when processing a branch to FNC_RETURN.

The possible values of this bit are:

- 0**
From Non-secure state.
- 1**
From Secure state.

D1.2.99 FPCAR, Floating-Point Context Address Register

The FPCAR characteristics are:

Purpose

Holds the location of the unpopulated Floating-point register space allocated on an exception stack frame.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present if the Floating-point Extension, or MVE, or both are implemented.

This register is **RES0** if neither the Floating-point Extension nor MVE are implemented.

Attributes

32-bit read/write register located at 0xE000EF38.

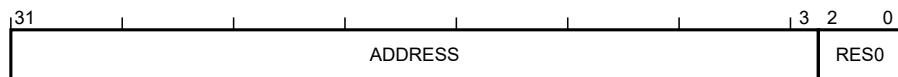
Secure software can access the Non-secure version of this register via FPCAR_NS located at 0xE002EF38. The location 0xE002EF38 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The FPCAR bit assignments are:

**ADDRESS, bits [31:3]**

Address. The location of the unpopulated Floating-point register space allocated on an exception stack frame.

This field resets to an **UNKNOWN** value on a Warm reset.

Bits [2:0]

Reserved, RES0.

D1.2.100 FPCCR, Floating-Point Context Control Register

The FPCCR characteristics are:

Purpose

Holds control data for the Floating Point Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present if the Floating-point Extension, or MVE, or both are implemented.

This register is **RES0** if neither the Floating-point Extension nor MVE are implemented.

Attributes

32-bit read/write register located at 0xE000EF34.

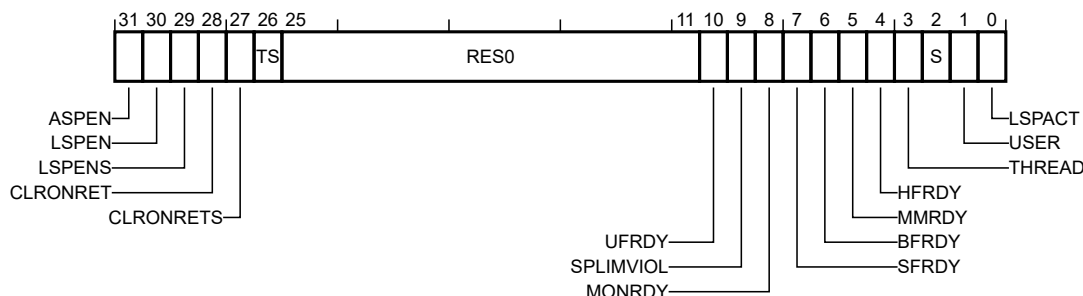
Secure software can access the Non-secure version of this register via FPCCR_NS located at 0xE002EF34. The location 0xE002EF34 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The FPCCR bit assignments are:



ASPEN, bit [31]

Automatic state preservation enable. When this bit is set to 1, execution of a Floating-point instruction sets the CONTROL.FPCA bit to 1.

This bit is banked between Security states.

The possible values of this bit are:

0

Executing a floating-point instruction has no effect on CONTROL.FPCA.

1

Executing a floating-point instruction sets CONTROL.FPCA to 1.

Setting this bit to 1 means the hardware automatically preserves Floating-point context on exception entry and restores it on exception return. As of version Armv8.1-M of the architecture Arm deprecates setting this field to 0.

This bit resets to one on a Warm reset.

LSPEN, bit [30]

Lazy Floating-point state preservation enable. Enables lazy Floating-point state preservation.

This bit is not banked between Security states.

The possible values of this bit are:

0

Disable automatic lazy Floating-point state save.

1

Enable automatic lazy Floating-point state save.

Writes to this bit from Non-secure state are ignored if LSPENS is set to 1.

This bit resets to one on a Warm reset.

LSPENS, bit [29]

Lazy Floating-point state preservation enable Secure. This bit controls whether the LSPEN bit is writable from the Non-secure state. This behaves as RAZ/WI when accessed from the Non-secure state.

This bit is not banked between Security states.

The possible values of this bit are:

0

LSPEN is readable and writable from both Security states.

1

LSPEN is readable from both Security states, but writes to LSPEN are ignored from the Non-secure state.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

CLRONRET, bit [28]

Clear on return. Clear Floating-point context on exception return.

This bit is not banked between Security states.

The possible values of this bit are:

0

Disabled.

1

Enabled.

When set to 1 the Floating-point context registers (S0 to S15, FPSCR, and VPR) are cleared on exception return (including tail chaining) if CONTROL.FPCA is set to 1 and FPCCR_S.LSPACT is set to 0. Writes to this bit from Non-secure state are ignored if CLRONRETS is set to one.

This bit resets to zero on a Warm reset.

CLRONRETS, bit [27]

Clear on return, Secure only. This bit controls whether the CLRONRET bit is writable from the Non-secure state.

This bit is not banked between Security states.

The possible values of this bit are:

0

The CLRONRET field is accessibly from both Security states.

1

The Non-secure view of the CLRONRET field is read-only.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

TS, bit [26]

Treat as Secure. Treat Floating-point registers as Secure enable.

This bit is not banked between Security states.

The possible values of this bit are:

0

Disabled.

1

Enabled.

When set to 0 the Floating-point registers are treated as Non-secure even when the PE is in Secure state and, therefore, the callee saved registers are never pushed to the stack. If the Floating-point registers never contain data that needs to be protected, clearing this flag can reduce interrupt latency. As this field changes how secure stack frames are interpreted, **UNPREDICTABLE** behavior can result if the state of this bit is not consistent with the current Secure stacks. Therefore, firmware must take care when modifying this value. This field behaves as RAZ/WI from the Non-secure state.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

Bits [25:11]

Reserved, **RES0**.

UFRDY, bit [10]

UsageFault enable. Indicates whether the software executing, when the PE allocated the Floating-point stack frame, was able to set the UsageFault exception to pending.

This bit is banked between Security states.

The possible values of this bit are:

0

Not able to set the UsageFault exception to pending.

1

Able to set the UsageFault exception to pending.

This bit resets to an **UNKNOWN** value on a Warm reset.

SPLIMVIOL, bit [9]

Stack pointer limit violation. This bit indicates whether the Floating-point context violates the stack pointer limit that was active when lazy Floating-point state preservation was activated. SPLIMVIOL modifies the lazy Floating-point state preservation behavior.

This bit is banked between Security states.

The possible values of this bit are:

0

The existing behavior is retained.

1

The memory accesses associated with the Floating-point state preservation are not performed. However if the Floating-point state is Secure and FPCCR.TS is set to 1 the registers are still zeroed and the Floating-point state is lost.

This bit resets to an **UNKNOWN** value on a Warm reset.

MONRDY, bit [8]

DebugMonitor ready. Indicates whether the software executing, when the PE allocated the Floating-point stack frame, was able to set the DebugMonitor exception to pending.

This bit is not banked between Security states.

The possible values of this bit are:

0

Not able to set the DebugMonitor exception to pending.

1

Able to set the DebugMonitor exception to pending.

If DEMCR.SDME is one this bit is RAZ/WI from Non-secure state

This bit resets to an **UNKNOWN** value on a Warm reset.

SFRDY, bit [7]

SecureFault ready. Indicates whether the software executing, when the PE allocated the Floating-point stack frame, was able to set the SecureFault exception to pending.

This bit is not banked between Security states.

This bit is RAZ/WI from Non-secure state.

This bit resets to an **UNKNOWN** value on a Warm reset.

BFRDY, bit [6]

BusFault ready. Indicates whether the software executing, when the PE allocated the Floating-point stack frame, was able to set the BusFault exception to pending.

This bit is not banked between Security states.

The possible values of this bit are:

0

Not able to set the BusFault exception to pending.

1

Able to set the BusFault exception to pending.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to an **UNKNOWN** value on a Warm reset.

MMRDY, bit [5]

MemManage ready. Indicates whether the software executing, when the PE allocated the Floating-point stack frame, was able to set the MemManage exception to pending.

This bit is banked between Security states.

The possible values of this bit are:

0

Not able to set the MemManage exception to pending.

1

Able to set the MemManage exception to pending.

This bit resets to an **UNKNOWN** value on a Warm reset.

HFRDY, bit [4]

HardFault ready. Indicates whether the software executing, when the PE allocated the Floating-point stack

frame, was able to set the HardFault exception to pending. If there is both a Secure HardFault and a Non-secure HardFault, this bit only applies to the Non-secure HardFault.

This bit is not banked between Security states.

The possible values of this bit are:

0

Not able to set the HardFault exception to pending.

1

Able to set the HardFault exception to pending.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to an **UNKNOWN** value on a Warm reset.

THREAD, bit [3]

Thread mode. Indicates the PE mode when it allocated the Floating-point stack frame.

This bit is banked between Security states.

The possible values of this bit are:

0

Handler mode.

1

Thread mode.

This bit is for fault handler information only and does not interact with the exception model.

This bit resets to an **UNKNOWN** value on a Warm reset.

S, bit [2]

Security. Security status of the Floating-point context. This bit is only present in the Secure version of the register. This bit is updated whenever lazy Floating-point state preservation is activated, or when a Floating-point instruction is executed.

This bit is not banked between Security states.

The possible values of this bit are:

0

Indicates the Floating-point context belongs to the Non-secure state.

1

Indicates the Floating-point context belongs to the Secure state.

This bit is RAZ/WI from Non-secure state.

This bit resets to one on a Warm reset.

USER, bit [1]

User privilege. Indicates the privilege level of the software executing when the PE allocated the Floating-point stack frame.

This bit is banked between Security states.

The possible values of this bit are:

0

Privileged.

1

Unprivileged.

This bit resets to an **UNKNOWN** value on a Warm reset.

LSPACT, bit [0]

Lazy state preservation active. Indicates whether lazy Floating-point state preservation is active.

This bit is banked between Security states.

The possible values of this bit are:

0

Lazy state preservation is not active.

1

Lazy state preservation is active.

This bit resets to zero on a Warm reset.

D1.2.101 FPCXT, Floating-point context payload

The FPCXT characteristics are:

Purpose

Values produced or consumed by instructions that provide access to the Floating-point context.

Usage constraints

None.

Configurations

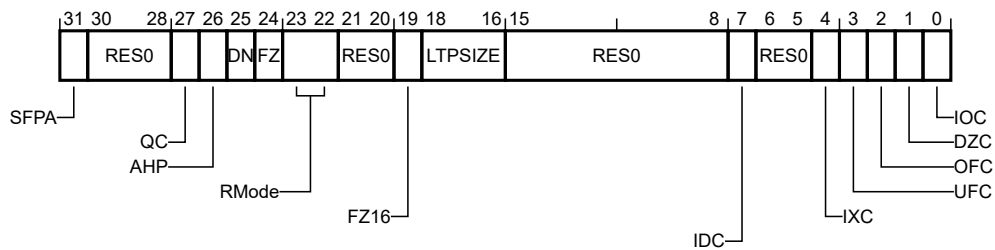
All.

Attributes

32-bit payload.

Field descriptions

The FPCXT bit assignments are:



SFPA, bit [31]

Secure Floating-point active. The value corresponds to CONTROL.SFPA.

Bits [30:28]

Reserved, **RES0**.

QC, bit [27]

Cumulative saturation bit. The value corresponds to FPSCR.QC.

AHP, bit [26]

Alternative half-precision control bit. The value corresponds to FPSCR.AHP.

DN, bit [25]

Default NaN mode control bit. The value corresponds to FPSCR.DN.

FZ, bit [24]

Flush-to-zero mode control for single and double precision Floating-point. The value corresponds to FPSCR.FZ.

RMode, bits [23:22]

Rounding mode control field. The value corresponds to FPSCR.RMode.

Bits [21:20]

Reserved, **RES0**.

FZ16, bit [19]

Flush-to-zero mode control bit on half-precision data-processing instructions. The value corresponds to FPSCR.FZ16.

LTPSIZE, bits [18:16]

The vector element size that is used when applying low-overhead-loop tail predication to vector instructions.

Bits [15:8]

Reserved, [RES0](#).

IDC, bit [7]

Input Denormal cumulative exception bit. The value corresponds to FPSCR.IDC.

Bits [6:5]

Reserved, [RES0](#).

IXC, bit [4]

Inexact cumulative exception bit. The value corresponds to FPSCR.IXC.

UFC, bit [3]

Underflow cumulative exception bit. The value corresponds to FPSCR.UFC.

OFC, bit [2]

Overflow cumulative exception bit. The value corresponds to FPSCR.OFC.

DZC, bit [1]

Divide by Zero cumulative exception bit. The value corresponds to FPSCR.DZC.

IOC, bit [0]

Invalid Operation cumulative exception bit. The value corresponds to FPSCR.IOC.

D1.2.102 FPDSCR, Floating-Point Default Status Control Register

The FPDSCR characteristics are:

Purpose

Holds the default values for the Floating-point status control data that the PE assigns to FPSCR when it creates a new Floating-point context.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present if the Floating-point Extension, or MVE, or both are implemented.

This register is **RES0** if neither the Floating-point Extension nor MVE are implemented.

Attributes

32-bit read/write register located at 0xE000EF3C.

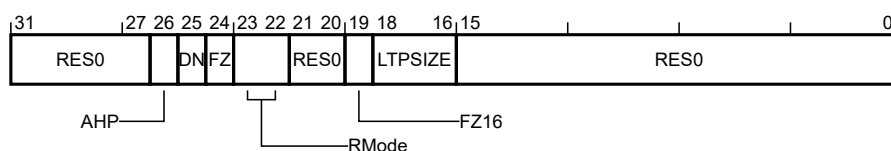
Secure software can access the Non-secure version of this register via FPDSCR_NS located at 0xE002EF3C. The location 0xE002EF3C is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The FPDSCR bit assignments are:



Bits [31:27]

Reserved, **RES0**.

AHP, bit [26]

Alternative half-precision. Default value for FPSCR.AHP.

If the Floating-point Extension is not implemented, this bit is **RES0**.

This bit resets to zero on a Warm reset.

DN, bit [25]

Default NaN. Default value for FPSCR.DN.

If the Floating-point Extension is not implemented, this bit is **RES0**.

This bit resets to zero on a Warm reset.

FZ, bit [24]

Flush-to-zero. Default value for FPSCR.FZ.

If the Floating-point Extension is not implemented, this bit is **RES0**.

This bit resets to zero on a Warm reset.

RMode, bits [23:22]

Rounding mode. Default value for FPSCR.RMode.

If the Floating-point Extension is not implemented, this field is [RES0](#).

This field resets to zero on a Warm reset.

Bits [21:20]

Reserved, [RES0](#).

FZ16, bit [19]

Flush-to-zero mode control bit on half-precision data-processing instructions. Default value for FPSCR.FZ16.

If the Floating-point Extension is not implemented, this bit is RAZ/WI.

If the Floating-point Extension is implemented without half-precision support, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

LTPSIZE, bits [18:16]

The vector element size used when applying low-overhead-loop tail predication to vector instructions. Default value for FPSCR.LTPSIZE.

If the Low Overhead Branch Extension is not implemented, this field is [RES0](#).

This field reads as 0x4.

Bits [15:0]

Reserved, [RES0](#).

D1.2.103 FPSCR, Floating-point Status and Control Register

The FPSCR characteristics are:

Purpose

Provides control of the Floating-point unit.

Usage constraints

Privileged and unprivileged access permitted.

Configurations

Present if the Floating-point Extension, or MVE, or both are implemented.

This register is [RES0](#) if neither the Floating-point Extension nor MVE are implemented.

Attributes

32-bit read/write special-purpose register.

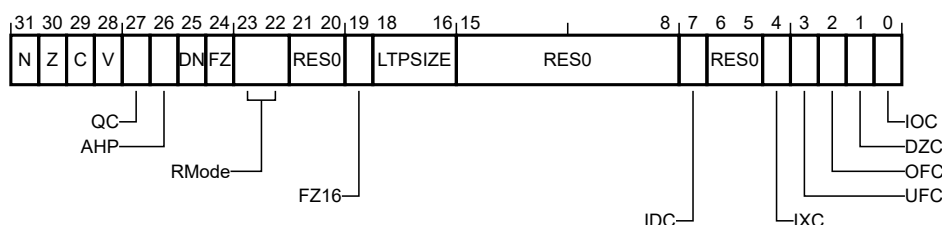
This register is not banked between Security states.

Preface

Writes to FPSCR can have side effects on various aspects of processor operation. All of these side effects are synchronous to FPSCR write. This means that they are guaranteed not to be visible to earlier instructions in the execution stream, and they are guaranteed to be visible to later instructions in the execution stream.

Field descriptions

The FPSCR bit assignments are:



N, bit [31]

Negative condition flag. When updated by a VCMP instruction, this bit indicates whether the result was less than.

The possible values of this bit are:

0

Compare result was not less than.

1

Compare result was less than.

See VCMP for details.

If the Floating-point Extension is not implemented, this bit is RAZ/WI.

This bit resets to an [UNKNOWN](#) value on a Warm reset.

Z, bit [30]

Zero condition flag. When updated by a VCMP instruction, this bit indicates whether the result was equal.

The possible values of this bit are:

0
Compare result was not equal.

1
Compare result was equal.

See VCMP for details.

If the Floating-point Extension is not implemented, this bit is RAZ/WI.

This bit resets to an **UNKNOWN** value on a Warm reset.

C, bit [29]

Carry condition flag. Accessed by the VCMP, VADC, and VSBC instructions. For VCMP this bit indicates whether the result was not less than. In VADC and VSBC this bit is used to hold the carry in/out flag.

The possible values of this bit are:

0
Compare result was less than.

1
Compare result was not less than.

See VCMP, VADC, and VSBC for details.

This bit resets to an **UNKNOWN** value on a Warm reset.

V, bit [28]

Overflow condition flag. When updated by a VCMP instruction, this bit indicates whether the result was unordered.

The possible values of this bit are:

0
Compare result was not unordered.

1
Compare result was unordered.

See VCMP for details.

If the Floating-point Extension is not implemented, this bit is RAZ/WI.

This bit resets to an **UNKNOWN** value on a Warm reset.

QC, bit [27]

Cumulative saturation bit. This bit is set to 1 to indicate that an MVE integer operation has saturated since 0 was last written to this bit.

If MVE is not implemented, this bit is **RES0**.

This bit resets to an **UNKNOWN** value on a Warm reset.

AHP, bit [26]

Alternative half-precision control bit. This bit controls how the PE interprets 16-bit Floating-point values.

The possible values of this bit are:

0
IEEE half-precision format selected.

1
Alternative half-precision format selected.

If the Floating-point Extension is not implemented, this bit is RAZ/WI.

This bit resets to an **UNKNOWN** value on a Warm reset.

DN, bit [25]

Default NaN mode control bit. This bit determines whether Floating-point operations propagate NaNs or use the Default NaN.

The possible values of this bit are:

0

NaN operands propagate through to the output of a Floating-point operation.

1

Any operation involving one of more NaNs returns the Default NaN.

If the Floating-point Extension is not implemented, this bit is RAZ/WI.

This bit resets to an **UNKNOWN** value on a Warm reset.

FZ, bit [24]

Flush-to-zero mode control for single and double precision Floating-point. This bit determines whether denormal Floating-point values are treated as though they are zero.

The possible values of this bit are:

0

Flush-to-zero mode disabled. Behavior of the Floating-point unit is fully compliant with the IEEE754 standard.

1

Flush-to-zero mode enabled.

If the Floating-point Extension is not implemented, this bit is RAZ/WI.

This bit resets to an **UNKNOWN** value on a Warm reset.

RMode, bits [23:22]

Rounding mode control field. This field determines what rounding mode is applied to Floating-point operations.

The possible values of this field are:

0b00

Round to Nearest (RN) mode.

0b01

Round towards Plus Infinity (RP) mode.

0b10

Round towards Minus Infinity (RM) mode.

0b11

Round towards Zero (RZ) mode.

If the Floating-point Extension is not implemented, this field is RAZ/WI.

This field resets to an **UNKNOWN** value on a Warm reset.

Bits [21:20]

Reserved, **RES0**.

FZ16, bit [19]

Flush-to-zero mode control bit on half-precision data-processing instructions.

The possible values of this bit are:

0

Flush-to-zero mode disabled. Behavior of the Floating-point unit is fully compliant with the IEEE 754 standard.

1

Flush-to-zero mode enabled.

The value of this bit applies to both scalar and MVE Floating-point half-precision calculations.

If the Floating-point Extension is not implemented, this bit is RAZ/WI.

If the Floating-point Extension is implemented without half-precision support, this bit is [RES0](#).

This bit resets to an [UNKNOWN](#) value on a Warm reset.

LTFSIZE, bits [18:16]

The vector element size used when applying low-overhead-loop tail predication to vector instructions.

The possible values of this field are:

0b000

8 bits.

0b001

16 bits.

0b010

32 bits.

0b011

64 bits.

0b100

Tail predication not applied.

All other values are reserved.

The loop hardware behaves as if this field had the value 4 (indicating no low-overhead-loop predication) if no floating-point context is active. This field reads as 4 and ignores writes if MVE is not implemented.

If the Low Overhead Branch Extension is not implemented, this field is [RES0](#).

This field resets to an [UNKNOWN](#) value on a Warm reset.

Bits [15:8]

Reserved, [RES0](#).

IDC, bit [7]

Input Denormal cumulative exception bit. This sticky flag records whether a Floating-point input denormal exception has been detected since last cleared.

The possible values of this bit are:

0

Input Denormal exception has not occurred since 0 was last written to this bit.

1

Input Denormal exception has occurred since 0 was last written to this bit.

If the Floating-point Extension is not implemented, this bit is [RES0](#).

This bit resets to an [UNKNOWN](#) value on a Warm reset.

Bits [6:5]

Reserved, [RES0](#).

IXC, bit [4]

Inexact cumulative exception bit. This sticky flag records whether a Floating-point inexact exception has been detected since last cleared.

The possible values of this bit are:

0
Inexact exception has not occurred since 0 was last written to this bit.

1
Inexact exception has occurred since 0 was last written to this bit.

If the Floating-point Extension is not implemented, this bit is **RES0**.

This bit resets to an **UNKNOWN** value on a Warm reset.

UFC, bit [3]

Underflow cumulative exception bit. This sticky flag records whether a Floating-point Underflow exception has been detected since last cleared.

The possible values of this bit are:

0
Underflow exception has not occurred since 0 was last written to this bit.

1
Underflow exception has occurred since 0 was last written to this bit.

If the Floating-point Extension is not implemented, this bit is **RES0**.

OFC, bit [2]

Overflow cumulative exception bit. This sticky flag records whether a Floating-point overflow exception has been detected since last cleared.

The possible values of this bit are:

0
Overflow exception has not occurred since 0 was last written to this bit.

1
Overflow exception has occurred since 0 was last written to this bit.

If the Floating-point Extension is not implemented, this bit is **RES0**.

This bit resets to an **UNKNOWN** value on a Warm reset.

DZC, bit [1]

Divide by Zero cumulative exception bit. This sticky flag records whether a Floating-point divide by zero exception has been detected since last cleared.

The possible values of this bit are:

0
Division by Zero exception has not occurred since 0 was last written to this bit.

1
Division by Zero exception has occurred since 0 was last written to this bit.

If the Floating-point Extension is not implemented, this bit is **RES0**.

This bit resets to an **UNKNOWN** value on a Warm reset.

IOC, bit [0]

Invalid Operation cumulative exception bit. This sticky flag records whether a Floating-point invalid operation exception has been detected since last cleared.

The possible values of this bit are:

0
Invalid Operation exception has not occurred since 0 was last written to this bit.

1
Invalid Operation exception has occurred since 0 was last written to this bit.

If the Floating-point Extension is not implemented, this bit is [RES0](#).

This bit resets to an [UNKNOWN](#) value on a Warm reset.

D1.2.104 FP_CIDR0, FP Component Identification Register 0

The FP_CIDR0 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

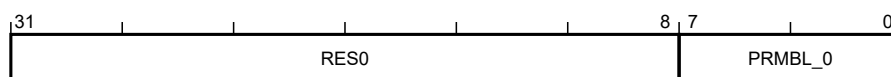
Attributes

32-bit read-only register located at 0xE0002FF0.

This register is not banked between Security states.

Field descriptions

The FP_CIDR0 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

PRMBL_0, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0D.

D1.2.105 FP_CIDR1, FP Component Identification Register 1

The FP_CIDR1 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

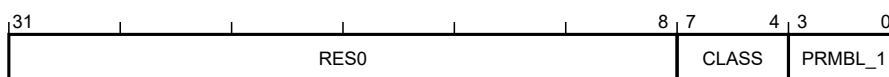
Attributes

32-bit read-only register located at 0xE0002FF4.

This register is not banked between Security states.

Field descriptions

The FP_CIDR1 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

CLASS, bits [7:4]

CoreSight component class. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x9.

PRMBL_1, bits [3:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0.

D1.2.106 FP_CIDR2, FP Component Identification Register 2

The FP_CIDR2 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

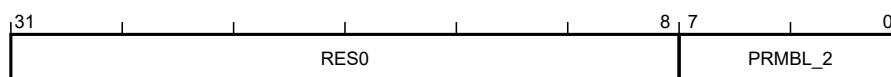
Attributes

32-bit read-only register located at 0xE0002FF8.

This register is not banked between Security states.

Field descriptions

The FP_CIDR2 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

PRMBL_2, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x05.

D1.2.107 FP_CIDR3, FP Component Identification Register 3

The FP_CIDR3 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

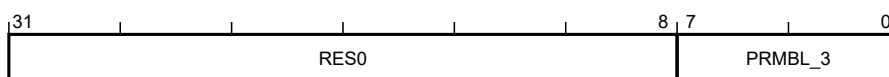
Attributes

32-bit read-only register located at 0xE0002FFC.

This register is not banked between Security states.

Field descriptions

The FP_CIDR3 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

PRMBL_3, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0xB1.

D1.2.108 FP_COMPn, Flash Patch Comparator Register, n = 0 - 125

The FP_COMP{0..125} characteristics are:

Purpose

Holds an address for comparison.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

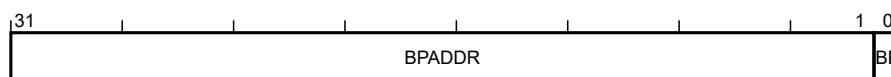
Attributes

32-bit read/write register located at $0xE0002008 + 4n$.

This register is not banked between Security states.

Field descriptions

The FP_COMP{0..125} bit assignments are:



BPADDR, bits [31:1]

Breakpoint address. Specifies bits[31:1] of the breakpoint instruction address.

BE, bit [0]

Breakpoint enable. Selects between remapping and breakpoint functionality.

The possible values of this bit are:

0

Breakpoint disabled.

1

Breakpoint enabled.

For backwards compatibility, when disabling a breakpoint software must write zero to the whole register.

This bit resets to zero on a Cold reset.

D1.2.109 FP_CTRL, Flash Patch Control Register

The FP_CTRL characteristics are:

Purpose

Provides FPB implementation information, and the global enable for the FPB unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

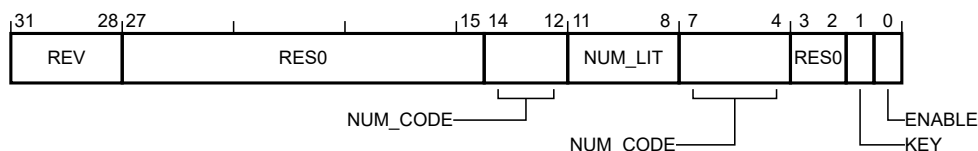
Attributes

32-bit read/write register located at 0xE0002000.

This register is not banked between Security states.

Field descriptions

The FP_CTRL bit assignments are:



REV, bits [31:28]

Revision. Flash Patch and Breakpoint Unit architecture revision.

The possible values of this field are:

0b0001

Flash Patch Breakpoint version 2 implemented.

All other values are reserved.

This field is read-only.

This field reads as 0b0001.

Bits [27:15]

Reserved, **RES0**.

NUM_CODE, bits [14:12,7:4]

Number of implemented code comparators. Indicates the number of implemented instruction address comparators. Zero indicates no Instruction Address comparators are implemented. The Instruction Address comparators are numbered from 0 to NUM_CODE - 1.

This field is read-only.

This field reads as an **IMPLEMENTATION DEFINED** value.

NUM_LIT, bits [11:8]

Number of literal comparators. This field is RAZ/WI. Remapping is not supported in Armv8-M.

Reserved, RAZ/WI

Bits [3:2]

Reserved, RES0.

KEY, bit [1]

FP_CTRL write-enable key. Writes to the FP_CTRL are ignored unless KEY is concurrently written to one.

The possible values of this bit are:

0

Concurrent write to FP_CTRL ignored.

1

Concurrent write to FP_CTRL permitted.

This bit reads-as-zero.

ENABLE, bit [0]

Flash Patch global enable. Enables the FPB.

The possible values of this bit are:

0

All FPB functionality disabled.

1

FPB enabled.

This bit resets to zero on a Cold reset.

D1.2.110 FP_DEVARCH, FPB Device Architecture Register

The FP_DEVARCH characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

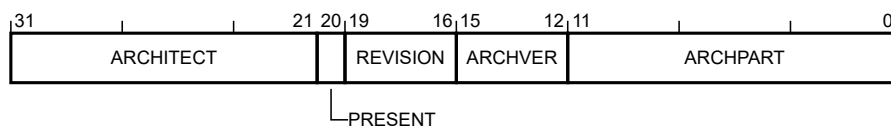
Attributes

32-bit read-only register located at 0xE0002FBC.

This register is not banked between Security states.

Field descriptions

The FP_DEVARCH bit assignments are:

**ARCHITECT, bits [31:21]**

Architect. Defines the architect of the component. Bits [31:28] are the JEP106 continuation code (JEP106 bank ID, minus 1) and bits [27:21] are the JEP106 ID code.

The possible values of this field are:

0x23B

JEP106 continuation code 0x4, ID code 0x3B. Arm Limited.

Other values are defined by the JEDEC JEP106 standard.

This field reads as 0x23B.

PRESENT, bit [20]

DEVARCH Present. Defines that the DEVARCH register is present.

The possible values of this bit are:

1

DEVARCH information present.

This bit reads as one.

REVISION, bits [19:16]

Revision. Defines the architecture revision of the component.

The possible values of this field are:

0b0000

FPB architecture v2.0.

This field reads as 0b0000.

ARCHVER, bits [15:12]

Architecture Version. Defines the architecture version of the component.

The possible values of this field are:

0b0001

FPB architecture v2.

ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHVER is ARCHID[15:12].

This field reads as 0b0001.

ARCHPART, bits [11:0]

Architecture Part. Defines the architecture of the component.

The possible values of this field are:

0xA03

FPB architecture.

ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHPART is ARCHID[11:0].

This field reads as 0xA03.

D1.2.111 FP_DEVTYPE, FPB Device Type Register

The FP_DEVTYPE characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

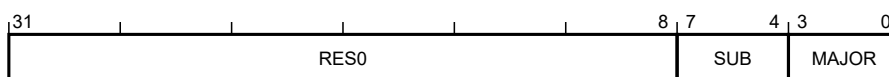
Attributes

32-bit read-only register located at 0xE0002FCC.

This register is not banked between Security states.

Field descriptions

The FP_DEVTYPE bit assignments are:



Bits [31:8]

Reserved, **RES0**.

SUB, bits [7:4]

Sub-type. Component sub-type.

The possible values of this field are:

0x0

Other.

This field reads as 0b0000.

MAJOR, bits [3:0]

Major type. Component major type.

The possible values of this field are:

0x0

Miscellaneous.

This field reads as 0b0000.

D1.2.112 FP_LAR, FPB Software Lock Access Register

The FP_LAR characteristics are:

Purpose

Provides CoreSight Software Lock control for the FPB, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank). It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

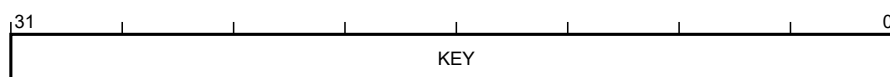
Attributes

32-bit write-only register located at 0xE0002FB0.

This register is not banked between Security states.

Field descriptions

The FP_LAR bit assignments are:



KEY, bits [31:0]

Lock Access control.

Writing the key value 0xC5ACCE55 to this field unlocks the lock, enabling write accesses to the registers of this component through a memory-mapped interface.

Writing any other value to this register locks the lock, disabling write accesses to the registers of this component through a memory mapped interface.

D1.2.113 FP_LSR, FPB Software Lock Status Register

The FP_LSR characteristics are:

Purpose

Provides CoreSight Software Lock status information for the FPB, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank). It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

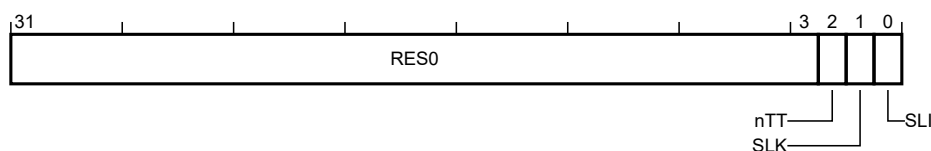
Attributes

32-bit read-only register located at 0xE0002FB4.

This register is not banked between Security states.

Field descriptions

The FP_LSR bit assignments are:



Bits [31:3]

Reserved, **RES0**.

nTT, bit [2]

Not thirty-two bit. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as zero.

SLK, bit [1]

Software Lock status. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

0

Lock clear. Software writes are permitted to the registers of this component.

1

Lock set. Software writes to the registers of this component are ignored, and reads have no side effects.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is [RES0](#).

This bit resets to one on a Cold reset.

SLI, bit [0]

Software Lock implemented. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

0

Software Lock not implemented or debugger access.

1

Software Lock is implemented and software access.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is RAZ.

This bit reads as an [IMPLEMENTATION DEFINED](#) value.

D1.2.114 FP_PIDR0, FP Peripheral Identification Register 0

The FP_PIDR0 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

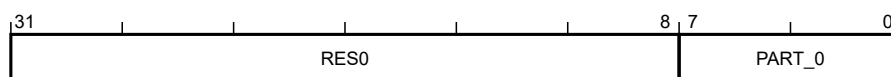
Attributes

32-bit read-only register located at 0xE0002FE0.

This register is not banked between Security states.

Field descriptions

The FP_PIDR0 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

PART_0, bits [7:0]

Part number bits [7:0]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.115 FP_PIDR1, FP Peripheral Identification Register 1

The FP_PIDR1 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

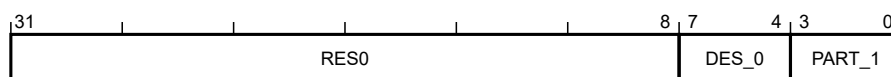
Attributes

32-bit read-only register located at 0xE0002FE4.

This register is not banked between Security states.

Field descriptions

The FP_PIDR1 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

DES_0, bits [7:4]

JEP106 identification code bits [3:0]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

PART_1, bits [3:0]

Part number bits [11:8]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an IMPLEMENTATION DEFINED value.

D1.2.116 FP_PIDR2, FP Peripheral Identification Register 2

The FP_PIDR2 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

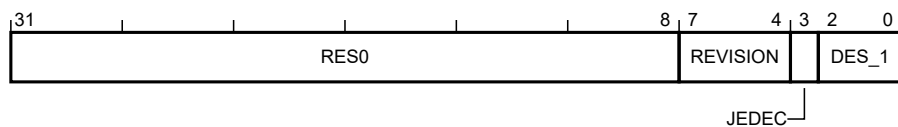
Attributes

32-bit read-only register located at 0xE0002FE8.

This register is not banked between Security states.

Field descriptions

The FP_PIDR2 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REVISION, bits [7:4]

Component revision. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

JEDEC, bit [3]

JEDEC assignee value is used. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as one.

DES_1, bits [2:0]

JEP106 identification code bits [6:4]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.117 FP_PIDR3, FP Peripheral Identification Register 3

The FP_PIDR3 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

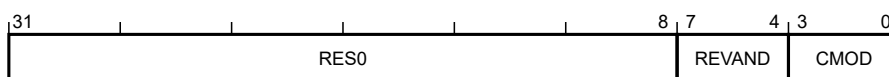
Attributes

32-bit read-only register located at 0xE0002FEC.

This register is not banked between Security states.

Field descriptions

The FP_PIDR3 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REVAND, bits [7:4]

RevAnd. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

CMOD, bits [3:0]

Customer Modified. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.118 FP_PIDR4, FP Peripheral Identification Register 4

The FP_PIDR4 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

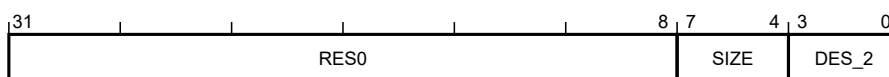
Attributes

32-bit read-only register located at 0xE0002FD0.

This register is not banked between Security states.

Field descriptions

The FP_PIDR4 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

SIZE, bits [7:4]

4KB count. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as zero.

DES_2, bits [3:0]

JEP106 continuation code. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.119 FP_PIDR5, FP Peripheral Identification Register 5

The FP_PIDR5 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

Attributes

32-bit read-only register located at 0xE0002FD4.

This register is not banked between Security states.

Field descriptions

The FP_PIDR5 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.120 FP_PIDR6, FP Peripheral Identification Register 6

The FP_PIDR6 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

Attributes

32-bit read-only register located at 0xE0002FD8.

This register is not banked between Security states.

Field descriptions

The FP_PIDR6 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.121 FP_PIDR7, FP Peripheral Identification Register 7

The FP_PIDR7 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the Flash Patch and Breakpoint Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

Attributes

32-bit read-only register located at 0xE0002FDC.

This register is not banked between Security states.

Field descriptions

The FP_PIDR7 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.122 FP_REMAP, Flash Patch Remap Register

The FP_REMAP characteristics are:

Purpose

Indicates whether the implementation supports Flash Patch remap and, if it does, holds the target address for remap.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the FPB is implemented.

This register is **RES0** if the FPB is not implemented.

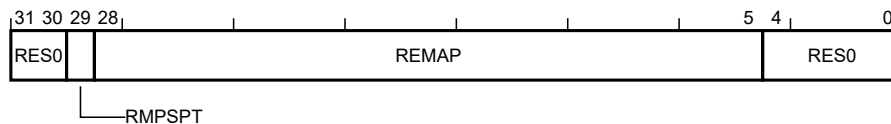
Attributes

32-bit read-only register located at 0xE0002004.

This register is not banked between Security states.

Field descriptions

The FP_REMAP bit assignments are:

**Bits [31:30]**

Reserved, RES0.

RMPSPT, bit [29]

Remap supported. This field is RAZ. Remapping is not supported in Armv8-M.

REMAP, bits [28:5]

Remap address.

Reserved, RES0.

Bits [4:0]

Reserved, RES0.

D1.2.123 HFSR, HardFault Status Register

The HFSR characteristics are:

Purpose

Shows the cause of any HardFaults.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read/write-one-to-clear register located at 0xE000ED2C.

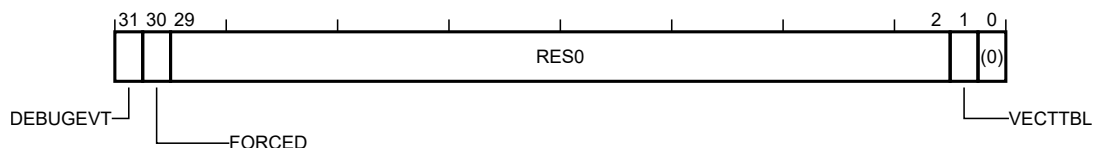
Secure software can access the Non-secure version of this register via HFSR_NS located at 0xE002ED2C. The location 0xE002ED2C is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The HFSR bit assignments are:



DEBUGEVT, bit [31]

Debug event. Indicates when a debug event has occurred.

The possible values of this bit are:

0

No debug event has occurred.

1

Debug event has occurred. The Debug Fault Status Register has been updated.

The PE sets this bit to 1 when a breakpoint is encountered and Halting debug is disabled and a DebugMonitor exception cannot be taken.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

FORCED, bit [30]

Forced. Indicates that a fault with configurable priority has been escalated to a HardFault exception, because it could not be made active, because of priority, or because it was disabled.

The possible values of this bit are:

0
No priority escalation has occurred.

1
Processor has escalated a configurable-priority exception to HardFault.
If AIRCR.BFHFNMINs is zero this bit is RAZ/WI from Non-secure state.
This bit resets to zero on a Warm reset.

Bits [29:2]
Reserved, [RES0](#).

VECTTBL, bit [1]
Vector table. Indicates when a fault has occurred because of a vector table read error on exception processing.
The possible values of this bit are:

0
No vector table read fault has occurred.

1
Vector table read fault has occurred.
If AIRCR.BFHFNMINs is zero this bit is RAZ/WI from Non-secure state.
This bit resets to zero on a Warm reset.

Bit [0]
Reserved, [RES0](#).

D1.2.124 ICIALLU, Instruction Cache Invalidate All to PoU

The ICIALLU characteristics are:

Purpose

Invalidate all instruction caches to PoU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

32-bit write-only register located at 0xE000EF50.

Secure software can access the Non-secure version of this register via ICIALLU_NS located at 0xE002EF50.

The location 0xE002EF50 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ICIALLU bit assignments are:



Ignored, bits [31:0]

The value written to this field is ignored. Ignored.

D1.2.125 ICIMVAU, Instruction Cache line Invalidate by Address to PoU

The ICIMVAU characteristics are:

Purpose

Invalidate instruction cache line by address to PoU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

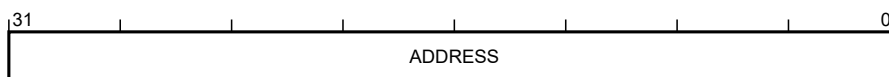
32-bit write-only register located at 0xE000EF58.

Secure software can access the Non-secure version of this register via ICIMVAU_NS located at 0xE002EF58. The location 0xE002EF58 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ICIMVAU bit assignments are:



ADDRESS, bits [31:0]

Address. Writing to this field initiates the maintenance operation for the address written.

D1.2.126 ICSR, Interrupt Control and State Register

The ICSR characteristics are:

Purpose

Controls and provides status information for NMI, PendSV, SysTick and interrupts.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED04.

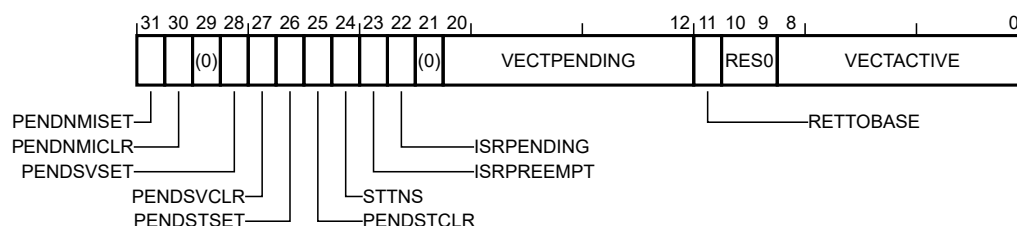
Secure software can access the Non-secure version of this register via ICSR_NS located at 0xE002ED04. The location 0xE002ED04 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

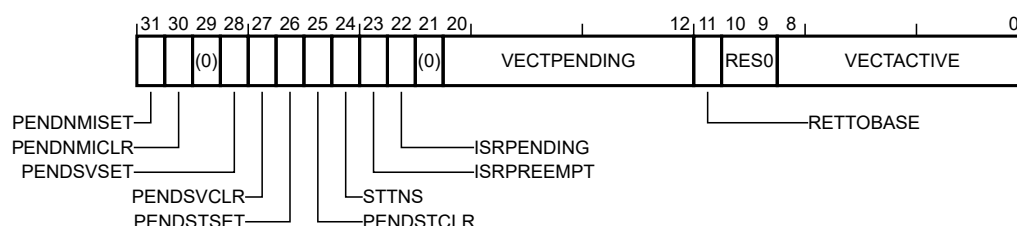
Field descriptions

The ICSR bit assignments are:

On a read:



On a write:



PENDNMISET, bit [31], on a write

Pend NMI set. Allows the NMI exception to be set as pending.

This bit is not banked between Security states.

The possible values of this bit are:

O

No effect.

1

Sets the NMI exception pending.

If both PENDNMISET and PENDNMICLR are written to one simultaneously, the pending state of the NMI exception becomes **UNKNOWN**.

This bit is write-one-to-set. Writes of zero are ignored.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

PENDNMISET, bit [31], on a read

Pend NMI set. Indicates whether the NMI exception is pending.

This bit is not banked between Security states.

The possible values of this bit are:

0

NMI exception not pending.

1

NMI exception pending.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

PENDNMICLR, bit [30]

Pend NMI clear. Allows the NMI exception pending state to be cleared.

This bit is not banked between Security states.

The possible values of this bit are:

0

No effect.

1

Clear pending status.

This bit is write-only, and reads-as-zero.

This bit is write-one-to-clear. Writes of zero are ignored.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

Bit [29]

Reserved, **RES0**.

PENDSVSET, bit [28], on a write

Pend PendSV set. Allows the PendSV exception for the selected Security state to be set as pending.

This bit is banked between Security states.

The possible values of this bit are:

0

No effect.

1

Sets the PendSV exception pending.

If both PENDSVSET and PENDSVCLR are written to one simultaneously, the pending state of the associated PendSV exception becomes **UNKNOWN**.

This bit is write-one-to-set. Writes of zero are ignored.

PENDSVSET, bit [28], on a read

Pend PendSV set. Indicates whether the PendSV for the selected Security state exception is pending.

This bit is banked between Security states.

The possible values of this bit are:

0

PendSV exception not pending.

1

PendSV exception pending.

This bit resets to zero on a Warm reset.

PENDSVCLR, bit [27]

Pend PendSV clear. Allows the PendSV exception pending state to be cleared for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

No effect.

1

Clear pending status.

This bit is write-only, and reads-as-zero.

This bit is write-one-to-clear. Writes of zero are ignored.

PENDSTSET, bit [26], on a write

Pend SysTick set. Allows the SysTick for the selected Security state exception to be set as pending.

If two SysTick timers are implemented this bit is banked between Security states.

If less than two SysTick timers are implemented this bit is not banked between Security states.

The possible values of this bit are:

0

No effect.

1

Sets the SysTick exception for the selected Security state pending.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is [RES0](#).

PENDSTSET, bit [26], on a read

Pend SysTick set. Indicates whether the SysTick for the selected Security state exception is pending.

If two SysTick timers are implemented this bit is banked between Security states.

If less than two SysTick timers are implemented this bit is not banked between Security states.

The possible values of this bit are:

0

SysTick exception not pending.

1

SysTick exception pending.

If both PENDSTSET and PENDSTCLR are written to one simultaneously, the pending state of the associated SysTick exception becomes [UNKNOWN](#).

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is [RES0](#).

This bit resets to zero on a Warm reset.

PENDSTCLR, bit [25]

Pend SysTick clear. Allows the SysTick exception pending state to be cleared for the selected Security state.

If two SysTick timers are implemented this bit is banked between Security states.

If less than two SysTick timers are implemented this bit is not banked between Security states.

The possible values of this bit are:

0

No effect.

1

Clear pending status.

This bit is write-only, and reads-as-zero.

This bit is write-one-to-clear. Writes of zero are ignored.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is [RES0](#).

STTNS, bit [24]

SysTick Targets Non-secure. Controls whether in a single SysTick implementation, the SysTick is Secure or Non-secure.

This bit is not banked between Security states.

The possible values of this bit are:

0

SysTick is Secure.

1

SysTick is Non-secure.

Behaves as RAZ/WI when either no SysTick or both SysTick timers are implemented. In a PE with the Main Extension and Security Extension this bit is [RES0](#). This bit is RAZ/WI when accessed from the Non-secure state.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

ISRPREEMPT, bit [23]

Interrupt preempt. Indicates whether a pending exception will be handled on exit from Debug state.

This bit is not banked between Security states.

The possible values of this bit are:

0

Will not handle.

1

Will handle a pending exception.

The value of this bit is [UNKNOWN](#) when not in Debug state.

This bit is read-only.

If neither Halting debug or the Main Extension are implemented, this bit is [RES0](#).

ISRPENDING, bit [22]

Interrupt pending. Indicates whether an external interrupt, generated by the NVIC, is pending.

This bit is not banked between Security states.

The possible values of this bit are:

0

No external interrupt pending.

1

External interrupt pending.

The value of DHCSR.C_MASKINTS is ignored.

This bit is read-only.

If neither Halting debug or the Main Extension are implemented, this bit is [RES0](#).

Bit [21]

Reserved, [RES0](#).

VECTPENDING, bits [20:12]

Vector pending. The exception number of the highest priority pending and enabled interrupt.

This field is not banked between Security states.

The possible values of this field are:

Zero

No pending and enabled exception.

Nonzero

Exception number.

From Armv8.1-M this value is 1 when accessed as Non-secure and a Secure exception is the highest priority pending exception.

If DHCSR.C_MASKINTS is set, the PendSV, SysTick, and configurable external interrupts are masked and will not be shown as pending in VECTPENDING.

This field is read-only.

RETTOBASE, bit [11]

Return to base. In Handler mode, indicates whether there is more than one active exception.

This bit is not banked between Security states.

The possible values of this bit are:

0

There is more than one active exception.

1

There is only one active exception.

In Thread mode the value of this bit is [UNKNOWN](#).

This bit is read-only.

If the Main Extension is not implemented, this bit is [RES0](#).

Bits [10:9]

Reserved, [RES0](#).

VECTACTIVE, bits [8:0]

Vector active. The exception number of the current executing exception.

This field is not banked between Security states.

The possible values of this field are:

Zero

Thread mode.

Nonzero

Exception number.

This value is the same as the IPSR Exception number. When the IPSR value has been set to 1 because of a function call to Non-secure state, this field is also set to 1.

This field is read-only.

If neither Halting debug or the Main Extension are implemented, this field is [RES0](#).

D1.2.127 ICTR, Interrupt Controller Type Register

The ICTR characteristics are:

Purpose

Provides information about the interrupt controller.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, it is **IMPLEMENTATION DEFINED** if this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

This register is always implemented.

Attributes

32-bit read-only register located at 0xE000E004.

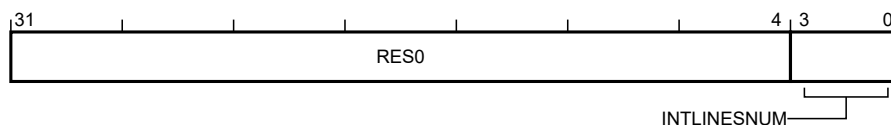
Secure software can access the Non-secure version of this register via ICTR_NS located at 0xE002E004.

The location 0xE002E004 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ICTR bit assignments are:

**Bits [31:4]**

Reserved, RES0.

INTLINESNUM, bits [3:0]

Interrupt line set number. The number, n , of the highest implemented NVIC_IPR n register is the minimum of ((8 * INTLINESNUM) + 7) and 123. For all other NVIC registers, the number, n , of the highest implemented register is equal to INTLINESNUM.

This field reads as an IMPLEMENTATION DEFINED value.

D1.2.128 ID_AFR0, Auxiliary Feature Register 0

The ID_AFR0 characteristics are:

Purpose

Provides information about the **IMPLEMENTATION DEFINED** features of the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000ED4C.

Secure software can access the Non-secure version of this register via ID_AFR0_NS located at 0xE002ED4C.

The location 0xE002ED4C is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_AFR0 bit assignments are:

31																16	15																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																
----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----	----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Bits [31:16]

Reserved, **RES0**.

IMPDEF m , bits $[4m+3:4m]$, for $m = 0$ to 3

IMPLEMENTATION DEFINED. **IMPLEMENTATION DEFINED** meaning.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.129 ID_DFR0, Debug Feature Register 0

The ID_DFR0 characteristics are:

Purpose

Provides top level information about the debug system.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000ED48.

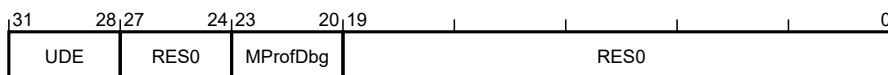
Secure software can access the Non-secure version of this register via ID_DFR0_NS located at 0xE002ED48.

The location 0xE002ED48 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_DFR0 bit assignments are:



UDE, bits [31:28]

Unprivileged Debug Extension. Indicates support for the Unprivileged Debug Extension.

The possible values of this field are:

0b0000

Unprivileged Debug Extension is not implemented.

0b0001

Unprivileged Debug Extension is implemented.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [27:24]

Reserved, **RES0**.

MProfDbg, bits [23:20]

M-Profile debug. Indicates the supported M-Profile debug architecture.

The possible values of this field are:

0b0000

Halting debug is not implemented.

0b0010

Armv8-M Debug architecture including Halting debug is implemented.

All other values are reserved.

This field reads as an IMPLEMENTATION DEFINED value.

Bits [19:0]

Reserved, RES0.

D1.2.130 ID_ISAR0, Instruction Set Attribute Register 0

The ID_ISAR0 characteristics are:

Purpose

Provides information about the instruction set implemented by the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000ED60.

Secure software can access the Non-secure version of this register via ID_ISAR0_NS located at 0xE002ED60. The location 0xE002ED60 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_ISAR0 bit assignments are:

31	28	27	24	23	20	19	16	15	12	11	8	7	4	3	0
RES0	Divide	Divide	Divide	Divide	Coproc	Coproc	CmpBranch	CmpBranch	BitField	BitField	BitCount	BitCount	RES0	RES0	RES0

Bits [31:28]

Reserved, [RES0](#).

Divide, bits [27:24]

Divide. Indicates the supported Divide instructions.

The possible values of this field are:

0b0001

Supports SDIV and UDIV instructions.

All other values are reserved.

This field reads as 0b0001.

Debug, bits [23:20]

Debug. Indicates the implemented Debug instructions.

The possible values of this field are:

0b0001

Supports BKPT instruction.

All other values are reserved.

This field reads as 0b0001.

Coproc, bits [19:16]

Coprocessor. Indicates the supported coprocessor instructions.

The possible values of this field are:

0b0000

No coprocessor instructions support other than FPU or MVE.

0b0100

Coprocessor instructions supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

CmpBranch, bits [15:12]

Compare and branch. Indicates the supported combined Compare and Branch instructions.

The possible values of this field are:

0b0001

Supports CBNZ and CBZ instructions.

0b0011

Supports CBNZ and CBZ instructions along with non-predicated low overhead looping (WLS, DLS, LE and LCTP) and branch future (BF, BFX, BFL, BFLX, and BFCSEL) instructions.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

BitField, bits [11:8]

Bit field. Indicates the supported bit field instructions.

The possible values of this field are:

0b0001

BFC, BFI, SBFX, and UBFX supported.

All other values are reserved.

This field reads as 0b0001.

BitCount, bits [7:4]

Bit count. Indicates the supported bit count instructions.

The possible values of this field are:

0b0001

CLZ supported.

All other values are reserved.

This field reads as 0b0001.

Bits [3:0]

Reserved, **RES0**.

D1.2.131 ID_ISAR1, Instruction Set Attribute Register 1

The ID_ISAR1 characteristics are:

Purpose

Provides information about the instruction set implemented by the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000ED64.

Secure software can access the Non-secure version of this register via ID_ISAR1_NS located at 0xE002ED64. The location 0xE002ED64 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_ISAR1 bit assignments are:

31	28	27	24	23	20	19	16	15	12	11					0								
RES0				Interwork				Immediate				IfThen				Extend				RES0			

Bits [31:28]

Reserved, [RES0](#).

Interwork, bits [27:24]

Interworking. Indicates the implemented interworking instructions.

The possible values of this field are:

0b0010

BLX, BX, and loads to PC interwork.

All other values are reserved.

This field reads as 0b0010.

Immediate, bits [23:20]

Immediate. Indicates the implemented for data-processing instructions with long immediates.

The possible values of this field are:

0b0001

ADDW, MOVW, MOVT, and SUBW supported.

0b0010

Same as 0b0001.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

IfThen, bits [19:16]

If-Then. Indicates the implemented If-Then instructions.

The possible values of this field are:

0b0001

IT instruction supported.

All other values are reserved.

This field reads as 0b0001.

Extend, bits [15:12]

Extend. Indicates the implemented Extend instructions.

The possible values of this field are:

0b0001

SXTB, SXTB, UXTB, and UXTH.

0b0010

Adds SXTB16, SXTAB, SXTAB16, SXTAH, UXTB16, UXTAB, UXTAB16, and UXTAH, DSP Extension only.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [11:0]

Reserved, **RES0**.

D1.2.132 ID_ISAR2, Instruction Set Attribute Register 2

The ID_ISAR2 characteristics are:

Purpose

Provides information about the instruction set implemented by the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

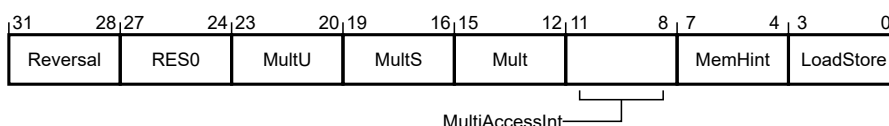
32-bit read-only register located at 0xE000ED68.

Secure software can access the Non-secure version of this register via ID_ISAR2_NS located at 0xE002ED68. The location 0xE002ED68 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_ISAR2 bit assignments are:



Reversal, bits [31:28]

Reversal. Indicates the implemented Reversal instructions.

The possible values of this field are:

0b0010

REV, REV16, REVSH and RBIT instructions supported.

All other values are reserved.

This field reads as 0b0010.

Bits [27:24]

Reserved, [RES0](#).

MultU, bits [23:20]

Multiply unsigned. Indicates the implemented advanced unsigned Multiply instructions.

The possible values of this field are:

0b0001

UMULL and UMLAL.

0b0010

Adds UMAAL, DSP Extension only.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

MultS, bits [19:16]

Multiply signed. Indicates the implemented advanced signed Multiply instructions.

The possible values of this field are:

0b0001

SMULL and SMLAL.

0b0011

Adds all saturating and DSP signed multiplies, DSP Extension only.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Mult, bits [15:12]

Multiplies. Indicates the implemented additional Multiply instructions.

The possible values of this field are:

0b0010

MUL, MLA, and MLS.

All other values are reserved.

This field reads as 0b0010.

MultiAccessInt, bits [11:8]

Multi-access instructions. Indicates the support for interruptible multi-access instructions.

The possible values of this field are:

0b0001

LDM and STM instructions are restartable.

0b0010

LDM and STM instructions, and if Armv8.1-M is implemented the CLRM instruction, are continuable.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

MemHint, bits [7:4]

Memory hints. Indicates the implemented Memory hint instructions.

The possible values of this field are:

0b0011

PLI and PLD instructions implemented.

All other values are reserved.

This field reads as 0b0011.

LoadStore, bits [3:0]

Load/store. Indicates the implemented additional load/store instructions.

The possible values of this field are:

0b0010

Supports load-acquire, store-release, and exclusive load and store instructions.

All other values are reserved.

This field reads as 0b0010.

D1.2.133 ID_ISAR3, Instruction Set Attribute Register 3

The ID_ISAR3 characteristics are:

Purpose

Provides information about the instruction set implemented by the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000ED6C.

Secure software can access the Non-secure version of this register via ID_ISAR3_NS located at 0xE002ED6C. The location 0xE002ED6C is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_ISAR3 bit assignments are:

31	28	27	24	23	20	19	16	15	12	11	8	7	4	3	0
RES0	TrueNOP			T32Copy		TabBranch		SynchPrim		SVC		SIMD		Saturate	

Bits [31:28]

Reserved, [RES0](#).

TrueNOP, bits [27:24]

True no-operation. Indicates the implemented true NOP instructions.

The possible values of this field are:

0b0001

NOP instruction and compatible hints implemented.

All other values are reserved.

This field reads as 0b0001.

T32Copy, bits [23:20]

T32 copy. Indicates the support for T32 non flag-setting MOV instructions.

The possible values of this field are:

0b0001

Encoding T1 of MOV (register) supports copying low register to low register.

All other values are reserved.

This field reads as 0b0001.

TabBranch, bits [19:16]

Table branch. Indicates the implemented Table Branch instructions.

The possible values of this field are:

0b0001

TBB and TBH implemented.

All other values are reserved.

This field reads as 0b0001.

SynchPrim, bits [15:12]

Synchronization primitives. Used in conjunction with ID_ISAR4.SynchPrim_frac to indicate the implemented synchronization primitive instructions.

The possible values of this field are:

0b0001

LDREX, STREX, LDREXB, STREXB, LDREXH, STREXH, and CLREX implemented.

All other values are reserved.

This field reads as 0b0001.

SVC, bits [11:8]

Supervisor Call. Indicates the implemented SVC instructions.

The possible values of this field are:

0b0001

SVC instruction implemented.

All other values are reserved.

This field reads as 0b0001.

SIMD, bits [7:4]

Single-instruction, multiple-data. Indicates the implemented SIMD instructions.

The possible values of this field are:

0b0001

SSAT, USAT, and Q-bit implemented.

0b0011

Adds all packed arithmetic and GE-bits, DSP Extension only.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Saturate, bits [3:0]

Saturate. Indicates the implemented saturating instructions.

The possible values of this field are:

0b0000

None implemented.

0b0001

QADD, QDADD, QDSUB, QSUB, and Q-bit implemented, DSP Extension only.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.134 ID_ISAR4, Instruction Set Attribute Register 4

The ID_ISAR4 characteristics are:

Purpose

Provides information about the instruction set implemented by the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

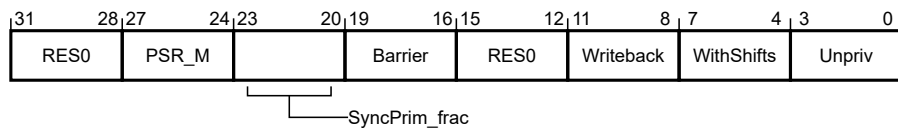
32-bit read-only register located at 0xE000ED70.

Secure software can access the Non-secure version of this register via ID_ISAR4_NS located at 0xE002ED70. The location 0xE002ED70 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_ISAR4 bit assignments are:



Bits [31:28]

Reserved, [RES0](#).

PSR_M, bits [27:24]

Program Status Registers M. Indicates the implemented M profile instructions to modify the PSRs.

The possible values of this field are:

0b0001

M profile forms of CPS, MRS, and MSR implemented.

All other values are reserved.

This field reads as 0b0001.

SyncPrim_frac, bits [23:20]

Synchronization primitives fractional. Used in conjunction with ID_ISAR3.SynchPrim to indicate the implemented synchronization primitive instructions.

The possible values of this field are:

0b0011

LDREX, STREX, CLREX, LDREXB, LDREXH, STREXB, and STREXH implemented.

All other values are reserved.

This field reads as 0b0011.

Barrier, bits [19:16]

Barrier. Indicates the implemented Barrier instructions.

The possible values of this field are:

0b0001

CSDB, DMB, DSB, ISB, PSSBB and SSBB barrier instructions implemented.

All other values are reserved.

This field reads as 0b0001.

Bits [15:12]

Reserved, [RES0](#).

Writeback, bits [11:8]

Writeback. Indicates the support for write-back addressing modes.

The possible values of this field are:

0b0001

All write-back addressing modes supported.

All other values are reserved.

This field reads as 0b0001.

WithShifts, bits [7:4]

With shifts. Indicates the support for write-back addressing modes.

The possible values of this field are:

0b0011

Support for constant shifts on load/store and other instructions.

All other values are reserved.

This field reads as 0b0011.

Unpriv, bits [3:0]

Unprivileged. Indicates the implemented unprivileged instructions.

The possible values of this field are:

0b0010

LDRBT, LDRHT, LDRSBT, LDRSHT, LDRT, STRBT, STRHT, and STRT implemented.

All other values are reserved.

This field reads as 0b0010.

D1.2.135 ID_ISAR5, Instruction Set Attribute Register 5

The ID_ISAR5 characteristics are:

Purpose

Provides information about the instruction set implemented by the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

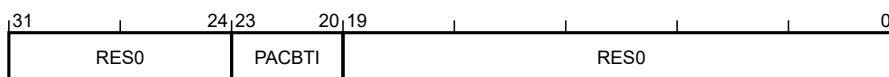
32-bit read-only register located at 0xE000ED74.

Secure software can access the Non-secure version of this register via ID_ISAR5_NS located at 0xE002ED74. The location 0xE002ED74 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_ISAR5 bit assignments are:



Bits [31:24]

Reserved, [RES0](#).

PACBTI, bits [23:20]

Pointer authentication algorithm. Indicates which version of QARMA or an [IMPLEMENTATION DEFINED](#) algorithm is implemented for address authentication in the PE.

The possible values of this field are:

0b0000

Address authentication and branch target identification is not implemented.

0b0001

Address authentication using the QARMA5 algorithm is implemented. Applies to all pointer authentication instructions.

0b0010

Address authentication using an [IMPLEMENTATION DEFINED](#) algorithm is implemented. Applies to all pointer authentication instructions.

0b0100

Address authentication using the QARMA3 algorithm is implemented. Applies to all pointer authentication instructions.

All other values are reserved.

If the PACBTI Extension is not implemented, this field is [RES0](#).

This field reads as an [IMPLEMENTATION DEFINED](#) value.

Bits [19:0]

Reserved, [RES0](#).

D1.2.136 ID_MMFR0, Memory Model Feature Register 0

The ID_MMFR0 characteristics are:

Purpose

Provides information about the implemented memory model and memory management support.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000ED50.

Secure software can access the Non-secure version of this register via ID_MMFR0_NS located at 0xE002ED50. The location 0xE002ED50 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_MMFR0 bit assignments are:

31		24	23	20	19	16	15	12	11	8	7	4	3	0
RES0				AuxReg		TCM		ShareLvl		OuterShr		PMSA		RES0

Bits [31:24]

Reserved, [RES0](#).

AuxReg, bits [23:20]

Auxiliary Registers. Indicates support for Auxiliary Control Registers.

The possible values of this field are:

0b0000

No Auxiliary Control Registers.

0b0001

Auxiliary Control Registers supported.

All other values are reserved.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

TCM, bits [19:16]

Tightly Coupled Memories. Indicates support for Tightly Coupled Memories (TCMs).

The possible values of this field are:

0b0000

None supported.

0b0001

TCMs supported with IMPLEMENTATION DEFINED control.

All other values are reserved.

This field reads as an IMPLEMENTATION DEFINED value.

ShareLvl, bits [15:12]

Shareability Levels. Indicates the number of Shareability levels implemented.

The possible values of this field are:

0b0000

One level of Shareability implemented.

0b0001

Two levels of Shareability implemented.

All other values are reserved.

This field reads as an IMPLEMENTATION DEFINED value.

OuterShr, bits [11:8]

Outermost Shareability. Indicates the outermost Shareability domain implemented.

The possible values of this field are:

0b0000

Implemented as Non-cacheable.

0b0001

Implemented with hardware coherency support.

0b1111

Shareability ignored.

All other values are reserved.

This field reads as an IMPLEMENTATION DEFINED value.

PMSA, bits [7:4]

Protected memory system architecture. Indicates support for the protected memory system architecture (PMSA).

The possible values of this field are:

0b0100

Supports PMSAv8.

All other values are reserved.

This field reads as 0b0100.

Bits [3:0]

Reserved, RES0.

D1.2.137 ID_MMFR1, Memory Model Feature Register 1

The ID_MMFR1 characteristics are:

Purpose

Provides information about the implemented memory model and memory management support.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000ED54.

Secure software can access the Non-secure version of this register via ID_MMFR1_NS located at 0xE002ED54. The location 0xE002ED54 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_MMFR1 bit assignments are:



Bits [31:0]

Reserved, [RES0](#).

D1.2.138 ID_MMFR2, Memory Model Feature Register 2

The ID_MMFR2 characteristics are:

Purpose

Provides information about the implemented memory model and memory management support.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

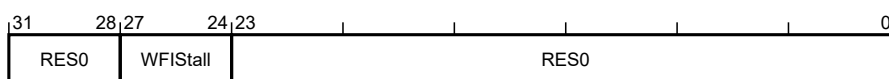
32-bit read-only register located at 0xE000ED58.

Secure software can access the Non-secure version of this register via ID_MMFR2_NS located at 0xE002ED58. The location 0xE002ED58 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_MMFR2 bit assignments are:



Bits [31:28]

Reserved, **RES0**.

WFISStall, bits [27:24]

WFI stall. Indicates the support for Wait For Interrupt (WFI) stalling.

The possible values of this field are:

0b0000

WFI never stalls.

0b0001

WFI has the ability to stall.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [23:0]

Reserved, **RES0**.

D1.2.139 ID_MMFR3, Memory Model Feature Register 3

The ID_MMFR3 characteristics are:

Purpose

Provides information about the implemented memory model and memory management support.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

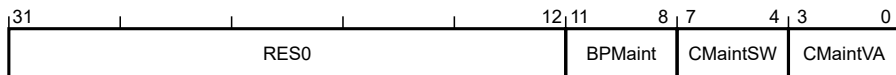
32-bit read-only register located at 0xE000ED5C.

Secure software can access the Non-secure version of this register via ID_MMFR3_NS located at 0xE002ED5C. The location 0xE002ED5C is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_MMFR3 bit assignments are:



Bits [31:12]

Reserved, [RES0](#).

BPMaint, bits [11:8]

Branch predictor maintenance. Indicates the supported branch predictor maintenance.

The possible values of this field are:

0b0000

None supported.

0b0001

Support for invalidate all of branch predictors.

All other values are reserved.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

CMaintSW, bits [7:4]

Cache maintenance set/way. Indicates the supported cache maintenance operations by set/way.

The possible values of this field are:

0b0000

None supported.

0b0001

Maintenance by set/way operations supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

CMaintVA, bits [3:0]

Cache maintenance by address. Indicates the supported cache maintenance operations by address.

The possible values of this field are:

0b0000

None supported.

0b0001

Maintenance by address and instruction cache invalidate all supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.140 ID_PFR0, Processor Feature Register 0

The ID_PFR0 characteristics are:

Purpose

Gives top-level information about the instruction set supported by the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000ED40.

Secure software can access the Non-secure version of this register via ID_PFR0_NS located at 0xE002ED40.

The location 0xE002ED40 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_PFR0 bit assignments are:

31	28	27						8	7			4	3	0
RAS		RES0							State1		State0			

RAS, bits [31:28]

RAS Extension. Identifies which version of the RAS extension is implemented.

The possible values of this field are:

0b0000

No RAS extension.

0b0010

Version 1 of the RAS extension implemented.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [27:8]

Reserved, **RES0**.

State1, bits [7:4]

T32 instruction set support.

The possible values of this field are:

0b0011

T32 instruction set including Thumb-2 Technology implemented.

All other values are reserved.

This field reads as 0b0011.

State0, bits [3:0]

A32 instruction set support.

The possible values of this field are:

0b0000

A32 instruction set not implemented.

All other values are reserved.

This field reads as 0b0000.

D1.2.141 ID_PFR1, Processor Feature Register 1

The ID_PFR1 characteristics are:

Purpose

Gives information about the programmers' model and Extensions support.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000ED44.

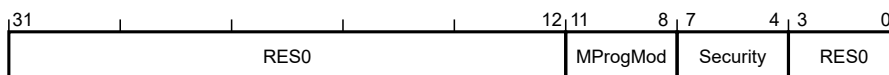
Secure software can access the Non-secure version of this register via ID_PFR1_NS located at 0xE002ED44.

The location 0xE002ED44 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The ID_PFR1 bit assignments are:



Bits [31:12]

Reserved, [RES0](#).

MProgMod, bits [11:8]

M programmers' model. Identifies support for the M-Profile programmers' model support.

The possible values of this field are:

0b0010

Two-stack programmers' model.

All other values are reserved.

This field reads as 0b0010.

Security, bits [7:4]

Security. Identifies whether the Security Extension is implemented.

The possible values of this field are:

0b0000

Security Extension not implemented.

0b0001

Security Extension implemented.

0b0011

Security Extension implemented with state handling instructions (VSCCLRM, CLRM, FPCXT access instructions and disabling SG Thread mode re-entrancy).

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [3:0]

Reserved, **RES0**.

D1.2.143 ITM_CIDR0, ITM Component Identification Register 0

The ITM_CIDR0 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is [RES0](#) if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is [RES0](#) if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

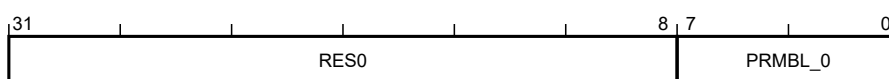
Attributes

32-bit read-only register located at 0xE0000FF0.

This register is not banked between Security states.

Field descriptions

The ITM_CIDR0 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

PRMBL_0, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0D.

D1.2.144 ITM_CIDR1, ITM Component Identification Register 1

The ITM_CIDR1 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

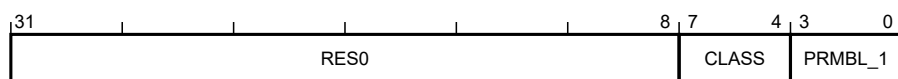
Attributes

32-bit read-only register located at 0xE0000FF4.

This register is not banked between Security states.

Field descriptions

The ITM_CIDR1 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

CLASS, bits [7:4]

CoreSight component class. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x9.

PRMBL_1, bits [3:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0.

D1.2.145 ITM_CIDR2, ITM Component Identification Register 2

The ITM_CIDR2 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is [RES0](#) if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is [RES0](#) if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

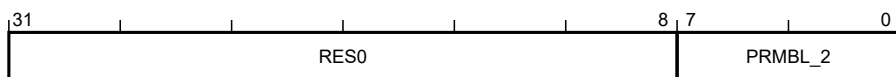
Attributes

32-bit read-only register located at 0xE0000FF8.

This register is not banked between Security states.

Field descriptions

The ITM_CIDR2 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

PRMBL_2, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x05.

D1.2.146 ITM_CIDR3, ITM Component Identification Register 3

The ITM_CIDR3 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is [RES0](#) if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is [RES0](#) if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

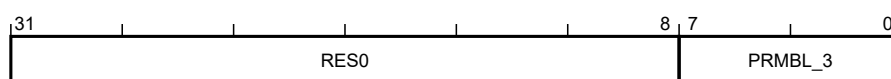
Attributes

32-bit read-only register located at 0xE0000FFC.

This register is not banked between Security states.

Field descriptions

The ITM_CIDR3 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

PRMBL_3, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0xB1.

D1.2.147 ITM_DEVARCH, ITM Device Architecture Register

The ITM_DEVARCH characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

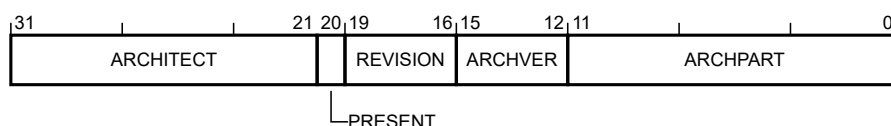
Attributes

32-bit read-only register located at 0xE0000FBC.

This register is not banked between Security states.

Field descriptions

The ITM_DEVARCH bit assignments are:



ARCHITECT, bits [31:21]

Architect. Defines the architect of the component. Bits [31:28] are the JEP106 continuation code (JEP106 bank ID, minus 1) and bits [27:21] are the JEP106 ID code.

The possible values of this field are:

0x23B

JEP106 continuation code 0x4, ID code 0x3B. Arm Limited.

Other values are defined by the JEDEC JEP106 standard.

This field reads as 0x23B.

PRESENT, bit [20]

DEVARCH Present. Defines that the DEVARCH register is present.

The possible values of this bit are:

1

DEVARCH information present.

This bit reads as one.

REVISION, bits [19:16]

Revision. Defines the architecture revision of the component.

The possible values of this field are:

0b0000

ITM architecture v2.0.

This field reads as 0b0000.

ARCHVER, bits [15:12]

Architecture Version. Defines the architecture version of the component.

The possible values of this field are:

0b0001

ITM architecture v2.

ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHVER is ARCHID[15:12].

This field reads as 0b0001.

ARCHPART, bits [11:0]

Architecture Part. Defines the architecture of the component.

The possible values of this field are:

0xA01

ITM architecture.

ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHPART is ARCHID[11:0].

This field reads as 0xA01.

D1.2.148 ITM_DEVTTYPE, ITM Device Type Register

The ITM_DEVTTYPE characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

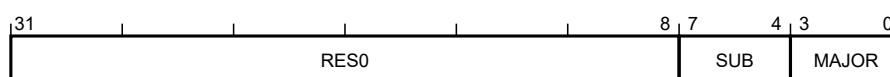
Attributes

32-bit read-only register located at 0xE000FCC.

This register is not banked between Security states.

Field descriptions

The ITM_DEVTTYPE bit assignments are:



Bits [31:8]

Reserved, **RES0**.

SUB, bits [7:4]

Sub-type. Component sub-type.

The possible values of this field are:

0x0

Other. Only permitted if the MAJOR field reads as 0x0.

0x4

Associated with a Bus, stimulus derived from bus activity. Only permitted if the MAJOR field reads as 0x3.

This field reads as an **IMPLEMENTATION DEFINED** value.

MAJOR, bits [3:0]

Major type. Component major type.

The possible values of this field are:

0x0

Miscellaneous.

0x3

Trace Source.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.149 ITM_LAR, ITM Software Lock Access Register

The ITM_LAR characteristics are:

Purpose

Provides CoreSight Software Lock control for the ITM, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted, but unprivileged writes are ignored.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank). It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

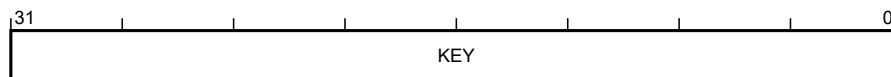
Attributes

32-bit write-only register located at 0xE0000FB0.

This register is not banked between Security states.

Field descriptions

The ITM_LAR bit assignments are:



KEY, bits [31:0]

Lock Access control.

Writing the key value 0xC5ACCE55 to this field unlocks the lock, enabling write accesses to the registers of this component through a memory-mapped interface.

Writing any other value to this register locks the lock, disabling write accesses to the registers of this component through a memory mapped interface.

D1.2.150 ITM_LSR, ITM Software Lock Status Register

The ITM_LSR characteristics are:

Purpose

Provides CoreSight Software Lock status information for the ITM, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank).

It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

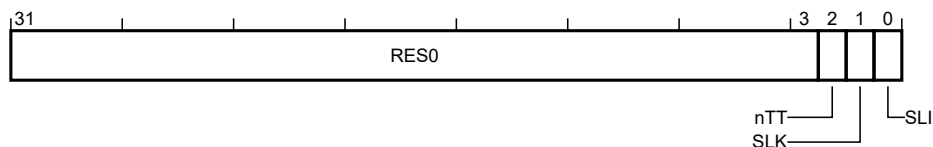
Attributes

32-bit read-only register located at 0xE0000FB4.

This register is not banked between Security states.

Field descriptions

The ITM_LSR bit assignments are:



Bits [31:3]

Reserved, **RES0**.

nTT, bit [2]

Not thirty-two bit. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as zero.

SLK, bit [1]

Software Lock status. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

0

Lock clear. Software writes are permitted to the registers of this component.

1

Lock set. Software writes to the registers of this component are ignored, and reads have no side effects.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is [RES0](#).

This bit resets to one on a Warm reset.

SLI, bit [0]

Software Lock implemented. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

0

Software Lock not implemented or debugger access.

1

Software Lock is implemented and software access.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is RAZ.

This bit reads as an [IMPLEMENTATION DEFINED](#) value.

D1.2.151 ITM_PIDR0, ITM Peripheral Identification Register 0

The ITM_PIDR0 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

Present only if the Main Extension is implemented.

This register is RES0 if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is RES0 if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is RES0 if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

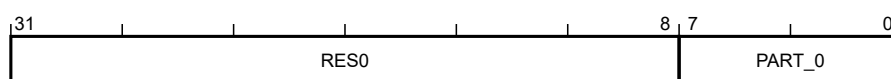
Attributes

32-bit read-only register located at 0xE000FE0.

This register is not banked between Security states.

Field descriptions

The ITM_PIDR0 bit assignments are:



Bits [31:8]

Reserved, RES0.

PART_0, bits [7:0]

Part number bits [7:0]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an IMPLEMENTATION DEFINED value.

D1.2.152 ITM_PIDR1, ITM Peripheral Identification Register 1

The ITM_PIDR1 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

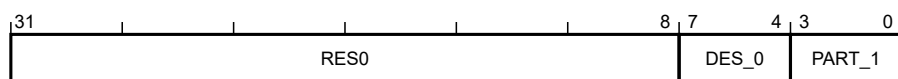
Attributes

32-bit read-only register located at 0xE0000FE4.

This register is not banked between Security states.

Field descriptions

The ITM_PIDR1 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

DES_0, bits [7:4]

JEP106 identification code bits [3:0]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

PART_1, bits [3:0]

Part number bits [11:8]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an IMPLEMENTATION DEFINED value.

D1.2.153 ITM_PIDR2, ITM Peripheral Identification Register 2

The ITM_PIDR2 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

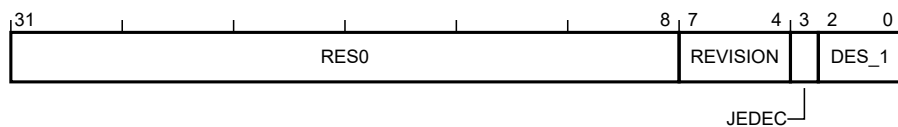
Attributes

32-bit read-only register located at 0xE0000FE8.

This register is not banked between Security states.

Field descriptions

The ITM_PIDR2 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REVISION, bits [7:4]

Component revision. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

JEDEC, bit [3]

JEDEC assignee value is used. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as one.

DES_1, bits [2:0]

JEP106 identification code bits [6:4]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.154 ITM_PIDR3, ITM Peripheral Identification Register 3

The ITM_PIDR3 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

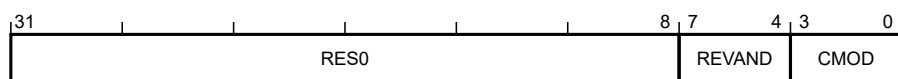
Attributes

32-bit read-only register located at 0xE0000FEC.

This register is not banked between Security states.

Field descriptions

The ITM_PIDR3 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

REVAND, bits [7:4]

RevAnd. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an IMPLEMENTATION DEFINED value.

CMOD, bits [3:0]

Customer Modified. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an IMPLEMENTATION DEFINED value.

D1.2.155 ITM_PIDR4, ITM Peripheral Identification Register 4

The ITM_PIDR4 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

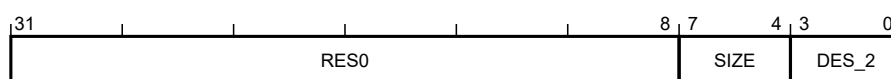
Attributes

32-bit read-only register located at 0xE0000FD0.

This register is not banked between Security states.

Field descriptions

The ITM_PIDR4 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

SIZE, bits [7:4]

4KB count. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as zero.

DES_2, bits [3:0]

JEP106 continuation code. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.156 ITM_PIDR5, ITM Peripheral Identification Register 5

The ITM_PIDR5 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is [RES0](#) if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is [RES0](#) if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

Attributes

32-bit read-only register located at 0xE0000FD4.

This register is not banked between Security states.

Field descriptions

The ITM_PIDR5 bit assignments are:



Bits [31:0]

Reserved, [RES0](#).

D1.2.157 ITM_PIDR6, ITM Peripheral Identification Register 6

The ITM_PIDR6 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is [RES0](#) if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is [RES0](#) if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

Attributes

32-bit read-only register located at 0xE0000FD8.

This register is not banked between Security states.

Field descriptions

The ITM_PIDR6 bit assignments are:



Bits [31:0]

Reserved, [RES0](#).

D1.2.158 ITM_PIDR7, ITM Peripheral Identification Register 7

The ITM_PIDR7 characteristics are:

Purpose

Provides CoreSight discovery information for the ITM.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if CoreSight identification is implemented.

This register is [RES0](#) if CoreSight identification is not implemented.

Present only if the ITM is implemented.

This register is [RES0](#) if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

Attributes

32-bit read-only register located at 0xE000FDC.

This register is not banked between Security states.

Field descriptions

The ITM_PIDR7 bit assignments are:



Bits [31:0]

Reserved, [RES0](#).

D1.2.159 ITM_STIMn, ITM Stimulus Port Register, n = 0 - 255

The ITM_STIM{0..255} characteristics are:

Purpose

Provides the interface for generating Instrumentation packets.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted, but unprivileged writes are ignored if ITM_TPR.PRIVMASK[n DIV 8] is set to one.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

All writes are ignored if ITM_TCR.ITMENA == 0 or ITM_TER{n DIV 32}.STIMENA[n MOD 32] == 0.

This register is word, halfword, and byte accessible.

Accesses that are not word aligned are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if the ITM is implemented.

This register is [RES0](#) if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

Attributes

32-bit read/write register located at 0xE0000000 + 4n.

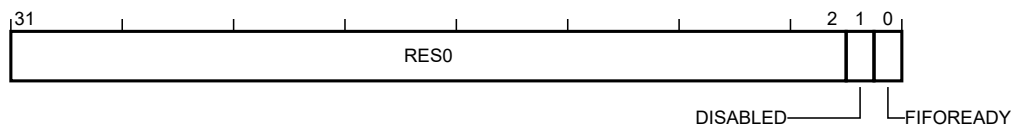
This register is not banked between Security states.

From Armv8.1-M onwards it is [IMPLEMENTATION DEFINED](#) whether a debugger write to this register is ignored when the PE is not in Debug state.

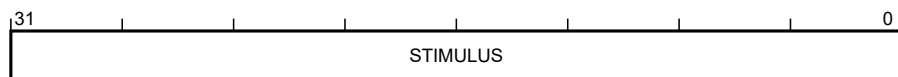
Field descriptions

The ITM_STIM{0..255} bit assignments are:

On a read:



On a write:



STIMULUS, bits [31:0], on a write

Stimulus data. Data to write to the stimulus port output buffer, for forwarding as an Instrumentation packet. The size of write access determines the type of Instrumentation packet generated.

Bits [31:2], on a read

Reserved, [RES0](#).

DISABLED, bit [1], on a read

Disabled. Indicates whether the stimulus port is enabled or disabled.

The possible values of this bit are:

0

Stimulus port and ITM are enabled.

1

Stimulus port or ITM is disabled.

FIFOREADY, bit [0], on a read

FIFO ready. Indicates whether the stimulus port can accept data.

The possible values of this bit are:

0

Stimulus port cannot accept data.

1

Stimulus port can accept at least one word.

D1.2.160 ITM_TCR, ITM Trace Control Register

The ITM_TCR characteristics are:

Purpose

Configures and controls transfers through the ITM interface.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted, but unprivileged writes are ignored.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

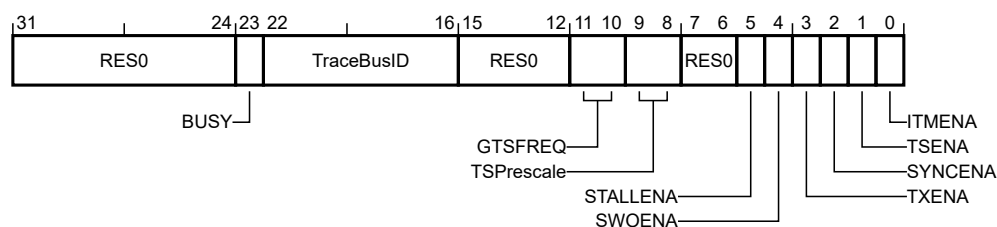
Attributes

32-bit read/write register located at 0xE0000E80.

This register is not banked between Security states.

Field descriptions

The ITM_TCR bit assignments are:



Bits [31:24]

Reserved, **RES0**.

BUSY, bit [23]

ITM busy. Indicates whether the ITM is currently processing events.

The possible values of this bit are:

0

ITM is not processing any events.

1

Events present and being drained.

Events means the ITM is generating or processing any of:

- Packets generated by the ITM from writes to Stimulus Ports.
- Other packets generated by the ITM itself.
- Packets generated by the DWT.

This bit is read-only.

TraceBusID, bits [22:16]

Trace bus identity. Identifier for multi-source trace stream formatting. If multi-source trace is in use, the debugger must write a unique nonzero trace ID value to this field.

The possible values of this field are:

0x00

Multi-source trace not in use.

0x01-0x6F

Unique trace ID value to be used for ITM trace packets.

All other values are reserved. If the ITM is the only trace source in the system, this field might be RAZ.

This field resets to an **UNKNOWN** value on a Cold reset.

Bits [15:12]

Reserved, **RES0**.

GTSFREQ, bits [11:10]

Global timestamp frequency. Defines how often the ITM generates a global timestamp, based on the global timestamp clock frequency, or disables generation of global timestamps.

The possible values of this field are:

0b00

Disable generation of Global Timestamp packets.

0b01

Generate timestamp request whenever the ITM detects a change in global timestamp counter bits $[N-1:7]$. This is approximately every 128 cycles.

0b10

Generate timestamp request whenever the ITM detects a change in global timestamp counter bits $[N-1:13]$. This is approximately every 8192 cycles.

0b11

Generate a timestamp after every packet, if the output FIFO is empty.

N is the size of the global timestamp counter.

If the implementation does not support global timestamping then these bits are reserved, RAZ/WI.

This field resets to zero on a Cold reset.

TSPrescale, bits [9:8]

Timestamp prescale. Local timestamp prescaler, used with the trace packet reference clock.

The possible values of this field are:

0b00

No prescaling.

0b01

Divide by 4.

0b10

Divide by 16.

0b11

Divide by 64.

If the processor does not implement the timestamp prescaler then these bits are reserved, RAZ/WI.

This field resets to zero on a Cold reset.

Bits [7:6]

Reserved, [RES0](#).

STALLENA, bit [5]

Stall enable. Stall the PE to guarantee delivery of Data Trace packets.

The possible values of this bit are:

0

Drop Hardware Source packets and generate an Overflow packet if the ITM output is stalled.

1

Stall the PE to guarantee delivery of Data Trace packets.

If stalling is not implemented, this bit is RAZ/WI.

SWOENA, bit [4]

SWO enable. Enables asynchronous clocking of the timestamp counter.

The possible values of this bit are:

0

Timestamp counter uses the processor system clock.

1

Timestamp counter uses asynchronous clock from the TPIU interface. The timestamp counter is held in reset while the output line is idle.

Which clocking modes are implemented is [IMPLEMENTATION DEFINED](#). If the implementation does not support both modes this bit is either RAZ or RAO, to indicate the implemented mode.

This bit resets to an [UNKNOWN](#) value on a Cold reset.

TXENA, bit [3]

Transmit enable. Enables forwarding of hardware event packets from the DWT unit or PMU to the ITM for output to the TPIU.

The possible values of this bit are:

0

Disabled.

1

Enabled.

It is [IMPLEMENTATION DEFINED](#) whether the DWT or PMU discards packets that cannot be forwarded to the ITM.

This bit resets to zero on a Cold reset.

Note

If a debugger changes this bit from 0 to 1, the DWT or PMU might forward a hardware event packet that it has previously generated.

SYNCENA, bit [2]

Synchronization enable. Enables Synchronization packet transmission for a synchronous TPIU.

The possible values of this bit are:

0

Disabled.

1

Enabled.

This bit resets to zero on a Cold reset.

Note

If a debugger sets this bit to 1 it must also configure DWT_CTRL.SYNCTAP for the correct synchronization speed.

TSENA, bit [1]

Timestamp enable. Enables Local timestamp generation.

The possible values of this bit are:

0

Disabled.

1

Enabled.

This bit resets to zero on a Cold reset.

ITMENA, bit [0]

ITM enable. Enables the ITM.

The possible values of this bit are:

0

Disabled.

1

Enabled.

This is the global enable for the ITM unit. A debugger must set this bit to 1 to permit writes to all Stimulus Port registers.

This bit resets to zero on a Cold reset.

D1.2.161 ITM_TERN, ITM Trace Enable Register, n = 0 - 7

The ITM_TERN{0..7} characteristics are:

Purpose

Provide an individual enable bit for each ITM_STIM register.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

Present only if the Main Extension is implemented.

This register is RES0 if the Main Extension is not implemented.

Present only if the ITM is implemented.

This register is RES0 if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

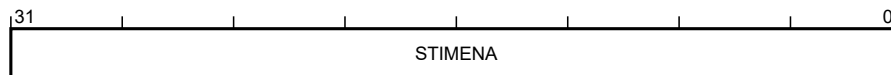
Attributes

32-bit read/write register located at $0xE0000E00 + 4n$.

This register is not banked between Security states.

Field descriptions

The ITM_TERN{0..7} bit assignments are:



STIMENA, bits [31:0]

Stimulus enable. For STIMENA[m] in ITM_TERN, controls whether stimulus port ITM_STIM<32n+m> is enabled.

The possible values of each bit are:

0

Stimulus port (32n + m) disabled.

1

Stimulus port (32n + m) enabled.

Bits corresponding to unimplemented stimulus ports are RAZ/WI. Unprivileged writes to ITM_TERN do not update STIMENA[m] if ITM_TPR.PRIVMASK[(32n+m) DIV 8] is set to 1.

This field resets to zero on a Cold reset.

D1.2.162 ITM_TPR, ITM Trace Privilege Register

The ITM_TPR characteristics are:

Purpose

Controls which stimulus ports can be accessed by unprivileged code.

Usage constraints

If the Main Extension is implemented, both privileged and unprivileged accesses are permitted, but unprivileged writes are ignored.

If the Main Extension is not implemented, unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if the ITM is implemented.

This register is **RES0** if the ITM is not implemented.

If the Main Extension is not implemented then the ITM is not implemented.

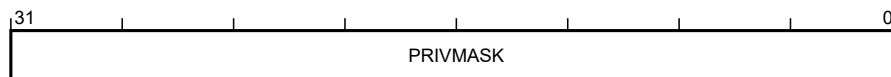
Attributes

32-bit read/write register located at 0xE0000E40.

This register is not banked between Security states.

Field descriptions

The ITM_TPR bit assignments are:

**PRIVMASK, bits [31:0]**

Privilege mask. For PRIVMASK[*m*], defines the access permissions of stimulus ports ITM_STIM<8*m*> to ITM_STIM<8*m*+7> inclusive.

The possible values of each bit are:

0

Unprivileged access permitted.

1

Privileged access only.

Bits corresponding to unimplemented stimulus ports are RAZ/WI.

This field resets to zero on a Cold reset.

D1.2.163 LO_BRANCH_INFO, Loop and branch tracking information

The LO_BRANCH_INFO characteristics are:

Purpose

Holds the cached loop end point and branching information.

Usage constraints

This register is not accessible from software.

Configurations

Present only if version Armv8.1-M of the architecture and LOB are implemented.

This cache is not implemented otherwise.

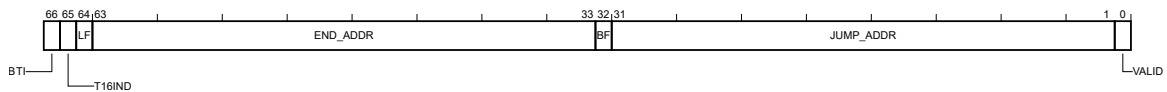
Attributes

67-bit read/write register.

This register is not banked between Security states.

Field descriptions

The LO_BRANCH_INFO bit assignments are:



BTI, bit [66]

When set this field indicates that the BF is a BTI setting instruction. EPSR.B will be set to one when BTI is enabled (see CONTROL register) and the LO_BRANCH_INFO.VALID bit is set.

T16IND, bit [65]

When set this field indicates that BF is a 16-bit T32 indirect branch. For BF and link instructions, this flag calculates the offset of the return address set in LR from the branch point.

LF, bit [64]

Link / forever. If BF is set, this field indicates that the link register is populated with a return address at the point the branch is taken. If BF is clear, this flag indicates a forever loop that does not decrement LR at the end of each loop iteration.

END_ADDR, bits [63:33]

The partial address of either the last instruction in a low-overhead-loop, or an upcoming branch set by a Branch Future instruction. If the VALID bit is set and this field matches the address of the next instruction, a branch back to the start of the loop is triggered (as specified by the JUMP_ADDR field).

BF, bit [32]

Indicates that the value in this register originates from a BF instruction.

JUMP_ADDR, bits [31:1]

The address to jump to when an end address match is detected.

VALID, bit [0]

The cached loop information in the rest of this register is only valid if this bit is set. The PE is permitted to clear this bit and invalidate the cache at any point.

This bit resets to zero on a Warm reset.

D1.2.164 LR, Link Register

The LR characteristics are:

Purpose

Exception and procedure call link register.

Usage constraints

Privileged and unprivileged access permitted.

Configurations

This register is always implemented.

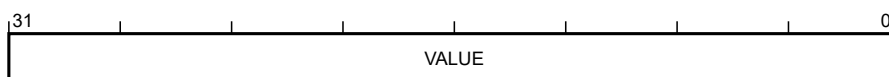
Attributes

32-bit read/write special-purpose register.

This register is not banked between Security states.

Field descriptions

The LR bit assignments are:



VALUE, bits [31:0]

Link register. 32-bit link register updated to hold a return address, FNC_RETURN or EXC_RETURN on a function call or exception entry. LR can be used as a general-purpose register.

This field resets to an **UNKNOWN** value on Warm reset when the Main Extension is not implemented.

This field resets to 0xFFFFFFFF on a Warm reset if the Main Extension is implemented.

D1.2.165 MAIR_ATTR, Memory Attribute Indirection Register Attributes

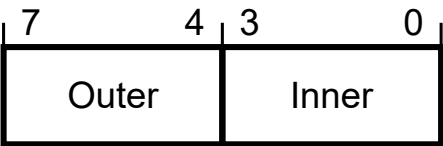
The MAIR_ATTR characteristics are:

- Purpose**
Defines the memory attribute encoding for use in the MPU_MAIR0 and MPU_MAIR1.
- Usage constraints**
None.
- Configurations**
All.
- Attributes**
8-bit payload.

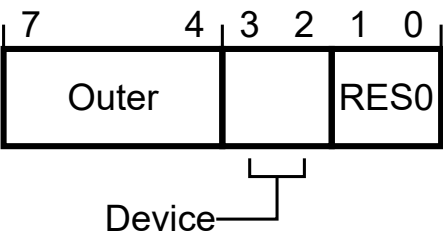
Field descriptions

The MAIR_ATTR bit assignments are:

When Outer != 0b0000:



When Outer == 0b0000:



- Outer, bits [7:4]**
Outer attributes. Specifies the Outer memory attributes.
- The possible values of this field are:
- 0b0000**
Device memory.
 - 0b00RW**
Normal memory, Outer Write-Through transient (RW!=0b00).
 - 0b0100**
Normal memory, Outer Non-cacheable.
 - 0b01RW**
Normal memory, Outer Write-Back Transient (RW!=0b00).
 - 0b10RW**
Normal memory, Outer Write-Through Non-transient.

0b11RW

Normal memory, Outer Write-Back Non-transient.

R and W specify the outer read and write allocation policy: 0 = do not allocate, 1 = allocate.

Device, bits [3:2], when Outer == 0b0000

Device attributes. Specifies the memory attributes for Device.

The possible values of this field are:

0b00

Device-nGnRnE.

0b01

Device-nGnRE.

0b10

Device-nGRE.

0b11

Device-GRE.

Bits [1:0], when Outer == 0b0000

Reserved, [RES0](#).

Inner, bits [3:0], when Outer != 0b0000

Inner attributes. Specifies the Inner memory attributes.

The possible values of this field are:

0b0000

[UNPREDICTABLE](#).

0b00RW

Normal memory, Inner Write-Through Transient (RW!=0b00).

0b0100

Normal memory, Inner Non-cacheable.

0b01RW

Normal memory, Inner Write-Back Transient (RW!=0b00).

0b10RW

Normal memory, Inner Write-Through Non-transient.

0b11RW

Normal memory, Inner Write-Back Non-transient.

R and W specify the inner read and write allocation policy: 0 = do not allocate, 1 = allocate.

D1.2.166 MMFAR, MemManage Fault Address Register

The MMFAR characteristics are:

Purpose

Shows the address of the memory location that caused an MPU fault.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read/write register located at 0xE000ED34.

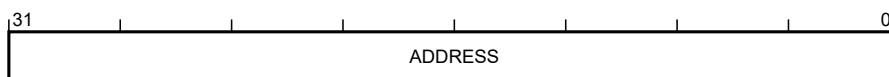
Secure software can access the Non-secure version of this register via MMFAR_NS located at 0xE002ED34. The location 0xE002ED34 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The MMFAR bit assignments are:



ADDRESS, bits [31:0]

Data address for an MemManage fault. This register is updated with the address of a location that produced a MemManage fault. The MMFSR shows the cause of the fault, and whether this field is valid. This field is valid only when MMFSR.MMFARVALID is set, otherwise it is **UNKNOWN**.

In implementations without unique BFAR, SFAR and MMFAR registers, the value of this register is **UNKNOWN** if BFSR.BFARVALID or SFSR.SFARVALID is set.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.167 MMFSR, MemManage Fault Status Register

The MMFSR characteristics are:

Purpose

Shows the status of MPU faults.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If the Main Extension is implemented, this register is word, halfword, and byte accessible.

If the Main Extension is not implemented, this register is word accessible only, halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

8-bit read/write-one-to-clear register located at 0xE000ED28.

Secure software can access the Non-secure version of this register via MMFSR_NS located at 0xE002ED28. The location 0xE002ED28 is **RES0** to software executing in Non-secure state and the debugger.

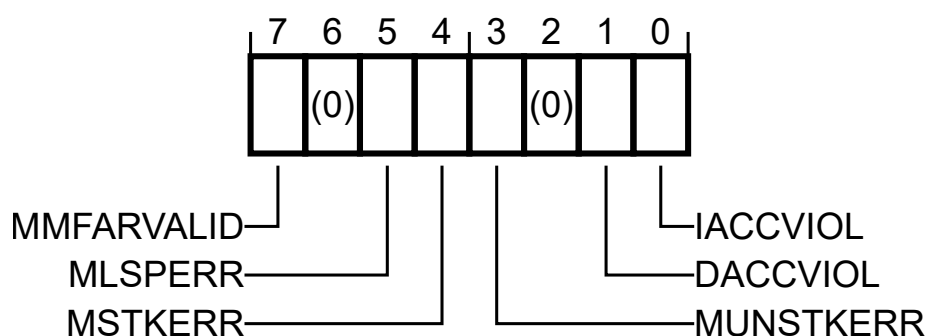
This register is banked between Security states.

This register is part of CFSR.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The MMFSR bit assignments are:



MMFARVALID, bit [7]

MMFAR valid flag. Indicates validity of the MMFAR register.

The possible values of this bit are:

0

MMFAR content not valid.

1

MMFAR content valid.

This bit resets to zero on a Warm reset.

Bit [6]

Reserved, [RES0](#).

MLSPERR, bit [5]

MemManage lazy Floating-point state preservation error flag. Records whether a MemManage fault occurred during lazy Floating-point state preservation.

The possible values of this bit are:

0

No MemManage occurred.

1

MemManage occurred.

This bit resets to zero on a Warm reset.

MSTKERR, bit [4]

MemManage stacking error flag. Records whether a derived MemManage fault occurred during exception entry stacking.

The possible values of this bit are:

0

No derived MemManage occurred.

1

Derived MemManage occurred during exception entry.

This bit resets to zero on a Warm reset.

MUNSTKERR, bit [3]

MemManage unstacking error flag. Records whether a derived MemManage fault occurred during exception return unstacking.

The possible values of this bit are:

0

No derived MemManage fault occurred.

1

Derived MemManage fault occurred during exception return.

This bit resets to zero on a Warm reset.

Bit [2]

Reserved, [RES0](#).

DACCVIOL, bit [1]

Data access violation flag. Records whether a data access violation has occurred.

The possible values of this bit are:

0

No MemManage fault on data access has occurred.

1

MemManage fault on data access has occurred.

A DACCVIOL will be accompanied by an MMFAR update.

This bit resets to zero on a Warm reset.

IACCVIOL, bit [0]

Instruction access violation. Records whether an instruction related memory access violation has occurred.

The possible values of this bit are:

0

No MemManage fault on instruction access has occurred.

1

MemManage fault on instruction access has occurred.

An IACCVIOL is only recorded if a faulted instruction is executed.

This bit resets to zero on a Warm reset.

D1.2.168 MPU_CTRL, MPU Control Register

The MPU_CTRL characteristics are:

Purpose

Enables the MPU and, when the MPU is enabled, controls whether the system address map is enabled as a background region for privileged accesses, and whether the MPU is enabled for HardFaults, NMIs, and exception handlers when FAULTMASK is set to 1.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

If no MPU regions are implemented, this register is RAZ/WI.

Attributes

32-bit read/write register located at 0xE000ED94.

To allow lock down of this register it is **IMPLEMENTATION DEFINED** whether this register is writable.

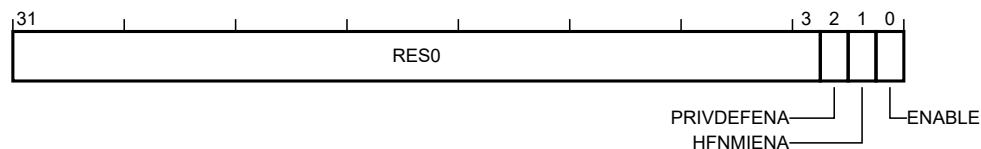
Secure software can access the Non-secure version of this register via MPU_CTRL_NS located at 0xE002ED94. The location 0xE002ED94 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The MPU_CTRL bit assignments are:

**Bits [31:3]**

Reserved, RES0.

PRIVDEFENA, bit [2]

Privileged default enable. Controls whether the system address map is enabled for privileged software.

The possible values of this bit are:

0

Use of the system address map disabled. Any instruction or data access that does not access a defined memory region faults.

1

Enables the system address map as a memory region for privileged accesses only.

When the **ENABLE** bit is set to 0, the PE ignores the **PRIVDEFENA** bit. If no regions are enabled and the **PRIVDEFENA** and **ENABLE** bits are set to 1, only privileged code can execute from the system address map.

This bit resets to zero on a Warm reset.

HFNMIENA, bit [1]

HardFault, NMI enable. Controls whether the PE with a requested execution priority of less than 0 accesses memory with the MPU enabled or disabled. This applies to accesses by the handlers of HardFault and NMI, and all accesses when FAULTMASK is set to 1.

The possible values of this bit are:

0

MPU disabled for these accesses.

1

MPU enabled for these accesses.

If HFNMIENA is set to 1 when ENABLE is set to 0, behavior is [UNPREDICTABLE](#).

This bit resets to zero on a Warm reset.

ENABLE, bit [0]

Enable. Enables the MPU.

The possible values of this bit are:

0

The MPU is disabled.

1

The MPU is enabled.

Disabling the MPU, by setting the ENABLE bit to 0, means that privileged and unprivileged accesses use the system address map.

This bit resets to zero on a Warm reset.

D1.2.169 MPU_MAIR0, MPU Memory Attribute Indirection Register 0

The MPU_MAIR0 characteristics are:

Purpose

Along with MPU_MAIR1, provides the memory attribute encodings corresponding to the AttrIdx values.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

If no MPU regions are implemented, this register is RAZ/WI.

Attributes

32-bit read/write register located at 0xE000EDC0.

To allow lock down of this register it is IMPLEMENTATION DEFINED whether this register is writable.

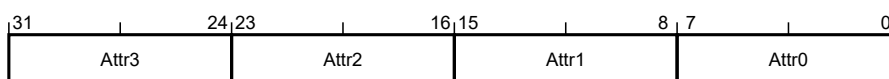
Secure software can access the Non-secure version of this register via MPU_MAIR0_NS located at 0xE002EDC0. The location 0xE002EDC0 is RES0 to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The MPU_MAIR0 bit assignments are:



Attrm, bits [8m+7:8m], for m = 0 to 3

Attribute m. Memory attribute encoding for MPU regions with an AttrIdx of m.

The possible values of this field are:

All

See MAIR_ATTR for encoding.

This field resets to an Architecturally UNKNOWN value on a Warm reset.

D1.2.170 MPU_MAIR1, MPU Memory Attribute Indirection Register 1

The MPU_MAIR1 characteristics are:

Purpose

Along with MPU_MAIR0, provides the memory attribute encodings corresponding to the AttrIdx values.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

If no MPU regions are implemented, this register is RAZ/WI.

Attributes

32-bit read/write register located at 0xE000EDC4.

To allow lock down of this register it is **IMPLEMENTATION DEFINED** whether this register is writable.

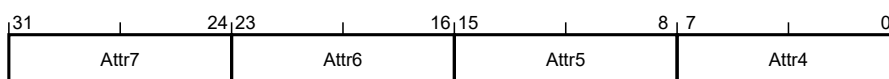
Secure software can access the Non-secure version of this register via MPU_MAIR1_NS located at 0xE002EDC4. The location 0xE002EDC4 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The MPU_MAIR1 bit assignments are:



Attrm, bits [8(m-4)+7:8(m-4)], for m = 4 to 7

Attribute m. Memory attribute encoding for MPU regions with an AttrIdx of m.

The possible values of this field are:

All

See MAIR_ATTR for encoding.

This field resets to an **Architecturally UNKNOWN** value on a Warm reset.

D1.2.171 MPU_RBAR, MPU Region Base Address Register

The MPU_RBAR characteristics are:

Purpose

Provides indirect read and write access to the base address of the currently selected MPU region for the selected Security state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

If no MPU regions are implemented, this register is RAZ/WI.

Attributes

32-bit read/write register located at 0xE000ED9C.

To allow lock down of this register it is **IMPLEMENTATION DEFINED** whether this register is writable.

Secure software can access the Non-secure version of this register via MPU_RBAR_NS located at 0xE002ED9C. The location 0xE002ED9C is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

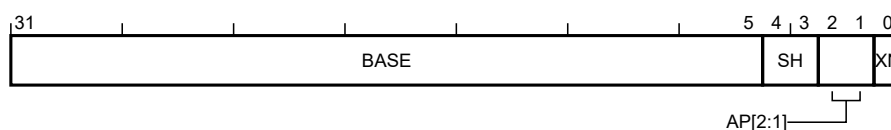
From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

This register provides access to the configuration of the MPU region selected by MPU_RNR.REGION for the appropriate Security state. The field descriptions apply to the currently selected region.

Field descriptions

The MPU_RBAR bit assignments are:



BASE, bits [31:5]

Base address. Contains bits [31:5] of the lower inclusive limit of the selected MPU memory region. This value is zero extended to provide the base address to be checked against.

It is **IMPLEMENTATION DEFINED** whether any of the BASE bits are WI

This field resets to an **Architecturally UNKNOWN** value on a Warm reset.

SH, bits [4:3]

Shareability. Defines the Shareability domain of this region for Normal memory.

The possible values of this field are:

0b00

Non-shareable.

0b10

Outer Shareable.

0b11

Inner Shareable.

All other values are reserved.

For any type of Device memory, the value of this field is ignored.

This field resets to an **UNKNOWN** value on a Warm reset.

AP[2:1], bits [2:1]

Access permissions. Defines the access permissions for this region.

The possible values of this field are:

0b00

Read/write by privileged code only.

0b01

Read/write by any privilege level.

0b10

Read-only by privileged code only.

0b11

Read-only by any privilege level.

This field resets to an **UNKNOWN** value on a Warm reset.

XN, bit [0]

Execute-never. Defines whether code can be executed from this region.

The possible values of this bit are:

0

Execution only permitted if read permitted.

1

Execution not permitted.

This bit resets to an **UNKNOWN** value on a Warm reset.

D1.2.172 MPU_RBAR_An, MPU Region Base Address Register Alias, n = 1 - 3

The MPU_RBAR_A{1..3} characteristics are:

Purpose

Provides indirect read and write access to the base address of the MPU region selected by MPU_RNR[7:2):(n[1:0]) for the selected Security state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

Present only if the Main Extension is implemented.

This register is RES0 if the Main Extension is not implemented.

If no MPU regions are implemented, this register is RAZ/WI.

Attributes

32-bit read/write register located at $0xE000EDA4 + 8(n-1)$.

To allow lock down of this register it is IMPLEMENTATION DEFINED whether this register is writable.

Secure software can access the Non-secure version of this register via MPU_RBAR_An_NS located at $0xE002EDA4 + 8(n-1)$. The location $0xE002EDA4 + 8(n-1)$ is RES0 to software executing in Non-secure state and the debugger.

This register is banked between Security states.

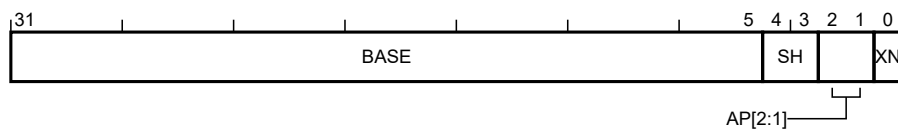
From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

This register is an alias of the MPU_RBAR register and provides access to the configuration of the MPU region selected by MPU_RNR.REGION had REGION[1:0] been set to n[1:0].

Field descriptions

The MPU_RBAR_A{1..3} bit assignments are:



BASE, bits [31:5]

Base address. Contains bits [31:5] of the lower inclusive limit of the selected MPU memory region. This value is zero extended to provide the base address to be checked against.

It is IMPLEMENTATION DEFINED whether any of the BASE bits are WI

This field resets to an Architecturally UNKNOWN value on a Warm reset.

SH, bits [4:3]

Shareability. Defines the Shareability domain of this region for Normal memory.

The possible values of this field are:

0b00

Non-shareable.

0b10

Outer Shareable.

0b11

Inner Shareable.

All other values are reserved.

For any type of Device memory, the value of this field is ignored.

This field resets to an **UNKNOWN** value on a Warm reset.

AP[2:1], bits [2:1]

Access permissions. Defines the access permissions for this region.

The possible values of this field are:

0b00

Read/write by privileged code only.

0b01

Read/write by any privilege level.

0b10

Read-only by privileged code only.

0b11

Read-only by any privilege level.

This field resets to an **UNKNOWN** value on a Warm reset.

XN, bit [0]

Execute-never. Defines whether code can be executed from this region.

The possible values of this bit are:

0

Execution only permitted if read permitted.

1

Execution not permitted.

This bit resets to an **UNKNOWN** value on a Warm reset.

D1.2.173 MPU_RLAR, MPU Region Limit Address Register

The MPU_RLAR characteristics are:

Purpose

Provides indirect read and write access to the limit address of the currently selected MPU region for the selected Security state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

If no MPU regions are implemented, this register is RAZ/WI.

Attributes

32-bit read/write register located at 0xE000EDA0.

To allow lock down of this register it is **IMPLEMENTATION DEFINED** whether this register is writable.

Secure software can access the Non-secure version of this register via MPU_RLAR_NS located at 0xE00EDA0. The location 0xE00EDA0 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

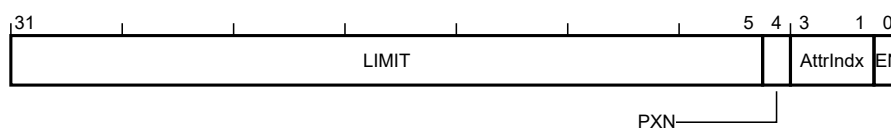
From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

This register provides access to the configuration of the MPU region selected by MPU_RNR.REGION for the appropriate Security state. The field descriptions apply to the currently selected region.

Field descriptions

The MPU RLAR bit assignments are:

**LIMIT, bits [31:5]**

Limit address. Contains bits [31:5] of the upper inclusive limit of the selected MPU memory region. This value is postfixed with 0x1F to provide the limit address to be checked against.

It is **IMPLEMENTATION DEFINED** whether any of the LIMIT bits are WI

This field resets to an Architecturally UNKNOWN value on a Warm reset.

PXN, bit [4]

Privileged execute-never. Defines whether code can be executed from this privileged region.

The possible values of this bit are:

O

Execution only permitted if read permitted.

1

Execution from a privileged mode is not permitted.

If version Armv8.1-M of the architecture is not implemented, this bit is [RES0](#).

This bit resets to an [UNKNOWN](#) value on a Warm reset.

AttrIndx, bits [3:1]

Attribute index. Associates a set of attributes in the MPU_MAIRO and MPU_MAIR1 fields.

This field resets to an [UNKNOWN](#) value on a Warm reset.

EN, bit [0]

Enable. Region enable.

The possible values of this bit are:

0

Region disabled.

1

Region enabled.

This bit resets to zero on a Warm reset.

D1.2.174 MPU_RLAR_An, MPU Region Limit Address Register Alias, n = 1 - 3

The MPU_RLAR_A{1..3} characteristics are:

Purpose

Provides indirect read and write access to the limit address of the currently selected MPU region selected by MPU_RNR[7:2]:(n[1:0]) for the selected Security state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

If no MPU regions are implemented, this register is RAZ/WI.

Attributes

32-bit read/write register located at $0xE00EDA8 + 8(n-1)$.

To allow lock down of this register it is **IMPLEMENTATION DEFINED** whether this register is writable.

Secure software can access the Non-secure version of this register via MPU_RLAR_An_NS located at $0xE00EDA8 + 8(n-1)$. The location $0xE00EDA8 + 8(n-1)$ is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

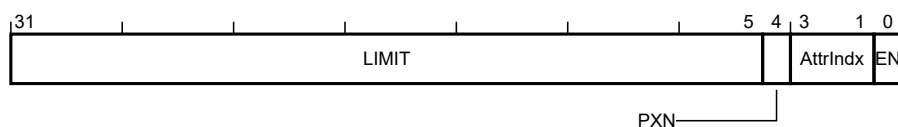
From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

This register is an alias of the MPU_RLAR register and provides access to the configuration of the MPU region selected by MPU_RNR.REGION had REGION[1:0] been set to $n[1:0]$.

Field descriptions

The MPU_RLAR_A{1..3} bit assignments are:

**LIMIT, bits [31:5]**

Limit address. Contains bits [31:5] of the upper inclusive limit of the selected MPU memory region. This value is postfixed with 0x1F to provide the limit address to be checked against.

It is **IMPLEMENTATION DEFINED** whether any of the LIMIT bits are WI

This field resets to an Architecturally UNKNOWN value on a Warm reset.

PXN, bit [4]

Privileged execute-never. Defines whether code can be executed from this privileged region.

The possible values of this bit are:

0
Execution only permitted if read permitted.

1
Execution from a privileged mode is not permitted.
If version Armv8.1-M of the architecture is not implemented, this bit is [RES0](#).
This bit resets to an [UNKNOWN](#) value on a Warm reset.

AttrIndx, bits [3:1]

Attribute index. Associates a set of attributes in the MPU_MAIRO and MPU_MAIR1 fields.
This field resets to an [UNKNOWN](#) value on a Warm reset.

EN, bit [0]

Enable. Region enable.
The possible values of this bit are:

0
Region disabled.

1
Region enabled.

This bit resets to zero on a Warm reset.

D1.2.175 MPU_RNR, MPU Region Number Register

The MPU_RNR characteristics are:

Purpose

Selects the region currently accessed by MPU_RBAR and MPU_RLAR.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

If no MPU regions are implemented, this register is RAZ/WI.

Attributes

32-bit read/write register located at 0xE000ED98.

To allow lock down of this register it is IMPLEMENTATION DEFINED whether this register is writable.

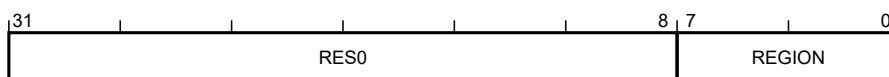
Secure software can access the Non-secure version of this register via MPU_RNR_NS located at 0xE002ED98. The location 0xE002ED98 is RES0 to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The MPU_RNR bit assignments are:



Bits [31:8]

Reserved, RES0.

REGION, bits [7:0]

Region number. Indicates the memory region accessed by MPU_RBAR and MPU_RLAR.

Writing a value corresponding to an unimplemented region is CONSTRAINED UNPREDICTABLE.

This field resets to an UNKNOWN value on a Warm reset.

D1.2.176 MPU_TYPE, MPU Type Register

The MPU_TYPE characteristics are:

Purpose

The MPU Type Register indicates how many regions the MPU for the selected Security state supports.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

This register is always implemented.

Attributes

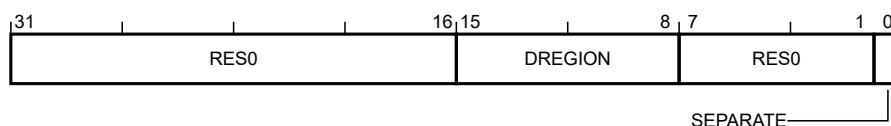
32-bit read-only register located at 0xE000ED90.

Secure software can access the Non-secure version of this register via MPU_TYPE_NS located at 0xE002ED90. The location 0xE002ED90 is RES0 to software executing in Non-secure state and the debugger.

This register is banked between Security states.

Field descriptions

The MPU_TYPE bit assignments are:



Bits [31:16]

Reserved, RES0.

DREGION, bits [15:8]

Data regions. Number of regions supported by the MPU.

If this field reads-as-zero, the PE does not implement an MPU for the selected Security state.

This field reads as an IMPLEMENTATION DEFINED value.

Bits [7:1]

Reserved, RES0.

SEPARATE, bit [0]

Separate. Indicates support for separate instructions and data address regions.

Armv8-M only supports unified MPU regions.

This bit reads as zero.

D1.2.177 MSPLIM, Main Stack Pointer Limit Register

The MSPLIM characteristics are:

Purpose

Holds the lower limit of the Main stack pointer.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

This register is always implemented.

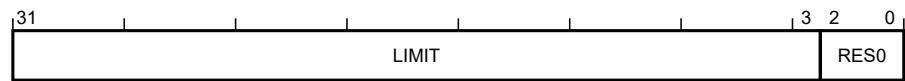
Attributes

32-bit read/write special-purpose register.

This register is banked between Security states.

Field descriptions

The MSPLIM bit assignments are:



LIMIT, bits [31:3]

Stack limit. Bits [31:3] of the Main stack pointer limit address for the selected Security state.

Many instructions and exception entry will generate an exception if the appropriate stack pointer would be updated to a value lower than this limit. If the Main Extension is not implemented, the Non-secure MSPLIM is [RES0](#).

This field resets to zero on a Warm reset.

Bits [2:0]

Reserved, [RES0](#).

D1.2.178 MVFR0, Media and VFP Feature Register 0

The MVFR0 characteristics are:

Purpose

Describes the features provided by the Floating-point Extension.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000EF40.

Secure software can access the Non-secure version of this register via MVFR0_NS located at 0xE002EF40.

The location 0xE002EF40 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The MVFR0 bit assignments are:

31	28, 27	24, 23	20, 19	16, 15	12, 11	8, 7	4, 3	0
FPRound	RES0	FPSqrt	FPDivide	RES0	FPDP	FPSP	SIMDReg	

FPRound, bits [31:28]

Floating-point rounding modes. Indicates the rounding modes supported by the Floating-point Extension.

The possible values of this field are:

0b0000

Not supported.

0b0001

All rounding modes supported.

All other values are reserved.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

Bits [27:24]

Reserved, [RES0](#).

FPSqrt, bits [23:20]

Floating-point square root. Indicates the support for Floating-point square root operations.

The possible values of this field are:

0b0000

Not supported.

0b0001

Supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

FPDivide, bits [19:16]

Floating-point divide. Indicates the support for Floating-point divide operations.

The possible values of this field are:

0b0000

Not supported.

0b0001

Supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [15:12]

Reserved, **RES0**.

FPDP, bits [11:8]

Floating-point double-precision. Indicates support for Floating-point double-precision operations.

The possible values of this field are:

0b0000

Not supported.

0b0010

Supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

FPSP, bits [7:4]

Floating-point single-precision. Indicates support for Floating-point single-precision operations.

The possible values of this field are:

0b0000

Not supported.

0b0010

Supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

SIMDReg, bits [3:0]

SIMD registers. Indicates size of Floating-Point Extension register file.

The possible values of this field are:

0b0001

16 x 64-bit registers.

All other values are reserved.

This field reads as 0b0001.

D1.2.179 MVFR1, Media and VFP Feature Register 1

The MVFR1 characteristics are:

Purpose

Describes the features provided by the Floating-point Extension.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000EF44.

Secure software can access the Non-secure version of this register via MVFR1_NS located at 0xE002EF44. The location 0xE002EF44 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The MVFR1 bit assignments are:

31	28,27	24,23	20,19	12,11	8,7	4,3	0
FMAC	FPHP	FP16	RES0	MVE	FPDNaN	FPFtZ	

FMAC, bits [31:28]

Fused multiply accumulate. Indicates whether the Floating-point Extension implements the fused multiply accumulate instructions.

The possible values of this field are:

0b0000

Not supported.

0b0001

Implemented.

All other values are reserved.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

FPHP, bits [27:24]

Floating-point half-precision conversion. Indicates whether the Floating-point Extension implements half-precision Floating-point conversion instructions.

The possible values of this field are:

0b0000

Not supported.

0b0001

Half-precision to single-precision implemented.

0b0010

Half-precision to single and double-precision implemented.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

FP16, bits [23:20]

Floating-point half-precision data processing. Indicates whether the Floating-point Extension implements half-precision floating-point data processing instructions.

The possible values of this field are:

0b0000

No Half-precision data processing support.

0b0001

Half-precision data processing instructions supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

Bits [19:12]

Reserved, **RES0**.

MVE, bits [11:8]

Indicates support for M-profile vector extension.

The possible values of this field are:

0b0000

Not supported.

0b0001

Supported, no Floating-point.

0b0010

Supported, with single-precision and half-precision Floating-point.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

FPDNaN, bits [7:4]

Floating-point default NaN. Indicates whether the Floating-point Extension implementation supports NaN propagation.

The possible values of this field are:

0b0000

Not supported.

0b0001

Propagation of NaN values supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

FPFtZ, bits [3:0]

Floating-point flush-to-zero. Indicates whether subnormals are always flushed-to-zero.

The possible values of this field are:

0b0000

Not supported.

0b0001

Full denormalized numbers arithmetic supported.

All other values are reserved.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.180 MVFR2, Media and VFP Feature Register 2

The MVFR2 characteristics are:

Purpose

Describes the features provided by the Floating-point Extension.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read-only register located at 0xE000EF48.

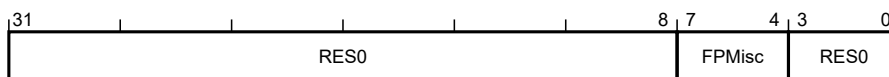
Secure software can access the Non-secure version of this register via MVFR2_NS located at 0xE002EF48.

The location 0xE002EF48 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The MVFR2 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

FPMisc, bits [7:4]

Floating-point miscellaneous. Indicates support for miscellaneous Floating-point features.

The possible values of this field are:

0b0000

Floating-point extensions not implemented.

0b0100

Selection, directed conversion to integer, VMINNM and VMAXNM supported.

All other values are reserved.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

Bits [3:0]

Reserved, [RES0](#).

D1.2.181 NSACR, Non-secure Access Control Register

The NSACR characteristics are:

Purpose

Defines the Non-secure access permissions for the Floating-point Extension and coprocessors CP0 to CP7. If MVE is implemented this register Specifies the Non-secure access permissions for MVE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read/write register located at 0xE000ED8C.

If the Security Extension is not implemented this register returns a value of 0x00000CFF.

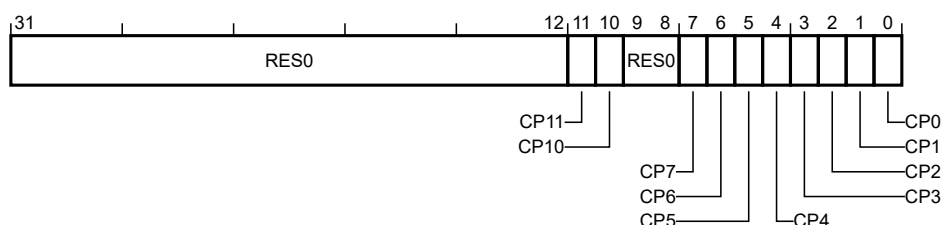
This register is RAZ/WI when accessed as Non-secure.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The NSACR bit assignments are:



Bits [31:12]

Reserved, **RES0**.

CP11, bit [11]

CP11 access. Enables Non-secure access to the Floating-point Extension and MVE.

Programming with a different value than that used for CP10 is **UNPREDICTABLE**.

If neither the Floating-point Extension or MVE are implemented, this bit is RAZ/WI.

This bit resets to an **Architecturally UNKNOWN** value on a Warm reset.

CP10, bit [10]

CP10 access. Enables Non-secure access to the Floating-point Extension and MVE.

The possible values of this bit are:

0

Non-secure accesses to the Floating-point Extension or MVE, unless otherwise specified, generate a NOCP UsageFault.

1

Non-secure access to the Floating-point Extension or MVE permitted.

If neither the Floating-point Extension or MVE are implemented, this bit is RAZ/WI.

This bit resets to an [Architecturally UNKNOWN](#) value on a Warm reset.

Bits [9:8]

Reserved, [RES0](#).

CPm, bit [m], for m = 0 to 7

CPm access. Enables Non-secure access to coprocessor CPm.

The possible values of this field are:

0

Non-secure accesses to this coprocessor generate a NOCP UsageFault.

1

Non-secure access to this coprocessor permitted.

A CPm bit is RAZ/WI if CPm is:

- Not implemented.
- Correspond to a coprocessor that is hardwired to operate in a single Security state.

This field resets to an [Architecturally UNKNOWN](#) value on a Warm reset.

D1.2.182 NVIC_IABRn, Interrupt Active Bit Register, n = 0 - 15

The NVIC IABR{0..15} characteristics are:

Purpose

For each group of 32 interrupts, shows the active state of each interrupt.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read-only register located at $0xE000E300 + 4n$.

Secure software can access the Non-secure version of this register via NVIC_IABRn_NS located at $0xE002E300 + 4n$. The location $0xE002E300 + 4n$ is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The NVIC_IABR{0..15} bit assignments are:

**ACTIVE, bits [31:0]**

Active state. For ACTIVE[m] in NVIC_IABR $_n$, indicates the active state for interrupt $32n+m$.

The possible values of each bit are:

0

Interrupt not active.

1

Interrupt is active.

Bits corresponding to unimplemented interrupts are [RES0](#). Bits corresponding to interrupts targeting Secure state are RAZ/WI from Non-secure.

This field resets to zero on a Warm reset.

D1.2.183 NVIC_ICERn, Interrupt Clear Enable Register, n = 0 - 15

The NVIC_ICER{0..15} characteristics are:

Purpose

Clears or reads the enabled state of each group of 32 interrupts.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

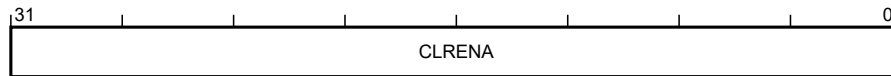
32-bit read/write-one-to-clear register located at $0xE000E180 + 4n$.

Secure software can access the Non-secure version of this register via NVIC_ICERn_NS located at $0xE002E180 + 4n$. The location $0xE002E180 + 4n$ is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The NVIC_ICER{0..15} bit assignments are:



CLRENA, bits [31:0], on a write

Clear enable. For CLRENA[m] in NVIC_ICERn, allows interrupt $32n + m$ to be disabled.

The possible values of each bit are:

0

No effect.

1

Disable interrupt $32n + m$.

Bits corresponding to unimplemented interrupts are **RES0**. Bits corresponding to interrupts targeting Secure state are RAZ/WI from Non-secure.

CLRENA, bits [31:0], on a read

Clear enable. For CLRENA[m] in NVIC_ICERn, indicates whether interrupt $32n + m$ is enabled.

The possible values of each bit are:

0

Interrupt $32n + m$ disabled.

1

Interrupt $32n + m$ enabled.

Bits corresponding to unimplemented interrupts are **RES0**. Bits corresponding to interrupts targeting Secure state are RAZ/WI from Non-secure.

This field resets to zero on a Warm reset.

D1.2.184 NVIC_ICPR_n, Interrupt Clear Pending Register, $n = 0 - 15$

The NVIC_ICPR{0..15} characteristics are:

Purpose

Clears or reads the pending state of each group of 32 interrupts.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

This register is always implemented.

Attributes

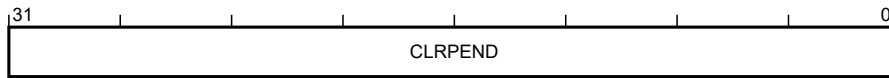
32-bit read/write-one-to-clear register located at $0xE000E280 + 4n$.

Secure software can access the Non-secure version of this register via NVIC_ICPR_n_NS located at $0xE002E280 + 4n$. The location $0xE002E280 + 4n$ is RES0 to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The NVIC_ICPR{0..15} bit assignments are:



CLRPEND, bits [31:0], on a write

Clear pending. For CLRPEND[m] in NVIC_ICPR_n, allows interrupt $32n + m$ to be unpended.

The possible values of each bit are:

0

No effect.

1

Clear pending state of interrupt $32n + m$.

Bits corresponding to unimplemented interrupts are RES0. Bits corresponding to interrupts targeting Secure state are RAZ/WI from Non-secure.

CLRPEND, bits [31:0], on a read

Clear pending. For CLRPEND[m] in NVIC_ICPR_n, indicates whether interrupt $32n + m$ is pending.

The possible values of each bit are:

0

Interrupt $32n + m$ is not pending.

1

Interrupt $32n + m$ is pending.

Bits corresponding to unimplemented interrupts are RES0. Bits corresponding to interrupts targeting Secure state are RAZ/WI from Non-secure.

D1.2.185 NVIC_IPRn, Interrupt Priority Register, n = 0 - 123

The NVIC_IPR{0..123} characteristics are:

Purpose

Sets or reads interrupt priorities.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If the Main Extension is implemented, this register is word, halfword, and byte accessible.

If the Main Extension is not implemented, this register is word accessible only, halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at $0xE000E400 + 4n$.

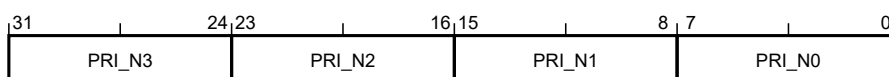
Secure software can access the Non-secure version of this register via NVIC_IPRn_NS located at $0xE002E400 + 4n$. The location $0xE002E400 + 4n$ is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The NVIC_IPR{0..123} bit assignments are:



PRI_Nm, bits [8m+7:8m], for m = 0 to 3

Priority 'N'+m. For register NVIC_IPRn, this field indicates and allows modification of the priority of interrupt number $4n+m$, or is **RES0** if the PE does not implement this interrupt.

If the PE implements fewer than 8 bits of priority, then the least significant bits of this field are RAZ/WI.

If interrupt number $4n+m$ targets Secure state, this field is RAZ/WI from Non-secure.

This field resets to zero on a Warm reset.

D1.2.186 NVIC_ISERn, Interrupt Set Enable Register, n = 0 - 15

The NVIC_ISER{0..15} characteristics are:

Purpose

Enables or reads the enabled state of each group of 32 interrupts.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

This register is always implemented.

Attributes

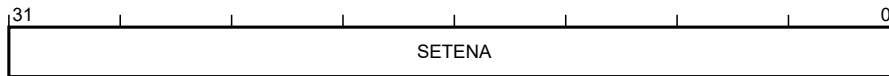
32-bit read/write-one-to-set register located at $0xE000E100 + 4n$.

Secure software can access the Non-secure version of this register via NVIC_ISERn_NS located at $0xE002E100 + 4n$. The location $0xE002E100 + 4n$ is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The NVIC_ISER{0..15} bit assignments are:



SETENA, bits [31:0], on a write

Set enable. For SETENA[m] in NVIC_ISERn, allows interrupt $32n + m$ to be set enabled.

The possible values of each bit are:

0

No effect.

1

Enable interrupt $32n + m$.

Bits corresponding to unimplemented interrupts are [RES0](#). Bits corresponding to interrupts targeting Secure state are RAZ/WI from Non-secure.

This field resets to zero on a Warm reset.

SETENA, bits [31:0], on a read

Set enable. For SETENA[m] in NVIC_ISERn, indicates whether interrupt $32n + m$ is enabled.

The possible values of each bit are:

0

Interrupt $32n + m$ disabled.

1

Interrupt $32n + m$ enabled.

Bits corresponding to unimplemented interrupts are [RES0](#). Bits corresponding to interrupts targeting Secure state are RAZ/WI from Non-secure.

This field resets to zero on a Warm reset.

D1.2.187 NVIC_ISPRn, Interrupt Set Pending Register, n = 0 - 15

The NVIC_ISPR{0..15} characteristics are:

Purpose

Enables or reads the pending state of each group of 32 interrupts.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

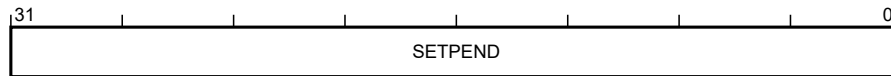
32-bit read/write-one-to-set register located at $0xE000E200 + 4n$.

Secure software can access the Non-secure version of this register via NVIC_ISPRn_NS located at $0xE002E200 + 4n$. The location $0xE002E200 + 4n$ is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The NVIC_ISPR{0..15} bit assignments are:



SETPEND, bits [31:0], on a write

Set pending. For SETPEND[m] in NVIC_ISPRn, allows interrupt $32n + m$ to be set pending.

The possible values of each bit are:

0

No effect.

1

Pend interrupt $32n + m$.

Bits corresponding to unimplemented interrupts are **RES0**. Bits corresponding to interrupts targeting Secure state are RAZ/WI from Non-secure.

This field is write-one-to-set. Writes of zero are ignored.

This field resets to zero on a Warm reset.

SETPEND, bits [31:0], on a read

Set pending. For SETPEND[m] in NVIC_ISPRn, indicates whether interrupt $32n + m$ is pending.

The possible values of each bit are:

0

Interrupt $32n + m$ is not pending.

1

Interrupt $32n + m$ pending.

Bits corresponding to unimplemented interrupts are [RES0](#). Bits corresponding to interrupts targeting Secure state are RAZ/WI from Non-secure.

This field resets to zero on a Warm reset.

D1.2.188 NVIC_ITNSn, Interrupt Target Non-secure Register, n = 0 - 15

The NVIC_ITNS{0..15} characteristics are:

Purpose

For each group of 32 interrupts, determines whether each interrupt targets Non-secure or Secure state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at $0xE000E380 + 4n$.

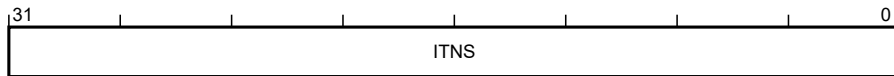
This register is RAZ/WI when accessed as Non-secure.

This register is not banked between Security states.

From Armv8.1-M onwards it is IMPLEMENTATION DEFINED whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The NVIC_ITNS{0..15} bit assignments are:



ITNS, bits [31:0]

Interrupt Targets Non-secure. For ITNS[m] in NVIC_ITNSn, this field indicates and allows modification of the target Security state for interrupt $32n+m$.

The possible values of each bit are:

0

Interrupt targets Secure state.

1

Interrupt targets Non-secure state.

Bits corresponding to unimplemented interrupts are RES0. It is IMPLEMENTATION DEFINED whether individual bits are WI and have an IMPLEMENTATION DEFINED constant value. Where an interrupt is configured to target Secure state, accesses to the associated fields in Non-secure versions of the NVIC_IABR, NVIC_ICER, NVIC_ISER, NVIC_ICPR, NVIC_IPR and NVIC_ISPR are RAZ/WI.

This field resets to zero on a Warm reset.

D1.2.189 PAC_KEY_P_n, Pointer Authentication Key Privileged, n = 0 - 3

The PAC_KEY_P_{0..3} characteristics are:

Purpose

Holds word n of the key used for PAC calculation in Privileged software.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

Present only if the PACBTI Extension is implemented.

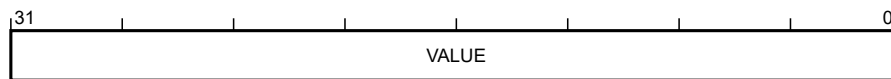
Attributes

32-bit read/write special-purpose register.

This register is banked between Security states.

Field descriptions

The PAC_KEY_P_{0..3} bit assignments are:

**VALUE, bits [31:0]**

Pointer authentication key, word n . Holds bits[31 + 32 n :32 n] of the 128-bit key.

This field resets to an Architecturally UNKNOWN value on a Warm reset.

D1.2.190 PAC_KEY_U_n, Pointer Authentication Key Unprivileged, n = 0 - 3

The PAC_KEY_U_{0..3} characteristics are:

Purpose

Holds word n of the key used for PAC calculation in Unprivileged software.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

Present only if the PACBTI Extension is implemented.

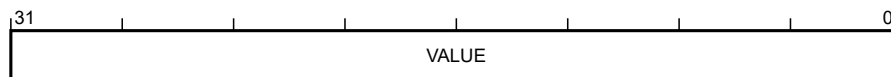
Attributes

32-bit read/write special-purpose register.

This register is banked between Security states.

Field descriptions

The PAC_KEY_U_{0..3} bit assignments are:

**VALUE, bits [31:0]**

Pointer authentication key, word n . Holds bits[31 + 32 n :32 n] of the 128-bit key.

This field resets to an **Architecturally UNKNOWN** value on a Warm reset.

D1.2.191 PC, Program Counter

The PC characteristics are:

Purpose

Holds the current Program Counter value.

Usage constraints

Privileged and unprivileged access permitted.

Configurations

This register is always implemented.

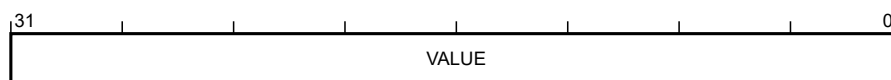
Attributes

32-bit read/write special-purpose register.

This register is not banked between Security states.

Field descriptions

The PC bit assignments are:



VALUE, bits [31:0]

Program Counter. Holds the address of the current instruction.

Software can refer to PC as R15.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.192 PMU_AUTHSTATUS, Performance Monitoring Unit Authentication Status Register

The PMU_AUTHSTATUS characteristics are:

Purpose

Provides information about the state of the **IMPLEMENTATION DEFINED** authentication interface for Performance Monitoring Units.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

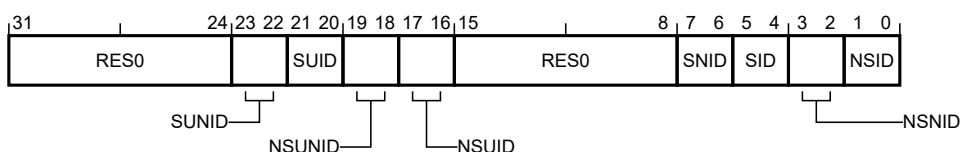
Attributes

32-bit read-only register located at 0xE0003FB8.

This register is not banked between Security states.

Field descriptions

The PMU_AUTHSTATUS bit assignments are:



Bits [31:24]

Reserved, **RES0**.

SUNID, bits [23:22]

Secure Unprivileged Non-invasive Debug Allowed. Indicates that Unprivileged Non-invasive debug is allowed for the Secure state.

The possible values of this field are:

0b00

Security Extension or Unprivileged Non-invasive Debug not implemented.

0b01

Reserved.

0b10

Secure Non-invasive debug prohibited.

0b11

Secure Non-invasive debug allowed in unprivileged mode.

If version Armv8.1 of the architecture and UDE are not implemented, this field is **RES0**.

SUID, bits [21:20]

Secure Unprivileged Invasive Debug Allowed. Indicates that Unprivileged Halting Debug is allowed for the Secure state.

The possible values of this field are:

0b00

Security Extension or Unprivileged Debug not implemented.

0b01

Reserved.

0b10

Secure halting debug prohibited.

0b11

Secure halting debug allowed in unprivileged mode.

This reflects the value of `UnprivHaltingDebugAllowed(TRUE) && !SecureHaltingDebugAllowed()`.

If version Armv8.1 of the architecture and UDE are not implemented, this field is [RES0](#).

NSUNID, bits [19:18]

Non-secure Unprivileged Non-invasive Debug Allowed. Indicates that Unprivileged Non-invasive Debug is allowed for the Non-secure state.

The possible values of this field are:

0b00

Unprivileged Non-invasive debug not implemented.

0b01

Reserved.

0b10

Non-secure Non-invasive debug prohibited.

0b11

Non-secure Non-invasive debug allowed in unprivileged mode.

If the Main Extension is not implemented, this field is [RES0](#).

NSUID, bits [17:16]

Non-secure Unprivileged Invasive Debug Allowed. Indicates that Unprivileged Halting Debug is allowed for the Non-secure state.

The possible values of this field are:

0b00

Unprivileged halting debug not implemented.

0b01

Reserved.

0b10

Non-secure halting debug prohibited.

0b11

Non-secure halting debug allowed in unprivileged mode.

This reflects the value of `UnprivHaltingDebugAllowed(FALSE) && !HaltingDebugAllowed()`.

If version Armv8.1 of the architecture and UDE are not implemented, this field is [RES0](#).

Bits [15:8]

Reserved, [RES0](#).

SNID, bits [7:6]

Secure Non-invasive Debug. Indicates whether Secure non-invasive debug is implemented and allowed.

The possible values of this field are:

0b00

Security Extension not implemented.

0b01

Reserved.

0b10

Security Extension implemented and Secure non-invasive debug prohibited.

0b11

Security Extension implemented and Secure non-invasive debug allowed.

SID, bits [5:4]

Secure Invasive Debug. Indicates whether Secure invasive debug is implemented and allowed.

The possible values of this field are:

0b00

Security Extension not implemented.

0b01

Reserved.

0b10

Security Extension implemented and Secure invasive debug prohibited.

0b11

Security Extension implemented and Secure invasive debug allowed.

NSNID, bits [3:2]

Non-secure Non-invasive Debug. Indicates whether Non-secure non-invasive debug is allowed.

The possible values of this field are:

0b0x

Reserved.

0b10

Non-secure non-invasive debug prohibited.

0b11

Non-secure non-invasive debug allowed.

NSID, bits [1:0]

Non-secure Invasive Debug. Indicates whether Non-secure invasive debug is allowed.

The possible values of this field are:

0b0x

Reserved.

0b10

Non-secure invasive debug prohibited.

0b11

Non-secure invasive debug allowed.

D1.2.193 PMU_CCFILTR, Performance Monitoring Unit Cycle Counter Filter Register

The PMU_CCFILTR characteristics are:

Purpose

This register is reserved for future use.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is [RES0](#) if the PMU is not implemented.

Attributes

32-bit read/write register located at 0xE000347C.

This register is not banked between Security states.

Field descriptions

The PMU_CCFILTR bit assignments are:



Bits [31:0]

Reserved, [RES0](#).

D1.2.194 PMU_CCNTR, Performance Monitoring Unit Cycle Counter Register

The PMU_CCNTR characteristics are:

Purpose

Holds the value of the cycle counter, which counts processor clock cycles.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

Attributes

32-bit read/write register located at 0xE000307C.

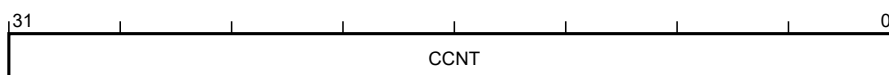
This register is not banked between Security states.

Preface

This register is an alias of the DWT_CYCCNT register.

Field descriptions

The PMU_CCNTR bit assignments are:



CCNT, bits [31:0]

Cycle count. The cycle count increments on every processor clock cycle.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.195 PMU_CIDR0, Performance Monitoring Unit Component Identification Register 0

The PMU_CIDR0 characteristics are:

Purpose

Provides information to identify a Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

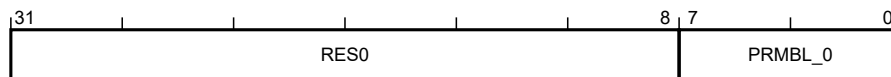
Attributes

32-bit read-only register located at 0xE0003FF0.

This register is not banked between Security states.

Field descriptions

The PMU_CIDR0 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

PRMBL_0, bits [7:0]

Preamble.

This field reads as 0x0D.

D1.2.196 PMU_CIDR1, Performance Monitoring Unit Component Identification Register 1

The PMU_CIDR1 characteristics are:

Purpose

Provides information to identify a Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is [RES0](#) if the PMU is not implemented.

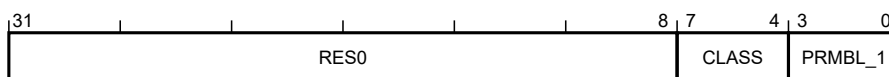
Attributes

32-bit read-only register located at 0xE0003FF4.

This register is not banked between Security states.

Field descriptions

The PMU_CIDR1 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

CLASS, bits [7:4]

Component class.

This field reads as 0x9.

PRMBL_1, bits [3:0]

Preamble.

This field reads as 0x0.

D1.2.197 PMU_CIDR2, Performance Monitoring Unit Component Identification Register 2

The PMU_CIDR2 characteristics are:

Purpose

Provides information to identify a Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is [RES0](#) if the PMU is not implemented.

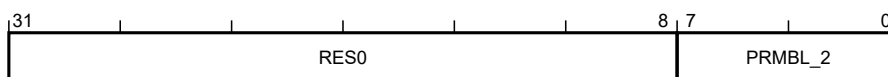
Attributes

32-bit read-only register located at 0xE0003FF8.

This register is not banked between Security states.

Field descriptions

The PMU_CIDR2 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

PRMBL_2, bits [7:0]

Preamble.

This field reads as 0x05.

D1.2.198 PMU_CIDR3, Performance Monitoring Unit Component Identification Register 3

The PMU_CIDR3 characteristics are:

Purpose

Provides information to identify a Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is [RES0](#) if the PMU is not implemented.

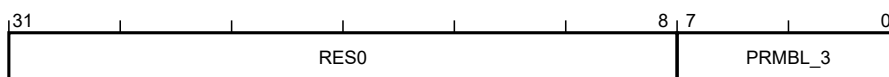
Attributes

32-bit read-only register located at 0xE0003FFC.

This register is not banked between Security states.

Field descriptions

The PMU_CIDR3 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

PRMBL_3, bits [7:0]

Preamble.

This field reads as 0xB1.

D1.2.199 PMU_CNTENCLR, Performance Monitoring Unit Count Enable Clear Register

The PMU_CNTENCLR characteristics are:

Purpose

Disables the Cycle Count Register, PMU_CCNTR, and any implemented event counters PMU_EVCNTR<n>. Reading this register shows which counters are enabled.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

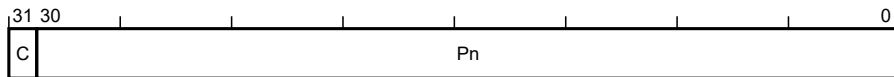
Attributes

32-bit read/write register located at 0xE0003C20.

This register is not banked between Security states.

Field descriptions

The PMU_CNTENCLR bit assignments are:



C, bit [31]

PMU_CCNTR disable bit. Disables the cycle counter register.

The possible values of this bit are:

0

When read, means the cycle counter is disabled. When written, has no effect.

1

When read, means the cycle counter is enabled. When written, disables the cycle counter.

This bit resets to zero on a Cold reset.

Pn, bits [30:0]

Event counter PMU_EVCNTR<n> disable bit. Disables PMU_EVCNTR<n>.

The possible values of this field are:

0

When read, means that PMU_EVCNTR<n> is disabled. When written, has no effect.

1

When read, means that PMU_EVCNTR<n> event counter is enabled. When written, disables PMU_EVCNTR<n>.

This field resets to zero on a Cold reset.

Note

Bits [30:N] are RAZ/WI, where N is the number of counters and the value of PMU_TYPE.N.

D1.2.200 PMU_CNTENSET, Performance Monitoring Unit Count Enable Set Register

The PMU_CNTENSET characteristics are:

Purpose

Enables the Cycle Count Register, PMU_CCNTR, and any implemented event counters PMU_EVCNTR<n>. Reading this register shows which counters are enabled.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is RES0 if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is RES0 if the PMU is not implemented.

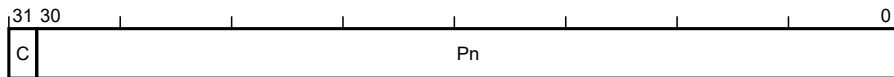
Attributes

32-bit read/write register located at 0xE0003C00.

This register is not banked between Security states.

Field descriptions

The PMU_CNTENSET bit assignments are:



C, bit [31]

PMU_CCNTR enable bit. Enables the cycle counter register.

The possible values of this bit are:

0

When read, means the cycle counter is disabled. When written, has no effect.

1

When read, means the cycle counter is enabled. When written, enables the cycle counter.

This bit resets to zero on a Cold reset.

Pn, bits [30:0]

Event counter PMU_EVCNTR<n> enable bit. Enables PMU_EVCNTR<n>.

The possible values of this field are:

0

When read, means that PMU_EVCNTR<n> is disabled. When written, has no effect.

1

When read, means that PMU_EVCNTR<n> event counter is enabled. When written, enables PMU_EVCNTR<n>.

This field resets to zero on a Cold reset.

Note

Bits [30:N] are RAZ/WI, where N is the number of counters and the value of PMU_TYPE.N.

D1.2.201 PMU_CTRL, Performance Monitoring Unit Control Register

The PMU_CTRL characteristics are:

Purpose

Configures and controls the Performance Monitoring Unit.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is [RES0](#) if the PMU is not implemented.

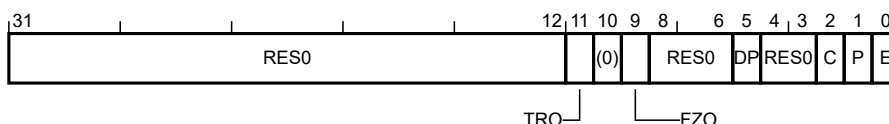
Attributes

32-bit read/write register located at 0xE0003E04.

This register is not banked between Security states.

Field descriptions

The PMU_CTRL bit assignments are:



Bits [31:12]

Reserved, [RES0](#).

TRO, bit [11]

Trace-on-overflow. Enable trace-on-overflow.

The possible values of this bit are:

0

Trace-on-overflow disabled.

1

Trace whenever any of the first eight counters overflows an 8-bit value.

This bit resets to an [UNKNOWN](#) value on a Warm reset.

Bit [10]

Reserved, [RES0](#).

FZO, bit [9]

Freeze-on-overflow. Stops events being counted once PMU_OVSCLR or PMU_OVSSET is nonzero.

The possible values of this bit are:

0

This bit has no effect on event counting.

1

While any bit of PMU_OVSCLR.Pn or PMU_OVSSET.Pn is nonzero, event counters do not count events.

This bit resets to an UNKNOWN value on a Warm reset.

Bits [8:6]

Reserved, RES0.

DP, bit [5]

Controls whether the cycle counter is disabled in Secure state. This bit is an alias of the DWT_CTRL.CYCDISS bit.

Bits [4:3]

Reserved, RES0.

C, bit [2]

Cycle counter reset. Reset the PMU_CCNTR counter.

The possible values of this bit are:

0

No action.

1

Reset PMU_CCNTR to zero.

Resetting PMU_CCNTR does not clear the PMU_CCNTR overflow bit to 0.

This bit is write-only.

This bit reads as zero.

P, bit [1]

Event counter reset. Reset event counters.

The possible values of this bit are:

0

No action.

1

Reset all event counters, not including PMU_CCNTR, to zero.

Resetting the event counters does not clear any overflow bits to 0.

This bit is write-only.

This bit reads as zero.

E, bit [0]

Enable. Enable the event counters.

The possible values of this bit are:

0

All counters, including PMU_CCNTR, are disabled.

1

All counters are enabled by PMU_CNTENSET.

This bit resets to zero on a Warm reset.

D1.2.202 PMU_DEVARCH, Performance Monitoring Unit Device Architecture Register

The PMU_DEVARCH characteristics are:

Purpose

Identifies the programmers' model architecture of the Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

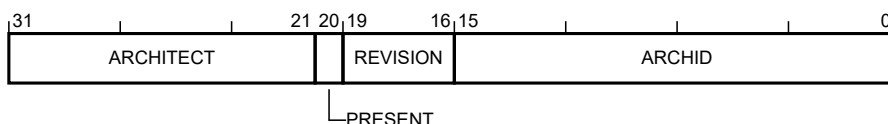
Attributes

32-bit read-only register located at 0xE0003FBC.

This register is not banked between Security states.

Field descriptions

The PMU_DEVARCH bit assignments are:



ARCHITECT, bits [31:21]

Defines the architecture of the component.

For Performance Monitoring Units, this is Arm Limited.

Bits [31:28] are the JEP 106 continuation code, 0x4.

Bits [27:21] are the JEP 106 ID code, 0x3B.

PRESENT, bit [20]

Determines the presence of DEVARCH. When set to 1, indicates that the DEVARCH is present.

This bit reads as 0x1.

REVISION, bits [19:16]

Defines the architecture revision.

For architectures defined by Arm this is the minor revision.

For Performance Monitoring Units, the revision defined by Armv8.1-M is 0x0.

All other values are reserved.

ARCHID, bits [15:0]

Defines this part to be an Armv8-M debug component.

For architectures defined by Arm this is further subdivided. For Performance Monitoring Units:

Bits [15:12] are the architecture version, 0x0.

Bits [11:0] are the architecture part number, 0xA06.

D1.2.203 PMU_DEVTYPE, Performance Monitoring Unit Device Type Register

The PMU_DEVTYPE characteristics are:

Purpose

Indicates to a debugger that this component is part of the Performance Monitoring Unit interface of the PE.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is [RES0](#) if the PMU is not implemented.

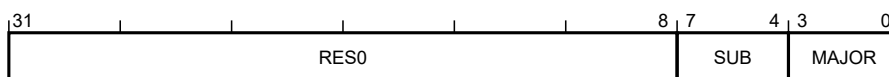
Attributes

32-bit read-only register located at 0xE0003FCC.

This register is not banked between Security states.

Field descriptions

The PMU_DEVTYPE bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

SUB, bits [7:4]

Subtype.

This field reads as 0x1.

MAJOR, bits [3:0]

Major type.

This field reads as 0x6.

D1.2.204 PMU_EVCNTRn, Performance Monitoring Unit Event Counter Register

The PMU_EVCNTR{0..30} characteristics are:

Purpose

Holds performance counter n, which counts events.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

Attributes

32-bit read/write register located at $0xE003000 + 4n$.

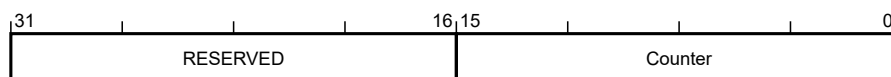
This register is not banked between Security states.

Preface

If n is greater than or equal to the number of accessible counters, reads and writes of this register are RES0.

Field descriptions

The PMU_EVCNTR{0..30} bit assignments are:

**RESERVED, bits [31:16]**

Reserved, RAZ/WI

Counter, bits [15:0]

Event counter n.

Value of event counter n, where n is the number of this register. n is a number in the range 0-30. The size of this counter is 16 bits.

The counter counts whenever the selected event occurs, and either of:

- SecureNoninvasiveDebugEnabled() == TRUE.
- The NS-Req for the operation is set to Non-secure and NoninvasiveDebugEnabled() == TRUE.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.205 PMU_EVTYPEn, Performance Monitoring Unit Event Type and Filter Register

The PMU_EVTYPER{0..30} characteristics are:

Purpose

Configures event counter n, where n is 0 to 30.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

Attributes

32-bit read/write register located at $0xE003400 + 4n$.

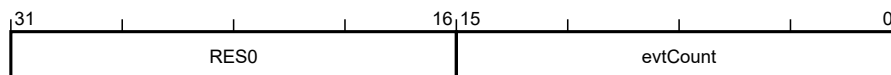
This register is not banked between Security states.

Preface

If n is greater than or equal to the number of accessible counters, reads and writes of this register are [RES0](#).

Field descriptions

The PMU_EVTYPER{0..30} bit assignments are:

**Bits [31:16]**

Reserved, RES0.

evtCount, bits [15:0]

Event to Count. The event number of the event that is counted by event counter PMU_EVCNTR<n>. If the associated counter does not support the event number that is written to this register, the value read back is UNKNOWN.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.206 PMU_INTENCLR, Performance Monitoring Unit Interrupt Enable Clear Register

The PMU_INTENCLR characteristics are:

Purpose

Disables the generation of interrupt requests on overflows from the Cycle Count Register, PMU_CCNTR, and the event counters, PMU_EVCNTR. Reading the register shows which overflow interrupt requests are enabled.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is RES0 if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is RES0 if the PMU is not implemented.

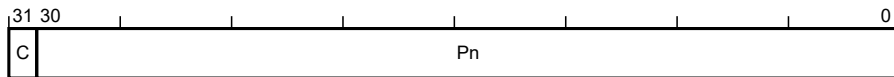
Attributes

32-bit read/write register located at 0xE0003C60.

This register is not banked between Security states.

Field descriptions

The PMU_INTENCLR bit assignments are:



C, bit [31]

PMU_CCNTR overflow interrupt request disable bit. Disable the overflow interrupt for the cycle counter.

The possible values of this bit are:

0

When read, means the cycle counter overflow interrupt request is disabled. When written, has no effect.

1

When read, means the cycle counter overflow interrupt request is enabled. When written, disables the cycle count overflow interrupt request.

This bit resets to zero on a Cold reset.

Pn, bits [30:0]

Event counter overflow interrupt request disable bit for PMU_EVCNTR<n>. Disable the overflow interrupt for PMU_EVCNTR<n>.

The possible values of this field are:

0

When read, means that the PMU_EVCNTR<n> event counter interrupt request is disabled. When written, has no effect.

1

When read, means that the PMU_EVCNTR<n> event counter interrupt request is enabled. When written, disables the PMU_EVCNTR<n> interrupt request.

This field resets to zero on a Cold reset.

Note

Bits [30:N] are RAZ/WI, where N is the number of counters and the value of PMU_TYPE.N.

D1.2.207 PMU_INTENSET, Performance Monitoring Unit Interrupt Enable Set Register

The PMU_INTENSET characteristics are:

Purpose

Enables the generation of interrupt requests on overflows from the Cycle Count Register, PMU_CCNTR, and the event counter, PMU_EVCNTR. Reading the register shows which overflow interrupt requests are enabled.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is [RES0](#) if the PMU is not implemented.

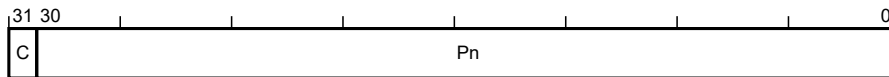
Attributes

32-bit read/write register located at 0xE0003C40.

This register is not banked between Security states.

Field descriptions

The PMU_INTENSET bit assignments are:



C, bit [31]

PMU_CCNTR overflow interrupt request enable bit. Enable the overflow interrupt for the cycle counter.

The possible values of this bit are:

0

When read, means the cycle counter overflow interrupt request is disabled. When written, has no effect.

1

When read, means the cycle counter overflow interrupt request is enabled. When written, enables the cycle count overflow interrupt request.

This bit resets to zero on a Cold reset.

Pn, bits [30:0]

Event counter overflow interrupt request enable bit for PMU_EVCNTR<n>. Enable the overflow interrupt for PMU_EVCNTR<n>.

The possible values of this field are:

0

When read, means that the PMU_EVCNTR<n> event counter interrupt request is disabled. When written, has no effect.

1

When read, means that the PMU_EVCNTR<n> event counter interrupt request is enabled. When written, enables the PMU_EVCNTR<n> interrupt request.

This field resets to zero on a Cold reset.

Note

Bits [30:N] are RAZ/WI, where N is the number of counters and the value of PMU_TYPE.N.

D1.2.208 PMU_OVSCLR, Performance Monitoring Unit Overflow Flag Status Clear Register

The PMU_OVSCLR characteristics are:

Purpose

Contains the state of the overflow bit for the Cycle Count Register, PMU_CCNTR, and each of the implemented event counters, PMU_EVCNTR<n>. Writing to this register clears these bits.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is RES0 if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is RES0 if the PMU is not implemented.

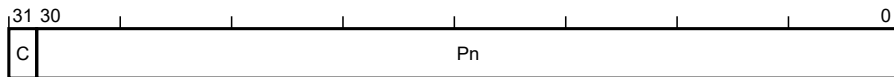
Attributes

32-bit read/write register located at 0xE0003C80.

This register is not banked between Security states.

Field descriptions

The PMU_OVSCLR bit assignments are:



C, bit [31]

PMU_CCNTR overflow bit. Clears the PMU_CCNTR overflow bit.

The possible values of this bit are:

0

When read, means the cycle counter has not overflowed. When written, has no effect.

1

When read, means the cycle counter has overflowed. When written, clears the overflow bit to 0.

This bit resets to zero on a Cold reset.

Pn, bits [30:0]

Event counter overflow clear bit for PMU_EVCNTR<n>. Clears the PMU_EVCNTR<n> overflow bit.

The possible values of this field are:

0

When read, means that the PMU_EVCNTR<n> event counter has not overflowed. When written, has no effect.

1

When read, means that the PMU_EVCNTR<n> event counter has overflowed. When written, clears the PMU_EVCNTR<n> overflow bit to 0.

This field resets to zero on a Cold reset.

Note

Bits [30:N] are RAZ/WI, where N is the number of counters and the value of PMU_TYPE.N.

D1.2.209 PMU_OVSSET, Performance Monitoring Unit Overflow Flag Status Set Register

The PMU_OVSSET characteristics are:

Purpose

Sets the state of the overflow bit for the Cycle Count Register, PMU_CCNTR, and each of the implemented event counters, PMU_EVCNTR<n>.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is [RES0](#) if the PMU is not implemented.

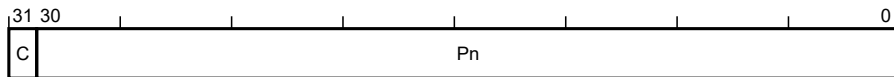
Attributes

32-bit read/write register located at 0xE0003CC0.

This register is not banked between Security states.

Field descriptions

The PMU_OVSSET bit assignments are:



C, bit [31]

PMU_CCNTR overflow bit. Set the overflow status for PMU_CCNTR.

The possible values of this bit are:

0

When read, means the cycle counter has not overflowed. When written, has no effect.

1

When read, means the cycle counter has overflowed. When written, sets the overflow bit to 1.

This bit resets to zero on a Cold reset.

Pn, bits [30:0]

Event counter overflow set bit for PMU_EVCNTR<n>. Set the overflow status for PMU_EVCNTR<n>.

The possible values of this field are:

0

When read, means that the PMU_EVCNTR<n> event counter has not overflowed. When written, has no effect.

1

When read, means that the PMU_EVCNTR<n> event counter has overflowed. When written, sets the PMU_EVCNTR<n> overflow bit to 1.

This field resets to zero on a Cold reset.

Note

Bits [30:N] are RAZ/WI, where N is the number of counters and the value of PMU_TYPE.N.

D1.2.210 PMU_PIDR0, Performance Monitoring Unit Peripheral Identification Register 0

The PMU_PIDR0 characteristics are:

Purpose

Provides information to identify a Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is [RES0](#) if the PMU is not implemented.

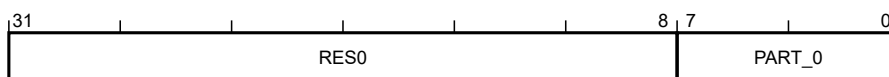
Attributes

32-bit read-only register located at 0xE0003FE0.

This register is not banked between Security states.

Field descriptions

The PMU_PIDR0 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

PART_0, bits [7:0]

Part number, least significant byte.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

D1.2.211 PMU_PIDR1, Performance Monitoring Unit Peripheral Identification Register 1

The PMU_PIDR1 characteristics are:

Purpose

Provides information to identify a Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

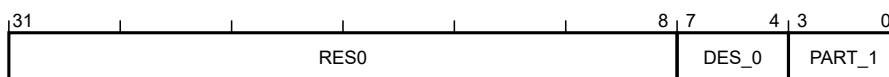
Attributes

32-bit read-only register located at 0xE0003FE4.

This register is not banked between Security states.

Field descriptions

The PMU_PIDR1 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

DES_0, bits [7:4]

Designer, least significant nibble of JEP106 ID code. For Arm Limited, this field is 0b1011.

This field reads as an **IMPLEMENTATION DEFINED** value.

PART_1, bits [3:0]

Part number, most significant nibble.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.212 PMU_PIDR2, Performance Monitoring Unit Peripheral Identification Register 2

The PMU_PIDR2 characteristics are:

Purpose

Provides information to identify a Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

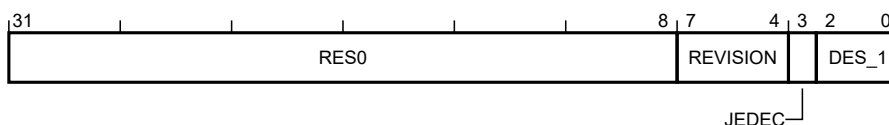
Attributes

32-bit read-only register located at 0xE0003FE8.

This register is not banked between Security states.

Field descriptions

The PMU_PIDR2 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REVISION, bits [7:4]

Part major revision. Parts can also use this field to extend Part number to 16-bits.

This field reads as an **IMPLEMENTATION DEFINED** value.

JEDEC, bit [3]

JEDEC. RAO. Indicates a JEP106 identity code is used.

DES_1, bits [2:0]

Designer, most significant bits of JEP106 ID code. For Arm Limited, this field is 0b011.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.213 PMU_PIDR3, Performance Monitoring Unit Peripheral Identification Register 3

The PMU_PIDR3 characteristics are:

Purpose

Provides information to identify a Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

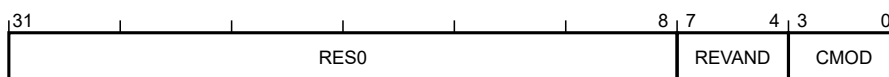
Attributes

32-bit read-only register located at 0xE0003FEC.

This register is not banked between Security states.

Field descriptions

The PMU_PIDR3 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REVAND, bits [7:4]

Part minor revision. Parts using PMU_PIDR2.REVISION as an extension to the Part number must use this field as a major revision number.

This field reads as an **IMPLEMENTATION DEFINED** value.

CMOD, bits [3:0]

Customer modified. Indicates someone other than the Designer has modified the component.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.214 PMU_PIDR4, Performance Monitoring Unit Peripheral Identification Register 4

The PMU_PIDR4 characteristics are:

Purpose

Provides information to identify a Performance Monitoring Unit component.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

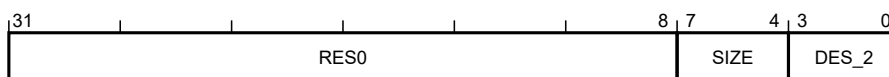
Attributes

32-bit read-only register located at 0xE0003FD0.

This register is not banked between Security states.

Field descriptions

The PMU_PIDR4 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

SIZE, bits [7:4]

Size of the component. RAZ. Log₂ of the number of 4KB pages from the start of the component to the end of the component ID registers.

DES_2, bits [3:0]

Designer, JEP106 continuation code, least significant nibble. For Arm Limited, this field is 0b0100.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.215 PMU_SWINC, Performance Monitoring Unit Software Increment Register

The PMU_SWINC characteristics are:

Purpose

Increments a counter that is configured to count the Software increment event, event 0x00.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is RES0 if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is RES0 if the PMU is not implemented.

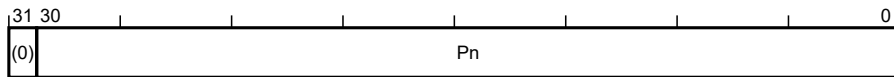
Attributes

32-bit write-only register located at 0xE0003CA0.

This register is not banked between Security states.

Field descriptions

The PMU_SWINC bit assignments are:



Bit [31]

Reserved, RES0.

Pn, bits [30:0]

Event counter software increment bit for PMU_EVCNTR<n>. An event counter n, configured for SW_INCR events, increments on every write to bit n of this field.

The possible values of this field are:

0

No action. The write to this bit is ignored.

1

A SW_INCR event is generated for event counter n.

Note

Bits [30:N] are WI, where N is the number of counters and the value of PMU_TYPE.N.

This field reads as zero.

D1.2.216 PMU_TYPE, Performance Monitoring Unit Type Register

The PMU_TYPE characteristics are:

Purpose

Contains information regarding what the Performance Monitoring Unit supports.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Present only if version Armv8.1 of the architecture and the PMU are implemented.

This register is **RES0** if the PMU is not implemented.

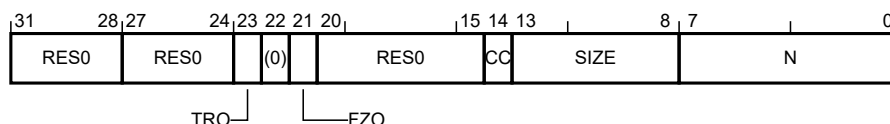
Attributes

32-bit read-only register located at 0xE0003E00.

This register is not banked between Security states.

Field descriptions

The PMU_TYPE bit assignments are:



Bits [31:28]

Reserved, **RES0**.

Bits [27:24]

Reserved, **RES0**.

TRO, bit [23]

Trace-on-overflow support. Identifies whether the trace-on-overflow function is supported.

The possible values of this bit are:

0

Trace-on-overflow not supported.

1

Trace-on-overflow supported.

This bit reads as one.

Bit [22]

Reserved, **RES0**.

FZO, bit [21]

Freeze-on-overflow support. Identifies whether the freeze-on-overflow mechanism is supported.

The possible values of this bit are:

0
Freeze-on-overflow mechanism not supported.

1
Freeze-on-overflow mechanism supported.

This bit reads as one.

Bits [20:15]
Reserved, [RES0](#).

CC, bit [14]
Cycle counter present. This bit is set if a dedicated cycle counter is present.
This bit reads as one.

SIZE, bits [13:8]
Size of counters. This field determines the spacing of counters in the memory-map.

Note

In Armv8-M this indicates all counters are word-aligned, as the largest counter is PMU_CCNTR with 32-bits.

This field reads as 0b011111.

N, bits [7:0]
Number of counters.
Number of counters implemented in addition to the cycle counter, PMU_CCNTR.
00000010 PMU_CCNTR and 2 event counters implemented.
00000011 PMU_CCNTR and 3 event counters implemented.
and so on up to 00011111, which indicates PMU_CCNTR and 31 event counters implemented.

Note

This field will be nonzero when the PMU is implemented, and serves to indicate the PMU is supported. The maximum number of event counters is 31, so bits[7:5] are always zero.

This field reads as an [IMPLEMENTATION DEFINED](#) value.

D1.2.217 PRIMASK, Exception Mask Register

The PRIMASK characteristics are:

Purpose

Provides access to the PE PRIMASK register.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

This register is always implemented.

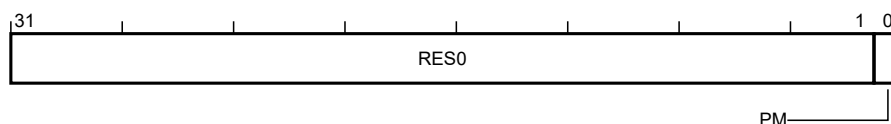
Attributes

32-bit read/write special-purpose register.

This register is banked between Security states.

Field descriptions

The PRIMASK bit assignments are:



Bits [31:1]

Reserved, [RES0](#).

PM, bit [0]

Exception mask register. Setting the Secure PRIMASK to one raises the execution priority to 0. Setting the Non-secure PRIMASK to one raises the execution priority to 0 if AIRCR.PRIS is clear, or 0x80 if AIRCR.PRIS is set.

The possible values of this bit are:

0

No effect on execution priority.

1

Boosts execution priority to either 0 or 0x80.

This bit resets to zero on a Warm reset.

D1.2.218 PSPLIM, Process Stack Pointer Limit Register

The PSPLIM characteristics are:

Purpose

Holds the lower limit for the Process stack pointer.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

This register is always implemented.

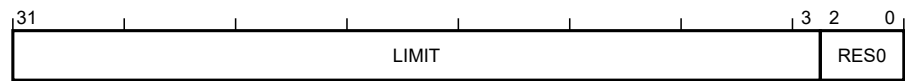
Attributes

32-bit read/write special-purpose register.

This register is banked between Security states.

Field descriptions

The PSPLIM bit assignments are:



LIMIT, bits [31:3]

Stack limit. Bits [31:3] of the Process stack limit address for the selected Security state.

Many instructions and exception entry will generate an exception if the appropriate stack pointer would be updated to a value lower than this limit. If the Main Extension is not implemented, the Non-secure PSPLIM is [RES0](#).

This field resets to zero on a Warm reset.

Bits [2:0]

Reserved, [RES0](#).

D1.2.219 Qn, Vector register, n = 0 - 7

The Q{0..7} characteristics are:

Purpose

Provides access to the Vector register file. These registers are an alias of the Floating-point register file.

Usage constraints

Privileged and unprivileged access permitted.

Configurations

Present only if version Armv8.1 of the architecture and MVE are implemented.

This register is [RES0](#) if MVE is not implemented.

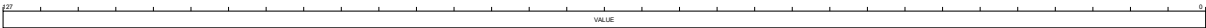
Attributes

128-bit read/write special-purpose register.

This register is not banked between Security states.

Field descriptions

The Q{0..7} bit assignments are:



VALUE, bits [127:0]

MVE Vector register value.

D1.2.220 Rn, General-Purpose Register, n = 0 - 12

The R{0..12} characteristics are:

Purpose

General-purpose register.

Usage constraints

Both privileged and unprivileged accesses are permitted.

This register is word, halfword, and byte accessible.

Configurations

This register is always implemented.

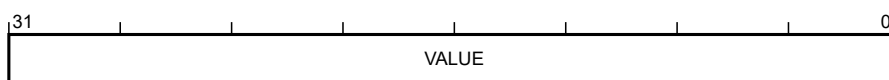
Attributes

32-bit read/write register.

This register is not banked between Security states.

Field descriptions

The R{0..12} bit assignments are:



VALUE, bits [31:0]

General purpose register value. Armv8-M implemented thirteen general-purpose 32-bit registers, R0 to R12.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.221 RETPSR, Combined Exception Return Program Status Registers

The RETPSR characteristics are:

Purpose

Value pushed to the stack on exception entry. On exception return this is used to restore the flags and other architectural state. This payload is also used for FNC_RETURN stacking, however in this case only some of the fields are used. See FunctionReturn() for details.

Usage constraints

None.

Configurations

All.

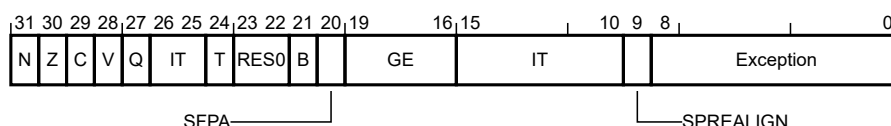
Attributes

32-bit payload.

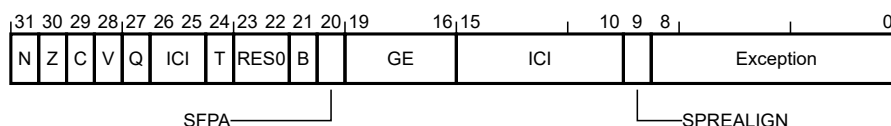
Field descriptions

The RETPSR bit assignments are:

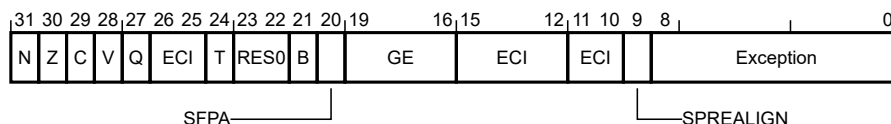
When {RETPSR[26:25], RETPSR[11:10]} != 0:



When {RETPSR[26:25], RETPSR[11:10]} == 0, and a multi-cycle load or store instruction was in progress when the exception was taken:



When {RETPSR[26:25], RETPSR[11:10]} == 0, and beat-wise vector instructions were in progress when the exception was taken:



N, bit [31]

Negative flag. Value corresponding to APSR.N.

Z, bit [30]

Zero flag. Value corresponding to APSR.Z.

C, bit [29]

Carry flag. Value corresponding to APSR.C.

V, bit [28]

Overflow flag. Value corresponding to APSR.V.

Q, bit [27]

Saturate flag. Value corresponding to APSR.Q.

T, bit [24]

T32 state. Value corresponding to EPSR.T.

Bits [23:22]

Reserved, **RES0**.

B, bit [21]

Branch target identification active. Value corresponding to EPSR.B.

SFPA, bit [20]

Secure Floating-point active. Value corresponding to CONTROL.SFPA.

GE, bits [19:16]

Greater-than or equal flag. Value corresponding to APSR.GE.

IT, bits [15:10,26:25] , when [{RETPSR[26:25], RETPSR[11:10]} != 0]

If-then flags. Value corresponding to EPSR.IT.

ICI, bits [26:25,15:10] , when [{RETPSR[26:25], RETPSR[11:10]} == 0, and a multi-cycle load or store instruction was in progress when the exception was taken]

Interrupt continuation flags. Value corresponding to EPSR.ICI.

ECI, bits [26:25, 11:10, 15:12] , when [{RETPSR[26:25], RETPSR[11:10]} == 0, and beat-wise vector instructions were in progress when the exception was taken]

Exception continuation flags for beat-wise vector instructions. Value corresponding to EPSR.ECI.

SPREALIGN, bit [9]

Stack-pointer re-align. Indicates whether the SP was re-aligned to an 8-byte alignment on exception entry.

The possible values of this bit are:

0

The stack pointer was 8-byte aligned before exception entry began, no special handling is required on exception return.

1

The stack pointer was only 4-byte aligned before exception entry. The exception entry realigned SP to 8-byte alignment by increasing the stack frame size by 4-bytes.

Exception, bits [8:0]

Exception number. Value corresponding to IPSR.Exception.

D1.2.222 REVIDR, Revision ID Register

The REVIDR characteristics are:

Purpose

Provides implementation-specific minor revision information.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

Configurations

Present only if version Armv8.1-M of the architecture is implemented.

This register is [RES0](#) if Armv8.1-M is not implemented.

Attributes

32-bit read-only register located at 0xE000ECFC.

Secure software can access the Non-secure version of this register via REVIDR_NS located at 0xE002ECFC. The location 0xE002ECFC is [RES0](#) to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The REVIDR bit assignments are:



IMPLEMENTATION DEFINED, bits [31:0]

[IMPLEMENTATION DEFINED](#). The contents of this field are [IMPLEMENTATION DEFINED](#).

D1.2.223 RFSR, RAS Fault Status Register

The RFSR characteristics are:

Purpose

Records syndrome information for a RAS exception.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

If AIRCR.BFHFNMINS is zero this register is RAZ/WI from Non-secure state.

Configurations

Present only if version Armv8.1 of the architecture and RAS are implemented.

This register is **RES0** if RAS is not implemented.

Attributes

32-bit read/write register located at 0xE000EF04.

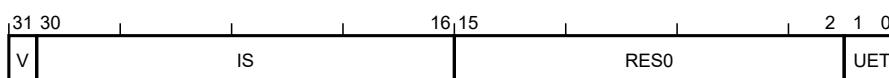
Secure software can access the Non-secure version of this register via RFSR_NS located at 0xE002EF04. The location 0xE002EF04 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The RFSR bit assignments are:



V, bit [31]

Valid. Indicates the register values are valid.

This bit is write-one-to-clear. Writes of zero are ignored.

This bit resets to zero on a Warm reset.

IS, bits [30:16]

IMPLEMENTATION DEFINED Syndrome. Contains additional **IMPLEMENTATION DEFINED** syndrome information.

Bits [15:2]

Reserved, **RES0**.

UET, bits [1:0]

Error Type. Describes the state of the processor after taking the RAS exception.

The possible values of this field are:

0b00

Uncontainable error (UC).

0b01

Unrecoverable error (UEU).

0b10

Restartable error (UEO).

0b11

Recoverable error (UER).

This field resets to zero on a Warm reset.

D1.2.224 Sn, Floating-point Single-precision register, n = 0 - 31

The S{0..31} characteristics are:

Purpose

Provides access to the floating-point register file. These registers are an alias of the Floating-point register file.

Usage constraints

Privileged and unprivileged access permitted.

Configurations

Present if the Floating-point Extension, or MVE, or both are implemented.

This register is [RES0](#) if neither the Floating-point Extension nor MVE are implemented.

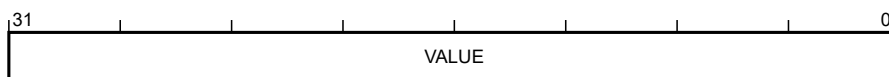
Attributes

32-bit read/write special-purpose register.

This register is not banked between Security states.

Field descriptions

The S{0..31} bit assignments are:



VALUE, bits [31:0]

Floating-point single-precision register value.

ENABLE, bit [0]

Enable. Enables the SAU.

The possible values of this bit are:

0

The SAU is disabled.

1

The SAU is enabled.

If this register resets to 1, the SAU region registers also reset to an **IMPLEMENTATION DEFINED** value.

This bit resets to an **IMPLEMENTATION DEFINED** value on a Warm reset.

D1.2.226 SAU_RBAR, SAU Region Base Address Register

The SAU_RBAR characteristics are:

Purpose

Provides indirect read and write access to the base address of the currently selected SAU region.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000EDDC.

To allow lock down of this register it is **IMPLEMENTATION DEFINED** whether this register is writable.

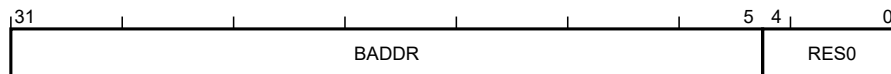
This register is RAZ/WI when accessed as Non-secure.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The SAU_RBAR bit assignments are:

**BADDR, bits [31:5]**

Base address. Holds bits [31:5] of the base address for the selected SAU region.

Bits [4:0] of the base address are defined as 0x00.

It is **IMPLEMENTATION DEFINED** whether any of the BADDR bits are WI

This field resets to an **IMPLEMENTATION DEFINED** value on a Warm reset.

Bits [4:0]

Reserved, RES0.

D1.2.227 SAU_RLAR, SAU Region Limit Address Register

The SAU_RLAR characteristics are:

Purpose

Provides indirect read and write access to the limit address of the currently selected SAU region.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000EDE0.

To allow lock down of this register it is **IMPLEMENTATION DEFINED** whether this register is writable.

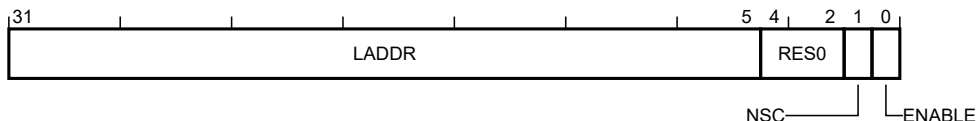
This register is RAZ/WI when accessed as Non-secure.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The SAU_RLAR bit assignments are:



LADDR, bits [31:5]

Limit address. Holds bits [31:5] of the limit address for the selected SAU region.

Bits [4:0] of the limit address are defined as 0x1F.

It is **IMPLEMENTATION DEFINED** whether any of the LADDR bits are WI

This field resets to an **IMPLEMENTATION DEFINED** value on a Warm reset.

Bits [4:2]

Reserved, **RES0**.

NSC, bit [1]

Non-secure callable. Controls whether Non-secure state is permitted to execute an SG instruction from this region.

The possible values of this bit are:

0

Region is marked with the Secure attribute and is not Non-secure callable.

1

Region is marked with the Secure attribute and is Non-secure callable.

This bit resets to an **IMPLEMENTATION DEFINED** value on a Warm reset.

ENABLE, bit [0]

Enable. SAU region enable.

The possible values of this bit are:

0

SAU region is disabled.

1

SAU region is enabled.

This bit resets to an **IMPLEMENTATION DEFINED** value on a Warm reset.

D1.2.228 SAU_RNR, SAU Region Number Register

The SAU_RNR characteristics are:

Purpose

Selects the region currently accessed by SAU_RBAR and SAU_RLAR.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000EDD8.

To allow lock down of this register it is **IMPLEMENTATION DEFINED** whether this register is writable.

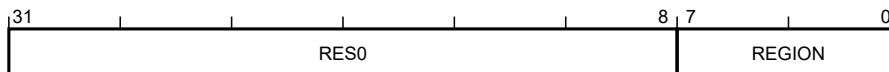
This register is RAZ/WI when accessed as Non-secure.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The SAU_RNR bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REGION, bits [7:0]

Region number. Indicates the SAU region accessed by SAU_RBAR and SAU_RLAR.

If no SAU regions are implemented, this field is **RES0**. Writing a value corresponding to an unimplemented region is **CONSTRAINED UNPREDICTABLE**.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.230 SCR, System Control Register

The SCR characteristics are:

Purpose

Sets or returns system control data.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED10.

Secure software can access the Non-secure version of this register via SCR_NS located at 0xE002ED10. The location 0xE002ED10 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The SCR bit assignments are:



Bits [31:5]

Reserved, **RES0**.

SEVONPEND, bit [4]

Send event on pend. Determines whether an interrupt assigned to the same Security state as the SEVONPEND bit transitioning from inactive state to pending state generates a wakeup event.

This bit is banked between Security states.

The possible values of this bit are:

0

Transitions from inactive to pending are not wakeup events.

1

Transitions from inactive to pending are wakeup events.

This bit resets to zero on a Warm reset.

SLEEPDEEPS, bit [3]

Sleep deep secure. This field controls whether the SLEEPDEEP bit is only accessible from the Secure state.

This bit is not banked between Security states.

The possible values of this bit are:

0

The SLEEPDEEP bit accessible from both Security states.

1

The SLEEPDEEP bit behaves as RAZ/WI when accessed from the Non-secure state.

This bit is only accessible from the Secure state, and behaves as RAZ/WI when accessed from the Non-secure state. If a PE does not implement the deep sleep state this bit behaves as RAZ/WI from both Security states.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

SLEEPDEEP, bit [2]

Sleep deep. Provides a qualifying hint indicating that waking from sleep might take longer. An implementation can use this bit to select between two alternative sleep states.

This bit is not banked between Security states.

The possible values of this bit are:

0

Selected sleep state is not deep sleep.

1

Selected sleep state is deep sleep.

Details of the implemented sleep states, if any, and details of the use of this bit, are [IMPLEMENTATION DEFINED](#). If the PE does not implement a deep sleep state then this bit can be RAZ/WI.

This bit resets to zero on a Warm reset.

SLEEPONEXIT, bit [1]

Sleep on exit. Determines whether, on an exit from an ISR that returns to the base level of execution priority, the PE enters a sleep state.

This bit is banked between Security states.

The possible values of this bit are:

0

Enter sleep state disabled.

1

Enter sleep state permitted.

The Secure version of this field is used if the Background state being returned to is the Secure state, otherwise the Non-secure version is used.

This bit resets to zero on a Warm reset.

Bit [0]

Reserved, [RES0](#).

D1.2.231 SFAR, Secure Fault Address Register

The SFAR characteristics are:

Purpose

Shows the address of the memory location that caused a Security violation.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read/write register located at 0xE000EDE8.

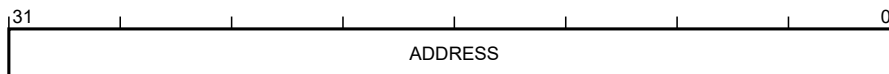
This register is RAZ/WI when accessed as Non-secure.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The SFAR bit assignments are:



ADDRESS, bits [31:0]

Address. The address of an access that caused an attribution unit violation. This field is only valid when SFSR.SFARVALID is set, otherwise the value in this register is **UNKNOWN**. This allows the actual flip flops associated with this register to be shared with other fault address registers. If an implementation chooses to share the storage in this way, care must be taken to not leak Secure address information to the Non-secure state. One way of achieving this is to share the SFAR register with the MMFAR_S register, which is not accessible to the Non-secure state.

In implementations without unique SFAR, BFAR and MMFAR registers, the value of this register is **UNKNOWN** if MMFSR.MMFARVALID or BFSR.BFARVALID is set.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.232 SFSR, Secure Fault Status Register

The SFSR characteristics are:

Purpose

Provides information about any security related faults.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

32-bit read/write-one-to-clear register located at 0xE00EDE4.

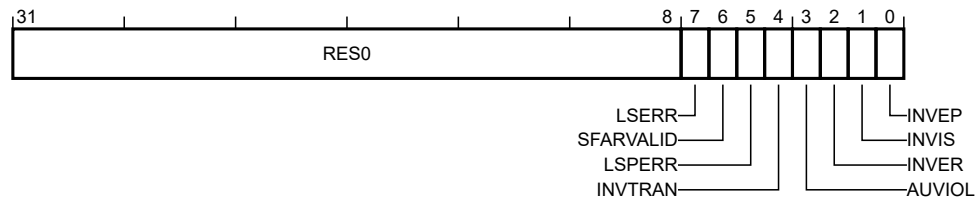
This register is RAZ/WI when accessed as Non-secure.

This register is not banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The SFSR bit assignments are:

**Bits [31:8]**

Reserved, RES0.

LSERR, bit [7]

Lazy state error flag. Sticky flag indicating that an error occurred during lazy Floating-point state preservation activation or deactivation.

The possible values of this bit are:

0

Error has not occurred.

1

Error has occurred.

This bit resets to zero on a Warm reset.

SFARVALID, bit [6]

Secure fault address valid. This bit is set when the SFAR register contains a valid value. As with similar fields, such as BFSR.BFARVALID and MMFSR.MMFARVALID, this bit can be cleared by other exceptions, such as BusFault.

The possible values of this bit are:

0
SFAR content not valid.

1
SFAR content valid.

This bit resets to zero on a Warm reset.

LSPERR, bit [5]

Lazy state preservation error flag. Sticky flag indicating that an SAU or IDAU violation occurred during the lazy Floating-point state preservation.

The possible values of this bit are:

0
Error has not occurred.

1
Error has occurred.

This bit resets to zero on a Warm reset.

INVTRAN, bit [4]

Invalid transition flag. Sticky flag indicating that an exception was raised due to a branch that was not flagged as being domain crossing causing a transition from Secure to Non-secure memory.

The possible values of this bit are:

0
Error has not occurred.

1
Error has occurred.

This bit resets to zero on a Warm reset.

AUVIOL, bit [3]

Attribution unit violation flag.

Sticky flag indicating that an attempt was made to access parts of the address space that are marked as Secure with NS-Req for the transaction set to Non-secure.

This bit is not set if the violation occurred during:

- Lazy state preservation, see LSPERR.
- Vector fetches.

The possible values of this bit are:

0
Error has not occurred.

1
Error has occurred.

This bit resets to zero on a Warm reset.

INVER, bit [2]

Invalid exception return flag. This can be caused by EXC_RETURN.DCRS being set to 0 when returning from an exception in the Non-secure state, or by EXC_RETURN.ES being set to 1 when returning from an exception in the Non-secure state.

The possible values of this bit are:

0
Error has not occurred.

1

Error has occurred.

This bit resets to zero on a Warm reset.

INVIS, bit [1]

Invalid integrity signature flag. This bit is set if the integrity signature in an exception stack frame is found to be invalid during the unstacking operation.

The possible values of this bit are:

0

Error has not occurred.

1

Error has occurred.

This bit resets to zero on a Warm reset.

INVEP, bit [0]

Invalid entry point. This bit is set if there is an invalid attempt to enter Secure state.

The possible values of this bit are:

0

Error has not occurred.

1

Error has occurred.

This bit resets to zero on a Warm reset.

D1.2.233 SHCSR, System Handler Control and State Register

The SHCSR characteristics are:

Purpose

Provides access to the active and pending status of system exceptions.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED24.

Secure software can access the Non-secure version of this register via SHCSR_NS located at 0xE002ED24. The location 0xE002ED24 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

Exception processing automatically updates the SHCSR fields. However, software can write to the register to add or remove the pending or active state of an exception. When updating the SHCSR, Arm recommends using a read-modify-write sequence, to avoid unintended effects on the state of the exception handlers.

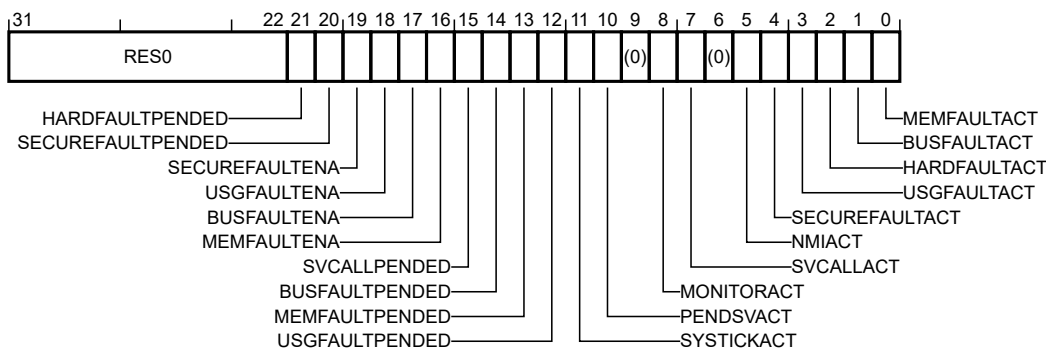
Removing the active state of an exception can change the current execution priority, and affect the exception return consistency checks. If software removes the active state, causing a change in current execution priority, this can defeat the architectural behavior that prevents an exception from preempting its own handler.

Pending state bits are set to one when an exception occurs and are cleared to zero when the exception becomes active.

Active state bits are set to one when the associated exception becomes active.

Field descriptions

The SHCSR bit assignments are:



Bits [31:22]

Reserved, **RES0**.

HARDFaultPENDED, bit [21]

HardFault exception pended state. This bit indicates and allows modification of the pending state of the HardFault exception corresponding to the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

HardFault exception not pending for the selected Security state.

1

HardFault exception pending for the selected Security state.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

Note

The Non-secure HardFault exception will not preempt if AIRCR.BFHFNMINS is set to zero.

SECUREFaultPENDED, bit [20]

SecureFault exception pended state. This bit indicates and allows modification of the pending state of the SecureFault exception.

This bit is not banked between Security states.

The possible values of this bit are:

0

SecureFault exception not pending.

1

SecureFault exception pending.

This bit is RAZ/WI from Non-secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

SECUREFaultENA, bit [19]

SecureFault exception enable. The value of this bit defines whether the SecureFault exception is enabled.

This bit is not banked between Security states.

The possible values of this bit are:

0

SecureFault exception disabled.

1

SecureFault exception enabled.

When disabled, exceptions that target SecureFault escalate to Secure state HardFault.

This bit is RAZ/WI from Non-secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

USGFaultENA, bit [18]

UsageFault exception enable. The value of this bit defines whether the UsageFault exception is enabled for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

UsageFault exception disabled for the selected Security state.

1

UsageFault exception enabled for the selected Security state.

When the UsageFault exception is disabled, exceptions targeting UsageFault escalate to HardFault.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

BUSFAULTENA, bit [17]

BusFault exception enable. The value of this bit defines whether the BusFault exception is enabled.

This bit is not banked between Security states.

The possible values of this bit are:

0

BusFault exception disabled.

1

BusFault exception enabled.

The BusFault exception is not banked between Security states. When the BusFault exception is disabled, exceptions targeting BusFault escalate to HardFault.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

MEMFAULTENA, bit [16]

MemManage exception enable. The value of this bit defines whether the MemManage exception is enabled for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

MemManage exception disabled for the selected Security state.

1

MemManage exception enabled for the selected Security state.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

Note

When the MemManage exception is disabled, exceptions targeting MemManage escalate to HardFault.

SVCALLPENDEd, bit [15]

SVCALL exception pending state. This bit indicates and allows modification of the pending state of the SVCALL exception for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0
SVCALL exception not pending for the selected Security state.

1
SVCALL exception pending for the selected Security state.

This bit resets to zero on a Warm reset.

BUSFAULTPENDEd, bit [14]

BusFault exception pended state. This bit indicates and allows modification of the pending state of the BusFault exception.

This bit is not banked between Security states.

The possible values of this bit are:

0
BusFault exception not pending.

1
BusFault exception pending.

The BusFault exception is not banked between Security states.

If AIRCR.BFHFNMINs is zero this bit is RAZ/WI from Non-secure state.

If the Main Extension is not implemented, this bit is **RES0**.

This bit resets to zero on a Warm reset.

MEMFAULTPENDEd, bit [13]

MemManage exception pended state. This bit indicates and allows modification of the pending state of the MemManage exception for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0
MemManage exception not pending for the selected Security state.

1
MemManage exception pending for the selected Security state.

If the Main Extension is not implemented, this bit is **RES0**.

This bit resets to zero on a Warm reset.

USGFAULTPENDEd, bit [12]

UsageFault exception pended state. The UsageFault exception is banked between Security states, this bit indicates and allows modification of the pending state of the UsageFault exception for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0
UsageFault exception not pending for the selected Security state.

1
UsageFault exception pending for the selected Security state.

If the Main Extension is not implemented, this bit is **RES0**.

This bit resets to zero on a Warm reset.

SYSTICKACT, bit [11]

SysTick exception active state. This bit indicates and allows modification of the active state of the SysTick exception for the selected Security state.

If two SysTick timers are implemented this bit is banked between Security states.

If less than two SysTick timers are implemented this bit is not banked between Security states.

The possible values of this bit are:

0

SysTick exception not active for the selected Security state.

1

SysTick exception active for the selected Security state.

If two timers are implemented, then SYSTICKACT is banked between Security states. If one timer is implemented this bit corresponds to the Secure state if ICSR.STTNS is zero, or the Non-secure state if ICSR.STTNS is one.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is [RES0](#).

This bit resets to zero on a Warm reset.

PENDSVACT, bit [10]

PendSV exception active state. This bit indicates and allows modification of the active state of the PendSV exception for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

PendSV exception not active for the selected Security state.

1

PendSV exception active for the selected Security state.

This bit resets to zero on a Warm reset.

Bit [9]

Reserved, [RES0](#).

MONITORACT, bit [8]

DebugMonitor exception active state. This bit indicates and allows modification of the active state of the DebugMonitor exception.

This bit is not banked between Security states.

The possible values of this bit are:

0

DebugMonitor exception not active.

1

DebugMonitor exception active.

If DEMCR.SDME is one this bit is RAZ/WI from Non-secure state

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

SVCALLACT, bit [7]

SVCALL exception active state. This bit indicates and allows modification of the active state of the SVCALL exception for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

SVCALL exception not active for the selected Security state.

1

SVCALL exception active for the selected Security state.

This bit resets to zero on a Warm reset.

Bit [6]

Reserved, [RES0](#).

NMIACT, bit [5]

NMI exception active state. This bit indicates and allows modification of the active state of the NMI exception.

This bit is not banked between Security states.

The possible values of this bit are:

0

NMI exception not active.

1

NMI exception active.

The NMI exception is not banked between Security states. This field behaves as writes ignored unless all the following are true:

- AIRCR.BFHFNMINS is set to one.
- This field is being written to zero.
- Either the access is from Secure software and targets the Non-secure alias or the access is from the debugger, DHCSR.S_SDE is one and the access targets the Non-secure bank of the register.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

SECUREFAULTACT, bit [4]

SecureFault exception active state. This bit indicates and allows modification of the active state of the SecureFault exception.

This bit is not banked between Security states.

The possible values of this bit are:

0

SecureFault exception not active.

1

SecureFault exception active.

This bit is RAZ/WI from Non-secure state.

This bit resets to zero on a Warm reset.

USGFAULTACT, bit [3]

UsageFault exception active state for the selected Security state. This bit indicates and allows modification of the active state of the UsageFault exception for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

UsageFault exception not active for the selected Security state.

1

UsageFault exception active for the selected Security state.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

HARDFFAULTACT, bit [2]

HardFault exception active state. Indicates and allows limited modification of the active state of the HardFault exception for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

HardFault exception not active for the selected Security state.

1

HardFault exception active for the selected Security state.

This field ignores writes when any of the following are true:

- The field is being written to one.
- The write targets the Secure instance of this field.
- The access is from Non-secure state or from a debugger when DHCSR.S_SDE is zero.

This bit resets to zero on a Warm reset.

BUSEFAULTACT, bit [1]

BusFault exception active state. This bit indicates and allows modification of the active state of the BusFault exception.

This bit is not banked between Security states.

The possible values of this bit are:

0

BusFault exception not active.

1

BusFault exception active.

The BusFault exception is not banked between Security states.

If AIRCR.BFHFNMINS is zero this bit is RAZ/WI from Non-secure state.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

MEMFAULTACT, bit [0]

MemManage exception active state for the selected Security state. This bit indicates and allows modification of the active state of the MemManage exception for the selected Security state.

This bit is banked between Security states.

The possible values of this bit are:

0

MemManage exception not active for the selected Security state.

1

MemManage exception active for the selected Security state.

If the Main Extension is not implemented, this bit is [RES0](#).

This bit resets to zero on a Warm reset.

D1.2.234 SHPR1, System Handler Priority Register 1

The SHPR1 characteristics are:

Purpose

Sets or returns priority for system handlers 4 - 7.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If the Main Extension is implemented, this register is word, halfword, and byte accessible.

If the Main Extension is not implemented, this register is word accessible only, halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

32-bit read/write register located at 0xE000ED18.

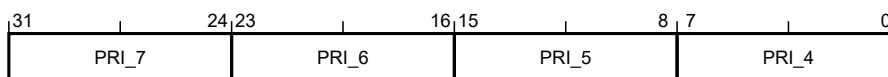
Secure software can access the Non-secure version of this register via SHPR1_NS located at 0xE002ED18. The location 0xE002ED18 is [RES0](#) to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

From Armv8.1-M onwards it is [IMPLEMENTATION DEFINED](#) whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The SHPR1 bit assignments are:



PRI_7, bits [31:24]

Priority 7. Priority of system handler 7, SecureFault.

This field is not banked between Security states.

If the PE implements fewer than 8 bits of priority, then the least significant bits of this field are RAZ/WI.

This field is RAZ/WI from Non-secure state.

This field resets to zero on a Warm reset.

PRI_6, bits [23:16]

Priority 6. Priority of system handler 6, UsageFault.

This field is banked between Security states.

If the PE implements fewer than 8 bits of priority, then the least significant bits of this field are RAZ/WI.

This field resets to zero on a Warm reset.

PRI_5, bits [15:8]

Priority 5. Priority of system handler 5, BusFault.

This field is not banked between Security states.

If the PE implements fewer than 8 bits of priority, then the least significant bits of this field are RAZ/WI.

If AIRCR.BFHFNMINS is zero this field is RAZ/WI from Non-secure state.

This field resets to zero on a Warm reset.

PRI_4, bits [7:0]

Priority 4. Priority of system handler 4, MemManage.

This field is banked between Security states.

If the PE implements fewer than 8 bits of priority, then the least significant bits of this field are RAZ/WI.

This field resets to zero on a Warm reset.

D1.2.235 SHPR2, System Handler Priority Register 2

The SHPR2 characteristics are:

Purpose

Sets or returns priority for system handlers 8 - 11.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If the Main Extension is implemented, this register is word, halfword, and byte accessible.

If the Main Extension is not implemented, this register is word accessible only, halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED1C.

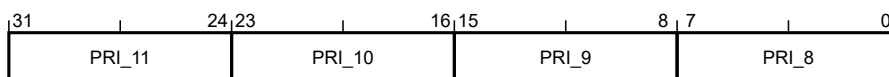
Secure software can access the Non-secure version of this register via SHPR2_NS located at 0xE002ED1C. The location 0xE002ED1C is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The SHPR2 bit assignments are:



PRI_11, bits [31:24]

Priority 11. Priority of system handler 11, SVCALL.

This field is banked between Security states.

If the PE implements fewer than 8 bits of priority, then the least significant bits of this field are RAZ/WI.

This field resets to zero on a Warm reset.

PRI_10, bits [23:16]

Reserved, **RES0**.

PRI_9, bits [15:8]

Reserved, **RES0**.

PRI_8, bits [7:0]

Reserved, **RES0**.

D1.2.236 SHPR3, System Handler Priority Register 3

The SHPR3 characteristics are:

Purpose

Sets or returns priority for system handlers 12 - 15.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If the Main Extension is implemented, this register is word, halfword, and byte accessible.

If the Main Extension is not implemented, this register is word accessible only, halfword and byte accesses are **UNPREDICTABLE**.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED20.

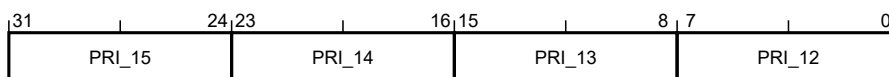
Secure software can access the Non-secure version of this register via SHPR3_NS located at 0xE002ED20. The location 0xE002ED20 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states on a bit by bit basis.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The SHPR3 bit assignments are:



PRI_15, bits [31:24]

Priority 15. Priority of system handler 15, SysTick.

If two SysTick timers are implemented this field is banked between Security states.

If less than two SysTick timers are implemented this field is not banked between Security states.

If the PE implements fewer than 8 bits of priority, then the least significant bits of this field are RAZ/WI. If one timer is implemented, this field corresponds to the Secure state if ICSR.STTNS is zero, or the Non-secure state if ICSR.STTNS is one.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this field is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this field is **RES0**.

This field resets to zero on a Warm reset.

PRI_14, bits [23:16]

Priority 14. Priority of system handler 14, PendSV.

This field is banked between Security states.

If the PE implements fewer than 8 bits of priority, then the least significant bits of this field are RAZ/WI.

This field resets to zero on a Warm reset.

PRI_13, bits [15:8]

Reserved, [RES0](#).

PRI_12, bits [7:0]

Priority 12. Priority of system handler 12, DebugMonitor.

This field is not banked between Security states.

If the PE implements fewer than 8 bits of priority, then the least significant bits of this field are RAZ/WI.

If DEMCR.SDME is one this field is RAZ/WI from Non-secure state

If the Main Extension is not implemented, this field is [RES0](#).

This field resets to zero on a Warm reset.

D1.2.237 SP, Current Stack Pointer Register

The SP characteristics are:

Purpose
Exception and procedure stack pointer register.

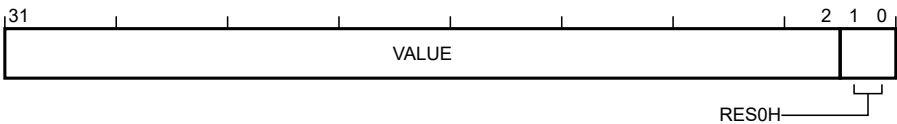
Usage constraints
Privileged and unprivileged access permitted.

Configurations
This register is always implemented.

Attributes
32-bit read/write special-purpose register.
This register is not banked between Security states.

Field descriptions

The SP bit assignments are:



VALUE, bits [31:2]
Stack pointer. Holds bits[31:2] of the stack pointer address. The current stack pointer is selected from one of MSP_NS, PSP_NS, MSP_S or PSP_S.
Software can refer to SP as R13.

Bits [1:0]
Reserved, [RES0H](#).

D1.2.238 SP_NS, Current Stack Pointer register (Non-secure)

The SP_NS characteristics are:

Purpose

Provides access to the current Non-secure stack pointer.

Usage constraints

Privileged and unprivileged access permitted.

Configurations

This register is always implemented.

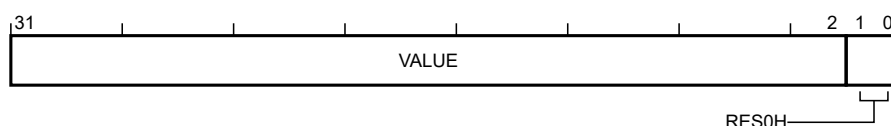
Attributes

32-bit read/write special-purpose register.

This register is not banked between Security states.

Field descriptions

The SP_NS bit assignments are:



VALUE, bits [31:2]

Stack pointer. Holds bits[31:2] of the current Non-secure stack pointer address. SP_NS is selected from one of MSP_NS or PSP_NS. Access to SP_NS is provided via MRS and MSR and is subject to stack limit checking.

Bits [1:0]

Reserved, [RES0H](#).

D1.2.239 STIR, Software Triggered Interrupt Register

The STIR characteristics are:

Purpose

Provides a mechanism for software to generate an interrupt.

Usage constraints

Unprivileged accesses generate a fault if CCR.USERSETMPEND is zero.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

Present only if the Main Extension is implemented.

This register is **RES0** if the Main Extension is not implemented.

Attributes

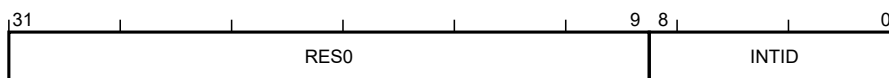
32-bit write-only register located at 0xE000EF00.

Secure software can access the Non-secure version of this register via STIR_NS located at 0xE002EF00. The location 0xE002EF00 is **RES0** to software executing in Non-secure state and the debugger.

This register is not banked between Security states.

Field descriptions

The STIR bit assignments are:

**Bits [31:9]**

Reserved, RES0.

INTID, bits [8:0], on a write

Interrupt ID. Indicates the interrupt to be pended. The value written is (ExceptionNumber - 16).

Writing to this register has the same effect as setting the NVIC_ISPR*n* bit corresponding to the interrupt to 1. Like NVIC_ISPR*n*, an attempt to pend an interrupt targeting Secure state from Non-secure is ignored.

INTID, bits [8:0], on a read

This field reads as zero.

D1.2.240 SYST_CALIB, SysTick Calibration Value Register

The SYST_CALIB characteristics are:

Purpose

Reads the SysTick timer calibration value and parameters for the selected Security state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if at least one SysTick timer is implemented.

This register is **RES0** if no SysTick timer is implemented.

Attributes

32-bit read-only register located at 0xE000E01C.

Secure software can access the Non-secure version of this register via SYST_CALIB_NS located at 0xE002E01C. The location 0xE002E01C is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

Preface

If the Main Extension is implemented then, two SysTick timers are implemented. If the Main Extension is not implemented, then it is **IMPLEMENTATION DEFINED** whether none, one or two SysTick timers are implemented. Where two SysTick timers are implemented, this register is banked. Where one SysTick timer is implemented, this register is not banked, and Non-secure accesses behave as RAZ/WI if ICSR.STTNS is clear. If no SysTick timer is implemented, both aliases of this register behave as **RES0**.

Field descriptions

The SYST_CALIB bit assignments are:



NOREF, bit [31]

No reference. Indicates whether the **IMPLEMENTATION DEFINED** reference clock is implemented.

The possible values of this bit are:

0

Reference clock is implemented.

1

Reference clock is not implemented.

When this bit is 1, the CLKSOURCE bit of the SYST_CSR register is forced to 1 and cannot be cleared to 0.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is **RES0**.

This bit resets to an **IMPLEMENTATION DEFINED** value on a Warm reset.

This bit reads as an **IMPLEMENTATION DEFINED** value.

SKEW, bit [30]

Skew. Indicates whether the 10ms calibration value is exact.

The possible values of this bit are:

0

TENMS calibration value is exact.

1

TENMS calibration value is inexact.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is **RES0**.

This bit resets to an **IMPLEMENTATION DEFINED** value on a Warm reset.

This bit reads as an **IMPLEMENTATION DEFINED** value.

Bits [29:24]

Reserved, **RES0**.

TENMS, bits [23:0]

Ten milliseconds. Optionally holds a reload value to be used for 10ms (100Hz) timing, subject to system clock skew errors. If this field is zero, the calibration value is not known.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this field is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this field is **RES0**.

This field resets to an **IMPLEMENTATION DEFINED** value on a Warm reset.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.241 SYST_CSR, SysTick Control and Status Register

The SYST_CSR characteristics are:

Purpose

Controls the SysTick timer and provides status data for the selected Security state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if at least one SysTick timer is implemented.

This register is **RES0** if no SysTick timer is implemented.

Attributes

32-bit read/write register located at 0xE000E010.

Secure software can access the Non-secure version of this register via SYST_CSR_NS located at 0xE002E010. The location 0xE002E010 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

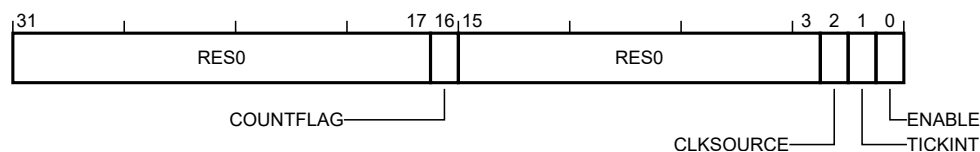
From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

If the Main Extension is implemented, then two SysTick timers are implemented. If the Main Extension is not implemented, then it is **IMPLEMENTATION DEFINED** whether none, one or two SysTick timers are implemented. Where two SysTick timers are implemented, this register is banked. Where one SysTick timer is implemented, this register is not banked, and Non-secure accesses behave as RAZ/WI if ICSR.STTNS is clear. If no SysTick timer is implemented, both aliases of this register behave as **RES0**.

Field descriptions

The SYST_CSR bit assignments are:



Bits [31:17]

Reserved, **RES0**.

COUNTFLAG, bit [16]

Count flag. Indicates whether the counter has counted to zero since the last read of this register.

The possible values of this bit are:

0

Timer has not counted to 0.

1

Timer has counted to 0.

COUNTFLAG is set to 1 by a count transition from 1 to 0. COUNTFLAG is cleared to 0 if software reads this bit as one, and by any write to the SYST_CVR for the selected Security state. Debugger reads do not clear the COUNTFLAG.

If set this bit clears to zero when read by software. Reads from the debugger do not clear this bit.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is [RES0](#).

This bit resets to zero on a Warm reset.

Bits [15:3]

Reserved, [RES0](#).

CLKSOURCE, bit [2]

Clock source. Indicates the SysTick clock source.

The possible values of this bit are:

0

Uses the [IMPLEMENTATION DEFINED](#) external reference clock.

1

Uses the PE clock.

If no external clock is implemented, this bit reads as 1 and ignores writes.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is [RES0](#).

This bit resets to an [IMPLEMENTATION DEFINED](#) value on a Warm reset.

TICKINT, bit [1]

Tick interrupt. Indicates whether counting to 0 causes the status of the SysTick exception to change to pending.

The possible values of this bit are:

0

Count to 0 does not affect the SysTick exception status.

1

Count to 0 changes the SysTick exception status to pending.

Changing the value of the counter to 0 by writing the SysTick does not change the status of the SysTick exception.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is [RES0](#).

This bit resets to zero on a Warm reset.

ENABLE, bit [0]

SysTick enable. Indicates the enabled status of the SysTick counter.

The possible values of this bit are:

0

Counter is disabled.

1

Counter is enabled.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this bit is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this bit is [RES0](#).

This bit resets to zero on a Warm reset.

D1.2.242 SYST_CVR, SysTick Current Value Register

The SYST_CVR characteristics are:

Purpose

Reads or clears the SysTick timer current counter value for the selected Security state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if at least one SysTick timer is implemented.

This register is **RES0** if no SysTick timer is implemented.

Attributes

32-bit read/write-to-clear register located at 0xE000E018.

Secure software can access the Non-secure version of this register via SYST_CVR_NS located at 0xE002E018. The location 0xE002E018 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

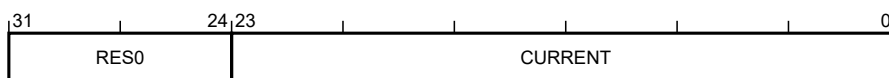
From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

If the Main Extension is implemented, then two SysTick timers are implemented. If the Main Extension is not implemented, then it is **IMPLEMENTATION DEFINED** whether none, one or two SysTick timers are implemented. Where two SysTick timers are implemented, this register is banked. Where one SysTick timer is implemented, this register is not banked, and Non-secure accesses behave as RAZ/WI if ICSR.STTNS is clear. If no SysTick timer is implemented, both aliases of this register behave as **RES0**.

Field descriptions

The SYST_CVR bit assignments are:



Bits [31:24]

Reserved, **RES0**.

CURRENT, bits [23:0], on a read

Current counter value. Provides the value of the SysTick timer counter for the selected Security state.

It is **IMPLEMENTATION DEFINED** whether the current counter value decrements if the PE is sleeping and SCR.SLEEPDEEP is set.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this field is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this field is **RES0**.

This field resets to an **UNKNOWN** value on a Warm reset.

CURRENT, bits [23:0], on a write

Reset counter value. Writing any value clears the SysTick timer counter for the selected Security state to zero.

D1.2.243 SYST_RVR, SysTick Reload Value Register

The SYST_RVR characteristics are:

Purpose

Provides access SysTick timer counter reload value for the selected Security state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

Configurations

Present only if at least one SysTick timer is implemented.

This register is **RES0** if no SysTick timer is implemented.

Attributes

32-bit read/write register located at 0xE000E014.

Secure software can access the Non-secure version of this register via SYST_RVR_NS located at 0xE002E014. The location 0xE002E014 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

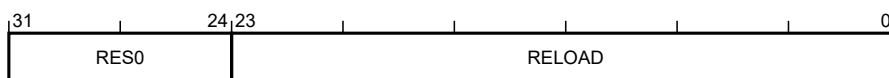
From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Preface

If the Main Extension is implemented, then two SysTick timers are implemented. If the Main Extension is not implemented, then it is **IMPLEMENTATION DEFINED** whether none, one or two SysTick timers are implemented. Where two SysTick timers are implemented, this register is banked. Where one SysTick timer is implemented, this register is not banked, and Non-secure accesses behave as RAZ/WI if ICSR.STTNS is clear. If no SysTick timer is implemented, both instances of this register behave as **RES0**.

Field descriptions

The SYST_RVR bit assignments are:



Bits [31:24]

Reserved, **RES0**.

RELOAD, bits [23:0]

Counter reload value. The value to load into the SYST_CVR for the selected Security state when the counter reaches 0.

If only one SysTick timer is implemented and ICSR.STTNS is clear, this field is RAZ/WI from Non-secure state.

If no SysTick timer is implemented this field is **RES0**.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.244 TPIU_ACPR, TPIU Asynchronous Clock Prescaler Register

The TPIU_ACPR characteristics are:

Purpose

Defines a prescaler value for the baud rate of the Serial Wire Output (SWO). Writing to the register automatically updates the prescale counter, immediately affecting the baud rate of the serial data output.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

If a debugger changes the register value while the TPIU is transmitting data, the effect on the output stream is **UNPREDICTABLE** and the required recovery process is **IMPLEMENTATION DEFINED**.

Configurations

Present only if the TPIU is implemented and supports SWO.

This register is **RES0** if the TPIU is not implemented or does not support SWO.

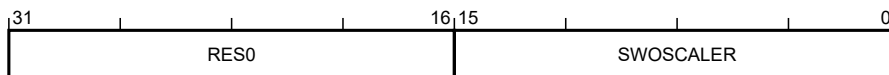
Attributes

32-bit read/write register located at 0xE0040010.

This register is not banked between Security states.

Field descriptions

The TPIU_ACPR bit assignments are:



Bits [31:16]

Reserved, **RES0**.

SWOSCALER, bits [15:0]

SWO and Parallel trace port baud rate prescaler. Sets the ratio between an **IMPLEMENTATION DEFINED** reference clock and the TPIU output clock rates. The prescaler always sets the ratio for the SWO output clock. When TPIU_DEVID.CPPT is one, the prescaler also sets the ratio for the Parallel trace port clock. The supported scaler value range is **IMPLEMENTATION DEFINED**, to a maximum scaler value of 0xFFFF. Unused bits of this field are RAZ/WI.

The possible values of this field are:

n

$$\text{SWO or Parallel trace port output clock} = \text{Asynchronous_Reference_Clock} / (n + 1).$$

This field resets to zero on a Cold reset.

D1.2.245 TPIU_CIDR0, TPIU Component Identification Register 0

The TPIU_CIDR0 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

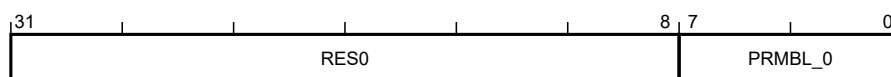
Attributes

32-bit read-only register located at 0xE0040FF0.

This register is not banked between Security states.

Field descriptions

The TPIU_CIDR0 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

PRMBL_0, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0D.

D1.2.246 TPIU_CIDR1, TPIU Component Identification Register 1

The TPIU_CIDR1 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

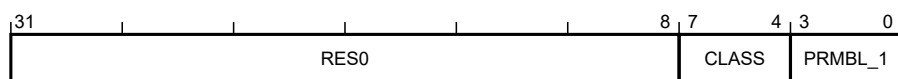
Attributes

32-bit read-only register located at 0xE0040FF4.

This register is not banked between Security states.

Field descriptions

The TPIU_CIDR1 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

CLASS, bits [7:4]

CoreSight component class. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x9.

PRMBL_1, bits [3:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x0.

D1.2.247 TPIU_CIDR2, TPIU Component Identification Register 2

The TPIU_CIDR2 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

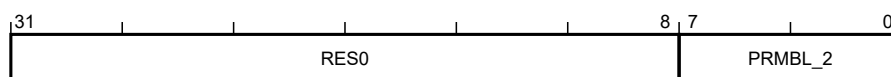
Attributes

32-bit read-only register located at 0xE0040FF8.

This register is not banked between Security states.

Field descriptions

The TPIU_CIDR2 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

PRMBL_2, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0x05.

D1.2.248 TPIU_CIDR3, TPIU Component Identification Register 3

The TPIU_CIDR3 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is [IMPLEMENTATION DEFINED](#) whether this register is accessible only to the debugger and [RES0](#) for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is [RES0](#) if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is [RES0](#) if the TPIU is not implemented.

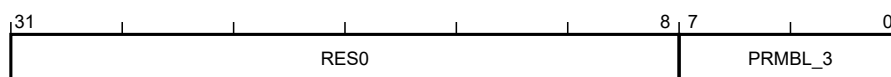
Attributes

32-bit read-only register located at 0xE0040FFC.

This register is not banked between Security states.

Field descriptions

The TPIU_CIDR3 bit assignments are:



Bits [31:8]

Reserved, [RES0](#).

PRMBL_3, bits [7:0]

CoreSight component identification preamble. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as 0xB1.

D1.2.249 TPIU_CLAIMCLR, TPIU Claim Tag Clear Register

The TPIU_CLAIMCLR characteristics are:

Purpose

In conjunction with TPIU_CLAIMSET, provides Claim Tag bits that can be separately set and cleared to indicate whether functionality is in use by a debug agent.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

If a debugger changes the register value while the TPIU is transmitting data, the effect on the output stream is **UNPREDICTABLE** and the required recovery process is **IMPLEMENTATION DEFINED**.

Configurations

Present only if the TPIU is implemented and supports SWO.

This register is **RES0** if the TPIU is not implemented or does not support SWO.

Present only if version Armv8.1-M of the architecture is implemented.

This register is **RES0** if Armv8.1-M is not implemented.

Attributes

32-bit read/write register located at 0xE0040FA4.

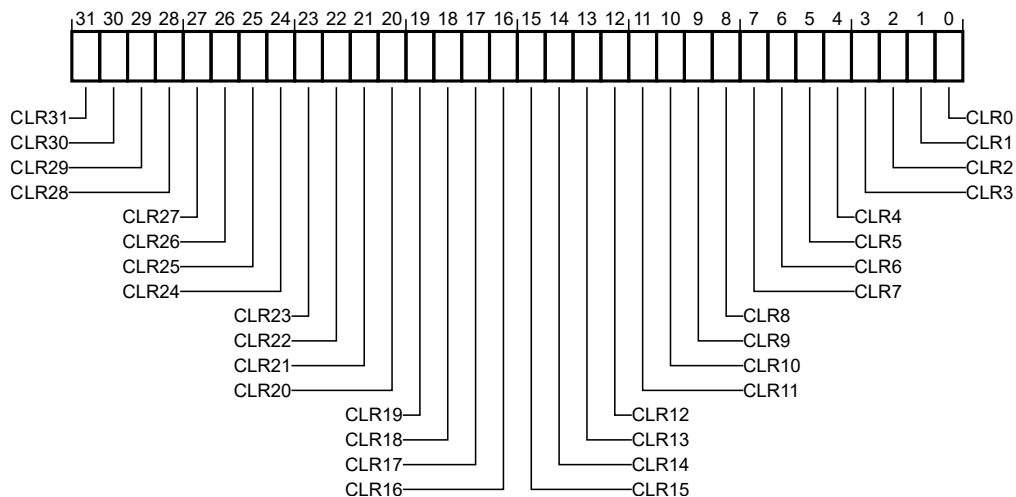
This register is not banked between Security states.

Preface

Unimplemented Claim Tag bits are RAZ/WI.

Field descriptions

The TPIU_CLAIMCLR bit assignments are:



CLR*m*, bit [*m*], for *m* = 0 to 31, on a read

Claim Tag Clear. Indicates the current status of Claim Tag bit *m*, and is used to set Claim Tag bit *m* to 0b0.

The possible values of this field are:

0b0

Claim Tag bit *m* is not set.

0b1

Claim Tag bit *m* is set.

The number of Claim Tag bits is indicated in TPIU_CLAIMSET.

This field resets to zero on a Cold reset.

CLR*m*, bit [*m*], for *m* = 0 to 31, on a write

Claim Tag Clear. Indicates the current status of Claim Tag bit *m*, and is used to set Claim Tag bit *m* to 0b0.

The possible values of this field are:

0b0

Ignored.

0b1

Clear Claim Tag bit *m* to 0b0.

The number of Claim Tag bits is indicated in TPIU_CLAIMSET.

This field is write-one-to-clear. Writes of zero are ignored.

D1.2.250 TPIU_CLAIMSET, TPIU Claim Tag Set Register

The TPIU_CLAIMSET characteristics are:

Purpose

In conjunction with TPIU_CLAIMCLR, provides Claim Tag bits that can be separately set and cleared to indicate whether functionality is in use by a debug agent.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

If a debugger changes the register value while the TPIU is transmitting data, the effect on the output stream is **UNPREDICTABLE** and the required recovery process is **IMPLEMENTATION DEFINED**.

Configurations

Present only if the TPIU is implemented and supports SWO.

This register is **RES0** if the TPIU is not implemented or does not support SWO.

Present only if version Armv8.1-M of the architecture is implemented.

This register is **RES0** if Armv8.1-M is not implemented.

Attributes

32-bit read/write register located at 0xE0040FA0.

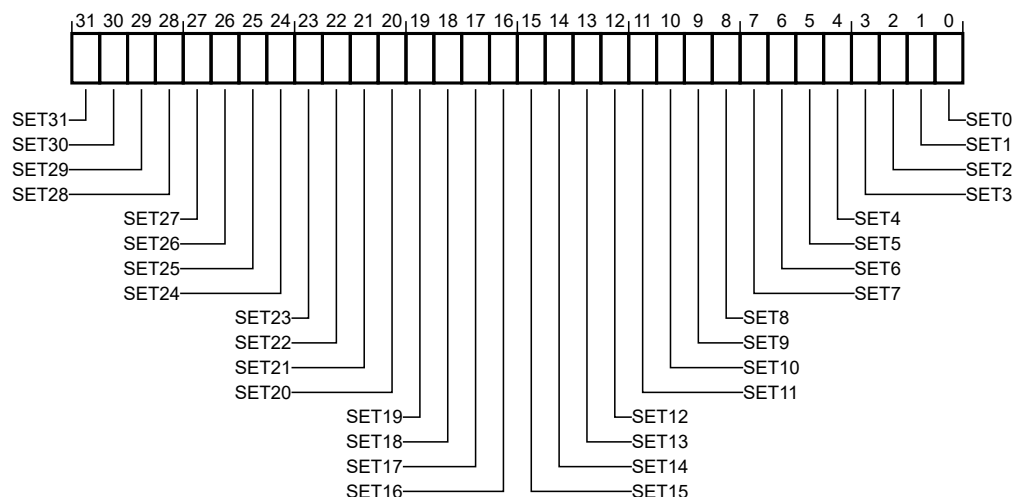
This register is not banked between Security states.

Preface

Unimplemented Claim Tag bits are RAZ/WI.

Field descriptions

The TPIU_CLAIMSET bit assignments are:



SET m , bit [m], for $m = 0$ to 31, on a read

Claim Tag Set. Indicates whether Claim Tag bit m is implemented, and is used to set Claim Tag bit m to 0b1.

The possible values of this field are:

0b0

Claim Tag bit m is not implemented.

0b1

Claim Tag bit m is implemented.

This field resets to 0xF on a Cold reset.

SET m , bit [m], for $m = 0$ to 31, on a write

Claim Tag Set. Indicates whether Claim Tag bit m is implemented, and is used to set Claim Tag bit m to 0b1.

The possible values of this field are:

0b0

Ignored.

0b1

Set Claim Tag bit m to 0b1.

This field is write-one-to-set. Writes of zero are ignored.

D1.2.251 TPIU_CSPSR, TPIU Current Parallel Port Sizes Register

The TPIU_CSPSR characteristics are:

Purpose

Controls the width of the parallel trace port.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

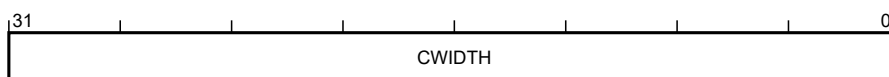
Attributes

32-bit read/write register located at 0xE0040004.

This register is not banked between Security states.

Field descriptions

The TPIU_CSPSR bit assignments are:



CWIDTH, bits [31:0]

Current width. CWIDTH[m] represents a parallel trace port width of (m+1).

The possible values of each bit are:

0

Width (N+1) is not the current parallel trace port width.

1

Width (N+1) is the current parallel trace port width.

A debugger must set only one bit is set to 1, and all others must be zero. The effect of writing a value with more than one bit set to 1 is **UNPREDICTABLE**. The effect of a write to an unsupported bit is **UNPREDICTABLE**.

This register resets to the value for the smallest supported parallel trace port size.

This field resets to an **IMPLEMENTATION DEFINED** value on a Cold reset.

D1.2.252 TPIU_DEVID, TPIU Device Identifier Register

The TPIU_DEVID characteristics are:

Purpose

Describes the TPIU to a debugger.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

Attributes

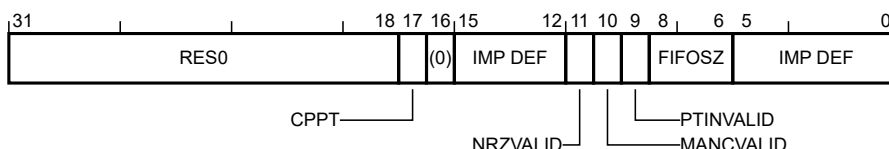
32-bit read-only register located at 0xE0040FC8.

This register is not banked between Security states.

In earlier versions of the M-Profile architecture, this register was named TPIU_TYPE.

Field descriptions

The TPIU_DEVID bit assignments are:



Bits [31:18]

Reserved, **RES0**.

CPPT, bit [17]

Clock Prescaler Parallel Trace. Indicates whether the Parallel trace port prescaler is controlled by TPIU_ACPR.

The possible values of this bit are:

0

Parallel trace port is not affected by the prescaler controlled by TPIU_ACPR.

1

Parallel trace port is affected by the prescaler controlled by TPIU_ACPR.

This bit is **RES0** if TPIU_DEVID.PTINVALID == 1.

This bit reads as an **IMPLEMENTATION DEFINED** value.

Bit [16]

Reserved, **RES0**.

Bits [15:12]

IMPLEMENTATION DEFINED.

NRZVALID, bit [11]

NRZ valid. Indicates support for SWO using UART/NRZ encoding.

The possible values of this bit are:

0

Not supported.

1

Supported.

This bit reads as an IMPLEMENTATION DEFINED value.

MANCVALID, bit [10]

Manchester valid. Indicates support for SWO using Manchester encoding.

The possible values of this bit are:

0

Not supported.

1

Supported.

This bit reads as an IMPLEMENTATION DEFINED value.

PTINVALID, bit [9]

Parallel Trace Interface invalid. Indicates support for parallel trace port operation.

The possible values of this bit are:

0

Supported.

1

Not supported.

This bit reads as an IMPLEMENTATION DEFINED value.

FIFOSZ, bits [8:6]

FIFO depth. Indicates the minimum implemented size of the TPIU output FIFO for trace data.

The possible values of this field are:

0

IMPLEMENTATION DEFINED FIFO depth.

Other

Minimum FIFO size is 2^{FIFOSZ} .

For example, a value of 0b011 indicates a FIFO size of at least $2^3 = 8$ bytes.

This field reads as an IMPLEMENTATION DEFINED value.

Bits [5:0]

IMPLEMENTATION DEFINED.

D1.2.253 TPIU_DEVTTYPE, TPIU Device Type Register

The TPIU_DEVTYPE characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

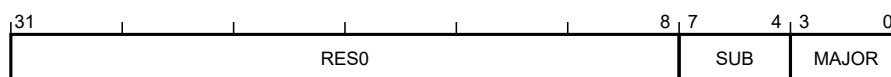
Attributes

32-bit read-only register located at 0xE0040FCC.

This register is not banked between Security states.

Field descriptions

The TPIU_DEVTYPE bit assignments are:

**Bits [31:8]**

Reserved, RES0.

SUB, bits [7:4]

Sub-type. Component sub-type.

The possible values of this field are:

0x0

Other. Only permitted if the MAJOR field reads as 0x0.

0x1

Trace port. Only permitted if the MAJOR field reads as 0x1.

This field reads as an IMPLEMENTATION DEFINED value.

MAJOR, bits [3:0]

Major type. Component major type.

The possible values of this field are:

0x0

Miscellaneous.

0x1

Trace sink.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.254 TPIU_FFCR, TPIU Formatter and Flush Control Register

The TPIU_FFCR characteristics are:

Purpose

Controls the TPIU formatter. This register might contain other formatter and flush control fields that are outside the scope of the architecture. Contact Arm for more information.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

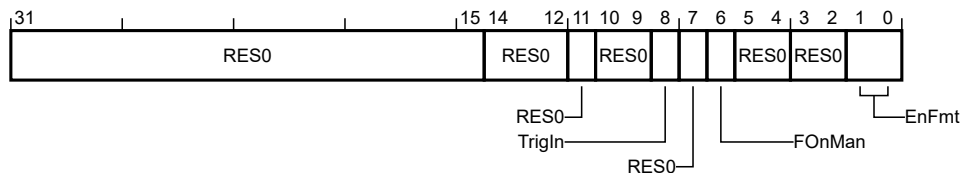
Attributes

32-bit read/write register located at 0xE0040304.

This register is not banked between Security states.

Field descriptions

The TPIU_FFCR bit assignments are:



Bits [31:15,11,7,3:2]

Reserved, **RES0**.

Bits [14:12]

Reserved for formatter stop controls.

Reserved, **RES0**.

Bits [10:9]

Reserved for additional trigger mark controls.

Reserved, **RES0**.

TrigIn, bit [8]

Trigger input asserted. Indicate a trigger on the trace port when an **IMPLEMENTATION DEFINED** TRIGIN signal is asserted.

It is **IMPLEMENTATION DEFINED** whether this bit is R/W or RAO.

This bit resets to zero on a Cold reset.

FOnMan, bit [6]

Flush On Manual. Setting this bit to 1 generates a flush. The TPIU clears the bit to 0 when the flush completes.

This bit resets to zero on a Cold reset.

Bits [5:4]

Reserved for additional flush controls.

Reserved, [RES0](#).

EnFmt, bits [1:0]

Formatter control. Selects the output formatting mode.

The possible values of this field are:

0b00

Bypass. Disable formatting. Only supported when SWO mode is selected. Only a single trace source is supported in bypass mode:

- If only a single trace source is connected to this TPIU, it is selected.
- If multiple sources (including the ITM) are implemented and connected to this TPIU, then all other trace sources, except for the ITM, must be disabled. Otherwise, the trace output is [UNPREDICTABLE](#).

All other trace sources are discarded.

0b10

Continuous. Enable formatting and embed triggers and null cycles in the formatted output.

All other values are reserved.

If no formatter is implemented, this field is [RES0](#). This field must be set to 0b10 when the parallel trace port is selected, or when using multiple trace sources. Changing the value of this field when TPIU_FFSR.FtStopped is 0 is [UNPREDICTABLE](#).

This field resets to zero on a Cold reset.

Note

An optional TRACECTL pin might be implemented as part of the parallel trace port that allows Bypass mode when using a parallel trace port and a further mode, EnFmt == 0b01. The CoreSight architecture describes EnFmt[1] as the EnFCont bit and EnFmt[0] as the EnFTC bit.

D1.2.255 TPIU_FFSR, TPIU Formatter and Flush Status Register

The TPIU_FFSR characteristics are:

Purpose

Shows the status and capabilities of the TPIU formatter.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

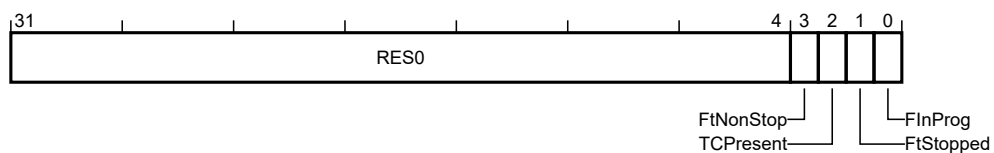
Attributes

32-bit read-only register located at 0xE0040300.

This register is not banked between Security states.

Field descriptions

The TPIU_FFSR bit assignments are:

**Bits [31:4]**

Reserved, RES0.

FtNonStop, bit [3]

Non-stop formatter. Indicates the formatter cannot be stopped.

The possible values of this bit are:

0

Formatter can be stopped.

1

Formatter cannot be stopped.

If no formatter is implemented, this bit is RAO.

TCPresent, bit [2]

TRACECTL present. Indicates presence of the TRACECTL pin.

The possible values of this bit are:

O

No TRACECTL pin is available. The data formatter must be used and only in continuous mode.

1

The optional TRACECTL pin is present.

If a parallel trace port is not implemented, this bit is RAZ.

Note

If a parallel trace port is implemented, Arm recommends the TRACECTL pin is not implemented.

FtStopped, bit [1]

Formatter stopped. Indicates the formatter is stopped.

The possible values of this bit are:

0

Formatter is enabled.

1

The formatter has received a stop request signal and all trace data and post-amble has been output. Any further trace data is ignored.

If no formatter is implemented, or the formatter cannot be stopped, this bit is RAZ.

FInProg, bit [0]

Flush in progress. Set to 1 when a flush is initiated and clears to zero when all data received before the flush is acknowledged has been output on the trace port. That is, the trace has been received at the sink, formatted, and output on the trace port.

D1.2.256 TPIU_LAR, TPIU Software Lock Access Register

The TPIU_LAR characteristics are:

Purpose

Provides CoreSight Software Lock control for the TPIU, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank). It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

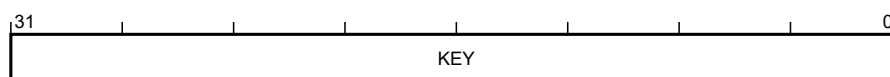
Attributes

32-bit write-only register located at 0xE0040FB0.

This register is not banked between Security states.

Field descriptions

The TPIU_LAR bit assignments are:



KEY, bits [31:0]

Lock Access control.

Writing the key value 0xC5ACCE55 to this field unlocks the lock, enabling write accesses to the registers of this component through a memory-mapped interface.

Writing any other value to this register locks the lock, disabling write accesses to the registers of this component through a memory mapped interface.

D1.2.257 TPIU_LSR, TPIU Software Lock Status Register

The TPIU_LSR characteristics are:

Purpose

Provides CoreSight Software Lock status information for the TPIU, see the *Arm® CoreSight™ Architecture Specification* for details.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

This register is RAZ/WI if accessed via the debugger, regardless of DAUTHCTRL.UIDAPEN (either bank). It is **IMPLEMENTATION DEFINED** whether a DAP error is generated.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

Present only if the optional Software Lock is implemented.

This register is RAZ/WI if the Software Lock is not implemented.

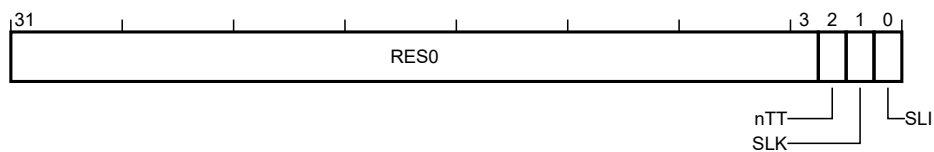
Attributes

32-bit read-only register located at 0xE0040FB4.

This register is not banked between Security states.

Field descriptions

The TPIU_LSR bit assignments are:



Bits [31:3]

Reserved, **RES0**.

nTT, bit [2]

Not thirty-two bit. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as zero.

SLK, bit [1]

Software Lock status. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

0

Lock clear. Software writes are permitted to the registers of this component.

1

Lock set. Software writes to the registers of this component are ignored, and reads have no side effects.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is [RES0](#).

This bit resets to one on a Cold reset.

SLI, bit [0]

Software Lock implemented. See the *Arm® CoreSight™ Architecture Specification*.

The possible values of this bit are:

0

Software Lock not implemented or debugger access.

1

Software Lock is implemented and software access.

For a debugger read of this register, or when the Software Lock is not implemented, this bit is RAZ.

This bit reads as an [IMPLEMENTATION DEFINED](#) value.

D1.2.258 TPIU_PIDR0, TPIU Peripheral Identification Register 0

The TPIU_PIDR0 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

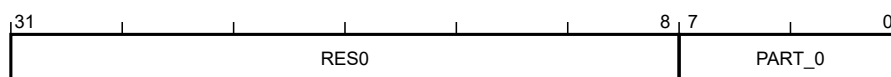
Attributes

32-bit read-only register located at 0xE0040FE0.

This register is not banked between Security states.

Field descriptions

The TPIU_PIDR0 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

PART_0, bits [7:0]

Part number bits [7:0]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.259 TPIU_PIDR1, TPIU Peripheral Identification Register 1

The TPIU_PIDR1 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

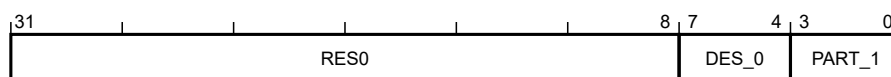
Attributes

32-bit read-only register located at 0xE0040FE4.

This register is not banked between Security states.

Field descriptions

The TPIU_PIDR1 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

DES_0, bits [7:4]

JEP106 identification code bits [3:0]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an IMPLEMENTATION DEFINED value.

PART_1, bits [3:0]

Part number bits [11:8]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an IMPLEMENTATION DEFINED value.

D1.2.260 TPIU_PIDR2, TPIU Peripheral Identification Register 2

The TPIU_PIDR2 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

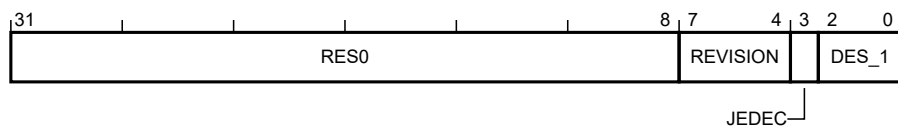
Attributes

32-bit read-only register located at 0xE0040FE8.

This register is not banked between Security states.

Field descriptions

The TPIU_PIDR2 bit assignments are:



Bits [31:8]

Reserved, **RES0**.

REVISION, bits [7:4]

Component revision. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

JEDEC, bit [3]

JEDEC assignee value is used. See the *Arm® CoreSight™ Architecture Specification*.

This bit reads as one.

DES_1, bits [2:0]

JEP106 identification code bits [6:4]. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.261 TPIU_PIDR3, TPIU Peripheral Identification Register 3

The TPIU_PIDR3 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

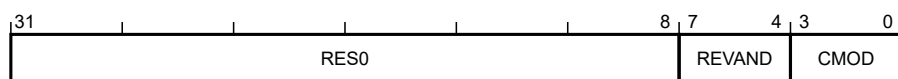
Attributes

32-bit read-only register located at 0xE0040FEC.

This register is not banked between Security states.

Field descriptions

The TPIU_PIDR3 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

REVAND, bits [7:4]

RevAnd. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

CMOD, bits [3:0]

Customer Modified. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an IMPLEMENTATION DEFINED value.

D1.2.262 TPIU_PIDR4, TPIU Peripheral Identification Register 4

The TPIU_PIDR4 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

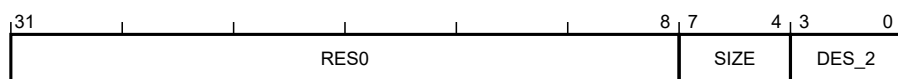
Attributes

32-bit read-only register located at 0xE0040FD0.

This register is not banked between Security states.

Field descriptions

The TPIU_PIDR4 bit assignments are:

**Bits [31:8]**

Reserved, RES0.

SIZE, bits [7:4]

4KB count. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as zero.

DES_2, bits [3:0]

JEP106 continuation code. See the *Arm® CoreSight™ Architecture Specification*.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.263 TPIU_PIDR5, TPIU Peripheral Identification Register 5

The TPIU_PIDR5 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

Attributes

32-bit read-only register located at 0xE0040FD4.

This register is not banked between Security states.

Field descriptions

The TPIU_PIDR5 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.264 TPIU_PIDR6, TPIU Peripheral Identification Register 6

The TPIU_PIDR6 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

Attributes

32-bit read-only register located at 0xE0040FD8.

This register is not banked between Security states.

Field descriptions

The TPIU_PIDR6 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.265 TPIU_PIDR7, TPIU Peripheral Identification Register 7

The TPIU_PIDR7 characteristics are:

Purpose

Provides CoreSight Unique Component Identifier information for the TPIU.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if CoreSight identification is implemented.

This register is **RES0** if CoreSight identification is not implemented.

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

Attributes

32-bit read-only register located at 0xE0040FDC.

This register is not banked between Security states.

Field descriptions

The TPIU_PIDR7 bit assignments are:



Bits [31:0]

Reserved, **RES0**.

D1.2.266 TPIU_PSCR, TPIU Periodic Synchronization Control Register

The TPIU_PSCR characteristics are:

Purpose

Defines the reload value for the Periodic Synchronization Counter register. The Periodic Synchronization Counter decrements for each byte that is output by the TPIU. If the formatter is implemented and enabled, the TPIU forces completion of the current frame when the counter reaches zero. It is **IMPLEMENTATION DEFINED** whether the TPIU forces all trace sources to generate synchronization packets when the counter reaches zero. Bytes generated by the TPIU as part of a Halfword synchronization packet or a Full frame synchronization packet are not counted.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present if the TPIU is implemented and DWT_CYCCNT is not implemented.

OPTIONAL if both the TPIU and DWT_CYCCNT are implemented.

This register is **RES0** if the TPIU is not implemented.

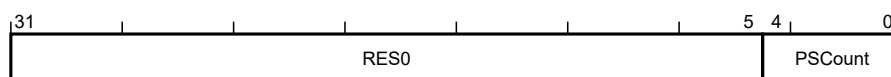
Attributes

32-bit read/write register located at 0xE0040308.

This register is not banked between Security states.

Field descriptions

The TPIU_PSCR bit assignments are:

**Bits [31:5]**Reserved, **RES0**.**PSCount, bits [4:0]**

Periodic Synchronization Count. Determines the reload value of the Periodic Synchronization Counter. The reload value takes effect the next time the counter reaches zero. Reads from this register return the reload value programmed into this register.

The possible values of this field are:

0b00000

Synchronization disabled.

0b00111

128 bytes.

0b01000

256 bytes.

...

...

0b11111

2^{31} bytes.

All other values are reserved.

The Periodic Synchronization Counter might have a maximum value smaller than 2^{31} . In this case, if the programmed reload value is greater than the maximum value, then the Periodic Synchronization Counter is reloaded with its maximum value and the TPIU will generate synchronization requests at this interval.

This field resets to 0xA on a Cold reset.

Note

In the CoreSight TPIU, TPIU_PSCR specifies the number of frames between synchronizations, each frame being 16 bytes. This definition of TPIU_PSCR specifies a number of bytes and is encoded as a power-of-two rather than a plain binary number.

D1.2.267 TPIU_SPPR, TPIU Selected Pin Protocol Register

The TPIU_SPPR characteristics are:

Purpose

Selects the protocol used for trace output.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are [UNPREDICTABLE](#).

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is [IMPLEMENTATION DEFINED](#) whether this register is accessible only to the debugger and [RES0](#) for software. Otherwise the register is accessible to the debugger and software.

If a debugger changes the register value while the TPIU is transmitting data, the effect on the output stream is [UNPREDICTABLE](#) and the required recovery process is [IMPLEMENTATION DEFINED](#).

Configurations

Present only if the TPIU is implemented and supports SWO.

This register is [RES0](#) if the TPIU is not implemented or does not support SWO.

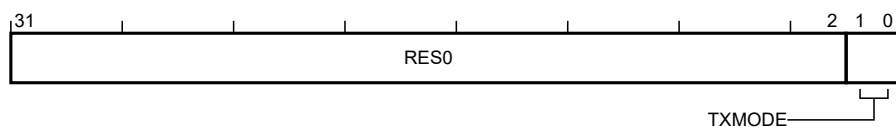
Attributes

32-bit read/write register located at 0xE00400F0.

This register is not banked between Security states.

Field descriptions

The TPIU_SPPR bit assignments are:



Bits [31:2]

Reserved, [RES0](#).

TXMODE, bits [1:0]

Transmit mode. Specifies the protocol for trace output from the TPIU.

The possible values of this field are:

0b00

Parallel trace port mode. This value is reserved if TPIU_DEVID.PTINVALID == 1.

0b01

Asynchronous SWO, using Manchester encoding. This value is reserved if TPIU_DEVID.MANCVALID == 0.

0b10

Asynchronous SWO, using NRZ encoding. This value is reserved if TPIU_DEVID.NRZVALID == 0.

All other values are reserved.

The effect of selecting a reserved value, or a mode that the implementation does not support, is [UNPREDICTABLE](#).

This field resets to an **IMPLEMENTATION DEFINED** value on a Cold reset.

D1.2.268 TPIU_SSPSR, TPIU Supported Parallel Port Sizes Register

The TPIU_SSPSR characteristics are:

Purpose

Indicates the supported parallel trace port sizes.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are **UNPREDICTABLE**.

From Armv8.1-M, this register is accessible through unprivileged DAP requests when DAUTHCTRL.UIDAPEN (either bank) is set.

If the Main Extension is not implemented then it is **IMPLEMENTATION DEFINED** whether this register is accessible only to the debugger and **RES0** for software. Otherwise the register is accessible to the debugger and software.

Configurations

Present only if the TPIU is implemented.

This register is **RES0** if the TPIU is not implemented.

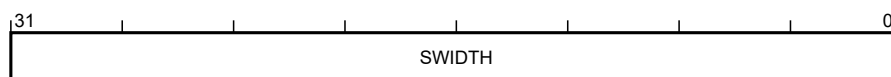
Attributes

32-bit read-only register located at 0xE0040000.

This register is not banked between Security states.

Field descriptions

The TPIU_SSPSR bit assignments are:



SWIDTH, bits [31:0]

Supported width. SWIDTH[*m*] indicates whether a parallel trace port width of (*m*+1) is supported.

The possible values of each bit are:

0

Parallel trace port width (*m*+1) not supported.

1

Parallel trace port width (*m*+1) supported.

The value of this register is **IMPLEMENTATION DEFINED**.

This field reads as an **IMPLEMENTATION DEFINED** value.

D1.2.269 TT_RESP, Test Target Response Payload

The TT_RESP characteristics are:

Purpose

Provides the response information from a TT, TTA, TTT, or TTAT instruction that is not [UNDEFINED](#).

Usage constraints

None.

Configurations

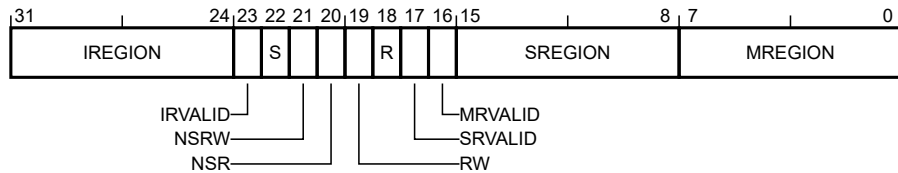
All.

Attributes

32-bit payload.

Field descriptions

The TT_RESP bit assignments are:



IREGION, bits [31:24]

IDAU region number. Indicates the IDAU region number containing the target address.

This field is zero if IRVALID is zero.

IRVALID, bit [23]

IREGION valid flag. For a Secure request, indicates the validity of the IREGION field.

The possible values of this bit are:

0

IREGION content not valid.

1

IREGION content valid.

This bit is always zero if the IDAU cannot provide a region number, the address is exempt from security attribution, or if the requesting TT or TTT variant was executed from the Non-secure state.

S, bit [22]

Security. For a Secure request, indicates the Security attribute of the target address.

The possible values of this bit are:

0

Target address is Non-secure.

1

Target address is Secure.

This bit is always zero if the requesting TT or TTT instruction was executed from the Non-secure state.

NSRW, bit [21]

Non-secure read and writable. Equal to RW AND NOT S. This field is only valid if the variant of the TT group of instructions was executed from Secure state and the RW field is valid.

NSR, bit [20]

Non-secure readable. Equal to R AND NOT S. This field is only valid if the variant of the TT group of instructions was executed from Secure state and the R field is valid.

RW, bit [19]

Read and writable.

Set to 1 if the address specified by the TT instruction variant can be read and written according to the permissions of the selected MPU when operating in the privilege level for the selected mode and selected Security state. For TTT and TTAT, this field returns the permissions for unprivileged access, regardless of whether the selected mode and state is privileged or unprivileged.

This field is invalid and RAZ if the TT instruction was executed from an unprivileged mode and the A flag was not specified. This field is also RAZ if the address matches multiple MPU regions.

R, bit [18]

Readable.

Read accessibility. Set to 1 if the address specified by the TT instruction variant can be read according to the permissions of the selected MPU when operating in the privilege level for the selected mode and selected Security state. For TTT and TTAT, this field returns the permissions for unprivileged access, regardless of whether the selected mode and state is privileged or unprivileged.

This field is invalid and RAZ if the TT instruction was executed from an unprivileged mode and the A flag was not specified. This field is also RAZ if the address matches multiple MPU regions.

SRVALID, bit [17]

SREGION valid flag. For a Secure request indicates validity of the SREGION field.

The possible values of this bit are:

0

SREGION content not valid.

1

SREGION content valid.

The SREGION field is invalid if any of the following are true:

- SAU_CTRL.ENABLE is set to zero.
- The address specified by the TT instruction variant field does not match any enabled SAU regions.
- The address specified matches multiple enabled SAU regions.
- The address specified by the TT instruction variant is exempt from the Secure memory attribution.
- The TT or TTT instruction variant was executed from the Non-secure state or the Security Extension is not implemented.

The TTA and TTAT instruction variants are **UNDEFINED** when executed from Non-secure state.

MRVALID, bit [16]

MREGION valid flag. Indicates validity of the MREGION field.

The possible values of this bit are:

0

MREGION content not valid.

1

MREGION content valid.

The MREGION field is invalid if any of the following is true:

- The MPU is not implemented or MPU_CTRL.ENABLE is set to zero.

- The address specified by the TT instruction variant does not match any enabled MPU regions.
- The address matched multiple MPU regions.
- The TT or TTT instruction variants, without the A flag specified, were executed from an unprivileged mode.

The TTA and TTAT instructions are **UNDEFINED** when executed from Non-secure state.

SREGION, bits [15:8]

SAU region number. Holds the SAU region that the address maps to.

This field is only valid if the instruction was executed from Secure state. This field is zero if SRVALID is 0.

MREGION, bits [7:0]

MPU region number. Holds the MPU region that the address maps to.

This field is zero if MRVALID is 0.

D1.2.270 UFSR, UsageFault Status Register

The UFSR characteristics are:

Purpose

Contains the status for some instruction execution faults, and for data access faults.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

If the Main Extension is implemented, this register is word, halfword, and byte accessible.

If the Main Extension is not implemented, this register is word accessible only, halfword and byte accesses are [UNPREDICTABLE](#).

Configurations

Present only if the Main Extension is implemented.

This register is [RES0](#) if the Main Extension is not implemented.

Attributes

16-bit read/write-one-to-clear register located at 0xE000ED2A.

Secure software can access the Non-secure version of this register via UFSR_NS located at 0xE002ED2A. The location 0xE002ED2A is [RES0](#) to software executing in Non-secure state and the debugger.

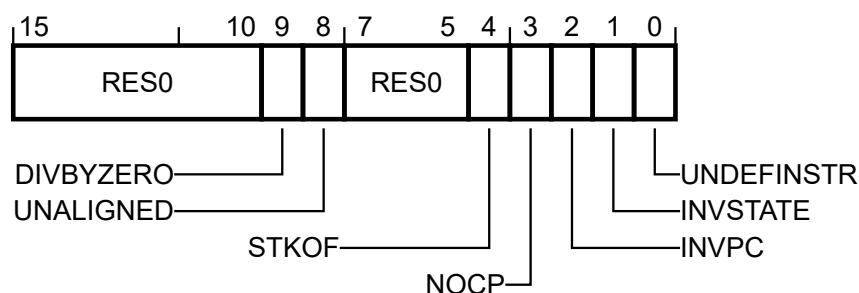
This register is banked between Security states.

This register is part of CFSR.

From Armv8.1-M onwards it is [IMPLEMENTATION DEFINED](#) whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The UFSR bit assignments are:



Bits [15:10]

Reserved, [RES0](#).

DIVBYZERO, bit [9]

Divide by zero flag. Sticky flag indicating whether an integer division by zero error has occurred.

The possible values of this bit are:

0

Error has not occurred.

1

Error has occurred.

This bit resets to zero on a Warm reset.

UNALIGNED, bit [8]

Unaligned access flag. Sticky flag indicating whether an unaligned access error has occurred.

The possible values of this bit are:

0
Error has not occurred.

1
Error has occurred.

This bit resets to zero on a Warm reset.

Bits [7:5]

Reserved, [RES0](#).

STKOF, bit [4]

Stack overflow flag. Sticky flag indicating whether a stack overflow error has occurred.

The possible values of this bit are:

0
Error has not occurred.

1
Error has occurred.

This bit resets to zero on a Warm reset.

NOCP, bit [3]

No coprocessor flag. Sticky flag indicating whether a coprocessor disabled or not present error has occurred.

The possible values of this bit are:

0
Error has not occurred.

1
Error has occurred.

This bit resets to zero on a Warm reset.

INVPC, bit [2]

Invalid PC flag. Sticky flag indicating whether an integrity check error has occurred.

The possible values of this bit are:

0
Error has not occurred.

1
Error has occurred.

This bit resets to zero on a Warm reset.

INVSTATE, bit [1]

Invalid state flag. Sticky flag indicating whether an EPSR.B, EPSR.T, EPSR.IT, or FPSCR.LTPSIZE validity error has occurred.

The possible values of this bit are:

0
Error has not occurred.

1
Error has occurred.

This bit resets to zero on a Warm reset.

UNDEFINSTR, bit [0]

UNDEFINED instruction flag. Sticky flag indicating whether an **UNDEFINED** instruction error has occurred.

The possible values of this bit are:

0

Error has not occurred.

1

Error has occurred.

This includes attempting to execute an **UNDEFINED** instruction associated with an enable coprocessor.

This bit resets to zero on a Warm reset.

D1.2.271 VPR, Vector Predication Status and Control Register

The VPR characteristics are:

Purpose

Holds the per element predication flags.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

Present only if version Armv8.1 of the architecture and MVE are implemented.

This register is [RES0](#) if MVE is not implemented.

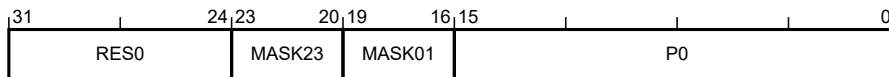
Attributes

32-bit read/write special-purpose register.

This register is not banked between Security states.

Field descriptions

The VPR bit assignments are:



Bits [31:24]

Reserved, [RES0](#).

MASK23, bits [23:20]

The VPT mask bits for beat 2 and 3.

The possible values of this field are:

0b0000

Not in a VPT block.

0b1000

In a VPT block which is valid for one more instruction. The predicate flags are not inverted.

0bx100

In a VPT block which is valid for two more instructions. If set, the x bit causes the predicate flags for beat 2 and 3 to be inverted between the corresponding instructions in the VPT block.

0bxxx10

In a VPT block which is valid for three more instructions. If set, the x bits cause the predicate flags for beat 2 and 3 to be inverted between the corresponding instructions in the VPT block.

0bxxxx1

In a VPT block which is valid for four more instructions. If set, the x bits cause the predicate flags for beat 2 and 3 to be inverted between the corresponding instructions in the VPT block.

If the PE executes a single beat per architecture tick, this field and the associated predicate flags are updated after beat 3 completes.

This field resets to an [UNKNOWN](#) value on a Warm reset.

MASK01, bits [19:16]

The VPT mask bits for beat 0 and 1.

The possible values of this field are:

0b0000

Not in a VPT block.

0b1000

VPT predication valid for one more instruction. The predicate flags are not inverted.

0bx100

In a VPT block which is valid for two more instructions. If set, the x bit causes the predicate flags for beat 0 and 1 to be inverted between the corresponding instructions in the VPT block.

0bxx10

In a VPT block which is valid for three more instructions. If set, the x bits cause the predicate flags for beat 0 and 1 to be inverted between the corresponding instructions in the VPT block.

0bxxx1

In a VPT block which is valid for four more instructions. If set, the x bits cause the predicate flags for beat 0 and 1 to be inverted between the corresponding instructions in the VPT block.

If the PE executes a single beat per architecture tick, this field and the associated predicate flags are updated after beat 1 completes.

This field resets to an **UNKNOWN** value on a Warm reset.

P0, bits [15:0]

Predication bits. Each group of 4 bits determines the predication of each of the 4 bytes within the corresponding beat, regardless of instruction data type. See the relevant instruction descriptions and pseudocode for information on how the predication affects execution.

The possible values of this field are:

0

The corresponding vector lane will be masked.

1

The corresponding vector lane will be active.

Unprivileged access to this field is permitted, see VMRS and VMSR instructions.

This field resets to an **UNKNOWN** value on a Warm reset.

D1.2.272 VTOR, Vector Table Offset Register

The VTOR characteristics are:

Purpose

Holds the vector table address for the selected Security state.

Usage constraints

Privileged access permitted only. Unprivileged accesses generate a fault.

This register is word accessible only. Halfword and byte accesses are UNPREDICTABLE.

Configurations

This register is always implemented.

Attributes

32-bit read/write register located at 0xE000ED08.

To allow lock down of this register it is **IMPLEMENTATION DEFINED** whether this register is writable.

Secure software can access the Non-secure version of this register via VTOR_NS located at 0xE002ED08. The location 0xE002ED08 is **RES0** to software executing in Non-secure state and the debugger.

This register is banked between Security states.

From Armv8.1-M onwards it is **IMPLEMENTATION DEFINED** whether a debugger write to this register is ignored when the PE is not in Debug state.

Field descriptions

The VTOR bit assignments are:

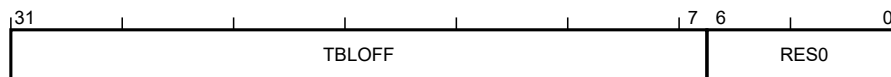
**TBLOFF, bits [31:7]**

Table offset. Bits [31:7] of the vector table address for the selected Security state.

It is **IMPLEMENTATION DEFINED** whether any of the TBLOFF bits are WI

This field resets to an **IMPLEMENTATION DEFINED** value on a Warm reset.

Bits [6:0]

Reserved, RES0.

D1.2.273 XPSR, Combined Program Status Registers

The XPSR characteristics are:

Purpose

Provides access to a combination of the APSR, EPSR and IPSR.

Usage constraints

Privileged access only. Unprivileged access is RAZ/WI, unless otherwise stated.

Configurations

This register is always implemented.

Attributes

32-bit read/write special-purpose register.

This register is not banked between Security states.

Field descriptions

The XPSR bit assignments are:

When {XPSR[26:25], XPSR[11:10]} != 0:

31	30	29	28	27	26	25	24	23	22	21	20	19	16	15	10	9	8	0
N	Z	C	V	Q	IT	T	RES0	B	(0)	GE	IT	(0)	Exception					

When {XPSR[26:25], XPSR[11:10]} == 0, and a multi-cycle load or store instruction is in progress:

31	30	29	28	27	26	25	24	23	22	21	20	19	16	15	10	9	8	0
N	Z	C	V	Q	ICI	T	RES0	B	(0)	GE	ICI	(0)	Exception					

When {XPSR[26:25], XPSR[11:10]} == 0, and more than one beat-wise vector instruction is in progress:

31	30	29	28	27	26	25	24	23	22	21	20	19	16	15	12	11	10	9	8	0
N	Z	C	V	Q	ECI	T	RES0	B	(0)	GE	ECI	ECI	(0)	Exception						

N, bit [31]

Negative flag. Reads or writes the current value of APSR.N.

Z, bit [30]

Zero flag. Reads or writes the current value of APSR.Z.

C, bit [29]

Carry flag. Reads or writes the current value of APSR.C.

V, bit [28]

Overflow flag. Reads or writes the current value of APSR.V.

Q, bit [27]

Saturate flag. Reads or writes the current value of APSR.Q.

T, bit [24]

T32 state. Reads or writes the current value of EPSR.T.

Bits [23:22]

Reserved, [RES0](#).

B, bit [21]

Branch target identification active. Reads or writes the current value of EPSR.B.

Bit [20]

Reserved, [RES0](#).

GE, bits [19:16]

Greater-than or equal flag. Reads or writes the current value of APSR.GE.

IT, bits [15:10,26:25] , when [{XPSR[26:25], XPSR[11:10]} != 0]

If-then flags. Reads or writes the current value of EPSR.IT.

ICI, bits [26:25,15:10] , when [{XPSR[26:25], XPSR[11:10]} == 0, and a multi-cycle load or store instruction is in progress]

Interrupt continuation flags. Reads or writes the current value of EPSR.ICI.

ECI, bits [26:25, 11:10, 15:12] , when [{XPSR[26:25], XPSR[11:10]} == 0, and more than one beat-wise vector instruction is in progress]

Exception continuation flags for beat-wise vector instructions. Reads or writes the current value of EPSR.ECI.

Bit [9]

Reserved, [RES0](#).

Exception, bits [8:0]

Exception number. Reads or writes the current value of IPSR.Exception.

Part E

Armv8-M Pseudocode

Chapter E1

Arm Pseudocode Definition

This chapter provides a definition of the pseudocode that this manual uses, and defines some *built-in* functions that the pseudocode uses. It contains the following sections:

[E1.1 *About the Arm pseudocode.*](#)

[E1.2 *Data types.*](#)

[E1.3 *Operators.*](#)

[E1.4 *Statements and control structures.*](#)

[E1.5 *Built in functions.*](#)

[E1.6 *Arm pseudocode definition index.*](#)

[E1.7 *Additional functions.*](#)

Note

This chapter is not a formal language definition for the pseudocode. It is a guide to help understand the use of Arm pseudocode.

E1.1 About the Arm pseudocode

The Arm pseudocode provides precise descriptions of some areas of the Arm architecture. This includes description of the decoding and operation of all valid instructions.

The following sections describe the Arm pseudocode in detail:

[E1.2 Data types.](#)

[E1.3 Operators.](#)

[E1.4 Statements and control structures.](#)

[E1.5 Built in functions](#) describes some built-in functions that the pseudocode functions use that this manual describes elsewhere.

[E1.6 Arm pseudocode definition index](#) contains the indexes to the pseudocode.

E1.1.1 General limitations of Arm pseudocode

Because of the limitations inherent in all pseudocode, the Arm pseudocode and pseudocode comments describe only one particular implementation of the architecture. There are several instances where a rule relaxes the behavior that a particular piece of pseudocode describes.

The pseudocode statements `EndOfInstruction()`, `SEE`, `UNDEFINED`, `CONSTRAINED_UNPREDICTABLE`, and `UNPREDICTABLE` indicate behavior that differs from that indicated by the pseudocode being executed. If one of the statements is encountered:

- `CONSTRAINED_UNPREDICTABLE`, and `UNPREDICTABLE` mean earlier behavior indicated by the pseudocode is only specified as occurring to the extent required to determine that the statement is executed. No subsequent behavior that the pseudocode indicates occurs.
- `EndOfInstruction()`, `SEE`, and `UNDEFINED` mean that the pseudocode will terminate execution of the current instruction and pseudocode execution continues from the exception catch.

For more information, see [E1.4.5 Special statements](#).

E1.2 Data types

This section describes:

[E1.2.1 General data type rules.](#)

[E1.2.2 Bitstrings.](#)

[E1.2.3 Integers.](#)

[E1.2.4 Reals.](#)

[E1.2.5 Booleans.](#)

[E1.2.6 Enumerations.](#)

[E1.2.7 Structures.](#)

[E1.2.8 Tuples.](#)

[E1.2.9 Arrays.](#)

E1.2.1 General data type rules

Arm architecture pseudocode is a strongly typed language. Every literal and variable is of one of the following types:

- Bitstring.
- Integer.
- Boolean.
- Real.
- Enumeration.
- Tuple.
- Struct.
- Array.

The syntax of a literal determines its type. A variable can be assigned to without an explicit declaration. The variable implicitly has the type of the assigned value. For example, the following assignments implicitly declare the variables `x`, `y` and `z` to have types integer, bitstring of length 1, and Boolean, respectively.

```
1 x = 1;  
2 y = '1';  
3 z = TRUE;
```

Variables can also have their types declared explicitly by preceding the variable name with the name of the type. The following example declares explicitly that a variable named `count` is an integer.

```
integer count;
```

This is most often done in function definitions for the arguments and the result of the function.

The remaining subsections describe each data type in more detail.

E1.2.2 Bitstrings

This section describes the bitstring data type.

Syntax

bits(N)

The type name of a bitstring of length 'N'.

bit

A synonym of **bits**(1).

Description

A bitstring is a finite-length string of 0s and 1s. Each length of bitstring is a different type. The minimum permitted length of a bitstring is 0.

Bitstring constants literals are written as a single quotation mark, followed by the string of 0s and 1s, followed by another single quotation mark. For example, the two constants literals of type **bit** are '0' and '1'. Spaces can be included in bitstrings for clarity.

The bits in a bitstring are numbered from left to right $N-1$ to 0. This numbering is used when accessing the bitstring using bitslices. In conversions to and from integers, bit $N-1$ is the MSByte and bit 0 is the LSByte. This order matches the order in which bitstrings derived from encoding diagrams are printed.

Every bitstring value has a left-to-right order, with the bits being numbered in standard *little-endian* order. That is, the leftmost bit of a bitstring of length N is bit $(N-1)$ and its right-most bit is bit 0. This order is used as the most-significant-to-least-significant bit order in conversions to and from integers. For bitstring constants and bitstrings that are derived from encoding diagrams, this order matches the way that they are printed.

Bitstrings are the only concrete data type in pseudocode, corresponding directly to the contents values that are manipulated in registers, memory locations, and instructions. All other data types are abstract.

E1.2.3 Integers

This section describes the data type for integer numbers.

Syntax

integer

The type name for the integer data type.

Description

Pseudocode integers are unbounded in size and can be either positive or negative. That is, they are mathematical integers rather than what computer languages and architectures commonly call integers. Computer integers are represented in pseudocode as bitstrings of the appropriate length, and the pseudocode provides functions to interpret those bitstrings as integers.

Integer literals are normally written in decimal form, such as 0, 15, -1234. They can also be written in C-style hexadecimal form, such as 0x55 or 0x80000000. Hexadecimal integer literals are treated as positive unless they have a preceding minus sign. For example, 0x80000000 is the integer $+2^{31}$. If -2^{31} needs to be written in hexadecimal, it must be written as -0x80000000.

E1.2.4 Reals

This section describes the data type for real numbers.

Syntax

real

The type name for the real data type.

Description

Pseudocode reals are unbounded in size and precision. That is, they are mathematical real numbers, not computer floating-point numbers. Computer floating-point numbers are represented in pseudocode as bitstrings of the appropriate length, and the pseudocode provides functions to interpret those bitstrings as reals.

Real constant literals are written in decimal form with a decimal point. This means `0` is an integer constant literal, but `0.0` is a real constant literal.

E1.2.5 Booleans

This section describes the boolean data type.

Syntax

boolean

The type name for the boolean data type.

`TRUE, FALSE`

The two values a boolean variable can take.

Description

A boolean is a logical `TRUE` or `FALSE` value.

Note

This is not the same type as `bit`, which is a bitstring of length 1. A boolean can only take on one of two values: `TRUE` or `FALSE`.

E1.2.6 Enumerations

This section describes the enumeration data type.

Syntax and examples

enumeration

Keyword to define a new enumeration type.

enumeration Example {Example_One, Example_Two, Example_Three};

A definition of a new enumeration that is called `Example`, which can take on the values `Example_One`, `Example_Two`, `Example_Three`.

Description

An enumeration is a defined set of named values.

An enumeration must contain at least one named value. A named value must not be shared between enumerations.

Enumerations must be defined explicitly, although a variable of an enumeration type can be declared implicitly by assigning one of the named values to it. By convention, each named value starts with the name of the enumeration followed by an underscore. The name of the enumeration is its *type name*, or *type*, and the named values are its possible *values*.

E1.2.7 Structures

This section describes the structure data type.

Syntax and examples

type

The keyword that is used to declare the structure data type.

```
type ShiftSpec is (bits(2) shift, integer amount):
```

An example definition for a new structure that is called ‘ShiftSpec’ that contains a bitstring member that is called ‘shift’ and an integer member called ‘amount’. Structure definitions must not be terminated with a semicolon.

```
ShiftSpec abc;
```

A declaration of a variable that is named ‘abc’ of type ‘ShiftSpec’.

```
abc.shift
```

Syntax to refer to the individual members within the structure variable.

Description

A structure is a compound data type composed of one or more data items. The data items can be of different data types. This can include compound data types. The data items of a structure are called its members and are named.

In the syntax section, the example defines a structure that is called `ShiftSpec` with two members. The first is a bitstring of length 2 named `shift` and the second is an integer that is named `amount`. After declaring a variable of that type that is named `abc`, the members of this structure are referred to as `abc.shift` and `abc.amount`.

Every definition of a structure creates a different type, even if the number and type of their members are identical. For example:

```
type ShiftSpec1 is (bits(2) shift, integer amount)
```

```
type ShiftSpec2 is (bits(2) shift, integer amount)
```

`ShiftSpec1` and `ShiftSpec2` are two different types despite having identical definitions. This means that the value in a variable of type `ShiftSpec1` cannot be assigned to variable of type `ShiftSpec2`.

E1.2.7.1 `_Type` and `_Type`

This subsection describes the data structure types for a particular register or payload.

Example

RETPSR_Type

The data structure of type RETPSR.

Description

By convention `_Type` declares a structure data type for a specific register or payload.

See the individual register descriptions for the fields that apply to a particular data structure.

E1.2.8 Tuples

This section describes the tuple data type.

Examples

```
(bits(32) shifter_result, bit shifter_carry_out)
```

An example of the tuple syntax.

```
(shift_t, shift_n) = ('00', 0);
```

An example of assigning values to a tuple.

Description

A tuple is an ordered set of data items, which are separated by commas and enclosed in parentheses. The items can be of different types and a tuple must contain at least one data item.

Tuples are often used as the return type for functions that return multiple results. For example, in the syntax section, the example tuple is the return type of the function `Shift_C()`, which performs a standard A32/T32 shift or rotation. Its return type is a tuple containing two data items, with the first of type `bits(32)`, and `bits(32)` the second of type `bit`.

Each tuple is a separate compound data type. The compound data type is represented as a comma-separated list of ordered data types between parentheses. This means that the example tuple at the start of this section is of type `(bits(32), bit)`. The general principle that types can be implied by an assignment extends to implying the type of the elements in the tuple. For example, in the syntax section, the example assignment implicitly declares:

- `shift_t` to be of type `bits(2)`.
- `shift_n` to be of type `integer`.
- `(shift_t, shift_n)` to be a tuple of type `(bits(2), integer)`.

E1.2.9 Arrays

This section describes the array data type.

Syntax

array

The type name for the array data type.

```
array data_type array_name[A..B];
```

```
array [A..B] of data_type array_name
```

Declaration of an array of type ‘data_type’, which might be compound data type. It is named ‘array_name’ and is indexed with an integer range from ‘A’ to ‘B’.

Description

An array is an ordered set of fixed size containing items of a single data type. This can include compound data types. Pseudocode arrays are indexed by either enumerations or integer ranges. An integer range is represented by the lower inclusive end of the range, then . . , then the upper inclusive end of the range.

For example:

The following example declares an array of 31 bitstrings of length 64, indexed from 0-30.

```
1 array bits(64) _R[0..30];
```

Arrays are always explicitly declared, and there is no notation for a constant literal array. Arrays always contain at least one element data item, because:

- Enumerations always contain at least one symbolic constant named value.
- Integer ranges always contain at least one integer.

An array declared with an enumeration type as the index must be accessed using enumeration values of that enumeration type. An array declared with an integer range type as the index must be accessed using integer values from that inclusive range. Accessing such an array with an integer value outside of the range is a coding error.

Pseudocode can also contain array-like functions such as `R[i]`, `MemU[address, size]`, or `Elem[vector, i, size ↪]`. These functions package up and abstract additional operations that are normally performed on accesses to the underlying arrays, such as register banking, memory protection, endian-dependent byte ordering, exclusive-access housekeeping and Advanced SIMD element processing. See [E1.4.2 Function and procedure calls](#).

E1.3 Operators

This section describes:

[E1.3.1 Relational operators.](#)

[E1.3.2 Boolean operators.](#)

[E1.3.3 Bitstring operators.](#)

[E1.3.4 Arithmetic operators.](#)

[E1.3.5 The assignment operator.](#)

[E1.3.6 Precedence rules.](#)

[E1.3.7 Conditional expressions.](#)

[E1.3.8 Operator polymorphism.](#)

E1.3.1 Relational operators

The following operations yield results of type **boolean**.

Equality and non-equality

If two variables *x* and *y* are of the same type, their values can be tested for equality by using the expression *x* == *y* and for non-equality by using the expression *x* != *y*. In both cases, the result is of type **boolean**.

Both *x* and *y* must be of type **bits(N)**, **real**, **enumeration**, **boolean**, or **integer**. Named values from an **enumeration** can only be compared if they are both from the same **enumeration**. An exception is that a bitstring can be tested for equality with an integer to allow a *d* == 15 test.

A special form of comparison is defined with a bitstring literal that can contain bit values '0', '1', and 'x'. Any bit with value 'x' is ignored in determining the result of the comparison. For example, if *opcode* is a 4-bit bitstring, the expression *opcode* == '1x0x' matches the values 1000, 1100, 1001, and 1101. This is known as a bitmask.

Note

This special form is permitted in the implied equality comparisons in the **when** parts of **case ... of ...** structures.

Comparisons

If *x* and *y* are integers or reals, then *x* < *y*, *x* <= *y*, *x* > *y*, and *x* >= *y* are less than, less than or equal, greater than, and greater than or equal comparisons between them, producing boolean results.

E1.3.1.1 Set membership with IN

<expression> **IN** {<set>} produces **TRUE** if <expression> is a member of <set>. Otherwise, it is **FALSE**. <set> must be a list of expressions that are separated by commas.

E1.3.2 Boolean operators

If *x* is a boolean expression, then !*x* is its logical inverse.

If *x* and *y* are boolean expressions, then *x* &&*y* is the result of ANDing them together. As in the C language, if *x* is **FALSE**, the result is determined to be **FALSE** without evaluating *y*.

Note

This is known as short circuit evaluation.

If x and y are **booleans**, then $x \ || \ y$ is the result of ORing them together. As in the C language, if x is **TRUE**, the result is determined to be **TRUE** without evaluating y .

Note

If x and y are **booleans** or boolean expressions, then the result of $x \ != \ y$ is the same as the result of exclusive-ORing x and y together. The operator **EOR** only accepts bitstring arguments.

E1.3.3 Bitstring operators

The following operations can be applied only to bitstrings.

Logical operations on bitstrings

If x is a bitstring, **NOT**(x) is the bitstring of the same length that is obtained by logically inverting every bit of x .

If x and y are bitstrings of the same length, $x \ \text{AND} \ y$, $x \ \text{OR} \ y$, and $x \ \text{EOR} \ y$ are the bitstrings of that same length that is obtained by logically ANDing, logically ORing, and exclusive-ORing corresponding bits of x and y together.

Bitstring concatenation and slicing

If x and y are bitstrings of lengths N and M respectively, then $x:y$ is the bitstring of length $N + M$ constructed by concatenating x and y in left-to-right order.

The bitstring slicing operator addresses specific bits in a bitstring. This can be used to create a new bitstring from extracted bits or to set the value of specific bits. Its syntax is $x[\text{integer_list}]$, where x is the integer or bitstring being sliced, and $[\text{integer_list}]$ is a comma-separated list of integers that are enclosed in square brackets. The length of the resulting bitstring is equal to the number of integers in $[\text{integer_list}]$. In $x[\text{integer_list}]$, each of the integers in $[\text{integer_list}]$ must be:

- ≥ 0 .
- $< \text{Len}(x)$ if x is a bitstring.

The definition of $x[\text{integer_list}]$ depends on whether integer_list contains more than one integer:

- If integer_list contains more than one integer, $x[i, j, k, \dots, n]$ is defined to be the concatenation:

```
1  x[i]: x[j]: x[k]: ...: x[n]
```

- If integer_list consists of just one integer i , $x[i]$ is defined to be:
 - If x is a bitstring, '0' if bit i of x is a zero and '1' if bit i of x is a one.
 - If x is an integer, and y is the unique integer in the range 0 to $2^{(i+1)} - 1$ that is congruent to x modulo $2^{(i+1)}$. Then $x[i]$ is '0' if $y < 2^i$ and '1' if $y \geq 2^i$.

Loosely, this definition treats an integer as equivalent to a sufficiently long two's complement representation of it as a bitstring.

The notation for a range expression is $i:j$ with $i \geq j$ is shorthand for the integers in order from i down to j , with both end values included. For example, `instr[31:28]` represents `instr[31, 30, 29, 28]`.

$x[\text{integer_list}]$ is assignable provided x is an assignable bitstring and no integer appears more than once in $[\text{integer_list}]$. In particular, $x[i]$ is assignable if x is an assignable bitstring and $0 \leq i < \text{Len}(x)$.

Encoding diagrams for registers frequently show named bits or multi-bit fields. For example, the encoding diagram for the APSR shows its bit[31] as N . In such cases, the syntax `APSR.N` is used as a more readable synonym for `APSR[31]` as named bits can be referred to with the same syntax as referring to members of a struct. A

comma-separated list of named bits enclosed in square brackets following the register name allows multiple bits to be addressed simultaneously.

For example, `APSR.[N, C, Q]` is synonymous with `APSR [31, 29, 27]`.

E1.3.4 Arithmetic operators

Most pseudocode arithmetic is performed on integer or real values, with operands obtained by conversions from bitstrings and results that are converted back to bitstrings. As these data types are the unbounded mathematical types, no issues arise about overflow or similar errors.

Unary plus and minus

If x is an integer or real, then $+x$ is x unchanged, $-x$ is x with its sign reversed. Both are of the same type as x .

Addition and subtraction

If x and y are integers or reals, $x + y$ and $x - y$ are their sum and difference. Both are of type **integer** if x and y are both of type **integer**, and **real** otherwise.

There are two cases where the types of x and y can be different. A bitstring and an integer can be added together to allow the operation `PC + 4`. An integer can be subtracted from a bitstring to allow the operation `PC - 2`.

If x and y are bitstrings of the same length N , so that $N = \text{Len}(x) = \text{Len}(y)$, then $x + y$ and $x - y$ are the least significant N bits of the results of converting x and y to integers and adding or subtracting them. Signed and unsigned conversions produce the same result:

```
1 x + y = (SInt(x) + SInt(y)) [N-1:0]
2       = (UInt(x) + UInt(y)) [N-1:0]
3 x - y = (SInt(x) - SInt(y)) [N-1:0]
4       = (UInt(x) - UInt(y)) [N-1:0]
```

If x is a bitstring of length N and y is an integer, $x + y$ and $x - y$ are the bitstrings of length N defined by $x + y = \rightarrow x + y[N-1:0]$ and $x - y = x - y[N-1:0]$. Similarly, if x is an integer and y is a bitstring of length M , $x + y$ and $x - y$ are the bitstrings of length M defined by $x + y = x[M-1:0] + y$ and $x - y = x[M-1:0] - y$.

Multiplication

If x and y are integers or reals, then $x * y$ is the product of x and y . It is of type **integer** if x and y are both of type **integer**, and **real** otherwise.

Division and modulo

If x and y are reals, then x/y is the result of dividing x by y , and is always of type **real**.

If x and y are integers, then $x \text{ DIV } y$ and $x \text{ MOD } y$ are defined by:

```
1 x DIV y = RoundDown(x/y)
2 x MOD y = x - y * (x DIV y)
```

It is a pseudocode error to use any of x/y , $x \text{ MOD } y$, or $x \text{ DIV } y$ in any context where y can be zero.

Scaling

If x and n are of type **integer**, then:

- $x \ll n = \text{RoundDown}(x * 2^n)$.
- $x \gg n = \text{RoundDown}(x * 2^{(-n)})$.

Raising to a power

If x is an integer or a real and n is an integer, then x^n is the result of raising x to the power of n , and:

- If x is of type **integer** then x^n is of type **integer**.
- If x is of type **real** then x^n is of type **real**.

E1.3.5 The assignment operator

The assignment operator is the `=` character, which assigns the value of the right-hand side to the left-hand side. An assignment statement takes the form:

```
<assignable_expression> = <expression>;
```

This following subsection defines valid expression syntax.

General expression syntax

An expression is one of the following:

- A literal.
- A variable, optionally preceded by a data type name to declare its type.
- The word **UNKNOWN** preceded by a data type name to declare its type.
- The result of applying a language-defined operator to other expressions.
- The result of applying a function to other expressions.

Variable names normally consist of alphanumeric and underscore characters, starting with an alphabetic or underscore character.

Each register that is defined in an Arm architecture specification defines a correspondingly named pseudocode bitstring variable, and that variable has the stated behavior of the register. For example, if a bit of a register is defined as RAZ/WI, then the corresponding bit of its variable reads as '0' and ignore writes.

An expression like `bits(32) UNKNOWN` indicates that the result of the expression is a value of the given type, but the architecture does not specify what value it is and software must not rely on such values. The value produced must not:

- Return information that cannot be accessed at the current or a lower level of privilege using instructions that are not UNPREDICTABLE or CONSTRAINED UNPREDICTABLE and do not return UNKNOWN values,
- Be promoted as providing any useful information to software.

Note

UNKNOWN values are similar to the definition of UNPREDICTABLE, but do not indicate that the entire architectural state becomes unspecified.

Only the following expressions are assignable. This means that these are the only expressions that can be placed on the left-hand side of an assignment:

- Variables.
- The results of applying some operators to other expressions.

The description of each language-defined operator that can generate an assignable expression specifies the circumstances under which it does so. For example, those circumstances might require that one or more of the expressions the operator operates on is an assignable expression.

- The results of applying array-like functions to other expressions. The description of an array-like function specifies the circumstances under which it can generate an assignable expression.

Note

If the right-hand side in an assignment is a function returning a tuple, an item in the assignment destination can be written as `-` to indicate that the corresponding item of the assigned tuple value is discarded. For example:

```
(shifted, -) = LSL_C(operand, amount);
```

The expression on the right-hand side itself can be a tuple. For example:

```
(x, y) = (function_1(), function_2());
```

Every expression has a data type.

- For a literal, this data type is determined by the syntax of the literal.
- For a variable, there are the following possible sources for the data type
 - An optional preceding data type name.
 - A data type the variable was given earlier in the pseudocode by recursive application of this rule.
 - A data type the variable is being given by assignment, either by direct assignment to the variable, or by assignment to a list of which the variable is a member.

It is a pseudocode error if none of these data type sources exists for a variable, or if more than one of them exists and they do not agree about the type.

- For a language-defined operator, the definition of the operator determines the data type.
- For a function, the definition of the function determines the data type.

E1.3.6 Precedence rules

The precedence rules for expressions are:

1. Literals, variables, and function invocations are evaluated with higher priority than any operators using their results, but see [E1.3.2 Boolean operators](#).
2. Operators on integers follow the normal operator precedence rules of *exponentiation before multiply/divide before add/subtract*, with sequences of multiply/divides or add/subtracts evaluated left-to-right.
3. Other expressions must be parenthesized to indicate operator precedence if ambiguity is possible, but do not need to be if all permitted precedence orders under the type rules necessarily lead to the same result. For example, if *i*, *j* and *k* are integer variables, *i* > 0 && *j* > 0 && *k* > 0 is acceptable, but *i* > 0 && *j* > 0 || *k* > 0 is not.

E1.3.7 Conditional expressions

If *x* and *y* are two values of the same type and *t* is a value of type **boolean**, then **if** *t* **then** *x* **else** *y* is an expression of the same type as *x* and *y* that produces *x* if *t* is **TRUE** and *y* if *t* is **FALSE**.

E1.3.8 Operator polymorphism

Operators in pseudocode can be polymorphic, with different functionality when applied to different data types. Each resulting form of an operator has a different prototype definition. For example, the operator `+` has forms that act on various combinations of integers, reals and bitstrings.

[Table E1-1](#) summarizes the operand types valid for each unary operator and the result type. [Table E1-2](#) summarizes the operand types valid for each binary operator and the result type.

Table E1-1, Result and operand types that are permitted for unary operators.

Operator	Operand Type	Result Type
-	integer	integer
	real	real
NOT	bits (N)	bits (N)
!	boolean	boolean

Table E1-2, Result and operand types that are permitted for binary operators.

Operator	First operand type	Second operand type	Result type
==	bits (N)	integer	boolean
	bits (N)	bits (N)	
	integer	integer	
	real	real	
	enumeration	enumeration	
	boolean	boolean	
!=	bits (N)	bits (N)	boolean
	integer	integer	
	real	real	
<, >	integer	integer	boolean
	<=, >=	real	
+, -	integer	integer	integer
	real	real	real
	bits (N)	bits (N)	bits (N)
	bits (N)	integer	
«, »	integer	integer	integer
	integer	integer	integer
*	real	real	real
	bits (N)	bits (N)	bits (N)
	real	real	real
/	integer	integer	integer
DIV	integer	integer	integer
MOD	integer	integer	integer
	bits (N)	integer	
&&,	boolean	boolean	boolean
AND, OR, EOR	bits (N)	bits (N)	bits (N)
^	integer	integer	integer
	real	integer	real

E1.4 Statements and control structures

This section describes the statements and program structures available in the pseudocode.

E1.4.1 Statements and Indentation

A simple statement is either an assignment, a function call, or a procedure call. Each statement must be terminated with a semicolon.

Indentation normally indicates the structure in compound statements. The statements that are contained in structures such as **if... then... else...** or procedure and function definitions are indented more deeply than the statement structure itself. The end of a compound statement structure and their end is indicated by returning to the original indentation level or less.

Indentation is normally done by four spaces for each level. Standard indentation uses four spaces for each level of indent.

E1.4.2 Function and procedure calls

This section describes how functions and procedures are defined and called in the pseudocode.

Procedure and function definitions

A procedure definition has the form:

```
1  <procedure name>(<argument prototypes>)  
2  <statement 1>;  
3  <statement 2>;  
4  ...  
5  <statement n>;
```

where `<argument prototypes>` consists of zero or more argument definitions, which are separated by commas. Each argument definition consists of a type name followed by the name of the argument.

Note

This first definition line is not terminated by a semicolon. This distinguishes it from a procedure call.

A function definition is similar, but also declares the return type of the function:

```
1  <return type> <function name>(<argument prototypes>)  
2  <statement 1>;  
3  <statement 2>;  
4  ...  
5  <statement n>;
```

Array-like functions are similar, but are written with square brackets and have two forms. These two forms exist because reading from and writing to an array element require different functions. They are frequently used in memory operations. An array-like function definition with a return type is equivalent to reading from an array. For example:

```
1  <return type> <function name>[<argument prototypes>]  
2  <statement 1>;  
3  <statement 2>;  
4  ...  
5  <statement n>;
```

Its related function definition with no return type is equivalent to writing to an array. For example:

```
1 <function name>[<argument prototypes>] =<value prototype>
2 <statement 1>;
3 <statement 2>;
4 ...
5 <statement n>;
```

The value prototype determines what data type can be written to the array. The two related functions must share the same name, but the value prototype and return type can be different.

Procedure calls

A procedure call has the form:

```
1 <procedure_name>(<arguments>);
```

Return statements

A procedure return has the form:

```
1 return;
```

A function return has the form:

```
1 return <expression>;
```

where <expression> is of the type declared in the function prototype line.

E1.4.3 Conditional control structures

This section describes how conditional control structures are used in the pseudocode.

if...then...else...

In addition to being a ternary operator, a multi-line **if...then...else...** structure can act as a control structure and has the form:

```
1 if <boolean_expression> then
2 <statement 1>;
3 <statement 2>;
4 ...
5 <statement n>;
6
7 elseif <boolean_expression> then
8 <statement a>;
9 <statement b>;
10 ...
11 <statement z>;
12 else
13 <statement A>;
14 <statement B>;
15 ...
16 <statement Z>;
```

The block of lines consisting of **elseif** and its indented statements is optional, and multiple **elseif** blocks can be used.

The block of lines consisting of **else** and its indented statements is optional.

Abbreviated one-line forms can be used when the **then** part, and in the **else** part if it is present, contain only simple statements such as:

```
1  if <boolean_expression> then <statement 1>;  
2  if <boolean_expression> then <statement 1>; else <statement A>;  
3  if <boolean_expression> then <statement 1>; <statement 2>; else <statement A>;
```

Note

In these forms, <statement 1>, <statement> 2>, and <statement A> must be terminated by semicolons. This and > the fact that the **else** part is optional distinguish its use as a > control structure from its use as a ternary operator.

case...of...

A case...of... structure has the form:

```
1  case <expression> of  
2  when <literal values1>  
3    <statement 1>;  
4    <statement 2>;  
5    ...  
6    <statement n>;  
7  
8  when <literal values2>  
9    <statement 1>;  
10   <statement 2>;  
11   ...  
12   <statement n>;  
13  
14   ...more "when" groups if required...  
15  
16  otherwise  
17    <statement A>;  
18    <statement B>;  
19    ...  
20    <statement Z>;
```

In this structure, <literal values1> and <literal values2> consist of literal values of the same type as <expression>, separated by commas. There can be additional **when** groups in the structure. Abbreviated one line forms of **when** and **otherwise** parts can be used when they contain only simple statements.

If <expression> has a bitstring type, the literal values can also include bitstring literals containing 'x' bits, known as bitmasks. For details, see Equality and non-equality.

E1.4.4 Loop control structures

This section describes the three loop control structures that are used in the pseudocode.

repeat...until...

A repeat...until... structure has the form:

```
1  repeat  
2    <statement 1>;  
3    <statement 2>;  
4    ...  
5    <statement n>;  
6  until <boolean_expression>;
```

It executes the statement block at least once, and the loop repeats until **<boolean expression>** evaluates to **TRUE**. Variables explicitly declared inside the loop body have scope local to that loop and might not be accessed outside the loop body.

while...do

A `while...do` structure has the form:

```
1 while <boolean_expression> do
2   <statement 1>;
3   <statement 2>;
4   ...
5   <statement n>;
```

It begins executing the statement block only if the boolean expression is true. The loop then runs until the expression is false.

for...

A `for...` structure has the form:

```
1 for <assignable_expression> = <integer_expr1> to <integer_expr2>
2   <statement 1>;
3   <statement 2>;
4   ...
5   <statement n>;
```

The `<assignable_expression>` is initialized to `<integer_expr1>` and compared to `<integer_expr2>`. If `<integer_expr1>` is less than `<integer_expr2>`, the loop body is executed and the `<assignable_expression>` is incremented by one. This repeats until `<assignable_expression>` is more than or equal to `<integer_expr2>`.

There is an alternate form:

for `<assignable_expression> = <integer_expr1> downto <integer_expr2>`

where `<integer_expr1>` is decremented after the loop body executes and continues until `<assignable_expression>` is less than or equal than `<integer_expr2>`.

Try...Catch

A `try...catch` structure has the following form:

```
1 try
2   <statement 1>;
3   <statement 2>;
4   ...
5   <statement n>;
6
7 catch <exception>
8   <statement a>;
9   <statement b>;
10  ...
11  <statement z>;
```

The purpose of this structure is to catch exceptions that are generated by the `try` statements.

E1.4.5 Special statements

This section describes statements with particular architecturally defined behaviors.

UNDEFINED

This subsection describes the statement:

`UNDEFINED;`

This statement indicates a pseudocode exception that will be caught by the `try...catch` block. When caught, this might result in an UNDEFINSTR UsageFault, NOP or NOCP UsageFault.

UNPREDICTABLE

This subsection describes the statement:

```
UNPREDICTABLE;
```

This statement indicates a special case that replaces the behavior that is defined by the current pseudocode, apart from behavior that is required to determine that the special case applies. The replacement behavior is UNPREDICTABLE.

CONSTRAINED_UNPREDICTABLE

This subsection describes the statement:

```
CONSTRAINED_UNPREDICTABLE;
```

This statement indicates a special case that replaces the behavior that is defined by the current pseudocode, apart from behavior that is required to determine that the special case applies. The replacement behavior is CONSTRAINED UNPREDICTABLE within the limits defined for each particular case, and might vary.

SEE . . .

This subsection describes the statement:

```
SEE <reference>;
```

This statement indicates a special case that replaces the behavior that is defined by the current instruction pseudocode, apart from behavior that is required to determine that the special case applies.

It usually refers to another instruction, but can also refer to another encoding or note of the same instruction.

IMPLEMENTATION_DEFINED

This subsection describes the statement:

```
IMPDEF { "<text>" };
```

This statement indicates a special case that provides an IMPLEMENTATION_DEFINED value or behavior. An optional <text> field can give more information.

E1.4.6 Comments

The pseudocode supports two styles of comments:

- `//` starts a comment that is terminated by the end of the line.
- `/*` starts a comment that is terminated by `*/`.

`/**` statements might not be nested, and the first `*/` ends the comment.

Note

Comment lines do not require a terminating semicolon.

E1.5 Built in functions

Bitstring length

If x is a bitstring:

- The bitstring length function `Len (x)` returns the length of x as an integer.

Converting bitstrings to integers

If x is a bitstring, `SInt ()` is the integer whose twos complement representation is x .

`UInt ()` is the integer whose unsigned representation is x .

E1.6 Arm pseudocode definition index

This section contains the following tables:

[Table E1-3](#) which contains the pseudocode data types.

[Table E1-4](#) which contains the pseudocode operators.

[Table E1-5](#) which contains the pseudocode keywords and control structures.

[Table E1-6](#) which contains the statements with special behaviors.

Table E1-3 Index of pseudocode data types

Keyword	Meaning
array	Type name for the array type
bit	Keyword equivalent to <code>bits(1)</code>
bits(N)	Type name for the bitstring of length N data type
boolean	Type name for the boolean data type
enumeration	Keyword to define a new enumeration type
integer	Type name for the integer data type
real	Type name for the real data type
type	Keyword to define a new structure

Table E1-4 Index of pseudocode operators

Operator	Meaning
–	Unary minus on integers or reals
	Subtraction of integers, reals, and bitstrings
	Used in the left-hand side of an assignment or a tuple to discard the result
+	Unary plus on integers or reals
	Addition of integers, reals, and bitstrings
.	Extract named member from a list
	Integer in bitstring extraction operator
:	Bitstring concatenation
	Integer range in bitstring extraction operator
!	Boolean NOT
!=	Comparison for inequality
(...)	Around arguments of procedure or function
[...]	Around array index
	Around arguments of array-like function
*	Multiplication of integers, reals and bitstrings
/	Division of integers and reals (real result)
&&	Boolean AND
<	<i>Less than</i> comparison of integers and reals
[...]	Slicing of specified bits or bitstring or integer
<<	Multiply integer by power of 2 (with rounding towards infinity)
<=	<i>Less than or equal</i> comparison of integers and reals
=	Assignment operator
==	Comparison for equality
>	<i>Greater than</i> comparison of integers and reals
>=	<i>Greater than or equal</i> comparison of integers and reals
>>	Divide integer by power of 2
	Boolean OR

$\wedge$	Exponential operator
AND	Bitwise AND of bitstrings
DIV	Quotient from integer division
EOR	Bitwise EOR of bitstrings
IN	Test membership of a certain expression in a set of values
MOD	Remainder from integer division
NOT	Bitwise inversion of bitstrings
OR	Bitwise OR of bitstrings

Table E1-5 Index of pseudocode keywords and control structures

Operator	Meaning
<code>/*...*/</code>	Comment delimiters
<code>//</code>	Introduces comment terminated by end of line
<code>case...of...</code>	Control structure
FALSE	One of two values a boolean can take (other than TRUE)
<code>for...=...to...</code>	Loop control structure, counting up from the initial value to the upper limit
<code>for...=...downto...</code>	Loop control structure, counting down from the initial value to the lower limit
<code>if...then...else...</code>	Condition expression selecting between two values
<code>if...then...else...</code>	Conditional control structure
<code>otherwise</code>	Introduces default in <code>case...of...</code> control structure
<code>repeat...until...</code>	Loop control structure that runs at least once until the termination condition is satisfied
<code>return</code>	Procedure or function return
TRUE	One of two values a boolean can take (other than FALSE)
<code>try...catch</code>	Control structure
<code>when</code>	Introduces a specific case in <code>case...of...</code> control structure
<code>while...do...</code>	Loop control structure that runs until the termination condition is satisfied

Table E1-6 Index of special statements

Keyword	Meaning
IMPLEMENTATION_DEFINED	Describes IMPLEMENTATION_DEFINED behavior.
SEE	Points to other pseudocode to use instead
UNDEFINED	Cause Undefined Instruction exception
UNKNOWN	Unspecified value
CONSTRAINED_UNPREDICTABLE	Unspecified behavior within limits
UNPREDICTABLE	Unspecified behavior

E1.7 Additional functions

The following functions are not listed in E2 Pseudocode specification, and are only described in this section.

E1.7.1 **IsSee()**

`IsSee()` returns TRUE if the exception variable that is passed to it was created because all the encodings that matched the instruction that was being decoded called SEE.

See [SEE...](#)

E1.7.2 **IsUndefined()**

`IsUndefined()` returns TRUE if the exception variable that is passed to it was created because either the instruction that was being decoded did not match any known encoding, or because one of the encodings that was matched called the special statement UNDEFINED.

See [UNDEFINED](#).

Chapter E2

Pseudocode Specification

This chapter specifies the Armv8-M pseudocode. It contains the following section:

[Alphabetical Pseudocode List](#)

E2.1 Alphabetical Pseudocode List

E2.1.1 `_AdvanceVPTState`

```
1 // Advances VPT state
2
3 boolean _AdvanceVPTState;
```

E2.1.2 `_CommitState`

```
1 // _CommitState()
2 // =====
3
4 boolean _CommitState;
```

E2.1.3 `_ITStateChanged`

```
1 // Indicates a write to ITSTATE
2
3 boolean _ITStateChanged;
```

E2.1.4 `_Mem`

```
1 // _Mem[] - non-assignment (read) form
2 // =====
3 // Perform single-copy atomic, aligned, little-endian read from physical memory.
4
5 (boolean, bits(8*size)) _Mem(AddressDescriptor memaddrdesc, integer size);
6
7 // _Mem[] - assignment (write) form
8 // =====
9 // Perform single-copy atomic, aligned, little-endian write to physical memory.
10
11 boolean _Mem(AddressDescriptor memaddrdesc, integer size, bits(8*size) value);
```

E2.1.5 `_NextInstrAddr`

```
1 // Address of next instruction to be fetched in case of branch type operation
2
3 bits(32) _NextInstrAddr;
```

E2.1.6 `_NextInstrITState`

```
1 // Updated ITSTATE for next instruction
2
3 ITSTATETYPE _NextInstrITState;
```

E2.1.7 `_PCChanged`

```
1 // Indicates a change in instruction fetch address due to branch type operations
2
3 boolean _PCChanged;
```

E2.1.8 `_PendingFetchFault`

```
1 // Indicates a pending fetch fault
2
3 boolean _PendingFetchFault;
```

E2.1.9 `_PendingReturnOperation`

```

1 // Indicate any pending exception returns
2
3 boolean _PendingReturnOperation;

```

E2.1.10 `_RName`

```

1 // The physical array of core registers.
2 // _R[RName_PC] is defined to be the address of the current instruction.
3 // The offset of 4 bytes is applied to it by the register access functions.
4
5 array bits(32) _RName[RNames];

```

E2.1.11 `_S`

```

1 // The 32-bit extension register bank for the Floating-point extension.
2
3 array bits(32) _S[0..31];

```

E2.1.12 `_SP`

```

1 // _SP()
2 // =====
3
4 // Non-assignment form
5
6 bits(32) _SP(RNames spreg)
7     assert ( (spreg == RNamesSP_Main_NonSecure) ||
8             ((spreg == RNamesSP_Main_Secure) && HaveSecurityExt()) ||
9             (spreg == RNamesSP_Process_NonSecure) ||
10            (spreg == RNamesSP_Process_Secure) && HaveSecurityExt()) );
11
12     return _RName[spreg][31:2]:'00';
13
14 // Assignment form
15
16 ExcInfo _SP(RNames spreg, boolean excEntry, boolean skipLimitCheck, bits(32) value)
17     assert ( (spreg == RNamesSP_Main_NonSecure) ||
18             ((spreg == RNamesSP_Main_Secure) && HaveSecurityExt()) ||
19             (spreg == RNamesSP_Process_NonSecure) ||
20            (spreg == RNamesSP_Process_Secure) && HaveSecurityExt()) );
21
22     excInfo = DefaultExcInfo();
23     if !skipLimitCheck && ViolatesSPLim(spreg, value) then
24         isSecure = ((spreg == RNamesSP_Main_Secure) ||
25                    (spreg == RNamesSP_Process_Secure));
26         // If the stack limit is violated during exception entry then the stack
27         // pointer is set to the limit value. This both prevents violations and
28         // ensures that the stack pointer is 8 byte aligned.
29         if excEntry then
30             _RName[spreg] = LookUpSPLim(spreg);
31
32         // Raise the appropriate exception and syndrome information.
33         if isSecure then
34             UFSR_S.STKOF = '1';
35         else
36             UFSR_NS.STKOF = '1';
37         // Create the exception. NOTE: If Main Extension is not implemented the fault always
38         // escalates to HardFault.
39         excInfo = CreateException(UsageFault, TRUE, isSecure);
40         if !excEntry then
41             HandleException(excInfo);
42     else
43         // Stack pointer only updated normally if limit not violated.

```

```

44     _RName[spreg] = value[31:2]:'00';
45     return excInfo;

```

E2.1.13 Abs

```

1  // Abs()
2  // =====
3
4  integer Abs(integer x)
5      return if x >= 0 then x else -x;
6
7  real Abs(real x)
8      return if x >= 0.0 then x else -x;

```

E2.1.14 AccessAttributes

```

1  // Memory access attributes
2
3  type AccessAttributes is (
4      boolean iswrite,           // TRUE for memory stores, FALSE for load accesses.
5      boolean ispriv,           // TRUE if the access is privileged, FALSE if unprivileged.
6      AccType acctype
7  )

```

E2.1.15 AccType

```

1  // Memory reference access type
2
3  enumeration AccType { AccType_NORMAL,           // Normal loads and stores.
4                        AccType_MVE,              // Loads and stores generated by MVE
5                                                         // instructions.
6                        AccType_ORDERED,          // Load-Acquire and Store-Release.
7                        AccType_STACK,           // HW generated stacking or unstacking
8                                                         // operation.
9                        AccType_LAZYFP,           // HW generated stacking due to lazy
10                                                         // floating-point state preservation.
11                        AccType_IFETCH,           // Instruction fetch.
12                        AccType_DBG,              // Loads and Stores generated by the Debugger.
13                        AccType_VECTABLE          // Vector table fetch.
14                };

```

E2.1.16 ActivateException

```

1  // ActivateException()
2  // =====
3
4  ActivateException(integer exceptionNumber, boolean excIsSecure)
5      // If the exception is Secure, directly entry the Secure state.
6      CurrentState = if excIsSecure
7                      then SecurityState_Secure else SecurityState_NonSecure;
8      IPSR.Exception = exceptionNumber[8:0];           // Update IPSR to this exception.
9                                                         // This also causes a transition to
10                                                         // privileged handler
11                                                         // mode as IPSR.Exception != 0.
12
13      if HaveMainExt() then
14          EPSR.IT = Zeros(8);                         // IT/ICI/ECI bits cleared.
15
16      if HavePACBTIExt() then
17          EPSR.B = '0';                                // Clear BTI state.
18
19      // PRIMASK, FAULTMASK, BASEPRI unchanged on exception entry.
20
21      if HaveMveOrFPEExt() then
22          CONTROL.FPCA = '0';                         // Floating-point Extension only
23          CONTROL.S.SFPA = '0';
24
25      CONTROL.SPSEL = '0';                             // CONTROL.SPSEL is updated to indicate
26                                                         // the selection of the Main stack pointer

```



```

22                                     // (SP_main), CONTROL.nPRIV unchanged.
23 // Transition exception from pending to active.
24 SetPending(exceptionNumber, excIsSecure, FALSE);
25 SetActive(exceptionNumber, excIsSecure, TRUE);

```

E2.1.17 ActiveFPState

```

1 // ActiveFPState()
2 // =====
3
4 boolean ActiveFPState()
5 // Is the floating-point state accessible
6 (active, -) = IsCPEnabled(10);
7
8 // Is floating-point lazy state preservation active.
9 if active then
10     if FPCCR.S == '1' then
11         lspact = FPCCR.LSPACT;
12     else
13         lspact = FPCCR_NS.LSPACT;
14     active = lspact == '0';
15
16 // Check ASPEN to determine if the PE or software is managing the
17 // floating-point state.
18 if active && FPCCR.ASPEN == '1' then
19     // If the PE is managing the floating-point state then FPCA can be used
20     // to indicate if the current context has an active floating-point state.
21     // Similarly SFPA is also checked to determine if the floating-point state
22     // is active for the Secure state.
23     active = CONTROL.FPCA == '1' && (!IsSecure() || CONTROL_S.SFPA == '1');
24 return active;

```

E2.1.18 AddressDescriptor

```

1 // Descriptor used to access the underlying memory array
2
3 type AddressDescriptor is (
4     MemoryAttributes memattr,
5     bits(32)          paddress, // Physical Address.
6     AccessAttributes accattr
7 )

```

E2.1.19 AddrType

```

1 // Indicates address type
2
3 enumeration AddrType { AddrType_NORMAL,
4                         AddrType_EXC_RETURN,
5                         AddrType_FNC_RETURN
6                         };

```

E2.1.20 AddWithCarry

```

1 // AddWithCarry()
2 // =====
3
4 (bits(N), bit, bit) AddWithCarry(bits(N) x, bits(N) y, bit carry_in)
5     unsigned_sum = UInt(x) + UInt(y) + UInt(carry_in);
6     signed_sum   = SInt(x) + SInt(y) + UInt(carry_in);
7     result       = unsigned_sum[N-1:0]; // same value as signed_sum[N-1:0].
8     carry_out    = if UInt(result) == unsigned_sum then '0' else '1';
9     overflow     = if SInt(result) == signed_sum then '0' else '1';
10    return (result, carry_out, overflow);

```

E2.1.21 AdvSIMDExpandImm

```

1 // AdvSIMDExpandImm()
2 // =====
3
4 bits(64) AdvSIMDExpandImm(bit op, bits(4) cmode, bits(8) imm8)
5
6     case cmode[3:1] of
7         when '000'
8             imm64 = Replicate(Zeros(24):imm8, 2);
9         when '001'
10            imm64 = Replicate(Zeros(16):imm8:Zeros(8), 2);
11         when '010'
12            imm64 = Replicate(Zeros(8):imm8:Zeros(16), 2);
13         when '011'
14            imm64 = Replicate(imm8:Zeros(24), 2);
15         when '100'
16            imm64 = Replicate(Zeros(8):imm8, 4);
17         when '101'
18            imm64 = Replicate(imm8:Zeros(8), 4);
19         when '110'
20             if cmode[0] == '0' then
21                 imm64 = Replicate(Zeros(16):imm8:Ones(8), 2);
22             else
23                 imm64 = Replicate(Zeros(8):imm8:Ones(16), 2);
24         when '111'
25             if cmode[0] == '0' && op == '0' then
26                 imm64 = Replicate(imm8, 8);
27             if cmode[0] == '0' && op == '1' then
28                 imm8a = Replicate(imm8[7], 8); imm8b = Replicate(imm8[6], 8);
29                 imm8c = Replicate(imm8[5], 8); imm8d = Replicate(imm8[4], 8);
30                 imm8e = Replicate(imm8[3], 8); imm8f = Replicate(imm8[2], 8);
31                 imm8g = Replicate(imm8[1], 8); imm8h = Replicate(imm8[0], 8);
32                 imm64 = imm8a:imm8b:imm8c:imm8d:imm8e:imm8f:imm8g:imm8h;
33             if cmode[0] == '1' && op == '0' then
34                 imm32 = imm8[7]:NOT(imm8[6]):Replicate(imm8[6],5):imm8[5:0]:Zeros(19);
35                 imm64 = Replicate(imm32, 2);
36             if cmode[0] == '1' && op == '1' then
37                 UNDEFINED;
38
39     return imm64;

```

E2.1.22 AlgorithmPAC

```

1 // Algorithm used for computing PAC
2
3 enumeration AlgorithmPAC {AlgorithmPAC_None, // PACBTI not implemented.
4                           AlgorithmPAC_QARMA3, // QARMA3 algorithm implemented.
5                           AlgorithmPAC_QARMA5, // QARMA5 algorithm implemented.
6                           AlgorithmPAC_IMPDEF // IMPLEMENTATION DEFINED algorithm
7                                           // implemented.
8                           };

```

E2.1.23 Align

```

1 // Align()
2 // =====
3
4 integer Align(integer x, integer y)
5     return y * (x DIV y);
6
7 bits(N) Align(bits(N) x, integer y)
8     return Align(UInt(x), y) [N-1:0];

```

E2.1.24 ArchVersion

```

1 // Indicates architecture version
2
3 enumeration ArchVersion {
4     Armv8p0,
5     Armv8p1
6 };

```

E2.1.25 ASR

```

1 // ASR()
2 // =====
3
4 bits(N) ASR(bits(N) x, integer shift)
5     assert shift >= 0;
6     if shift == 0 then
7         result = x;
8     else
9         (result, -) = ASR_C(x, shift);
10    return result;

```

E2.1.26 ASR_C

```

1 // ASR_C()
2 // =====
3
4 (bits(N), bit) ASR_C(bits(N) x, integer shift)
5     assert shift > 0;
6     extended_x = SignExtend(x, shift+N);
7     result = extended_x[shift+N-1:shift];
8     carry_out = extended_x[shift-1];
9     return (result, carry_out);

```

E2.1.27 BeatComplete

```

1 // BeatComplete
2 // =====
3
4 // The BeatComplete value indicates whether the 4 beats from 2 instructions have
5 // been performed. The flags are packed into an 8 bit value as follows:
6 // bit 0: beat 0 of instruction 0
7 // bit 1: beat 1 of instruction 0
8 // ...
9 // bit 4: beat 0 of instruction 1
10 // bit 5: beat 1 of instruction 1
11 // ...
12 //
13 // NOTE: The beat execution rules mean that only a few flag combinations are
14 //       valid.
15
16 // Non-assignment form
17 bits(8) BeatComplete
18     bits(8) beatComplete;
19     case EPSR.ECI of
20         when '00000000' beatComplete = '0000 0000';
21         when '00000001' beatComplete = '0000 0001';
22         when '00000010' beatComplete = '0000 0011';
23         when '00000100' beatComplete = '0000 0111';
24         when '00000101' beatComplete = '0001 0111';
25         otherwise
26             assert(FALSE);
27     return beatComplete;
28
29
30 // Assignment form
31 BeatComplete = bits(8) value

```

```

32     case value of
33         when '0000 0000' EPSR.ECI = 0[7:0];
34         when '0000 0001' EPSR.ECI = 1[7:0];
35         when '0000 0011' EPSR.ECI = 2[7:0];
36         when '0000 0111' EPSR.ECI = 4[7:0];
37         when '0001 0111' EPSR.ECI = 5[7:0];
38         otherwise
39             assert(FALSE);

```

E2.1.28 BeatSchedule

```

1  // BeatSchedule()
2  // =====
3
4  type InstInfoType is (
5      bits(32)    Opcode,
6      integer     Length,
7      boolean     Valid,
8      InstrType   Type
9  )
10
11 array [0..MAX_OVERLAPPING_INSTRS-1] of InstInfoType _InstInfo;
12 integer _InstID;
13 integer _BeatID;

```

E2.1.29 BigEndian

```

1  // BigEndian()
2  // =====
3
4  boolean BigEndian(bits(32) startAddress, integer size)
5      // If AIRCR.ENDIANNESS is 0 then the PE is in little endian mode.
6      if AIRCR.ENDIANNESS == '0' then
7          return FALSE;
8      // ...otherwise the PE is in big endian mode, however; the PPB
9      // space (0xE0000000 to 0xE00FFFFF) is always little endian.
10     endAddress = startAddress + size;
11     startPpbAccess = IsPPB(startAddress);
12     endPpbAccess = IsPPB(endAddress);
13     // If an access crosses the PPB boundary then it is
14     // CONSTRAINED_UNPREDICTABLE if the PE is in big endian mode.
15     if startPpbAccess != endPpbAccess then
16         CONSTRAINED_UNPREDICTABLE;
17     return !startPpbAccess;

```

E2.1.30 BigEndianReverse

```

1  // BigEndianReverse()
2  // =====
3
4  bits(8*N) BigEndianReverse (bits(8*N) value, integer N)
5      assert N == 1 || N == 2 || N == 4;
6      bits(8*N) result;
7      case N of
8          when 1
9              result[7:0] = value[7:0];
10         when 2
11             result[15:8] = value[7:0];
12             result[7:0] = value[15:8];
13         when 4
14             result[31:24] = value[7:0];
15             result[23:16] = value[15:8];
16             result[15:8] = value[23:16];
17             result[7:0] = value[31:24];
18     return result;

```

E2.1.31 BitCount

```

1 // BitCount()
2 // =====
3
4 integer BitCount(bits(N) x)
5     integer result = 0;
6     for i = 0 to N-1
7         if x[i] == '1' then
8             result = result + 1;
9     return result;

```

E2.1.32 BitReverseShiftRight

```

1 // BitReverseShiftRight()
2 // =====
3
4 bits(N) BitReverseShiftRight(bits(N) x, integer R)
5     reversed = Zeros(N);
6     if R > N then
7         R = N;
8     for i = 0 to R-1
9         reversed[R-i-1] = x[i];
10    return reversed;

```

E2.1.33 BranchCall

```

1 // BranchCall()
2 // =====
3
4 BranchCall(bits(32) address, boolean allowNonSecure, boolean setBti)
5     // If in the Secure state and transitions to the Non-secure state are allowed
6     // then the target state is specified by the LSB of the target address.
7     if HaveSecurityExt() && allowNonSecure && IsSecure() then
8         EPSR.T = '1';
9         if address[0] == '0' then
10            CurrentState = SecurityState_NonSecure;
11            if HaveMveOrFPEExt() then CONTROL_S.SFPA = '0';
12            if HaveLOBExt() then
13                LO_BRANCH_INFO.VALID = '0';
14        else
15            EPSR.T = address[0];
16            // If EPSR.T == 0 then an exception is taken on the next
17            // instruction: UsageFault('Invalid State') if the Main Extension is
18            // implemented; HardFault otherwise.
19
20    BranchTo(address[31:1]:'0', FALSE, setBti);

```

E2.1.34 BranchReturn

```

1 // BranchReturn()
2 // =====
3
4 ExcInfo BranchReturn(bits(32) address, boolean allowNonSecure, boolean setBti)
5     exc = DefaultExcInfo();
6
7     case IsReturn(address) of
8         when AddrType_NORMAL
9             BranchCall(address, allowNonSecure, setBti);
10        when AddrType_FNC_RETURN
11            // Unlike exception return, any faults raised during a FNC_RETURN
12            // unstacking are raised synchronously with the instruction that triggered
13            // the unstacking.
14            exc = FunctionReturn();
15        when AddrType_EXC_RETURN

```

```

16         // If enabled, the IESB contains asynchronous RAS / BusFault errors to the
17         // exception context.
18         if AIRCR.IESB == '1' then
19             exc = SynchronizeBusFault();
20         // The actual exception return is performed when the
21         // current instruction completes. This is because faults that occur
22         // during the exception return are handled differently from faults
23         // raised during the instruction execution.
24         if exc.fault == NoFault then
25             PendReturnOperation(address);
26
27     return exc;
28
29 ExcInfo BranchReturn(bits(32) address, boolean allowNonSecure)
30     return BranchReturn(address, allowNonSecure, FALSE);

```

E2.1.35 BranchTo

```

1 // BranchTo()
2 // =====
3
4 BranchTo(bits(32) address, boolean commit, boolean setBti)
5     if HaveLOBExt() then
6         // Any branch between a branch future instruction and the associated
7         // branch point invalidates the branch info cache.
8         if LO_BRANCH_INFO.VALID == '1' && LO_BRANCH_INFO.BF == '1' then
9             LO_BRANCH_INFO.VALID = '0';
10
11         // EPSR.B is set to ensure that the target of the branch is either a BTI clearing
12         // instruction or a BTI agnostic instruction.
13         if setBti then
14             EPSR.B = '1';
15
16         // Sets the address to fetch the next instruction from. NOTE: The current PC
17         // is not changed directly as this would modify the result of
18         // ThisInstrAddr(), which would cause the wrong return addresses to be used
19         // for some types of exception. The actual update of the PC is done in the
20         // InstructionAdvance() function after the instruction finishes executing.
21         _NextInstrAddr = address[31:1]:'0';
22         _PCChanged = TRUE;
23         // Clear any pending exception returns
24         _PendingReturnOperation = FALSE;
25
26         if commit then
27             // This directly commits the change to the PC, so ThisInstrAddr()
28             // and NextInstrAddr() both point to the target address. Used for exception
29             // returns and resets so the state is consistent before the next instruction
30             // (or exception) is taken.
31             _RName[RNamesPC] = _NextInstrAddr;
32
33 BranchTo(bits(32) address)
34     BranchTo(address, FALSE, FALSE);

```

E2.1.36 BTIEnabled

```

1 // BTIEnabled()
2 // =====
3 // Check whether BTI is enabled.
4
5 boolean BTIEnabled(boolean isSecure)
6     enabled = FALSE;
7     // The PACBTI Extension must be implemented in order to use BTI functionality.
8     if HavePACBTIExt() then
9         control = if isSecure then CONTROL_S else CONTROL_NS;
10        if CurrentModeIsPrivileged(isSecure) then
11            enabled = control.BTI_EN == '1'; // Privileged BTI enabled
12        else

```

```

13         enabled = control.UBTI_EN == '1'; // Unprivileged BTI enabled
14         return enabled;
15
16
17 boolean BTIEnabled()
18     return BTIEnabled(IsSecure());

```

E2.1.37 BusFaultBarrier

```

1 // BusFaultBarrier()
2 // =====
3
4 // Forces any latent BusFault (both RAS and non-RAS) to be recognised.
5 // This function returns TRUE if a previously unrecognised BusFault was detected.
6 boolean BusFaultBarrier();

```

E2.1.38 CallSupervisor

```

1 // CallSupervisor()
2 // =====
3
4 CallSupervisor()
5     // Clear the LOB cache so that the return address in the SVC exception stack frame is
6     // guaranteed to be the instruction directly after the SVC.
7     if HaveLOBExt() then
8         LO_BRANCH_INFO.VALID = '0';
9     excInfo = CreateException(SVCall);
10    HandleException(excInfo);

```

E2.1.39 CanDebugAccessFP

```

1 // CanDebugAccessFP()
2 // =====
3
4 boolean CanDebugAccessFP()
5     canAccessFP = (!HaveSecurityExt() || DHCSR.S_SDE == '1' ||
6         (FPCCR.S.S == '0' && NSACR.CP10 == '1'));
7
8     // Unprivileged-only debug for the state associated with the floating-point
9     // context restricts access through CPACR checking and if a lazy context is active.
10    if HaveUDE() then
11        if FPCCR.S.S == '1' && DHCSR.S_SUIDE == '1' then
12            canAccessFP = canAccessFP && CPACR.S.CP10 == '11' && FPCCR.S.LSPACT != '1';
13        elseif FPCCR.S.S == '0' && DHCSR.S_NSUIDE == '1' then
14            canAccessFP = canAccessFP && CPACR_NS.CP10 == '11' && FPCCR_NS.LSPACT != '1';
15    return canAccessFP;

```

E2.1.40 CanHaltOnEvent

```

1 // CanHaltOnEvent()
2 // =====
3
4 boolean CanHaltOnEvent(boolean is_secure, boolean isPriv)
5     if !HaveSecurityExt() then assert !is_secure;
6
7     if !HaveHaltingDebug() || Halted || DHCSR.C_DEBUGEN == '0' then return FALSE;
8
9     if is_secure then
10        if DHCSR.S_SDE == '1' && (!HaveUDE() || DHCSR.S_SUIDE == '0') then return TRUE;
11    else
12        if HaltingDebugAllowed() then return TRUE;
13
14    return (!isPriv && UnprivHaltingDebugEnabled(is_secure));

```

E2.1.41 CanPendMonitorOnEvent

```

1 // CanPendMonitorOnEvent()
2 // =====
3
4 boolean CanPendMonitorOnEvent(boolean isSecure, boolean checkPri, boolean checkEn,
5                               boolean isPriv, boolean checkSecure)
6     if !HaveSecurityExt() then assert !isSecure;
7
8     result = HaveDebugMonitor() && !CanHaltOnEvent(isSecure, isPriv) && !Halted;
9
10    if checkEn then
11        if HaveUDE() && !isPriv then
12            result = result && (DEMC.R.MON_EN == '1' || DEMC.R.UMON_EN == '1');
13        else
14            result = result && DEMC.R.MON_EN == '1';
15
16    if checkSecure then
17        result = result && (!isSecure || DEMC.R.SDME == '1');
18
19    if checkPri then
20        result = result && MonitorCanPreempt();
21
22    return result;

```

E2.1.42 CdeImpDefValue

```

1 // CdeImpDefValue()
2 // =====
3 // IMPLEMENTATION DEFINED value functions for the Custom Datapath Extension.
4
5 bits(size) CdeImpDefValue(bits(N) instr);
6 bits(size) CdeImpDefValue(bits(N) instr, bits(M) opa);
7 bits(size) CdeImpDefValue(bits(N) instr, bits(M) opa, bits(L) opb);
8 bits(size) CdeImpDefValue(bits(N) instr, bits(M) opa, bits(L) opb, bits(K) opc);
9
10 bits(size) CdeImpDefValue(bits(N) instr, integer curBeat, bits(4) elmtMask);
11 bits(size) CdeImpDefValue(bits(N) instr, bits(M) opa, integer curBeat, bits(4) elmtMask);
12 bits(size) CdeImpDefValue(bits(N) instr, bits(M) opa, bits(L) opb, integer curBeat,
13                           bits(4) elmtMask);
14 bits(size) CdeImpDefValue(bits(N) instr, bits(M) opa, bits(L) opb, bits(K) opc,
15                           integer curBeat, bits(4) elmtMask);

```

E2.1.43 CheckCDEDecodeFaults

```

1 // CheckCDEDecodeFaults()
2 // =====
3 // Check whether the coprocessors and features required by CDE are implemented and enabled.
4
5 CheckCDEDecodeFaults(integer cp, boolean isMve)
6     // Check whether access to the CDE coprocessor is enabled.
7     CheckCPDecodeFaults(cp);
8
9     // Check whether access to CP10 is enabled
10    CheckCPDecodeFaults(10);
11
12    if isMve then
13        // MVE extension is required.
14        if MVFR1.MVE == '0000' then UNDEFINED;
15    else
16        // Either MVE or the Floating-point extension are required.
17        if MVFR1.MVE == '0000' && MVFR0.FPSP == '0000' then UNDEFINED;

```

E2.1.44 CheckCPDecodeFaults


```

1 // CheckCPDecodeFaults()
2 // =====
3
4 CheckCPDecodeFaults(integer cp)
5     // Is the co-processor enabled, this may raise a NOCP UsageFault
6     excInfo = CheckCPEnabled(cp);
7     HandleException(excInfo);

```

E2.1.45 CheckCPEnabled

```

1 // CheckCPEnabled()
2 // =====
3
4 ExcInfo CheckCPEnabled(integer cp, boolean privileged, boolean secure)
5     (enabled, toSecure) = IsCPEnabled(cp, privileged, secure);
6     if !enabled then
7         if toSecure then
8             UFSR_S.NOCP = '1';
9         else
10            UFSR_NS.NOCP = '1';
11            excInfo = CreateException(UsageFault, TRUE, toSecure);
12        else
13            excInfo = DefaultExcInfo();
14        return excInfo;
15
16 ExcInfo CheckCPEnabled(integer cp)
17     return CheckCPEnabled(cp, CurrentModeIsPrivileged(), IsSecure());

```

E2.1.46 CheckDecodeFaults

```

1 // CheckDecodeFaults()
2 // =====
3 // Check and raise faults in the correct order for MVE and floating-point instructions.
4
5 CheckDecodeFaults(ExtType extType)
6     // Is the instruction in the co-processor space
7     (isCP, cpNumber) = IsCPInstruction(ThisInstr());
8     assert(isCP);
9
10    // Is the co-processor enabled, this may raise a NOCP UsageFault
11    CheckCPDecodeFaults(cpNumber);
12
13    // Check if the type of instruction is supported.
14    case extType of
15        when ExtType_HpFp      if MVFR1.FP16 == '0000' then UNDEFINED;
16        when ExtType_SpFp      if MVFR0.FPSP == '0000' then UNDEFINED;
17        when ExtType_DpFp      if MVFR0.FPDP == '0000' then UNDEFINED;
18        when ExtType_Mve        if MVFR1.MVE == '0000' then UNDEFINED;
19        when ExtType_MveOrFp    if MVFR1.MVE != '0010' then UNDEFINED;
20        when ExtType_MveOrFp
21            // Raises an UNDEFINED fault if MVE and the Floating-point extension are
22            // not implemented.
23            if MVFR1.MVE == '0000' && MVFR0.FPSP == '0000' then UNDEFINED;
24        when ExtType_MveOrDpFp
25            if MVFR1.MVE == '0000' && MVFR0.FPDP == '0000' then UNDEFINED;
26        when ExtType_Unknown
27            otherwise assert(FALSE);

```

E2.1.47 CheckFPDecodeFaults

```

1 // CheckFPDecodeFaults()
2 // =====
3
4 CheckFPDecodeFaults(bits(2) size)
5     // Checks the size field and identifies the correct ExtType.

```

```

6      case size of
7          when '00' CheckDecodeFaults(ExtType_Unknown);
8          when '01' CheckDecodeFaults(ExtType_HpFp);
9          when '10' CheckDecodeFaults(ExtType_SpFp);
10         when '11' CheckDecodeFaults(ExtType_DpFp);

```

E2.1.48 CheckPermission

```

1  // CheckPermission()
2  // =====
3
4  ExcInfo CheckPermission(Permissions perms, bits(32) address, AccType acctype,
5                          boolean iswrite, boolean ispriv, boolean isSecure)
6      if !perms.apValid then
7          fault = TRUE;
8      elseif (perms.xn == '1') && (acctype == AccType_IFETCH) then
9          fault = TRUE;
10     else
11         case perms.ap of
12             when '00' fault = !ispriv;
13             when '01' fault = FALSE;
14             when '10' fault = !ispriv || iswrite;
15             when '11' fault = iswrite;
16             otherwise UNPREDICTABLE;
17
18     // If a fault occurred generate the syndrome info and create the exception.
19     if fault then
20         // Create and write out the syndrome information on implementations with
21         // the Main Extension.
22         if HaveMainExt() then
23             MMFSR_Type fsr = Zeros(8);
24             case acctype of
25                 when AccType_IFETCH
26                     fsr.IACCVIOL = '1';
27                 when AccType_STACK
28                     if iswrite then
29                         fsr.MSTKERR = '1';
30                     else
31                         fsr.MUNSTKERR = '1';
32                 when AccType_LAZYFP
33                     fsr.MLSPEER = '1';
34                 when AccType_NORMAL, AccType_MVE, AccType_ORDERED
35                     fsr.MMFARVALID = '1';
36                     fsr.DACCVIOL = '1';
37                 when AccType_DBG
38                     // DAP errors do not set syndrome
39             otherwise
40                 assert(FALSE);
41
42         // Write the syndrome information to the correct instance of banked
43         // registers.
44         if isSecure then
45             MMFSR_S = MMFSR_S OR fsr;
46             if fsr.MMFARVALID == '1' then
47                 MMFAR_S = address;
48         else
49             MMFSR_NS = MMFSR_NS OR fsr;
50             if fsr.MMFARVALID == '1' then
51                 MMFAR_NS = address;
52
53         // Create the exception. NOTE: If Main Extension is not implemented the fault
54         // escalates to a HardFault.
55         excInfo = CreateException(MemManage, TRUE, isSecure);
56     else
57         excInfo = DefaultExcInfo();
58     return excInfo;

```

E2.1.49 ClearEventRegister

```

1 // ClearEventRegister
2 // =====
3 // Clears the Event register.
4
5 ClearEventRegister();

```

E2.1.50 ClearExclusiveByAddress

```

1 // ClearExclusiveByAddress
2 // =====
3 // Clear the global exclusive monitor for all PEs, except for the PE specified
4 // by processorid for which an address region including any of size bytes
5 // starting from address has had a request for an exclusive access.
6
7 ClearExclusiveByAddress(bits(32) address, integer exclprocessorid, integer size);

```

E2.1.51 ClearExclusiveLocal

```

1 // ClearExclusiveLocal()
2 // =====
3 // Clear local exclusive monitor records for the PE.
4
5 ClearExclusiveLocal(integer processorid);

```

E2.1.52 ClearInFlightInstructions

```

1 // ClearInFlightInstructions()
2 // =====
3
4 ClearInFlightInstructions()
5     for i = 0 to MAX_OVERLAPPING_INSTRS-1
6         _InstInfo[i].Valid = FALSE;

```

E2.1.53 ComparePriorities

```

1 // ComparePriorities()
2 // =====
3
4 boolean ComparePriorities(integer exc0Pri, integer exc0Number, boolean exc0IsSecure,
5                           integer exc1Pri, integer exc1Number, boolean exc1IsSecure)
6     if exc0Pri != exc1Pri then
7         takeE0 = exc0Pri < exc1Pri;
8     elseif exc0Number != exc1Number then
9         takeE0 = exc0Number < exc1Number;
10    elseif exc0IsSecure != exc1IsSecure then
11        takeE0 = exc0IsSecure;
12    else
13        // The two exceptions have exactly the same priority, so exception 0
14        // cannot be taken in preference to exception 1.
15        takeE0 = FALSE;
16    return takeE0;
17
18
19 boolean ComparePriorities(ExcInfo exc0Info, boolean groupPri,
20                           integer exc1Pri, integer exc1Number, boolean exc1IsSecure)
21     exc0Pri = ExceptionPriority(exc0Info.fault, exc0Info.isSecure, groupPri);
22     return ComparePriorities(exc0Pri, exc0Info.fault, exc0Info.isSecure,
23                             exc1Pri, exc1Number, exc1IsSecure);

```

E2.1.54 ComputePAC

```

1 // ComputePAC()
2 // =====
3 // QARMA cryptographic algorithm.
4 // When an IMPLEMENTATION DEFINED cryptographic algorithm is used, the
5 // input arguments must match the input arguments of this function.
6
7 array bits(64) RC[0..4];
8
9 bits(64) ComputePAC(bits(64) data, bits(64) modifier, bits(64) key0, bits(64) key1)
10     bits(64) workingval;
11     bits(64) runningmod;
12     bits(64) roundkey;
13     bits(64) modk0;
14     constant bits(64) Alpha = 0xC0AC29B7C97C50DD[63:0];
15
16     RC[0] = 0x0000000000000000[63:0];
17     RC[1] = 0x13198A2E03707344[63:0];
18     RC[2] = 0xA4093822299F31D0[63:0];
19     RC[3] = 0x082EFA98EC4E6C89[63:0];
20     RC[4] = 0x452821E638D01377[63:0];
21
22     // The QARMA algorithm is available in two configurations:
23     // 1. QARMA5 - Performs 5 iterations.
24     // 2. QARMA3 - Performs 3 iterations.
25     integer MAX_ITER = if (HaveAlgorithmPAC() == AlgorithmPAC_QARMA5) then 4 else 2;
26
27     modk0 = key0[0]:key0[63:2]:(key0[63] EOR key0[1]);
28     runningmod = modifier;
29     workingval = data EOR key0;
30     for i = 0 to MAX_ITER
31         roundkey = key1 EOR runningmod;
32         workingval = workingval EOR roundkey;
33         workingval = workingval EOR RC[i];
34         if i > 0 then
35             workingval = PACCellShuffle(workingval);
36             workingval = PACMult(workingval);
37             if (HaveAlgorithmPAC() == AlgorithmPAC_QARMA5) then
38                 workingval = PACSubQ5(workingval);
39             else
40                 workingval = PACSubQ3(workingval);
41                 runningmod = TweakShuffle(runningmod[63:0]);
42         roundkey = modk0 EOR runningmod;
43         workingval = workingval EOR roundkey;
44         workingval = PACCellShuffle(workingval);
45         workingval = PACMult(workingval);
46         if (HaveAlgorithmPAC() == AlgorithmPAC_QARMA5) then
47             workingval = PACSubQ5(workingval);
48         else
49             workingval = PACSubQ3(workingval);
50             workingval = PACCellShuffle(workingval);
51             workingval = PACMult(workingval);
52             workingval = key1 EOR workingval;
53             workingval = PACCellInvShuffle(workingval);
54             if (HaveAlgorithmPAC() == AlgorithmPAC_QARMA5) then
55                 workingval = PACInvSub(workingval);
56             else
57                 workingval = PACSubQ3(workingval);
58             workingval = PACMult(workingval);
59             workingval = PACCellInvShuffle(workingval);
60             workingval = workingval EOR key0;
61             workingval = workingval EOR runningmod;
62         for i = 0 to MAX_ITER
63             if (HaveAlgorithmPAC() == AlgorithmPAC_QARMA5) then
64                 workingval = PACInvSub(workingval);
65             else
66                 workingval = PACSubQ3(workingval);
67             if i < MAX_ITER then
68                 workingval = PACMult(workingval);
69                 workingval = PACCellInvShuffle(workingval);

```

```

70     runningmod = TweakInvShuffle(runningmod[63:0]);
71     roundkey   = key1 EOR runningmod;
72     workingval = workingval EOR RC[MAX_ITER-i];
73     workingval = workingval EOR roundkey;
74     workingval = workingval EOR Alpha;
75     workingval = workingval EOR modk0;
76
77     return workingval;

```

E2.1.55 Cond

```

1  // Condition code definitions
2  // =====
3
4  constant bits(4) CondEQ = 0x0[3:0];
5  constant bits(4) CondNE = 0x1[3:0];
6  constant bits(4) CondCS = 0x2[3:0];
7  constant bits(4) CondCC = 0x3[3:0];
8  constant bits(4) CondMI = 0x4[3:0];
9  constant bits(4) CondPL = 0x5[3:0];
10 constant bits(4) CondVS = 0x6[3:0];
11 constant bits(4) CondVC = 0x7[3:0];
12 constant bits(4) CondHI = 0x8[3:0];
13 constant bits(4) CondLS = 0x9[3:0];
14 constant bits(4) CondGE = 0xA[3:0];
15 constant bits(4) CondLT = 0xB[3:0];
16 constant bits(4) CondGT = 0xC[3:0];
17 constant bits(4) CondLE = 0xD[3:0];
18 constant bits(4) CondAL = 0xE[3:0];
19 constant bits(4) CondNV = 0xF[3:0];

```

E2.1.56 ConditionHolds

```

1  // ConditionHolds()
2  // =====
3
4  boolean ConditionHolds(bits(3) shortCond, bit n, bit z, bit c, bit v)
5      // Expand the short condition to the standard 4 bit representation.
6      case shortCond of
7          when '000' cond = CondEQ;
8          when '001' cond = CondNE;
9          when '010' cond = CondCS;
10         when '011' cond = CondHI;
11         when '100' cond = CondGE;
12         when '101' cond = CondLT;
13         when '110' cond = CondGT;
14         when '111' cond = CondLE;
15         return ConditionHolds(cond, n, z, c, v, TRUE);
16
17 boolean ConditionHolds(bits(4) cond)
18     return ConditionHolds(cond, APSR.N, APSR.Z, APSR.C, APSR.V, FALSE);
19
20 boolean ConditionHolds(bits(4) cond, bit n, bit z, bit c, bit v, boolean allowNV)
21     // Evaluate base condition.
22     case cond[3:1] of
23         when '000' result = (z == '1');           // EQ or NE
24         when '001' result = (c == '1');           // CS or CC
25         when '010' result = (n == '1');           // MI or PL
26         when '011' result = (v == '1');           // VS or VC
27         when '100' result = (c == '1') && (z == '0'); // HI or LS
28         when '101' result = (n == v);             // GE or LT
29         when '110' result = (n == v) && (z == '0'); // GT or LE
30         when '111' result = TRUE;                 // AL or possibly NV
31
32     // The LSB of the condition code is an invert flag. Some situations prohibit
33     // execute-never, and treat it the same as execute always. This applies the
34     // invert taking into account whether the inverse of always is allowed.

```

```

35     if cond[0] == '1' && (cond != CondNV || allowNV) then
36         result = !result;
37     return result;

```

E2.1.57 ConditionPassed

```

1  // ConditionPassed()
2  // =====
3
4  boolean ConditionPassed()
5      return ConditionPassed(CurrentCond());
6
7  boolean ConditionPassed(bits(4) cond)
8      passed = ConditionHolds(cond);
9      return passed;

```

E2.1.58 ConstrainUnpredictable

```

1  // ConstrainUnpredictable()
2  // =====
3  // Return the appropriate Constraint result to control the caller's behavior.
4  // The return value is IMPLEMENTATION DEFINED within a permitted list for
5  // each UNPREDICTABLE case.
6  // (The permitted list is determined by an assert or case statement at the call site).
7
8  Constraint ConstrainUnpredictable(Unpredictable which);

```

E2.1.59 ConstrainUnpredictableBits

```

1  // ConstrainUnpredictableBits()
2  // =====
3  // This is a variant of ConstrainUnpredictable for when the result can be Constraint_UNKNOWN.
4  // If the result is Constraint_UNKNOWN then the function also returns UNKNOWN value, but that
5  // value is always an allocated value; that is, one for which the behavior is not itself
6  // CONSTRAINED.
7
8  (Constraint, bits(width)) ConstrainUnpredictableBits(Unpredictable which);

```

E2.1.60 ConstrainUnpredictableBool

```

1  // ConstrainUnpredictableBool()
2  // =====
3  // This is a wrapper for UNPREDICTABLE cases where the constrained result is
4  // either TRUE or FALSE.
5
6  boolean ConstrainUnpredictableBool(Unpredictable which);

```

E2.1.61 ConstrainUnpredictableInteger

```

1  // ConstrainUnpredictableInteger()
2  // =====
3  // This is a variant of ConstrainUnpredictable for when the result can be Constraint_UNKNOWN.
4  // If the result is Constraint_UNKNOWN then the function also returns an UNKNOWN value in
5  // the range low to high, inclusive.
6
7  (Constraint, integer) ConstrainUnpredictableInteger(integer low, integer high,
8                                                         Unpredictable which);

```

E2.1.62 ConsumeExcStackFrame

```

1 // ConsumeExcStackFrame()
2 // =====
3
4 ConsumeExcStackFrame(EXC_RETURN_Type excReturn, bit fourByteAlign)
5 // Calculate the size of the integer part of the stack frame.
6 toSecure = HaveSecurityExt() && excReturn.S == '1';
7 if toSecure && (excReturn.ES == '0' ||
8               excReturn.DCRS == '0') then
9     framesize = 0x48;
10 else
11     framesize = 0x20;
12 // Add on the size of the floating-point part of the stack frame if present
13 if HaveMveOrFPEExt() && excReturn.FType == '0' then
14     if toSecure && FPCCR.S.TS == '1' then
15         framesize = framesize + 0x88;
16     else
17         framesize = framesize + 0x48;
18
19 // Update stack pointer. NOTE: Stack pointer limit not checked on exception
20 // return as stack pointer guaranteed to be ascending not descending.
21 mode = if excReturn.Mode == '1' then PMode_Thread else PMode_Handler;
22 spName = LookUpSP_with_security_mode(toSecure, mode);
23 exc = _SP(spName, FALSE, TRUE, (_SP(spName) + framesize) OR
24                                     ZeroExtend(fourByteAlign:'00',32));
25 assert exc.fault == NoFault;

```

E2.1.63 ConsumptionOfSpeculativeDataBarrier

```

1 // Consumption of Speculative Data Barrier
2 // =====
3 // Perform a Consumption of Speculative Data Barrier operation.
4
5 ConsumptionOfSpeculativeDataBarrier();

```

E2.1.64 Coproc_Accepted

```

1 // Coproc_Accepted
2 // =====
3 // Check whether a coprocessor accepts an instruction.
4
5 boolean Coproc_Accepted(integer cp_num, bits(32) instr);

```

E2.1.65 Coproc_DoneLoading

```

1 // Coproc_DoneLoading
2 // =====
3 // Check whether enough 32-bit words have been loaded for an LDC instruction.
4
5 boolean Coproc_DoneLoading(integer cp_num, bits(32) instr);

```

E2.1.66 Coproc_DoneStoring

```

1 // Coproc_DoneStoring
2 // =====
3 // Check whether enough 32-bit words have been stored for a STC instruction.
4
5 boolean Coproc_DoneStoring(integer cp_num, bits(32) instr);

```

E2.1.67 Coproc_GetOneWord

```

1 // Coproc_GetOneWord
2 // =====
3 // Gets the 32-bit word for an MRC instruction from the coprocessor.

```

```

4
5 bits(32) Coproc_GetOneWord(integer cp_num, bits(32) instr);

```

E2.1.68 Coproc_GetTwoWords

```

1 // Coproc_GetTwoWords
2 // =====
3 // Get two 32-bit words for an MRRC instruction from the coprocessor.
4
5 (bits(32), bits(32)) Coproc_GetTwoWords(integer cp_num, bits(32) instr);

```

E2.1.69 Coproc_GetWordToStore

```

1 // Coproc_GetWordToStore
2 // =====
3 // Gets the next 32-bit word to store for an STC instruction from the coprocessor.
4
5 bits(32) Coproc_GetWordToStore(integer cp_num, bits(32) instr);

```

E2.1.70 Coproc_InternalOperation

```

1 // Coproc_InternalOperation
2 // =====
3 // Instructs a coprocessor to perform the internal operation requested
4 // by a CDP instruction.
5
6 Coproc_InternalOperation(integer cp_num, bits(32) instr);

```

E2.1.71 Coproc_SendLoadedWord

```

1 // Coproc_SendLoadedWord
2 // =====
3 // Sends a loaded 32-bit word for an LDC instruction to the coprocessor.
4
5 Coproc_SendLoadedWord(bits(32) word, integer cp_num, bits(32) instr);

```

E2.1.72 Coproc_SendOneWord

```

1 // Coproc_SendOneWord
2 // =====
3 // Sends the 32-bit word for an MCR instruction to the coprocessor.
4
5 Coproc_SendOneWord(bits(32) word, integer cp_num, bits(32) instr);

```

E2.1.73 Coproc_SendTwoWords

```

1 // Coproc_SendTwoWords
2 // =====
3 // Send two 32-bit words for an MCRR instruction to the coprocessor.
4
5 Coproc_SendTwoWords(bits(32) word2, bits(32) word1, integer cp_num, bits(32) instr);

```

E2.1.74 CoprocType

```

1 // CoprocType
2 // =====
3 // Returns the architecture defined enumeration of the instruction set
4 // supported by the given coprocessor space.
5
6 CPDef CoprocType(integer coproc)

```



```

7      // The CDE extension defines two encoding patterns:
8      // - CP_GCP    : The architected coprocessor encodings for MRC, MCR,
9      //              CDP and other coprocessor instructions.
10     // - CP_CDEv1 : Version 1 of the Custom Datapath Extension.
11     coprocType = CP_GCP;
12     if coproc >= 0 && coproc <= 7 then
13         if boolean IMPLEMENTATION_DEFINED "CDE enabled coprocessor" then
14             coprocType = CP_CDEv1;
15     return coprocType;

```

E2.1.75 CountLeadingSignBits

```

1  // CountLeadingSignBits()
2  // =====
3
4  integer CountLeadingSignBits(bits(N) x)
5      return CountLeadingZeroBits(x[N-1:1] EOR x[N-2:0]);

```

E2.1.76 CountLeadingZeroBits

```

1  // CountLeadingZeroBits()
2  // =====
3
4  integer CountLeadingZeroBits(bits(N) x)
5      return N - 1 - HighestSetBit(x);

```

E2.1.77 CPDef

```

1  // CPDef
2  // =====
3  // The CDE extension defines two encoding patterns
4
5  enumeration CPDef { CP_GCP,    // The architected coprocessor encodings for MRC, MCR, CDP and
6                          // other coprocessor instructions.
7                          CP_CDEv1 // Version 1 of the Custom Datapath Extension.
8                          };

```

E2.1.78 CreateException

```

1  // CreateException()
2  // =====
3
4  ExcInfo CreateException(integer exception, boolean forceSecurity,
5                          boolean isSecure, boolean isSynchronous)
6
7      // Work out the effective target Security state of the exception.
8      if HaveSecurityExt() then
9          if !forceSecurity then
10             isSecure = ExceptionTargetsSecure(exception, isSecure);
11      else
12          isSecure = FALSE;
13
14      // An implementation without Security Extensions cannot cause a fault targetting
15      // Secure state.
16      assert HaveSecurityExt() || !isSecure;
17
18      // Get the remaining exception details.
19      (escalateToHf, termInst) = ExceptionDetails(exception, isSecure, isSynchronous);
20
21      // Fill in the default exception info
22      info = DefaultExcInfo();
23      info.fault = exception;
24      info.termInst = termInst;
25      info.origFault = exception;

```

```

26     info.origFaultIsSecure = isSecure;
27
28     // Check for HardFault escalation
29     // In some cases (for example faults during lazy floating-point state preservation)
30     // the decision to escalate below is ignored and instead based on the info.origFault*
31     // fields and other factors.
32     if escalateToHf && info.fault != HardFault then
33         // Update the exception info with the escalation details, including
34         // whether there's a change in destination Security state.
35         info.fault      = HardFault;
36         isSecure        = ExceptionTargetsSecure(HardFault, isSecure);
37         (escalateToHf, -) = ExceptionDetails(HardFault, isSecure, isSynchronous);
38
39     // If the requested exception was already a HardFault then the PE cannot escalate
40     // to a HardFault, so lockup. NOTE: Asynchronous BusFaults never cause
41     // lockups, if the BusFault is disabled it escalates to a HardFault that is
42     // pending.
43     if escalateToHf && isSynchronous && info.fault == HardFault then
44         info.lockup = TRUE;
45
46     // Fill in the remaining exception info.
47     info.isSecure = isSecure;
48     return info;
49
50 ExcInfo CreateException(integer exception, boolean forceSecurity, boolean isSecure)
51     return CreateException(exception, forceSecurity, isSecure, TRUE);
52
53 ExcInfo CreateException(integer exception)
54     return CreateException(exception, FALSE, IsSecure(), TRUE);

```

E2.1.79 CreatePAC

```

1 // CreatePAC()
2 // =====
3 // Returns a 32-bit value containing the pointer authentication code, where the pointer
4 // authentication code is derived using a cryptographic algorithm as a combination of
5 // x, y and the cryptographic key. The cryptographic key is selected on the basis of the
6 // current Security state and privilege level.
7
8 bits(32) CreatePAC(bits(32) x, bits(32) y)
9     // Where <x> is the pointer and <y> is the modifier. Both are zero extended to 64 bits.
10     bits(64) extPtr  = Zeros(32) : x;
11     bits(64) modifier = Zeros(32) : y;
12
13     // Get the key to use. The top 64bits are the whitening key.
14     key = PACKey(IsSecure(), CurrentModeIsPrivileged());
15     // Return the 32 least significant bits of the computed PAC value.
16     return ComputePAC(extPtr, modifier, key[127:64], key[63:0])[31:0];

```

E2.1.80 CurrentCond

```

1 // CurrentCond()
2 // =====
3 // Returns condition specifier of current instruction.
4
5 bits(4) CurrentCond();

```

E2.1.81 CurrentMode

```

1 // CurrentMode()
2 // =====
3
4 PMode CurrentMode()
5     return if IPSR.Exception == NoFault[8:0] then PMode_Thread else PMode_Handler;

```

E2.1.82 CurrentModelsPrivileged

```

1 // CurrentModeIsPrivileged()
2 // =====
3
4 boolean CurrentModeIsPrivileged()
5     return CurrentModeIsPrivileged(IsSecure());
6
7 boolean CurrentModeIsPrivileged(boolean isSecure)
8     nPriv = if isSecure then CONTROL_S.nPRIV else CONTROL_NS.nPRIV;
9     return (CurrentMode() == PMode_Handler || nPriv == '0');
```

E2.1.83 CX_op0

```

1 // CX_op0
2 // =====
3
4 bits(size) CX_op0(bits(32) instr, integer size)
5     assert size IN {32, 64};
6
7     // Custom data path returning IMPLEMENTATION DEFINED value based on
8     // instruction opcode only.
9     return CdeImpDefValue(instr);
```

E2.1.84 CX_op1

```

1 // CX_op1
2 // =====
3
4 bits(size) CX_op1(bits(32) instr, bits(N) opa, integer size)
5     assert N IN {32, 64};
6     assert size IN {32, 64};
7
8     // Custom data path returning IMPLEMENTATION DEFINED value based on
9     // instruction opcode and single 32-bit or 64-bit operand, opa, only.
10    return CdeImpDefValue(instr, opa);
```

E2.1.85 CX_op2

```

1 // CX_op2
2 // =====
3
4 bits(size) CX_op2(bits(32) instr, bits(N) opa, bits(32) opb, integer size)
5     assert N IN {32, 64};
6     assert size IN {32, 64};
7
8     // Custom data path returning IMPLEMENTATION DEFINED value based on instruction
9     // opcode and two 32-bit or 64-bit operands, opa and opb, only.
10    return CdeImpDefValue(instr, opa, opb);
```

E2.1.86 CX_op3

```

1 // CX_op3
2 // =====
3
4 bits(size) CX_op3(bits(32) instr, bits(N) opa, bits(32) opb, bits(32) opc, integer size)
5     assert N IN {32, 64};
6     assert size IN {32, 64};
7
8     // Custom data path returning IMPLEMENTATION DEFINED value based on instruction
9     // opcode and three 32-bit or 64-bit operands, opa, opb and opc, only.
10    return CdeImpDefValue(instr, opa, opb, opc);
```

E2.1.87 D

```

1 // D[]
2 // ===
3
4 // Non-assignment form
5
6 bits(64) D[integer n]
7     assert n >= 0 && n <= 31;
8     return _S[(n*2)+1]:_S[n*2];
9
10 // Assignment form
11
12 D[integer n] = bits(64) value
13     assert n >= 0 && n <= 31;
14     _S[(n*2)+1] = value[63:32];
15     _S[n*2]     = value[31:0];
16     return;

```

E2.1.88 DAPCheck

```

1 // DAPCheck()
2 // =====
3
4 (boolean, boolean, boolean) DAPCheck(bits(32) address, boolean reqPriv, boolean reqSecure)
5
6     assert(HaveSecurityExt() || !reqSecure);
7
8     // DAP accesses to SCS registers select the bank to access based on DSCSR.SBRSEL*, and
9     // not the security attribute of the DAP access itself.
10    // If DHCSR.S_SDE is 0 then DSCSR.SBRSELEN reads as 1 and DSCSR.SBRSEL reads as 0. This
11    // forces the Non-secure bank of the registers to be read if Secure debug is not
12    // permitted.
13    if IsSCS(address) then
14        if DSCSR.SBRSELEN == '0' then
15            isSecure = IsSecure();
16        else
17            isSecure = DSCSR.SBRSEL == '1';
18    else
19        // DAP access falls back to Nonsecure when secure debug disabled.
20        isSecure = reqSecure && DHCSR.S_SDE == '1';
21
22    udeEn = (isSecure && DHCSR.S_SUIDE == '1') || (!isSecure && DHCSR.S_NSUIDE == '1');
23    isPriv = reqPriv && !udeEn;
24    err = TRUE; // Deny-default policy, with exemptions below.
25
26    // Allow access if halting debug or UDE permissions are available, and let the
27    // target handle access based on privilege/security.
28    if HaltingDebugEnabled() || udeEn then
29        err = FALSE;
30
31    elseif IsPPB(address) then
32        // Handle accesses based on NonInvasiveDebugEnabled or
33        // region-specific rules.
34        niden = !NoninvasiveDebugEnabled(TRUE);
35        case address of
36            when '1110 0000 0000 xxxx xxxx 1111 1011 0xxx'
37                err = niden; // CoreSight software lock
38            when '1110 0000 0000 xxxx xxxx 1111 1101 xxxx'
39                err = FALSE; // All ID registers RO
40            when '1110 0000 0000 xxxx xxxx 1111 111x xxxx'
41                err = FALSE; // All ID registers RO
42            when '1110 0000 0000 0000 0000 xxxx xxxx xxxx'
43                err = niden; // ITM
44            when '1110 0000 0000 0000 0001 xxxx xxxx xxxx'
45                err = niden; // DWT
46            when '1110 0000 0000 0000 0011 xxxx xxxx xxxx'

```

```

47         err = niden; // PMU
48     when '1110 0000 0000 0100 0000 xxxx xxxx xxxx'
49         err = FALSE; // TPIU
50     when '1110 0000 0000 0100 0001 xxxx xxxx xxxx'
51         err = FALSE; // ETM
52     when '1110 0000 0000 1111 1111 xxxx xxxx xxxx'
53         err = FALSE; // ROM Table
54     otherwise
55         address_uint = UInt(address);
56         if address_uint >= 0xE0042000 && address_uint <= 0xE00FEFFF then
57             err = boolean IMPLEMENTATION_DEFINED "IMPDEF DAP region";
58
59         if DAUTHCTRL_S.UIDAPEN == '1' || DAUTHCTRL_NS.UIDAPEN == '1' then
60             err = FALSE;
61             isPriv = FALSE;
62
63     else // NOT PPB
64         if UInt(address) >= 0xE0100000 then
65             err = !NoninvasiveDebugAllowed(TRUE);
66
67         if DAUTHCTRL_S.UIDAPEN == '1' || DAUTHCTRL_NS.UIDAPEN == '1' then
68             err = TRUE;
69             isPriv = FALSE;
70
71     return (isSecure, isPriv, err);

```

E2.1.89 DataMemoryBarrier

```

1 // DataMemoryBarrier()
2 // =====
3 // Perform a Data Memory Barrier operation.
4
5 DataMemoryBarrier(bits(4) option);

```

E2.1.90 DataSynchronizationBarrier

```

1 // DataSynchronizationBarrier
2 // =====
3 // Perform a data synchronization barrier operation.
4
5 DataSynchronizationBarrier(bits(4) option);

```

E2.1.91 DeActivate

```

1 // DeActivate()
2 // =====
3
4 DeActivate(integer returningExceptionNumber, boolean targetDomainSecure)
5     // To prevent the execution priority remaining negative (and therefore
6     // masking HardFault) when returning from NMI or HardFault with a corrupted
7     // IPSR value, the active bits corresponding to the execution priority are
8     // cleared if the raw execution priority (in other words the priority before
9     // FAULTMASK and other priority boosting is considered) is negative.
10    rawPri = RawExecutionPriority();
11    if rawPri == -1 then
12        SetActive(HardFault, AIRCR.BFHFNMINS == '0', FALSE);
13    elseif rawPri == -2 then
14        SetActive(NMI, AIRCR.BFHFNMINS == '0', FALSE);
15    elseif rawPri == -3 then
16        SetActive(HardFault, TRUE, FALSE);
17    else
18        secure = HaveSecurityExt() && targetDomainSecure;
19        SetActive(returningExceptionNumber, secure, FALSE);
20
21    /* PRIMASK and BASEPRI unchanged on exception exit */

```

```

22     if HaveMainExt() && rawPri >= 0 then
23         // Clear FAULTMASK for exception Security domain on any return except
24         // NMI and HardFault.
25         if HaveSecurityExt() && targetDomainSecure then
26             FAULTMASK_S.FM = '0';
27         else
28             FAULTMASK_NS.FM = '0';
29     return;

```

E2.1.92 Debug_authentication

```

1 // In the recommended CoreSight interface, there are four signals for external debug
2 // authentication, DBGGEN, SPIDEN, NIDEN and SPNIDEN. Each signal is active-HIGH.
3
4 signal DBGGEN;
5 signal SPIDEN;
6 signal NIDEN;
7 signal SPNIDEN;

```

E2.1.93 DebugCanMaskInts

```

1 // DebugCanMaskInts()
2 // =====
3
4 boolean DebugCanMaskInts(boolean secure)
5     if !HaltingDebugAllowed() || DHCSR.C_DEBUGEN == '0' then
6         return FALSE;
7     elsif secure && DHCSR.S_SDE == '0' then
8         return FALSE;
9     elsif HaveUDE() && secure && DHCSR.S_SUIDE == '1' then
10        return FALSE;
11    elsif HaveUDE() && DHCSR.S_NSUIDE == '1' then
12        return FALSE;
13    else
14        return DHCSR.C_MASKINTS == '1';

```

E2.1.94 DebugEventCause

```

1 // Causes for debug event generation.
2
3 enumeration DebugEventCause {DebugEventCause_BKPT, DebugEventCause_FPB, DebugEventCause_EXT};

```

E2.1.95 DebugRegisterTransfer

```

1 // DebugRegisterTransfer()
2 // =====
3
4 DebugRegisterTransfer(bits(8) reg, boolean isWrite)
5     isSecure = IsSecure();
6     unprivDbgS = HaveUDE() && DHCSR.S_SUIDE == '1';
7     unprivDbgNS = HaveUDE() && DHCSR.S_NSUIDE == '1';
8     unprivDbg = if isSecure then unprivDbgS else unprivDbgNS;
9
10    if ((UInt(reg) >= UInt(DCRSR_REGSEL_R_LOW) && UInt(reg) <= UInt(DCRSR_REGSEL_R_HIGH)) ||
11        reg == DCRSR_REGSEL_LR) then
12        if isWrite then
13            R[UInt(reg)] = DCRDR;
14        else
15            DCRDR = R[UInt(reg)];
16
17    elsif reg == DCRSR_REGSEL_SP then
18        if isWrite then
19            // This requires skipping stack limit checking,
20            // hence a direct _RName access is used.

```

```

21     _RName[LookupRName(UInt(reg))] = DCRDR[31:2]:'00';
22     else
23         DCRDR = _RName[LookupRName(UInt(reg))];
24
25     elsif reg == DCRSR_REGSEL_DBGRETADDR then
26         if isWrite then
27             BranchTo(DCRDR, TRUE, FALSE);
28         else
29             DCRDR = _RName[RNamesPC];
30
31     elsif reg == DCRSR_REGSEL_XPSR then
32         if isWrite then
33             EAPSR = DCRDR[31:0];
34         else
35             DCRDR[31:0] = XPSR; // When accessed in unprivileged state the value
36                                 // of IPSR is 0.
37
38     elsif reg == DCRSR_REGSEL_SP_MAIN then
39         if isSecure then
40             spName = RNamesSP_Main_Secure;
41         else
42             spName = RNamesSP_Main_NonSecure;
43         if isWrite then
44             if !unprivDbg then
45                 - = _SP(spName, FALSE, TRUE, DCRDR);
46             else
47                 if !unprivDbg then
48                     DCRDR = _SP(spName);
49                 else
50                     DCRDR = Zeros();
51
52     elsif reg == DCRSR_REGSEL_SP_PROCESS then
53         if isSecure then
54             spName = RNamesSP_Process_Secure;
55         else
56             spName = RNamesSP_Process_NonSecure;
57         if isWrite then
58             - = _SP(spName, FALSE, TRUE, DCRDR);
59         else
60             DCRDR = _SP(spName);
61
62     elsif reg == DCRSR_REGSEL_STATE then
63         if isWrite then
64             if !unprivDbg then
65                 CONTROL[7:0] = DCRDR[31:24];
66                 if HaveMainExt() then
67                     FAULTMASK[7:0] = DCRDR[23:16];
68                     BASEPRI[7:0] = DCRDR[15:8];
69                     PRIMASK[7:0] = DCRDR[7:0];
70             else
71                 CONTROL.SFPA = DCRDR[27]; // NOTE: SFPA is RAZ/WI when PE is in NS state.
72                 CONTROL.FPCA = DCRDR[26];
73                 if HavePACBTIExt() then
74                     CONTROL.UPAC_EN = DCRDR[31];
75                     CONTROL.UBTI_EN = DCRDR[29];
76             else
77                 if !unprivDbg then
78                     DCRDR[31:24] = CONTROL[7:0];
79                     if HaveMainExt() then
80                         DCRDR[23:16] = FAULTMASK[7:0];
81                         DCRDR[15:8] = BASEPRI[7:0];
82                     else
83                         DCRDR[23:8] = Zeros(16);
84                         DCRDR[7:0] = PRIMASK[7:0];
85                 else
86                     CONTROL_Type control = Zeros(32);
87                     control.SFPA = CONTROL.SFPA; // NOTE: SFPA is RAZ/WI when PE is in NS state.
88                     control.FPCA = CONTROL.FPCA;
89                     if HavePACBTIExt() then

```

```

90         control.UPAC_EN = CONTROL.UPAC_EN;
91         control.UBTI_EN = CONTROL.UBTI_EN;
92         DCRDR = control[7:0] : Zeros(24);
93
94     elsif reg == DCRSR_REGSEL_MSP_NS && HaveSecurityExt() then
95         if isWrite then
96             // Unprivileged-only debug is restricted even if MSP is being used by
97             // unprivileged execution, a safe restriction that removes the
98             // requirement to check other conditions here.
99             if !unprivDbgNS then
100                 - = _SP(RNamesSP_Main_NonSecure, FALSE, TRUE, DCRDR);
101             else
102                 if !unprivDbgNS then
103                     DCRDR = _SP(RNamesSP_Main_NonSecure);
104                 else
105                     DCRDR[31:0] = Zeros();
106
107     elsif reg == DCRSR_REGSEL_PSP_NS && HaveSecurityExt() then
108         if isWrite then
109             - = _SP(RNamesSP_Process_NonSecure, FALSE, TRUE, DCRDR);
110         else
111             DCRDR = _SP(RNamesSP_Process_NonSecure);
112
113     elsif reg == DCRSR_REGSEL_MSP_S && HaveSecurityExt() then
114         if isWrite then
115             // Unprivileged-only debug is restricted even if MSP is being used by
116             // unprivileged execution, a safe restriction that removes the
117             // requirement to check other conditions here.
118             if DHCSR.S_SDE == '1' && !unprivDbgS then
119                 - = _SP(RNamesSP_Main_Secure, FALSE, TRUE, DCRDR);
120             else
121                 if DHCSR.S_SDE == '1' && !unprivDbgS then
122                     DCRDR = _SP(RNamesSP_Main_Secure);
123                 else
124                     DCRDR = Zeros(32);
125
126     elsif reg == DCRSR_REGSEL_PSP_S && HaveSecurityExt() then
127         if isWrite then
128             if DHCSR.S_SDE == '1' then
129                 - = _SP(RNamesSP_Process_Secure, FALSE, TRUE, DCRDR);
130             else
131                 if DHCSR.S_SDE == '1' then
132                     DCRDR = _SP(RNamesSP_Process_Secure);
133                 else
134                     DCRDR = Zeros(32);
135
136     elsif reg == DCRSR_REGSEL_MSPLIM_S && HaveSecurityExt() then
137         if isWrite then
138             if DHCSR.S_SDE == '1' && !unprivDbgS then
139                 MSPLIM_S = DCRDR[31:0];
140             else
141                 if DHCSR.S_SDE == '1' && !unprivDbgS then
142                     DCRDR[31:0] = MSPLIM_S;
143                 else
144                     DCRDR = Zeros(32);
145
146     elsif reg == DCRSR_REGSEL_PSPLIM_S && HaveSecurityExt() then
147         if isWrite then
148             if DHCSR.S_SDE == '1' then
149                 PSPLIM_S = DCRDR[31:0];
150             else
151                 if DHCSR.S_SDE == '1' then
152                     DCRDR[31:0] = PSPLIM_S;
153                 else
154                     DCRDR = Zeros(32);
155
156     elsif reg == DCRSR_REGSEL_MSPLIM_NS && HaveMainExt() then
157         if isWrite then
158             if !unprivDbgNS then

```



```

159         MSPLIM_NS = DCRDR[31:0];
160     else
161         if !unprivDbgNS then
162             DCRDR[31:0] = MSPLIM_NS;
163         else
164             DCRDR[31:0] = Zeros();
165
166     elsif reg == DCRSR_REGSEL_PSPLIM_NS && HaveMainExt() then
167         if isWrite then
168             PSPLIM_NS = DCRDR[31:0];
169         else
170             DCRDR[31:0] = PSPLIM_NS;
171
172     elsif reg == DCRSR_REGSEL_FPSCR && (HaveFPExt() || HaveMve()) then
173         if isWrite then
174             if CanDebugAccessFP() then
175                 FPSCR = DCRDR[31:0];
176             else
177                 if CanDebugAccessFP() then
178                     DCRDR[31:0] = FPSCR;
179                 else
180                     DCRDR = Zeros(32);
181
182     elsif reg == DCRSR_REGSEL_STATE_S && HaveSecurityExt() then
183         if isWrite then
184             if DHCSR.S_SDE == '1' then
185                 if !unprivDbgS then
186                     CONTROL_S[7:0] = DCRDR[31:24];
187                     if HaveMainExt() then
188                         FAULTMASK_S[7:0] = DCRDR[23:16];
189                         BASEPRI_S[7:0] = DCRDR[15:8];
190                         PRIMASK_S[7:0] = DCRDR[7:0];
191                     else
192                         CONTROL_S.SFPA = DCRDR[27];
193                         CONTROL_S.FPCA = DCRDR[26];
194                         if HavePACBTIExt() then
195                             CONTROL_S.UPAC_EN = DCRDR[31];
196                             CONTROL_S.UBTI_EN = DCRDR[29];
197                     else
198                         if DHCSR.S_SDE == '1' then
199                             if !unprivDbgS then
200                                 DCRDR[31:24] = CONTROL_S[7:0];
201                                 if HaveMainExt() then
202                                     DCRDR[23:16] = FAULTMASK_S[7:0];
203                                     DCRDR[15:8] = BASEPRI_S[7:0];
204                                 else
205                                     DCRDR[23:8] = Zeros(16);
206                                     DCRDR[7:0] = PRIMASK_S[7:0];
207                             else
208                                 CONTROL_Type control = Zeros(32);
209                                 control.SFPA = CONTROL_S.SFPA;
210                                 control.FPCA = CONTROL_S.FPCA;
211                                 if HavePACBTIExt() then
212                                     control.UPAC_EN = CONTROL_S.UPAC_EN;
213                                     control.UBTI_EN = CONTROL_S.UBTI_EN;
214                                 DCRDR = control[7:0] : Zeros(24);
215                         else
216                             DCRDR = Zeros(32);
217
218     elsif reg == DCRSR_REGSEL_STATE_NS && HaveSecurityExt() then
219         if isWrite then
220             if !unprivDbgNS then
221                 CONTROL_NS[7:0] = DCRDR[31:24]; // NOTE: SFPA is RAZ/WI.
222                 if HaveMainExt() then
223                     FAULTMASK_NS[7:0] = DCRDR[23:16];
224                     BASEPRI_NS[7:0] = DCRDR[15:8];
225                     PRIMASK_NS[7:0] = DCRDR[7:0];
226                 else
227                     CONTROL_NS.FPCA = DCRDR[26];

```

```

228         if HavePACBTIExt() then
229             CONTROL_NS.UPAC_EN = DCRDR[31];
230             CONTROL_NS.UBTI_EN = DCRDR[29];
231     else
232         if !unprivDbgNS then
233             DCRDR[31:24] = CONTROL_NS[7:0]; // NOTE: SFPA is RAZ/WI.
234             if HaveMainExt() then
235                 DCRDR[23:16] = FAULTMASK_NS[7:0];
236                 DCRDR[15:8] = BASEPRI_NS[7:0];
237             else
238                 DCRDR[23:8] = Zeros(16);
239                 DCRDR[7:0] = PRIMASK_NS[7:0];
240         else
241             CONTROL_Type control = Zeros(32);
242             control.FPCA = CONTROL_NS.FPCA;
243             if HavePACBTIExt() then
244                 control.UPAC_EN = CONTROL_NS.UPAC_EN;
245                 control.UBTI_EN = CONTROL_NS.UBTI_EN;
246             DCRDR = control[7:0] : Zeros(24);
247
248     elseif reg == DCRSR_REGSEL_VPR && HaveMve() then
249         if isWrite then
250             if CanDebugAccessFP() then
251                 VPR = DCRDR[31:0];
252             else
253                 if CanDebugAccessFP() then
254                     DCRDR = VPR[31:0];
255                 else
256                     DCRDR = Zeros(32);
257
258     elseif (UInt(reg) >= UInt(DCSR_REGSEL_S_LOW) &&
259             UInt(reg) <= UInt(DCSR_REGSEL_S_HIGH) && (HaveFPEExt() || HaveMve())) then
260         if isWrite then
261             if CanDebugAccessFP() then
262                 _S[UInt(reg[5:0])] = DCRDR;
263             else
264                 if CanDebugAccessFP() then
265                     DCRDR = _S[UInt(reg[5:0])];
266                 else
267                     DCRDR = Zeros(32);
268
269     elseif (UInt(reg) >= UInt(DCSR_REGSEL_PAC_KEY_P_LOW) &&
270             UInt(reg) <= UInt(DCSR_REGSEL_PAC_KEY_U_HIGH) && HavePACBTIExt()) then
271         if isWrite then
272             if !unprivDbg then
273                 PACKeyReg[reg[2:0], isSecure] = DCRDR;
274             else
275                 if !unprivDbg then
276                     DCRDR = PACKeyReg[reg[2:0], isSecure];
277                 else
278                     DCRDR = Zeros();
279
280     elseif (UInt(reg) >= UInt(DCSR_REGSEL_PAC_KEY_P_S_LOW) &&
281             UInt(reg) <= UInt(DCSR_REGSEL_PAC_KEY_U_S_HIGH) &&
282             HavePACBTIExt() && HaveSecurityExt()) then
283         if isWrite then
284             if DHCSR.S_SDE == '1' && !unprivDbgS then
285                 PACKeyReg[reg[2:0], TRUE] = DCRDR;
286             else
287                 if DHCSR.S_SDE == '1' && !unprivDbgS then
288                     DCRDR = PACKeyReg[reg[2:0], TRUE];
289                 else
290                     DCRDR = Zeros();
291
292     elseif (UInt(reg) >= UInt(DCSR_REGSEL_PAC_KEY_P_NS_LOW) &&
293             UInt(reg) <= UInt(DCSR_REGSEL_PAC_KEY_U_NS_HIGH) &&
294             HavePACBTIExt() && HaveSecurityExt()) then
295         if isWrite then
296             if !unprivDbgNS then

```

```

297     PACKeyReg[reg[2:0], FALSE] = DCRDR;
298     else
299         if !unprivDbgNS then
300             DCRDR = PACKeyReg[reg[2:0], FALSE];
301         else
302             DCRDR = Zeros();
303
304     else
305         DCRDR = bits(32) UNKNOWN;

```

E2.1.96 DecodeExecute

```

1  // DecodeExecute
2  // =====
3  // Decode instruction and execute.
4
5  DecodeExecute(bits(32) instr, bits(32) pc, boolean isT16, bits(4) defaultCond);

```

E2.1.97 DecodeImmShift

```

1  // DecodeImmShift()
2  // =====
3
4  (SRTYPE, integer) DecodeImmShift(bits(2) sr_type, bits(5) imm5)
5
6      case sr_type of
7          when '00'
8              shift_t = SRTYPE_LSL; shift_n = UInt(imm5);
9          when '01'
10             shift_t = SRTYPE_LSR; shift_n = if imm5 == '00000' then 32 else UInt(imm5);
11          when '10'
12             shift_t = SRTYPE_ASR; shift_n = if imm5 == '00000' then 32 else UInt(imm5);
13          when '11'
14             if imm5 == '00000' then
15                 shift_t = SRTYPE_RRX; shift_n = 1;
16             else
17                 shift_t = SRTYPE_ROR; shift_n = UInt(imm5);
18
19      return (shift_t, shift_n);

```

E2.1.98 DecodeRegShift

```

1  // DecodeRegShift()
2  // =====
3
4  SRTYPE DecodeRegShift(bits(2) sr_type)
5      case sr_type of
6          when '00' shift_t = SRTYPE_LSL;
7          when '01' shift_t = SRTYPE_LSR;
8          when '10' shift_t = SRTYPE_ASR;
9          when '11' shift_t = SRTYPE_ROR;
10     return shift_t;

```

E2.1.99 DefaultCond

```

1  // DefaultCond()
2  // =====
3
4  bits(4) DefaultCond()
5      // If in an IT block use the IT condition, otherwise set the condition to
6      // always (in other words 0xE).
7      // NOTE: This is only the default condition, as it may be overridden by an
8      // explicit condition code in the instruction itself.
9      if ITSTATE[3:0] == Zeros(4) then

```

```

10     cond = 0xE[3:0];
11     else
12         cond = ITSTATE[7:4];
13     return cond;

```

E2.1.100 DefaultExcInfo

```

1  // DefaultExcInfo()
2  // =====
3
4  ExcInfo DefaultExcInfo()
5      ExcInfo exc;
6
7      exc.fault                = NoFault;
8      exc.origFault           = NoFault;
9      exc.derivedFrom         = NoFault;
10     exc.isSecure             = boolean UNKNOWN;
11     exc.origFaultIsSecure    = boolean UNKNOWN;
12     exc.derivedFromIsSecure = boolean UNKNOWN;
13     exc.isTerminal           = FALSE;
14     exc.inExcTaken           = FALSE;
15     exc.lockup                = FALSE;
16     exc.termInst             = TRUE;
17     return exc;

```

E2.1.101 DefaultMemoryAttributes

```

1  // DefaultMemoryAttributes()
2  // =====
3
4  MemoryAttributes DefaultMemoryAttributes(bits(32) address)
5
6      MemoryAttributes memattrs;
7
8      case address[31:29] of
9          when '000'
10             memattrs.memtype = MemType_Normal;
11             memattrs.device = DeviceType UNKNOWN;
12             memattrs.innerattrs = '10';
13             memattrs.innerhints = '10';
14             memattrs.shareable = FALSE;
15          when '001'
16             memattrs.memtype = MemType_Normal;
17             memattrs.device = DeviceType UNKNOWN;
18             memattrs.innerattrs = '11';
19             memattrs.innerhints = '11';
20             memattrs.shareable = FALSE;
21          when '010'
22             memattrs.memtype = MemType_Device;
23             memattrs.device = DeviceType_nGnRE;
24             memattrs.innerattrs = '00';
25             memattrs.innerhints = '00';
26             memattrs.shareable = TRUE;
27          when '011'
28             memattrs.memtype = MemType_Normal;
29             memattrs.device = DeviceType UNKNOWN;
30             memattrs.innerattrs = '11';
31             memattrs.innerhints = '11';
32             memattrs.shareable = FALSE;
33          when '100'
34             memattrs.memtype = MemType_Normal;
35             memattrs.device = DeviceType UNKNOWN;
36             memattrs.innerattrs = '10';
37             memattrs.innerhints = '10';
38             memattrs.shareable = FALSE;
39          when '101'
40             memattrs.memtype = MemType_Device;

```

```

41         memattrs.device = DeviceType_nGnRE;
42         memattrs.innerattrs = '00';
43         memattrs.innerhints = '00';
44         memattrs.shareable = TRUE;
45     when '110'
46         memattrs.memtype = MemType_Device;
47         memattrs.device = DeviceType_nGnRE;
48         memattrs.innerattrs = '00';
49         memattrs.innerhints = '00';
50         memattrs.shareable = TRUE;
51     when '111'
52         if address[28:20] == '00000000' then
53             memattrs.memtype = MemType_Device;
54             memattrs.device = DeviceType_nGnRnE;
55             memattrs.innerattrs = '00';
56             memattrs.innerhints = '00';
57             memattrs.shareable = TRUE;
58         else
59             memattrs.memtype = MemType_Device;
60             memattrs.device = DeviceType_nGnRE;
61             memattrs.innerattrs = '00';
62             memattrs.innerhints = '00';
63             memattrs.shareable = TRUE;
64
65     // Outer attributes are the same as the inner attributes in all cases.
66     memattrs.outerattrs = memattrs.innerattrs;
67     memattrs.outerhints = memattrs.innerhints;
68     memattrs.outershareable = memattrs.shareable;
69     memattrs.innertransient = FALSE;
70     memattrs.outertransient = FALSE;
71
72     // Setting as UNKNOWN by default. This flag will be overwritten based on
73     // SAU and IDAU checking in SecurityCheck().
74     memattrs.NS = boolean UNKNOWN;
75     return memattrs;

```

E2.1.102 DefaultPermissions

```

1  // DefaultPermissions()
2  // =====
3
4  Permissions DefaultPermissions(bits(32) address)
5
6      Permissions perms;
7
8      perms.ap          = '01';
9      perms.apValid     = TRUE;
10     perms.region      = Zeros(8);
11     perms.regionValid = FALSE;
12
13     case address[31:29] of
14         when '000'
15             perms.xn = '0';
16         when '001'
17             perms.xn = '0';
18         when '010'
19             perms.xn = '1';
20         when '011'
21             perms.xn = '0';
22         when '100'
23             perms.xn = '0';
24         when '101'
25             perms.xn = '1';
26         when '110'
27             perms.xn = '1';
28         when '111'
29             perms.xn = '1';
30

```

31 **return** perms;

E2.1.103 DerivedLateArrival

```

1  // DerivedLateArrival()
2  // =====
3
4  DerivedLateArrival(integer pePriority, integer peNumber, boolean peIsSecure, ExcInfo deInfo,
5                     integer oeNumber, boolean oeIsSecure, EXC_RETURN_Type excReturn)
6      // PE: the pre-empted exception - before exception entry.
7      // OE: the original exception - exception entry.
8      // DE: the derived exception - fault on exception entry.
9
10     // Get the priorities of the exceptions
11     // xePriority: the lower the value, the higher the priority.
12     oePriority = ExceptionPriority(oeNumber, oeIsSecure, FALSE);
13     // NOTE: Comparison of dePriority against PE priority and possible
14     // escalation to HardFault has already occurred. See CreateException().
15
16     // Work out which fault to take, and what the target Security state is.
17     if deInfo.isTerminal then
18         // Derived exception is terminal and prevents the original exception
19         // being taken (for example fault on vector fetch). As a result the
20         // derived exception is treated as a HardFault.
21         targetIsSecure = deInfo.isSecure;
22         targetFault    = deInfo.fault;
23         // If the derived fault does not have sufficient priority to pre-empt
24         // lockup instead of taking it.
25         if !ComparePriorities(deInfo, FALSE, oePriority, oeNumber, oeIsSecure) then
26             _ = ExceptionTaken(oeNumber, deInfo.inExcTaken, oeIsSecure,
27                               IgnoreFaults_ALL, excReturn);
28             // Since execution of original exception cannot be started, lockup
29             // at the current priority level. That is the priority of the original
30             // exception.
31             Lockup(TRUE);
32     elseif ComparePriorities(deInfo, FALSE, oePriority, oeNumber, oeIsSecure) then
33         // Derive exception has a higher priority (that is a lower value) than the
34         // original exception, so the derived exception first. Tail-chaining is
35         // IMPLEMENTATION DEFINED.
36         targetFault    = deInfo.fault;
37         targetIsSecure = deInfo.isSecure;
38     else
39         // If the derived exception caused a lockup then this must be handled
40         // now as the lockup cannot be pended until the original exception
41         // returns.
42         if deInfo.lockup then
43             // Lockup at the priority of the original exception being entered.
44             _ = ExceptionTaken(oeNumber, deInfo.inExcTaken, oeIsSecure,
45                               IgnoreFaults_ALL, excReturn);
46             Lockup(TRUE);
47         else
48             // DE will be pended below, start execution of the OE.
49             targetFault    = oeNumber;
50             targetIsSecure = oeIsSecure;
51
52     // If none of the tests above have triggered a lockup (which would have
53     // terminated execution of the pseudocode) then the derived exception
54     // must be pended and any escalation syndrome info generated.
55     if HaveMainExt() &&
56         (deInfo.fault == HardFault) &&
57         (deInfo.origFault != HardFault) then
58         HFSR.FORCED = '1';
59     SetPending(deInfo.fault, deInfo.isSecure, TRUE);
60
61     // Take the target exception. NOTE: None terminal faults are ignored when
62     // handling the derived exception, allowing forward progress to be made.
63     (excInfo, excReturn) = ExceptionTaken(targetFault, deInfo.inExcTaken,
64                                           targetIsSecure, IgnoreFaults_STACK, excReturn);

```

```

65 // If trying to take the resulting exception results in another fault, then handle
66 // the derived fault.
67 if excInfo.fault != NoFault then
68     DerivedLateArrival(pePriority, peNumber, peIsSecure, excInfo, targetFault,
69                       targetIsSecure, excReturn);

```

E2.1.104 DeviceType

```

1 // Types of memory
2
3 enumeration DeviceType {DeviceType_GRE, DeviceType_nGRE,
4                       DeviceType_nGnRE, DeviceType_nGnRnE};

```

E2.1.105 DWT_AddressCompare

```

1 // DWT_AddressCompare()
2 // =====
3 // Returns a pair of values. The first result is whether the (masked) addresses are equal,
4 // where the access address (addr) is masked according to DWT_FUNCTION[n].DATAVSIZ and the
5 // comparator address (compaddr) is masked according to the access size. The second result
6 // is whether the (unmasked) addr is greater than the (unmasked) compaddr.
7
8 (boolean,boolean) DWT_AddressCompare(bits(32) addr, bits(32) compaddr, integer size,
9                                     integer compsize)
10 // addr must be a multiple of size. Unaligned accesses are split into smaller accesses.
11 assert Align(addr, size) == addr;
12
13 // compaddr must be a multiple of compsize
14 if Align(compaddr, compsize) != compaddr then UNPREDICTABLE;
15
16 addrmatch = (Align(addr, compsize) == Align(compaddr, size));
17 addrgreater = (UInt(addr) > UInt(compaddr));
18 return (addrmatch, addrgreater);

```

E2.1.106 DWT_CycCountMatch

```

1 // DWT_CycCountMatch
2 // =====
3 // Check for DWT cycle count match. This is called for each increment of
4 // DWT_CYCCNT.
5
6 DWT_CycCountMatch()
7     boolean trigger_debug_event = FALSE;
8     boolean debug_event = FALSE;
9     N = UInt(DWT_CTRL.NUMCOMP);
10    if N == 0 then return; // No comparator support
11    secure_match = IsSecure() && DWT_CTRL.CYCDISS == '1';
12    for i = 0 to N-1
13        if IsDWTConfigUnpredictable(i) then UNPREDICTABLE;
14        if DWT_FUNCTION[i].MATCH == '0001' && DWT_ValidMatch(i, secure_match, FindPriv())
15            && DWT_CYCCNT == DWT_COMP[i] then
16            DWT_FUNCTION[i].MATCHED = '1';
17            debug_event = DWT_FUNCTION[i].ACTION == '01';
18            trigger_debug_event = trigger_debug_event || debug_event;
19
20    // Setting the debug event if at least one comparator matches.
21    if trigger_debug_event then
22        // There is no special behaviour required for a cycle count match,
23        // so treat it as if it were a normal memory access.
24        debug_event = SetDWTDebugEvent(secure_match, FindPriv(), AccType_NORMAL);
25    return;

```

E2.1.107 DWT_DataAddressMatch

```

1 // DWT_DataAddressMatch()
2 // =====
3 // Check for match of access at "daddr". "dsize", "read", "NSreq", and "privilege" are the
4 // attributes for the access. Note that for a load or store instruction, "NSreq" is the
5 // current Security state of the PE, but this is not necessarily true for a hardware stack
6 // push/pop or vector table access. "NSreq" might not be the same as the "NSattr"
7 // attribute the PE finally uses to make the access.
8 // If comparators 'm' and 'm+1' form an Data Address Range comparator, then this function
9 // returns the range match result when N=m+1.
10
11 boolean DWT_DataAddressMatch(integer N, bits(32) daddr, integer dsize, boolean read,
12                             boolean NSreq, boolean priv_match)
13     assert N < UInt(DWT_CTRL.NUMCOMP) && dsize IN {1,2,4} && Align(daddr, dsize) == daddr;
14
15     valid_match = DWT_ValidMatch(N, !NSreq, priv_match);
16     valid_addr = DWT_FUNCTION[N].MATCH == 'x1xx';
17
18     if valid_match && valid_addr then
19         if N != UInt(DWT_CTRL.NUMCOMP)-1 then
20             linked_to_addr = DWT_FUNCTION[N+1].MATCH == '0111';    // Data Address Limit
21         else
22             linked_to_addr = FALSE;
23
24         case DWT_FUNCTION[N].MATCH[1:0] of
25             when '00' match_lsc = TRUE;    linked = FALSE;
26             when '01' match_lsc = !read;    linked = FALSE;
27             when '10' match_lsc = read;    linked = FALSE;
28             when '11'
29
30                 case DWT_FUNCTION[N-1].MATCH[1:0] of
31                     when '00' match_lsc = TRUE;    linked = TRUE;
32                     when '01' match_lsc = !read;    linked = TRUE;
33                     when '10' match_lsc = read;    linked = TRUE;
34
35             if !linked_to_addr then
36                 vsize = 2^UInt(DWT_FUNCTION[N].DATAVSIZE);
37                 (match_eq, match_gt) = DWT_AddressCompare(daddr, DWT_COMP[N], dsize, vsize);
38
39             if linked then
40                 valid_match = DWT_ValidMatch(N-1, !NSreq, priv_match);
41                 (lower_eq, lower_gt) = DWT_AddressCompare(daddr, DWT_COMP[N-1], dsize, 1);
42                 match_addr = valid_match && (lower_eq || lower_gt) && !match_gt;
43             else
44                 match_addr = match_eq;
45         else
46             match_addr = FALSE;
47
48         match = match_addr && match_lsc;
49     else
50         match = FALSE;
51
52     return match;

```

E2.1.108 DWT_DataMatch

```

1 // DWT_DataMatch()
2 // =====
3 // Perform varioius Data match checks for DWT.
4
5 DWT_DataMatch(bits(32) daddr, integer dsize, bits(32) dvalue, boolean read, boolean NSreq,
6              boolean priv_match, AccType acctype)
7
8     boolean trigger_debug_event = FALSE;
9     boolean debug_event = FALSE;
10
11     if !HaveDWT() || IsZero(DWT_CTRL.NUMCOMP) then return; // No comparator support
12
13     for i = 0 to UInt(DWT_CTRL.NUMCOMP) - 1

```



```

14     if IsDWTConfigUnpredictable(i) then UNPREDICTABLE;
15     daddr_match = DWT_DataAddressMatch(i, daddr, dsize, read, NSreq, priv_match);
16     dvalue_match = DWT_DataValueMatch(i, daddr, dvalue, dsize, read, NSreq, priv_match);
17
18     // Data Address and Data Address Limit.
19     if daddr_match && DWT_FUNCTION[i].MATCH == '01xx' then
20         // Data Address.
21         if DWT_FUNCTION[i].MATCH != '0111' then
22             DWT_FUNCTION[i].MATCHED = '1';
23             debug_event = DWT_FUNCTION[i].ACTION == '01';
24
25         // Data Address with Data Address Limit.
26         else
27             //ith comparator.
28             DWT_FUNCTION[i].MATCHED = bit UNKNOWN;
29             // (i-1)th comparator.
30             DWT_FUNCTION[i-1].MATCHED = '1';
31             debug_event = DWT_FUNCTION[i-1].ACTION == '01';
32
33     // Data Value and Linked Data Value.
34     if dvalue_match && DWT_FUNCTION[i].MATCH == '10xx' then
35         // Data Value.
36         if DWT_FUNCTION[i].MATCH != '1011' then
37             DWT_FUNCTION[i].MATCHED = '1';
38             debug_event = DWT_FUNCTION[i].ACTION == '01';
39
40         // For Linked Data Value, daddr_match will be TRUE for [i-1].
41         else
42             DWT_FUNCTION[i].MATCHED = '1';
43             debug_event = DWT_FUNCTION[i].ACTION == '01';
44
45     // Data Address with Value.
46     if daddr_match && DWT_FUNCTION[i].MATCH == '11xx' then
47         DWT_FUNCTION[i].MATCHED = '1';
48         // No debug_event generated in the case of Data Address with Value.
49
50     trigger_debug_event = trigger_debug_event || debug_event;
51
52     // Setting the debug event if at least one comparator matches.
53     if trigger_debug_event then
54         debug_event = SetDWTDebugEvent(!NSreq, priv_match, acctype);
55
56     return;

```

E2.1.109 DWT_DataValueMatch

```

1  // DWT_DataValueMatch()
2  // =====
3  // Check for match of "dvalue" at "daddr". "dsize", "read" and "NSreq"
4  // and "priv_match" are the attributes for the access. Note that for a load or store
5  // instruction, "NSreq" is the current Security state of the PE, but this is not
6  // necessarily true for a hardware stack push/pop or vector table access. "NSreq"
7  // might not be the same as the "NSattr" attribute the PE finally uses to make the access.
8
9  boolean DWT_DataValueMatch(integer N, bits(32) daddr, bits(32) dvalue, integer dsize,
10                             boolean read, boolean NSreq, boolean priv_match)
11
12     assert N < UInt(DWT_CTRL.NUMCOMP) && dsize IN {1,2,4} && Align(daddr,dsize) == daddr;
13
14     valid_match = DWT_ValidMatch(N, !NSreq, priv_match);
15     valid_data = DWT_FUNCTION[N].MATCH[3:2] == '10';
16
17     if valid_match && valid_data then
18         case DWT_FUNCTION[N].MATCH[1:0] of
19             when '00' match_lsc = TRUE;   linked = FALSE;
20             when '01' match_lsc = !read;  linked = FALSE;
21             when '10' match_lsc = read;   linked = FALSE;
22             when '11'
23                 case DWT_FUNCTION[N-1].MATCH[1:0] of

```

```

23         when '00' match_lsc = TRUE;   linked = TRUE;
24         when '01' match_lsc = !read;  linked = TRUE;
25         when '10' match_lsc = read;   linked = TRUE;
26
27         vsize = 2^UInt(DWT_FUNCTION[N].DATAVSIZE);
28
29         // Determine which bytes of dvalue to look at in the comparison.
30         if linked then
31             byte_mask = '0000'; // Filled in below if there is an address match
32             if DWT_DataAddressMatch(N-1, daddr, dsize, read, NSreq, priv_match) then
33                 case (vsize,dsize) of
34                     when (1,1) byte_mask[0] = '1';
35                     when (1,2) byte_mask[UInt(DWT_COMP[N-1][0])] = '1';
36                     when (1,4) byte_mask[UInt(DWT_COMP[N-1][1:0])] = '1';
37                     when (2,2) byte_mask[1:0] = '11';
38                     when (2,4)
39                         byte_mask[UInt(DWT_COMP[N-1][1:0])+1:UInt(DWT_COMP[N-1][1:0])] = '11';
40                     when (4,4) byte_mask = '1111';
41                     otherwise byte_mask = '0000'; // vsize > dsize: no match
42             else
43                 case dsize of
44                     when 1 byte_mask = '0001';
45                     when 2 byte_mask = '0011';
46                     when 4 byte_mask = '1111';
47
48             // Perform bitwise mask on the candidate data value.
49             bit_mask = (if HasArchVersion(Armv8p1) then DWT_VMASK[N] else Zeros(32));
50             dvalue = (dvalue AND NOT bit_mask);
51
52             // Split both values into byte lanes: DCBA and dcba.
53             // This function relies on the values being correctly replicated across DWT_COMP[N].
54             D = dvalue[31:24]; C = dvalue[23:16]; B = dvalue[15:8]; A = dvalue[7:0];
55             d = DWT_COMP[N][31:24]; c = DWT_COMP[N][23:16];
56             b = DWT_COMP[N][15:8]; a = DWT_COMP[N][7:0];
57
58             // Partial results.
59             D_d = byte_mask[3] == '1' && D == d;
60             C_c = byte_mask[2] == '1' && C == c;
61             B_b = byte_mask[1] == '1' && B == b;
62             A_a = byte_mask[0] == '1' && A == a;
63
64             // Combined partial results.
65             BA_ba = B_b && A_a;
66             DC_dc = D_d && C_c;
67             DCBA_dcba = D_d && C_c && B_b && A_a;
68
69             // Generate full results.
70             case (vsize,dsize) of
71                 when (1,-) match_data = D_d || C_c || B_b || A_a;
72                 when (2,2), (2,4) match_data = DC_dc || BA_ba;
73                 when (4,4) match_data = DCBA_dcba;
74                 otherwise match_data = FALSE; // vsize > dsize: no match
75
76             match = match_data && match_lsc;
77         else
78             match = FALSE;
79
80         return match;

```

E2.1.110 DWT_InstructionAddressMatch

```

1 // DWT_InstructionAddressMatch()
2 // =====
3 // Check for match of instruction access at "Iaddr".
4 // If comparators 'm' and 'm+1' form an Instruction Address Range comparator, then this
5 // function returns the range match when N=m+1.
6
7 boolean DWT_InstructionAddressMatch(integer N, bits(32) Iaddr, boolean isSecure,

```

```

8         boolean isPriv)
9     assert N < UInt(DWT_CTRL.NUMCOMP) && Align(Iaddr, 2) == Iaddr;
10
11     valid_match = DWT_ValidMatch(N, isSecure, isPriv);
12     valid_instr = DWT_FUNCTION[N].MATCH == '001x';
13
14     if valid_match && valid_instr then
15         if N != UInt(DWT_CTRL.NUMCOMP)-1 then
16             linked_to_instr = DWT_FUNCTION[N+1].MATCH == '0011';
17         else
18             linked_to_instr = FALSE;
19
20         if DWT_FUNCTION[N].MATCH == '0011' then
21             linked = TRUE;
22         else
23             linked = FALSE;
24
25         if !linked_to_instr then
26             (match_eq, match_gt) = DWT_AddressCompare(Iaddr, DWT_COMP[N], 2, 2);
27             if linked then
28                 valid_match = DWT_ValidMatch(N-1, isSecure, isPriv);
29                 (lower_eq, lower_gt) = DWT_AddressCompare(Iaddr, DWT_COMP[N-1], 2, 2);
30                 match_addr = valid_match && (lower_eq || lower_gt) && !match_gt;
31             else
32                 match_addr = match_eq;
33         else
34             match_addr = FALSE;
35         match = match_addr;
36     else
37         match = FALSE;
38
39     return match;

```

E2.1.111 DWT_InstructionMatch

```

1 // DWT_InstructionMatch()
2 // =====
3 // Perform various Instruction Address checks for DWT.
4
5 DWT_InstructionMatch(bits(32) Iaddr, boolean isSecure, boolean isPriv, integer size)
6
7     boolean trigger_debug_event = FALSE;
8     boolean debug_event = FALSE;
9
10    if !HaveDWT() || IsZero(DWT_CTRL.NUMCOMP) then return; // No comparator support.
11
12    for i = 0 to UInt(DWT_CTRL.NUMCOMP) - 1
13        if IsDWTConfigUnpredictable(i) then UNPREDICTABLE;
14        instr_addr_match = DWT_InstructionAddressMatch(i, Iaddr, isSecure, isPriv);
15        if instr_addr_match then
16            // Instruction Address.
17            if DWT_FUNCTION[i].MATCH == '0010' then
18                DWT_FUNCTION[i].MATCHED = '1';
19                debug_event = DWT_FUNCTION[i].ACTION == '01';
20
21            // Instruction Address Limit.
22            elseif DWT_FUNCTION[i].MATCH == '0011' then
23                DWT_FUNCTION[i].MATCHED = bit UNKNOWN;
24                DWT_FUNCTION[i-1].MATCHED = '1';
25                debug_event = DWT_FUNCTION[i-1].ACTION == '01';
26
27            trigger_debug_event = trigger_debug_event || debug_event;
28
29    // If address A matches but if A+2 does not match for a 4-byte instruction, it
30    // is CONSTRAINED UNPREDICTABLE whether a match is generated.
31    elseif size == 4 && DWT_InstructionAddressMatch(i, Iaddr + 2, isSecure, isPriv) then
32        trigger_debug_event = ConstrainUnpredictableBool(Unpredictable_DWTBreakpoint);
33

```

```

34     if trigger_debug_event then
35         debug_event = SetDWTDebugEvent(isSecure, isPriv, AccType_IFETCH);
36     return;

```

E2.1.112 DWT_ValidMatch

```

1  // DWT_ValidMatch()
2  // =====
3  // Returns TRUE if this match is permitted by the current authentication controls,
4  // FALSE otherwise.
5
6  boolean DWT_ValidMatch(integer N, boolean secure_match, boolean priv_match)
7      if !HaveSecurityExt() then assert !secure_match;
8
9      // Check for disabled.
10     if ((secure_match && !SecureNoninvasiveDebugAllowed(priv_match)) ||
11         !NoninvasiveDebugAllowed(priv_match)) ||
12         DEMCR.TRCENA == '0' ||
13         DWT_FUNCTION[N].MATCH == '0000') then
14         return FALSE;
15
16     // Check for Debug event.
17     if DWT_FUNCTION[N].ACTION == '01' then
18         hlt_en = CanHaltOnEvent(secure_match, priv_match);
19         // Ignore priority when checking whether DebugMonitor activates DWT matches.
20         mon_en = (HaveDebugMonitor() && CanPendMonitorOnEvent(secure_match, FALSE, TRUE,
21                                                                 priv_match, TRUE));
22         return (hlt_en || mon_en);
23     else
24         // Otherwise trace or trigger event.
25         return !secure_match || SecureNoninvasiveDebugAllowed(priv_match);

```

E2.1.113 Elem

```

1  // Elem[]
2  // =====
3
4  // Non-assignment form
5
6  bits(size) Elem[bits(N) vector, integer e, integer size]
7      assert e >= 0 && (e+1)*size <= N;
8      return vector[(e+1)*size-1:e*size];
9
10 bits(size) Elem[bits(N) vector, integer e]
11     return Elem[vector, e, size];
12
13 // Assignment form
14
15 Elem[bits(N) &vector, integer e, integer size] = bits(size) value
16     assert e >= 0 && (e+1)*size <= N;
17     vector[(e+1)*size-1:e*size] = value;
18     return;
19
20 Elem[bits(N) &vector, integer e] = bits(size) value
21     Elem[vector, e, size] = value;
22     return;

```

E2.1.114 EndOfInstruction

```

1  // EndOfInstruction
2  // =====
3  // Terminates the processing of current instruction.
4
5  EndOfInstruction();

```

E2.1.115 EventRegistered

```
1 // EventRegistered
2 // =====
3 // Returns TRUE if PE Event Register is set to 1 and FALSE otherwise.
4
5 boolean EventRegistered();
```

E2.1.116 ExceptionActiveBitCount

```
1 // ExceptionActiveBitCount()
2 // =====
3
4 integer ExceptionActiveBitCount()
5     integer count = 0;
6     for i = 0 to MaxExceptionNum()
7         for j = 0 to 1
8             if IsActiveForState(i, j == 0) then
9                 count = count + 1;
10    return count;
```

E2.1.117 ExceptionDetails

```
1 // ExceptionDetails()
2 // =====
3
4 (boolean, boolean) ExceptionDetails(integer exception, boolean isSecure,
5                                     boolean isSynchronous)
6     // Is the exception subject to escalation.
7     case exception of
8         when HardFault
9             termInst = TRUE;
10            canPend = TRUE;
11            canEscalate = TRUE;
12        when MemManage
13            termInst = TRUE;
14            if HaveMainExt() then
15                val = if isSecure then SHCSR_S else SHCSR_NS;
16                canPend = val.MEMFAULTENA == '1';
17            else
18                canPend = FALSE;
19            canEscalate = TRUE;
20        when BusFault
21            termInst = isSynchronous;
22            canPend = if HaveMainExt()
23                    then SHCSR_S.BUSFAULTENA == '1' else FALSE;
24            // Async BusFaults only escalate if they are disabled.
25            canEscalate = termInst || !canPend;
26        when UsageFault
27            termInst = TRUE;
28            if HaveMainExt() then
29                val = if isSecure then SHCSR_S else SHCSR_NS;
30                canPend = val.USGFAULTENA == '1';
31            else
32                canPend = FALSE;
33            canEscalate = TRUE;
34        when SecureFault
35            termInst = TRUE;
36            canPend = if HaveMainExt()
37                    then SHCSR_S.SECUREFAULTENA == '1' else FALSE;
38            canEscalate = TRUE;
39        when SVCall
40            termInst = FALSE;
41            canPend = TRUE;
42            canEscalate = TRUE;
43        when DebugMonitor
```

```

44         termInst      = TRUE;
45         // Only used by a BKPT or FPB event.
46         canPend       = (HaveMainExt() && CanPendMonitorOnEvent(isSecure, TRUE, TRUE,
47                                                                    FindPriv(), TRUE));
48         canEscalate = TRUE; // HardFault escalation for a BKPT
49     otherwise
50         termInst      = FALSE;
51         canEscalate = FALSE;
52
53     // If the fault can escalate then check if exception can be taken immediately, or whether
54     // it should escalate.
55     // NOTE: In some cases (for example faults during lazy floating-point state preservation)
56     // the priority comparison below is ignored and the decision to escalate or not is
57     // based on other factors.
58     escalateToHf = FALSE;
59     if canEscalate then
60         execPri = ExecutionPriority();
61         excePri = ExceptionPriority(exception, isSecure, TRUE);
62         if (excePri >= execPri) || !canPend then
63             escalateToHf = TRUE;
64
65     return (escalateToHf, termInst);

```

E2.1.118 ExceptionEnabled

```

1  // ExceptionEnabled()
2  // =====
3
4  boolean ExceptionEnabled(integer exception, boolean secure)
5      assert 1 <= exception && exception < NUMEXN;
6      if secure && !HaveSecurityExt() then
7          enabled = FALSE;
8      elseif exception < 16 then
9          val = if secure then _SHCSR_S else _SHCSR_NS;
10         case exception of
11             when Reset
12                 enabled = TRUE;
13             when NMI
14                 enabled = secure == (AIRCR_S.BFHFNMINS == '0');
15             when HardFault
16                 enabled = secure || AIRCR_S.BFHFNMINS == '1';
17             when MemManage
18                 enabled = val.MEMFAULTENA == '1';
19             when BusFault
20                 enabled = ((_SHCSR_S.BUSFAULTENA == '1') &&
21                            (secure == (AIRCR_S.BFHFNMINS == '0')));
22             when UsageFault
23                 enabled = val.USGFAULTENA == '1';
24             when SecureFault
25                 enabled = secure && _SHCSR_S.SECUREFAULTENA == '1';
26             when SVCcall
27                 enabled = TRUE;
28             when DebugMonitor
29                 enabled = ((secure == (DEMCR.SDME == '1')) &&
30                            !InstructionsInFlight());
31             when PendSV
32                 enabled = !DebugCanMaskInts(secure);
33             when SysTick
34                 enabled = ((!IsExceptionTargetConfigurable(SysTick) ||
35                            (secure == (_ICSR_S.STTNS == '0')) &&
36                            !DebugCanMaskInts(secure)));
37             otherwise
38                 enabled = FALSE;
39     else
40         enabled = (IrqEnabled[exception-16] && !DebugCanMaskInts(secure) &&
41                   ((NVIC_ITNS[exception-16] == '0') == secure));
42
43     return enabled;

```

E2.1.119 ExceptionEntry

```

1 // ExceptionEntry()
2 // =====
3 // Exception entry is modified according to the behavior of a derived
4 // exception, see DerivedLateArrival() also.
5
6 (ExcInfo, EXC_RETURN_Type) ExceptionEntry(integer exceptionType, boolean toSecure)
7
8     // PushStack() can abandon memory accesses if a fault occurs during the stacking
9     // sequence.
10    (exc, partialExcReturn) = PushStack();
11    if exc.fault == NoFault then
12        (exc, partialExcReturn) = ExceptionTaken(exceptionType, FALSE, toSecure,
13                                                IgnoreFaults_NONE, partialExcReturn);
14    return (exc, partialExcReturn);

```

E2.1.120 ExceptionPriority

```

1 // ExceptionPriority()
2 // =====
3
4 integer ExceptionPriority(integer n, boolean isSecure, boolean groupPri)
5     if HaveMainExt() then
6         assert 1 <= n && n < 512;
7     else
8         assert 1 <= n && n < 48;
9
10    if n == Reset then // Reset.
11        result = -4;
12    elseif n == NMI then // NMI.
13        result = -2;
14    elseif n == HardFault then // HardFault.
15        if isSecure && AIRCR.BFHFNMINS == '1' then
16            result = -3;
17        else
18            result = -1;
19    elseif HaveMainExt() && n == MemManage then // MemManage.
20        result = UInt(if isSecure then SHPR1_S.PRI_4 else SHPR1_NS.PRI_4);
21    elseif HaveMainExt() && n == BusFault then // BusFault.
22        result = UInt(SHPR1_S.PRI_5);
23    elseif HaveMainExt() && n == UsageFault then // UsageFault.
24        result = UInt(if isSecure then SHPR1_S.PRI_6 else SHPR1_NS.PRI_6);
25    elseif HaveMainExt() && n == SecureFault then // SecureFault.
26        result = UInt(SHPR1_S.PRI_7);
27    elseif n == SVCcall then // SVCcall.
28        result = UInt(if isSecure then SHPR2_S.PRI_11 else SHPR2_NS.PRI_11);
29    elseif HaveMainExt() && n == DebugMonitor then // DebugMonitor.
30        result = UInt(SHPR3_S.PRI_12);
31    elseif n == PendSV then // PendSV.
32        result = UInt(if isSecure then SHPR3_S.PRI_14 else SHPR3_NS.PRI_14);
33    elseif n == SysTick // SysTick.
34        && ((HaveSysTick() == 2) ||
35            (HaveSysTick() == 1 && ((_ICSR_S.STTNS == '0') == isSecure))) then
36        result = UInt(if isSecure then SHPR3_S.PRI_15 else SHPR3_NS.PRI_15);
37    elseif n >= 16 then // External interrupt (n-16).
38        r = (n - 16) DIV 4;
39        v = n MOD 4;
40        result = UInt(NVIC_IPR[r][v*8+7:v*8]);
41    else // Reserved exceptions.
42        result = 256;
43
44    assert result IN {-4 .. 256};
45
46    // Negative priorities (in other words Reset, NMI, and HardFault)
47    // are not affected by PRIGROUP or PRIS.
48    if result >= 0 then

```

```

49 // Include the PRIGROUP effect.
50 if HaveMainExt() && groupPri then
51     integer subgroupshift;
52     if isSecure then
53         subgroupshift = UInt(AIRCR_S.PRIGROUP);
54     else
55         subgroupshift = UInt(AIRCR_NS.PRIGROUP);
56     integer groupvalue = 2 << subgroupshift;
57     integer subgroupvalue = result MOD groupvalue;
58     result = result - subgroupvalue;
59
60     PriSnsPri = RestrictedNSPri();
61     if (AIRCR_S.PRIS == '1') && !isSecure then
62         result = (result >> 1) + PriSnsPri;
63
64     assert result IN {-4 .. 256};
65     return result;

```

E2.1.121 ExceptionReturn

```

1 // ExceptionReturn()
2 // =====
3
4 (ExcInfo, EXC_RETURN_Type, boolean) ExceptionReturn(EXC_RETURN_Type excReturn)
5     integer returningExceptionNumber = UInt(IPSr.Exception);
6
7     (exc, excReturn) = ValidateExceptionReturn(excReturn, returningExceptionNumber);
8     if exc.fault != NoFault then
9         return (exc, excReturn, FALSE);
10
11     if HaveSecurityExt() then
12         excSecure = excReturn.ES == '1';
13         retToSecure = excReturn.S == '1';
14     else
15         excSecure = FALSE;
16         retToSecure = FALSE;
17
18     // Restore SPSEL for the Security state we are returning from.
19     if excSecure then
20         CONTROL_S.SPSEL = excReturn.SPSEL;
21     else
22         CONTROL_NS.SPSEL = excReturn.SPSEL;
23
24     returningExcIsSecure = excReturn.ES == '1';
25     DeActivate(returningExceptionNumber, returningExcIsSecure);
26
27     // If requested, clear the scratch floating-point values left in the
28     // Floating-point context before returning or tail-chaining.
29     if HaveMveOrFPEExt() && FPCCR.CLRONRET == '1' && CONTROL.FPCA == '1' then
30         if FPCCR.SLSPACT == '1' then
31             SFSR.LSERR = '1';
32             exc = CreateException(SecureFault);
33             return (exc, excReturn, FALSE);
34         else
35             // Check if we have permission to clear the registers.
36             if HasArchVersion(Armv8pl) then
37                 exc = CheckCPEEnabled(10, TRUE, returningExcIsSecure);
38                 if exc.fault != NoFault then
39                     return (exc, excReturn, FALSE);
40
41             // Clear the floating-point / MVE registers
42             InvalidateFPRegs(TRUE, FALSE);
43
44     // If tail-chaining is supported, check if there is a pending exception with
45     // sufficient priority to be taken now. This check is done after the
46     // previous exception is deactivated so the priority of the previous
47     // exception does not mask any pending exceptions.
48     // The position of TailChain() within this function is the earliest point

```



```

49 // at which a tailchain is architecturally visible. Unless otherwise
50 // stated, tail-chaining from a later point is permissible.
51 if boolean IMPLEMENTATION_DEFINED "Tail chaining supported" then
52     (takeException, exception, excIsSecure) = PendingExceptionDetails();
53     if takeException then
54         (exc, excReturn) = TailChain(exception, excIsSecure, excReturn);
55         return (exc, excReturn, TRUE);
56
57 // Return to the background Security state
58 if HaveSecurityExt() then
59     CurrentState = if retToSecure
60                     then SecurityState_Secure else SecurityState_NonSecure;
61
62 // Sleep-on-exit performs equivalent behavior to the WFI instruction.
63 // The position of SleepOnExit() within this function is the earliest point
64 // at which it can be performed. Performing SleepOnExit from a later point
65 // is permissible.
66 if (excReturn.Mode == '1' && SCR.SLEEPONEXIT == '1' &&
67     ExceptionActiveBitCount() == 0) then
68     SleepOnExit(); // IMPLEMENTATION DEFINED
69
70 // Pop the stack and raise any exceptions that are generated.
71 exc = PopStack(excReturn);
72 if exc.fault == NoFault then
73     ClearExclusiveLocal(ProcessorID());
74     ClearInFlightInstructions();
75     SetEventRegister(); // See WFE instruction for more details
76     InstructionSynchronizationBarrier('1111');
77
78 return (exc, excReturn, FALSE);

```

E2.1.122 ExceptionTaken

```

1 // ExceptionTaken()
2 // =====
3
4 (ExcInfo, EXC_RETURN_Type) ExceptionTaken(integer exceptionNumber, boolean doTailChain,
5     boolean excIsSecure, IgnoreFaultsType ignoreFaults,
6     EXC_RETURN_Type excReturn)
7     assert(HaveSecurityExt() || !excIsSecure);
8
9 // If the background code was running in the Secure state there are some
10 // additional steps that might need to be taken to protect the Additional state
11 // context registers.
12 exc = DefaultExcInfo();
13 if HaveSecurityExt() && excReturn.S == '1' then
14     if excIsSecure then // Transitioning to Secure state
15         // If tail-chaining is from Non-secure to Secure, then the Additional
16         // state context registers are already on stack. Set excReturn.DCRS
17         // accordingly
18         if doTailChain && excReturn.ES == '0' then
19             excReturn.DCRS = '0';
20     else // Transitioning to Non-secure state.
21         // If the Additional state context registers are not already on the stack,
22         // push them now.
23         if excReturn.DCRS == '1' && !(doTailChain && excReturn.ES == '0') then
24             exc = PushCalleeStack(doTailChain, excReturn);
25         // Going to Non-secure exception. Set excReturn.DCRS to default
26         // value
27         excReturn.DCRS = '1';
28
29 // Finalise excReturn value
30 if excIsSecure then
31     excReturn.SPSEL = CONTROL_S.SPSEL;
32     excReturn.ES = '1';
33 else
34     excReturn.SPSEL = CONTROL_NS.SPSEL;
35     excReturn.ES = '0';

```

```

36     LR = excReturn;
37
38     // Register clearing
39     // State context registers: These registers are cleared if exception
40     // targets the Non-secure state, otherwise they are UNKNOWN. As of Armv8.1 the
41     // registers are always cleared if the Security extension is implemented.
42     // NOTE: The original values were pushed to the stack.
43     if HaveSecurityExt() && (!excIsSecure || HasArchVersion(Armv8p1)) then
44         callerRegValue = Zeros(32);
45     else
46         callerRegValue = bits(32) UNKNOWN;
47     for n = 0 to 3
48         R[n] = callerRegValue;
49     R[12] = callerRegValue;
50     EAPSR = callerRegValue;
51     // Additional state context registers: If the background code was in the Secure
52     // state these registers are cleared if the exception targets the Non-secure state,
53     // and UNKNOWN if it targets the Secure state and the registers have been
54     // pushed to the stack (as indicated by EXC_RETURN.DCRS).
55     //
56     // NOTE: Additional state context registers are preserved if the background code is
57     // Non-secure, or when the exception is Secure and the values have not
58     // been pushed to the stack.
59     if HaveSecurityExt() && excReturn.S == '1' then
60         if excIsSecure then
61             if excReturn.DCRS == '0' then
62                 for n = 4 to 11
63                     R[n] = bits(32) UNKNOWN;
64             else
65                 for n = 4 to 11
66                     R[n] = Zeros();
67
68     // If enabled, the IESB contains asynchronous RAS or BusFault errors to the background
69     // context. This is conditional on there being no stacking faults - if there are, the
70     // errors will be synchronized when the subsequent exception is raised.
71     if AIRCR.IESB == '1' then
72         exc = MergeExcInfo(exc, SynchronizeBusFault());
73
74     // If no errors so far (or errors that can be ignored) load the vector address.
75     if exc.fault == NoFault || ignoreFaults != IgnoreFaults_NONE then
76         (exc, start) = Vector[exceptionNumber, excIsSecure];
77
78     // The state or mode of processor is not updated if an exception is raised
79     // during the entry sequence.
80     if exc.fault == NoFault || ignoreFaults == IgnoreFaults_ALL then
81         ActivateException(exceptionNumber, excIsSecure);
82         SCS_UpdateStatusRegs();
83         ClearExclusiveLocal(ProcessorID());
84         ClearInFlightInstructions();
85         SetEventRegister(); // See WFE instruction for details.
86         InstructionSynchronizationBarrier('1111');
87         // Start execution of handler
88         EPSR.T = start[0];
89         // If EPSR.T == 0 then an exception is taken on the next
90         // instruction: UsageFault('Invalid State') if the Main Extension is
91         // implemented; HardFault otherwise.
92         BranchTo(start[31:1]:'0', TRUE, FALSE);
93
94     if exc.fault != NoFault then
95         exc.inExcTaken = TRUE;
96         exc.derivedFrom = exceptionNumber;
97         exc.derivedFromIsSecure = excIsSecure;
98
99     return (exc, excReturn);

```

E2.1.123 ExceptionTargetsSecure

```

1 // ExceptionTargetsSecure()

```

```

2 // =====
3
4 // Determine the default Security state an exception is expected to target if the
5 // exception is not forced to a specific domain.
6
7 boolean ExceptionTargetsSecure(integer exceptionNumber, boolean isSecure)
8     if !HaveSecurityExt() then
9         return FALSE;
10
11     boolean targetSecure = FALSE;
12     case exceptionNumber of
13         when NMI
14             targetSecure = AIRCR.BFHFNMINS == '0';
15
16         when HardFault
17             targetSecure = AIRCR.BFHFNMINS == '0' || isSecure;
18
19         when MemManage
20             targetSecure = isSecure;
21
22         when BusFault
23             targetSecure = AIRCR.BFHFNMINS == '0';
24
25         when UsageFault
26             targetSecure = isSecure;
27
28         when SecureFault
29             // SecureFault always targets Secure state.
30             targetSecure = TRUE;
31
32         when SVCall
33             targetSecure = isSecure;
34
35         when DebugMonitor
36             targetSecure = DEMCR.SDME == '1';
37
38         when PendSV
39             // This state should be unreachable as PendSV is a banked interrupt
40             // and it is directly pended for the correct Security state, so this
41             // function is not called for this exception.
42             assert FALSE;
43
44         when SysTick
45             if HaveSysTick() != 1 then
46                 // If there is a SysTick for each Security state, then the exception
47                 // targets the Security state associated with the SysTick instance
48                 // that raised the exception.
49                 // This state should be unreachable as SysTick exception is banked
50                 // and it is directly pended for the correct Security state. This
51                 // function can only be called when 1 SysTick is implemented.
52                 assert FALSE;
53             else
54                 // SysTick target Security state is configurable.
55                 targetSecure = _ICSR_S.STTNS == '0';
56
57         otherwise
58             if exceptionNumber >= 16 then
59                 // Interrupts target the state defined by the NVIC_ITNS register.
60                 targetSecure = NVIC_ITNS[exceptionNumber - 16] == '0';
61
62     return targetSecure;

```

E2.1.124 ExclInfo

```

1 // Exception information
2
3 type ExclInfo is (
4     integer fault, // The ID of the resulting fault, or NoFault (or 0)

```

```

5          // if no fault occurred.
6      integer origFault,          // The ID of the original fault raised before
7          // escalation is considered.
8      integer derivedFrom,       // If origFault is a derived exception this field indicates
9          // the ID of the exception that triggered this derived
10         // fault. This field is only valid if inExcTaken is TRUE.
11      boolean isSecure,          // TRUE if the fault targets the Secure state.
12      boolean origFaultIsSecure, // TRUE if the original fault raised targeted
13         // Secure state.
14      boolean derivedFromIsSecure, // True if the exception this fault was derived from
15         // targeted the Secure state.
16      boolean isTerminal,        // Set to TRUE for derived faults (for example an
17         // exception on exception entry) that prevent the
18         // original exception being entered (for example a
19         // BusFault whilst fetching the exception vector address).
20      boolean inExcTaken,        // TRUE if the exception occurred during ExceptionTaken().
21         // This is used to determine if the LR update and the
22         // Additional state context stacking operations have
23         // been performed, and therefore whether the derived
24         // exception should be treated as a tail-chain.
25      boolean lockup,            // Set to TRUE if the exception should cause a lockup.
26      boolean termInst           // Set to TRUE if the exception should cause the
27         // instruction to be terminated.
28 )

```

E2.1.125 ExclusiveMonitorsPass

```

1  // ExclusiveMonitorsPass()
2  // =====
3
4  boolean ExclusiveMonitorsPass(bits(32) address, integer size)
5
6      // It is IMPLEMENTATION DEFINED whether the detection of memory aborts happens
7      // before or after the check on the local Exclusive Monitor. As a result a failure
8      // of the local monitor can occur on some implementations even if the memory
9      // access would give a memory abort.
10
11     if address != Align(address, size) then
12         UFSR.UNALIGNED = '1';
13         excInfo = CreateException(UsageFault);
14     else
15         (excInfo, memaddrdesc) = ValidateAddress(address, AccType_NORMAL,
16             FindPriv(), IsSecure(), TRUE, TRUE);
17         HandleException(excInfo);
18
19     passed = IsExclusiveLocal(memaddrdesc.address, ProcessorID(), size);
20     if memaddrdesc.memattrs.shareable then
21         passed = passed && IsExclusiveGlobal(memaddrdesc.address, ProcessorID(), size);
22     // Local monitor remains in, or transitions to the Open Access state.
23     ClearExclusiveLocal(ProcessorID());
24     return passed;

```

E2.1.126 ExecBeats

```

1  // ExecBeats()
2  // =====
3
4  ExecBeats()
5      // PEs are not constrained to following the beat execution pattern shown in
6      // this function. Any pattern is permitted providing it meets the following
7      // requirements:
8      //      1) The new pattern of completed beats in representable as a valid ECI
9      //         value.
10     //      2) The beat execution rules are not violated (see specification).
11     //      3) All ECI encodings are accepted as inputs, even if the PE cannot
12     //         generate that ECI value.
13     newBeatComplete = BeatComplete;

```

```

14     beatInstIdx      = 0;
15     for instId = 0 to MAX_OVERLAPPING_INSTRS-1
16         if _InstInfo[instId].Valid then
17             _InstID      = instId;
18             _CurrentInstrExecState = GetInstrExecState(instId);
19             InstStateCheck(ThisInstr());
20             // Find the first ticks worth of beats that is not complete
21             beatBits      = Elem[newBeatComplete, beatInstIdx, MAX_BEATS];
22             baseBeatId = 0;
23             beatIndex      = 0;
24             while Elem[beatBits, beatIndex, BEATS_PER_TICK] == Ones(BEATS_PER_TICK) do
25                 beatIndex      = beatIndex + 1;
26                 baseBeatId = baseBeatId + BEATS_PER_TICK;
27
28             // Perform all the beats in this tick for the current instruction.
29             for beatInTick = 0 to BEATS_PER_TICK-1
30                 beatId = baseBeatId + beatInTick;
31                 // Only perform the beat if it has not already been completed.
32                 beatFlagIdx = (beatInstIdx * MAX_BEATS) + beatId;
33                 if newBeatComplete[beatFlagIdx] == '0' then
34                     _BeatID      = beatId;
35                     _AdvanceVPTState = TRUE;
36                     cond          = DefaultCond();
37                     DecodeExecute(ThisInstr(), ThisInstrAddr(),
38                                   ThisInstrLength() == 2, cond);
39                     // Advance the VPT state for the current beat if the instruction
40                     // did not update the mask directly (indicated by the instruction
41                     // clearing _AdvanceVPTState).
42                     if _AdvanceVPTState then
43                         VPTAdvance(beatId);
44
45                     // Update the beat complete state. If the instruction is now complete
46                     // advance the beat complete flags.
47                     newBeatComplete[beatFlagIdx] = '1';
48                     if newBeatComplete[MAX_BEATS-1:0] == Ones(MAX_BEATS) then
49                         // A whole instruction has been completed, set the flag to indicate
50                         // that the PC etc must be advanced. Only one instruction should
51                         // ever be committed per tick, so assert that an instruction has
52                         // not already been committed.
53                         assert(!_CommitState);
54                         _CommitState      = TRUE;
55                         // Remove the beat bits for the newly completed instruction and
56                         // adjust the index used for the newBeatComplete bit vector
57                         // accordingly.
58                         newBeatComplete = LSR(newBeatComplete, MAX_BEATS);
59                         beatInstIdx      = beatInstIdx - 1;
60                     // Update the beat complete flags. This is done after each beat in the
61                     // tick, so that if an exception occurs it correctly indicates which
62                     // beats of which instruction have been executed before the exception
63                     // occurred.
64                     BeatComplete = newBeatComplete;
65
66                     // Increment the index of the group of beat bits to point to the next instruction to
67                     // be processed.
68                     beatInstIdx = beatInstIdx + 1;

```

E2.1.127 ExecuteFPCheck

```

1 // ExecuteFPCheck()
2 // =====
3
4 ExecuteFPCheck()
5 // Preserve any lazy floating-point state.
6 PreserveFPState();
7 // It is an IMPLEMENTATION_DEFINED choice on whether an interrupt can
8 // be taken after triggering lazy floating-point state preservation by
9 // an instruction, but before updating the ownership of the
10 // floating-point registers.

```

```

11
12 // Update the ownership of the floating-point context.
13 FPCCR_S.S = if IsSecure() then '1' else '0';
14
15 // Update CONTROL.FPCA, and create new floating-point context
16 // if this has been enabled by setting FPCCR.ASPEN to 1.
17 if FPCCR.ASPEN == '1' &&
18 (CONTROL.FPCA == '0' || (IsSecure() && CONTROL_S.SFPA == '0')) then
19     CONTROL.FPCA = '1';
20     if IsSecure() then
21         CONTROL_S.SFPA = '1';
22     FPSCR = FPDSCR[31:0];
23     VPR = Zeros();
24     // If the instruction is abandoned due to an exception, then it is an
25     // IMPLEMENTATION_DEFINED choice whether the floating-point context
26     // created by that instruction is preserved or not.
27
28 return;

```

E2.1.128 ExecutionPriority

```

1 // ExecutionPriority()
2 // =====
3 // Determine the current execution priority.
4
5 integer ExecutionPriority()
6
7     boostedpri = HighestPri(); // Priority influence of BASEPRI, PRIMASK and FAULTMASK.
8
9 // Calculate boosted priority effect due to BASEPRI for both Security states.
10 PriSNsPri = RestrictedNSPri();
11 if HaveMainExt() then
12     if UInt(BASEPRI_NS.BASEPRI) != 0 then
13         basepri = UInt(BASEPRI_NS[7:0]);
14         // Include the PRIGROUP effect
15         subgroupshift = UInt(AIRCR_NS.PRIGROUP);
16         groupvalue = 2 << subgroupshift;
17         subgroupvalue = basepri MOD groupvalue;
18         boostedpri = basepri - subgroupvalue;
19         if AIRCR_S.PRIS == '1' then
20             boostedpri = (boostedpri >> 1) + PriSNsPri;
21
22     if UInt(BASEPRI_S.BASEPRI) != 0 then
23         basepri = UInt(BASEPRI_S.BASEPRI);
24         // Include the PRIGROUP effect
25         subgroupshift = UInt(AIRCR_S.PRIGROUP);
26         groupvalue = 2 << subgroupshift;
27         subgroupvalue = basepri MOD groupvalue;
28         basepri = basepri - subgroupvalue;
29         if boostedpri > basepri then
30             boostedpri = basepri;
31
32 // Calculate boosted priority effect due to PRIMASK for both Security states.
33 if PRIMASK_NS.PM == '1' then
34     if AIRCR_S.PRIS == '0' then
35         boostedpri = 0;
36     else
37         if boostedpri > PriSNsPri then
38             boostedpri = PriSNsPri;
39
40 if PRIMASK_S.PM == '1' then
41     boostedpri = 0;
42
43 // Calculate boosted priority effect due to FAULTMASK for both Security states.
44 if HaveMainExt() then
45     if FAULTMASK_NS.FM == '1' then
46         if AIRCR.BFHFNMINS == '0' then
47             if AIRCR_S.PRIS == '0' then

```

```

48         boostedpri = 0;
49     else
50         if boostedpri > PriSnsPri then
51             boostedpri = PriSnsPri;
52     else
53         boostedpri = -1;
54
55     if FAULTMASK_S.FM == '1' then
56         boostedpri = if AIRCR.BFHFNMINS == '0' then -1 else -3;
57
58     // Finally calculate the resultant priority after boosting.
59     rawExecPri = RawExecutionPriority();
60     if boostedpri < rawExecPri then
61         priority = boostedpri;
62     else
63         priority = rawExecPri;
64
65     assert priority IN {-4 .. 256};
66     return priority;

```

E2.1.129 Extend

```

1 // Extend()
2 // =====
3
4 bits(N) Extend(bits(M) x, integer N, boolean unsigned)
5     return if unsigned then ZeroExtend(x, N) else SignExtend(x, N);
6
7 bits(N) Extend(bits(M) x, boolean unsigned)
8     return Extend(x, N, unsigned);

```

E2.1.130 ExternalInvasiveDebugEnabled

```

1 // ExternalInvasiveDebugEnabled()
2 // =====
3 // Return TRUE if Halting debug is enabled by the
4 // IMPLEMENTATION DEFINED authentication interface.
5
6 boolean ExternalInvasiveDebugEnabled()
7     // In the recommended interface, ExternalInvasiveDebugEnabled returns the state of
8     // the DBGEN signal.
9     return DBGEN == HIGH;

```

E2.1.131 ExternalNoninvasiveDebugEnabled

```

1 // ExternalNoninvasiveDebugEnabled()
2 // =====
3 // Return TRUE if non-invasive debug is enabled by the IMPLEMENTATION DEFINED authentication
4 // interface.
5
6 boolean ExternalNoninvasiveDebugEnabled()
7     // In the recommended interface, ExternalNoninvasiveDebugEnabled() returns the state
8     // of the (DBGEN OR NIDEN) signal.
9     return ExternalInvasiveDebugEnabled() || NIDEN == HIGH;

```

E2.1.132 ExternalSecureInvasiveDebugEnabled

```

1 // ExternalSecureInvasiveDebugEnabled()
2 // =====
3 // Return TRUE if Secure Halting debug is enabled by the IMPLEMENTATION DEFINED
4 // authentication interface.
5
6 boolean ExternalSecureInvasiveDebugEnabled()
7     // In the recommended interface, ExternalSecureInvasiveDebugEnabled() returns the

```

```

8      // state of the (DBGEN AND SPIDEN) signal.
9      return ExternalInvasiveDebugEnabled() && SPIDEN == HIGH;

```

E2.1.133 ExternalSecureNoninvasiveDebugEnabled

```

1  // ExternalSecureNoninvasiveDebugEnabled()
2  // =====
3  // Return TRUE if Secure non-invasive debug is enabled by the IMPLEMENTATION DEFINED
4  // authentication interface.
5
6  boolean ExternalSecureNoninvasiveDebugEnabled()
7      // In the recommended interface, ExternalSecureNoninvasiveDebugEnabled() returns
8      // the state of the (DBGEN OR NIDEN) AND (SPIDEN OR SPNIDEN) signal.
9      return ExternalNoninvasiveDebugEnabled() && (SPIDEN == HIGH || SPNIDEN == HIGH);

```

E2.1.134 ExternalSecureSelfHostedDebugEnabled

```

1  // ExternalSecureSelfHostedDebugEnabled()
2  // =====
3  // Return TRUE if Secure self-hosted debug is enabled by the IMPLEMENTATION DEFINED
4  // authentication interface.
5
6  boolean ExternalSecureSelfHostedDebugEnabled()
7      // In the recommended interface, ExternalSecureSelfHostedDebugEnabled() returns the
8      // state of the (DBGEN AND SPIDEN) signal.
9      return DBGEN == HIGH && SPIDEN == HIGH;

```

E2.1.135 ExtType

```

1  // Types of ISA extension
2
3  enumeration ExtType {ExtType_Mve,
4                      ExtType_MveFp,
5                      ExtType_MveOrFp,
6                      ExtType_MveOrDpFp,
7                      ExtType_Unknown,
8                      ExtType_HpFp,
9                      ExtType_SpFp,
10                     ExtType_DpFp};

```

E2.1.136 FaultNumbers

```

1  // Fault Numbers
2  // =====
3
4  // The fault numbers are a subset of ExceptionNumber and can be one of the
5  // following values:
6  constant integer NoFault      = 0;
7  constant integer Reset       = 1;
8  constant integer NMI         = 2;
9  constant integer HardFault   = 3;
10 constant integer MemManage    = 4;
11 constant integer BusFault    = 5;
12 constant integer UsageFault  = 6;
13 constant integer SecureFault = 7;
14 constant integer SVCall      = 11;
15 constant integer DebugMonitor = 12;
16 constant integer PendSV      = 14;
17 constant integer SysTick     = 15;

```

E2.1.137 FetchInstr


```

1  // FetchInstr()
2  // =====
3
4  (bits(32), boolean) FetchInstr(bits(32) addr)
5      // NOTE: It is CONSTRAINED UNPREDICTABLE whether otherwise valid sequential
6      //         instruction fetches that cross from Non-secure to Secure memory
7      //         generate a INVEP SecureFault, or transition normally.
8      sgOpcode = 0xE97FE97F[31:0];
9
10     // Assume we have a fetch fault so that if we end a cycle early the next
11     // cycle will capture this fact.
12     _PendingFetchFault = TRUE;
13
14     hw1Attr = SecurityCheck(addr, TRUE, IsSecure());
15     // Fetch the 16-bit T32 instruction, or the first half of a T32.
16     hw1Instr = MemI[addr];
17
18     isT16 = UInt(hw1Instr[15:11]) < UInt('11101');
19     len = if isT16 then 2 else 4;
20
21     // Checking for FPB Breakpoint on instructions.
22     if HaveFPB() && FPB_CheckBreakPoint(addr, len, TRUE, IsSecure()) then
23         GenerateDebugEventResponse(DebugEventCause_FPB);
24
25     // If the T bit is clear then the instruction cannot be decoded.
26     if EPSR.T == '0' then
27         // Attempted NS->S domain crossings with the T bit clear raise an INVEP
28         // SecureFault
29         if !IsSecure() && !hw1Attr.ns then
30             SFSR.INVEP = '1';
31             excInfo = CreateException(SecureFault);
32         else
33             UFSR.INVSTATE = '1';
34             excInfo = CreateException(UsageFault);
35             HandleException(excInfo);
36
37     // Implementations are permitted to terminate the fetch process early if a
38     // domain crossing is being attempted and the first 16bits of the opcode
39     // is not the first part of the SG instruction.
40     if boolean IMPLEMENTATION_DEFINED "Early SG check" then
41         if !IsSecure() && !hw1Attr.ns && (hw1Instr != sgOpcode[31:16]) then
42             SFSR.INVEP = '1';
43             excInfo = CreateException(SecureFault);
44             HandleException(excInfo);
45
46     // NOTE: Implementations are also permitted to terminate the fetch process
47     //         at this point with an UNDEFINSTR UsageFault if the first 16bit is
48     //         an undefined T32 prefix.
49
50     // If the data fetched is the top half of a T32 instruction fetch the bottom
51     // 16 bits.
52     if isT16 then
53         instr = Zeros(16) : hw1Instr;
54     else
55         hw2Attr = SecurityCheck(addr+2, TRUE, IsSecure());
56         // The following test covers 2 possible fault conditions:-
57         // 1) NS code branching to a T32 instruction where the first half is in
58         //    NS memory, and the second half is in S memory.
59         // 2) NS code branching to a T32 instruction in S & NSC memory, but
60         //    where the second half of the instruction is in NS memory.
61         if !IsSecure() && (hw1Attr.ns != hw2Attr.ns) then
62             SFSR.INVEP = '1';
63             excInfo = CreateException(SecureFault);
64             HandleException(excInfo);
65
66         // Fetch the second half of T32 instruction.
67         instr = hw1Instr : MemI[addr+2];
68
69     // Raise a fault if an otherwise valid NS->S transition that does not land on

```

```

70 // an SG instruction.
71 if !IsSecure() && !hwlAttr.ns && (instr != sgOpcode) then
72     SFSR.INVEP = '1';
73     excInfo = CreateException(SecureFault);
74     HandleException(excInfo);
75
76 if EPSR.B == '1' then
77     // BTI clearing instruction masks.
78     // When BTI is set, these instructions do not cause an INVSTATE UsageFault.
79     btiClearing = instr IN {sgOpcode, // SG.
80                                     '111100111010xxx10x0x00000001111', // BTI.
81                                     '111100111010xxx10x0x00000001101'}; // PACBTI.
82     bkptIgnore = hwlInstr == '1011110xxxxxxx'; // BKPT.
83     // A BKPT instruction ignores the EPSR.B bit.
84     if !btiClearing && !bkptIgnore then
85         // Generate a fault due to a BTI error.
86         UFSR.INVSTATE = '1';
87         excInfo = CreateException(UsageFault);
88         HandleException(excInfo);
89
90 // If we get to here we have no fetch fault.
91 _PendingFetchFault = FALSE;
92
93 return (instr, isT16);

```

E2.1.138 FindMemPriv

```

1 // FindMemPriv()
2 // =====
3
4 Privilege FindMemPriv()
5     return FindMemPriv(FindPriv());
6
7 Privilege FindMemPriv(boolean privileged)
8     return if privileged then Privilege_Priv else Privilege_Unpriv;

```

E2.1.139 FindPriv

```

1 // FindPriv()
2 // =====
3
4 boolean FindPriv()
5     return CurrentModeIsPrivileged();

```

E2.1.140 FixedToFP

```

1 // FixedToFP()
2 // =====
3
4 bits(N) FixedToFP(bits(M) operand, integer N, integer fraction_bits, boolean unsigned,
5                   boolean round_to_nearest, boolean fpscr_controlled)
6     return FixedToFP(operand, N, fraction_bits, unsigned, round_to_nearest,
7                       fpscr_controlled, FALSE);
8
9 bits(N) FixedToFP(bits(M) operand, integer N, integer fraction_bits, boolean unsigned,
10                  boolean round_to_nearest, boolean fpscr_controlled, boolean predicated)
11     assert N IN {16, 32, 64};
12     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
13     if round_to_nearest then fpscr_val.RMode = FPSCR_RMode_RN;
14     int_operand = if unsigned then UInt(operand) else SInt(operand);
15     real_operand = Real(int_operand) / 2.0^fraction_bits;
16     if real_operand == 0.0 then
17         result = FPZero('0', N);
18     else
19         result = FPRound(real_operand, N, fpscr_val, predicated);
20     return result;

```

E2.1.141 FPAbs

```

1 // FPAbs()
2 // =====
3
4 bits(N) FPAbs(bits(N) operand)
5     assert N IN {16,32,64};
6     return '0' : operand[N-2:0];

```

E2.1.142 FPAAdd

```

1 // FPAAdd()
2 // =====
3
4 bits(N) FPAAdd(bits(N) op1, bits(N) op2, boolean fpscr_controlled)
5     return FPAAdd(op1, op2, fpscr_controlled, FALSE);
6
7 bits(N) FPAAdd(bits(N) op1, bits(N) op2, boolean fpscr_controlled, boolean predicated)
8     assert N IN {16,32,64};
9     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
10    (type1,sign1,value1) = FPUnpack(op1, fpscr_val, predicated);
11    (type2,sign2,value2) = FPUnpack(op2, fpscr_val, predicated);
12    (done,result) = FPProcessNaNs(type1, type2, op1, op2, fpscr_val, predicated);
13    if !done then
14        inf1 = (type1 == FPType_Infinity);  inf2 = (type2 == FPType_Infinity);
15        zero1 = (type1 == FPType_Zero);    zero2 = (type2 == FPType_Zero);
16        if inf1 && inf2 && sign1 == NOT(sign2) then
17            result = FPDefaultNaN(N);
18            FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
19        elsif (inf1 && sign1 == '0') || (inf2 && sign2 == '0') then
20            result = FPInfinity('0', N);
21        elsif (inf1 && sign1 == '1') || (inf2 && sign2 == '1') then
22            result = FPInfinity('1', N);
23        elsif zero1 && zero2 && sign1 == sign2 then
24            result = FPZero(sign1, N);
25        else
26            result_value = value1 + value2;
27            if result_value == 0.0 then // Sign of exact zero result depends
28                                    // on rounding mode
29                result_sign = if fpscr_val.RMode == FPSCR_RMode_RM then '1' else '0';
30                result = FPZero(result_sign, N);
31            else
32                result = FPRound(result_value, N, fpscr_val, predicated);
33    return result;

```

E2.1.143 FPB_CheckBreakPoint

```

1 // FPB_CheckBreakPoint
2 // =====
3 // Check for Flash Patch Break point.
4
5 boolean FPB_CheckBreakPoint(bits(32) iaddr, integer size,
6                             boolean is_ifetch, boolean is_secure)
7
8     match = FPB_CheckMatchAddress(iaddr);
9     if !match && size == 4 && FPB_CheckMatchAddress(iaddr + 2) then
10         match = ConstrainUnpredictableBool(Unpredictable_FPBKbreakpoint);
11     return match;

```

E2.1.144 FPB_CheckMatchAddress

```

1 // FPB_CheckMatchAddress
2 // =====
3 // Flash Patch breakpoint instruction address comparison.
4

```

```

5  boolean FPB_CheckMatchAddress(bits(32) iaddr)
6
7      if FP_CTRL.ENABLE == '0' then return FALSE; // FPB not enabled.
8
9      // Instruction Comparator.
10     num_addr_cmp = UInt(FP_CTRL.NUM_CODE);
11     if num_addr_cmp == 0 then return FALSE;      // No comparator support.
12
13     for N = 0 to (num_addr_cmp - 1)
14         if FP_COMP[N].BE == '1' then            // Breakpoint enabled.
15             if iaddr[31:1] == FP_COMP[N].BPADDR then
16                 return TRUE;
17
18     return FALSE;

```

E2.1.145 FPCompare

```

1  // FPCompare()
2  // =====
3
4  (bit, bit, bit, bit) FPCompare(bits(N) op1, bits(N) op2, boolean quiet_nan_exc,
5                                boolean fpscr_controlled)
6      return FPCompare(op1, op2, quiet_nan_exc, fpscr_controlled, FALSE);
7
8  (bit, bit, bit, bit) FPCompare(bits(N) op1, bits(N) op2, boolean quiet_nan_exc,
9                                boolean fpscr_controlled, boolean predicated)
10
11     assert N IN {16,32,64};
12     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
13     (type1,-,value1) = FPUnpack(op1, fpscr_val, predicated);
14     (type2,-,value2) = FPUnpack(op2, fpscr_val, predicated);
15     if type1 == FPType_SNaN || type1 == FPType_QNaN ||
16        type2 == FPType_SNaN || type2 == FPType_QNaN then
17         result = ('0','0','1','1');
18         if type1==FPType_SNaN || type2==FPType_SNaN || quiet_nan_exc then
19             FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
20     else
21         // All non-NaN cases can be evaluated on the values produced by FPUnpack().
22         if value1 == value2 then
23             result = ('0','1','1','0');
24         elseif value1 < value2 then
25             result = ('1','0','0','0');
26         else // value1 > value2
27             result = ('0','0','1','0');
28     return result;

```

E2.1.146 FPConvertNaN

```

1  // FPConvertNaN()
2  // =====
3  //
4  // For half-precision data it ignores AHP, and observes FZ16.
5  // Calls FPConvertNaNBase() which demotes any input SNaN to a QNaN.
6
7  bits(N) FPConvertNaN(bits(N) fpval, boolean fpscr_controlled)
8      fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
9      fpscr_val.AHP = '0';
10     return FPConvertNaNBase(fpval, fpscr_val);

```

E2.1.147 FPConvertNaNBase

```

1  // FPConvertNaNBase()
2  // =====
3  //
4  // Demotes any input SNaN to a QNaN and ensures that any comparison between a number and
5  // a NaN, always returns the number and not the NaN.

```

```

6
7 bits(N) FPConvertNaNBase(bits(N) fpval, FPSCR_Type fpscr_val)
8     assert N IN {16,32,64};
9
10     if N == 16 then
11         exp16 = fpval[14:10];
12         frac16 = fpval[9:0];
13         if IsOnes(exp16) && fpscr_val.AHP == '0' then // Infinity or NaN in IEEE format.
14             if !IsZero(frac16) then // NaN in IEEE format
15                 if frac16[9] == '0' then // if the value is an SNaN.
16                     fpval[9] = '1'; // Convert the value to a QNaN.
17                     FPProcessException(FPExc_InvalidOp, fpscr_val);
18
19     elseif N == 32 then
20         exp32 = fpval[30:23];
21         frac32 = fpval[22:0];
22         if IsOnes(exp32) && !IsZero(frac32) then // NaN in IEEE format
23             if frac32[22] == '0' then // if the value is an SNaN.
24                 fpval[22] = '1'; // Convert the value to a QNaN.
25                 FPProcessException(FPExc_InvalidOp, fpscr_val);
26
27     else // N == 64
28         exp64 = fpval[62:52];
29         frac64 = fpval[51:0];
30         if IsOnes(exp64) && !IsZero(frac64) then // NaN in IEEE format
31             if frac64[51] == '0' then // if the value is an SNaN.
32                 fpval[51] = '1'; // Convert the value to a QNaN.
33                 FPProcessException(FPExc_InvalidOp, fpscr_val);
34
35     return fpval;

```

E2.1.148 FPDefaultNaN

```

1 // FPDefaultNaN()
2 // =====
3
4 bits(N) FPDefaultNaN(integer N)
5     assert N IN {16,32,64};
6     integer E = if N == 16 then 5 elseif N == 32 then 8 else 11;
7     constant integer F = N - E - 1;
8     bits(E) exp;
9     bits(F) frac;
10    sign = '0';
11    exp = Ones(E);
12    frac = '1':Zeros(F-1);
13    return sign : exp : frac;

```

E2.1.149 FPDiv

```

1 // FPDiv()
2 // =====
3
4 bits(N) FPDiv(bits(N) op1, bits(N) op2, boolean fpscr_controlled)
5     assert N IN {16,32,64};
6     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
7     (fp_type1,sign1,value1) = FPUnpack(op1, fpscr_val);
8     (fp_type2,sign2,value2) = FPUnpack(op2, fpscr_val);
9     (done,result) = FPProcessNaNs(fp_type1, fp_type2, op1, op2, fpscr_val);
10    if !done then
11        inf1 = (fp_type1 == FType_Infinity); inf2 = (fp_type2 == FType_Infinity);
12        zero1 = (fp_type1 == FType_Zero); zero2 = (fp_type2 == FType_Zero);
13        if (inf1 && inf2) || (zero1 && zero2) then
14            result = FPDefaultNaN(N);
15            FPProcessException(FPExc_InvalidOp, fpscr_val);
16        elseif inf1 || zero2 then
17            result_sign = if sign1 == sign2 then '0' else '1';
18            result = FPInfinity(result_sign, N);

```

```

19         if !inf1 then FPProcessException(FPExc_DivideByZero, fpscr_val);
20     elseif zero1 || inf2 then
21         result_sign = if sign1 == sign2 then '0' else '1';
22         result = FPZero(result_sign, N);
23     else
24         result = FPRound(value1/value2, N, fpscr_val);
25     return result;

```

E2.1.150 FPDoubleToHalf

```

1  // FPDoubleToHalf()
2  // =====
3
4  bits(16) FPDoubleToHalf(bits(64) operand, boolean fpscr_controlled)
5      fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
6      (fp_type, sign, value) = FPUnpackCV(operand, fpscr_val);
7      if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
8          if fpscr_val.AHP == '1' then
9              result = FPZero(sign, 16);
10         elseif fpscr_val.DN == '1' then
11             result = FPDefaultNaN(16);
12         else
13             result = sign : '11111 1' : operand[50:42];
14             if fp_type == FPType_SNaN || fpscr_val.AHP == '1' then
15                 FPProcessException(FPExc_InvalidOp, fpscr_val);
16     elseif fp_type == FPType_Infinity then
17         if fpscr_val.AHP == '1' then
18             result = sign : Ones(15);
19             FPProcessException(FPExc_InvalidOp, fpscr_val);
20         else
21             result = FPInfinity(sign, 16);
22     elseif fp_type == FPType_Zero then
23         result = FPZero(sign, 16);
24     else
25         result = FPRoundCV(value, 16, fpscr_val);
26     return result;

```

E2.1.151 FPDoubleToSingle

```

1  // FPDoubleToSingle()
2  // =====
3
4  bits(32) FPDoubleToSingle(bits(64) operand, boolean fpscr_controlled)
5      fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
6      (fp_type, sign, value) = FPUnpackCV(operand, fpscr_val);
7      if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
8          if fpscr_val.DN == '1' then
9              result = FPDefaultNaN(32);
10         else
11             result = sign : '11111111 1' : operand[50:29];
12             if fp_type == FPType_SNaN then
13                 FPProcessException(FPExc_InvalidOp, fpscr_val);
14     elseif fp_type == FPType_Infinity then
15         result = FPInfinity(sign, 32);
16     elseif fp_type == FPType_Zero then
17         result = FPZero(sign, 32);
18     else
19         result = FPRoundCV(value, 32, fpscr_val);
20     return result;

```

E2.1.152 FPExc

```

1  // Floating-point exceptions
2  // =====
3

```

```

4 enumeration FPExc {FPExc_InvalidOp, FPExc_DivideByZero, FPExc_Overflow,
5                   FPExc_Underflow, FPExc_Inexact, FPExc_InputDenorm};

```

E2.1.153 FPHalfToDouble

```

1 // FPHalfToDouble()
2 // =====
3
4 bits(64) FPHalfToDouble(bits(16) operand, boolean fpscr_controlled)
5     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
6     (fp_type,sign,value) = FPUnpackCV(operand, fpscr_val);
7     if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
8         if fpscr_val.DN == '1' then
9             result = FPDefaultNaN(64);
10        else
11            result = sign : '1111111111 1' : operand[8:0] : Zeros(42);
12            if fp_type == FPType_SNaN then
13                FPProcessException(FPExc_InvalidOp, fpscr_val);
14            elsif fp_type == FPType_Infinity then
15                result = FPInfinity(sign, 64);
16            elsif fp_type == FPType_Zero then
17                result = FPZero(sign, 64);
18            else
19                result = FPRoundCV(value, 64, fpscr_val); // Rounding will be exact
20            return result;

```

E2.1.154 FPHalfToSingle

```

1 // FPHalfToSingle()
2 // =====
3
4 bits(32) FPHalfToSingle(bits(16) operand, boolean fpscr_controlled)
5     return FPHalfToSingle(operand, fpscr_controlled, FALSE);
6
7 bits(32) FPHalfToSingle(bits(16) operand, boolean fpscr_controlled, boolean predicated)
8     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
9     (fp_type,sign,value) = FPUnpackCV(operand, fpscr_val, predicated);
10    if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
11        if fpscr_val.DN == '1' then
12            result = FPDefaultNaN(32);
13        else
14            result = sign : '11111111 1' : operand[8:0] : Zeros(13);
15            if fp_type == FPType_SNaN then
16                FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
17            elsif fp_type == FPType_Infinity then
18                result = FPInfinity(sign, 32);
19            elsif fp_type == FPType_Zero then
20                result = FPZero(sign, 32);
21            else
22                result = FPRoundCV(value, 32, fpscr_val, predicated); // Rounding will be exact
23            return result;

```

E2.1.155 FPInfinity

```

1 // FPInfinity()
2 // =====
3
4 bits(N) FPInfinity(bit sign, integer N)
5     assert N IN {16,32,64};
6     integer E = if N == 16 then 5 elsif N == 32 then 8 else 11;
7     constant integer F = N - E - 1;
8     bits(E) exp;
9     bits(F) frac;
10    exp = Ones(E);
11    frac = Zeros(F);
12    return sign : exp : frac;

```

E2.1.156 FPMMax

```

1 // FPMMax()
2 // =====
3
4 bits(N) FPMMax(bits(N) op1, bits(N) op2, boolean fpscr_controlled)
5     return FPMMax(op1, op2, fpscr_controlled, FALSE);
6
7 bits(N) FPMMax(bits(N) op1, bits(N) op2, boolean fpscr_controlled, boolean predicated)
8     assert N IN {16,32,64};
9     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
10    (fp_type1,sign1,value1) = FPUnpack(op1, fpscr_val, predicated);
11    (fp_type2,sign2,value2) = FPUnpack(op2, fpscr_val, predicated);
12    (done,result) = FPProcessNaNs(fp_type1, fp_type2, op1, op2, fpscr_val, predicated);
13    if !done then
14        if value1 > value2 then
15            (fp_type,sign,value) = (fp_type1,sign1,value1);
16        else
17            (fp_type,sign,value) = (fp_type2,sign2,value2);
18        if fp_type == FPType_Infinity then
19            result = FPInfinity(sign, N);
20        elsif fp_type == FPType_Zero then
21            sign = sign1 AND sign2; // Use most positive sign
22            result = FPZero(sign, N);
23        else
24            result = FPRound(value, N, fpscr_val, predicated);
25    return result;

```

E2.1.157 FPMMaxNormal

```

1 // FPMMaxNormal()
2 // =====
3
4 bits(N) FPMMaxNormal(bit sign, integer N)
5     assert N IN {16,32,64};
6     integer E = if N == 16 then 5 elsif N == 32 then 8 else 11;
7     constant integer F = N - E - 1;
8     bits(E) exp;
9     bits(F) frac;
10    exp = Ones(E-1):'0';
11    frac = Ones(F);
12    return sign : exp : frac;

```

E2.1.158 FPMMaxNum

```

1 // FPMMaxNum()
2 // =====
3
4 bits(N) FPMMaxNum(bits(N) op1, bits(N) op2, boolean fpscr_controlled)
5     return FPMMaxNum(op1, op2, fpscr_controlled, FALSE);
6
7 bits(N) FPMMaxNum(bits(N) op1, bits(N) op2, boolean fpscr_controlled, boolean predicated)
8     assert N IN {16,32,64};
9
10    fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
11    (type1,-,-) = FPUnpack(op1, fpscr_val, predicated);
12    (type2,-,-) = FPUnpack(op2, fpscr_val, predicated);
13
14    // Treat a single quiet-NaN as -Infinity
15    if type1 == FPType_QNaN && type2 != FPType_QNaN then
16        op1 = FPInfinity('1', N);
17    elsif type1 != FPType_QNaN && type2 == FPType_QNaN then
18        op2 = FPInfinity('1', N);
19
20    return FPMMax(op1, op2, fpscr_controlled, predicated);

```


E2.1.159 FPMIn

```

1  // FPMIn()
2  // =====
3
4  bits(N) FPMIn(bits(N) op1, bits(N) op2, boolean fpSCR_controlled)
5      return FPMIn(op1, op2, fpSCR_controlled, FALSE);
6
7  bits(N) FPMIn(bits(N) op1, bits(N) op2, boolean fpSCR_controlled, boolean predicated)
8      assert N IN {16,32,64};
9      fpSCR_val = if fpSCR_controlled then FPSCR else StandardFPSCRValue();
10     (fp_type1,sign1,value1) = FPUnpack(op1, fpSCR_val, predicated);
11     (fp_type2,sign2,value2) = FPUnpack(op2, fpSCR_val, predicated);
12     (done,result) = FPProcessNaNs(fp_type1, fp_type2, op1, op2, fpSCR_val, predicated);
13     if !done then
14         if value1 < value2 then
15             (fp_type,sign,value) = (fp_type1,sign1,value1);
16         else
17             (fp_type,sign,value) = (fp_type2,sign2,value2);
18         if fp_type == FPType_Infinity then
19             result = FPInfinity(sign, N);
20         elsif fp_type == FPType_Zero then
21             sign = sign1 OR sign2; // Use most negative sign
22             result = FPZero(sign, N);
23         else
24             result = FPRound(value, N, fpSCR_val, predicated);
25     return result;

```

E2.1.160 FPMInNum

```

1  // FPMInNum()
2  // =====
3
4  bits(N) FPMInNum(bits(N) op1, bits(N) op2, boolean fpSCR_controlled)
5      return FPMInNum(op1, op2, fpSCR_controlled, FALSE);
6
7  bits(N) FPMInNum(bits(N) op1, bits(N) op2, boolean fpSCR_controlled, boolean predicated)
8      assert N IN {16,32,64};
9
10     fpSCR_val = if fpSCR_controlled then FPSCR else StandardFPSCRValue();
11     (fp_type1,-,-) = FPUnpack(op1, fpSCR_val, predicated);
12     (fp_type2,-,-) = FPUnpack(op2, fpSCR_val, predicated);
13
14     // Treat a single quiet-NaN as +Infinity
15     if fp_type1 == FPType_QNaN && fp_type2 != FPType_QNaN then
16         op1 = FPInfinity('0', N);
17     elsif fp_type1 != FPType_QNaN && fp_type2 == FPType_QNaN then
18         op2 = FPInfinity('0', N);
19
20     return FPMIn(op1, op2, fpSCR_controlled, predicated);

```

E2.1.161 FPMul

```

1  // FPMul()
2  // =====
3
4  bits(N) FPMul(bits(N) op1, bits(N) op2, boolean fpSCR_controlled)
5      return FPMul(op1, op2, fpSCR_controlled, FALSE);
6
7  bits(N) FPMul(bits(N) op1, bits(N) op2, boolean fpSCR_controlled, boolean predicated)
8      assert N IN {16,32,64};
9      fpSCR_val = if fpSCR_controlled then FPSCR else StandardFPSCRValue();
10     (type1,sign1,value1) = FPUnpack(op1, fpSCR_val, predicated);
11     (type2,sign2,value2) = FPUnpack(op2, fpSCR_val, predicated);
12     (done,result) = FPProcessNaNs(type1, type2, op1, op2, fpSCR_val, predicated);
13     if !done then

```

```

14     inf1 = (type1 == FPType_Infinity);  inf2 = (type2 == FPType_Infinity);
15     zero1 = (type1 == FPType_Zero);    zero2 = (type2 == FPType_Zero);
16     if (inf1 && zero2) || (zero1 && inf2) then
17         result = FPDefaultNaN(N);
18         FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
19     elseif inf1 || inf2 then
20         result_sign = if sign1 == sign2 then '0' else '1';
21         result = FPInfinity(result_sign, N);
22     elseif zero1 || zero2 then
23         result_sign = if sign1 == sign2 then '0' else '1';
24         result = FPZero(result_sign, N);
25     else
26         result = FPRound(value1*value2, N, fpscr_val, predicated);
27     return result;

```

E2.1.162 FPMulAdd

```

1  // FPMulAdd()
2  // =====
3  // Calculates addend + op1*op2 with a single rounding.
4
5  bits(N) FPMulAdd(bits(N) addend, bits(N) op1, bits(N) op2, boolean fpscr_controlled)
6      return FPMulAdd(addend, op1, op2, fpscr_controlled, FALSE);
7
8  bits(N) FPMulAdd(bits(N) addend, bits(N) op1, bits(N) op2, boolean fpscr_controlled,
9      boolean predicated)
10     assert N IN {16,32,64};
11     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
12     (typeA,signA,valueA) = FPUnpack(addend, fpscr_val, predicated);
13     (type1,sign1,value1) = FPUnpack(op1, fpscr_val, predicated);
14     (type2,sign2,value2) = FPUnpack(op2, fpscr_val, predicated);
15     inf1 = (type1 == FPType_Infinity); zero1 = (type1 == FPType_Zero);
16     inf2 = (type2 == FPType_Infinity); zero2 = (type2 == FPType_Zero);
17     (done,result) = FPProcessNaNs3(typeA, type1, type2, addend, op1, op2, fpscr_val,
18         predicated);
19
20     if typeA == FPType_QNaN && ((inf1 && zero2) || (zero1 && inf2)) then
21         result = FPDefaultNaN(N);
22         FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
23
24     if !done then
25         infA = (typeA == FPType_Infinity);  zeroA = (typeA == FPType_Zero);
26
27         // Determine sign and type product will have if it does not cause an Invalid
28         // Operation.
29         signP = if sign1 == sign2 then '0' else '1';
30         infP = inf1 || inf2;
31         zeroP = zero1 || zero2;
32
33         // Non SNaN-generated Invalid Operation cases are multiplies of zero by infinity and
34         // additions of opposite-signed infinities.
35         if (inf1 && zero2) || (zero1 && inf2) || (infA && infP && signA == NOT(signP)) then
36             result = FPDefaultNaN(N);
37             FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
38
39         // Other cases involving infinities produce an infinity of the same sign.
40         elseif (infA && signA == '0') || (infP && signP == '0') then
41             result = FPInfinity('0', N);
42         elseif (infA && signA == '1') || (infP && signP == '1') then
43             result = FPInfinity('1', N);
44
45         // Cases where the result is exactly zero and its sign is not determined by the
46         // rounding mode are additions of same-signed zeros.
47         elseif zeroA && zeroP && signA == signP then
48             result = FPZero(signA, N);
49
50         // Otherwise calculate numerical result and round it.
51     else

```

```

52     result_value = valueA + (value1 * value2);
53     if result_value == 0.0 then // Sign of exact zero result depends
54                               // on rounding mode
55         result_sign = if fpscr_val.RMode == FPSCR_RMode_RM then '1' else '0';
56         result = FPZero(result_sign, N);
57     else
58         result = FPRound(result_value, N, fpscr_val, predicated);
59
60     return result;

```

E2.1.163 FPNeg

```

1 // FPNeg()
2 // =====
3
4 bits(N) FPNeg(bits(N) operand)
5     assert N IN {16,32,64};
6     return NOT(operand[N-1]) : operand[N-2:0];

```

E2.1.164 FPProcessException

```

1 // FPProcessException()
2 // =====
3 // The 'fpscr_val' argument supplies FPSCR control bits. Status information is
4 // updated directly in FPSCR where appropriate.
5
6 FPProcessException(FPExc exception, FPSCR_Type fpscr_val)
7     FPProcessException(exception, fpscr_val, FALSE);
8     return;
9
10 FPProcessException(FPExc exception, FPSCR_Type fpscr_val, boolean predicated)
11     // Get appropriate FPSCR bit numbers.
12     if !predicated then
13         case exception of
14             when FPExc_InvalidOp      enable = 8;    cumul = 0;
15             when FPExc_DivideByZero   enable = 9;    cumul = 1;
16             when FPExc_Overflow        enable = 10;   cumul = 2;
17             when FPExc_Underflow       enable = 11;   cumul = 3;
18             when FPExc_Inexact         enable = 12;   cumul = 4;
19             when FPExc_InputDenorm     enable = 15;   cumul = 7;
20         if fpscr_val[enable] == '1' then
21             IMPLEMENTATION_DEFINED "floating-point trap handling";
22         else
23             FPSCR[cumul] = '1';
24     return;

```

E2.1.165 FPProcessNaN

```

1 // FPProcessNaN()
2 // =====
3 // The 'fpscr_val' argument supplies FPSCR control bits. Status information is
4 // updated directly in FPSCR where appropriate.
5
6 bits(N) FPProcessNaN(FPType fp_type, bits(N) operand, FPSCR_Type fpscr_val)
7     return FPProcessNaN(fp_type, operand, fpscr_val, FALSE);
8
9 bits(N) FPProcessNaN(FPType fp_type, bits(N) operand, FPSCR_Type fpscr_val,
10                     boolean predicated)
11     assert N IN {16,32,64};
12     if N == 16 then topfrac = 9;
13     elsif N == 32 then topfrac = 22;
14     else topfrac = 51;
15     result = operand;
16     if fp_type == FPType_SNaN then
17         result[topfrac] = '1';

```

```

18     FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
19     if fpscr_val.DN == '1' then // DefaultNaN requested
20         result = FPDefaultNaN(N);
21     return result;

```

E2.1.166 FPProcessNaNs

```

1  // FPProcessNaNs()
2  // =====
3  // The boolean part of the return value says whether a NaN has been found and
4  // processed. The bits(N) part is only relevant if it has and supplies the
5  // result of the operation.
6  //
7  // The 'fpscr_val' argument supplies FPSCR control bits. Status information is
8  // updated directly in FPSCR where appropriate.
9
10 (boolean, bits(N)) FPProcessNaNs(FPType type1, FPType type2, bits(N) op1, bits(N) op2,
11                                 bits(32) fpscr_val)
12     return FPProcessNaNs(type1, type2, op1, op2, fpscr_val, FALSE);
13
14 (boolean, bits(N)) FPProcessNaNs(FPType type1, FPType type2, bits(N) op1, bits(N) op2,
15                                 bits(32) fpscr_val, boolean predicated)
16     assert N IN {16,32,64};
17     if type1 == FPType_SNaN then
18         done = TRUE; result = FPProcessNaN(type1, op1, fpscr_val, predicated);
19     elseif type2 == FPType_SNaN then
20         done = TRUE; result = FPProcessNaN(type2, op2, fpscr_val, predicated);
21     elseif type1 == FPType_QNaN then
22         done = TRUE; result = FPProcessNaN(type1, op1, fpscr_val, predicated);
23     elseif type2 == FPType_QNaN then
24         done = TRUE; result = FPProcessNaN(type2, op2, fpscr_val, predicated);
25     else
26         done = FALSE; result = Zeros(N); // 'Don't care' result
27     return (done, result);

```

E2.1.167 FPProcessNaNs3

```

1  // FPProcessNaNs3()
2  // =====
3  // The boolean part of the return value says whether a NaN has been found and
4  // processed. The bits(N) part is only relevant if it has and supplies the
5  // result of the operation.
6  //
7  // The 'fpscr_val' argument supplies FPSCR control bits. Status information is
8  // updated directly in FPSCR where appropriate.
9
10 (boolean, bits(N)) FPProcessNaNs3(FPType type1, FPType type2, FPType type3,
11                                   bits(N) op1, bits(N) op2, bits(N) op3,
12                                   bits(32) fpscr_val)
13     return FPProcessNaNs3(type1, type2, type3, op1, op2, op3, fpscr_val, FALSE);
14
15 (boolean, bits(N)) FPProcessNaNs3(FPType type1, FPType type2, FPType type3,
16                                   bits(N) op1, bits(N) op2, bits(N) op3,
17                                   bits(32) fpscr_val, boolean predicated)
18     assert N IN {16,32,64};
19     if type1 == FPType_SNaN then
20         done = TRUE; result = FPProcessNaN(type1, op1, fpscr_val, predicated);
21     elseif type2 == FPType_SNaN then
22         done = TRUE; result = FPProcessNaN(type2, op2, fpscr_val, predicated);
23     elseif type3 == FPType_SNaN then
24         done = TRUE; result = FPProcessNaN(type3, op3, fpscr_val, predicated);
25     elseif type1 == FPType_QNaN then
26         done = TRUE; result = FPProcessNaN(type1, op1, fpscr_val, predicated);
27     elseif type2 == FPType_QNaN then
28         done = TRUE; result = FPProcessNaN(type2, op2, fpscr_val, predicated);
29     elseif type3 == FPType_QNaN then
30         done = TRUE; result = FPProcessNaN(type3, op3, fpscr_val, predicated);

```

```

31     else
32         done = FALSE; result = Zeros(N); // 'Do not care' result
33     return (done, result);

```

E2.1.168 FPRound

```

1 // FPRound()
2 // =====
3 // Used by data processing and int/fixed <-> floating-point conversion instructions.
4 // For half-precision data it ignores AHP, and observes FZ16.
5
6 bits(N) FPRound(real value, integer N, FPSCR_Type fpSCR_val)
7     return FPRound(value, N, fpSCR_val, FALSE);
8
9 bits(N) FPRound(real value, integer N, FPSCR_Type fpSCR_val, boolean predicated)
10     fpSCR_val.AHP = '0';
11     return FPRoundBase(value, N, fpSCR_val, predicated);

```

E2.1.169 FPRoundBase

```

1 // FPRoundBase()
2 // =====
3 // The 'fpSCR_val' argument supplies FPSCR control bits. Status information is
4 // updated directly in FPSCR where appropriate.
5
6 bits(N) FPRoundBase(real value, integer N, FPSCR_Type fpSCR_val, boolean predicated)
7     assert N IN {16,32,64};
8     assert value != 0.0;
9
10    // Obtain format parameters - minimum exponent, numbers of exponent and fraction bits.
11    integer E = if N == 16 then 5 elsif N == 32 then 8 else 11;
12    minimum_exp = 2 - 2^(E-1);
13    constant integer F = N - E - 1;
14
15    // Split value into sign, unrounded mantissa and exponent.
16    if value < 0.0 then
17        sign = '1'; mantissa = -value;
18    else
19        sign = '0'; mantissa = value;
20    exponent = 0;
21    while mantissa < 1.0 do
22        mantissa = mantissa * 2.0; exponent = exponent - 1;
23    while mantissa >= 2.0 do
24        mantissa = mantissa / 2.0; exponent = exponent + 1;
25
26    // Deal with flush-to-zero.
27    if ((N != 16 && fpSCR_val.FZ == '1') || (N == 16 && fpSCR_val.FZ16 == '1')) &&
28        exponent < minimum_exp then
29        result = FPZero(sign, N);
30        if !predicated then FPSCR.UFC = '1'; // Flush-to-zero never generates a trapped
31                                           // exception.
32    else
33
34        // Start creating the exponent value for the result. Start by biasing the actual
35        // exponent so that the minimum exponent becomes 1, lower values 0 (indicating
36        // possible underflow).
37        biased_exp = Max(exponent - minimum_exp + 1, 0);
38        if biased_exp == 0 then mantissa = mantissa / 2.0^(minimum_exp - exponent);
39
40        // Get the unrounded mantissa as an integer, and the "units in last place"
41        // rounding error.
42        int_mant = RoundDown(mantissa * 2.0^F); // if biased_exp == 0, < 2.0^F
43                                           // otherwise >= 2.0^F
44        error = mantissa * 2.0^F - Real(int_mant);
45
46        // Underflow occurs if exponent is too small before rounding, and result is inexact
47        // or the Underflow exception is trapped.

```

```

48     if biased_exp == 0 && error != 0.0 then
49         FPProcessException(FPExc_Underflow, fpscr_val, predicated);
50
51     // Round result according to rounding mode.
52     case fpscr_val.RMode of
53         when FPSCR_RMode_RN // Round to Nearest (rounding to even if exactly halfway)
54             round_up = (error > 0.5 || (error == 0.5 && int_mant[0] == '1'));
55             overflow_to_inf = TRUE;
56         when FPSCR_RMode_RP // Round towards Plus Infinity
57             round_up = (error != 0.0 && sign == '0');
58             overflow_to_inf = (sign == '0');
59         when FPSCR_RMode_RM // Round towards Minus Infinity
60             round_up = (error != 0.0 && sign == '1');
61             overflow_to_inf = (sign == '1');
62         when FPSCR_RMode_RZ // Round towards Zero
63             round_up = FALSE;
64             overflow_to_inf = FALSE;
65     if round_up then
66         int_mant = int_mant + 1;
67         if int_mant == 2^F then // Rounded up from denormalized to normalized
68             biased_exp = 1;
69         if int_mant == 2^(F+1) then // Rounded up to next exponent
70             biased_exp = biased_exp + 1; int_mant = int_mant DIV 2;
71
72     // Deal with overflow and generate result.
73     if N != 16 || fpscr_val.AHP == '0' then // Single, double or IEEE half precision
74         if biased_exp >= 2^E - 1 then
75             result = if overflow_to_inf then FPInfinity(sign,N) else FPMMaxNormal(sign,N);
76             FPProcessException(FPExc_Overflow, fpscr_val, predicated);
77             error = 1.0; // Ensure that an Inexact exception occurs
78         else
79             result = sign : biased_exp[E-1:0] : int_mant[F-1:0];
80     else // Alternative half precision
81         if biased_exp >= 2^E then
82             result = sign : Ones(N-1);
83             FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
84             error = 0.0; // Ensure that an Inexact exception does not occur
85         else
86             result = sign : biased_exp[E-1:0] : int_mant[F-1:0];
87
88     // Deal with Inexact exception.
89     if error != 0.0 then
90         FPProcessException(FPExc_Inexact, fpscr_val, predicated);
91
92     return result;

```

E2.1.170 FPRoundCV

```

1 // FPRoundCV()
2 // =====
3 // Used for floating-point <-> floating-point conversion instructions.
4 // For half-precision data processing operations the FZ16 bit
5 // is ignored and the AHP bit is observed.
6
7 bits(N) FPRoundCV(real value, integer N, FPSCR_Type fpscr_val)
8     return FPRoundCV(value, N, fpscr_val, FALSE);
9
10 bits(N) FPRoundCV(real value, integer N, FPSCR_Type fpscr_val, boolean predicated)
11     fpscr_val.FZ16 = '0';
12     return FPRoundBase(value, N, fpscr_val, predicated);

```

E2.1.171 FPRoundInt

```

1 // FPRoundInt()
2 // =====
3 // Round floating-point value to nearest integral floating-point value
4 // using given rounding mode. If exact is TRUE, set inexact flag if result

```

```

5  // is not numerically equal to given value.
6
7  bits(N) FPRoundInt(bits(N) op, bits(2) rmode, boolean away, boolean exact,
8                    boolean fpscr_controlled)
9      return FPRoundInt(op, rmode, away, exact, fpscr_controlled, FALSE);
10
11 bits(N) FPRoundInt(bits(N) op, bits(2) rmode, boolean away, boolean exact,
12                  boolean fpscr_controlled, boolean predicated)
13     assert N IN {16,32,64};
14     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
15
16     // Unpack using FPSCR to determine if subnormals are flushed-to-zero.
17     (fp_type,sign,value) = FPUnpack(op, fpscr_val, predicated);
18
19     if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
20         result = FPProcessNaN(fp_type, op, fpscr_val, predicated);
21     elsif fp_type == FPType_Infinity then
22         result = FPInfinity(sign, N);
23     elsif fp_type == FPType_Zero then
24         result = FPZero(sign, N);
25     else
26         // extract integer component
27         int_result = RoundDown(value);
28         error = value - Real(int_result);
29
30         // Determine whether supplied rounding mode requires an increment.
31         case rmode of
32             when '00' // Round to nearest, ties to even
33                 round_up = (error > 0.5 || (error == 0.5 && int_result[0] == '1'));
34             when '01' // Round towards Plus Infinity
35                 round_up = (error != 0.0);
36             when '10' // Round towards Minus Infinity
37                 round_up = FALSE;
38             when '11' // Round towards Zero
39                 round_up = (error != 0.0 && int_result < 0);
40
41         if away then // Round towards Zero, ties away.
42             round_up = (error > 0.5 || (error == 0.5 && int_result >= 0));
43
44         if round_up then int_result = int_result + 1;
45
46         // Convert integer value into an equivalent real value.
47         real_result = Real(int_result);
48
49         // Re-encode as a floating-point value, result is always exact.
50         if real_result == 0.0 then
51             result = FPZero(sign, N);
52         else
53             result = FPRound(real_result, N, fpscr_val, predicated);
54
55         // Generate inexact exceptions.
56         if error != 0.0 && exact then
57             FPProcessException(FPExc_Inexact, fpscr_val, predicated);
58
59     return result;

```

E2.1.172 FPSingleToDouble

```

1  // FPSingleToDouble()
2  // =====
3
4  bits(64) FPSingleToDouble(bits(32) operand, boolean fpscr_controlled)
5      fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
6      (fp_type,sign,value) = FPUnpackCV(operand, fpscr_val);
7      if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
8          if fpscr_val.DN == '1' then
9              result = FPDefaultNaN(64);
10         else

```

```

11     result = sign : '1111111111 1' : operand[21:0] : Zeros(29);
12     if fp_type == FPType_SNaN then
13         FPProcessException(FPExc_InvalidOp, fpscr_val);
14     elsif fp_type == FPType_Infinity then
15         result = FPInfinity(sign, 64);
16     elsif fp_type == FPType_Zero then
17         result = FPZero(sign, 64);
18     else
19         result = FPRoundCV(value, 64, fpscr_val); // Rounding will be exact
20     return result;

```

E2.1.173 FPSingleToHalf

```

1 // FPSingleToHalf()
2 // =====
3
4 bits(16) FPSingleToHalf(bits(32) operand, boolean fpscr_controlled)
5     return FPSingleToHalf(operand, fpscr_controlled, FALSE);
6
7 bits(16) FPSingleToHalf(bits(32) operand, boolean fpscr_controlled, boolean predicated)
8     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
9     (fp_type, sign, value) = FPUnpackCV(operand, fpscr_val, predicated);
10    if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
11        if fpscr_val.AHP == '1' then
12            result = FPZero(sign, 16);
13        elsif fpscr_val.DN == '1' then
14            result = FPDefaultNaN(16);
15        else
16            result = sign : '11111 1' : operand[21:13];
17            if fp_type == FPType_SNaN || fpscr_val.AHP == '1' then
18                FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
19            elsif fp_type == FPType_Infinity then
20                if fpscr_val.AHP == '1' then
21                    result = sign : Ones(15);
22                    FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
23                else
24                    result = FPInfinity(sign, 16);
25            elsif fp_type == FPType_Zero then
26                result = FPZero(sign, 16);
27            else
28                result = FPRoundCV(value, 16, fpscr_val, predicated);
29    return result;

```

E2.1.174 FPSqrt

```

1 // FPSqrt()
2 // =====
3
4 bits(N) FPSqrt(bits(N) operand)
5     assert N IN {16, 32, 64};
6     (fp_type, sign, value) = FPUnpack(operand, FPSCR);
7     if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
8         result = FPProcessNaN(fp_type, operand, FPSCR);
9     elsif fp_type == FPType_Zero then
10        result = FPZero(sign, N);
11    elsif fp_type == FPType_Infinity && sign == '0' then
12        result = FPInfinity(sign, N);
13    elsif sign == '1' then
14        result = FPDefaultNaN(N);
15        FPProcessException(FPExc_InvalidOp, FPSCR);
16    else
17        result = FPRound(Sqrt(value), N, FPSCR);
18    return result;

```

E2.1.175 FPSub


```

1  // FPSub()
2  // =====
3
4  bits(N) FPSub(bits(N) op1, bits(N) op2, boolean fpscr_controlled)
5      return FPSub(op1, op2, fpscr_controlled, FALSE);
6
7  bits(N) FPSub(bits(N) op1, bits(N) op2, boolean fpscr_controlled, boolean predicated)
8      assert N IN {16,32,64};
9      fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
10     (fp_type1,sign1,value1) = FPUnpack(op1, fpscr_val, predicated);
11     (fp_type2,sign2,value2) = FPUnpack(op2, fpscr_val, predicated);
12     (done,result) = FPProcessNaNs(fp_type1, fp_type2, op1, op2, fpscr_val, predicated);
13     if !done then
14         inf1 = (fp_type1 == FPType_Infinity); inf2 = (fp_type2 == FPType_Infinity);
15         zero1 = (fp_type1 == FPType_Zero); zero2 = (fp_type2 == FPType_Zero);
16         if inf1 && inf2 && sign1 == sign2 then
17             result = FPDefaultNaN(N);
18             FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
19         elsif (inf1 && sign1 == '0') || (inf2 && sign2 == '1') then
20             result = FPInfinity('0', N);
21         elsif (inf1 && sign1 == '1') || (inf2 && sign2 == '0') then
22             result = FPInfinity('1', N);
23         elsif zero1 && zero2 && sign1 == NOT(sign2) then
24             result = FPZero(sign1, N);
25         else
26             result_value = value1 - value2;
27             if result_value == 0.0 then // Sign of exact zero result depends
28                                     // on rounding mode
29                 result_sign = if fpscr_val.RMode == FPSCR_RMode_RM then '1' else '0';
30                 result = FPZero(result_sign, N);
31             else
32                 result = FPRound(result_value, N, fpscr_val, predicated);
33     return result;

```

E2.1.176 FPToFixed

```

1  // FPToFixed()
2  // =====
3
4  bits(M) FPToFixed(bits(N) operand, integer M, integer fraction_bits, boolean unsigned,
5                  boolean round_towards_zero, boolean fpscr_controlled)
6      return FPToFixed(operand, M, fraction_bits, unsigned, round_towards_zero,
7                      fpscr_controlled, FALSE);
8
9  bits(M) FPToFixed(bits(N) operand, integer M, integer fraction_bits, boolean unsigned,
10                  boolean round_towards_zero, boolean fpscr_controlled,
11                  boolean predicated)
12      assert N IN {16,32,64};
13      fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
14      if round_towards_zero then fpscr_val.RMode = FPSCR_RMode_RZ;
15      (fp_type, -, value) = FPUnpack(operand, fpscr_val, predicated);
16
17      // For NaNs and infinities, FPUnpack() has produced a value that will round to the
18      // required result of the conversion. Also, the value produced for infinities will
19      // cause the conversion to overflow and signal an Invalid Operation floating-point
20      // exception as required. NaNs must also generate such a floating-point exception.
21      if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
22          FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
23
24      // Scale value by specified number of fraction bits, then start rounding to an integer
25      // and determine the rounding error.
26      value = value * 2.0^fraction_bits;
27      int_result = RoundDown(value);
28      error = value - Real(int_result);
29
30      // Apply the specified rounding mode.
31      case fpscr_val.RMode of
32          when FPSCR_RMode_RN // Round to Nearest (rounding to even if exactly halfway)

```

```

33     round_up = (error > 0.5 || (error == 0.5 && int_result[0] == '1'));
34     when FPSCR_RMode_RP // Round towards Plus Infinity
35         round_up = (error != 0.0);
36     when FPSCR_RMode_RM // Round towards Minus Infinity
37         round_up = FALSE;
38     when FPSCR_RMode_RZ // Round towards Zero
39         round_up = (error != 0.0 && int_result < 0);
40     if round_up then int_result = int_result + 1;
41
42     // Bitstring result is the integer result saturated to the destination size, with
43     // saturation indicating overflow of the conversion (signaled as an Invalid
44     // Operation floating-point exception).
45     (result, overflow) = SatQ(int_result, M, unsigned);
46     if overflow then
47         FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
48     elsif error != 0.0 then
49         FPProcessException(FPExc_Inexact, fpscr_val, predicated);
50
51     return result;

```

E2.1.177 FPToFixedDirected

```

1  // FPToFixedDirected()
2  // =====
3
4  bits(M) FPToFixedDirected(bits(N) op, integer fbits, boolean unsigned, bits(2) round_mode,
5                          boolean fpscr_controlled)
6      return FPToFixedDirected(op, fbits, unsigned, round_mode, fpscr_controlled, FALSE);
7
8  bits(M) FPToFixedDirected(bits(N) op, integer fbits, boolean unsigned, bits(2) round_mode,
9                          boolean fpscr_controlled, boolean predicated)
10     assert N IN {16,32,64};
11
12     fpscr_val = if fpscr_controlled then FPSCR else StandardFPSCRValue();
13
14     // Unpack using FPSCR to determine if subnormals are flushed-to-zero.
15     (fp_type, -, value) = FPUnpack(op, fpscr_val, predicated);
16
17     // If NaN, set cumulative flag or take exception
18     if fp_type == FPType_SNaN || fp_type == FPType_QNaN then
19         FPProcessException(FPExc_InvalidOp, FPSCR, predicated);
20
21     // Scale by fractional bits and produce integer rounded towards
22     // minus-infinity.
23     value = value * 2.0^fbits;
24     int_result = RoundDown(value);
25     error = value - Real(int_result);
26
27     // Determine whether supplied rounding mode requires an increment.
28     case round_mode of
29         when '00' // ties away
30             round_up = (error > 0.5 || (error == 0.5 && int_result >= 0));
31         when '01' // nearest even
32             round_up = (error > 0.5 || (error == 0.5 && int_result[0] == '1'));
33         when '10' // plus infinity
34             round_up = (error != 0.0);
35         when '11' // neg infinity
36             round_up = FALSE;
37
38     if round_up then int_result = int_result + 1;
39
40     // Generate saturated result and exceptions.
41     (result, overflow) = SatQ(int_result, M, unsigned);
42
43     if overflow then
44         FPProcessException(FPExc_InvalidOp, fpscr_val, predicated);
45     elsif error != 0.0 then
46         FPProcessException(FPExc_Inexact, fpscr_val, predicated);

```

```
47     return result;
```

E2.1.178 FPType

```
1 // FPType
2 // =====
3
4 // Type of floating-point value. Floating-point values are categorized into one
5 // of the following type during unpacking.
6
7 enumeration FPType {FPType_Nonzero, FPType_Zero, FPType_Infinity, FPType_QNaN, FPType_SNaN};
```

E2.1.179 FPUnpack

```
1 // FPUnpack()
2 // =====
3 //
4 // Used by data processing and int/fixed <-> floating-point conversion instructions.
5 // For half-precision data it ignores AHP, and observes FZ16.
6
7 (FPType, bit, real) FPUnpack(bits(N) fpval, FPSCR_Type fpscr_val)
8     return FPUnpack(fpval, fpscr_val, FALSE);
9
10 (FPType, bit, real) FPUnpack(bits(N) fpval, FPSCR_Type fpscr_val, boolean predicated)
11     fpscr_val.AHP = '0';
12     return FPUnpackBase(fpval, fpscr_val, predicated);
```

E2.1.180 FPUnpackBase

```
1 // FPUnpackBase()
2 // =====
3 //
4 // Unpack a floating-point number into its type, sign bit and the real number
5 // that it represents. The real number result has the correct sign for numbers
6 // and infinities, is very large in magnitude for infinities, and is 0.0 for
7 // NaNs. (These values are chosen to simplify the description of comparisons
8 // and conversions).
9 //
10 // The 'fpscr_val' argument supplies FPSCR control bits. Status information is
11 // updated directly in FPSCR where appropriate.
12
13 (FPType, bit, real) FPUnpackBase(bits(N) fpval, FPSCR_Type fpscr_val, boolean predicated)
14     assert N IN {16,32,64};
15
16     if N == 16 then
17         sign    = fpval[15];
18         expl6   = fpval[14:10];
19         frac16  = fpval[9:0];
20         if IsZero(expl6) then
21             // Produce zero if value is zero or flush-to-zero is selected.
22             if IsZero(frac16) || fpscr_val.FZ16 == '1' then
23                 fp_type = FPType_Zero; value = 0.0;
24             else
25                 fp_type = FPType_Nonzero; value = 2.0^-14 * (Real(UInt(frac16)) * 2.0^-10);
26         elseif IsOnes(expl6) && fpscr_val.AHP == '0' then // Infinity or NaN in IEEE format
27             if IsZero(frac16) then
28                 fp_type = FPType_Infinity; value = 2.0^1000000;
29             else
30                 fp_type = if frac16[9] == '1' then FPType_QNaN else FPType_SNaN;
31                 value    = 0.0;
32         else
33             fp_type = FPType_Nonzero;
34             value    = 2.0^(UInt(expl6)-15) * (1.0 + Real(UInt(frac16)) * 2.0^-10);
35
36     elseif N == 32 then
```

```

37
38     sign    = fpval[31];
39     exp32   = fpval[30:23];
40     frac32  = fpval[22:0];
41     if IsZero(exp32) then
42         // Produce zero if value is zero or flush-to-zero is selected.
43         if IsZero(frac32) || fpscr_val.FZ == '1' then
44             fp_type = FPType_Zero; value = 0.0;
45             if !IsZero(frac32) then // Denormalized input flushed to zero
46                 FPProcessException(FPExc_InputDenorm, fpscr_val, predicated);
47         else
48             fp_type = FPType_Nonzero; value = 2.0^-126 * (Real(UInt(frac32)) * 2.0^-23);
49     elseif IsOnes(exp32) then
50         if IsZero(frac32) then
51             fp_type = FPType_Infinity; value = 2.0^1000000;
52         else
53             fp_type = if frac32[22] == '1' then FPType_QNaN else FPType_SNaN;
54             value = 0.0;
55     else
56         fp_type = FPType_Nonzero;
57         value = 2.0^(UInt(exp32)-127) * (1.0 + Real(UInt(frac32)) * 2.0^-23);
58
59     else // N == 64.
60
61         sign    = fpval[63];
62         exp64   = fpval[62:52];
63         frac64  = fpval[51:0];
64         if IsZero(exp64) then
65             // Produce zero if value is zero or flush-to-zero is selected.
66             if IsZero(frac64) || fpscr_val.FZ == '1' then
67                 fp_type = FPType_Zero; value = 0.0;
68                 if !IsZero(frac64) then // Denormalized input flushed to zero
69                     FPProcessException(FPExc_InputDenorm, fpscr_val, predicated);
70             else
71                 fp_type = FPType_Nonzero;
72                 value = 2.0^-1022 * (Real(UInt(frac64)) * 2.0^-52);
73         elseif IsOnes(exp64) then
74             if IsZero(frac64) then
75                 fp_type = FPType_Infinity;
76                 value = 2.0^1000000;
77             else
78                 fp_type = if frac64[51] == '1' then FPType_QNaN else FPType_SNaN;
79                 value = 0.0;
80         else
81             fp_type = FPType_Nonzero;
82             value = 2.0^(UInt(exp64)-1023) * (1.0 + Real(UInt(frac64)) * 2.0^-52);
83
84     if sign == '1' then value = -value;
85     return (fp_type, sign, value);

```

E2.1.181 FPUnpackCV

```

1 // FPUnpackCV()
2 // =====
3 //
4 // Used for floating-point <-> floating-point conversion instructions.
5 // For half-precision data ignores FZ16 and observes AHP.
6
7 (FPType, bit, real) FPUnpackCV(bits(N) fpval, FPSCR_Type fpscr_val)
8     return FPUnpackCV(fpval, fpscr_val, FALSE);
9
10 (FPType, bit, real) FPUnpackCV(bits(N) fpval, FPSCR_Type fpscr_val, boolean predicated)
11     fpscr_val.FZ16 = '0';
12     return FPUnpackBase(fpval, fpscr_val, predicated);

```

E2.1.182 FPZero

```

1 // FPZero()
2 // =====
3
4 bits(N) FPZero(bit sign, integer N)
5     assert N IN {16,32,64};
6     integer E = if N == 16 then 5 elsif N == 32 then 8 else 11;
7     constant integer F = N - E - 1;
8     bits(E) exp;
9     bits(F) frac;
10    exp = Zeros(E);
11    frac = Zeros(F);
12    return sign : exp : frac;

```

E2.1.183 FunctionReturn

```

1 // FunctionReturn()
2 // =====
3
4 ExcInfo FunctionReturn()
5     exc = DefaultExcInfo();
6
7     // Pull the return address and IPSR off the Secure stack.
8     mode = CurrentMode();
9     spName = LookUpSP_with_security_mode(TRUE, mode);
10    framePtr = _SP(spName);
11    if !IsAligned(framePtr, 8) then UNPREDICTABLE;
12    // Only stack locations, not the load order are architected
13    RETPSR_Type newPSR;
14    if exc.fault == NoFault then (exc, newPSR) = Stack(framePtr, 4, spName, mode);
15    if exc.fault == NoFault then (exc, newPC) = Stack(framePtr, 0, spName, mode);
16
17    // Check the IPSR value that has been unstacked is consistent with the current
18    // mode, and being originally called from the Secure state.
19    // NOTE: It is IMPLEMENTATION DEFINED whether this check is performed before
20    // or after the load of the return address above.
21    if (exc.fault == NoFault) &&
22        !(((IPSR.Exception == 0[8:0]) && (newPSR.Exception == 0[8:0])) ||
23          ((IPSR.Exception == 1[8:0]) && (newPSR.Exception != 0[8:0]))) then
24        if HaveMainExt() then
25            UFSR_S.INVPC = '1';
26            // Create the exception. NOTE: If the Main Extension is not implemented
27            // then the fault always escalates to a HardFault.
28            exc = CreateException(UsageFault, TRUE, TRUE);
29            // The IPSR value is set as UNKNOWN if the IPSR value is not supported by the PE.
30            excNum = UInt(newPSR.Exception);
31            validIPSR = excNum IN {0, 1, NMI, HardFault, SVCcall, PendSV, SysTick};
32            if !validIPSR && HaveMainExt() then
33                validIPSR = excNum IN {MemManage, BusFault, UsageFault, SecureFault, DebugMonitor};
34            if !validIPSR && !IsIrqValid(excNum) then
35                newPSR.Exception = bits(9) UNKNOWN;
36
37            // Only consume the function return stack frame and update the XPSR and PC
38            // if no faults occurred.
39            if exc.fault == NoFault then
40                // Transition to the Secure state.
41                CurrentState = SecurityState_Secure;
42                // Update stack pointer. NOTE: Stack pointer limit not checked on function
43                // return as stack pointer guaranteed to be ascending not descending.
44                exc = _SP(spName, FALSE, TRUE, framePtr + 8);
45                assert exc.fault == NoFault;
46
47                IPSR.Exception = newPSR.Exception;
48                CONTROL_S.SFPA = newPSR.SFPA;
49                // IT/ICI/ECI/LOB data cleared to prevent Non-secure code interfering with
50                // Secure execution.
51                if HaveMainExt() then
52                    ITSTATE = Zeros(8);
53                if HaveLOBExt() then

```

```

54         LO_BRANCH_INFO.VALID = '0';
55         // If EPSR.T == 0, a UsageFault('Invalid State') or a HardFault is taken
56         // on the next instruction depending on whether the Main Extension is
57         // is implemented or not.
58         EPSR.T = newPC[0];
59         BranchTo(newPC[31:1]:'0');
60     return exc;

```

E2.1.184 GenerateCoproprocessorException

```

1  // GenerateCoproprocessorException()
2  // =====
3
4  GenerateCoproprocessorException()
5      UFSR.UNDEFINSTR = '1';
6      excInfo = CreateException(UsageFault);
7      HandleException(excInfo);

```

E2.1.185 GenerateDebugEventResponse

```

1  // GenerateDebugEventResponse()
2  // =====
3  // Generate a debug event response based on the PE configuration.
4
5  GenerateDebugEventResponse(DebugEventCause cause)
6      if CanHaltOnEvent(IsSecure(), FindPriv()) then
7          DHCSR.C_HALT = '1';
8          if cause != DebugEventCause_EXT then
9              DFSR.BKPT = '1';
10             // Internally generated debug events halt synchronously and terminate
11             // the current instruction. Halt() will be called following completion
12             // of in flight instructions.
13             EndOfInstruction();
14         else
15             DFSR.EXTERNAL = '1';
16
17     elseif cause == DebugEventCause_BKPT then
18         // The Breakpoint debug event is set unconditionally.
19         DFSR.BKPT = '1';
20         // A DebugMonitor exception is generated. This exception can be escalated to
21         // a HardFault when evaluated in ExceptionDetails().
22         exceptionSecurity = IsSecure() || DEMCR.SDME == '1';
23         excInfo = CreateException(DebugMonitor, TRUE, exceptionSecurity);
24         HandleException(excInfo);
25
26     elseif CanPendMonitorOnEvent(IsSecure(), TRUE, TRUE, FindPriv(), FALSE) then
27         if cause != DebugEventCause_EXT then
28             DFSR.BKPT = '1';
29         else
30             DFSR.EXTERNAL = '1';
31             DEMCR.MON_PEND = '1';
32             if cause == DebugEventCause_FPB then
33                 EndOfInstruction();

```

E2.1.186 GenerateIntegerZeroDivide

```

1  // GenerateIntegerZeroDivide()
2  // =====
3
4  GenerateIntegerZeroDivide()
5      UFSR.DIVBYZERO = '1';
6      excInfo = CreateException(UsageFault);
7      HandleException(excInfo);

```

E2.1.187 GetActiveChains

```

1 // GetActiveChains()
2 // =====
3
4 integer GetActiveChains()
5     count = 0;
6     if HaveMve() then
7         for i = 0 to MAX_OVERLAPPING_INSTRS-1
8             if _InstInfo[i].Valid then
9                 count = count + 1;
10    return count;

```

E2.1.188 GetCurInstrBeat

```

1 // GetCurInstrBeat()
2 // =====
3
4 (integer, bits(4)) GetCurInstrBeat()
5     assert HaveMve();
6     // By default assume all lanes are active.
7     elmtMask = Ones(4);
8
9     // If VPT active apply the predicate flags in VPR.P0.
10    if VPTActive() then
11        elmtMask = elmtMask AND Elem[VPR.P0, _BeatID, 4];
12
13    // LOB truncation may override the flags on the last iteration of a loop
14    // LTPSIZE < 4 is a proxy for knowing if loop and tail predication is active.
15    ltpsize = if _CurrentInstrExecState.ResetLTPSize then 4 else LTPSIZE;
16    if ltpsize < 4 && IsLastLowOverheadLoop() then
17        loopCount = _CurrentInstrExecState.LoopCount;
18        predSize = ltpsize;
19        fullMask = ZeroExtend(Ones(UInt(loopCount[4-predSize:0] : Zeros(predSize))), 16);
20        elmtMask = elmtMask AND Elem[fullMask, _BeatID, 4];
21    return (_BeatID, elmtMask);

```

E2.1.189 GetInstrExecState

```

1 // GetInstrExecState()
2 // =====
3
4 INSTR_EXEC_STATE_Type GetInstrExecState(integer next)
5     // next = 0: returns current (committed) state.
6     // next > 0: returns n-th state from now.
7     assert (next >= 0 && next < MAX_BEATS);
8     INSTR_EXEC_STATE_Type state;
9
10    // 1) Next == 0: current committed state
11    state.FetchAddr = _RName[RNamesPC];
12    state.ITState = EPSR.IT;
13    state.L = '0';
14    state.T16IND = '0';
15    state.BTI = '0';
16    state.LoopCount = LR;
17    state.LOBranchInfoValid = LO_BRANCH_INFO.VALID;
18    state.ResetLTPSize = FALSE;
19
20    // 2) Determine speculative future.
21    for i = 1 to next
22        // Handle normal PC changes BEFORE LOB handling.
23        if _PCChanged && i == 1 then
24            state.FetchAddr = _NextInstrAddr;
25        else
26            state.FetchAddr = state.FetchAddr + ThisInstrLength(i-1);
27
28    // If the IT state has been directly modified return that value as the
29    // next state, otherwise advance the IT state normally.
30    if _ITStateChanged && i == 1 then

```

```

31     state.ITState = _NextInstrITState;
32     elseif InITBlock(state.ITState) then
33         state.ITState = ITAdvance(state.ITState);
34         // When an ICI compatible instruction that is outside of an IT block completes, the
35         // ICI bits are cleared. Since the EPSR.ICI and EPSR.IT fields overlap, clearing the
36         // ICI bits is accomplished by setting ITState to zero.
37     elseif HasArchVersion(Armv8p1) && IsLoadStoreClearMultInstruction(ThisInstr(i-1)) then
38         state.ITState = Zeros(8);
39
40     // Check if loop or branch triggers PC change (unless normal PC change).
41     if (!_PCChanged) && HaveLOBExt() then
42         state = HandleLO(state);
43
44     return state;

```

E2.1.190 GetMveScalarReadRegs

```

1  bits(16) GetMveScalarReadRegs(bits(32) instr)
2      assert(IsMveBeatWiseInstruction(instr));
3      regs = Zeros(16);
4      if (instr == '111x1110100xxxx0xxxx111x0x0xxx1' ||
5          instr == '111x11101010xxxx0xxxx111x0x0xxx1') then // VABAV-R.QQ-T1
6          Elem[regs, UInt( instr[15:12] ), 1] = '1';
7      elseif (instr == '111011100x0xxxx1xxx0111x100xxxx' ||
8          instr == '111011100x10xxx1xxx0111x100xxxx') then // VADD-Q.Qx-T2
9          Elem[regs, UInt( instr[3:0] ), 1] = '1';
10     elseif (instr == '111x11101110xx01xxxx111100x0xxx0' ||
11         instr == '111x1110110xxx01xxx111100x0xxx0' ||
12         instr == '111x111010xxxx01xxx111100x0xxx0') then // VADDLV-RR.Q-T1
13         Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
14         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
15     elseif (instr == '111x111011110x01xxxx111100x0xxx0' ||
16         instr == '111x11101111001xxx111100x0xxx0') then // VADDV-R.Q-T1
17         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
18     elseif (instr == '111x11100x11xxx0xxx0111x100xxxx') then // VADD_af-Q.Qx-T2
19         Elem[regs, UInt( instr[3:0] ), 1] = '1';
20     elseif (instr == '111111100x10xxx1xxx1110x110xxxx' ||
21         instr == '111111100x0xxxx1xxx1110x110xxxx') then // VBRSR-Q.Qx-T1
22         Elem[regs, UInt( instr[3:0] ), 1] = '1';
23     elseif (instr == '111x11100011xxx1000x111x1x010xx' ||
24         instr == '111x11100011xxx1000x111x1x00xxx' ||
25         instr == '111x11100011xxx1000x111x1x011x0' ||
26         instr == '111x11100011xxx1000x111x1x01x1x') then // VCMP_f-P.Qx-T2
27         Elem[regs, UInt( instr[3:0] ), 1] = '1';
28     elseif (instr == '11111110000xxxx100001111x100xxxx' ||
29         instr == '111111100010xxx100001111x100xxxx') then // VCMP_i-P.Qx-T4
30         Elem[regs, UInt( instr[3:0] ), 1] = '1';
31     elseif (instr == '111111100010xxx100001111x110xxxx' ||
32         instr == '11111110000xxxx100001111x110xxxx') then // VCMP_i-P.Qx-T5
33         Elem[regs, UInt( instr[3:0] ), 1] = '1';
34     elseif (instr == '11111110000xxxx10001111x1x0xxxx' ||
35         instr == '111111100010xxx10001111x1x0xxxx') then // VCMP_i-P.Qx-T6
36         Elem[regs, UInt( instr[3:0] ), 1] = '1';
37     elseif (instr == '1111000000xx0xxx11101xxxxxxxxxx1' ||
38         instr == '1111000000xx111011101xxxxxxxxxx1' ||
39         instr == '1111000000xx110x11101xxxxxxxxxx1' ||
40         instr == '1111000000xx10xx11101xxxxxxxxxx1') then // VCTP-R-T1
41         Elem[regs, UInt( instr[19:16] ), 1] = '1';
42     elseif (instr == '111011100x10xxx1xxx1111x110111x' ||
43         instr == '111011100x0xxxx1xxx1111x110111x') then // VDDUP-Q.RI-T2
44         Elem[regs, UInt( instr[19:17]:'0' ), 1] = '1';
45     elseif (instr == '111011100x10xxx1xxx1111x1100xxx' ||
46         instr == '111011100x0xxx1xxx1111x11010xx' ||
47         instr == '111011100x0xxx1xxx1111x1100xxx' ||
48         instr == '111011100x10xxx1xxx1111x110110x' ||
49         instr == '111011100x0xxx1xxx1111x110110x' ||
50         instr == '111011100x10xxx1xxx1111x11010xx') then // VDDUP-Q.RRI-T1
51         Elem[regs, UInt( instr[3:1]:'1' ), 1] = '1';

```



```

52     Elem[regs, UInt( instr[19:17]:'0' ), 1] = '1';
53     elseif (instr == '111011101x10xxx0xxx1011x0x1xxxx') then // VDUP-Q.R-T1
54     Elem[regs, UInt( instr[15:12] ), 1] = '1';
55     elseif (instr == '111x11100x11xxx1xxx1110x100xxxx') then // VFMAS_fs-Q.QR-T1
56     Elem[regs, UInt( instr[3:0] ), 1] = '1';
57     elseif (instr == '111x11100x11xxx1xxx01110x100xxxx') then // VFMA_sf-Q.QR-T1
58     Elem[regs, UInt( instr[3:0] ), 1] = '1';
59     elseif (instr == '111x11100x10xxx0xxx01111x100xxxx' ||
60             instr == '111x11100x0xxx0xxx01111x100xxxx') then // VHADD-Q.QR-T2
61     Elem[regs, UInt( instr[3:0] ), 1] = '1';
62     elseif (instr == '111x11100x10xxx0xxx1111x100xxxx' ||
63             instr == '111x11100x0xxx0xxx1111x100xxxx') then // VHSUB-Q.QR-T2
64     Elem[regs, UInt( instr[3:0] ), 1] = '1';
65     elseif (instr == '111011100x10xxx1xxx01111x11011x' ||
66             instr == '111011100x0xxx1xxx01111x11011x') then // VIDUP-Q.RI-T2
67     Elem[regs, UInt( instr[19:17]:'0' ), 1] = '1';
68     elseif (instr == '111011100x0xxx1xxx01111x1100xxx' ||
69             instr == '111011100x10xxx1xxx01111x11010xx' ||
70             instr == '111011100x0xxx1xxx01111x11010xx' ||
71             instr == '111011100x0xxx1xxx01111x110110x' ||
72             instr == '111011100x10xxx1xxx01111x110110x' ||
73             instr == '111011100x10xxx1xxx01111x1100xxx') then // VIDUP-Q.RRI-T1
74     Elem[regs, UInt( instr[3:1]:'1' ), 1] = '1';
75     Elem[regs, UInt( instr[19:17]:'0' ), 1] = '1';
76     elseif (instr == '111111001xx1xxxxxxx1110xxxxxx0' ||
77             instr == '111111001xx1xxxxxxx11110xxxxxx0') then // VLD2-Q.RI-T1
78     Elem[regs, UInt( instr[19:16] ), 1] = '1';
79     elseif (instr == '111111001xx1xxxxxxx11110xxxxxx1' ||
80             instr == '111111001xx1xxxxxxx11110xxxxxx1') then // VLD4-Q.RI-T1
81     Elem[regs, UInt( instr[19:16] ), 1] = '1';
82     elseif (instr == '111x110xx0110xxxxxx01110xxxxxxx' ||
83             instr == '111x110xx0110xxxxxx011110xxxxxxx' ||
84             instr == '111x1101x0x10xxxxxx01110xxxxxxx' ||
85             instr == '111x1101x0x10xxxxxx011110xxxxxxx') then // VLDR-Q.RI-T1
86     Elem[regs, UInt( instr[18:16] ), 1] = '1';
87     elseif (instr == '111x1101x0x11xxxxxx011110xxxxxxx' ||
88             instr == '111x110xx0111xxxxxx011110xxxxxxx' ||
89             instr == '111x110xx0111xxxxxx01110xxxxxxx' ||
90             instr == '111x1101x0x11xxxxxx01110xxxxxxx') then // VLDR-Q.RI-T2
91     Elem[regs, UInt( instr[18:16] ), 1] = '1';
92     elseif (instr == '1110110xxx11xxxxxx111100xxxxxx' ||
93             instr == '11101101xxx1xxxxxx111100xxxxxx') then // VLDR-Q.RI-T5
94     Elem[regs, UInt( instr[19:16] ), 1] = '1';
95     elseif (instr == '1110110xxx11xxxxxx11101xxxxxx' ||
96             instr == '11101101xxx1xxxxxx111101xxxxxx') then // VLDR-Q.RI-T6
97     Elem[regs, UInt( instr[19:16] ), 1] = '1';
98     elseif (instr == '1110110xxx1xxxxxx11110xxxxxxx' ||
99             instr == '1110110xxx11xxxxxx11110xxxxxxx') then // VLDR-Q.RI-T7
100    Elem[regs, UInt( instr[19:16] ), 1] = '1';
101    elseif (instr == '111x11001x01xxxxxx0111xx0x0xxx') then // VLDR_v-Q.xx-T1
102    Elem[regs, UInt( instr[19:16] ), 1] = '1';
103    elseif (instr == '111x11001x01xxxxxx0111xx0x1xxx') then // VLDR_v-Q.xx-T2
104    Elem[regs, UInt( instr[19:16] ), 1] = '1';
105    elseif (instr == '111x11001x01xxxxxx0111xx1x0xxx') then // VLDR_v-Q.xx-T3
106    Elem[regs, UInt( instr[19:16] ), 1] = '1';
107    elseif (instr == '111x11001x01xxxxxx0111xx1x1xxx') then // VLDR_v-Q.xx-T4
108    Elem[regs, UInt( instr[19:16] ), 1] = '1';
109    elseif (instr == '111x111011101110xxx111100x0xxx0') then // VMAXNMV_f-R.Q-T1
110    Elem[regs, UInt( instr[15:12] ), 1] = '1';
111    elseif (instr == '111x1110111011100xxx111100x0xxx0') then // VMAXNMV_f-R.Q-T2
112    Elem[regs, UInt( instr[15:12] ), 1] = '1';
113    elseif (instr == '111x111011100x10xxx111100x0xxx0' ||
114            instr == '111x111011101010xxx111100x0xxx0') then // VMAXV-R.Q-T1
115    Elem[regs, UInt( instr[15:12] ), 1] = '1';
116    elseif (instr == '111x111011100x00xxx111100x0xxx0' ||
117            instr == '111x111011101000xxx111100x0xxx0') then // VMAXV-R.Q-T2
118    Elem[regs, UInt( instr[15:12] ), 1] = '1';
119    elseif (instr == '111x111011101110xxx111110x0xxx0') then // VMINNMV_f-R.Q-T1
120    Elem[regs, UInt( instr[15:12] ), 1] = '1';

```

```

121     elsif (instr == '111x1101101100xxxx11110x0xxx0') then // VMINNMV_f-R.Q-T2
122         Elem[regs, UInt( instr[15:12] ), 1] = '1';
123     elsif (instr == '111x1101100x10xxxx11110x0xxx0' ||
124           instr == '111x1101101010xxxx11110x0xxx0') then // VMINV-R.Q-T1
125         Elem[regs, UInt( instr[15:12] ), 1] = '1';
126     elsif (instr == '111x1101100x00xxxx11110x0xxx0' ||
127           instr == '111x1101101000xxxx11110x0xxx0') then // VMINV-R.Q-T2
128         Elem[regs, UInt( instr[15:12] ), 1] = '1';
129     elsif (instr == '111x1100x0xxxx1xxx0110x100xxxx' ||
130           instr == '111x1100x10xxx1xxx0110x100xxxx') then // VMLA-Q.QR-T1
131         Elem[regs, UInt( instr[3:0] ), 1] = '1';
132     elsif (instr == '111x1101111xxxxxxx1110x0x0xxx0') then // VMLADAV-R.QQ-T1
133         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
134     elsif (instr == '111x1101111xxx0xxxx111x0x0xxx0') then // VMLADAV-R.QQ-T2
135         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
136     elsif (instr == '111x11010xxxxxxxxxx1110x0x0xxx0' ||
137           instr == '111x110110xxxxxxxxxx1110x0x0xxx0' ||
138           instr == '111x110110xxxxxxxxxx1110x0x0xxx0') then // VMLALDAV-RR.QQ-T1
139         Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
140         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
141     elsif (instr == '111x1100x0xxxx1xxx1110x100xxxx' ||
142           instr == '111x1100x10xxx1xxx1110x100xxxx') then // VMLAS-Q.QR-T1
143         Elem[regs, UInt( instr[3:0] ), 1] = '1';
144     elsif (instr == '11101101111xxxxxxx1110x0x0xxx1') then // VMLSDAV-R.QQ-T1
145         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
146     elsif (instr == '11111101111xxx0xxxx1110x0x0xxx1') then // VMLSDAV-R.QQ-T2
147         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
148     elsif (instr == '11101101110xxxxxxxx1110x0x0xxx1' ||
149           instr == '1110110110xxxxxxxx1110x0x0xxx1' ||
150           instr == '111011010xxxxxxxx1110x0x0xxx1') then // VMLSIDAV-RR.QQ-T1
151         Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
152         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
153     elsif (instr == '11101100xx0xxxxxxxx1011xxx1xxxx') then // VMOV_rs-Q.R-T1
154         Elem[regs, UInt( instr[15:12] ), 1] = '1';
155     elsif (instr == '111011000x01xxxxxxxx0111xxxxxxxx') then // VMOV_vrr-Q.RR-T1
156         Elem[regs, UInt( instr[3:0] ), 1] = '1';
157         Elem[regs, UInt( instr[19:16] ), 1] = '1';
158     elsif (instr == '11101100x10xxx1xxx1110x110xxxx' ||
159           instr == '11101100x0xxx1xxx1110x110xxxx') then // VMUL-Q.Qx-T2
160         Elem[regs, UInt( instr[3:0] ), 1] = '1';
161     elsif (instr == '111x1100x11xxx1xxx01110x110xxxx') then // VMUL_af-Q.QR-T2
162         Elem[regs, UInt( instr[3:0] ), 1] = '1';
163     elsif (instr == '11111100110xxx1xxx0111x100xxxx' ||
164           instr == '1111110000xxx10010111x100xxxx' ||
165           instr == '11111100010xxx11xx0111x100xxxx' ||
166           instr == '11111100010xxx10010111x100xxxx' ||
167           instr == '1111110010xxx1xxx0111x100xxxx' ||
168           instr == '11111100010xxx101x0111x100xxxx' ||
169           instr == '1111110000xxx101x0111x100xxxx' ||
170           instr == '1111110000xxx1xx0111x111x100xxxx') then // VPT-P.Qx-T4
171         Elem[regs, UInt( instr[3:0] ), 1] = '1';
172     elsif (instr == '1111110000xxx101x0111x110xxxx' ||
173           instr == '11111100010xxx11xx0111x110xxxx' ||
174           instr == '1111110000xxx11xx0111x110xxxx' ||
175           instr == '11111100010xxx10010111x110xxxx' ||
176           instr == '11111100110xxx1xxx0111x110xxxx' ||
177           instr == '1111110000xxx10010111x110xxxx' ||
178           instr == '1111110010xxx1xxx0111x110xxxx' ||
179           instr == '11111100010xxx101x0111x110xxxx') then // VPT-P.Qx-T5
180         Elem[regs, UInt( instr[3:0] ), 1] = '1';
181     elsif (instr == '11111100010xxx10011111x1x0xxxx' ||
182           instr == '1111110000xxx101x1111x1x0xxxx' ||
183           instr == '1111110000xxx11xx1111x1x0xxxx' ||
184           instr == '11111100110xxx1xxx1111x1x0xxxx' ||
185           instr == '1111110000xxx10011111x1x0xxxx' ||
186           instr == '1111110010xxx1xxx1111x1x0xxxx' ||
187           instr == '11111100010xxx101x1111x1x0xxxx' ||
188           instr == '11111100010xxx11xx1111x1x0xxxx') then // VPT-P.Qx-T6
189         Elem[regs, UInt( instr[3:0] ), 1] = '1';

```

```

190     elsif (instr == '111x11100011xxx101xx1111x1x00xxx' ||
191             instr == '111x11100111xxx1xxx1111x1x01x1x' ||
192             instr == '111x11100011xxx11xxx1111x1x011x0' ||
193             instr == '111x11100011xxx101xx1111x1x010xx' ||
194             instr == '111x11100011xxx11xxx1111x1x010xx' ||
195             instr == '111x11100011xxx11xxx1111x1x00xxx' ||
196             instr == '111x11100111xxx1xxx1111x1x011x0' ||
197             instr == '111x11100011xxx1001x1111x1x01x1x' ||
198             instr == '111x11100111xxx1xxx1111x1x010xx' ||
199             instr == '111x11100011xxx1001x1111x1x011x0' ||
200             instr == '111x11100011xxx101xx1111x1x01x1x' ||
201             instr == '111x11100111xxx1xxx1111x1x00xxx' ||
202             instr == '111x11100011xxx101xx1111x1x011x0' ||
203             instr == '111x11100011xxx1001x1111x1x010xx' ||
204             instr == '111x11100011xxx1001x1111x1x00xxx' ||
205             instr == '111x11100011xxx11xxx1111x1x01x1x') then // VPT_f-P.Qx-T2
206         Elem[regs, UInt( instr[3:0] ), 1] = '1';
207     elsif (instr == '111x11100x10xxx0xxx0111x110xxxx' ||
208             instr == '111x11100x0xxx0xxx0111x110xxxx') then // VQADD-Q.Qx-T2
209         Elem[regs, UInt( instr[3:0] ), 1] = '1';
210     elsif (instr == '111x11100x0xxx0xxx01110x110xxxx' ||
211             instr == '111x11100x10xxx0xxx01110x110xxxx') then // VQDMLAH-Q.QR-T1
212         Elem[regs, UInt( instr[3:0] ), 1] = '1';
213     elsif (instr == '111x11100x10xxx0xxx01110x110xxxx' ||
214             instr == '111x11100x0xxx0xxx01110x110xxxx') then // VQDMLAH-Q.QR-T2
215         Elem[regs, UInt( instr[3:0] ), 1] = '1';
216     elsif (instr == '111x11100x10xxx0xxx11110x110xxxx' ||
217             instr == '111x11100x0xxx0xxx11110x110xxxx') then // VQDMLASH-Q.QR-T1
218         Elem[regs, UInt( instr[3:0] ), 1] = '1';
219     elsif (instr == '111x11100x0xxx0xxx11110x110xxxx' ||
220             instr == '111x11100x10xxx0xxx11110x110xxxx') then // VQDMLASH-Q.QR-T2
221         Elem[regs, UInt( instr[3:0] ), 1] = '1';
222     elsif (instr == '111011100x10xxx1xxx01110x110xxxx' ||
223             instr == '111011100x0xxx1xxx01110x110xxxx') then // VQDMULH-Q.QR-T3
224         Elem[regs, UInt( instr[3:0] ), 1] = '1';
225     elsif (instr == '111111100x10xxx1xxx01110x110xxxx' ||
226             instr == '111111100x0xxx1xxx01110x110xxxx') then // VQDMULH-Q.QR-T4
227         Elem[regs, UInt( instr[3:0] ), 1] = '1';
228     elsif (instr == '111x11100x11xxx0xxx1111x110xxxx') then // VQDMULL-Q.Qx-T2
229         Elem[regs, UInt( instr[3:0] ), 1] = '1';
230     elsif (instr == '111x11100x111011xxx111101110xxxx' ||
231             instr == '111x11100x110x11xxx111101110xxxx') then // VQRSHL-Q.Qx-T2
232         Elem[regs, UInt( instr[3:0] ), 1] = '1';
233     elsif (instr == '111x11100x111001xxx111101110xxxx' ||
234             instr == '111x11100x110x01xxx111101110xxxx') then // VQSHL-Q.Qx-T1
235         Elem[regs, UInt( instr[3:0] ), 1] = '1';
236     elsif (instr == '111x11100x0xxx0xxx1111x110xxxx' ||
237             instr == '111x11100x10xxx0xxx1111x110xxxx') then // VQSUB-Q.Qx-T2
238         Elem[regs, UInt( instr[3:0] ), 1] = '1';
239     elsif (instr == '111x111010xxxx0xxx1111x0x0xxx0' ||
240             instr == '111x1110110xxxx0xxx1111x0x0xxx0') then // VRMLALDAVH-RR.QQ-T1
241         Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
242         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
243     elsif (instr == '111111101110xxx0xxx11110x0x0xxx1' ||
244             instr == '11111110110xxx0xxx11110x0x0xxx1' ||
245             instr == '1111111010xxxx0xxx11110x0x0xxx1') then // VRMLSLDAVH-RR.QQ-T1
246         Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
247         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
248     elsif (instr == '111x11100x111011xxx111100110xxxx' ||
249             instr == '111x11100x110x11xxx111100110xxxx') then // VRSHL-Q.Qx-T2
250         Elem[regs, UInt( instr[3:0] ), 1] = '1';
251     elsif (instr == '111x11100x110x01xxx111100110xxxx' ||
252             instr == '111x11100x111001xxx111100110xxxx') then // VSHL-Q.Qx-T2
253         Elem[regs, UInt( instr[3:0] ), 1] = '1';
254     elsif (instr == '111011101x1xxxxxxxx011111100xxxx') then // VSHLC-Q.RI-T1
255         Elem[regs, UInt( instr[3:0] ), 1] = '1';
256     elsif (instr == '111111001xx0xxxxxxxx1110xxxxxxxx0' ||
257             instr == '111111001xx0xxxxxxxx11110xxxxxxxx0') then // VST2-Q.RI-T1
258         Elem[regs, UInt( instr[19:16] ), 1] = '1';

```

```

259     elsif (instr == '111111001xx0xxxxxx11110xxxxxx1' ||
260           instr == '111111001xx0xxxxxx1110xxxxxx1') then // VST4-Q.RI-T1
261         Elem[regs, UInt( instr[19:16] ), 1] = '1';
262     elsif (instr == '111x1101x0x00xxxxxx01110xxxxxx' ||
263           instr == '111x1101x0x00xxxxxx01110xxxxxx' ||
264           instr == '111x110xx0100xxxxxx01110xxxxxx' ||
265           instr == '111x110xx0100xxxxxx01110xxxxxx') then // VSTR-Q.RI-T1
266         Elem[regs, UInt( instr[18:16] ), 1] = '1';
267     elsif (instr == '111x110xx0101xxxxxx01110xxxxxx' ||
268           instr == '111x1101x0x01xxxxxx01110xxxxxx' ||
269           instr == '111x1101x0x01xxxxxx01110xxxxxx' ||
270           instr == '111x110xx0101xxxxxx01110xxxxxx') then // VSTR-Q.RI-T2
271         Elem[regs, UInt( instr[18:16] ), 1] = '1';
272     elsif (instr == '11101101xxx0xxxxxx111100xxxxxx' ||
273           instr == '1110110xxx10xxxxxx111100xxxxxx') then // VSTR-Q.RI-T5
274         Elem[regs, UInt( instr[19:16] ), 1] = '1';
275     elsif (instr == '11101101xxx0xxxxxx111101xxxxxx' ||
276           instr == '1110110xxx10xxxxxx111101xxxxxx') then // VSTR-Q.RI-T6
277         Elem[regs, UInt( instr[19:16] ), 1] = '1';
278     elsif (instr == '11101101xxx0xxxxxx11110xxxxxx' ||
279           instr == '1110110xxx10xxxxxx11110xxxxxx') then // VSTR-Q.RI-T7
280         Elem[regs, UInt( instr[19:16] ), 1] = '1';
281     elsif (instr == '111x11001x00xxxxxx011xx0x0xxx' ) then // VSTR_v-Q.xx-T1
282         Elem[regs, UInt( instr[19:16] ), 1] = '1';
283     elsif (instr == '111x11001x00xxxxxx011xx0x1xxx' ) then // VSTR_v-Q.xx-T2
284         Elem[regs, UInt( instr[19:16] ), 1] = '1';
285     elsif (instr == '111x11001x00xxxxxx011xx1x0xxx' ) then // VSTR_v-Q.xx-T3
286         Elem[regs, UInt( instr[19:16] ), 1] = '1';
287     elsif (instr == '111x11001x00xxxxxx011xx1x1xxx' ) then // VSTR_v-Q.xx-T4
288         Elem[regs, UInt( instr[19:16] ), 1] = '1';
289     elsif (instr == '111011100x0xxxx1xxx1111x100xxxx' ||
290           instr == '111011100x10xxx1xxx1111x100xxxx') then // VSUB-Q.Qx-T2
291         Elem[regs, UInt( instr[3:0] ), 1] = '1';
292     elsif (instr == '111x11100x11xxx0xxx1111x100xxxx') then // VSUB_af-Q.Qx-T2
293         Elem[regs, UInt( instr[3:0] ), 1] = '1';
294     elsif (instr == '111x1110110xxxxxx01110x0xxx0' ||
295           instr == '111x111010xxxxxx01110x0xxx0' ||
296           instr == '111x1110110xxxxxx01110x0xxx0') then // alias VMLALV-RR.QQ-T1
297         Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
298         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
299     elsif (instr == '111x1110111xxxxxx01110x0xxx0') then // alias VMLAV-R.QQ-T1
300         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
301     elsif (instr == '111x1110110xxxx0xxx0111x0x0xxx0' ||
302           instr == '111x111010xxxx0xxx0111x0x0xxx0') then // alias VRMLALVH-RR.QQ-T1
303         Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
304         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
305     return regs;

```

E2.1.191 GetMveScalarWriteRegs

```

1  bits(16) GetMveScalarWriteRegs(bits(32) instr)
2  assert(IsMveBeatWiseInstruction(instr));
3  regs = Zeros(16);
4  if (instr == '111x1110100xxxx0xxx1111x0x0xxx1' ||
5     instr == '111x11101010xxx0xxx1111x0x0xxx1') then // VABAV-R.QQ-T1
6     Elem[regs, UInt( instr[15:12] ), 1] = '1';
7  elsif (instr == '111x11101110xx01xxx111100x0xxx0' ||
8        instr == '111x1110110xxx01xxx111100x0xxx0' ||
9        instr == '111x111010xxxx01xxx111100x0xxx0') then // VADDLV-RR.Q-T1
10     Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
11     Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
12  elsif (instr == '111x111011110x01xxx111100x0xxx0' ||
13        instr == '111x11101111001xxx111100x0xxx0') then // VADDV-R.Q-T1
14     Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
15  elsif (instr == '111011100x10xxx1xxx1111x110111x' ||
16        instr == '111011100x0xxxx1xxx1111x110111x') then // VDDUP-Q.RI-T2
17     Elem[regs, UInt( instr[19:17]:'0' ), 1] = '1';
18  elsif (instr == '111011100x10xxx1xxx1111x1100xxx' ||

```

```

19         instr == '111011100x0xxxx1xxx1111x11010xx' ||
20         instr == '111011100x0xxxx1xxx1111x1100xxx' ||
21         instr == '111011100x10xxx1xxx1111x110110x' ||
22         instr == '111011100x0xxxx1xxx1111x110110x' ||
23         instr == '111011100x10xxx1xxx1111x11010xx') then // VDDUP-Q.RRI-T1
24         Elem[regs, UInt( instr[19:17]:'0' ), 1] = '1';
25     elseif (instr == '111011100x10xxx1xxx01111x110111x' ||
26             instr == '111011100x0xxxx1xxx01111x110111x') then // VIDUP-Q.RI-T2
27         Elem[regs, UInt( instr[19:17]:'0' ), 1] = '1';
28     elseif (instr == '111011100x0xxxx1xxx01111x1100xxx' ||
29             instr == '111011100x10xxx1xxx01111x11010xx' ||
30             instr == '111011100x0xxxx1xxx01111x11010xx' ||
31             instr == '111011100x0xxxx1xxx01111x110110x' ||
32             instr == '111011100x10xxx1xxx01111x110110x' ||
33             instr == '111011100x10xxx1xxx01111x1100xxx') then // VIDUP-Q.RRI-T1
34         Elem[regs, UInt( instr[19:17]:'0' ), 1] = '1';
35     elseif (instr == '111111001xx1xxxxxxx11110xxxxxxx0' ||
36             instr == '111111001xx1xxxxxxx11110xxxxxxx0') then // VLD2-Q.RI-T1
37         Elem[regs, UInt( instr[19:16] ), 1] = '1';
38     elseif (instr == '111111001xx1xxxxxxx11110xxxxxxx1' ||
39             instr == '111111001xx1xxxxxxx11110xxxxxxx1') then // VLD4-Q.RI-T1
40         Elem[regs, UInt( instr[19:16] ), 1] = '1';
41     elseif (instr == '111x110xx0110xxxxxx01110xxxxxxx' ||
42             instr == '111x110xx0110xxxxxx01110xxxxxxx' ||
43             instr == '111x1101x0110xxxxxx01110xxxxxxx' ||
44             instr == '111x1101x0110xxxxxx01110xxxxxxx') then // VLDR-Q.RI-T1
45         Elem[regs, UInt( instr[18:16] ), 1] = '1';
46     elseif (instr == '111x1101x011xxxxxx01110xxxxxxx' ||
47             instr == '111x110xx011xxxxxx01110xxxxxxx' ||
48             instr == '111x110xx011xxxxxx01110xxxxxxx' ||
49             instr == '111x1101x011xxxxxx01110xxxxxxx') then // VLDR-Q.RI-T2
50         Elem[regs, UInt( instr[18:16] ), 1] = '1';
51     elseif (instr == '1110110xxx1xxxxxxx111100xxxxxxx' ||
52             instr == '11101101xxx1xxxxxxx111100xxxxxxx') then // VLDR-Q.RI-T5
53         Elem[regs, UInt( instr[19:16] ), 1] = '1';
54     elseif (instr == '1110110xxx1xxxxxxx111101xxxxxxx' ||
55             instr == '11101101xxx1xxxxxxx111101xxxxxxx') then // VLDR-Q.RI-T6
56         Elem[regs, UInt( instr[19:16] ), 1] = '1';
57     elseif (instr == '11101101xxx1xxxxxxx11110xxxxxxx' ||
58             instr == '1110110xxx1xxxxxxx11110xxxxxxx') then // VLDR-Q.RI-T7
59         Elem[regs, UInt( instr[19:16] ), 1] = '1';
60     elseif (instr == '111x111011101110xxxx111100x0xxx0') then // VMAXNMV_f-R.Q-T1
61         Elem[regs, UInt( instr[15:12] ), 1] = '1';
62     elseif (instr == '111x1110111011100xxx111100x0xxx0') then // VMAXNMV_f-R.Q-T2
63         Elem[regs, UInt( instr[15:12] ), 1] = '1';
64     elseif (instr == '111x111011100x10xxx111100x0xxx0' ||
65             instr == '111x111011101010xxx111100x0xxx0') then // VMAXV-R.Q-T1
66         Elem[regs, UInt( instr[15:12] ), 1] = '1';
67     elseif (instr == '111x111011100x00xxx111100x0xxx0' ||
68             instr == '111x111011101000xxx111100x0xxx0') then // VMAXV-R.Q-T2
69         Elem[regs, UInt( instr[15:12] ), 1] = '1';
70     elseif (instr == '111x111011101110xxxx111110x0xxx0') then // VMINNMV_f-R.Q-T1
71         Elem[regs, UInt( instr[15:12] ), 1] = '1';
72     elseif (instr == '111x1110111011100xxx111110x0xxx0') then // VMINNMV_f-R.Q-T2
73         Elem[regs, UInt( instr[15:12] ), 1] = '1';
74     elseif (instr == '111x111011100x10xxx111110x0xxx0' ||
75             instr == '111x111011101010xxx111110x0xxx0') then // VMINV-R.Q-T1
76         Elem[regs, UInt( instr[15:12] ), 1] = '1';
77     elseif (instr == '111x111011100x00xxx111110x0xxx0' ||
78             instr == '111x111011101000xxx111110x0xxx0') then // VMINV-R.Q-T2
79         Elem[regs, UInt( instr[15:12] ), 1] = '1';
80     elseif (instr == '111x11101111xxxxxxx1110x0xxx0') then // VMLADAV-R.QQ-T1
81         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
82     elseif (instr == '111x11101111xxx0xxx1111x0xxx0') then // VMLADAV-R.QQ-T2
83         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
84     elseif (instr == '111x111010xxxxxxx1110x0xxx0' ||
85             instr == '111x11101110xxxxxxx1110x0xxx0' ||
86             instr == '111x1110110xxxxxxx1110x0xxx0') then // VMLALDAV-RR.QQ-T1
87         Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';

```

```

88     Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
89     elsif (instr == '11101110111xxxxxxx1110x0x0xxx1') then // VMLSDAV-R.QQ-T1
90         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
91     elsif (instr == '11111110111xxx0xxxxx1110x0x0xxx1') then // VMLSDAV-R.QQ-T2
92         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
93     elsif (instr == '111011101110xxxxxxx1110x0x0xxx1' ||
94             instr == '11101110110xxxxxxx1110x0x0xxx1' ||
95             instr == '1110111010xxxxxxx1110x0x0xxx1') then // VMLSLDAV-RR.QQ-T1
96         Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
97         Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
98     elsif (instr == '111011000x00xxxxxx0111xxxxxxx') then // VMOV_rrv-RR.Q-T1
99         Elem[regs, UInt( instr[3:0] ), 1] = '1';
100        Elem[regs, UInt( instr[19:16] ), 1] = '1';
101    elsif (instr == '11101110xxx1xxxxxxx1011xxx1xxxx') then // VMOV_sr-Q.R-T1
102        Elem[regs, UInt( instr[15:12] ), 1] = '1';
103    elsif (instr == '111x111010xxxx0xxxx111x0x0xxx0' ||
104            instr == '111x1110110xxxx0xxxx111x0x0xxx0') then // VRMLALDAVH-RR.QQ-T1
105        Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
106        Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
107    elsif (instr == '111111101110xxx0xxxx1110x0x0xxx1' ||
108            instr == '11111110110xxxx0xxxx1110x0x0xxx1' ||
109            instr == '1111111010xxxx0xxxx1110x0x0xxx1') then // VRMLSLDAVH-RR.QQ-T1
110        Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
111        Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
112    elsif (instr == '111011101x1xxxxxxx01111100xxxx') then // VSHLC-Q.RI-T1
113        Elem[regs, UInt( instr[3:0] ), 1] = '1';
114    elsif (instr == '111111001xx0xxxxxx1110xxxxxxx0' ||
115            instr == '111111001xx0xxxxxx11110xxxxxxx0') then // VST2-Q.RI-T1
116        Elem[regs, UInt( instr[19:16] ), 1] = '1';
117    elsif (instr == '111111001xx0xxxxxx11110xxxxxx1' ||
118            instr == '111111001xx0xxxxxx11110xxxxxxx1') then // VST4-Q.RI-T1
119        Elem[regs, UInt( instr[19:16] ), 1] = '1';
120    elsif (instr == '111x1101x0x00xxxxxx01110xxxxxxx' ||
121            instr == '111x1101x0x00xxxxxx011110xxxxxxx' ||
122            instr == '111x110xx0100xxxxxx011110xxxxxxx' ||
123            instr == '111x110xx0100xxxxxx01110xxxxxxx') then // VSTR-Q.RI-T1
124        Elem[regs, UInt( instr[18:16] ), 1] = '1';
125    elsif (instr == '111x110xx010xxxxxx011110xxxxxxx' ||
126            instr == '111x1101x0x01xxxxxx01110xxxxxxx' ||
127            instr == '111x1101x0x01xxxxxx011110xxxxxxx' ||
128            instr == '111x110xx010xxxxxx01110xxxxxxx') then // VSTR-Q.RI-T2
129        Elem[regs, UInt( instr[18:16] ), 1] = '1';
130    elsif (instr == '11101101xxx0xxxxxx111100xxxxxxx' ||
131            instr == '1110110xxx10xxxxxx111100xxxxxxx') then // VSTR-Q.RI-T5
132        Elem[regs, UInt( instr[19:16] ), 1] = '1';
133    elsif (instr == '11101101xxx0xxxxxx111101xxxxxxx' ||
134            instr == '1110110xxx10xxxxxx111101xxxxxxx') then // VSTR-Q.RI-T6
135        Elem[regs, UInt( instr[19:16] ), 1] = '1';
136    elsif (instr == '11101101xxx0xxxxxx111110xxxxxxx' ||
137            instr == '1110110xxx10xxxxxx111110xxxxxxx') then // VSTR-Q.RI-T7
138        Elem[regs, UInt( instr[19:16] ), 1] = '1';
139    elsif (instr == '111x1110110xxxxxxx01110x0x0xxx0' ||
140            instr == '111x111010xxxxxxx01110x0x0xxx0' ||
141            instr == '111x11101110xxxxxxx01110x0x0xxx0') then // alias VMLALV-RR.QQ-T1
142        Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
143        Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
144    elsif (instr == '111x1110111xxxxxxx01110x0x0xxx0') then // alias VMLAV-R.QQ-T1
145        Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
146    elsif (instr == '111x1110110xxxx0xxx0111x0x0xxx0' ||
147            instr == '111x111010xxxx0xxx0111x0x0xxx0') then // alias VRMLALVH-RR.QQ-T1
148        Elem[regs, UInt( instr[22:20]:'1' ), 1] = '1';
149        Elem[regs, UInt( instr[15:13]:'0' ), 1] = '1';
150    return regs;

```

E2.1.192 Halt

```

1 // Halt()
2 // =====

```

```

3
4 Halt()
5     // Halt
6     Halted = TRUE;
7
8     // Clear lockup state.
9     LockedUp = FALSE;
10
11     // Upon entering debug state, S_REGRDY becomes valid hence must be set to '1'.
12     DHCSR.S_REGRDY = '1';
13
14     // Any pending return operation is cleared and can be re-pended on
15     // exit from Debug State.
16     _PendingReturnOperation = FALSE;
17
18     // Clear all remaining in flight instructions.
19     ClearInFlightInstructions();

```

E2.1.193 Halted

```

1 // Indicates the PE is in Debug State
2
3 boolean Halted;

```

E2.1.194 HaltingDebugAllowed

```

1 // HaltingDebugAllowed()
2 // =====
3
4 boolean HaltingDebugAllowed()
5     return ExternalInvasiveDebugEnabled() || Halted;

```

E2.1.195 HandleException

```

1 // HandleException()
2 // =====
3
4 HandleException(ExcInfo excInfo)
5     if excInfo.fault != NoFault then
6         if excInfo.lockup then
7             Lockup(excInfo.termInst);
8         else
9             // If the fault escalated to a HardFault update the syndrome info.
10            if HaveMainExt() && excInfo.fault == HardFault then
11                if excInfo.origFault == DebugMonitor then
12                    HFSR.DEBUGEVT = '1';
13                elseif excInfo.origFault != HardFault then
14                    HFSR.FORCED = '1';
15
16            // If the exception does not cause a lockup set the exception pending
17            // and potentially terminate execution of the current instruction.
18            SetPending(excInfo.fault, excInfo.isSecure, TRUE);
19            if excInfo.termInst then
20                EndOfInstruction();

```

E2.1.196 HandleExceptionTransitions

```

1 // HandleExceptionTransitions()
2 // =====
3
4 boolean HandleExceptionTransitions()
5     // Check for, and process any exception returns that were requested. This
6     // must be done after the instruction has completed so any exceptions
7     // raised during the exception return do not interfere with the execution of

```



```

8      // the instruction that cause the exception return (for example a POP
9      // causing an excReturn value to be written to the PC must adjust SP
10     // even if the exception return caused by the POP raises a fault).
11     excStackFramePresent      = FALSE;
12     tailChainedException      = FALSE;
13     EXC_RETURN_Type excReturn = NextInstrAddr();
14     if _PendingReturnOperation then
15         _PendingReturnOperation = FALSE;
16         (excInfo, excReturn, tailChainedException) = ExceptionReturn(excReturn);
17         // Handle any faults raised during exception return.
18         if excInfo.fault != NoFault then
19             // If the fault occurred during exception return then the previous
20             // exception stack frame will not have been consumed.
21             excStackFramePresent = TRUE;
22             // Either lockup, or pend the fault if it can be taken.
23             if excInfo.lockup then
24                 // Check if the fault occurred on exception return, or whether it
25                 // occurred during a tail-chained exception entry. This is
26                 // because Lockups on exception return have to be handled
27                 // differently.
28                 if excInfo.inExcTaken then
29                     // Fault occurred whilst tail-chaining into a new exception,
30                     // lockup at the priority of the exception that we attempted
31                     // to enter by transitioning it to being active.
32                     SetPending(excInfo.derivedFrom, excInfo.derivedFromIsSecure, FALSE);
33                     SetActive(excInfo.derivedFrom, excInfo.derivedFromIsSecure, TRUE);
34                 else
35                     // If the fault occurred during exception return then the
36                     // register state is UNKNOWN. This is due to the fact that
37                     // an unknown amount of the exception stack frame might have
38                     // been restored.
39                     for n = 0 to 12
40                         R[n] = bits(32) UNKNOWN;
41                     LR = bits(32) UNKNOWN;
42                     XPSR = bits(32) UNKNOWN;
43                     if HaveMveOrFPEExt() then
44                         InvalidateFRegs(FALSE, TRUE);
45                     // If lockup is entered as a result of an exception return
46                     // fault the original exception is deactivated. Therefore
47                     // the stack pointer must be updated to consume the
48                     // exception stack frame to keep the stack depth consistent
49                     // with the number of active exceptions. NOTE: The XPSR SP
50                     // alignment flag is UNKNOWN, assume it was zero.
51                     ConsumeExcStackFrame(excReturn, '0');
52                     excStackFramePresent = FALSE;
53                     // IPSR from stack is UNKNOWN, set IPSR based on mode
54                     // specified in EXC_RETURN.
55                     IPSR.Exception = (if excReturn.Mode == '1' then NoFault
56                                     else HardFault)[8:0];
57                     if HaveMveOrFPEExt() then
58                         CONTROL.FPCA = NOT(excReturn.FType);
59                         CONTROL_S.SFPA = bit UNKNOWN;
60                     Lockup(FALSE);
61                 else
62                     // Set syndrome if fault escalated to a HardFault.
63                     if HaveMainExt() &&
64                         (excInfo.fault == HardFault) &&
65                         (excInfo.origFault != HardFault) then
66                         HFSR.FORCED = '1';
67                         SetPending(excInfo.fault, excInfo.isSecure, TRUE);
68
69     // If there is a pending exception with sufficient priority take it now.
70     // This is done before committing PC and ITSTATE changes caused by the
71     // previous instruction so that the committed architecture state reflects
72     // the context the instruction was executed in.
73     (takeException, exception, excIsSecure) = PendingExceptionDetails();
74     if takeException then
75         // If a fault occurred during an exception return then the exception
76         // stack frame will already be on the stack, as a result entry to the

```



```

77     // next exception is treated as if it were a tail-chain.
78     pePriority = ExecutionPriority();
79     peException = UInt(IPSr.Exception);
80     peIsSecure = IsSecure();
81     if excStackFramePresent then
82         // If the fault occurred during ExceptionTaken() then LR will have
83         // been updated with the new exception return value. To keep excReturn
84         // consistent with the state of the exception stack frame we need to
85         // use the updated version in this case. If no updates have occurred
86         // then the excReturn value from the previous exception return is
87         // used.
88         if excInfo.inExcTaken then
89             excReturn = LR;
90             (excInfo, excReturn) = TailChain(exception, excIsSecure, excReturn);
91         else
92             (excInfo, excReturn) = ExceptionEntry(exception, excIsSecure);
93         // Handle any derived faults that have occurred.
94         if excInfo.fault != NoFault then
95             DerivedLateArrival(pePriority, peException, peIsSecure, excInfo,
96                               exception, excIsSecure, excReturn);
97
98     return takeException || tailChainedException;

```

E2.1.197 HandleLO

```

1  // HandleLO()
2  // =====
3
4  INSTR_EXEC_STATE_Type HandleLO(INSTR_EXEC_STATE_Type state)
5      // The default state for the link and BTI bits is FALSE.
6      state.L = '0';
7      state.BTI = '0';
8
9      // If valid branch info matches the fetch address update the LOB state and
10     // fetch address accordingly.
11     if state.LOBranInfoValid == '1' then
12         if LO_BRANCH_INFO.END_ADDR == state.FetchAddr[31:1] then
13             state.L = LO_BRANCH_INFO.BF AND LO_BRANCH_INFO.LF;
14             state.T16IND = LO_BRANCH_INFO.T16IND;
15             state.BTI = LO_BRANCH_INFO.BTI;
16             // Conditions for LOB handling in an IT block.
17             if InITBlock(state.ITState) then
18                 // The BF b_label is allowed to be the last instruction in an IT block.
19                 // As the BF branch occurs before this instruction is executed, the ITSTATE
20                 // needs to be updated as if the end of the IT block had been reached.
21                 if LO_BRANCH_INFO.BF == '1' then
22                     state.ITState = Zeros(8);
23                 else
24                     // If LO_BRANCH_INFO is valid and a low overhead branch is handled,
25                     // then the behavior is CONSTRAINED UNPREDICTABLE.
26                     CONSTRAINED_UNPREDICTABLE;
27             // Branch cache address matched, branch to offset specified.
28             if LO_BRANCH_INFO.BF == '1' ||
29               (LO_BRANCH_INFO.BF == '0' && LO_BRANCH_INFO.LF == '1') ||
30               !IsLastLowOverheadLoop(state) then
31                 state.FetchAddr = LO_BRANCH_INFO.JUMP_ADDR: '0';
32             // If the branch is due to a BF instruction invalidate the branch
33             // info so spurious branches do not occur.
34             if LO_BRANCH_INFO.BF == '1' then
35                 // Clear the branch cache.
36                 state.LOBranInfoValid = '0';
37             elseif LO_BRANCH_INFO.LF == '0' then
38                 // Looping mode: Decrement the loop counter unless this is the
39                 // last iteration, in which case looping mode is exited.
40                 if !IsLastLowOverheadLoop(state) then
41                     state.LoopCount = state.LoopCount - (1 << (4 - LTPSIZE))[31:0];
42                 else
43                     // LO_BRANCH_INFO.VALID does not need to be cleared at the end

```

```

44         // of the loop.
45         //
46         // Skip over LE at the end of the loop.
47         state.FetchAddr = state.FetchAddr + 4;
48         // Reset LTPSIZE if it is accessible, which will be the case
49         // for all predicated loops as the LETP instruction will
50         // have forced the state to be accessible and all operations
51         // that can cause the state to be inaccessible require a CSE
52         // which will invalidate LO_BRANCH_INFO.
53         if ActiveFPState() then
54             state.ResetLTPSize = TRUE;
55     return state;

```

E2.1.198 HasArchVersion

```

1 // HasArchVersion()
2 // =====
3
4 // Return TRUE if the implemented architecture includes the extensions defined in the
5 // specified architecture version.
6
7 boolean HasArchVersion(ArchVersion version)
8     return version == Armv8p0 || boolean IMPLEMENTATION_DEFINED "Architecture version";

```

E2.1.199 HaveAlgorithmPAC

```

1 // HaveAlgorithmPAC
2 // =====
3 // Checks which PAC algorithm is implemented by quereing ID_ISAR5.PACBTI.
4
5 AlgorithmPAC HaveAlgorithmPAC();

```

E2.1.200 HaveDebugMonitor

```

1 // HaveDebugMonitor()
2 //=====
3
4 boolean HaveDebugMonitor()
5     return HaveMainExt();

```

E2.1.201 HaveDSPExt

```

1 // HaveDSPExt()
2 // =====
3 // Check whether DSP Extension is implemented.
4
5 boolean HaveDSPExt();

```

E2.1.202 HaveDWT

```

1 // HaveDWT()
2 // =====
3 // Check whether Data Watchpoint and Trace unit is implemented.
4
5 boolean HaveDWT();

```

E2.1.203 HaveFPB

```

1 // HaveFPB()
2 // =====
3 // Check whether Flash Patch and Breakpoint unit is implemented.
4
5 boolean HaveFPB();

```

E2.1.204 HaveFPEExt

```

1 // HaveFPEExt ()
2 // =====
3 // Check whether Floating Point Extension is implemented.
4
5 boolean HaveFPEExt ();

```

E2.1.205 HaveHaltingDebug

```

1 // HaveHaltingDebug ()
2 // =====
3 // Check whether Halting debug implemented.
4
5 boolean HaveHaltingDebug ();

```

E2.1.206 HaveITM

```

1 // HaveITM ()
2 // =====
3 // Check whether Instrumentation Trace Macrocell is implemented.
4
5 boolean HaveITM ();

```

E2.1.207 HaveLOBExt

```

1 // HaveLOBExt ()
2 // =====
3 // Check whether the Low Overhead Loops and Branch Future Extension is implemented.
4
5 boolean HaveLOBExt ();

```

E2.1.208 HaveMainExt

```

1 // HaveMainExt ()
2 // =====
3 // Check whether the Main Extension is implemented.
4
5 boolean HaveMainExt ();

```

E2.1.209 HaveMve

```

1 // HaveMve ()
2 // =====
3 // Check whether the M-profile Vector Extension is implemented.
4
5 boolean HaveMve ();

```

E2.1.210 HaveMveOrFPEExt

```

1 // HaveMveOrFPEExt ()
2 // =====
3
4 boolean HaveMveOrFPEExt ()
5     return HaveFPEExt () || HaveMve ();

```

E2.1.211 HavePACBTIExt

```

1 // HavePACBTIExt ()
2 // =====
3 // Checks whether the PACBTI Extension is implemented.
4
5 boolean HavePACBTIExt ()
6     return (HaveMainExt () &&
7             HaveAlgorithmPAC () IN {AlgorithmPAC_QARMA3,
8                                     AlgorithmPAC_QARMA5,
9                                     AlgorithmPAC_IMPDEF});

```

E2.1.212 HaveSecurityExt

```

1 // HaveSecurityExt ()
2 // =====
3 // Check whether the implementation has the Security Extension.
4
5 boolean HaveSecurityExt ();

```

E2.1.213 HaveSysTick

```

1 // HaveSysTick ()
2 // =====
3 // Returns the number of SysTick instances (0, 1 or 2).
4
5 integer HaveSysTick ();

```

E2.1.214 HaveUDE

```

1 // HaveUDE ()
2 // =====
3 // Check whether the Unprivileged Debug Extension is implemented.
4
5 boolean HaveUDE ();

```

E2.1.215 HighestPri

```

1 // HighestPri ()
2 // =====
3 // Priority of Thread mode with no active exceptions.
4
5 integer HighestPri ()
6     // The value is PriorityMax + 1 = 256 (configurable priority maximum
7     // bit field is 8 bits).
8     return 256;

```

E2.1.216 HighestSetBit

```

1 // HighestSetBit ()
2 // =====
3
4 integer HighestSetBit (bits (N) x)
5     for i = N-1 downto 0
6         if x[i] == '1' then return i;
7     return -1;

```

E2.1.217 Hint_Debug

```

1 // Hint_Debug
2 // =====
3 // Generate a hint to the debug system.
4
5 Hint_Debug (bits (4) option);

```

E2.1.218 Hint_PreloadData

```

1 // Hint_PreloadData
2 // =====
3 // Performs a preload data hint.
4
5 Hint_PreloadData(bits(32) address);

```

E2.1.219 Hint_PreloadDataForWrite

```

1 // Hint_PreloadDataForWrite
2 // =====
3 // Performs a preload data hint for write.
4
5 Hint_PreloadDataForWrite(bits(32) address);

```

E2.1.220 Hint_PreloadInstr

```

1 // Hint_PreloadInstr
2 // =====
3 // Performs a preload instructions hint.
4
5 Hint_PreloadInstr(bits(32) address);

```

E2.1.221 Hint_Yield

```

1 // Hint_Yield
2 // =====
3 // Performs a Yield hint.
4
5 Hint_Yield();

```

E2.1.222 IDAUCheck

```

1 // IDAUCheck
2 // =====
3 // Query IDAU(Implementation Defined Attribution Unit) for attribution information.
4
5 (boolean, boolean, boolean, bits(8), boolean) IDAUCheck(bits(32) address);

```

E2.1.223 IgnoreFaultsType

```

1 // Indicates Ignore Faults Types
2 // =====
3
4 enumeration IgnoreFaultsType { IgnoreFaults_NONE,
5                               IgnoreFaults_STACK,
6                               IgnoreFaults_ALL };

```

E2.1.224 InITBlock

```

1 // InITBlock()
2 // =====
3
4 boolean InITBlock(ITSTATEType itState)
5     return (itState[3:0] != '0000');
6
7 boolean InITBlock()
8     return InITBlock(ITSTATE);

```

E2.1.225 InstrCanChain

```
1  // InstrCanChain()
2  // =====
3
4  boolean InstrCanChain(bits(32) instr)
5      // Check if the instruction is a chainable instruction, and if it
6      // is a chained memory operation.
7      isChainMem = IsMveLoadStoreInstruction(instr);
8      // Get a list of registers read and written by this instruction
9      // (these are bitstring where the index of each set bit indicates a used register).
10     isMve      = IsMveBeatWiseInstruction(instr);
11     isCde      = IsCdeBeatWiseInstruction(instr);
12     canChain   = (isMve || isCde) && !InITBlock();
13
14     // Calculate the scalar reg access masks for beatwise instructions. There are no CDE
15     // instructions that access scalar registers so the default values of 0 are valid.
16     instrScalarReads = 0[15:0];
17     instrScalarWrites = 0[15:0];
18     if isMve then
19         instrScalarReads = GetMveScalarReadRegs(instr);
20         instrScalarWrites = GetMveScalarWriteRegs(instr);
21
22     // If an instruction attempts to modify its own tail predication mask it is
23     // CONSTRAINED_UNPREDICTABLE whether an UNDEFINSTR UsageFault is raised, or whether
24     // the instruction executes with an UNKNOWN predication value.
25     if LO_BRANCH_INFO.VALID == '1' && LO_BRANCH_INFO.BF == '0' &&
26        instrScalarWrites[14] == '1' && LTFSIZE < 4 &&
27        ConstrainUnpredictable(Unpredictable_MVETailPredMod) == Constraint_UNDEF then
28        UFSR.UNDEFINSTR = '1';
29        HandleException(CreateException(UsageFault));
30
31     // Memory operations cannot chain with other memory operations.
32     if canChain && isChainMem then
33         for i = 0 to MAX_OVERLAPPING_INSTRS-1
34             if _InstInfo[i].Valid && IsMveLoadStoreInstruction(ThisInstr(i)) then
35                 canChain = FALSE;
36
37     // Scalar dependencies must be tracked.
38     if canChain then
39         // Get a list of all registers read or written by other in-flight instructions.
40         otherScalarReads = 0[15:0];
41         otherScalarWrites = 0[15:0];
42         for i = 0 to MAX_OVERLAPPING_INSTRS-1
43             if _InstInfo[i].Valid && _InstInfo[i].Type == InstrType_BEATWISE_MVE then
44                 // Only MVE beatwise instructions access scalar registers
45                 otherScalarReads = (otherScalarReads OR
46                                     GetMveScalarReadRegs(_InstInfo[i].Opcode));
47                 otherScalarWrites = (otherScalarWrites OR
48                                     GetMveScalarWriteRegs(_InstInfo[i].Opcode));
49         // Determine if there is any overlap between the registers read and written,
50         // if so chaining is impossible.
51         if ((instrScalarReads AND otherScalarWrites) != 0[15:0]) ||
52            ((instrScalarWrites AND otherScalarReads) != 0[15:0]) then
53             canChain = FALSE;
54
55     // LR chaining restrictions
56     if (canChain &&
57         _InstInfo[0].Valid &&
58         LO_BRANCH_INFO.VALID == '1' &&
59         LO_BRANCH_INFO.BF == '0') then
60         // Check if any instruction in the chain writes to LR, and get the index
61         // of the last instruction in the chain.
62         lastValidId = 0;
63         lrWrite = FALSE;
64         for i = 0 to MAX_OVERLAPPING_INSTRS-1
65             if _InstInfo[i].Valid then
66                 lastValidId = i;
67                 if _InstInfo[i].Type == InstrType_BEATWISE_MVE then
```

```

68         lrWrite = (lrWrite ||
69                     GetMveScalarWriteRegs(_InstInfo[i].Opcode)[14] == '1');
70
71         // Do not chain the next instruction if the end of the loop body has been
72         // reached and either one of the existing chained instructions writes to
73         // LR, or the new instruction reads or writes to LR.
74         instState = GetInstrExecState(lastValidId);
75         nextSeqAddr = instState.FetchAddr + _InstInfo[lastValidId].Length[31:0];
76         if LO_BRANCH_INFO.END_ADDR == nextSeqAddr[31:1] then
77             canChain = !(lrWrite
78                         ||
79                         instrScalarReads[14] == '1' ||
80                         instrScalarWrites[14] == '1');
81
82         // Two instructions reading/writing FPSCR carry bit should not chain with each other.
83         // Only MVE beatwise instructions access FPSCR.
84         if isMve && canChain && IsMveAccessFPSCR_C(instr) then
85             for i = 0 to MAX_OVERLAPPING_INSTRS-1
86                 if _InstInfo[i].Valid && _InstInfo[i].Type == InstrType_BEATWISE_MVE then
87                     if IsMveAccessFPSCR_C(_InstInfo[i].Opcode) then
88                         canChain = FALSE;
89
90         // Branch future chaining restrictions.
91         if (canChain
92             &&
93             _InstInfo[0].Valid
94             &&
95             LO_BRANCH_INFO.VALID == '1' &&
96             LO_BRANCH_INFO.BF == '1') then
97             // Get the index of the last instruction in the chain.
98             lastValidId = 0;
99             for i = 0 to MAX_OVERLAPPING_INSTRS-1
100                 if _InstInfo[i].Valid then
101                     lastValidId = i;
102             // Do not chain the next instruction if execution has reached a BF branch
103             // point.
104             instState = GetInstrExecState(lastValidId);
105             nextSeqAddr = instState.FetchAddr + _InstInfo[lastValidId].Length[31:0];
106             canChain = LO_BRANCH_INFO.END_ADDR != nextSeqAddr[31:1];
107
108         // Implementations can choose not to chain an instruction.
109         if canChain then
110             canChain = boolean IMPLEMENTATION_DEFINED "Chain instruction";
111
112         return canChain;

```

E2.1.226 InstrExecState

```

1 // Indicates instruction execution state
2 // =====
3
4 type INSTR_EXEC_STATE_Type is (
5     bits(32)    FetchAddr,
6     ITSTATEType ITState,
7     bit         L,
8     bit         T16IND,
9     bit         BTI,
10    bits(32)    LoopCount,
11    bit         LOBranchInfoValid,
12    boolean     ResetLTPSize
13 )
14 INSTR_EXEC_STATE_Type _CurrentInstrExecState;

```

E2.1.227 InstrType

```

1 // Beatwise or not instruction type
2
3 enumeration InstrType { InstrType_NON_BEATWISE, // Non Beatwise.
4                         InstrType_BEATWISE_MVE, // Beatwise MVE instruction type.
5                         InstrType_BEATWISE_CDE  // Beatwise CDE instruction type.

```

```
6      };
```

E2.1.228 InstructionAdvance

```
1  // InstructionAdvance()
2  // =====
3
4  InstructionAdvance()
5      // Instruction getting old (or scalar instruction).
6      // Commit next state back to the registers.
7      INSTR_EXEC_STATE_Type next = GetInstrExecState(1);
8      if HaveLOBExt() then
9          if next.LOBranchInfoValid == '1' then
10             LR = next.LoopCount;
11          else
12             LO_BRANCH_INFO.VALID = '0';
13          if next.L == '1' then
14             // Set LR to return to the return address. The offset of the
15             // return address depends on whether the originating BF
16             // instruction assumed there would be a T32 or a 16-bit T32
17             // instruction after the branch point. See BF documentation for
18             // details.
19             retAddr = ThisInstrAddr() + ThisInstrLength();
20             retAddr = retAddr + (if next.T16IND == '1' then 2 else 4);
21             LR = retAddr[31:1] : '1';
22          if BTIEnabled() && next.BTI == '1' then
23             // When BTI is enabled and LO_BRANCH_INFO.BTI was set by BFX (!= LR) or BFLX
24             // then the EPSR.B bit is set to one, requiring a BTI clearing instruction at
25             // the target address of the branch.
26             EPSR.B = '1';
27          if HaveMve() && next.ResetLTPSize then
28             FPSCR.LTPSIZE = 4[2:0];
29          _RName[RNamesPC] = next.FetchAddr;
30          if HaveMainExt() then
31             EPSR.IT = next.ITState;
32
33          // Mark an instruction as having retired.
34          DHCSR.S_RETIRE_ST = '1';
35
36          // Advance the instruction FIFO.
37          for i = 0 to MAX_OVERLAPPING_INSTRS-1
38             if i == MAX_OVERLAPPING_INSTRS-1 then
39                 _InstInfo[i].Valid = FALSE;
40             else
41                 _InstInfo[i] = _InstInfo[i+1];
```

E2.1.229 InstructionExecute

```
1  // InstructionExecute()
2  // =====
3
4  // If fetchNew is set then fetch and execute new instructions, otherwise only
5  // continue execution of inflight beats.
6  boolean InstructionExecute(boolean fetchNew)
7      try
8          // Attempt to execute the next instruction. Start by setting up the state.
9          _InstID = 0;
10         _BeatID = 0;
11         execFault = FALSE;
12         activeChains = GetActiveChains();
13         _CurrentInstrExecState = GetInstrExecState(activeChains);
14         // Fetch the instruction
15         pc = ThisInstrAddr();
16         if fetchNew then
17             (instr, is16bit) = FetchInstr(pc);
18             len = if is16bit then 2 else 4;
19         // Capture security and privilege
```



```

20     isSecure = IsSecure();
21     isPriv   = FindPriv();
22
23     // Assume we have an execution fault until we know otherwise, so that we
24     // know whether to override any pending fetch fault we may have.
25     execFault = TRUE;
26
27     // If a chain is being executed then the current instruction can be
28     // added to the chain if it is chainable. If the instruction is not
29     // chainable (for example because its a scalar instruction) then
30     // it is not executed, and the next beat(s) of the flight chained
31     // instructions are executed. This process is repeated on the next
32     // architecture tick, and when the chain has completed the non-chainable
33     // instruction can be executed.
34     // NOTE: Chainable instructions are not allowed to chain if inside
35     //       an IT block.
36     if fetchNew then
37         chainableInst = (IsMveBeatWiseInstruction(instr) ||
38                         IsCdeBeatWiseInstruction(instr)) && !InITBlock();
39     else
40         chainableInst = FALSE;
41     if HaveMve() && (chainableInst || activeChains > 0) then
42         // A new instruction can only be chained if a power of 2 number
43         // of beats have completed. Also allow an instruction to be
44         // started if the ECI information indicates that an instruction
45         // is in progress, but the corresponding slot in the instruction
46         // queue is empty, which can occur on exception return.
47         chainableExecPoint = (EPSR.ECI == 0[7:0]) || (EPSR.ECI == 2[7:0]);
48         if !chainableExecPoint then
49             beatStatus = BeatComplete;
50             for instId = 0 to MAX_OVERLAPPING_INSTRS-1
51                 if ((Elem[beatStatus, instId, MAX_BEATS] != Zeros(MAX_BEATS)) &&
52                     !_InstInfo[instId].Valid) then
53                     chainableExecPoint = TRUE;
54             if chainableExecPoint && fetchNew then
55                 if InstrCanChain(instr) then
56                     SetThisInstrDetails(instr, len);
57             // In beatwise execution, updating _CommitState is done inside ExecBeats.
58             ExecBeats();
59
60     elseif fetchNew then
61         // Scalar instruction, execute instructions normally.
62         SetThisInstrDetails(instr, len);
63         InstStateCheck(instr);
64         DecodeExecute(instr, pc, is16bit, DefaultCond());
65         // Scalar instructions, and MVE instructions in IT blocks do not
66         // have beat behavior so commit straight away
67         _CommitState = TRUE;
68
69     // Check for DWT match.
70     if fetchNew && IsDWTEnabled(isSecure, isPriv) then
71         DWT_InstructionMatch(pc, isSecure, isPriv, len);
72
73     // If we get to here we have no execution fault.
74     execFault = FALSE;
75
76 catch exn
77     // Do not catch UNPREDICTABLE or internal errors.
78     when IsSEE(exn) || IsUNDEFINED(exn)
79         // Unallocated instructions in the NOP hint space and instructions
80         // that fail their condition tests are treated like NOPs.
81         nopHint16 = instr == '000000000000000101111111xxxx0000';
82         nopHint32 = (instr == '11110011010111110000000xxxxxxx') && HaveMainExt();
83         if ConditionHolds(CurrentCond()) && !(nopHint16 || nopHint32) then
84             _CommitState = FALSE;
85             toSecure     = IsSecure();
86             // Unallocated instructions in the coprocessor space behave as NOCP
87             // if the coprocessor is disabled.
88             (isCp, cpNum) = IsCPIInstruction(instr);

```

```

89         if isCp then
90             (cpEnabled, cpFaultState) = IsCPEnabled(cpNum);
91             // In v8.0 the PE is permitted to not decode the CP space if it is
92             // disabled or not implemented. This behaviour is required from v8.1 onwards.
93             // From v8.1, some instructions are not considered to be part of the
94             // CP encoding space for the purposes of this check.
95             skipable = ( (HasArchVersion(Armv8p1) ||
96                 boolean IMPLEMENTATION_DEFINED "Skip CP space decode") &&
97                 !(HasArchVersion(Armv8p1) &&
98                     IsCpInstructionSecureOnly(instr)) );
99             if isCp && !cpEnabled && skipable then
100                 toSecure = cpFaultState;
101                 if toSecure then
102                     UFSR_S.NOCP = '1';
103                 else
104                     UFSR_NS.NOCP = '1';
105             else
106                 UFSR.UNDEFINSTR = '1';
107
108             // If Main Extension is not implemented the fault will escalate
109             // to a HardFault.
110             excInfo = CreateException(UsageFault, TRUE, toSecure);
111             // Prevent EndOfInstruction() being called in
112             // HandleException() as the instruction has already been
113             // terminated so there is no need to throw the exception
114             // again.
115             excInfo.termInst = FALSE;
116             HandleException(excInfo);
117         else
118             // If the instruction condition does not pass then this
119             // behaves as a NOP, as such PC must be advanced. Since
120             // vector instructions are not chained (so only one
121             // instruction is in flight) when inside an IT
122             // block, they are also committed here.
123             _CommitState = TRUE;
124         when IsExceptionTaken(exn)
125             // If an exception is thrown then it was before _CommitState was
126             // set to true, so no additional actions are required in this
127             // catch block.
128
129         return execFault;

```

E2.1.230 InstructionsInFlight

```

1 // InstructionsInFlight()
2 // =====
3
4 boolean InstructionsInFlight()
5     // If there is more than one active chain and it is not just a single active
6     // scalar instruction then there are instructions in flight.
7     return GetActiveChains() != 0 && (GetActiveChains() != 1 ||
8         _InstInfo[0].Type == InstrType_BEATWISE_MVE ||
9         _InstInfo[0].Type == InstrType_BEATWISE_CDE);

```

E2.1.231 InstructionSynchronizationBarrier

```

1 // InstructionSynchronizationBarrier()
2 // =====
3 // Perform an instruction synchronization barrier operation.
4
5 InstructionSynchronizationBarrier(bits(4) option);

```

E2.1.232 InstStateCheck

```

1 // InstStateCheck()

```

```

2 // =====
3
4 InstStateCheck(bits(32) instr)
5 // Check for IT,ICI,ECI bits that are not permitted for the current
6 // instruction. NOTE EPSR.ICI and EPSR.ECI overlap with EPSR.IT.
7 isLE = IsLEInstruction(instr);
8 validICI = (EPSR.ICI[7:6] == Zeros(2) && EPSR.ICI[1:0] == Zeros(2));
9 validECI = UInt(EPSR.ECI) < 6 && EPSR.ECI[3:0] != '0011';
10 valid = ( InITBlock()                                     ||
11           EPSR.IT == Zeros(8)                             ||
12           IsBKPTInstruction(instr)                         ||
13           ( validICI &&                                     IsLoadStoreClearMultInstruction(instr)) ||
14           ( validECI && HaveMve() && (IsMveBeatWiseInstruction(instr)
15                                     IsCdeBeatWiseInstruction(instr)
16                                     isLE) )
17           );
18 if !valid then
19     UFSR.INVSTATE = '1';
20     excInfo = CreateException(UsageFault);
21     HandleException(excInfo);

```

E2.1.233 Int

```

1 // Int()
2 // =====
3
4 integer Int(bits(N) x, boolean unsigned)
5     result = if unsigned then UInt(x) else SInt(x);
6     return result;

```

E2.1.234 IntegerZeroDivideTrappingEnabled

```

1 // IntegerZeroDivideTrappingEnabled()
2 // =====
3
4 boolean IntegerZeroDivideTrappingEnabled()
5     // DIV_0_TRP bit in CCR is RAZ/WI if Main Extension is not implemented
6     return CCR.DIV_0_TRP == '1';

```

E2.1.235 InvalidateFPRegs

```

1 // InvalidateFPRegs()
2 // =====
3
4 InvalidateFPRegs(boolean shouldClear, boolean doCallee)
5     clearValue = if shouldClear then Zeros(32) else bits(32) UNKNOWN;
6
7     for i = 0 to 15
8         S[i] = clearValue;
9         if doCallee then S[i+16] = clearValue;
10    FPSCR = clearValue;
11    VPR = clearValue;

```

E2.1.236 InVPTBlock

```

1 // InVPTBlock()
2 // =====
3
4 boolean InVPTBlock()
5     if _BeatID IN {0, 1} then
6         return VPR.MASK01 != '0000';
7     else
8         return VPR.MASK23 != '0000';

```

E2.1.237 IsAccessible

```
1 // IsAccessible()
2 // =====
3
4 (bit, bit, bits(8), boolean) IsAccessible(bits(32) address, boolean forceunpriv,
5                                         boolean isSecure)
6     bit write;
7     bit read;
8
9     // Work out which privilege level the current mode in the Non-secure state
10    // is subject to.
11    if forceunpriv then
12        isPrivileged = FALSE;
13    else
14        isPrivileged = CurrentModeIsPrivileged(isSecure);
15    (-, perms) = MPUCheck(address, AccType_NORMAL, isPrivileged, isSecure);
16    if !perms.apValid then
17        write = '0';
18        read = '0';
19    else
20        case perms.ap of
21            when '00' (write, read) = if isPrivileged then ('1','1') else ('0','0');
22            when '01' (write, read) = ('1','1') ;
23            when '10' (write, read) = if isPrivileged then ('0','1') else ('0','0');
24            when '11' (write, read) = ('0','1');
25    return (write, read, perms.region, perms.regionValid);
```

E2.1.238 IsActiveForState

```
1 // IsActiveForState()
2 // =====
3
4 boolean IsActiveForState(integer exception, boolean isSecure)
5     if !HaveSecurityExt() then
6         isSecure = FALSE;
7     // If the exception is configurable then check which domain it
8     // currently targets. If it is not configurable then the active flags can be
9     // used directly.
10    if IsExceptionTargetConfigurable(exception) then
11        // The active bit for exceptions that are not banked between the Security
12        // states are modelled as two bits to be compatible with the other exceptions
13        // that have a bit for each Security state.
14        assert ExceptionActive[exception][0] == ExceptionActive[exception][1];
15        active = ((ExceptionActive[exception][0] == '1') &&
16                (ExceptionTargetsSecure(exception, isSecure) == isSecure));
17    else
18        idx = if isSecure then 0 else 1;
19        active = ExceptionActive[exception][idx] == '1';
20    return active;
```

E2.1.239 IsAligned

```
1 // IsAligned()
2 // =====
3
4 boolean IsAligned(bits(32) address, integer size)
5     assert size IN {1,2,4,8};
6     mask = (size-1)[31:0]; // Integer to bit string conversion.
7     return IsZero(address AND mask);
```

E2.1.240 IsBKPTInstruction

```
1 // IsBKPTInstruction()
2 // =====
```

```

3 // Checks whether the instruction is a breakpoint.
4
5 boolean IsBKPTInstruction(bits(32) instr)
6     return instr == '0000 0000 0000 0000 1011 1110 xxxxxxxx';

```

E2.1.241 IsCdeBeatWiseInstruction

```

1 // IsCdeBeatWiseInstruction()
2 // =====
3 // Checks whether the instruction is a CDE Beatwise instruction.
4
5 boolean IsCdeBeatWiseInstruction(bits(32) instr)
6     // Quick check if the instruction is NOT a beatwise CDE instruction.
7     if (instr AND 0xee000840[31:0]) != 0xec000040[31:0] then return FALSE;
8     // Merge VCX1_vector and VCX2_vector
9     if instr == '111x110x0x1xxxxxxxxxx0xxx1xxxxxx' then return TRUE;
10    // VCX3_vector
11    if instr == '111x110x1xxxxxxxxxx0xxx1xxxxxx' then return TRUE;
12    return FALSE;

```

E2.1.242 IsCPEnabled

```

1 // IsCPEnabled()
2 // =====
3
4 (boolean, boolean) IsCPEnabled(integer cp, boolean privileged, boolean secure)
5     // Check Coprocessor Access Control Register for permission to use coprocessor.
6     boolean enabled;
7     boolean forceToSecure = FALSE;
8
9     cpacr = if secure then CPACR_S else CPACR_NS;
10    case cpacr[(cp*2)+1:cp*2] of
11        when '00'
12            enabled = FALSE;
13        when '01'
14            enabled = privileged;
15        when '10'
16            UNPREDICTABLE;
17        when '11' // Access permitted by CPACR.
18            enabled = TRUE;
19
20    if enabled && HaveSecurityExt() then
21        // Check if access is forbidden by NSACR.
22        if !secure && NSACR[cp] == '0' then
23            enabled = FALSE;
24            forceToSecure = TRUE;
25
26    // Check if the coprocessor state unknown flag.
27    if enabled && CPPWR_S[cp*2] == '1' then
28        enabled = FALSE;
29        // Check SUS bit to determine the target state of any fault.
30        forceToSecure = CPPWR_S[(cp*2)+1] == '1';
31
32    return (enabled, secure || forceToSecure);
33
34 (boolean, boolean) IsCPEnabled(integer cp)
35     return IsCPEnabled(cp, CurrentModeIsPrivileged(), IsSecure());

```

E2.1.243 IsCPInstruction

```

1 // IsCPInstruction()
2 // =====
3
4 (boolean, integer) IsCPInstruction(bits(32) instr)
5     isCp = instr IN { '111x1110xxxxxxxxxxxxxxxxxxxxxxxx',

```

```

6      '111x110xxxxxxxxxxxxxxxxxxxxxxxxx' };
7      cpNum = if isCp then UInt(instr[11:8]) else integer UNKNOWN;
8      // CP 11 controlled by CP10 enables.
9      // As of v8.1 CP8, 9, 14 and 15 are also controlled by CP10 enables.
10     if ( cpNum IN {11} ||
11         (cpNum IN {8, 9, 14, 15} && HasArchVersion(Armv8p1)) ) then
12         cpNum = 10;
13     // From v8.1 the encoding space outside the CDP space used by MVE instructions
14     // is also classed as coprocessor space and is associated with CP10.
15     if instr IN { '111x111xxxxxxxxxxxxxxxxxxxxx' } && HasArchVersion(Armv8p1) then
16         isCp = TRUE;
17         cpNum = 10;
18     return (isCp, cpNum);

```

E2.1.244 IsCpInstructionSecureOnly

```

1  // IsCpInstructionSecureOnly()
2  // =====
3
4  boolean IsCpInstructionSecureOnly(bits(32) instr)
5      // For VLLDM, VLSTM, and VSCCLRM instructions
6      // UNDEFINSTR UsageFault takes precedence over NOCP.
7      isCp = instr IN { '111011000011xxxx00001010x0000000', // VLLDM
8                       '111011000010xxxx00001010x0000000', // VLSTM
9                       '111011001x01111xxxx1011xxxxxxx0', // VSCCLRM T1
10                      '111011001x01111xxxx1010xxxxxxx', // VSCCLRM T2
11                      '1110110xx1x1xxxx11x01111xxxxxxx', // VLDR accessing FPCXT_*
12                      '1110110xx1x0xxxx11x01111xxxxxxx', // VSTR accessing FPCXT_*
13                      '1110111011111xxxxx101000010000', // VMRS accessing FPCXT_*
14                      '11101110111011xxxxx101000010000' }; // VMSR accessing FPCXT_*
15
16     return isCp;

```

E2.1.245 IsDebugState

```

1  // IsDebugState
2  // =====
3
4  boolean IsDebugState()
5      return Halted;

```

E2.1.246 IsDWTConfigUnpredictable

```

1  // IsDWTConfigUnpredictable()
2  // =====
3  // Checks for some of the UNPREDICTABLE cases for various combination of MATCH and
4  // ACTION for each comparator.
5
6  boolean IsDWTConfigUnpredictable(integer N)
7
8      no_trace = (!HaveMainExt() || DWT_CTRL.NOTRCPKT == '1' || !HaveITM());
9
10     // First pass check of MATCH field - coarse checks
11     case DWT_FUNCTION[N].MATCH of
12         when '0000' // Disabled.
13             return FALSE;
14         when '0001' // Cycle counter match.
15             if (!HaveMainExt() ||
16                 DWT_CTRL.NOCYCCONT == '1' ||
17                 DWT_FUNCTION[N].ID[0] == '0') then
18                 return TRUE;
19         when '001x' // Instruction address.
20             if (DWT_FUNCTION[N].ID[1] == '0' ||
21                 DWT_FUNCTION[N].DATAVSIZE != '01' ||
22                 DWT_COMP[N][0] == '1') then
23                 return TRUE;

```

```

24     when '01xx' // Data address.
25         mask = (1 << UInt(DWT_FUNCTION[N].DATAVSIZE)) - 1;
26         if (DWT_FUNCTION[N].ID[3] == '0' ||
27             ((DWT_COMP[N][3:0] AND mask[3:0]) != '0000')) then
28             return TRUE;
29     when '1100', '1101', '1110' // Data address with value.
30         if no_trace then return TRUE;
31         mask = (1 << UInt(DWT_FUNCTION[N].DATAVSIZE)) - 1;
32         if (DWT_FUNCTION[N].ID[3] == '0' ||
33             ((DWT_COMP[N][3:0] AND mask[3:0]) != '0000')) then
34             return TRUE;
35     when '10xx' // Data value.
36         Vsize = 2^UInt(DWT_FUNCTION[N].DATAVSIZE);
37         if (!HaveMainExt() || DWT_FUNCTION[N].ID[2] == '0' ||
38             (Vsize != 4 && DWT_COMP[N][31:16] != DWT_COMP[N][15:0]) ||
39             (Vsize == 1 && DWT_COMP[N][15:8] != DWT_COMP[N][7:0])) then
40             return TRUE;
41         if (HasArchVersion(Armv8p1) &&
42             (!IsZero(DWT_VMASK[N] AND DWT_COMP[N]) ||
43              (Vsize != 4 && DWT_VMASK[N][31:16] != DWT_VMASK[N][15:0]) ||
44              (Vsize == 1 && DWT_VMASK[N][15:8] != DWT_VMASK[N][7:0]))) then
45             return TRUE;
46     otherwise
47         return TRUE;
48
49 // Second pass MATCH check - linked and limit comparators.
50 case DWT_FUNCTION[N].MATCH of
51     when '0011' // Instruction address limit.
52         if N == 0 then return TRUE;
53         elseif (DWT_FUNCTION[N].ID[4] == '0' ||
54                 DWT_FUNCTION[N-1].MATCH IN {'0001', '0011', '01xx', '1xxx'}) ||
55                 UInt(DWT_COMP[N]) <= UInt(DWT_COMP[N-1])) then
56             return TRUE;
57         if DWT_FUNCTION[N-1].MATCH == '0000' then return FALSE;
58     when '0111' // Data address limit.
59         if N == 0 then return TRUE;
60         elseif (DWT_FUNCTION[N].ID[4] == '0' ||
61                 DWT_FUNCTION[N-1].MATCH IN {'0001', '001x', '0111', '10xx'}) ||
62                 (!HaveMainExt() && DWT_FUNCTION[N-1].MATCH == '11xx') ||
63                 DWT_FUNCTION[N].DATAVSIZE != '00' ||
64                 DWT_FUNCTION[N-1].DATAVSIZE != '00' ||
65                 UInt(DWT_COMP[N]) <= UInt(DWT_COMP[N-1])) then
66             return TRUE;
67         if DWT_FUNCTION[N-1].MATCH == '0000' then return FALSE;
68     when '1011' // Linked data value.
69         if N == 0 then return TRUE;
70         elseif (DWT_FUNCTION[N].ID[4] == '0' ||
71                 DWT_FUNCTION[N-1].MATCH IN {'0001', '001x', '0111', '10xx'}) ||
72                 DWT_FUNCTION[N].DATAVSIZE != DWT_FUNCTION[N-1].DATAVSIZE) then
73             return TRUE;
74         if DWT_FUNCTION[N-1].MATCH == '0000' then return FALSE;
75     otherwise
76         // No limitations in second pass.
77
78 // Check if DATAVSIZE is permitted.
79 if DWT_FUNCTION[N].DATAVSIZE == '11' then return TRUE;
80
81 // Check the ACTION is allowed for the MATCH type.
82 case DWT_FUNCTION[N].ACTION of
83     when '00' // CMPMATCH trigger only.
84         if DWT_FUNCTION[N].MATCH IN {'1100', '1101', '1110'} then
85             return TRUE;
86         if N > 0 && !HaveMainExt() then
87             if (DWT_FUNCTION[N].MATCH == '0011' &&
88                 DWT_FUNCTION[N-1].MATCH == '0010' &&
89                 DWT_FUNCTION[N-1].ACTION == '10') then
90                 return TRUE;
91             if (DWT_FUNCTION[N].MATCH == '0111' &&
92                 DWT_FUNCTION[N-1].MATCH == '01xx' &&

```

```

93         DWT_FUNCTION[N-1].ACTION == '1x') then
94             return TRUE;
95     when '01' // Debug event.
96         if DWT_FUNCTION[N].MATCH IN {'0011', '0111', '1100', '1101', '1110'} then
97             return TRUE;
98         if N > 0 then
99             if (DWT_FUNCTION[N].MATCH == '1011' &&
100                DWT_FUNCTION[N-1].MATCH == '01xx' &&
101                DWT_FUNCTION[N-1].ACTION == '01') then
102                 return TRUE;
103     when '10' // Data Trace Match or Data Value packet.
104         if no_trace || DWT_FUNCTION[N].MATCH IN {'0011', '0111'} then
105             return TRUE;
106         if N > 0 then
107             if (DWT_FUNCTION[N].MATCH == '1011' &&
108                DWT_FUNCTION[N-1].MATCH == '01xx' &&
109                DWT_FUNCTION[N-1].ACTION == '10') then
110                 return TRUE;
111     when '11' // Other Data Trace packet.
112         if (no_trace ||
113            DWT_FUNCTION[N].MATCH IN {'0010', '1000', '1001', '1010', '1011'}) then
114             return TRUE;
115         if N > 0 then
116             if (DWT_FUNCTION[N].MATCH == '0011' &&
117                DWT_FUNCTION[N-1].MATCH == '0010' &&
118                (DWT_FUNCTION[N-1].ACTION != '00' || !HaveMainExt())) then
119                 return TRUE;
120             if (DWT_FUNCTION[N].MATCH == '0111' &&
121                DWT_FUNCTION[N-1].MATCH == '01xx' &&
122                (DWT_FUNCTION[N-1].ACTION IN {'01', '10'} ||
123                 (DWT_FUNCTION[N-1].ACTION != '00' && !HaveMainExt()))) then
124                 return TRUE;
125
126     return FALSE; // Passes checks.

```

E2.1.247 IsDWTEEnabled

```

1 // IsDWTEEnabled()
2 // =====
3 // Check whether DWT is enabled.
4
5 boolean IsDWTEEnabled(boolean isSecure, boolean isPriv)
6     return ( HaveDWT() && DEMCR.TRCENA == '1' &&
7              ((!isSecure && NoninvasiveDebugAllowed(isPriv)) ||
8               SecureNoninvasiveDebugAllowed(isPriv)) );

```

E2.1.248 IsExceptionTargetConfigurable

```

1 // IsExceptionTargetConfigurable()
2 // =====
3
4 boolean IsExceptionTargetConfigurable(integer e)
5     if HaveSecurityExt() then
6         case e of
7             when NMI
8                 configurable = TRUE;
9             when BusFault
10                configurable = TRUE;
11             when DebugMonitor
12                configurable = TRUE;
13             when SysTick
14                // If there is only 1 SysTick instance then the target domain is
15                // configurable.
16                configurable = HaveSysTick() == 1;
17             otherwise
18                // Exceptions numbers lower than 16 that are not listed in this
19                // function are not configurable in this context.

```



```

20         configurable = e >= 16;
21     else
22         configurable = FALSE;
23     return configurable;

```

E2.1.249 IsExclusiveGlobal

```

1  // IsExclusiveGlobal
2  // =====
3  // Checks if PE has marked in a global record an address range as "exclusive access
4  // requested" that covers at least the size bytes from address.
5
6  boolean IsExclusiveGlobal(bits(32) address, integer processorid, integer size);

```

E2.1.250 IsExclusiveLocal

```

1  // IsExclusiveLocal
2  // =====
3  // Checks if PE has marked in a local record an address range as "exclusive access
4  // requested" that covers at least the size bytes from address.
5
6  boolean IsExclusiveLocal(bits(32) address, integer processorid, integer size);

```

E2.1.251 IsFirstBeat

```

1  // IsFirstBeat()
2  // =====
3
4  boolean IsFirstBeat()
5      return _BeatID == 0;

```

E2.1.252 IsIrqValid

```

1  // IsIrqValid()
2  // =====
3  // Check whether given exception number denotes a valid external interrupt
4  // implemented by PE.
5
6  boolean IsIrqValid(integer e);

```

E2.1.253 IsLastBeat

```

1  // IsLastBeat()
2  // =====
3
4  boolean IsLastBeat()
5      return _BeatID >= (MAX_BEATS - 1);

```

E2.1.254 IsLastLowOverheadLoop

```

1  // IsLastLowOverheadLoop()
2  // =====
3
4  boolean IsLastLowOverheadLoop()
5      return IsLastLowOverheadLoop(_CurrentInstrExecState);
6
7  boolean IsLastLowOverheadLoop(INSTR_EXEC_STATE_Type state)
8      // This does not check whether a loop is currently active.
9      // If the PE were in a loop, would this be the last one?
10     return UInt(state.LoopCount) <= (1 << (4 - LTPSIZE));

```

E2.1.255 IsLEInstruction

```
1 // IsLEInstruction()
2 // =====
3 // Checks whether the instruction is a loop end instruction.
4
5 boolean IsLEInstruction(bits(32) instr)
6     return instr IN {'1111000000xx11111100xxxxxxxxxx1'};
```

E2.1.256 IsLoadStoreClearMultInstruction

```
1 // IsLoadStoreClearMultInstruction()
2 // =====
3 // Checks whether the instruction is a clear multiple or a load/store multiple.
4
5 boolean IsLoadStoreClearMultInstruction(bits(32) instr)
6     isLSCM = instr IN {'0000000000000001100xxxxxxxxxx', // LDM_T1,STM_T1.
7                       '0000000000000001011x10xxxxxxxxxx', // LDM_T3,STM_T2, and aliases.
8                       '1110100xx0xxxxxxxxxxxxxxxxxxxxxx', // Load/store/clear mul Scalar.
9                       '1110110xxxxxxxxxxxxx101xxxxxxxxxx'}; // Load/store/clear mul
10                                                                // Floating-point.
11
12     // False positives due to the masks used in isLSCM.
13     notLSCM = instr IN {'1110100000xxxxxxxxxxxxxxxxxxxxxx', // UNALLOCATED.
14                        '1110100110xxxxxxxxxxxxxxxxxxxxxx', // UNALLOCATED.
15                        '11101100010xxxxxxxxxx101x00x1xxxx', // UNALLOCATED.
16                        '11101100010xxxxxxxxxx101xxx0xxxx', // UNALLOCATED.
17                        '11101100000xxxxxxxxxx101xxxxxxxxxx', // UNALLOCATED.
18                        '111011000011xxxx00001010x0000000', // VLDM.
19                        '111011000010xxxx00001010x0000000', // VLSTM.
20                        '11101101xx01xxxxxxxxxx101xxxxxxxxxx', // VLDR.
21                        '11101101xx00xxxxxxxxxx101xxxxxxxxxx', // VSTR.
22                        '111011011x1xxxxxxxxxx101xxxxxxxxxx'}; // VMOV.
23
24     return (isLSCM && !notLSCM);
```

E2.1.257 IsMveAccessFPSCR_C

```
1 boolean IsMveAccessFPSCR_C(bits(32) instr)
2     assert(IsMveBeatWiseInstruction(instr));
3     if (instr == '111011100x11xxx0xxxx1111x0x0xxx0') then // VADC-Q.QQ-T1
4         return TRUE;
5     if (instr == '111111100x11xxx0xxxx1111x0x0xxx0') then // VSBC-Q.QQ-T1
6         return TRUE;
7     return FALSE;
```

E2.1.258 IsMveBeatWiseInstruction

```
1 boolean IsMveBeatWiseInstruction(bits(32) instr)
2     if (instr AND 0xe0000000[31:0]) != 0xe0000000[31:0] then return FALSE;
3
4     if instr == '1111000000xx111011101xxxxxxxxxx1' then return TRUE;
5     if instr == '111x1110110xxx01xxx111100x0xxx0' then return TRUE;
6     if instr == '111x11101010xxx0xxxx1111x0x0xxx1' then return TRUE;
7     if instr == '111111100x0xxxx1xxx1111xxx0xxxx' then return TRUE;
8     if instr == '111x11100x11xxx1xxx1111x1x01x1x' then return TRUE;
9     if instr == '111x1110xxxxxx0xxxx1111x0x0xxx0' then return TRUE;
10    if instr == '111x11101x11000xxxx111101x0xxx0' then return TRUE;
11    if instr == '111x11110xxxxxx0xxx000x01xxxxx0' then return TRUE;
12    if instr == '11111111x11x00xxx00x00x1x0xxx0' then return TRUE;
13    if instr == '111x1110100xxxx0xxxx1111x0x0xxx1' then return TRUE;
14    if instr == '111x11100x11xxx1xxx1111x1x010xx' then return TRUE;
15    if instr == '111x11100x11xxx1xxx1111x0x0xxxx' then return TRUE;
16    if instr == '111x11100x11xxx1xxx1111x1x00xxx' then return TRUE;
17    if instr == '111111110xxxxxx0xxx01011x1x0xxx0' then return TRUE;
18    if instr == '111111110xxxxxx0xxx01000x1x0xxx0' then return TRUE;
19    if instr == '111111001xxxxxxxxxx11110xxxxxx' then return TRUE;
```

```

20     if instr == '111x11101x0xxxxxxx111x1x0xxx' then return TRUE;
21     if instr == '111x11111x01xxxxxxx011x01x1xxx0' then return TRUE;
22     if instr == '111111111x11xx11xxx000xxx1x0xxx0' then return TRUE;
23     if instr == '111x11100x11xxx1xxx01110x1x0xxx' then return TRUE;
24     if instr == '111111110x1xxx0xxx0111x1x1xxx0' then return TRUE;
25     if instr == '111111100x10xxx0xxx1110x0x0xxx' then return TRUE;
26     if instr == '111x11100x0xxx1xxx1110x1x0xxx' then return TRUE;
27     if instr == '111011111xxxxxxx0010101x1xxx0' then return TRUE;
28     if instr == '111111100x11xx1xxx111x1x01101' then return TRUE;
29     if instr == '111111111x1xx11xxx0011x1x0xxx0' then return TRUE;
30     if instr == '11101101xxxxxxx11110xxxxxxx' then return TRUE;
31     if instr == '111x11111x000xxxxxxx00x1001x1xxx' then return TRUE;
32     if instr == '111x11101x111001xxx111101x0xxx0' then return TRUE;
33     if instr == '111x11111x001xxxxx011x01x1xxx0' then return TRUE;
34     if instr == '111x11111x000xxxxx0x11101x1xxx' then return TRUE;
35     if instr == '111x11101x1xx001xxx111101x0xxx0' then return TRUE;
36     if instr == '111011110x1xxx0xxx01100x1x1xxx0' then return TRUE;
37     if instr == '111x11100x10xxx0xxx111x1x0xxx' then return TRUE;
38     if instr == '111100000xx110x11101xxxxxxx1' then return TRUE;
39     if instr == '111011110x0xxx0xxx01101x1x0xxx0' then return TRUE;
40     if instr == '111x11111x1xxxxxxx011x01x1xxx0' then return TRUE;
41     if instr == '111111111xxxxxxx0011001x1xxx0' then return TRUE;
42     if instr == '111x11100x111001xxx111010x0xxx1' then return TRUE;
43     if instr == '111x1101x0xxxxxxx011110xxxxxxx' then return TRUE;
44     if instr == '111x11111xxxxxxx000x001x1xxx0' then return TRUE;
45     if instr == '111111111x1xx00xxx0011x1x0xxx0' then return TRUE;
46     if instr == '1111110x1x0xxx0xxx01000x1x0xxx0' then return TRUE;
47     if instr == '111011110x1xxx0xxx00001x1x1xxx0' then return TRUE;
48     if instr == '111x11110x10xxx0xxx00110x1xxxxx0' then return TRUE;
49     if instr == '111x110xx01xxxxxxx01110xxxxxxx' then return TRUE;
50     if instr == '111111110x0xxx0xxx011x1x1x1xxx0' then return TRUE;
51     if instr == '111011110xxxxx0xxx01000x1x0xxx0' then return TRUE;
52     if instr == '111111111xxxxxxx0010x01x1xxx0' then return TRUE;
53     if instr == '111111110x00xxx0xxx00001x1x1xxx0' then return TRUE;
54     if instr == '111x11101x1xx1xxxxx111101x0xxx0' then return TRUE;
55     if instr == '111011110xxxxx0xxx01011x1x0xxx0' then return TRUE;
56     if instr == '111011000x0xxxxxxx0111xxxxxxx' then return TRUE;
57     if instr == '111x11100x1110x1xxx11110x110xxx' then return TRUE;
58     if instr == '111x11100x11xxx0xxx1110x0x0xxx' then return TRUE;
59     if instr == '111x11101x1xxxxxxx111101x0xxx0' then return TRUE;
60     if instr == '111x11110xxxxx0xxx00001x1x0xxx0' then return TRUE;
61     if instr == '111011101x1xxxxxxx01111100xxx' then return TRUE;
62     if instr == '111011100xx0xxxxxxx1011xxx1xxx' then return TRUE;
63     if instr == '111011110xxxxx0xxx01001x1x1xxx0' then return TRUE;
64     if instr == '111x11100x111x11xxx1110x0x0xxx1' then return TRUE;
65     if instr == '111x110xx01xxxxxxx011110xxxxxxx' then return TRUE;
66     if instr == '111011100x10xxx0xxx1110x0x0xxx' then return TRUE;
67     if instr == '111x11100x0xxx0xxx111x1x0xxx' then return TRUE;
68     if instr == '111x11100x11xx1xxx111x1x011x0' then return TRUE;
69     if instr == '111x11100x110xx1xxx11110x110xxx' then return TRUE;
70     if instr == '111011110x0xxx0xxx01100x1x1xxx0' then return TRUE;
71     if instr == '111111101xxxxx0xxx1110x0x0xxx1' then return TRUE;
72     if instr == '111111001xxxxxxx111110xxxxxxx' then return TRUE;
73     if instr == '11111100x11xx1xxx0111x0x0xxx1' then return TRUE;
74     if instr == '111111111x1xx0xxx0000101x0xxx0' then return TRUE;
75     if instr == '111x11111x000xxxxx0xxx0111xxx' then return TRUE;
76     if instr == '111x111010xxx01xxx111100x0xxx0' then return TRUE;
77     if instr == '1111110xxx1xxx0xxx01000x1x0xxx0' then return TRUE;
78     if instr == '11101101xxxxxxx111110xxxxxxx' then return TRUE;
79     if instr == '111x11111x000xxxxx0xxx01x1xxx' then return TRUE;
80     if instr == '111x11101111001xxx111100x0xxx0' then return TRUE;
81     if instr == '111011110x0xxx0xxx00001x1x1xxx0' then return TRUE;
82     if instr == '111x11100x11xx0xxx111x0x0xxx1' then return TRUE;
83     if instr == '111111111x11000xxx0010111x0xxx0' then return TRUE;
84     if instr == '111x11110x0xxx0xxx00110x1xxxxx0' then return TRUE;
85     if instr == '111x11110xxxxx0xxx0011x1x0xxx0' then return TRUE;
86     if instr == '111x11100x1110x1xxx111000x0xxx1' then return TRUE;
87     if instr == '111x11110x1xxx0xxx01101x1x0xxx0' then return TRUE;
88     if instr == '111100000xx10xx11101xxxxxxx1' then return TRUE;

```

```

89     if instr == '11111111x11xx10xxx001xxx1x0xxx0' then return TRUE;
90     if instr == '111x11100x11xxx1xxx111xx0x0xxx0' then return TRUE;
91     if instr == '111011101x10xxx0xxx1011x0x1xxx' then return TRUE;
92     if instr == '111x11100x110xx1xxx1110x0x0xxx1' then return TRUE;
93     if instr == '111x11100x11xxx1xxx11110x100xxx' then return TRUE;
94     if instr == '111011100x10xxx1xxx1111x1x0xxx' then return TRUE;
95     if instr == '111x11110x10xxx0xxx0010xx1xxx0' then return TRUE;
96     if instr == '111x111011110x01xxx111100x0xxx0' then return TRUE;
97     if instr == '111x11001x0xxxxxxxx011xxxxxxxx' then return TRUE;
98     if instr == '111x11101xxxxxxxx1110x0x0xxx0' then return TRUE;
99     if instr == '1110110xxx1xxxxxxxx11110xxxx' then return TRUE;
100    if instr == '111x11111xxxxxxxx0011101x1xxx0' then return TRUE;
101    if instr == '111011100x0xxx1xxx1111x1x0xxx' then return TRUE;
102    if instr == '111x11110x0xxx0xxx0010xx1xxx0' then return TRUE;
103    if instr == '111100000xx0xxx11101xxxxxxxx1' then return TRUE;
104    if instr == '111x11100x11xx0xxx1111x1x0xxx' then return TRUE;
105    if instr == '111x11100x0xxxxxxxx1110x0x0xxx' then return TRUE;
106    if instr == '111x11100x10xxx1xxx1110xxx0xxx' then return TRUE;
107    if instr == '11111101xxxxxxxx1111xxxxxxxx' then return TRUE;
108    if instr == '11101110xxx1xxxxxxxx1011xxx1xxx' then return TRUE;
109    if instr == '111x1101x0xxxxxxxx01110xxxxxxxx' then return TRUE;
110    if instr == '111x11101110xx01xxx111100x0xxx0' then return TRUE;
111    if instr == '1110110xxx1xxxxxxxx11110xxxxxxxx' then return TRUE;
112    if instr == '111011101xxxxxxxx1110x0x0xxx1' then return TRUE;
113    if instr == '111011000x0xxxxxxxx11110xxxxxxxx' then return TRUE;
114    if instr == '11111111x11xx01xxx00x1x1x0xxx0' then return TRUE;
115    if instr == '111111100x10xxx1xxx111xxx0xxx' then return TRUE;
116    return FALSE;

```

E2.1.259 IsMveLoadStoreInstruction

```

1  boolean IsMveLoadStoreInstruction(bits(32) instr)
2      if instr == '111x1101x0xxxxxxxx011110xxxx' then return TRUE;
3      if instr == '111x110xx01xxxxxxxx01110xxxx' then return TRUE;
4      if instr == '11111101xxxxxxxx1111xxxxxxxx' then return TRUE;
5      if instr == '111111001xxxxxxxx11110xxxx' then return TRUE;
6      if instr == '11101101xxxxxxxx111110xxxx' then return TRUE;
7      if instr == '111x110xx01xxxxxxxx01110xxxx' then return TRUE;
8      if instr == '111x11001x0xxxxxxxx011xxxxxxxx' then return TRUE;
9      if instr == '111x1101x0xxxxxxxx01110xxxx' then return TRUE;
10     if instr == '1110110xxx1xxxxxxxx11110xxxx' then return TRUE;
11     if instr == '1110110xxx1xxxxxxxx11110xxxx' then return TRUE;
12     if instr == '111111001xxxxxxxx11110xxxx' then return TRUE;
13     if instr == '11101101xxxxxxxx11110xxxx' then return TRUE;
14     return FALSE;

```

E2.1.260 IsOnes

```

1  // IsOnes()
2  // =====
3
4  boolean IsOnes(bits(N) x)
5      return x == Ones(N);

```

E2.1.261 IsPPB

```

1  // IsPPB()
2  // =====
3
4  boolean IsPPB(bits(32) address)
5      return address[31:20] == 0xE00[11:0];

```

E2.1.262 IsReqExcPriNeg

```

1 // IsReqExcPriNeg()
2 // =====
3
4 boolean IsReqExcPriNeg(boolean secure)
5     // This function checks if the requested execution priority is negative for
6     // the specified security domain. That is, NMI or HardFault is active, or
7     // FAULTMASK is set. It does not take account of AIRCR.PRIS so returns TRUE
8     // if FAULTMASK_NS is set even if PRIS is set to restrict Non-secure priorities
9     // to the range 0x80-0x7E.
10    neg = IsActiveForState(NMI, secure) || IsActiveForState(HardFault, secure);
11    if HaveMainExt() then
12        faultmask = if secure then FAULTMASK_S else FAULTMASK_NS;
13        if faultmask.FM == '1' then
14            neg = TRUE;
15    return neg;
16
17
18 boolean IsReqExcPriNeg(boolean secure, AccType acctype)
19     // If the access is due to lazy floating-point state preservation the FPCCR flag
20     // indicating whether a HardFault could be taken is used to determine if the
21     // priority should be considered to be negative rather than the current
22     // execution priority.
23     if acctype == AccType_LAZYFP then
24         neg = FPCCR_S.HFRDY == '0';
25     else
26         neg = IsReqExcPriNeg(secure);
27     return neg;

```

E2.1.263 IsReturn

```

1 // IsReturn()
2 // =====
3
4 AddrType IsReturn(bits(32) address)
5     addrtype = AddrType_NORMAL;
6
7     if (HaveSecurityExt() && address=='1111 1110 1111 1111 1111 1111 1111 111x') then
8         addrtype = AddrType_FNC_RETURN;
9
10    elsif CurrentMode() == PEmode_Handler && address[31:24] == '11111111' then
11        addrtype = AddrType_EXC_RETURN;
12    return addrtype;

```

E2.1.264 IsSCS

```

1 // IsSCS()
2 // =====
3
4 boolean IsSCS(bits(32) address)
5     return ( (address[31:12] == 0xE000E[19:0]) ||
6             (address[31:12] == 0xE002E[19:0]) );

```

E2.1.265 IsSecure

```

1 // IsSecure()
2 // =====
3
4 boolean IsSecure()
5     return HaveSecurityExt() && CurrentState == SecurityState_Secure;

```

E2.1.266 IsZero

```

1 // IsZero()
2 // =====

```

```

3
4 boolean IsZero(bits(N) x)
5     return x == Zeros(N);

```

E2.1.267 IsZeroBit

```

1 // IsZeroBit()
2 // =====
3
4 bit IsZeroBit(bits(N) x)
5     return if IsZero(x) then '1' else '0';

```

E2.1.268 ITAdvance

```

1 // ITAdvance()
2 // =====
3
4 ITSTATEType ITAdvance(ITSTATEType itState)
5     // If the mask field (in other words the bottom 4 bits) are zero then the ITSTATE bits
6     // hold ECI information and therefore the normal state advancement should
7     // not take place.
8     if itState[3:0] == '1000' then
9         itState = '00000000';
10    elseif itState[3:0] != '0000' then
11        itState[4:0] = LSL(itState[4:0], 1);
12    return itState;

```

E2.1.269 ITSTATE

```

1 // ITSTATE
2 // =====
3
4 ITSTATEType ITSTATE
5     return ThisInstrITState();
6
7 ITSTATE = ITSTATEType value
8     // Writes to ITSTATE do not take effect immediately, instead they change the
9     // value returned by NextInstrITState().
10    _NextInstrITState = value;
11    _ITStateChanged = TRUE;

```

E2.1.270 ITSTATEType

```

1 // If-Then execution state bits for the T32 IT instruction.
2
3 type ITSTATEType = bits(8);

```

E2.1.271 LastInITBlock

```

1 // LastInITBlock()
2 // =====
3
4 boolean LastInITBlock()
5     return (ITSTATE[3:0] == '1000');

```

E2.1.272 LoadWritePC

```

1 // LoadWritePC()
2 // =====
3
4 LoadWritePC(bits(32) address, integer baseReg, bits(32) baseRegVal, boolean baseRegUpdate,

```

```

5         boolean spLimCheck, boolean setBti)
6
7     if baseRegUpdate then
8         oldBaseVal = R[baseReg];
9         if spLimCheck then
10            RSPCheck[baseReg] = baseRegVal;
11        else
12            R[baseReg] = baseRegVal;
13
14        // Attempt to update the PC, which may result in a fault.
15        excInfo = BranchReturn(address, FALSE, setBti);
16
17        if baseRegUpdate && excInfo.fault != NoFault then
18            // Restore the previous base reg value, SP limit checking is not performed
19            if baseReg == 13 then
20                exc = _SP(LookUpRName(baseReg), FALSE, TRUE, oldBaseVal);
21                assert exc.fault == NoFault;
22            else
23                R[baseReg] = oldBaseVal;
24
25        HandleException(excInfo);
26
27    LoadWritePC(bits(32) address, integer baseReg, bits(32) baseRegVal, boolean baseRegUpdate,
28               boolean spLimCheck)
29        LoadWritePC(address, baseReg, baseRegVal, baseRegUpdate, spLimCheck, FALSE);

```

E2.1.273 LockedUp

```

1 // Indicates the PE is locked up
2
3 boolean LockedUp;

```

E2.1.274 Lockup

```

1 // Lockup()
2 // =====
3
4 Lockup(boolean termInst)
5     LockedUp = TRUE;
6     // Branch to the lockup address.
7     BranchTo(0xEFFFFFFE[31:0], TRUE, FALSE);
8     // Invalidate the instruction buffer and set the length of the current
9     // instruction to zero so NextInstrAddr() reports the correct lockup
10    // address.
11    ClearInFlightInstructions();
12    // If requested, terminate execution of the pseudo code for this
13    // instruction.
14    if termInst then
15        EndOfInstruction();

```

E2.1.275 LookUpRName

```

1 // LookUpRName()
2 // =====
3
4 RNames LookUpRName(integer n)
5     case n of
6         when 0 result = RNames0;
7         when 1 result = RNames1;
8         when 2 result = RNames2;
9         when 3 result = RNames3;
10        when 4 result = RNames4;
11        when 5 result = RNames5;
12        when 6 result = RNames6;
13        when 7 result = RNames7;

```

```

14     when 8 result = RNames8;
15     when 9 result = RNames9;
16     when 10 result = RNames10;
17     when 11 result = RNames11;
18     when 12 result = RNames12;
19     when 13 result = LookUpSP();
20     when 14 result = RNamesLR;
21     when 15 result = RNamesPC;
22     otherwise assert(FALSE);
23     return result;

```

E2.1.276 LookUpSP

```

1 // LookUpSP()
2 // =====
3
4 RNames LookUpSP()
5     return LookUpSP_with_security_mode(IsSecure(), CurrentMode());

```

E2.1.277 LookUpSP_with_security_mode

```

1 // LookUpSP_with_security_mode()
2 // =====
3
4 RNames LookUpSP_with_security_mode(boolean isSecure, PMode mode)
5     RNames sp;
6     bit spSel;
7
8     // Get the SPSEL bit corresponding to the Security state requested.
9     if isSecure then
10         spSel = CONTROL_S.SPSEL;
11     else
12         spSel = CONTROL_NS.SPSEL;
13
14     // Determine which stack pointer should be used.
15     if spSel == '1' && mode == PMode_Thread then
16         if isSecure then
17             sp = RNamesSP_Process_Secure;
18         else
19             sp = RNamesSP_Process_NonSecure;
20     else
21         if isSecure then
22             sp = RNamesSP_Main_Secure;
23         else
24             sp = RNamesSP_Main_NonSecure;
25     return sp;

```

E2.1.278 LookUpSPLim

```

1 // LookUpSPLim()
2 // =====
3
4 bits(32) LookUpSPLim(RNames spreg)
5     case spreg of
6         when RNamesSP_Main_Secure limit = MSPLIM_S.LIMIT:'000';
7         when RNamesSP_Process_Secure limit = PSPLIM_S.LIMIT:'000';
8         when RNamesSP_Main_NonSecure
9             limit = if HaveMainExt() then MSPLIM_NS.LIMIT:'000' else Zeros(32);
10        when RNamesSP_Process_NonSecure
11            limit = if HaveMainExt() then PSPLIM_NS.LIMIT:'000' else Zeros(32);
12        otherwise
13            assert (FALSE);
14
15    return limit;

```


E2.1.279 LowestSetBit

```

1 // LowestSetBit()
2 // =====
3
4 integer LowestSetBit(bits(N) x)
5     for i = 0 to N-1
6         if x[i] == '1' then return i;
7     return N;

```

E2.1.280 LR

```

1 // LR
2 // ==
3
4 // Non-assignment form
5 bits(32) LR
6     return RName[RNamesLR];
7
8 // Assignment form
9
10 LR = bits(32) value
11     RName[RNamesLR] = value;

```

E2.1.281 LSL

```

1 // LSL()
2 // =====
3
4 bits(N) LSL(bits(N) x, integer shift)
5     assert shift >= 0;
6     if shift == 0 then
7         result = x;
8     else
9         (result, -) = LSL_C(x, shift);
10    return result;

```

E2.1.282 LSL_C

```

1 // LSL_C()
2 // =====
3
4 (bits(N), bit) LSL_C(bits(N) x, integer shift)
5     assert shift > 0;
6     extended_x = x : Zeros(shift);
7     result = extended_x[N-1:0];
8     carry_out = extended_x[N];
9     return (result, carry_out);
10
11 (bits(N), bits(M)) LSL_C(bits(N) x, bits(M) carry_in, integer shift)
12     assert shift > 0 && shift <= M;
13     cin = LSL(carry_in, M - shift);
14     extended_x = LSL(Zeros(M) : x : cin, shift);
15     result = extended_x[N+ M-1:M ];
16     carry_out = extended_x[N+2*M-1:N+M];
17     return (result, carry_out);

```

E2.1.283 LSR

```

1 // LSR()
2 // =====
3
4 bits(N) LSR(bits(N) x, integer shift)

```

```

5      assert shift >= 0;
6      if shift == 0 then
7          result = x;
8      else
9          (result, -) = LSR_C(x, shift);
10     return result;

```

E2.1.284 LSR_C

```

1  // LSR_C()
2  // =====
3
4  (bits(N), bit) LSR_C(bits(N) x, integer shift)
5      assert shift > 0;
6      extended_x = ZeroExtend(x, shift+N);
7      result = extended_x[shift+N-1:shift];
8      carry_out = extended_x[shift-1];
9      return (result, carry_out);

```

E2.1.285 LTPSIZE

```

1  // LTPSIZE - non-assignment form
2  // =====
3
4  integer LTPSIZE
5      if HaveMve() && ActiveFPState() then
6          size = UInt(FPSCR.LTPSIZE);
7      else
8          // Full vector length, so no loop tail predication.
9          size = 4;
10     return size;

```

E2.1.286 MAIRDecode

```

1  // MAIRDecode()
2  // =====
3
4  MemoryAttributes MAIRDecode(bits(8) attrfield, bits(2) sh)
5      // Converts the MAIR attributes to orthogonal attribute and
6      // hint fields.
7      MemoryAttributes memattrs;
8      // Decoding MAIR0 or MAIR1 Registers.
9      if attrfield[7:4] == '0000' then
10         unpackinner = FALSE;
11         memattrs.memtype = MemType_Device;
12         memattrs.shareable = TRUE;
13         memattrs.outershareable = TRUE;
14         memattrs.innerattrs = bits(2) UNKNOWN;
15         memattrs.outerattrs = bits(2) UNKNOWN;
16         memattrs.innerhints = bits(2) UNKNOWN;
17         memattrs.outerhints = bits(2) UNKNOWN;
18         memattrs.innertransient = boolean UNKNOWN;
19         memattrs.outertransient = boolean UNKNOWN;
20         case attrfield[3:0] of
21             when '0000' memattrs.device = DeviceType_nGnRnE;
22             when '0100' memattrs.device = DeviceType_nGnRE;
23             when '1000' memattrs.device = DeviceType_nGRE;
24             when '1100' memattrs.device = DeviceType_GRE;
25             otherwise UNPREDICTABLE;
26         else
27             unpackinner = TRUE;
28             memattrs.memtype = MemType_Normal;
29             memattrs.device = DeviceType UNKNOWN;
30             memattrs.outerhints = attrfield[5:4];
31             memattrs.shareable = sh[1] == '1';

```

```

32     memattrs.outershareable = sh == '10';
33     if sh == '01' then UNPREDICTABLE;
34
35     if attrfield[7:6] == '00' then
36         memattrs.outerattrs = '10';
37         memattrs.outertransient = TRUE;
38     elsif attrfield[7:6] == '01' then
39         if attrfield[5:4] == '00' then
40             memattrs.outerattrs = '00';
41             memattrs.outertransient = FALSE;
42         else
43             memattrs.outerattrs = '11';
44             memattrs.outertransient = TRUE;
45     else
46         memattrs.outerattrs = attrfield[7:6];
47         memattrs.outertransient = FALSE;
48     if unpackinner then
49         if attrfield[3:0] == '0000' then UNPREDICTABLE;
50     else
51         if attrfield[3:2] == '00' then
52             memattrs.innerattrs = '10';
53             memattrs.innerhints = attrfield[1:0];
54             memattrs.innertransient = TRUE;
55         elsif attrfield[3:2] == '01' then
56             memattrs.innerhints = attrfield[1:0];
57             if attrfield[1:0] == '00' then
58                 memattrs.innerattrs = '00';
59                 memattrs.innertransient = FALSE;
60             else
61                 memattrs.innerattrs = '11';
62                 memattrs.innertransient = TRUE;
63         elsif attrfield[3:2] == '10' then
64             memattrs.innerhints = attrfield[1:0];
65             memattrs.innerattrs = '10';
66             memattrs.innertransient = FALSE;
67         elsif attrfield[3:2] == '11' then
68             memattrs.innerhints = attrfield[1:0];
69             memattrs.innerattrs = '11';
70             memattrs.innertransient = FALSE;
71         else UNPREDICTABLE;
72     return memattrs;

```

E2.1.287 MarkExclusiveGlobal

```

1 // MarkExclusiveGlobal
2 // =====
3 // Records in a global record that PE has requested "exclusive access" covering
4 // at least size bytes from the address.
5
6 MarkExclusiveGlobal(bits(32) address, integer processorid, integer size);

```

E2.1.288 MarkExclusiveLocal

```

1 // MarkExclusiveLocal
2 // =====
3 // Records in a local record that PE has requested "exclusive access" covering
4 // at least size bytes from the address.
5
6 MarkExclusiveLocal(bits(32) address, integer processorid, integer size);

```

E2.1.289 Max

```

1 // Max()
2 // =====
3

```

```

4 integer Max(integer a, integer b)
5     return if a >= b then a else b;
6
7 real Max(real a, real b)
8     return if a >= b then a else b;

```

E2.1.290 MaxExceptionNum

```

1 // MaxExceptionNum()
2 // =====
3 // Returns the maximum exception number supported.
4
5 integer MaxExceptionNum()
6     if HaveMainExt() then
7         return 511;
8     else
9         return 47;

```

E2.1.291 MemA

```

1 // MemA[]
2 // =====
3
4 bits(8*size) MemA[bits(32) address, integer size]
5     return MemA_with_priv(address, size, FindMemPriv(), TRUE);
6
7 MemA[bits(32) address, integer size] = bits(8*size) value
8     MemA_with_priv(address, size, FindMemPriv(), TRUE) = value;
9     return;

```

E2.1.292 MemA_MVE

```

1 // MemA_MVE[]
2 // =====
3
4 // Non-assignment form
5
6 bits(8*size) MemA_MVE[bits(32) address, integer size]
7     (excInfo, value) = MemA_with_priv_security(address, size, AccType_MVE,
8                                             FindMemPriv(), IsSecure(), TRUE);
9     HandleException(excInfo);
10    return value;
11
12 // Assignment form
13
14 MemA_MVE[bits(32) address, integer size] = bits(8*size) value
15     excInfo = MemA_with_priv_security(address, size, AccType_MVE, FindMemPriv(),
16                                     IsSecure(), TRUE, value);
17     HandleException(excInfo);
18

```

E2.1.293 MemA_with_priv

```

1 // MemA_with_priv[]
2 // =====
3
4 // Non-assignment form
5
6 bits(8*size) MemA_with_priv[bits(32) address, integer size, Privilege privileged,
7                             boolean aligned]
8     (excInfo, value) = MemA_with_priv_security(address, size, AccType_NORMAL,
9                                             privileged, IsSecure(), aligned);
10    HandleException(excInfo);
11    return value;

```

```

12
13 // Assignment form
14
15 MemA_with_priv[bits(32) address, integer size, Privilege privileged,
16               boolean aligned] = bits(8*size) value
17     excInfo = MemA_with_priv_security(address, size, AccType_NORMAL, privileged,
18                                     IsSecure(), aligned, value);
19     HandleException(excInfo);

```

E2.1.294 MemA_with_priv_security

```

1 // MemA_with_priv_security()
2 // =====
3
4 // Non-assignment form
5
6 (ExcInfo, bits(8*size)) MemA_with_priv_security(bits(32) address, integer size,
7                                               AccType acctype, Privilege privileged,
8                                               boolean secure, boolean aligned)
9
10 // Check alignment.
11 excInfo = DefaultExcInfo();
12 if !IsAligned(address, size) then
13     if HaveMainExt() then
14         if secure then
15             UFSR_S.UNALIGNED = '1';
16         else
17             UFSR_NS.UNALIGNED = '1';
18         // Create the exception. NOTE: If Main Extension is not implemented the fault
19         // always escalates to a HardFault.
20         excInfo = CreateException(UsageFault, TRUE, secure);
21
22 // Check permissions and get attributes
23 if excInfo.fault == NoFault then
24     (excInfo, memaddrdesc) = ValidateAddress(address, acctype,
25                                             privileged == Privilege_Priv, secure,
26                                             FALSE, aligned);
27
28 if excInfo.fault == NoFault then
29     // Memory array access, and sort out endianness.
30     (error, value) = _Mem(memaddrdesc, size);
31
32 if privileged == Privilege_ForcedUnpriv then
33     dwtPriv = CurrentModeIsPrivileged(secure);
34 else
35     dwtPriv = privileged == Privilege_Priv;
36
37 // Check if a synchronous BusFault occurred. NOTE: Asynchronous BusFaults are handled
38 // in RaiseAsyncBusFault().
39 if error then
40     value = bits(8*size) UNKNOWN;
41     if HaveMainExt() then
42         case acctype of
43             when AccType_VECTABLE
44                 HFSR.VECTTBL = '1';
45                 excInfo = CreateException(HardFault, TRUE, AIRCR.BFHFNMIN == '0');
46             when AccType_STACK
47                 BFSR.UNSTKERR = '1';
48                 excInfo = CreateException(BusFault, FALSE, secure);
49             when AccType_NORMAL, AccType_MVE, AccType_ORDERED
50                 BFAR.ADDRESS = address;
51                 BFSR.BFARVALID = '1';
52                 BFSR.PRECISERR = '1';
53                 // Generate BusFault exception if it cannot be ignored.
54                 if !IsReqExcPriNeg(secure) || (CCR.BFHFNMIN == '0') then
55                     excInfo = CreateException(BusFault, FALSE, secure);
56             otherwise
57                 // Some access types do not call this function
58                 assert(FALSE);

```

```

58
59 // Check for Watchpoint Match.
60 elseif (IsDWTEnabled(secure, dwtPriv) &&
61         (acctype != AccType_VECTABLE ||
62          !boolean IMPLEMENTATION_DEFINED "Ignore Vector table fetch")) then
63     // Instructions with the access type unprivileged access the DWT with the current
64     // privilege mode, for other accesses the access type specifies the privilege.
65     bits(32) dvalue = ZeroExtend(value);
66     DWT_DataMatch(address, size, dvalue, TRUE, !secure, dwtPriv, acctype);
67
68     if BigEndian(address, size) then
69         value = BigEndianReverse(value, size);
70
71     return (excInfo, value);
72
73 // Assignment form
74
75 ExcInfo MemA_with_priv_security(bits(32) address, integer size, AccType acctype,
76                                Privilege privileged, boolean secure, boolean aligned,
77                                bits(8*size) value)
78
79 // Check alignment.
80 excInfo = DefaultExcInfo();
81 if !IsAligned(address, size) then
82     if HaveMainExt() then
83         if secure then
84             UFSR_S.UNALIGNED = '1';
85         else
86             UFSR_NS.UNALIGNED = '1';
87     // Create the exception. NOTE: If Main Extension is not implemented the fault
88     // always escalates to a HardFault.
89     excInfo = CreateException(UsageFault, TRUE, secure);
90
91 // Check permissions and get attributes
92 if excInfo.fault == NoFault then
93     (excInfo, memaddrdesc) = ValidateAddress(address, acctype,
94                                             privileged == Privilege_Priv, secure,
95                                             TRUE, aligned);
96
97 if excInfo.fault == NoFault then
98     // Effect on exclusives.
99     if memaddrdesc.memattrs.shareable then
100         ClearExclusiveByAddress(memaddrdesc.paddress,
101                                ProcessorID(), size); // see Note
102
103 // Sort out endianness, then memory array access.
104 if BigEndian(address, size) then
105     value = BigEndianReverse(value, size);
106
107 // Check for Watchpoint Match.
108 if privileged == Privilege_ForcedUnpriv then
109     dwtPriv = CurrentModeIsPrivileged(secure);
110 else
111     dwtPriv = privileged == Privilege_Priv;
112 if IsDWTEnabled(secure, dwtPriv) then
113     // Instructions with the access type unprivileged access the DWT with the current
114     // privilege mode, for other accesses the access type specifies the privilege.
115     bits(32) dvalue = ZeroExtend(value);
116     DWT_DataMatch(address, size, dvalue, FALSE, !secure, dwtPriv, acctype);
117
118 if _Mem(memaddrdesc, size, value) then
119     // Synchronous BusFault occurred. NOTE: Asynchronous BusFaults are handled
120     // in RaiseAsyncBusFault().
121     if HaveMainExt() then
122         case acctype of
123             when AccType_STACK
124                 BFSR.STKERR = '1';
125                 excInfo = CreateException(BusFault, FALSE, secure);
126             when AccType_LAZYFP
127                 BFSR.LSPERR = '1';

```

```

127         excInfo      = CreateException(BusFault, FALSE, secure);
128     when AccType_NORMAL, AccType_MVE, AccType_ORDERED
129         BFAR.ADDRESS  = address;
130         BFSR.BFARVALID = '1';
131         BFSR.PRECISERR = '1';
132         // Generate BusFault exception if it cannot be ignored.
133         if !IsReqExcPriNeg(secure) || (CCR.BFHFNMIGN == '0') then
134             excInfo = CreateException(BusFault, FALSE, secure);
135         otherwise
136             // Some access types do not call this function.
137             assert(FALSE);
138     return excInfo;

```

E2.1.295 MemD_with_priv_security

```

1  // MemD_with_priv_security()
2  // =====
3
4  // Non-assignment form
5  (boolean, bits(8*size)) MemD_with_priv_security(AddressDescriptor attr, integer size)
6      // Debugger accesses always specify their required privilege level or Security states,
7      // but can be demoted.
8      (secure, privileged, error) = DAPCheck(attr.address, attr.accattrs.ispriv,
9                                          !attr.memattrs.NS);
10
11  if !error then
12      (excInfo, memaddrdesc) = ValidateAddress(attr.address, AccType_DBG,
13                                              privileged, secure,
14                                              attr.accattrs.iswrite,
15                                              IsAligned(attr.address, size));
16
17      if (secure && DHCSR.S_SUIDE == '0') || (!secure && DHCSR.S_NSUIDE == '0') then
18          // Inherit memory attributes from IMPDEF debugger interface if not
19          // accessed via UDE. The debugger-specified security attributes (NS-Req) is
20          // replaced by the NS-Attr obtained from the SAU/IDAU.
21          memNS = memaddrdesc.memattrs.NS;
22          memaddrdesc.memattrs = attr.memattrs;
23          memaddrdesc.accattrs.acctype = attr.accattrs.acctype;
24          memaddrdesc.memattrs.NS = memNS;
25
26      error = (excInfo.fault != NoFault);
27  if !error then
28      (error, value) = _Mem(memaddrdesc, size);
29  if error then
30      value = bits(8*size) UNKNOWN;
31
32      // No exception is generated here since the debugger should not be able to cause
33      // exceptions in the PE. Instead, the caller should check against NoFault and return
34      // that information to the debugger.
35      return (error, value);
36
37
38  // Assignment form
39  boolean MemD_with_priv_security(AddressDescriptor attr, integer size, bits(8*size) value)
40      // Debugger accesses always specify their required privilege level or Security states,
41      // but can be demoted.
42      (secure, privileged, error) = DAPCheck(attr.address,
43                                          attr.accattrs.ispriv,
44                                          !attr.memattrs.NS);
45
46  if !error then
47      (excInfo, memaddrdesc) = ValidateAddress(attr.address,
48                                              AccType_DBG,
49                                              privileged,
50                                              secure,
51                                              attr.accattrs.iswrite,
52                                              IsAligned(attr.address, size));
53

```

```

54     if (secure && DHCSR.S_SUIDE == '0') || (!secure && DHCSR.S_NSUIDE == '0') then
55         // Inherit memory attributes from IMPDEF debugger interface if not
56         // accessed through the UDE. The debugger-specified security attributes
57         // (NS-Req) is replaced by the NS-Attr obtained from the SAU/IDAU.
58         memNS = memaddrdesc.memattrs.NS;
59         memaddrdesc.memattrs = attr.memattrs;
60         memaddrdesc.acctype = attr.acctype;
61         memaddrdesc.memattrs.NS = memNS;
62
63         error = (excInfo.fault != NoFault);
64     if !error then
65         error = _Mem(memaddrdesc, size, value);
66
67         // No exception is generated here since the debugger should not be able to cause
68         // exceptions in the PE. Instead, the caller should check against NoFault and return
69         // that information to the debugger.
70     return error;

```

E2.1.296 MemI

```

1  // MemI()
2  // =====
3
4  bits(16) MemI[bits(32) address]
5      // Check permissions and get attributes
6      // NOTE: The privilege flag passed to ValidateAddress may be overridden if
7      // the Security of the memory is different from the current Security
8      // state, for example when performing a Non-secure to Secure function call.
9      (excInfo, memaddrdesc) = ValidateAddress(address, AccType_IFETCH, FindPriv(),
10                                              IsSecure(), FALSE, TRUE);
11
12     if excInfo.fault == NoFault then
13         (error, value) = _Mem(memaddrdesc, 2);
14         if error then
15             value = bits(16) UNKNOWN;
16             BFSR.IBUSERR = '1';
17             // Create the exception. NOTE: If the Main Extension is not implemented the fault
18             // always escalates to a HardFault.
19             excInfo = CreateException(BusFault);
20         HandleException(excInfo);
21     return value;

```

E2.1.297 MemO

```

1  // MemO[] - non-assignment form
2  // =====
3
4  bits(8*size) MemO[bits(32) address, integer size]
5      (excInfo, value) = MemA_with_priv_security(address, size, AccType_ORDERED,
6                                              FindMemPriv(), IsSecure(), TRUE);
7      HandleException(excInfo);
8      return value;
9
10
11 // MemO[] - assignment form
12 // =====
13
14 MemO[bits(32) address, integer size] = bits(8*size) value
15     excInfo = MemA_with_priv_security(address, size, AccType_ORDERED, FindMemPriv(),
16                                     IsSecure(), TRUE, value);
17     HandleException(excInfo);

```

E2.1.298 MemoryAttributes

```

1  // v8-M Memory Attributes
2  type MemoryAttributes is (

```



```

3     MemType memtype,
4     DeviceType device,           // For Device memory.
5     bits(2) innerattrs,         // The possible encodings for each attributes field are:
6     bits(2) outerattrs,         // '00' = Non-cacheable, '01' = RESERVED,
7                                 // '10' = Write-Through, '11' = Write-Back.
8     bits(2) innerhints,         // The possible encodings for the hints are as follows
9     bits(2) outerhints,         // '00' = No-Allocate, '01' = Write-Allocate,
10                                // '10' = Read-Allocate, '11' = Read-Allocate and Write-Allocate.
11     boolean NS,                 // TRUE if Non-secure, else FALSE.
12     boolean innertransient,
13     boolean outertransient,
14     boolean shareable,
15     boolean outershareable
16 )

```

E2.1.299 MemType

```

1 // Types of memory
2
3 enumeration MemType {MemType_Normal, MemType_Device};

```

E2.1.300 MemU

```

1 // MemU[]
2 // =====
3
4 // Non-assignment form, used for memory reads
5 // =====
6
7 bits(8*size) MemU[bits(32) address, integer size]
8     if HaveMainExt() then
9         return MemU_with_priv[address, size, FindMemPriv()];
10    else
11        return MemA[address, size];
12
13
14 // Assignment form, used for memory writes
15 // =====
16
17 MemU[bits(32) address, integer size] = bits(8*size) value
18     if HaveMainExt() then
19         MemU_with_priv[address, size, FindMemPriv()] = value;
20    else
21        MemA[address, size] = value;
22    return;

```

E2.1.301 MemU_unpriv

```

1 // MemU_unpriv[]
2 // =====
3
4 // Non-assignment form
5
6 bits(8*size) MemU_unpriv[bits(32) address, integer size]
7     return MemU_with_priv[address, size, Privilege_ForcedUnpriv];
8
9 // Assignment form
10
11 MemU_unpriv[bits(32) address, integer size] = bits(8*size) value
12     MemU_with_priv[address, size, Privilege_ForcedUnpriv] = value;
13    return;

```

E2.1.302 MemU_with_priv

```

1  // MemU_with_priv[]
2  // =====
3  // Due to single-copy atomicity constraints, the aligned accesses are distinguished from
4  // the unaligned accesses:
5  // * Aligned accesses are performed at their size.
6  // * Unaligned accesses are expressed as a set of bytes.
7
8  // Non-assignment form
9
10 bits(8*size) MemU_with_priv[bits(32) address, integer size, Privilege privileged]
11
12     bits(8*size) value;
13     // Do aligned access, take alignment fault, or do sequence of bytes.
14     if address == Align(address, size) then
15         value = MemA_with_priv[address, size, privileged, TRUE];
16     elsif CCR.UNALIGN_TRP == '1' then
17         UFSR.UNALIGNED = '1';
18         excInfo = CreateException(UsageFault);
19         HandleException(excInfo);
20     else // If unaligned access.
21         for i = 0 to size-1
22             value[8*i+7:8*i] = MemA_with_priv[address+i, 1, privileged, FALSE];
23             if BigEndian(address, size) then
24                 value = BigEndianReverse(value, size);
25
26     return value;
27
28 // Assignment form
29
30 MemU_with_priv[bits(32) address, integer size, Privilege privileged] = bits(8*size) value
31
32     // Do aligned access, take alignment fault, or do sequence of bytes.
33     if address == Align(address, size) then
34         MemA_with_priv[address, size, privileged, TRUE] = value;
35     elsif CCR.UNALIGN_TRP == '1' then
36         UFSR.UNALIGNED = '1';
37         excInfo = CreateException(UsageFault);
38         HandleException(excInfo);
39     else // If unaligned access.
40         if BigEndian(address, size) then
41             value = BigEndianReverse(value, size);
42         for i = 0 to size-1
43             MemA_with_priv[address+i, 1, privileged, FALSE] = value[8*i+7:8*i];
44
45     return;

```

E2.1.303 MergeExcInfo

```

1  // MergeExcInfo()
2  // =====
3
4  ExcInfo MergeExcInfo(ExcInfo a, ExcInfo b)
5      // The ExcInfo structure is used to determine which exception should be
6      // taken, and how it should be handled (mainly in the case of derived
7      // exceptions).
8      if (b.fault == NoFault) || (a.isTerminal && !b.isTerminal) then
9          exc = a;
10     elsif (a.fault == NoFault) || (b.isTerminal && !a.isTerminal) then
11         exc = b;
12     elsif (a.fault == b.fault) && (a.isSecure == b.isSecure) then
13         exc = a;
14     else
15         // Propagate the fault with the highest priority (lowest numerical
16         // value).
17         aPri = ExceptionPriority(a.fault, a.isSecure, FALSE);
18         bPri = ExceptionPriority(b.fault, b.isSecure, FALSE);
19
20         // Compare the exception priority values. Exception with the highest priority, which

```

```

21      // is the lowest numerical value, is taken and the other exception may be pended.
22      if aPri < bPri then
23          exc = a;
24          pend = b;
25      elseif bPri < aPri then
26          exc = b;
27          pend = a;
28      // If both priority values are equal, the exception numbers are compared.
29      // The exception with the lowest exception number is taken and the other
30      // exception may be pended.
31      elseif a.fault < b.fault then
32          exc = a;
33          pend = b;
34      elseif b.fault < a.fault then
35          exc = b;
36          pend = a;
37      // If the two exception numbers are equal, the Secure exception is taken and the
38      // Non-secure exception may be pended.
39      elseif a.isSecure && !b.isSecure then
40          exc = a;
41          pend = b;
42      // In any other case exception (b) is taken and exception (a) is pended.
43      else
44          exc = b;
45          pend = a;
46
47      // It is IMPLEMENTATION_DEFINED whether all exceptions generated are visible or not.
48      // If visible, the highest priority exception will become active and lower priority
49      // exceptions will get pended.
50      if boolean IMPLEMENTATION_DEFINED "Overridden exceptions pended" then
51          SetPending(pend.fault, pend.isSecure, TRUE);
52      return exc;

```

E2.1.304 Min

```

1  // Min()
2  // =====
3
4  integer Min(integer a, integer b)
5      return if a <= b then a else b;
6
7  real Min(real a, real b)
8      return if a <= b then a else b;

```

E2.1.305 MonitorCanPreempt

```

1  // MonitorCanPreempt()
2  // =====
3
4  boolean MonitorCanPreempt()
5      monSecure = HaveSecurityExt() && DEMCR.SDME == '1';
6      return ExceptionPriority(DebugMonitor, monSecure, TRUE) < ExecutionPriority();

```

E2.1.306 MPUCheck

```

1  // MPUCheck()
2  // =====
3
4  (MemoryAttributes, Permissions) MPUCheck(bits(32) address, AccType acctype,
5      boolean ispriv, boolean secure)
6
7      assert(HaveSecurityExt() || !secure);
8      MemoryAttributes attributes;
9      Permissions perms;
10     attributes = DefaultMemoryAttributes(address);

```

```

11 perms      = DefaultPermissions(address);
12 // Assume no valid MPU region and not using system address map.
13 hit        = FALSE;
14 mpuCtrl    = if secure then MPU_CTRL_S else MPU_CTRL_NS;
15
16 // Determine what MPU permissions should apply based on access type and MPU
17 // configuration.
18 if acctype == AccType_VECTABLE || IsPPB(address) then
19     hit = TRUE; // Use system address map for PPB and vector table lookups.
20 elseif acctype == AccType_DBG && secure && DHCSR.S_SUIDE == '0' then
21     hit = TRUE; // Use the debugger-provided memory attributes.
22 elseif acctype == AccType_DBG && !secure && DHCSR.S_NSUIDE == '0' then
23     hit = TRUE; // Use the debugger-provided memory attributes.
24 elseif mpuCtrl.ENABLE == '0' then
25     if mpuCtrl.HFNMIENA == '1' then UNPREDICTABLE;
26     else hit = TRUE; // Always use system address map if MPU disabled.
27 elseif mpuCtrl.HFNMIENA == '0' && IsReqExcPriNeg(secure, acctype) then
28     hit = TRUE; // Optionally use default for HardFault, NMI and FAULTMASK.
29 else // MPU is enabled so check each individual region.
30     if (mpuCtrl.PRIVDEFENA == '1') && ispriv then
31         hit = TRUE; // Optional default as background for Privileged accesses.
32
33     regionMatched = FALSE;
34     mpuType       = if secure then MPU_TYPE_S else MPU_TYPE_NS;
35     for r = 0 to (UInt(mpuType.DREGION) - 1)
36         if secure then
37             rbar = __MPU_RBAR_S[r];
38             rlar = __MPU_RLAR_S[r];
39         else
40             rbar = __MPU_RBAR_NS[r];
41             rlar = __MPU_RLAR_NS[r];
42
43     // MPU region enabled so perform checks.
44     if rlar.EN == '1' then
45         if (UInt(address) >= UInt(rbar.BASE : '00000')) &&
46             (UInt(address) <= UInt(rlar.LIMIT : '11111')) then
47             // Flag error if multiple regions match.
48             if regionMatched then
49                 perms.regionValid = FALSE;
50                 perms.region      = Zeros(8);
51                 hit               = FALSE;
52             else
53                 regionMatched     = TRUE;
54                 perms.ap          = rbar.AP;
55                 if (rbar.XN == '1') || (ispriv && (rlar.PXN == '1')) then
56                     perms.xn = '1';
57                 else
58                     perms.xn = '0';
59                 perms.region      = r[7:0];
60                 perms.regionValid = TRUE;
61                 hit              = TRUE;
62                 sh               = rbar.SH;
63
64     // Parsing MAIR0/1 Register fields.
65     index      = UInt(rlar.AttrIdx);
66     mair       = (if secure then MPU_MAIR1_S : MPU_MAIR0_S else
67                   MPU_MAIR1_NS : MPU_MAIR0_NS);
68     attrfield  = mair[8*index+7:8*index];
69     // Decoding MAIR0/1 field and populating memory attributes.
70     attributes = MAIRDecode(attrfield, sh);
71
72 // MVE accesses to device memory are relaxed to GRE.
73 if acctype == AccType_MVE && attributes.memtype == MemType_Device then
74     attributes.device = DeviceType_GRE;
75 if address[31:29] == '111' then // Enforce System space execute-never.
76     perms.xn = '1';
77 if !hit then // Access not allowed if there is no MPU match and use
78     // of the system address map is disabled.
79     perms.apValid = FALSE;

```

```
80     return (attributes, perms);
```

E2.1.307 NextInstrAddr

```
1 // NextInstrAddr()
2 // =====
3
4 bits(32) NextInstrAddr()
5     return GetInstrExecState(1).FetchAddr;
```

E2.1.308 NextInstrITState

```
1 // NextInstrITState()
2 // =====
3
4 ITSTATEType NextInstrITState()
5     if HaveMainExt() then
6         nextState = GetInstrExecState(1).ITState;
7     else
8         nextState = Zeros(8);
9     return nextState;
```

E2.1.309 NoninvasiveDebugAllowed

```
1 // NoninvasiveDebugAllowed()
2 // =====
3
4 boolean NoninvasiveDebugAllowed(boolean isPriv)
5     if ExternalNoninvasiveDebugEnabled() then
6         return TRUE;
7     elsif (!isPriv && UnprivHaltingDebugEnabled(FALSE)) then
8         return TRUE;
9     elsif HaltingDebugEnabled() then
10        return TRUE;
11    else
12        return FALSE;
13
14
15 boolean NoninvasiveDebugAllowed()
16     return NoninvasiveDebugAllowed(CurrentModeIsPrivileged());
```

E2.1.310 Ones

```
1 // Ones()
2 // =====
3
4 bits(N) Ones(integer N)
5     return Replicate('1', N);
6
7 bits(N) Ones()
8     return Ones(N);
```

E2.1.311 PACCellInvShuffle

```
1 // PACCellInvShuffle()
2 // =====
3
4 bits(64) PACCellInvShuffle(bits(64) inData)
5     bits(64) outData;
6     outData[3:0] = inData[15:12];
7     outData[7:4] = inData[27:24];
8     outData[11:8] = inData[51:48];
9     outData[15:12] = inData[39:36];
```

```

10     outData[19:16] = inData[59:56];
11     outData[23:20] = inData[47:44];
12     outData[27:24] = inData[7:4];
13     outData[31:28] = inData[19:16];
14     outData[35:32] = inData[35:32];
15     outData[39:36] = inData[55:52];
16     outData[43:40] = inData[31:28];
17     outData[47:44] = inData[11:8];
18     outData[51:48] = inData[23:20];
19     outData[55:52] = inData[3:0];
20     outData[59:56] = inData[43:40];
21     outData[63:60] = inData[63:60];
22     return outData;

```

E2.1.312 PACCellShuffle

```

1  // PACCellShuffle()
2  // =====
3
4  bits(64) PACCellShuffle(bits(64) inData)
5      bits(64) outData;
6      outData[3:0] = inData[55:52];
7      outData[7:4] = inData[27:24];
8      outData[11:8] = inData[47:44];
9      outData[15:12] = inData[3:0];
10     outData[19:16] = inData[31:28];
11     outData[23:20] = inData[51:48];
12     outData[27:24] = inData[7:4];
13     outData[31:28] = inData[43:40];
14     outData[35:32] = inData[35:32];
15     outData[39:36] = inData[15:12];
16     outData[43:40] = inData[59:56];
17     outData[47:44] = inData[23:20];
18     outData[51:48] = inData[11:8];
19     outData[55:52] = inData[39:36];
20     outData[59:56] = inData[19:16];
21     outData[63:60] = inData[63:60];
22     return outData;

```

E2.1.313 PACEnabled

```

1  // PACEnabled()
2  // =====
3  // Check whether PAC is enabled.
4
5  boolean PACEnabled()
6      enabled = FALSE;
7      // The PACBTI Extension must be implemented in order to use PAC functionality.
8      if HavePACBTIExt() then
9          if CurrentModeIsPrivileged() then
10             pacEn = CONTROL.PAC_EN; // Current Security state privileged PAC enabled.
11          else
12             pacEn = CONTROL.UPAC_EN; // Current Security state unprivileged PAC enabled.
13
14             if pacEn == '1' then enabled = TRUE;
15
16      return enabled;

```

E2.1.314 PACInvSub

```

1  // PACInvSub()
2  // =====
3
4  bits(64) PACInvSub(bits(64) Tinput)
5      // This is a 4-bit substitution from the PRINCE-family cipher.

```

```

6
7     bits(64) Toutput;
8     for i = 0 to 15
9         case Tinput[4*i+3:4*i] of
10             when '0000' Toutput[4*i+3:4*i] = '0101';
11             when '0001' Toutput[4*i+3:4*i] = '1110';
12             when '0010' Toutput[4*i+3:4*i] = '1101';
13             when '0011' Toutput[4*i+3:4*i] = '1000';
14             when '0100' Toutput[4*i+3:4*i] = '1010';
15             when '0101' Toutput[4*i+3:4*i] = '1011';
16             when '0110' Toutput[4*i+3:4*i] = '0001';
17             when '0111' Toutput[4*i+3:4*i] = '1001';
18             when '1000' Toutput[4*i+3:4*i] = '0010';
19             when '1001' Toutput[4*i+3:4*i] = '0110';
20             when '1010' Toutput[4*i+3:4*i] = '1111';
21             when '1011' Toutput[4*i+3:4*i] = '0000';
22             when '1100' Toutput[4*i+3:4*i] = '0100';
23             when '1101' Toutput[4*i+3:4*i] = '1100';
24             when '1110' Toutput[4*i+3:4*i] = '0111';
25             when '1111' Toutput[4*i+3:4*i] = '0011';
26     return Toutput;

```

E2.1.315 PACKey

```

1 // PACKey[]
2 // =====
3
4 // Non-assignment forms
5
6 bits(128) PACKey[boolean isSecure, boolean isPriv]
7     if isSecure then
8         key = if isPriv then PAC_KEY_P_S else PAC_KEY_U_S;
9     else
10        key = if isPriv then PAC_KEY_P_NS else PAC_KEY_U_NS;
11    return key;
12
13 bits(32) PACKeyReg[bits(3) index, boolean isSecure]
14 // Access PAC_KEY_U if index[2] is 0, otherwise PAC_KEY_P.
15 return Elem[PACKey[isSecure, index[2] == '0'], UInt(index[1:0]), 32];
16
17
18 // Assignment forms
19
20 PACKey[boolean isSecure, boolean isPriv] = bits(128) value
21     if isSecure then
22         if isPriv then
23             PAC_KEY_P_S = value;
24         else
25             PAC_KEY_U_S = value;
26     else
27         if isPriv then
28             PAC_KEY_P_NS = value;
29         else
30             PAC_KEY_U_NS = value;
31
32 PACKeyReg[bits(3) index, boolean isSecure] = bits(32) value
33 // Access PAC_KEY_U if index[2] is 0, otherwise PAC_KEY_P.
34 Elem[PACKey[isSecure, index[2] == '0'], UInt(index[1:0]), 32] = value;

```

E2.1.316 PACKeys

```

1 // Pointer authentication keys
2
3 bits(128) PAC_KEY_P_S;
4 bits(128) PAC_KEY_P_NS;
5 bits(128) PAC_KEY_U_S;
6 bits(128) PAC_KEY_U_NS;

```

E2.1.317 PACMult

```
1 // PACMult ()
2 // =====
3
4 bits(64) PACMult(bits(64) Sinput)
5     bits(4)  t0;
6     bits(4)  t1;
7     bits(4)  t2;
8     bits(4)  t3;
9     bits(64) Soutput;
10
11     for i = 0 to 3
12         t0[3:0] = (RotCell(Sinput[4*(i+8)+3:4*(i+8)], 1) EOR
13                     RotCell(Sinput[4*(i+4)+3:4*(i+4)], 2));
14         t0[3:0] = t0[3:0] EOR RotCell(Sinput[4*(i)+3:4*(i)], 1);
15         t1[3:0] = (RotCell(Sinput[4*(i+12)+3:4*(i+12)], 1) EOR
16                     RotCell(Sinput[4*(i+4)+3:4*(i+4)], 1));
17         t1[3:0] = t1[3:0] EOR RotCell(Sinput[4*(i)+3:4*(i)], 2);
18         t2[3:0] = (RotCell(Sinput[4*(i+12)+3:4*(i+12)], 2) EOR
19                     RotCell(Sinput[4*(i+8)+3:4*(i+8)], 1));
20         t2[3:0] = t2[3:0] EOR RotCell(Sinput[4*(i)+3:4*(i)], 1);
21         t3[3:0] = (RotCell(Sinput[4*(i+12)+3:4*(i+12)], 1) EOR
22                     RotCell(Sinput[4*(i+8)+3:4*(i+8)], 2));
23         t3[3:0] = t3[3:0] EOR RotCell(Sinput[4*(i+4)+3:4*(i+4)], 1);
24         Soutput[4*i+3:4*i] = t3[3:0];
25         Soutput[4*(i+4)+3:4*(i+4)] = t2[3:0];
26         Soutput[4*(i+8)+3:4*(i+8)] = t1[3:0];
27         Soutput[4*(i+12)+3:4*(i+12)] = t0[3:0];
28     return Soutput;
```

E2.1.318 PACSubQ3

```
1 // PACSubQ3 ()
2 // =====
3
4 bits(64) PACSubQ3(bits(64) Tinput)
5     // This is a 4-bit substitution from the PRINCE-family cipher.
6     bits(64) Toutput;
7     for i = 0 to 15
8         case Tinput[4*i+3:4*i] of
9             when '0000' Toutput[4*i+3:4*i] = '1010';
10            when '0001' Toutput[4*i+3:4*i] = '1101';
11            when '0010' Toutput[4*i+3:4*i] = '1110';
12            when '0011' Toutput[4*i+3:4*i] = '0110';
13            when '0100' Toutput[4*i+3:4*i] = '1111';
14            when '0101' Toutput[4*i+3:4*i] = '0111';
15            when '0110' Toutput[4*i+3:4*i] = '0011';
16            when '0111' Toutput[4*i+3:4*i] = '0101';
17            when '1000' Toutput[4*i+3:4*i] = '1001';
18            when '1001' Toutput[4*i+3:4*i] = '1000';
19            when '1010' Toutput[4*i+3:4*i] = '0000';
20            when '1011' Toutput[4*i+3:4*i] = '1100';
21            when '1100' Toutput[4*i+3:4*i] = '1011';
22            when '1101' Toutput[4*i+3:4*i] = '0001';
23            when '1110' Toutput[4*i+3:4*i] = '0010';
24            when '1111' Toutput[4*i+3:4*i] = '0100';
25     return Toutput;
```

E2.1.319 PACSubQ5

```
1 // PACSubQ5 ()
2 // =====
3
4 bits(64) PACSubQ5(bits(64) Tinput)
5     // This is a 4-bit substitution from the PRINCE-family cipher.
```



```

6      bits(64) Toutput;
7      for i = 0 to 15
8          case Tinput[4*i+3:4*i] of
9              when '0000' Toutput[4*i+3:4*i] = '1011';
10             when '0001' Toutput[4*i+3:4*i] = '0110';
11             when '0010' Toutput[4*i+3:4*i] = '1000';
12             when '0011' Toutput[4*i+3:4*i] = '1111';
13             when '0100' Toutput[4*i+3:4*i] = '1100';
14             when '0101' Toutput[4*i+3:4*i] = '0000';
15             when '0110' Toutput[4*i+3:4*i] = '1001';
16             when '0111' Toutput[4*i+3:4*i] = '1110';
17             when '1000' Toutput[4*i+3:4*i] = '0011';
18             when '1001' Toutput[4*i+3:4*i] = '0111';
19             when '1010' Toutput[4*i+3:4*i] = '0100';
20             when '1011' Toutput[4*i+3:4*i] = '0101';
21             when '1100' Toutput[4*i+3:4*i] = '1101';
22             when '1101' Toutput[4*i+3:4*i] = '0010';
23             when '1110' Toutput[4*i+3:4*i] = '0001';
24             when '1111' Toutput[4*i+3:4*i] = '1010';
25      return Toutput;

```

E2.1.320 PC

```

1  // PC - non-assignment form
2  // =====
3  bits(32) PC
4  return RName[RNamesPC];

```

E2.1.321 PEmode

```

1  // The PE execution modes.
2
3  enumeration PEmode {PEmode_Thread, PEmode_Handler};

```

E2.1.322 PendingDebugHalt

```

1  // PendingDebugHalt()
2  // =====
3
4  boolean PendingDebugHalt()
5  return CanHaltOnEvent(IsSecure(), FindPriv()) && DHCSR.C_HALT == '1';

```

E2.1.323 PendingDebugMonitor

```

1  // PendingDebugMonitor()
2  // =====
3
4  boolean PendingDebugMonitor()
5  // If the current execution priority is below DebugMonitor and generating a DebugMonitor
6  // exception is allowed, and MON_PEND is set, then return TRUE. Otherwise return FALSE.
7  if DEMCR.MON_PEND == '0' then
8      return FALSE;
9  elseif !CanPendMonitorOnEvent(IsSecure(), TRUE, FALSE, FindPriv(), FALSE) then
10     return FALSE;
11  else
12     return TRUE;

```

E2.1.324 PendingExceptionDetails

```

1  // PendingExceptionDetails
2  // =====
3  // Determines whether to take a pending exception or not. This is done based
4  // on current execution priority and the priority of pending exceptions that

```

```

5 // are not masked by DHCSR.C_MASKINTS.
6 // Returns whether any pending exception is to be taken, and, if so, the
7 // exception number for the highest priority unmasked exception, and
8 // whether this exception is Secure.
9
10 (boolean, integer, boolean) PendingExceptionDetails();

```

E2.1.325 PendReturnOperation

```

1 // PendReturnOperation()
2 // =====
3
4 PendReturnOperation(bits(32) returnValue)
5     _NextInstrAddr      = returnValue;
6     _PCChanged          = TRUE;
7     _PendingReturnOperation = TRUE;
8     return;

```

E2.1.326 Permissions

```

1 // Access permissions descriptor
2
3 type Permissions is (
4     boolean apValid,      // TRUE when ap is valid, else FALSE.
5     bits(2) ap,          // Access Permission bits, if valid.
6     bit xn,              // Execute Never bit.
7     boolean regionValid, // TRUE if the region number is valid, else FALSE.
8     bits(8) region       // The MPU region number, if valid.
9 )

```

E2.1.327 PMU_CounterIncrement

```

1 // PMU_CounterIncrement()
2 // =====
3 // Increments PMU counters associated with the specified event as needed.
4
5 constant integer CYCLE_COUNTER_ID = 31;
6
7 PMU_CounterIncrement(PmuEventType eventId, integer counterId)
8     // If the counter is disabled it does not need incrementing. Early-exit.
9     if PMU_CTRL.E == '0' || DEMCR.TRCENA == '0' then
10         return;
11
12     // Frozen, event counters do not increment. The cycle counter continues to increment.
13     if IsDebugState() ||
14         (counterId != 31 && !IsZero(PMU_OVSSET.Pn) && PMU_CTRL.FZO == '1') then
15         return;
16
17     pmuAllowedState = (((!IsSecure() && NoninvasiveDebugAllowed(FindPriv())) ||
18         SecureNoninvasiveDebugAllowed(FindPriv())) ||
19         eventId == PmuEventType_SW_INCR);
20
21     // If the PMU_CTRL.DP bit is not set, the dedicated cycle counter is
22     // enabled in Secure state.
23     if (counterId == CYCLE_COUNTER_ID && eventId == PmuEventType_CPU_CYCLES) then
24         if (!IsSecure() || PMU_CTRL.DP == '0') then
25             pmuAllowedState = TRUE;
26         else
27             pmuAllowedState = FALSE;
28
29     // Prohibited, counters do not increment
30     if !pmuAllowedState then
31         return;
32
33     // The cycle counter will increment whenever it is enabled and there is a

```

```

34 // CPU_CYCLE event.
35 if counterId == CYCLE_COUNTER_ID then
36     if PMU_CNTESET.C == '1' && eventId == PmuEventType_CPU_CYCLES then
37         newValue = UInt(PMU_CCNTR) + 1;
38         PMU_CCNTR = newValue[31:0];
39         - = PMU_HandleOverflow(counterId, newValue, 32);
40
41 // Other counters will increment if they are enabled,
42 // are configured to respond to that event.
43 elseif (PMU_CNTESET.Pn[counterId] == '1' &&
44         PMU_EVTYPER[counterId] == PmuEvent(eventId)) then
45     newValue = UInt(PMU_EVCNTR[counterId]) + 1;
46     PMU_EVCNTR[counterId].Counter = newValue[15:0];
47     if PMU_HandleOverflow(counterId, newValue, 16) then
48         // If this is an EVEN counter, look at possible chaining.
49         if counterId[0] == '0' &&
50            PMU_CNTESET.Pn[counterId + 1] == '1' &&
51            PMU_EVTYPER[counterId + 1] == PmuEvent(PmuEventType_CHAIN) then
52             // Configured as chaining counter, increment
53             newValueChain = UInt(PMU_EVCNTR[counterId + 1]) + 1;
54             PMU_EVCNTR[counterId + 1].Counter = newValueChain[15:0];
55             - = PMU_HandleOverflow(counterId + 1, newValueChain, 16);
56
57 PMU_CounterIncrement(PmuEvent eventId)
58 // If all counters are globally disabled,
59 // they do not need incrementing. Early-exit.
60 if PMU_CTRL.E == '0' || DEMCR.TRCENA == '0' then
61     return;
62 for i = 0 to UInt(PMU_TYPE.N) - 1
63     PMU_CounterIncrement(eventId, i);
64 PMU_CounterIncrement(eventId, CYCLE_COUNTER_ID);

```

E2.1.328 PMU_HandleOverflow

```

1 // PMU_HandleOverflow()
2 // =====
3 // Handles the overflow of a specified counter.
4
5 boolean PMU_HandleOverflow(integer counterId, integer newValue, integer overflowBit)
6 // Handle trace-on-overflow if the lower 8-bits of any of the first 8 counters
7 // overflows. This only occurs if trace-on-overflow is enabled.
8 // If multiple trace packets are waiting to be issued, their contents can be
9 // merged into a single packet.
10 if counterId < 8 && PMU_CTRL.TRO == '1' && newValue[8] != (newValue - 1)[8] then
11     PMU_EmitTrace(counterId);
12
13 // Has the counter actually overflowed?
14 if newValue[overflowBit] == '1' then
15     PMU_OVSET[counterId] = '1';
16     // If enabled, generate a debug event with 'PMU' syndrome.
17     if PMU_INTENSET[counterId] == '1' then
18         isSecure = FALSE;
19         isPriv = CurrentModeIsPrivileged(isSecure);
20         if DHCSR.C_PMOV == '1' && CanHaltOnEvent(isSecure, isPriv) then
21             DHCSR.C_HALT = '1';
22             DFSR.PMU = '1';
23         elseif CanPendMonitorOnEvent(isSecure, FALSE, TRUE, isPriv, TRUE) then
24             DEMCR.MON_PEND = '1';
25             DFSR.PMU = '1';
26     return TRUE;
27 return FALSE;

```

E2.1.329 PolynomialMult

```

1 // PolynomialMult()
2 // =====
3

```

```

4  bits(M+N) PolynomialMult(bits(M) op1, bits(N) op2)
5      result = Zeros(M+N);
6      extended_op2 = Zeros(M) : op2;
7      for i=0 to M-1
8          if op1[i] == '1' then
9              result = result EOR LSL(extended_op2, i);
10     return result;

```

E2.1.330 PopStack

```

1  // PopStack()
2  // =====
3
4  ExcInfo PopStack(EXC_RETURN_Type excReturn)
5      constant integer intCallerFrameSize = 0x20;
6      constant integer intCalleeFrameSize = 0x28;
7      constant integer fpCallerFrameSize = 0x48;
8
9      // NOTE: All stack accesses are performed as Unprivileged accesses if
10     // returning to Thread mode and CONTROL.nPRIV is 1 for the destination
11     // Security state.
12     mode = if excReturn.Mode == '1' then PMode_Thread else PMode_Handler;
13     toSecure = HaveSecurityExt() && excReturn.S == '1';
14     spName = LookUpSP_with_security_mode(toSecure, mode);
15     frameptr = _SP(spName);
16     if !IsAligned(frameptr, 8) then UNPREDICTABLE;
17
18     // only stack locations, not the load order, are architected
19
20     // Pop the Additional state context registers, when returning from a Non-secure
21     // exception or a Secure exception that followed a Non-secure exception and
22     // therefore still has the Additional state context register state on the stack.
23     exc = DefaultExcInfo();
24     if toSecure && (excReturn.ES == '0' ||
25         excReturn.DCRS == '0') then
26         // Check the integrity signature, and if so is it correct.
27         expectedSig = 0xFEFA125B[31:0];
28         if HaveMveOrFPEExt() then
29             expectedSig[0] = excReturn.FType;
30             (exc, integritySig) = Stack(frameptr, 0x0, spName, mode);
31         if exc.fault == NoFault && integritySig != expectedSig then
32             if HaveMainExt() then
33                 SFSR.INVIS = '1';
34             // Create the exception. NOTE: If Main Extension is not implemented the fault
35             // always escalates to a HardFault.
36             return CreateException(SecureFault);
37
38         if exc.fault == NoFault then (exc, R[4]) = Stack(frameptr, 0x8, spName, mode);
39         if exc.fault == NoFault then (exc, R[5]) = Stack(frameptr, 0xC, spName, mode);
40         if exc.fault == NoFault then (exc, R[6]) = Stack(frameptr, 0x10, spName, mode);
41         if exc.fault == NoFault then (exc, R[7]) = Stack(frameptr, 0x14, spName, mode);
42         if exc.fault == NoFault then (exc, R[8]) = Stack(frameptr, 0x18, spName, mode);
43         if exc.fault == NoFault then (exc, R[9]) = Stack(frameptr, 0x1C, spName, mode);
44         if exc.fault == NoFault then (exc, R[10]) = Stack(frameptr, 0x20, spName, mode);
45         if exc.fault == NoFault then (exc, R[11]) = Stack(frameptr, 0x24, spName, mode);
46         frameptr = frameptr + intCalleeFrameSize;
47
48     // Unstack the State context saved registers, possibly including
49     // the floating-point registers.
50     RETPSR_Type psr;
51     if exc.fault == NoFault then (exc, R[0]) = Stack(frameptr, 0x0, spName, mode);
52     if exc.fault == NoFault then (exc, R[1]) = Stack(frameptr, 0x4, spName, mode);
53     if exc.fault == NoFault then (exc, R[2]) = Stack(frameptr, 0x8, spName, mode);
54     if exc.fault == NoFault then (exc, R[3]) = Stack(frameptr, 0xC, spName, mode);
55     if exc.fault == NoFault then (exc, R[12]) = Stack(frameptr, 0x10, spName, mode);
56     if exc.fault == NoFault then (exc, LR) = Stack(frameptr, 0x14, spName, mode);
57     if exc.fault == NoFault then (exc, pc) = Stack(frameptr, 0x18, spName, mode);
58     if exc.fault == NoFault then (exc, psr) = Stack(frameptr, 0x1C, spName, mode);

```

```

59     frameOffset = intCallerFrameSize;
60     BranchTo(pc, TRUE, psr.B == '1');
61
62     // Check the XPSR value that has been unstacked is consistent with the mode
63     // being returned to.
64     excNum = UInt(psr.Exception);
65     if (exc.fault == NoFault) &&
66         ((mode == PMode_Handler) == (excNum == 0)) then
67         if HaveMainExt() then
68             UFSR.INVPC = '1';
69         // Create the exception. NOTE: If Main Extension is not implemented the fault
70         // always escalates to a HardFault.
71         return CreateException(UsageFault);
72
73     // The IPSR value is set as UNKNOWN if the unstacked IPSR value is not
74     // supported by the PE.
75     validIPSR = excNum IN {0, 1, NMI, HardFault, SVCall, PendSV, SysTick};
76     if !validIPSR && HaveMainExt() then
77         validIPSR = excNum IN {MemManage, BusFault, UsageFault, SecureFault, DebugMonitor};
78
79     // Check also whether excNum is an external interrupt supported by PE.
80     if !validIPSR && !IsIrqValid(excNum) then
81         psr.Exception = bits(9) UNKNOWN;
82
83     if HaveMveOrFPExt() then
84         if excReturn.FType == '0' then
85             // Raise a fault and skip Floating-point operations if requested to expose
86             // Secure Floating-point state to the Non-secure code.
87             if !toSecure && FPCCR_S.LSPACT == '1' then
88                 SFSR.LSERR = '1';
89                 newExc = CreateException(SecureFault);
90                 // It is IMPLEMENTATION DEFINED whether a MemManage fault is dropped if
91                 // a SecureFault is generated subsequently. If the MemManage fault is
92                 // not dropped the exceptions will be taken based on exception
93                 // priority as described in MergeExcInfo().
94                 if boolean IMPLEMENTATION_DEFINED "Drop previously generated exceptions" then
95                     exc = newExc;
96                 else
97                     exc = MergeExcInfo(exc, newExc);
98             else
99                 lspact = if toSecure then FPCCR_S.LSPACT else FPCCR_NS.LSPACT;
100                 if lspact == '1' then // State in floating-point is still valid.
101                     if exc.fault == NoFault then
102                         if toSecure then
103                             FPCCR_S.LSPACT = '0';
104                         else
105                             FPCCR_NS.LSPACT = '0';
106                         // Tail-chaining during an exception return after this point where
107                         // LSPACT is cleared is not permissible.
108                     else
109                         if exc.fault == NoFault then
110                             nPriv = if toSecure then CONTROL_S.nPRIV else CONTROL_NS.nPRIV;
111                             isPriv = mode == PMode_Handler || nPriv == '0';
112                             exc = CheckCPEnabled(10, isPriv, toSecure);
113
114                     // If an implementation abandons the unstacking of the Floating-point
115                     // Extension registers to tail-chain into a fault or late arriving
116                     // interrupt it must clear any Floating-point registers that
117                     // would have been unstacked.
118                     // NOTE: The requirement to clear the registers only applies
119                     // to implementations that include the Security Extension.
120                     // The Floating-point Extension registers that would have been
121                     // unstacked become UNKNOWN in implementations that do not include the
122                     // Security Extension.
123                     if exc.fault == NoFault then
124                         for i = 0 to 15
125                             if exc.fault == NoFault then
126                                 (exc, S[i]) = Stack(framePtr, frameOffset + (4*i),
127                                                         spName, mode);

```

```

128         if exc.fault == NoFault then
129             (exc, FPSCR) = Stack(frameptr, frameOffset + 0x40,
130                                 spName, mode);
131         if exc.fault == NoFault then
132             if HaveMve() then
133                 (exc, VPR) = Stack(frameptr, frameOffset + 0x44,
134                                   spName, mode);
135             elsif boolean IMPLEMENTATION_DEFINED "No MVE VPR mem access" then
136                 (exc, -) = Stack(frameptr, frameOffset + 0x44,
137                                 spName, mode);
138             frameOffset = frameOffset + fpCallerFrameSize;
139
140             pushFPCalleeRegs = toSecure && FPCCR_S.TS == '1';
141             if pushFPCalleeRegs then
142                 for i = 0 to 15
143                     if exc.fault == NoFault then
144                         (exc, S[i+16]) = Stack(frameptr, frameOffset + (4*i),
145                                                spName, mode);
146
147             if exc.fault != NoFault then
148                 InvalidateFPRegs(HaveSecurityExt(), pushFPCalleeRegs);
149
150             CONTROL.FPCA = NOT(excReturn.FType);
151
152             // If there was not a fault then move the stack pointer to consume the
153             // exception stack frame. NOTE: If a exception return fault occurs and
154             // results in a lockup the stack pointer is updated. This special case is
155             // handled at the point lockup is entered and not here.
156             if exc.fault == NoFault then
157                 ConsumeExcStackFrame(excReturn, psr.SPREALIGN);
158
159             if HaveDSPEExt() then
160                 APSR.GE = psr.GE;
161             if IsSecure() then
162                 CONTROL_S.SFPA = psr.SFPA;
163                 IPSR.Exception = psr.Exception;           // Load valid IPSR bits from memory.
164                 EPSR.T = psr.T;                           // Load valid EPSR bits from memory.
165             if HaveMainExt() then
166                 APSR[31:27] = psr[31:27];                 // Load valid APSR bits from memory.
167                 SetITSTATEAndCommit(psr.IT);             // Load valid ITSTATE from memory
168                                                         // (also handles ICI and ECI).
169             else
170                 APSR[31:28] = psr[31:28];                 // Load valid APSR bits from memory.
171             return exc;

```

E2.1.331 PreserveFPState

```

1 // PreserveFPState()
2 // =====
3
4 PreserveFPState()
5     // Check if there is any lazy floating-point state to be preserved.
6     isSecure = FPCCR_S.S == '1';
7     lspact = if isSecure then FPCCR_S.LSPACT else FPCCR_NS.LSPACT;
8
9     if lspact == '1' then
10         // Preserve floating-point state using address, privilege
11         // and relative priorities recorded during original stacking.
12         // Derived exceptions are handled by TakePreserveFPEXception().
13
14         // The checks usually performed for stacking using ValidateAddress()
15         // are performed, with the value of ExecutionPriority()
16         // overridden by -1 if FPCCR.HFRDY == '0'.
17
18         if isSecure then
19             ispriv = FPCCR_S.USER == '0';
20             splimviol = FPCCR_S.SPLIMVIOL == '1';
21             fpcar = FPCAR_S;

```

```

22     else
23         ispriv      = FPCCR_NS.USER      == '0';
24         splimviol   = FPCCR_NS.SPLIMVIOL == '1';
25         fpcar       = FPCAR_NS;
26         Privilege memPriv = FindMemPriv(ispriv);
27
28         // Check if the background context had access to the FPU.
29         excInfo = CheckCPEnabled(10, ispriv, isSecure);
30
31         // Only perform the memory accesses if the stack limit has not been violated
32         bfExcInfo = DefaultExcInfo();
33         if !splimviol && excInfo.fault == NoFault then
34             // RAS or BusFault errors raised before lazy floating-point stacking are
35             // barriered if IESB is enabled. Errors that are Synchronized at this point
36             // belong to the current context (the context that executed the instruction
37             // that triggered the lazystacking), are handled normally and not by
38             // TakePreserveFPEException.
39             if AIRCR.IESB == '1' then
40                 HandleException(SynchronizeBusFault());
41
42             // Whether these stores are interruptible is IMPLEMENTATION DEFINED.
43             // Only the stack locations, not the store order, are architected.
44             for i = 0 to 15
45                 if excInfo.fault == NoFault then
46                     addr      = fpcar + (4*i);
47                     excInfo = MemA_with_priv_security(addr, 4, AccType_LAZYFP, memPriv,
48                                                         isSecure, TRUE, S[i]);
49
50             if excInfo.fault == NoFault then
51                 addr      = fpcar + 0x40;
52                 excInfo = MemA_with_priv_security(addr, 4, AccType_LAZYFP, memPriv,
53                                                         isSecure, TRUE, FPSCR);
54             addr = fpcar + 0x44;
55             if excInfo.fault == NoFault && HaveMve() then
56                 excInfo = MemA_with_priv_security(addr, 4, AccType_LAZYFP, memPriv,
57                                                         isSecure, TRUE, VPR);
58             elseif boolean IMPLEMENTATION_DEFINED "No MVE VPR mem access" then
59                 excInfo = MemA_with_priv_security(addr, 4, AccType_LAZYFP, memPriv,
60                                                         isSecure, TRUE, bits(32) UNKNOWN);
61
62             if isSecure && FPCCR_S.TS == '1' then
63                 for i = 0 to 15
64                     if excInfo.fault == NoFault then
65                         addr      = fpcar + (4*i) + 0x48;
66                         excInfo = MemA_with_priv_security(addr, 4, AccType_LAZYFP, memPriv,
67                                                         TRUE, TRUE, S[i+16]);
68
69             // If IESB is enabled, barrier RAS or BusFault errors raised during lazy
70             // floating-point stacking to the original context. If errors do occur,
71             // BFSR.LSPERR is set.
72             if AIRCR.IESB == '1' then
73                 bfExcInfo = SynchronizeBusFault(AccType_LAZYFP);
74
75             // Handle any faults that have occurred.
76             termInst = FALSE;
77             if excInfo.fault != NoFault then
78                 termInst = termInst || TakePreserveFPEException(excInfo);
79             if bfExcInfo.fault != NoFault then
80                 termInst = termInst || TakePreserveFPEException(bfExcInfo);
81
82             // If exception with sufficient priority to pre-empt current instruction execution is
83             // raised during floating-point state preserve, then termInst will be true and
84             // execution of the current instruction should be terminated by calling
85             // EndOfInstruction(). If the exception results in a lockup state, termInst
86             // will also be true.
87             if termInst then
88                 EndOfInstruction();
89             else
90                 // If the floating-point state is being treated as Secure then the registers

```

```

91         // are zeroed.
92         InvalidateFPRegs(isSecure && FPCCR_S.TS == '1', isSecure && FPCCR_S.TS == '1');
93
94     // In case of NoFault or, on successful return from TakePreserveFPEException(),
95     // the current instruction execution continues and FPCCR.LSPACT will be cleared.
96     // If the stores are interrupted, the register content and LSPACT remain unchanged.
97     if (HasArchVersion(Armv8p1) &&
98         boolean IMPLEMENTATION_DEFINED "Clear both LSPACTs" then
99         FPCCR_S.LSPACT = '0';
100        FPCCR_NS.LSPACT = '0';
101    else
102        if isSecure then
103            FPCCR_S.LSPACT = '0';
104        else
105            FPCCR_NS.LSPACT = '0';

```

E2.1.332 Privilege

```

1 // Memory privilege type
2
3 enumeration Privilege {
4     Privilege_Priv,           // Privileged memory request.
5     Privilege_ForcedUnpriv,   // Deprivileged memory request.
6     Privilege_Unpriv         // Unprivileged memory request.
7 };

```

E2.1.333 ProcessorID

```

1 // ProcessorID
2 // =====
3 // Returns an integer that uniquely identifies the executing PE in the system.
4
5 integer ProcessorID();

```

E2.1.334 PushCalleeStack

```

1 // PushCalleeStack()
2 // =====
3
4 ExcInfo PushCalleeStack(boolean doTailChain, EXC_RETURN_Type excReturn)
5     // Allocate space of the correct stack. NOTE: If the PE is tail-chaining the PE should
6     // check excReturn instead of CONTROL.SPSEL to determine which stack to use,
7     // as SPSEL can report the wrong stack in tail-chaining cases.
8     if doTailChain then
9         if excReturn.Mode == '0' then
10             mode = PMode_Handler;
11             spName = RNamesSP_Main_Secure;
12         else
13             mode = PMode_Thread;
14             if excReturn.SPSEL == '1' then
15                 spName = RNamesSP_Process_Secure;
16             else
17                 spName = RNamesSP_Main_Secure;
18         else
19             spName = LookUpSP();
20             mode = CurrentMode();
21
22     // Calculate the address of the base of the Additional state context frame.
23     bits(32) frameptr = _SP(spName) - 0x28;
24
25     /* only the stack locations, not the store order, are architected */
26     // Write out integrity signature.
27     if HaveMveOrFPExt() then
28         integritySig = 0xFEFA125A[31:1] : excReturn.FType;
29     else

```



```

30     integritySig = 0xFEFA125B[31:0];
31     exc = Stack(frameptr, 0x0, spName, mode, integritySig);
32     // Stack Additional state context registers
33     if exc.fault == NoFault then exc = Stack(frameptr, 0x8, spName, mode, R[4]);
34     if exc.fault == NoFault then exc = Stack(frameptr, 0xC, spName, mode, R[5]);
35     if exc.fault == NoFault then exc = Stack(frameptr, 0x10, spName, mode, R[6]);
36     if exc.fault == NoFault then exc = Stack(frameptr, 0x14, spName, mode, R[7]);
37     if exc.fault == NoFault then exc = Stack(frameptr, 0x18, spName, mode, R[8]);
38     if exc.fault == NoFault then exc = Stack(frameptr, 0x1C, spName, mode, R[9]);
39     if exc.fault == NoFault then exc = Stack(frameptr, 0x20, spName, mode, R[10]);
40     if exc.fault == NoFault then exc = Stack(frameptr, 0x24, spName, mode, R[11]);
41
42     // Update the stack pointer.
43     spExc = _SP(spName, TRUE, FALSE, frameptr);
44     return MergeExcInfo(exc, spExc);

```

E2.1.335 PushStack

```

1  // PushStack()
2  // =====
3
4  (ExcInfo, EXC_RETURN_Type) PushStack()
5      constant integer intFrameSize      = 0x20;
6      constant integer fpCallerFrameSize = 0x48;
7      constant integer fpCalleeFrameSize = 0x40;
8
9      boolean pushFPCallerFrame = HaveMveOrFPEExt() && CONTROL.FPCA == '1';
10     boolean pushFPCalleeFrame = pushFPCallerFrame && IsSecure() && FPCCR_S.TS == '1';
11
12     integer framesize = intFrameSize;
13     // In the case where a NOCP usage fault is generated, floating-point stack space
14     // is not allocated.
15     if IsSecure() || NSACR.CP10 == '1' then
16         if pushFPCallerFrame then framesize = framesize + fpCallerFrameSize;
17         if pushFPCalleeFrame then framesize = framesize + fpCalleeFrameSize;
18
19     /* allocate space on the correct stack */
20     bits(1) frameptralign;
21     frameptralign = SP[2];
22     frameptr      = (SP - framesize) AND NOT(ZeroExtend('100', 32));
23     spName        = LookUpSP();
24
25     // Prepare architecture state for stacking
26     RETPSR_Type retpsr = XPSR[31:0];
27     retpsr.SPREALIGN   = frameptralign;
28     retpsr.SFPA        = if IsSecure() then CONTROL_S.SFPA else '0';
29     mode               = CurrentMode();
30
31     /* only the stack locations, not the store order, are architected */
32     exc                = Stack(frameptr, 0x0, spName, mode, R[0]);
33     if exc.fault == NoFault then exc = Stack(frameptr, 0x4, spName, mode, R[1]);
34     if exc.fault == NoFault then exc = Stack(frameptr, 0x8, spName, mode, R[2]);
35     if exc.fault == NoFault then exc = Stack(frameptr, 0xC, spName, mode, R[3]);
36     if exc.fault == NoFault then exc = Stack(frameptr, 0x10, spName, mode, R[12]);
37     if exc.fault == NoFault then exc = Stack(frameptr, 0x14, spName, mode, LR);
38     // Push the address of the next instruction, this is the raw PC value, without
39     // the +4 that is observed when reading PC from within instructions.
40     if exc.fault == NoFault then exc = Stack(frameptr, 0x18, spName, mode, _RName[RNamesPC]);
41     if exc.fault == NoFault then exc = Stack(frameptr, 0x1C, spName, mode, retpsr);
42     frameOffset = intFrameSize;
43
44     if pushFPCallerFrame then
45         newExc = DefaultExcInfo();
46         // LSPACT should not be active at the same time as CONTROL.FPCA. This
47         // is a possible attack scenario so raise a SecureFault.
48         lspact = if FPCCR_S.S == '1' then FPCCR_S.LSPACT else FPCCR_NS.LSPACT;
49         if HaveSecurityExt() && lspact == '1' then
50             SFSR.LSERR = '1';

```

```

51     newExc      = CreateException(SecureFault);
52     elsif !IsSecure() && NSACR.CP10 == '0' then
53         UFSR_S.NOCP = '1';
54         newExc      = CreateException(UsageFault, TRUE, TRUE);
55     elsif FPCCR.LSPEN == '0' then
56         if exc.fault == NoFault then
57             exc = CheckCPEnabled(10);
58
59         if exc.fault == NoFault then
60             for i = 0 to 15
61                 if exc.fault == NoFault then
62                     exc = Stack(frameptr, frameOffset + (4*i), spName, mode, S[i]);
63                 if exc.fault == NoFault then
64                     exc = Stack(frameptr, frameOffset + 0x40, spName, mode, FPSCR);
65                 if exc.fault == NoFault then
66                     if HaveMve() then
67                         exc = Stack(frameptr, frameOffset + 0x44, spName, mode, VPR);
68                     elsif boolean IMPLEMENTATION_DEFINED "No MVE VPR mem access" then
69                         exc = Stack(frameptr, frameOffset + 0x44, spName, mode,
70                                 bits(32) UNKNOWN);
71                 frameOffset = frameOffset + fpCallerFrameSize;
72
73             if pushFPCalleeFrame then
74                 for i = 0 to 15
75                     if exc.fault == NoFault then
76                         exc = Stack(frameptr, frameOffset+(4*i), spName, mode, S[i+16]);
77
78             (cpEnabled, -) = IsCPEnabled(10);
79             if cpEnabled then
80                 InvalidateFPRegs(pushFPCalleeFrame, pushFPCalleeFrame);
81         else
82             UpdateFPCCR(frameptr + frameOffset, TRUE);
83
84     if newExc.fault != NoFault then
85         // It is IMPLEMENTATION_DEFINED whether to drop the earlier MemFault
86         // if the Secure fault or NOCP fault is also generated subsequently.
87         // If MemFault is not dropped, it will be merged with the Secure or NOCP fault
88         // based on exception priority as per MergeExcInfo().
89         if boolean IMPLEMENTATION_DEFINED "Drop previously generated exceptions" then
90             exc = newExc;
91         else
92             exc = MergeExcInfo(exc, newExc);
93
94     // Set the stack pointer to be at the bottom of the new stack frame.
95     spExc = _SP(spName, TRUE, FALSE, frameptr);
96     exc = MergeExcInfo(exc, spExc);
97
98     // Some excReturn bits (for example ES, SPSEL) are set by ExceptionTaken.
99     EXC_RETURN_Type partialExcReturn = 0xFFFFFFFF8[31:0];
100     partialExcReturn.S      = if IsSecure()          then '1'          else '0';
101     partialExcReturn.FType = if HaveMveOrFPExt()      then NOT(CONTROL.FPCA) else '1';
102     partialExcReturn.Mode  = if mode == PEMode_Thread then '1'          else '0';
103     return (exc, partialExcReturn);

```

E2.1.336 Q

```

1 // Q[] - non-assignment forms
2 // =====
3
4 bits(32) Q[integer idx, integer beat]
5     assert idx >= 0 && idx <= 7;
6     assert beat >= 0 && beat <= 3;
7     return S[(idx * 4) + beat];
8
9
10 // Q[] - assignment forms
11 // =====
12

```

```

13 Q[integer idx, integer beat] = bits(32) value
14     assert idx >= 0 && idx <= 7;
15     assert beat >= 0 && beat <= 3;
16     S[(idx * 4) + beat] = value;

```

E2.1.337 R

```

1 // R[]
2 // ===
3
4 // Non-assignment form
5
6 bits(32) R[integer n]
7     return RName[LookupRName(n)];
8
9 // Assignment form
10
11 R[integer n] = bits(32) value
12     assert n != 15;
13     RName[LookupRName(n)] = value;
14     return;

```

E2.1.338 RaiseAsyncBusFault

```

1 // RaiseAsyncBusFault()
2 // =====
3
4 RaiseAsyncBusFault()
5     if HaveMainExt() then
6         BFSR.IMPRESISERR = '1';
7
8         // On asynchronous bus faults, implementations might choose to populate RFSR with
9         // corresponding syndrome, and are required to do so for RAS faults. It is recommended
10        // these are marked as contained where reasonable, and only marked as uncontained when
11        // no guarantees can be made with respect to IESB containment, for example in the case
12        // of a latent error in dirty cache lines that is only uncovered on eviction.
13        if HasArchVersion(Armv8p1) && boolean IMPLEMENTATION_DEFINED "RFSR populated" then
14            RFSR.V = '1';
15            RFSR.IS = RasImpDefValue(15);
16            RFSR.UET = RasImpDefValue(2);
17
18        // To ensure errors are containable asynchronous BusFaults escalate as if they were
19        // synchronous if implicit error synchronization barriers are enabled.
20        handleSynchronously = AIRCR.IESB == '1';
21        excInfo = CreateException(BusFault, FALSE, IsSecure(), handleSynchronously);
22        HandleException(excInfo);

```

E2.1.339 RasImpDefValue

```

1 // RasImpDefValue()
2 // =====
3 // IMPLEMENTATION_DEFINED value functions used to populate RAS-related syndrome.
4
5 bits(size) RasImpDefValue(integer size);

```

E2.1.340 RawExecutionPriority

```

1 // RawExecutionPriority()
2 // =====
3 // Determine the current execution priority without the effect of priority boosting.
4
5 integer RawExecutionPriority()
6     execPri = HighestPri();
7     for i = 2 to MaxExceptionNum() // IPSR values of the exception handlers.

```

```

8         for j = 0 to 1           // Check both Non-secure and Secure exceptions.
9             secure = (j == 0);
10            if IsActiveForState(i, secure) then
11                // PRIGROUP effect applied in ExceptionPriority.
12                effectivePriority = ExceptionPriority(i, secure, TRUE);
13                if effectivePriority < execPri then
14                    execPri = effectivePriority;
15            assert execPri IN {-4 .. 256};
16            return execPri;

```

E2.1.341 Replicate

```

1 // Replicate()
2 // =====
3
4 bits(M*N) Replicate(bits(M) x, integer N);
5
6 bits(N) Replicate(bits(M) x)
7     assert N MOD M == 0;
8     return Replicate(x, N DIV M);

```

E2.1.342 ResetRegs

```

1 // ResetRegs
2 // =====
3 // Sets all registers that have architecturally-defined reset
4 // values to those values.
5
6 ResetRegs();

```

E2.1.343 RestrictedNSPri

```

1 // RestrictedNSPri()
2 // =====
3 // The priority to which Non-secure exceptions are restricted if AIRCR.PRIS is set.
4
5 integer RestrictedNSPri()
6     return 0x80;

```

E2.1.344 RF

```

1 // RF[] - non-assignment form
2 // =====
3
4 bits(32) RF[integer n]
5     assert n >= 0 && n <= 15;
6
7     // Returns the selected general-purpose register for indices less than 15,
8     // or the APSR Condition flags for the index 15.
9
10    if n < 15 then
11        result = R[n];
12    else
13        result = APSR[31:28] : Zeros(28);
14
15    return result;
16
17 // RF[] - assignment form
18 // =====
19
20 RF[integer n] = bits(32) value
21     assert n >= 0 && n <= 15;
22
23     // Assigns a value to the selected general-purpose register for indices

```

```

24 // less than 15, or the APSR Condition flags for the index 15.
25
26 if n < 15 then
27     R[n] = value;
28 else
29     APSR.N = value[31];
30     APSR.Z = value[30];
31     APSR.C = value[29];
32     APSR.V = value[28];

```

E2.1.345 RFD

```

1 // RFD[] - non-assignment form
2 // =====
3
4 bits(64) RFD[integer n]
5     assert n >= 0 && n <= 14;
6     assert n[0] == '0';
7
8     // Returns the selected general-purpose register pair
9     // Register pairs containing SP or PC are UNPREDICTABLE.
10    if n > 10 then UNPREDICTABLE;
11
12    result = R[n+1]:R[n];
13    return result;
14
15 // RFD[] - assignment form
16 // =====
17
18 RFD[integer n] = bits(64) value
19     assert n >= 0 && n <= 14;
20     assert n[0] == '0';
21
22     // Assigns a value to the selected general-purpose register pair
23     // Register pairs containing SP or PC are UNPREDICTABLE.
24     if n > 10 then UNPREDICTABLE;
25
26     R[n+1] = value[63:32];
27     R[n] = value[31:0];

```

E2.1.346 RName

```

1 // RName[] - assignment form
2 // =====
3
4 RName[RNames reg] = bits(32) value
5     case reg of
6         when {RNamesSP_Main_NonSecure, RNamesSP_Process_NonSecure,
7              RNamesSP_Main_Secure, RNamesSP_Process_Secure}
8             // It is IMPLEMENTATION DEFINED whether stack pointer limit checking
9             // is performed for instructions that were previously UNPREDICTABLE
10            // when modifying the stack pointer.
11            applyLimit = boolean IMPLEMENTATION_DEFINED "SPLim check UNPRED instructions";
12            exc = _SP(reg, FALSE, !applyLimit, value);
13            assert applyLimit || exc.fault == NoFault;
14         when RNamesPC
15             // Direct PC writes not supported, PC updates must go through
16             // LoadWritePC(), BranchReturn() or similar function.
17             assert FALSE;
18         otherwise
19             _RName[reg] = value;
20     return;
21
22 // RName[] - non-assignment form
23 // =====
24
25 bits(32) RName[RNames reg]

```

```

26     bits(32) result;
27     case reg of
28         when {RNamesSP_Main_NonSecure, RNamesSP_Process_NonSecure,
29              RNamesSP_Main_Secure,    RNamesSP_Process_Secure}
30             result = _RName[reg][31:2]:'00';
31         when RNamesPC
32             result = _RName[RNamesPC] + 4;
33         otherwise
34             result = _RName[reg];
35     return result;

```

E2.1.347 RNames

```

1 // The names of the core registers. SP is a Banked register.
2
3 enumeration RNames {RNames0, RNames1, RNames2, RNames3, RNames4, RNames5, RNames6,
4                    RNames7, RNames8, RNames9, RNames10, RNames11, RNames12,
5                    RNamesSP_Main_Secure, RNamesSP_Main_NonSecure,
6                    RNamesLR, RNamesPC,
7                    RNamesSP_Process_NonSecure, RNamesSP_Process_Secure};

```

E2.1.348 ROR

```

1 // ROR()
2 // =====
3
4 bits(N) ROR(bits(N) x, integer shift)
5     if shift == 0 then
6         result = x;
7     else
8         (result, -) = ROR_C(x, shift);
9     return result;

```

E2.1.349 ROR_C

```

1 // ROR_C()
2 // =====
3
4 (bits(N), bit) ROR_C(bits(N) x, integer shift)
5     assert shift != 0;
6     m = shift MOD N;
7     result = LSR(x,m) OR LSL(x,N-m);
8     carry_out = result[N-1];
9     return (result, carry_out);

```

E2.1.350 RotCell

```

1 // RotCell()
2 // =====
3
4 bits(4) RotCell(bits(4) inCell, integer amount)
5     bits(8) tmp;
6     bits(4) outCell;
7
8     assert amount < 4 && amount > 0;
9     tmp[7:0] = inCell[3:0]:inCell[3:0];
10    outCell = tmp[7-amount:4-amount];
11    return outCell;

```

E2.1.351 RoundDown

```

1 // RoundDown()
2 // =====
3
4 integer RoundDown(real x);

```

E2.1.352 RoundTowardsZero

```

1 // RoundTowardsZero()
2 // =====
3
4 integer RoundTowardsZero(real x)
5     return if x == 0.0 then 0 else if x > 0.0 then RoundDown(x) else RoundUp(x);

```

E2.1.353 RoundUp

```

1 // RoundUp()
2 // =====
3
4 integer RoundUp(real x);

```

E2.1.354 RRX

```

1 // RRX()
2 // =====
3
4 bits(N) RRX(bits(N) x, bit carry_in)
5     (result, -) = RRX_C(x, carry_in);
6     return result;

```

E2.1.355 RRX_C

```

1 // RRX_C()
2 // =====
3
4 (bits(N), bit) RRX_C(bits(N) x, bit carry_in)
5     result = carry_in : x[N-1:1];
6     carry_out = x[0];
7     return (result, carry_out);

```

E2.1.356 RSPCheck

```

1 // RSPCheck[] - assignment form
2 // =====
3
4 RSPCheck[integer n] = bits(32) value
5     if n == 13 then
6         - = _SP(LookUpSP(), FALSE, FALSE, value);
7     else
8         R[n] = value;
9     return;

```

E2.1.357 RZ

```

1 // RZ[] -- Read R15 as zero
2 // =====
3
4 bits(32) RZ[integer n]
5     assert n >= 0 && n <= 15;
6     if n == 15 then
7         return Zeros(32);
8     else
9         return R[n];

```

E2.1.358 S

```
1 // S[]
2 // ===
3
4 // Non-assignment form
5
6 bits(32) S[integer n]
7     assert n >= 0 && n <= 31;
8     return _S[n];
9
10 // Assignment form
11
12 S[integer n] = bits(32) value
13     assert n >= 0 && n <= 31;
14     _S[n] = value;
15     return;
```

E2.1.359 Sat

```
1 // Sat()
2 // =====
3
4 bits(N) Sat(integer i, integer N, boolean unsigned)
5     result = if unsigned then UnsignedSat(i, N) else SignedSat(i, N);
6     return result;
```

E2.1.360 SatQ

```
1 // SatQ()
2 // =====
3
4 (bits(N), boolean) SatQ(integer i, integer N, boolean unsigned)
5     (result, sat) = if unsigned then UnsignedSatQ(i, N) else SignedSatQ(i, N);
6     return (result, sat);
```

E2.1.361 SAttributes

```
1 // Security attributes associated with an address
2
3 type SAttributes is (
4     boolean nsc,           // Non-secure callability of an address. FALSE = not
5                             // callable from the Non-secure state.
6     boolean ns,           // Security of an address FALSE = Secure, TRUE = Non-secure.
7     bits(8) sregion,      // The SAU region number.
8     boolean srvalid,      // Set to TRUE if the SAU region number is valid.
9     bits(8) iregion,      // The IDAU region number.
10    boolean irvalid       // Set to TRUE if the IDAU region number is valid.
11 )
```

E2.1.362 SCS_UpdateStatusRegs

```
1 // SCS_UpdateStatusRegs()
2 // =====
3 // Update status registers in the System Control Space (SCS).
4
5 SCS_UpdateStatusRegs();
```

E2.1.363 SecureDebugMonitorAllowed


```

1 // SecureDebugMonitorAllowed()
2 // =====
3
4 boolean SecureDebugMonitorAllowed()
5     if DAUTHCTRL_S.FSDMA == '1' then
6         return TRUE;
7     elseif DAUTHCTRL_S.SPIDENSEL == '1' then
8         return DAUTHCTRL_S.INTSPIDEN == '1';
9     else
10        return ExternalSecureSelfHostedDebugEnabled();

```

E2.1.364 SecureHaltingDebugEnabled

```

1 // SecureHaltingDebugEnabled()
2 // =====
3
4 boolean SecureHaltingDebugEnabled()
5     if HaltingDebugEnabled() == FALSE then
6         return FALSE;
7     elseif DAUTHCTRL_S.SPIDENSEL == '1' then
8         return DAUTHCTRL_S.INTSPIDEN == '1';
9     else
10        return ExternalSecureInvasiveDebugEnabled();

```

E2.1.365 SecureNoninvasiveDebugEnabled

```

1 // SecureNoninvasiveDebugEnabled()
2 // =====
3
4 boolean SecureNoninvasiveDebugEnabled(boolean isPriv)
5     assert HaveSecurityExt();
6     if DHCSR.S_SDE == '1' && (!HaveUDE() || DHCSR.S_SUIDE == '0') then
7         return TRUE;
8     elseif !isPriv && UnprivHaltingDebugEnabled(TRUE) then
9         return TRUE;
10    elseif !NoninvasiveDebugEnabled(isPriv) then
11        return FALSE;
12    elseif DAUTHCTRL_S.SPIDENSEL == '1' then
13        return DAUTHCTRL_S.INTSPIDEN == '1';
14    else
15        return ExternalSecureNoninvasiveDebugEnabled();
16
17 boolean SecureNoninvasiveDebugEnabled()
18     return SecureNoninvasiveDebugEnabled(CurrentModeIsPrivileged());

```

E2.1.366 SecurityCheck

```

1 // SecurityCheck()
2 // =====
3
4 SAttributes SecurityCheck(bits(32) address, boolean isinstrfetch, boolean isSecure)
5     SAttributes result;
6     addr = UInt(address);
7
8     // Setup default attributes
9     result.ns = !HaveSecurityExt();
10    result.nsc = FALSE;
11    result.sregion = Zeros(8);
12    result.srvalid = FALSE;
13    result.iregion = Zeros(8);
14    result.irvalid = FALSE;
15    idauExempt = FALSE;
16    idauNs = TRUE;
17    idauNsc = TRUE;

```

```

18
19 // If an IMPLEMENTATION DEFINED memory security attribution unit is present
20 // query it and override defaults set above. The IDAU is subject to the same
21 // 32byte minimum region granularity as the SAU or MPU.
22 // NOTE: The defaults above are set such that the IDAU has no effect on the
23 // SAU.
24 if boolean IMPLEMENTATION_DEFINED "IDAU present" then
25     (idauExempt,
26     idauNs,
27     idauNsc,
28     result.iregion,
29     result.irvalid) = IDAUCheck(address[31:5]:'00000');
30
31 // The 0xF0000000 -> 0xFFFFFFFF is always Secure for instruction fetches
32 if isinstrfetch && (address[31:28] == '1111') then
33     // Use default attributes defined above
34
35 // Check if the address is exempt from SAU/IDAU checking.
36 elseif idauExempt || // IDAU specified exemption.
37     (isinstrfetch && (address[31:28] == '1110')) || // Whole 0xExxxxxxx range
38     // exempt for IFetch.
39     ((addr >= 0xE0000000) && (addr <= 0xE0003FFF)) || // ITM, DWT, FPB, PMU.
40     ((addr >= 0xE0005000) && (addr <= 0xE0005FFF)) || // RAS error record registers.
41     ((addr >= 0xE000E000) && (addr <= 0xE000EFFF)) || // SCS.
42     ((addr >= 0xE002E000) && (addr <= 0xE002EFFF)) || // SCS NS alias.
43     ((addr >= 0xE0040000) && (addr <= 0xE0041FFF)) || // TPIU, ETM.
44     ((addr >= 0xE00FF000) && (addr <= 0xE00FFFFF)) then // ROM table.
45     // Memory security reported as NS-Req, and no region information is supplied.
46     result.ns = !isSecure;
47     result.irvalid = FALSE;
48
49 else
50     // If the SAU is enabled check its regions.
51     if SAU_CTRL.ENABLE == '1' then
52         boolean multiRegionHit = FALSE;
53         for r = 0 to (UInt(SAU_TYPE.SREGION) - 1)
54             if SAU_REGION[r].ENABLE == '1' then
55                 // SAU region enabled so perform checks
56                 bits(32) base_address = SAU_REGION[r].BADDR:'00000';
57                 bits(32) limit_address = SAU_REGION[r].LADDR:'11111';
58                 if ((UInt(base_address) <= addr) &&
59                     (UInt(limit_address) >= addr)) then
60                     if result.srvalid then
61                         multiRegionHit = TRUE;
62                     else
63                         result.ns = SAU_REGION[r].NSC == '0';
64                         result.nsc = SAU_REGION[r].NSC == '1';
65                         result.srvalid = TRUE;
66                         result.sregion = r[7:0];
67
68                 // If multiple regions are hit then report memory as Secure and not
69                 // Non-secure callable. Also do not report any region number
70                 // information.
71                 if multiRegionHit then
72                     result.ns = FALSE;
73                     result.nsc = FALSE;
74                     result.sregion = Zeros(8);
75                     result.srvalid = FALSE;
76
77                 // SAU disabled, check if whole address space should be marked as
78                 // Non-secure.
79                 elseif SAU_CTRL.ALLNS == '1' then
80                     result.ns = TRUE;
81
82                 // Override the internal setting if the external attribution unit
83                 // reports more restrictive attributes.
84                 if !idauNs then
85                     if result.ns || (!idauNsc && result.nsc) then
86                         result.ns = FALSE;

```

```

87         result.nsc = idauNsc;
88
89     return result;

```

E2.1.367 SecurityState

```

1  // Type and definition of the current Security state of PE
2
3  enumeration SecurityState {SecurityState_NonSecure, SecurityState_Secure};
4  SecurityState CurrentState;

```

E2.1.368 SendEvent

```

1  // SendEvent
2  // =====
3  // Performs a send event by setting the Event Register of
4  // every PE in multiprocessor system.
5
6  SendEvent();

```

E2.1.369 SerializeVFP

```

1  // SerializeVFP
2  // =====
3  // Ensures that any exceptional conditions in previous floating-point
4  // instructions have been detected.
5
6  SerializeVFP();

```

E2.1.370 SetActive

```

1  // SetActive()
2  // =====
3
4  SetActive(integer exception, boolean isSecure, boolean setNotClear)
5      if !HaveSecurityExt() then
6          isSecure = FALSE;
7          // If the exception target Security state is configurable there is only one active
8          // bit. To represent this the Non-secure and Secure instances of the active
9          // flags in the array are always set to the same value.
10         if IsExceptionTargetConfigurable(exception) then
11             if ExceptionTargetsSecure(exception, boolean UNKNOWN) == isSecure then
12                 ExceptionActive[exception] = if setNotClear then '11' else '00';
13         else
14             idx = if isSecure then 0 else 1;
15             ExceptionActive[exception][idx] = if setNotClear then '1' else '0';

```

E2.1.371 SetDWTDebugEvent

```

1  // SetDWTDebugEvent()
2  // =====
3  // Set a pending debug event to the PE.
4
5  boolean SetDWTDebugEvent(boolean secure_match, boolean priv_match, AccType acctype)
6      boolean isLAZYFP = acctype == AccType_LAZYFP;
7      if CanHaltOnEvent(secure_match, priv_match) then
8          DHCSR.C_HALT = '1';
9          DFSR.DWTTRAP = '1';
10         return TRUE;
11     elsif (HaveMainExt()
12         CanPendMonitorOnEvent(secure_match, !isLAZYFP, TRUE, priv_match, TRUE) &&
13         (!isLAZYFP || (isLAZYFP && FPCCR.S.MONRDY == '1'))) then
14         DEMCR.MON_PEND = '1';

```

```

15     DFSR.DWTTRAP = '1';
16     return TRUE;
17 else
18     return FALSE;

```

E2.1.372 SetEventRegister

```

1 // SetEventRegister()
2 // =====
3 // Set the Event Register of the current PE.
4
5 SetEventRegister();

```

E2.1.373 SetExclusiveMonitors

```

1 // SetExclusiveMonitors()
2 // =====
3
4 SetExclusiveMonitors(bits(32) address, integer size)
5
6     boolean isSecure = CurrentState == SecurityState_Secure;
7     if address != Align(address, size) then
8         UFSR.UNALIGNED = '1';
9         excInfo = CreateException(UsageFault, FALSE, isSecure);
10    else
11        (excInfo, memaddrdesc) = ValidateAddress(address, AccType_NORMAL, FindPriv(),
12                                                isSecure, FALSE, TRUE);
13        HandleException(excInfo);
14
15    if memaddrdesc.memattrs.shareable then
16        MarkExclusiveGlobal(memaddrdesc.paddress, ProcessorID(), size);
17
18    MarkExclusiveLocal(memaddrdesc.paddress, ProcessorID(), size);

```

E2.1.374 SetITSTATEAndCommit

```

1 // SetITSTATEAndCommit()
2 // =====
3
4 SetITSTATEAndCommit(ITSTATEType it)
5     // This function directly commits the change to the ITSTATE, so ThisInstrITSTATE()
6     // and NextInstrITSTATE() both point to the target address.
7     _NextInstrITState = it;
8     _ITStateChanged = TRUE;
9     EPSR.IT = it;
10    return;

```

E2.1.375 SetPending

```

1 // SetPending()
2 // =====
3
4 SetPending(integer exception, boolean isSecure, boolean setNotClear)
5     if !HaveSecurityExt() then
6         isSecure = FALSE;
7         // If the exception target Security state is configurable there is only one pending
8         // bit. To represent this, the Non-secure and Secure instances of the pending
9         // flags in the array are always set to the same value.
10        if IsExceptionTargetConfigurable(exception) then
11            ExceptionPending[exception] = if setNotClear then '11' else '00';
12        else
13            idx = if isSecure then 0 else 1;
14            ExceptionPending[exception][idx] = if setNotClear then '1' else '0';

```

E2.1.376 SetThisInstrDetails

```

1  // SetThisInstrDetails
2  // =====
3
4  SetThisInstrDetails(bits(32) opcode, integer len)
5      // Insert the instruction into the queue at the first free slot. For an
6      // instruction with no beat behavior this should always be the first slot.
7      // NOTE: MVE instructions in IT blocks do not have beat-wise execution.
8      i = 0;
9      isMveInst = IsMveBeatWiseInstruction(opcode);
10     isCdeInst = IsCdeBeatWiseInstruction(opcode);
11     isBeatInst = (isMveInst || isCdeInst) && !InITBlock();
12     repeat
13         emptySlot = !_InstInfo[i].Valid;
14         if emptySlot && (isBeatInst || i == 0) then
15             _InstInfo[i].Valid = TRUE;
16             _InstInfo[i].Length = len;
17             _InstInfo[i].Opcode = opcode;
18             _InstInfo[i].Type = (if isMveInst then InstrType_BEATWISE_MVE else
19                                 if isCdeInst then InstrType_BEATWISE_CDE else
20                                     InstrType_NON_BEATWISE);
21             i = i + 1;
22     until emptySlot || (!isBeatInst && i > 0) || (i >= MAX_OVERLAPPING_INSTRS);

```

E2.1.377 SetVPTMask

```

1  // SetVPTMask()
2  // =====
3
4  SetVPTMask(integer beat, bits(4) mask)
5      // Only one mask field is available for each pair of beats.
6      assert beat[0] == '1';
7      Elem[VPR.[MASK23, MASK01], beat DIV 2, 4] = mask;
8      // Since the mask has been modified do not advance the VPT state after this
9      // instruction beat.
10     _AdvanceVPTState = FALSE;

```

E2.1.378 Shift

```

1  // Shift()
2  // =====
3
4  bits(N) Shift(bits(N) value, SRType sr_type, integer amount, bit carry_in)
5      (result, -) = Shift_C(value, sr_type, amount, carry_in);
6      return result;

```

E2.1.379 Shift_C

```

1  // Shift_C()
2  // =====
3
4  (bits(N), bit) Shift_C(bits(N) value, SRType sr_type, integer amount, bit carry_in)
5      assert !(sr_type == SRType_RRX && amount != 1);
6
7      if amount == 0 then
8          (result, carry_out) = (value, carry_in);
9      else
10         case sr_type of
11             when SRType_LSL
12                 (result, carry_out) = LSL_C(value, amount);
13             when SRType_LSR
14                 (result, carry_out) = LSR_C(value, amount);
15             when SRType_ASR
16                 (result, carry_out) = ASR_C(value, amount);

```

```

17         when SRTYPE_ROR
18             (result, carry_out) = ROR_C(value, amount);
19         when SRTYPE_RRX
20             (result, carry_out) = RRX_C(value, carry_in);
21
22     return (result, carry_out);

```

E2.1.380 SignedSat

```

1 // SignedSat()
2 // =====
3
4 bits(N) SignedSat(integer i, integer N)
5     (result, -) = SignedSatQ(i, N);
6     return result;

```

E2.1.381 SignedSatQ

```

1 // SignedSatQ()
2 // =====
3
4 (bits(N), boolean) SignedSatQ(integer i, integer N)
5     if i > 2^(N-1) - 1 then
6         result = 2^(N-1) - 1; saturated = TRUE;
7     elsif i < -(2^(N-1)) then
8         result = -(2^(N-1)); saturated = TRUE;
9     else
10        result = i; saturated = FALSE;
11    return (result[N-1:0], saturated);

```

E2.1.382 SignExtend

```

1 // SignExtend()
2 // =====
3
4 bits(N) SignExtend(bits(M) x, integer N)
5     assert N >= M;
6     return Replicate(x[M-1], N-M) : x;
7
8 bits(N) SignExtend(bits(M) x)
9     return SignExtend(x, N);

```

E2.1.383 Sleeping

```

1 // Indicates the PE is sleeping
2
3 boolean Sleeping;

```

E2.1.384 SleepOnExit

```

1 // SleepOnExit()
2 // =====
3 // Optionally returns PE to a power-saving mode on return from the only
4 // active exception.
5
6 SleepOnExit();

```

E2.1.385 SP

```

1 // SP
2 // ==
3
4 // Non-assignment form
5
6 bits(32) SP
7     return R[13];
8
9 // Assignment form
10
11 SP = bits(32) value
12     RSPCheck[13] = value;

```

E2.1.386 SP_Main

```

1 // SP_Main
2 // =====
3
4 // Non-assignment form
5
6 bits(32) SP_Main
7     value = if IsSecure() then SP_Main_Secure else SP_Main_NonSecure;
8     return value;
9
10 // Assignment form
11
12 SP_Main = bits(32) value
13     if IsSecure() then
14         SP_Main_Secure = value;
15     else
16         SP_Main_NonSecure = value;

```

E2.1.387 SP_Main_NonSecure

```

1 // SP_Main_NonSecure
2 // =====
3
4 // Non-assignment form
5
6 bits(32) SP_Main_NonSecure
7     return _SP(RNamesSP_Main_NonSecure);
8
9 // Assignment form
10
11 SP_Main_NonSecure = bits(32) value
12     - = _SP(RNamesSP_Main_NonSecure, FALSE, FALSE, value);

```

E2.1.388 SP_Main_Secure

```

1 // SP_Main_Secure
2 // =====
3
4 // Non-assignment form
5
6 bits(32) SP_Main_Secure
7     return _SP(RNamesSP_Main_Secure);
8
9 // Assignment form
10
11 SP_Main_Secure = bits(32) value
12     - = _SP(RNamesSP_Main_Secure, FALSE, FALSE, value);

```

E2.1.389 SP_Process

```

1 // SP_Process
2 // =====
3
4 // Non-assignment form
5
6 bits(32) SP_Process
7     value = if IsSecure()
8             then SP_Process_Secure else SP_Process_NonSecure;
9     return value;
10
11 // Assignment form
12
13 SP_Process = bits(32) value
14     if IsSecure() then
15         SP_Process_Secure = value;
16     else
17         SP_Process_NonSecure = value;

```

E2.1.390 SP_Process_NonSecure

```

1 // SP_Process_NonSecure
2 // =====
3
4 // Non-assignment form
5
6 bits(32) SP_Process_NonSecure
7     return _SP(RNamesSP_Process_NonSecure);
8
9 // Assignment form
10
11 SP_Process_NonSecure = bits(32) value
12     - = _SP(RNamesSP_Process_NonSecure, FALSE, FALSE, value);

```

E2.1.391 SP_Process_Secure

```

1 // SP_Process_Secure
2 // =====
3
4 // Non-assignment form
5
6 bits(32) SP_Process_Secure
7     return _SP(RNamesSP_Process_Secure);
8
9 // Assignment form
10
11 SP_Process_Secure = bits(32) value
12     - = _SP(RNamesSP_Process_Secure, FALSE, FALSE, value);

```

E2.1.392 SpeculativeSynchronizationBarrier

```

1 // Speculative Synchronisation Barrier
2 // =====
3 // Perform a Speculative Synchronization Barrier.
4
5 SpeculativeSynchronizationBarrier();

```

E2.1.393 SRTYPE

```

1 // Different types of shift and rotate operations
2
3 enumeration SRTYPE {SRTYPE_LSL, SRTYPE_LSR, SRTYPE_ASR, SRTYPE_ROR, SRTYPE_RRX};

```

E2.1.394 Stack


```

1 // Stack
2 // =====
3
4 // Assignment form
5
6 ExcInfo Stack(bits(32) frameptr, integer offset, RNames spreg, PMode mode, bits(32) value)
7 // This function is used to perform register stacking operations that are
8 // done around exception handling. If the stack pointer is below the stack
9 // pointer limit but the access itself is above the limit it is
10 // IMPLEMENTATION DEFINED whether the write is performed. If the
11 // address of access is below the limit the access is not performed
12 // regardless of the stack pointer value.
13 if !ViolatesSPLim(spreg, frameptr) then
14     doAccess = TRUE;
15 else
16     doAccess = boolean IMPLEMENTATION_DEFINED "Push non-violating locations";
17
18 address = frameptr + offset;
19 if doAccess && !ViolatesSPLim(spreg, address) then
20     secure = ((spreg == RNamesSP_Main_Secure) ||
21              (spreg == RNamesSP_Process_Secure));
22     // Work out if the stack operations should be privileged or not.
23     if secure then
24         isPriv = CONTROL_S.nPRIV == '0';
25     else
26         isPriv = CONTROL_NS.nPRIV == '0';
27     isPriv = isPriv || (mode == PMode_Handler);
28     // Finally perform the memory operations.
29     excInfo = MemA_with_priv_security(address, 4, AccType_STACK,
30                                     FindMemPriv(isPriv), secure, TRUE, value);
31 else
32     excInfo = DefaultExcInfo();
33 return excInfo;
34
35 // Non-assignment form
36
37 (ExcInfo, bits(32)) Stack(bits(32) frameptr, integer offset, RNames spreg, PMode mode)
38     secure = ((spreg == RNamesSP_Main_Secure) ||
39              (spreg == RNamesSP_Process_Secure));
40     // Work out if the stack operations should be privileged or not.
41     if secure then
42         isPriv = CONTROL_S.nPRIV == '0';
43     else
44         isPriv = CONTROL_NS.nPRIV == '0';
45     isPriv = isPriv || (mode == PMode_Handler);
46     // Finally perform the memory operations.
47     address = frameptr + offset;
48     (excInfo, value) = MemA_with_priv_security(address, 4, AccType_STACK,
49                                             FindMemPriv(isPriv), secure, TRUE);
50 return (excInfo, value);

```

E2.1.395 StandardFPSCRValue

```

1 // StandardFPSCRValue()
2 // =====
3
4 FPSCR_Type StandardFPSCRValue()
5     return '00000' : FPSCR.AHP : '110000' : FPSCR.FZ16 : '00000000000000000000';

```

E2.1.396 SteppingDebug

```

1 // SteppingDebug()
2 // =====
3
4 SteppingDebug(boolean isSecure, boolean isPriv, boolean haltAllowed, boolean monitorAllowed)
5 // Process step requests, stepping must be avoided if a pending event is
6 // already in flight.

```

```

7      if (haltAllowed      && CanHaltOnEvent(isSecure, isPriv) &&
8          DHCSR.C_STEP == '1' && DHCSR.C_HALT == '0') then
9          // If C_STEP is set then pend a debug halt for the next instruction.
10         DHCSR.C_HALT = '1';
11         DFSR.HALTED = '1';
12     elsif (monitorAllowed && CanPendMonitorOnEvent(isSecure, TRUE, TRUE, isPriv, TRUE) &&
13         DEMCR.MON_STEP == '1' && DEMCR.MON_PEND == '0') then
14         // If MON_STEP is set then pend the an exception for the next instruction.
15         DEMCR.MON_PEND = '1';
16         DFSR.HALTED = '1';

```

E2.1.397 SynchronizeBusFault

```

1  // SynchronizeBusFault()
2  // =====
3
4  ExcInfo SynchronizeBusFault()
5      return SynchronizeBusFault(FALSE);
6
7  ExcInfo SynchronizeBusFault(AccType acctype)
8      return SynchronizeBusFault(acctype == AccType_LAZYFP);
9
10 ExcInfo SynchronizeBusFault(boolean isLazyStatePreservation)
11     // Force any latent BusFaults to be recognised.
12     faultDetected = BusFaultBarrier();
13     if faultDetected then
14         if isLazyStatePreservation then
15             BFSR.LSPERR = '1';
16         else
17             BFSR.IMPRESERR = '1';
18             // To ensure errors are containable, asynchronous BusFaults escalate as if they were
19             // synchronous if Implicit Error Synchronization Barriers are enabled.
20             handleSync = AIRCR.IESB == '1';
21             excInfo = CreateException(BusFault, FALSE, IsSecure(), handleSync);
22     else
23         excInfo = DefaultExcInfo();
24     return excInfo;

```

E2.1.398 T32ExpandImm

```

1  // T32ExpandImm()
2  // =====
3
4  bits(32) T32ExpandImm(bits(12) imm12)
5
6     // APSR.C argument to following function call does not affect the imm32 result.
7     (imm32, -) = T32ExpandImm_C(imm12, APSR.C);
8
9     return imm32;

```

E2.1.399 T32ExpandImm_C

```

1  // T32ExpandImm_C()
2  // =====
3
4  (bits(32), bit) T32ExpandImm_C(bits(12) imm12, bit carry_in)
5
6     if imm12[11:10] == '00' then
7
8         case imm12[9:8] of
9             when '00'
10                 imm32 = ZeroExtend(imm12[7:0], 32);
11             when '01'
12                 if imm12[7:0] == '00000000' then UNPREDICTABLE;
13                 imm32 = '00000000' : imm12[7:0] : '00000000' : imm12[7:0];

```

```

14         when '10'
15             if imm12[7:0] == '00000000' then UNPREDICTABLE;
16             imm32 = imm12[7:0] : '00000000' : imm12[7:0] : '00000000';
17         when '11'
18             if imm12[7:0] == '00000000' then UNPREDICTABLE;
19             imm32 = imm12[7:0] : imm12[7:0] : imm12[7:0] : imm12[7:0];
20             carry_out = carry_in;
21
22     else
23
24         unrotated_value = ZeroExtend('1':imm12[6:0], 32);
25         (imm32, carry_out) = ROR_C(unrotated_value, UInt(imm12[11:7]));
26
27     return (imm32, carry_out);

```

E2.1.400 TailChain

```

1 // TailChain()
2 // =====
3
4 (ExcInfo, EXC_RETURN_Type) TailChain(integer exceptionNumber,
5                                     boolean excIsSecure,
6                                     EXC_RETURN_Type excReturn)
7
8 // Refresh LR with the excReturn value, ready for the next exception.
9 if !HaveMveOrFPEExt() then
10     excReturn.FType = '1';
11     excReturn.PREFIX = Ones(8);
12
13 return ExceptionTaken(exceptionNumber, TRUE, excIsSecure, IgnoreFaults_NONE, excReturn);

```

E2.1.401 TakePreserveFPException

```

1 // TakePreserveFPException()
2 // =====
3
4 boolean TakePreserveFPException(ExcInfo excInfo)
5     assert HaveMveOrFPEExt();
6     assert excInfo.origFault IN {DebugMonitor, SecureFault, MemManage, BusFault, UsageFault};
7
8 // Get the details of the original fault so that any escalation to HardFault or Lockup
9 // based on the current execution priority is ignored. Escalation is performed manually
10 // against the FPCCR.*RDY fields below.
11 exception = excInfo.origFault;
12 isSecure = excInfo.origFaultIsSecure;
13 fpccr = if isSecure then FPCCR_S else FPCCR_NS;
14
15 if FPCCR_S.MONRDY == '1' && FPCCR_S.HFRDY == '0' then UNPREDICTABLE;
16 if FPCCR_S.BFRDY == '1' && FPCCR_S.HFRDY == '0' then UNPREDICTABLE;
17 if FPCCR_S.SFRDY == '1' && FPCCR_S.HFRDY == '0' then UNPREDICTABLE;
18 if fpccr.UFRDY == '1' && FPCCR_S.HFRDY == '0' then UNPREDICTABLE;
19 if fpccr.MMRDY == '1' && FPCCR_S.HFRDY == '0' then UNPREDICTABLE;
20 if exception == DebugMonitor && FPCCR_S.MONRDY == '0' then
21     // ignore DebugMonitor exception
22     return FALSE;
23
24 // Handle exception specific details like escalation and syndrome information.
25 case exception of
26     when MemManage
27         escalate = fpccr.MMRDY == '0';
28     when UsageFault
29         escalate = fpccr.UFRDY == '0';
30     when BusFault
31         escalate = FPCCR_S.BFRDY == '0';
32     when SecureFault
33         escalate = FPCCR_S.SFRDY == '0';
34     otherwise
35         escalate = FALSE;

```

```

36     if escalate then
37         exception = HardFault;
38         // Faults that originally targeted the Secure state still target the
39         // Secure state even if HardFault normally targets Non-secure.
40         isSecure = isSecure || ExceptionTargetsSecure(HardFault, isSecure);
41
42         // Check if the exception is enabled and has sufficient priority to
43         // preempt and be taken straight away.
44         termInst = FALSE;
45         if (ExceptionPriority(exception, isSecure, TRUE) < ExecutionPriority()) &&
46             ExceptionEnabled(exception, isSecure) then
47             if escalate then
48                 HFSR.FORCED = '1';
49                 // Set the exception pending and terminate the current instruction. This
50                 // leaves Floating-point disabled (that is CONTROL.FPCA set to 0) and
51                 // prevents the preempting exception entry reserving space for a redundant
52                 // floating-point state.
53                 SetPending(exception, isSecure, TRUE);
54                 termInst = TRUE;
55             else
56                 // If the reason the exception cannot preempt is because of the fact that
57                 // HardFault could not be entered by the context the floating-point state
58                 // belongs to then enter the lockup state.
59                 if FPCCR_S.HFRDY == '0' then
60                     Lockup(FALSE); // Lockup at current priority, lock-up address = 0xEFFFFFFE.
61                     termInst = TRUE;
62                 else
63                     if escalate then
64                         HFSR.FORCED = '1';
65                         // Set the exception pending so it will be taken after the current
66                         // handler returns.
67                         SetPending(exception, isSecure, TRUE);
68     return termInst;

```

E2.1.402 TakeReset

```

1 // TakeReset ()
2 // =====
3
4 TakeReset ()
5 // If the Security Extension is implemented the PE resets into Secure state.
6 // If the Security Extension is not implemented the PE resets into Non-secure state.
7 if HaveSecurityExt () then
8     CurrentState = SecurityState_Secure;
9 else
10    CurrentState = SecurityState_NonSecure;
11
12 ResetRegs(); // Catch-all function for System Control Space reset.
13 if HaveMainExt () then
14     LR = Ones (32); // Preset to an illegal exception return value.
15     SetITSTATEAndCommit (Zeros (8)); // IT/ICI bits cleared.
16 else
17     LR = bits (32) UNKNOWN; // Value must be initialised by software.
18
19 if HaveMve () then
20     VPR = bits (32) UNKNOWN;
21
22 // Reset internal run state
23 Halted = FALSE;
24 LockedUp = FALSE;
25
26 // Initialize the Floating-point Extension
27 if HaveMveOrFPExt () then
28     for i = 0 to 31
29         S[i] = bits (32) UNKNOWN;
30
31 for i = 0 to MaxExceptionNum() // All exceptions Inactive.
32     ExceptionActive[i] = '00';

```

```

33     ClearExclusiveLocal(ProcessorID()); // Synchronization (LDREX* / STREX*) monitor support.
34     ClearEventRegister();               // See WFE instruction for more information.
35     ClearInFlightInstructions();
36     for i = 0 to 12
37         R[i] = bits(32) UNKNOWN;
38
39     // Clearing stack limit registers
40     if HaveMainExt() then
41         MSPLIM_NS = Zeros(32);
42         PSPLIM_NS = Zeros(32);
43     if HaveSecurityExt() then
44         MSPLIM_S = Zeros(32);
45         PSPLIM_S = Zeros(32);
46
47     // Load the initial value of the stack pointer and the reset value from the
48     // vector table. The order of the loads is IMPLEMENTATION DEFINED.
49     (excSp, sp) = Vector[0, HaveSecurityExt()];
50     (excRst, start) = Vector[Reset, HaveSecurityExt()];
51     if excSp.fault != NoFault || excRst.fault != NoFault then
52         SetActive(HardFault, HaveSecurityExt(), TRUE);
53         Lockup(TRUE);
54
55     // Initialize the stack pointers and start execution at the reset vector.
56     if HaveSecurityExt() then
57         SP_Main_Secure = sp;
58         SP_Main_NonSecure = ((bits(30) UNKNOWN):'00');
59         SP_Process_Secure = ((bits(30) UNKNOWN):'00');
60     else
61         SP_Main_NonSecure = sp;
62         SP_Process_NonSecure = ((bits(30) UNKNOWN):'00');
63     EPSR.T = start[0];
64     BranchTo(start, TRUE, FALSE);
65
66     // Trigger a debug event even if resetting into Secure state.
67     if DHCSR.C_DEBUGEN == '1' && DEMCR.VC_CORERESET == '1' &&
68         (HasArchVersion(Armv8p1) || CanHaltOnEvent(FALSE, TRUE)) then
69         DHCSR.C_HALT = '1';
70         DFSR.VCATCH = '1';

```

E2.1.403 ThisInstr

```

1 // ThisInstr()
2 // =====
3
4 bits(32) ThisInstr()
5     return ThisInstr(if HaveMve() then _InstID else 0);
6
7 bits(32) ThisInstr(integer instID)
8     if !_InstInfo[instID].Valid then
9         return bits(32) UNKNOWN;
10    return _InstInfo[instID].Opcode;

```

E2.1.404 ThisInstrAddr

```

1 // ThisInstrAddr()
2 // =====
3
4 bits(32) ThisInstrAddr()
5     return _CurrentInstrExecState.FetchAddr;

```

E2.1.405 ThisInstrITState

```

1 // ThisInstrITState()
2 // =====
3

```

```

4 ITSTATEType ThisInstrITState()
5     if HaveMainExt() then
6         value = _CurrentInstrExecState.ITState;
7     else
8         value = Zeros(8);
9     return value;

```

E2.1.406 ThisInstrLength

```

1 // ThisInstrLength()
2 // =====
3
4 integer ThisInstrLength()
5     return ThisInstrLength(if HaveMve() then _InstID else 0);
6
7 integer ThisInstrLength(integer instID)
8     if !_InstInfo[instID].Valid then
9         return 0;
10    return _InstInfo[instID].Length;

```

E2.1.407 TopLevel

```

1 // TopLevel()
2 // =====
3
4 // This function is called one time for each tick the PE is not in a sleep
5 // state. It handles all instruction processing, including fetching the opcode,
6 // decode and execute. It also handles pausing execution when in the lockup
7 // state.
8 TopLevel()
9     // Process any pending reset.
10    if AIRCR_S.SYSRESETREQ == '1' then
11        TakeReset();
12
13    UpdateDebugEnable();
14    isSecure          = IsSecure();
15    isPriv            = FindPriv();
16    execFault         = FALSE;
17    haltSteppingAllowed = TRUE;
18    monitorSteppingAllowed = TRUE;
19
20    // Reset some globals so that PC/ITSTATE changes can be detected.
21    _ITStateChanged = FALSE;
22    _PCChanged      = FALSE;
23    _CommitState    = FALSE;
24
25    // If we have no instructions in flight then any pending fetch fault was
26    // handled in the previous cycle.
27    if !InstructionsInFlight() then
28        _PendingFetchFault = FALSE;
29
30    // If the PE is halted then do nothing, otherwise process the next
31    // instruction.
32    if !IsDebugState() then
33        // If not locked up, process the next instruction, or just the in flight
34        // beats if a halt or monitor exception is pending.
35        if !LockedUp then
36            // Debug can prevent new fetches if there a pending halt or DebugMonitor
37            // exception. However in-flight instructions need to be completed before we
38            // handle the debug event. In some corner cases (for example, if the debugger
39            // has modified ECI) this may require fetching new instructions to complete
40            // all outstanding beats (as indicated by ECI).
41            debugAllowsFetch = ( (!PendingDebugHalt() && !PendingDebugMonitor()) ||
42                                (EPSR.ECI[7:4] == Zeros(4) &&
43                                 EPSR.ECI[3:0] != Zeros(4) &&
44                                 !InstructionsInFlight()) );
45            // Execute remaining in-flight instructions without fetching new ones if

```

```

46         // there is a pending fetch fault, or debug is preventing the fetch.
47         execFault = InstructionExecute(debugAllowsFetch && !_PendingFetchFault);
48         // Advance the PC and commit instruction state if not locked up, an
49         // exception return is not about to be performed, and the current
50         // instruction has not been terminated by a fault.
51         if _CommitState && !_PendingReturnOperation then
52             InstructionAdvance();
53
54         // If a debug halt was requested and there are no active vector chains
55         // then halt if allowed to do so.
56         if !InstructionsInFlight() && PendingDebugHalt() then
57             Halt();
58
59         // If the execution priority after executing the instruction, but before
60         // handling exception transitions, prevents the DebugMonitor exception preempting,
61         // then prevent monitor stepping. This makes sure that monitor stepping executes
62         // at least one instruction in a preemptable context before reentering the
63         // DebugMonitor.
64         if !MonitorCanPreempt() then
65             monitorSteppingAllowed = FALSE;
66
67     elseif DHCSR.C_HALT == '0' then
68         // Resume the PE if a resume from debug halt was requested.
69         Halted = FALSE;
70         DHCSR.S_RESTART_ST = '1';
71         // If the PE has only just exited debug state prevent stepping immediately
72         // re-entering without executing an instruction first.
73         haltSteppingAllowed = FALSE;
74
75     if !IsDebugState() then
76         try
77             // Process and take any pending exceptions. If there is a pending
78             // fetch fault, only process it if we either have no instructions in
79             // flight, or there is a execution fault.
80             if !_PendingFetchFault || !InstructionsInFlight() || execFault then
81                 if HandleExceptionTransitions() then
82                     // If an exception has been taken, process any step request now,
83                     // not on the next instruction.
84                     SteppingDebug(isSecure, isPriv, TRUE, TRUE);
85
86                     // Pend vector catch debug state when needed.
87                     VectorCatchDebug(IsSecure(), FindPriv());
88                 else
89                     SteppingDebug(isSecure, isPriv,
90                                 haltSteppingAllowed, monitorSteppingAllowed);
91
92             // If the PC has moved away from the lockup address (for example,
93             // because an NMI has been taken) leave the lockup state.
94             if LockedUp && NextInstrAddr() != 0xEFFFFFFE[31:0] then
95                 LockedUp = FALSE;
96
97         catch exn
98             // Do not catch UNPREDICTABLE or internal errors.
99             when IsExceptionTaken(exn)
100                 // The correct architectural behavior for any exceptions is
101                 // performed inside HandleExceptionTransitions. So no
102                 // additional actions are required in this catch block.

```

E2.1.408 TTResp

```

1  // TTResp()
2  // =====
3
4  bits(32) TTResp(bits(32) address, boolean alt, boolean forceunpriv)
5      TT_RESP_Type resp = Zeros();
6
7      // Only allow security checks if currently in Secure state.
8      if IsSecure() then

```

```

9      sAttributes = SecurityCheck(address, FALSE, IsSecure());
10     if sAttributes.srvalid then
11         resp.SREGION = sAttributes.sregion;
12         resp.SRVALID = '1';
13     if sAttributes.irvalid then
14         resp.IREGION = sAttributes.iregion;
15         resp.IRVALID = '1';
16     addrSecure = if sAttributes.ns then '0' else '1';
17     resp.S      = addrSecure;
18
19     // MPU region information only available when privileged or when
20     // inspecting the other MPU state.
21     other_domain = (alt != IsSecure());
22     if CurrentModeIsPrivileged() || alt then
23         (write, read, region, hit) = IsAccessible(address, forceunpriv, other_domain);
24         if hit then
25             resp.MREGION = region;
26             resp.MRVALID = '1';
27         resp.R = read;
28         resp.RW = write;
29         if IsSecure() then
30             resp.NSR = read AND NOT addrSecure;
31             resp.NSRW = write AND NOT addrSecure;
32
33     return resp;

```

E2.1.409 TweakCellInvRot

```

1  // TweakCellInvRot()
2  // =====
3
4  bits(4) TweakCellInvRot(bits(4) inCell)
5      bits(4) outCell;
6      outCell[3] = inCell[2];
7      outCell[2] = inCell[1];
8      outCell[1] = inCell[0];
9      outCell[0] = inCell[0] EOR inCell[3];
10     return outCell;

```

E2.1.410 TweakCellRot

```

1  // TweakCellRot()
2  // =====
3
4  bits(4) TweakCellRot(bits(4) inCell)
5      bits(4) outCell;
6      outCell[3] = inCell[0] EOR inCell[1];
7      outCell[2] = inCell[3];
8      outCell[1] = inCell[2];
9      outCell[0] = inCell[1];
10     return outCell;

```

E2.1.411 TweakInvShuffle

```

1  // TweakInvShuffle()
2  // =====
3
4  bits(64) TweakInvShuffle(bits(64) inData)
5      bits(64) outData;
6      outData[3:0] = TweakCellInvRot(inData[51:48]);
7      outData[7:4] = inData[55:52];
8      outData[11:8] = inData[23:20];
9      outData[15:12] = inData[27:24];
10     outData[19:16] = inData[3:0];
11     outData[23:20] = inData[7:4];

```



```

12     outData[27:24] = TweakCellInvRot(inData[11:8]);
13     outData[31:28] = inData[15:12];
14     outData[35:32] = TweakCellInvRot(inData[31:28]);
15     outData[39:36] = TweakCellInvRot(inData[63:60]);
16     outData[43:40] = TweakCellInvRot(inData[59:56]);
17     outData[47:44] = TweakCellInvRot(inData[19:16]);
18     outData[51:48] = inData[35:32];
19     outData[55:52] = inData[39:36];
20     outData[59:56] = inData[43:40];
21     outData[63:60] = TweakCellInvRot(inData[47:44]);
22     return outData;

```

E2.1.412 TweakShuffle

```

1  // TweakShuffle()
2  // =====
3
4  bits(64) TweakShuffle(bits(64) inData)
5      bits(64) outData;
6      outData[3:0] = inData[19:16];
7      outData[7:4] = inData[23:20];
8      outData[11:8] = TweakCellRot(inData[27:24]);
9      outData[15:12] = inData[31:28];
10     outData[19:16] = TweakCellRot(inData[47:44]);
11     outData[23:20] = inData[11:8];
12     outData[27:24] = inData[15:12];
13     outData[31:28] = TweakCellRot(inData[35:32]);
14     outData[35:32] = inData[51:48];
15     outData[39:36] = inData[55:52];
16     outData[43:40] = inData[59:56];
17     outData[47:44] = TweakCellRot(inData[63:60]);
18     outData[51:48] = TweakCellRot(inData[3:0]);
19     outData[55:52] = inData[7:4];
20     outData[59:56] = TweakCellRot(inData[43:40]);
21     outData[63:60] = TweakCellRot(inData[39:36]);
22     return outData;

```

E2.1.413 UnprivHaltingDebugEnabled

```

1  // UnprivHaltingDebugEnabled()
2  // =====
3
4  boolean UnprivHaltingDebugEnabled(boolean isSecure)
5      return UnprivHaltingDebugEnabled(isSecure) && !CurrentModeIsPrivileged(isSecure);

```

E2.1.414 UnprivHaltingDebugEnabled

```

1  // UnprivHaltingDebugEnabled()
2  // =====
3  // Returns TRUE if unprivileged halting debug is enabled for the indicated
4  // Security state, FALSE otherwise.
5
6  boolean UnprivHaltingDebugEnabled(boolean isSecure)
7      if !HaveUDE() || (!HaveSecurityExt() && isSecure) then
8          return FALSE;
9      elseif HaveSecurityExt() then
10         uiden = DAUTHCTRL_S.UIDEN == '1';
11         // Secure unprivileged debug also grants Non-secure unprivileged debug.
12         if !isSecure then uiden = uiden || DAUTHCTRL_NS.UIDEN == '1';
13         return uiden;
14     else
15         return DAUTHCTRL.UIDEN == '1';

```

E2.1.415 UnsignedSat

```

1 // UnsignedSat()
2 // =====
3
4 bits(N) UnsignedSat(integer i, integer N)
5     (result, -) = UnsignedSatQ(i, N);
6     return result;

```

E2.1.416 UnsignedSatQ

```

1 // UnsignedSatQ()
2 // =====
3
4 (bits(N), boolean) UnsignedSatQ(integer i, integer N)
5     if i > 2^N - 1 then
6         result = 2^N - 1; saturated = TRUE;
7     elseif i < 0 then
8         result = 0; saturated = TRUE;
9     else
10        result = i; saturated = FALSE;
11    return (result[N-1:0], saturated);

```

E2.1.417 UpdateDebugEnable

```

1 // UpdateDebugEnable()
2 // =====
3 // Update DHCSR.S_SDE, DEMCR.SDME, and unprivileged debug enables for each instruction.
4
5 UpdateDebugEnable()
6     // DHCSR.S_SDE and unprivileged debug enables are frozen if the PE is in Debug state.
7     if !Halted then
8         nsUide = UnprivHaltingDebugAllowed(FALSE);
9         sUide = UnprivHaltingDebugAllowed(TRUE);
10        DHCSR.S_SDE = if sUide || SecureHaltingDebugAllowed() then '1' else '0';
11        DHCSR.S_SUIDE = if sUide && !SecureHaltingDebugAllowed() then '1' else '0';
12        DHCSR.S_NSUIDE = if nsUide && !HaltingDebugAllowed() then '1' else '0';
13
14    // DEMCR.SDME is frozen if DebugMonitor is active or pending.
15    if HaveDebugMonitor() &&
16        ExceptionActive[DebugMonitor] == '00' &&
17        DEMCR.MON_PEND == '0' then
18        DEMCR.SDME = if SecureDebugMonitorAllowed() then '1' else '0';

```

E2.1.418 UpdateFPCCR

```

1 // UpdateFPCCR()
2 // =====
3
4 UpdateFPCCR(bits(32) frameptr, boolean applySpLim)
5     assert(HaveMveOrFPExt());
6
7     FPCAR.ADDRESS = frameptr[31:3];
8     // Flag if the context address violates the stack pointer limit. If the
9     // limit has been violated PreserveFPState() will zero the registers if
10    // required, but will not save the context to the stack.
11    if applySpLim && ViolatesSPLim(LookUpSP(), frameptr) then
12        FPCCR.SPLIMVIOL = '1';
13    else
14        FPCCR.SPLIMVIOL = '0';
15    FPCCR.LSPACT = '1';
16
17    execPri = ExecutionPriority();
18    isSecure = IsSecure();
19    FPCCR_S.S = if isSecure then '1' else '0';
20    if CurrentModeIsPrivileged() then
21        FPCCR.USER = '0';

```

```

22     else
23         FPCCR.USER = '1';
24     if CurrentMode() == PMode_Thread then
25         FPCCR.THREAD = '1';
26     else
27         FPCCR.THREAD = '0';
28     if execPri > -1 then
29         FPCCR_S.HFRDY = '1';
30     else
31         FPCCR_S.HFRDY = '0';
32     targetSecure = AIRCR.BFHFNMINS == '0';
33     busfaultpri = ExceptionPriority(BusFault, targetSecure, FALSE);
34     if SHCSR_S.BUSFAULTENA == '1' && execPri > busfaultpri then
35         FPCCR_S.BFRDY = '1';
36     else
37         FPCCR_S.BFRDY = '0';
38     memfaultpri = ExceptionPriority(MemManage, isSecure, FALSE);
39     if SHCSR_S.MEMFAULTENA == '1' && execPri > memfaultpri then
40         FPCCR_S.MMRDY = '1';
41     else
42         FPCCR_S.MMRDY = '0';
43     usagefaultpri = ExceptionPriority(UsageFault, FALSE, FALSE);
44     if SHCSR_NS.USGFAULTENA == '1' && execPri > usagefaultpri then
45         FPCCR_NS.UFRDY = '1';
46     else
47         FPCCR_NS.UFRDY = '0';
48     usagefaultpri = ExceptionPriority(UsageFault, TRUE, FALSE);
49     if SHCSR_S.USGFAULTENA == '1' && execPri > usagefaultpri then
50         FPCCR_S.UFRDY = '1';
51     else
52         FPCCR_S.UFRDY = '0';
53     if HaveSecurityExt() then
54         securefaultpri = ExceptionPriority(SecureFault, TRUE, FALSE);
55         if SHCSR_S.SECUREFAULTENA == '1' && execPri > securefaultpri then
56             FPCCR_S.SFRDY = '1';
57         else
58             FPCCR_S.SFRDY = '0';
59     if CanPendMonitorOnEvent(isSecure, TRUE, TRUE, FindPriv(), TRUE) then
60         FPCCR_S.MONRDY = '1';
61     else
62         FPCCR_S.MONRDY = '0';
63     return;

```

E2.1.419 ValidateAddress

```

1  // ValidateAddress()
2  // =====
3
4  (ExcInfo, AddressDescriptor) ValidateAddress(bits(32) address, AccType acctype,
5                                             boolean ispriv, boolean secure,
6                                             boolean iswrite, boolean aligned)
7
8  AddressDescriptor result;
9  Permissions        perms;
10 ns                 = boolean UNKNOWN;
11 excInfo             = DefaultExcInfo();
12
13 // Security checking and MPU bank selection if Security Extensions are present.
14 if HaveSecurityExt() then
15     // Check SAU and IDAU for given address.
16     isInstrfetch = acctype == AccType_IFETCH;
17     sAttrib       = SecurityCheck(address, isInstrfetch, secure);
18     if isInstrfetch then
19         ns = sAttrib.ns;
20         secureMpu = !sAttrib.ns;
21         // Override the privilege flag supplied with the a value based on the
22         // privilege associated with the current mode and the Security state
23         // of the MPU being queried. This can be different from the value this
24         // function is called with, because CONTROL.nPRIV is banked between

```

```

24         // Security states.
25         ispriv = CurrentModeIsPrivileged(secureMpu);
26     else
27         ns      = !secure || sAttrib.ns;
28         secureMpu = secure;
29     else
30         ns      = TRUE;
31         secureMpu = FALSE;
32
33     // Get memory attribute information from MPU. Note that NS information
34     // in the memory attribute is set by SAU or IDAU and is updated after getting
35     // attribute values from MPU.
36     (result.memattrs, perms) = MPUCheck(address, acctype, ispriv, secureMpu);
37     // Updating NS information got from SAU or IDAU memory attributes.
38     result.memattrs.NS      = ns;
39
40     // Generate UNALIGNED UsageFault exception if access to Device memory is unaligned.
41     if !aligned && result.memattrs.memtype == MemType_Device && perms.apValid == TRUE then
42         if acctype != AccType_DBG then
43             if secure then
44                 UFSR_S.UNALIGNED = '1';
45             else
46                 UFSR_NS.UNALIGNED = '1';
47             excInfo = CreateException(UsageFault, FALSE, secure);
48
49     if excInfo.fault == NoFault && HaveSecurityExt() then
50         // Check if there is a SAU or IDAU violation and, if so, update the fault syndrome.
51         case acctype of
52             when AccType_IFETCH
53                 if secure then
54                     if sAttrib.ns then
55                         // Invalid exit from the Secure state
56                         SFSR.INVTRAN = '1';
57                         excInfo      = CreateException(SecureFault);
58                     else
59                         if !sAttrib.ns && !sAttrib.nsc then
60                             // Invalid entry to the Secure state
61                             SFSR.INVEP = '1';
62                             excInfo    = CreateException(SecureFault);
63             when AccType_VECTABLE
64                 if !secure && !sAttrib.ns then
65                     HFSR.VECTTBL = '1';
66                     // Vector fetch faults raise a HardFault directly, but because this
67                     // fault is caused by an SAU or IDAU violation it always targets
68                     // the Secure state.
69                     excInfo      = CreateException(HardFault, TRUE, TRUE);
70             when AccType_DBG
71                 if !secure && !sAttrib.ns then
72                     // DAP accesses result in a error being returned to the DAP without any
73                     // syndrome being set.
74                     excInfo      = CreateException(SecureFault);
75             when AccType_NORMAL, AccType_MVE, AccType_ORDERED, AccType_STACK
76                 if !secure && !sAttrib.ns then
77                     SFSR.AUVIOL = '1';
78                     SFSR.SFARVALID = '1';
79                     SFAR          = address;
80                     excInfo      = CreateException(SecureFault);
81             when AccType_LAZYFP
82                 if !secure && !sAttrib.ns then
83                     SFSR.LSPERR = '1';
84                     SFSR.SFARVALID = '1';
85                     SFAR          = address;
86                     excInfo      = CreateException(SecureFault);
87             otherwise
88                 assert(FALSE);
89
90     result.paddress      = address;
91     result.accattrs.iswrite = iswrite;
92     result.accattrs.ispriv  = ispriv;

```

```

93     result.accattrs.acctype = acctype;
94
95     if excInfo.fault == NoFault then
96         excInfo = CheckPermission(perms, address, acctype, iswrite, ispriv, secureMpu);
97
98     return (excInfo, result);

```

E2.1.420 ValidateExceptionReturn

```

1  // ValidateExceptionReturn()
2  // =====
3
4  (ExcInfo, EXC_RETURN_Type) ValidateExceptionReturn(EXC_RETURN_Type excReturn,
5                                                    integer returningExceptionNumber)
6
7  boolean error          = FALSE;
8  assert CurrentMode() == PMode_Handler;
9  if !IsOnes(excReturn[23:7]) || excReturn[1] != '0' then
10     UNPREDICTABLE;
11  if !HaveMveOrFPExt() && excReturn.FType == '0' then
12     UNPREDICTABLE;
13  if !HaveSecurityExt() && (excReturn.S      == '1' ||
14                          excReturn.ES     == '1' ||
15                          excReturn.DCRS == '0') then
16
17     UNPREDICTABLE;
18
19  // Security specific validation
20  if HaveSecurityExt() then
21     // If exception return is an invalid attempt to return from Non-secure
22     // state with EXC_RETURN.ES set as '1', then a SecureFault is raised.
23     exceptionWasSecure = excReturn.ES == '1';
24     if CurrentState == SecurityState_NonSecure && excReturn.ES == '1' then
25         error          = TRUE;
26         // excReturn.ES is used below to control which exception to
27         // deactivate, and which CONTROL.SPSEL to update. Force it to the
28         // correct value so the code below functions correctly even if the
29         // Non-secure state returned an invalid excReturn value.
30         // Similarly the exception to deactivate below is actually Non-secure.
31         excReturn.ES     = '0';
32         exceptionWasSecure = FALSE;
33
34     // Check DCRS bit not used in for Non-secure exceptions.
35     if !exceptionWasSecure && excReturn.DCRS == '0' then
36         error = TRUE;
37
38     if error then
39         SFSR.INVER = '1';
40         exceptionNumber = SecureFault;
41     else
42         exceptionWasSecure = FALSE;
43
44     // Check returning from an inactive handler.
45     if !error then
46         if !IsActiveForState(returningExceptionNumber, exceptionWasSecure) then
47             error = TRUE;
48             if HaveMainExt() then
49                 UFSR.INVPC = '1';
50                 exceptionNumber = UsageFault;
51             else
52                 exceptionNumber = HardFault;
53
54     if error then
55         DeActivate(returningExceptionNumber, exceptionWasSecure);
56         if HaveSecurityExt() && exceptionWasSecure then
57             CONTROL_S.SPSEL = excReturn.SPSEL;
58         else
59             CONTROL_NS.SPSEL = excReturn.SPSEL;
60         // Escalates to HardFault if requested fault is disabled, or has
61         // insufficient priority, or if Main Extension is not implemented.

```

```

60     excInfo = CreateException(exceptionNumber, FALSE, IsSecure());
61     else
62         excInfo = DefaultExcInfo();
63     return (excInfo, excReturn);

```

E2.1.421 ValidatePAC

```

1  // ValidatePAC()
2  // =====
3  // Returns TRUE if the pointer authentication code in Z matches the pointer authentication
4  // code derived using a cryptographic algorithm as a combination of X, Y and the
5  // cryptographic key. The correct cryptographic key is selected in CreatePAC(), on the
6  // basis of the current Security state and privilege level.
7
8  boolean ValidatePAC(bits(32) X, bits(32) Y, bits(32) Z)
9      return Z == CreatePAC(X, Y);

```

E2.1.422 VCX_op0

```

1  // VCX_op0
2  // =====
3
4  bits(size) VCX_op0(bits(32) instr, integer size)
5      return VCX_op0(instr, size, FALSE, integer UNKNOWN, bits(4) UNKNOWN);
6
7  bits(size) VCX_op0(bits(32) instr, integer size, boolean isBeatWise, integer curBeat,
8                  bits(4) elmtMask)
9      assert size IN {32, 64};
10
11     // Custom data path returning IMPLEMENTATION DEFINED value based on
12     // instruction opcode only.
13     if isBeatWise then
14         return CdeImpDefValue(instr, curBeat, elmtMask);
15     else
16         return CdeImpDefValue(instr);

```

E2.1.423 VCX_op1

```

1  // VCX_op1
2  // =====
3
4  bits(size) VCX_op1(bits(32) instr, bits(N) opa, integer size)
5      return VCX_op1(instr, opa, size, FALSE, integer UNKNOWN, bits(4) UNKNOWN);
6
7  bits(size) VCX_op1(bits(32) instr, bits(N) opa, integer size, boolean isBeatWise,
8                  integer curBeat, bits(4) elmtMask)
9      assert N IN {32, 64, 128};
10     assert size IN {32, 64, 128};
11
12     // Custom data path returning IMPLEMENTATION DEFINED value based on instruction
13     // opcode and single 32-bit, 64-bit, or 128-bit operand, opa, only.
14     if isBeatWise then
15         return CdeImpDefValue(instr, opa, curBeat, elmtMask);
16     else
17         return CdeImpDefValue(instr, opa);

```

E2.1.424 VCX_op2

```

1  // VCX_op2
2  // =====
3
4  bits(size) VCX_op2(bits(32) instr, bits(N) opa, bits(N) opb, integer size)
5      return VCX_op2(instr, opa, opb, size, FALSE, integer UNKNOWN, bits(4) UNKNOWN);
6

```

```

7  bits(size) VCX_op2(bits(32) instr, bits(N) opa, bits(N) opb, integer size,
8      boolean isBeatWise, integer curBeat, bits(4) elmtMask)
9      assert N      IN {32, 64, 128};
10     assert size IN {32, 64, 128};
11
12     // Custom data path returning IMPLEMENTATION DEFINED value based on instruction
13     // opcode and two 32-bit or 64-bit operands, opa and opb, only.
14     if isBeatWise then
15         return CdeImpDefValue(instr, opa, opb, curBeat, elmtMask);
16     else
17         return CdeImpDefValue(instr, opa, opb);

```

E2.1.425 VCX_op3

```

1  // VCX_op3
2  // =====
3
4  bits(size) VCX_op3(bits(32) instr, bits(N) opa, bits(N) opb, bits(N) opc, integer size)
5      return VCX_op3(instr, opa, opb, opc, size, FALSE, integer UNKNOWN, bits(4) UNKNOWN);
6
7  bits(size) VCX_op3(bits(32) instr, bits(N) opa, bits(N) opb, bits(N) opc, integer size,
8      boolean isBeatWise, integer curBeat, bits(4) elmtMask)
9      assert N      IN {32, 64, 128};
10     assert size IN {32, 64, 128};
11
12     // Custom data path returning IMPLEMENTATION DEFINED value based on instruction
13     // opcode and three 32-bit, 64-bit, or 128-bit operands, opa, opb and opc, only.
14     if isBeatWise then
15         return CdeImpDefValue(instr, opa, opb, opc, curBeat, elmtMask);
16     else
17         return CdeImpDefValue(instr, opa, opb, opc);

```

E2.1.426 Vector

```

1  // Vector[]
2  // =====
3
4  (ExcInfo, bits(32)) Vector(integer exceptionNumber, boolean isSecure)
5      // Calculate the address of the entry in the vector table.
6      vtor = if isSecure then VTOR_S else VTOR_NS;
7      addr = (vtor.TBLOFF:'0000000') + 4 * exceptionNumber;
8      // Fetch the vector with the correct privilege and security
9      (exc, vector) = MemA_with_priv_security(addr, 4, AccType_VECTABLE,
10          Privilege_Priv, isSecure, TRUE);
11      // Faults that prevent the vector being fetched are terminal and prevent
12      // the exception handler from being entered.
13      if exc.fault != NoFault then
14          exc.isTerminal = TRUE;
15      return (exc, vector);

```

E2.1.427 VectorCatchDebug

```

1  // VectorCatchDebug()
2  // =====
3
4  VectorCatchDebug(boolean isSecure, boolean isPriv)
5      vectorEvt = FALSE;
6
7      if CanHaltOnEvent(isSecure, isPriv) then
8          case UInt(IPSr.Exception) of
9              when HardFault
10                 vectorEvt = ((DEMCr.VC_HARDERR == '1' && (HFSr.FORCED      == '1' ||
11                     HFSr.DEBUGEVT == '1')) ||
12                     (DEMCr.VC_INTERR == '1' && (HFSr.VECTBL == '1')));
13

```

```

14         when MemManage
15             vectorEvt = ((DEMCR.VC_MMERR == '1' && (MMFSR.IACCVIOL == '1' ||
16                                                         MMFSR.DACCVIOL == '1')) ||
17                         (DEMCR.VC_INTERR == '1' && (MMFSR.MSTKERR == '1' ||
18                                                         MMFSR.MUNSTKERR == '1' ||
19                                                         MMFSR.MLSPERR == '1'))));
20
21         when BusFault
22             vectorEvt = ((DEMCR.VC_BUSERR == '1' && (BFSR.IBUSERR == '1' ||
23                                                         BFSR.PRECISERR == '1' ||
24                                                         BFSR.IMPRESISERR == '1')) ||
25                         (DEMCR.VC_INTERR == '1' && (BFSR.STKERR == '1' ||
26                                                         BFSR.UNSTKERR == '1' ||
27                                                         BFSR.LSPERR == '1'))));
28
29         when UsageFault
30             vectorEvt = ((DEMCR.VC_STATERR == '1' && (UFSR.UNDEFINSTR == '1' ||
31                                                         UFSR.INVPC == '1' ||
32                                                         UFSR.INVSTATE == '1')) ||
33                         (DEMCR.VC_CHKERR == '1' && (UFSR.UNALIGNED == '1' ||
34                                                         UFSR.DIVBYZERO == '1')) ||
35                         (DEMCR.VC_INTERR == '1' && (UFSR.STKOF == '1')) ||
36                         (DEMCR.VC_NOCPERR == '1' && (UFSR.NOCP == '1'))));
37
38         when SecureFault
39             vectorEvt = ((DEMCR.VC_SFERR == '1' && (SFSR.INVEP == '1' ||
40                                                         SFSR.INVIS == '1' ||
41                                                         SFSR.INVER == '1' ||
42                                                         SFSR.AUVIOL == '1' ||
43                                                         SFSR.INVTRAN == '1' ||
44                                                         SFSR.LSPERR == '1' ||
45                                                         SFSR.LSERR == '1'))));
46
47         otherwise
48             // No other exceptions trigger vector catch
49
50         if vectorEvt then
51             DHCSR.C_HALT = '1';
52             DFSR.VCATCH = '1';

```

E2.1.428 VFPExcBarrier

```

1 // VFPExcBarrier
2 // =====
3 // Ensures that all floating-point exception processing has completed.
4
5 VFPExcBarrier();

```

E2.1.429 VFPEExpandImm

```

1 // VFPEExpandImm()
2 // =====
3
4 bits(N) VFPEExpandImm(bits(8) imm8, integer N)
5     assert N IN {16, 32, 64};
6     integer E = if N == 16 then 5 elsif N == 32 then 8 else 11;
7     constant integer F = N - E - 1;
8     bits(E-2) exp;
9     bits(F+2) frac;
10    sign = imm8[7];
11    exp = NOT(imm8[6]) : Replicate(imm8[6], E-3);
12    frac = imm8[5:0] : Zeros(F-4);
13    return sign : exp : frac;

```

E2.1.430 VFPNegMul


```

1 // Different types of floating-point multiply and negate operations
2
3 enumeration VFPNegMul {VFPNegMul_VNMLA, VFPNegMul_VNMLS, VFPNegMul_VNMUL};

```

E2.1.431 VFPSmallRegisterBank

```

1 // VFPSmallRegisterBank()
2 // =====
3 // Returns TRUE because the floating-point implementation only provides access to
4 // 16 double-precision registers
5
6 boolean VFPSmallRegisterBank()
7     return TRUE;

```

E2.1.432 ViolatesSPLim

```

1 // ViolatesSPLim()
2 // =====
3
4 boolean ViolatesSPLim(RNames spreg, bits(32) value)
5     isSecure = ((spreg == RNamesSP_Main_Secure) || (spreg == RNamesSP_Process_Secure));
6
7     // Check CCR.STKOFHFNMI to determine if the limit should actually be
8     // applied. When checking if CCR.STKOFHFNMI should apply the requested
9     // execution priority is considered, and AIRCR.PRIS is ignored.
10    assert (!isSecure || HaveSecurityExt());
11    if HaveMainExt() && IsReqExcPriNeg(isSecure) then
12        ignLimit = if isSecure then CCR_S.STKOFHFNMI else CCR_NS.STKOFHFNMI;
13        applyLimit = (ignLimit == '0');
14    else
15        applyLimit = TRUE;
16
17    return applyLimit && (UInt(value) < UInt(LookUpSPLim(spreg)));

```

E2.1.433 VPTActive

```

1 // VPTActive()
2 // =====
3
4 boolean VPTActive()
5     return VPTActive(_BeatID);
6
7 boolean VPTActive(integer beat)
8     return Elem[VPR[23:16], beat DIV 2, 4] != Zeros(4);

```

E2.1.434 VPTAdvance

```

1 // VPTAdvance()
2 // =====
3
4 VPTAdvance(integer beat)
5     maskID = beat DIV 2;
6     vptState = Elem[VPR[23:16], maskID, 4];
7     if vptState == '1000' then
8         vptState = Zeros(4);
9     elseif vptState != '0000' then
10        (vptState, inv) = LSL_C(vptState, 1);
11        // Invert the predicate flags for this beat if the bit shifted out of
12        // the VPT state was 1.
13        if inv == '1' then
14            Elem[VPR.P0, beat, 4] = NOT Elem[VPR.P0, beat, 4];
15        // Since the mask fields are grouped in pairs only update the mask on every
16        // odd numbered beat.
17        if beat[0] == '1' then
18            Elem[VPR[23:16], maskID, 4] = vptState;

```

E2.1.435 WaitForEvent

```

1 // WaitForEvent
2 // =====
3 // Optionally suspends execution until a WFE wakeup event or reset occurs,
4 // or until some earlier time if the implementation chooses.
5
6 WaitForEvent();

```

E2.1.436 WaitForInterrupt

```

1 // WaitForInterrupt
2 // =====
3 // Optionally suspends execution until a WFI wakeup event or reset occurs, or
4 // until some earlier time if the implementation chooses.
5
6 WaitForInterrupt();

```

E2.1.437 ZeroExtend

```

1 // ZeroExtend()
2 // =====
3
4 bits(N) ZeroExtend(bits(M) x, integer N)
5     assert N >= M;
6     return Zeros(N-M) : x;
7
8 bits(N) ZeroExtend(bits(M) x)
9     return ZeroExtend(x, N);

```

E2.1.438 Zeros

```

1 // Zeros()
2 // =====
3
4 bits(N) Zeros(integer N)
5     return Replicate('0', N);
6
7 bits(N) Zeros()
8     return Zeros(N);

```

Part F

Debug Packet Protocols

Chapter F1

ITM and DWT Packet Protocol Specification

This chapter describes the protocol for packets that send the data generated by the ITM and DWT to an external debugger. It contains the following sections:

- [About the ITM and DWT packets.](#)
- [Alphabetical list of DWT and ITM packets.](#)

F1.1 About the ITM and DWT packets

The following sections give an overview of the ITM and DWT packets and how the TPIU transmits them:

- [Uses of ITM and DWT packets](#)
- [ITM and DWT protocol packet headers](#)
- [Packet transmission by the trace sink](#)

Note

This chapter describes packet transmission by a trace sink such as a TPIU. The ITM can send packets to any suitable trace sink. Regardless of the actual trace sink used, the ITM formats the packets as described in this chapter.

F1.1.1 Uses of ITM and DWT packets

The ITM sends a packet to the trace sink when:

- Software writes to a stimulus register. This generates a [Instrumentation packet](#).
- The hardware generates a Protocol packet. Protocol packets include timestamps and synchronization packets.
- It receives a packet from the DWT, for forwarding to the trace sink.

The DWT sends a packet to the ITM for forwarding to the trace sink when:

- A DWT comparator matches and generates one or more Data Trace packets.
- It samples the PC.
- One of the performance profile counters wraps.

This chapter describes the packet protocol used.

F1.1.2 ITM and DWT protocol packet headers

[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]	Description
0	0	0	0	0	0	0	0	Synchronization packet
0	1	1	1	0	0	0	0	Overflow packet
0	$\neq 0b000$ && $\neq 0b111$			0	0	0	0	Local Timestamp 2 packet
1	0	0	1	0	1	0	0	Global Timestamp 1 packet
1	0	1	1	0	1	0	0	Global Timestamp 2 packet
1	1	x	x	0	0	0	0	Local Timestamp 1 packet
x	x	x	x	1	x	0	0	Extension Packet
0	0	0	0	0	1	0	1	Event Counter Packet
0	1	x	x	0	1	0	1	Date Trace Match Packet
0	0	0	0	1	1	1	0	Exception Trace Packet
0	1	x	x	0	1	$\neq 0b00$		Data Trace PC Value packet
0	1	x	x	1	1	$\neq 0b00$		Data Trace Data Address packet
1	0	x	x	x	1	$\neq 0b00$		Data Trace Data Value packet
x	x	x	x	x	0	$\neq 0b00$		Instrumentation packet
0	0	0	1	0	1	x	1	Periodic PC Sample packet
0	0	0	1	1	1	0	1	PMU Overflow packet

F1.1.3 Packet transmission by the trace sink

The trace sink either:

- Forms the packets into frames, as required by the *Arm@CoreSight™ Architecture Specification*.
- Transmits the packets over a serial port.

For each packet, the trace sink transmits:

- The header byte first, followed by any payload bytes.
- Each byte of the packet *least significant bit* (LSB) first.

Figures in this chapter show each packet as a sequence of bytes, with the LSB of each byte to the right and the *most significant bit* (MSB) to the left. [Convention for packet descriptions](#) shows this convention, and how it relates to data transmission for a packet with a header byte and two payload bytes.

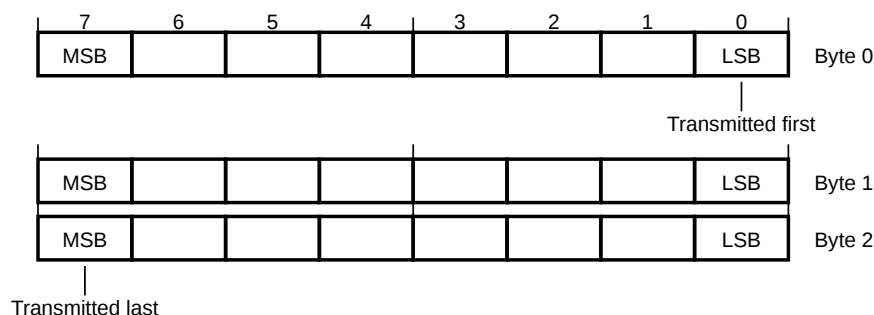


Figure F1.1: Convention for packet descriptions

In some sections, the figures are split into separate figures for the header byte and payload bytes. For instance, where the number of payload bytes varies according to a field in the header.

The ITM merges the packets from the ITM and DWT with the Local and Global timestamp, Synchronization, and other Protocol packets, and forwards them to the trace sink as a single data stream. The trace sink then merges this data stream with the data from the ETM, if implemented.

Arm reserves the right to add more trace packets in the future and makes no guarantees as to their format. When encountering an unknown packet, trace decoders should wait for a Synchronization packet before restarting their decode.

F1.2 Alphabetical list of DWT and ITM packets

F1.2.1 Data Trace Data Address packet

The Data Trace Data Address packet characteristics are:

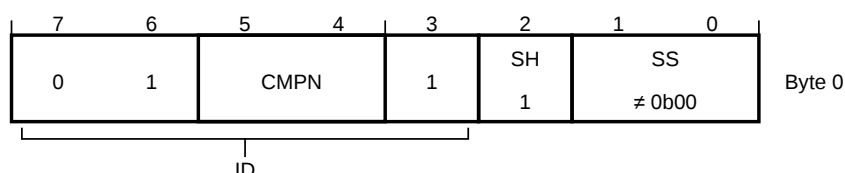
Purpose Indicates a DWT comparator generated a match, and the address that matched. Data Address packets are only generated for Data Address range comparator pairs. The address might be compressed. However, it is not required that Short and Medium packets are generated when the address bits match.

Attributes Multi-part Hardware source packet comprising:

- 8-bit header.
- 8, 16, or 32-bit payload.

F1.2.1.1 Data Trace Data Address packet header

The Data Trace Data Address packet header bit assignments are:



ID, byte 0 bits [7:3] Hardware Source packet type. Bits [7:3] discriminate between Hardware Source packet types. The defined values of this field are:

0b01xx1 Data Trace Data Address packet.

This field reads as 0b01xx1.

CMPN, byte 0 bits [5:4] DWT comparator index. Defines which comparator generated a match. Data Trace Data Address packets can be compressed relative to the value in DWT_COMP<CMPN>. The number of traced bits is indicated by the SS field. The remainder of the address bits comes from DWT_COMP<CMPN>. Either comparator in a Data Address range comparator pair can be used.

SH, byte 0 bit [2] Source. The defined values of this bit are:

1 Hardware source packet.

This bit reads as one.

SS, byte 0 bits [1:0] Size. The defined values of this field are:

0b01 Short Data Address packet.

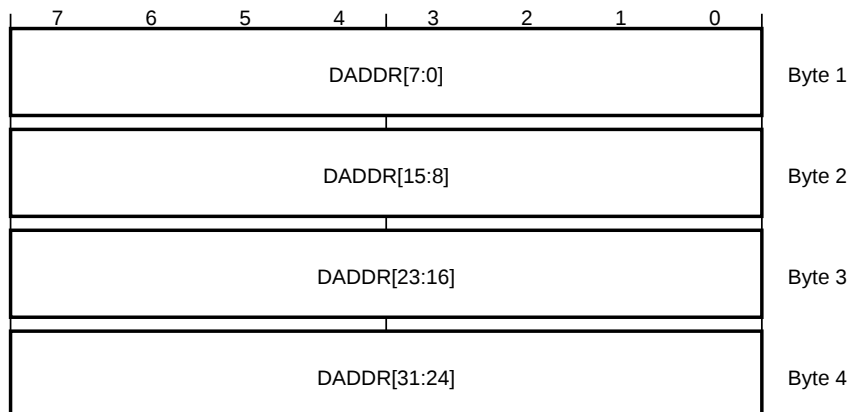
0b10 Medium Data Address packet.

0b11 Long Data Address packet.

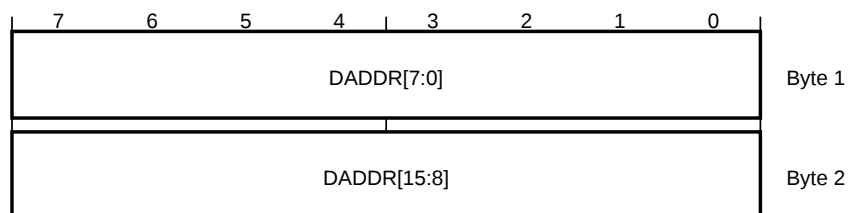
The value 0b00 encodes a Protocol packet.

F1.2.1.2 Data Trace Data Address packet payload

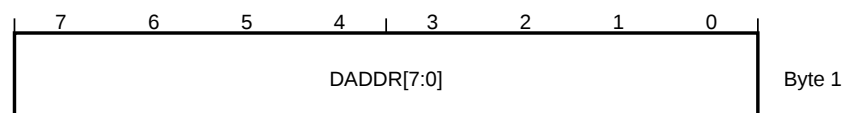
When Long Data Address packet, SS == 0b11, the Data Trace Data Address packet payload bit assignments are:



When Medium Data Address packet, SS == 0b10, the Data Trace Data Address packet payload bit assignments are:



When Short Data Address packet, SS == 0b01, the Data Trace Data Address packet payload bit assignments are:



DADDR[31:0], bytes <4:1>, when Long Data Address packet, SS == 0b11 Data address.

DADDR[15:0], bytes <2:1>, when Medium Data Address packet, SS == 0b10 Data address. DADDR[31:16] == DWT_COMP<CMPN>[31:16].

DADDR[7:0], byte <1>, when Short Data Address packet, SS == 0b01 Data address. DADDR[31:8] == DWT_COMP<CMPN>[31:8].

F1.2.2 Data Trace Data Value packet

The Data Trace Data Value packet characteristics are:

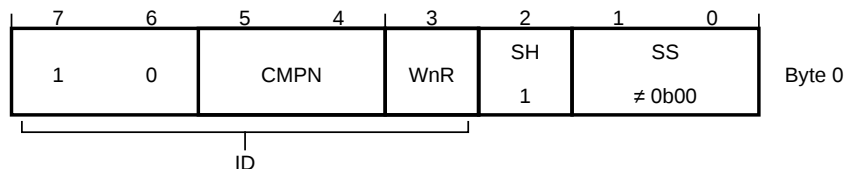
Purpose Indicates a DWT comparator generated a match, and the value that matched.

Attributes Multi-part Hardware source packet comprising:

- 8-bit header.
- 8, 16, or 32-bit payload.

F1.2.2.1 Data Trace Data Value packet header

The Data Trace Data Value packet header bit assignments are:



ID, byte 0 bits [7:3] Hardware Source packet type. Bits [7:3] discriminate between Hardware Source packet types. The defined values of this field are:

0b10xxx Data Trace Data Value packet.

This field reads as 0b10xxx.

CMPN, byte 0 bits [5:4] DWT comparator index. Defines which comparator generated a match.

WnR, byte 0 bit [3] Write-not-read. The defined values of this bit are:

0 Read.

1 Write.

SH, byte 0 bit [2] Source. The defined values of this bit are:

1 Hardware source packet.

This bit reads as one.

SS, byte 0 bits [1:0] Size. The defined values of this field are:

0b01 Byte Data Value packet.

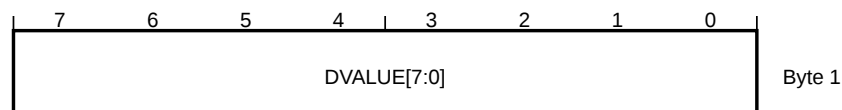
0b10 Halfword Data Value packet.

0b11 Word Data Value packet.

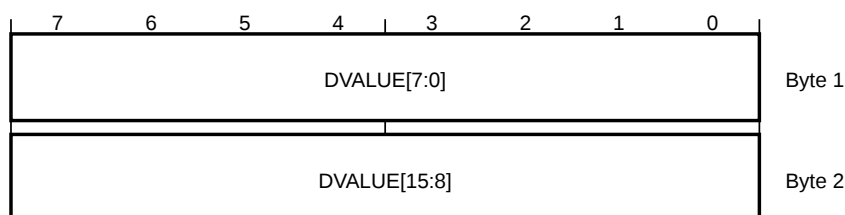
The value 0b00 encodes a Protocol packet.

F1.2.2.2 Data Trace Data Value packet payload

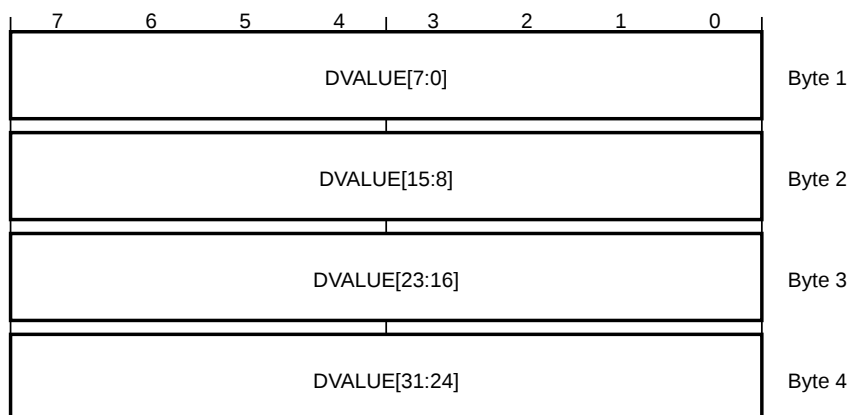
When Byte Data Value packet, SS == 0b01, the Data Trace Data Value packet payload bit assignments are:



When Halfword Data Value packet, SS == 0b10, the Data Trace Data Value packet payload bit assignments are:



When Word Data Value packet, SS == 0b11, the Data Trace Data Value packet payload bit assignments are:



DVALUE[31:0], bytes <4:1>, when Word Data Value packet, SS == 0b11 Word data value.

DVALUE[15:0], byte 1 bits [15:0], when Halfword Data Value packet, SS == 0b10 Halfword data value.

DVALUE[7:0], byte <1>, when Byte Data Value packet, SS == 0b01 Byte data value.

F1.2.3 Data Trace Match packet

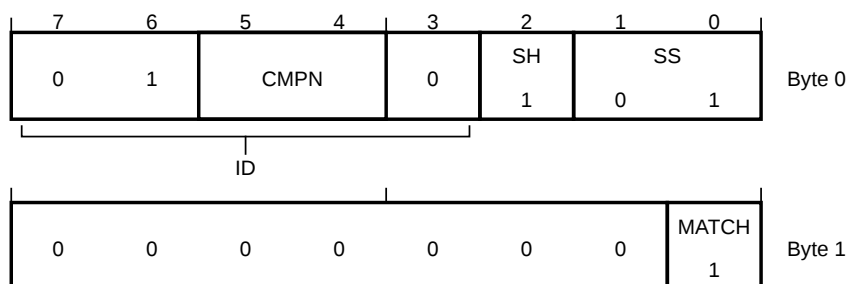
The Data Trace Match packet characteristics are:

Purpose Indicates a DWT comparator generated a match.

Attributes 16-bit Hardware source packet.

Field descriptions

The Data Trace Match packet bit assignments are:



Byte 1 bits [7:1] This field reads as 0b0000000.

MATCH, byte 1 bit [0] Data Trace Match packet. Discriminates between the [Data Trace PC Value packet](#) and the Data Trace Match packet. The defined values of this bit are:

1 Data Trace Match packet.

This bit reads as one.

ID, byte 0 bits [7:3] Hardware Source packet type. Bits [7:3] discriminate between Hardware Source packet types. The defined values of this field are:

0b01xx0 [Data Trace PC Value packet](#) or Data Trace Match packet.

Bit [0] of byte 1 discriminates between the [Data Trace PC Value packet](#) and the Data Trace Match packet.

This field reads as 0b01xx0.

CMPN, byte 0 bits [5:4] DWT comparator index. Defines which comparator generated a match.

SH, byte 0 bit [2] Source. The defined values of this bit are:

- 1 Hardware source packet.

This bit reads as one.

SS, byte 0 bits [1:0] Size. The defined values of this field are:

0b01 Source packet, 1-byte payload, 2-byte packet.

The value 0b00 encodes a Protocol packet. All other values are reserved.

This field reads as 0b01.

F1.2.4 Data Trace PC Value packet

The Data Trace PC Value packet characteristics are:

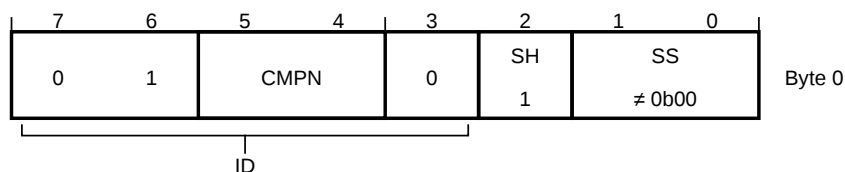
Purpose Indicates a DWT comparator generated a match, and the address of the instruction that matched. The address might be compressed. However, it is not required that Short and Medium packets are generated when the address bits match.

Attributes Multi-part Hardware source packet comprising:

- 8-bit header.
- 8, 16, or 32-bit payload.

F1.2.4.1 Data Trace PC Value packet header

The Data Trace PC Value packet header bit assignments are:



ID, byte 0 bits [7:3] Hardware Source packet type. Bits [7:3] discriminate between Hardware Source packet types. The defined values of this field are:

0b01xx0 Data Trace PC Value packet or [Data Trace Match packet](#).

Bit [0] of byte 1 discriminates between the Data Trace PC Value packet and the [Data Trace Match packet](#).

This field reads as 0b01xx0.

CMPN, byte 0 bits [5:4] DWT comparator index. Defines which comparator generated a match. Data Trace PC Value packets can be compressed relative to the value in DWT_COMP<CMPN>. The number of traced bits is indicated by the SS field. The remainder of the address bits comes from DWT_COMP<CMPN>. Either comparator in an Instruction Address range comparator pair can be used.

SH, byte 0 bit [2] Source. The defined values of this bit are:

1 Hardware source packet.

This bit reads as one.

SS, byte 0 bits [1:0] Size. The defined values of this field are:

0b01 Short PC Value packet.

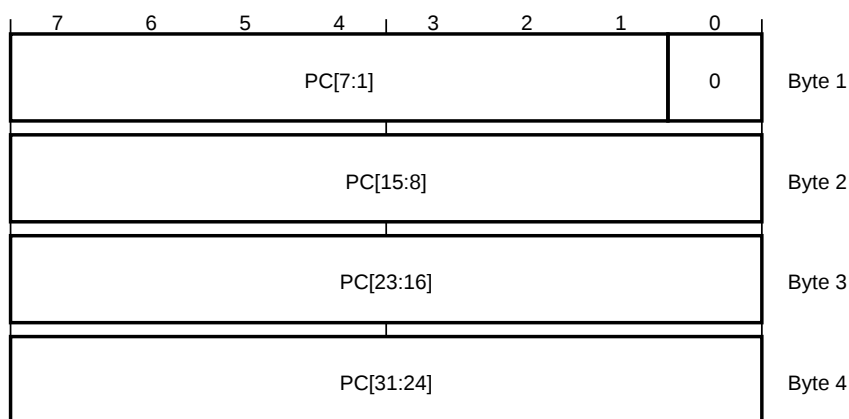
0b10 Medium PC Value packet.

0b11 Long PC Value packet.

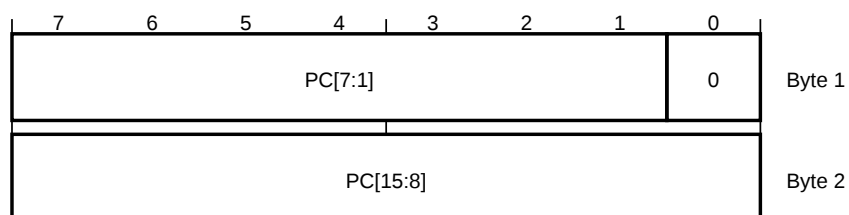
The value 0b00 encodes a Protocol packet.

F1.2.4.2 Data Trace PC Value packet payload

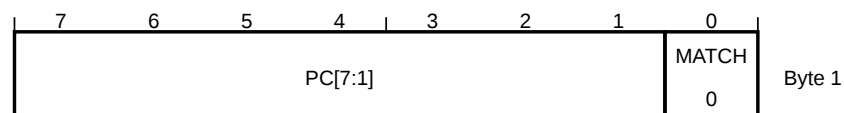
When Long PC Value packet, SS == 0b11, the Data Trace PC Value packet payload bit assignments are:



When Medium PC Value packet, SS == 0b10, the Data Trace PC Value packet payload bit assignments are:



When Short PC Value packet, SS == 0b01, the Data Trace PC Value packet payload bit assignments are:



PC[31:1], bytes <4:2>, byte 1 bits [7:1], when Long PC Value packet, SS == 0b11 Instruction address.

PC[15:1], byte <2>, byte 1 bits [7:1], when Medium PC Value packet, SS == 0b10 Instruction address.
PC[31:16] == DWT_COMP<CMPN>[31:16].

PC[7:1], byte 1 bits [7:1], when Short PC Value packet, SS == 0b01 Instruction address. PC[31:8] == DWT_COMP<CMPN>[31:8].

MATCH, byte 1 bit [0] [Data Trace Match packet](#). Discriminates between the Data Trace PC Value packet and the [Data Trace Match packet](#). The defined values of this bit are:

0 Data Trace PC Value packet.

This bit reads as zero.

F1.2.5 Event Counter packet

The Event Counter packet characteristics are:

Purpose Indicates one or more DWT counters wraps through zero.

Attributes 16-bit Hardware source packet.

Field descriptions

The Event Counter packet bit assignments are:

7	6	5	4	3	2	1	0	
ID				SH	SS			Byte 0
0	0	0	0	0	1	0	1	
0		0	Cyc	Fold	LSU	Sleep	Exc	CPI
								Byte 1

Byte 1 bits [7:6] This field reads-as-zero.

Cyc, byte 1 bit [5] POSTCNT timer decremented to zero. See DWT_CTRL for more information on the POSTCNT timer.

Fold, byte 1 bit [4] DWT_FOLDCNT counter wrapped from 0xFF to zero.

LSU, byte 1 bit [3] DWT_LSUNCT counter wrapped from 0xFF to zero.

Sleep, byte 1 bit [2] DWT_SLEEP CNT counter wrapped from 0xFF to zero.

Exc, byte 1 bit [1] DWT_EXCCNT counter wrapped from 0xFF to zero.

CPI, byte 1 bit [0] DWT_CPICNT counter wrapped from 0xFF to zero.

ID, byte 0 bits [7:3] Hardware Source packet type. Bits [7:3] discriminate between Hardware Source packet types. The defined values of this field are:

0b00000 Event Counter packet.

This field reads as 0b00000.

SH, byte 0 bit [2] Source. The defined values of this bit are:

1 Hardware source packet.

This bit reads as one.

SS, byte 0 bits [1:0] Size. The defined values of this field are:

0b01 Source packet, 1-byte payload, 2-byte packet.

The value 0b00 encodes a Protocol packet. All other values are reserved.

This field reads as 0b01.

F1.2.6 Exception Trace packet

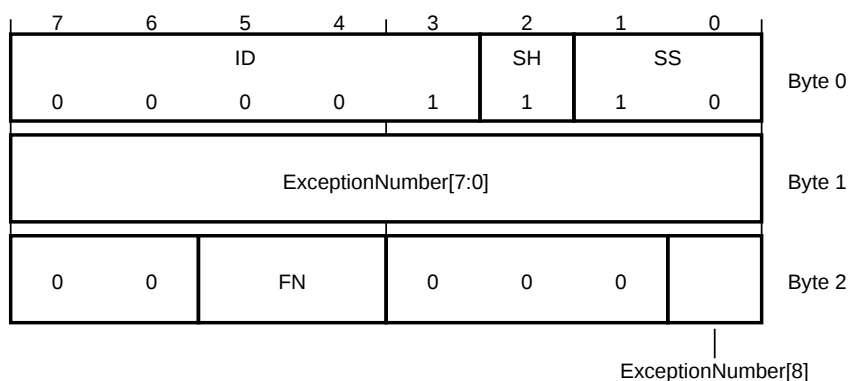
The Exception Trace packet characteristics are:

Purpose Indicates the PE has entered, exited or returned to an exception.

Attributes 24-bit Hardware source packet.

Field descriptions

The Exception Trace packet bit assignments are:



Byte 2 bits [7:6,3:1] This field reads-as-zero.

FN, byte 2 bits [5:4] Function. The defined values of this field are:

- 0b01** Entered exception indicated by ExceptionNumber.
 - 0b10** Exited exception indicated by ExceptionNumber.
 - 0b11** Returned to exception indicated by ExceptionNumber.
- All other values are reserved.

ExceptionNumber, byte 2 bit [0], byte <1> The exception number.

ID, byte 0 bits [7:3] Hardware Source packet type. Bits [7:3] discriminate between Hardware Source packet types. The defined values of this field are:

0b00001 Exception Trace packet.

This field reads as 0b00001.

SH, byte 0 bit [2] Source. The defined values of this bit are:

- 1** Hardware source packet.

This bit reads as one.

SS, byte 0 bits [1:0] Size. The defined values of this field are:

0b10 Source packet, 2-byte payload, 3-byte packet.

The value 0b00 encodes a Protocol packet. All other values are reserved.

This field reads as 0b10.

F1.2.7 Extension packet

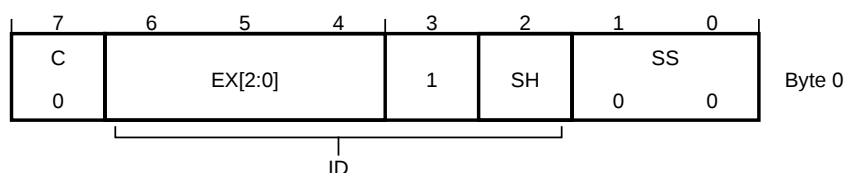
The Extension packet characteristics are:

Purpose An Extension packet provides additional information about the identified source. The amount of information required determines the number of payload bytes, 0-4. The architecture only defines one use of the Extension packet, to provide a Stimulus port page number. For this use, SH == 0, and a single byte Extension packet is emitted.

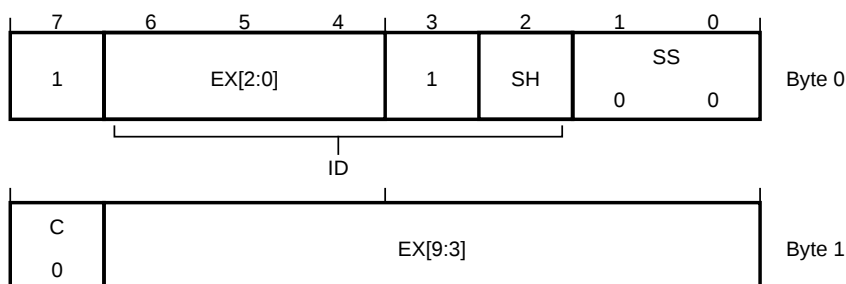
Attributes 8, 16, 24, 32, or 40-bit Protocol packet.

Field descriptions

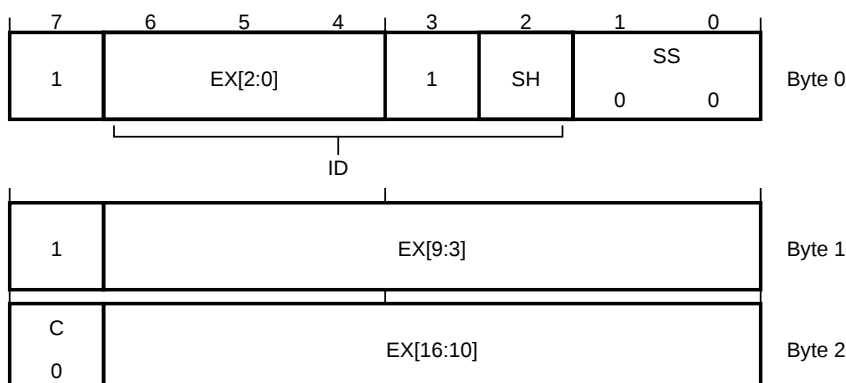
When 1-byte packet, the Extension packet bit assignments are:



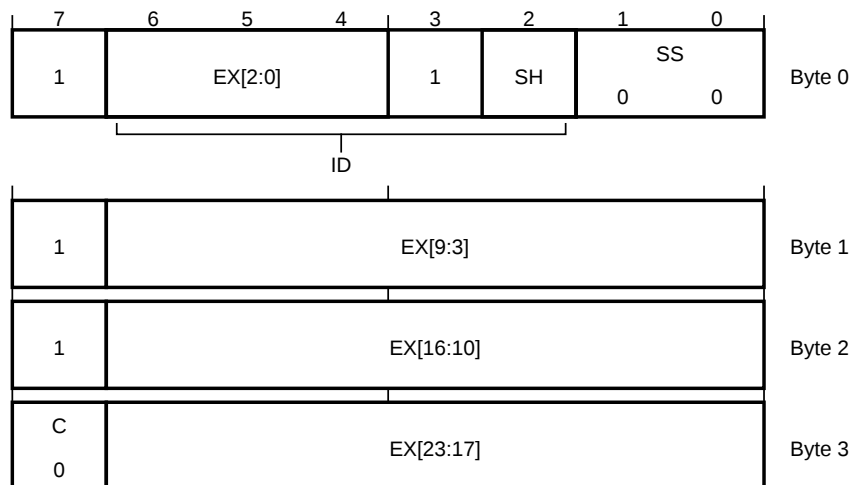
When 2-byte packet, the Extension packet bit assignments are:



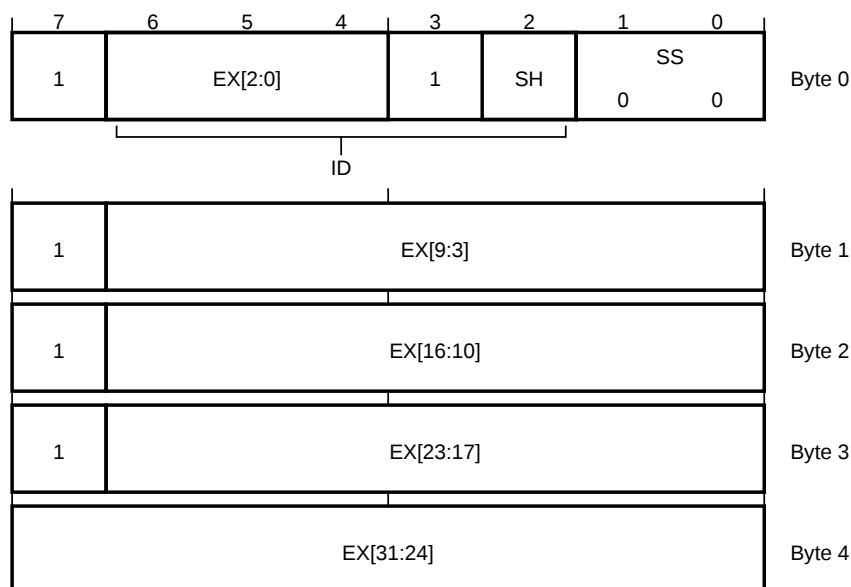
When 3-byte packet, the Extension packet bit assignments are:



When 4-byte packet, the Extension packet bit assignments are:



When 5-byte packet, the Extension packet bit assignments are:



EX, byte <4>, byte 3 bits [6:0], byte 2 bits [6:0], byte 1 bits [6:0], byte 0 bits [6:4] Extension information. If SH == 1, then EX defines PAGE, the Stimulus port page number.

This is a 32-bit field. If the Extension packet is shorter than 5 bytes, the most significant bits are zero.

C, byte 3 bit [7], byte 2 bit [7], byte 1 bit [7], byte 0 bit [7] Continuation bit. The defined values of this field are:

0 Last byte of the packet.

1 Another byte follows.

ID, byte 0 bits [6:2] Protocol packet type. Bits [6:2] discriminate between Protocol packet types. The defined values of this field are:

0bxxx1x Extension packet.

This field reads as 0bxxx1x.

SH, byte 0 bit [2] Source. The defined values of this bit are:

0 Extension packet for [Instrumentation packet](#).

1 Extension packet for Hardware source packet.

SS, byte 0 bits [1:0] Packet type. The defined values of this field are:

0b00 Protocol packet.

Other values encode different sizes of Hardware and Software source packets. This field reads as 0b00.

F1.2.8 Global Timestamp 1 packet

The Global Timestamp 1 packet characteristics are:

Purpose Contains the least significant bits of the global timestamp value. The ITM might compress this value if it is not generating a full timestamp by omitting significant bits if they are unchanged from the previous timestamp value.

Attributes Multi-part Protocol packet comprising:

- 8-bit header.
- 8, 16, 24, or 32-bit payload.

F1.2.8.1 Global Timestamp 1 packet header

The Global Timestamp 1 packet header bit assignments are:

7	6	5	4	3	2	1	0	
C	ID				SS			Byte 0
1	0	0	1	0	1	0	0	

C, byte 0 bit [7] Continuation bit. This bit reads as one.

ID, byte 0 bits [6:2] Protocol packet type. Bits [6:2] discriminate between Protocol packet types. The defined values of this field are:

0b00101 Global Timestamp 1 packet.

This field reads as 0b00101.

SS, byte 0 bits [1:0] Packet type. The defined values of this field are:

0b00 Protocol packet.

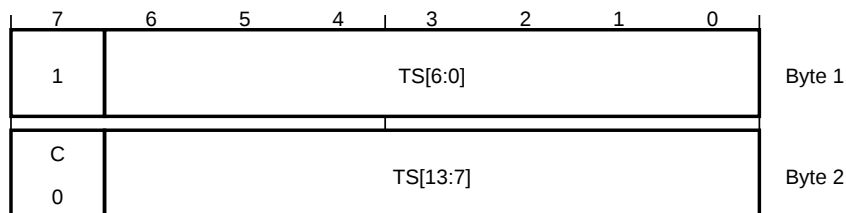
Other values encode different sizes of Hardware and Software source packets. This field reads as 0b00.

F1.2.8.2 Global Timestamp 1 packet payload

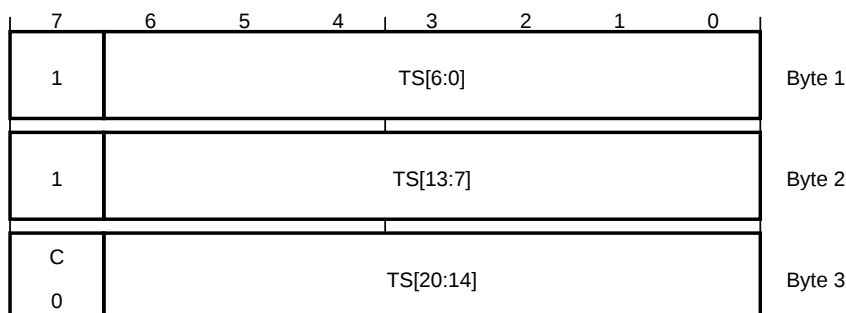
When 7-bit timestamp, the Global Timestamp 1 packet payload bit assignments are:

7	6	5	4	3	2	1	0	
C	TS[6:0]							Byte 1
0								

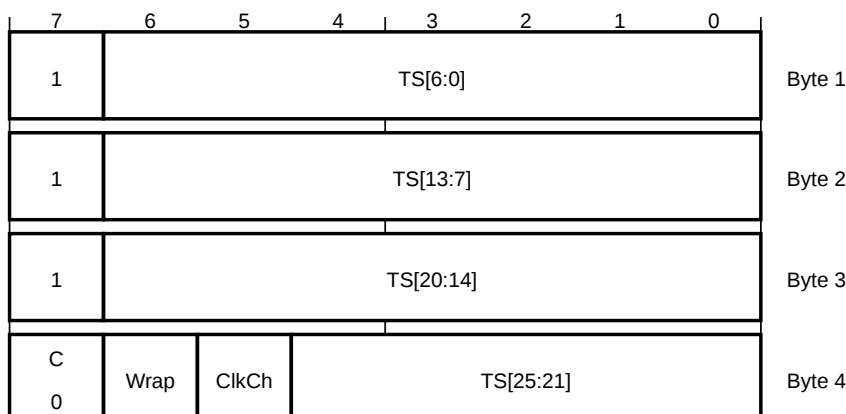
When 14-bit timestamp, the Global Timestamp 1 packet payload bit assignments are:



When 21-bit timestamp, the Global Timestamp 1 packet payload bit assignments are:



When 26-bit or full timestamp, the Global Timestamp 1 packet payload bit assignments are:



C, byte 4 bit [7], byte 3 bit [7], byte 2 bit [7], byte 1 bit [7] Continuation bit. The defined values of this field are:

- 0 Last byte of the packet.
- 1 Another byte follows.

Wrap, byte 4 bit [6], when 26-bit or full timestamp Wrapped. The defined values of this bit are:

- 0 The value of global timestamp bits TS[47:26] or TS[63:26] have not changed since the last [Global Timestamp 2 packet](#) output by the ITM.
- 1 The value of global timestamp bits TS[47:26] or TS[63:26] have changed since the last [Global Timestamp 2 packet](#) output by the ITM.

ClkCh, byte 4 bit [5], when 26-bit or full timestamp Clock change. The defined values of this bit are:

- 0 The system has not asserted the clock change input to the processor since the last time the ITM generated a Global Timestamp packet.

- 1 The system has asserted the clock change input to the processor since the last time the ITM generated a Global Timestamp packet.

Note

When the clock change input to the processor is asserted, the ITM must output a full 48-bit or 64-bit global timestamp value.

TS[25:0], byte 4 bits [4:0], byte 3 bits [6:0], byte 2 bits [6:0], byte 1 bits [6:0] Global Timestamp. The timestamp is 64 or 48 bits. If the Global Timestamp 1 packet is shorter than 5 bytes, the most-significant bits of the timestamp have not changed since the last Global Timestamp 1 packet output by the ITM. If the Global Timestamp 1 packet is 5 bytes, the Wrap bit defines whether most-significant bits have unchanged since the last [Global Timestamp 2 packet](#) output by the ITM.

F1.2.9 Global Timestamp 2 packet

The Global Timestamp 2 packet characteristics are:

Purpose Provides the most significant bits of a full 48 or 64-bit timestamp.

Attributes Multi-part Protocol packet comprising:

- 8-bit header.
- 32 or 48-bit payload.

F1.2.9.1 Global Timestamp 2 packet header

The Global Timestamp 2 packet header bit assignments are:

7	6	5	4	3	2	1	0	
C	ID					SS		Byte 0
1	0	1	1	0	1	0	0	

C, byte 0 bit [7] Continuation bit. This bit reads as one.

ID, byte 0 bits [6:2] Protocol packet type. Bits [6:2] discriminate between Protocol packet types. The defined values of this field are:

0b01101 Global Timestamp 2 packet.

This field reads as 0b01101.

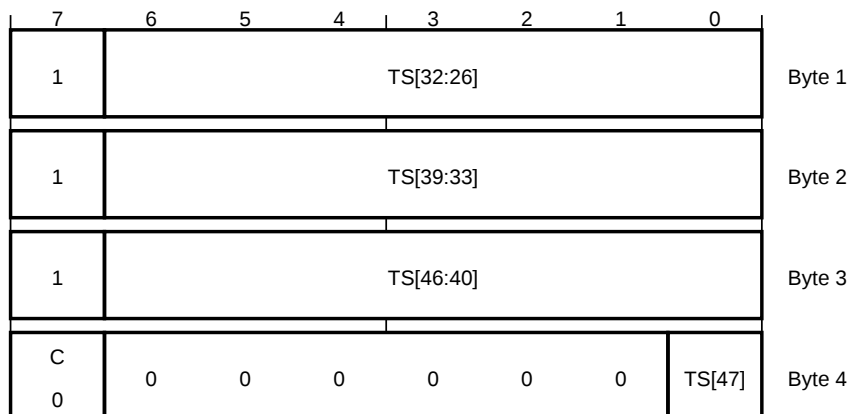
SS, byte 0 bits [1:0] Packet type. The defined values of this field are:

0b00 Protocol packet.

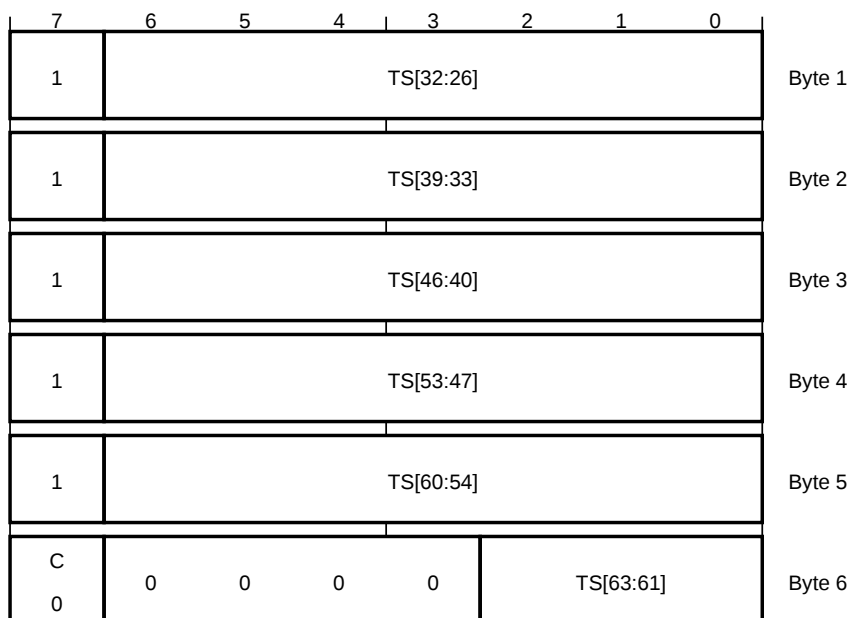
Other values encode different sizes of Hardware and Software source packets. This field reads as 0b00.

F1.2.9.2 Global Timestamp 2 packet payload

When 48-bit Global Timestamp 2 packet, the Global Timestamp 2 packet payload bit assignments are:



When 64-bit Global Timestamp 2 packet, the Global Timestamp 2 packet payload bit assignments are:



C, byte 6 bit [7], byte 5 bit [7], byte 4 bit [7], byte 3 bit [7], byte 2 bit [7], byte 1 bit [7] Continuation bit.

The defined values of this field are:

0 Last byte of the packet.

1 Another byte follows.

Byte 6 bits [6:3], when 64-bit Global Timestamp 2 packet This field reads-as-zero.

Byte 4 bits [6:1], when 48-bit Global Timestamp 2 packet This field reads-as-zero.

TS[47:26] , byte 4 bit [0], byte 3 bits [6:0], byte 2 bits [6:0], byte 1 bits [6:0], when 48-bit Global Timestamp 2 packet

Most significant bits of the Global Timestamp.

TS[63:26] , byte 6 bits [2:0], byte 5 bits [6:0], byte 4 bits [6:0], byte 3 bits [6:0], byte 2 bits [6:0], byte 1 bits [6:0], when 64-bit Global Timestamp 2 packet

Most significant bits of the Global Timestamp.

F1.2.10 Instrumentation packet

The Instrumentation packet characteristics are:

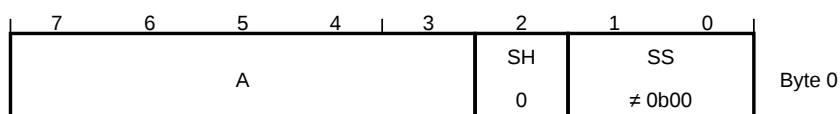
Purpose A software write to an ITM stimulus port generates an Instrumentation packet.

Attributes Multi-part Software source packet comprising:

- 8-bit header.
- 8, 16, or 32-bit payload.

F1.2.10.1 Instrumentation packet header

The Instrumentation packet header bit assignments are:



A, byte 0 bits [7:3] Port number, 0-31.

SH, byte 0 bit [2] Source. The defined values of this bit are:

0 Instrumentation packet (Software source).

This bit reads as zero.

SS, byte 0 bits [1:0] Size. The defined values of this field are:

0b01 Byte Instrumentation packet.

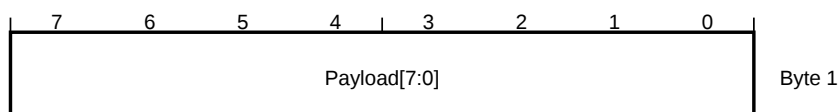
0b10 Halfword Instrumentation packet.

0b11 Word Instrumentation packet.

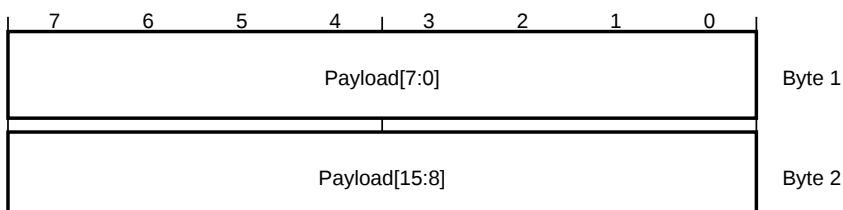
The value 0b00 encodes a Protocol packet.

F1.2.10.2 Instrumentation packet payload

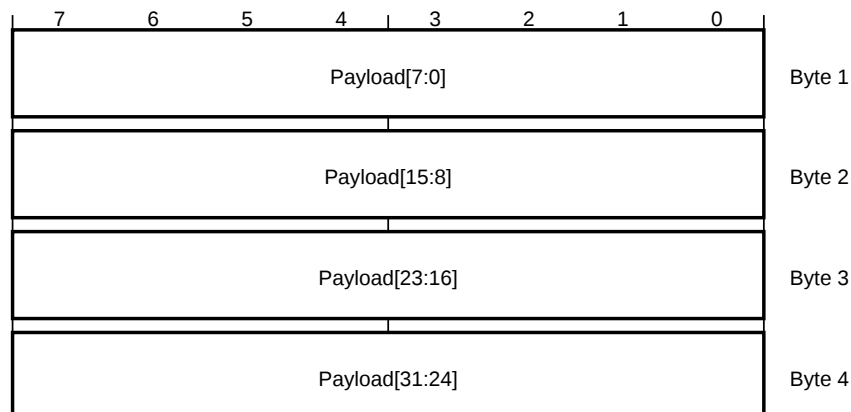
When Byte Instrumentation packet, SS == 0b01, the Instrumentation packet payload bit assignments are:



When Halfword Instrumentation packet, SS == 0b10, the Instrumentation packet payload bit assignments are:



When Word Instrumentation packet, SS == 0b11, the Instrumentation packet payload bit assignments are:



Payload[31:0], bytes <4:1>, when **Word Instrumentation packet**, SS == **0b11** Payload value.

Payload[15:0], byte 1 bits [15:0], when **Halfword Instrumentation packet**, SS == **0b10** Payload value.

Payload[7:0], byte <1>, when **Byte Instrumentation packet**, SS == **0b01** Payload value.

F1.2.11 Local Timestamp 1 packet

The Local Timestamp 1 packet characteristics are:

Purpose A Local Timestamp 1 packet encodes timestamp information, for generic control and synchronization, based on a timestamp counter in the ITM. To reduce the trace bandwidth:

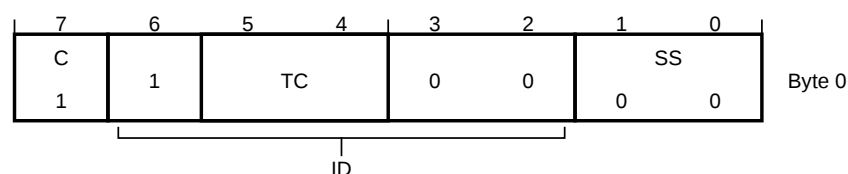
- The local timestamping scheme uses delta timestamps. Whenever the ITM outputs a Local timestamp packet, it clears its timestamp counter to zero, meaning each local timestamp value gives the interval since the generation of the previous Local timestamp packet.
- The Local Timestamp 1 packet length, 1-5 bytes, depends on the timestamp value.
- If the ITM outputs the local timestamp synchronously to the corresponding ITM or DWT data, and the timestamp value is in the range 1-6, the ITM uses the [Local Timestamp 2 packet](#).

Attributes Multi-part Protocol packet comprising:

- 8-bit header.
- 8, 16, 24, or 32-bit payload.

F1.2.11.1 Local Timestamp 1 packet header

The Local Timestamp 1 packet header bit assignments are:



C, byte 0 bit [7] Continuation bit. This bit reads as one.

ID, byte 0 bits [6:2] Protocol packet type. Bits [6:2] discriminate between Protocol packet types. The defined values of this field are:

0b1xx00 Local Timestamp 1 packet.

This field reads as 0b1xx00.

TC, byte 0 bits [5:4] Indicates the relationship between the generation of the Local timestamp packet and the corresponding ITM or DWT data packet. The defined values of this field are:

0b00 The local timestamp value is synchronous to the corresponding ITM or DWT data. The value in the TS field is the timestamp counter value when the ITM or DWT packet is generated.

0b01 The local timestamp value is delayed relative to the ITM or DWT data. The value in the TS field is the timestamp counter value when the Local timestamp packet is generated.

Note

The local timestamp value corresponding to the previous ITM or DWT packet is *unknown*, but must be between the previous and current local timestamp values.

0b10 Output of the ITM or DWT packet corresponding to this Local timestamp packet is delayed relative to the associated event. The value in the TS field is the timestamp counter value when the ITM or DWT packets is generated.

This encoding indicates that the ITM or DWT packet was delayed relative to other trace output packets.

0b11 Output of the ITM or DWT packet corresponding to this Local timestamp packet is delayed relative to the associated event, and this Local timestamp packet is delayed relative to the ITM or DWT data. This is a combination of the conditions indicated by values 0b01 and 0b10.

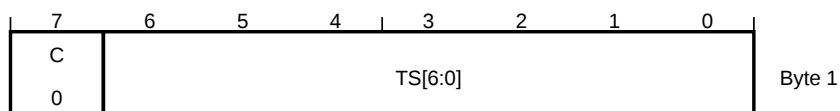
SS, byte 0 bits [1:0] Packet type. The defined values of this field are:

0b00 Protocol packet.

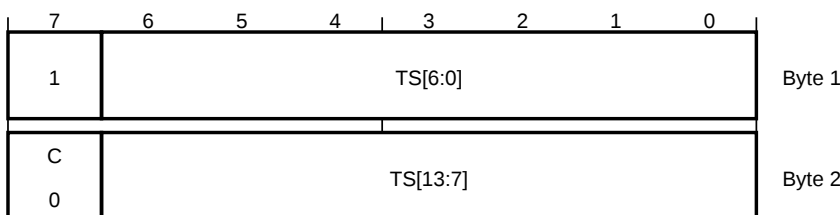
Other values encode different sizes of Hardware and Software source packets. This field reads as 0b00.

F1.2.11.2 Local Timestamp 1 packet payload

When 7-bit timestamp, the Local Timestamp 1 packet payload bit assignments are:



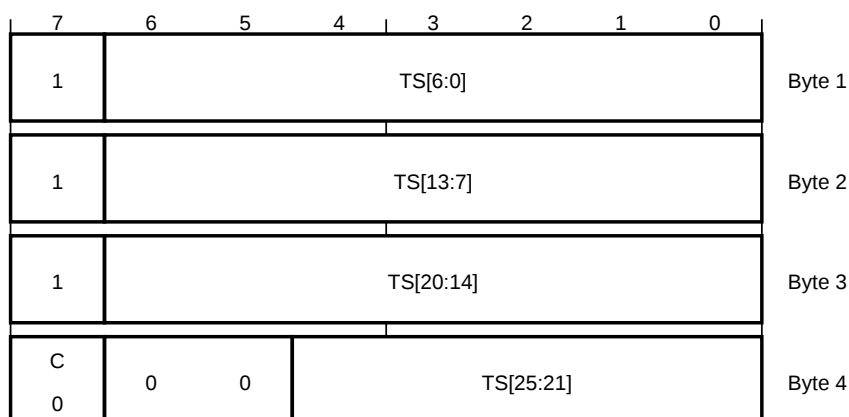
When 14-bit timestamp, the Local Timestamp 1 packet payload bit assignments are:



When 21-bit timestamp, the Local Timestamp 1 packet payload bit assignments are:



When 28-bit timestamp, the Local Timestamp 1 packet payload bit assignments are:



C, byte 4 bit [7], byte 3 bit [7], byte 2 bit [7], byte 1 bit [7] Continuation bit. The defined values of this field are:

- 0 Last byte of the packet.
- 1 Another byte follows.

Byte 4 bits [6:5], when 28-bit timestamp This field reads-as-zero.

TS, byte 4 bits [4:0], byte 3 bits [6:0], byte 2 bits [6:0], byte 1 bits [6:0] Local Timestamp.

The timestamp is 28 bits. If the Local Timestamp 1 packet is shorter than 5 bytes, the most significant bits of the timestamp are zero.

F1.2.12 Local Timestamp 2 packet

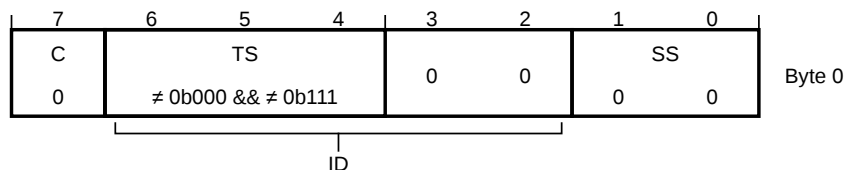
The Local Timestamp 2 packet characteristics are:

Purpose If the ITM outputs the Local Timestamp synchronously to the corresponding ITM or DWT data, and the required timestamp value is in the range 1-6, it uses the Local Timestamp 2 packet. For more information, see [Local Timestamp 1 packet](#).

Attributes 8-bit Protocol packet.

Field descriptions

The Local Timestamp 2 packet bit assignments are:



C, byte 0 bit [7] Continuation bit. This bit reads as zero.

ID, byte 0 bits [6:2] Protocol packet type. Bits [6:2] discriminate between Protocol packet types. The defined values of this field are:

0b00000 See [Synchronization packet](#).

0bxxx00 For all other values of 0bxxx. Local Timestamp 2 packet.

0b11100 See [Overflow packet](#).

This field reads as 0bxxx00.

TS, byte 0 bits [6:4] Local timestamp value, in the range 0b001 to 0b110.

SS, byte 0 bits [1:0] Packet type. The defined values of this field are:

0b00 Protocol packet.

Other values encode different sizes of Hardware and Software source packets. This field reads as 0b00.

F1.2.13 Overflow packet

The Overflow packet characteristics are:

Purpose The ITM outputs an Overflow packet if:

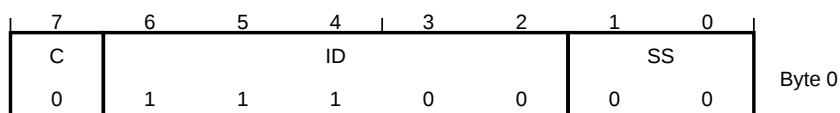
- Software writes to a Stimulus Port register when the stimulus port output buffer is full.
- The DWT attempts to generate a Hardware source packet when the DWT output buffer is full.
- The Local timestamp counter overflows.

The Overflow packet comprises a header with no payload.

Attributes 8-bit Protocol packet.

Field descriptions

The Overflow packet bit assignments are:



C, byte 0 bit [7] Continuation bit. This bit reads as zero.

ID, byte 0 bits [6:2] Protocol packet type. Bits [6:2] discriminate between Protocol packet types. The defined values of this field are:

0b11100 Overflow packet.

This field reads as 0b11100.

SS, byte 0 bits [1:0] Packet type. The defined values of this field are:

0b00 Protocol packet.

Other values encode different sizes of Hardware and Software source packets. This field reads as 0b00.

F1.2.14 Periodic PC Sample packet

The Periodic PC Sample packet characteristics are:

Purpose The DWT unit generates PC samples at fixed time intervals, with an accuracy of one clock cycle. The POSTCNT counter period determines the PC sampling interval. Software configures the [DWT_CTRL.CYCTAP](#) and [DWT_CTRL.POSTINIT](#) fields to determine how POSTCNT relates to [DWT_CYCCNT](#). The [DWT_CTRL.PCSAMPLENA](#) bit enables PC sampling.

Attributes Multi-part Hardware source packet comprising:

- 8-bit header.
- 8 or 32-bit payload.

F1.2.14.1 Periodic PC Sample packet header

The Periodic PC Sample packet header bit assignments are:

7	6	5	4	3	2	1	0
ID				SH	SS		
0	0	0	1	0	1	1	

Byte 0

ID, byte 0 bits [7:3] Discriminator ID. The defined values of this field are:

0b00010 Periodic PC Sample packet.

This field reads as 0b00010.

SH, byte 0 bit [2] Source. The defined values of this bit are:

1 Hardware source packet.

This bit reads as one.

SS, byte 0 bits [1:0] Size. The defined values of this field are:

0b01 Source packet, 1-byte payload, 2-byte packet.

0b11 Source packet, 4-byte payload, 5-byte packet.

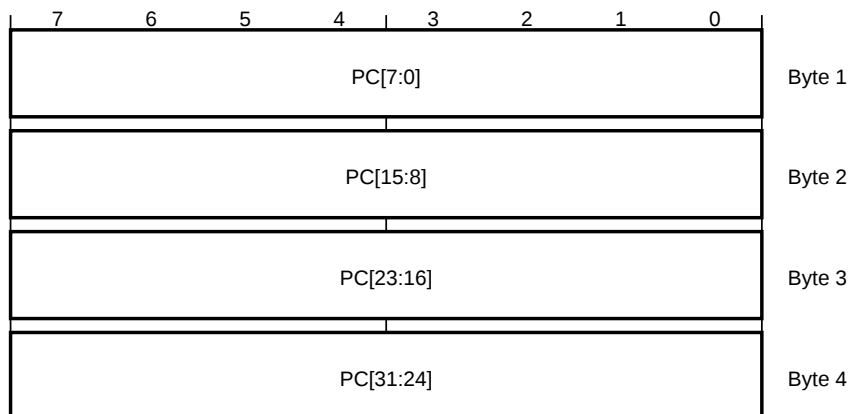
SS == 0b10 is invalid for a Periodic PC Sample packet.

The value 0b00 encodes a Protocol packet.

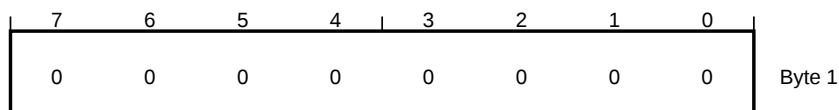
This field reads as 0bx1.

F1.2.14.2 Periodic PC Sample packet payload

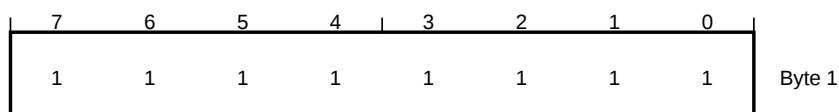
When Allowed and not sleeping, SS == 0b11, the Periodic PC Sample packet payload bit assignments are:



When Allowed and sleeping, SS == 0b01, the Periodic PC Sample packet payload bit assignments are:



When Prohibited, SS == 0b01, the Periodic PC Sample packet payload bit assignments are:



PC, bytes <4:1>, when Allowed and not sleeping, SS == 0b11 Periodic PC sample value.

Byte <1>, when Allowed and sleeping, SS == 0b01 This field reads as 0b00000000.

Byte <1>, when Prohibited, SS == 0b01 This field reads as 0b11111111.

F1.2.15 PMU overflow packet

The PMU overflow packet characteristics are:

Purpose For each counter n , if the lower eight bits of that counter overflow, the associated OV_n of the PMU overflow packet is set. If multiple counters overflow in the same period, multiple bits might be set. If there are fewer than 8 general-purpose counters, the associated PMU overflow packet bit is always zero.

Attributes 8 bit protocol packet.

Field descriptions

The PMU overflow packet bit assignments are:

7	6	5	4	3	2	1	0	
ID				SH	SS			Byte 0
0	0	0	1	1	1	0	1	
								Byte 1
OV7	OV6	OV5	OV4	OV3	OV2	OV1	OV0	

Byte 0 bits [7:0] Packet Header

Byte 1 bits [7:0] OV_n

F1.2.16 Synchronization packet

The Synchronization packet characteristics are:

Purpose A Synchronization packet provides a unique pattern in the bit stream. Trace capture hardware can identify this pattern and use it to identify the alignment of packet bytes in the bitstream.

Attributes 48-bit Protocol packet.

A Synchronization packet is at least forty-seven 0 bits followed by single 1 bit. This section describes the smallest possible Synchronization packet.

Field descriptions

The Synchronization packet bit assignments are:

7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	Byte 0
0	0	0	0	0	0	0	0	Byte 1
0	0	0	0	0	0	0	0	Byte 2
0	0	0	0	0	0	0	0	Byte 3
0	0	0	0	0	0	0	0	Byte 4
1	0	0	0	0	0	0	0	Byte 5

Byte 5 bit [7] Indicates the end of the Synchronization packet. This bit reads as one.

Byte 5 bits [6:0], bytes <4:1> This field reads-as-zero.

Byte <0> This field reads as 0b00000000.

Part G

Glossary

AAPCS

Procedure Call Standard for the Arm Architecture.

Additional floating-point context registers

The Additional floating-point state context registers are as follows:

- [S16](#) to [S31](#).

See also: [Floating-point context](#)

Additional state context registers

The Additional state context registers are as follows:

- [R4](#) to [R11](#).
- The Integrity Signature.

See also: [State context registers](#)

Address dependency

An address dependency exists when the value that is returned by a read computes the address of a subsequent access. An address dependency exists even if the value that is returned by the first read does not change the address of the second read or write.

See [B7.2.3 Ordering and observability](#)

Addressing mode

Means a method for generating the memory address that is used by a load/store instruction.

Aligned

A data item that is stored at an address that is exactly divisible by the highest power of 2 that divides exactly into its size in bytes. Aligned halfwords, words, and doublewords therefore have addresses that are divisible by 2, 4 and 8 respectively.

An aligned access is one where the address of the access is aligned to the size of each element of the access.

Application Program Status Register (APSR)

The register containing those bits that deliver status information about the results of instructions, the N, Z, C, and V bits of the XPSR. In an implementation that includes the DSP extension, the APSR includes the GE bits that provide status information from DSP operations.

See also [B3.5 XPSR, APSR, IPSR, and EPSR](#).

APSR

See Application Program Status Register.

Architecturally executed

An instruction is architecturally executed only if it would be executed in a simple sequential execution of the program. When such an instruction has been executed and retired it has been *architecturally executed*. Any instruction that, in a simple sequential execution of a program, is treated as a NOP because it fails its condition code check, is an architecturally executed instruction.

In a PE that performs Speculative execution, an instruction is not architecturally executed if the PE discards the results of a Speculative execution.

See also [Condition code check](#), [Simple sequential execution](#).

Architecturally UNKNOWN

An Architecturally UNKNOWN value is a value that is not defined by the architecture but must meet the requirements of the definition of UNKNOWN. Implementations can define the value of the field, but are not required to do so.

See also [IMPLEMENTATION DEFINED](#).

Architecture tick

An atomic unit of execution. In the Armv8.0-M architecture, most instructions are considered atomic units for execution (they are either performed or not performed). The most notable exceptions are instructions that support ICI behavior.

Associativity

See [Cache associativity](#)

Asynchronous exception

An exception is described as asynchronous if any of the following apply:

- The exception is not generated as a result of direct execution or attempted execution of the instruction stream.
- The return address presented to the exception handler is not guaranteed to indicate the instruction that caused the exception.
- The exception is imprecise.

See [Imprecise exception](#), [Synchronous exception](#).

Atomicity

Describes either single-copy atomicity or multi-copy atomicity. [B7.2.1 Single-copy atomicity](#) defines these forms of atomicity for the Arm architecture.

See also [Multi-copy atomicity](#), [Single-copy atomicity](#).

Attributability

A PMU event that is caused by the PE counting the PMU event is Attributable. If an agent other than the PE that is counting the PMU events causes a PMU event then that PMU event is Unattributable.

A PMU event is either Attributable or Unattributable. If the PMU event is Attributable, it is further defined whether the PMU event is Attributable to:

- The current Security state of the PE.
- The privilege level.
- When the PE is in Debug state, operations issued to the PE by the Debugger through the external debug interface.

Attribution Unit (AU)

The combination of the Secure Attribution Unit (SAU) and the Implementation Defined Attribution Unit (IDAU).

See also [Chapter B10 The Armv8-M Protected Memory System Architecture](#).

AU

See [Attribution unit](#).

Availability

Readiness for correct service.

Background state

The state of the PE before the last (previous) preemption occurred.

Banked register

A register that has multiple instances, with the instance that is in use depending on the PE mode, Security state, or other PE state.

Base register

A register that is specified by a load/store instruction that is used as the base value for the address calculation for the instruction. Depending on the instruction and its addressing mode, an offset can be added to or subtracted from the base register value to form the address that is sent to memory.

Base register Write-Back

Describes writing back a modified value to the base register used in an address calculation.

Baseboard Management Controller

A PE dedicated to system control and monitoring.

Beat

The execution of a 1/4 of an MVE vector operation. Because the vector length is 128 bits, one beat of a vector add instruction equates to computing 32 bits of result data. This is independent of lane width. For example, if a lane width is 8 bits, then a single beat of a vector add instruction would perform four 8-bit additions.

See also [B5.4 Beats](#).

Behaves as if

Where this manual indicates that a PE *behaves as if* a certain condition applies, all descriptions of the operation of the PE must be re-evaluated taking account of that condition, together with any other conditions that affect operation.

BF branch point

The gap between the two instructions. Specifically, the gap between the instruction that immediately precedes the instruction at the `<b_label>` of a BF instruction and the instruction that is identified by the `<b_label>` of a BF instruction..

See also [B3.29 Branch future](#).

Big-endian memory

Means that, for example:

- A byte or halfword at a word-aligned address is the most significant byte or halfword in the word at that address.
- A byte at a halfword-aligned address is the most significant byte in the halfword at that address.

See also [B7.5 Endianness](#), [Little-endian memory](#).

Blocking

Describes an operation that does not permit following instructions to be executed before the operation completes.

A non-blocking operation can permit following instructions to be executed before the operation completes, and in the event of encountering an exception does not signal an exception to the PE. This enables implementations to retire following instructions while the non-blocking operation is executing, without the need to retain precise PE state.

Branch prediction

Is where a PE selects a future execution path to fetch along. For example, after a branch instruction, the PE can choose to speculatively fetch either the instruction following the branch or the instruction at the branch target.

See also [Prefetching](#).

Breakpoint

A debug event that is triggered by the execution of a particular instruction, which is specified by one or both of the address of the instruction and the state of the PE when the instruction is executed.

BTI

Branch Target Identification. The mechanism used for creating and identifying valid branch landing pads.

BTI clearing

Branch Target Identification clearing instruction. Any instruction that clears the [EPSR.B](#) bit to zero.

BTI setting

Branch Target Identification setting instruction. Any instruction that sets the [EPSR.B](#) bit to one.

Byte

An 8-bit data item.

Cache associativity

The number of locations in a cache set to which an address can be assigned. Each location is identified by its *way* value.

Cache level

The position of a cache in the cache hierarchy. In the Arm architecture, the lower numbered levels are those closest to the PE. For more information, see [B7.19 Caches](#).

Cache line

The basic unit of storage in a cache. Its size in words is always a power of two, usually four or eight words. A cache line must be aligned to a suitable memory boundary. A *memory cache line* is a block of memory locations with the same size and alignment as a cache line. Memory cache lines are sometimes loosely called cache lines.

Cache sets

Areas of a cache, which is divided up to simplify and speed up the process of determining whether a cache hit occurs. The number of cache sets is always a power of two. The term cache sets is a common convention for describing cache memories, and this description must not be treated as defining a property of the cache.

Cache way

A cache way consists of one cache line from each cache set. The cache ways are indexed from 0 to (Associativity-1). Each cache line in a cache way is chosen to have the same index as the cache way. For example, cache way *n* consists of the cache line with index *n* from each cache set. The term cache way is a common convention for describing cache memories, and this description must not be treated as defining a property of the cache.

Cache write-back granule

The maximum size of the memory that can be overwritten. In some implementations, the [CTR](#) identifies the Cache Write-Back Granule.

Catastrophic failure

A failure with harmful consequences that are orders of magnitude, or even incommensurably, higher than the benefit provided by correct service delivery.

Chained vector instruction

An instruction that is subject to beat-wise execution.

Coherence order

See [Coherent](#)

Coherent

Data accesses from a set of observers to a byte in memory are coherent if accesses to that byte in memory by the members of that set of observers are consistent with there being a single total order of all writes to that byte in memory by all members of the set of observers. This single total order of all to writes to that memory location is the *coherence order* for that byte in memory.

Completer

An agent in a computing system that responds to and completes a memory transaction that was initiated by a Requester.

See also [Requester](#)

Condition code check

The process of determining whether a conditional instruction executes normally or is treated as a NOP. For an instruction that includes a condition code field, that field is compared with the condition flags to determine whether the instruction is executed normally. For a T32 instruction in an IT block, the value of [EPSR.IT](#) determines whether the instruction is executed normally.

See also [Condition code field](#), [Condition flags](#), [Conditional execution](#).

Condition code field

A 4-bit field in an instruction that specifies the condition under which the instruction executes.

See also [Condition code check](#).

Condition flags

The N, Z, C, and V bits of [APSR](#), or [XPSR](#). See [B3.5 XPSR, APSR, IPSR, and EPSR](#) for more information.

See also [Condition code check](#).

Conditional execution

When a conditional instruction starts executing, if the condition code check returns TRUE, the instruction executes normally. Otherwise, it is treated as a NOP. See [C1.3 Conditional execution](#).

See also [Condition code check](#).

Configuration

Settings that are made on reset, or immediately after reset, and normally expected to remain static throughout program execution.

CONSTRAINED UNPREDICTABLE

Where an instruction can result in UNPREDICTABLE behavior, the Armv8 architecture specifies a narrow range of permitted behaviors. This range is the range of CONSTRAINED UNPREDICTABLE behavior. All implementations that are compliant with the architecture must follow the CONSTRAINED UNPREDICTABLE behavior within the limits defined for each particular case, and this behavior might vary.

In body text, the term CONSTRAINED UNPREDICTABLE is shown in SMALLCAPS.

See also [UNPREDICTABLE](#).

Containable

An error that is not uncontained. A Containable error is also referred to as a Contained error.

Context switch

The saving and restoring of computational state when switching between different threads or processes. In this manual, the term context switch describes any situation where the context is switched by an operating system and might or might not include changes to the address space.

Context synchronization event

A Context synchronization event guarantees visibility of any change to any register described in the architecture. Following a Context synchronization event a completed write to a register is visible to an indirect read by an instruction appearing in program order after the context synchronization event.

Control dependency

A control dependency exists when the data value that is returned by a read access determines the condition flags, and the values of the flags determine the address of a subsequent read access. This address determination might be through conditional execution, or through the evaluation of a branch.

Corrected

An error that is detected by hardware and that hardware can correct. This is also referred to as a Correctable error.

Countable error

An error that is detected and recorded by hardware by incrementing a counter.

Cross beat

An operation that requires operands from different beats to produce the output for a single beat. For example, a vector widening operation would be cross beat, whereas a vector addition would not be.

Cross Trigger Interface

A debug component that is not part of the Armv8-M architecture.

CTI

See [Cross Trigger Interface](#).

DAP

Debug Access Port.

Data independent timing

The time that it takes to execute a piece of code where the time is not a function of the data being operated on.

See also [B3.34 Data independent timing](#).

Data Watchpoint and Trace (DWT)

The Data Watchpoint and Trace unit is a component of Armv8-M debug that optionally provides a number of trace, sampling, and profiling functions.

See also [B14.2 Data Watchpoint and Trace unit](#).

DCB

See [Debug Control Block](#).

Debug Control Block (DCB)

A region in the System Control Space that is assigned to registers that support debug features.

See also [System Control Space](#).

Debugger

In most of this manual, *debugger* refers to any agent that is performing debug. However, some parts of the manual require a more rigorous definition, and define debugger locally. See [Chapter B13 Debug](#).

Deferred

An error that has not been silently propagated but does not require immediate action at the producer. The error might have passed from the producer to the consumer.

Deprecated

Something that is present in the Arm architecture for backwards compatibility. Whenever possible software must avoid using deprecated features. Features that are deprecated but are not optional are present in current implementations of the Arm architecture, but might not be present, or might be deprecated and OPTIONAL, in future versions of the Arm architecture.

See also [OPTIONAL](#).

Detected

An error that has been detected and signaled to a consumer.

Detected Uncorrectable

A detected error that cannot be corrected and causes failure.

Digital signal processing (DSP)

Algorithms for processing signals that have been sampled and converted to digital form. DSP algorithms often use saturated arithmetic.

Direct access

A read or write of a register.

DIT

See [Data independent timing](#).

Domain

In the Arm architecture, *domain* is used in the following contexts.

Shareability domain Defines a set of observers for which the Shareability attributes make the data or unified caches transparent for data accesses.

Power domain Defines a block of logic with a single, common, power supply.

Double-precision value

Consists of two consecutive 32-bit words that are interpreted as a basic double-precision floating-point number according to the *IEEE Standard for Floating-point Arithmetic*.

Doubleword

A 64-bit data item. Doublewords are normally at least word-aligned in Arm systems.

Doubleword-aligned

Means that the address is divisible by 8.

DSP

See [Digital signal processing](#).

DWT

See [Data Watchpoint and Trace](#).

ECC

Error Correction Code

EDC

Error Detection Code

Effective Value

A register control field, meaning a field in a register that controls some aspect of the behavior, can be described as having an *Effective value*:

- In some cases, the description of a particular control *a* specifies that when control *a* is active it causes a register control field *b* to be treated as having a fixed value for all purposes other than direct reads, or direct reads and direct writes, of the register containing control field *b*. When control *a* is active that fixed value is described as the *Effective value* of register control field *b*.

In other cases, a register control field *b* is not implemented or is not accessible, but behavior of the PE is as if control field *b* was implemented and accessible, and had a particular value. In this case, that value is the *Effective value* of register control field *b*.

Where a register control field is introduced in a particular version of the architecture, and is not implemented in an earlier version of the architecture, typically it will have an *Effective value* in that earlier version of the architecture.

- Otherwise, the *Effective value* of a register control field is the value of that field.

Element

The data that is put into a lane.

See also [Lane](#).

Embedded Trace Macrocell (ETM)

A component of the Arm CoreSight debug and trace solution. An ETM provides non-invasive trace of PE operation.

See [B14.3 Embedded Trace Macrocell](#).

Endianness

An aspect of the system memory mapping. For more information, see [B7.5 Endianness](#).

See also [Big-endian memory](#) and [Little-endian memory](#).

EPSR

See [Execution Program Status Register](#).

Error

Deviation from correct service or a correct value.

Error propagation

Passing an error from a producer to a consumer.

Error record

Data recorded about an error, usually by hardware.

ETM

See [Embedded Trace Macrocell](#)

Exception

Handles an event. For example, an exception could handle an external interrupt or an undefined instruction.

Exception vector

A fixed address that contains the address of the first instruction of the corresponding exception handler.

Exception-continuable instruction

Any instruction that can cause the PE to set the [EPSR.ECI](#) fields.

Execution Program Status Register (EPSR)

A register that contains the Execution state bits and is part of the XPSR.

See also [B3.5 XPSR, APSR, IPSR, and EPSR](#).

Execution stream

The stream of instructions that would have been executed by sequential execution of the program.

Explicit access

A read from, or a write to memory or memory-mapped registers, generated by a load or store instruction that is executed by the PE or an access by the debugger.

Failure

The event of deviation from correct service.

Fault

An exception that is generated because of some form of system error. A BusFault might be a RAS fault.

Fault injection

The deliberate injection of faults into a system for testing.

Fault prevention

Designing a system to avoid faults.

Fault removal

Logic or other mechanisms for detecting faults and correcting or bypassing their effect.

Field Replaceable Unit

The smallest unit that can be replaced without return to base.

Flash Patch and Breakpoint Unit

The Flash Patch and Breakpoint unit supports setting breakpoints on instruction fetches.

See also [B14.5 Flash Patch and Breakpoint unit](#).

Floating-point context

The Floating-point context is as follows:

- [S0-S15](#).
- [FPSCR](#).
- [VPR](#).

See also: [Additional floating-point state context](#).

Flush-to-zero mode

A processing mode that optimizes the performance of some floating-point algorithms by replacing the denormalized operands and Intermediate results with zeros, without significantly affecting the accuracy of their final results.

FPB

See [Flash Patch and Breakpoint Unit](#).

FRU

See [Field Replaceable Unit](#).

General-purpose registers

The registers that the base instructions use for processing:

- The general-purpose registers are [R0-R12](#). R13-R14 are the [SP](#) and [LR](#), respectively. For more information, see [B3.3 Registers](#).

See also [High registers](#), [Low registers](#).

Halfword

A 16-bit data item. Halfwords are normally halfword-aligned in Arm systems.

Halfword-aligned

Means that the address is divisible by 2.

Hardware fault

A fault that originates in or affects hardware.

High registers

The general-purpose registers [R8-R14](#). Most 16-bit T32 instructions cannot access the high registers.

Note

In some contexts, *high registers* refers to R8-R15, meaning R8-R14 and the [PC](#).

See also [General-purpose registers](#), [Low registers](#).

ICI

See [Interrupt-continuable instruction](#).

If-Then block (IT block)

An IT block is a block of up to four instructions following an *If-Then* (IT) instruction. Each instruction in the block is conditional. The conditions for the instructions are either all the same, or some are the inverse of others.

Immediate and offset fields

Are unsigned unless otherwise stated.

Immediate value

A value that is encoded directly in the instruction and used as numeric data when the instruction is executed. Many T32 instructions can be used with an immediate argument.

IMP DEF

An abbreviation that is used in diagrams to indicate that one or more bits have IMPLEMENTATION DEFINED behavior.

IMPLEMENTATION DEFINED

Means that the behavior is not architecturally defined, but must be defined and documented by individual implementations.

In body text, the term IMPLEMENTATION DEFINED is shown in SMALLCAPS.

Implicit access

An access that is not an explicit access. For example:

- Instruction fetches.
- The setting of a syndrome flag in response to an exception being raised.

See also [Explicit access](#).

Imprecise exception

An exception that is generated as the result of a system error. An imprecise exception is reported at the time that is asynchronous to the instruction that caused it.

Index register

A register that is specified in some load and store instructions. The value of this register is used as an offset to be added to or subtracted from the base register value to form the address that is sent to memory. Some instruction forms permit the index register value to be shifted before the addition or subtraction.

Indirect access

A read or write of a register that is not a direct access.

For example, an indirect write to a register might occur as the side effect of executing an instruction that does not perform a direct write to the register, or because of some operation that is performed by an external agent.

See also [Direct access](#)

Infected

Being in error.

Inline literals

These are constant addresses and other data items that are held in the same area as the software itself. They are automatically generated by compilers, and can also appear in assembler code.

Instrumentation Trace Macrocell (ITM)

A component of the Arm CoreSight debug and trace solution. An ITM provides a memory-mapped register interface that applications can use to write logging or event words to a trace sink.

See [B14.1 Instrumentation Trace Macrocell](#).

Interrupt Program Status Register (IPSR)

The register that provides status information on whether an application thread or exception handler is executing on the processor. If an exception handler is executing, the register provides information on the exception type. The register is part of the XPSR.

See also [B3.5 XPSR, APSR, IPSR, and EPSR](#).

Interrupt Service Routine

The procedure that handles an interrupt.

Interrupt-continuable instruction

Multicycle load or store instructions that can be interrupted part way through their execution. After the interrupt service routine has completed, execution of the partially executed instruction can be resumed and the instruction is not required to be restarted from the beginning. An interruption of a multicycle load or store instructions will cause the PE to set the [EPSR.ICI](#) fields.

See also [Exception-continuable instruction](#)

Interworking

A method of working that permits branches between software using the A32 and T32 instruction sets in the Armv8-A architecture. For Armv8-M, interworking is described in [C1.4.7 Instruction set, interworking and interstating support](#).

IPSR

See [Interrupt Program Status Register](#).

Isolation

Limiting the impact of an error only to components that actually try to use corrupted data.

ISR

See [Interrupt Service Routine](#).

ITM

See [Instrumentation Trace Macrocell](#).

Landing pad

[BTI](#) landing pad instruction. A landing pad instruction is a [BTI clearing](#) instruction.

Not all operations that clear the [EPSR.B](#) bit to zero are landing pads. For example, the [EPSR.B](#) is cleared automatically on exception entry but this operations is not a [landing pad](#) or a [BTI clearing](#) instruction.

The following instructions are valid [BTI](#) landing pads:

- [BTI](#).
- [SG](#).
- [PACBTI](#).

Lane

A section of a vector register or operation.

Latent fault

An error that is present in a system but not yet detected.

Level

See [Cache level](#).

Level of Coherence (LoC)

The last level of cache that must be cleaned or invalidated when cleaning or invalidating to the point of coherence.

See also [Cache level](#), [Point of Coherency](#).

Level of Unification, Inner Shareable (LoUIS)

The last level of cache that must be cleaned or invalidated when cleaning or invalidating to the point of unification for the Inner Shareable Shareability domain.

See also [Cache level](#), [Point of Unification](#).

Level of Unification, uniprocessor (LoUU)

For a PE, the last level of cache that must be cleaned or invalidated when cleaning or invalidating to the point of unification for that PE.

See also [Cache level](#), [Point of Unification](#).

Line

See [Cache line](#).

Little-endian memory

Means that, for example:

- A byte or halfword at a word-aligned address is the least significant byte or halfword in the word at that address.
- A byte at a halfword-aligned address is the least significant byte in the halfword at that address.

See also [Big-endian memory](#), [B7.5 Endianness](#).

Load/store architecture

An architecture where data-processing operations only operate on register contents, not directly on memory contents.

LoC

See [Level of Coherence](#).

Lockup

A PE state where the PE stops executing instructions in response to an error for which escalation to an appropriate HardFault handler is not possible because of the current execution priority. For more information, see [B3.33 Lockup](#).

LoUIS

See [Level of Unification, Inner Shareable](#).

LoUU

See [Level of Unification, uniprocessor](#).

Low registers

General-purpose registers R0-R7. Unlike the high registers, all T32 instructions can access the Low registers.

LR hazard

An LR hazard occurs when LR is written while another instruction, other than `LE` or `LETP`, is accessing LR.

M-Profile Vector Extension

An optional part of the Armv8.1-M architecture that supports both integer and floating-point data types.

Memory barriers

The term memory barrier is the general term that is applied to an instruction, or sequence of instructions, that forces synchronization events by a PE regarding retiring Load/Store instructions. For more information, see [B7.2.9 Memory barriers](#).

Memory coherency

The problem of ensuring that when a memory location is read, either by a data read or an instruction fetch, the value that is actually obtained is always the value that was most recently written to the location. This can be difficult when there are multiple possible physical locations, such as main memory and at least one of a write buffer and one or more levels of cache.

Memory hint

A memory hint instruction provides advance information to memory systems about future memory accesses, without actually loading or storing any data to or from the register file. `PLD` and `PLI` are the only memory hint instructions that are defined in Armv8-M.

Memory Protection Unit (MPU)

A hardware unit whose registers provide simple control of a limited number of protection regions in memory, for more information, see [Chapter B10 The Armv8-M Protected Memory System Architecture](#).

Minor failure

A failure with harmful consequences that are of a similar cost to the benefits that are provided by correct service delivery.

MPU

See [Chapter B10 The Armv8-M Protected Memory System Architecture](#).

Multi-copy atomicity

The form of atomicity that is described in [B7.2.2 Multi-copy atomicity](#).

See also [Atomicity](#), [Single-copy atomicity](#).

MVE

See [M-Profile Vector Extension](#).

NaN

Not a Number. A floating-point value that can be used when neither a numeric value nor an infinity is appropriate. A NaN can be a *quiet* NaN, that propagate through most floating-point operations, or a *signaling* NaN, that causes an Invalid Operation floating-point exception when used. For more information, see the IEEE Standard for Floating-point Arithmetic.

Node

A component that detects an error is called a node.

Non-Return-to-Zero (NRZ)

A physical layer signaling scheme that is used on asynchronous communication ports

NRZ

See [Non-Return-to-Zero](#).

Obsolete

Obsolete indicates something that is no longer supported by Arm. When an architectural feature is described as obsolete, this indicates that the architecture has no support for that feature, although an earlier version of the architecture did support it.

Offset addressing

Means that the memory address is formed by adding or subtracting an offset to or from the base register value.

OPTIONAL

When applied to a feature of the architecture, OPTIONAL indicates a feature that is not required in an implementation of the Arm architecture:

- If a feature is OPTIONAL and deprecated, this indicates that the feature is being phased out of the architecture. Arm expects such a feature to be included in a new implementation only if there is a known backwards-compatibility reason for the inclusion of the feature.

A feature that is OPTIONAL and deprecated might not be present in future versions of the architecture.

- A feature that is OPTIONAL but not deprecated is, typically, a feature added to a version of the Arm architecture after the initial release of that version of the architecture. Arm recommends that such features are included in all new implementations of the architecture.

In body text, these meanings of the term OPTIONAL are shown in SMALLCAPS.

Note: Do not confuse these Arm-specific uses of OPTIONAL with other uses of OPTIONAL, where it has its usual meaning. These include:

- Optional arguments in the syntax of many instructions.
- Behavior that is determined by an implementation choice.

See also [Deprecated](#).

PAC

Pointer Authentication Code. The hash generated by the cryptographic algorithm. PAC also refers to the process used for generating the hash.

PE

See [Processing element](#).

Performance Monitoring Unit

An optional non-invasive debug component that allows events to be identified and counted. See [Chapter B15 The Performance Monitors Extension](#)

Persistent fault

A fault that is not transient.

Physical address (PA)

An address that identifies a location in the physical memory map.

PMU

See [Performing Monitoring Unit](#).

PoC

See [Point of Coherency](#).

Point of coherency (PoC)

For a particular PA, the point at which all agents that can access memory are guaranteed to see the same copy of a memory location.

Point of unification (PoU)

For a particular PE, the point by which the instruction and data caches of that PE are guaranteed to see the same copy of a memory location.

Poisoned

State that has been marked as being in error so that subsequent consumption of the state signals a detected error to a consumer.

Post-indexed addressing

Means that the memory address is the base register value, but an offset is added to or subtracted from the base register value and the result is written back to the base register.

PoU

See [Point of Unification](#).

PPB

Private Peripheral Bus

Pre-indexed addressing

Means that the memory address is formed in the same way as for offset addressing, but the memory address is also written back to the base register.

Precise exception

An exception is described as precise when the exception handler receives the PE state and memory system state that is consistent with the PE having executed all instructions up to, but not including, the point in the instruction stream where the exception was taken and none afterwards.

Prefetching

Prefetching refers to speculatively fetching instructions or data from the memory system. In particular, instruction prefetching is the process of fetching instructions from memory before the instructions that precede them, in simple sequential execution of the program, have finished executing. Prefetching an instruction does not mean that the instruction has to be executed.

In this manual, references to instruction or data fetching apply also to prefetching, unless the context explicitly indicates otherwise.

See also [Simple sequential execution](#).

Privileged access

Memory systems typically differentiate between privileged and unprivileged accesses, and support more restrictive permissions for unprivileged accesses. Some instructions can be used only by privileged software.

Processing element (PE)

The abstract machine that is defined in the Arm architecture, as documented in an Arm Architecture Reference Manual. A PE implementation compliant with the Arm architecture must conform with the behaviors described in the corresponding Arm Architecture Reference Manual.

Program Status Registers (XPSR)

XPSR is the term that is used to describe the combination of the APSR, EPSR, and IPSR into a single 32-bit Program Status Register.

See also [B3.5 XPSR, APSR, IPSR, and EPSR](#).

Protection granule

A quantum of memory for which an EDC or ECC provides detection or correction.

Protection region

A memory region whose position, size, and other properties are defined by Memory Protection Unit registers.

Protection Unit

See [Memory Protection Unit](#)

Pseudo-instruction

UAL assembler syntax that assembles to an instruction encoding that is expected to disassemble to a different assembler syntax, and is described in this manual under that other syntax. For example, `MOV<Rd>, <Rm>, LSL #<n>` is a pseudo-instruction that is expected to disassemble as `LSL<Rd>, <Rm>, #<n>`.

See also [Chapter C1 Instruction Set Overview](#).

Quadword

A 128-bit data item. Quadwords are normally at least word-aligned in Arm systems.

Quadword-aligned

Means that the address is divisible by 16.

Quiet NaN

A NaN that propagates unchanged through most floating-point operations.

RAO

See [Read-As-One](#).

RAO/SBOP

In versions of the Arm architecture before Armv8, Read-As-One, Should-Be-One-or-Preserved on writes.

In Armv8, RES1 replaces this description.

See also [UNK/SBOP](#), [Read-As-One](#), [RES1](#), [Should-Be-One-or-Preserved \(SBOP\)](#).

RAO/WI

Read-As-One, Writes Ignored.

Hardware must implement the field as Read-As-One, and must ignore writes to the field.

Software can rely on the field reading as all 1s, and on writes being ignored.

This description can apply to a single bit that reads as 1, or to a field that reads as all 1s.

See also [Read-As-One](#).

RAS

Reliability, Availability, and Serviceability. See [Chapter B16 Reliability, Availability, and Serviceability \(RAS\) Extension](#)

RAZ

See [Read-As-Zero](#).

RAZ/SBZP

In versions of the Arm architecture before Armv8, Read-As-Zero, Should-Be-Zero-or-Preserved on writes.

In Armv8, RES0 replaces this description.

See also [UNK/SBZP](#), [Read-As-Zero](#), [RES0](#), [Should-Be-Zero-or-Preserved \(SBOP\)](#).

RAZ/WI

Read-As-Zero, Writes Ignored.

Hardware must implement the field as Read-As-Zero, and must ignore writes to the field.

Software can rely on the field reading as all 0s, and on writes being ignored.

This description can apply to a single bit that reads as 0, or to a field that reads as all 0s.

See also [Read-As-Zero](#).

Read, modify, write

In a read, modify, write instruction sequence, a value is read to a general-purpose register, the relevant fields that are updated in that register, and the new value that is written back.

Read-allocate cache

A cache in which a cache miss on reading data causes a cache line to be allocated into the cache.

Read-As-One (RAO)

Hardware must implement the field as reading as all 1s.

Software:

- Can rely on the field reading as all 1s.
- Must use a [SBOP](#) policy to write to the field.

This description can apply to a single bit that reads as 1, or to a field that reads as all 1s. It applies only to a bit or field that is read-only.

See also [RAO/SBOP](#), [RAO/WI](#), [RES1](#).

Read-As-Zero (RAZ)

Hardware must implement the field as reading as all 0s.

Software:

- Can rely on the field reading as all 0s
- Must use a [SBZP](#) policy to write to the field.

This description can apply to a single bit that reads as 0, or to a field that reads as all 0s. It applies only to a bit or field that is read-only.

See also [RAZ/SBZP](#), [RAZ/WI](#), [RES0](#).

Recoverable

A contained error that must be corrected to allow the correct operation of the system or smaller parts of the system to continue.

See also [B16.3 Generating error exceptions](#).

Register data dependency

A register data dependency exists between a first data value and a second data value when either:

- The register that holds the first data value is used in the calculation of the second data value, and the calculation between the first data value and the second data value does not consist of either:
 - A conditional branch whose condition is determined by the first data value.
 - A conditional selection, move, or computation whose condition is determined by the first data value, where the input data values for the selection, move, or computation do not have a data dependency on the first data value.
- There is a register data dependency between the first data value and a third data value, and between the third data value and the second data value.

See [B7.2.3 Ordering and observability](#)

Reliability

Continuity of correct service.

Requester

An agent in a computing system that is capable of initiating memory transactions.

See also [Completer](#)

RES0

A reserved bit or field with [Should-Be-Zero-or-Preserved](#) behavior, or equivalent read-only or write-only behavior. Used for fields in register descriptions, and for fields in architecturally defined data structures that are held in memory.

Within the architecture, there are some cases where a register bit or field:

- Is RES0 in some defined architectural context.
- Has different defined behavior in a different architectural context.

Note

RES0 is not used in descriptions of instruction encodings.

This means the definition of RES0 for fields in read/write registers is:

If a bit is RES0 in all contexts

For a bit in a read/write register, it is IMPLEMENTATION DEFINED whether:

1. The bit is hardwired to 0. In this case:
 - Reads of the bit always return 0.
 - Writes to the bit are ignored.
2. The bit can be written. In this case:
 - An indirect write to the register sets the bit to 0.
 - A read of the bit returns the last value that is successfully written, by either a direct or an indirect write, to the bit.

If the bit has not been successfully written since reset, then the read of the bit returns the reset value if there is one, or otherwise returns an UNKNOWN value.

- A direct write to the bit must update a storage location that is associated with the bit.
- The value of the bit must have no effect on the operation of the PE, other than determining the value read back from the bit, unless this manual explicitly defines additional properties for the bit.

Whether RES0 bits or fields follow behavior 1 or behavior 2 is IMPLEMENTATION DEFINED on a bit-by-bit basis.

If a bit is RES0 only in some contexts

For a bit in a read/write register, when the bit is described as RES0:

- An indirect write to the register sets the bit to 0.
- A read of the bit must return the value last successfully written to the bit, by either a direct or an indirect write, regardless of the use of the register when the bit was written.

If the bit has not been successfully written since reset, then the read of the bit returns the reset value if there is one, or otherwise returns an UNKNOWN value.

- A direct write to the bit must update a storage location that is associated with the bit.
- While the use of the register is such that the bit is described as RES0, the value of the bit must have no effect on the operation of the PE, other than determining the value read back from that bit, unless this manual explicitly defines additional properties for the bit.

Considering only contexts that apply to a particular implementation, if there is a context in which a bit is defined as RES0, another context in which the same bit is defined as RES1, and no context in which the bit is defined as a functional bit, then it is IMPLEMENTATION DEFINED whether:

- Writes to the bit are ignored, and reads of the bit return an UNKNOWN value.
- The value of the bit can be written, and a read returns the last value that is written to the bit.

The RES0 description can apply to bits or fields that are read-only, or are write-only:

- For a read-only bit, RES0 indicates that the bit reads as 0, but software must treat the bit as UNKNOWN.
- For a write-only bit, RES0 indicates that software must treat the bit as [SBZ](#).

A bit that is RES0 in a context is reserved for possible future use in that context. To preserve forward compatibility, software:

- Must not rely on the bit reading as 0.
- Must use an SBZP policy to write to the bit.

This RES0 description can apply to a single bit, or to a field for which each bit of the field must be treated as RES0.

In body text, the term RES0 is shown in SMALLCAPS.

See also [Read-As-Zero](#), [RES1](#), [Should-Be-Zero-or-Preserved](#), [UNKNOWN](#).

RES0H

A reserved bit or field with [Should-Be-Zero-or-Preserved \(SBZP\)](#). This behavior uses the *Hardwired to 0* subset of the [RES0](#) definition.

RES1

A reserved bit or field with [Should-Be-One-or-Preserved](#) behavior, or equivalent read-only or write-only behavior. Used for fields in register descriptions, and for fields in architecturally defined data structures that are held in memory.

Within the architecture, there are some cases where a register bit or field:

- Is RES1 in some defined architectural context.
- Has different defined behavior in a different architectural context.

Note

RES1 is not used in descriptions of instruction encodings.

This means the definition of RES1 for fields in read/write registers is:

If a bit is RES1 in all contexts

For a bit in a read/write register, it is IMPLEMENTATION DEFINED whether:

1. The bit is hardwired to 1. In this case:
 - Reads of the bit always return 1.
 - Writes to the bit are ignored.
2. The bit can be written. In this case:
 - An indirect write to the register sets the bit to 1.
 - A read of the bit returns the last value that is successfully written, by either a direct or an indirect write, to the bit.

If the bit has not been successfully written since reset, then the read of the bit returns the reset value if there is one, or otherwise returns an UNKNOWN value.

- A direct write to the bit must update a storage location that is associated with the bit.
- The value of the bit must have no effect on the operation of the PE, other than determining the value read back from the bit, unless this manual explicitly defines additional properties for the bit.

Whether RES1 bits or fields follow behavior 1 or behavior 2 is IMPLEMENTATION DEFINED on a bit-by-bit basis.

If a bit is RES1 only in some contexts

For a bit in a read/write register, when the bit is described as RES1:

- An indirect write to the register sets the bit to 1.
- A read of the bit must return the value last successfully written to the bit, regardless of the use of the register when the bit was written.

Note

As indicated in this list, this value might be written by an indirect write to the register.

If the bit has not been successfully written since reset, then the read of the bit returns the reset value if there is one, or otherwise returns an UNKNOWN value.

- A direct write to the bit must update a storage location that is associated with the bit.
- While the use of the register is such that the bit is described as RES1, the value of the bit must have no effect on the operation of the PE, other than determining the value read back from that bit, unless this manual explicitly defines additional properties for the bit.

Considering only contexts that apply to a particular implementation, if there is a context in which a bit is defined as RES0, another context in which the same bit is defined as RES1, and no context in which the bit is defined as a functional bit, then it is IMPLEMENTATION DEFINED whether:

- Writes to the bit are ignored, and reads of the bit return an UNKNOWN value.
- The value of the bit can be written, and a read returns the last value that is written to the bit.

The RES1 description can apply to bits or fields that are read-only, or are write-only:

- For a read-only bit, RES1 indicates that the bit reads as 1, but software must treat the bit as UNKNOWN.
- For a write-only bit, RES1 indicates that software must treat the bit as [SBO](#).

A bit that is RES1 in a context is reserved for possible future use in that context. To preserve forward compatibility, software:

- Must not rely on the bit reading as 1.
- Must use an [SBOP](#) policy to write to the bit.

This RES1 description can apply to a single bit, or to a field for which each bit of the field must be treated as RES1.

In body text, the term RES1 is shown in SMALLCAPS.

See also [Read-As-One](#), [RES0](#), [Should-Be-One-or-Preserved](#), [UNKNOWN](#).

RES1H

A reserved bit or field with [Should-Be-One-or-Preserved \(SBOP\)](#) behavior. This behavior uses the *Hardwired to 1* subset of the [RES1](#) definition.

Reserved

Unless otherwise stated:

- Instructions that are reserved or that access reserved registers have UNPREDICTABLE or CONSTRAINED UNPREDICTABLE behavior.
- Bit positions that are described as reserved are:
 - In an RW or WO register, RES0.
 - In an RO register, UNK.

See also [CONSTRAINED UNPREDICTABLE](#), [RES0](#), [RES1](#), [UNDEFINED](#), [UNK](#), [UNPREDICTABLE](#).

Restartable

A contained error that does not immediately impact correct operation. Usually this means correct operation of the system, but it can also be used in other contexts to describe correct operation of a smaller part.

See also [B16.3 Generating error exceptions](#).

Return Link

A value relating to the return address.

RISC

Reduced Instruction Set Computer.

Rounding error

The value of the rounded result of an arithmetic operation minus the exact result of the operation.

Rounding mode

Specifies how the exact result of a floating-point operation is rounded to a value that is representable in the destination format. The rounding modes are defined by the *IEEE Standard for Floating-point Arithmetic*.

Saturated arithmetic

Integer arithmetic in which a result that would be greater than the largest representable number is set to the largest representable number, and a result that would be less than the smallest representable number is set to the smallest representable number. Signed saturated arithmetic is often used in DSP algorithms. It contrasts with the normal signed integer arithmetic used in Arm processors, in which overflowing results wrap around from $+2^{31} - 1$ to -2^{31} or the opposite way.

SBO

See [Should-Be-One](#).

SBOP

See [Should-Be-One-or-Preserved](#).

SBZ

See [Should-Be-Zero](#).

SBZP

See [Should-Be-Zero-or-Preserved](#)

Security hole

A mechanism by which execution at the current level of privilege can achieve an outcome that cannot be achieved at the current or a lower level of privilege using instructions that are not UNPREDICTABLE and are not CONSTRAINED UNPREDICTABLE. The Arm architecture forbids security holes.

See also [CONSTRAINED UNPREDICTABLE](#), [UNPREDICTABLE](#).

Self-modifying code

Code that writes one or more instructions to memory and then executes them. When using self-modifying code, cache maintenance and barrier instructions must be used to ensure synchronization.

Serial Wire Output (SWO)

An asynchronous TPIU port supporting one or both of the NRZ and Manchester encodings.

Serial Wire Viewer (SWV)

The combination of an SWO and at least one of a DWT unit or an ITM, providing data tracing capability.

Service failure mode

A mode entered to reduce the severity of an error.

Serviceability

The ability to undergo modification and repairs.

Set

See [Cache sets](#).

Should-Be-One (SBO)

Hardware must ignore writes to the field.

Arm strongly recommends that software writes the field as all 1s. If software writes a value that is not all 1s, it must expect an UNPREDICTABLE or CONSTRAINED UNPREDICTABLE result.

This description can apply to a single bit that should be written as 1, or to a field that should be written as all 1s.

See also [CONSTRAINED UNPREDICTABLE](#), [UNPREDICTABLE](#).

Should-Be-One-or-Preserved (SBOP)

From the introduction of the Armv8 architecture, the description *Should-Be-One-or-Preserved* is superseded by RES1.

Hardware must ignore writes to the field.

If software has read the field since the PE implementing the field was last reset and initialized, it must preserve the value of the field by writing the value that it previously read from the field. Otherwise, it must write the field as all 1s.

If software writes a value to the field that is not a value that is previously read for the field and is not all 1s, it must expect an UNPREDICTABLE or CONSTRAINED UNPREDICTABLE result.

This description can apply to a single bit that should be written as its preserved value or as 1, or to a field that should be written as its preserved value or as all 1s.

See also [CONSTRAINED UNPREDICTABLE](#), [UNPREDICTABLE](#).

Should-Be-Zero (SBZ)

Hardware must ignore writes to the field.

Arm strongly recommends that software writes the field as all 0s. If software writes a value that is not all 0s, it must expect an UNPREDICTABLE or CONSTRAINED UNPREDICTABLE result.

This description can apply to a single bit that should be written as 0, or to a field that should be written as all 0s.

See also [CONSTRAINED UNPREDICTABLE](#), [UNPREDICTABLE](#).

Should-Be-Zero-or-Preserved (SBZP)

From the introduction of the Armv8 architecture, the description *Should-Be-Zero -or-Preserved* is superseded by RES0.

Hardware must ignore writes to the field.

If software has read the field since the PE implementing the field was last reset and initialized, it must preserve the value of the field by writing the value that it previously read from the field. Otherwise, it must write the field as all 0s.

If software writes a value to the field that is not a value that is previously read for the field and is not all 0s, it must expect an UNPREDICTABLE or CONSTRAINED UNPREDICTABLE result.

This description can apply to a single bit that should be written as its preserved value or as 0, or to a field that should be written as its preserved value or as all 0s.

See also [CONSTRAINED UNPREDICTABLE](#), [UNPREDICTABLE](#).

Signaling NaNs

Cause an Invalid Operation exception whenever any floating-point operation receives a signaling NaN as an operand. Signaling NaNs can be used in debugging, to track down some uses of uninitialized variables.

Signed data types

Represent an integer in the range -2^{N-1} to $+2^{N-1} - 1$, using two's complement format.

Signed immediate and offset fields

Are encoded in two's complement notation unless otherwise stated.

Silent data corruption

An error that is not detected by hardware or software.

Silently propagated

An error that is passed from place to place without being signaled as a detected error.

SIMD

Single-Instruction, Multiple-Data.

Simple sequential execution

The behavior of an implementation that fetches, decodes and completely executes each instruction before proceeding to the next instruction. Such an implementation performs no Speculative accesses to memory, including to instruction memory. The implementation does not pipeline any phase of execution. In practice, this is the theoretical execution model that the architecture is based on, and Arm does not expect this model to correspond to a realistic implementation of the architecture.

Single peripheral

A single peripheral is a region of memory of an IMPLEMENTATION DEFINED size that is defined by the peripheral.

Single-copy atomicity

The form of atomicity that is described in [B7.2.1 Single-copy atomicity](#).

See also [Atomicity](#), [Multi-copy atomicity](#).

Single-precision value

A 32-bit word that is interpreted as a basic single-precision floating-point number according to the IEEE Standard for Floating-point Arithmetic.

Software fault

A fault that originates in and affects software.

Spatial locality

The observed effect that after a program has accessed a memory location, it is likely to also access nearby memory locations in the near future. Caches with multi-word cache lines exploit this effect to improve performance.

Special-purpose register

One of a specified set of registers for which all direct and indirect reads and writes to the register appear to occur in program order relative to other instructions, without the need for any explicit synchronization. For more information, see [B3.3 Registers](#).

Speculative writes

All of the following are Speculative writes:

- Writes generated by store instructions that appear in the Execution stream after a branch that is not architecturally resolved.
- Writes generated by store instructions that appear in the Execution stream after an instruction where a synchronous exception condition has not been architecturally resolved.
- Writes generated by conditional store instructions for which the conditions for the instruction have not been architecturally resolved.
- Writes generated by store instructions for which the data being written comes from a register that has not been architecturally committed.

State context registers

The State context registers are as follows:

- [R0](#) to [R3](#).
- [R12](#).
- [LR](#).
- [RETPSR](#).
- The `ReturnAddress`.

See also [Additional state context registers](#)

Synchronous exception

An exception is described as synchronous if all of the following apply:

- The exception generated as a result of direct execution or attempted execution of the instruction stream.
- The return address presented to the exception handler is guaranteed to indicate the instruction that caused the exception.
- The exception is precise.

See also [Asynchronous exception](#), [Precise exception](#).

System Control Block (SCB)

An address region in the System Control Space, which is used for key feature control and configuration that is associated with the exception model.

See also [System Control Space](#).

System Control Space (SCS)

A region of the memory map that is reserved for system control and configuration registers.

See also [Debug Control Block](#), [B8.3 The System Control Space \(SCS\)](#).

T32 instruction

One or two halfwords that specify an operation to be performed by a PE. T32 instructions must be halfword-aligned. For more information, see [Chapter C1 Instruction Set Overview](#).

T32 instructions were previously called Thumb instructions.

Tail-chaining

An optimization that removes unstacking and stacking operations. For more information, see [B3.26 Tail-chaining](#).

Temporal locality

The observed effect that after a program has accesses a memory location, it is likely to access the same memory location again in the near future. Caches exploit this effect to improve performance.

TPIU

See [Trace Port Interface Unit](#).

Trace Port Interface Unit (TPIU)

A component of the Arm CoreSight debug and trace solution. A TPIU provides an external interface for one or more trace sources in the processor implementation.

See [B14.4 Trace Port Interface Unit](#).

Transient fault

A fault that is not persistent.

UAL

See [Unified Assembler Language](#).

Unaligned

An unaligned access is an access where the address of the access is not aligned to the size of an element of the access.

Unaligned memory accesses

Are memory accesses that are not, or might not be, appropriately halfword-aligned, word-aligned, or doubleword-aligned.

Unallocated

Except where otherwise stated in this manual, an instruction encoding is unallocated if the architecture does not assign a specific function to the entire bit pattern of the instruction, but instead describes it as `CONSTRAINED UNPREDICTABLE`, `UNDEFINED`, `UNPREDICTABLE`, or as an unallocated hint instruction.

A bit in a register is unallocated if the architecture does not assign a function to that bit.

See also [CONSTRAINED UNPREDICTABLE](#), [UNPREDICTABLE](#), [UNDEFINED](#).

Uncontainable

An error that has been, or might have been, silently propagated. This is also referred to as an Uncontained error.

See also [B16.3 Generating error exceptions](#).

UNDEFINED

Indicates an instruction that generates an Undefined Instruction exception.

In body text, the term UNDEFINED is shown in SMALLCAPS.

See also [Chapter C1 Instruction Set Overview](#).

Undetected fault

See [Latent fault](#).

Unified Assembler Language

The assembler language that is introduced with Thumb-2 technology that is used in this manual. See [Chapter C1 Instruction Set Overview](#) for details.

Unified cache

Is a cache that is used for both processing instruction fetches and processing data loads and stores.

Unindexed addressing

Means addressing in which the base register value is used directly as the address to send to memory, without adding or subtracting an offset. In most types of load/store instruction, unindexed addressing is performed by using offset addressing with an immediate offset of 0.

In the M-Profile, the *LDC*, *LDC2*, *STC*, and *STC2* instructions have an explicit unindexed addressing mode that permits the offset field in the instruction to specify additional coprocessor options.

UNK

An abbreviation indicating that software must treat a field as containing an UNKNOWN value.

Hardware must implement the bit as read as 0, or all 0s for a multi-bit field. Software must not rely on the field reading as zero.

See also [UNKNOWN](#).

UNK/SBOP

Hardware must implement the field as Read-As-One, and must ignore writes to the field.

Software must not rely on the field reading as all 1s, and except for writing back to the register it must treat the value as if it is UNKNOWN. Software must use an SBOP policy to write to the field.

This description can apply to a single bit that should be written as its preserved value or as 1, or to a field that should be written as its preserved value or as all 1s.

See also [Read-as-One](#), [Should-Be-One-or-Preserved](#), [UNKNOWN](#).

UNK/SBZP

Hardware must implement the bit as Read-As-Zero, and must ignore writes to the field.

Software must not rely on the field reading as all 0s, and except for writing back to the register must treat the value as if it is UNKNOWN. Software must use an SBZP policy to write to the field.

This description can apply to a single bit that should be written as its preserved value or as 0, or to a field that should be written as its preserved value or as all 0s.

See also [Read-as-Zero](#), [Should-Be-Zero-or-Preserved](#), [UNKNOWN](#).

UNKNOWN

An UNKNOWN value does not contain valid data, and can vary from moment to moment, instruction to instruction, and implementation to implementation. An UNKNOWN value must not return information that cannot be accessed at the current or lower level of privilege using instructions that are not UNPREDICTABLE, is not CONSTRAINED UNPREDICTABLE, and do not return UNKNOWN values.

An UNKNOWN value can vary from moment to moment, and instruction to instruction, unless it has previously been assigned, other than at reset, to one of the following registers:

- Any of the general-purpose registers.
- Any of the Advanced SIMD and floating-point registers.
- Any of the [APSR.{N, Z, C, V}](#) flags.

An UNKNOWN value must not be documented or promoted as having a defined value or effect.

In body text, the term UNKNOWN is shown in SMALLCAPS.

See also [CONSTRAINED UNPREDICTABLE](#), [UNDEFINED](#), [UNK](#), [UNPREDICTABLE](#).

UNPREDICTABLE

Means the behavior cannot be relied on. UNPREDICTABLE behavior must not perform any function that cannot be performed at the current or a lower level of privilege or security using instructions that are not UNPREDICTABLE.

UNPREDICTABLE behavior must not be documented or promoted as having a defined effect.

An instruction that is UNPREDICTABLE can be implemented as UNDEFINED.

In body text, the term UNPREDICTABLE is shown in SMALLCAPS.

See also [CONSTRAINED UNPREDICTABLE](#), [UNDEFINED](#).

Unrecoverable

A contained error that is not recoverable. Continued correct operation is generally not possible. Usually this means correct operation of the system, but it can also be used in other contexts to describe correct operation of a smaller part. Systems might use high-level recovery techniques to work around an unrecoverable yet contained error in a component so that the system recovers from the error.

See also [B16.3 Generating error exceptions](#).

Unsigned data types

Represent a non-negative integer in the range 0 to $+2^{N-1} - 1$, using normal binary format.

Watchpoint

A debug event that is triggered by an access to memory, which is specified in terms of the address of the location in memory being accessed.

Way

See [Cache way](#).

WI

Writes Ignored. In a register that software can write to, a WI attribute that is applied to a bit or field indicates that the bit or field ignores the value that is written by software and retains the value it had before that write.

See also [RAO/WI](#), [RAZ/WI](#), [RES0](#), [RES1](#).

Word

A 32-bit data item. Words are normally word-aligned in Arm systems.

Word-aligned

Means that the address is divisible by 4.

Write buffer

A block of high-speed memory that optimizes stores to main memory.

Write-Allocate cache

A cache in which a cache miss on storing data causes a cache line to be allocated into the cache.

Write-back cache

A cache in which when a cache hit occurs on a store access, the data is only written to the cache. Data in the cache can therefore be more up-to-date than data in main memory. Any such data is written back to main memory when the cache line is cleaned or reallocated. Another common term for a write-back cache is a *copy-back cache*.

Write-one-to-clear

Writing 1 to the relevant bit clears it to 0. Writing 0 to the bit has no effect.

Write-one-to-set

Writing 1 to the relevant bit sets it to 1. Writing 0 to the bit has no effect.

Write-Through cache

A cache in which when a cache hit occurs on a store access, the data is written both to the cache and to main memory. This is normally done using a write buffer, to avoid slowing down the PE.

XPSR

See [Program Status Registers \(XPSR\)](#).